Linear Algebra From the Ground Up

A Reader for Before *and* After Strang

From Geometric Intuition to Modern Applications

June 2026

Preface

A book to read before *and* after Strang

There is a moment, somewhere in a first linear algebra course, when a student writes down a matrix equation $\mathbf{Ax} = \mathbf{b}$ (a compact shorthand for a system of simultaneous equations), solves it correctly, gets full marks... and still has no idea what just happened. The symbols moved around the page in the legal ways. The answer matched the back of the book. But the *meaning* never arrived. Nothing in the calculation felt like it was *about* anything.

This book exists to make sure that does not happen to you. Or, if it has already happened, to undo it.

Gilbert Strang's lectures and his *Introduction to Linear Algebra* are, deservedly, the way an enormous number of people have learned this subject. They are warm, geometric, and full of the sense that linear algebra is one connected idea rather than a list of tricks. If you have not met them yet, you have a genuine pleasure ahead of you. So why another book?

Because Strang moves *fast*, and he assumes you are ready to move with him. In his very first lecture, he draws both a column picture and a row picture of a system of equations. If you have never thought of a list of numbers as an *arrow*, or of a matrix as a *machine that moves space around*, that opening hour can go by like scenery from a train window: beautiful, but gone before you could hold onto it. This book is the slow walk through the same country *before* you take the train, and the deeper hike into the mountains *after*.

Intuition

The single picture this whole book is built to install in your head:

> **A matrix is a function that takes in vectors and spits out vectors – and it does so in the one geometrically simplest way possible: it keeps the grid of space straight, evenly spaced, and pinned at the origin.**

Everything else flows from that one sentence, looked at from a different angle. Determinants measure how areas or volumes scale, eigenvalues mark the directions a matrix stretches without rotating, and rank counts the true dimensions of the output. From the same picture come the singular value decomposition (SVD, revealing every matrix as rotate-stretch-rotate), least squares (finding the best compromise when no exact solution exists), and the mathematics behind modern machine learning. If you finish a chapter and feel lost, come back to this box. It is true on page 1 and it is true on the last page.

Before Strang

Read this book before Strang and its job is to build the intuition that makes his course *click* on the first pass instead of the third. When Strang says "the column space is everything you can reach," you will already have spent an afternoon reaching for things and noticing what you could and could not touch. When he writes $\mathbf{A} = \mathbf{LU}$ – splitting a matrix into lower- and upper-triangular factors through elimination (systematically clearing equations) – you will already feel elimination as a sequence of physical shears on a grid, not as bookkeeping. You will walk into his classroom not as a beginner but as someone who already has the mental furniture and just needs the precise names for it.

After Strang

Read this book after Strang and its job changes. Now you know the vocabulary, you have solved the systems, you have computed the eigenvalues by hand. What you may *not* yet have is a unified view of how the four fundamental subspaces (the spaces of inputs, outputs, and directions sent to zero), the spectral theorem (showing that symmetric matrices stretch space along perpendicular axes), and the singular value decomposition fit together as facets of one geometric structure. That same structure powers principal component analysis (finding the main axes of variation in high-dimensional data), recommender systems, stable neural-network training, and the compression of the image on your screen right now. The later chapters of this book are written for the reader who is ready to see the whole structure at once and to watch it turn into technology.

Focus

Hold onto two promises for the whole book:

1. **No formula appears before its picture.** You will see what a thing *means* before you are asked to compute it.
2. **No formula is asserted that we can derive in a few clear lines.** When we can show you *why*, we will, deriving key results rather than stating them without justification.

Who this book is for

You are smart, you are motivated, and you may be seeing this material rigorously for the first time. That is exactly the reader I am writing to. Concretely, this book is for you if any of the following fit:

- You are about to take a linear algebra course (Strang's or any other) and you want a solid geometric intuition before the formal algebra starts.
- You took such a course, passed it, and have the uncomfortable feeling that you can compute things you cannot *see*.
- You came to linear algebra through machine learning, data science, computer graphics, or signal processing, and you are tired of treating "the matrix stuff" as a black box you import from a library.
- You simply think mathematics is one of the most beautiful things humans have made, and you want to feel why this corner of it is so often called the most useful mathematics ever invented.

What I assume you already have

Very little, and all of it is the kind of thing you can refresh as you go.

- **High-school algebra.** You are comfortable with symbols standing for numbers, with solving $2x + 1 = 7$, and with the idea of a function as a machine: feed it an input, get an output.
- **A little coordinate geometry.** You have seen the xy-plane, plotted a point, drawn a line $y = mx + b$. We will rebuild everything else.
- **Willingness to draw.** Not skill – willingness. A shaky arrow on the back of an envelope is worth more here than a paragraph of prose.

You do *not* need calculus. Where calculus deepens a point, it lives in a clearly marked box you can skip without losing the thread. You do not need any prior programming, though if you can run a few lines of Python or use a spreadsheet, some of the labs become more fun.

Common trap

The most common way to misread this book is to treat the worked numerical examples as something to *watch* rather than something to *do*. Reading "$3 \cdot 2 + (-1) \cdot 4 = 2$" and nodding is not the same as understanding. Keep a pen moving. Recompute the small examples yourself, and when a box says *Compute it yourself*, actually stop and do it.

The philosophy: geometry, then algebra, then proof

Almost every important idea in this book is introduced three times, in this order, and the order is intentional.

First, geometry

We start with a *picture*. An arrow, a grid, a plane through the origin, a square getting squashed into a parallelogram. The goal of the first pass is not correctness in any formal sense; it is to give you something to *look at* in your mind's eye, so that the symbols which come later have somewhere to land. When you can close your eyes and *see* a matrix tilting the plane, you are no longer memorizing – you are describing something you can watch.

Then, algebra

Once the picture is steady, we make it precise with coordinates and arithmetic. We ask: if the picture is true, what must the *numbers* do? This is where the small, fully worked 2×2 and 3×3 examples live, traced line by line, because the picture only becomes trustworthy once you have watched it agree with the arithmetic in concrete cases.

Finally, proof

A picture can mislead. A handful of examples can be a coincidence. So the third pass asks the adult question: *is it always true, and why?* Our proofs are gentle but real. They are usually just the geometric idea written carefully enough that a skeptic could not wriggle out. You will find that, done this way, a proof is not a hurdle after the understanding – it *is* the understanding, pinned down so it cannot drift.

Intuition

A tiny illustration of the three passes, so the philosophy is not just a slogan. Take the claim: *rotating the plane by* $90°$ *never changes a vector's length.*

Geometry. Spin an arrow about the origin like the hand of a clock. It points somewhere new; it is plainly just as long. Length cannot care which way you face.

Algebra. Write the arrow's coordinates as a vertical column $\begin{bmatrix} x \\ y \end{bmatrix}$. A horizontal step of length x rotates $90°$ into a vertical step of length x, while a vertical step of length y rotates $90°$ left to become $-y$, so rotation sends $\begin{bmatrix} x \\ y \end{bmatrix}$ to $\begin{bmatrix} -y \\ x \end{bmatrix}$. Its squared length is $(-y)^2 + x^2 = x^2 + y^2$ – the same number we started with. Try it on $\begin{bmatrix} 3 \\ 1 \end{bmatrix}$: it goes to $\begin{bmatrix} -1 \\ 3 \end{bmatrix}$, and $9 + 1 = 1 + 9 = 10$. Length preserved.

Proof. For *every* $\begin{bmatrix} x \\ y \end{bmatrix}$ the swap-and-negate sends $x^2 + y^2$ to $(-y)^2 + x^2$, and addition does not care about order, so the two sums are identical. Always. No exceptions. The picture was telling the truth.

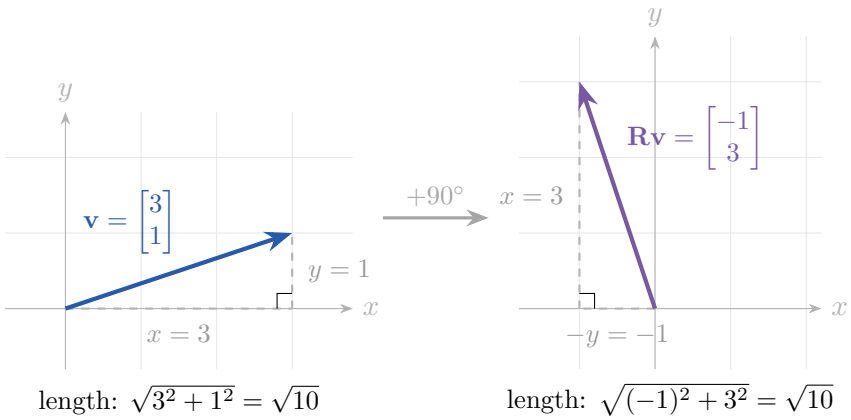


Figure 1: The three passes in a single picture. Rotating the vector counterclockwise by $90°$ spins the right triangle as a rigid piece. The horizontal step $x = 3$ turns upright to become the new vertical coordinate, while the vertical step $y = 1$ pivots left to become -1. The legs swap and one negates, but the hypotenuse is the same piece of geometry: $\sqrt{x^2 + y^2} = \sqrt{(-y)^2 + x^2} = \sqrt{10}$. The algebra tracks what the rigid rotation did.

Pause

Mental pause: seeing the signs. Before reading further, close your eyes and rotate the vector $\begin{bmatrix}0\\2\end{bmatrix}$ by 90° counterclockwise. Where does it land? Now check the formula $\begin{bmatrix}x\\y\end{bmatrix} \mapsto \begin{bmatrix}-y\\x\end{bmatrix}$: with $x = 0$ and $y = 2$, it gives $\begin{bmatrix}-2\\0\end{bmatrix}$. Does the formula agree with your mental picture? It always will.

Walking question

Before you read another page, take this on a walk. You have two arrows lying flat on a table, starting from the same point. You are allowed to *add* them (tip to tail) and to *scale* them (stretch or flip). Which points on the table can you reach by adding and scaling? When can you reach *every* point, and when are you stuck on a single line forever? You will have just discovered, on your own, the central themes of the first third of this book: *span* (the entire territory reachable by scaling and adding), *independence* (having no redundant arrows pointing along paths already covered), and *dimension* (the minimum number of independent directions needed to map the space). Notice how far you get with no formula at all.

The colored boxes: a visual language

This book talks to you in two registers. The main text walks forward steadily. Around it sit *colored boxes*, each a different kind of voice – a nudge, a warning, a moment of geometric focus, a bridge to the wider world. They are part of the main argument, not optional sidebars to skim. Here is what each color signals:

Intuition

Intuition (green). The core mental picture. When you see green, slow down and try to *see* the thing being described before you read on. If you remember nothing else from a section, remember its green boxes.

Focus

Focus (blue). The one or two sentences worth holding onto from a stretch of text. If you were making flashcards, these are the fronts of the cards.

Common trap

Common trap (red). A classic mistake – one that almost everyone makes, including people who go on to use this subject professionally. Seeing the trap named in advance is often what keeps you out of it.

Walking question

Walking question (orange). A prompt to chew on away from the page, ideally while doing something else. The best linear algebra thinking I have ever done happened while walking, not while staring. These questions have no deadline; let them simmer.

Compute it yourself

Compute it yourself (purple). A small, do-it-by-hand 2×2 or 3×3 lab. Pen, paper, two minutes. These are the reps that turn watching into knowing.

Formal

Formal (gray). The precise statement – a definition or theorem stated exactly, the way you would need it to prove something or to look it up later. When you want to be sure of the *letter* of an idea, find its gray box.

Before/after Strang

Before/after Strang (deep blue). An explicit bridge to how Strang frames the same idea. *Before* you have taken his course, these boxes tell you what to listen for. *After*, they connect his vocabulary – pivots (the leading nonzero numbers that survive each elimination step to anchor the solution), column space (all vectors reachable as combinations of a matrix's columns), the four fundamental subspaces – to the way we have built things here, so the two pictures fuse into one. Example: where we will speak of "the directions a matrix flattens to zero," Strang calls that the *null space*, and you will hear him say "$\mathbf{Ax} = \mathbf{0}$" for exactly the same idea.

Modern application

Modern application (magenta). Where this idea shows up in the world right now – in machine learning, data science, computer graphics, signals, networks, or statistics. A taste of why people sometimes call linear algebra "the language of data." For instance: every time a streaming service guesses a film you will like, it is approximating a giant matrix of ratings by a product of two thin ones – a low-rank matrix approximation computed via the singular value decomposition. We will derive this construction completely in the later chapters of the book.

Connection to other math

Connection to other math (teal). A bridge to a neighboring country: calculus, geometry, group theory (the formal mathematics of symmetry), probability, differential equations (equations tracking how quantities change over continuous time), functional analysis (where functions themselves are treated as infinite-dimensional vectors). Linear algebra is the hub that these spokes meet at, and seeing the spokes is part of seeing why it matters so much. For example: the derivative you met in calculus is simply the *best linear approximation* to a curve at a point – which is to say, calculus is the art of replacing complicated functions by matrices, locally, and linear algebra is what you do with those matrices once you have them.

Why this matters

Why this matters (cyan). A pause to say plainly why a result is important or mathematically elegant – the part a hurried course skips, and the part that, for many people, provides the deepest motivation. Mathematics is not only useful; seeing how its pieces lock together is half the reward.

History and connection

History and connection (yellow). A little context on where an idea came from and who fought to understand it – because the ideas were hard-won by real people, and knowing that they were once confused too is strangely encouraging.

Pause

Pause (plain). A breath. A short recap, or permission to stop, reread, and let things settle before going on. If a chapter ever feels like too much at once, the Pause boxes are designed to be safe places to rest.

Reading note

Reading note (gray). A small piece of advice about *how* to read a particular passage – what to skim, what to linger on, what is safe to skip on a first pass and return to later.

Focus

You do not need to memorize this list. The colors are chosen so that, within a chapter or two, you will know what kind of voice is speaking before you have read a single word – green means *see this*, red means *careful*, magenta means *the real world*, teal means *the rest of math*. The page itself will start telling you how to read it.

Pictures are first-class citizens here

The diagrams in this book are essential to the explanations. For example, a figure that shows the unit square getting sheared into a parallelogram directly represents the determinant, which is simply the factor by which area got scaled. You will see, drawn carefully and referenced from the prose:

- vectors as arrows, and their sums as journeys laid tip to tail;
- linear combinations and *spans* – the lines and planes you can reach;
- the action of a matrix on a whole grid, and on the humble unit square;
- *projections* dropping a point straight down onto a line or a plane;
- subspaces nested inside one another, and the famous four of them;
- eigenvector directions – the special arrows a matrix does not turn;
- rotations, reflections, and shears, each with its own visual fingerprint;

- and, as the book's quiet destination, the singular value decomposition as *rotate, stretch, rotate* – the claim that *every* matrix, no matter how complicated, is only ever those three simple moves in sequence.

Below is the kind of small, plain figure you will meet on nearly every page. Nothing fancy: just an arrow, its double, and their sum, so that the very first operations of linear algebra – scaling and adding – are things you have *seen* and not only read.

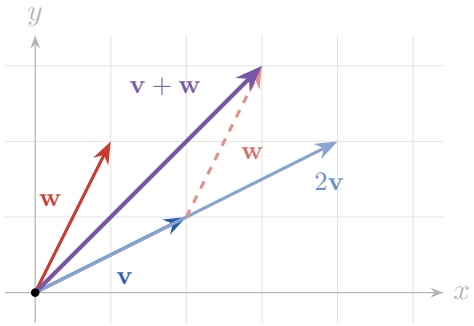


Figure 2: Two operations are the whole foundation. Scaling: $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ stretches to $2\mathbf{v} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$ along the same line. Adding: walk along $\mathbf{v}$, then continue along $\mathbf{w} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ (dashed), and you arrive at $\mathbf{v} + \mathbf{w} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$. Every span, every matrix, every theorem in this book is built out of just these two moves.

Figure 2 is doing real work. The dashed copy of $\mathbf{w}$ *is* the reason vector addition is "tip to tail"; the two shades of blue *are* the reason scaling stays on one line. Whenever a figure appears, slow down and ask what claim it is quietly making.

Here is a second taste of a figure-as-argument – one we will return to so often that it is almost the mascot of the book. Take the humble unit square, the one cornered at the origin with sides $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, and feed it to the matrix $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$. The matrix sends each corner to a new place, and the square slides into a *parallelogram*. Watch what stays true and what changes.

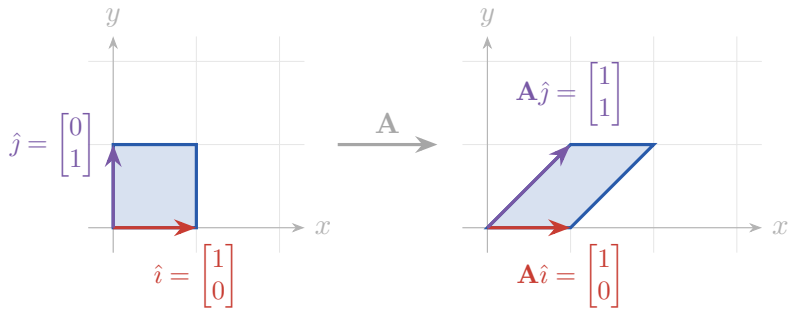


Figure 3: A matrix is a motion of space. The shear $\mathbf{A} = \begin{bmatrix}1 & 1\\0 & 1\end{bmatrix}$ leaves the red side $\begin{bmatrix}1\\0\end{bmatrix}$ alone and tilts the purple side $\begin{bmatrix}0\\1\end{bmatrix}$ over to $\begin{bmatrix}1\\1\end{bmatrix}$. The square becomes a slanted parallelogram, yet its area is unchanged: base 1, height 1, area 1. That unchanged area is the *determinant*, here equal to 1. We have not defined the determinant yet – but you have already seen it.

Figure 3 shows the determinant at work before we have defined it: *see the geometry first, then name it.* When the formula $\det\begin{bmatrix}a & b\\c & d\end{bmatrix} = ad - bc$ finally arrives, it will be the caption to a figure you already trust, not a rule to memorize.

Compute it yourself

A thirty-second warm-up, to prove to yourself that the figure tells the truth. On the same axes, draw $\mathbf{v} = \begin{bmatrix}2\\1\end{bmatrix}$ and $\mathbf{w} = \begin{bmatrix}1\\2\end{bmatrix}$, then form $\mathbf{v} + \mathbf{w}$ two ways: once by walking $\mathbf{v}$ then $\mathbf{w}$, and once by walking $\mathbf{w}$ then $\mathbf{v}$. You should land on $\begin{bmatrix}3\\3\end{bmatrix}$ both times. You have just *seen* that vector addition does not care about order – a fact we will lean on constantly, and which you now believe because you watched it, not because anyone told you.

How the parts build

The book climbs in deliberate stages. You can read it straight through, and that is what I recommend on a first pass, because each part quietly installs the tools the next one stands on. Here is an overview of how the topics connect:

1. **Vectors and the spaces they live in.** We begin where everything begins: a vector as an arrow and as a list of numbers, the two operations of scaling and adding, and the geometry of *span* – the lines, planes, and spaces you can build. This is where "before Strang" readers get the furniture that the column and row pictures will later sit on.

2. **Matrices as motions.** A matrix stops being a grid of numbers and becomes a *transformation* – a way of moving all of space at once while keeping the grid straight. Matrix multiplication becomes *composition* of motions, and suddenly the strange "row times column" rule is the only rule it could possibly have been.

3. **Solving and structure.** Systems of equations, elimination, rank, and the four fundamental subspaces – the skeleton Strang builds his whole course around. We earn each piece geometrically first, so the algebra of elimination feels like watching a shape simplify rather than pushing symbols.

4. **Formalizing geometry.** Here we build the tools of measurement: length, angle, the dot product (multiplying two vectors to measure how closely they align), *orthogonality* (perpendicularity, meeting at 90°), and *projection* (dropping a perpendicular shadow onto a subspace). Those tools immediately give us a central method of applied linear algebra, *least squares*: finding the closest compromise when no exact solution exists, a staple of data analysis and engineering.

5. **Eigenvalues and the heart of a matrix.** The special directions a matrix merely stretches without turning (eigenvectors) and the scaling factors along them (eigenvalues), why they reveal a matrix's deepest behavior, and how they let us understand repeated processes – powers, dynamics, the long-run fate of a system.

6. **The singular value decomposition.** Every matrix as *rotate, stretch, rotate*. From here we can finally see, in full, the machinery under principal component analysis, data compression, and a great deal of modern machine learning.

Before/after Strang

If you are reading this *before* Strang: stages 1 through 3 above (Parts I–III) are your runway. By the time you have them, his opening lectures will feel like meeting old friends under new names. If you are reading *after* Strang, feel free to skim the early chapters for any geometric pictures you may have rushed past. Spend your main energy in stages 4 through 6 (Parts IV–VIII), where we go deeper than a first course usually can – especially into the SVD, which Strang rightly calls the high point of the subject and which receives a full, unhurried treatment here.

Why this matters

Here is the promise that the structure above is really making. By the final part, six ideas that looked separate at the start – linear combinations, matrix multiplication, rank, orthogonal projection, eigenvalues, and singular values – will have collapsed into *one* idea seen from six sides. That collapse, the moment a sprawling subject suddenly closes into a single clean shape in your mind, is one of the great pleasures mathematics has to offer. It is worth the climb. I have tried to build every step so that you actually get to feel it.

Two of those six ideas deserve a face before you meet them by name, because they are the ones students most often compute without ever seeing. Figure 4 shows them side by side: a *projection* on the left, an *eigenvector* on the right.

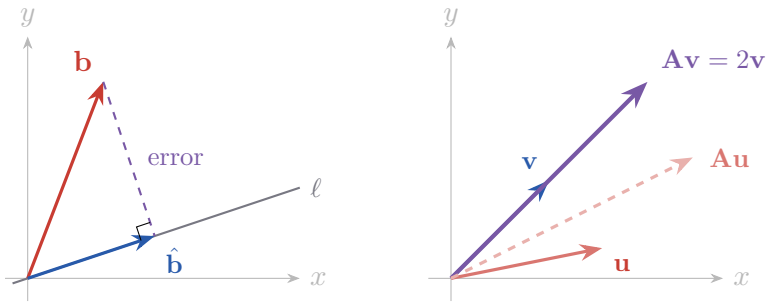


Figure 4: Two pictures the book earns slowly. **Left:** the projection $\hat{\mathbf{b}}$ is the shadow of $\mathbf{b}$ cast straight down onto the line ℓ; the purple dashed segment is the *error*, and it meets ℓ at a right angle – that perpendicularity is the entire secret of least squares. **Right:** most arrows get turned by a matrix (red $\mathbf{u} \to \mathbf{Au}$), but a special one, the *eigenvector* $\mathbf{v}$, is only stretched – here $\mathbf{Av} = 2\mathbf{v}$, doubled but not rotated.

Compute it yourself

A two-minute numeric preview, so the words above have a number behind them. Let $\mathbf{A} = \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Multiply, row by row: the top entry is $1.5 \cdot 1 + 0.5 \cdot 1 = 2$, and the bottom entry is $0.5 \cdot 1 + 1.5 \cdot 1 = 2$. So $\mathbf{Av} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} = 2\mathbf{v}$: the arrow doubled and did not turn. Now try $\mathbf{u} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$ and watch it get *bent* instead: $\mathbf{Au} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, which no longer points along the x-axis. You have just told an eigenvector from an ordinary vector by hand – the entire fifth part of the book is the slow, careful version of this one calculation.

How to read this book

A few practical suggestions, earned from watching people learn this material.

- **Keep a pen in your hand.** I will say it once more because it is the single highest-leverage habit: redo the small examples, attempt the purple labs, and sketch the figures yourself. Passive reading produces the illusion of understanding and very little of the real thing.
- **Let the orange questions breathe.** The walking questions are not graded and not urgent. Carry one around for a day. The understanding it produces is deeper than anything you can get by reading the next paragraph faster.
- **When you are lost, go back to green.** If a section dissolves into fog, find the nearest Intuition box and the nearest figure, and rebuild the picture before pushing on. Almost all confusion in this subject is a picture that slipped, not a calculation that broke.
- **Do the exercises.** Every chapter ends with a graded set, from warm-ups to genuine challenges, mixing computation, geometry, and a proof or two. They are where the chapter's ideas stop being things you have read and become things you can use. You do not need to do all of them,

but do not skip them wholesale – pick a warm-up, a geometry problem, and a hard one from each chapter at minimum.

- **It is fine to be confused for a while.** Confusion is expected; it is the feeling of your mind reshaping itself around a genuinely new idea. Every mathematician you admire has sat exactly where you are sitting, stuck on exactly the thing you are stuck on. Stay with it. Take a walk. Come back. The picture will steady.

History and connection

Take heart from the history. The objects in this book did not arrive clean, polished, or obvious. In 1679, Gottfried Wilhelm Leibniz wrote a visionary letter to Christiaan Huygens (the Dutch mathematician and physicist who developed the wave theory of light), expressing frustration with René Descartes' coordinate geometry (pinning every point down by numerical distances along rigid perpendicular axes x, y, z). Leibniz lamented that algebra could only speak of numbers, forcing mathematicians to chop geometric space into arbitrary coordinate lists. He dreamed of a true *characteristica geometrica* – a coordinate-free geometric language of position and direction that could operate directly on geometric figures and spatial motions without the artificial framework of numerical coordinates.

Huygens was unimpressed, dismissing the proposal as impractical, and Leibniz's dream languished for more than a century and a half. At the time, mathematicians possessed only synthetic Euclidean geometry (compass and straightedge) and Cartesian coordinates; they had no algebraic machinery for directed arrows or higher dimensions. In 1844, a quiet gymnasium (high school) teacher in Stettin named Hermann Grassmann published *Die Lineare Ausdehnungslehre* (The Theory of Linear Extension). Working in near-total isolation, Grassmann invented the core architecture of modern linear algebra: abstract n-dimensional spaces (spaces with more than three independent directions of movement), linear independence (sets of directions with no redundant arrows), and span (the territory reachable by scaling and adding vectors). In the same work he introduced bases (minimal sets of reference arrows defining a coordinate system), dimension (the count of independent directions), and linear transformations (motions of space that keep straight lines straight and the origin fixed).

The mathematical establishment's reaction was brutal: total silence. August Ferdinand Möbius (famed for the one-sided Möbius strip) declined to review it, writing that Grassmann's philosophical prose gave him a headache. Carl Friedrich Gauss (one of the greatest mathematicians in history) praised Grassmann's talent in private correspondence but noted that he had contemplated similar ideas decades prior without publishing them. Grassmann's book sat unread on publisher shelves for thirty years; deeply discouraged, he abandoned mathematics altogether to master Sanskrit linguistics, where he achieved international renown. It was only in the late 1870s and 1880s, when Giuseppe Peano (who wrote down the formal axioms of vector spaces) and Josiah Willard Gibbs (the physicist who adapted vectors for electromagnetism) popularized his ideas, that the mathematical world realized Grassmann had single-handedly built the geometry of the future.

Grassmann's insights extended geometric thinking beyond three dimensions, creating a universal language that applies equally well to physical velocities, economic inputs, quantum states, and high-dimensional data. Yet pure coordinate-free geometry, for all its conceptual elegance, is notoriously difficult to calculate with directly. Human spatial intuition falters past three dimensions, and an algebra of pure directional rays cannot balance an engineering load, predict an asteroid orbit, or train a computer model without concrete numerical arrays, coordinate

bases, and matrix algorithms. Bridging that divide between geometric intuition and practical calculation is the plan: Chapter 1 gives the executive view of what linear algebra is about, and Chapter 2 grounds vectors as geometric steps and lists of numbers.

Connection to other math

One last reason to keep going, for the reader who wonders where this leads. The "keep the grid straight" idea at the center of this book does not stop at arrows in the plane. Replace arrows with *functions* and the same machinery becomes *functional analysis*, the language of quantum mechanics and of the Fourier transform (which breaks complicated signals down into pure sine and cosine frequencies) that lives inside every audio and image codec. Replace addition with *composition* and you brush up against *group theory* (the formal mathematics of symmetry). Linear algebra sits at the center of pure and applied mathematics, connecting geometry, analysis, probability, optimization, and computation.

Warm-up exercises

A preface has no theorems to drill, but it can still ask you to pick up a pen – which is what linear algebra is about. None of these needs anything past high-school algebra and the two figures above. Treat them as a gentle handshake with the book's habits: *draw first, compute second, and trust the picture once the numbers agree with it.*

1. **(Mental check: scaling to zero.)** If you take any vector $\mathbf{v} = \begin{bmatrix} x \\ y \end{bmatrix}$ and scale it by 0, where does the resulting arrow land? Can any choice of starting vector $\mathbf{v}$ avoid ending up at the origin? Now think in reverse: what happens if you start at the origin vector $\mathbf{0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and scale it by a million?

2. **(Mental check: does order matter?)** Suppose you stretch a vector to three times its original length, and then rotate it 90° counterclockwise. A friend takes the identical starting vector, rotates it 90° counterclockwise first, and then stretches it to three times its length. Without writing down any numbers, picture both sequences of moves in your head. Do you and your friend arrive at the exact same arrow? Why is this a hallmark of a *linear* operation?

3. **(Tip to tail.)** On graph paper draw $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\mathbf{w} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$ from the origin. Form $\mathbf{v} + \mathbf{w}$ by the tip-to-tail rule, read its coordinates off the page, and then check by adding componentwise. Do the two answers match? They had better.

4. **(Scaling stays on a line.)** With the same $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, plot $\frac{1}{2}\mathbf{v}$, $2\mathbf{v}$, and $-\mathbf{v}$. Convince yourself by eye that all four tips lie on a single line through the origin. Which one points the opposite way, and why?

5. **(Rotation preserves length, again.)** The text claimed a 90° turn sends $\begin{bmatrix} x \\ y \end{bmatrix}$ to $\begin{bmatrix} -y \\ x \end{bmatrix}$. Apply it to $\begin{bmatrix} 4 \\ 3 \end{bmatrix}$, then to the result, then twice more. After four turns, where are you? Compute the

length $\sqrt{x^2 + y^2}$ at each step and confirm it never changes.

6. **(A shear, by hand.)** Using $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ from Figure 3, compute where the four corners of the unit square $\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ land. Plot the image. You should recover the slanted parallelogram of the figure – and you can check its area is still 1 by counting base times height.

7. **(Spot the eigenvector.)** For $\mathbf{A} = \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{bmatrix}$, the purple lab found that $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ is merely stretched. Test the candidate $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ the same way: compute $\mathbf{A} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Is it a scalar multiple of $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$? If so, by what factor? (You will have found this matrix's *second* eigenvector without any of the machinery – a small preview of how clean Part VI turns out to be.)

8. **(A walking question, on paper for once.)** You have two arrows in the plane, not pointing along the same line. Argue in a sentence or two why you can reach *every* point of the plane by scaling and adding them – and what goes wrong the moment the two arrows *do* share a line. This is the seed of *span* and *independence*; say it in your own words now, and watch the book sharpen your wording later.

A note of welcome

Linear algebra has a reputation for being dry – a gauntlet of row reductions to survive on the way to something more interesting. I think that reputation is a tragedy of bad first encounters, and this book is my attempt to give you a good one. Underneath the row reductions is a subject of startling beauty and almost unreasonable usefulness: the mathematics that lets us understand data, render worlds, model the physical universe, and – right now, in our moment – build machines that learn. You are about to see how a single, simple idea about keeping space straight blossoms into all of that.

So sharpen a pencil. Find some scrap paper. We are going to start, quite literally, from the ground up – first with the bird's-eye view of why linear mathematics works so well, and then with a single arrow drawn from the origin and the two simplest things you can do to it.

Welcome. Let us begin.

Contents

Preface **iii**

I Seeing Vectors (Before Strang) **1**

1 What Linear Algebra Is Really About **3**

1.1 What Makes Linearity Special . . . 3

1.2 Vectors: Things You Can Scale and Add . . . 5

1.2.1 Vectors that are not arrows . . . 7

1.3 Linear Combinations and Span: Reachable Territory . . . 7

1.4 The Two Views of a Matrix: Rows and Columns . . . 9

1.4.1 The row picture: intersecting lines . . . 9

1.4.2 The column picture: mixing ingredients . . . 10

1.5 Linear Transformations: Matrices That Move Space . . . 11

1.5.1 A gallery of transformations . . . 12

1.5.2 Composition is matrix multiplication . . . 13

1.6 Where This All Leads . . . 14

1.7 Exercises . . . 17

2 Vectors: Arrows, Lists, and Points **19**

2.1 An Arrow Is a Step, Not a Place . . . 19

2.2 A List Is How You Compute With an Arrow . . . 21

2.3 Adding Vectors: Walk, Then Walk Again . . . 21

2.4 Scaling Vectors: Stretch, Shrink, Flip . . . 23

2.5 A Point Is Where an Arrow's Tip Lands . . . 24

2.6 The Zero Vector: Standing Still . . . 25

2.7 From the Plane to $\mathbb{R}^n$ . . . 26
2.8 Unit Vectors and the Standard Basis . . . 28
2.9 Vectors as Data: A Row of Features . . . 30
2.10 Subtraction and the Arrow Between Two Points . . . 32
2.11 What to Carry Forward . . . 33
2.12 Exercises . . . 35

3 Linear Combinations, Span, and Geometry 37
3.1 Scaling and Adding: The Two Moves . . . 37
3.2 The Linear Combination . . . 38
3.3 Span: The Set of All Combinations . . . 39
3.3.1 Span of one vector: a line (usually) . . . 40
3.3.2 Span of two vectors: a plane (usually) . . . 40
3.3.3 Span of three vectors: all of space (usually) . . . 41
3.4 A Fully Worked Span Question . . . 42
3.5 The Column Picture: $\mathbf{Ax}$ as a Combination of Columns . . . 43
3.5.1 What the column picture says about solving $\mathbf{Ax} = \mathbf{b}$ . . . 45
3.6 Reaching a Target $\mathbf{b}$ . . . 45
3.7 When a New Vector Adds Nothing: Dependence, Previewed . . . 47
3.7.1 Counting: why you cannot span $\mathbb{R}^3$ with two vectors . . . 49
3.8 Summary . . . 51
3.9 Exercises . . . 52

4 Dot Products, Length, and Angle 55
4.1 The dot product, two ways . . . 55
4.1.1 The algebraic definition . . . 55
4.1.2 Length lives inside the dot product . . . 57
4.2 The geometric meaning: length times length times cosine . . . 58
4.2.1 Deriving the cosine formula from the law of cosines . . . 58
4.2.2 Solving for the angle . . . 59
4.3 Orthogonality: perpendicular means dot product zero . . . 60
4.3.1 The Pythagorean theorem, reborn . . . 61
4.4 Unit vectors and direction . . . 62

4.5 A first look at projection . . . 63
4.6 Two great inequalities . . . 64
4.6.1 Cauchy–Schwarz . . . 64
4.6.2 The triangle inequality . . . 65
4.7 Modern application: cosine similarity . . . 66
4.8 Worked summary example . . . 68
4.9 Exercises . . . 70

5 Matrices as Machines: Linear Transformations 73
5.1 A Matrix Is a Function on Vectors . . . 73
5.1.1 The picture: dots moving to dots . . . 75
5.2 Linearity: The One Property That Makes a Matrix a Matrix . . . 75
5.2.1 Why matrices are automatically linear . . . 77
5.3 What the Columns Tell You . . . 77
5.3.1 From basis images to the whole map . . . 78
5.3.2 Columns and the unit square . . . 79
5.4 The Action on the Grid . . . 80
5.5 A Tour of 2×2 Machines . . . 81
5.5.1 Scalings: stretch and squash . . . 81
5.5.2 Rotations . . . 82
5.5.3 Reflections . . . 84
5.5.4 Shears . . . 84
5.5.5 The identity and the zero map . . . 86
5.6 Composition: Doing One Machine After Another . . . 86
5.7 Cost and Intuition Recap . . . 88
5.8 Exercises . . . 89

II Solving Linear Systems 93

6 Systems of Linear Equations and Their Geometry 95
6.1 Two pictures of one system . . . 96
6.1.1 The row picture: lines that cross . . . 96
6.1.2 The column picture: arrows that add up . . . 97

6.1.3 Matrix form: the whole thing as $A\mathbf{x} = \mathbf{b}$ . . . 98
6.2 A fully worked 2×2, both ways . . . 99
6.3 The three possibilities, and why there are only three . . . 100
6.3.1 One solution: lines cross, columns spread . . . 100
6.3.2 No solution: parallel lines, collinear columns missing the target . . . 100
6.3.3 Infinitely many: same line twice, target on the column line . . . 101
6.3.4 Why never exactly two? . . . 102
6.4 Singular versus nonsingular: the determinant in 2D . . . 102
6.4.1 Deriving the 2×2 inverse formula . . . 104
6.5 Up to three dimensions: planes instead of lines . . . 104
6.5.1 A worked 3×3 with a clean answer . . . 105
6.5.2 How a 3×3 system can fail . . . 105
6.6 The homogeneous shadow: $A\mathbf{x} = \mathbf{0}$ . . . 106
6.7 A modern application: when "solve $A\mathbf{x} = \mathbf{b}$" is the whole job . . . 108
6.8 Building intuition that lasts . . . 109
6.9 Summary . . . 109
6.10 Exercises . . . 111

7 Gaussian Elimination **113**
7.1 The picture: collapsing planes onto a corner . . . 113
7.1.1 The three legal moves . . . 114
7.1.2 From equations to a grid of numbers . . . 115
7.2 Elimination to upper-triangular form . . . 115
7.2.1 Pivots and multipliers . . . 115
7.2.2 A 3x3 traced fully . . . 116
7.2.3 Back-substitution: climbing the staircase . . . 117
7.3 Counting the work, and why triangles are cheap . . . 118
7.4 Row exchanges: when the pivot is in the wrong place . . . 119
7.5 What goes wrong: zero pivots that will not move . . . 120
7.5.1 Case one: no solution (an inconsistent system) . . . 120
7.5.2 Case two: infinitely many solutions . . . 121
7.6 Reduced row echelon form: the cleanest possible answer . . . 122
7.6.1 From triangle to RREF, traced . . . 123

7.7 Elimination is matrix multiplication . . . 124
7.7.1 The replacement matrix . . . 124
7.7.2 The other two as matrices . . . 125
7.7.3 The whole elimination as one product . . . 125
7.7.4 Where permutations land: $\mathbf{PA} = \mathbf{LU}$ . . . 126
7.8 The full algorithm, in one place . . . 126
7.8.1 One more 3x3, with a twist . . . 127
7.9 What elimination is really telling you . . . 128
7.10 Exercises . . . 130

8 Matrix Multiplication and Inverses 133
8.1 Where Matrix Multiplication Comes From . . . 133
8.1.1 Composition forces the rule . . . 133
8.1.2 Why the shapes have to match . . . 135
8.2 Four Views of the Same Product . . . 136
8.2.1 View 1: the dot-product (entry) view . . . 137
8.2.2 View 2: the column view (combinations of **A**'s columns) . . . 137
8.2.3 View 3: the row view (combinations of **B**'s rows) . . . 138
8.2.4 View 4: the outer-product (sum-of-rank-one) view . . . 138
8.3 The Algebra of Multiplication . . . 140
8.3.1 It is associative (and the reason is composition) . . . 140
8.3.2 It is NOT commutative . . . 140
8.3.3 The rules that DO hold . . . 142
8.4 The Identity: the Machine That Does Nothing . . . 142
8.5 The Inverse: Undoing a Machine . . . 142
8.5.1 The 2×2 formula . . . 143
8.5.2 The geometry of singular: area collapse . . . 144
8.5.3 The big equivalence (collect everything) . . . 144
8.6 Computing the Inverse with Gauss–Jordan . . . 145
8.6.1 The idea: solve n systems at once . . . 145
8.6.2 A full 3×3 worked example . . . 146
8.6.3 What singular looks like in the algorithm . . . 148
8.7 The Inverse of a Product: Socks and Shoes . . . 148

8.8 Exercises . . . 151

9 LU Factorization **153**

9.1 The picture: triangular machines are easy machines . . . 153
9.2 Elimination, watched closely . . . 155
9.3 Why it works: elimination is left-multiplication by shears . . . 156
9.4 Solving $\mathbf{Ax} = \mathbf{b}$: two slides instead of one sweep . . . 158
9.5 Many right-hand sides: the real reason to factor . . . 160
9.6 Row exchanges: when a pivot is missing . . . 161
9.6.1 Pivoting for Numerical Stability . . . 162
9.7 Pulling out the pivots: $\mathbf{A} = \mathbf{LDU}$ . . . 163
9.8 The cost of elimination, counted exactly . . . 164
9.9 When LU fails, and when it does not . . . 165
9.10 What to carry forward . . . 166
9.11 Exercises . . . 168

10 Transposes and Permutations **171**

10.1 The transpose: flipping across the diagonal . . . 171
10.1.1 The definition, with a picture . . . 171
10.1.2 The transpose turns a column into a dot-product machine . . . 173
10.2 The algebra of transposes . . . 173
10.3 The transpose is the adjoint . . . 175
10.4 Symmetric matrices . . . 177
10.4.1 When a matrix equals its own mirror image . . . 177
10.4.2 $\mathbf{A}^\mathsf{T}\mathbf{A}$ is always symmetric — and it is everywhere . . . 177
10.4.3 Splitting any square matrix into symmetric plus antisymmetric . . . 179
10.5 Permutation matrices: shuffling without distorting . . . 179
10.5.1 Reordering the rows of the identity . . . 179
10.5.2 Multiplying by $\mathbf{P}$ on the left vs. on the right . . . 180
10.5.3 How many are there, and how do they multiply? . . . 181
10.6 The inverse of a permutation is its transpose . . . 181
10.7 Putting transposes and permutations together . . . 183
10.7.1 Looking forward: from matrix arithmetic to vector spaces . . . 184

10.8 Exercises . . . 186

III Vector Spaces **189**

11 Vector Spaces and Subspaces **191**

11.1 What "space" is supposed to capture . . . 191

11.1.1 The two operations we keep using . . . 191

11.1.2 Why bother abstracting? . . . 192

11.2 The axioms of a vector space (made friendly) . . . 192

11.2.1 Two small theorems you get for free . . . 194

11.3 The home example: $\mathbb{R}^n$ . . . 195

11.4 Spaces that are not columns of numbers . . . 196

11.4.1 Matrices form a vector space . . . 196

11.4.2 Polynomials form a vector space . . . 197

11.4.3 Functions form a vector space . . . 197

11.5 Subspaces: vector spaces hiding inside vector spaces . . . 198

11.5.1 The subspace test . . . 199

11.5.2 A worked subspace and a worked non-subspace . . . 199

11.6 The complete list of subspaces of $\mathbb{R}^3$ . . . 201

11.7 Spans are subspaces . . . 202

11.8 Operations on subspaces . . . 204

11.9 Looking ahead . . . 205

11.9.1 The bridge to Chapter 12: how matrices manufacture subspaces . . . 205

Exercises . . . 208

12 Column Space and Nullspace **211**

12.1 The column space: what can come out . . . 211

12.1.1 Reading a matrix–vector product as a combination of columns . . . 211

12.1.2 Definition and the picture . . . 212

12.1.3 Why the column space is a subspace . . . 213

12.1.4 The shape of the column space depends on the columns . . . 214

12.2 The nullspace: what gets crushed to zero . . . 215

12.2.1 Definition and the picture . . . 215

12.2.2 Why the nullspace is a subspace . . . 215
12.3 Finding the nullspace by elimination . . . 216
12.3.1 Elimination preserves the nullspace . . . 216
12.3.2 Pivots, free variables, and special solutions . . . 217
12.3.3 Why the special solutions span the whole nullspace . . . 218
12.3.4 A rank-one example tying both spaces together . . . 219
12.4 The two spaces as a solvability ledger . . . 220
12.5 A modern application and a connection . . . 222
12.6 Reading the column space off the echelon form (a caution) . . . 223
12.7 Summary . . . 225
12.8 Exercises . . . 226

13 The Complete Solution to Ax = b **229**
13.1 One picture to hold onto . . . 229
13.2 Why particular plus nullspace is everything . . . 230
13.3 Existence: is **b** in the column space? . . . 232
13.4 Uniqueness: is the nullspace trivial? . . . 233
13.5 Rank decides the shape . . . 234
13.5.1 The four cases, one at a time . . . 235
13.6 A full worked example: building the complete solution . . . 236
13.7 Reading the four cases off your worked example . . . 237
13.8 The two “nice” rectangular extremes . . . 238
13.8.1 Full column rank: at most one solution . . . 238
13.8.2 Full row rank: always at least one solution . . . 239
13.9 Recipe: the complete solution in five steps . . . 240
13.10 Counting, sanity checks, and what can go wrong . . . 241
13.11 Summary . . . 242
13.12 Exercises . . . 243

14 Independence, Basis, and Dimension **245**
14.1 The redundant vector: a picture of dependence . . . 245
14.2 Linear independence, defined precisely . . . 246
14.3 The independence test you will actually run . . . 248

14.4 Spanning sets: reaching everything . . . 250
14.5 Basis: independent and spanning . . . 251
14.6 Dimension is well-defined . . . 253
14.7 Coordinates in a basis . . . 255
14.8 A basis for the column space . . . 256
14.9 A basis for the nullspace . . . 258
14.10 Bases in other vector spaces . . . 259
14.11 Putting it together . . . 260
Exercises . . . 261

15 The Four Fundamental Subspaces 263
15.1 Two spaces for the inputs, two for the outputs . . . 263
15.2 The column space: where the outputs live . . . 264
15.3 The nullspace: inputs mapped to zero . . . 266
15.4 The row space: inputs that govern the output . . . 267
15.5 The left nullspace: the unreachable output directions . . . 268
15.6 Orthogonality: the two spaces in each room are perpendicular . . . 270
15.7 The big picture . . . 271
15.8 Reading all four bases off one elimination . . . 273
15.9 A modern application: where data lives and what is unreachable . . . 275
15.10 Worked summary example, all four at once . . . 275
15.11 What we now know . . . 278
15.12 Exercises . . . 278

16 Rank and the Fundamental Theorem of Linear Algebra 281
16.1 Rank is the true size of a matrix . . . 281
16.1.1 Reading rank off the echelon form . . . 283
16.2 Rank-nullity: the conservation law . . . 283
16.2.1 Why it is true, from the echelon form . . . 284
16.3 Row rank equals column rank . . . 285
16.3.1 The decisive picture: **A** as a short product . . . 286
16.4 The four subspaces, paired by perpendicularity . . . 288
16.5 Rank-one matrices: elementary outer products . . . 291

16.5.1 Every matrix is a sum of rank-one pieces . . . 292
16.6 Exercises . . . 294

IV Orthogonality **297**

17 Orthogonality and Orthogonal Subspaces **299**
17.1 The dot product remembers the angle . . . 299
17.2 From vectors to subspaces . . . 301
17.3 The orthogonal complement . . . 302
17.4 The key identity: $\text{null}(\mathbf{A}) = \text{row}(\mathbf{A})^{\perp}$ . . . 304
17.5 The two orthogonal pairs among the four subspaces . . . 305
17.6 Orthonormal sets and coordinate expansions . . . 307
17.7 Orthogonal matrices and isometries . . . 309
17.7.1 What orthogonal matrices preserve . . . 310
17.8 Splitting a vector into perpendicular parts . . . 312
17.9 Exercises . . . 314

18 Projections **317**
18.1 Projecting onto a line . . . 317
18.1.1 Finding the projection from the right angle . . . 318
18.1.2 Packaging the projection as a matrix . . . 319
18.2 Why the perpendicular is the closest point . . . 320
18.3 Projecting onto a subspace . . . 321
18.3.1 The normal equations . . . 321
18.3.2 Why $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ is invertible when the columns are independent . . . 323
18.4 The two properties that define a projection . . . 323
18.5 A fully worked plane in three dimensions . . . 325
18.6 Two special cases worth checking . . . 327
18.7 Where this is heading: the inconsistent system . . . 328
18.8 What to carry forward . . . 329
18.9 Exercises . . . 330

19 Least Squares and Regression **333**

19.1 Why $\mathbf{Ax} = \mathbf{b}$ usually has no solution . . . 333
19.2 What "closest" should mean: the error vector . . . 335
19.3 The geometry: projection onto the column space . . . 336
19.4 Deriving the normal equations . . . 337
19.5 Worked example: fitting a line . . . 340
19.6 Fitting a parabola, and any other model linear in its parameters . . . 342
19.7 Measuring the fit: residuals and R^2 . . . 343
19.8 Least squares as the engine of statistics and machine learning . . . 345
19.9 The projection matrix, revisited . . . 346
19.10Summary . . . 347

20 Gram-Schmidt and QR **351**

20.1 The wish: straighten a crooked basis . . . 351
20.2 Gram-Schmidt for two vectors . . . 352
20.3 Gram-Schmidt in general . . . 354
20.4 A fully worked three-vector example . . . 355
20.5 The QR factorization . . . 357
20.5.1 QR for our three-vector example . . . 358
20.5.2 Properties of orthogonal matrices . . . 359
20.6 Why orthonormal columns make least squares easy . . . 359
20.7 Classical versus modified Gram-Schmidt . . . 361
20.8 Cost and intuition recap . . . 363
20.9 Exercises . . . 364

V Determinants **367**

21 Determinants: Volume and Sign **369**

21.1 The Unit Box and Its Image . . . 369
21.2 The 2×2 Determinant: Deriving $ad - bc$ . . . 370
21.3 Sign and Orientation: Tracking Reflections . . . 373
21.4 $\det = 0$ Means Collapse . . . 374
21.5 The 3×3 Determinant: Volume of the Box . . . 376
21.6 How Row Operations Change the Determinant . . . 377

21.7 The Product Rule: $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$ . . . 379
21.8 Putting It Together . . . 382
21.9 Exercises . . . 383

22 Cofactors and Cramer's Rule **385**
22.1 Peeling a Determinant Apart: Minors and Cofactors . . . 386
22.1.1 The Cofactor Expansion Theorem . . . 387
22.1.2 A worked 3×3 expansion . . . 387
22.1.3 Cofactors and the cost of computing . . . 389
22.2 The Determinant of Special Matrices . . . 390
22.2.1 Triangular matrices: just multiply the diagonal . . . 390
22.2.2 A worked diagonal and triangular check . . . 391
22.2.3 A few more determinants you can read off . . . 391
22.3 The Adjugate and the Inverse Formula . . . 392
22.3.1 Why the adjugate identity holds . . . 393
22.3.2 The 2×2 inverse, finally explained . . . 394
22.3.3 A full 3×3 inverse by cofactors . . . 394
22.4 Cramer's Rule . . . 395
22.4.1 Where it comes from . . . 396
22.4.2 A worked 2×2 system . . . 396
22.4.3 A worked 3×3 system . . . 397
22.5 The Geometry of Cramer's Rule: Each Unknown Is a Ratio of Volumes . . . 397
22.6 When These Formulas Help, and When Elimination Wins . . . 399
22.7 Looking Back . . . 401
22.8 Exercises . . . 401

VI Eigenvalues **403**

23 Eigenvalues and Eigenvectors **405**
23.1 Directions a matrix does not turn . . . 405
23.2 Finding the eigenvalues: the characteristic equation . . . 408
23.2.1 The full recipe, walked through twice . . . 408
23.3 Triangular and diagonal matrices: eigenvalues for free . . . 410

23.4 Trace and determinant: the eigenvalues' fingerprints . . . 411
23.5 What the eigenvalues do to the picture . . . 412
23.6 Special matrices and their eigenvalues . . . 414
23.7 Complex eigenvalues: when a matrix is really a rotation . . . 416
23.8 Why eigenvectors of distinct eigenvalues are independent . . . 418
23.9 A worked 3×3 . . . 419
23.10Pulling it together . . . 422
Exercises . . . 423

24 Diagonalization **425**
24.1 The Big Idea: Work in the Eigenbasis . . . 425
24.2 The Factorization $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$ . . . 426
24.2.1 A first worked example . . . 428
24.3 Why Powers Become Easy . . . 429
24.3.1 The long-run behavior you can read off . . . 430
24.4 The Geometry: Three Clean Steps . . . 431
24.5 Which Matrices Diagonalize? Independent Eigenvectors . . . 432
24.5.1 Algebraic versus geometric multiplicity . . . 433
24.6 Defective Matrices: When Diagonalization Fails . . . 434
24.7 Application: Solving the Fibonacci Recurrence . . . 436
24.8 The Recipe and the Cautions . . . 438
24.8.1 Cost and intuition recap . . . 440
24.8.2 Looking forward: continuous systems and the matrix exponential . . . 440
Exercises . . . 441

25 Differential Equations and the Matrix Exponential **443**
25.1 The One-Dimensional Story First . . . 444
25.2 Coupled Equations: Why a Matrix Appears . . . 445
25.3 One Eigenvector at a Time: Decoupling by Modes . . . 446
25.3.1 Adding modes: the general solution by superposition . . . 447
25.3.2 A fully worked example . . . 447
25.4 Stability: Reading the Future from the Eigenvalues . . . 449
25.5 The Matrix Exponential: One Object for the Whole Flow . . . 452

25.5.1 Properties, and the one trap . . . 453
25.6 Computing $e^{\mathbf{A}t}$ by Diagonalization . . . 454
25.7 When You Cannot Diagonalize . . . 456
25.8 A Modern Application . . . 457
25.9 The Geometry of the Flow . . . 458
Summary . . . 461
25.10Exercises . . . 461

26 Symmetric Matrices and the Spectral Theorem 463
26.1 What Makes a Matrix Symmetric . . . 464
26.1.1 Where symmetric matrices come from . . . 464
26.2 The Picture: Stretching Along Perpendicular Axes . . . 465
26.3 Real Eigenvalues . . . 466
26.4 Orthogonal Eigenvectors . . . 467
26.5 The Spectral Theorem . . . 468
26.5.1 Reading the factorization as a sum of projections . . . 469
26.5.2 The geometry: rotate, stretch, rotate back . . . 470
26.5.3 Why it is true: a proof sketch . . . 471
26.6 Computing the Spectral Decomposition by Hand . . . 472
26.7 The Spectral Theorem and Quadratic Forms . . . 473
26.8 Why Symmetry Is Everywhere: A Modern Application . . . 475
26.9 A Gallery of Symmetric Behaviors . . . 475
26.10Summary . . . 477
26.11Exercises . . . 477

27 Positive Definite Matrices 481
27.1 The energy of a matrix . . . 481
27.1.1 From a number to a matrix . . . 481
27.1.2 The definition . . . 482
27.1.3 Three quadratic surfaces . . . 483
27.2 Test 1: eigenvalues . . . 484
27.3 Test 2: pivots . . . 484
27.4 Test 3: leading minors (the determinant test) . . . 486

27.5 The geometry: ellipses and ellipsoids . . . 487
27.6 Cholesky: the square root of a matrix . . . 488
27.7 Positive definiteness in optimization . . . 490
27.8 Covariance: positive definiteness in statistics . . . 491
27.9 A gallery of consequences . . . 492
27.10Exercises . . . 494

28 Similarity and Jordan Form **497**
28.1 Similarity: The Same Operator in a New Basis . . . 498
28.1.1 The change-of-basis picture . . . 498
28.1.2 Similarity is an equivalence relation . . . 499
28.2 What Similar Matrices Share . . . 499
28.2.1 Eigenvalues, determinant, trace, rank . . . 499
28.2.2 Eigenvectors transform too . . . 500
28.3 When Diagonalization Fails . . . 501
28.3.1 Algebraic versus geometric multiplicity . . . 501
28.3.2 The canonical defective matrix . . . 502
28.4 Generalized Eigenvectors: Filling the Gap . . . 502
28.4.1 What the chain does to the matrix . . . 503
28.5 The Jordan Form . . . 504
28.5.1 Jordan blocks . . . 504
28.5.2 The big theorem . . . 504
28.5.3 A worked 3×3 example . . . 506
28.6 The Geometry of a Jordan Block . . . 508
28.6.1 Nilpotent matrices are pure Jordan . . . 509
28.7 A Modern Application: Stability of Iterations . . . 510
28.8 Putting It All Together . . . 511
28.9 Exercises . . . 513

VII The SVD and Beyond (After Strang) **515**

29 The Singular Value Decomposition **517**
29.1 The Picture First: What a Matrix Does to a Circle . . . 517

29.2 From the Picture to the Factorization . . . 519
29.3 The Formal Statement . . . 520
29.3.1 The economy (thin) SVD . . . 521
29.4 Where the Pieces Come From: $\mathbf{A}^\mathsf{T}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^\mathsf{T}$ . . . 522
29.4.1 Why the recipe is forced (a real proof) . . . 523
29.5 A Fully Worked 2×2 Example . . . 524
29.6 The SVD and the Four Fundamental Subspaces . . . 525
29.7 Best Low-Rank Approximation: Eckart–Young . . . 527
29.7.1 Why the Frobenius error is what it is . . . 528
29.8 The Pseudoinverse: Solving the Unsolvable . . . 529
29.9 Symmetry, Eigenvalues, and the SVD . . . 530
29.10How It Is Computed (and Why Not the Obvious Way) . . . 532
29.11Summary: One Picture, Many Doors . . . 533
29.12Exercises . . . 533

30 Principal Component Analysis and Data 535

30.1 Data as a cloud of points . . . 536
30.1.1 The data matrix . . . 536
30.1.2 The mean, and why we center . . . 537
30.2 The covariance matrix . . . 538
30.2.1 Variance along one feature . . . 538
30.2.2 Covariance: do two features move together? . . . 538
30.2.3 Packing every covariance into one matrix . . . 539
30.3 Principal components: the directions of greatest spread . . . 540
30.3.1 The variance of a projection . . . 540
30.3.2 Maximize the spread: an eigenvalue problem . . . 541
30.3.3 Doing it by hand on the five points . . . 543
30.4 Variance explained and dimensionality reduction . . . 543
30.4.1 The total variance is the trace . . . 543
30.4.2 Reducing dimensions: 2D down to 1D . . . 544
30.4.3 Two faces of the same optimum . . . 545
30.4.4 How many components? The scree plot . . . 546
30.5 PCA is the SVD of the centered data . . . 547

30.5.1 Why a practitioner never forms **C** . . . 547
30.5.2 Low-rank approximation is dimensionality reduction . . . 548
30.6 PCA in the wild . . . 549
30.6.1 Compressing and denoising images . . . 549
30.6.2 Eigenfaces . . . 549
30.7 The recipe, start to finish . . . 550
Exercises . . . 552

31 Linear Transformations and Change of Basis **555**
31.1 The Operator and Its Matrix Are Two Different Things . . . 555
31.2 Linear Maps, Abstractly . . . 557
31.3 Coordinates: A Vector's Address in a Basis . . . 559
31.4 The Matrix of a Transformation in Given Bases . . . 560
31.4.1 A non-square example: bases on input and output . . . 562
31.5 Change of Basis: Translating Between Two Address Systems . . . 563
31.5.1 The two pictures of a basis matrix . . . 564
31.6 The Same Operator in Different Coordinates . . . 565
31.7 Similarity, Revisited . . . 567
31.8 Choosing the Right Basis: The Natural Language of an Operator . . . 568
31.8.1 Eigenvectors: the basis that diagonalizes . . . 568
31.8.2 Singular vectors: the basis that works for every matrix . . . 569
31.9 Putting It All Together: One Operator in Three Bases . . . 570
31.10Summary . . . 571
Exercises . . . 573

32 Complex Vectors, Hermitian Matrices, and the Fourier Matrix **575**
32.1 Why real numbers are not enough . . . 576
32.1.1 A two-minute refresher on complex numbers . . . 576
32.2 Complex vectors and the right inner product . . . 577
32.2.1 The naive dot product fails . . . 578
32.2.2 The fix: conjugate one side . . . 578
32.2.3 Conjugate symmetry: the price of the fix . . . 579
32.3 The conjugate transpose . . . 580

32.4 Hermitian matrices: the complex symmetric . . . 580
32.4.1 Real eigenvalues, for free . . . 581
32.4.2 Orthogonal eigenvectors and the complex spectral theorem . . . 582
32.5 Unitary matrices: the complex orthogonal . . . 582
32.5.1 Unitary means length-preserving . . . 583
32.6 The discrete Fourier matrix . . . 584
32.6.1 Roots of unity: n points evenly on the circle . . . 585
32.6.2 Building the matrix . . . 586
32.6.3 Why **F** is (almost) unitary . . . 587
32.7 From the matrix to the FFT . . . 588
32.7.1 The one identity that makes it work . . . 588
32.8 A unified picture . . . 590
32.9 Unitary and Hermitian Matrices as the Language of Quantum Mechanics . . . 591
32.9.1 Quantum states as unit vectors in $\mathbb{C}^n$ . . . 591
32.9.2 Quantum gates as unitary matrices . . . 592
32.9.3 Physical observables as Hermitian matrices . . . 592
32.10Power and limits . . . 593
32.11Exercises . . . 594

VIII Linear Algebra in the Modern World 597

33 Numerical Linear Algebra 599

33.1 Floating point: numbers with a budget . . . 600
33.1.1 The cardinal sin: catastrophic cancellation . . . 601
33.2 Conditioning: how delicate is the problem? . . . 602
33.2.1 The condition number, made precise . . . 602
33.3 Stability: elimination versus QR . . . 605
33.3.1 Why elimination needs pivoting . . . 606
33.3.2 Backward stability, in one sentence . . . 607
33.3.3 Why QR provides numerical stability . . . 607
33.4 Iterative methods: solving without inverting . . . 608
33.4.1 Jacobi: everyone updates from last round . . . 609

33.4.2 Gauss-Seidel: use the news as soon as you hear it . . . 609
33.4.3 Conjugate gradient: the heavyweight . . . 611
33.5 The power method: finding the top eigenvalue . . . 612
33.6 Practical warnings to carry with you . . . 613
33.7 The computational horizon: where numerical precision meets learning . . . 614
33.8 Exercises . . . 615

34 The Linear Algebra of Machine Learning 617
34.1 Data is a matrix . . . 617
34.1.1 Centering and the mean as a vector . . . 619
34.1.2 The Gram matrix: all the dot products at once . . . 620
34.2 Linear regression: geometric least squares . . . 620
34.2.1 The setup . . . 620
34.2.2 Why projection? The column-space picture . . . 621
34.2.3 A fully worked tiny fit . . . 622
34.2.4 When the columns nearly collide: regularization . . . 623
34.3 Vectors that mean something: embeddings and similarity . . . 623
34.3.1 From words to vectors . . . 623
34.3.2 A worked similarity . . . 624
34.3.3 Word arithmetic and the meaning of differences . . . 625
34.4 A neural network layer is $\mathbf{W}\mathbf{x} + \mathbf{b}$, then a bend . . . 626
34.4.1 The two-step recipe . . . 626
34.4.2 Why the bend is non-negotiable . . . 626
34.4.3 A layer computed by hand . . . 628
34.4.4 Classification and logistic regression . . . 629
34.5 Training: gradient descent and the gradient as a vector . . . 630
34.5.1 The loss surface . . . 630
34.5.2 The gradient points uphill . . . 630
34.5.3 Why $-\nabla L$ is steepest: a one-line derivation . . . 631
34.5.4 Gradient descent on our regression . . . 631
34.6 Backpropagation: the multivariable chain rule as matrix multiplication . . . 632
34.6.1 The Jacobian: the local linear map . . . 632
34.6.2 Chain rule as Jacobian product . . . 632

34.6.3 Why multiply starting from the loss (backwards through the net) . . . 632
34.7 Attention: scaled dot products and matrix weights . . . 634
34.7.1 Queries, keys, values . . . 634
34.7.2 The core formula, read piece by piece . . . 634
34.8 Low rank: why a little structure goes a long way . . . 636
34.8.1 Rank as the number of independent directions . . . 636
34.8.2 Low-rank approximation: keep the loud directions . . . 636
34.8.3 Principal component analysis . . . 637
34.9 Exercises . . . 639

35 Graphs, Networks, and Markov Chains 641
35.1 From pictures to matrices . . . 641
35.1.1 The adjacency matrix . . . 642
35.1.2 Directed graphs and the adjacency matrix that is not symmetric . . . 643
35.1.3 The incidence matrix: edges meet nodes . . . 644
35.2 The graph Laplacian . . . 645
35.2.1 Why "Laplacian"? The connection to calculus . . . 646
35.2.2 The energy form: what $\mathbf{x}^{\mathsf{T}}\mathbf{L}\mathbf{x}$ measures . . . 647
35.3 Eigenvalues of the Laplacian: hearing connectivity . . . 648
35.3.1 The Fiedler value: connectivity by degree . . . 648
35.3.2 Weighted graphs . . . 650
35.4 Random walks and Markov chains . . . 650
35.4.1 Distributions evolve by matrix multiplication . . . 653
35.4.2 The stationary distribution is a left eigenvector . . . 654
35.4.3 When is there a unique stationary distribution it converges to? . . . 655
35.5 PageRank: importance as an eigenvector . . . 656
35.5.1 Two snags and the "teleport" that fixes both . . . 657
35.5.2 The power method: how it is actually computed . . . 658
35.6 Looking back . . . 659
Exercises . . . 660

36 Optimization and Quadratic Forms 663
36.1 Curved surfaces and their bottoms . . . 663

36.2 Quadratic forms: $\mathbf{x}^\top \mathbf{A}\mathbf{x}$ . . . 664
36.2.1 Why symmetric, and why the factor of two . . . 665
36.2.2 A first worked form and its level curves . . . 666
36.3 Diagonalizing the form: the principal axes . . . 667
36.4 Reading the shape: definiteness . . . 668
36.4.1 Checking definiteness without computing eigenvalues . . . 669
36.5 Gradients, Hessians, and where the marble stops . . . 670
36.6 Positive definite means convex means one bottom . . . 672
36.6.1 Solving the bowl directly . . . 673
36.7 Least squares is the gradient of a bowl . . . 674
36.8 Constrained optimization: a first look at Lagrange . . . 675
36.8.1 Optimization recovers eigenvalues . . . 676
36.9 Constrained convex optimization, duality, and the KKT conditions . . . 678
36.9.1 Why convexity matters geometrically . . . 678
36.9.2 Inequality constraints as physical walls . . . 679
36.9.3 The Karush–Kuhn–Tucker (KKT) conditions . . . 681
36.9.4 Lagrange duality and Slater's condition . . . 682
36.9.5 A worked two-dimensional quadratic program . . . 683
36.10Putting it together: a worked classification . . . 686
36.11What to carry forward . . . 686
Exercises . . . 687

37 Connections: Groups, Tensors, and Functional Analysis 691

37.1 Matrix groups: the geometry that linear algebra preserves . . . 692
37.1.1 $SO(2)$ is a circle, and that circle is the key to everything . . . 694
37.1.2 The complex cousin: unitary matrices . . . 696
37.1.3 Representation theory: letting a group act . . . 697
37.2 Tensors: linearity in several slots at once . . . 699
37.2.1 The dot product is a bilinear form . . . 699
37.2.2 The tensor product: building bigger spaces by multiplication . . . 700
37.2.3 The determinant is an alternating multilinear form . . . 701
37.3 Functional analysis: linear algebra in infinitely many dimensions . . . 702
37.3.1 Functions are vectors . . . 702

37.3.2 Fourier series: expanding in the orthonormal basis you cannot see 703
37.3.3 The repair: completeness and Hilbert space 704
37.4 Looking back, looking out 706
Exercises 707

IX Mathematical Background 709

38 Appendix: Sets, Functions, and Fields 711
38.1 Sets: collections of mathematical objects 711
38.1.1 What a set is, and the three things you do with it 711
38.1.2 Numbers you already trust, named 712
38.1.3 Combining sets 713
38.1.4 Ordered pairs and the product $\mathbb{R}^n$ 714
38.2 Functions: mappings between sets 715
38.2.1 Domain, codomain, rule 715
38.2.2 Injective, surjective, bijective 715
38.2.3 Why this is the spine of linear algebra 717
38.2.4 Inverses and composition 717
38.3 Fields: the numbers you are allowed to scale by 718
38.3.1 The question hiding behind "scalar" 718
38.3.2 The definition, friendly first 719
38.3.3 Who passes, who fails 720
38.3.4 Why $\mathbb{R}$ and why $\mathbb{C}$ 721
38.4 Just enough proof literacy 722
38.4.1 Reading the logical connectives 722
38.4.2 The handful of proof shapes 723
38.5 Power and limits 724
38.6 Exercises 725

39 Appendix: Sums, Complex Numbers, and Calculus 727
39.1 Sigma Notation: A Loop on Paper 727
39.1.1 The three rules that make sigma algebra work 728
39.1.2 Four closed-form sums worth memorizing 729

39.1.3 Double sums and why matrix multiplication looks the way it does 731
39.2 Complex Numbers: Rotation Made into Arithmetic 732
39.2.1 The number i and the complex plane 732
39.2.2 Arithmetic: add like vectors, multiply by rotating 734
39.2.3 Polar form and Euler's formula 735
39.3 Just Enough Calculus 737
39.3.1 The derivative as a slope, and as a best linear fit 737
39.3.2 From slope to optimization: where the derivative is zero 739
39.3.3 Gradients: the derivative for many variables 739
39.3.4 A little integration 741
39.4 Exercises 744

Part I

Seeing Vectors (Before Strang)

Chapter 1

What Linear Algebra Is Really About

In the Preface, we promised pictures before formulas – so before we compute a determinant or row-reduce a matrix, let us agree on what this whole subject is *for*. Most people meet linear algebra as a pile of procedures: solve this system, invert this matrix, find these eigenvalues. The procedures are real and we will learn them carefully. But if that is all you take away, you will have memorized the grammar of a language without ever hearing it spoken. In this book we start with the geometric *pictures*, so that each algebraic procedure has a clear visual meaning.

So here is the one-sentence version, which we will spend the rest of the chapter unpacking:

Focus

Linear algebra is the mathematics of **vectors**, the **linear combinations** you can build from them, and the **linear transformations** that move whole spaces around while keeping straight lines straight and the origin fixed. Almost everything else is a consequence of those three ideas.

If those terms are new, picture a **vector** as a displacement – an arrow pointing from here to there – and a **linear combination** as a recipe that scales a few arrows and lays them tip to tail. Picture a **linear transformation** as a rule that reshapes the entire plane or space while keeping grid lines straight and evenly spaced. By the end of this chapter, these three words will each carry a sharp, instinctive mental image, and you will understand why the adjective *linear* is the most useful kind of structure a piece of mathematics can have.

1.1 What Makes Linearity Special

Mathematics is full of functions. Some are wild: $\sin x$ wiggles forever, e^x explodes, $1/x$ blows up at the origin. Linear algebra deliberately restricts attention to well-behaved operations – and discovers that they govern a wide range of mathematics and practical applications.

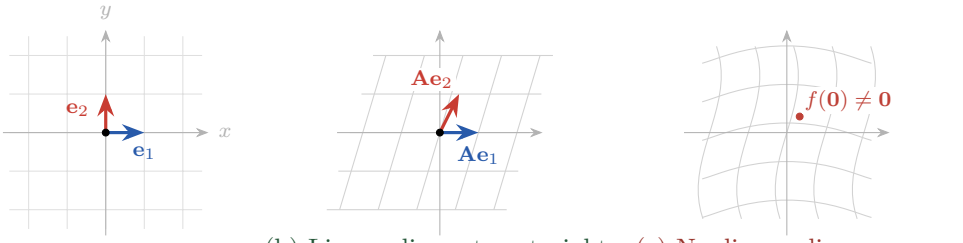
What does it mean for an operation to be *linear*? There are just two fundamental moves you are allowed to make: you may **scale** (stretch or shrink a thing by a number) and you may **add** (combine two things tip to tail). A linear operation is one that does not care about the order in which you do these: scaling-then-transforming gives the same answer as transforming-then-scaling, and the same for adding. Grid lines stay straight and evenly spaced.

Intuition

Picture a function f as a machine. It is *linear* if it respects two promises:

- **Scaling:** if you feed it twice as much, you get twice as much out. $f(2\mathbf{v}) = 2f(\mathbf{v})$.
- **Adding:** if you feed it $\mathbf{u}$ plus $\mathbf{v}$, you get $f(\mathbf{u})$ plus $f(\mathbf{v})$.

A linear machine cannot tell the difference between "combine first, then process" and "process each piece, then combine." That single property is what makes linear systems analytically tractable.



(a) Original grid

(b) Linear: lines stay straight, parallels stay parallel, **0** fixed

(c) Nonlinear: lines curve, spacing warps, origin wanders

Figure 1.1: The visual geometry of linearity. (a) A regular square Cartesian grid with unit steps $\mathbf{e}_1$ and $\mathbf{e}_2$ along the axes. (b) A linear transformation moves the grid but keeps all grid lines completely straight, keeps parallel lines parallel, and keeps the origin firmly pinned in place. (c) A nonlinear map bends and curves the grid, destroying the predictable combination structure.

Why should we care so much about such a restrictive family? There are three main reasons.

First, linear things are predictable. If I know what a linear machine does to a few basic inputs, I know what it does to *every* input, because every input is a combination of the basic ones and the machine respects combinations. You will see this turn into the entire theory of matrices: a matrix is simply a record of "what the machine did to the basic inputs," and that record is enough to reconstruct everything.

Second, the world is full of linear things, or things we choose to approximate as linear. The current through a circuit is a linear function of the voltages. The position of a robot arm is (locally) a linear function of its joint angles. The output of one layer of a neural network is a linear function of its input, wrapped in a simple nonlinear squish. Even when nature is curved, calculus tells us that *up close* everything looks linear – a derivative is exactly a "best linear approximation." In multivariable calculus, that local linear approximation is represented by a matrix of derivatives.

Third, linear problems are the ones we can actually solve. We have complete, reliable theories for linear systems. No comparably general theory exists for arbitrary nonlinear systems. So an enormous amount of applied mathematics follows a standard strategy: approximate a difficult nonlinear problem by a sequence of linear problems. Newton's method, gradient descent, the simplex algorithm, and the optimization routines that train modern machine learning models all rely on this local linearization.

Pause

Pause and test your intuition: Is the ordinary line $f(x) = 2x + 1$ a linear function in the sense of linear algebra?
In everyday speech and high-school algebra, we call anything with a straight graph "linear." But test our two promises:

- **Test scaling:** double the input from $x = 1$ to $x = 2$. At $x = 1$, the function gives $f(1) = 2(1) + 1 = 3$. If the machine scaled properly, doubling the input would double the output to 6. But at $x = 2$, we get $f(2) = 2(2) + 1 = 5 \neq 6$. The scaling promise is broken.
- **Test the origin:** feed in zero. We get $f(0) = 1 \neq 0$. But any genuinely linear map must keep the origin nailed down, because $f(0 \cdot x) = 0 \cdot f(x) = 0$.

So $f(x) = 2x + 1$ is what mathematicians call an *affine* function – a linear operation ($2x$) plus an offset shift ($+1$). In linear algebra, a function is linear only if it preserves scaling and addition, which forces its graph to pass directly through the origin.

Why this matters

"Linear" sounds like a restriction – no curves allowed – yet that restriction is exactly what makes the theory comprehensive. We can completely classify linear maps: determine when solutions exist, compute inverses, and decompose transformations into rotations and scalings. Few branches of mathematics are so thoroughly understood.

1.2 Vectors: Things You Can Scale and Add

Everything starts with the vector. You may have met vectors in physics as "arrows with magnitude and direction," and that picture is excellent – we will use it constantly. But the deeper definition is not about arrows at all. A vector is simply **a thing you are allowed to scale and add**, where scaling and adding obey the standard rules. The arrow is one example; there are many others, and recognizing them is a key skill.

Let us anchor the arrow picture first, because pictures come first in this book. Take the ordinary two-dimensional coordinate plane, written $\mathbb{R}^2$ because each point is specified by two real numbers (with $\mathbb{R}$ denoting the real number line). A vector $\mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ is an instruction: "go 3 to the right and 1 up." We draw it as an arrow from the origin to the point $(3, 1)$, as in Figure 1.2.

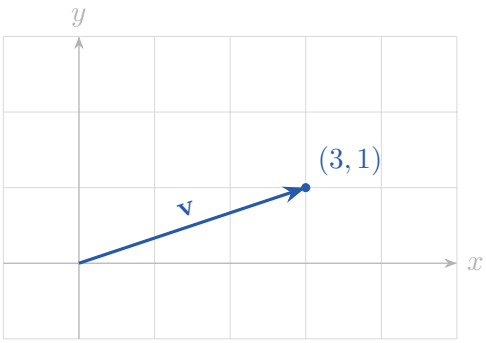


Figure 1.2: A vector in $\mathbb{R}^2$ as an arrow from the origin. The vector $\mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ encodes the instruction "3 right, 1 up."

Scaling. Multiply a vector by a number (a *scalar*) and you stretch the arrow. The number 2 doubles its length; $\frac{1}{2}$ halves it; -1 flips it to point the opposite way. The direction is preserved (or exactly reversed); only the length changes.

$$2\begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \end{bmatrix}, \qquad -\begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ -1 \end{bmatrix}, \qquad \tfrac{1}{3}\begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{1}{3} \end{bmatrix}.$$

Adding. To add two vectors, lay them tip to tail. Walk along the first, then continue along the second; the sum is the single arrow from where you started to where you ended.

$$\begin{bmatrix} 3 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}.$$

Figure 1.3 shows both operations side by side.

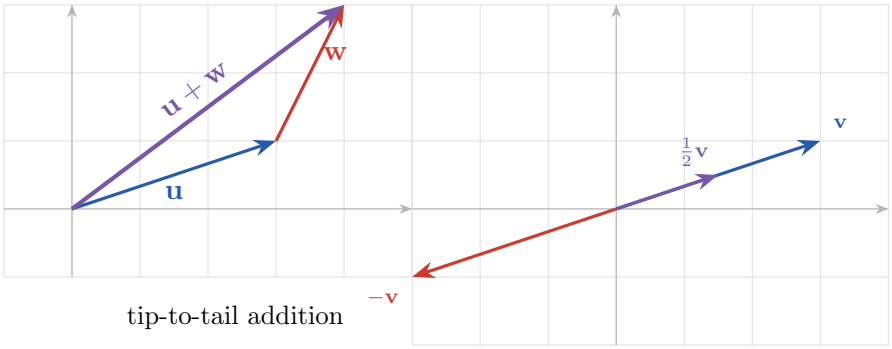


Figure 1.3: Left: vector addition is tip-to-tail. Right: scaling stretches ($\frac{1}{2}\mathbf{v}$) or flips ($-\mathbf{v}$) without changing the line the vector lives on.

That is the entire arithmetic of vectors: scale and add. Notice we never multiplied two vectors together to get a third vector – that is deliberately *not* part of the basic structure (it comes later, and in more than one flavor). The plainness is the point.

Formal

A **real vector space** is a set V together with an addition $\mathbf{u}+\mathbf{v}$ and a scalar multiplication $c\,\mathbf{v}$ (with $c \in \mathbb{R}$) such that addition is commutative and associative, there is a zero vector $\mathbf{0}$ with $\mathbf{v}+\mathbf{0}=\mathbf{v}$, every $\mathbf{v}$ has a negative $-\mathbf{v}$, and scaling distributes over both kinds of sum: $c(\mathbf{u}+\mathbf{v}) = c\mathbf{u}+c\mathbf{v}$ and $(c+d)\mathbf{v} = c\mathbf{v}+d\mathbf{v}$, with $1\,\mathbf{v}=\mathbf{v}$. The familiar example is $\mathbb{R}^n$, the set of columns of n real numbers, but the definition deliberately mentions no arrows – so that other things can qualify.

1.2.1 Vectors that are not arrows

The abstract definition pays off immediately, because many different mathematical objects satisfy the vector space axioms. Each example below is a collection of objects you can scale and add, obeying the same rules, so the entire machinery of linear algebra applies directly to it.

- **Lists of numbers.** A point in $\mathbb{R}^{784}$ is a 28×28 grayscale image flattened into one long column. Scaling brightens it; adding overlays two images. Classifiers routinely treat images as vectors.
- **Functions.** You can add two functions ($(f+g)(x) = f(x)+g(x)$) and scale one ($(3f)(x) = 3f(x)$). So functions form a vector space. This is the doorway to Fourier series and to all of functional analysis.
- **Polynomials.** The polynomials of degree at most 2 form a 3-dimensional vector space; 1, x, and x^2 play the role of the basic directions.
- **Solutions of a linear differential equation.** If y_1 and y_2 both solve $y''+y=0$, so does $3y_1-2y_2$. The solution set is a vector space, which is exactly why such equations are so tractable.

Connection to other math

That functions are vectors is one of the great unifying ideas in mathematics. Once you accept it, a Fourier series – an infinite sum that breaks any periodic waveform into sinusoidal harmonics – is no longer an ad hoc trick; it is simply writing a function as a *linear combination* of the basic "direction" functions $\sin nx$ and $\cos nx$, exactly as you write $\begin{bmatrix}4\\3\end{bmatrix} = 4\begin{bmatrix}1\\0\end{bmatrix}+3\begin{bmatrix}0\\1\end{bmatrix}$ in the plane. The infinite sum requires analysis to make rigorous, but the underlying linear-algebraic idea is identical. Calculus, differential equations, signal processing, and quantum mechanics all rest on this single observation, and we will return to it once the finite-dimensional picture is solid.

1.3 Linear Combinations and Span: Reachable Territory

Now combine the two operations. A **linear combination** of vectors $\mathbf{v}_1,\dots,\mathbf{v}_k$ is any vector you can build by scaling each one and adding the results:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k, \qquad c_1,\dots,c_k \in \mathbb{R}.$$

The numbers c_i are the "amounts" of each ingredient. This is *the* central construction of linear algebra; nearly every later concept in the subject builds directly on linear combinations.

The natural follow-up question is: starting from a given set of vectors, scaling and adding as much as you like, *where can you get to*? The set of all reachable points is called the **span**.

Intuition

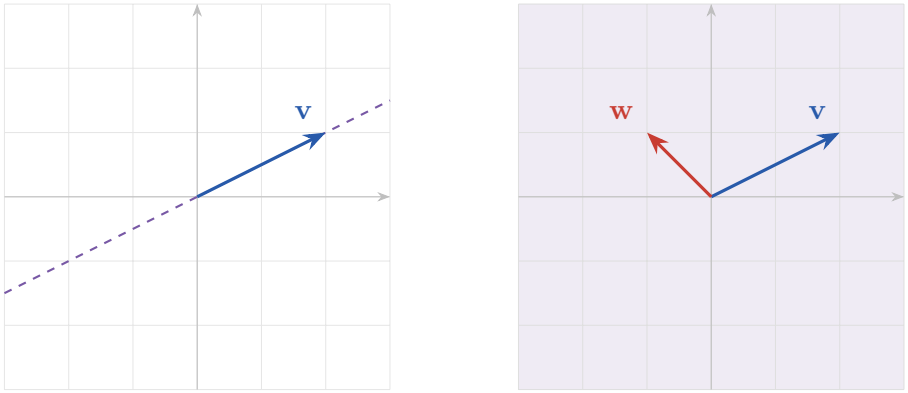
Think of your vectors as the only moves available in a game. A linear combination is one sequence of moves; the span is the entire region of the board you can ever reach. Span is the answer to "what's possible from here?"

The geometry is easiest to see in $\mathbb{R}^2$, so start there.

One nonzero vector spans a line. Take $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. Every multiple $c\mathbf{v}$ lands somewhere on the line through the origin in the direction of $\mathbf{v}$. You can reach any point on that line and no point off it. Its span is a one-dimensional subspace: a line through the origin.

Two vectors usually span the whole plane. Add $\mathbf{w} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$. Now you have two independent directions. By choosing how much of each you take, you can reach *any* point in $\mathbb{R}^2$. For instance, to reach $\begin{bmatrix} 1 \\ 4 \end{bmatrix}$ we solve $c\begin{bmatrix} 2 \\ 1 \end{bmatrix} + d\begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$, which gives $c = \frac{5}{3}$, $d = \frac{7}{3}$. The span is all of $\mathbb{R}^2$.

But two vectors that point the same way span only a line. If $\mathbf{w}$ had been $\begin{bmatrix} 4 \\ 2 \end{bmatrix} = 2\mathbf{v}$, it would add nothing new – it lies on the line $\mathbf{v}$ already reaches. Two vectors span a plane only when they point in genuinely different directions. This is your first glimpse of *linear independence*, the idea that some vectors are redundant. Figure 1.4 contrasts the two cases.



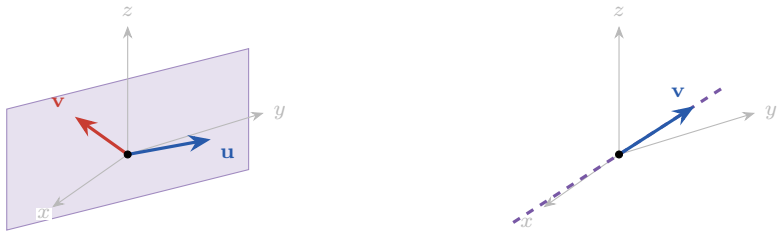
span of one vector: a line

span of two independent vectors: all of $\mathbb{R}^2$

Figure 1.4: Span is reachable territory. One vector reaches a line; two independent vectors reach the whole plane; a vector that merely repeats a direction reaches nothing new.

The same story scales up. In three-dimensional space $\mathbb{R}^3$, one vector spans a line, two independent vectors span a plane through the origin, and three independent vectors span all of space. These lines and planes through the origin are our first examples of **subspaces** – flat sheets, passing through $\mathbf{0}$,

that are closed under scaling and adding. Subspaces are where vectors live, and recognizing them will let us answer "how many solutions does this system have?" with a glance instead of a calculation. Figure 1.5 shows the two proper subspaces of $\mathbb{R}^3$: a line and a plane, both threaded through the origin.



two independent vectors span a plane one vector spans a line

Figure 1.5: Subspaces of $\mathbb{R}^3$ pass through the origin. Two independent vectors sweep out a plane (left); a single nonzero vector sweeps out a line (right). Add a third independent vector and the plane fills out to all of $\mathbb{R}^3$.

Walking question

Take a walk and turn this over. In $\mathbb{R}^3$, you hold three vectors. Sometimes they span all of space, sometimes only a plane, sometimes only a line. *What geometric thing has to be true for three vectors to fail to fill space?* Try to describe it without any formula – in terms of arrows lying flat. You are inventing the idea of linear dependence on your own, and it will feel obvious once you say it out loud.

1.4 The Two Views of a Matrix: Rows and Columns

We have been talking about vectors floating freely. Linear algebra gets its real power when we package vectors into a grid – a **matrix** – and ask what a system of equations is really saying. There are two complementary ways to read any linear system, and switching between them fluently is one of the most valuable skills this book can give you. Strang famously builds his whole course around getting you to see both. One small system will serve as our laboratory:

$$\begin{aligned} 2x - y &= 0, \\ x + y &= 3. \end{aligned} \quad \Longleftrightarrow \quad \underbrace{\begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 0 \\ 3 \end{bmatrix}}_{\mathbf{b}}.$$

1.4.1 The row picture: intersecting lines

Read the system one equation at a time, and each equation is a constraint. The first equation, $2x - y = 0$, is satisfied by all points (x, y) on a particular line. The second, $x + y = 3$, picks out another line. A solution to the *system* must satisfy both, so it must lie on *both* lines at once – it is their intersection, drawn in Figure 1.6.

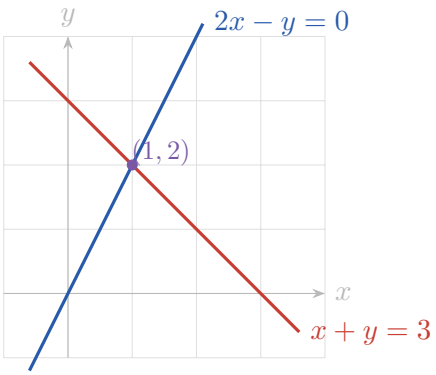


Figure 1.6: The row picture. Each equation is a line; the solution is where the lines cross, at $(x, y) = (1, 2)$.

The two lines cross at $(1, 2)$, so $x = 1$, $y = 2$. Geometrically a solution is an *intersection of hyperplanes* – lines in $\mathbb{R}^2$, planes in $\mathbb{R}^3$, and higher-dimensional flats beyond. This view makes the possible outcomes intuitive: two lines usually meet in one point, but parallel lines never meet (no solution) and identical lines meet everywhere (infinitely many). Every strange thing a linear system can do is, in the row picture, just lines arranging themselves.

1.4.2 The column picture: mixing ingredients

Now read the *same* matrix equation a completely different way – down the columns. Multiplying $\mathbf{Ax}$ is, by definition, taking x of the first column plus y of the second:

$$\begin{bmatrix} x \\ y \end{bmatrix} \longmapsto x \begin{bmatrix} 2 \\ 1 \end{bmatrix} + y \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

So the equation $\mathbf{Ax} = \mathbf{b}$ is asking: *what combination of the two column vectors produces* $\mathbf{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$*?* This is the span question from the previous section, in a new guise. The unknowns x, y are no longer coordinates of a mystery point; they are the *amounts* of each column we must mix. Figure 1.7 shows the winning recipe.

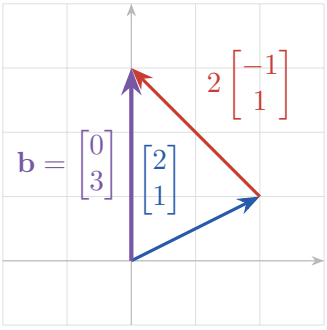


Figure 1.7: The column picture. We seek the recipe $x \begin{bmatrix} 2 \\ 1 \end{bmatrix} + y \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$. One scoop of the blue column plus two scoops of the red column lands exactly on $\mathbf{b}$, so $x = 1$, $y = 2$.

And indeed $1\cdot\begin{bmatrix}2\\1\end{bmatrix}+2\cdot\begin{bmatrix}-1\\1\end{bmatrix}=\begin{bmatrix}0\\3\end{bmatrix}$. Same answer, $x=1,\ y=2$, arrived at by a totally different mental route. The row picture intersected two lines; the column picture mixed two ingredient-vectors. Both are correct, and a fluent reader flips between them depending on which makes the current question easy.

Focus

Two ways to read $\mathbf{Ax}=\mathbf{b}$, and you want both fluent:

- **Row view:** each row is an equation, a hyperplane; the solution is where all the hyperplanes intersect. Best for: "does a solution exist, and is it unique?"
- **Column view:** $\mathbf{Ax}$ is a linear combination of $\mathbf{A}$'s columns; solving asks which recipe of columns builds $\mathbf{b}$. Best for: "what can this matrix produce?" – the span of the columns, which we will name the *column space*.

Common trap

The single most common early mistake is to think $\mathbf{Ax}$ means "multiply each number of $\mathbf{A}$ by each number of $\mathbf{x}$ somehow." It does not. **$\mathbf{Ax}$ is a linear combination of the columns of $\mathbf{A}$, with the entries of $\mathbf{x}$ as the weights.** Burn the column picture into your mind now and matrix multiplication will never again feel like an arbitrary rule. We will derive the row-by-column "dot product" shortcut later, but it is a *consequence* of the column picture, not the meaning of it.

Compute it yourself

Do this one by hand before reading on. Take $\mathbf{A}=\begin{bmatrix}1&2\\3&1\end{bmatrix}$ and $\mathbf{b}=\begin{bmatrix}5\\5\end{bmatrix}$.

1. **Row picture:** write the two equations, draw the two lines, and read off where they cross.
2. **Column picture:** draw the columns $\begin{bmatrix}1\\3\end{bmatrix}$ and $\begin{bmatrix}2\\1\end{bmatrix}$ as arrows, and find by eye roughly how many scoops of each you need to reach $\begin{bmatrix}5\\5\end{bmatrix}$.
3. Check that both pictures give the same (x,y). (Solving $x+2y=5$ and $3x+y=5$ gives $x=1,\ y=2$; one scoop of the first column and two of the second indeed land on $\begin{bmatrix}5\\5\end{bmatrix}$.)

Seeing one answer emerge from two unrelated-looking pictures is the moment linear algebra clicks.

1.5 Linear Transformations: Matrices That Move Space

So far a matrix has been a static array of coefficients. The deepest shift in viewpoint – the one this whole book is organized around – is to see a matrix as a **verb**, not a noun. A matrix *does something*: it takes in a vector and spits out a vector. It is a function from space to space. And not just any function – a **linear transformation**, obeying the two promises from the very start of this chapter.

Intuition

Stop thinking of $\mathbf{Ax}$ as "a calculation" and start thinking of it as "$\mathbf{A}$ *acts on* $\mathbf{x}$." The matrix $\mathbf{A}$ grabs the whole plane and moves it – stretching, rotating, shearing, flipping – and $\mathbf{Ax}$ is wherever the point $\mathbf{x}$ ends up. Linearity means the grid lines stay straight and evenly spaced, and the origin stays nailed down. Squares may become parallelograms, but they never become curved blobs.

The cleanest way to see this: a linear transformation is completely determined by what it does to the basic direction vectors $\mathbf{e}_1 = \begin{bmatrix}1\\0\end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix}0\\1\end{bmatrix}$. Why? Because every vector is a combination of them, $\begin{bmatrix}x\\y\end{bmatrix} = x\mathbf{e}_1 + y\mathbf{e}_2$, and a linear map respects combinations:

$$\mathbf{A}\begin{bmatrix}x\\y\end{bmatrix} = \mathbf{A}(x\mathbf{e}_1 + y\mathbf{e}_2) = x\,(\mathbf{Ae}_1) + y\,(\mathbf{Ae}_2).$$

So if you tell me where the two basic arrows land, I can find where *any* arrow lands. And those two landing spots are precisely the *columns* of the matrix. That is what a matrix really is: a compact record of where the basic directions go.

1.5.1 A gallery of transformations

Watch four matrices act on the unit square (the grid spanned by $\mathbf{e}_1$ and $\mathbf{e}_2$), collected in Figure 1.8. Each picture shows the square before and after the matrix grabs it.

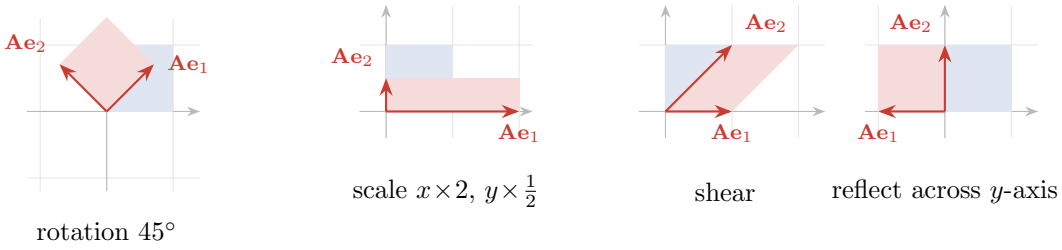


Figure 1.8: Four linear transformations acting on the unit square (blue) and producing the image (red). In every case straight lines stay straight, parallel lines stay parallel, and the origin stays put – the signature of linearity. The red arrows are the new homes of $\mathbf{e}_1$ and $\mathbf{e}_2$, i.e. the columns of the matrix.

The matrices that produce these are

$$\underbrace{\begin{bmatrix}\frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2}\\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2}\end{bmatrix}}_{\text{rotation}},\quad \underbrace{\begin{bmatrix}2 & 0\\ 0 & \frac{1}{2}\end{bmatrix}}_{\text{scaling}},\quad \underbrace{\begin{bmatrix}1 & 1\\ 0 & 1\end{bmatrix}}_{\text{shear}},\quad \underbrace{\begin{bmatrix}-1 & 0\\ 0 & 1\end{bmatrix}}_{\text{reflection}}.$$

Read each one column by column and confirm the picture. In the shear, for instance, $\mathbf{e}_1 = \begin{bmatrix}1\\0\end{bmatrix}$ stays put (first column $\begin{bmatrix}1\\0\end{bmatrix}$) while $\mathbf{e}_2 = \begin{bmatrix}0\\1\end{bmatrix}$ slides over to $\begin{bmatrix}1\\1\end{bmatrix}$ (second column) – which is exactly the slanting you see.

Example 1.1. Trace one all the way through. Apply the shear $\mathbf{S} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ to $\mathbf{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ two ways.

Column (combination) view:

$$\mathbf{S}\begin{bmatrix} 2 \\ 3 \end{bmatrix} = 2\begin{bmatrix} 1 \\ 0 \end{bmatrix} + 3\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} + \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}.$$

Row (dot-product shortcut) view: multiply matching entries and add across the top row, $\langle (1,1),\ (2,3) \rangle = 1 \cdot 2 + 1 \cdot 3 = 5$, and across the bottom row, $\langle (0,1),\ (2,3) \rangle = 0 \cdot 2 + 1 \cdot 3 = 3$. Same answer, $\begin{bmatrix} 5 \\ 3 \end{bmatrix}$. The point $(2,3)$ slid horizontally by an amount equal to its height – which is precisely what a shear does.

Before/after Strang

Strang opens his course by hammering on the row picture and the column picture of $\mathbf{Ax} = \mathbf{b}$, and he is right to: that double vision is the foundation. This book agrees and goes one step further *before* you meet him. We frame the matrix as a *transformation* – a verb acting on space – from the very first chapter, because that view is what later makes eigenvalues ("directions the transformation merely stretches"), the SVD ("every transformation is rotate-stretch-rotate"), and determinants ("the factor by which areas grow") feel like one connected story instead of four disconnected tricks. When you reach Strang's lectures, his row and column pictures will already be two familiar faces of the single idea you started with here. And *after* Strang, when you return for the SVD and data chapters, this transformation lens is the thread that ties it all together.

1.5.2 Composition is matrix multiplication

One more central result: if you apply transformation $\mathbf{B}$ and then transformation $\mathbf{A}$, the combined effect "$\mathbf{A}$ after $\mathbf{B}$" is itself a linear transformation – and the matrix that represents it is exactly the product $\mathbf{AB}$. This explains why matrix multiplication is defined as it is: it is designed so that composing two linear transformations corresponds to multiplying their matrices. Function composition *is* matrix multiplication. We will derive the coordinate formulas from this geometric principle in Chapters 5 and 8.

Figure 1.9 shows the geometry: applying $\mathbf{B}$ reshapes the space once, and applying $\mathbf{A}$ reshapes the result. The composite map that takes the starting grid straight to the final destination in one leap is $\mathbf{AB}$.

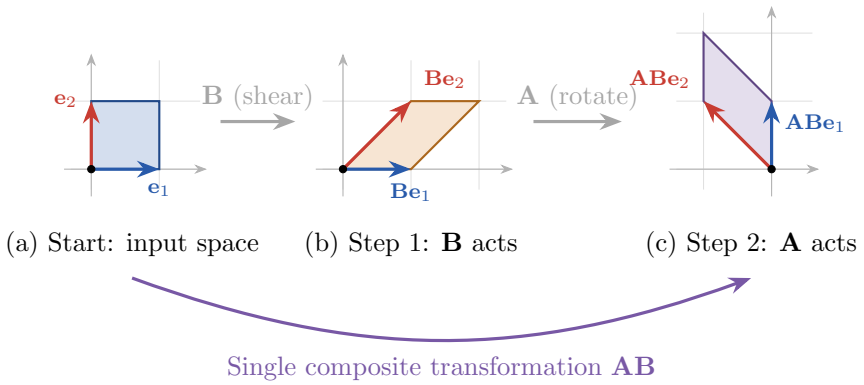


Figure 1.9: Matrix multiplication represents the composition of linear transformations. Applying **B** (a shear) followed by **A** (a 90° rotation) transforms the space in two stages. The product matrix **AB** encodes the transition from the original input grid (a) straight to the final rotated shape (c) in a single step.

Connection to other math

This makes the set of invertible $n \times n$ matrices into a *group* under multiplication – the general linear group $GL_n(\mathbb{R})$. An invertible matrix is one whose transformation can be fully undone without flattening space, and a group is a mathematical collection of actions that can be combined and reversed. Here, the group operation is "compose these two motions of space." Rotations alone form a smaller group; rotations and reflections together form another. Geometry, in the program launched by Felix Klein in 1872, *is* the study of which properties survive a group of transformations. Linear algebra is therefore the gateway to symmetry, and the symmetry groups it houses run all the way through crystallography, particle physics, and computer graphics.

1.6 Where This All Leads

We have planted three seeds: vectors, the combinations you build from them (span), and the linear transformations that move them around (matrices). Almost every later chapter is one of these seeds grown tall. Here is the shape of what is coming.

- **Solving systems and the four subspaces.** Elimination, rank, column space and null space – the complete answer to "how many solutions does $\mathbf{Ax} = \mathbf{b}$ have, and why." Every question lives in the row/column duality of this chapter.
- **Orthogonality and projection.** Orthogonality is the rigorous geometric word for perpendicularity – meeting at a clean 90° right angle. Projection asks: what is the closest point in a subspace to a given vector? This single question is the engine of least-squares fitting in applied statistics.
- **Determinants.** One number that reads off the area- or volume-scaling factor of a transformation – and tells you instantly whether it can be undone.
- **Eigenvalues and eigenvectors.** The special directions a transformation merely stretches without turning. They explain how systems evolve over time, why some structures resonate,

and how Google originally ranked the web.

- **The Singular Value Decomposition.** A foundational theorem of matrix analysis: *every* matrix factors into rotate, stretch, rotate (Figure 1.10). The SVD is the mathematical engine of data compression, recommendation systems, and principal component analysis.

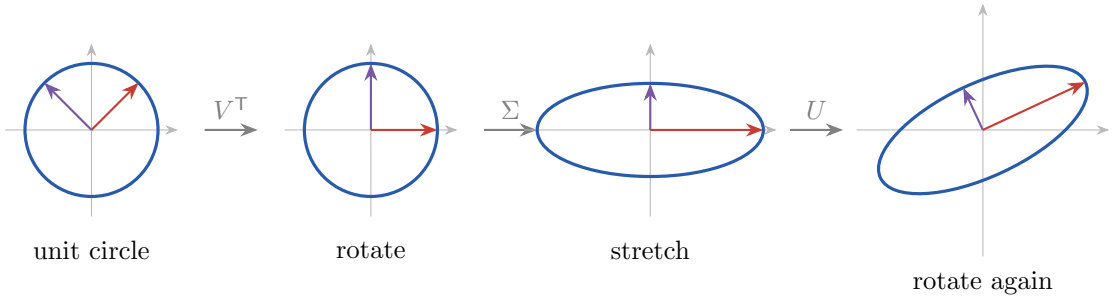


Figure 1.10: A preview of the Singular Value Decomposition $\mathbf{A} = U\Sigma V^\mathsf{T}$. Every matrix, however complicated, does only this to space: rotate (V^T, where the superscript $^\mathsf{T}$ marks the transpose we will meet in Chapter 10), stretch along perpendicular axes (Σ), and rotate again (U). The unit circle always lands on an ellipse, and its axis lengths are the singular values.

Modern application

Linear algebra in machine learning. A feedforward neural network computes a sequence of affine transformations interleaved with coordinate-wise nonlinearities. Each layer takes its input vector $\mathbf{x}$, computes $\mathbf{W}\mathbf{x} + \mathbf{b}$ – a linear transformation followed by a shift – and then applies a simple nonlinear function such as $\max(0, \cdot)$ to each entry. The weight matrices $\mathbf{W}$ store the learned parameters; training adjusts the numerical entries of these matrices via gradient descent. When such a model has billions of parameters, essentially all of them live inside matrices, and evaluation requires multiplying matrices by vectors at scale. The attention mechanism in transformer architectures is built from weight matrices ($\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$) and matrix products. GPUs power modern deep learning primarily because they are specialized hardware for parallel matrix multiplication.

The same vocabulary reaches everywhere else too. In computer graphics, every rotation and zoom of a 3D scene is a matrix acting on vertices. In networks, the connections of a graph form a matrix whose eigenvectors reveal communities and importance. In statistics, a covariance matrix's eigenvectors are the principal directions of your data. In signal processing, the Fourier transform is a change of basis into sinusoidal harmonics.

Why this matters

By the end of this book, you will be able to look at a system of equations, a high-dimensional dataset, a geometric transformation, or an optimization step, and recognize the same mathematical structure: vectors combined by weights, and linear maps transforming space.

History and connection

Linear algebra grew out of a two-century search for the right algebra to describe geometry. In 1637, René Descartes published *La Géométrie*, introducing coordinate geometry: curves and lines were converted into equations involving real numbers (x, y, z). While Descartes' coordinates were a colossal breakthrough, they came with a cost: geometry was reduced to mechanical algebra on coordinate numbers, obscuring the intrinsic geometric objects – the directed arrows, the planes, the physical forces – under a mountain of coordinate-dependent algebra.

Two centuries later, mathematicians searched for a direct algebra of space. On October 16, 1843, while walking along the Royal Canal in Dublin, Sir William Rowan Hamilton had a flash of inspiration and carved his equations for *quaternions* into the stone of Brougham Bridge: $i^2 = j^2 = k^2 = ijk = -1$. Quaternions are a four-dimensional number system that extends the complex numbers by adding three imaginary units to encode three-dimensional rotations directly. But they were complicated hybrids, mixing a single real number (scalar part) with a three-dimensional spatial direction (vector part) into an unwieldy package. In parallel, the German schoolteacher Hermann Grassmann published his *Ausdehnungslehre* (Theory of Extension) in 1844, inventing vector spaces of arbitrary dimension and abstract geometric products, though his dense philosophical style was so difficult that contemporaries ignored it for decades. In 1858, Arthur Cayley formalized the **matrix** as an independent algebraic object, showing that multiplying rectangular arrays corresponds to composing linear transformations. In the 1880s, the American physicist Josiah Willard Gibbs at Yale and the English electrical engineer Oliver Heaviside independently realized that James Clerk Maxwell's theory of electromagnetism was weighed down by quaternion notation. Gibbs and Heaviside discarded the full quaternion product and split its useful pieces into two separate operations. From the scalar part they extracted the *dot product* (multiplying two vectors to yield a single number measuring how closely they align), and from the vector part they extracted the *cross product* (combining two three-dimensional vectors into a perpendicular vector whose length equals the parallelogram area between them).

A ferocious controversy erupted, known as the *Great Vector-Quaternion War* of the 1890s. Peter Guthrie Tait, Hamilton's fiercest disciple, attacked Gibbs and Heaviside in the pages of *Nature*, denouncing their vector system as a "monstrous hybrid" and a "mutilated quaternion." Heaviside fired back with characteristic wit: "Quaternions were born with a great deal of promise, but they never made good... Vector analysis is what the physical mathematician wants, in his daily work with electromagnetic fields, fluid dynamics, and mechanics." Heaviside and Gibbs prevailed: vector analysis became the universal language of physics and engineering. Treating vectors as primary objects and matrices as linear transformations of space transformed modern mathematics. Instead of drowning in separate scalar equations for every coordinate, an entire coupled system across hundreds of variables can be written in four symbols: $\mathbf{Ax} = \mathbf{b}$. Linear transformations provide an exact geometric framework for rotations, shears, projections, and deformations of space. Yet relying solely on the mental picture of spatial arrows hits a wall the moment we venture beyond three dimensions into image processing, econometrics, or neural networks with millions of parameters. To harness the geometry of high-dimensional space without getting lost, we need formal algebraic coordinates and systematic algorithms to solve coupled constraints. That bridge begins in Chapter 2 (Section 2.1), which formalizes vectors in $\mathbb{R}^n$ by uniting the arrow, the coordinate list, and the data point, before Chapter 3 (Definition 3.1) develops linear combinations and span.

Focus

The three ideas to carry out of this chapter.

1. A **vector** is anything you can scale and add; arrows in the plane are the picture, but images, functions, and solution sets are vectors too.
2. A **linear combination** mixes vectors by scaling and adding; the set of all reachable combinations is the **span**, and $\mathbf{Ax} = \mathbf{b}$ asks whether $\mathbf{b}$ is in the span of $\mathbf{A}$'s columns.
3. A **matrix is a linear transformation** – a verb that moves space, recorded by where it sends the basic directions. Its columns *are* those destinations.

Hold these three and the rest of the book is elaboration.

Before turning the page to Chapter 2, test these three ideas on the exercises below.

1.7 Exercises

Exercise 1.1 (Warmup: scale and add)**.** Let $\mathbf{u} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$. Compute $2\mathbf{u}+\mathbf{v}$, $\mathbf{u}-3\mathbf{v}$, and $\frac{1}{2}(\mathbf{u}+\mathbf{v})$. Sketch $\mathbf{u}$, $\mathbf{v}$, and $2\mathbf{u}+\mathbf{v}$ as arrows from the origin, and confirm the tip-to-tail picture for the sum.

Exercise 1.2 (Spans in the plane)**.** For each pair, decide whether the span is a line or all of $\mathbb{R}^2$, and justify geometrically.

$$\text{(a) } \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \end{bmatrix}; \qquad \text{(b) } \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}; \qquad \text{(c) } \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Exercise 1.3 (Both pictures of one system)**.** Consider $\begin{aligned} x + 2y &= 4 \\ 3x - y &= 5 \end{aligned}$

1. Draw the row picture (two lines) and estimate the intersection.
2. Write the column picture: which combination of $\begin{bmatrix} 1 \\ 3 \end{bmatrix}$ and $\begin{bmatrix} 2 \\ -1 \end{bmatrix}$ gives $\begin{bmatrix} 4 \\ 5 \end{bmatrix}$?
3. Solve algebraically and check that the row and column answers agree.

Exercise 1.4 (Reading a matrix as a verb)**.** The matrix $\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ sends $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ to which vectors? Apply $\mathbf{A}$ to $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and to $\begin{bmatrix} 3 \\ 0 \end{bmatrix}$. What familiar motion of the plane does $\mathbf{A}$ perform? Sketch the unit square and its image to confirm.

Exercise 1.5 (Identify the transformation)**.** For each matrix, describe in words what it does to the plane (stretch, rotate, reflect, shear, collapse) by looking at where it sends $\mathbf{e}_1$ and $\mathbf{e}_2$:

$$\text{(a) } \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}, \quad \text{(b) } \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \quad \text{(c) } \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \text{(d) } \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

One of these flattens the whole plane onto a line. Which, and onto which line?

Exercise 1.6 (Functions are vectors)**.** Let V be the set of all functions of the form $f(x) = a + b\cos x + c\sin x$.

1. Show that if $f, g \in V$ then $f + g \in V$ and $2f \in V$, so V is closed under scaling and adding.
2. Argue that V behaves like $\mathbb{R}^3$: each function corresponds to the triple (a, b, c). Which three "basic direction" functions play the role of $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$?

Exercise 1.7 (Linearity, tested)**.** Decide whether each map $T : \mathbb{R}^2 \to \mathbb{R}^2$ is linear by checking the two promises $T(c\mathbf{x}) = cT(\mathbf{x})$ and $T(\mathbf{x}+\mathbf{y}) = T(\mathbf{x}) + T(\mathbf{y})$.

$$\text{(a) } T\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2x \\ x - y \end{bmatrix}; \quad \text{(b) } T\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + 1 \\ y \end{bmatrix}; \quad \text{(c) } T\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} xy \\ 0 \end{bmatrix}.$$

For each linear one, write down its matrix. For each nonlinear one, name the exact promise it breaks.

Exercise 1.8 (Composition by hand)**.** Let $\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ (rotate 90°) and $\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ (flip vertically). Apply "$\mathbf{F}$ then $\mathbf{R}$" to $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and to $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ by tracking the vectors through both steps. Then apply "$\mathbf{R}$ then $\mathbf{F}$." Do the two orders give the same result? What does your answer tell you about whether matrix multiplication can be commutative?

Exercise 1.9 (Proof: linearity forces the origin to stay put)**.** Suppose T is linear. Using only the scaling promise, prove that $T(\mathbf{0}) = \mathbf{0}$. (*Hint:* $\mathbf{0} = 0 \cdot \mathbf{0}$.) Explain in one sentence why this means the map $\mathbf{x} \mapsto \mathbf{x} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ – a pure shift – can never be linear.

Exercise 1.10 (Challenge: when do three vectors fill space?)**.** You hold three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$ in $\mathbb{R}^3$. Argue geometrically that their span is all of $\mathbb{R}^3$ exactly when none of them lies in the plane spanned by the other two. Give an explicit example of three vectors whose span is only a plane, and verify that one of them is a linear combination of the other two. (You are rediscovering linear dependence – the subject of a coming chapter.)

Exercise 1.11 (Challenge: a recipe is unique when directions differ)**.** Suppose $\mathbf{v}$ and $\mathbf{w}$ in $\mathbb{R}^2$ point in different directions (neither is a multiple of the other). Prove that any target $\mathbf{b}$ can be written as $c\mathbf{v} + d\mathbf{w}$ in *exactly one* way. (*Hint:* if $c\mathbf{v} + d\mathbf{w} = c'\mathbf{v} + d'\mathbf{w}$, move everything to one side and use that $\mathbf{v}, \mathbf{w}$ are not parallel.) Connect this to why the column picture of a 2×2 system with independent columns has a unique solution.

Chapter 2

Vectors: Arrows, Lists, and Points

In Chapter 1, we stepped back to take in the bird's-eye view: linear algebra is the mathematics of vectors, the linear combinations you can build from them, and the transformations that move them. But to build that world, we have to start on the ground. Every big idea in this book grows out of one small object: the *vector*. Before we can talk about matrices (machines that transform space), spans (the shapes reachable by mixing directions), projections, eigenvalues, or the singular value decomposition that quietly powers modern machine learning, we have to be completely comfortable with vectors. It is not enough merely to manipulate them mechanically — you want to know them the way you know the number 3. You can picture 3, you can add it, you can scale it, you can spot it in the wild. By the end of this chapter you should feel the same easy familiarity with a vector like $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

Here is the gentle surprise that organizes the whole chapter. A vector is *three things at once*, and the three pictures always agree:

- an **arrow** — a directed step, with a length and a heading;
- a **list** of numbers — something you can type into a computer;
- a **point** — a location in space, or a single data record.

We will look at the same little objects from all three angles until switching between them becomes automatic. That flexibility is not a side benefit; it *is* the skill. Geometry gives you intuition, the list of numbers gives you computation, and viewing vectors as points gives you the bridge to data.

2.1 An Arrow Is a Step, Not a Place

Start with the most physical picture. A vector is an *arrow*: it has a length and a direction, and that is *all* it has. Draw an arrow that goes two units right and one unit up. That arrow is a vector. It says "move two right and one up," and it says nothing else.

Intuition

A vector is a *displacement* — a recipe for moving. "Walk three blocks east and one block north" is a vector. It does not care where you start. The same instruction moves you the same way whether you begin at your house or at the library. So the very same arrow can be drawn anywhere on the page; sliding it around without rotating or stretching it does not change the vector. Length and direction are everything; position is nothing.

This is the single most common stumbling block at the very start, so let us name it now.

Common trap

An arrow is not glued to the origin. A vector has no fixed home. The two arrows below — one starting at the origin, one starting somewhere else — are the *same vector*, because they have the same length and the same direction. We often *choose* to draw a vector starting at the origin (we call that the *standard* or *position* representative), but that is a convenience, not a law.

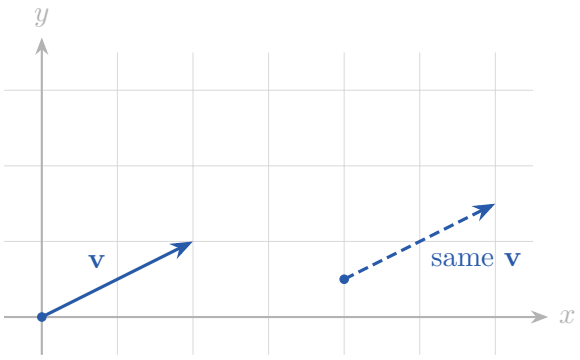


Figure 2.1: One vector, two drawings. Both arrows go "2 right, 1 up," so both represent the same vector $\mathbf{v}$. The solid arrow is the standard representative anchored at the origin; the dashed copy is the same step taken elsewhere.

Why insist on this so early? Because the arrow-as-step picture is exactly what makes vector *addition* obvious in a moment. If a vector were stuck at the origin, adding two of them would be a mystery. As a step, it is just "do this move, then do that one."

Length and direction, made precise

The *length* (or *magnitude*, or *norm*) of an arrow is how far it reaches: a single positive number telling you the total distance from tail to tip. For the arrow that goes 2 right and 1 up, the Pythagorean theorem gives length $\sqrt{2^2 + 1^2} = \sqrt{5} \approx 2.236$. We write this $\|\mathbf{v}\|$. The *direction* is the heading the arrow points. We will make both of these fully precise in Chapter 4 (Section 4.1); for now, hold the picture: *length plus direction equals arrow equals vector.*

Pause

Pause and think. What is the length of $\begin{bmatrix} -3 \\ 4 \end{bmatrix}$? Before reaching for paper: $(-3)^2 + 4^2 = 9 + 16 = 25$, so its length is $\sqrt{25} = 5$. Notice how the minus sign vanishes under squaring: stepping 3 units west adds the exact same distance as stepping 3 units east. Length only cares about distance traveled, never about the sign.

2.2 A List Is How You Compute With an Arrow

Geometry is wonderful for intuition, but you cannot type an arrow into a computer. So we pin a coordinate grid onto the plane and *measure* the arrow. The arrow that goes 2 right and 1 up gets recorded as the ordered list of its measurements:

$$\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

The top number, 2, is how far the arrow reaches in the x-direction; the bottom number, 1, is how far in the y-direction. These numbers are the *components* (or *coordinates*, or *entries*) of the vector.

Focus

The arrow is the *idea*; the list is the *measurement* of that idea in a chosen coordinate system. Change the geometry and the list changes with it; the underlying displacement is what they both point at. We will write vectors as *columns* almost everywhere in this book, because columns are what matrices act on (you will see why in the chapter on matrices as machines).

A small but important convention: the *order* matters. The list $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ is the arrow "2 right, 1 up," while $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ is the arrow "1 right, 2 up." Different lists, different arrows. A vector is an *ordered* tuple of numbers.

Remark 2.1. You will see vectors written as columns $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and, to save space in running text, as rows $(2, 1)$ or transposed columns $(2, 1)^{\mathsf{T}}$. They mean the same arrow. The distinction between a true *row vector* and a true *column vector* will start to matter when we multiply by matrices and take transposes; until then, read a row in prose as "the column written sideways."

2.3 Adding Vectors: Walk, Then Walk Again

Now the payoff for thinking of a vector as a step. To add two vectors, *take one step, then take the other from where you landed.* Geometrically: place the tail of the second arrow at the head of the first; the sum is the new arrow from the very start to the very end. This is the famous *tip-to-tail* rule.

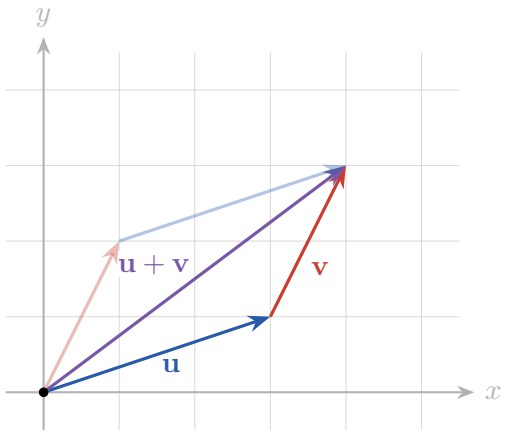


Figure 2.2: Tip-to-tail addition. Start at the origin, walk along $\mathbf{u} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, then walk along $\mathbf{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ from where you landed. The purple arrow from start to finish is $\mathbf{u} + \mathbf{v} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$. The faint arrows show that adding in the other order ($\mathbf{v}$ first) lands in the same place: the two routes complete a *parallelogram*.

In coordinates this is straightforward: *add the matching components*.

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 + 1 \\ 1 + 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}.$$

The geometric "walk then walk" and the algebraic "add componentwise" are the same operation seen from two sides. The total rightward motion is the sum of the two rightward motions $(3 + 1)$; the total upward motion is the sum of the two upward motions $(1 + 2)$. That is the entire content of vector addition.

Intuition

The faint parallelogram in Figure 2.2 is worth a long look. Going $\mathbf{u}$ then $\mathbf{v}$ and going $\mathbf{v}$ then $\mathbf{u}$ both land at the opposite corner. So $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$ is not an arbitrary rule to memorize — it simply says that walking around either side of a parallelogram reaches the same opposite corner.

Example 2.1. Three vectors in $\mathbb{R}^3$. Let

$$\mathbf{a} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -1 \\ 3 \\ 1 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 4 \\ -2 \\ 0 \end{bmatrix}.$$

Then

$$\mathbf{a} + \mathbf{b} + \mathbf{c} = \begin{bmatrix} 1 + (-1) + 4 \\ 0 + 3 + (-2) \\ 2 + 1 + 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ 3 \end{bmatrix}.$$

Notice we never had to picture three arrows in 3D at once. Componentwise addition does not care how many dimensions there are, and it does not care how many vectors you chain together — just add each coordinate down its own column. This indifference to dimension is what will let us, two chapters from now, work in $\mathbb{R}^{784}$ (the space of 28×28 images) without blinking.

2.4 Scaling Vectors: Stretch, Shrink, Flip

The second fundamental operation is multiplying a vector by an ordinary number. In this context a plain number is called a *scalar*, because its job is to *scale* the arrow — to change its length without changing the line it lies along.

Geometrically, $2\mathbf{v}$ is the arrow $\mathbf{v}$ stretched to twice its length, pointing the same way. The vector $\frac{1}{2}\mathbf{v}$ is half as long. And $-\mathbf{v}$ is the same length but pointing the *opposite* way — a negative scalar flips the arrow around.

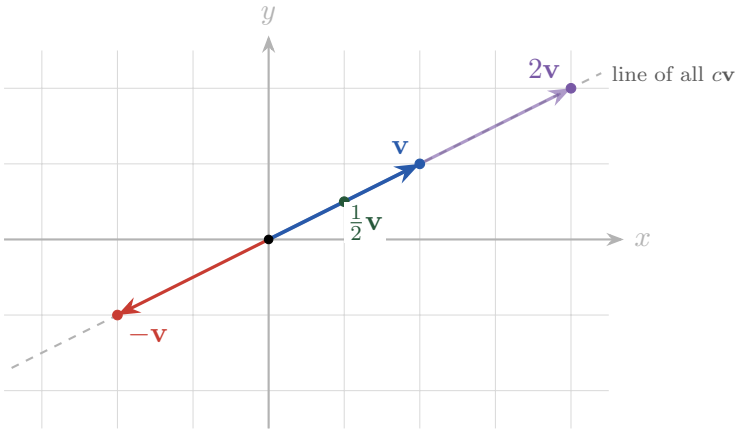


Figure 2.3: Scalar multiples of $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ all lie on one straight line through the origin. Positive scalars keep the heading and alter the length; the negative scalar -1 flips the arrow to point the opposite way. The dashed line is the collection of *all* scalar multiples $c\mathbf{v}$.

In coordinates, again, it could not be simpler: *multiply every component by the scalar.*

$$2\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, \qquad \tfrac{1}{2}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}, \qquad -\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}.$$

Stretching the arrow to double length doubles how far right *and* how far up it reaches; that is why each component is multiplied by the same 2.

Focus

The set of all scalar multiples of a single nonzero vector $\mathbf{v}$ — that is, $\{t\,\mathbf{v} : t \in \mathbb{R}\}$ — is a *line through the origin*. Sweep t from $-\infty$ to $+\infty$ and the tip of $t\mathbf{v}$ traces out the whole line. This is your first *subspace* — a flat slice through the origin that never escapes itself under scaling and addition (studied formally in Chapter 11) — and it is the geometric meaning of the word "span" that the next chapter is built around.

Common trap

Scaling a vector is not the same as moving its tip by a fixed amount. The vector $2\mathbf{v}$ is *not* "$\mathbf{v}$ with each component plus 2." Doubling means *multiply*, not add. From $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$, scaling

by 2 gives $\begin{bmatrix}4\\2\end{bmatrix}$, while adding 2 to each entry would give $\begin{bmatrix}4\\3\end{bmatrix}$ — a different arrow pointing a different direction. Multiplication scales along the line; addition (of a constant vector) slides off it.

Combining the two: linear combinations

Addition and scaling are the only two operations vectors come with, and almost everything in this book is built by using them together. An expression like

$$3\mathbf{u} - 2\mathbf{v}$$

means: stretch $\mathbf{u}$ by 3, stretch $\mathbf{v}$ by 2 and flip it, then add the results tip-to-tail. With $\mathbf{u} = \begin{bmatrix}1\\2\end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix}3\\1\end{bmatrix}$,

$$3\mathbf{u} - 2\mathbf{v} = 3\begin{bmatrix}1\\2\end{bmatrix} - 2\begin{bmatrix}3\\1\end{bmatrix} = \begin{bmatrix}3\\6\end{bmatrix} - \begin{bmatrix}6\\2\end{bmatrix} = \begin{bmatrix}-3\\4\end{bmatrix}.$$

A *linear combination* is a mix formed by scaling each vector by some weight and adding the results together: choose how much of each direction you want, then add them up. It is so central that Chapter 3 (starting at Definition 3.1) is devoted entirely to it. For now just notice that we already have everything we need to build it: scale, then add.

Pause

Pause and think. If $\mathbf{u} = \begin{bmatrix}1\\0\end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix}0\\1\end{bmatrix}$, what linear combination yields the target vector $\begin{bmatrix}4\\-3\end{bmatrix}$? In your head: $4\mathbf{u} + (-3)\mathbf{v} = 4\begin{bmatrix}1\\0\end{bmatrix} - 3\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}4\\-3\end{bmatrix}$. The coordinates 4 and -3 are literally the scaling knobs that build the vector from basic directions.

2.5 A Point Is Where an Arrow's Tip Lands

Here is the third face of a vector, and the one that opens the door to data. Take the arrow $\begin{bmatrix}2\\1\end{bmatrix}$ and draw it in standard position — tail at the origin. Its tip lands at a specific spot in the plane: the location two units right and one unit up. That *location* is the point $(2, 1)$.

Intuition

Arrow and point, two sides of one coin. Anchor a vector at the origin and its tip names a point. Conversely, any point names the vector that reaches it from the origin. So $\begin{bmatrix}2\\1\end{bmatrix}$ *is* the displacement "2 right, 1 up" *and* the location $(2, 1)$, depending on whether you care about the journey or the destination. Same list of numbers, two readings.

This identification is so useful that we usually stop distinguishing them. When we say "the point $\mathbf{x}$ in $\mathbb{R}^2$," we mean the location named by the vector $\mathbf{x}$. The arrow reading is best when you are *moving* (adding displacements, following a velocity); the point reading is best when you are *locating* (a data record, a pixel, a position). Train yourself to flip between them at will.

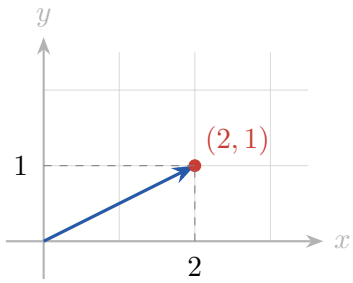


Figure 2.4: The arrow $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ in standard position and the point $(2, 1)$ where its tip lands. The dashed lines read off the components as coordinates. Arrow and point are the same vector, seen as a journey or as a destination.

Connection to other math

This is exactly Descartes' great idea, *analytic geometry*: pin a coordinate grid onto the plane so that every *geometric* point becomes an *algebraic* list of numbers, and vice versa. Linear algebra is, in a real sense, Descartes' idea taken seriously and pushed to every dimension at once. When you later study calculus on curves and surfaces, or solve differential equations whose "state" is a vector, you are leaning on this same arrow $\leftrightarrow$ point dictionary — the position, velocity, and acceleration of a moving particle are all vectors anchored at the same moving point.

2.6 The Zero Vector: Standing Still

There is one special vector we must single out: the *zero vector*, written $\mathbf{0}$. It is the vector all of whose components are 0:

$$\mathbf{0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in \mathbb{R}^2, \qquad \mathbf{0} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^3, \qquad \text{and so on.}$$

As an arrow it is the "arrow" of length zero: the instruction "do not move." As a point it is the origin. It is the only vector with no direction at all, and it plays the role for vectors that the number 0 plays for numbers.

Focus

The zero vector is the *additive identity*: adding it changes nothing,

$$\mathbf{v} + \mathbf{0} = \mathbf{v} \qquad \text{for every } \mathbf{v}.$$

"Move by $\mathbf{v}$, then don't move" lands you exactly where "move by $\mathbf{v}$" did. And every vector has an *opposite* that undoes it: $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$, because walking out along $\mathbf{v}$ and back along

$-\mathbf{v}$ returns you to start.

Common trap

The zero vector $\mathbf{0}$ is not the number 0. One is a list of numbers (an arrow, a point); the other is a single scalar. They are written almost the same and they cooperate — $0\,\mathbf{v} = \mathbf{0}$ for every $\mathbf{v}$ — but they live in different places. Writing "$\mathbf{v} = 0$" when you mean the zero *vector* is a venial sin we all commit; just know which 0 you mean. A subtler trap: a subspace (a flat slice through the origin that is closed under scaling and addition, studied in Chapter 11) must always contain $\mathbf{0}$. A line through the origin is a subspace; a parallel line that misses the origin is not, because it lacks the zero vector.

2.7 From the Plane to $\mathbb{R}^n$

Everything so far has lived in the plane, which we call $\mathbb{R}^2$: the set of all lists of *two* real numbers. The list picture has one decisive advantage: nothing stops at two.

A vector in three-dimensional space $\mathbb{R}^3$ is a list of three numbers, $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$, an arrow in the room you are sitting in. We can still draw it (Figure 2.5). Then the pictures run out — nobody can sketch four perpendicular axes — but the *lists* sail right on. A vector in $\mathbb{R}^n$ is simply an ordered list of n real numbers,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad x_1, \ldots, x_n \in \mathbb{R}.$$

We call n the *dimension* of the space, and x_i the i-th *component*.

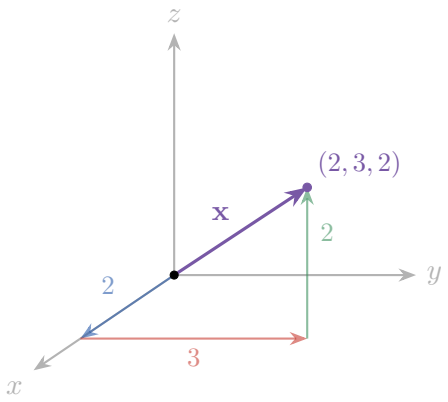


Figure 2.5: A vector $\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 2 \end{bmatrix}$ in $\mathbb{R}^3$, assembled from its three components: walk 2 units along the x-axis, then 3 along y, then 2 up along the z-axis. The purple arrow from the origin to the corner is the vector; tip-to-tail addition of the three axis steps still builds it, exactly as in the plane.

Formal

Definition 2.1 (Real n-space)**.** For a positive integer n, *real n-space* $\mathbb{R}^n$ is the set of all ordered n-tuples of real numbers,

$$\mathbb{R}^n = \left\{ \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} : x_1, \ldots, x_n \in \mathbb{R} \right\}.$$

Its elements are *vectors*. Addition and scalar multiplication are defined *componentwise*: for $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and a scalar $c \in \mathbb{R}$,

$$(\mathbf{x} + \mathbf{y})_i = x_i + y_i, \qquad (c\,\mathbf{x})_i = c\,x_i, \qquad i = 1, \ldots, n.$$

The zero vector $\mathbf{0}$ has every component equal to 0.

These componentwise definitions are chosen so that the algebra matches the geometry of arrows in every dimension. And they obey a short list of rules that we will lean on constantly.

Formal

Proposition 2.1 (The vector arithmetic rules)**.** *For all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^n$ and all scalars $c, d \in \mathbb{R}$:*

(commutativity)	$\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$,
(associativity)	$(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$,
(identity)	$\mathbf{v} + \mathbf{0} = \mathbf{v}$,
(inverse)	$\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$,
(distributivity over vectors)	$c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$,
(distributivity over scalars)	$(c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$,
(compatibility)	$c(d\mathbf{v}) = (cd)\mathbf{v}$,
(unit scalar)	$1\mathbf{v} = \mathbf{v}$.

These eight rules are the axioms of a *vector space*: any collection of objects that can be scaled and added together without ever leaving the set, while obeying these eight common-sense rules of arithmetic (a structure we will explore thoroughly in Chapter 11). The point to absorb now is how cheaply they are proved: each one is just an ordinary number fact applied in every slot at once.

Proof of commutativity, as a sample. Compare the two sides component by component. For each i,

$$(\mathbf{u} + \mathbf{v})_i = u_i + v_i = v_i + u_i = (\mathbf{v} + \mathbf{u})_i,$$

where the middle equality is just commutativity of ordinary real-number addition. Since the two vectors agree in every component, they are equal. The other seven rules fall the same way: write out one component, use the matching rule for real numbers, and you are done. □

Why this matters

Pause on what just happened. The geometry of arrows in the flat plane gave us rules — parallelograms, stretching, flipping. We translated those rules into componentwise arithmetic. And that arithmetic makes *perfect sense in* $\mathbb{R}^{100}$ *or* $\mathbb{R}^{1,000,000}$, where no arrow can be drawn. So the algebra has carried the geometry past the edge of what we can see. This is what gives linear algebra its reach: it lets you reason *geometrically* about spaces far too big to picture, by trusting that the same rules still hold. When you later hear that a deep network manipulates vectors in a 4096-dimensional space, do not panic — it is the parallelogram law and stretching, just with more slots.

2.8 Unit Vectors and the Standard Basis

Some vectors are special enough to earn names. A *unit vector* is any vector of length exactly 1. Unit vectors are the natural way to talk about *pure direction*: strip a vector of its length and what remains is a unit vector pointing the same way.

To turn any nonzero vector into a unit vector, divide it by its own length — a move called *normalizing*:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}.$$

The little hat $\hat{}$ traditionally marks a unit vector. For example, with $\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ we have $\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$, so

$$\hat{\mathbf{v}} = \frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix},$$

and you can check $\|\hat{\mathbf{v}}\| = \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = \sqrt{1} = 1$. Same direction as $\mathbf{v}$, length trimmed to 1.

Common trap

You cannot normalize the zero vector. Its length is 0, and dividing by 0 is undefined — which matches the geometry, since $\mathbf{0}$ has no direction to point a unit vector along. Normalizing is a move reserved for *nonzero* vectors.

What is a basis?

Before we name these vectors, let us pause on the second word in this section's title: **basis**. What does that mean?

In plain terms, a *basis* is a coordinate scaffold: a minimal set of benchmark directions that lets you assemble *every* vector in the space uniquely by scaling and adding. Think of it as a set of standard measuring rulers or basic building blocks. If you have enough benchmark directions pointing in genuinely different ways, every single vector in the space can be described by an exact recipe of steps along those rulers. (We will define and explore general bases rigorously in Chapter 14, where you will see that a space can be measured against many different coordinate scaffolds.)

In $\mathbb{R}^n$, the friendliest and most natural scaffold of all comes directly from the coordinate axes we already draw. The *standard basis* consists of the unit vectors pointing straight along each axis: $\mathbf{e}_1$ along the first axis, $\mathbf{e}_2$ along the second, and so on. We call it "standard" because its measuring rods line up with the grid we use to write down numbers.

Formal

Definition 2.2 (Standard basis of $\mathbb{R}^n$)**.** The *standard basis* of $\mathbb{R}^n$ is the collection of unit vectors $\{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n\}$, where the i-th *standard basis vector* $\mathbf{e}_i$ has a 1 in the i-th slot and 0 in all other slots:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \qquad \ldots, \qquad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}.$$

In $\mathbb{R}^3$, these are

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

In physics and engineering the $\mathbb{R}^3$ versions are often written $\hat{\imath}, \hat{\jmath}, \hat{k}$.

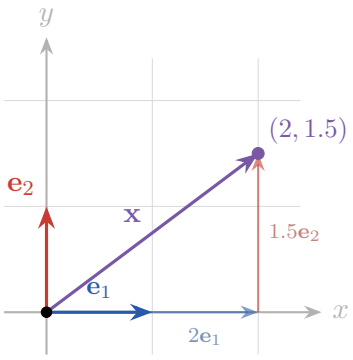


Figure 2.6: The standard basis $\mathbf{e}_1, \mathbf{e}_2$ of $\mathbb{R}^2$ and the vector $\mathbf{x} = \begin{bmatrix} 2 \\ 1.5 \end{bmatrix} = 2\mathbf{e}_1 + 1.5\,\mathbf{e}_2$. The faint arrows show $\mathbf{x}$ being assembled step by step: scale $\mathbf{e}_1$ by 2, scale $\mathbf{e}_2$ by 1.5, and add.

Here is why these particular vectors matter so much. *Every* vector in $\mathbb{R}^n$ is a linear combination of the standard basis vectors, and its coordinates *are* the coefficients:

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + \cdots + x_n\mathbf{e}_n.$$

For instance $\begin{bmatrix} 2 \\ 1.5 \end{bmatrix} = 2\mathbf{e}_1 + 1.5\,\mathbf{e}_2$ in $\mathbb{R}^2$, and $\begin{bmatrix} 4 \\ -1 \\ 7 \end{bmatrix} = 4\mathbf{e}_1 - \mathbf{e}_2 + 7\mathbf{e}_3$ in $\mathbb{R}^3$.

Focus

This is the geometric meaning of "a vector is a list of numbers." The list $\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$ is really a set of *instructions*: take x_1 steps along $\mathbf{e}_1$, x_2 steps along $\mathbf{e}_2$, and so on, then add them up. The standard basis is the implicit framework behind every coordinate you have ever written. When we get to *change of basis*, we will swap this framework for a different one — and the same arrow will get a different list. The arrow is real; the list depends on the basis you measure it against.

Compute it yourself

Verify the assembly by hand in $\mathbb{R}^3$. Let $\mathbf{x} = \begin{bmatrix} 4 \\ -1 \\ 7 \end{bmatrix}$. Compute the linear combination $4\mathbf{e}_1 - \mathbf{e}_2 + 7\mathbf{e}_3$ slot by slot:

$$4\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + 7\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 7 \end{bmatrix} = \begin{bmatrix} 4 \\ -1 \\ 7 \end{bmatrix}.$$

The $\mathbf{e}_i$ act like labelled buckets: scalar i drops straight into slot i and nowhere else. That clean separation is exactly what makes coordinates work.

2.9 Vectors as Data: A Row of Features

So far a vector has been a thing in space. But the very same object — an ordered list of numbers — is how the modern world stores information, and this is where linear algebra stops being geometry class and starts being the language of data.

A single record about something in the world is a list of measured numbers, and that list *is* a vector. The trick is to let each coordinate axis stand for a different *feature* rather than a direction in physical space.

Example 2.2 (A house as a vector)**.** Describe a house by three numbers: its area in square metres, its number of bedrooms, and its age in years. A particular house then becomes the vector

$$\mathbf{h} = \begin{bmatrix} 120 \\ 3 \\ 15 \end{bmatrix} \in \mathbb{R}^3 \quad \longleftrightarrow \quad (120\ \text{m}^2,\ \ 3\ \text{bedrooms},\ \ 15\ \text{years old}).$$

A whole neighbourhood is a *cloud of points* in $\mathbb{R}^3$, one point per house. Suddenly geometric questions become data questions: "which two houses are most alike?" becomes "which two points are closest?"; "what's a typical house?" becomes "where is the center of the cloud?" — which, we will see, is just the *average* of the vectors, $\frac{1}{N}(\mathbf{h}_1 + \cdots + \mathbf{h}_N)$, vector addition and scaling doing real work.

Once data is geometry, the operations of this chapter take on plain-English meanings. Adding vectors and dividing by N *averages* records into a prototype (the centroid). Scaling a feature axis

reweights or *rescales* a feature (multiply the age coordinate by a constant to convert years to months). Subtracting one vector from another, $\mathbf{h}_2 - \mathbf{h}_1$, gives the *difference* record — how house 2 differs from house 1, feature by feature — and its length measures how far apart they are.

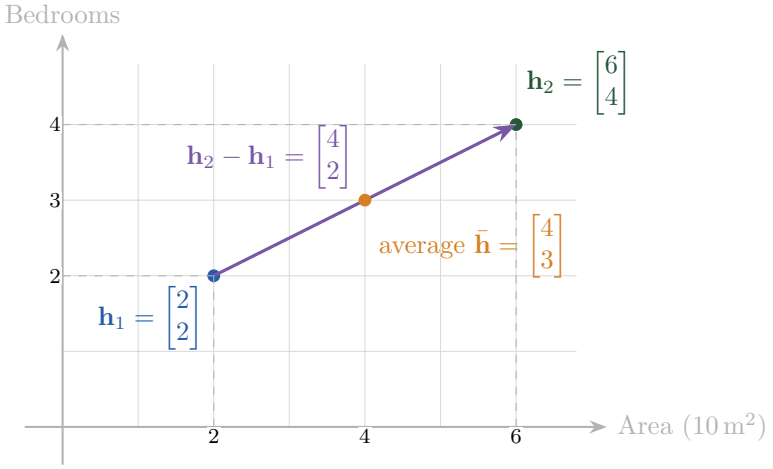


Figure 2.7: Vectors as data records. In this two-feature space (area and bedrooms), every house is a point. The average vector $\bar{\mathbf{h}} = \frac{1}{2}(\mathbf{h}_1 + \mathbf{h}_2)$ sits at the geometric midpoint (the center of the data), while the difference arrow $\mathbf{h}_2 - \mathbf{h}_1$ measures exactly how house 2 differs from house 1 across every feature.

Modern application

Word embeddings and the geometry of meaning. Modern language models begin by turning every word into a vector — often in a space like $\mathbb{R}^{768}$ — chosen so that words used in similar ways land near each other. The astonishing part is that the *arrow* picture survives: directions in this space carry meaning, and the tip-to-tail addition of this chapter does real semantic work. The famous example is

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}.$$

Read it as displacements: the arrow from "man" to "king" is roughly the same arrow as from "woman" to "queen" — a "royalty" displacement — so adding it to "woman" lands you near "queen." That is the parallelogram of Figure 2.2, operating in 768 dimensions on the meanings of words. Every recommendation engine, search ranker, and chatbot you touch is, at bottom, adding and scaling vectors like the ones in this chapter.

Before/after Strang

Before Strang: Strang opens his course by asking what you can build from a few vectors using addition and scalar multiplication — *linear combinations* — and he draws the picture before he writes a single equation. This chapter is the warm-up that makes that opening feel inevitable: if you are at home with arrows, lists, and points, his first lecture will read like a story you already know the shape of.

After Strang: when you return to this material having seen the whole arc, reread the data section above. The cloud of feature vectors in Example 2.2 is exactly the object that the

singular value decomposition (factoring a matrix into pure geometric stretches and rotations) and principal component analysis (discovering the main directions of spread in a dataset) take apart. Strang's recurring slogan "column space" is, right from the start, a statement about which points your feature vectors can reach by addition and scaling (the set of all outputs reachable by combining a matrix's columns). It all begins here.

2.10 Subtraction and the Arrow Between Two Points

We close the geometric tour with one more picture that we will use over and over: subtraction. By definition,

$$\mathbf{u} - \mathbf{v} = \mathbf{u} + (-\mathbf{v}),$$

so to subtract, flip the second arrow and add. Componentwise, of course, you just subtract matching entries:

$$\begin{bmatrix} 5 \\ 2 \end{bmatrix} - \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 4 \\ -1 \end{bmatrix}.$$

But the picture is the thing to remember. Think of $\mathbf{u}$ and $\mathbf{v}$ as *points* (tips of arrows from the origin). Then $\mathbf{u} - \mathbf{v}$ is the arrow that points *from* $\mathbf{v}$ *to* $\mathbf{u}$: the displacement you would walk to get from the second point to the first.

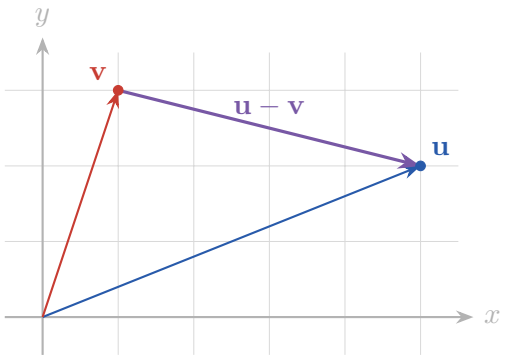
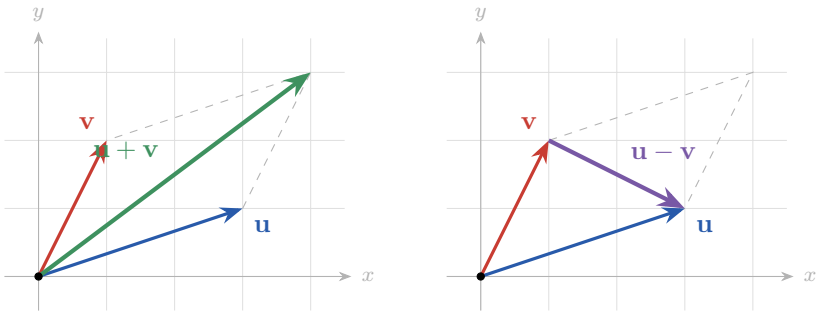


Figure 2.8: Subtraction as the arrow between points. With $\mathbf{u}$ and $\mathbf{v}$ drawn from the origin, $\mathbf{u} - \mathbf{v}$ is the displacement *from the tip of* $\mathbf{v}$ *to the tip of* $\mathbf{u}$. Check tip-to-tail: $\mathbf{v} + (\mathbf{u} - \mathbf{v}) = \mathbf{u}$, so this arrow really does carry you from $\mathbf{v}$ to $\mathbf{u}$.

This little fact does an enormous amount of work later. The arrow between two data points is their *difference*, and its length is their *distance* — the engine behind nearest-neighbor search, clustering, and the residuals of least-squares fitting. Whenever you see $\mathbf{u} - \mathbf{v}$ in a formula, picture the arrow running from the tip of $\mathbf{v}$ to the tip of $\mathbf{u}$.



Addition: main diagonal from origin to far vertex

Subtraction: cross diagonal directed from tip of $\mathbf{v}$ to tip of $\mathbf{u}$

Figure 2.9: The two diagonals of the parallelogram. Left: the sum $\mathbf{u}+\mathbf{v}$ stretches along the main diagonal from the origin out to the opposite corner. Right: the difference $\mathbf{u}-\mathbf{v}$ bridges the other diagonal, pointing from the tip of $\mathbf{v}$ directly to the tip of $\mathbf{u}$.

Walking question

Take this one on a walk. You are standing at point $\mathbf{v} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$ and your friend is at $\mathbf{u} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$. *Without computing*, which way do you point to face them — and roughly how far is the walk? Now check: the displacement is $\mathbf{u}-\mathbf{v} = \begin{bmatrix} 4 \\ -1 \end{bmatrix}$ (four blocks toward larger x, one block toward smaller y), a walk of length $\sqrt{4^2+(-1)^2} = \sqrt{17} \approx 4.1$ blocks. Did your arm point the right way before you did any arithmetic? If so, the arrow picture has taken root.

2.11 What to Carry Forward

You now hold the same object three ways, and this flexibility will be very useful.

Focus

The chapter in six lines.

- A vector is an *arrow* (length and direction), a *list* of numbers (its components in a coordinate system), and a *point* (where the arrow's tip lands from the origin) — one object, three readings.
- **Addition** is tip-to-tail / componentwise: $(\mathbf{u}+\mathbf{v})_i = u_i + v_i$. It is commutative because parallelograms are symmetric.
- **Scaling** stretches, shrinks, or flips along a line through the origin: $(c\mathbf{v})_i = c\,v_i$.
- The **zero vector 0** is "don't move," the origin, and the additive identity — and it is not the scalar 0.
- $\mathbb{R}^n$ is just lists of n numbers; the geometry of arrows survives into dimensions you cannot draw.
- The **standard basis** $\mathbf{e}_i$ turns coordinates into instructions: $\mathbf{x} = x_1\mathbf{e}_1 + \cdots + x_n\mathbf{e}_n$.

History and connection

The idea of a vector began not in abstract algebra, but in the practical mechanics of weights, ropes, and celestial motions. In 1586, the Flemish engineer and mathematician Simon Stevin published *De Beghinselen der Weeghconst* (The Principles of Statics). Stevin analyzed the balance of weights on inclined ramps and discovered the *parallelogram of forces*: two simultaneous mechanical pulls combine into a single effective pull along the diagonal of the parallelogram they form. A century later, in 1687, Isaac Newton made this the very first corollary to his laws of motion in the *Principia Mathematica* (Corollary 1): "A body by two forces conjoined will describe the diagonal of a parallelogram in the same time that it would describe the sides by those forces apart."

For the next two centuries, mathematicians wrestled with how to formalize these physical directed segments. In 1835, the Italian mathematician Giusto Bellavitis introduced the concept of *equipollence*: two directed line segments represent the same mathematical entity if they have the same length and direction, regardless of where they sit on the plane. This is our modern concept of a free vector.

Viewing vectors purely as geometric arrows is intuitive in two and three dimensions, but it is insufficient when moving to modern scientific computing and data science. In economics, finance, or machine learning, vectors live in $\mathbb{R}^{100}$ or $\mathbb{R}^{10,000}$ (asset returns, word embeddings, image pixels), where there is no physical paper on which to sketch a high-dimensional arrow. Conversely, treating a vector as merely a detached list of numbers $(x_1, \ldots, x_n)$ without geometric structure is just as risky: if you rotate your coordinate frame or change measuring units, the numbers shift completely even though the underlying physical displacement or force remains identical. The resolution – championed by Hermann Grassmann (1844) and formalized by Giuseppe Peano (1888) – is to recognize that coordinates only have meaning relative to a chosen basis:

$$\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + \cdots + x_n\mathbf{e}_n.$$

The basis vectors $\mathbf{e}_i$ fix the coordinate frame; the vector $\mathbf{x}$ is the intrinsic geometric object.

Unifying arrows, lists, and points gave physics and mathematics a single coherent language for multi-component quantities: displacements, velocities, forces, field gradients, and multidimensional data records. We can scale them, add them, and manipulate them with straightforward componentwise arithmetic. Yet an individual vector by itself is a static object. Knowing how to scale and add single vectors tells us how to name destinations, but it does not tell us what happens when an entire space is transformed, nor does it tell us how to systematically determine which vectors can be formed from an entire collection of directions. That deeper structure unfolds in Chapter 3 (Definition 3.1), which investigates what happens when we scale and add sets of vectors systematically: the geometry of linear combinations and the span.

The next chapter takes the one operation we built everything from — "scale, then add" — and asks the question that organizes all of linear algebra: starting from a handful of vectors, *which other vectors can you reach*? That set of reachable vectors is the *span*, and chasing it down leads straight to lines, planes, subspaces, and the four fundamental spaces at the heart of Strang's course. Keep the three pictures close; we are about to put them to work.

2.12 Exercises

Exercise 2.1 (Warm-up arithmetic and mental checks)**.** Let $\mathbf{u} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$. Compute componentwise: (a) $\mathbf{u} + \mathbf{v}$; (b) $\mathbf{u} - \mathbf{v}$; (c) $3\mathbf{u}$; (d) $2\mathbf{u} - \frac{1}{2}\mathbf{v}$; (e) $\mathbf{u} + \mathbf{v} + \mathbf{0}$.

(f) *Mental checks*: Without touching pen or paper, what is the length of $\begin{bmatrix} 0 \\ -7 \end{bmatrix}$? What is the length of $\begin{bmatrix} 3 \\ -4 \end{bmatrix}$? If a vector $\mathbf{w}$ satisfies $3\mathbf{w} = \begin{bmatrix} 9 \\ -6 \end{bmatrix}$, what is $\mathbf{w}$?

Exercise 2.2 (Draw the parallelogram)**.** With $\mathbf{u} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, draw a coordinate grid and carefully sketch $\mathbf{u}$, $\mathbf{v}$, $\mathbf{u} + \mathbf{v}$, and $\mathbf{u} - \mathbf{v}$, all from the origin (draw $\mathbf{u} - \mathbf{v}$ also as the arrow between the appropriate tips). Confirm visually that $\mathbf{u}$, $\mathbf{v}$, and $\mathbf{u} + \mathbf{v}$ form three corners of a parallelogram, and identify the fourth corner.

Exercise 2.3 (Normalizing)**.** Find the length of each vector and then its unit vector $\hat{\mathbf{v}}$: (a) $\begin{bmatrix} 6 \\ 8 \end{bmatrix}$; (b) $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$; (c) $\begin{bmatrix} 2 \\ -1 \\ 2 \end{bmatrix}$ in $\mathbb{R}^3$. For (b), why does $\hat{\mathbf{v}} = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}$ make sense geometrically?

Exercise 2.4 (Coordinates are coefficients)**.** Write each vector explicitly as a linear combination of standard basis vectors: (a) $\begin{bmatrix} 5 \\ -2 \end{bmatrix}$ in $\mathbb{R}^2$; (b) $\begin{bmatrix} 0 \\ 3 \\ -7 \end{bmatrix}$ in $\mathbb{R}^3$; (c) $\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$ in $\mathbb{R}^4$. Then, going the other way, write out the single vector equal to $2\mathbf{e}_1 - 5\mathbf{e}_3$ in $\mathbb{R}^3$.

Exercise 2.5 (Scaling versus shifting)**.** A classmate claims that "doubling a vector means adding 2 to each entry." Using $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, compute both the true $2\mathbf{v}$ and the classmate's "$\mathbf{v} + \begin{bmatrix} 2 \\ 2 \end{bmatrix}$," draw both results, and explain in one or two sentences the geometric difference between scaling and shifting.

Exercise 2.6 (Midpoint and average)**.** Two points are $\mathbf{p} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\mathbf{q} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$. (a) Compute the midpoint $\frac{1}{2}(\mathbf{p} + \mathbf{q})$ and verify it sits halfway along the segment from $\mathbf{p}$ to $\mathbf{q}$. (b) Show in general that $\frac{1}{2}(\mathbf{p} + \mathbf{q})$ is the midpoint by writing it as $\mathbf{p} + \frac{1}{2}(\mathbf{q} - \mathbf{p})$ and reading that as "start at $\mathbf{p}$, walk halfway toward $\mathbf{q}$."

Exercise 2.7 (A house cloud)**.** Three houses are recorded as $\mathbf{h}_1 = \begin{bmatrix} 100 \\ 2 \\ 30 \end{bmatrix}$, $\mathbf{h}_2 = \begin{bmatrix} 140 \\ 3 \\ 10 \end{bmatrix}$, $\mathbf{h}_3 = \begin{bmatrix} 120 \\ 4 \\ 20 \end{bmatrix}$ (area, bedrooms, age). (a) Compute the "average house" $\bar{\mathbf{h}} = \frac{1}{3}(\mathbf{h}_1 + \mathbf{h}_2 + \mathbf{h}_3)$. (b) Compute the difference vectors $\mathbf{h}_2 - \mathbf{h}_1$ and $\mathbf{h}_3 - \mathbf{h}_1$ and state, in words, how houses 2 and 3 each differ from house 1.

Exercise 2.8 (The arrow between points)**.** For $\mathbf{u} = \begin{bmatrix} 7 \\ 1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$, find the displacement from $\mathbf{v}$ to $\mathbf{u}$ and the displacement from $\mathbf{u}$ to $\mathbf{v}$. How are the two related, and why does that match the picture of flipping an arrow?

Exercise 2.9 (Prove a rule)**.** Prove the distributive law $c(\mathbf{u}+\mathbf{v}) = c\mathbf{u} + c\mathbf{v}$ for all $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and all scalars $c \in \mathbb{R}$, by comparing the two sides one component at a time (imitate the sample proof of commutativity in the text). Which single fact about ordinary real numbers does your proof rely on?

Exercise 2.10 (When is a multiple a unit vector?)**.** Let $\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$. Find *every* scalar c for which $c\mathbf{v}$ is a unit vector. (Hint: $\|c\mathbf{v}\| = |c|\,\|\mathbf{v}\|$; set this equal to 1.) How many such c are there, and where do the two resulting vectors point?

Exercise 2.11 (Collinear or not — a geometric eye)**.** Three points are $\mathbf{a} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$, $\mathbf{c} = \begin{bmatrix} 7 \\ 4 \end{bmatrix}$. By comparing the displacement $\mathbf{b} - \mathbf{a}$ with $\mathbf{c} - \mathbf{a}$, decide whether the three points lie on a single straight line. (They are collinear exactly when one displacement is a scalar multiple of the other — connect this to the line-through-the-origin picture of scalar multiples.)

Exercise 2.12 (The triangle inequality: why straight is shortest)**.** In any triangle, the length of any one side cannot exceed the sum of the lengths of the other two sides.

(a) Draw $\mathbf{u} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}$ tip-to-tail. Compute $\|\mathbf{u}\|$, $\|\mathbf{v}\|$, and $\|\mathbf{u}+\mathbf{v}\|$. Verify that the strict inequality $\|\mathbf{u}+\mathbf{v}\| < \|\mathbf{u}\| + \|\mathbf{v}\|$ holds.

(b) When does equality hold: $\|\mathbf{u}+\mathbf{v}\| = \|\mathbf{u}\| + \|\mathbf{v}\|$? State the precise geometric condition on $\mathbf{u}$ and $\mathbf{v}$ in plain English, and verify it with a simple numerical example.

Chapter 3

Linear Combinations, Span, and Geometry

In the last chapter you met a vector as three things at once: a list of numbers, a point in space, and an arrow from the origin. Now we put scaling and addition together into the central operation of linear algebra: the *linear combination*.

Almost everything in this subject is a version of one question: *starting from a few vectors, where can I get to by scaling them and adding them up?* The set of reachable destinations has a name – the **span** – and the shapes it takes (a point, a line, a plane, all of space) are the geometry hiding inside every system of equations, every matrix, every dataset. Once you can picture what a span looks like, half of Strang's course will feel natural the moment you meet it.

This chapter is short on machinery and long on pictures, so take it slowly.

3.1 Scaling and Adding: The Two Moves

You only ever do two things to vectors in linear algebra, and you met both already. They are worth naming explicitly, because the entire chapter is built from these two moves and nothing else.

Move one: scaling. Take a vector $\mathbf{v}$ and a number c (a *scalar*). Then $c\,\mathbf{v}$ stretches $\mathbf{v}$ by the factor c. If $c > 1$ it grows; if $0 < c < 1$ it shrinks; if $c < 0$ it flips to point the other way; if $c = 0$ it collapses to the origin. Notice that scaling never lets $\mathbf{v}$ leave its own line. No matter what c you pick, $c\,\mathbf{v}$ sits somewhere on the infinite line through the origin and the tip of $\mathbf{v}$.

Move two: adding. Take two vectors and add them tip-to-tail (the parallelogram rule from the last chapter). Addition lets you leave a single line and explore sideways.

Intuition

Scaling moves you *along* a direction. Adding lets you *mix* directions. A linear combination is exactly this: pick how much of each direction you want (scaling), then mix them (adding). The rest of the chapter watches what shapes you can reach by doing this.

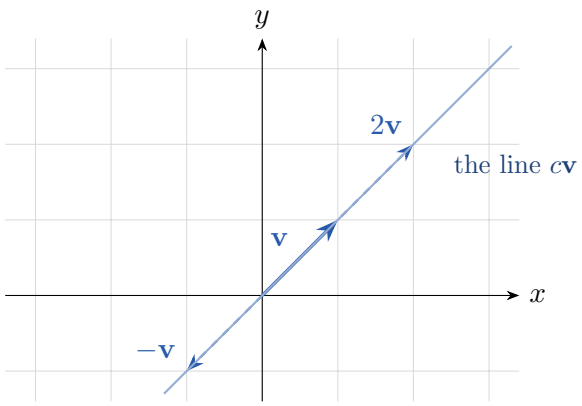


Figure 3.1: Scaling alone. Every scalar multiple $c\,\mathbf{v}$ of a single nonzero vector lands on one line through the origin. Scaling can stretch, shrink, or flip, but it cannot escape the line.

3.2 The Linear Combination

Combine the two moves into one definition. Take vectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k$ and scalars $c_1, c_2, \ldots, c_k$. The expression

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

is called a **linear combination** of the vectors. The scalars c_i are the **weights** (or coefficients). The word "linear" is doing real work: the only operations allowed are scaling (linear in each weight) and adding. No squaring a vector, no multiplying two vectors together, no taking a sine. Just scale and add.

Definition 3.1 (Linear combination)**.** A vector $\mathbf{b}$ is a **linear combination** of $\mathbf{v}_1, \ldots, \mathbf{v}_k \in \mathbb{R}^n$ if there exist scalars $c_1, \ldots, c_k \in \mathbb{R}$ such that

$$\mathbf{b} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k.$$

Make it concrete. Take two vectors in the plane,

$$\mathbf{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Here is the linear combination with weights $c_1 = 2$ and $c_2 = 1$:

$$2\begin{bmatrix} 2 \\ 1 \end{bmatrix} + 1\begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

And with weights $c_1 = 1$, $c_2 = -1$:

$$1\begin{bmatrix} 2 \\ 1 \end{bmatrix} - 1\begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Each choice of weights gives a destination, and different weights give different destinations. The central question of the chapter: as we sweep the weights over *all* possible real numbers, what is the full set of destinations?

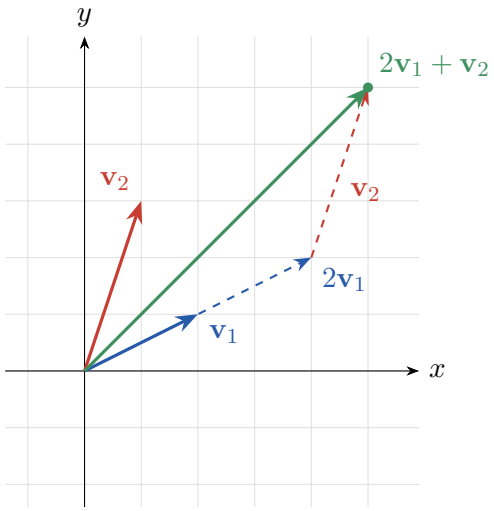


Figure 3.2: The linear combination $2\mathbf{v}_1 + \mathbf{v}_2 = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$ built tip-to-tail: walk out $2\mathbf{v}_1$ (blue, doubled), then add $\mathbf{v}_2$ (red) from there. The green arrow is the result.

Focus

A linear combination is a weighted sum: each scalar determines how much of a direction to include, and their vector sum is the result. The span (defined next) is the collection of all points reachable by varying these weights across all real values.

Quick mental check. Suppose your building blocks are the standard coordinate vectors $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. What weights do you need to reach $\begin{bmatrix} 7 \\ -4 \end{bmatrix}$? You do not have to solve any equations: $7\mathbf{e}_1 - 4\mathbf{e}_2 = \begin{bmatrix} 7 \\ -4 \end{bmatrix}$. The coordinates of a vector *are* its weights in the recipe of the standard axes.

3.3 Span: The Set of All Combinations

Now the central object of the chapter.

Definition 3.2 (Span)**.** The **span** of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$, written $\text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$, is the set of *all* their linear combinations:

$$\text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\} = \{\, c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k : c_1, \ldots, c_k \in \mathbb{R} \,\}.$$

Read that as a verb and a noun. As a verb: the vectors *span* a region – they reach it, they cover it. As a noun: the span *is* that region, the complete set of reachable points. The weights are the dials; the span is everywhere the dials can take you.

A span is never a random scatter of points. It is always one of a short list of clean geometric shapes: the origin alone, a line, a plane, all of 3-space, and so on up. We will walk up that ladder one rung at a time, because the ladder *is* the geometry of linear algebra.

3.3.1 Span of one vector: a line (usually)

Start with a single nonzero vector $\mathbf{v}$. Its span is the set of all $c\,\mathbf{v}$. We already saw this in Figure 3.1: scaling one vector traces out a line through the origin. So

$$\text{span}\{\mathbf{v}\} = \text{the line through the origin in the direction of } \mathbf{v}.$$

There is one exception, and it matters. If $\mathbf{v} = \mathbf{0}$, then $c\,\mathbf{0} = \mathbf{0}$ for every c. The span of the zero vector consists solely of the origin, $\{\mathbf{0}\}$. Scaling the zero vector produces only the zero vector.

Common trap

"A line" here always means a line *through the origin*. The span of one vector can never be a line that misses the origin, because $c = 0$ always produces $\mathbf{0}$, so the origin is in *every* span. If someone hands you a line that does not pass through the origin and asks "is this a span?" the answer is no. (It is a *shifted* line – an affine line – and we will meet those when we solve $\mathbf{Ax} = \mathbf{b}$ for nonzero $\mathbf{b}$.)

Quick mental check. Is the origin $\mathbf{0}$ in the span of *any* collection of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$? Yes, always: set every weight to zero ($0\mathbf{v}_1 + \cdots + 0\mathbf{v}_k = \mathbf{0}$). No matter what vectors you choose, their span is anchored at the origin.

3.3.2 Span of two vectors: a plane (usually)

Now take two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ and ask for $\text{span}\{\mathbf{v}_1, \mathbf{v}_2\} = \{c_1\mathbf{v}_1 + c_2\mathbf{v}_2\}$.

Think of it as a grid. Sliding c_1 moves you along the $\mathbf{v}_1$ direction; sliding c_2 moves you along the $\mathbf{v}_2$ direction. With two independent dials you can reach any point on a flat sheet – a **plane** through the origin. The two vectors lay down a (possibly skewed) coordinate grid, and the span is the whole infinite sheet that grid lives in.

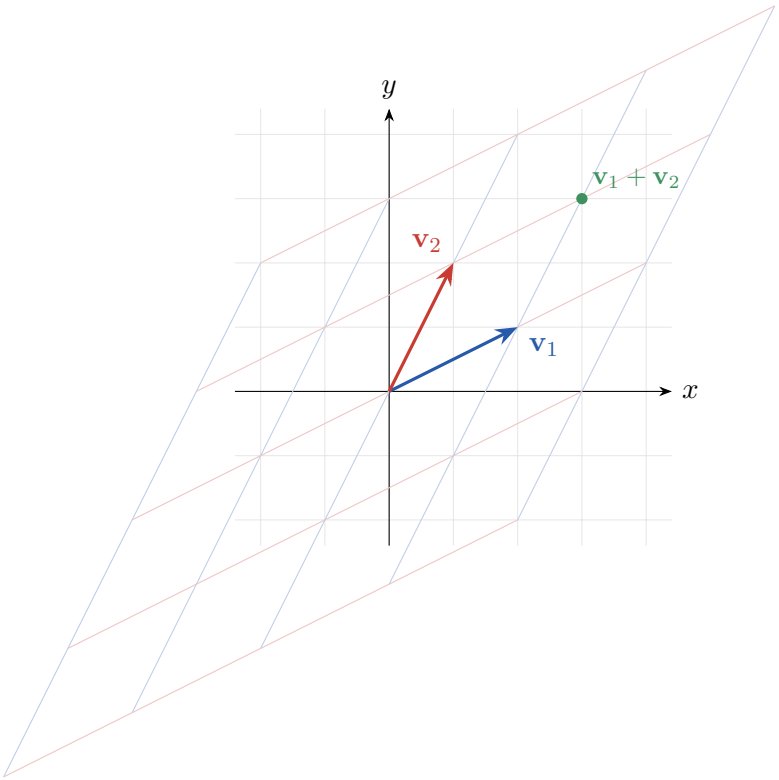


Figure 3.3: Two non-parallel vectors lay down a skewed grid. Sliding the two weights c_1, c_2 reaches every point of the plane – here, in $\mathbb{R}^2$, that plane is the entire flat page. The blue and red families of lines are the "coordinate lines" of the grid.

In $\mathbb{R}^2$ that plane *is* all of $\mathbb{R}^2$: two non-parallel vectors in the plane span the whole plane. (We will prove this cleanly below.) But the same two vectors living in $\mathbb{R}^3$ span a genuine 2-dimensional plane floating inside 3-space – a flat sheet through the origin, with all of 3-space stretching above and below it that you cannot reach.

The exception again. What if $\mathbf{v}_2$ happens to lie on the same line as $\mathbf{v}_1$ – say $\mathbf{v}_2 = 3\mathbf{v}_1$? Then the second dial gives you nothing new. Anything you build, $c_1\mathbf{v}_1 + c_2(3\mathbf{v}_1) = (c_1 + 3c_2)\mathbf{v}_1$, is still just a multiple of $\mathbf{v}_1$. The span collapses back down to a single line. Two vectors span a plane only when they point in genuinely different directions.

3.3.3 Span of three vectors: all of space (usually)

Three vectors in $\mathbb{R}^3$ that point in three genuinely different directions span all of $\mathbb{R}^3$. Three dials, three independent directions, and you can reach every point in space.

But watch the exceptions stack up, because they are the whole story of *dependence*:

- If the third vector $\mathbf{v}_3$ already lies in the plane spanned by $\mathbf{v}_1$ and $\mathbf{v}_2$, it adds no new direction. The span stays a plane.
- If all three lie on one line, the span is just that line.
- If all three are $\mathbf{0}$, the span is the origin.

Focus

The ladder of spans: *point, line, plane, space, ...* Each new vector either bumps you up one rung (it points somewhere genuinely new) or it does nothing (it was already reachable). That binary – "new direction" versus "nothing new" – is the seed of *linear independence*, the topic of a coming chapter. You are meeting it here, geometrically, before its name.

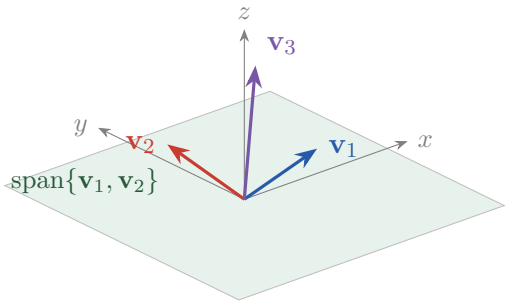


Figure 3.4: In $\mathbb{R}^3$: $\mathbf{v}_1, \mathbf{v}_2$ span the shaded plane through the origin. A third vector $\mathbf{v}_3$ that pokes *out* of that plane is a brand new direction – together the three span all of $\mathbb{R}^3$. If $\mathbf{v}_3$ had lain flat in the plane, it would have added nothing.

Walking question

Picture a single vector in $\mathbb{R}^3$, drawn as an arrow. Its span is a line. Now picture sweeping that line around – how many genuinely *different* lines through the origin are there? Infinitely many, one for each direction. Now: how many genuinely different *planes* through the origin are there in $\mathbb{R}^3$? Also infinitely many. A span in $\mathbb{R}^3$ is therefore determined by two things: *its dimension and its orientation.* On your walk, try to feel the difference between "what dimension is this span" and "which way is it tilted." Both questions matter; the first is about how many independent directions, the second about orientation.

3.4 A Fully Worked Span Question

Definitions are cheap. Computing one span end to end by hand teaches more than ten of them, so let us do exactly that.

Example 3.1 (Is $\mathbf{b}$ in the span?)**.** Let

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}.$$

Is $\mathbf{b} \in \operatorname{span}\{\mathbf{v}_1, \mathbf{v}_2\}$? In words: can we hit the target $\mathbf{b}$ by mixing $\mathbf{v}_1$ and $\mathbf{v}_2$?

Being in the span means there *exist* weights c_1, c_2 with

$$c_1 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + c_2 \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}.$$

Reading off each coordinate turns this single vector equation into two scalar equations – one per row:

$$\begin{aligned} c_1 + 3c_2 &= 5 \quad \text{(top row)}, \\ 2c_1 + c_2 &= 0 \quad \text{(bottom row)}. \end{aligned}$$

From the second equation, $c_2 = -2c_1$. Substitute into the first:

$$c_1 + 3(-2c_1) = 5 \implies c_1 - 6c_1 = 5 \implies -5c_1 = 5 \implies c_1 = -1.$$

Then $c_2 = -2(-1) = 2$. Now *check* – always check:

$$-1\begin{bmatrix}1\\2\end{bmatrix} + 2\begin{bmatrix}3\\1\end{bmatrix} = \begin{bmatrix}-1\\-2\end{bmatrix} + \begin{bmatrix}6\\2\end{bmatrix} = \begin{bmatrix}5\\0\end{bmatrix} = \mathbf{b}.\checkmark$$

Yes. $\mathbf{b}$ is in the span, reached with weights $(-1, 2)$. And because $\mathbf{v}_1, \mathbf{v}_2$ point in different directions, they span *all* of $\mathbb{R}^2$ – so this would have worked for any target $\mathbf{b}$ we picked.

Focus

Notice the equivalence here: "is $\mathbf{b}$ in the span?" is the same question as "does this system of linear equations have a solution?" That link connects the geometry of spans and planes to the algebra of linear systems, and we use it throughout the next several chapters.

3.5 The Column Picture: Ax as a Combination of Columns

What follows is, in Strang's own words, the most important idea in the whole subject. It reframes matrix-times-vector so that it *becomes* a linear combination, and the reframing tends to stick once you have seen it.

Take a matrix and a vector:

$$\mathbf{A} = \begin{bmatrix}1 & 3\\2 & 1\end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix}c_1\\c_2\end{bmatrix}.$$

The usual "rows times the vector" recipe for $\mathbf{Ax}$ gives

$$\mathbf{Ax} = \begin{bmatrix}1 & 3\\2 & 1\end{bmatrix}\begin{bmatrix}c_1\\c_2\end{bmatrix} = \begin{bmatrix}1\cdot c_1 + 3\cdot c_2\\2\cdot c_1 + 1\cdot c_2\end{bmatrix}.$$

Now *regroup that right-hand side by columns* instead of by rows:

$$\begin{bmatrix}c_1 + 3c_2\\2c_1 + c_2\end{bmatrix} = \begin{bmatrix}c_1\\2c_1\end{bmatrix} + \begin{bmatrix}3c_2\\c_2\end{bmatrix} = c_1\begin{bmatrix}1\\2\end{bmatrix} + c_2\begin{bmatrix}3\\1\end{bmatrix}.$$

Those two final vectors are the columns of $\mathbf{A}$: $\begin{bmatrix}1\\2\end{bmatrix}$ is the first column, $\begin{bmatrix}3\\1\end{bmatrix}$ the second. So

$$\mathbf{Ax} = c_1\,(\text{column 1 of } \mathbf{A}) + c_2\,(\text{column 2 of } \mathbf{A}).$$

Formal

The column rule for Ax. If $\mathbf{A}$ has columns $\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_n$ (each a vector in $\mathbb{R}^m$) and $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$, then

$$\mathbf{Ax} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

Ax is the linear combination of the columns of A with weights taken from x.

This is the column picture. The entries of $\mathbf{x}$ are exactly the *weights* in a linear combination, and the things being combined are the columns. Multiplying by a matrix *is* taking a linear combination of its columns – nothing more.

Figure 3.5 draws this for the system we solved by hand in Example 3.1: the columns of $\mathbf{A}$ are the two arrows you are allowed to scale, and $\mathbf{Ax} = \mathbf{b}$ just asks you to dial the weights until tip-to-tail you land exactly on the target $\mathbf{b}$.

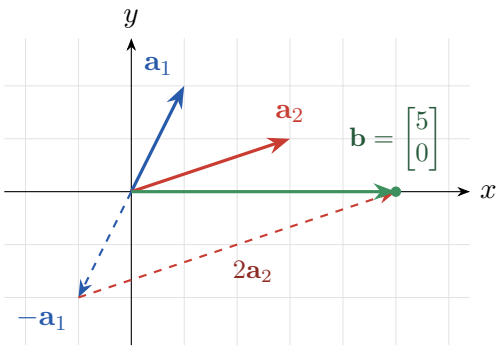


Figure 3.5: The column picture of $\mathbf{Ax} = \mathbf{b}$ for $\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$. Solving the system means choosing weights so that $x_1\mathbf{a}_1 + x_2\mathbf{a}_2$ reaches the green target. The weights $(-1, 2)$ from Example 3.1 do it: walk $-\mathbf{a}_1$ (down-left), then add $2\mathbf{a}_2$, and you land on $\mathbf{b}$.

Intuition

There are two ways to read $\mathbf{Ax}$, and a master keeps both in mind:

- **Row picture:** each entry of the output is computed row by row as a dot product (multiplying matching entries across the row and down $\mathbf{x}$, then adding them up). Good for computing one number at a time, and for thinking of each equation as a line or plane.
- **Column picture:** the whole output is a linear combination of the columns, with weights $\mathbf{x}$. Good for thinking about *which outputs are reachable at all.*

The row picture answers "what is the answer?" The column picture answers "which answers are even possible?" This chapter lives in the column picture.

Compute it yourself

Do this one with pen and paper before reading on. Let

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

Compute $\mathbf{Ax}$ *twice*: once the row way (multiplying matching entries across each row and adding), once the column way (1 times column one, plus 2 times column two, minus 1 times column three, added up). You should get the same vector both times – $\begin{bmatrix} 1 \\ 6 \end{bmatrix}$. Seeing the two routes land on the same point is important.

Quick mental check. What happens if you multiply an $m \times n$ matrix $\mathbf{A}$ by the first standard coordinate vector $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$? By the column rule,

$$\mathbf{Ae}_1 = 1\mathbf{a}_1 + 0\mathbf{a}_2 + \cdots + 0\mathbf{a}_n = \mathbf{a}_1.$$

Multiplying by $\mathbf{e}_1$ plucks out the first column of $\mathbf{A}$. In general, $\mathbf{Ae}_j$ extracts the j-th column of $\mathbf{A}$ – a fact we will rely on across the entire subject.

3.5.1 What the column picture says about solving $\mathbf{Ax} = \mathbf{b}$

Here is the payoff. The equation $\mathbf{Ax} = \mathbf{b}$ asks: *find weights* $\mathbf{x}$ *so that the linear combination of the columns equals* $\mathbf{b}$. In our column language:

$$\mathbf{Ax} = \mathbf{b} \iff x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n = \mathbf{b} \iff \mathbf{b} \in \operatorname{span}\{\mathbf{a}_1, \ldots, \mathbf{a}_n\}.$$

Read that chain slowly. **The system $\mathbf{Ax} = \mathbf{b}$ has a solution if and only if $\mathbf{b}$ lies in the span of the columns of $\mathbf{A}$.** The span of the columns is so important it gets a name – the **column space** of $\mathbf{A}$, written $\operatorname{col}(\mathbf{A})$ – and an entire chapter later in the book. You are meeting it here as "the set of all reachable right-hand sides."

Focus

Solvability is a geometry question. "Can I solve $\mathbf{Ax} = \mathbf{b}$?" is *exactly* "is the target $\mathbf{b}$ sitting inside the span of the columns?" If $\mathbf{b}$ is in the column space, a solution exists. If $\mathbf{b}$ floats outside it, no choice of weights will ever reach it, and the system is unsolvable – not because you were not clever enough, but because the target is geometrically out of reach.

3.6 Reaching a Target b

A few small cases turn solvability into pictures, and the geometry tells you the answer before any algebra does.

Case 1: columns span everything. Take

$$\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}.$$

Its columns $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\begin{bmatrix} 3 \\ 1 \end{bmatrix}$ point in different directions, so they span all of $\mathbb{R}^2$. Therefore *every* target $\mathbf{b} \in \mathbb{R}^2$ is reachable – $\mathbf{Ax} = \mathbf{b}$ is solvable for every $\mathbf{b}$, exactly as we saw in Example 3.1. The column space fills the whole plane; nothing is out of reach.

Case 2: columns collapse to a line. Take

$$\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

Its second column $\begin{bmatrix} 2 \\ 4 \end{bmatrix}$ is exactly twice the first column $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. They lie on the same line. So no matter what weights x_1, x_2 you choose,

$$x_1 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + x_2 \begin{bmatrix} 2 \\ 4 \end{bmatrix} = (x_1 + 2x_2) \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

is always a multiple of $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. The column space is a single line – the line through the origin in direction $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. The system $\mathbf{Bx} = \mathbf{b}$ is solvable *only* for targets $\mathbf{b}$ sitting on that line, and unsolvable for every $\mathbf{b}$ off it.

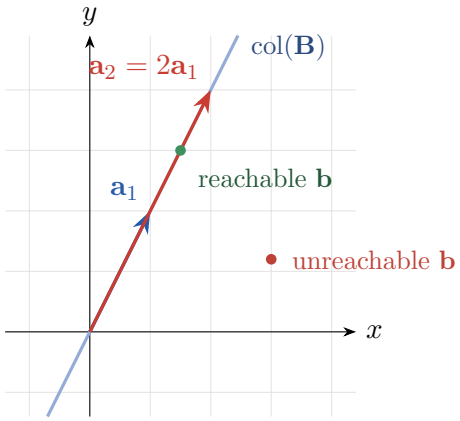


Figure 3.6: When the columns are parallel, the column space collapses to a line. Targets on the line (green) are reachable; targets off the line (red) are forever out of reach, no matter the weights.

Example 3.2 (Reading solvability off the geometry)**.** For $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ above, is $\mathbf{Bx} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$ solvable? Is $\mathbf{Bx} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ solvable?

First target. $\begin{bmatrix} 3 \\ 6 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ sits right on the line, so yes – pick $x_1 = 3, x_2 = 0$ (or $x_1 = 1, x_2 = 1$, or infinitely many others, since $x_1 + 2x_2 = 3$ is one equation in two unknowns). Reachable, in infinitely many ways.

Second target. $\begin{bmatrix}3\\1\end{bmatrix}$ is *not* a multiple of $\begin{bmatrix}1\\2\end{bmatrix}$ (that would need the second entry to be 6, not 1). It floats off the line. No weights reach it. Unsolvable. The algebra confirms it: the two equations $x_1 + 2x_2 = 3$ and $2x_1 + 4x_2 = 1$ say $2(x_1 + 2x_2)$ equals both 6 and 1 – a flat contradiction.

Common trap

A square matrix does *not* guarantee a solution. Both $\mathbf{A}$ and $\mathbf{B}$ above are 2×2, but $\mathbf{A}$ solves for every $\mathbf{b}$ while $\mathbf{B}$ fails for most. What matters is not just that the matrix is square, but whether its columns span the whole target space or collapse onto a lower-dimensional subspace. Always ask "what do the columns span?" before assuming a solution exists.

Modern application

Color, recipes, and recommender systems. Linear combinations are how machines mix and reach things.

A screen pixel is a linear combination of three column vectors – red, green, blue "primaries." The weights (r, g, b) are the dials. The span of those three columns is the set of *all displayable colors* (the monitor's gamut). A color outside that span literally cannot be shown on the screen – it is an unreachable $\mathbf{b}$, exactly like the red dot in Figure 3.6. Wider-gamut displays add primaries that point in "newer directions," enlarging the span.

The same idea powers recommender systems. In a model like matrix factorization, each user's taste is a short vector of weights, and a predicted rating is a linear combination of a handful of hidden "taste direction" vectors. The whole catalog of predictable preferences is the span of those directions. When Netflix or Spotify says "because you liked X," it is reporting which combination of latent direction-vectors your taste landed on. The dimension of that span – how many genuinely different taste directions the model keeps – is a design knob (the "rank"), and you will meet it precisely when we get to the SVD and PCA.

3.7 When a New Vector Adds Nothing: Dependence, Previewed

We keep bumping into the same phenomenon: sometimes a new vector enlarges the span, sometimes it is dead weight. The full theory comes later, but the geometry is already in plain view, so it is worth naming the pattern now.

Definition 3.3 (Adds nothing / redundant – informal)**.** A vector $\mathbf{v}$ is **redundant** for a collection if it already lies in the span of the others – that is, $\mathbf{v}$ is itself a linear combination of them. Adding a redundant vector does not change the span.

Two quick litmus tests you can run by eye:

- **One vector is a multiple of another** ($\mathbf{v}_2 = c\,\mathbf{v}_1$): redundant. They share a line. This is the case of $\mathbf{B}$ above.
- **One vector lies in the plane of two others** ($\mathbf{v}_3 = a\mathbf{v}_1 + b\mathbf{v}_2$): redundant. It adds no new direction out of that plane.

When *no* vector in the set is redundant – when every one of them points somewhere the others cannot reach – we call the set **linearly independent**. When at least one is redundant, the set is

linearly dependent. That is the whole geometric content of those two intimidating words, and you understand it already.

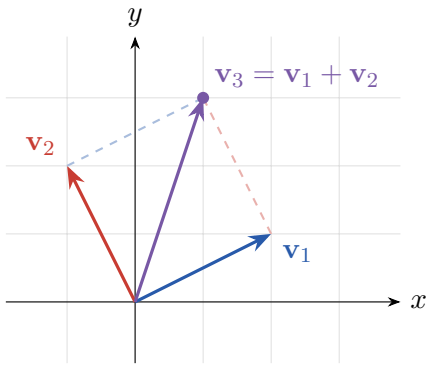


Figure 3.7: Redundancy in $\mathbb{R}^2$. Two non-parallel vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ already span the entire plane. Any third vector $\mathbf{v}_3$ lies in that plane and can be built as a linear combination of them ($1\mathbf{v}_1 + 1\mathbf{v}_2$ here). Because $\mathbf{v}_3$ points nowhere new, it is redundant – the set $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ is linearly dependent.

Theorem 3.1 (Two non-parallel vectors span all of $\mathbb{R}^2$)**.** *Let $\mathbf{v}_1, \mathbf{v}_2 \in \mathbb{R}^2$ be nonzero and non-parallel (neither is a scalar multiple of the other). Then* $\operatorname{span}\{\mathbf{v}_1, \mathbf{v}_2\} = \mathbb{R}^2$*: every vector in the plane is some linear combination of them.*

Proof. Write $\mathbf{v}_1 = \begin{bmatrix} a \\ c \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} b \\ d \end{bmatrix}$, and let $\mathbf{b}^\star = \begin{bmatrix} p \\ q \end{bmatrix}$ be *any* target. We must find weights x_1, x_2 with $x_1\mathbf{v}_1 + x_2\mathbf{v}_2 = \mathbf{b}^\star$, i.e.

$$\begin{aligned} ax_1 + bx_2 &= p, \\ cx_1 + dx_2 &= q. \end{aligned}$$

Solve by elimination. Multiply the first equation by d, the second by b, and subtract:

$$(ad - bc)\, x_1 = dp - bq.$$

The number $ad - bc$ is the *determinant* of the 2×2 matrix with columns $\mathbf{v}_1, \mathbf{v}_2$ – a number measuring the signed area of the parallelogram they span. The non-parallel hypothesis is exactly the statement $ad - bc \neq 0$: if the vectors were parallel ($\mathbf{v}_2 = \lambda\mathbf{v}_1$), their components would be proportional ($b = \lambda a$ and $d = \lambda c$), forcing $ad - bc = a(\lambda c) - (\lambda a)c = 0$, collapsing that area to zero. Since $ad - bc \neq 0$, the denominator is safe and we may divide:

$$x_1 = \frac{dp - bq}{ad - bc}, \qquad \text{and similarly} \qquad x_2 = \frac{aq - cp}{ad - bc}.$$

These weights exist for *every* (p, q), so every target is reachable. Hence the span is all of $\mathbb{R}^2$. □

Remark 3.1. Notice the chain of equivalences in that proof: "the vectors are non-parallel" (geometry), "$ad - bc \neq 0$" (algebra), "the columns span $\mathbb{R}^2$" (span), and "$\mathbf{Ax} = \mathbf{b}$ is solvable for every $\mathbf{b}$" (linear systems) all state the exact same condition. Much of linear algebra consists of translating fluently among these four views.

3.7.1 Counting: why you cannot span $\mathbb{R}^3$ with two vectors

Here is a fact your geometric intuition already insists on.

Proposition 3.1 (Too few vectors cannot span)**.** *The span of k vectors in $\mathbb{R}^n$ is at most k-dimensional. In particular, fewer than n vectors can never span all of $\mathbb{R}^n$.*

We will only sketch the reasoning now (the airtight proof waits for the dimension chapter). Each vector you add can push the span up by *at most* one dimension – one new direction at most. Starting from nothing (dimension 0), after k vectors you have climbed at most k rungs of the point-line-plane-space ladder. So two vectors span at most a plane; they sit flat inside $\mathbb{R}^3$ and can never fill it. You need at least three vectors to span $\mathbb{R}^3$, at least n to span $\mathbb{R}^n$. Conversely, more vectors than necessary guarantees redundancy: any $n+1$ vectors in $\mathbb{R}^n$ *must* be dependent, because the $(n+1)$-th has nowhere new to point.

Quick mental check. Can four vectors in $\mathbb{R}^3$ ever be linearly independent? By Proposition 3.1, no. Three independent vectors already fill all of 3-space; any fourth vector has nowhere new to point, so it is trapped in the span of the first three and must be redundant.

Connection to other math

Spans show up far past arrows in $\mathbb{R}^n$. Once you recognize the pattern "set of all combinations of a few generators," you start seeing it everywhere.

- **Polynomials.** The functions $1, t, t^2$ are "vectors," and their span is every quadratic $a + bt + ct^2$. Calculus students compute in this span constantly without naming it. Taylor series live in the span of $1, t, t^2, t^3, \ldots$
- **Differential equations.** The solutions of $y'' + y = 0$ are exactly the linear combinations $c_1 \cos t + c_2 \sin t$ – a 2-dimensional span of two "basis solutions." Solving a linear ODE is finding generators and taking their span.
- **Signals and Fourier.** A sound wave is (approximately) a linear combination of pure sine tones. The span of the basic tones is the space of signals you can build; an equalizer is just you, turning the weight dials.
- **Geometry of subspaces.** A line through the origin, a plane through the origin, and all of $\mathbb{R}^3$ are all examples of the same construction: spans of 1, 2, and 3 independent vectors. Taking spans produces *subspaces*, the subject of the middle chapters of this book.

Whenever you see "all combinations of a few building blocks," you are looking at a span, and everything in this chapter applies.

Before/after Strang

Before Strang: when he opens his course with the "column picture" and insists $\mathbf{Ax}$ is a combination of columns, you will already own it – you derived it above by regrouping $\begin{bmatrix} c_1 + 3c_2 \\ 2c_1 + c_2 \end{bmatrix}$ into columns. His famous line, "linear algebra is about $\mathbf{Ax} = \mathbf{b}$, and the question is whether $\mathbf{b}$ is a combination of the columns," is precisely the chain we built in the section on reaching a target.

After Strang: when you return for the *column space* $\mathrm{col}(\mathbf{A})$ and the *four fundamental*

subspaces, recognize that the column space is simply the span of the columns introduced here, studied as a subspace in its own right. The rank of a matrix is the dimension of this span – the number of linearly independent directions its columns span. Every subsequent theorem about rank is fundamentally a theorem about the dimension of a span.

Why this matters

Notice the structural economy. Two basic operations – scaling and adding – generate the foundational geometric vocabulary of the subject: lines, planes, higher-dimensional subspaces, and the criterion for whether an equation $\mathbf{Ax} = \mathbf{b}$ is solvable. From these operations alone, we obtain a precise dictionary connecting geometric spans to algebraic systems, laying the groundwork for linear independence, dimension, and rank.

History and connection

The idea that spaces of any dimension can be generated by taking linear combinations of a few directions belongs above all to Hermann Grassmann (1809–1877), a secondary school teacher in Stettin, Prussia. In the 1830s, mathematicians and physicists were stuck in a computational bottleneck: problems in tidal forces, rotation, and mechanics were buried under mountains of coordinate calculations. If you rotate your coordinate axes or shift your origin, every single (x, y, z) number changes, even though the physical object and the forces acting on it have not changed at all. The only available tools were Cartesian coordinates (grids of numbers pegged to fixed perpendicular axes, introduced by René Descartes) and classical Euclidean geometry, which had no language for spaces beyond three dimensions. Grassmann wanted an algebra of geometry that worked directly on directed geometric quantities in four, five, or n dimensions without coordinates.

In his masterwork of 1844, *Die Lineare Ausdehnungslehre* ("The Theory of Linear Extension"), Grassmann showed that linear combinations $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$ sweep out flat linear spaces: lines, planes, and higher-dimensional hyperplanes (flat slices of space that have one dimension less than the ambient space). Because scaling and adding vectors never bend straight lines, these combinations always produce flat subspaces with zero intrinsic curvature.

Grassmann's book was decades ahead of its time, written in dense, philosophical German without numerical worked examples. When he submitted it to the University of Marburg hoping for a professorship, the faculty dismissed it as unreadable mysticism. Even August Ferdinand Möbius (the Leipzig mathematician best known for the one-sided Möbius strip) confessed he could not follow Grassmann's terminology. The mathematical community refused to believe that algebraic operations could be performed on geometric magnitudes without coordinates. Disheartened by three decades of obscurity, Grassmann abandoned mathematics in mid-life and turned to historical linguistics, where he achieved world renown by publishing a monumental 1,700-page dictionary of the Rigveda and discovering Grassmann's Law of Indo-European phonetics. Only in 1888, eleven years after Grassmann's death, did the Italian mathematician Giuseppe Peano extract the modern axioms of vector spaces and linear combinations from Grassmann's work.

Grassmann's notion of span gave linear systems a geometric formulation: solving $\mathbf{Ax} = \mathbf{b}$ means asking whether the target $\mathbf{b}$ lies inside the subspace spanned by the columns of $\mathbf{A}$. Today, the frontier of linear combinations has shifted to computational scale. When a dataset has millions of columns, testing whether a combination exists is easy, but two harder problems

remain open and active. The first is *sparse recovery*: finding the recipe that hits or approximates a target $\mathbf{b}$ using the fewest nonzero weights, which is the core challenge of compressed sensing and MRI imaging and is NP-hard in general. The second is *lattice problems*: finding the shortest vector when weights are restricted to integers rather than real numbers, a problem so computationally difficult that modern post-quantum cryptography rests on its hardness.

3.8 Summary

- A **linear combination** $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$ is built from only two moves: *scaling* (along a direction) and *adding* (mixing directions). The c_i are weights.
- The **span** is the set of *all* linear combinations – every destination the weights can reach. It always passes through the origin.
- Geometrically the span climbs a ladder: *point* (zero vectors), *line* (one direction), *plane* (two independent directions), *space*, and on up. Each new vector adds at most one rung.
- A vector that already lies in the others' span is **redundant**; it adds nothing. "No redundancy" is **linear independence**; this is the geometric seed of independence, rank, and dimension.
- **Column picture:** $\mathbf{Ax} = x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n$ is a linear combination of the columns of $\mathbf{A}$ with weights $\mathbf{x}$.
- Therefore $\mathbf{Ax} = \mathbf{b}$ is solvable *exactly when* $\mathbf{b}$ lies in the span of the columns (the **column space**). Solvability is a geometry question.

Pause

Cost and intuition recap. The *intuition* is a single sentence: a span is everywhere you can get to by scaling and adding, and it always grows by clean rungs – point, line, plane, space. The *cost* is just as small: to test whether a target $\mathbf{b}$ lies in $\text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$, you do not search forever – you set up one linear system, $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{b}$ (n equations, k unknowns), and solve it once. "In the span?" costs exactly "does this system have a solution?" – no more. Hold those two together: the picture (rungs of a ladder) tells you *what* a span is; the system (solve once, then check) tells you *how much* it costs to use one.

Looking ahead: the missing ruler and protractor

We now have a complete geometric picture of reachability: scaling and adding sweeps out flat spans, and solving $\mathbf{Ax} = \mathbf{b}$ asks whether the target $\mathbf{b}$ lands on that flat sheet.

Now notice what is missing. Suppose $\mathbf{b}$ does *not* lie in the span – like the unreachable red dot in Figure 3.6, or an overdetermined system in data science where measurements are corrupted by noise. What do we do then?

In practice, a target is almost never perfectly reachable. If we cannot reach $\mathbf{b}$ exactly, the natural engineer's question is: *which reachable point comes closest?* And how far away is it?

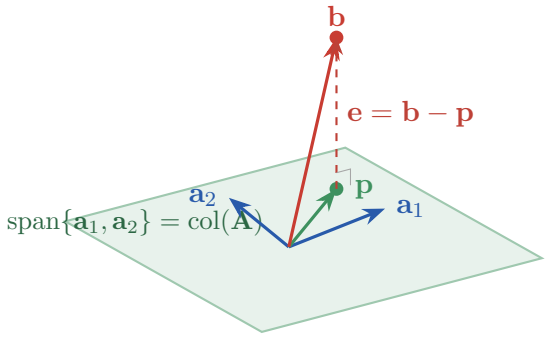


Figure 3.8: The geometric puzzle that motivates Chapter 4 (Section 4.1). When a target $\mathbf{b}$ (red) floats outside the column space col($\mathbf{A}$) (green plane), no linear combination can reach it: $\mathbf{Ax} = \mathbf{b}$ has no exact solution. The best reachable substitute is the projection $\mathbf{p}$ (green), which drops a perpendicular shadow onto the plane so that the error $\mathbf{e} = \mathbf{b} - \mathbf{p}$ is as short as possible. But measuring lengths and perpendicular angles requires an operation that addition and scaling cannot provide – the dot product.

Addition and scaling alone cannot answer that question. They build the flat stage, but they come with neither a ruler to measure distances nor a protractor to measure angles. To find the closest point $\mathbf{p}$ on the span to $\mathbf{b}$, we must drop a perpendicular line from $\mathbf{b}$ down onto the plane (Figure 3.8). But "perpendicular" is an angle concept, and "closest" is a length concept. Neither exists until we introduce a way to measure them.

In Chapter 4 (Section 4.1) we introduce the single operation that supplies both the ruler and the protractor: the **dot product**. Multiplying matching entries and adding them up will seem almost too simple, but inside that tiny sum lies the entire metric geometry of space: lengths, angles, perpendicularity, and the orthogonal projections that let us solve unsolvable systems.

3.9 Exercises

Exercise 3.1 (Warmup combinations)**.** Let $\mathbf{u} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ and $\mathbf{w} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$. Compute each linear combination: (a) $3\mathbf{u} + \mathbf{w}$; (b) $2\mathbf{u} - 2\mathbf{w}$; (c) $0\mathbf{u} + 5\mathbf{w}$; (d) $-\mathbf{u} - \mathbf{w}$. For (c), what does the weight 0 on $\mathbf{u}$ tell you about where the result lands?

Exercise 3.2 (Solve for the weights)**.** Find weights c_1, c_2 so that $c_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$. Then verify your answer by substituting back. (Hint: add and subtract the two scalar equations.)

Exercise 3.3 (In the span or not?)**.** Let $\mathbf{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} -4 \\ -2 \end{bmatrix}$. (a) What is span$\{\mathbf{v}_1, \mathbf{v}_2\}$ geometrically – a point, a line, or a plane? Why? (b) Is $\begin{bmatrix} 6 \\ 3 \end{bmatrix}$ in the span? Is $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ in the span? Justify each answer geometrically, then confirm algebraically.

Exercise 3.4 (Column picture by hand)**.** Let $\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 4 \\ -1 \\ 2 \end{bmatrix}$. Compute $\mathbf{Ax}$ as a

linear combination of the columns of $\mathbf{A}$ (write out "4 times column one, plus ..."). Then state what span of the three columns is, and explain why the third column is redundant.

Exercise 3.5 (Reaching a target)**.** For $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix}$: (a) Describe $\text{col}(\mathbf{A})$ (the span of the columns) as a specific line through the origin. (b) Is $\mathbf{Ax} = \begin{bmatrix} 2 \\ 6 \end{bmatrix}$ solvable? Is $\mathbf{Ax} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$ solvable? Answer by checking whether each target lies on the line from (a). (c) For the solvable case, find *all* solutions $\mathbf{x}$, and explain geometrically why there are infinitely many.

Exercise 3.6 (The third vector decides)**.** In $\mathbb{R}^3$, let $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$. (a) Describe $\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$ as a specific plane. (b) For $\mathbf{v}_3 = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix}$, what is $\text{span}\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$? Did $\mathbf{v}_3$ add a new direction? (c) For $\mathbf{v}_3 = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$, what is $\text{span}\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ now? What changed?

Exercise 3.7 (A span that misses the origin?)**.** Your friend claims the line $y = x + 1$ in the plane is the span of some single vector. Prove your friend is wrong in one sentence. (Hint: which point is in *every* span, and is it on this line?)

Exercise 3.8 (Counting argument)**.** Explain, in your own words and with a picture, why three vectors in $\mathbb{R}^2$ *must* be linearly dependent (at least one is redundant). Then exhibit explicit weights, not all zero, with $c_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + c_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + c_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, demonstrating the dependence concretely.

Exercise 3.9 (Same span, different generators)**.** Show that $\text{span}\left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\} = \text{span}\left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\}$ by proving each side contains the other. (To show "$\subseteq$," write a general vector of one span as a combination of the other set's vectors.) What does this say about the idea that a span has many possible "generating sets"?

Exercise 3.10 (Proof: span is closed under combination)**.** Let $S = \text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$. Suppose $\mathbf{p}$ and $\mathbf{q}$ are both in S. Prove that $\alpha\mathbf{p} + \beta\mathbf{q}$ is also in S for any scalars α, β. (Write $\mathbf{p}$ and $\mathbf{q}$ as combinations of the $\mathbf{v}_i$, then combine.) This "closed under linear combination" property is the defining feature of a *subspace* – you have just proved every span is one.

Exercise 3.11 (Challenge: build any vector two ways)**.** Let $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $\mathbf{v}_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. (a) Find two *different* sets of weights (c_1, c_2, c_3) that both produce $\begin{bmatrix} 3 \\ 2 \end{bmatrix}$. (b) Explain why having a redundant vector in the set is exactly what makes more than one set of weights possible. Connect this to why $\mathbf{Bx} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$ in Example 3.2 had infinitely many solutions.

Chapter 4

Dot Products, Length, and Angle

In Chapter 3, we explored how vectors combine: scaling them, adding them tip-to-tail, and tracing out the flat worlds of their span—lines through the origin, sheets in space, and hyperplanes. But notice what was missing. We could tell whether an arrow lived on a plane or spanned a whole space, yet we never once asked how *long* an arrow is or at what *angle* two arrows meet. Those are metric questions—questions of size and shape—and up to now our algebra has had no way to answer them. Addition and scaling alone know about directions and grid lines, but they are completely blind to length and angle.

This chapter introduces the one new operation that answers both questions: the **dot product**. It is a humble-looking gadget – multiply matching coordinates and add them up – but it carries the entire metric geometry of $\mathbb{R}^n$ inside it. Length is a dot product. Angle is a dot product. The word "perpendicular" becomes the single equation $\mathbf{u} \cdot \mathbf{v} = 0$. Once you see that the same small number gives the cosine of an angle, you hold the machinery behind projections, least squares, Fourier series, and the cosine similarity a search engine uses to judge whether two documents are about the same thing.

We will build it geometrically first – arrows, lengths, the law of cosines – then make it precise, then push it to the two great inequalities (Cauchy–Schwarz and the triangle inequality) that hold in *every* dimension. Slow down here. The dot product is small, but it is the hinge the rest of the book swings on.

4.1 The dot product, two ways

4.1.1 The algebraic definition

Here is the whole operation in one line. Given two vectors in $\mathbb{R}^n$ with the same number of entries,

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}, \qquad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix},$$

their **dot product** is the single number

$$\mathbf{u} \cdot \mathbf{v} \;=\; u_1v_1 + u_2v_2 + \cdots + u_nv_n \;=\; \sum_{i=1}^{n} u_iv_i.$$

Multiply matching entries, add up the results. The inputs are two vectors; the output is one *scalar* (a single real number, as opposed to a list of numbers). For that reason the dot product is sometimes called the *scalar product*. (You will also see it written $\langle \mathbf{u}, \mathbf{v} \rangle$, called the *inner product*; later, in Chapter 10, when we meet row vectors and matrix multiplication, we will also write this exact calculation as a row multiplied into a column. All these notations mean the same arithmetic.)

Example 4.1. Let $\mathbf{u} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$. Then

$$\mathbf{u} \cdot \mathbf{v} = (3)(2) + (4)(-1) = 6 - 4 = 2.$$

And in three dimensions, with $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}$,

$$\mathbf{a} \cdot \mathbf{b} = (1)(2) + (2)(0) + (2)(-1) = 2 + 0 - 2 = 0.$$

That zero reveals something fundamental: by the end of this chapter you will read it as "**a** and **b** are perpendicular."

Mental check. Take five seconds right now: what is $\begin{bmatrix} 2 \\ 5 \end{bmatrix} \cdot \begin{bmatrix} -5 \\ 2 \end{bmatrix}$ in your head? You get $(2)(-5) + (5)(2) = -10 + 10 = 0$. Swapping the two coordinates and flipping one sign always creates a perpendicular vector in $\mathbb{R}^2$ because $(a)(-b) + (b)(a) = 0$. Keep that reflex handy.

Focus

The dot product takes two vectors of the *same dimension* and returns one number. You can only dot $\mathbf{u} \cdot \mathbf{v}$ when $\mathbf{u}$ and $\mathbf{v}$ have the same number of components. Dotting a vector in $\mathbb{R}^2$ with a vector in $\mathbb{R}^3$ is meaningless – there is no rule that pairs up their entries.

A few properties fall straight out of the definition, and we will lean on all of them constantly.

Formal

For all vectors $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^n$ and every scalar $c \in \mathbb{R}$:

1. **Symmetry:** $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$.
2. **Linearity in each slot:** $(\mathbf{u} + \mathbf{w}) \cdot \mathbf{v} = \mathbf{u} \cdot \mathbf{v} + \mathbf{w} \cdot \mathbf{v}$ and $(c\,\mathbf{u}) \cdot \mathbf{v} = c\,(\mathbf{u} \cdot \mathbf{v})$.
3. **Positivity:** $\mathbf{u} \cdot \mathbf{u} = u_1^2 + \cdots + u_n^2 \geq 0$, with equality if and only if $\mathbf{u} = \mathbf{0}$.

Proof. Each is one line of algebra. Symmetry: $u_iv_i = v_iu_i$ term by term. Linearity: $(u_i + w_i)v_i = u_iv_i + w_iv_i$, then sum over i; and $(cu_i)v_i = c(u_iv_i)$. Positivity: $\mathbf{u} \cdot \mathbf{u} = \sum_i u_i^2$ is a sum of squares, hence ≥ 0, and a sum of squares is zero exactly when every term is zero, i.e. every $u_i = 0$. □

Property 3 is what links algebra to geometric distance. A sum of squares can never be negative, and it is zero only for the zero vector. That single fact is what makes $\mathbf{u} \cdot \mathbf{u}$ deserve to be called "length squared," which is where we go next.

4.1.2 Length lives inside the dot product

Look again at $\mathbf{u}\cdot\mathbf{u} = u_1^2 + \cdots + u_n^2$. In the plane, with $\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$, this is exactly $u_1^2 + u_2^2$ – the inside of the Pythagorean theorem. The hypotenuse of the right triangle with legs u_1 and u_2 is the length of the arrow $\mathbf{u}$. So:

$$\|\mathbf{u}\| \;=\; \sqrt{u_1^2 + u_2^2} \;=\; \sqrt{\mathbf{u}\cdot\mathbf{u}}.$$

Nothing about that derivation cared that we were in two dimensions. We simply *define* length in $\mathbb{R}^n$ to be the same formula, and the definition is forced on us the moment we want Pythagoras to keep holding.

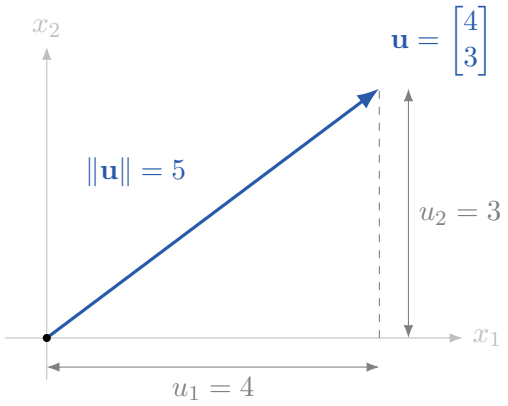


Figure 4.1: Length is Pythagoras: $\|\mathbf{u}\| = \sqrt{4^2 + 3^2} = \sqrt{25} = 5$. The dot product $\mathbf{u}\cdot\mathbf{u} = 16 + 9 = 25$ is the length *squared*.

Definition 4.1. The **length** (or **norm**, or **magnitude**) of $\mathbf{u} \in \mathbb{R}^n$ is

$$\|\mathbf{u}\| \;=\; \sqrt{\mathbf{u}\cdot\mathbf{u}} \;=\; \sqrt{u_1^2 + u_2^2 + \cdots + u_n^2}.$$

A vector with $\|\mathbf{u}\| = 1$ is called a **unit vector**.

Because $\mathbf{u}\cdot\mathbf{u} \geq 0$, that square root is always of a nonnegative number, so $\|\mathbf{u}\|$ is a well-defined real number ≥ 0, and it equals 0 only for $\mathbf{u} = \mathbf{0}$. Length behaves exactly as length should.

Example 4.2. $\left\| \begin{bmatrix} 3 \\ 4 \end{bmatrix} \right\| = \sqrt{9+16} = 5.$ $\left\| \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \right\| = \sqrt{1+4+4} = 3.$ $\left\| \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right\| = \sqrt{4} = 2.$ The last one is a useful sanity check: the all-ones vector in $\mathbb{R}^n$ has length $\sqrt{n}$, which grows without bound as the dimension does.

Common trap

The dot product $\mathbf{u}\cdot\mathbf{u}$ is length *squared*, not length. People constantly drop the square root, or, worse, take the square root of the *individual* coordinates. There is exactly one square root, and it goes *around the whole sum*: $\|\mathbf{u}\| = \sqrt{u_1^2 + \cdots + u_n^2}$, never $|u_1| + \cdots + |u_n|$. (That second thing is a different, also useful, way to measure size – but it is not the length we get from the

dot product.)

One small but heavily used fact: scaling a vector scales its length by the *absolute value* of the scalar. For any $c \in \mathbb{R}$,

$$\|c\,\mathbf{u}\| = \sqrt{(c\mathbf{u}) \cdot (c\mathbf{u})} = \sqrt{c^2\,(\mathbf{u} \cdot \mathbf{u})} = |c|\,\|\mathbf{u}\|.$$

Doubling an arrow doubles its length; negating it ($c = -1$) flips its direction but leaves its length unchanged.

4.2 The geometric meaning: length times length times cosine

We now have an *algebraic* number, $\mathbf{u} \cdot \mathbf{v}$, and we have used the special case $\mathbf{u} \cdot \mathbf{u}$ to recover length. But what does the general dot product of two *different* vectors mean geometrically? The answer is one of the most useful formulas in mathematics:

$$\boxed{\mathbf{u} \cdot \mathbf{v} \;=\; \|\mathbf{u}\|\,\|\mathbf{v}\|\,\cos\theta}$$

where θ is the angle between the two arrows. Here is why it holds: the identity is an algebraic restatement of the law of cosines.

Intuition

The dot product measures *how much two arrows point the same way*, weighted by how long they are. If they point in nearly the same direction, $\cos\theta$ is near $+1$ and the dot product is large and positive. If they are perpendicular, $\cos\theta = 0$ and the dot product is 0. If they point in opposite directions, $\cos\theta$ is near -1 and the dot product is large and negative. The sign of $\mathbf{u} \cdot \mathbf{v}$ alone tells you whether the angle between the arrows is acute, right, or obtuse (Figure 4.2).

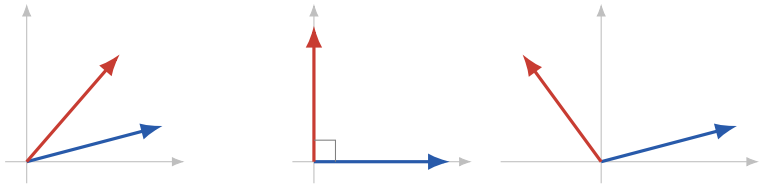


Figure 4.2: The sign of the dot product is the sign of $\cos\theta$. Arrows that lean the same way give a positive dot product (acute angle); perpendicular arrows give zero (right angle); arrows that lean apart give a negative dot product (obtuse angle).

4.2.1 Deriving the cosine formula from the law of cosines

Place both arrows tail-to-tail at the origin. The vector $\mathbf{u} - \mathbf{v}$ is the arrow that runs from the tip of $\mathbf{v}$ to the tip of $\mathbf{u}$ – it closes the triangle whose other two sides are $\mathbf{u}$ and $\mathbf{v}$ (Figure 4.3).

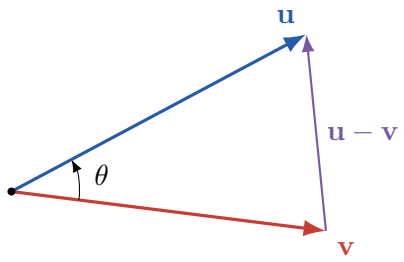


Figure 4.3: Two arrows tail-to-tail at the origin enclose an angle θ. The third side of the triangle is $\mathbf{u} - \mathbf{v}$, and the law of cosines relates its length to $\|\mathbf{u}\|$, $\|\mathbf{v}\|$, and $\cos\theta$.

The law of cosines from trigonometry says that in a triangle with side lengths a, b, c and angle θ opposite the side of length c,

$$c^2 = a^2 + b^2 - 2ab\cos\theta.$$

Apply it to our triangle, with $a = \|\mathbf{u}\|$, $b = \|\mathbf{v}\|$, and the opposite side $c = \|\mathbf{u} - \mathbf{v}\|$:

$$\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 - 2\,\|\mathbf{u}\|\,\|\mathbf{v}\|\cos\theta.$$

Now compute that left side with the dot product, using linearity and symmetry:

$$\begin{aligned}\|\mathbf{u} - \mathbf{v}\|^2 &= (\mathbf{u} - \mathbf{v}) \cdot (\mathbf{u} - \mathbf{v}) \\ &= \mathbf{u}\cdot\mathbf{u} - \mathbf{u}\cdot\mathbf{v} - \mathbf{v}\cdot\mathbf{u} + \mathbf{v}\cdot\mathbf{v} \\ &= \|\mathbf{u}\|^2 - 2\,(\mathbf{u}\cdot\mathbf{v}) + \|\mathbf{v}\|^2.\end{aligned}$$

Set the two expressions for $\|\mathbf{u} - \mathbf{v}\|^2$ equal. The $\|\mathbf{u}\|^2$ and $\|\mathbf{v}\|^2$ on both sides cancel, leaving

$$-2\,(\mathbf{u}\cdot\mathbf{v}) = -2\,\|\mathbf{u}\|\,\|\mathbf{v}\|\cos\theta,$$

and dividing by -2 gives precisely

$$\mathbf{u}\cdot\mathbf{v} = \|\mathbf{u}\|\,\|\mathbf{v}\|\,\cos\theta.$$

That is the whole derivation. The algebraic dot product and the geometric "length times length times cosine" are two faces of the same number, glued together by the law of cosines.

Why this matters

Pause on how surprising this is. The left-hand side, $\sum_i u_i v_i$, is pure arithmetic – it never mentions angles. The right-hand side is pure geometry – lengths and a cosine. The identity says these always agree. This is the bridge between coordinates and geometry, and it is exactly the bridge that lets a computer, which only ever knows lists of numbers, reason about angles, similarity, and perpendicularity it can never "see."

4.2.2 Solving for the angle

Turn the boxed formula around to get the angle from the coordinates. As long as neither vector is zero,

$$\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}, \qquad \theta = \arccos\left(\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}\right).$$

Example 4.3. Find the angle between $\mathbf{u} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ (the vectors from Example 4.1). We already found $\mathbf{u} \cdot \mathbf{v} = 2$. The lengths are

$$\|\mathbf{u}\| = \sqrt{9+16} = 5, \qquad \|\mathbf{v}\| = \sqrt{4+1} = \sqrt{5} \approx 2.236.$$

So

$$\cos\theta = \frac{2}{5\sqrt{5}} \approx 0.1789, \qquad \theta \approx \arccos(0.1789) \approx 79.7^\circ.$$

The dot product was positive, so we expected an acute angle, and 79.7° just barely qualifies – the arrows are nearly, but not quite, perpendicular.

Compute it yourself

Compute by hand, then sketch each pair as arrows and check the sign of the angle against the sign of the dot product.

1. $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and the angle between them. (Answer: dot $= 1$, angle $= 45^\circ$.)
2. $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ 2 \end{bmatrix}$. What is the angle? What does the value of the dot product tell you before you compute the angle?
3. $\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, then $\cos\theta$, then θ. (This is the angle between a face diagonal and the space diagonal of a cube.)

4.3 Orthogonality: perpendicular means dot product zero

The cosine formula hands us the cleanest definition in linear algebra. Two nonzero arrows are perpendicular exactly when the angle between them is 90°, i.e. $\cos\theta = 0$. Since $\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$ and the lengths are nonzero, the dot product is zero precisely when $\cos\theta = 0$:

$$\mathbf{u} \perp \mathbf{v} \quad \Longleftrightarrow \quad \mathbf{u} \cdot \mathbf{v} = 0.$$

Definition 4.2. Two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ are **orthogonal** (the fancy word for perpendicular) when $\mathbf{u} \cdot \mathbf{v} = 0$. By this definition the zero vector is orthogonal to *every* vector, since $\mathbf{0} \cdot \mathbf{v} = 0$ always.

We trade the geometric word "perpendicular" for the algebraic test "dot product $= 0$," and the trade is a fantastic deal: the algebraic test works in any number of dimensions, where we can no longer draw the right angle but the equation still makes perfect sense. Two vectors in $\mathbb{R}^{100}$ can be orthogonal; you will never picture it, but you can always check it.

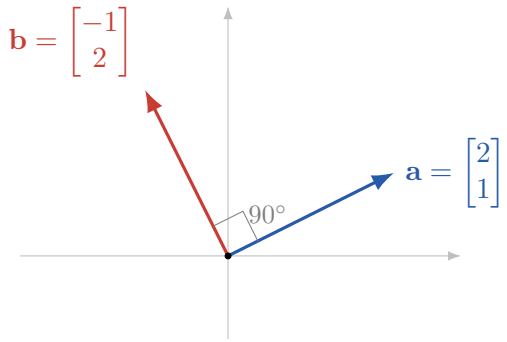


Figure 4.4: $\mathbf{a} \cdot \mathbf{b} = (2)(-1) + (1)(2) = 0$, so the arrows meet at a right angle. Orthogonality is just a zero dot product.

Example 4.4. Back in Example 4.1 we found $\begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix} = 0$. Those two three-dimensional arrows are orthogonal: they meet at a perfect right angle in space, and even though a flat sketch on paper might not make that 90° angle obvious, the dot product confirms it in one line.

Before/after Strang

Before Strang: when he opens his lectures by saying "perpendicular means the dot product is zero," you will already own that sentence – you will have *derived* it from the cosine formula rather than been told to memorize it. *After Strang:* this one idea, "$\mathbf{u} \cdot \mathbf{v} = 0$," is the seed of orthogonal subspaces, the four fundamental subspaces meeting at right angles, orthogonal matrices, projections, and the Gram–Schmidt process. Whenever Strang draws two subspaces crossing at 90°, the dot product of this chapter is doing the actual work underneath.

4.3.1 The Pythagorean theorem, reborn

Orthogonality gives the cleanest possible version of Pythagoras. If $\mathbf{u} \cdot \mathbf{v} = 0$, expand the squared length of the sum:

$$\|\mathbf{u} + \mathbf{v}\|^2 = (\mathbf{u} + \mathbf{v}) \cdot (\mathbf{u} + \mathbf{v}) = \|\mathbf{u}\|^2 + 2\,(\mathbf{u} \cdot \mathbf{v}) + \|\mathbf{v}\|^2.$$

The middle term $2(\mathbf{u} \cdot \mathbf{v})$ vanishes precisely when the vectors are orthogonal, leaving

$$\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 \qquad \text{exactly when } \mathbf{u} \perp \mathbf{v}.$$

This is the Pythagorean theorem, stated for vectors, derived in two lines, and valid in every dimension. The converse holds too: if $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$, then the cross term must be zero, so $\mathbf{u} \cdot \mathbf{v} = 0$ and the vectors are orthogonal. Pythagoras and orthogonality are the *same* statement.

Mental check. If $\mathbf{u}$ and $\mathbf{v}$ are orthogonal, how does the length of $\mathbf{u} + \mathbf{v}$ compare to the length of $\mathbf{u} - \mathbf{v}$? In your head: $\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 - 2(\mathbf{u} \cdot \mathbf{v}) + \|\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$. The cross term vanishes again, so $\|\mathbf{u} + \mathbf{v}\| = \|\mathbf{u} - \mathbf{v}\|$. Geometrically, the two diagonals of a rectangle have the exact same length. When the vectors are not perpendicular, the parallelogram tilts, making one diagonal longer than the other.

4.4 Unit vectors and direction

In the formula $\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta$, the number mixes two separate geometric ideas: how long the arrows are, and what angle they form. Often we care only about the *direction* an arrow points, not its length. To strip out the length, divide a nonzero vector by its own norm:

$$\hat{\mathbf{u}} \;=\; \frac{\mathbf{u}}{\|\mathbf{u}\|}.$$

This is called **normalizing u**, and the result $\hat{\mathbf{u}}$ (read "**u** hat") is a unit vector pointing the same way as **u**, because

$$\|\hat{\mathbf{u}}\| = \left\| \frac{1}{\|\mathbf{u}\|}\mathbf{u} \right\| = \frac{1}{\|\mathbf{u}\|}\,\|\mathbf{u}\| = 1,$$

using the scaling rule $\|c\mathbf{u}\| = |c| \, \|\mathbf{u}\|$ with the positive scalar $c = 1/\|\mathbf{u}\|$.

Example 4.5. Normalize $\mathbf{u} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$. Its length is 5, so $\hat{\mathbf{u}} = \frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$, and indeed $0.6^2 + 0.8^2 = 0.36 + 0.64 = 1$. ✓

Mental check. If **u** has length 10, what is the length of $\frac{1}{10}\mathbf{u}$? It is 1. What is the length of $-3\hat{\mathbf{u}}$? By the scaling rule, $|-3|\|\hat{\mathbf{u}}\| = 3$. If you double **u**, what happens to its normalized version $\hat{\mathbf{u}}$? Absolutely nothing: doubling the arrow doubles both the numerator and its length in the denominator, returning the exact same unit arrow.

Unit vectors make the cosine formula especially clean. If $\hat{\mathbf{u}}$ and $\hat{\mathbf{v}}$ are both unit vectors, then $\|\hat{\mathbf{u}}\| = \|\hat{\mathbf{v}}\| = 1$ and the formula collapses to

$$\hat{\mathbf{u}} \cdot \hat{\mathbf{v}} = \cos\theta.$$

For unit vectors, *the dot product simply is the cosine of the angle between them* (Figure 4.5). Hold onto that – it is the entire idea behind cosine similarity at the end of the chapter.

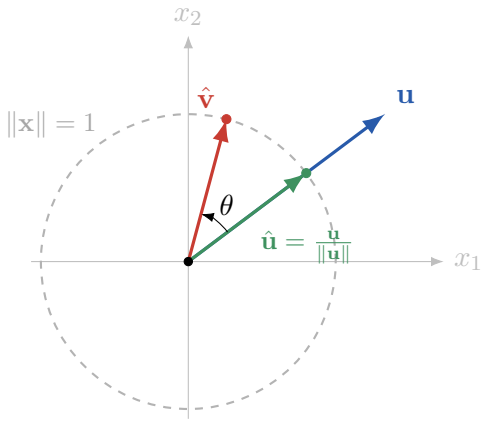


Figure 4.5: Normalizing pulls any nonzero arrow **u** back onto the unit circle $\|\mathbf{x}\| = 1$ along its own line. When two vectors already have unit length, $\|\hat{\mathbf{u}}\| = \|\hat{\mathbf{v}}\| = 1$, their dot product simplifies directly to the cosine of the angle between them: $\hat{\mathbf{u}} \cdot \hat{\mathbf{v}} = \cos\theta$.

Focus

Two complementary readings of the dot product:

- $\mathbf{u} \cdot \mathbf{u} = \|\mathbf{u}\|^2$ (length squared), and
- $\hat{\mathbf{u}} \cdot \hat{\mathbf{v}} = \cos\theta$ (cosine of the angle), once you have normalized.

Length and angle, the two pieces of metric geometry, both come straight out of this one operation.

4.5 A first look at projection

Here is a preview of one of the most important uses of the dot product; a whole later chapter is devoted to it, but the seed belongs here. Ask: *how much of* $\mathbf{u}$ *points along the direction of* $\mathbf{v}$*?*

Picture shining a light straight down onto the line through $\mathbf{v}$ and reading off the shadow $\mathbf{u}$ casts on that line (Figure 4.6). That shadow is the **projection** of $\mathbf{u}$ onto $\mathbf{v}$.

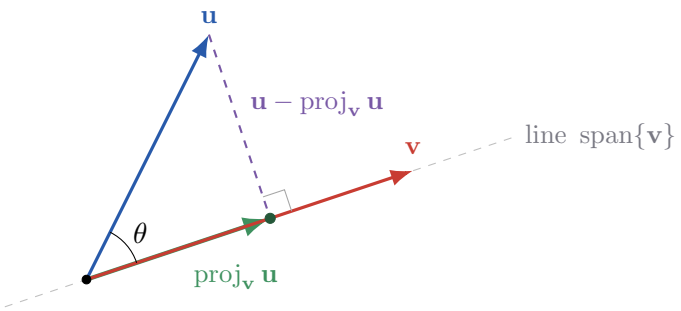


Figure 4.6: The projection $\text{proj}_{\mathbf{v}}\,\mathbf{u}$ is the shadow $\mathbf{u}$ casts onto the line through $\mathbf{v}$. The dropped perpendicular (dashed purple) is the error $\mathbf{u} - \text{proj}_{\mathbf{v}}\,\mathbf{u}$, which meets the line at a right angle. The signed length of the shadow is $\|\mathbf{u}\|\cos\theta$.

From the right triangle in the picture, the (signed) length of that shadow is the adjacent side, $\|\mathbf{u}\|\cos\theta$. But $\|\mathbf{u}\|\cos\theta = \dfrac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{v}\|}$ straight from the cosine formula. So the length of the shadow is

$$\|\mathbf{u}\|\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{v}\|},$$

and the shadow as an actual vector – this length, pointed along the unit direction vector $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$ – is

$$\text{proj}_{\mathbf{v}}\,\mathbf{u} \;=\; \left(\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{v}\|}\right)\hat{\mathbf{v}} \;=\; \left(\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{v}\|}\right)\frac{\mathbf{v}}{\|\mathbf{v}\|} \;=\; \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{v}\|^2}\,\mathbf{v} \;=\; \frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\,\mathbf{v}.$$

Example 4.6. Project $\mathbf{u} = \begin{bmatrix}1\\3\end{bmatrix}$ onto $\mathbf{v} = \begin{bmatrix}2\\0\end{bmatrix}$. Here $\mathbf{u}\cdot\mathbf{v} = 2$ and $\mathbf{v}\cdot\mathbf{v} = 4$, so

$$\text{proj}_{\mathbf{v}}\,\mathbf{u} = \tfrac{2}{4}\begin{bmatrix}2\\0\end{bmatrix} = \tfrac{1}{2}\begin{bmatrix}2\\0\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix}.$$

That is exactly right: the shadow of $\begin{bmatrix}1\\3\end{bmatrix}$ on the horizontal axis is $\begin{bmatrix}1\\0\end{bmatrix}$ – it keeps the horizontal part and drops the vertical part, just as the picture promised.

Mental check. What is the projection of an arrow $\mathbf{u}$ onto itself? Using the formula, $\text{proj}_{\mathbf{u}}\,\mathbf{u} = \frac{\mathbf{u}\cdot\mathbf{u}}{\mathbf{u}\cdot\mathbf{u}}\mathbf{u} = 1\,\mathbf{u} = \mathbf{u}$. The shadow of an arrow along its own line is the entire arrow.

Intuition

Every appearance of the fraction $\dfrac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}$ later in this book – in least squares, in regression, in Gram–Schmidt – is this same shadow. The dot product on top measures the overlap of $\mathbf{u}$ with $\mathbf{v}$; the dot product on the bottom rescales by the length of $\mathbf{v}$ so that the answer does not change if you happen to write $\mathbf{v}$ longer or shorter. Projection is the dot product earning its keep.

4.6 Two great inequalities

We have been writing $\cos\theta = \dfrac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$ as if it is automatically well-defined. But a cosine must live between -1 and 1. For the formula to hold, the fraction on the right had better never escape $[-1, 1]$ – that is, we need $|\mathbf{u}\cdot\mathbf{v}| \le \|\mathbf{u}\|\|\mathbf{v}\|$. This is the celebrated **Cauchy–Schwarz inequality**, and it holds in every dimension, with pure algebra, no pictures required.

4.6.1 Cauchy–Schwarz

Before plunging into the algebra, look back at the right triangle in Figure 4.6. The hypotenuse has length $\|\mathbf{u}\|$, and the shadow along the line of $\mathbf{v}$ has length $\|\text{proj}_{\mathbf{v}}\,\mathbf{u}\| = \frac{|\mathbf{u}\cdot\mathbf{v}|}{\|\mathbf{v}\|}$. In any right triangle, a leg is never longer than the hypotenuse:

$$\frac{|\mathbf{u}\cdot\mathbf{v}|}{\|\mathbf{v}\|} \;\le\; \|\mathbf{u}\|.$$

Multiplying both sides by $\|\mathbf{v}\|$ gives $|\mathbf{u}\cdot\mathbf{v}| \le \|\mathbf{u}\|\,\|\mathbf{v}\|$ at once. A shadow can never be longer than the stick that cast it. When can the shadow equal the stick? Only when the arrow lies completely flat along the line, so that $\mathbf{u}$ and $\mathbf{v}$ are parallel.

That geometric picture is completely convincing in two and three dimensions. The algebraic proof below guarantees that this same bound holds in 100 dimensions, or in abstract vector spaces where triangles cannot be drawn.

Theorem 4.1 (Cauchy–Schwarz)**.** *For all $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$,*

$$|\mathbf{u}\cdot\mathbf{v}| \;\le\; \|\mathbf{u}\|\,\|\mathbf{v}\|.$$

Equality holds if and only if $\mathbf{u}$ and $\mathbf{v}$ are parallel (one is a scalar multiple of the other, including the case where one is $\mathbf{0}$).

Proof. If $\mathbf{v} = \mathbf{0}$ both sides are zero and there is nothing to prove, so assume $\mathbf{v} \neq \mathbf{0}$. The trick is to use positivity (property 3): the squared length of *any* vector is ≥ 0. Consider, for a real number t, the vector $\mathbf{u} - t\,\mathbf{v}$, and expand its squared length:

$$0 \;\le\; \|\mathbf{u} - t\mathbf{v}\|^2 = (\mathbf{u} - t\mathbf{v})\cdot(\mathbf{u} - t\mathbf{v}) = \|\mathbf{u}\|^2 - 2t\,(\mathbf{u}\cdot\mathbf{v}) + t^2\,\|\mathbf{v}\|^2.$$

Read the right-hand side as a quadratic function of t: $q(t) = (\|\mathbf{v}\|^2)t^2 - 2(\mathbf{u}\cdot\mathbf{v})t + \|\mathbf{u}\|^2$. Because the squared length is nonnegative for *every* real number t, $q(t) \geq 0$ always. The graph of $q(t)$ is a parabola opening upward that either touches the horizontal axis once or hovers entirely above it. A parabola that never dips below the axis cannot have two distinct real roots, which forces its discriminant $\Delta = b^2 - 4ac$ to be ≤ 0:

$$\big(-2(\mathbf{u}\cdot\mathbf{v})\big)^2 - 4\,\|\mathbf{v}\|^2\,\|\mathbf{u}\|^2 \leq 0.$$

That is $4(\mathbf{u}\cdot\mathbf{v})^2 \leq 4\,\|\mathbf{u}\|^2\|\mathbf{v}\|^2$. Divide by 4 and take square roots:

$$|\mathbf{u}\cdot\mathbf{v}| \leq \|\mathbf{u}\|\,\|\mathbf{v}\|.$$

Equality means the discriminant is exactly zero, so the parabola touches the axis at a single root t_0, giving $\|\mathbf{u} - t_0\mathbf{v}\|^2 = 0$. By positivity, this forces $\mathbf{u} - t_0\mathbf{v} = \mathbf{0}$, so $\mathbf{u} = t_0\mathbf{v}$: the vectors are parallel. □

Why this matters

That proof used *only* the three properties in our Formal box – symmetry, linearity, positivity. It never once peeked at coordinates or drew an angle. So the inequality holds not just in $\mathbb{R}^n$ but in *any* setting with a dot-product-like operation obeying those three rules: spaces of functions, spaces of random variables, infinite-dimensional spaces. The same five lines prove that the correlation coefficient in statistics lies in $[-1, 1]$ and that overlap integrals in quantum mechanics are bounded. One little quadratic, enormous reach.

Reading note

A theorem with three names. Augustin-Louis Cauchy proved the algebraic version for finite lists of numbers in 1821; Viktor Bunyakovsky extended it to integrals in 1859; and Hermann Schwarz rediscovered the integral version in 1888 while studying minimal surfaces (soap-film shapes). In many textbooks it is called the Cauchy–Bunyakovsky–Schwarz (CBS) inequality.

With Cauchy–Schwarz in hand, the angle formula is finally legitimate: the fraction $\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$ genuinely lies in $[-1, 1]$, so there really is an angle $\theta \in [0, \pi]$ with that cosine. *This is why we are allowed to define angle by the dot product even in dimensions we cannot picture.*

4.6.2 The triangle inequality

The second great inequality is the one your intuition already trusts: the straight path is the shortest, so going from a tail to a tip directly is no longer than detouring. In vector language, $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$.

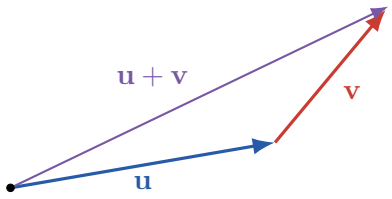


Figure 4.7: The direct trip $\mathbf{u}+\mathbf{v}$ is never longer than the detour $\|\mathbf{u}\| + \|\mathbf{v}\|$ through the corner. That is the triangle inequality.

Theorem 4.2 (Triangle inequality)**.** *For all* $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$*,*

$$\|\mathbf{u}+\mathbf{v}\| \;\le\; \|\mathbf{u}\| + \|\mathbf{v}\|.$$

Proof. Square the left side and expand:

$$\|\mathbf{u}+\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + 2\,(\mathbf{u}\cdot\mathbf{v}) + \|\mathbf{v}\|^2.$$

Now bound the cross term. Since $\mathbf{u}\cdot\mathbf{v} \le |\mathbf{u}\cdot\mathbf{v}|$, and by Cauchy–Schwarz $|\mathbf{u}\cdot\mathbf{v}| \le \|\mathbf{u}\|\|\mathbf{v}\|$,

$$\|\mathbf{u}+\mathbf{v}\|^2 \le \|\mathbf{u}\|^2 + 2\,\|\mathbf{u}\|\|\mathbf{v}\| + \|\mathbf{v}\|^2 = \big(\|\mathbf{u}\| + \|\mathbf{v}\|\big)^2.$$

Both sides are nonnegative, so taking square roots preserves the inequality: $\|\mathbf{u}+\mathbf{v}\| \le \|\mathbf{u}\|+\|\mathbf{v}\|$. □

Notice that the triangle inequality follows directly from Cauchy–Schwarz – the cross term is exactly the dot product, and Cauchy–Schwarz is precisely the tool that bounds it. The two inequalities are not separate facts; the second is a corollary of the first.

Walking question

Take a walk and turn this over: equality in the triangle inequality, $\|\mathbf{u}+\mathbf{v}\| = \|\mathbf{u}\|+\|\mathbf{v}\|$, means the detour is no longer than the direct route – the "corner" has flattened out. Geometrically, when must that happen? Trace the proof: equality forces equality in Cauchy–Schwarz, which forces $\mathbf{u}$ and $\mathbf{v}$ parallel, *and* forces $\mathbf{u}\cdot\mathbf{v} \ge 0$, so they point the *same* way, not opposite ways. Does that match your picture of when two arrows lie perfectly end to end along a single line?

Connection to other math

The two properties $\|\mathbf{u}\| \ge 0$ (with equality only at $\mathbf{0}$) and $\|\mathbf{u}+\mathbf{v}\| \le \|\mathbf{u}\|+\|\mathbf{v}\|$, together with $\|c\mathbf{u}\| = |c|\|\mathbf{u}\|$, are exactly the axioms a mathematician demands of any "length" – they define what is called a *norm*, and a norm in turn defines a notion of *distance*, $d(\mathbf{u},\mathbf{v}) = \|\mathbf{u}-\mathbf{v}\|$, satisfying $d(\mathbf{u},\mathbf{w}) \le d(\mathbf{u},\mathbf{v}) + d(\mathbf{v},\mathbf{w})$. This is the doorway from linear algebra into *metric spaces, analysis, and topology*: the moment you can measure distance and trust the triangle inequality, you can talk about limits, continuity, and convergence. Our humble dot product is where all of that begins.

4.7 Modern application: cosine similarity

Why does any of this matter to a search engine, a recommender, or a large language model? Because those systems turn things – documents, images, words – into vectors, and then must decide which vectors are *similar*. The dot product, normalized, is the standard measure of that similarity, and it works for exactly the reason we just proved: the normalized dot product is a cosine, so it lives in $[-1, 1]$ and reads as "how much do these point the same way."

Modern application

Cosine similarity. Represent a document as a vector of word counts. Suppose our vocabulary is just {`linear`, `algebra`, `cooking`} and

$$\mathbf{d}_1 = \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix} \text{ (a math note)}, \quad \mathbf{d}_2 = \begin{bmatrix} 8 \\ 6 \\ 0 \end{bmatrix} \text{ (a longer math note)}, \quad \mathbf{d}_3 = \begin{bmatrix} 0 \\ 1 \\ 5 \end{bmatrix} \text{ (a recipe)}.$$

The **cosine similarity** of two documents is the cosine of the angle between their vectors,

$$\cos\theta = \frac{\mathbf{d}_i \cdot \mathbf{d}_j}{\|\mathbf{d}_i\| \, \|\mathbf{d}_j\|}.$$

Compare the two math notes:

$$\mathbf{d}_1 \cdot \mathbf{d}_2 = 32 + 18 + 0 = 50, \quad \|\mathbf{d}_1\| = 5, \quad \|\mathbf{d}_2\| = 10, \quad \cos\theta = \frac{50}{50} = 1.$$

A perfect score of 1: even though $\mathbf{d}_2$ has twice the word counts (it is simply a longer document), it points in exactly the same direction, so cosine similarity declares the two notes identical in *topic*. That length-insensitivity is desired here – we do not want a document to look different just because it is longer. Now compare the math note to the recipe:

$$\mathbf{d}_1 \cdot \mathbf{d}_3 = 0 + 3 + 0 = 3, \quad \|\mathbf{d}_3\| = \sqrt{26} \approx 5.10, \quad \cos\theta = \frac{3}{5 \cdot 5.10} \approx 0.118.$$

Nearly orthogonal – a similarity near zero – which is exactly what we want: the recipe is about something else. The same arithmetic, run over vectors with hundreds or thousands of coordinates (one per word, or per learned "embedding" dimension, where an embedding is a list of coordinates representing the meaning of a concept in machine learning), is how modern search, recommendation, and retrieval systems find the items closest in meaning to your query. Underneath the buzzwords, it is $\sum_i u_i v_i$ divided by two lengths.

To see why length-insensitivity is the right behaviour, drop the (here empty) `cooking` axis and plot the three document vectors in the `linear`–`algebra` plane (Figure 4.8). The two math notes $\mathbf{d}_1$ and $\mathbf{d}_2$ lie on the *same ray* from the origin – $\mathbf{d}_2$ is just $\mathbf{d}_1$ stretched – so the angle between them is 0 and the cosine is 1. The recipe $\mathbf{d}_3$ has almost all of its weight on the `cooking` axis (out of the page), so once that axis is restored the full angle to either math note is close to 90° and the cosine is close to 0.

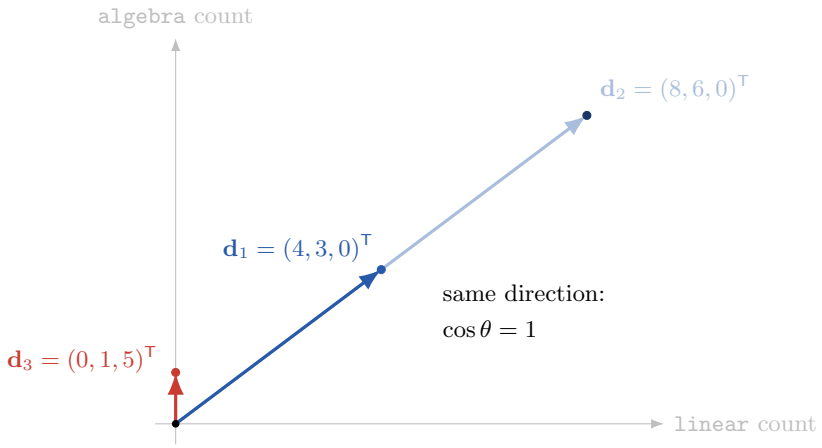


Figure 4.8: The three document vectors projected onto the `linear`–`algebra` plane (the `cooking` coordinate is the out-of-page third axis, where only $\mathbf{d}_3$ has weight). Cosine similarity reads *direction*, not size: $\mathbf{d}_1$ and $\mathbf{d}_2$ share a ray and score 1, while $\mathbf{d}_3$ lives mostly along the out-of-page `cooking` axis, so in full space it is nearly perpendicular to both and scores near 0.

Common trap

Cosine similarity ignores length *on purpose*. Sometimes that is wrong for your problem – if magnitude carries real information (total sales, signal energy, how many times a user clicked), then normalizing it away throws out the very thing you cared about. Choose deliberately: cosine similarity (angle only) versus a raw dot product or Euclidean distance (both of which depend on length). The dot product gives you both ingredients; *you* decide which to keep.

4.8 Worked summary example

Let us run one example through every idea in the chapter, end to end. Take

$$\mathbf{u} = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}, \qquad \mathbf{v} = \begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}.$$

Dot product. $\mathbf{u} \cdot \mathbf{v} = (2)(0) + (2)(3) + (1)(4) = 0 + 6 + 4 = 10$.

Lengths. $\|\mathbf{u}\| = \sqrt{4 + 4 + 1} = \sqrt{9} = 3, \quad \|\mathbf{v}\| = \sqrt{0 + 9 + 16} = \sqrt{25} = 5.$

Angle. $\cos\theta = \dfrac{10}{3 \cdot 5} = \dfrac{10}{15} = \dfrac{2}{3} \approx 0.667$, so $\theta = \arccos(0.667) \approx 48.2°$. Positive dot product, acute angle: consistent.

Cauchy–Schwarz check. $|\mathbf{u} \cdot \mathbf{v}| = 10$ and $\|\mathbf{u}\|\|\mathbf{v}\| = 15$, and indeed $10 \leq 15$. ✓

Triangle inequality check. $\mathbf{u} + \mathbf{v} = \begin{bmatrix} 2 \\ 5 \\ 5 \end{bmatrix}$, so $\|\mathbf{u} + \mathbf{v}\| = \sqrt{4 + 25 + 25} = \sqrt{54} \approx 7.35$, while $\|\mathbf{u}\| + \|\mathbf{v}\| = 3 + 5 = 8$. Indeed $7.35 \leq 8$. ✓

Unit vector. $\hat{\mathbf{u}} = \frac{1}{3}\begin{bmatrix}2\\2\\1\end{bmatrix} = \begin{bmatrix}2/3\\2/3\\1/3\end{bmatrix}$, and $(2/3)^2 + (2/3)^2 + (1/3)^2 = \frac{4+4+1}{9} = 1$. ✓

Projection of u onto v.

$$\text{proj}_{\mathbf{v}}\,\mathbf{u} = \frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\,\mathbf{v} = \frac{10}{25}\begin{bmatrix}0\\3\\4\end{bmatrix} = \tfrac{2}{5}\begin{bmatrix}0\\3\\4\end{bmatrix} = \begin{bmatrix}0\\6/5\\8/5\end{bmatrix}.$$

Sanity check: the shadow should be shorter than $\mathbf{u}$. Its length is $\frac{2}{5}\|\mathbf{v}\| = \frac{2}{5}\cdot 5 = 2$, which is indeed less than $\|\mathbf{u}\| = 3$. ✓Every tool in the chapter, one vector pair, all consistent.

Pause

If that last example felt like a lot, that is the right feeling – it deliberately touched everything. The thing to carry forward is small: **one operation**, $\mathbf{u}\cdot\mathbf{v} = \sum_i u_i v_i$. From it, length is $\sqrt{\mathbf{u}\cdot\mathbf{u}}$, angle is $\arccos\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$, perpendicular is $\mathbf{u}\cdot\mathbf{v} = 0$, and projection is $\frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\mathbf{v}$.
Cost. A dot product in $\mathbb{R}^n$ is n multiplications and $n-1$ additions – linear in the dimension, $O(n)$ work. Length costs one dot product plus a square root; an angle costs two more lengths and an arccos; a projection costs two dot products and a scaling. Everything in this chapter is cheap, which is exactly why cosine similarity scales to vectors with thousands of coordinates and to millions of documents.

History and connection

The dot product is the bridge where algebra meets geometry. For centuries, length and angle were considered the exclusive domain of Euclidean synthetic geometry—proving theorems with compass, straightedge, and classical trigonometry without coordinates. In 1821, Augustin-Louis Cauchy published his landmark textbook *Cours d'Analyse*, where he proved the algebraic inequality for finite sums: $(\sum a_i b_i)^2 \le (\sum a_i^2)(\sum b_i^2)$. Cauchy viewed this strictly as an arithmetic fact about lists of numbers. In 1859, the Ukrainian mathematician Viktor Bunyakovsky extended it to continuous integrals, and in 1888 Hermann Schwarz rediscovered the integral version while studying minimal surfaces (soap-film shapes that take the smallest possible surface area for a given boundary).
Meanwhile, the geometric meaning of the dot product was trapped inside Sir William Rowan Hamilton's 1843 quaternions—a four-dimensional number system extending complex numbers by packaging a scalar real part and a three-dimensional vector part into a single package. Hamilton's product of two spatial vectors (quaternions with zero scalar part) yielded:

$$\mathbf{uv} \;=\; -\mathbf{u}\cdot\mathbf{v} + \mathbf{u}\times\mathbf{v},$$

where $\mathbf{u}\times\mathbf{v}$ is the cross product (an arrow perpendicular to both $\mathbf{u}$ and $\mathbf{v}$). Notice the minus sign: in Hamilton's system, the scalar piece was inherently negative, which made computing geometric lengths awkward and required confusing sign corrections throughout mechanics.
In 1881, Josiah Willard Gibbs at Yale broke free from quaternion dogma. He split Hamilton's package in two, defining the dot product $\mathbf{u}\cdot\mathbf{v}$ as a positive-definite operation (strictly positive whenever $\mathbf{u} \neq \mathbf{0}$) equal to $\|\mathbf{u}\|\|\mathbf{v}\|\cos\theta$, while treating orthogonal projection $\text{proj}_{\mathbf{v}}\,\mathbf{u} = \frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\mathbf{v}$ as the geometric shadow cast by $\mathbf{u}$ onto the line through $\mathbf{v}$.
Gibbs' notation sparked the "Quaternion War" of the 1890s in the pages of *Nature*. Peter Guthrie Tait, Hamilton's disciple, attacked Gibbs' separate products as a "monstrosity," ar-

guing that discarding the unified quaternion product destroyed a precious division algebra (a rare number system where every nonzero element can be divided by). Gibbs and the British physicist Oliver Heaviside argued that mathematical tools should serve physics and geometry rather than rigid algebraic conventions. In particular, Gibbs and Heaviside showed that the dot product is universal: while quaternion multiplication is restricted to four dimensions, the dot product works directly in $\mathbb{R}^n$ for every n, and extends naturally to infinite-dimensional function spaces. This provided the geometric framework for Maxwell's electromagnetism, Einstein's relativity, and quantum mechanics.

Today, the dot product algebraically unifies three foundational geometric concepts: length ($\|\mathbf{v}\| = \sqrt{\mathbf{v} \cdot \mathbf{v}}$), angle ($\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$), and orthogonal projection ($\text{proj}_{\mathbf{v}}\,\mathbf{u} = \frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\mathbf{v}$). In high dimensions, geometry behaves unexpectedly. As dimension n climbs into the hundreds or thousands, almost every pair of randomly chosen vectors is nearly perpendicular—a counter-intuitive reality known as the *concentration of measure* (where nearly all the volume of a high-dimensional sphere gathers in a thin strip around its equator). How to arrange k unit vectors in $\mathbb{R}^n$ so that the largest dot product between any pair is as small as possible—the *spherical code* problem—remains unsolved in general dimensions. And in modern computing, searching high-dimensional vector embeddings by cosine similarity without comparing every pair is an active algorithmic challenge.

Finally, notice what the dot product does and does not do. It measures lengths and angles between given vectors, but it does not move space itself. To rotate, shear, stretch, or project an entire space at once, we need functions that take in vectors and return new ones—the subject of Chapter 5 (Section 5.1), where matrices act as linear transformations.

4.9 Exercises

Exercise 4.1. Compute each dot product and say whether the angle between the vectors is acute, right, or obtuse.

$$\text{(a)}\ \begin{bmatrix}1\\2\end{bmatrix}\cdot\begin{bmatrix}3\\1\end{bmatrix}, \qquad \text{(b)}\ \begin{bmatrix}2\\-3\end{bmatrix}\cdot\begin{bmatrix}3\\2\end{bmatrix}, \qquad \text{(c)}\ \begin{bmatrix}1\\1\\1\end{bmatrix}\cdot\begin{bmatrix}1\\1\\-3\end{bmatrix}.$$

Exercise 4.2. Find the length of each vector and then normalize it to a unit vector.

$$\text{(a)}\ \begin{bmatrix}6\\8\end{bmatrix}, \qquad \text{(b)}\ \begin{bmatrix}1\\-2\\2\end{bmatrix}, \qquad \text{(c)}\ \begin{bmatrix}1\\1\\1\\1\end{bmatrix}.$$

Exercise 4.3. Find the angle (in degrees, to one decimal place) between $\mathbf{u} = \begin{bmatrix}1\\0\\1\end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix}1\\1\\0\end{bmatrix}$. These are two face diagonals of a cube meeting at a corner; does your answer match the 60° you might guess from the geometry?

Exercise 4.4. For what value(s) of k are $\begin{bmatrix}2\\k\end{bmatrix}$ and $\begin{bmatrix}3\\-1\end{bmatrix}$ orthogonal? For what value(s) of k are they parallel?

Exercise 4.5. Project $\mathbf{u} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ onto $\mathbf{v} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, and then onto $\mathbf{w} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Add your two projection vectors. What do you get, and why does it make sense?

Exercise 4.6. Let $\mathbf{u} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$. Verify the Cauchy–Schwarz inequality and the triangle inequality numerically for this pair. Are either of them close to equality? Explain in one sentence what would have to be true for equality.

Exercise 4.7. Show that for any vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$,

$$\|\mathbf{u} + \mathbf{v}\|^2 + \|\mathbf{u} - \mathbf{v}\|^2 = 2\|\mathbf{u}\|^2 + 2\|\mathbf{v}\|^2.$$

(This is the *parallelogram law*.) Then draw a parallelogram with sides $\mathbf{u}$ and $\mathbf{v}$ and explain which lengths the four terms represent. *Hint:* expand both squared norms using the dot product and watch the cross terms.

Exercise 4.8. Prove that if $\mathbf{u}$ is orthogonal to both $\mathbf{v}$ and $\mathbf{w}$, then $\mathbf{u}$ is orthogonal to every linear combination $a\mathbf{v} + b\mathbf{w}$. *Hint:* use linearity of the dot product. (This little fact is the reason orthogonality plays so nicely with subspaces later on.)

Exercise 4.9. A unit vector in the plane can be written $\hat{\mathbf{u}} = \begin{bmatrix} \cos\alpha \\ \sin\alpha \end{bmatrix}$ and another as $\hat{\mathbf{v}} = \begin{bmatrix} \cos\beta \\ \sin\beta \end{bmatrix}$. Compute $\hat{\mathbf{u}} \cdot \hat{\mathbf{v}}$ directly from the coordinates, and use the fact that this equals $\cos\theta$ (with θ the angle between them) to re-derive the angle-subtraction formula $\cos(\alpha - \beta) = \cos\alpha\cos\beta + \sin\alpha\sin\beta$. A trig identity, falling out of the dot product.

Exercise 4.10. Three documents over the vocabulary {`cat`, `dog`, `stock`, `market`} are

$$\mathbf{a} = \begin{bmatrix} 3 \\ 2 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 6 \\ 4 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ 0 \\ 4 \\ 3 \end{bmatrix}.$$

Compute the cosine similarity of each pair. Which two documents are most similar, and why does $\mathbf{a}$ versus $\mathbf{b}$ score 1 even though their word counts differ?

Exercise 4.11 (Challenge)**.** Use Cauchy–Schwarz to prove that for any real numbers $a_1, \dots, a_n$,

$$(a_1 + a_2 + \cdots + a_n)^2 \;\le\; n\,(a_1^2 + a_2^2 + \cdots + a_n^2).$$

Hint: apply the inequality to $\mathbf{a} = \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix}$ and a very well-chosen second vector. When is it an equality? (This is one route to the fact that the average of squares is at least the square of the average.)

Exercise 4.12 (Challenge)**.** The angle formula lets us define angle in any dimension. Let $\mathbf{u}$, $\mathbf{v}$, $\mathbf{w}$ be three unit vectors and let $\theta_{uv}, \theta_{vw}, \theta_{uw}$ be the pairwise angles. Argue, using the triangle inequality applied to arcs on the unit sphere (or the cosine formula and a little trig), that $\theta_{uw} \le \theta_{uv} + \theta_{vw}$. Why is this the "triangle inequality for angles," and where have you seen its flat-space cousin in this chapter?

Chapter 5

Matrices as Machines: Linear Transformations

So far our vectors have been static arrows resting in space. You have added them, scaled them, combined them into spans, and used the dot product to measure their lengths, angles, and projections. But throughout all of that, the space itself sat still. Now we ask the question that turns linear algebra from static geometry into a living, moving world: *what if space itself can move? What if a matrix is not a static grid of numbers, but a* ***verb****?*

A matrix represents a transformation. You supply an input vector, and it produces an output vector. Apply it to $\begin{bmatrix}1\\0\end{bmatrix}$ and see where it lands. Apply it to $\begin{bmatrix}0\\1\end{bmatrix}$ and observe the result. Extending this across the entire plane, the transformation moves all of space at once: rotating, stretching, reflecting, or collapsing it. Everything else in this book (elimination, inverses, eigenvectors, the SVD) builds on understanding what these linear transformations do and how to decompose them.

This chapter builds that picture from the ground up. We start with the geometry of a single transformation, see why linearity preserves grids, and examine the basic motions of the plane: rotations, reflections, shears, and scalings.

5.1 A Matrix Is a Function on Vectors

Here is the main idea in one example. Take the matrix

$$\mathbf{A} = \begin{bmatrix}2 & 0\\0 & 3\end{bmatrix},$$

and feed it the vector $\mathbf{x} = \begin{bmatrix}1\\1\end{bmatrix}$. How do we feed a vector into a matrix? Each row of $\mathbf{A}$ takes a dot product with $\mathbf{x}$ – the very operation we mastered in the previous chapter:

$$\mathbf{Ax} = \begin{bmatrix}2 & 0\\0 & 3\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix} = \begin{bmatrix}\begin{bmatrix}2 & 0\end{bmatrix}\cdot\begin{bmatrix}1\\1\end{bmatrix}\\ \begin{bmatrix}0 & 3\end{bmatrix}\cdot\begin{bmatrix}1\\1\end{bmatrix}\end{bmatrix} = \begin{bmatrix}2\cdot 1 + 0\cdot 1\\ 0\cdot 1 + 3\cdot 1\end{bmatrix} = \begin{bmatrix}2\\3\end{bmatrix}.$$

Each row computes a dot product with $\mathbf{x}$ to produce one coordinate of the output. The matrix took the arrow pointing to $(1, 1)$ and returned the arrow pointing to $(2, 3)$: it doubled the horizontal coordinate and tripled the vertical coordinate. Applied to any other vector, it scales the coordinates in the same way. The matrix is a rule that assigns to *every* input vector $\mathbf{x}$ an output vector $\mathbf{Ax}$. That is precisely what a function is. In geometry, a function whose inputs and outputs are vectors is called a **transformation** – it transforms space by mapping every vector to an output vector.

Intuition

Think of $\mathbf{Ax}$ geometrically as **$\mathbf{A}$ acting on $\mathbf{x}$** – the transformation $\mathbf{A}$ mapping the point $\mathbf{x}$ to its destination $\mathbf{Ax}$. The matrix describes the overall transformation of space; the vector $\mathbf{x}$ is one point being mapped.

We will write this as a function and give it a name. An $m \times n$ matrix $\mathbf{A}$ defines a function

$$T : \mathbb{R}^n \to \mathbb{R}^m, \qquad T(\mathbf{x}) = \mathbf{Ax}.$$

It takes in vectors with n entries and produces vectors with m entries – the column count n is the dimension of the input space, while the row count m is the dimension of the output space. A 2×2 matrix maps the plane to the plane. A 3×2 matrix maps the plane into 3-dimensional space. A 1×3 matrix maps 3-space to a single real number (a line). The *dimensions* of the matrix tell you which spaces it connects before you compute a single entry.

Focus

$\mathbf{A}$ is $m \times n \implies$ input lives in $\mathbb{R}^n$ (the columns), output lives in $\mathbb{R}^m$ (the rows). Read the shape first: it tells you where vectors come from and where they go.

Example 5.1 (Three machines, one vector)**.** Let $\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and watch three different matrices act on it.

A flip across the vertical axis, $\mathbf{F} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$:

$$\mathbf{Fx} = \begin{bmatrix} -1 \cdot 2 + 0 \\ 0 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

The x-coordinate changed sign; the point hopped to the other side of the y-axis.

A horizontal shear, $\mathbf{S} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$:

$$\mathbf{Sx} = \begin{bmatrix} 1 \cdot 2 + 1 \cdot 1 \\ 0 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

The height 1 slid the point to the right by 1.

A collapse onto the x-axis, $\mathbf{P} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$:

$$\mathbf{Px} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

The vertical part was erased; every point now lives on the horizontal line. The same input vector lands in three different places depending on which matrix acts on it.

5.1.1 The picture: dots moving to dots

The cleanest way to *see* a transformation is to draw a few input points and draw arrows to where they land. Figure 5.1 shows the doubling machine $\mathbf{A} = \mathrm{diag}(2, 3)$ acting on a small cloud of points.

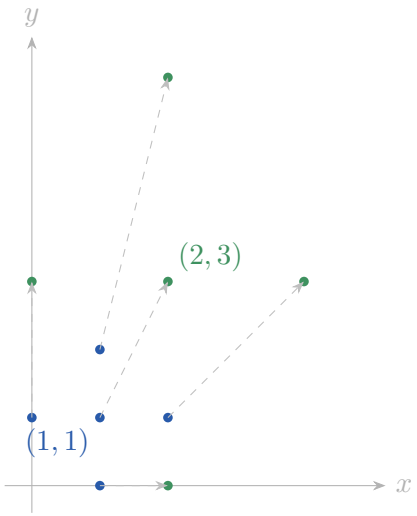


Figure 5.1: The machine $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ acting on a handful of points. Blue dots are inputs, green dots are outputs, and each dashed arrow shows where a point is dragged. Horizontal distances double, vertical distances triple.

A cloud of dots tells the truth but it gets cluttered fast. The trick that makes transformations *readable* is to draw a whole **grid** and watch what the machine does to it. Because (as we will prove in a moment) the machine moves straight lines to straight lines and evenly spaced lines to evenly spaced lines, the image of a square grid is again a grid – but possibly tilted, stretched, or sheared. We will lean on this picture constantly.

5.2 Linearity: The One Property That Makes a Matrix a Matrix

Not every function on vectors comes from a matrix. The function that adds $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ to everything, or the function that maps $\mathbf{x}$ to the vector of squared entries, are perfectly good functions – but they are not matrices. The functions that *are* matrices are exactly the ones with a special, beautiful property called **linearity**.

Definition 5.1 (Linear transformation)**.** A function $T : \mathbb{R}^n \to \mathbb{R}^m$ is **linear** if for all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and all scalars $c \in \mathbb{R}$:

$$\begin{aligned} &\textbf{(additivity)} && T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v}), \\ &\textbf{(homogeneity)} && T(c\,\mathbf{u}) = c\,T(\mathbf{u}). \end{aligned}$$

Read these geometrically. *Additivity* says: if you build $\mathbf{u} + \mathbf{v}$ by the parallelogram rule and *then* transform, you get the same arrow as transforming $\mathbf{u}$ and $\mathbf{v}$ separately and *then* adding. The machine

doesn't care whether you combine before or after (Figure 5.2). *Homogeneity* says: stretching the input by c stretches the output by the same c. Double the input arrow, double the output arrow. The machine respects scaling.

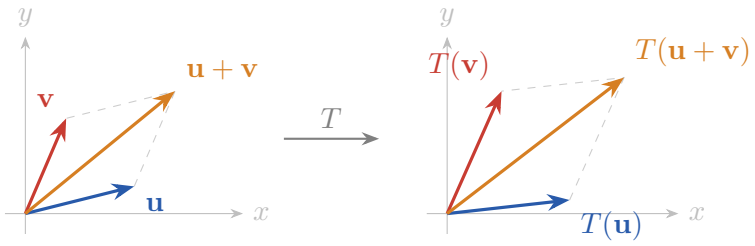


Figure 5.2: The geometric meaning of additivity: the parallelogram rule survives the transformation. Adding $\mathbf{u}$ and $\mathbf{v}$ first and then applying T reaches the exact same arrow as transforming $\mathbf{u}$ and $\mathbf{v}$ individually and adding their images: $T(\mathbf{u}+\mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$.

Intuition

Linearity is the statement that **the grid stays a grid**. Straight, evenly spaced lines go to straight, evenly spaced lines, and the origin stays put. No bending, no curving, no off-center shift. If you can picture a sheet of graph paper being stretched and sheared but never crumpled, you are picturing a linear transformation.

The two rules combine into one clean statement worth memorizing: a linear map sends linear combinations to linear combinations,

$$T(c\,\mathbf{u} + d\,\mathbf{v}) = c\,T(\mathbf{u}) + d\,T(\mathbf{v}).$$

This is the heart of everything. It says T *commutes* with the basic operations of linear algebra – adding and scaling – which are the only operations we have. To say two operations **commute** means doing them in either order yields the identical result: you can combine first and then transform, or transform first and then combine. A linear map is a function that preserves vector addition and scalar multiplication.

Common trap

"Linear" here does *not* mean "graph is a line." The high-school line $y = mx + b$ is **not** a linear map when $b \neq 0$, because it fails homogeneity: $T(0) = b \neq 0$, but every linear map must send $\mathbf{0}$ to $\mathbf{0}$ (set $c = 0$ in homogeneity). An **affine map** is a linear transformation followed by a rigid shift (a translation): it keeps straight lines straight, but it drags the origin away from $(0, 0)$. Linear maps always pin the origin in place.

Example 5.2 (Checking linearity directly)**.** Is rotation by 90° counterclockwise a linear map? Call it R. Geometrically R takes $\begin{bmatrix} a \\ b \end{bmatrix}$ to $\begin{bmatrix} -b \\ a \end{bmatrix}$ (we will derive this carefully in Section 5.5.2). Check additivity:

$$R\left(\begin{bmatrix} a_1 \\ b_1 \end{bmatrix} + \begin{bmatrix} a_2 \\ b_2 \end{bmatrix}\right) = R\begin{bmatrix} a_1 + a_2 \\ b_1 + b_2 \end{bmatrix} = \begin{bmatrix} -(b_1 + b_2) \\ a_1 + a_2 \end{bmatrix} = \begin{bmatrix} -b_1 \\ a_1 \end{bmatrix} + \begin{bmatrix} -b_2 \\ a_2 \end{bmatrix} = R\begin{bmatrix} a_1 \\ b_1 \end{bmatrix} + R\begin{bmatrix} a_2 \\ b_2 \end{bmatrix}. \checkmark$$

Check homogeneity:

$$R\left(c\begin{bmatrix}a\\b\end{bmatrix}\right) = R\begin{bmatrix}ca\\cb\end{bmatrix} = \begin{bmatrix}-cb\\ca\end{bmatrix} = c\begin{bmatrix}-b\\a\end{bmatrix} = c\,R\begin{bmatrix}a\\b\end{bmatrix}. \checkmark$$

Both hold, so rotation is linear. Geometrically this is obvious: spinning the whole plane rigidly about the origin keeps the grid a grid.

Example 5.3 (A function that is *not* linear)**.** Let $T\begin{bmatrix}a\\b\end{bmatrix} = \begin{bmatrix}a^2\\b\end{bmatrix}$. Try homogeneity with $c = 2$ and $\mathbf{u} = \begin{bmatrix}1\\0\end{bmatrix}$:

$$T(2\mathbf{u}) = T\begin{bmatrix}2\\0\end{bmatrix} = \begin{bmatrix}4\\0\end{bmatrix}, \qquad \text{but} \qquad 2\,T(\mathbf{u}) = 2\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}2\\0\end{bmatrix}.$$

Since $\begin{bmatrix}4\\0\end{bmatrix} \neq \begin{bmatrix}2\\0\end{bmatrix}$, the map fails homogeneity. Squaring bends the grid, so T is not linear and cannot be written as a matrix.

5.2.1 Why matrices are automatically linear

Every matrix gives a linear map, and this property falls right out of how matrix–vector products work. Two facts about the product $\mathbf{Ax}$ – which we can take as the defining algebra of matrices – are

$$\mathbf{A}(\mathbf{u}+\mathbf{v}) = \mathbf{Au} + \mathbf{Av} \qquad \text{and} \qquad \mathbf{A}(c\,\mathbf{u}) = c\,(\mathbf{Au}).$$

Compare with Definition 5.1: these are *exactly* additivity and homogeneity. So $T(\mathbf{x}) = \mathbf{Ax}$ is always linear. Let us verify the first one by direct computation in 2×2, so you trust the machinery. Write $\mathbf{A} = \begin{bmatrix}a & b\\c & d\end{bmatrix}$, $\mathbf{u} = \begin{bmatrix}u_1\\u_2\end{bmatrix}$, $\mathbf{v} = \begin{bmatrix}v_1\\v_2\end{bmatrix}$:

$$\mathbf{A}(\mathbf{u}+\mathbf{v}) = \begin{bmatrix}a & b\\c & d\end{bmatrix}\begin{bmatrix}u_1+v_1\\u_2+v_2\end{bmatrix} = \begin{bmatrix}a(u_1+v_1)+b(u_2+v_2)\\c(u_1+v_1)+d(u_2+v_2)\end{bmatrix}.$$

Distribute and regroup by $\mathbf{u}$ and $\mathbf{v}$:

$$= \begin{bmatrix}(au_1+bu_2)+(av_1+bv_2)\\(cu_1+du_2)+(cv_1+dv_2)\end{bmatrix} = \begin{bmatrix}au_1+bu_2\\cu_1+du_2\end{bmatrix} + \begin{bmatrix}av_1+bv_2\\cv_1+dv_2\end{bmatrix} = \mathbf{Au}+\mathbf{Av}. \checkmark$$

The only thing we used was the distributive law for ordinary numbers. Homogeneity follows just as quickly by pulling the scalar out of each term:

$$\mathbf{A}(c\,\mathbf{u}) = \begin{bmatrix}a & b\\c & d\end{bmatrix}\begin{bmatrix}cu_1\\cu_2\end{bmatrix} = \begin{bmatrix}a(cu_1)+b(cu_2)\\c(cu_1)+d(cu_2)\end{bmatrix} = c\begin{bmatrix}au_1+bu_2\\cu_1+du_2\end{bmatrix} = c\,(\mathbf{Au}). \checkmark$$

Matrices are linear because arithmetic is distributive.

The centerpiece of the chapter is that the converse holds too: *every* linear map from $\mathbb{R}^n$ to $\mathbb{R}^m$ is given by some matrix. There are no other linear maps hiding out there. To see why, and to learn how to *build* the matrix, we read the columns.

5.3 What the Columns Tell You

Here is the single most useful sentence in this chapter:

Focus

The columns of a matrix are the images of the standard basis vectors. Column j of $\mathbf{A}$ is exactly $\mathbf{A}\mathbf{e}_j$ – where the input arrow $\mathbf{e}_j$ lands. To know a linear map, you only need to know where it sends the basis vectors. Everything else follows by linearity.

Recall the **standard basis vectors** of $\mathbb{R}^2$ – the unit-length measuring sticks pointing along each coordinate axis: $\mathbf{e}_1 = \begin{bmatrix}1\\0\end{bmatrix}$ (one step east) and $\mathbf{e}_2 = \begin{bmatrix}0\\1\end{bmatrix}$ (one step north). Multiply a matrix by $\mathbf{e}_1$ and watch what survives:

$$\begin{bmatrix}a & b\\ c & d\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}a\cdot 1 + b\cdot 0\\ c\cdot 1 + d\cdot 0\end{bmatrix} = \begin{bmatrix}a\\c\end{bmatrix} = \text{first column.}$$

Likewise $\mathbf{A}\mathbf{e}_2 = \begin{bmatrix}b\\d\end{bmatrix}$ is the second column. Picking out $\mathbf{e}_j$ simply *reads off* column j. This is why we say the columns are the images of the basis vectors: feeding the machine the pure "east" arrow returns its first column; feeding it the pure "north" arrow returns its second.

Pause

Mental check: If a machine $\mathbf{A}$ sends $\mathbf{e}_1$ to $\begin{bmatrix}5\\0\end{bmatrix}$ and $\mathbf{e}_2$ to $\begin{bmatrix}0\\-2\end{bmatrix}$, what is the matrix $\mathbf{A}$? (Pause before reading on: just drop the two destination arrows in as the two columns: $\mathbf{A} = \begin{bmatrix}5 & 0\\0 & -2\end{bmatrix}$.)

5.3.1 From basis images to the whole map

Now watch how a single matrix's behavior on the two basis vectors determines its behavior on *every* vector. Any $\mathbf{x} = \begin{bmatrix}x_1\\x_2\end{bmatrix}$ is a linear combination of the basis vectors:

$$\mathbf{x} = \begin{bmatrix}x_1\\x_2\end{bmatrix} = x_1\begin{bmatrix}1\\0\end{bmatrix} + x_2\begin{bmatrix}0\\1\end{bmatrix} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2.$$

Apply T and use linearity (Definition 5.1) to slide T through the combination:

$$T(\mathbf{x}) = T(x_1\mathbf{e}_1 + x_2\mathbf{e}_2) = x_1\,T(\mathbf{e}_1) + x_2\,T(\mathbf{e}_2).$$

Read that last expression aloud: *the output is the same combination of the basis images.* The numbers x_1, x_2 that built the input out of $\mathbf{e}_1, \mathbf{e}_2$ now rebuild the output out of $T(\mathbf{e}_1), T(\mathbf{e}_2)$. So if you stack the basis images as columns into a matrix $\mathbf{A} = [\,T(\mathbf{e}_1)\ \ T(\mathbf{e}_2)\,]$, then

$$T(\mathbf{x}) = x_1\,(\text{col } 1) + x_2\,(\text{col } 2) = \mathbf{A}\mathbf{x}.$$

Formal

Every linear map is a matrix. Let $T : \mathbb{R}^n \to \mathbb{R}^m$ be linear. Define the $m \times n$ matrix $\mathbf{A}$ whose j-th column is $T(\mathbf{e}_j)$. Then $T(\mathbf{x}) = \mathbf{A}\mathbf{x}$ for all $\mathbf{x} \in \mathbb{R}^n$, and this $\mathbf{A}$ is the only matrix that works.

Proof sketch. Write $\mathbf{x} = \sum_j x_j \mathbf{e}_j$. By linearity $T(\mathbf{x}) = \sum_j x_j T(\mathbf{e}_j) = \sum_j x_j (\text{col } j) = \mathbf{A}\mathbf{x}$. Uniqueness: if $\mathbf{B}$ also represents T, then column j of $\mathbf{B}$ is $\mathbf{B}\mathbf{e}_j = T(\mathbf{e}_j) =$ column j of $\mathbf{A}$, so $\mathbf{B} = \mathbf{A}$. □

That is a lot of payoff for very little work. An object you might think needs infinitely many instructions – "tell me where *every* vector goes" – is pinned down by a finite table of numbers: just say where the n basis vectors go. Linearity fills in all the rest.

Why this matters

A linear map carries infinite information (a rule for the whole continuous plane) yet is compressed, with zero loss, into a tiny 2×2 box of four numbers. That compression is the reason linear algebra is computable at all. It is also the seed of every later idea in this book: change of basis is choosing *which* vectors to track, eigenvectors are the inputs the machine treats most simply, and the SVD finds the basis in which the machine looks like pure stretching. All of it grows from "store the images of the basis."

Compute it yourself

Build the matrix from its action. Suppose a linear map T on $\mathbb{R}^2$ satisfies $T\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}3\\1\end{bmatrix}$ and $T\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}-1\\2\end{bmatrix}$.

(a) Write down $\mathbf{A}$ by stacking these as columns.

(b) Use it to compute $T\begin{bmatrix}4\\5\end{bmatrix}$ both by the formula $4\,T(\mathbf{e}_1) + 5\,T(\mathbf{e}_2)$ and by the matrix product $\mathbf{A}\begin{bmatrix}4\\5\end{bmatrix}$, and check that they agree.

Answer: $\mathbf{A} = \begin{bmatrix}3 & -1\\1 & 2\end{bmatrix}$, and both routes give $\begin{bmatrix}7\\14\end{bmatrix}$.

5.3.2 Columns and the unit square

The columns have a direct geometric interpretation. The standard basis vectors $\mathbf{e}_1, \mathbf{e}_2$ are the two sides of the **unit square** – the square with corners $(0,0), (1,0), (1,1), (0,1)$. Their images $\mathbf{A}\mathbf{e}_1, \mathbf{A}\mathbf{e}_2$ are the two columns of $\mathbf{A}$, and they are the two sides of the **output parallelogram** the unit square becomes. Drawing the columns as arrows draws the entire transformation, because the rest of the grid is forced to follow.

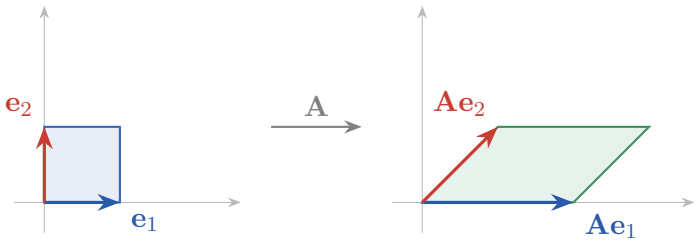


Figure 5.3: Reading $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$ off the unit square. The columns $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ are the images of $\mathbf{e}_1$ and $\mathbf{e}_2$; they form the two sides of the parallelogram that the square becomes. The whole grid follows.

Walking question

Picture the unit square and its two edge-arrows $\mathbf{e}_1, \mathbf{e}_2$. Now imagine sliding the tip of $\mathbf{e}_2$ straight up to $(0, 3)$ while leaving $\mathbf{e}_1$ alone. What 2×2 matrix did you just describe, and what happens to the square's area? Then imagine instead swinging $\mathbf{e}_2$ down until it points *along* $\mathbf{e}_1$. What is the matrix now, and why has the square gone flat?

5.4 The Action on the Grid

Let us make good on the promise that the image of a grid is a grid. Suppose $\mathbf{A}\mathbf{e}_1 = \mathbf{a}_1$ and $\mathbf{A}\mathbf{e}_2 = \mathbf{a}_2$ are the two columns. A grid point with integer coordinates (m, n) is the input $m\mathbf{e}_1 + n\mathbf{e}_2$, and by linearity it lands at

$$\mathbf{A}(m\mathbf{e}_1 + n\mathbf{e}_2) = m\,\mathbf{a}_1 + n\,\mathbf{a}_2.$$

So the original square lattice $\{m\mathbf{e}_1 + n\mathbf{e}_2\}$ becomes the *sheared* lattice $\{m\mathbf{a}_1 + n\mathbf{a}_2\}$ – still a perfectly regular grid, just built from the new edge-vectors $\mathbf{a}_1$ and $\mathbf{a}_2$ instead of the old $\mathbf{e}_1, \mathbf{e}_2$. Straight lines stay straight (a line is $\mathbf{p} + t\mathbf{d}$, and $\mathbf{A}(\mathbf{p} + t\mathbf{d}) = \mathbf{A}\mathbf{p} + t\,\mathbf{A}\mathbf{d}$ is again a line), and parallel lines stay parallel. This is the visual signature of linearity.

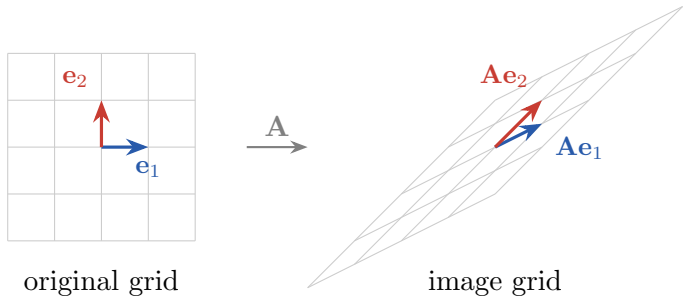


Figure 5.4: A linear map turns the square grid into a slanted grid. With $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0.5 & 1 \end{bmatrix}$ the basis arrows tilt to the new columns $\begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$, and every grid line follows. Lines stay straight and parallels stay parallel – the grid is sheared, never crumpled.

Before/after Strang

In Strang's course you will spend a lot of time on the equation $\mathbf{Ax} = \mathbf{b}$: *which* input $\mathbf{x}$ produces a target output $\mathbf{b}$? This chapter is the other half of that conversation. Here we hold $\mathbf{A}$ fixed and watch it sweep over *all* inputs at once – the machine's view. When Strang draws the "column picture" of $\mathbf{Ax} = \mathbf{b}$ as a combination of columns, he is using exactly our identity $\mathbf{Ax} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2$. Carrying the moving-grid picture in beforehand makes his row/column pictures feel inevitable instead of arbitrary.

5.5 A Tour of 2×2 Machines

We now meet the named transformations of the plane. For each one, the recipe is the same and it never gets old: *find where* $\mathbf{e}_1$ *and* $\mathbf{e}_2$ *go, and stack those as columns.* Keep Figure 5.3 in mind – every matrix below is just a different pair of edge-arrows for the unit square.

5.5.1 Scalings: stretch and squash

The simplest machine multiplies each coordinate by a fixed factor. A **scaling** with horizontal factor s and vertical factor t sends $\mathbf{e}_1 \mapsto \begin{bmatrix} s \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 \mapsto \begin{bmatrix} 0 \\ t \end{bmatrix}$, so

$$\mathbf{D} = \begin{bmatrix} s & 0 \\ 0 & t \end{bmatrix}, \qquad \mathbf{D}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} sx \\ ty \end{bmatrix}.$$

This is a **diagonal matrix**, $\mathbf{D} = \operatorname{diag}(s, t)$ – a matrix whose only nonzero entries lie along the main diagonal from top-left to bottom-right, meaning it scales each coordinate axis independently without mixing them. When $s = t$ it is a uniform zoom (everything grows by the same factor, shapes preserved). When $s \neq t$ the plane stretches more in one direction than the other – a circle becomes an ellipse, as in Figure 5.5. Our opening example $\operatorname{diag}(2, 3)$ was exactly this: double across, triple up.

Pause

Mental check: What does the matrix $\operatorname{diag}(1, -1)$ do to a vector $\begin{bmatrix} x \\ y \end{bmatrix}$? (Pause: it leaves x untouched and flips y to $-y$ – reflecting every point across the horizontal axis.)

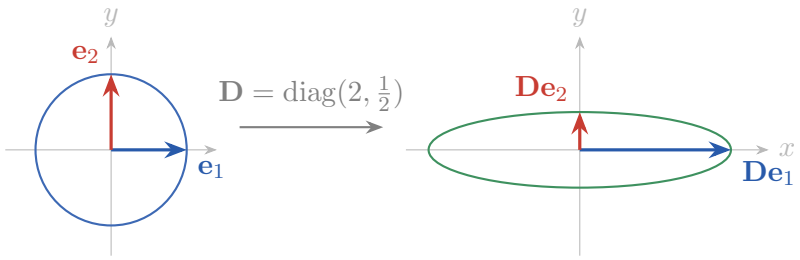


Figure 5.5: A non-uniform scaling $\mathbf{D} = \mathrm{diag}(2, \frac{1}{2})$ pulls the unit circle into an ellipse: $\mathbf{e}_1$ doubles in length while $\mathbf{e}_2$ is halved. The semi-axes of the ellipse are exactly the diagonal factors. The area changes by the product $2 \cdot \frac{1}{2} = 1$, so this particular stretch keeps area fixed even though it badly distorts the shape.

A negative factor flips that axis: $\mathrm{diag}(-1, 1)$ is the reflection across the y-axis from Example 5.1. A zero factor collapses that axis entirely: $\mathrm{diag}(1, 0)$ flattens the whole plane onto the x-axis, which is the projection $\mathbf{P}$ from the same example.

Example 5.4 (A non-uniform scaling, step by step)**.** Take $\mathbf{D} = \mathrm{diag}(\frac{1}{2}, 2)$. The unit square's corner $(1, 1)$ goes to $(\frac{1}{2}, 2)$; the square (area 1) becomes a rectangle $\frac{1}{2}$ wide and 2 tall (area 1 still, since $\frac{1}{2} \cdot 2 = 1$). The horizontal direction is squeezed, the vertical stretched, and – a preview of determinants – the area is unchanged because the product of the diagonal factors is 1.

5.5.2 Rotations

A **rotation** spins the whole plane rigidly about the origin by an angle θ (counterclockwise for $\theta > 0$). Lengths and angles are preserved; the grid turns like a record on a turntable. To find its matrix, we only need to know where the two basis arrows swing to.

Spin $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, which points east, by θ. Its tip traces the unit circle, so it lands at $\begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix}$. Spin $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, which points north, by θ; it lands $90°$ ahead of the first arrow, at $\begin{bmatrix} -\sin\theta \\ \cos\theta \end{bmatrix}$. Stack these as columns:

$$\boxed{\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.}$$

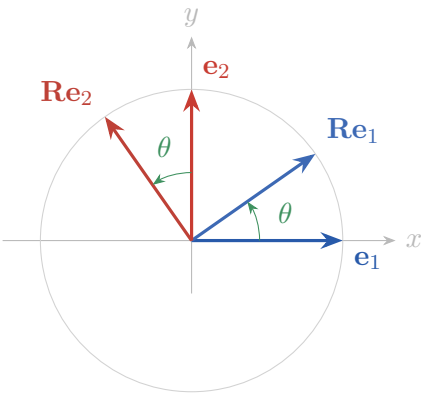


Figure 5.6: Building the rotation matrix from basis images. The east arrow $\mathbf{e}_1$ swings to $(\cos\theta, \sin\theta)$ and the north arrow $\mathbf{e}_2$ swings to $(-\sin\theta, \cos\theta)$. These two images are the columns of $\mathbf{R}_\theta$.

Example 5.5 (Rotation by 90°)**.** With $\theta = 90°$ we have $\cos 90° = 0$, $\sin 90° = 1$, so

$$\mathbf{R}_{90°} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{R}_{90°} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} -b \\ a \end{bmatrix},$$

matching the rotation we checked in Example 5.2. The east arrow $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ goes to north $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, and north goes to west $\begin{bmatrix} -1 \\ 0 \end{bmatrix}$ – exactly a quarter turn.

Example 5.6 (Rotation by 45°)**.** Here $\cos 45° = \sin 45° = \frac{1}{\sqrt{2}} \approx 0.707$, so

$$\mathbf{R}_{45°} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}.$$

Apply it to $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$: out comes $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, the arrow pointing exactly along the diagonal – a 45° tilt, as promised. Note the columns each have length 1 and are perpendicular; rotations never change size or shape, only orientation.

Connection to other math

The rotation matrices encode the angle-addition formulas of trigonometry. Rotating by α and then by β must be the same as rotating once by $\alpha + \beta$, i.e. $\mathbf{R}_\beta \mathbf{R}_\alpha = \mathbf{R}_{\alpha+\beta}$. Multiply the two matrices out and match the top-left entry:

$$\cos(\alpha + \beta) = \cos\beta\cos\alpha - \sin\beta\sin\alpha.$$

The angle-sum identity you memorized in high school is just the statement that *composing rotations adds angles*. The same product is how complex multiplication rotates the plane ($e^{i\alpha}e^{i\beta} = e^{i(\alpha+\beta)}$), a bridge to the complex-numbers chapter and, later, to the Fourier transform.

5.5.3 Reflections

A **reflection** flips the plane across a line through the origin, as if that line were a mirror. Points on the mirror stay fixed; everything else swaps to the opposite side. The easiest is reflection across the x-axis: it keeps $\mathbf{e}_1$ and flips $\mathbf{e}_2$ to $\begin{bmatrix} 0 \\ -1 \end{bmatrix}$, giving

$$\mathbf{M}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

More generally, reflection across the line through the origin making angle ϕ with the x-axis has matrix

$$\mathbf{M}_\phi = \begin{bmatrix} \cos 2\phi & \sin 2\phi \\ \sin 2\phi & -\cos 2\phi \end{bmatrix}.$$

(You can derive this by rotating the mirror line down to the x-axis, flipping, and rotating back – a nice composition exercise once we have the product.) The 2ϕ instead of ϕ is the famous quirk of reflections: a mirror tilted by ϕ flips directions through twice that angle.

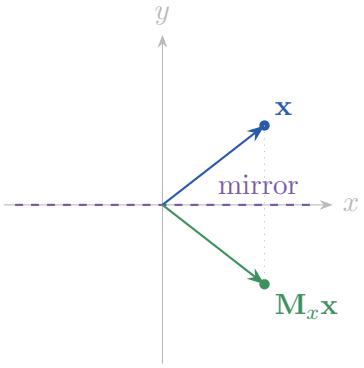


Figure 5.7: Reflection across the x-axis. The mirror (the x-axis) is fixed; the point $\mathbf{x} = (0.9, 0.7)$ is sent straight down to its mirror image $(0.9, -0.7)$. The horizontal coordinate is unchanged, the vertical coordinate flips sign.

Common trap

Rotations and reflections both keep lengths the same, so it is tempting to lump them together. The difference is *orientation*. A rotation preserves handedness: a counterclockwise loop stays counterclockwise. A reflection reverses it: it turns a left hand into a right hand. Algebraically, the determinant of a rotation is $+1$ and of a reflection is -1 – a sign you will meet again in the determinants chapter. If you ever confuse the two, ask: does my shape come out mirror-flipped? If yes, it was a reflection.

5.5.4 Shears

A **shear** slides points parallel to one axis by an amount proportional to their distance from that axis – think of pushing the top of a deck of cards sideways while the bottom stays put. A horizontal

shear keeps $\mathbf{e}_1$ fixed and tilts $\mathbf{e}_2$ to $\begin{bmatrix} k \\ 1 \end{bmatrix}$:

$$\mathbf{S}_k = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}, \qquad \mathbf{S}_k \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + ky \\ y \end{bmatrix}.$$

A point's height y is unchanged, but it slides right by ky – higher points slide farther. The horizontal axis stays put (when $y = 0$ nothing moves); the rest of the plane leans like italic type. Notice that a shear does *not* change area: the parallelogram the unit square becomes has the same base and height as the square (look back at Figure 5.3 – base 1, height 1). This area-preservation is the determinant $\det \mathbf{S}_k = 1$ again.

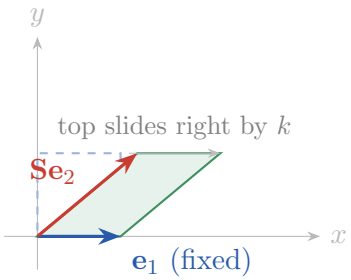


Figure 5.8: A horizontal shear $\mathbf{S}_k$ with $k = 1.2$. The base $\mathbf{e}_1$ is nailed in place; the top edge slides right in proportion to its height. The dashed square becomes the green parallelogram – same base, same height, so the same area.

Modern application

These fundamental geometric transformations appear throughout scientific computing. *Computer graphics and games* build all 2D and 3D motion – rotating a character, scaling a sprite, skewing a shadow – out of exactly these matrices, applied to the vertices of meshes; a GPU does millions of these matrix–vector products per frame. *Image processing* uses shears as a practical tool for image rotation: a rotation can be factored into three shears, and shearing an image corresponds to shifting rows or columns of pixels in memory. *Data augmentation* for training neural networks deliberately applies small random rotations, scalings, reflections, and shears to each training image so the model learns features that are invariant to these transformations.

Compute it yourself

Compute these 2×2 actions by hand and *sketch* the unit square's image for each.

(a) $\mathbf{R}_{180^\circ} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$ on $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$.

(b) The shear $\mathbf{S}_2 = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ on $\begin{bmatrix} 3 \\ 1 \end{bmatrix}$ and on $\begin{bmatrix} 3 \\ 0 \end{bmatrix}$ – notice the second is unmoved. Why?

(c) The reflection $\mathbf{M}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ applied twice to $\begin{bmatrix} a \\ b \end{bmatrix}$. What do you get, and what does that say about $\mathbf{M}_x \mathbf{M}_x$?

Answers: (a) $\begin{bmatrix}-1\\-2\end{bmatrix}$. (b) $\begin{bmatrix}5\\1\end{bmatrix}$ and $\begin{bmatrix}3\\0\end{bmatrix}$; the second has height 0, and a shear moves nothing on the axis it fixes. (c) back to $\begin{bmatrix}a\\b\end{bmatrix}$, so reflecting twice is the identity, $\mathbf{M}_x^2 = \mathbf{I}$.

5.5.5 The identity and the zero map

Two extreme machines bookend the tour. The **identity** $\mathbf{I} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}$ sends every basis vector to itself, so $\mathbf{Ix} = \mathbf{x}$: it does nothing, the "leave it alone" machine. The **zero matrix 0** sends every vector to $\mathbf{0}$: it crushes all of space to a single point. Between "change nothing" and "destroy everything" live all the interesting transformations.

5.6 Composition: Doing One Machine After Another

Machines are made to be chained. Apply $\mathbf{B}$ to a vector, then apply $\mathbf{A}$ to the result: "first $\mathbf{B}$, then $\mathbf{A}$." The combined effect is itself a linear map (a composition of linear maps is linear – chase the definitions), so by Section 5.3 it *must* be some matrix. Which matrix? That matrix is the **product** $\mathbf{AB}$, defined precisely so that

$$\mathbf{A}(\mathbf{Bx}) = (\mathbf{AB})\mathbf{x} \quad \text{for every } \mathbf{x}.$$

This is the real reason matrix multiplication is defined the way it is – the row- times-column rule is not arbitrary, it is engineered to make "compose the machines" come out right. We will derive the full rule in the multiplication chapter; here we just watch it work on the cleanest case and absorb the order.

Example 5.7 (Rotate, then rotate: angles add)**.** Compose a 90° rotation after another 90° rotation. Each is $\mathbf{R}_{90^\circ} = \begin{bmatrix}0 & -1\\1 & 0\end{bmatrix}$, and the combined machine is found by tracking the basis vectors through both steps. Start with $\mathbf{e}_1 = \begin{bmatrix}1\\0\end{bmatrix}$:

$$\begin{bmatrix}1\\0\end{bmatrix} \xrightarrow{\mathbf{R}_{90^\circ}} \begin{bmatrix}0\\1\end{bmatrix} \xrightarrow{\mathbf{R}_{90^\circ}} \begin{bmatrix}-1\\0\end{bmatrix}.$$

And $\mathbf{e}_2 = \begin{bmatrix}0\\1\end{bmatrix}$:

$$\begin{bmatrix}0\\1\end{bmatrix} \xrightarrow{\mathbf{R}_{90^\circ}} \begin{bmatrix}-1\\0\end{bmatrix} \xrightarrow{\mathbf{R}_{90^\circ}} \begin{bmatrix}0\\-1\end{bmatrix}.$$

Stacking the two final images as columns gives the combined matrix $\begin{bmatrix}-1 & 0\\0 & -1\end{bmatrix} = \mathbf{R}_{180^\circ}$. Two quarter turns make a half turn – exactly what geometry demands, and exactly the angle-addition we met in the rotation connection box.

Common trap

Order matters: AB usually does not equal BA. Putting on socks then shoes is not the same as shoes then socks. For a concrete clash, take a reflection $\mathbf{M}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ and a 90° rotation $\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$. Tracking $\mathbf{e}_1$: "rotate then reflect" sends $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \to \begin{bmatrix} 0 \\ 1 \end{bmatrix} \to \begin{bmatrix} 0 \\ -1 \end{bmatrix}$, while "reflect then rotate" sends $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \to \begin{bmatrix} 1 \\ 0 \end{bmatrix} \to \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Different outputs, so the two orders are different machines. Also note **AB** means "**B** first" – the matrix nearest the vector acts first. The notation runs right to left, like function composition $f(g(x))$.

Figure 5.9 draws this clash step by step across two tracks. Tracking where the same arrow $\mathbf{e}_1$ travels under both compositions reveals that the two paths end at opposite destinations.

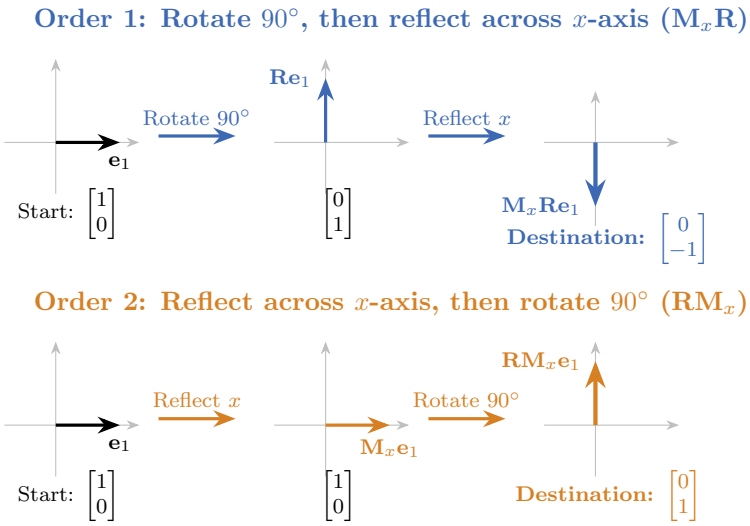


Figure 5.9: Matrix multiplication does not commute because the order of physical operations matters. Top track: rotating $\mathbf{e}_1$ by 90° sends it to $(0, 1)$, and then reflecting across the x-axis flips it down to $(0, -1)$. Bottom track: reflecting $\mathbf{e}_1$ across the x-axis leaves it at $(1, 0)$, and then rotating by 90° swings it up to $(0, 1)$. The two compositions finish at completely opposite destinations: $\mathbf{M}_x\mathbf{R} \neq \mathbf{R}\mathbf{M}_x$.

Before/after Strang

After Strang, you will recognize this section as the doorway to nearly everything deep. Factoring a matrix is composition read backwards: $\mathbf{A} = \mathbf{L}\mathbf{U}$ says "$\mathbf{A}$ is a shear-like machine after another," $\mathbf{A} = \mathbf{Q}\mathbf{R}$ splits it into an orthogonal map and a triangular factor, and the singular value decomposition $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ (the SVD) shows that *every* matrix is a rotation, then a pure scaling, then another rotation. Three of the machines from this very chapter – rotate, stretch, rotate – compose to give *any* linear map at all. Hold onto that picture; it is where the book is headed.

Why this matters

Once you read an $m \times n$ matrix as a transformation pinned down by where it sends the basis vectors, every later topic becomes a geometric question about that motion. Solving $\mathbf{Ax} = \mathbf{b}$ asks which input lands on $\mathbf{b}$, eigenvectors pick out the directions the motion leaves unrotated, and the SVD finds perpendicular axes along which any matrix acts as a pure stretch.

Connection to other math

The area-scaling factor of a linear map is the bridge to multivariable calculus. A smooth nonlinear map bends the grid, but zoom in close enough to any point and it looks linear – its local linear approximation is the *Jacobian matrix* of partial derivatives. The factor by which that Jacobian scales tiny areas (its determinant) is precisely the $\det \mathbf{A}$ you are about to rediscover in Exercise 5.12, and it is exactly the $|\partial(x, y)/\partial(u, v)|$ that appears when you change variables in a double integral. "Matrices as machines" is the linearized skeleton of all of differential calculus in several variables.

5.7 Cost and Intuition Recap

- **A matrix is a verb.** An $m \times n$ matrix $\mathbf{A}$ is the function $T(\mathbf{x}) = \mathbf{Ax}$ sending $\mathbb{R}^n \to \mathbb{R}^m$. Read the shape first: n columns in, m rows out (Section 5.1).
- **Linearity is "the grid stays a grid."** $T(c\mathbf{u} + d\mathbf{v}) = cT(\mathbf{u}) + dT(\mathbf{v})$, and every linear map fixes the origin (Section 5.2).
- **The columns are the basis images.** Column j is $\mathbf{Ae}_j$; knowing where the n basis arrows go pins down the entire map (Section 5.3). Cost to store a linear map on $\mathbb{R}^n \to \mathbb{R}^m$: just mn numbers.
- **Cost of one action.** Computing $\mathbf{Ax}$ for an $m \times n$ matrix takes mn multiplications and $m(n-1)$ additions – about $2mn$ arithmetic operations.
- **The named machines.** Scalings $\mathrm{diag}(s, t)$, rotations $\mathbf{R}_\theta$, reflections $\mathbf{M}_\phi$, shears $\mathbf{S}_k$ – each is just a choice of two edge-arrows for the unit square (Section 5.5).
- **Composition is multiplication.** "First $\mathbf{B}$, then $\mathbf{A}$" is the matrix $\mathbf{AB}$, and order matters: $\mathbf{AB} \neq \mathbf{BA}$ in general (Section 5.6).

Together, these ideas complete the forward picture: you feed the machine an input arrow $\mathbf{x}$, and it carries it to its destination $\mathbf{Ax}$. Yet in real applications, we frequently face the opposite problem: we observe the destination $\mathbf{b}$ and must reconstruct the secret input $\mathbf{x}$ that arrived there. Running a machine backward – solving $\mathbf{Ax} = \mathbf{b}$ – is the central challenge of linear algebra, and the next chapter takes it head-on.

History and connection

Before matrices had a name, mathematicians had been computing with determinants for over a century. A **determinant** is a single number calculated from a square grid of numbers that diagnoses whether a set of linear equations has a unique solution. In 1850, the English mathematician James Joseph Sylvester (1814–1897) coined the word **matrix**. Writing in the

Philosophical Magazine, Sylvester chose the term from the Latin *matrix*, meaning "womb" or "breeding ground," because he viewed the rectangular array of numbers as a source from which smaller square determinants (minors) could be extracted. To Sylvester, the determinants were primary and the grid was bookkeeping.

Sylvester's close friend and collaborator Arthur Cayley (1821–1895) treated the matrix as an algebraic object in its own right. In his 1855 note and his landmark 1858 paper *A Memoir on the Theory of Matrices*, Cayley defined algebraic operations directly on matrices rather than just on their determinants. He was investigating **coordinate substitutions** – algebraic rules that rename variables to describe geometric shifts (like replacing (x, y) with (u, v) via $x = au + bv$ and $y = cu + dv$). Cayley asked: what happens when you chain two substitutions together, replacing (u, v) in turn by (s, t) via $u = \alpha s + \beta t$ and $v = \gamma s + \delta t$? By substituting the second system into the first and regrouping, Cayley discovered the universal row-times-column rule for multiplying two matrices.

This new algebra was a significant departure from classical arithmetic. For centuries, multiplication had been assumed to be **commutative** ($ab = ba$ – order never alters the product). But Cayley proved that matrix multiplication is generally non-commutative ($\mathbf{AB} \neq \mathbf{BA}$), reflecting the geometric fact that composing spatial transformations in different orders produces different results. Matrix algebra also admits **zero divisors**: two nonzero matrices can multiply to yield the all-zero matrix ($\mathbf{AB} = \mathbf{0}$ with $\mathbf{A} \neq \mathbf{0}$ and $\mathbf{B} \neq \mathbf{0}$). In ordinary arithmetic, $a \cdot b = 0$ implies $a = 0$ or $b = 0$; in matrix algebra, two non-trivial transformations can compose to collapse an entire space to the origin.

Treating linear transformations as algebraic objects in their own right had wide consequences. In 1925, Werner Heisenberg, Max Born, and Pascual Jordan realized that matrices were the exact language of quantum mechanics, where physical quantities like position and momentum fail to commute. Today, matrices form the computational engine of computer graphics, robotics, and machine learning.

Computational questions about these machines remain open. Cayley's row-times-column rule multiplies two $n \times n$ matrices using about n^3 scalar multiplications. In 1969, Volker Strassen showed it could be done in $O(n^{2.81})$ operations. Today, the theoretical exponent has been driven down near 2.371, but whether two matrices can be multiplied in near-$O(n^2)$ time – the time it takes to read the entries – remains an open problem in computer science. Beyond multiplication, while running a matrix forward ($\mathbf{x} \mapsto \mathbf{Ax}$) is effortless arithmetic, running it backward – finding which input $\mathbf{x}$ produces a target output $\mathbf{b}$ – requires untangling coupled equations. That inverse challenge takes center stage in the next chapter, which explores systems of linear equations through the twin perspectives of the row picture and the column picture.

5.8 Exercises

Exercise 5.1 (Warmup: act on a vector)**.** Let $\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 1 & 3 \end{bmatrix}$. Compute $\mathbf{A}\begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{A}\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, and $\mathbf{A}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Verify that the third is the sum of the first two, and explain which property of linear maps guarantees this.

Exercise 5.2 (Build the matrix from its action)**.** A linear map T on $\mathbb{R}^2$ satisfies $T\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$

and $T\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}0\\4\end{bmatrix}$. Write down its matrix, then compute $T\begin{bmatrix}3\\2\end{bmatrix}$ two ways (as a combination of basis images and as a matrix product) and confirm they agree.

Exercise 5.3 (Linear or not?)**.** For each map, decide whether it is linear; if not, give a specific numerical counterexample to additivity or homogeneity. (a) $T\begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}x+y\\x-y\end{bmatrix}$. (b) $T\begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}x\\y+1\end{bmatrix}$. (c) $T\begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}xy\\0\end{bmatrix}$. (d) $T\begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}2x\\0\end{bmatrix}$.

Exercise 5.4 (Name that machine)**.** Identify the geometric effect of each matrix (scaling, rotation, reflection, or shear), and state where it sends the unit square. (a) $\begin{bmatrix}0&1\\1&0\end{bmatrix}$, (b) $\begin{bmatrix}3&0\\0&3\end{bmatrix}$, (c) $\begin{bmatrix}1&0\\-2&1\end{bmatrix}$, (d) $\begin{bmatrix}0&-1\\1&0\end{bmatrix}$. (Hint for (a): where do $\mathbf{e}_1$ and $\mathbf{e}_2$ go? What line is the mirror?)

Exercise 5.5 (Rotation practice)**.** Write the rotation matrix $\mathbf{R}_\theta$ for $\theta = 30°$ using $\cos 30° = \frac{\sqrt{3}}{2}$, $\sin 30° = \frac{1}{2}$. Apply it to $\begin{bmatrix}2\\0\end{bmatrix}$ and verify the output has length 2 (rotations preserve length).

Exercise 5.6 (Composition by tracking the basis)**.** Let $\mathbf{S} = \begin{bmatrix}1&1\\0&1\end{bmatrix}$ (shear) and $\mathbf{M} = \begin{bmatrix}1&0\\0&-1\end{bmatrix}$ (reflection). By tracking $\mathbf{e}_1$ and $\mathbf{e}_2$ through both steps, find the matrix for "first $\mathbf{S}$, then $\mathbf{M}$" and the matrix for "first $\mathbf{M}$, then $\mathbf{S}$." Confirm they are different, illustrating that order matters.

Exercise 5.7 (Reflection twice)**.** Let $\mathbf{M}$ be reflection across the x-axis. By applying it twice to a general $\begin{bmatrix}a\\b\end{bmatrix}$, show that $\mathbf{M}^2 = \mathbf{I}$. A transformation that equals its own undo button (its own inverse, meaning applying it twice returns every point to its start: $T^2 = \mathbf{I}$) is called an **involution**; name two other machines from this chapter that are involutions.

Exercise 5.8 (Collapse and information loss)**.** Let $\mathbf{P} = \begin{bmatrix}1&0\\0&0\end{bmatrix}$. (a) Where does $\mathbf{P}$ send the unit square? (b) Find two *different* input vectors that $\mathbf{P}$ sends to the same output, and explain geometrically why $\mathbf{P}$ cannot be undone. (This is your first encounter with a non-invertible machine and its null space.)

Exercise 5.9 (Proof: linear maps fix the origin)**.** Using only the homogeneity axiom, prove that every linear map satisfies $T(\mathbf{0}) = \mathbf{0}$. Then use additivity to prove $T(-\mathbf{v}) = -T(\mathbf{v})$. Explain in one sentence why this rules out the affine map $T(\mathbf{x}) = \mathbf{x} + \begin{bmatrix}1\\1\end{bmatrix}$ being linear.

Exercise 5.10 (Proof: composition is linear)**.** Let $S, T : \mathbb{R}^2 \to \mathbb{R}^2$ be linear. Prove that their composition $(S \circ T)(\mathbf{x}) = S(T(\mathbf{x}))$ is linear by verifying additivity and homogeneity directly. (This is why the product of two matrices is again a matrix.)

Exercise 5.11 (Challenge: derive the general reflection)**.** Reflection across the line through the origin at angle ϕ can be built in three steps: rotate the line down to the x-axis by $-\phi$, reflect across the x-axis, and rotate back up by $+\phi$. Using the rule that a composite map's matrix is the product of the individual matrices, multiply $\mathbf{R}_\phi \, \mathbf{M}_x \, \mathbf{R}_{-\phi}$ and apply double-angle identities to obtain $\mathbf{M}_\phi = \begin{bmatrix}\cos 2\phi & \sin 2\phi \\ \sin 2\phi & -\cos 2\phi\end{bmatrix}$.

Exercise 5.12 (Challenge: area scaling)**.** For each matrix, draw the image of the unit square and compute its area: $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, $\begin{bmatrix} 1 & 4 \\ 0 & 1 \end{bmatrix}$, $\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$. Conjecture a formula for the area of the output parallelogram in terms of the entries a, b, c, d of a general 2×2 matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ (noting that geometric area is always positive: consider $|ad - bc|$). You have just rediscovered the determinant.

Part II

Solving Linear Systems

Chapter 6

Systems of Linear Equations and Their Geometry

In Chapter 5, we watched matrices act as machines: feed in an input vector $\mathbf{x}$, and the machine hands back an output vector $\mathbf{b} = A\mathbf{x}$. That was the forward question – run the transformation and see where space lands. Solving a system of linear equations is the reverse question: you are handed the destination vector $\mathbf{b}$, and you must deduce which input vector(s) $\mathbf{x}$ the machine took there:

$$A\mathbf{x} = \mathbf{b}.$$

Did exactly one input land at $\mathbf{b}$? Did a whole family of inputs collapse onto it? Or is $\mathbf{b}$ somewhere the machine can never reach?

Before you ever wrote matrices as machines, you likely first encountered this question in elementary algebra: "two pencils and a notebook cost \$5, three pencils and two notebooks cost \$8, find the price of each." You may have solved it by substitution or by adding equations to eliminate a variable.

This chapter asks you to slow down and *look* at what you were doing, because hidden inside that humble word problem is the entire geometric skeleton of linear algebra. A system like

$$\begin{aligned} 2x + 1y &= 5, \\ 3x + 2y &= 8 \end{aligned}$$

can be *seen* in two different ways, and the whole subject grows out of holding both pictures in your head at once. One picture is about *rows* – lines and planes crossing. The other is about *columns* – arrows adding up. They describe the same equations and give the same answer, yet they feel like two different universes. Learning to flip between them at will is the skill that makes everything after this chapter click.

We will draw both pictures carefully, in two and three dimensions, with small numbers you can check by hand. Then we will ask the three questions that organize all of linear algebra:

- When does a system have *exactly one* solution?
- When does it have *none*?
- When does it have *infinitely many*?

In linear algebra, those are the *only* three possibilities – a linear system can never have, say, exactly two solutions. We call a square matrix *nonsingular* when its columns stay spread out to span the whole space, and *singular* when they collapse onto a lower-dimensional line or plane.

6.1 Two pictures of one system

Let us fix a concrete system and never let go of it:

$$\begin{aligned} 2x + 1y &= 5, \\ 3x + 2y &= 8. \end{aligned} \tag{6.1}$$

There are two ways to read these symbols, and both are correct and useful. Most people are taught only the first and never told the second exists, which is a shame, because the second is the one that grows up into the rest of the book.

6.1.1 The row picture: lines that cross

Read (6.1) one row at a time. The first row, $2x + 1y = 5$, is a *constraint* on the pair (x, y). The set of all points (x, y) that satisfy it is a straight line in the plane. The second row, $3x + 2y = 8$, is a different constraint, a different line. A pair (x, y) solves the *system* exactly when it sits on *both* lines at once – that is, at the point where the two lines cross.

Intuition

The row picture turns "solve the system" into "find where the lines meet." Each equation is a line (in 2 unknowns) or a plane (in 3 unknowns). A solution is a point common to all of them.

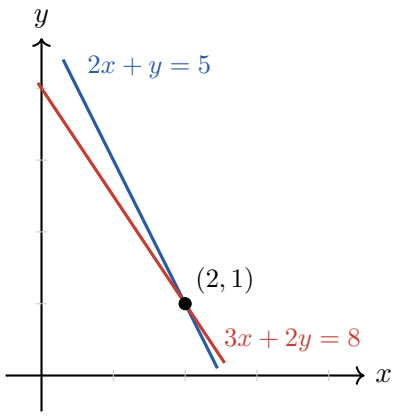


Figure 6.1: The row picture of system (6.1). Each equation is a line; the solution is the single point $(2, 1)$ where they cross.

You can read the answer right off Figure 6.1: the lines cross at $(x, y) = (2, 1)$. Check it: $2(2)+1 = 5$, yes; $3(2) + 2(1) = 8$, yes. Two lines in a plane, if they are not parallel, meet in exactly one point. That "if they are not parallel" is the seed of everything that can go wrong, and we will return to it.

6.1.2 The column picture: arrows that add up

Now read the *same* system (6.1) a different way – not row by row, but *column by column*. Stack the two equations into a single vector equation:

$$\begin{bmatrix} 2x + 1y \\ 3x + 2y \end{bmatrix} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}.$$

The left side is a sum of two pieces, one carrying x and one carrying y:

$$x \begin{bmatrix} 2 \\ 3 \end{bmatrix} + y \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}.$$

This equation says: take the vector $\mathbf{a}_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$, the vector $\mathbf{a}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, and the target $\mathbf{b} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}$. The question is no longer "where do two lines cross?" It is now:

Focus

How much of $\mathbf{a}_1$ *and how much of* $\mathbf{a}_2$ *must I add to land exactly on* $\mathbf{b}$*?* The unknowns x and y are the *amounts*. A linear system asks you to build the target vector $\mathbf{b}$ out of the *columns* of the matrix.

The columns of the system are the arrows $\mathbf{a}_1, \mathbf{a}_2$. We are looking for the recipe $x\,\mathbf{a}_1 + y\,\mathbf{a}_2$ that reaches $\mathbf{b}$. This is a *linear combination* of the columns – exactly the operation you met when we first talked about span. Solving a linear system *is* asking whether $\mathbf{b}$ lies in the span of the columns, and if so, with what weights.

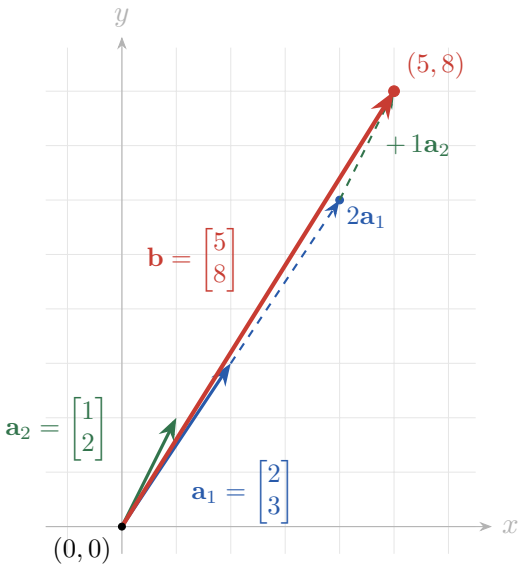


Figure 6.2: The column picture of system (6.1). We need $x\,\mathbf{a}_1 + y\,\mathbf{a}_2 = \mathbf{b}$. The answer $x = 2$, $y = 1$ says: walk twice along $\mathbf{a}_1$ (reaching $(4, 6)$), then once along $\mathbf{a}_2$, and you arrive exactly at $\mathbf{b} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}$.

The two pictures use the exact same numbers, read across the rows or down the columns. In the row picture, each row $(2, 1)$ and $(3, 2)$ defines a *line*, and the right-hand side $(5, 8)$ sets where those

lines sit. In the column picture, each column $(2, 3)$ and $(1, 2)$ is an *arrow* we scale and add to reach the single *target arrow* $(5, 8)$.

Common trap

The single most common confusion in week one of linear algebra: mixing up the rows and the columns of $\begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}$. The *rows* $(2, 1), (3, 2)$ are coefficients of the two equations – they make the *lines*. The *columns* $\begin{bmatrix} 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ are the arrows you combine – they make the *span*. When you read off a column of a matrix, read *down*.

6.1.3 Matrix form: the whole thing as $A\mathbf{x} = \mathbf{b}$

Both pictures collapse into one tidy statement. Collect the columns side by side into a *matrix* A, stack the unknowns into a vector $\mathbf{x}$, and the targets into $\mathbf{b}$:

$$A = \begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 5 \\ 8 \end{bmatrix},$$

so that the system is simply

$$A\mathbf{x} = \mathbf{b}.$$

The notation hides the duality but does not destroy it. The product $A\mathbf{x}$ can be computed two ways, and they are exactly our two pictures:

- **Row way:** the i-th entry of $A\mathbf{x}$ is row i of A dotted with $\mathbf{x}$. Setting each dotted result equal to the matching entry of $\mathbf{b}$ gives back the original equations (the lines).
- **Column way:** $A\mathbf{x} = x\,(\text{col } 1) + y\,(\text{col } 2)$, a linear combination of the columns (the arrows).

Keep both readings of $A\mathbf{x}$ alive. When we discuss whether a solution exists, the column reading dominates; when we actually *compute* one by elimination, the row reading dominates.

Before/after Strang

Before Strang: when you open his book, the very first lecture is "the geometry of linear equations," and it is built on precisely this row-versus-column split. Strang emphasizes that the column picture is the perspective students most often overlook, and that "$A\mathbf{x}$ is a linear combination of the columns of A" is the central takeaway. Mastering this duality makes his geometric introduction to linear systems immediate and intuitive. *After Strang:* the column picture is also where the four fundamental subspaces are born – the set of all reachable $\mathbf{b}$ is the *column space*, which you will meet soon.

Pause

Pause and think: two columns in three dimensions. Before reading on, picture a

system with 3 equations and 2 unknowns:

$$x\begin{bmatrix}a_1\\a_2\\a_3\end{bmatrix}+y\begin{bmatrix}b_1\\b_2\\b_3\end{bmatrix}=\begin{bmatrix}c_1\\c_2\\c_3\end{bmatrix}.$$

What does the row picture look like? Each equation is a plane in $\mathbb{R}^3$. Now look at the column picture: you are given just *two* column arrows in three-dimensional space $\mathbb{R}^3$. All their combinations $x\mathbf{a}_1 + y\mathbf{a}_2$ fill at most a flat 2D sheet through the origin. Unless the 3D target arrow $\mathbf{b}$ happens by sheer coincidence to lie on that exact sheet, *no solution can possibly exist*. The column picture makes it obvious at a single glance why "tall" systems (more equations than unknowns) rarely have exact solutions.

6.2 A fully worked 2×2, both ways

Let us solve (6.1) from scratch with bare algebra, so the geometry has numbers under it.

Example 6.1 (Substitution, the row way)**.** From the first equation, $y = 5 - 2x$. Substitute into the second:

$$3x + 2(5 - 2x) = 8 \implies 3x + 10 - 4x = 8 \implies -x = -2 \implies x = 2.$$

Then $y = 5 - 2(2) = 1$. So $(x, y) = (2, 1)$, matching the crossing point in Figure 6.1. Substitution is the row picture made mechanical: we used equation one to pin y to x (slide along the first line), then forced the second line.

Example 6.2 (Combination, the column way)**.** We want weights x, y with $x\begin{bmatrix}2\\3\end{bmatrix}+y\begin{bmatrix}1\\2\end{bmatrix}=\begin{bmatrix}5\\8\end{bmatrix}$. Cancel the second column $\begin{bmatrix}1\\2\end{bmatrix}$ by taking 2 times the top coordinate minus 1 times the bottom coordinate (since $2 \cdot 1 - 1 \cdot 2 = 0$). Applied to both sides, $(2 \cdot 2 - 1 \cdot 3)x = 2 \cdot 5 - 1 \cdot 8$, and canceling the first column similarly gives

$$x = \frac{2 \cdot 5 - 1 \cdot 8}{2 \cdot 2 - 1 \cdot 3} = \frac{10 - 8}{4 - 3} = 2, \qquad y = \frac{2 \cdot 8 - 3 \cdot 5}{2 \cdot 2 - 1 \cdot 3} = \frac{16 - 15}{1} = 1.$$

Same answer, $x = 2$, $y = 1$. Notice the denominator $2 \cdot 2 - 1 \cdot 3 = 1$. That number is called the *determinant* of A (written $\det A$), because it literally *determines* whether the system has a clean, unique recipe. Back in Chapter 5, we viewed matrices as active machines transforming space. The determinant is the *area scaling factor* of that machine: it measures how much any 2D region expands or shrinks under the transformation (with a minus sign if the orientation flips). When this scaling factor is nonzero, the column arrows enclose real area, meaning they stay spread out and can reach any target $\mathbf{b}$ in the plane. When it is zero, the area has collapsed to zero – the matrix has squashed the entire 2D plane onto a 1D line – and trouble begins. We will construct determinants in full generality in Chapter 21, but in 2×2 the geometric meaning is already right here in front of us.

Compute it yourself

Solve $\begin{bmatrix}1 & 2\\ 1 & 3\end{bmatrix}\begin{bmatrix}x\\ y\end{bmatrix} = \begin{bmatrix}4\\ 5\end{bmatrix}$ **twice**: once by substitution (find the crossing point of the two lines) and once by combination (what multiples of $\begin{bmatrix}1\\ 1\end{bmatrix}$ and $\begin{bmatrix}2\\ 3\end{bmatrix}$ reach $\begin{bmatrix}4\\ 5\end{bmatrix}$?). You should get $x = 2,\ y = 1$ both ways. Then sketch both pictures on scrap paper. The goal here is to see how two different geometric constructions produce the same coordinates.

6.3 The three possibilities, and why there are only three

So far our lines crossed once and our columns reached the target once. But depending on the matrix and target vector, a system can also have *no* solution or *infinitely many*. These three outcomes are the only possibilities. Let us examine all three in the 2×2 setting and observe how both pictures reflect each outcome.

6.3.1 One solution: lines cross, columns spread

This is the case we just did. Two lines with *different slopes* cross at exactly one point. In the column picture, the two arrows $\mathbf{a}_1, \mathbf{a}_2$ point in *different directions*, so by scaling and adding them you can reach *any* target $\mathbf{b}$ in the plane – and in exactly one way.

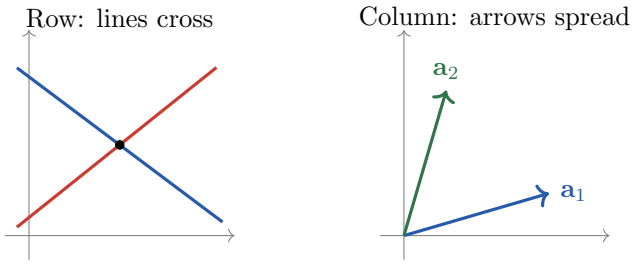


Figure 6.3: *Nonsingular* case. Different slopes / different directions. Exactly one solution for every $\mathbf{b}$.

6.3.2 No solution: parallel lines, collinear columns missing the target

Take a system whose two lines are *parallel but not equal*:

$$\begin{aligned} 1x + 2y &= 3, \\ 1x + 2y &= 7. \end{aligned} \tag{6.2}$$

The rows have the same left side, $x + 2y$, but demand two different values, 3 and 7. No point can make $x + 2y$ equal 3 and 7 at once. The two lines never meet – they are parallel. *No solution.*

Now flip to columns. The system is $x\begin{bmatrix}1\\ 1\end{bmatrix} + y\begin{bmatrix}2\\ 2\end{bmatrix} = \begin{bmatrix}3\\ 7\end{bmatrix}$. But $\begin{bmatrix}1\\ 1\end{bmatrix}$ and $\begin{bmatrix}2\\ 2\end{bmatrix}$ point the *same way* – the second is just twice the first. Every combination $x\begin{bmatrix}1\\ 1\end{bmatrix} + y\begin{bmatrix}2\\ 2\end{bmatrix} = (x + 2y)\begin{bmatrix}1\\ 1\end{bmatrix}$ lands somewhere on

the single line through $\begin{bmatrix}1\\1\end{bmatrix}$. But the target $\begin{bmatrix}3\\7\end{bmatrix}$ is *off* that line (it would need its two entries equal, and $3 \neq 7$). So no combination reaches it.

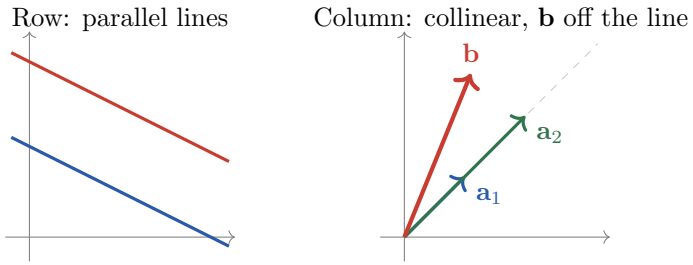


Figure 6.4: *No solution.* The lines are parallel; equivalently, the columns are collinear (both on the dashed line) and the target **b** does not lie on that line.

6.3.3 Infinitely many: same line twice, target on the column line

Keep the columns collinear, but now put the target *on* their line:

$$\begin{aligned} 1x + 2y &= 3, \\ 2x + 4y &= 6. \end{aligned} \tag{6.3}$$

The second equation is just twice the first – it is the *same line* written twice. Any point on the line $x + 2y = 3$ satisfies both. There are infinitely many such points: $x = 3 - 2t$, $y = t$ for every real t. In the column picture, $\begin{bmatrix}1\\2\end{bmatrix}$ and $\begin{bmatrix}2\\4\end{bmatrix}$ are collinear (the second is twice the first), and the target $\begin{bmatrix}3\\6\end{bmatrix}$ *is* on that line, so it can be reached – but in infinitely many ways, because you can trade amount of one column for amount of the other.

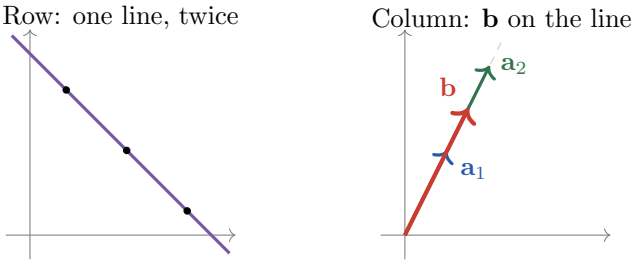


Figure 6.5: *Infinitely many solutions.* The two equations are the same line; the columns are collinear and **b** lies on their common line, so it is reachable in infinitely many ways.

Focus

Read off the pattern. Whether the system has 0, 1, or ∞ solutions is decided by two yes/no questions:

1. **Are the columns spread out** (point in different directions)? If *yes*, you can reach *every* **b**, uniquely: one solution, always.
2. If the columns have **collapsed** (collinear), then it depends on **b**: if **b** lies on their line,

infinitely many solutions; if not, *none*.

"Spread out vs. collapsed" is the heart of the matter. It has a name: *nonsingular* vs. *singular*.

6.3.4 Why never exactly two?

A student always asks: why can't a system have exactly two solutions, or seventeen? Here is the clean reason, and it is worth seeing once.

Proposition 6.1 (Zero, one, or infinity – never in between)**.** *If a linear system $A\mathbf{x} = \mathbf{b}$ has two distinct solutions, it has infinitely many.*

Proof. Suppose $\mathbf{p}$ and $\mathbf{q}$ are both solutions, $\mathbf{p} \neq \mathbf{q}$, so $A\mathbf{p} = \mathbf{b}$ and $A\mathbf{q} = \mathbf{b}$. Consider the whole *line* through them: for any real number t, set $\mathbf{x}_t = \mathbf{p} + t(\mathbf{q} - \mathbf{p})$. Then, using that A distributes over sums and pulls out scalars (linearity),

$$A\mathbf{x}_t = A\mathbf{p} + t\,A(\mathbf{q} - \mathbf{p}) = \mathbf{b} + t(A\mathbf{q} - A\mathbf{p}) = \mathbf{b} + t(\mathbf{b} - \mathbf{b}) = \mathbf{b}.$$

So *every* point $\mathbf{x}_t$ on the line is a solution, and since $\mathbf{q} - \mathbf{p} \neq \mathbf{0}$ these are all different. Two solutions force a whole line of them – hence infinitely many. □

Intuition

Geometrically: the solution set of a linear system is never a scattering of isolated dots. It is always "flat" – a single point, or a whole line, or a plane, or empty. Linearity forbids curved or finite-but-multiple solution sets. That flatness is exactly why the answer count is 0, 1, or ∞ and nothing else.

Pause

Pause and think: circles vs. lines. A straight line and a circle in the plane can cross at *two* distinct points. Why can't two equations in a linear system ever do that? Where in the proof of Proposition 6.1 did linearity rule out two dots? Notice where linearity entered: $A(\mathbf{p} + t(\mathbf{q} - \mathbf{p})) = A\mathbf{p} + tA(\mathbf{q} - \mathbf{p})$. Because the matrix machine acts linearly on weighted combinations, every single point along the entire line through $\mathbf{p}$ and $\mathbf{q}$ is forced to map to $\mathbf{b}$. A circle equation like $x^2 + y^2 = 1$ is *nonlinear* – it curves, so the points between two solutions fall off the circle. Linear equations are flat; they can never curve back to produce a finite handful of isolated dots.

6.4 Singular versus nonsingular: the determinant in 2D

We keep saying "spread out" and "collapsed." Let us make it a number. For a 2×2 matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

the *determinant* is $\det A = ad - bc$. As previewed in Chapter 5 and proved in full generality in Chapter 21, this algebraic expression computes the *signed area* of the parallelogram spanned by the

two column vectors. In our present story, it serves as the ultimate *collapse detector*: it tells us whether the matrix machine keeps the 2D plane spread out or squashes it flat.

Formal

The 2×2 matrix $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ is **nonsingular** (invertible) iff $\det A = ad - bc \neq 0$, and **singular** iff $\det A = 0$. When nonsingular, $A\mathbf{x} = \mathbf{b}$ has the unique solution

$$\mathbf{x} = A^{-1}\mathbf{b}, \qquad A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Let us see *why* $ad - bc$ is the right number, geometrically. The two columns of A are $\mathbf{a}_1 = \begin{bmatrix} a \\ c \end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix} b \\ d \end{bmatrix}$. These two arrows are the two sides of a parallelogram. The *area* of that parallelogram is exactly $|ad - bc|$. Now:

- If the columns point in different directions, the parallelogram has positive area, $ad - bc \neq 0$: *spread out*, nonsingular.
- If the columns are collinear, the parallelogram is squashed flat, area 0, so $ad - bc = 0$: *collapsed*, singular.

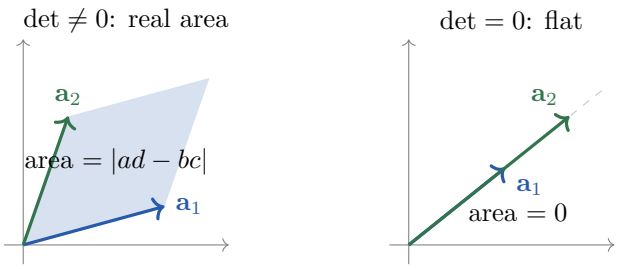


Figure 6.6: The determinant is the signed area of the column parallelogram. Nonzero area means the columns span the plane (one solution for every $\mathbf{b}$); zero area means they have collapsed to a line (the singular case).

Check our three examples against $ad - bc$:

- Flagship (6.1): $\det \begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix} = 2\cdot 2 - 1\cdot 3 = 1 \neq 0$. Nonsingular – one solution. ✓
- No-solution (6.2): $\det \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} = 1\cdot 2 - 2\cdot 1 = 0$. Singular – and here $\mathbf{b}$ missed the line, so zero solutions.
- Infinite (6.3): $\det \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = 1\cdot 4 - 2\cdot 2 = 0$. Singular – and here $\mathbf{b}$ sat on the line, so infinitely many.

The determinant alone separates "nonsingular" (always one solution) from "singular" (either zero or infinitely many, depending on $\mathbf{b}$). To decide between those last two you must look at $\mathbf{b}$; the

determinant cannot, because it knows only about A.

6.4.1 Deriving the 2×2 inverse formula

Where does $A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ come from? Let us solve $\begin{bmatrix} a & b \\ c & d \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} e \\ f \end{bmatrix}$ by standard elimination, assuming $ad - bc \neq 0$. The equations are

$$ax + by = e, \qquad cx + dy = f.$$

Multiply the first by d, the second by b, and subtract to eliminate y:

$$(ad - bc)x = de - bf \implies x = \frac{de - bf}{ad - bc}.$$

Multiply the first by c, the second by a, and subtract to eliminate x:

$$(bc - ad)y = ce - af \implies y = \frac{af - ce}{ad - bc}.$$

Stacking these,

$$\begin{bmatrix} x \\ y \end{bmatrix} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}\begin{bmatrix} e \\ f \end{bmatrix},$$

which is precisely $\mathbf{x} = A^{-1}\mathbf{b}$. The formula is simply two lines of elimination with the division by $ad - bc$ performed at the end. When $ad - bc = 0$, this involves division by zero. Singular matrices have no inverse because dividing by zero is undefined.

Common trap

"$\det A = 0$ means no solution" is *wrong*, and it is a trap students fall into constantly. $\det A = 0$ means the unique-solution machine breaks; the system then has *either* no solution *or* infinitely many. You must inspect $\mathbf{b}$ (does it lie in the span of the columns?) to know which. Nonsingular is a property of A alone; the zero-versus-infinity split needs $\mathbf{b}$.

Connection to other math

The quantity $ad - bc$ is your first determinant, and determinants reach far beyond linear algebra. In multivariable *calculus*, the Jacobian determinant is exactly this area factor: it tells you how a change of variables stretches or squashes little area elements, which is why $dx\,dy = |\det J|\,du\,dv$ appears in every change-of-variables integral. A coordinate change is "nonsingular" (locally invertible) precisely where its Jacobian determinant is nonzero – the same spread-out-versus-collapsed dichotomy you are learning here, now applied to infinitesimal boxes.

6.5 Up to three dimensions: planes instead of lines

Everything generalizes cleanly to three unknowns; only the cast of characters grows. An equation like $ax + by + cz = d$ in three unknowns is no longer a line – it is a *plane* in $\mathbb{R}^3$. A system of three such equations asks for a point lying on all three planes at once.

Intuition

In $\mathbb{R}^3$, each linear equation is a plane. The row picture of a 3×3 system is *three planes*, and a solution is a point common to all three. Generically three planes meet at a single point – the corner of a room where two walls and the floor come together.

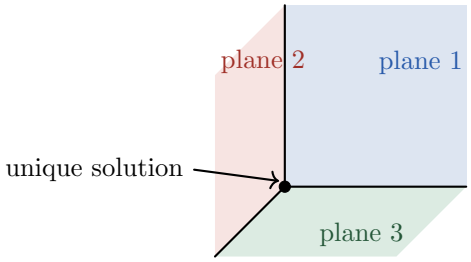


Figure 6.7: The row picture in $\mathbb{R}^3$: three planes meeting at one point, like two walls and the floor meeting at the corner of a room. That corner is the unique solution.

The column picture also survives intact, and arguably gets *more* useful. A 3×3 system

$$A\mathbf{x} = \mathbf{b}, \qquad x \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \end{bmatrix} + y \begin{bmatrix} a_{12} \\ a_{22} \\ a_{32} \end{bmatrix} + z \begin{bmatrix} a_{13} \\ a_{23} \\ a_{33} \end{bmatrix} = \mathbf{b},$$

asks: can the target arrow $\mathbf{b} \in \mathbb{R}^3$ be built as a combination of the three column arrows? If the three columns are *spread out* – not lying in any common plane – they span all of $\mathbb{R}^3$, and every $\mathbf{b}$ is reachable in exactly one way. That is the nonsingular case.

6.5.1 A worked 3×3 with a clean answer

Example 6.3 (Three planes, one corner)**.** Solve

$$\begin{aligned} x + y + z &= 6, \\ 2y + z &= 5, \\ 2z &= 4. \end{aligned}$$

This system is already in "triangular" shape, so we can read it from the bottom up – *back-substitution*. The last equation gives $2z = 4$, so $z = 2$. Plug into the middle: $2y + 2 = 5$, so $y = \frac{3}{2}$. Plug both into the top: $x + \frac{3}{2} + 2 = 6$, so $x = \frac{5}{2}$. The unique solution is $(x, y, z) = \left(\frac{5}{2}, \frac{3}{2}, 2\right)$.

The determinant of a triangular matrix is the product of its diagonal, $1 \cdot 2 \cdot 2 = 4 \neq 0$, so the system is nonsingular – three planes truly crossing at the single corner $\left(\frac{5}{2}, \frac{3}{2}, 2\right)$. Triangular systems are the reward at the end of Gaussian elimination, which is the subject of the next chapter; here we simply enjoy how easy they are.

6.5.2 How a 3×3 system can fail

In $\mathbb{R}^3$ the ways to fail are richer, but they still sort into "no solution" and "infinitely many," both flavors of *singular*.

No solution. Three planes can fail to share a common point in several shapes – two of them parallel, or three forming a triangular prism (each pair meets in a line, but the three intersection lines run parallel like the three edges of a triangular prism, so no single point lies on all three). Algebraically, elimination produces a contradiction such as $0 = 1$.

Infinitely many. If the three planes share a whole *line* (think of three pages of a book hinged on the same spine), every point on that line solves the system: a line's worth of solutions. If they all coincide as the *same* plane, you get a plane's worth. Elimination reveals this as a "free variable" you can set to anything.

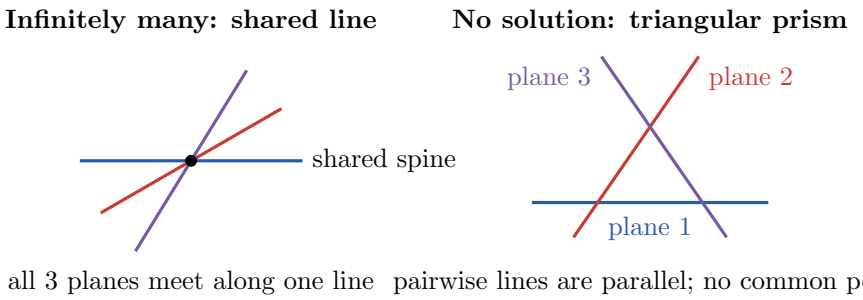


Figure 6.8: Two ways three planes fail to meet at a single corner, seen edge-on. *Left:* all three planes hinge along the same line (the spine of an open book), giving a whole line of solutions (infinitely many). *Right:* the planes form a triangular prism; each pair intersects in a line, but the three lines are parallel and never meet, leaving no point common to all three (no solution).

Focus

The $0/1/\infty$ trichotomy is dimension-independent. In $\mathbb{R}^n$:

- **Nonsingular** ($\det A \neq 0$, columns span $\mathbb{R}^n$): exactly one solution for *every* $\mathbf{b}$.
- **Singular** ($\det A = 0$, columns trapped in a smaller flat): *either* no solution (when $\mathbf{b}$ escapes that flat) *or* infinitely many (when $\mathbf{b}$ lies inside it).

Same story as 2×2, with "line" upgraded to "plane" and "parallelogram" to "parallelepiped" whose volume is $\det A$.

6.6 The homogeneous shadow: $A\mathbf{x} = \mathbf{0}$

There is a special right-hand side that controls all the others: the zero vector. The system $A\mathbf{x} = \mathbf{0}$ is called *homogeneous* (meaning all equations are balanced to zero on the right). It always has at least the obvious solution $\mathbf{x} = \mathbf{0}$ (the input that does nothing lands at the origin). The real question is whether it has *nontrivial* solutions – nonzero inputs that the matrix machine nevertheless crushes down to the origin $\mathbf{0}$.

Theorem 6.1 (Singular $\Leftrightarrow$ nonzero solutions to $A\mathbf{x} = \mathbf{0}$)**.** *A square matrix A is singular if and only if $A\mathbf{x} = \mathbf{0}$ has a solution $\mathbf{x} \neq \mathbf{0}$.*

Sketch. If A is nonsingular it has an inverse, so $\mathbf{x} = A^{-1}\mathbf{0} = \mathbf{0}$ is the *only* solution – no nontrivial one exists. Conversely, if the columns of A are collapsed (singular), some nontrivial combination of them

equals $\mathbf{0}$; the weights of that combination form an $\mathbf{x} \neq \mathbf{0}$ with $A\mathbf{x} = \mathbf{0}$. For our flagship $A = \begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}$ (nonsingular), only $\mathbf{0}$ works. For the collapsed $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, the combination $2\begin{bmatrix} 1 \\ 2 \end{bmatrix} - 1\begin{bmatrix} 2 \\ 4 \end{bmatrix} = \mathbf{0}$ exposes $\mathbf{x} = \begin{bmatrix} 2 \\ -1 \end{bmatrix} \neq \mathbf{0}$. □

This is the bridge between the row and column pictures and the rest of the book. The set of all input vectors that the matrix machine squashes down into the zero vector $\mathbf{0}$ is called the *null space* (or kernel) of A – literally, the collection of inputs annihilated to zero. We will study the null space as a vector subspace in Chapter 12 (for now, picture it as the directions that get flattened to a point). When A is nonsingular, the machine preserves the full space: only $\mathbf{0}$ maps to $\mathbf{0}$, so the null space contains nothing except the origin $\{\mathbf{0}\}$, often called the *trivial* null space. But when A is singular, the machine flattens at least one dimension: a whole line, plane, or higher-dimensional flat of nonzero vectors is crushed into the origin. This connects back to Proposition 6.1:

Why this matters

Every solution of $A\mathbf{x} = \mathbf{b}$ has the shape

$$\mathbf{x} = \underbrace{\mathbf{x}_{\text{particular}}}_{\text{one solution}} + \underbrace{\mathbf{x}_{\text{null}}}_{\text{anything killed by } A}.$$

The particular part gets you to $\mathbf{b}$; the null-space part is the freedom to wander without changing where you land. If the null space is just $\{\mathbf{0}\}$ (nonsingular), there is no freedom – one solution. If the null space is a line or a plane (singular), that same flat shape is copied onto the solution set, giving infinitely many. *The geometry of the solutions is the geometry of the null space, shifted.* This one decomposition quietly organizes the chapters to come.

Figure 6.9 draws this for a singular system whose null space is a line: the full solution set is that same line, picked up and slid over so it passes through one particular solution $\mathbf{x}_{\text{particular}}$.

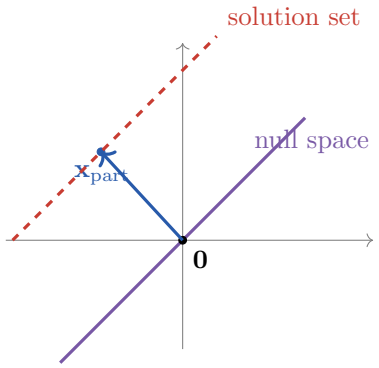


Figure 6.9: Solution set = null space, shifted. The solid purple line is the null space (all $\mathbf{x}$ with $\mathbf{Ax} = \mathbf{0}$). The dashed red line is the full solution set of $\mathbf{Ax} = \mathbf{b}$: the *same direction*, slid over to pass through one particular solution $\mathbf{x}_{\text{part}}$. A trivial null space (just $\{\mathbf{0}\}$) shifts to a single solution.

Connection to other math

This "particular plus homogeneous" shape is not special to matrices – it is the signature of *any* linear operator. In a first course on *differential equations* you meet exactly the same theorem: the general solution of $y'' + y = \cos t$ is one particular solution (say $y_p = \frac{t}{2}\sin t$) plus the general solution of the homogeneous equation $y'' + y = 0$ (here $c_1 \cos t + c_2 \sin t$). The homogeneous solutions play the role of the null space, and the particular solution plays the role of $\mathbf{x}_{\text{part}}$. The map "apply the differential operator $L = \frac{d^2}{dt^2} + 1$" is linear in precisely the sense $L(\alpha f + \beta g) = \alpha Lf + \beta Lg$, so the proof is word-for-word the one we just gave for $A\mathbf{x} = \mathbf{b}$. Once you see that solution sets of all linear problems are "a null space, shifted," the two subjects fuse.

6.7 A modern application: when "solve $A\mathbf{x} = \mathbf{b}$" is the whole job

Modern application

Networks, circuits, and the structure under deep learning. Solving linear systems is the main computational step in many areas of science and engineering.
Circuits and networks. Kirchhoff's laws for an electrical network – "current in equals current out at every node, voltage drops around every loop sum to zero" – are a linear system $A\mathbf{x} = \mathbf{b}$ whose unknowns are the currents. The *same* matrix shape (a graph's incidence matrix) governs water in pipes, traffic on roads, and forces in a bridge truss. Engineers solving for "will this bridge stand?" are, underneath, asking whether a structural stiffness matrix is nonsingular and what $A^{-1}\mathbf{b}$ says about deflections.
Computer graphics. Every time a 3D scene is rendered, the engine solves small linear systems to find where a ray pierces a triangle, or to blend bone transforms for an animated character. The "does the ray hit?" test is exactly a 3×3 singular-versus-nonsingular check: a singular system means the ray is parallel to the surface and grazes by.
Machine learning. Fitting a linear model – the regression that underlies a shocking amount of applied ML – is set up as $A\mathbf{x} = \mathbf{b}$, where rows are data examples and $\mathbf{x}$ is the vector of weights. Usually there are far more equations than unknowns and *no exact solution exists* (the $\mathbf{b}$ of real data does not lie in the column space). The fix, *least squares*, finds the $\mathbf{x}$ that gets closest – it asks "what is the nearest reachable point to $\mathbf{b}$ in the span of the columns?" That question is the column picture of this chapter, asked when the target is just out of reach. Even the giant nonlinear models you hear about are trained by repeatedly solving (or approximately solving) linear systems for the update direction. The trichotomy you learned today – spread out vs. collapsed columns – reappears as "is this problem well-conditioned, or has the data collapsed onto a lower-dimensional flat?" That collapse, called multicollinearity, is one of the most common ways a real-world model goes haywire.

Figure 6.10 shows the least-squares idea in one picture. The reachable targets – all combinations of the columns – fill a plane (the column space). When the data vector $\mathbf{b}$ floats *off* that plane, no exact solution exists, so we drop a perpendicular and solve for the nearest reachable point $\hat{\mathbf{b}}$ instead. That "drop a perpendicular onto the column space" is the whole of linear regression in one move; we make it precise when we reach projections.

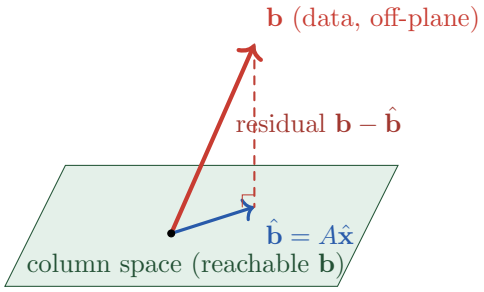


Figure 6.10: Least squares as the column picture with the target out of reach. The reachable vectors form a flat (the column space). When $\mathbf{b}$ lies off it, we solve $A\mathbf{x} = \hat{\mathbf{b}}$ for the nearest point $\hat{\mathbf{b}}$, the perpendicular foot. The dashed residual is what regression makes as small as possible.

6.8 Building intuition that lasts

Walking question

Take a walk and carry exactly this image. You are handed two arrows on the ground, $\mathbf{a}_1$ and $\mathbf{a}_2$, and a flag planted at point $\mathbf{b}$. Your job: reach the flag by walking some amount along $\mathbf{a}_1$ and some amount along $\mathbf{a}_2$ (backwards is allowed). Ask yourself:

- If the two arrows point in genuinely different directions, can you always reach *any* flag? In how many ways?
- If the two arrows point the *same* way, which flags can you reach, and which are forever out of reach?

When you can answer these without paper, you understand the column picture – and therefore the existence-and-uniqueness theory of linear systems – better than most people who have passed the course.

Remark 6.1 (Two pictures, one habit). Whenever you encounter $A\mathbf{x} = \mathbf{b}$, switch fluently between the two pictures. To determine if a solution *exists*, examine the columns: is $\mathbf{b}$ in their span? To *compute* the numerical solution, work with the rows using elimination. To determine if the solution is *unique*, check whether the columns are linearly independent ($\det A \neq 0$).

6.9 Summary

- A linear system $A\mathbf{x} = \mathbf{b}$ has **two geometric readings**. The *row picture* sees each equation as a line (in $\mathbb{R}^2$) or plane (in $\mathbb{R}^3$); a solution is a common intersection point. The *column picture* sees $A\mathbf{x}$ as a linear combination of the columns; a solution is a recipe of weights reaching $\mathbf{b}$.
- Compute $A\mathbf{x}$ two ways: *rows dotted with* $\mathbf{x}$, or *a combination of the columns*. Both are the same product.
- Exactly three outcomes are possible: **one**, **none**, or **infinitely many** solutions. Two distinct solutions force a whole line of them (Proposition 6.1).
- **Nonsingular** ($\det A \neq 0$): columns spread out / span the space; one solution for every $\mathbf{b}$;

$\mathbf{x} = A^{-1}\mathbf{b}$. **Singular** ($\det A = 0$): columns collapsed onto a smaller flat; then *no* solution if $\mathbf{b}$ escapes that flat, *infinitely many* if $\mathbf{b}$ lies inside it.

- The determinant $ad - bc$ (in 2×2) is the signed area of the column parallelogram: zero area = collapse = singular.
- The homogeneous system $A\mathbf{x} = \mathbf{0}$ measures the freedom: its nonzero solutions (the null space) are exactly the wiggle room that turns one solution into infinitely many.

Reading note

Cost and intuition recap. The 2×2 inverse formula costs a handful of multiplications, and Cramer-style formulas exist in every dimension – but their cost explodes (an $n \times n$ determinant by the naive expansion costs on the order of $n!$ operations, hopeless past $n \approx 10$). The practical route is *elimination*, which solves $A\mathbf{x} = \mathbf{b}$ in about $\frac{1}{3}n^3$ operations – the central algorithm of the next chapter. So carry two things forward: the geometric intuition (rows are lines/planes, columns are arrows, singular means collapsed) and the computational awareness that the method of solution matters greatly as dimension n increases.

History and connection

The systematic solution of linear systems is one of the oldest mathematical practices in human history. Long before European algebraists manipulated abstract symbols, Chinese civil servants in the Han Dynasty (compiled around 150 BCE by Zhang Cang and Geng Shoushan) recorded the method in *Jiuzhang Suanshu* (The Nine Chapters on the Mathematical Art). In Chapter 8, titled *Fangcheng* ("The Rectangular Array"), the authors solved a system of three linear equations in three agricultural unknowns (bundles of good, medium, and poor grain) by laying out the coefficients on a counting board using bamboo rods.

The Han mathematicians arranged equations into columns on the board and used red bamboo rods for positive quantities (surpluses) and black rods for negative quantities (deficits). This stands as the earliest recorded use of negative numbers in mathematical history. It also sparked one of mathematics' longest historical controversies: while Han scholars operated with negative numbers naturally as debts and deficits, European mathematicians resisted them for another 1,500 years. As late as the sixteenth and seventeenth centuries, Girolamo Cardano called negative solutions "fictitious" and René Descartes labeled them "false roots" in his algebra. By systematically subtracting multiples of one column from another, the Han mathematicians reduced their counting board to triangular form and read off the answers by back-substitution. For centuries in Europe, linear equations remained pure algebraic drudgery – tedious variable cancellation on paper. In the eighteenth century, Colin Maclaurin and Gabriel Cramer introduced determinant formulas (*Cramer's rule*, expressing each unknown as a ratio of determinants). It seemed like a theoretical triumph, but it concealed a computational trap: evaluating an $n \times n$ determinant by standard permutation expansion takes on the order of $n!$ operations (where each step multiplies all n choices together). For a modest 25×25 system, $25! \approx 1.5 \times 10^{25}$ operations would outlast the age of the universe on any modern supercomputer. When Carl Friedrich Gauss needed to calculate the orbit of the newly discovered asteroid Ceres in 1801 from a tiny handful of noisy telescope sightings, he abandoned determinants and revived systematic elimination – reducing the cost to roughly $\frac{1}{3}n^3$ operations and establishing the foundation of computational astronomy.

When geometry entered the subject, textbooks almost exclusively taught the *row picture*:

lines crossing at a point, or planes meeting at a corner. Yet the row picture breaks down in higher dimensions: human vision can picture three planes in 3D, but nobody can visualize four *hyperplanes* (flat three-dimensional slices of four-dimensional space) crossing in $\mathbb{R}^4$, let alone fifty hyperplanes in $\mathbb{R}^{50}$. The modern perspective championed by Gilbert Strang elevates the **column picture**:

$$x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n = \mathbf{b}.$$

In the column picture, the geometry never changes as the dimension grows. Whether in $\mathbb{R}^2$ or $\mathbb{R}^{1000}$, you are always asking the exact same picture-first question: can you scale the given arrows $\mathbf{a}_i$ and link them tip-to-tail to reach the target arrow $\mathbf{b}$? Solvability is reachability in the span of the columns.

What remains open? While Gaussian elimination reliably solves an $n \times n$ system in $O(n^3)$ operations, mathematicians in 1969 were surprised when Volker Strassen proved that $O(n^3)$ is not the optimal speed limit: he discovered a recursive algorithm that multiplies and inverts matrices in $O(n^{2.807})$ operations. Today, decades of research (by Don Coppersmith, Shmuel Winograd, Virginia Vassilevska Williams, Josh Alman, and others) have pushed the matrix multiplication and inversion exponent ω below 2.372. The open question in computational linear algebra is whether $\omega = 2$ – that is, whether solving any $n \times n$ system can ultimately run in near-$O(n^2)$ time, the time it takes just to read the matrix. Elimination begins with the classic engine in Chapter 7 (Section 7.1).

6.10 Exercises

Exercise 6.1 (Mental warm-ups)**.** Answer each question in your head before touching paper:

(a) A system has 4 equations and 2 unknowns. In the column picture, can the two column arrows ever span all of four-dimensional space $\mathbb{R}^4$? What does this tell you about whether a random target $\mathbf{b} \in \mathbb{R}^4$ has an exact solution?

(b) If a 2×2 matrix A has $\det A = 0$, does $A\mathbf{x} = \mathbf{b}$ necessarily have no solution? Explain in one sentence.

(c) If $A\mathbf{x} = \mathbf{0}$ has *only* the trivial solution $\mathbf{x} = \mathbf{0}$, can $A\mathbf{x} = \mathbf{b}$ ever have two distinct solutions? Why or why not?

Exercise 6.2 (Both pictures, by hand)**.** For the system $3x + y = 9$, $x + 2y = 8$: (a) draw the row picture (two lines) and read off the crossing point; (b) write it as $x\begin{bmatrix}3\\1\end{bmatrix} + y\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}9\\8\end{bmatrix}$ and find the weights; (c) confirm both answers agree.

Exercise 6.3 (The determinant decides)**.** Compute $\det A$ for each matrix and state, *without solving*, whether $A\mathbf{x} = \mathbf{b}$ is guaranteed to have a unique solution for every $\mathbf{b}$:

$$\begin{bmatrix}4 & 2\\1 & 3\end{bmatrix}, \qquad \begin{bmatrix}2 & 6\\1 & 3\end{bmatrix}, \qquad \begin{bmatrix}0 & 5\\-2 & 0\end{bmatrix}.$$

For any singular one, find a $\mathbf{b}$ giving no solution and a $\mathbf{b}$ giving infinitely many.

Exercise 6.4 (Make it fail two ways)**.** Start from $x + 3y = 4$. Add a *second* equation so that the system has (a) no solution; (b) infinitely many solutions; (c) exactly one solution. For each, name the geometric relationship between the two lines.

Exercise 6.5 (Column reach)**.** The columns of A are $\mathbf{a}_1 = \begin{bmatrix}1\\2\end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix}2\\4\end{bmatrix}$. (a) Describe, as a set, all vectors $\mathbf{b}$ for which $A\mathbf{x} = \mathbf{b}$ has a solution. (b) For $\mathbf{b} = \begin{bmatrix}3\\6\end{bmatrix}$, give *two* different solutions $\mathbf{x}$, and then describe *all* of them in the form $\mathbf{x}_{\text{particular}} + t\,\mathbf{x}_{\text{null}}$ (where $A\mathbf{x}_{\text{null}} = \mathbf{0}$).

Exercise 6.6 (Back-substitution in $\mathbb{R}^3$)**.** Solve, top-triangular style,

$$2x + y - z = 3, \qquad 3y + 2z = 4, \qquad 4z = 8,$$

and verify your point satisfies all three equations. What is det of the coefficient matrix, and why does that guarantee uniqueness?

Exercise 6.7 (Three planes, no corner)**.** Consider $x + y + z = 1$, $x + y + z = 2$, $x - y = 0$. Show by inspection that the system has no solution, and explain in words what the three planes are doing (which two are parallel?).

Exercise 6.8 (Derive your own inverse)**.** Without quoting the formula, solve $ax+by = e$, $cx+dy = f$ by elimination to re-derive $x = \frac{de-bf}{ad-bc}$ and $y = \frac{af-ce}{ad-bc}$. At which step does the assumption $ad-bc \neq 0$ become essential, and what goes wrong if it fails?

Exercise 6.9 (Zero, one, or infinity – a proof)**.** Suppose $A\mathbf{p} = \mathbf{b}$ and $A\mathbf{q} = \mathbf{b}$ with $\mathbf{p} \neq \mathbf{q}$. Prove that $\mathbf{p} + t(\mathbf{q} - \mathbf{p})$ solves $A\mathbf{x} = \mathbf{b}$ for *every* real t, citing the two linearity rules you use. Conclude that a linear system cannot have exactly two solutions.

Exercise 6.10 (The homogeneous shadow)**.** For $A = \begin{bmatrix}1 & 2 & 3\\2 & 4 & 6\\1 & 1 & 1\end{bmatrix}$: (a) notice that row 2 is exactly twice row 1, meaning the second equation provides no new constraint beyond the first. Use this geometric redundancy to explain why the three planes cannot intersect at a single unique corner, and conclude that A must be singular. (b) Now find a nonzero $\mathbf{x}$ with $A\mathbf{x} = \mathbf{0}$. (Hint: a nonzero vector with $A\mathbf{x} = \mathbf{0}$ records a combination of the *columns* that cancels out to zero; look for weights x_1, x_2, x_3, not all zero, with $x_1\begin{bmatrix}1\\2\\1\end{bmatrix} + x_2\begin{bmatrix}2\\4\\1\end{bmatrix} + x_3\begin{bmatrix}3\\6\\1\end{bmatrix} = \mathbf{0}$. You can check that $\mathbf{x} = \begin{bmatrix}1\\-2\\1\end{bmatrix}$ works.) (c) What does the existence of this $\mathbf{x}$ tell you about the number of solutions of $A\mathbf{x} = \mathbf{b}$ *when a solution exists at all*?

Exercise 6.11 (Challenge: a parametric crossing)**.** Consider $x + y = 1$, $x + ky = 2$, where k is a real parameter. (a) For which value(s) of k is the coefficient matrix singular? (b) For that k, does the system have no solution or infinitely many? (c) For all other k, solve for (x, y) in terms of k and describe what happens to the solution geometrically as $k \to 1$. (Watch the crossing point fly off to infinity as the lines become parallel.)

Chapter 7

Gaussian Elimination

In the previous chapter, we saw linear systems $\mathbf{Ax} = \mathbf{b}$ through two geometric lenses: intersecting hyperplanes in the row picture, and combining column arrows in the column picture. Both tell us what a solution means, but neither gives a practical way to compute it when a system has ten or a thousand equations.

We need a systematic method: *Gaussian elimination*. The strategy is simple: add multiples of one equation to another to eliminate variables until the system is triangular, then solve from the bottom up by back-substitution. Start, as always, with a picture.

7.1 The picture: collapsing planes onto a corner

Recall the two geometric faces of a linear system. In the row picture, each equation is a constraint that cuts down the space of candidates. In two dimensions, two non-parallel lines cross at a single point; in three dimensions, three tilted planes meet at a point. Solving the system means *pinning down that shared point*.

Gaussian elimination finds that point through a direct strategy: **we replace the tilted equations with simpler, axis-aligned equations that have the exact same solution, until the coordinates can be read off one by one.**

In the row picture, each step gently swings a tilted plane until it becomes parallel to a coordinate plane, without ever budging the intersection point. In the column picture, that same step introduces zeros into the column vectors, uncoupling the variables so that the final column alone accounts for the bottom coordinate of $\mathbf{b}$, letting us solve the weights in reverse order.

Intuition

Imagine three tilted planes in space crossing at a single hidden point. You cannot see where they meet because the planes are at awkward angles. Gaussian elimination gently rotates and tilts the equations – without ever moving the crossing point – until one plane becomes perfectly horizontal (it fixes z outright), the next pins down y once z is known, and the last pins down x once y and z are known. The crossing point never budges. We have only changed our *description* of it, from three vague clues into three sharp ones.

The key guarantee is that the crossing point never moves. Everything we do will be a *solution-preserving* operation: it changes the equations but not their common solution. That claim is the foundation everything else rests on, so let us pin it down.

Figure 7.1 shows the simplest case – two lines in the plane – before and after one elimination step. The replacement move swings the second line into a horizontal one (it now constrains only y), and the crossing point stays exactly where it was.

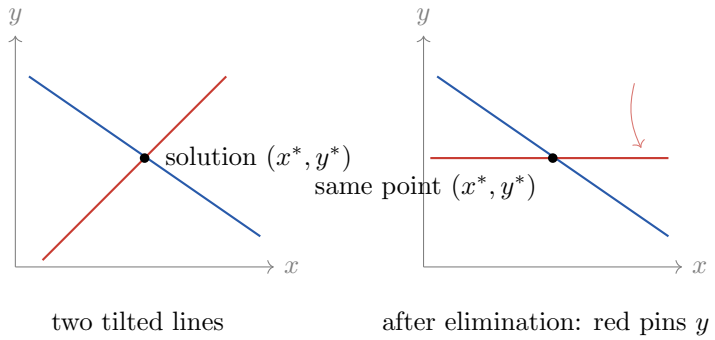


Figure 7.1: Elimination in the plane. One replacement step swings the red line into a horizontal one that fixes y outright, while the crossing point (the solution) does not move. Back-substitution then reads x off the blue line.

7.1.1 The three legal moves

We are allowed exactly three operations on the equations of a system. Each one is reversible, and each one leaves the set of solutions completely unchanged.

Focus

The three elementary row operations.

1. **Scaling:** multiply an equation by a nonzero number.
2. **Replacement:** add a multiple of one equation to another equation.
3. **Exchange:** swap the positions of two equations.

Every one of these is reversible, so none of them can lose or invent solutions.

Why do these preserve the solution set? Consider row replacement. Suppose a point (x, y, z) satisfies equation E_1 and equation E_2. Then it certainly satisfies $E_2 + cE_1$ for any number c, because both sides of E_1 are equal at that point, so adding c times a true equation to another true equation keeps it true. Conversely, if a point satisfies E_1 and $E_2 + cE_1$, we can *recover* E_2 as $(E_2 + cE_1) - cE_1$, so the point satisfies E_2 too. The two systems have identical solution sets. Scaling by a nonzero number and swapping are reversible for the obvious reasons. (Scaling by zero is forbidden precisely because it is *not* reversible – it would erase an equation.)

Common trap

The forbidden move is multiplying an equation by **zero**. "$0 = 0$" is true everywhere, so it throws away the information that equation carried, and you can never get it back. Every legal

move must be undoable. Keep that test in your head: if you could not reverse it, you may not do it.

7.1.2 From equations to a grid of numbers

Writing x, y, z and plus signs over and over is a waste of ink. The variables just hold places; what matters are the coefficients. So we strip the system down to its numbers and stack them in a rectangular grid. The system

$$\begin{aligned} 2x + 1y + 1z &= 5 \\ 4x + 1y + 0z &= 6 \\ -2x + 2y + 1z &= 1 \end{aligned} \qquad \longleftrightarrow \qquad \left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 4 & 1 & 0 & 6 \\ -2 & 2 & 1 & 1 \end{array}\right].$$

The vertical bar separates the coefficient matrix $\mathbf{A}$ from the right-hand side $\mathbf{b}$. The whole grid is the *augmented matrix*, written $[\,\mathbf{A} \mid \mathbf{b}\,]$. Each row is one equation; each of the first three columns belongs to one variable. The three row operations now become operations on rows of numbers – which is exactly what a computer wants.

7.2 Elimination to upper-triangular form

The goal of the elimination phase is to make the coefficient matrix *upper triangular*: every entry below the main diagonal becomes zero. Why that shape? Because an upper-triangular system is a staircase you can climb from the bottom. The last equation involves only the last variable, so you solve it immediately. The second-to-last involves only the last two, and you already know one of them. And so on up to the top. This climb is called *back-substitution*, and it solves line by line without guesswork once the triangle is built. Figure 7.2 shows the shape we are driving toward: zeros below the diagonal, pivots on it.

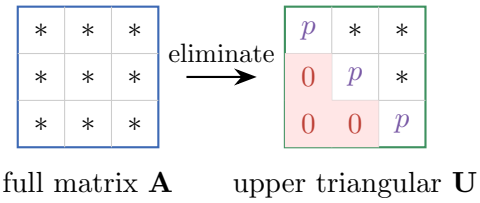


Figure 7.2: Elimination drives every entry below the diagonal to zero (shaded region). The surviving diagonal entries marked p are the *pivots*.

7.2.1 Pivots and multipliers

The mechanics are simple. We work column by column, left to right. In each column we pick a *pivot*: the diagonal entry we will use to knock out everything below it. To clear an entry below the pivot, we subtract the right multiple of the pivot's row.

Focus

To eliminate the entry in row i, column j, using the pivot in row j, column j, the **multiplier** is

$$\ell_{ij} = \frac{\text{entry to kill}}{\text{pivot}} = \frac{a_{ij}}{a_{jj}},$$

and the operation is: *new row* i = *old row* $i - \ell_{ij} \times$ (pivot row j). Choosing ℓ_{ij} this way makes the target entry become exactly $a_{ij} - \ell_{ij}\, a_{jj} = a_{ij} - a_{ij} = 0$. Hold onto these multipliers; they encode the entire elimination and will reappear in Chapter 9 (Section 9.1) as the entries of the matrix $\mathbf{L}$.

Pause

Pause and think: Suppose you reach the second row and find a zero in its pivot position. Could you fix it by adding a multiple of row 1? What would happen to column 1 if you did?

The answer is no: adding row 1 would dump nonzero entries right back into column 1 of row 2, destroying the zero just created. Previously computed zeros must be preserved. That is why the algorithm always looks *below* for pivots, never above.

7.2.2 A 3x3 traced fully

Let us run the example above all the way through. Watch every multiplier.

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 4 & 1 & 0 & 6 \\ -2 & 2 & 1 & 1 \end{array}\right].$$

The first pivot is the 2 in the top-left corner. We use it to clear the rest of column 1.

Step 1. Kill the 4 in row 2. Multiplier $\ell_{21} = 4/2 = 2$. Replace row 2 with (row 2) $-2\cdot$(row 1):

$$[\,4,\ 1,\ 0 \mid 6\,] - 2[\,2,\ 1,\ 1 \mid 5\,] = [\,0,\ -1,\ -2 \mid -4\,].$$

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 0 & -1 & -2 & -4 \\ -2 & 2 & 1 & 1 \end{array}\right].$$

Step 2. Kill the -2 in row 3. Multiplier $\ell_{31} = -2/2 = -1$. Replace row 3 with (row 3) $-(-1)\cdot$(row 1) = (row 3) + (row 1):

$$[\,-2,\ 2,\ 1 \mid 1\,] + [\,2,\ 1,\ 1 \mid 5\,] = [\,0,\ 3,\ 2 \mid 6\,].$$

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 0 & -1 & -2 & -4 \\ 0 & 3 & 2 & 6 \end{array}\right].$$

Column 1 is clean. Now we move to the second pivot, the -1 now sitting in row 2, column 2, and use it to clear column 2 below.

Step 3. Kill the 3 in row 3. Multiplier $\ell_{32} = 3/(-1) = -3$. Replace row 3 with (row 3) $-(-3)\cdot$(row 2) = (row 3) $+3\cdot$(row 2):

$$[\,0,\ 3,\ 2 \mid 6\,] + 3[\,0,\ -1,\ -2 \mid -4\,] = [\,0,\ 0,\ -4 \mid -6\,].$$

$$\mathbf{U} = \left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 0 & -1 & -2 & -4 \\ 0 & 0 & -4 & -6 \end{array}\right].$$

We have reached upper-triangular form. The three pivots are 2, -1, -4, all sitting on the diagonal. The forward sweep is done.

7.2.3 Back-substitution: climbing the staircase

Now read the triangle from the bottom up. The last row says

$$-4z = -6 \quad \Longrightarrow \quad z = \tfrac{3}{2}.$$

The middle row says $-y - 2z = -4$. Plug in $z = 3/2$:

$$-y - 2 \cdot \tfrac{3}{2} = -4 \implies -y - 3 = -4 \implies -y = -1 \implies y = 1.$$

The top row says $2x + y + z = 5$. Plug in $y = 1$ and $z = 3/2$:

$$2x + 1 + \tfrac{3}{2} = 5 \implies 2x = \tfrac{5}{2} \implies x = \tfrac{5}{4}.$$

So the three tilted planes meet at the single point $\left(\frac{5}{4}, 1, \frac{3}{2}\right)$. You should always sanity-check the original equations. The first: $2 \cdot \frac{5}{4} + 1 + \frac{3}{2} = \frac{5}{2} + 1 + \frac{3}{2} = 5$. Good. The second: $4 \cdot \frac{5}{4} + 1 + 0 = 5 + 1 = 6$. Good. The third: $-2 \cdot \frac{5}{4} + 2 \cdot 1 + \frac{3}{2} = -\frac{5}{2} + 2 + \frac{3}{2} = 1$. Good.

Figure 7.3 shows this triangular climb in action. Once the zeros are locked in place below the diagonal, the bottom equation hands you the last variable immediately. Substituting that value upward determines the next variable, and so on up the triangular system until the entire solution vector is assembled.

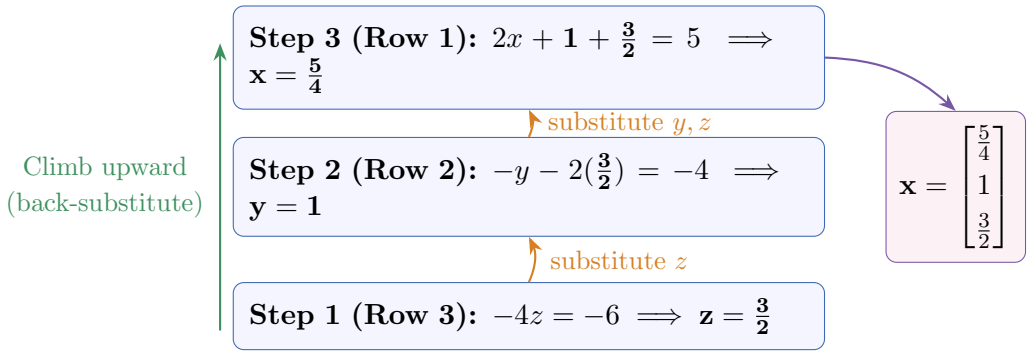


Figure 7.3: Back-substitution climbs the triangular system from the bottom up. Row 3 fixes z immediately. Feeding $z = \frac{3}{2}$ into Row 2 isolates $y = 1$. Feeding both into Row 1 isolates $x = \frac{5}{4}$, assembling the complete solution vector $\mathbf{x}$.

Focus

The two phases. *Forward elimination* reduces the matrix to an upper triangle $\mathbf{U}$, recording multipliers as it goes. *Back- substitution* then climbs the triangle from the bottom, solving one variable at each step. Together they solve the system. Forward elimination is the expensive part; back-substitution requires fewer operations.

Before/after Strang

Strang opens his course with exactly this: "the geometry of linear equations" followed immediately by "elimination with matrices." If you are reading this *before* Strang, you now have the geometric view – planes pinned down one coordinate at a time – that clarifies the reasoning behind each algebraic step. If you are reading *after* Strang, notice that we are being deliberate about the multipliers ℓ_{ij}; they are recorded systematically rather than discarded. They are the basis of the $\mathbf{A} = \mathbf{LU}$ factorization that Chapter 9 (Section 9.1) makes precise, and they encode the entire elimination as a single matrix.

Compute it yourself

Solve this 2×2 system by hand, tracing both phases:

$$3x + 2y = 7$$
$$6x + 5y = 16.$$

Pivot on the 3, find the multiplier $6/3 = 2$, eliminate, then back-substitute. You should get $y = 2$, $x = 1$. Check it against both original equations before moving on – the checking habit is half the skill.

7.3 Counting the work, and why triangles are cheap

Why is this the algorithm everyone actually uses? Cost. For an $n \times n$ system, forward elimination does on the order of $\frac{1}{3}n^3$ arithmetic operations – one nested loop over rows, columns, and the entries to update. Back-substitution does only about $\frac{1}{2}n^2$. The forward sweep dominates completely, as Figure 7.4 illustrates.

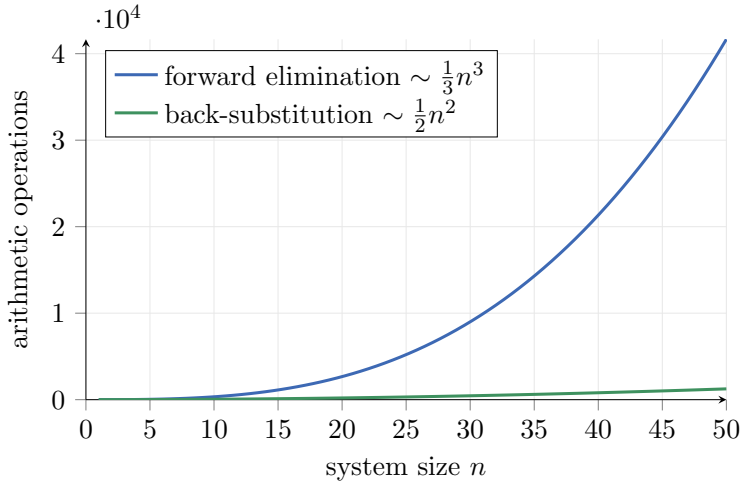


Figure 7.4: The two phases compared. The cubic forward sweep (blue) outruns the quadratic back-substitution (green) so fast that for any sizeable system the climb up the triangle is essentially free. Doubling n multiplies the blue curve by eight.

Why this matters

That $\frac{1}{3}n^3$ is one of the most consequential numbers in computing. Doubling the size of your system makes elimination eight times slower, not twice – which is why large sparse systems (where most coefficients are zero) require special algorithms that avoid filling in those zeros. But for dense systems, standard elimination remains, asymptotically, among the most effective algorithms for general use.

Connection to other math

The n^3 scaling is the same wall you hit when multiplying two matrices the naive way, and it is no coincidence. Solving $\mathbf{Ax} = \mathbf{b}$, inverting $\mathbf{A}$, and multiplying matrices are all $O(n^3)$ problems that turn out to be equivalent in difficulty up to constants – a fact from the theory of algorithmic complexity. Calculus students will recognize the family resemblance to how the cost of a triple integral grows with resolution: each new dimension multiplies the work. Elimination is what high-dimensional bookkeeping costs.

7.4 Row exchanges: when the pivot is in the wrong place

So far every pivot we needed was conveniently nonzero and sitting on the diagonal. Reality is not always so kind. Sometimes a zero shows up exactly where we wanted a pivot. Consider

$$\left[\begin{array}{ccc|c} 0 & 2 & 1 & 4 \\ 1 & 1 & 1 & 6 \\ 2 & 1 & 3 & 11 \end{array}\right].$$

The top-left entry is 0. We cannot use it as a pivot – the multiplier would be undefined (division by zero). But this is easily resolved. The equations are simply listed in an order that placed a zero on the diagonal. We exchange two rows to bring a nonzero number into the pivot position. Exchange row 1 and row 2:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 2 & 1 & 4 \\ 2 & 1 & 3 & 11 \end{array}\right].$$

Now the pivot is 1, and elimination proceeds. Clear column 1: row 3 already has a 2 to remove, multiplier $\ell_{31} = 2/1 = 2$, so row 3 becomes (row 3) $-2\cdot$(row 1) $= [\,0,\, -1,\, 1 \mid -1\,]$. Row 2 already starts with 0, so nothing to do there.

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 2 & 1 & 4 \\ 0 & -1 & 1 & -1 \end{array}\right].$$

Second pivot is 2 in row 2. Clear below: multiplier $\ell_{32} = -1/2$, so row 3 becomes (row 3) $-(-\frac{1}{2})\cdot$(row 2) = (row 3) $+\frac{1}{2}\cdot$(row 2) $= [\,0,\, 0,\, \frac{3}{2} \mid 1\,]$.

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 2 & 1 & 4 \\ 0 & 0 & \frac{3}{2} & 1 \end{array}\right].$$

Back-substitution: $\frac{3}{2}z = 1$ gives $z = \frac{2}{3}$; then $2y + \frac{2}{3} = 4$ gives $y = \frac{5}{3}$; then $x + \frac{5}{3} + \frac{2}{3} = 6$ gives $x = \frac{11}{3}$.

Focus

A zero in the pivot position is **not** necessarily a sign of trouble. If there is a nonzero entry *below* it in the same column, just swap rows to bring that entry up, and carry on. The swap costs nothing and changes no solution – you are only reordering the equations.

Modern application

Computers exchange rows even when they do not strictly have to, and the reason is *numerical stability* – keeping tiny, inevitable rounding errors in computer arithmetic from blowing up into completely wrong answers. Dividing by a pivot that is tiny – say 0.0001 – multiplies any rounding error in that row by ten thousand, and that magnified noise can quickly swamp the true answer.

So practical solvers use *partial pivoting*: before clearing a column, scan all the available rows on or below the diagonal, and swap the row with the largest absolute entry into the pivot position. This guarantees that every multiplier satisfies $|\ell_{ij}| \leq 1$, ensuring that row subtractions never amplify errors. Every time `numpy.linalg.solve`, MATLAB's backslash, or a LAPACK routine solves a system, partial pivoting runs automatically. It connects textbook math with exact fractions and scientific software working in finite floating-point arithmetic.

7.5 What goes wrong: zero pivots that will not move

Row exchange rescues us when a pivot position holds a zero but the column below has something nonzero to swap in. What if the *entire* column below (and including) the pivot position is zero? Then no swap can produce a pivot. This is not merely a bookkeeping detail; it indicates a structural property of the system. The geometry has changed.

7.5.1 Case one: no solution (an inconsistent system)

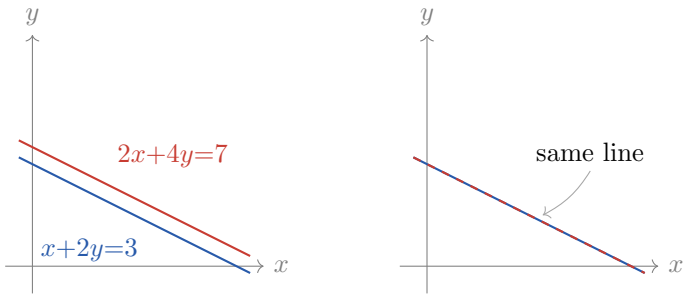
Consider

$$\begin{aligned} x + 2y &= 3 \\ 2x + 4y &= 7. \end{aligned} \qquad \left[\begin{array}{cc|c} 1 & 2 & 3 \\ 2 & 4 & 7 \end{array}\right].$$

Pivot on the 1. Multiplier $\ell_{21} = 2$. Row 2 becomes (row 2) $-2\cdot$(row 1) $= [\,0,\, 0 \mid 1\,]$.

$$\left[\begin{array}{cc|c} 1 & 2 & 3 \\ 0 & 0 & 1 \end{array}\right].$$

The second row now reads $0x + 0y = 1$, that is, $0 = 1$. There is no point in the plane where 0 equals 1. The system has **no solution**. Geometrically the two lines $x + 2y = 3$ and $2x + 4y = 7$ are *parallel*: same slope, different intercepts, never crossing. Elimination found this for us automatically – a row of zeros on the left with a nonzero on the right is the algebraic fingerprint of parallel, non-intersecting geometry. The left panel of Figure 7.5 shows the two parallel lines.



no solution (parallel) infinitely many

Figure 7.5: Two ways a 2×2 system fails to have one neat solution. Left: parallel lines that never meet, giving no solution (elimination uncovers the contradiction $0 = 1$). Right: both equations represent the exact same line, giving infinitely many solutions (elimination leaves the redundant identity $0 = 0$, signaling a free variable).

7.5.2 Case two: infinitely many solutions

Now change just the right-hand side:

$$\begin{aligned} x + 2y &= 3 \\ 2x + 4y &= 6. \end{aligned} \qquad \left[\begin{array}{cc|c} 1 & 2 & 3 \\ 2 & 4 & 6 \end{array}\right] \xrightarrow{R_2 - 2R_1} \left[\begin{array}{cc|c} 1 & 2 & 3 \\ 0 & 0 & 0 \end{array}\right].$$

This time the last row is $0 = 0$ – automatically true, adding no new constraint. The two equations defined the same line (the second is just twice the first), so every point on $x + 2y = 3$ is a solution (the right panel of Figure 7.5). There are **infinitely many**. We say y is a *free variable*: it can be anything, and then $x = 3 - 2y$ follows. The solution is a whole line's worth of points.

Focus

A zeroed-out pivot column forces a fork in the road. Read off the right-hand side of that dead row:

- left side all zero, right side **nonzero** $\Rightarrow$ "$0 = c$" with $c \neq 0$: **no solution** (inconsistent).
- left side all zero, right side **zero** $\Rightarrow$ "$0 = 0$": a **free variable** and **infinitely many** solutions.

A square matrix that produces fewer than n pivots is called **singular**: it has no inverse, and the system either fails or has a whole family of answers. A full set of n nonzero pivots means **A** is **invertible** and the solution is unique.

Common trap

Do not confuse the two failure modes. *Both* produce a row whose left side is all zeros. The distinction lies in the augmented entry on the right: a contradictory row like "$0 = 5$" indicates *no solution*; a row like "$0 = 0$" indicates a *free variable*. Make sure to inspect the augmented column before classifying the system.

Walking question

Take a walk and picture three planes in space. List, in your head, all the ways they can fail to meet at one point: all three parallel; two parallel and one crossing; three planes meeting pairwise in three parallel lines (forming a triangular prism); all three sharing a whole line (like pages meeting along a book's spine); all three being the same plane. Which of these give *no* solution and which give *infinitely many*? Elimination will sort every one of these cases into the two categories above. Notice which geometric configuration each case corresponds to.

7.6 Reduced row echelon form: the cleanest possible answer

Upper-triangular form plus back-substitution is enough to *solve* a system. There is a tidier endpoint, though, one that makes the answer machine-readable and gives us the natural language for the deeper structure coming in later chapters. Instead of stopping when the matrix is triangular, we push on until it is as clean as algebra allows. The result is the *reduced row echelon form*, almost always abbreviated **RREF**.

First, the looser notion. A matrix is in *row echelon form* when:

1. all-zero rows (if any) sit at the bottom, and
2. each row's leading nonzero entry (its *pivot*) is strictly to the right of the pivot in the row above – so the pivots march down and to the right in a staircase.

That is just upper-triangular generalized to non-square and rank-deficient matrices. To reach the *reduced* form we ask for two more things:

3. every pivot equals 1 (scale each pivot row), and
4. each pivot is the *only* nonzero entry in its column – we clear *upward* as well as downward.

Figure 7.6 shows the resulting staircase, with leading 1s stepping down and to the right and free columns in between.

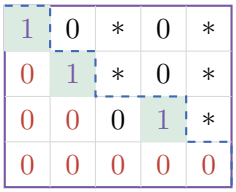


Figure 7.6: The staircase of a reduced row echelon form. Each leading 1 (a pivot, purple) sits strictly right of the one above; everything directly below a pivot is forced to zero (red), and clearing upward makes each pivot the lone nonzero in its column. Columns without a pivot ($*$ entries) belong to free variables.

Formal

A matrix **R** is in **reduced row echelon form** when it satisfies four conditions: (i) every zero row sits below every nonzero row, and (ii) the leading entry of each nonzero row is 1 (a *pivot* or leading 1). In addition, (iii) each leading 1 lies strictly to the right of the leading 1 above it, and (iv) each leading 1 is the only nonzero entry in its column. For a given matrix this form is **unique** – you reach the same **R** no matter which legal moves you choose along the way.

Because **R** depends only on **A** and not on the elimination path, every pivot count and free-variable pattern we read off **R** is an intrinsic property of **A** itself.

7.6.1 From triangle to RREF, traced

Take our very first result and finish the job. We had

$$\mathbf{U} = \left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 0 & -1 & -2 & -4 \\ 0 & 0 & -4 & -6 \end{array}\right].$$

Scale each pivot to 1. Divide row 1 by 2, row 2 by -1, row 3 by -4:

$$\left[\begin{array}{ccc|c} 1 & \frac{1}{2} & \frac{1}{2} & \frac{5}{2} \\ 0 & 1 & 2 & 4 \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right].$$

Clear upward in column 3. Row 2 becomes (row 2) $-2\cdot$(row 3); row 1 becomes (row 1) $-\frac{1}{2}\cdot$(row 3):

$$\left[\begin{array}{ccc|c} 1 & \frac{1}{2} & 0 & \frac{7}{4} \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right].$$

Clear upward in column 2. Row 1 becomes (row 1) $-\frac{1}{2}\cdot$(row 2):

$$\mathbf{R} = \left[\begin{array}{ccc|c} 1 & 0 & 0 & \frac{5}{4} \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right].$$

In this final form, the left block is the identity matrix, and the solution appears directly in the augmented column: $x = \frac{5}{4}$, $y = 1$, $z = \frac{3}{2}$ – matching the values found by back-substitution. *RREF incorporates the back-substitution step directly into the row operations.*

Intuition

Upper-triangular form produces a system ready for back-substitution. RREF completes those substitutions during elimination, leaving the solution directly visible in the augmented column. The trade-off is extra arithmetic up front (clearing upward) in exchange for an explicit readout. For solving a single system by hand, stopping at triangular form is usually faster. For analyzing the matrix itself – its rank, free variables, and null space – RREF provides the standard

canonical form used throughout subsequent chapters.

Compute it yourself

Reduce this matrix all the way to RREF by hand. It has a free variable, so the identity will *not* appear – watch where the staircase breaks.

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 2 & 4 & 0 & 6 \\ 1 & 2 & 3 & 0 \end{array}\right].$$

(Hint: column 2 will never get a pivot, so y is free. You should find that z is pinned to a single value while x depends on y. You will get $z = -1$ and $x = 3 - 2y$.)

7.7 Elimination is matrix multiplication

Here is the idea that lifts elimination from "a procedure" to "a piece of linear algebra," and it is the bridge to everything that follows. **Every single row operation is the same as multiplying on the left by a special matrix.** These special matrices are called *elementary matrices*, and once you see them, the whole algorithm becomes a product of matrices.

The trick to building an elementary matrix is almost too simple: *do the row operation to the identity matrix*, and the result is the matrix that performs that operation on *anything* when you left-multiply by it.

7.7.1 The replacement matrix

Recall our very first step: row 2 $\leftarrow$ row 2 $-2\cdot$row 1, on a 3-row matrix. Apply that operation to $\mathbf{I}_3$. Start from

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \text{do row } 2 \leftarrow \text{row } 2 - 2 \cdot \text{row } 1, \quad \mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The only change from the identity is the -2 sitting in position $(2, 1)$ – the multiplier, with a minus sign, parked exactly where the entry it kills used to be. Now multiply $\mathbf{E}_{21}$ against the original augmented matrix and watch it perform the operation:

$$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 4 & 1 & 0 & 6 \\ -2 & 2 & 1 & 1 \end{array}\right] = \left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 0 & -1 & -2 & -4 \\ -2 & 2 & 1 & 1 \end{array}\right].$$

Row 1 and row 3 pass through untouched (the identity rows copy them); row 2 comes out as old-row-2 minus twice row-1. Exactly Step 1 from before. Let us verify the new row 2 by hand: its first entry is $(-2)(2) + (1)(4) + (0)(-2) = -4 + 4 = 0$; its second is $(-2)(1) + (1)(1) = -1$; its third $(-2)(1) + (1)(0) = -2$; the augmented entry $(-2)(5) + (1)(6) = -4$. The -2 in $\mathbf{E}_{21}$ *is* the multiplier.

Focus

The elementary matrix for "row $i \leftarrow$ row i $-\ell\cdot$ row j" is the identity with the single extra entry $-\ell$ placed at position (i, j). Left-multiplying by it performs the operation. To *undo* it, flip the sign: $\mathbf{E}^{-1}$ is the identity with $+\ell$ at (i, j). (Subtracting $\ell\cdot$row j and then adding it back is the identity.)

Pause

Pause and think: If an elementary matrix $\mathbf{E}$ subtracts $3\times$ (row 1) from row 2, what does its inverse $\mathbf{E}^{-1}$ do? You do not need any algebraic formula – how do you undo that operation in plain words?

Add $3\times$ (row 1) back to row 2. Inverting an elementary row replacement changes only the sign of the multiplier: -3 becomes $+3$.

7.7.2 The other two as matrices

A **scaling** of row i by c is the identity with c in place of the 1 at position (i, i). A **row exchange** of rows i and j is the identity with rows i and j swapped; such a matrix is called a *permutation matrix*, written $\mathbf{P}$. For example, swapping rows 1 and 2 of a 3-row matrix is done by

$$\mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{P}\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} b \\ a \\ c \end{bmatrix}.$$

Each of these is invertible (scaling by c undoes with $1/c$; a swap undoes with the same swap), which is the matrix version of "every legal move is reversible."

7.7.3 The whole elimination as one product

Now string the steps together. Our full forward sweep was three replacement operations: $\mathbf{E}_{21}$, then $\mathbf{E}_{31}$, then $\mathbf{E}_{32}$. Applying them in order to $\mathbf{A}$ means left-multiplying one after another:

$$\mathbf{E}_{32}\,\mathbf{E}_{31}\,\mathbf{E}_{21}\,\mathbf{A} = \mathbf{U}.$$

Read it right to left, the order the operations actually happened. A single matrix $\mathbf{E} = \mathbf{E}_{32}\mathbf{E}_{31}\mathbf{E}_{21}$ captures the entire forward elimination. And because each $\mathbf{E}$ is invertible, we can move them to the other side:

$$\mathbf{A} = \mathbf{E}_{21}^{-1}\,\mathbf{E}_{31}^{-1}\,\mathbf{E}_{32}^{-1}\,\mathbf{U} = \mathbf{L}\,\mathbf{U}.$$

That product of inverse-elementary matrices is lower triangular, and it is the matrix $\mathbf{L}$ – whose entries are simply the multipliers ℓ_{ij} themselves, sign restored. We have arrived at the $\mathbf{A} = \mathbf{LU}$ factorization. Chapter 9 (Section 9.1) is devoted to exploring that factorization in full; for now, the headline is enough.

Why this matters

This is the moment elimination stops being arithmetic and becomes *algebra*. A sequence of row subtractions is expressed as a matrix equation $\mathbf{EA} = \mathbf{U}$. Procedures become objects; objects can be inverted, composed, factored, and analyzed. The viewpoint that an algebraic operation is represented by a matrix recurs throughout the book: rotations are matrices, projections are matrices, and derivatives on polynomial spaces are matrices. Representing an elimination step as an elementary matrix $\mathbf{E}$ connects computational procedures directly to linear algebra theory.

Before/after Strang

"Elimination = left-multiplication by elementary matrices, and undoing it gives $\mathbf{A} = \mathbf{LU}$" is the exact hinge on which Strang's first weeks turn. If you are heading into his course, you are now ahead: you have already met $\mathbf{E}$, $\mathbf{P}$, and $\mathbf{L}$ as concrete grids of numbers. If you are revisiting after Strang, notice we got $\mathbf{L}$ for *free* – the multipliers we were already computing *are* its entries, no extra work. That is why elimination and factorization are two views of one act.

7.7.4 Where permutations land: $\mathbf{PA} = \mathbf{LU}$

When the elimination needs a row swap, a permutation matrix $\mathbf{P}$ joins the product. Collecting all the needed swaps into a single $\mathbf{P}$ and applying them first, one can show every square matrix satisfies $\mathbf{PA} = \mathbf{LU}$ for some permutation $\mathbf{P}$, lower-triangular $\mathbf{L}$, and upper-triangular $\mathbf{U}$. That is the fully general statement of what Gaussian elimination produces, and it is exactly what a real solver computes and stores. We will prove and exploit it in Chapter 9 (Section 9.1); the point here is that nothing about row exchanges breaks the "elimination is matrix multiplication" story – swaps are just more elementary matrices.

7.8 The full algorithm, in one place

Here is the whole procedure as one recipe you can run on any system.

Focus

Gaussian elimination, end to end.

1. Form the augmented matrix $[\,\mathbf{A} \mid \mathbf{b}\,]$.
2. *Forward sweep.* For each column j from left to right:
   - If the pivot position (j, j) is zero, swap in a nonzero entry from below. If the whole column below is zero, move on – this column has no pivot (a free variable lives here).
   - For each row i below the pivot, compute $\ell_{ij} = a_{ij}/a_{jj}$ and replace row i by row i $-\ell_{ij}\cdot$ (pivot row).
3. Inspect the result. A row "$0 \;\cdots\; 0 \mid c$" with $c \neq 0$ means *no solution*. Otherwise, a

column without a pivot means a *free variable* and infinitely many solutions.

4. *Solve.* Back-substitute up the triangle; or, for the cleanest readout, continue to RREF and read the answer off directly.

7.8.1 One more 3x3, with a twist

Here is a system where a swap is forced *mid-stream*, not just at the start. This is the case students most often fumble, so trace it slowly.

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 5 & 12 \\ 1 & 3 & 0 & 8 \end{array}\right].$$

Pivot on the top-left 1. Clear column 1. Row 2 $\leftarrow$ row 2 $-2\cdot$row 1 $= [\,0,\,0,\,3 \mid 0\,]$; row 3 $\leftarrow$ row 3 $-1\cdot$row 1 $= [\,0,\,2,\,-1 \mid 2\,]$.

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 0 & 3 & 0 \\ 0 & 2 & -1 & 2 \end{array}\right].$$

Now the second pivot position $(2,2)$ holds a 0. But the column below it has a 2 in row 3 – so we swap rows 2 and 3:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 2 \\ 0 & 0 & 3 & 0 \end{array}\right].$$

And we are already upper triangular – the swap handed us a clean triangle. Back-substitute: $3z = 0$ so $z = 0$; then $2y - 0 = 2$ so $y = 1$; then $x + 1 + 0 = 6$ so $x = 5$. Check the original middle equation: $2 \cdot 5 + 2 \cdot 1 + 5 \cdot 0 = 12$. Good. Notice the pivots ended up being $1, 2, 3$ – three of them, all nonzero – so this matrix is invertible and the solution is the unique point $(5, 1, 0)$.

Common trap

When a pivot position is zero, do *not* divide by it and do *not* give up. First look down the column. Nonzero entry below? Swap. Nothing below? That column simply has no pivot; slide your attention right to the next column while staying in the same row, and keep going. Dividing by the zero is the single most common fatal error – it produces nonsense and corrupts everything downstream.

Connection to other math

Gaussian elimination is not confined to the real numbers. The only thing the algorithm ever does is add, subtract, multiply, and divide by nonzero pivots – so it runs verbatim over *any field*: the rationals, the complex numbers, or the finite field of integers modulo a prime. That last case is the engine of modern coding theory and cryptography: solving linear systems over $\mathbb{F}_2$ (arithmetic mod 2, where $1 + 1 = 0$) decodes error-correcting codes and breaks linear ciphers. The schoolroom trick, run over a two-element number system, is doing real work inside your hard drive's error correction and your phone's signal decoder. Same algorithm, different field.

7.9 What elimination is really telling you

Beyond solving a single right-hand side, the number of pivots a matrix produces – counted across the whole forward sweep, swaps and all – is one of the most important numbers attached to it. It is called the *rank*, and it answers questions the original grid of numbers hid from view:

- **How many genuinely independent equations are there?** Exactly the number of pivots. Rows that elimination flattens to zero were redundant – they carried no new information.
- **Is the square system uniquely solvable?** Yes if and only if every diagonal position earns a pivot – n pivots for an $n \times n$ matrix. That is the same as $\mathbf{A}$ being invertible.
- **How many free variables are there?** The number of columns minus the number of pivots. Each free variable is a direction you can slide along without leaving the solution set – the dimension of the family of answers.

Every one of these ideas – rank, invertibility, free variables, the dimension of the solution set – gets its own full chapter later.

Why this matters

One forward sweep of row operations does far more than solve a single right-hand side: its pivots count the independent equations, test whether $\mathbf{A}^{-1}$ exists, count the free variables, and assemble the factorization $\mathbf{A} = \mathbf{LU}$.

Where do we go next? In this chapter, we saw that row operations correspond to multiplications on the left by elementary matrices $\mathbf{E}$, and inverting those operations begins to assemble the matrix factorization $\mathbf{A} = \mathbf{LU}$. In the next chapter, we examine matrix multiplication and inverses more broadly. What does it mean to multiply one matrix by another as a unified geometric transformation? Why does the row-by-column rule work the way it does? And when a square matrix has a full set of n pivots, how do we find a single inverse matrix $\mathbf{A}^{-1}$ that undoes $\mathbf{A}$ directly, turning $\mathbf{Ax} = \mathbf{b}$ into $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$? The next chapter, *Matrix Multiplication and Inverses*, builds that algebraic foundation.

History and connection

Although named after Carl Friedrich Gauss, systematic elimination began much earlier. Around 150 BCE, Chinese scholars compiling *The Nine Chapters on the Mathematical Art* solved agricultural tax and grain-yield problems involving several simultaneous conditions. Working with bamboo counting rods arranged in rectangular arrays on a grid board (*Fangcheng*), they repeatedly subtracted multiples of one column from another to clear entries below the diagonal – performing the forward sweep of elimination over eighteen centuries before Europe codified it.

In 1801, Gaussian elimination took its modern shape through celestial mechanics. On New Year's Day, the Italian astronomer Giuseppe Piazzi spotted a dim, wandering body in the night sky – the dwarf planet Ceres. Piazzi tracked it for only 41 days across a tiny nine-degree arc before it was lost in the blinding glare of the Sun. Astronomers across Europe despaired of finding it again: they had only a handful of noisy, imperfect observations, and Newton's gravitational laws led to non-linear orbital equations that were impossible to solve directly.

The 24-year-old Carl Friedrich Gauss took up the challenge. Developing the *method of least squares* – a technique for finding the single best-fitting mathematical curve through a cloud

of noisy, imperfect observations – Gauss set up a system of six coupled linear equations in six unknown orbital parameters. With only pen, paper, and logarithms as tools, he organized the arithmetic into systematic forward elimination and backward substitution, cutting the hand computation down to manageable hours. On New Year's Eve 1801, guided by Gauss's calculations, astronomers pointed their telescopes into the night sky and found Ceres within half a degree of his predicted position. Gauss later published the systematic algorithm in 1809 in *Theoria Motus Corporum Coelestium* while calculating the orbit of the asteroid Pallas.

Eighty years later, in 1888, the German geodesist Wilhelm Jordan was tackling *geodesy* – the science of accurately measuring the Earth's geometric shape and mapping regional networks of surveying triangles. Facing hundreds of error-prone equations, Jordan extended Gauss's procedure by using each pivot to eliminate entries *upward* as well as downward. This drove the matrix straight to reduced row echelon form (RREF) or the identity matrix, allowing surveyors to read off coordinates directly without a separate back-substitution climb. The combined algorithm has been known as *Gauss–Jordan elimination* ever since.

In pure mathematics with exact fractions, elimination succeeds whenever pivots are nonzero. But on digital computers working in *floating-point arithmetic* – where numbers are stored with a finite number of digits and rounded off after every operation – naive elimination is dangerous. If a pivot is tiny (say 10^{-5}), dividing by it creates enormous multipliers (10^5), and subsequent row subtractions can magnify microscopic roundoff errors until they completely swamp the true solution.

In 1947, computing pioneers John von Neumann and Herman Goldstine published a paper warning that Gaussian elimination on digital computers would suffer catastrophic, exponential roundoff growth, predicting that errors would explode on matrices with more than 15 or 20 variables. That sparked an urgent controversy over whether linear algebra could ever run reliably on electronic computers. Alan Turing (in his 1948 paper *Rounding-Off Errors in Matrix Processes*) and later James H. Wilkinson (in 1961) resolved the crisis using *backward error analysis*. Rather than asking how far the computed answer strays from the true solution, backward error analysis asks what slightly perturbed problem the computed answer solves *exactly*. Using this perspective, they proved that **partial pivoting** – swapping rows at each step to seat the largest available entry as the pivot – keeps all multipliers within $|\ell_{ij}| \leq 1$, making Gaussian elimination numerically stable for almost all practical matrices.

Gaussian elimination provided science with its first universal, $O(n^3)$ algorithm: it solves systems of any size, checks whether solutions exist, tracks families of solutions when free variables appear, and measures the matrix's rank. Yet two major frontiers remain open today:

- **The optimal complexity exponent:** Is solving an $n \times n$ system fundamentally an $O(n^3)$ problem, or can it be done essentially in $O(n^2)$ time – as fast as reading the n^2 entries of the matrix? In 1969, Volker Strassen showed that matrix multiplication (and thus elimination) can be carried out in $O(n^{2.81})$ operations. Theoretical computer scientists have pushed that theoretical exponent down near $O(n^{2.37155})$, but the true optimal exponent ω remains one of the most famous open problems in computer science.
- **Worst-case stability vs. speed:** Although partial pivoting runs in cheap $O(n^2)$ extra time and never fails on practical problems, artificial matrices exist (such as Kahan's matrix) where its roundoff theoretically blows up exponentially. *Complete pivoting* (swapping both rows and columns) guarantees provable stability for every matrix, but searching the entire submatrix costs an expensive $O(n^3)$ overhead. Finding a pivoting strategy that combines the speed of partial pivoting with stronger theoretical stability guarantees

remains an active research direction in numerical linear algebra.

There is another practical bottleneck: if you solve $\mathbf{Ax} = \mathbf{b}$ and subsequently change only the right-hand side $\mathbf{b}$ (such as testing multiple wind loads on the same bridge structure), running naive elimination from scratch wastes $O(n^3)$ work by repeating the exact same row operations on $\mathbf{A}$. Overcoming this repetitive cost requires packaging elimination into matrix operators: Chapter 8 (Section 8.1) examines matrix multiplication and inverses, paving the way for Chapter 9 (Section 9.1) to introduce LU factorization ($\mathbf{A} = \mathbf{LU}$), which stores the multipliers so that any new right-hand side can be solved in $O(n^2)$ time.

7.10 Exercises

Exercise 7.1 (Mental checks: true or false?)**.** Decide whether each statement is true or false, and give a one-sentence geometric or algebraic reason:

(a) Multiplying an entire equation by a nonzero constant changes the line or plane it represents in space.

(b) In an upper-triangular system $\mathbf{Ux} = \mathbf{c}$, back-substitution solves the system without computing any new multipliers ℓ_{ij}.

(c) If an $n \times n$ system produces fewer than n nonzero pivots during elimination, it is guaranteed to have no solution.

(d) Swapping two rows of an augmented matrix changes the coordinates (x, y, z) of the intersection point.

Exercise 7.2 (Warm-up, fully traced)**.** Solve by Gaussian elimination, showing the multiplier at every step and then back-substituting:

$$\begin{aligned} x + 2y + z &= 4 \\ 2x + 5y + 3z &= 11 \\ 2x + 3y + 2z &= 8. \end{aligned}$$

Verify your answer in all three original equations.

Exercise 7.3 (A forced swap)**.** Run elimination on

$$\left[\begin{array}{ccc|c} 0 & 1 & 2 & 3 \\ 1 & 2 & 1 & 4 \\ 3 & 1 & 0 & 5 \end{array}\right].$$

You must exchange rows before you can begin. Identify which swap, reach upper-triangular form, and back-substitute. State the three pivots.

Exercise 7.4 (Diagnose the failure)**.** For each system, run elimination far enough to decide whether it has no solution, exactly one, or infinitely many. If infinitely many, describe the solution set (name the free variable and write the others in terms of it).

(a) $\begin{aligned} x + y &= 2 \\ 3x + 3y &= 5, \end{aligned}$ (b) $\begin{aligned} x - y &= 1 \\ -2x + 2y &= -2. \end{aligned}$

Exercise 7.5 (All the way to RREF)**.** Take the upper-triangular matrix

$$\left[\begin{array}{ccc|c} 3 & 6 & 1 & 0 \\ 0 & 2 & 4 & 8 \\ 0 & 0 & 5 & 10 \end{array}\right]$$

and reduce it to RREF: scale each pivot to 1 and clear upward. Read off x, y, z from the final augmented column.

Exercise 7.6 (Elementary matrices by hand)**.** Write down the 3×3 elementary matrix $\mathbf{E}$ that performs "row 3 $\leftarrow$ row 3 $-4\cdot$ row 1." Multiply it against $\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 4 & 2 & 0 \end{bmatrix}$ and confirm by direct multiplication that the product has a zero in position $(3, 1)$. Then write $\mathbf{E}^{-1}$ and verify $\mathbf{E}^{-1}\mathbf{E} = \mathbf{I}$.

Exercise 7.7 (A free variable in 3D)**.** Solve

$$\begin{aligned} x + y + z &= 3 \\ 2x + 2y + 3z &= 8 \\ x + y + 2z &= 5, \end{aligned}$$

and notice that column 2 never produces a pivot. Identify the free variable and write the full family of solutions as a point plus a multiple of a direction vector. (Geometrically, the three planes share a whole line.)

Exercise 7.8 (Tiny-pivot instability)**.** Consider, with exact arithmetic,

$$\begin{aligned} 0.001\, x + y &= 1 \\ x + y &= 2. \end{aligned}$$

(a) Solve exactly. (b) Now pretend your calculator keeps only 3 significant digits and eliminate *without* swapping – using 0.001 as the pivot. Watch the answer for x degrade. (c) Swap the two rows first (partial pivoting) and redo with 3 digits. Explain why the swapped version is far more accurate.

Exercise 7.9 (Counting operations)**.** Argue that clearing column 1 of an $n \times n$ matrix costs about n^2 multiply-add operations (you touch each of the $n-1$ rows below the pivot, and each touched row has about n entries). Summing the shrinking columns, explain informally why the whole forward sweep costs on the order of $\frac{1}{3}n^3$. (Hint: $\sum_{k=1}^{n} k^2 \approx \frac{1}{3}n^3$.)

Exercise 7.10 (Uniqueness of RREF – a proof)**.** Suppose two different sequences of legal row operations turn the same matrix $\mathbf{A}$ into row echelon forms $\mathbf{R}_1$ and $\mathbf{R}_2$. Argue that $\mathbf{R}_1$ and $\mathbf{R}_2$ have their pivots in exactly the same columns. (Hint: a column is a pivot column precisely when it is *not* a linear combination of the columns to its left, and row operations do not change which columns are combinations of which earlier columns. You may take this last fact as given for now – we prove it carefully in the chapter on the column space.)

Exercise 7.11 (Elimination over a tiny field)**.** Work over $\mathbb{F}_2$, where the only numbers are 0 and 1 and $1 + 1 = 0$. Solve

$$\begin{aligned} x + y \quad\quad &= 1 \\ x \quad\quad + z &= 0 \\ y + z &= 1, \end{aligned}$$

using exactly the same elimination, but with mod-2 arithmetic (so subtracting equals adding). Find all solutions. This provides a simple illustration of decoding an error-correcting code.

Chapter 8

Matrix Multiplication and Inverses

In the last chapter, Gaussian elimination gave us a systematic way to solve $\mathbf{Ax} = \mathbf{b}$. We treated the matrix $\mathbf{A}$ as a scaffold of equations, using row operations to simplify the system until we could read off the unknown vector $\mathbf{x}$. But behind that row-by-row arithmetic lives an active geometric reality: a matrix is also a *machine*. Feed in a vector $\mathbf{x}$, and the matrix moves it somewhere new, spitting out $\mathbf{Ax}$.

This chapter is about two natural questions that follow immediately.

First: what happens if you run a vector through *two* machines in a row? Push $\mathbf{x}$ through $\mathbf{B}$, then push the result through $\mathbf{A}$. You get $\mathbf{A}(\mathbf{Bx})$. It turns out there is a single matrix — we will call it $\mathbf{AB}$ — that performs both steps in one shot. Finding that matrix *is* matrix multiplication. Everything about how you multiply matrices, including the rule that feels so arbitrary the first time you see it, is forced on us the moment we demand $(\mathbf{AB})\mathbf{x} = \mathbf{A}(\mathbf{Bx})$.

Second: can we ever *undo* a machine? If $\mathbf{A}$ sent $\mathbf{x}$ to $\mathbf{b}$, is there a machine that takes $\mathbf{b}$ back to $\mathbf{x}$? That machine, when it exists, is the *inverse* $\mathbf{A}^{-1}$. We will see exactly when it exists, what it means geometrically, and how to compute it with the same elimination you already know.

These two ideas are the grammar of everything that comes later. Factorizations like $\mathbf{A} = \mathbf{LU}$ and $\mathbf{A} = \mathbf{QR}$ are sentences in this grammar: they say "this complicated machine is really these simpler machines, one after another." The grammar is worth learning carefully.

8.1 Where Matrix Multiplication Comes From

8.1.1 Composition forces the rule

Forget formulas for a moment. Here is the only thing we want.

Intuition

A matrix is a machine that transforms vectors. If $\mathbf{B}$ is one machine and $\mathbf{A}$ is another, then doing $\mathbf{B}$ *first* and $\mathbf{A}$ *second* is itself some transformation. **Matrix multiplication is the name of that combined transformation.** The product $\mathbf{AB}$ is defined to be the single matrix that produces $\mathbf{A}(\mathbf{Bx})$ for every $\mathbf{x}$.

Notice the order already looks backwards: $\mathbf{B}$ acts first but is written on the right, next to the $\mathbf{x}$ it touches first. That is not a typo and it is not a convention someone could have chosen differently. It comes straight from how we write function composition: $f(g(x))$ applies g first, and g sits on the inside, closest to x. Matrices inherit that.

Let us make it concrete in two dimensions. Take

$$\mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \qquad \text{(stretch horizontally by 2)},$$

$$\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \qquad \text{(rotate by } 90^\circ \text{ counterclockwise)}.$$

Take a test vector $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Run it through $\mathbf{B}$ first:

$$\mathbf{Bx} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Now run *that* through $\mathbf{A}$:

$$\mathbf{A}(\mathbf{Bx}) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

So the two-step journey sends $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ to $\begin{bmatrix} -1 \\ 2 \end{bmatrix}$. Can we find a *single* matrix $\mathbf{C}$ that does this directly? Yes, and there is a quick trick for reading it off. A matrix is completely determined by what it does to the basis vectors $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, because the columns of any matrix *are* the images of the basis vectors. So just push $\mathbf{e}_1$ and $\mathbf{e}_2$ through the whole pipeline and collect the answers as columns.

$$\mathbf{Be}_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad \mathbf{A}(\mathbf{Be}_1) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

$$\mathbf{Be}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad \mathbf{A}(\mathbf{Be}_2) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Stack those two images as columns:

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} 0 & -1 \\ 2 & 0 \end{bmatrix}.$$

Let us check it on our test vector: $\mathbf{C} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \cdot 1 + (-1) \cdot 1 \\ 2 \cdot 1 + 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$. Exactly the two-step answer. The single machine $\mathbf{C}$ reproduces "stretch then rotate" perfectly.

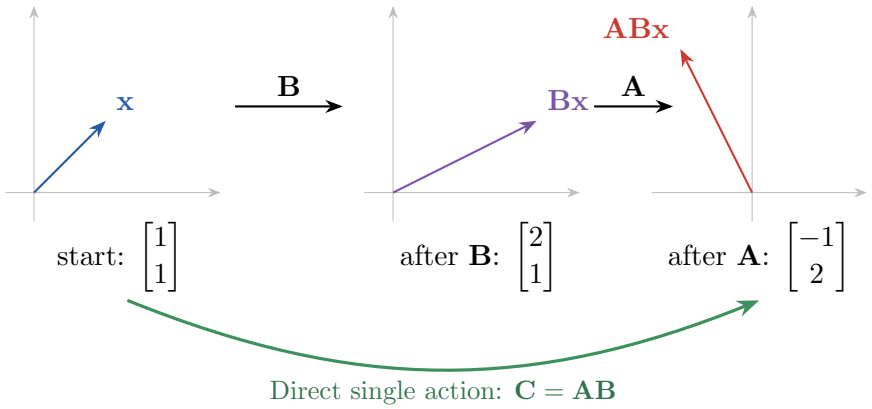


Figure 8.1: Two machines in a row. First **B** stretches, then **A** rotates. The single matrix **AB** does the whole trip at once.

Focus

To build **AB**, send each column of **B** through **A**. The j-th column of **AB** is **A** times the j-th column of **B**. In symbols, if $\mathbf{B} = [\,\mathbf{b}_1\ \mathbf{b}_2\ \cdots\ \mathbf{b}_n\,]$, then

$$\mathbf{AB} = [\,\mathbf{Ab}_1\ \ \mathbf{Ab}_2\ \ \cdots\ \ \mathbf{Ab}_n\,].$$

This is the single most useful way to think about a matrix product. Hold onto it.

8.1.2 Why the shapes have to match

For **AB** to make sense, the output of **B** must be a legal input for **A**. If **B** produces vectors in $\mathbb{R}^m$ (it has m rows), then **A** must accept vectors in $\mathbb{R}^m$ (it has m columns). That is the famous shape rule:

$$\underbrace{\mathbf{A}}_{p\times m}\ \underbrace{\mathbf{B}}_{m\times n} = \underbrace{\mathbf{AB}}_{p\times n}.$$

The inner dimensions (m and m) must agree and they cancel; the outer dimensions (p and n) survive as the shape of the product. There is nothing to memorize here if you remember the machines: **B** maps $\mathbb{R}^n \to \mathbb{R}^m$, **A** maps $\mathbb{R}^m \to \mathbb{R}^p$, so the composition maps $\mathbb{R}^n \to \mathbb{R}^p$.

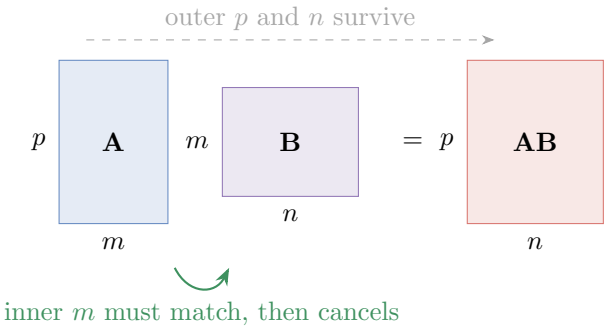


Figure 8.2: The shape rule as a picture. The right edge of $\mathbf{A}$ (m columns) must meet the left edge of $\mathbf{B}$ (m rows); the shared inner dimension cancels, and the outer dimensions p and n become the shape of $\mathbf{AB}$.

Common trap

"$\mathbf{AB}$ exists" does *not* mean "$\mathbf{BA}$ exists." If $\mathbf{A}$ is 2×3 and $\mathbf{B}$ is 3×4, then $\mathbf{AB}$ is 2×4 — fine — but $\mathbf{BA}$ asks a 3×4 to feed a 2×3, and $4\neq 2$, so it is undefined. Order matters before we even get to the values.

Pause

Pause and think: Suppose $\mathbf{A}$ is 3×4 and $\mathbf{B}$ is 4×3. Does $\mathbf{AB}$ exist? Does $\mathbf{BA}$ exist? What are their shapes? Take a moment to predict both before checking.
Answer: Both exist. The inner dimensions match in both orders ($4=4$ for $\mathbf{AB}$, and $3=3$ for $\mathbf{BA}$). But their shapes are completely different: $\mathbf{AB}$ is 3×3, while $\mathbf{BA}$ is 4×4. They do not even live in the same dimension.

8.2 Four Views of the Same Product

There is one product $\mathbf{AB}$, but four ways to see it, and a fluent reader can switch between them at will. Each view illuminates a different chapter later in the book, so we walk through all four with the same small example. Throughout this section let

$$\mathbf{A}=\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix},\qquad \mathbf{B}=\begin{bmatrix}5 & 6\\ 7 & 8\end{bmatrix}.$$

We will compute $\mathbf{AB}$ four times and get the same answer each time:

$$\mathbf{AB}=\begin{bmatrix}19 & 22\\ 43 & 50\end{bmatrix}.$$

8.2.1 View 1: the dot-product (entry) view

This is the entry-by-entry formula commonly used in manual calculations. The entry in row i, column j of $\mathbf{AB}$ is the dot product of row i of $\mathbf{A}$ with column j of $\mathbf{B}$:

$$(\mathbf{AB})_{ij} = \sum_k \mathbf{A}_{ik}\,\mathbf{B}_{kj} = (\text{row } i \text{ of } \mathbf{A}) \cdot (\text{column } j \text{ of } \mathbf{B}).$$

Let us do all four entries of our example, slowly.

$$(\mathbf{AB})_{11} = \begin{bmatrix}1\\2\end{bmatrix} \cdot \begin{bmatrix}5\\7\end{bmatrix} = 1\cdot 5 + 2\cdot 7 = 5 + 14 = 19,$$

$$(\mathbf{AB})_{12} = \begin{bmatrix}1\\2\end{bmatrix} \cdot \begin{bmatrix}6\\8\end{bmatrix} = 1\cdot 6 + 2\cdot 8 = 6 + 16 = 22,$$

$$(\mathbf{AB})_{21} = \begin{bmatrix}3\\4\end{bmatrix} \cdot \begin{bmatrix}5\\7\end{bmatrix} = 3\cdot 5 + 4\cdot 7 = 15 + 28 = 43,$$

$$(\mathbf{AB})_{22} = \begin{bmatrix}3\\4\end{bmatrix} \cdot \begin{bmatrix}6\\8\end{bmatrix} = 3\cdot 6 + 4\cdot 8 = 18 + 32 = 50.$$

Intuition

"Row i dotted with column j." To find a single entry you reach across the row of the left matrix and down the column of the right matrix, multiply pairwise, and add. It is the most mechanical view, perfect for hand computation, but the worst for *understanding* — it hides the geometry. Use the next three views for meaning.

Reading note

The cost. This view also lets us count the work. Multiplying a $p \times m$ by an $m \times n$ matrix fills $p\,n$ entries, and each entry is a dot product of length m: about m multiplications and m additions. So a full product costs on the order of $p\,m\,n$ multiply–adds, and for two $n \times n$ matrices that is roughly n^3 operations. Keep that n^3 in mind; it is why the order you multiply a long chain matters, and why factored views and the outer-product view (breaking matrices into simple, one-dimensional slices) are not just pretty but *cheaper*.

8.2.2 View 2: the column view (combinations of A's columns)

This is the view from Section 8.1: each column of $\mathbf{AB}$ is $\mathbf{A}$ acting on the corresponding column of $\mathbf{B}$. And remember what $\mathbf{A}$ times a vector means — it is a *linear combination of the columns of* $\mathbf{A}$, with weights given by the entries of that vector.

Write $\mathbf{A} = [\,\mathbf{a}_1\ \mathbf{a}_2\,]$ with $\mathbf{a}_1 = \begin{bmatrix}1\\3\end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix}2\\4\end{bmatrix}$. Column 1 of $\mathbf{B}$ is $\begin{bmatrix}5\\7\end{bmatrix}$, so:

$$\text{col 1 of } \mathbf{AB} \;=\; 5\,\mathbf{a}_1 + 7\,\mathbf{a}_2 = 5\begin{bmatrix}1\\3\end{bmatrix} + 7\begin{bmatrix}2\\4\end{bmatrix} = \begin{bmatrix}5\\15\end{bmatrix} + \begin{bmatrix}14\\28\end{bmatrix} = \begin{bmatrix}19\\43\end{bmatrix}.$$

And column 2 of $\mathbf{B}$ is $\begin{bmatrix}6\\8\end{bmatrix}$, so:

$$\text{col 2 of } \mathbf{AB} \;=\; 6\,\mathbf{a}_1 + 8\,\mathbf{a}_2 = 6\begin{bmatrix}1\\3\end{bmatrix} + 8\begin{bmatrix}2\\4\end{bmatrix} = \begin{bmatrix}6\\18\end{bmatrix} + \begin{bmatrix}16\\32\end{bmatrix} = \begin{bmatrix}22\\50\end{bmatrix}.$$

Same matrix $\begin{bmatrix}19 & 22\\43 & 50\end{bmatrix}$, built column by column.

Focus

Every column of AB lives in the column space of A. The *column space* $\text{col}(\mathbf{A})$ is the territory of all vectors you can reach by taking linear combinations of $\mathbf{A}$'s columns. Its dimension is the matrix's *rank* — the count of truly independent directions the matrix can output.
Because each column of $\mathbf{AB}$ is built strictly from $\mathbf{A}$'s columns, the product cannot explore any direction that $\mathbf{A}$ could not already reach on its own. In symbols, $\text{col}(\mathbf{AB}) \subseteq \text{col}(\mathbf{A})$. Consequently, the number of independent output directions cannot increase: $\text{rank}(\mathbf{AB}) \leq \text{rank}(\mathbf{A})$. You can never create new directions out of thin air by multiplying on the right.

8.2.3 View 3: the row view (combinations of B's rows)

By perfect symmetry, each *row* of $\mathbf{AB}$ is a combination of the *rows* of $\mathbf{B}$, with weights from the corresponding row of $\mathbf{A}$.

Row 1 of $\mathbf{A}$ is $[\,1\;\;2\,]$. Rows of $\mathbf{B}$ are $\mathbf{r}_1 = [\,5\;\;6\,]$ and $\mathbf{r}_2 = [\,7\;\;8\,]$:

$$\text{row 1 of } \mathbf{AB} = 1\cdot[\,5\;6\,] + 2\cdot[\,7\;8\,] = [\,5\;6\,] + [\,14\;16\,] = [\,19\;\;22\,].$$

Row 2 of $\mathbf{A}$ is $[\,3\;\;4\,]$:

$$\text{row 2 of } \mathbf{AB} = 3\cdot[\,5\;6\,] + 4\cdot[\,7\;8\,] = [\,15\;18\,] + [\,28\;32\,] = [\,43\;\;50\,].$$

Focus

Mirror image of View 2: **every row of AB lives in the row space of B.** The *row space* $\text{row}(\mathbf{B})$ is the span of $\mathbf{B}$'s rows. Multiplying on the *left* by $\mathbf{A}$ only mixes together the existing rows of $\mathbf{B}$, so it cannot create new row directions either: $\text{rank}(\mathbf{AB}) \leq \text{rank}(\mathbf{B})$.
Combining both views gives the fundamental bottleneck of matrix multiplication:

$$\text{rank}(\mathbf{AB}) \leq \min\big(\text{rank}(\mathbf{A}),\, \text{rank}(\mathbf{B})\big).$$

A product can never have more independent output directions than either of its factors.

8.2.4 View 4: the outer-product (sum-of-rank-one) view

Here is the view that looks strangest and pays off the most later. Instead of building the answer entry by entry, build it as a *sum of whole matrices*. Multiply column k of $\mathbf{A}$ by row k of $\mathbf{B}$ — a

column times a row is a full matrix, an *outer product* — and add up over k:

$$\mathbf{AB} = \sum_k (\text{col } k \text{ of } \mathbf{A})(\text{row } k \text{ of } \mathbf{B}).$$

A column times a row, like $\begin{bmatrix}1\\3\end{bmatrix}\begin{bmatrix}5 & 6\end{bmatrix}$, fills out a whole matrix:

$$\begin{bmatrix}1\\3\end{bmatrix}\begin{bmatrix}5 & 6\end{bmatrix} = \begin{bmatrix}1\cdot 5 & 1\cdot 6\\ 3\cdot 5 & 3\cdot 6\end{bmatrix} = \begin{bmatrix}5 & 6\\ 15 & 18\end{bmatrix}.$$

That is an outer product. It has *rank one*: every row is a scalar multiple of $\begin{bmatrix}5 & 6\end{bmatrix}$, and every column is a scalar multiple of $\begin{bmatrix}1\\3\end{bmatrix}$. Geometrically, a rank-one matrix is the simplest non-zero linear machine imaginable — it squashes the entire plane onto a single line. Now form the second one from column 2 of $\mathbf{A}$ and row 2 of $\mathbf{B}$:

$$\begin{bmatrix}2\\4\end{bmatrix}\begin{bmatrix}7 & 8\end{bmatrix} = \begin{bmatrix}14 & 16\\ 28 & 32\end{bmatrix}.$$

Add the two rank-one pieces:

$$\mathbf{AB} = \begin{bmatrix}5 & 6\\ 15 & 18\end{bmatrix} + \begin{bmatrix}14 & 16\\ 28 & 32\end{bmatrix} = \begin{bmatrix}19 & 22\\ 43 & 50\end{bmatrix}.$$

Same answer, fourth time. The product is a *layered sum of simple rank-one slabs.*

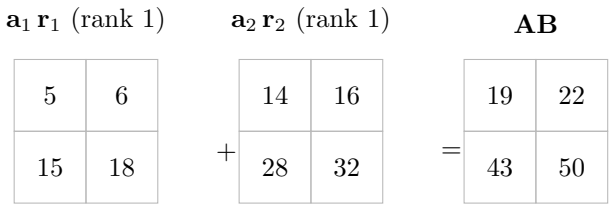


Figure 8.3: The outer-product view: $\mathbf{AB}$ is a sum of rank-one layers, one per column of $\mathbf{A}$ paired with the matching row of $\mathbf{B}$.

Modern application

Low-rank approximation and recommender systems. The outer-product view is the gateway to the most important idea in modern data science: a big matrix can often be *well approximated* by a short sum of rank-one layers. The singular value decomposition (Chapter 29) writes any matrix as $\mathbf{A} = \sum_k \sigma_k\,\mathbf{u}_k\mathbf{v}_k^{\mathsf{T}}$, a sum of outer products weighted by the singular values σ_k. Keeping only the few largest terms gives the best low-rank approximation. When Netflix predicts how you would rate an unseen film, it is modeling the giant users-by-movies rating matrix as a short sum of such layers: a few hidden "taste directions" $\mathbf{u}_k$ times a few "genre directions" $\mathbf{v}_k^{\mathsf{T}}$. Image compression, latent semantic analysis in search, and the embeddings inside language models all live in this same picture. View 4 is where it starts.

Walking question

Take a walk and consider why all four views must agree. Notice how they partition the same calculation: the entries $(\mathbf{AB})_{ij} = \sum_k \mathbf{A}_{ik}\mathbf{B}_{kj}$ share the same sum over k. If you fix j and collect all i, you get columns (View 2). If you fix i and collect all j, you get rows (View 3). And if you pull out one k at a time across all i, j at once, you get the rank-one layers (View 4). Four views, one shared sum, grouped differently.

8.3 The Algebra of Multiplication

Now that we know what the product *is*, what rules does it obey? Some are comforting, one is famously not.

8.3.1 It is associative (and the reason is composition)

Theorem 8.1 (Associativity)**.** *For any matrices of compatible shapes,*

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC}).$$

Proving this by expanding triple sums of subscripts verifies the identity algebraically, but obscures why it is true. The conceptual reason is composition. Each matrix represents a function. **ABC** means "apply **C**, then **B**, then **A**," and that ordered sequence of transformations does not depend on grouping. Grouping the first two or the last two does not change which transformation acts when. Associativity of matrix multiplication is just associativity of function composition.

Formal

Why composition is associative. For functions, $((f \circ g) \circ h)(x) = (f \circ g)(h(x)) = f(g(h(x)))$ and $(f \circ (g \circ h))(x) = f((g \circ h)(x)) = f(g(h(x)))$. Both equal $f(g(h(x)))$. Since a matrix is exactly the function $\mathbf{x} \mapsto \mathbf{Ax}$, and matrix product is composition, associativity is inherited. No subscripts required.

Why this matters

Associativity does more work than it first appears to. It is the reason a chain like **ABCD** is unambiguous, so factorizations (**LU**, **QR**, $\mathbf{U\Sigma V}^\top$) are even meaningful. It is the reason you can choose the *cheapest order* to multiply a long chain — multiplying $\mathbf{A}(\mathbf{Bx})$ as matrix-times-vector costs far less than forming **AB** first — a technique that powers efficient deep-learning forward passes. One algebraic law has consequences across the entire subject.

8.3.2 It is NOT commutative

Theorem 8.2 (Order matters)**.** *In general* $\mathbf{AB} \neq \mathbf{BA}$.

The cleanest proof is one example, and our opening machines already do it. Rotating then stretching is genuinely different from stretching then rotating. Let

$$\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \text{ (rotate } 90°\text{)}, \qquad \mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \text{ (stretch } x\text{)}.$$

We already found $\mathbf{AB} = \begin{bmatrix} 0 & -1 \\ 2 & 0 \end{bmatrix}$. Now the other order:

$$\mathbf{BA} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 1 & 0 \end{bmatrix}.$$

These are different matrices. Watch them act on $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. With $\mathbf{AB}$: stretch $\mathbf{e}_1$ to $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$, then rotate to $\begin{bmatrix} 0 \\ 2 \end{bmatrix}$. With $\mathbf{BA}$: rotate $\mathbf{e}_1$ to $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, then stretch the x-coordinate (which is already 0) to get $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$. One ends at $\begin{bmatrix} 0 \\ 2 \end{bmatrix}$, the other at $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$. The stretch lands on a different direction depending on whether the rotation happened before or after it.



$\mathbf{AB}\,\mathbf{e}_1$
stretch *then* rotate

$\mathbf{BA}\,\mathbf{e}_1$
rotate *then* stretch

Figure 8.4: Non-commutativity you can see. The same starting vector $\mathbf{e}_1$ ends up in different places depending on the order of the rotation and the stretch.

Common trap

Because $\mathbf{AB} \neq \mathbf{BA}$, ordinary algebra identities can fail. For matrices,

$$(\mathbf{A} + \mathbf{B})^2 = \mathbf{A}^2 + \mathbf{AB} + \mathbf{BA} + \mathbf{B}^2,$$

not $\mathbf{A}^2 + 2\mathbf{AB} + \mathbf{B}^2$, because you may not merge $\mathbf{AB}$ with $\mathbf{BA}$. Likewise $(\mathbf{AB})^2 = \mathbf{ABAB}$, which you cannot rearrange into $\mathbf{A}^2\mathbf{B}^2$. When you find yourself "swapping factors," stop and ask whether you earned it.

Pause

Pause and think: Why is $(\mathbf{A} + \mathbf{B})(\mathbf{A} - \mathbf{B})$ almost never equal to $\mathbf{A}^2 - \mathbf{B}^2$?
Answer: Expand carefully using the distributive laws:

$$(\mathbf{A} + \mathbf{B})(\mathbf{A} - \mathbf{B}) = \mathbf{A}^2 - \mathbf{AB} + \mathbf{BA} - \mathbf{B}^2.$$

In scalar arithmetic, the cross terms $-ab + ba$ cancel to zero. But for matrices, $-\mathbf{AB} + \mathbf{BA}$ cancels *only* when $\mathbf{AB} = \mathbf{BA}$. When order matters, familiar difference-of-squares formulas break down.

8.3.3 The rules that DO hold

Multiplication is still well behaved in every way except commuting. For compatible shapes and any scalar c:

- **Distributive (left):** $\mathbf{A}(\mathbf{B}+\mathbf{C}) = \mathbf{AB}+\mathbf{AC}$.
- **Distributive (right):** $(\mathbf{A}+\mathbf{B})\mathbf{C} = \mathbf{AC}+\mathbf{BC}$.
- **Scalars slide:** $c(\mathbf{AB}) = (c\mathbf{A})\mathbf{B} = \mathbf{A}(c\mathbf{B})$.
- **Associative:** $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.

Each follows because the entries are built from sums and products of real numbers, where these laws already hold; the matrix versions just package many scalar identities at once. Keep both distributive laws separate in your mind — with no commuting, "left" and "right" are genuinely different operations.

8.4 The Identity: the Machine That Does Nothing

Every kind of multiplication has a "1." For $n \times n$ matrices it is the *identity matrix* $\mathbf{I}$ — ones on the diagonal, zeros elsewhere:

$$\mathbf{I}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Intuition

$\mathbf{I}$ is the machine that leaves every vector exactly where it is: $\mathbf{Ix} = \mathbf{x}$ for all $\mathbf{x}$. Its columns are the basis vectors $\mathbf{e}_1, \ldots, \mathbf{e}_n$ sitting in their home positions. Geometrically it does nothing at all — and that is precisely why it acts as the "1" of matrix multiplication.

For any matrix $\mathbf{A}$ of the right shape, $\mathbf{IA} = \mathbf{A}$ and $\mathbf{AI} = \mathbf{A}$. Quick check via the column view: the j-th column of $\mathbf{AI}$ is $\mathbf{A}$ times the j-th column of $\mathbf{I}$, which is $\mathbf{Ae}_j$ — and $\mathbf{Ae}_j$ is just the j-th column of $\mathbf{A}$. So $\mathbf{AI}$ has the same columns as $\mathbf{A}$. Done.

Remark 8.1. $\mathbf{I}$ is the rare matrix that commutes with *everything*: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$. So is any scalar multiple $c\mathbf{I}$ (a uniform scaling). Apart from these, expecting two matrices to commute is the exception, not the rule.

8.5 The Inverse: Undoing a Machine

We can now ask the chapter's second big question. The machine $\mathbf{A}$ sends $\mathbf{x}$ to $\mathbf{Ax}$. Is there a machine that sends it back?

Definition 8.1 (Inverse matrix)**.** A square matrix $\mathbf{A}$ is *invertible* (or *nonsingular*) if there is a square matrix $\mathbf{A}^{-1}$ of the same size with

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \qquad \text{and} \qquad \mathbf{A}\,\mathbf{A}^{-1} = \mathbf{I}.$$

If no such matrix exists, $\mathbf{A}$ is *singular*.

Intuition

$\mathbf{A}^{-1}$ is the "rewind" machine. Run $\mathbf{x}$ forward through $\mathbf{A}$ to get $\mathbf{b} = \mathbf{A}\mathbf{x}$; run $\mathbf{b}$ through $\mathbf{A}^{-1}$ and you are back at $\mathbf{x}$, because $\mathbf{A}^{-1}\mathbf{b} = \mathbf{A}^{-1}\mathbf{A}\mathbf{x} = \mathbf{I}\mathbf{x} = \mathbf{x}$. The inverse exists exactly when the forward machine loses no information — when you could always tell, from the output, what the input must have been.

This information-loss picture explains exactly *when* an inverse exists. If $\mathbf{A}$ ever sends two different inputs to the same output, the rewind is impossible: seeing the output, you cannot know which input it came from. So $\mathbf{A}$ is invertible exactly when it is *one-to-one* (different inputs never collide). For a square matrix, being one-to-one turns out to be equivalent to hitting every target in the space, having linearly independent columns, and possessing a nonzero *determinant*.

The *determinant*, written $\det \mathbf{A}$, is a single number that measures the factor by which $\mathbf{A}$ stretches or shrinks area (in two dimensions) or volume (in higher dimensions). If $\det \mathbf{A} = 0$, space is flattened into a lower dimension, destroying information and making an inverse impossible. We will explore determinants systematically in Chapter 21 (Section 21.1), but in two dimensions the formula is immediate.

8.5.1 The 2×2 formula

In two dimensions there is a clean closed form worth memorizing.

Theorem 8.3 (Inverse of a 2×2)**.** *Let* $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$. *The area-scaling factor of* $\mathbf{A}$ *is its* 2×2 *determinant,*

$$\det \mathbf{A} = ad - bc.$$

If $\det \mathbf{A} \neq 0$*, then* $\mathbf{A}$ *is invertible and*

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

If $ad - bc = 0$*, then* $\mathbf{A}$ *has no inverse.*

Proof. Multiply and see. Let $\mathbf{M} = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$. Then

$$\mathbf{A}\mathbf{M} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \begin{bmatrix} ad - bc & -ab + ab \\ cd - dc & -bc + ad \end{bmatrix} = (ad - bc)\,\mathbf{I}.$$

So if $ad - bc \neq 0$ we may divide by it, and $\frac{1}{ad-bc}\mathbf{M}$ is the inverse. If $ad - bc = 0$ then $\mathbf{A}\mathbf{M} = \mathbf{0}$ with $\mathbf{M} \neq \mathbf{0}$ unless $\mathbf{A} = \mathbf{0}$; an invertible $\mathbf{A}$ could be cancelled to force $\mathbf{M} = \mathbf{0}$, a contradiction — so $\mathbf{A}$ cannot be invertible. □

Compute it yourself

Invert $\mathbf{A} = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$ by the formula. The determinant is $4 \cdot 6 - 7 \cdot 2 = 24 - 14 = 10$. Swap the diagonal, negate the off-diagonal, divide by 10:

$$\mathbf{A}^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}.$$

Now *check it* (always check):

$$\mathbf{A}\mathbf{A}^{-1} = \frac{1}{10} \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \frac{1}{10} \begin{bmatrix} 24-14 & -28+28 \\ 12-12 & -14+24 \end{bmatrix} = \frac{1}{10} \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} = \mathbf{I}. \checkmark$$

8.5.2 The geometry of singular: area collapse

Why is $\det \mathbf{A} = ad - bc$ the make-or-break number? Because, as you will prove in the determinant chapter, $|\det \mathbf{A}|$ is the *area* of the parallelogram that $\mathbf{A}$ makes from the unit square — the factor by which $\mathbf{A}$ scales area.

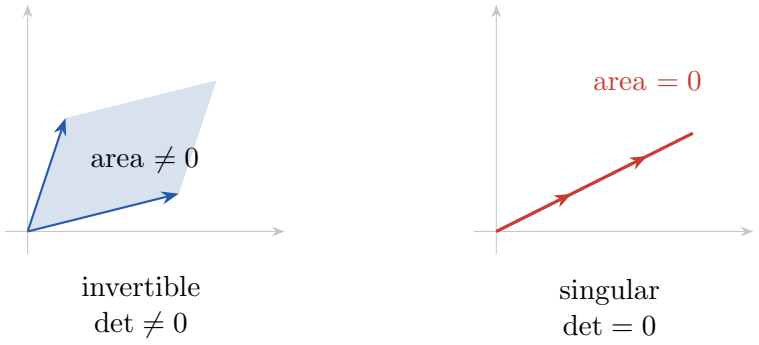


Figure 8.5: Determinant as area. An invertible matrix turns the unit square into a parallelogram with nonzero area. A singular matrix flattens it onto a line — the columns are parallel, area is zero, and the collapse cannot be undone.

Intuition

If $\det \mathbf{A} = 0$, the matrix squashes the plane onto a line (or a point). A whole direction's worth of vectors gets crushed to zero. Once an entire line of inputs lands on a single output, no machine can pull them back apart — the rewind would have to send one point to many. **Zero determinant means lost information means no inverse.** That is the geometry behind "singular."

8.5.3 The big equivalence (collect everything)

For square matrices, a long list of statements all say the same thing. You have met some already; the rest arrive in the chapters on subspaces, rank, and determinants. We gather them so you can see the unity.

Theorem 8.4 (The Invertible Matrix Theorem, first installment)**.** *For a square matrix* $\mathbf{A}$ *of size* $n \times n$*, the following are equivalent — all true together or all false together:*

1. $\mathbf{A}$ *is invertible.*
2. $\mathbf{Ax} = \mathbf{b}$ *has exactly one solution for every* $\mathbf{b}$*.*
3. $\mathbf{Ax} = \mathbf{0}$ *has only the trivial solution* $\mathbf{x} = \mathbf{0}$*.*
4. *The columns of* $\mathbf{A}$ *are linearly independent.*
5. *The columns of* $\mathbf{A}$ *span all of* $\mathbb{R}^n$*.*
6. $\mathbf{A}$ *row-reduces to the identity* $\mathbf{I}$ *(it has* n *pivots).*
7. $\det \mathbf{A} \neq 0$*.*

We will not prove the whole chain here — several links wait for later chapters — but the spine is the rewind picture. Invertibility (1) is exactly "no information lost," which is exactly "nothing nonzero gets crushed to zero" (3), which is exactly "independent columns" (4). Everything else hangs from that spine.

Before/after Strang

Strang loves to organize the whole course around this equivalence; he often calls it "the most important theorem in linear algebra" and returns to it again and again, adding a new equivalent statement each time a new concept appears (pivots, rank, nullspace, eigenvalues, even singular values). **Before Strang:** read this list as seven ways of saying "the machine is reversible." **After Strang:** you will be able to prove every arrow, and you will have added "0 is not an eigenvalue" and "all singular values positive" to the bottom of the list.

8.6 Computing the Inverse with Gauss–Jordan

The 2×2 formula does not scale — the 3×3 closed form is already ugly, and beyond that, hopeless. We need an algorithm. Happily it is just elimination, run with a clever bookkeeping trick.

8.6.1 The idea: solve n systems at once

What is the inverse, really? Its columns $\mathbf{x}_1, \ldots, \mathbf{x}_n$ must satisfy

$$\mathbf{Ax}_1 = \mathbf{e}_1, \quad \mathbf{Ax}_2 = \mathbf{e}_2, \quad \ldots, \quad \mathbf{Ax}_n = \mathbf{e}_n,$$

because then $\mathbf{A}[\,\mathbf{x}_1 \ \cdots \ \mathbf{x}_n\,] = [\,\mathbf{e}_1 \ \cdots \ \mathbf{e}_n\,] = \mathbf{I}$, which says exactly that $[\,\mathbf{x}_1 \ \cdots \ \mathbf{x}_n\,] = \mathbf{A}^{-1}$. So finding the inverse is the same as solving n linear systems that share the same left-hand side $\mathbf{A}$. Since elimination on $\mathbf{A}$ is identical across all n systems, we do it once on all the right-hand sides simultaneously.

Focus

The Gauss–Jordan recipe. Form the wide *augmented matrix* $[\,\mathbf{A} \mid \mathbf{I}\,]$. Row-reduce until the left block becomes $\mathbf{I}$. Whatever the right block has become is $\mathbf{A}^{-1}$:

$$[\,\mathbf{A} \mid \mathbf{I}\,] \xrightarrow{\text{row operations}} [\,\mathbf{I} \mid \mathbf{A}^{-1}\,].$$

If the left block cannot be reduced to $\mathbf{I}$ (a zero row appears), then $\mathbf{A}$ is singular and has no inverse.

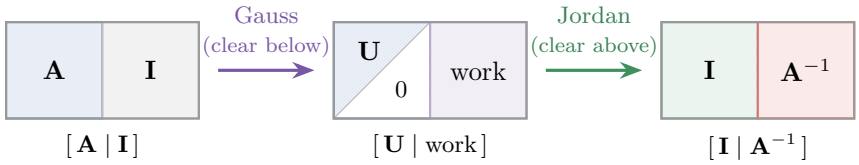


Figure 8.6: The two sweeps of Gauss–Jordan elimination. Standard Gaussian elimination sweeps *downward* to clear entries below pivots, converting $\mathbf{A}$ into an upper-triangular matrix $\mathbf{U}$. Jordan elimination then sweeps *upward* to clear entries above pivots and scale each pivot to 1. The identity matrix on the right passively records every row operation, emerging as the inverse $\mathbf{A}^{-1}$.

Why does the right block come out as $\mathbf{A}^{-1}$? Every row operation is itself multiplication on the left by some elementary matrix $\mathbf{E}$. If the full sequence is $\mathbf{E} = \mathbf{E}_k \cdots \mathbf{E}_2\mathbf{E}_1$, then reducing the left block to $\mathbf{I}$ means $\mathbf{EA} = \mathbf{I}$, so $\mathbf{E} = \mathbf{A}^{-1}$. But the *same* $\mathbf{E}$ acts on the right block, turning $\mathbf{I}$ into $\mathbf{EI} = \mathbf{E} = \mathbf{A}^{-1}$. The augmented matrix is a machine for applying $\mathbf{A}^{-1}$ to both blocks at once; we just watch what it does to the identity.

8.6.2 A full 3×3 worked example

Let us invert

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 2 \\ 1 & 0 & 0 \end{bmatrix}.$$

Start by augmenting with $\mathbf{I}$:

$$\left[\begin{array}{ccc|ccc} 2 & 1 & 1 & 1 & 0 & 0 \\ 1 & 3 & 2 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \end{array}\right].$$

Step 1. The bottom row is the simplest, so swap rows 1 and 3 to put a clean pivot 1 in the top-left ($R_1 \leftrightarrow R_3$):

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 3 & 2 & 0 & 1 & 0 \\ 2 & 1 & 1 & 1 & 0 & 0 \end{array}\right].$$

Step 2. Clear the rest of column 1. Replace $R_2 \to R_2 - R_1$ and $R_3 \to R_3 - 2R_1$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 3 & 2 & 0 & 1 & -1 \\ 0 & 1 & 1 & 1 & 0 & -2 \end{array}\right].$$

Step 3. We want a clean pivot in column 2. Row 3 has a 1 there, so swap $R_2 \leftrightarrow R_3$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & -2 \\ 0 & 3 & 2 & 0 & 1 & -1 \end{array}\right].$$

Step 4. Clear below the second pivot: $R_3 \to R_3 - 3R_2$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & -2 \\ 0 & 0 & -1 & -3 & 1 & 5 \end{array}\right].$$

Step 5. Make the third pivot $+1$: $R_3 \to -R_3$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & -2 \\ 0 & 0 & 1 & 3 & -1 & -5 \end{array}\right].$$

Step 6. Now eliminate *upward* to clear above the third pivot. Only R_2 has an entry there: $R_2 \to R_2 - R_3$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & -2 & 1 & 3 \\ 0 & 0 & 1 & 3 & -1 & -5 \end{array}\right].$$

The left block is now $\mathbf{I}$ (column 1 was already clean above the pivot). Read off the right block:

$$\mathbf{A}^{-1} = \begin{bmatrix} 0 & 0 & 1 \\ -2 & 1 & 3 \\ 3 & -1 & -5 \end{bmatrix}.$$

Always verify. Multiply $\mathbf{A}\mathbf{A}^{-1}$ and demand $\mathbf{I}$:

$$\begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 2 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ -2 & 1 & 3 \\ 3 & -1 & -5 \end{bmatrix}.$$

Row 1: $(2\cdot 0 + 1\cdot(-2) + 1\cdot 3,\ \ 2\cdot 0 + 1\cdot 1 + 1\cdot(-1),\ \ 2\cdot 1 + 1\cdot 3 + 1\cdot(-5)) = (1, 0, 0)$. Row 2: $(0 - 6 + 6,\ 0 + 3 - 2,\ 1 + 9 - 10) = (0, 1, 0)$. Row 3: $(0 + 0 + 0,\ 0,\ 1 + 0 + 0) = (0, 0, 1)$. Together that is exactly $\mathbf{I}$. ✓

Common trap

Two classic Gauss–Jordan slips. First, **do the row operation to the whole row** — both blocks, every time. It is easy to update the left and forget the right. Second, watch your **direction**: ordinary elimination only clears *below* each pivot (giving an upper-triangular $\mathbf{U}$); for the inverse you must also clear *above*, all the way to a clean $\mathbf{I}$. "Gauss" goes down; "Jordan" adds the trip back up.

Compute it yourself

Your turn, by hand. Invert $\mathbf{A} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ with $[\,\mathbf{A} \mid \mathbf{I}\,]$. It is upper triangular, so you only ever clear upward; no row swaps needed. (Answer: $\mathbf{A}^{-1} = \begin{bmatrix} 1 & -2 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$ — check it.) Notice how the -2 and the $+2$ track the two elimination steps; the inverse remembers exactly what you undid.

8.6.3 What singular looks like in the algorithm

If $\mathbf{A}$ is singular, elimination cannot manufacture a missing pivot. Somewhere a row of the left block becomes all zeros. The moment that happens, stop — there is no inverse. Try it on $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ (note row 2 is twice row 1, so $\det = 4 - 4 = 0$):

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 2 & 4 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \to R_2 - 2R_1} \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 0 & -2 & 1 \end{array}\right].$$

The left block has a zero row; it can never become $\mathbf{I}$. The algorithm itself *detects* singularity — you do not need to compute a determinant first. The collapse you saw geometrically (the two columns $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\begin{bmatrix} 2 \\ 4 \end{bmatrix}$ are parallel, so the unit square flattens onto a line) shows up as that zero row.

8.7 The Inverse of a Product: Socks and Shoes

We close with one identity you will use constantly, and it has the most memorable mnemonic in the subject.

Theorem 8.5 (Inverse of a product)**.** *If* $\mathbf{A}$ *and* $\mathbf{B}$ *are both invertible (and the same size), then* $\mathbf{AB}$ *is invertible and*

$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}.$$

The order reverses.

Proof. Just multiply the proposed inverse against $\mathbf{AB}$ and watch it collapse, using associativity to

regroup:

$$(\mathbf{AB})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}(\mathbf{B}\mathbf{B}^{-1})\mathbf{A}^{-1} = \mathbf{A}\,\mathbf{I}\,\mathbf{A}^{-1} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

The same cancellation from the other side gives $(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{AB}) = \mathbf{I}$. By Definition 8.1, $\mathbf{B}^{-1}\mathbf{A}^{-1}$ is the inverse of $\mathbf{AB}$. □

Intuition

Socks and shoes. In the morning you put on socks ($\mathbf{B}$), then shoes ($\mathbf{A}$). To undo it at night you cannot take off the socks first — the shoes are in the way. You reverse the order: shoes off ($\mathbf{A}^{-1}$), then socks off ($\mathbf{B}^{-1}$). To undo a composition, undo the *last* thing first. That is exactly $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.

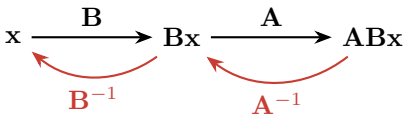


Figure 8.7: Forward (black) you apply $\mathbf{B}$ then $\mathbf{A}$. To rewind (red) you must peel them off in reverse: $\mathbf{A}^{-1}$ first, then $\mathbf{B}^{-1}$. Hence $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.

The pattern extends to any chain: $(\mathbf{ABC})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$, reverse the whole order. The same reversal governs transposes, $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$, which you will meet in Chapter 10 – two different operations, one shared "order reverses" rhythm.

Connection to other math

Groups. The invertible $n \times n$ matrices, with multiplication, form a *group* — the *general linear group* $GL_n(\mathbb{R})$. The four axioms are exactly what this chapter established: multiplication is associative (Theorem 8.1), there is an identity $\mathbf{I}$, every element has an inverse, and the product of two invertibles is invertible with $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$. The order-reversing inverse rule is a *general* feature of groups, not a quirk of matrices — it holds for symmetries of a cube, permutations of a deck, and the rotations of space. When physicists speak of symmetry groups and Lie groups underlying the laws of nature, GL_n and its subgroups (rotations SO_n, volume-preserving maps SL_n) are the founding examples. You have just verified the group axioms by hand.

Remark 8.2 (A useful shortcut you have earned). To *prove* a square matrix $\mathbf{C}$ is invertible with inverse $\mathbf{D}$, you do not have to check both $\mathbf{CD} = \mathbf{I}$ and $\mathbf{DC} = \mathbf{I}$. For square matrices, either one alone implies the other (a fact that follows from the equivalence in Theorem 8.4). That is why the one-sided check in the proof above was enough. Be careful, though: this rescue is for *square* matrices only. Non-square matrices can have a one-sided inverse on the left or the right but never both at once.

Looking Back, Looking Ahead

Two questions, two answers. *What is two machines in a row?* A single matrix $\mathbf{AB}$, best understood through the four views — entries, columns, rows, and rank-one layers — and obeying every algebra

rule except commuting. *Can we undo a machine?* Only when it loses no information; then the rewind is $\mathbf{A}^{-1}$, computed by Gauss–Jordan, and products rewind in reverse order.

Focus

Cost and intuition, side by side. The dot-product view tells you the *price*: an $n \times n$ product is about n^3 multiply–adds, and inverting an $n \times n$ matrix by Gauss–Jordan is the same n^3 order of work (elimination on n right-hand sides at once). The other three views tell you the *meaning*: columns of $\mathbf{AB}$ live in $\text{col}(\mathbf{A})$, rows live in $\text{row}(\mathbf{B})$, and the whole product is a sum of rank-one layers. Price comes from View 1; understanding — and every shortcut that beats the naive n^3 — comes from Views 2–4.

These tools turn the next several chapters into a story of *factoring* transformations into simpler ones. $\mathbf{A} = \mathbf{LU}$ records elimination itself as a product. $\mathbf{A} = \mathbf{QR}$ separates rotation from stretch. And the singular value decomposition, $\mathbf{A} = \mathbf{U\Sigma V}^{\mathsf{T}}$, shows that *every* matrix decomposes into a rotation, followed by axis-aligned stretching, followed by another rotation. All of these structural factorizations are built on the foundations of matrix multiplication and inversion.

History and connection

The algebraic language of matrix multiplication and inversion was developed by Arthur Cayley in his 1858 paper *A Memoir on the Theory of Matrices*. Cayley was wrestling with a practical algebraic headache: *linear substitutions*. In the geometry and algebra of the 1850s, mathematicians frequently changed coordinate systems by substituting one set of linear equations into another. Doing two substitutions in succession meant filling pages with tedious, error-prone expansions of coefficients by hand. Cayley realized that if you pack the coefficients of each substitution into a single rectangular grid, the entire two-step substitution collapses into a single algebraic product of two grids.

Before Cayley, mathematicians already worked with *determinants* (invented a century earlier by Gabriel Cramer and Colin Maclaurin to solve systems of equations). Sylvester had coined "matrix" in 1850 as a table from which square determinant minors were extracted; Cayley turned the array itself into an algebraic object equipped with addition, multiplication, and inverses ($\mathbf{AA}^{-1} = \mathbf{I}$).

Yet this new algebra triggered immediate controversy. For thousands of years, arithmetic had obeyed the sacred rule that order does not matter ($ab = ba$). Cayley's product broke that rule: $\mathbf{AB} \neq \mathbf{BA}$. Many contemporaries dismissed matrices as an unnatural, bizarre curiosity. Cayley himself believed his matrices were an exercise in pure abstract art, famously confessing that he developed matrix algebra solely for its internal beauty and doubted it would ever have practical utility in the physical world.

Physics soon proved him wrong. In 1925, Werner Heisenberg formulated quantum mechanics by tracking energy transitions in atoms, inventing strange multiplication tables that did not commute. When physicist Max Born saw Heisenberg's bizarre tables, he recognized that Heisenberg had independently rediscovered Cayley's non-commutative matrix algebra. Matrix mechanics became the very language of the quantum universe.

A second controversy erupted a century later with the rise of digital computers. Generations of textbooks had taught Gabriel Cramer's 1750 determinant rule ($x_i = \det(\mathbf{A}_i)/\det(\mathbf{A})$) for solving $\mathbf{Ax} = \mathbf{b}$. Cramer's rule is elegant on paper, but a computational nightmare: evaluating $n + 1$ determinants by expansion takes $(n + 1)!$ arithmetic operations — for a 25×25 system, that exceeds the age of the universe. When early programmers abandoned

Cramer, they fell into an opposite trap: they computed $\mathbf{A}^{-1}$ explicitly and multiplied $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. In the 1960s, numerical pioneers George Forsythe and Cleve Moler campaigned to end this habit. Computing $\mathbf{A}^{-1}$ takes three times as many operations as elimination, doubles accumulated roundoff error, and destroys *sparsity*. In engineering simulations (heat flow, power grids, structural mechanics), matrices often contain millions of entries that are almost entirely zeros (sparse matrices), requiring very little memory. But the inverse $\mathbf{A}^{-1}$ is almost always **completely dense**: every single entry fills with nonzero numbers, instantly exhausting computer RAM.

What remains open today is the ultimate speed of matrix multiplication itself. For over a century, everyone assumed multiplying two $n \times n$ matrices inherently required roughly n^3 operations (the dot-product view). In 1969, Volker Strassen showed that multiplying 2×2 matrices requires only 7 scalar multiplications instead of 8, yielding an $O(n^{2.807})$ algorithm. Strassen proved that n^3 is not an immutable law of nature. Since then, a half-century race has driven the theoretical exponent ω down to approximately $n^{2.37155}$ (through breakthroughs by Josh Alman, Virginia Vassilevska Williams, Ran Duan, and others). Whether the true limit is $\omega = 2$ — meaning multiplying two $n \times n$ matrices takes only $O(n^2)$ work, no more expensive than simply reading their entries — remains one of the greatest unsolved problems in mathematics and theoretical computer science.

8.8 Exercises

Exercise 8.1 (Warm-up by hand)**.** Compute $\mathbf{AB}$ and $\mathbf{BA}$ for $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$. Confirm they are different, and say in one sentence why two shears in different orders give different results.

Exercise 8.2 (All four views)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 1 & 4 \\ 2 & 0 \end{bmatrix}$, compute $\mathbf{AB}$ *four ways*: (a) entry by entry (dot products), (b) as combinations of $\mathbf{A}$'s columns, (c) as combinations of $\mathbf{B}$'s rows, (d) as a sum of two outer products. Verify all four agree.

Exercise 8.3 (Shapes)**.** $\mathbf{A}$ is 3×2, $\mathbf{B}$ is 2×5, $\mathbf{C}$ is 5×3. Which of $\mathbf{AB}$, $\mathbf{BC}$, $\mathbf{CA}$, $\mathbf{ABC}$, $\mathbf{CAB}$ are defined, and what is each one's shape? Which products $\mathbf{XY}$ from this list are square?

Exercise 8.4 (2×2 inverse)**.** Use the formula to invert $\mathbf{A} = \begin{bmatrix} 3 & 5 \\ 1 & 2 \end{bmatrix}$, then check $\mathbf{AA}^{-1} = \mathbf{I}$. What is $\det \mathbf{A}$, and how does it appear in the answer?

Exercise 8.5 (Gauss–Jordan)**.** Invert $\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 1 \end{bmatrix}$ by augmenting with $\mathbf{I}$ and row-reducing to $[\,\mathbf{I} \mid \mathbf{A}^{-1}\,]$. Show every step, and verify your answer.

Exercise 8.6 (Detecting singular)**.** Run Gauss–Jordan on $\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$ far enough to show it is singular. Identify the row that forces the collapse, and describe geometrically what $\mathbf{A}$ does to the unit cube.

Exercise 8.7 (Socks and shoes, with numbers)**.** Let $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$. Compute $\mathbf{A}^{-1}$, $\mathbf{B}^{-1}$, and $\mathbf{AB}$. Verify directly that $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$, and check that $\mathbf{A}^{-1}\mathbf{B}^{-1}$ gives the *wrong* answer (it is not the inverse of $\mathbf{AB}$).

Exercise 8.8 (Rank cannot grow)**.** Using the column view, explain why every column of $\mathbf{AB}$ is a linear combination of the columns of $\mathbf{A}$. Conclude that $\operatorname{rank}(\mathbf{AB}) \le \operatorname{rank}(\mathbf{A})$. Then give a 2×2 example where $\mathbf{A} \neq \mathbf{0}$, $\mathbf{B} \neq \mathbf{0}$, yet $\mathbf{AB} = \mathbf{0}$ — so rank can *drop* all the way to zero.

Exercise 8.9 (Inverse of a diagonal matrix)**.** Let $\mathbf{D} = \operatorname{diag}(d_1, \dots, d_n)$. Show that $\mathbf{D}$ is invertible if and only if every $d_i \neq 0$, and that then $\mathbf{D}^{-1} = \operatorname{diag}(1/d_1, \dots, 1/d_n)$. What goes wrong, geometrically and in Gauss–Jordan, if some $d_i = 0$?

Exercise 8.10 (Proof: inverses are unique)**.** Suppose $\mathbf{B}$ and $\mathbf{C}$ both satisfy $\mathbf{BA} = \mathbf{I}$ and $\mathbf{AC} = \mathbf{I}$. Prove $\mathbf{B} = \mathbf{C}$. (Hint: simplify the product $\mathbf{BAC}$ in two ways, using associativity.) Conclude that a matrix has at most one inverse.

Exercise 8.11 (Challenge: a 2×2 identity)**.** For an invertible 2×2 matrix $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, prove the surprisingly clean fact that

$$\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \big((\operatorname{tr} \mathbf{A})\, \mathbf{I} - \mathbf{A} \big),$$

where $\operatorname{tr} \mathbf{A} = a + d$. (This is the 2×2 case of the Cayley–Hamilton theorem, which you will meet again after eigenvalues. Just expand both sides.)

Exercise 8.12 (Challenge: commuting with everything)**.** Prove that if a square matrix $\mathbf{M}$ satisfies $\mathbf{MA} = \mathbf{AM}$ for *every* matrix $\mathbf{A}$ of the same size, then $\mathbf{M} = c\mathbf{I}$ for some scalar c. (Hint: try $\mathbf{A} = \mathbf{e}_i \mathbf{e}_j^{\mathsf{T}}$, the matrix with a single 1 in position (i, j), and see what commuting forces on the entries of $\mathbf{M}$.)

Chapter 9

LU Factorization

Two chapters ago, you learned to solve $\mathbf{Ax} = \mathbf{b}$ by Gaussian elimination: sweep down the matrix, knocking out the entries below each pivot, until what remains is upper triangular and easy to back-solve. In the last chapter, you saw that row operations can be packaged as matrix multiplications, and that solving a system is intimately tied to inverses. Elimination works. But there is a quiet flaw hidden inside the standard pencil-and-paper routine: every time someone hands you a new right-hand side $\mathbf{b}$, you seem forced to start the entire elimination sweep again from scratch.

This chapter fixes that. The key insight is simple once you see it: *the work of elimination can be bottled up and reused.* All the row operations you performed – every "subtract 3 times row 1 from row 2" – can be recorded once, packaged into a single matrix, and replayed instantly on any new $\mathbf{b}$. That packaging is the LU factorization. We will discover that elimination factors the matrix into two triangular pieces,

$$\mathbf{A} = \mathbf{L}\,\mathbf{U},$$

a **l**ower triangular matrix $\mathbf{L}$ (nonzero entries only on and below the main diagonal) times an **u**pper triangular matrix $\mathbf{U}$ (nonzero entries only on and above the main diagonal). Once you have $\mathbf{L}$ and $\mathbf{U}$, solving a system takes two fast triangular solves instead of a full elimination sweep. This factorization is the standard algorithm that solves linear systems inside scientific and engineering software.

We will build it picture first, show how the multipliers form $\mathbf{L}$, handle row exchanges, and count operations to see why factoring once and reusing beats recomputing.

9.1 The picture: triangular machines are easy machines

Before any formulas, hold onto a feeling for what triangular matrices *do* to space.

A matrix is a machine that takes a vector in and pushes a vector out. An *upper triangular* matrix

$$\mathbf{U} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

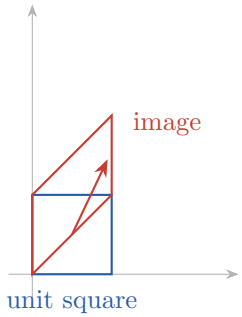
has all its action "above the diagonal." Look at what it does to the standard basis. The first basis vector $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ goes to $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$, staying flat on the x-axis – the x-axis is left as a line through the origin,

just stretched. The second basis vector $\begin{bmatrix}0\\1\end{bmatrix}$ goes to $\begin{bmatrix}1\\3\end{bmatrix}$, which leans to the right. Upper triangular means: *the first axis stays put as a direction; later axes are allowed to spill into the earlier ones.*

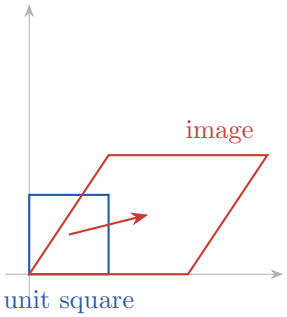
A *lower triangular* matrix

$$\mathbf{L} = \begin{bmatrix}1 & 0\\3 & 1\end{bmatrix}$$

is the mirror image. Here $\begin{bmatrix}1\\0\end{bmatrix}$ goes to $\begin{bmatrix}1\\3\end{bmatrix}$ (the first axis spills *upward* into the second) while $\begin{bmatrix}0\\1\end{bmatrix}$ stays put as $\begin{bmatrix}0\\1\end{bmatrix}$. This particular $\mathbf{L}$, with 1's on the diagonal, is a **shear**: it slides the plane vertically, keeping the vertical direction fixed while preserving area. We met shears as one of the basic deformations a matrix can apply; now they take on a starring role.



lower triangular: shear



upper triangular: stretch + lean

Figure 9.1: Triangular matrices are the simplest non-trivial deformations. A lower triangular matrix with unit diagonal is a pure shear (left): one axis stays fixed, the other slides along it. An upper triangular matrix stretches the first axis and lets later axes lean into it (right). Factoring $\mathbf{A} = \mathbf{LU}$ writes a general distortion as a stretch-and-lean followed by a shear.

Intuition

LU factorization says: *any* (nice enough) square matrix's action can be done in two simple stages. First apply $\mathbf{U}$ – stretch the space and let later coordinates lean into earlier ones. Then apply $\mathbf{L}$ – a shear that slides earlier coordinates into later ones. Two triangular slides, composed, can reproduce any general matrix. Elimination is the act of *discovering* those two slides.

Why do we care that the pieces are triangular? Because triangular systems are the easiest systems in the world to solve. If $\mathbf{U}$ is upper triangular, the bottom equation involves only the last unknown, so you read it off directly; the second-to-last equation then involves only the last two, and you already know one of them; and so on *up* the list. That is **back substitution**. If $\mathbf{L}$ is lower triangular, the same trick runs top-to-bottom: **forward substitution**. No searching, no choices – just a slide. LU's whole strategy is to trade one hard system for two trivial ones.

Pause

Pause and think: Look at the upper triangular system $\begin{bmatrix}2 & 1\\ 0 & 3\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} = \begin{bmatrix}5\\ 6\end{bmatrix}$. Can you see why it takes zero elimination steps to solve? The bottom equation $3x_2 = 6$ immediately gives $x_2 = 2$. With x_2 in hand, the top equation $2x_1 + (2) = 5$ gives $x_1 = 3/2$. Back substitution is simply reading off answers from bottom to top without ever modifying the matrix.

9.2 Elimination, watched closely

Let us do a single careful elimination and *watch what we throw away*, because the thing we usually throw away is exactly **L**.

Take

$$\mathbf{A} = \begin{bmatrix}2 & 1 & 1\\ 4 & 3 & 3\\ 8 & 7 & 9\end{bmatrix}.$$

The first **pivot** – the leading nonzero entry in the active row that we use as a lever to clear out all entries below it – is the 2 in the top-left corner. To clear the column below it we ask: what multiple of row 1 must we subtract from row 2 to kill the 4? The answer is $4/2 = 2$. We call this ratio the **multiplier** and name it ℓ_{21} (the multiplier that acts on row 2 using row 1):

$$\ell_{21} = \frac{4}{2} = 2, \qquad R_2 \leftarrow R_2 - 2\,R_1.$$

Likewise to kill the 8 in row 3 we need $\ell_{31} = 8/2 = 4$, and $R_3 \leftarrow R_3 - 4\,R_1$. After these two operations:

$$\begin{bmatrix}2 & 1 & 1\\ 4 & 3 & 3\\ 8 & 7 & 9\end{bmatrix} \longrightarrow \begin{bmatrix}2 & 1 & 1\\ 0 & 1 & 1\\ 0 & 3 & 5\end{bmatrix}.$$

The new second pivot is the 1 now sitting in position $(2, 2)$. To clear the 3 below it,

$$\ell_{32} = \frac{3}{1} = 3, \qquad R_3 \leftarrow R_3 - 3\,R_2,$$

giving

$$\begin{bmatrix}2 & 1 & 1\\ 0 & 1 & 1\\ 0 & 3 & 5\end{bmatrix} \longrightarrow \begin{bmatrix}2 & 1 & 1\\ 0 & 1 & 1\\ 0 & 0 & 2\end{bmatrix} = \mathbf{U}.$$

That triangular matrix on the right is **U**. Three multipliers did all the work:

$$\ell_{21} = 2, \quad \ell_{31} = 4, \quad \ell_{32} = 3.$$

Now assemble those multipliers into a lower triangular matrix, with 1's on the diagonal and each ℓ_{ij} sitting in the exact position (i, j) where it was used:

$$\mathbf{L} = \begin{bmatrix}1 & 0 & 0\\ \ell_{21} & 1 & 0\\ \ell_{31} & \ell_{32} & 1\end{bmatrix} = \begin{bmatrix}1 & 0 & 0\\ 2 & 1 & 0\\ 4 & 3 & 1\end{bmatrix}.$$

Claim: $\mathbf{A} = \mathbf{LU}$. Let us check by direct multiplication.

$$\mathbf{LU} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix}.$$

Row 1 of $\mathbf{L}$ is $(1, 0, 0)$, so row 1 of the product is just row 1 of $\mathbf{U}$: $(2, 1, 1)$. Row 2 of $\mathbf{L}$ is $(2, 1, 0)$, so row 2 of the product is $2 \cdot (2, 1, 1) + 1 \cdot (0, 1, 1) = (4, 3, 3)$. Row 3 of $\mathbf{L}$ is $(4, 3, 1)$, so row 3 is $4 \cdot (2, 1, 1) + 3 \cdot (0, 1, 1) + 1 \cdot (0, 0, 2) = (8, 7, 9)$. Stacking,

$$\mathbf{LU} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix} = \mathbf{A}.\checkmark$$

It worked for a clean structural reason. The multipliers you used to *destroy* the lower part of $\mathbf{A}$ are exactly the numbers that, placed in $\mathbf{L}$, *rebuild* it.

Focus

Hold onto three sentences.

- $\mathbf{U}$ is what elimination leaves behind: the upper triangular matrix of pivots.
- $\mathbf{L}$ is the bookkeeping: 1's on the diagonal, and the multiplier ℓ_{ij} = (entry being cleared)/(pivot) in position (i, j).
- No minus signs in $\mathbf{L}$. You *subtracted* ℓ_{ij} times a row during elimination, but $\mathbf{L}$ stores the plain multiplier with a + sign.

Common trap

The single most common LU mistake is putting minus signs into $\mathbf{L}$. During elimination you carried out $R_3 \leftarrow R_3 - 3R_2$, so it feels natural to write -3 in $\mathbf{L}$. **Do not.** The matrix that *performs* the subtraction is $\mathbf{E}$ with a -3; but $\mathbf{L} = \mathbf{E}^{-1}$ *undoes* the subtraction, and undoing a subtraction is an addition. So $\mathbf{L}$ holds $+3$. We will see exactly why in the next section. Quick sanity check: if every multiplier is positive (you only ever subtracted positive multiples), then $\mathbf{L}$ should be all positive below the diagonal.

9.3 Why it works: elimination is left-multiplication by shears

The claim "$\mathbf{L}$ stores the multipliers" deserves a real derivation. The idea behind it is clean: each elementary row operation is itself a matrix, and undoing it simply requires reversing the sign of the multiplier.

Recall that performing a row operation on $\mathbf{A}$ is the same as *left*-multiplying $\mathbf{A}$ by an **elementary matrix** – the identity matrix with a single modification that executes one row operation when it multiplies another matrix from the left. The operation $R_2 \leftarrow R_2 - 2R_1$ on a 3×3 matrix is multiplication by

$$\mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

(Read it row by row: new row 2 $= -2\cdot$ old row 1 $+1\cdot$ old row 2. Exactly the operation.) Our full elimination was three such steps, applied left to right:

$$\mathbf{E}_{32}\,\mathbf{E}_{31}\,\mathbf{E}_{21}\,\mathbf{A} = \mathbf{U},$$

with

$$\mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},\ \mathbf{E}_{31} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix},\ \mathbf{E}_{32} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{bmatrix}.$$

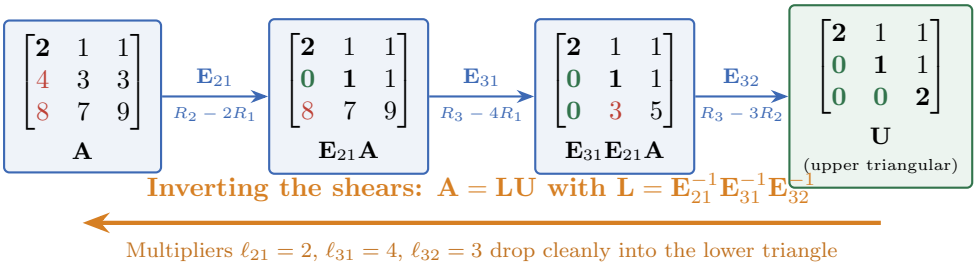


Figure 9.2: Gaussian elimination as a sequence of elementary shears. Each elementary matrix $\mathbf{E}_{ij}$ applies a shear that zeroes out the entry below the diagonal in position (i, j) (marked in green). Inverting the sequence in reverse order reconstitutes $\mathbf{A} = \mathbf{LU}$, where the lower-triangular matrix $\mathbf{L}$ bundles all the positive multipliers with no mutual interference.

Solve for $\mathbf{A}$ by moving the $\mathbf{E}$'s to the other side. Each one is invertible (elimination is reversible – you can always add back what you subtracted), so

$$\mathbf{A} = \mathbf{E}_{21}^{-1}\,\mathbf{E}_{31}^{-1}\,\mathbf{E}_{32}^{-1}\,\mathbf{U}.$$

Now the two gifts. **Gift one:** inverting a single shear just flips the sign of its off-diagonal entry, because to undo "subtract 2 times row 1" you "add 2 times row 1":

$$\mathbf{E}_{21}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ +2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

There is the plus sign the trap warned about. **Gift two:** multiplying these particular inverse shears together does not mix anything up – the multipliers simply drop into their slots with no interaction. Watch:

$$\mathbf{E}_{21}^{-1}\,\mathbf{E}_{31}^{-1}\,\mathbf{E}_{32}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} = \mathbf{L}.$$

Observe this result. If you had multiplied the original forward elementary matrices together in the order they were applied,

$$\mathbf{E}_{32}\mathbf{E}_{31}\mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ +2 & -3 & 1 \end{bmatrix},$$

an unexpected $+2$ appears in position $(3, 1)$: $(-3)(-2) - 4 = +2$. The row operations scrambled each other. But when you *invert* each matrix and *reverse* the product order to form $\mathbf{L} = \mathbf{E}_{21}^{-1}\mathbf{E}_{31}^{-1}\mathbf{E}_{32}^{-1}$,

that interaction vanishes completely. Every multiplier ℓ_{ij} drops cleanly into its own position (i, j) with a positive sign. This is why Strang calls $\mathbf{L}$ the "clean" matrix and $\mathbf{E}$ the "messy" one.

So $\mathbf{A} = \mathbf{LU}$, with $\mathbf{L}$ exactly the multiplier table. The reason no cross-terms appear is worth stating once, because it is the key reason this works.

Formal

Why the multipliers do not interfere. When you eliminate in the natural order – column 1 first (top to bottom), then column 2, and so on – every inverse elementary matrix $\mathbf{E}_{ij}^{-1}$ adds a multiple of an *earlier* row j to a *later* row i (so $i > j$). Multiplying two such matrices, the entry ℓ_{ij} of one never gets touched by the other, because the second factor only modifies rows using columns that the first factor has already finished with. Concretely, for unit lower triangular elementary matrices that each have a single off-diagonal entry below the diagonal, the product is just the matrix containing *all* those entries at once:

$$\prod(\mathbf{E}_{ij}^{-1}) = \mathbf{I} + \sum_{i>j} \ell_{ij}\, \mathbf{e}_i \mathbf{e}_j^{\mathsf{T}} = \mathbf{L}.$$

Here $\mathbf{e}_i$ denotes the i-th standard basis vector (with 1 in row i and zeros elsewhere), so the outer product matrix $\mathbf{e}_i\mathbf{e}_j^{\mathsf{T}}$ has a single 1 in row i, column j and zeros everywhere else. This is special to the with-the-grain ordering. Eliminate out of order and the clean "read off the multipliers" rule breaks.

Walking question

Take a walk and chew on this. You did three subtractions to get to $\mathbf{U}$. To get back to $\mathbf{A}$ you must undo them *in reverse order* – last subtraction undone first. Yet $\mathbf{L}$ lists the multipliers in *forward* order with no scrambling. How can the order not matter for assembling $\mathbf{L}$ even though it absolutely matters for the row operations themselves? (Hint: think about which rows each step reads from versus writes to. A step writes to a row no later step ever reads.)

Before/after Strang

Before Strang: if you have not yet taken his course, just absorb the slogan "$\mathbf{A} = \mathbf{LU}$, where $\mathbf{U}$ is elimination's output and $\mathbf{L}$ is the multipliers." That single sentence will make his Lecture on elimination feel like a story whose ending you already know.
After Strang: Strang loves to write elimination as $\mathbf{EA} = \mathbf{U}$ and then flip it to $\mathbf{A} = \mathbf{E}^{-1}\mathbf{U} = \mathbf{LU}$, emphasizing that $\mathbf{L}$ is the *nicer* matrix because no minus signs survive. He also stresses $\mathbf{LDU}$, which we reach in Section 9.7: pull the pivots out of $\mathbf{U}$ into a diagonal $\mathbf{D}$ so that both triangular factors have unit diagonals and the asymmetry between them disappears.

9.4 Solving $\mathbf{Ax} = \mathbf{b}$: two slides instead of one sweep

Now we collect the payoff. We want $\mathbf{Ax} = \mathbf{b}$, and we know $\mathbf{A} = \mathbf{LU}$, so

$$\mathbf{LUx} = \mathbf{b}.$$

Introduce a helper vector $\mathbf{c} = \mathbf{U}\mathbf{x}$. Then the single problem splits into a chain of two triangular problems:

$$\mathbf{L}\mathbf{c} = \mathbf{b} \quad \text{(forward, solve for } \mathbf{c}\text{)}, \qquad \mathbf{U}\mathbf{x} = \mathbf{c} \quad \text{(backward, solve for } \mathbf{x}\text{)}.$$

First slide down through $\mathbf{L}$ to find $\mathbf{c}$, then slide up through $\mathbf{U}$ to find $\mathbf{x}$. Let us do it with our matrix and a concrete right-hand side $\mathbf{b} = \begin{bmatrix} 5 \\ 11 \\ 27 \end{bmatrix}$.

Forward substitution, $\mathbf{L}\mathbf{c} = \mathbf{b}$.

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 5 \\ 11 \\ 27 \end{bmatrix}.$$

Row 1: $c_1 = 5$. Row 2: $2c_1 + c_2 = 11 \Rightarrow c_2 = 11 - 2(5) = 1$. Row 3: $4c_1 + 3c_2 + c_3 = 27 \Rightarrow c_3 = 27 - 20 - 3 = 4$. So $\mathbf{c} = \begin{bmatrix} 5 \\ 1 \\ 4 \end{bmatrix}$.

Back substitution, $\mathbf{U}\mathbf{x} = \mathbf{c}$.

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \\ 4 \end{bmatrix}.$$

Row 3: $2x_3 = 4 \Rightarrow x_3 = 2$. Row 2: $x_2 + x_3 = 1 \Rightarrow x_2 = 1 - 2 = -1$. Row 1: $2x_1 + x_2 + x_3 = 5 \Rightarrow 2x_1 = 5 - (-1) - 2 = 4 \Rightarrow x_1 = 2$. So

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 2 \end{bmatrix}.$$

Check against the original: $\mathbf{A}\mathbf{x} = \begin{bmatrix} 2 \\ 4 \\ 8 \end{bmatrix}(2) + \begin{bmatrix} 1 \\ 3 \\ 7 \end{bmatrix}(-1) + \begin{bmatrix} 1 \\ 3 \\ 9 \end{bmatrix}(2) = \begin{bmatrix} 4 - 1 + 2 \\ 8 - 3 + 6 \\ 16 - 7 + 18 \end{bmatrix} = \begin{bmatrix} 5 \\ 11 \\ 27 \end{bmatrix} = \mathbf{b}$. ✓

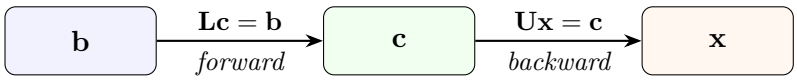


Figure 9.3: Solving with a factorization is a two-stop journey. Slide forward through the lower triangular $\mathbf{L}$ to convert $\mathbf{b}$ into the intermediate $\mathbf{c}$, then slide backward through the upper triangular $\mathbf{U}$ to land on the answer $\mathbf{x}$. Both slides are mechanical; neither involves any elimination.

The key structural benefit lies in the computational cost. Building $\mathbf{L}$ and $\mathbf{U}$ requires the full elimination effort – once. The two triangular substitutions require far fewer operations and can be repeated for each new right-hand side $\mathbf{b}$. We will quantify this cost soon, but the practical advantage is clear: **factor once, substitute many times**.

Compute it yourself

By hand, factor

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 6 & 5 \end{bmatrix}$$

into $\mathbf{LU}$. (Multiplier $\ell_{21} = 6/3 = 2$; subtract 2 times row 1 from row 2.) You should get $\mathbf{L} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$, $\mathbf{U} = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$. Then solve $\mathbf{Ax} = \begin{bmatrix} 4 \\ 11 \end{bmatrix}$ by forward-then-back substitution and confirm $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (you should get the intermediate $\mathbf{c} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$). Finally multiply $\mathbf{LU}$ back out and watch $\mathbf{A}$ reappear.

Pause

Pause and think: Suppose you only ever had to solve $\mathbf{Ax} = \mathbf{b}$ for a *single* right-hand side $\mathbf{b}$, and never again. Does LU factorization save any arithmetic over ordinary Gaussian elimination? **Answer:** No. Factoring $\mathbf{A}$ into $\mathbf{LU}$ costs $\sim \frac{1}{3}n^3$ operations, which is exactly the cost of running elimination on the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$. The two triangular substitutions cost $\sim n^2$, identical to updating $\mathbf{b}$ and back-substituting. LU does not make a single solve faster; its advantage is that for every *subsequent* right-hand side, the $\frac{1}{3}n^3$ elimination cost does not need to be repeated.

9.5 Many right-hand sides: the real reason to factor

Here is the situation that makes LU indispensable. Suppose you must solve

$$\mathbf{Ax}_1 = \mathbf{b}_1, \quad \mathbf{Ax}_2 = \mathbf{b}_2, \quad \ldots, \quad \mathbf{Ax}_k = \mathbf{b}_k,$$

with the *same* matrix $\mathbf{A}$ but many different right-hand sides. This is the common case, not the exception. In circuit simulation $\mathbf{A}$ is the network and each $\mathbf{b}$ is a different set of driving voltages. In a Newton solver for a nonlinear problem, $\mathbf{A}$ is a Jacobian reused across iterations. In finite-element structural analysis $\mathbf{A}$ is the stiffness matrix and each $\mathbf{b}$ is a different load case.

The naive approach re-runs elimination k times: a recompute. The LU approach factors $\mathbf{A} = \mathbf{LU}$ *once* and then does k cheap forward/back pairs. Since elimination is far more expensive than substitution, the savings grow without bound as k grows.

There is a clean way to package "all the right-hand sides at once." Put them as columns of a matrix $\mathbf{B} = [\mathbf{b}_1 \ \mathbf{b}_2 \ \cdots \ \mathbf{b}_k]$ and solve $\mathbf{AX} = \mathbf{B}$ column by column, reusing the one factorization. A particularly important special case is $\mathbf{B} = \mathbf{I}$: solving $\mathbf{AX} = \mathbf{I}$ gives $\mathbf{X} = \mathbf{A}^{-1}$. So the matrix inverse is just LU applied to the n right-hand sides $\mathbf{e}_1, \ldots, \mathbf{e}_n$.

Common trap

This last fact is also a warning. To *solve* $\mathbf{Ax} = \mathbf{b}$ you almost never want to form $\mathbf{A}^{-1}$ first and multiply. Computing $\mathbf{A}^{-1}$ is like doing n full solves and then throwing in an extra matrix-vector multiply at the end – more work, and numerically worse. The professional habit is: factor once, then forward/back-substitute. If someone's code contains `inv(A)*b`, they are

doing it the slow, fragile way; the right call is a triangular solve. “Never invert; always factor.”

Modern application

Real-time engineering and the power grid. An electrical power grid is modeled by a large sparse system $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ encodes the network admittances (which buses connect to which, and how strongly) and $\mathbf{b}$ encodes the loads and generation at this instant. The grid's wiring – the matrix $\mathbf{A}$ – changes only when a line trips or a switch flips, which is rare. But the loads $\mathbf{b}$ change every few seconds as demand fluctuates. So operators factor $\mathbf{A} = \mathbf{LU}$ once (for sparse matrices a specialized sparse LU keeps $\mathbf{L}$ and $\mathbf{U}$ almost as sparse as $\mathbf{A}$) and then re-solve thousands of times a day with fresh $\mathbf{b}$'s by cheap substitution. This same “factor once, substitute often” pattern drives implicit time-stepping in fluid and climate simulation, interior-point optimization, real-time robotics control, and the linear solves buried inside training large statistical models. LU is one of the most-executed numerical algorithms in human history precisely because of this reuse.

9.6 Row exchanges: when a pivot is missing

Everything above assumed elimination glided through without ever hitting a zero where it needed a pivot. Real matrices are not so polite. Consider

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}.$$

The would-be first pivot is the top-left 0. You cannot divide by zero to form a multiplier, so elimination stalls – not because the matrix is bad (it is perfectly invertible) but because the rows are simply in an inconvenient order. The cure is obvious: swap the two rows.

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \xrightarrow{R_1 \leftrightarrow R_2} \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}.$$

Now there is a healthy pivot 2, and the matrix is already upper triangular (Figure 9.4). A row swap is itself a matrix – a **permutation matrix** $\mathbf{P}$, which is the identity with its rows reordered:

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{PA} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}.$$

Left-multiplying by $\mathbf{P}$ reorders the rows of whatever it hits. Permutation matrices are wonderfully simple: each row and each column has exactly one 1 and the rest 0, they just shuffle, and their inverse is their transpose ($\mathbf{P}^{-1} = \mathbf{P}^{\mathsf{T}}$, since undoing a shuffle is a shuffle).

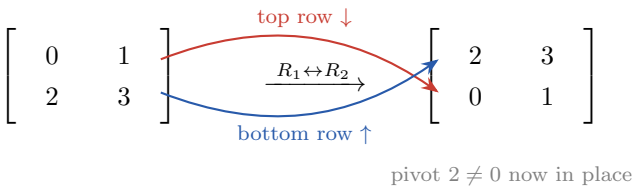


Figure 9.4: A row exchange is the whole content of a permutation matrix $\mathbf{P}$. The zero in the top-left pivot slot is swapped out for a nonzero entry by interchanging the two rows; left-multiplying by $\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ does exactly this. After the swap the matrix is already upper triangular, so elimination is done.

The fully general statement of the factorization, valid for *every* invertible matrix, builds the permutation in:

Formal

LU with partial pivoting. For any invertible square matrix $\mathbf{A}$ there is a permutation matrix $\mathbf{P}$, a unit lower triangular $\mathbf{L}$ (ones on the diagonal), and an upper triangular $\mathbf{U}$ such that

$$\mathbf{PA} = \mathbf{LU}.$$

Read it as: *if you reorder the rows of* $\mathbf{A}$ *in the right way up front, the reordered matrix factors cleanly into* $\mathbf{LU}$. The permutation $\mathbf{P}$ records all the swaps elimination chose to make.

To solve $\mathbf{Ax} = \mathbf{b}$ from $\mathbf{PA} = \mathbf{LU}$, multiply the system by $\mathbf{P}$ first: $\mathbf{PAx} = \mathbf{Pb}$, i.e. $\mathbf{LUx} = \mathbf{Pb}$. So you permute the right-hand side the same way, then do the usual forward/back substitution. Permuting $\mathbf{b}$ is free – it is just reading its entries in a different order.

9.6.1 Pivoting for Numerical Stability

Here is the deeper reason real software *always* pivots, even when no pivot is exactly zero. A *tiny* pivot is nearly as bad as a zero pivot. Suppose

$$\mathbf{A} = \begin{bmatrix} 0.0001 & 1 \\ 1 & 1 \end{bmatrix}.$$

Use the tiny 0.0001 as a pivot: the multiplier is $\ell_{21} = 1/0.0001 = 10000$, and clearing row 2 gives $u_{22} = 1 - 10000 \cdot 1 = -9999$. That huge multiplier amplifies any rounding error in the data by a factor of 10^4. The fix is **partial pivoting**: before using a column, swap the row with the *largest* available entry into the pivot position, so every multiplier satisfies $|\ell_{ij}| \leq 1$. Swapping rows here puts the 1 on top, the multiplier becomes a tame 0.0001, and the computation stays accurate. So $\mathbf{P}$ is doing double duty: it rescues you from exact zeros, and it keeps the arithmetic numerically stable. We return to this in the numerical chapter; for now, just know that $\mathbf{PA} = \mathbf{LU}$ is what your computer actually does.

Example 9.1 (A full $\mathbf{PA} = \mathbf{LU}$)**.** Factor $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ with partial pivoting. Column 1 has entries

1 and 3; the larger magnitude is 3, in row 2, so swap first:

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{PA} = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}.$$

Now eliminate: multiplier $\ell_{21} = 1/3$, and $R_2 \leftarrow R_2 - \frac{1}{3}R_1$ gives $u_{22} = 2 - \frac{1}{3} \cdot 4 = \frac{2}{3}$. Hence

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ \frac{1}{3} & 1 \end{bmatrix}, \qquad \mathbf{U} = \begin{bmatrix} 3 & 4 \\ 0 & \frac{2}{3} \end{bmatrix}.$$

Verify: $\mathbf{LU} = \begin{bmatrix} 3 & 4 \\ 1 & \frac{4}{3} + \frac{2}{3} \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix} = \mathbf{PA}$. ✓

Connection to other math

The sign of a permutation, and the determinant. A permutation matrix is the matrix face of a permutation of $\{1, \ldots, n\}$, the objects group theory calls the symmetric group S_n (the set of all $n!$ ways to reorder n distinct items). Every permutation is a product of swaps, and although the number of swaps is not unique, its *parity* (even or odd) is – a fixed fact about the permutation. That parity is the *sign*, $\det \mathbf{P} = \pm 1$. This links straight to determinants: $\mathbf{PA} = \mathbf{LU}$ gives

$$\det \mathbf{A} = (\det \mathbf{P})\,(\det \mathbf{L})\,(\det \mathbf{U}) = (\pm 1) \cdot 1 \cdot (u_{11} u_{22} \cdots u_{nn}),$$

because det of a triangular matrix is the product of its diagonal, and $\mathbf{L}$ has unit diagonal. So the determinant is just $\pm$ the product of the pivots – which is exactly how numerical libraries compute determinants: factor, multiply the pivots, fix the sign from the swaps. One $\mathbf{PA} = \mathbf{LU}$ factorization hands us the solver, the row-swap sign, and the determinant at once.

9.7 Pulling out the pivots: $\mathbf{A} = \mathbf{LDU}$

There is a mild asymmetry in $\mathbf{A} = \mathbf{LU}$ that bothers people, and fixing it reveals structure. In our example

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix}, \qquad \mathbf{U} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix},$$

the $\mathbf{L}$ has 1's on its diagonal but $\mathbf{U}$ carries the pivots $2, 1, 2$ on its diagonal. Factor those pivots out of $\mathbf{U}$ into a diagonal matrix $\mathbf{D}$, leaving a $\mathbf{U}$ with unit diagonal too:

$$\mathbf{U} = \underbrace{\begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}}_{\mathbf{D}} \underbrace{\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{U}'},$$

where each row of $\mathbf{U}$ was divided by its pivot. So

$$\mathbf{A} = \mathbf{L}\,\mathbf{D}\,\mathbf{U}', \qquad \mathbf{D} = \operatorname{diag}(2, 1, 2),$$

with $\mathbf{L}$ and $\mathbf{U}'$ both unit-triangular and all the scaling living in $\mathbf{D}$. This $\mathbf{LDU}$ form is the symmetric-looking sibling of $\mathbf{LU}$, and it pays off for symmetric matrices ($\mathbf{A} = \mathbf{A}^\mathsf{T}$, matrices that are mirror images of themselves across the main diagonal): if $\mathbf{A} = \mathbf{A}^\mathsf{T}$ then $\mathbf{U}' = \mathbf{L}^\mathsf{T}$ and the factorization collapses to $\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^\mathsf{T}$. When all pivots in $\mathbf{D}$ are strictly positive as well (meaning the matrix is **positive definite**, stretching every nonzero vector without flattening or reflecting any direction), you can split $\mathbf{D} = \sqrt{\mathbf{D}}\sqrt{\mathbf{D}}$ and absorb the square roots into the triangular factors. That yields $\mathbf{A} = \mathbf{R}^\mathsf{T}\mathbf{R}$ (or $\mathbf{L}_C\mathbf{L}_C^\mathsf{T}$ with lower triangular $\mathbf{L}_C = \mathbf{L}\sqrt{\mathbf{D}}$): the **Cholesky factorization**, a symmetric, half-the-work specialization of LU that powers least squares and Gaussian statistics. We will meet it again with positive definite matrices.

Why this matters

Writing $\mathbf{A} = \mathbf{LU}$ turns a list of row-reduction steps into an equation between matrices, and any symmetry in $\mathbf{A}$ shows up immediately in the factors: pulling out the pivots gives $\mathbf{LDU}'$; symmetry ($\mathbf{A} = \mathbf{A}^\mathsf{T}$) makes the upper factor mirror the lower ($\mathbf{L}\mathbf{D}\mathbf{L}^\mathsf{T}$); and positive pivots let us split $\mathbf{D}$ in half to get the Cholesky form $\mathbf{L}_C\mathbf{L}_C^\mathsf{T}$.

9.8 The cost of elimination, counted exactly

We have been saying "elimination is expensive, substitution is cheap." Let us make that quantitative, because the numbers are the entire justification for the LU strategy.

Count multiply-and-subtract operations ("flops"). To clear column 1 of an $n \times n$ matrix you process rows 2 through n – that is $n-1$ rows – and for each you compute one multiplier and update about n entries, so roughly $(n-1)\cdot n$ operations. Clearing column 2 works on the remaining $(n-1)\times(n-1)$ block, costing about $(n-2)(n-1)$. In general, eliminating column k costs about $(n-k)(n-k+1)$. Summing,

$$\text{elimination cost} \;\approx\; \sum_{k=1}^{n-1}(n-k)^2 \;\approx\; \sum_{j=1}^{n-1} j^2 \;\approx\; \frac{n^3}{3}.$$

The leading term is $\frac{1}{3}n^3$. **Cubic.** Double the size of the matrix and elimination costs eight times as much.

Now count a triangular solve. Back substitution on $\mathbf{U}$: the last unknown is free, the next costs 1 multiply, the next 2, and so on up to about n, giving

$$\sum_{k=1}^{n} k \approx \frac{n^2}{2}.$$

Forward substitution on $\mathbf{L}$ costs the same $\frac{1}{2}n^2$. So one full solve from a finished factorization is about n^2 operations. **Quadratic.**

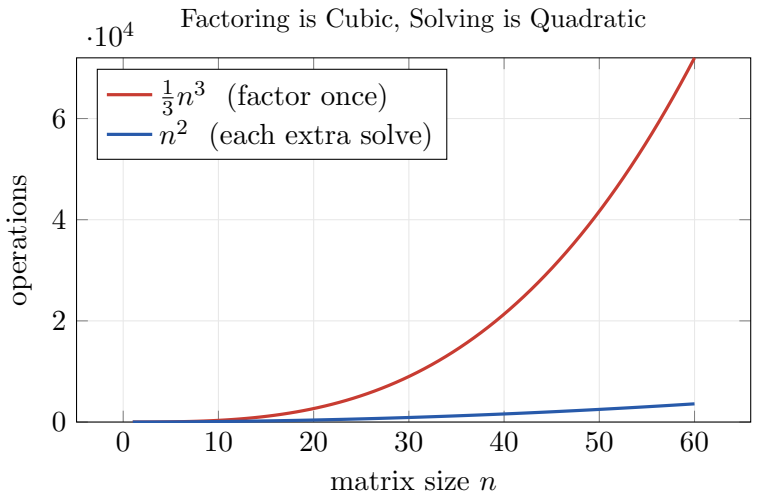


Figure 9.5: The two costs that justify LU. Building the factorization is cubic ($\sim \frac{1}{3}n^3$); each subsequent solve with a new right-hand side is only quadratic ($\sim n^2$). For large n the gap is enormous, so reusing the factorization across many right-hand sides is where the savings live.

The ratio of these tells the whole story. The expensive factorization costs about $\frac{1}{3}n^3$; each additional solve costs about n^2. Their ratio is

$$\frac{\frac{1}{3}n^3}{n^2} = \frac{n}{3}.$$

So one factorization "is worth" roughly $n/3$ solves. For a modest $n = 300$, the factor step costs as much as a hundred substitution steps. If you have a hundred right-hand sides and you re-eliminate each time, you do a hundred cubic operations; if you factor once and substitute, you do one cubic plus a hundred quadratics – essentially the cost of a *single* naive solve. That is the difference between a job finishing in a second and finishing in a minute.

Example 9.2 (Counting the win)**.** Take $n = 1000$ and $k = 500$ right-hand sides. Re-eliminating each time costs about $500 \times \frac{1}{3}(1000)^3 \approx 1.7 \times 10^{11}$ operations. Factoring once and substituting costs about $\frac{1}{3}(1000)^3 + 500 \times (1000)^2 \approx 3.3 \times 10^8 + 5 \times 10^8 \approx 8.3 \times 10^8$ operations. The LU strategy is roughly 200 times faster here – and the advantage only widens as k grows, because every extra right-hand side is nearly free.

Focus

Summary of LU complexity: **factoring is cubic, solving is quadratic.** The cubic cost is paid once, after which each additional right-hand side requires only a quadratic triangular solve. Whenever the same matrix appears with multiple right-hand sides, factorization provides substantial computational savings.

9.9 When LU fails, and when it does not

A short, careful accounting of the boundaries.

Pure LU (no permutation) exists exactly when elimination never needs a swap, i.e. when every pivot turns out nonzero without reordering. A clean sufficient condition: if every *leading principal*

submatrix of $\mathbf{A}$ (the top-left 1×1, 2×2, ..., $k \times k$ blocks) is invertible, then $\mathbf{A} = \mathbf{LU}$ exists and is unique. Uniqueness is easy to believe: if $\mathbf{L}_1\mathbf{U}_1 = \mathbf{L}_2\mathbf{U}_2$ then $\mathbf{L}_2^{-1}\mathbf{L}_1 = \mathbf{U}_2\mathbf{U}_1^{-1}$; the left side is unit lower triangular, the right side is upper triangular, and the only matrix that is both is the identity – so $\mathbf{L}_1 = \mathbf{L}_2$ and $\mathbf{U}_1 = \mathbf{U}_2$.

PA = LU always exists for an invertible $\mathbf{A}$. There is always *some* reordering of the rows that makes pivots appear, because invertibility guarantees a nonzero entry to promote in each column as you go.

Singular matrices can still be reduced to an upper triangular $\mathbf{U}$, but $\mathbf{U}$ will have a zero pivot somewhere, signaling that $\mathbf{Ax} = \mathbf{b}$ either has no solution or infinitely many. That is a story for the chapters on rank and the four subspaces; LU's clean two-slide solve assumes the pivots are all there.

Remark 9.1. A subtle point worth flagging: the matrix $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$ from Section 9.6 is invertible yet has *no* plain $\mathbf{LU}$ factorization – its leading 1×1 block is the zero matrix $[0]$, which is not invertible, so the leading-submatrix test fails at the very first step. This is precisely the case the permutation rescues. "No $\mathbf{LU}$" is a statement about row order, not about the matrix being defective.

Before/after Strang

After Strang: when Strang asks "which matrices have an $\mathbf{LU}$ without row exchanges?", the answer is the leading-submatrix condition above. He frames the general case as $\mathbf{PA} = \mathbf{LU}$ and emphasizes that this is the version your software performs. He also connects $\mathbf{A} = \mathbf{LU}$ to the *four subspaces* later: the pivots in $\mathbf{U}$ reveal the rank, and the columns of $\mathbf{L}$ relate to the column space. Keep this chapter handy – it is the computational engine those structural chapters run on.

9.10 What to carry forward

- **Elimination is factorization.** Running Gaussian elimination on $\mathbf{A}$ produces $\mathbf{A} = \mathbf{LU}$: $\mathbf{U}$ is the upper triangular result, and $\mathbf{L}$ collects the multipliers (with a plus sign) below a unit diagonal.
- **Solve in two slides.** $\mathbf{Ax} = \mathbf{b}$ becomes $\mathbf{Lc} = \mathbf{b}$ (forward) then $\mathbf{Ux} = \mathbf{c}$ (backward). Each slide is mechanical.
- **Factor once, reuse forever.** New right-hand sides need only the two cheap substitutions. This is the entire reason LU dominates practical computing.
- **Row exchanges give PA = LU.** Pivoting handles missing pivots and prevents numerical errors from blowing up by keeping multipliers ≤ 1 in magnitude.
- **Cost.** Factoring is $\sim \frac{1}{3}n^3$ (cubic); each solve is $\sim n^2$ (quadratic). One factorization is worth about $n/3$ solves.
- **Never invert; always factor.** To solve a system, do not compute $\mathbf{A}^{-1}$; factor and substitute. It is faster and more accurate.

You now own the algorithm that quietly sits underneath "solve this linear system" in every serious numerical library. When the theories of linear algebra need to compute, this is the engine they reach for.

Notice what this journey required: whenever a zero blocked our path, we had to swap rows using a permutation matrix $\mathbf{P}$, arriving at $\mathbf{PA} = \mathbf{LU}$. And when $\mathbf{A}$ was symmetric, we factored it into $\mathbf{LDL}^\mathsf{T}$, where the transpose operation $^\mathsf{T}$ appeared naturally. But what *is* a transpose geometrically? And how do permutation matrices behave as algebraic objects under multiplication, inversion, and signs? The next chapter, *Transposes and Permutations*, steps squarely into that world: flipping matrices across their main diagonals, reordering coordinates, and establishing the formal rules for the inner products, orthogonality, and symmetric decompositions that power the rest of the book.

History and connection

The modern formulation of elimination as matrix factorization ($\mathbf{A} = \mathbf{LU}$) was established by Alan Turing in his 1948 paper, *Rounding-Off Errors in Matrix Processes*. Turing, working on the design of the ACE (Automatic Computing Engine) at the British National Physical Laboratory, wanted to know whether early digital computers could solve systems of one hundred linear equations without roundoff error destroying the results. In 1947, John von Neumann and Herman Goldstine had published an alarming theoretical analysis suggesting that rounding errors might grow exponentially, convincing many mathematicians that solving large systems on digital computers was doomed to produce nonsense.

Turing answered that pessimism directly. With access only to slow electromechanical calculators and acoustic delay-line computer memory holding mere hundreds of numbers, Turing introduced the matrix condition number (a single number measuring how sensitive a system's output is to tiny perturbations in its inputs) and proved that Gaussian elimination is mathematically equivalent to factoring a matrix into triangular pieces: $\mathbf{A} = \mathbf{LU}$. By pairing factorization with partial pivoting, Turing showed that multipliers stay bounded by 1 and that elimination is highly stable. In 1961, James Wilkinson completed the mathematical circle by establishing backward error bounds – rigorous proofs that the computed floating-point answer is the exact mathematical solution to a slightly perturbed physical problem.

An important precursor came from military geodesy (the science of measuring Earth's curvature to triangulate positions and draft accurate maps). In 1910, the French artillery officer and surveyor André-Louis Cholesky developed an ingenious method for solving the symmetric, positive-definite normal equations ($\mathbf{A}^\mathsf{T}\mathbf{Ax} = \mathbf{A}^\mathsf{T}\mathbf{b}$, which arise when finding best-fit lines through experimental data). Cholesky was killed in action in August 1918 during the closing months of World War I, but his fellow officer Commandant Benoit published Cholesky's manuscript posthumously in 1924. The Cholesky factorization ($\mathbf{A} = \mathbf{LL}^\mathsf{T}$) remains the gold standard for symmetric positive-definite systems, requiring only half the arithmetic and half the memory of standard LU.

In early computers, memory was so scarce that storing a second $n \times n$ matrix during elimination was impossible. Programmers used a clean structural fact: the diagonal of $\mathbf{L}$ is entirely 1's, so it never needs to be stored. As elimination creates each zero below the pivot in column j, the multiplier ℓ_{ij} can be written directly into the memory register where that zero was just created. At the end of the sweep, the original array $\mathbf{A}$ holds both $\mathbf{L}$ (below the diagonal) and $\mathbf{U}$ (on and above the diagonal) in place with zero additional memory overhead.

A fascinating theoretical puzzle remains open to this day: Wilkinson proved that in the mathematical worst case, partial pivoting allows errors to grow by a factor of 2^{n-1}, which would render a 100×100 solve completely inaccurate. Yet in over seventy years of practical computing, an authentic physical matrix that triggers this exponential catastrophe has almost never appeared in the wild. Why physical systems are so immune to this worst-case behavior remains an active research topic, partially explained by the smoothed analysis of Daniel

Spielman and Shang-Hua Teng. At the same time, whether an $n \times n$ system can ultimately be solved in near-quadratic $\mathcal{O}(n^2)$ time or strictly requires the full complexity of matrix multiplication (currently $\mathcal{O}(n^{2.37})$) remains one of the deepest open problems in theoretical computer science.

Even with pivoting, the triangular factors $\mathbf{L}$ and $\mathbf{U}$ are not orthogonal – they can distort lengths and angles. Handling general rectangular data, least squares, and symmetric systems demands permutations and transposes: Chapter 10 (Definition 10.1) introduces transposes and permutations systematically, completing the full decomposition $\mathbf{PA} = \mathbf{LU}$ and opening the path to symmetric matrices, orthogonality, and the geometry of subspaces in Part III.

9.11 Exercises

Exercise 9.1 (Mental check: already triangular)**.** If a matrix $\mathbf{A}$ is already upper triangular, what are its factors $\mathbf{L}$ and $\mathbf{U}$ in the factorization $\mathbf{A} = \mathbf{LU}$? What if $\mathbf{A}$ is a diagonal matrix?

Exercise 9.2 (Mental check: multipliers at a glance)**.** Without picking up a pen, state the multiplier ℓ_{21} needed to eliminate the bottom-left entry of $\begin{bmatrix} 5 & 2 \\ 15 & 8 \end{bmatrix}$, and write down the matrix $\mathbf{L}$.

Exercise 9.3 (Warmup factorization)**.** Factor $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 4 & 9 \end{bmatrix}$ into $\mathbf{LU}$ by hand. State the multiplier you used, and verify your answer by multiplying $\mathbf{LU}$ back out.

Exercise 9.4 (Two slides)**.** Using your factorization from the previous exercise, solve $\mathbf{Ax} = \begin{bmatrix} 5 \\ 21 \end{bmatrix}$ by forward substitution ($\mathbf{Lc} = \mathbf{b}$) followed by back substitution ($\mathbf{Ux} = \mathbf{c}$). Show $\mathbf{c}$ and $\mathbf{x}$.

Exercise 9.5 (A 3×3 by hand)**.** Factor

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 5 \\ 4 & 6 & 8 \end{bmatrix}$$

into $\mathbf{LU}$. Record all three multipliers $\ell_{21}, \ell_{31}, \ell_{32}$, and confirm $\mathbf{LU} = \mathbf{A}$.

Exercise 9.6 (No minus signs)**.** A student factors $\mathbf{A} = \begin{bmatrix} 2 & 4 \\ 6 & 3 \end{bmatrix}$ and writes $\mathbf{L} = \begin{bmatrix} 1 & 0 \\ -3 & 1 \end{bmatrix}$. Without recomputing from scratch, explain how you know the sign is wrong, give the correct $\mathbf{L}$, and state the general rule for the sign of the entries of $\mathbf{L}$.

Exercise 9.7 (Permutation needed)**.** Show that $\mathbf{A} = \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix}$ has no plain $\mathbf{LU}$ factorization. Then find a permutation $\mathbf{P}$ so that $\mathbf{PA} = \mathbf{LU}$, and give $\mathbf{P}$, $\mathbf{L}$, and $\mathbf{U}$ explicitly.

Exercise 9.8 (Partial pivoting for stability)**.** Consider $\mathbf{A} = \begin{bmatrix} 0.001 & 1 \\ 1 & 2 \end{bmatrix}$. (a) Factor it *without* pivoting and report the multiplier ℓ_{21} and the entry u_{22}. (b) Now factor $\mathbf{PA}$ with the rows swapped (partial pivoting) and report the new multiplier. (c) In one sentence, say why the second factorization is the one a numerical library prefers.

Exercise 9.9 (Pull out the pivots)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 6 \\ 6 & 20 \end{bmatrix}$, find $\mathbf{LU}$, then pull the pivots out to write $\mathbf{A} = \mathbf{LDU}'$ with both triangular factors having unit diagonal. Verify that $\mathbf{A}$ is symmetric and that, consequently, $\mathbf{U}' = \mathbf{L}^{\mathsf{T}}$.

Exercise 9.10 (Determinant from pivots)**.** Using the factorization from Exercise 9.5, compute $\det \mathbf{A}$ as the product of the pivots (the diagonal of $\mathbf{U}$). Then state how the answer would change if one row exchange had been required.

Exercise 9.11 (Counting the cost)**.** You must solve $\mathbf{Ax} = \mathbf{b}$ for an $n \times n$ matrix with $n = 200$, across $k = 50$ different right-hand sides. (a) Estimate the total operation count if you re-run elimination ($\sim \frac{1}{3}n^3$ each) every time. (b) Estimate it if you factor once ($\sim \frac{1}{3}n^3$) and then do k substitutions ($\sim n^2$ each). (c) What is the approximate speedup factor?

Exercise 9.12 (Why not invert? (short proof/argument))**.** Explain, in terms of operation counts, why solving $\mathbf{Ax} = \mathbf{b}$ by first computing $\mathbf{A}^{-1}$ and forming $\mathbf{A}^{-1}\mathbf{b}$ is more expensive than factoring and substituting. (Hint: how many right-hand sides does computing $\mathbf{A}^{-1}$ implicitly solve, and how does that compare to the one right-hand side you actually care about?)

Exercise 9.13 (Uniqueness, a clean proof)**.** Suppose a matrix has two unit-lower/upper factorizations $\mathbf{L}_1\mathbf{U}_1 = \mathbf{L}_2\mathbf{U}_2$, where $\mathbf{L}_1, \mathbf{L}_2$ are unit lower triangular and $\mathbf{U}_1, \mathbf{U}_2$ are upper triangular with nonzero diagonals. Prove $\mathbf{L}_1 = \mathbf{L}_2$ and $\mathbf{U}_1 = \mathbf{U}_2$. (Hint: rearrange to $\mathbf{L}_2^{-1}\mathbf{L}_1 = \mathbf{U}_2\mathbf{U}_1^{-1}$ and argue about what kind of matrix each side must be.)

Exercise 9.14 (Block thinking, a challenge)**.** Let $\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}$ in 2×2 block form with $\mathbf{A}_{11}$ invertible. Show that

$$\mathbf{A} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{A}_{21}\mathbf{A}_{11}^{-1} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{0} & \mathbf{S} \end{bmatrix}, \qquad \mathbf{S} = \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}.$$

This is one step of block elimination; $\mathbf{S}$ is the *Schur complement*. Identify which factor plays the role of $\mathbf{L}$ and which plays $\mathbf{U}$, and explain how this generalizes the scalar multiplier $\ell_{21} = a_{21}/a_{11}$.

Chapter 10

Transposes and Permutations

In Chapter 9, we watched Gaussian elimination run into an awkward obstacle: whenever a zero landed in the pivot position on the diagonal, elimination could not divide by zero and ground to a halt. The rescue was simple: swap two rows and keep going. We collected those row exchanges into a single shuffle matrix $\mathbf{P}$, producing the complete factorization $\mathbf{PA} = \mathbf{LU}$.

At the time, $\mathbf{P}$ seemed like mere emergency housekeeping. But row swaps and their constant partner, the *transpose*, are far from bookkeeping: they carry an enormous amount of geometry. The first operation is the *transpose*: flip a matrix across its main diagonal so that rows become columns. The second is a *permutation*: shuffle the rows (or columns) of a matrix without changing any of the numbers, only their addresses.

The transpose is the operation that connects a matrix to the dot product. It is the reason $\langle \mathbf{Ax}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{A}^\mathsf{T}\mathbf{y} \rangle$, which is the single most useful identity in all of applied linear algebra. It is the source of symmetric matrices and of $\mathbf{A}^\mathsf{T}\mathbf{A}$, the matrix behind least squares, covariance, and the singular value decomposition. And permutations are our *first* orthogonal matrices — rigid rearrangements of space that preserve every length and every angle. By the end of this chapter you will see the transpose not as a typographical flip but as the geometric "adjoint": the matrix that does to the output space exactly what $\mathbf{A}$ does to the input space, but in reverse.

Let us build all of this from the ground up, pictures first.

10.1 The transpose: flipping across the diagonal

10.1.1 The definition, with a picture

Start with the most concrete possible thing. Take a matrix and reflect it across the diagonal that runs from the top-left corner down to the bottom-right. Whatever was in row i, column j lands in row j, column i.

Definition 10.1 (Transpose)**.** Let $\mathbf{A}$ be an $m \times n$ matrix with entries a_{ij} (row i, column j). Its *transpose* $\mathbf{A}^\mathsf{T}$ is the $n \times m$ matrix whose (i, j) entry is a_{ji}:

$$(\mathbf{A}^\mathsf{T})_{ij} = a_{ji}.$$

Rows of $\mathbf{A}$ become columns of $\mathbf{A}^\mathsf{T}$, and columns of $\mathbf{A}$ become rows of $\mathbf{A}^\mathsf{T}$.

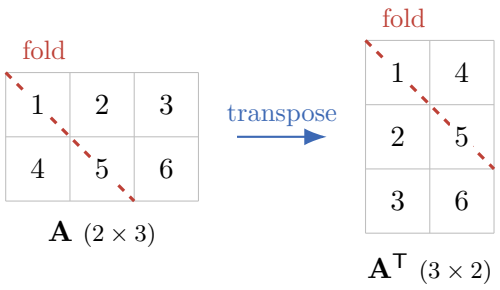
Here is a 2×3 matrix and its 3×2 transpose:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad \mathbf{A}^{\mathsf{T}} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Look at where each number went. The first *row* of $\mathbf{A}$, namely $(1, 2, 3)$, became the first *column* of $\mathbf{A}^{\mathsf{T}}$. The first column of $\mathbf{A}$, namely $\begin{bmatrix} 1 \\ 4 \end{bmatrix}$, became the first row of $\mathbf{A}^{\mathsf{T}}$. The diagonal entries that sit on the fold — here only $a_{11} = 1$ and $a_{22} = 5$ — do not move. Everything else swaps across the fold.

Intuition

Picture the matrix as a grid of tiles. Lay a mirror along the main diagonal (top-left to bottom-right) and the transpose is what you see in the mirror. A tall thin matrix becomes a short wide one. A row vector becomes a column vector. Nothing is added, scaled, or destroyed — the same numbers just trade places across the fold.



$\mathbf{A}$ (2×3) $\qquad$ $\mathbf{A}^{\mathsf{T}}$ (3×2)

Figure 10.1: The transpose reflects the grid across the main diagonal. Row $(1, 2, 3)$ of $\mathbf{A}$ becomes the first column of $\mathbf{A}^{\mathsf{T}}$. The shape flips from 2×3 to 3×2.

A few immediate sanity checks. If $\mathbf{A}$ is $m \times n$ then $\mathbf{A}^{\mathsf{T}}$ is $n \times m$ — the shape flips. Transposing twice brings you home: $(\mathbf{A}^{\mathsf{T}})^{\mathsf{T}} = \mathbf{A}$, because reflecting across the diagonal and then reflecting again undoes itself. And a single column vector $\mathbf{x}$ (which we think of as $n \times 1$) transposes into a single *row*, $\mathbf{x}^{\mathsf{T}}$, of shape $1 \times n$. That last fact is the key, because a row times a column is exactly a dot product.

Focus

Hold onto three reflexes:

- $(\mathbf{A}^{\mathsf{T}})_{ij} = a_{ji}$: swap the indices.
- Shape flips: $m \times n \to n \times m$.
- $(\mathbf{A}^{\mathsf{T}})^{\mathsf{T}} = \mathbf{A}$: transpose is its own undo.

Pause

Pause and think: If a matrix $\mathbf{M}$ has shape 4×7, what is the shape of $(\mathbf{M}^{\mathsf{T}})^{\mathsf{T}}$? Which entries of $\mathbf{M}$ never change position when you transpose? *Answer:* Transposing once produces 7×4; transposing again returns the original 4×7. Only the diagonal entries $m_{11}, m_{22}, m_{33}, m_{44}$ stay glued in place, because swapping row index i with column index i leaves the address

unchanged.

10.1.2 The transpose turns a column into a dot-product machine

Why should anyone care about flipping a grid? Here is the first real reason. Recall the dot product of two column vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$:

$$\langle \mathbf{u},\, \mathbf{v} \rangle \;=\; u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

Now write $\mathbf{u}$ as a column and transpose it into a row. Matrix-multiply that row by the column $\mathbf{v}$:

$$\mathbf{u}^{\mathsf{T}}\mathbf{v} = \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n = \langle \mathbf{u},\, \mathbf{v} \rangle.$$

So the dot product *is* a transpose-times-a-vector. The little operation "$\mathbf{u}^{\mathsf{T}}\mathbf{v}$" is matrix multiplication of a $1 \times n$ row by an $n \times 1$ column, producing a 1×1 number. This is more than a notational convenience; it lets every dot-product fact be written as a matrix product and back again. We will lean on it constantly.

Example 10.1 (A dot product as $\mathbf{u}^{\mathsf{T}}\mathbf{v}$)**.** Let $\mathbf{u} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 4 \\ 2 \end{bmatrix}$. Then

$$\mathbf{u}^{\mathsf{T}}\mathbf{v} = \begin{bmatrix} 2 & -1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ 2 \end{bmatrix} = (2)(1) + (-1)(4) + (3)(2) = 2 - 4 + 6 = 4.$$

And $\|\mathbf{u}\|^2 = \mathbf{u}^{\mathsf{T}}\mathbf{u} = 4 + 1 + 9 = 14$. A vector dotted with itself, written $\mathbf{u}^{\mathsf{T}}\mathbf{u}$, is the squared length. Remember that — $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ later is just this idea applied to every pair of columns at once.

Notice the order of multiplication carefully. Row times column, $(1 \times n)(n \times 1)$, produces a 1×1 scalar: the inner product (dot product). If you reverse the order to column times row, $(n \times 1)(1 \times n)$, you obtain an $n \times n$ grid of numbers: the *outer product* $\mathbf{u}\mathbf{v}^{\mathsf{T}}$. Every row of an outer product is a scaled copy of $\mathbf{v}^{\mathsf{T}}$, and every column is a scaled copy of $\mathbf{u}$. Keep row-times-column (number) sharply distinct from column-times-row (rank-one matrix).

10.2 The algebra of transposes

Before we go further geometrically we need the rulebook. There are four rules; three are easy and one is the famous "socks and shoes" reversal. We will derive each, because a formula you can derive in three lines is a formula you will never misremember.

Theorem 10.1 (Rules of the transpose)**.** *For matrices of compatible sizes and a scalar c:*

1. $(\mathbf{A} + \mathbf{B})^{\mathsf{T}} = \mathbf{A}^{\mathsf{T}} + \mathbf{B}^{\mathsf{T}}$.

2. $(c\,\mathbf{A})^{\mathsf{T}} = c\,\mathbf{A}^{\mathsf{T}}$.
3. $(\mathbf{A}^{\mathsf{T}})^{\mathsf{T}} = \mathbf{A}$.
4. $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$ *(the order reverses).*

Proof. Rules 1–3 are entry-chasing. For (1), the (i,j) entry of $(\mathbf{A}+\mathbf{B})^{\mathsf{T}}$ is by definition the (j,i) entry of $\mathbf{A}+\mathbf{B}$, which is $a_{ji}+b_{ji}$, which is the (i,j) entry of $\mathbf{A}^{\mathsf{T}}+\mathbf{B}^{\mathsf{T}}$. Rule (2) is the same with a scalar pulled along. Rule (3) we already saw: reflecting twice is the identity.

Rule (4) is the one worth doing carefully. Let $\mathbf{A}$ be $m \times n$ and $\mathbf{B}$ be $n \times p$, so $\mathbf{AB}$ is $m \times p$ and $(\mathbf{AB})^{\mathsf{T}}$ is $p \times m$. The (i,j) entry of $(\mathbf{AB})^{\mathsf{T}}$ is the (j,i) entry of $\mathbf{AB}$:

$$\big((\mathbf{AB})^{\mathsf{T}}\big)_{ij} = (\mathbf{AB})_{ji} = \sum_{k=1}^{n} a_{jk}\,b_{ki}.$$

Now compute the (i,j) entry of $\mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$. The (i,k) entry of $\mathbf{B}^{\mathsf{T}}$ is b_{ki}, and the (k,j) entry of $\mathbf{A}^{\mathsf{T}}$ is a_{jk}, so

$$(\mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}})_{ij} = \sum_{k=1}^{n} (\mathbf{B}^{\mathsf{T}})_{ik}(\mathbf{A}^{\mathsf{T}})_{kj} = \sum_{k=1}^{n} b_{ki}\,a_{jk}.$$

The two sums are identical (multiplication of scalars commutes), so the matrices are equal. □

Common trap

The order really does reverse. $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$, *not* $\mathbf{A}^{\mathsf{T}}\mathbf{B}^{\mathsf{T}}$. A quick way to see why it *has* to reverse: if $\mathbf{A}$ is 2×3 and $\mathbf{B}$ is 3×4, then $\mathbf{A}^{\mathsf{T}}\mathbf{B}^{\mathsf{T}}$ tries to multiply a 3×2 by a 4×3 — the inner dimensions 2 and 4 do not match, so it is not even defined. Only $\mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$ (4×3 times 3×2) makes sense.

A standard mnemonic illustrates the reversal: to take off your socks and shoes you reverse the order in which you put them on: shoes first, then socks. Transposing a product is the same — whatever went on last comes off first. Combined with the inverse rule $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ from Chapter 8, you get a clean fact: transpose and inverse commute with each other, $(\mathbf{A}^{\mathsf{T}})^{-1} = (\mathbf{A}^{-1})^{\mathsf{T}}$, a matrix so common it gets its own name, $\mathbf{A}^{-\mathsf{T}}$.

Compute it yourself

Verify the reversal rule directly. Take

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}.$$

Compute $\mathbf{AB}$, then transpose it. Separately compute $\mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$. They must match. Then compute the wrong thing, $\mathbf{A}^{\mathsf{T}}\mathbf{B}^{\mathsf{T}}$, and watch it disagree. (You should find $(\mathbf{AB})^{\mathsf{T}} = \left[\begin{smallmatrix} 11 & 4 \\ 10 & 5 \end{smallmatrix}\right]$.)

Why does this reversal happen geometrically? Think of $\mathbf{ABx}$ as a two-stage assembly line: first $\mathbf{B}$ transforms $\mathbf{x}$, and then $\mathbf{A}$ transforms $\mathbf{Bx}$. When we want to pull a measurement backward from the final output space back to the starting space (as we are about to see), we must pull it backward through $\mathbf{A}$ first, and only then backward through $\mathbf{B}$. The last operation to act in the forward direction is the first that must be undone in the backward direction.

Pause

Pause and think: What is the transpose of a three-matrix product $(\mathbf{ABC})^\mathsf{T}$? *Answer:* Group the first two factors: $((\mathbf{AB})\mathbf{C})^\mathsf{T} = \mathbf{C}^\mathsf{T}(\mathbf{AB})^\mathsf{T} = \mathbf{C}^\mathsf{T}\mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$. The reversal rule applies to products of any length: reverse the factor order completely and transpose each matrix.

10.3 The transpose is the adjoint

Now comes the identity that explains *why* the transpose is the right notion and not just a formatting trick.

Theorem 10.2 (The transpose moves across the inner product)**.** *Let* $\mathbf{A}$ *be an* $m \times n$ *real matrix,* $\mathbf{x} \in \mathbb{R}^n$*, and* $\mathbf{y} \in \mathbb{R}^m$*. Then*

$$\langle \mathbf{Ax},\, \mathbf{y} \rangle \;=\; \langle \mathbf{x},\, \mathbf{A}^\mathsf{T}\mathbf{y} \rangle.$$

Proof. Write both sides as row-times-column products and use the reversal rule. The left side is

$$\langle \mathbf{Ax},\, \mathbf{y} \rangle = (\mathbf{Ax})^\mathsf{T}\mathbf{y} = \mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{y},$$

using rule (4): $(\mathbf{Ax})^\mathsf{T} = \mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}$. The right side is

$$\langle \mathbf{x},\, \mathbf{A}^\mathsf{T}\mathbf{y} \rangle = \mathbf{x}^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{y}) = \mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{y}.$$

Both equal the same 1×1 quantity $\mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{y}$. Done. □

The proof is two lines, but its meaning is large. The matrix $\mathbf{A}$ takes vectors from the input space $\mathbb{R}^n$ over to the output space $\mathbb{R}^m$. The transpose $\mathbf{A}^\mathsf{T}$ runs the other way, from $\mathbb{R}^m$ back to $\mathbb{R}^n$. The theorem says: it does not matter whether you push $\mathbf{x}$ forward through $\mathbf{A}$ and then measure it against $\mathbf{y}$ in the output space, or pull $\mathbf{y}$ backward through $\mathbf{A}^\mathsf{T}$ and then measure it against $\mathbf{x}$ in the input space. You get the same number. The transpose is the unique matrix that lets you "hand the matrix across" the inner product.

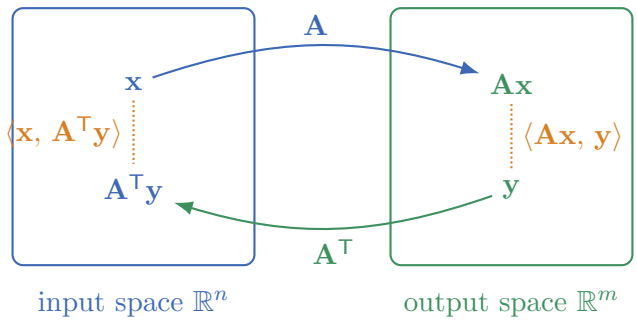


Figure 10.2: The adjoint relation. Push $\mathbf{x}$ forward by $\mathbf{A}$ and pair with $\mathbf{y}$ on the right; or pull $\mathbf{y}$ back by $\mathbf{A}^\mathsf{T}$ and pair with $\mathbf{x}$ on the left. The two dotted pairings give the same number.

Why this matters

This is the reason mathematicians often call $\mathbf{A}^\mathsf{T}$ the *adjoint* of $\mathbf{A}$. In more advanced settings (infinite-dimensional spaces, differential operators, quantum mechanics) the matrix entries vanish entirely and the *only* thing that survives is exactly this relation: the adjoint $\mathbf{A}^*$ of an operator $\mathbf{A}$ is *defined* as the operator satisfying $\langle \mathbf{Ax}, \mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{A}^*\mathbf{y}\rangle$ for all $\mathbf{x}, \mathbf{y}$. The matrix transpose is the finite-dimensional foundation of this general concept in analysis. When you meet $\frac{d}{dx}$ and its adjoint $-\frac{d}{dx}$ later, you will recognize this same picture.

Before/after Strang

Before Strang: when he writes $\mathbf{A}^\mathsf{T}$, do not read it merely as a flipped array of numbers. Read it as the matrix that maps outputs back to inputs while preserving dot products. That reading makes the four fundamental subspaces (coming a few chapters from now) feel inevitable rather than mysterious: the column space of $\mathbf{A}^\mathsf{T}$ is the row space of $\mathbf{A}$, and orthogonality between the subspaces is just Theorem 10.2 with one side set to zero. **After Strang:** this identity is the engine of backpropagation, of the normal equations, and of the adjoint method in optimization and PDEs.

Modern application

Backpropagation is a transpose. A layer of a neural network computes $\mathbf{z} = \mathbf{Wx}$ (followed by a nonlinearity we will ignore here). Training needs the gradient of a scalar loss L with respect to the input, given the gradient with respect to the output. The chain rule says: if $\mathbf{g} = \partial L/\partial \mathbf{z}$ is the "upstream gradient" in the output space, then the gradient in the input space is

$$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\mathsf{T}\mathbf{g}.$$

The forward pass multiplies by $\mathbf{W}$; the backward pass multiplies by $\mathbf{W}^\mathsf{T}$. That is exactly Theorem 10.2: gradients are covectors that live in the output space, and the transpose is what pulls them back to the input space. Every deep-learning framework implements "the backward of a matmul is a matmul by the transpose" — the whole training loop is the adjoint relation applied layer by layer.

Pause

Pause and think: If $\mathbf{A}$ is a 3×2 matrix, what spaces do $\mathbf{x}$ and $\mathbf{y}$ live in for the adjoint identity $\langle \mathbf{Ax}, \mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{A}^\mathsf{T}\mathbf{y}\rangle$? In which space is each dot product evaluated? *Answer:* For $\mathbf{Ax}$ to make sense, $\mathbf{x} \in \mathbb{R}^2$, so the output $\mathbf{Ax} \in \mathbb{R}^3$. Thus $\mathbf{y} \in \mathbb{R}^3$. The left-hand dot product $\langle \mathbf{Ax}, \mathbf{y}\rangle$ takes place in the 3-dimensional output space $\mathbb{R}^3$. On the right-hand side, $\mathbf{A}^\mathsf{T}$ has shape 2×3, so $\mathbf{A}^\mathsf{T}\mathbf{y} \in \mathbb{R}^2$; the dot product $\langle \mathbf{x}, \mathbf{A}^\mathsf{T}\mathbf{y}\rangle$ takes place in the 2-dimensional input space $\mathbb{R}^2$. Two dot products in two different dimensions, yet the resulting number is identical.

10.4 Symmetric matrices

Now that we understand how the transpose moves operators across inner products, a natural question arises: what happens when transposing a matrix does not change it at all?

10.4.1 When a matrix equals its own mirror image

A matrix that is unchanged by transposing is the most important special case in the entire subject.

Definition 10.2 (Symmetric matrix)**.** A square matrix $\mathbf{A}$ is *symmetric* if $\mathbf{A}^\mathsf{T} = \mathbf{A}$, that is, $a_{ij} = a_{ji}$ for all i, j. Geometrically, it is unchanged when reflected across its main diagonal.

For example,

$$\mathbf{S} = \begin{bmatrix} 2 & 7 & -1 \\ 7 & 0 & 3 \\ -1 & 3 & 5 \end{bmatrix}$$

is symmetric: the entry above the diagonal at position $(1, 2)$ is 7, and so is the entry below at $(2, 1)$. The whole matrix is determined by its diagonal and the triangle above it; the lower triangle is just a copy folded over. An $n \times n$ symmetric matrix has only $n + \binom{n}{2} = \binom{n+1}{2}$ free numbers instead of n^2.

Intuition

A symmetric matrix is one where "the influence of i on j equals the influence of j on i." Think of a spring connecting two masses: the force the first exerts on the second equals the force the second exerts on the first. Or a road network where the distance from town i to town j is the same as from j to i. Symmetry encodes a two-way, reciprocal relationship — and it is no accident that such matrices have the nicest possible eigenvectors (real, orthogonal), a fact we will prove in the spectral-theorem chapter.

10.4.2 $\mathbf{A}^\mathsf{T}\mathbf{A}$ is always symmetric — and it is everywhere

Here is a manufacturing process for symmetric matrices that works on *any* matrix, square or not. Take any $m \times n$ matrix $\mathbf{A}$ and form $\mathbf{A}^\mathsf{T}\mathbf{A}$.

Proposition 10.1 (The fundamental symmetric matrices)**.** *For any $m \times n$ matrix $\mathbf{A}$, both $\mathbf{A}^\mathsf{T}\mathbf{A}$ ($n \times n$) and $\mathbf{A}\mathbf{A}^\mathsf{T}$ ($m \times m$) are symmetric.*

Proof. Apply the transpose rules. Using $(\mathbf{X}\mathbf{Y})^\mathsf{T} = \mathbf{Y}^\mathsf{T}\mathbf{X}^\mathsf{T}$ and $(\mathbf{X}^\mathsf{T})^\mathsf{T} = \mathbf{X}$,

$$(\mathbf{A}^\mathsf{T}\mathbf{A})^\mathsf{T} = \mathbf{A}^\mathsf{T}(\mathbf{A}^\mathsf{T})^\mathsf{T} = \mathbf{A}^\mathsf{T}\mathbf{A}.$$

Since $\mathbf{A}^\mathsf{T}\mathbf{A}$ equals its own transpose, it is symmetric. The same computation with the roles swapped gives $(\mathbf{A}\mathbf{A}^\mathsf{T})^\mathsf{T} = \mathbf{A}\mathbf{A}^\mathsf{T}$. □

Work one out and see what the entries mean.

Example 10.2 ($\mathbf{A}^\mathsf{T}\mathbf{A}$ collects all the dot products of the columns)**.** Let

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix} \quad \text{with columns} \quad \mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \ \mathbf{a}_2 = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}.$$

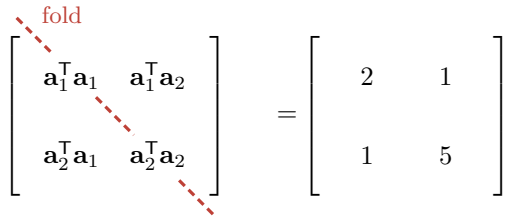
Then

$$\mathbf{A}^\mathsf{T}\mathbf{A} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 5 \end{bmatrix}.$$

Read the entries: the $(1,1)$ entry $2 = \mathbf{a}_1^\mathsf{T}\mathbf{a}_1 = \|\mathbf{a}_1\|^2$; the $(2,2)$ entry $5 = \|\mathbf{a}_2\|^2$; the off-diagonal $1 = \mathbf{a}_1^\mathsf{T}\mathbf{a}_2 = \mathbf{a}_2^\mathsf{T}\mathbf{a}_1$. In general the (i,j) entry of $\mathbf{A}^\mathsf{T}\mathbf{A}$ is the dot product $\mathbf{a}_i^\mathsf{T}\mathbf{a}_j$ of column i with column j. The matrix is symmetric precisely because $\mathbf{a}_i^\mathsf{T}\mathbf{a}_j = \mathbf{a}_j^\mathsf{T}\mathbf{a}_i$. This grid of all pairwise dot products is so useful it has a name: the *Gram matrix* of the columns.

Focus

$\mathbf{A}^\mathsf{T}\mathbf{A}$ is the table of all dot products between the columns of $\mathbf{A}$. Diagonal entries are squared lengths; off-diagonal entries measure how aligned two columns are (zero means perpendicular). Symmetry is automatic because dot products do not care about order (Figure 10.3).



grid of all pairwise dot products = $\mathbf{A}^\mathsf{T}\mathbf{A}$

Figure 10.3: The Gram matrix from Example 10.2. Entry (i,j) is $\mathbf{a}_i^\mathsf{T}\mathbf{a}_j$, the dot product of column i with column j. Reflecting across the dashed diagonal leaves it unchanged because $\mathbf{a}_i^\mathsf{T}\mathbf{a}_j = \mathbf{a}_j^\mathsf{T}\mathbf{a}_i$: the matrix is symmetric by construction.

Modern application

Covariance matrices and PCA. Suppose you have a data set: m samples, each with n measured features, stacked as the rows of a matrix $\mathbf{X}$ (so $\mathbf{X}$ is $m \times n$). After subtracting the mean of each column, the matrix

$$\mathbf{C} = \tfrac{1}{m-1}\,\mathbf{X}^\mathsf{T}\mathbf{X}$$

is the *covariance matrix* of the features. Its (i,j) entry is the covariance between feature i and feature j; its diagonal holds the variances. It is symmetric for exactly the reason in Example 10.2, and that symmetry guarantees real eigenvalues and orthogonal eigenvectors — the principal components. Principal Component Analysis, the core tool for dimensionality reduction, is exactly the spectral analysis of a symmetric $\mathbf{X}^\mathsf{T}\mathbf{X}$. The transpose is what turns raw data into reciprocal, analyzable structure.

10.4.3 Splitting any square matrix into symmetric plus antisymmetric

A nice structural fact, and a good exercise in the rules. A matrix with $\mathbf{A}^\mathsf{T} = -\mathbf{A}$ is called *antisymmetric* (or skew-symmetric); its diagonal must be zero and entries flip sign across the fold. Every square matrix splits uniquely into a symmetric part and an antisymmetric part:

$$\mathbf{A} = \underbrace{\tfrac{1}{2}(\mathbf{A} + \mathbf{A}^\mathsf{T})}_{\text{symmetric}} + \underbrace{\tfrac{1}{2}(\mathbf{A} - \mathbf{A}^\mathsf{T})}_{\text{antisymmetric}}.$$

Check the first piece is symmetric: $\big(\tfrac{1}{2}(\mathbf{A} + \mathbf{A}^\mathsf{T})\big)^\mathsf{T} = \tfrac{1}{2}(\mathbf{A}^\mathsf{T} + \mathbf{A})$, the same thing. Check the second is antisymmetric: $\big(\tfrac{1}{2}(\mathbf{A} - \mathbf{A}^\mathsf{T})\big)^\mathsf{T} = \tfrac{1}{2}(\mathbf{A}^\mathsf{T} - \mathbf{A})$, which is the negative of the original. Add the two pieces and the $\mathbf{A}^\mathsf{T}$ terms cancel, returning $\mathbf{A}$.

Connection to other math

This decomposition is the matrix mirror of even-plus-odd in calculus. Any function splits as $f(x) = \tfrac{1}{2}\big(f(x) + f(-x)\big) + \tfrac{1}{2}\big(f(x) - f(-x)\big)$, an even part plus an odd part, exactly as a matrix splits into a transpose-even (symmetric) part plus a transpose-odd (antisymmetric) part. In both cases an *involution* — an operation that serves as its own undo when applied twice, like negating an input ($x \mapsto -x$) or reflecting across a mirror — carves the space into a $+1$ piece and a -1 piece. The antisymmetric 3×3 matrices, by the way, are exactly the cross-product operators $\mathbf{v} \mapsto \mathbf{w} \times \mathbf{v}$, which is why rotations in $\mathbb{R}^3$ come from antisymmetric generators. Symmetry and antisymmetry under the transpose are a recurring theme you will keep meeting.

10.5 Permutation matrices: shuffling without distorting

10.5.1 Reordering the rows of the identity

We now switch to the second star of the chapter. A permutation matrix is what you get by taking the identity matrix and rearranging its rows. Nothing more. Yet these matrices are the cleanest possible example of an *orthogonal* matrix, our first rigid motion of space.

Definition 10.3 (Permutation matrix)**.** A *permutation matrix* $\mathbf{P}$ is a square matrix obtained by permuting the rows (equivalently, the columns) of the identity $\mathbf{I}$. Equivalently, $\mathbf{P}$ has exactly one 1 in each row and each column and zeros elsewhere.

The smallest nontrivial example swaps two coordinates. In $\mathbb{R}^2$,

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{P}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} y \\ x \end{bmatrix}.$$

It takes the point (x, y) to (y, x) — a reflection across the diagonal line $y = x$. In $\mathbb{R}^3$ a permutation can do something more interesting, a *cycle*:

$$\mathbf{P} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \qquad \mathbf{P}\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} z \\ x \\ y \end{bmatrix}.$$

The old third coordinate becomes the new first, the old first becomes the new second, and the old second becomes the new third: a 3-cycle. Notice that whatever $\mathbf{P}$ does, it never adds, scales, or blends coordinates. It only relocates them. A permutation does only one thing: it rigidly relabels the coordinate axes.

Because $\mathbf{P}\mathbf{e}_j$ is the jth column of $\mathbf{P}$, the action on the standard basis is the fastest way to read a permutation. Here the columns of $\mathbf{P}$ are $\mathbf{e}_2, \mathbf{e}_3, \mathbf{e}_1$, so $\mathbf{e}_1 \mapsto \mathbf{e}_2 \mapsto \mathbf{e}_3 \mapsto \mathbf{e}_1$: the three axes chase each other around a single loop (Figure 10.4).

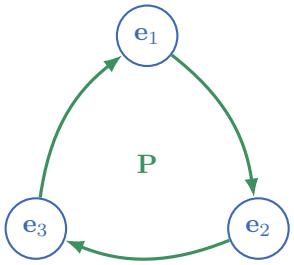


Figure 10.4: The 3-cycle reads off the columns of $\mathbf{P}$: column j is $\mathbf{P}\mathbf{e}_j$. Here $\mathbf{e}_1 \mapsto \mathbf{e}_2 \mapsto \mathbf{e}_3 \mapsto \mathbf{e}_1$, one closed loop. The transpose $\mathbf{P}^{\mathsf{T}}$ runs every arrow backward, which is why it undoes the shuffle.

Intuition

A permutation matrix is a perfect shuffle of the coordinate axes. The standard basis vectors $\mathbf{e}_1, \ldots, \mathbf{e}_n$ get permuted among themselves and sent back to standard basis vectors — none is bent or stretched. The unit cube goes to the unit cube; only the labels on the corners move. This is why permutations preserve every length and every angle.

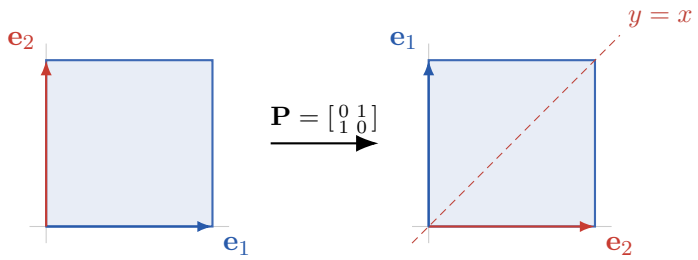


Figure 10.5: The swap $\mathbf{P} = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ exchanges the two axes. It is a reflection across the line $y = x$: the square is unchanged as a set, but $\mathbf{e}_1$ and $\mathbf{e}_2$ trade places.

10.5.2 Multiplying by P on the left vs. on the right

Remember the handedness of this action: multiplying *on the left* by $\mathbf{P}$ permutes the *rows* of whatever it hits; multiplying *on the right* permutes the *columns*. Watch it with the swap matrix:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} c & d \\ a & b \end{bmatrix} \quad \text{(rows swapped)}, \qquad \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} b & a \\ d & c \end{bmatrix} \quad \text{(columns swapped)}.$$

This is exactly the row-exchange operation you met during elimination: when a pivot is zero you swap rows, and that swap is recorded as a left multiplication by a permutation matrix. (In $\mathbf{PA} = \mathbf{LU}$ the $\mathbf{P}$ is precisely the bookkeeping of which rows got exchanged.)

10.5.3 How many are there, and how do they multiply?

There are exactly $n!$ permutation matrices of size $n \times n$, one for each way to reorder n items: $\mathbf{P}$ for $n = 2$ has $2! = 2$ members (identity and the swap), $n = 3$ has $3! = 6$, and so on. The product of two permutation matrices is another permutation matrix (do one shuffle, then the other — still a shuffle). The identity matrix is a permutation (the do-nothing shuffle). And every permutation matrix has an inverse that is also a permutation matrix (reversing the shuffle undoes it). A collection of operations with these properties — closure under multiplication, associativity, an identity element, and an inverse for every member — is what mathematicians call a *group*: a self-contained algebraic system describing symmetries.

Connection to other math

The $n \times n$ permutation matrices form a group under multiplication, and it is an exact copy of the *symmetric group* S_n — the group of all rearrangements of n symbols. This is the historical birthplace of group theory (Galois, permuting the roots of a polynomial). So when you multiply two permutation matrices you are composing two shuffles, and the noncommutativity you may notice (doing swap-then-cycle differs from cycle-then-swap) is the same noncommutativity that makes S_3 the smallest nonabelian group. In fact, by Cayley's theorem, every finite group can be realized as a group of permutation matrices.

10.6 The inverse of a permutation is its transpose

Here the two halves of the chapter meet. To undo a shuffle you shuffle back, and the "shuffle back" matrix is precisely the *transpose* of the original.

Theorem 10.3 (Permutations are orthogonal)**.** *Every permutation matrix* $\mathbf{P}$ *satisfies*

$$\mathbf{P}^{\mathsf{T}}\mathbf{P} = \mathbf{I}, \qquad \textit{equivalently} \qquad \mathbf{P}^{-1} = \mathbf{P}^{\mathsf{T}}.$$

Proof. The columns of $\mathbf{P}$ are the standard basis vectors $\mathbf{e}_1, \ldots, \mathbf{e}_n$ in *some* order; say column j of $\mathbf{P}$ is $\mathbf{e}_{\sigma(j)}$ for some rearrangement σ. The (i, j) entry of $\mathbf{P}^{\mathsf{T}}\mathbf{P}$ is the dot product of column i with column j of $\mathbf{P}$:

$$(\mathbf{P}^{\mathsf{T}}\mathbf{P})_{ij} = \mathbf{e}_{\sigma(i)}^{\mathsf{T}}\,\mathbf{e}_{\sigma(j)}.$$

Standard basis vectors are orthonormal: $\mathbf{e}_a^{\mathsf{T}}\mathbf{e}_b$ is 1 when $a = b$ and 0 otherwise. Since σ is a one-to-one rearrangement, $\sigma(i) = \sigma(j)$ exactly when $i = j$. So the dot product is 1 on the diagonal and 0 off it. That is the identity: $\mathbf{P}^{\mathsf{T}}\mathbf{P} = \mathbf{I}$. Hence $\mathbf{P}^{\mathsf{T}}$ is a left inverse of the square matrix $\mathbf{P}$, so it is the inverse. □

This is your *first orthogonal matrix*. A square matrix $\mathbf{Q}$ is called *orthogonal* when $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$, which means its columns are *orthonormal*: every column has length 1 (normal) and every pair of distinct columns is perpendicular (orthogonal, with dot product zero). Theorem 10.3 says permutation

matrices are orthogonal, and they are the most transparent example: their columns are literally the standard basis, shuffled, which is the most orthonormal a set of vectors can be.

Example 10.3 (Transpose undoes the cycle)**.** Take the 3-cycle from before and its transpose:

$$\mathbf{P} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \qquad \mathbf{P}^{\mathsf{T}} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

Recall $\mathbf{P}$ sends $(x, y, z) \mapsto (z, x, y)$. Its transpose sends $(x, y, z) \mapsto (y, z, x)$ — the opposite cycle. Compose them:

$$\mathbf{P}^{\mathsf{T}}\mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}.$$

Shuffle forward, shuffle back, home again. For a plain swap like $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ the matrix equals its own transpose, so the swap is its own inverse — which matches the picture: reflecting across $y = x$ twice does nothing.

Why this matters

Notice how the two halves of this chapter lock together. The transpose was defined so that $\langle \mathbf{Qx}, \mathbf{Qy} \rangle = \langle \mathbf{x}, \mathbf{Q}^{\mathsf{T}}\mathbf{Qy} \rangle$. Preserving every dot product therefore requires $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$, which immediately forces $\mathbf{Q}^{-1} = \mathbf{Q}^{\mathsf{T}}$. Permutations are the discrete prototype of this rule, preserving lengths by shuffling axes; in Chapter 17, rotations and reflections will do the same thing continuously.

The geometric payoff is that orthogonal matrices — and permutations in particular — preserve lengths. Here is the direct verification.

Proposition 10.2 (Orthogonal matrices preserve length and angle)**.** *If* $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$*, then for all* $\mathbf{x}, \mathbf{y}$*,*

$$(\mathbf{Qx})^{\mathsf{T}}(\mathbf{Qy}) = \mathbf{x}^{\mathsf{T}}\mathbf{y}, \qquad \textit{and in particular} \qquad \|\mathbf{Qx}\| = \|\mathbf{x}\|.$$

Proof. Use the adjoint relation (Theorem 10.2) directly:

$$(\mathbf{Qx})^{\mathsf{T}}(\mathbf{Qy}) = \mathbf{x}^{\mathsf{T}}\mathbf{Q}^{\mathsf{T}}\mathbf{Q}\,\mathbf{y} = \mathbf{x}^{\mathsf{T}}\mathbf{I}\,\mathbf{y} = \mathbf{x}^{\mathsf{T}}\mathbf{y}.$$

Setting $\mathbf{y} = \mathbf{x}$ gives $\|\mathbf{Qx}\|^2 = \|\mathbf{x}\|^2$, and lengths are nonnegative, so $\|\mathbf{Qx}\| = \|\mathbf{x}\|$. □

For a permutation this is obvious in hindsight: $\mathbf{Px}$ has the same numbers as $\mathbf{x}$, just in a different order, and reordering the entries cannot change $\sqrt{x_1^2 + \cdots + x_n^2}$ (Figure 10.6). The proposition says the abstract reason (orthogonality) agrees with the concrete one (same numbers, reshuffled). The determinant of a permutation matrix is therefore ± 1 — it is $+1$ for an even number of swaps and -1 for an odd number, the "sign of the permutation" that will reappear when we build determinants.

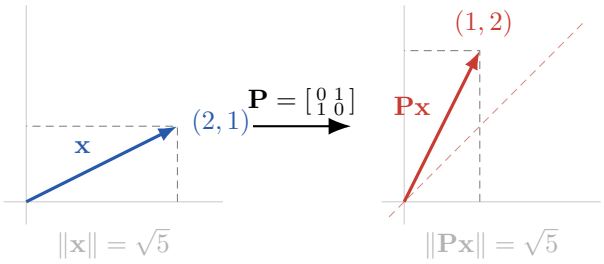


Figure 10.6: A permutation only relabels coordinates, so it lands $\mathbf{x} = (2, 1)$ on $\mathbf{Px} = (1, 2)$, the reflection across $y = x$. Same two numbers, same length $\sqrt{2^2 + 1^2} = \sqrt{5}$. Orthogonality ($\mathbf{P}^\mathsf{T}\mathbf{P} = \mathbf{I}$) is the abstract name for this length preservation.

Walking question

Take a walk and turn this over. A permutation matrix preserves all lengths and angles, so as a rigid motion it is some rotation or reflection of space. Question: which permutations are pure rotations (determinant $+1$) and which include a reflection (determinant -1)? Try the 2×2 swap (is exchanging x and y a rotation, or a flip?) and the 3-cycle. Can a single transposition ever be a rotation of the plane? What does its determinant tell you?

Pause

Pause and think: Why must every orthogonal matrix have determinant equal to $+1$ or -1? *Answer:* Take the determinant of both sides of $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$. Using $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$ and $\det(\mathbf{Q}^\mathsf{T}) = \det(\mathbf{Q})$, we find $\big(\det(\mathbf{Q})\big)^2 = \det(\mathbf{I}) = 1$, which forces $\det(\mathbf{Q}) = \pm 1$. A determinant of $+1$ means the transformation preserves orientation (like a pure rotation); a determinant of -1 means orientation is reversed (including a reflection).

10.7 Putting transposes and permutations together

The two ideas interlock in elimination, which plays a central role in subsequent chapters. When Gaussian elimination needs to avoid a zero pivot, it swaps rows — a left multiplication by a permutation $\mathbf{P}$. The clean statement of elimination "with row exchanges" is the factorization

$$\mathbf{PA} = \mathbf{LU},$$

where $\mathbf{P}$ is a permutation collecting all the swaps, $\mathbf{L}$ is lower triangular, and $\mathbf{U}$ is upper triangular. Because $\mathbf{P}^{-1} = \mathbf{P}^\mathsf{T}$, you can move the permutation to the other side for free: $\mathbf{A} = \mathbf{P}^\mathsf{T}\mathbf{LU}$. No inverse needs computing; the transpose *is* the inverse. This is the practical payoff of Theorem 10.3 and it is why numerical libraries store a permutation as a list of swaps rather than as a matrix — transposing a list of swaps is just reading it backward.

Example 10.4 (A row swap rescuing a pivot)**.** Consider

$$\mathbf{A} = \begin{bmatrix} 0 & 2 \\ 3 & 4 \end{bmatrix}.$$

The top-left pivot is 0, so elimination cannot proceed. Swap the rows with $\mathbf{P} = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$:

$$\mathbf{PA} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 0 & 2 \end{bmatrix},$$

which now has a healthy pivot 3 in the top-left and is already upper triangular (here $\mathbf{L} = \mathbf{I}$, $\mathbf{U} = \mathbf{PA}$). To recover $\mathbf{A}$ itself, multiply by $\mathbf{P}^{\mathsf{T}} = \mathbf{P}$ on the left again — the swap is its own undo (Figure 10.7).

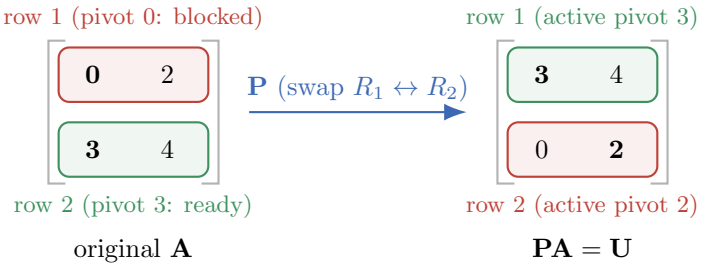


Figure 10.7: Permutation matrix as an elimination pivot rescue. In $\mathbf{A}$ (left), the top-left pivot is 0, so Gaussian elimination cannot proceed without division by zero. Multiplying on the left by the row-swap permutation $\mathbf{P}$ exchanges row 1 and row 2 (right). The resulting matrix $\mathbf{PA}$ has a valid pivot 3 in place and is already in upper-triangular echelon form $\mathbf{U}$.

Common trap

A subtle but common error: people write $\mathbf{A} = \mathbf{LUP}$ or sprinkle the permutation on the right. The swaps in elimination act on *rows*, so $\mathbf{P}$ must sit on the *left*: $\mathbf{PA} = \mathbf{LU}$. Putting it on the right would permute columns, which is a different operation (it reorders the unknowns, not the equations). Left for rows, right for columns — keep that straight and the factorization comes out right.

Formal

Collecting the main identities of this chapter, all for real matrices:

- $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$ and $(\mathbf{A}^{\mathsf{T}})^{\mathsf{T}} = \mathbf{A}$.
- $\langle \mathbf{Ax}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{A}^{\mathsf{T}}\mathbf{y} \rangle$ for all $\mathbf{x}, \mathbf{y}$ (the adjoint relation, which characterizes $\mathbf{A}^{\mathsf{T}}$ uniquely).
- $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and $\mathbf{AA}^{\mathsf{T}}$ are symmetric; the (i, j) entry of $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ is $\mathbf{a}_i^{\mathsf{T}}\mathbf{a}_j$ (the Gram matrix of the columns).
- A permutation matrix has one 1 per row and column; it is orthogonal, $\mathbf{P}^{\mathsf{T}}\mathbf{P} = \mathbf{I}$, so $\mathbf{P}^{-1} = \mathbf{P}^{\mathsf{T}}$.
- Orthogonal matrices ($\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$) preserve dot products, lengths, and angles; their determinant is ± 1.

10.7.1 Looking forward: from matrix arithmetic to vector spaces

Throughout the first ten chapters, our calculations have revolved around concrete systems of equations and grids of numbers: row reductions, matrix products, inverses, factorizations like $\mathbf{LU}$ and $\mathbf{PA} = \mathbf{LU}$, and now the transpose $\mathbf{A}^{\mathsf{T}}$.

Yet every rectangular matrix $\mathbf{A}$ of shape $m \times n$ connects two geometric worlds at once: it pushes vectors forward from $\mathbb{R}^n$ to $\mathbb{R}^m$, while its transpose $\mathbf{A}^\mathsf{T}$ pulls vectors backward from $\mathbb{R}^m$ to $\mathbb{R}^n$. Some inputs get stretched into new directions; others get squashed completely to zero. What are the shapes of the regions formed by all reachable outputs? What are the shapes of the directions that get crushed?

To answer these questions, we must leave behind mechanical entry-by-entry arithmetic and step into the conceptual foundation of the entire subject: *vector spaces and subspaces*. In Chapter 11, we begin Part III by exploring these geometric worlds where vectors live, add, and scale. That path leads directly to Gilbert Strang's celebrated "Big Picture of Linear Algebra" in Chapter 15, where the transpose $\mathbf{A}^\mathsf{T}$ returns to claim center stage: proving that the directions $\mathbf{A}$ squashes to zero are strictly perpendicular to the directions $\mathbf{A}^\mathsf{T}$ can reach.

History and connection

The transpose began as a humble bookkeeping device for organizing algebraic formulas. In 1855, the British mathematician Arthur Cayley introduced matrices to study how algebraic equations transform when you change geometric coordinates. Cayley was tired of writing out endless systems of equations variable by variable; he realized that flipping a rectangular table across its main diagonal turned row equations into column equations, calling the result the "transposed matrix." For decades, textbooks treated Cayley's transpose as little more than a convenient typing shortcut.

The true depth of the transpose emerged in the early twentieth century, when David Hilbert, Stefan Banach, and Frigyes Riesz developed *functional analysis* — the study of infinite-dimensional spaces where the "vectors" are continuous functions and the "matrices" are differential equations or integrals. In continuous spaces, there are no grid tiles to flip across a diagonal. How do you transpose a derivative? Riesz and Banach answered by looking at *dual spaces* — collections of linear measurement probes that turn vectors into scalars, like measuring the temperature or velocity of a system at a point. The *Riesz Representation Theorem* showed that in any space equipped with an inner product (a dot product for functions), every linear measurement probe corresponds to taking the inner product with a unique vector. This led to the modern definition of the *adjoint*: the transpose $\mathbf{A}^\mathsf{T}$ is not defined by its table of entries, but by the fundamental balance $\langle \mathbf{Ax}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{A}^\mathsf{T}\mathbf{y} \rangle$. The finite matrix transpose is simply the Euclidean coordinate shadow of this universal geometric relationship.

In numerical computation, permutation matrices sparked a fierce debate during the dawn of electronic computers in the 1950s. Pioneers like Alan Turing and John von Neumann feared that Gaussian elimination was fundamentally unstable, warning that tiny floating-point rounding errors would cascade and ruin the solution. In 1961, the British numerical analyst James Wilkinson proved that keeping pivots large via row swaps — Gaussian elimination with partial pivoting, $\mathbf{PA} = \mathbf{LU}$ — controls this error growth. Yet Wilkinson uncovered a famous controversy: in the theoretical worst case, partial pivoting can amplify errors by a factor of 2^{n-1}, an exponential worst-case bound. To this day, mathematicians and computer scientists analyze why this worst-case behavior almost never occurs in practice: on real-world engineering problems, $\mathbf{PA} = \mathbf{LU}$ behaves with reliable numerical stability.

Through the transpose we define symmetric matrices ($\mathbf{A}^\mathsf{T} = \mathbf{A}$), which represent observables in quantum mechanics, stress tensors in continuum mechanics, and covariance matrices in statistics, as well as the Gram matrix $\mathbf{A}^\mathsf{T}\mathbf{A}$, which is always symmetric and *positive semidefinite* — meaning it never assigns negative energy to any vector, $\mathbf{x}^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{x} = \|\mathbf{Ax}\|^2 \geq 0$.

This identity is the foundation of least-squares data fitting, the normal equations, and back-propagation in deep learning.
Yet deep questions about permutations and transposes remain active research frontiers today. The celebrated *Graph Isomorphism* problem asks whether two network graphs share the exact same connection structure — which is mathematically identical to asking whether their connection matrices $\mathbf{A}$ and $\mathbf{B}$ are related by a permutation matrix: $\mathbf{P}^\mathsf{T}\mathbf{A}\mathbf{P} = \mathbf{B}$. In 2015, László Babai shook theoretical computer science by proving that graph isomorphism can be solved in quasipolynomial time, but whether a truly efficient polynomial-time algorithm exists remains a famous open question. Meanwhile, in massive scientific simulations, finding the ideal row permutation $\mathbf{P}$ to minimize "fill-in" (preventing zero entries from accidentally turning into expensive nonzeros during $\mathbf{PA} = \mathbf{LU}$) is known to be NP-complete, driving ongoing research into heuristic graph algorithms for supercomputers.

10.8 Exercises

Exercise 10.1 (Warmup flips)**.** Write down the transpose of each matrix and state its shape.

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 0 \\ 5 & 3 & 7 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 4 \\ 4 & 9 \end{bmatrix}, \qquad \mathbf{c} = \begin{bmatrix} 3 \\ -2 \\ 6 \end{bmatrix}.$$

Which of these (if any) is symmetric, and how can you tell at a glance?

Exercise 10.2 (Reversal by hand)**.** Let $\mathbf{A} = \left[\begin{smallmatrix} 2 & 0 \\ 1 & 3 \end{smallmatrix}\right]$ and $\mathbf{B} = \left[\begin{smallmatrix} 1 & 5 \\ 0 & 2 \end{smallmatrix}\right]$. Compute $\mathbf{AB}$, $(\mathbf{AB})^\mathsf{T}$, $\mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$, and $\mathbf{A}^\mathsf{T}\mathbf{B}^\mathsf{T}$. Confirm which two agree and explain in one sentence why the fourth one is the odd one out.

Exercise 10.3 (Dot products as transposes)**.** Let $\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ -2 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}$. Compute $\mathbf{u}^\mathsf{T}\mathbf{v}$, $\mathbf{v}^\mathsf{T}\mathbf{u}$, $\|\mathbf{u}\|^2 = \mathbf{u}^\mathsf{T}\mathbf{u}$, and the outer product $\mathbf{u}\mathbf{v}^\mathsf{T}$ (a 3×3 matrix). Is the outer product symmetric? Why or why not?

Exercise 10.4 (Building Gram matrices)**.** For $\mathbf{A} = \left[\begin{smallmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ compute $\mathbf{A}^\mathsf{T}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^\mathsf{T}$. Verify both are symmetric. Identify which entry of $\mathbf{A}^\mathsf{T}\mathbf{A}$ is the dot product of the two columns of $\mathbf{A}$, and check it by computing that dot product directly.

Exercise 10.5 (Symmetric plus antisymmetric)**.** Split $\mathbf{A} = \left[\begin{smallmatrix} 4 & 1 \\ 7 & 2 \end{smallmatrix}\right]$ into its symmetric part $\frac{1}{2}(\mathbf{A} + \mathbf{A}^\mathsf{T})$ and its antisymmetric part $\frac{1}{2}(\mathbf{A} - \mathbf{A}^\mathsf{T})$. Add them back to confirm you recover $\mathbf{A}$. What must the diagonal of any antisymmetric matrix be, and why?

Exercise 10.6 (Permutations and their actions)**.** Let $\mathbf{P} = \left[\begin{smallmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{smallmatrix}\right]$. Describe in words what $\mathbf{Px}$ does to $\mathbf{x} = (x_1, x_2, x_3)$. Compute $\mathbf{P}^\mathsf{T}$ and verify $\mathbf{P}^\mathsf{T}\mathbf{P} = \mathbf{I}$. Is this $\mathbf{P}$ its own inverse? Explain using the cycle structure.

Exercise 10.7 (Left versus right)**.** With $\mathbf{P} = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ and an arbitrary $\mathbf{M} = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$, compute $\mathbf{PM}$ and $\mathbf{MP}$. State precisely what each does to $\mathbf{M}$ and connect it to row swaps versus column swaps in elimination.

Exercise 10.8 (Adjoint by example)**.** Let $\mathbf{A} = \left[\begin{smallmatrix} 1 & 2 \\ 0 & 1 \\ 3 & 0 \end{smallmatrix}\right]$, $\mathbf{x} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, and $\mathbf{y} = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}$. Compute $\mathbf{Ax}$ and $\langle \mathbf{Ax}, \mathbf{y} \rangle$; then compute $\mathbf{A}^\mathsf{T}\mathbf{y}$ and $\langle \mathbf{x}, \mathbf{A}^\mathsf{T}\mathbf{y} \rangle$. Confirm the two numbers agree, illustrating Theorem 10.2.

Exercise 10.9 (Counting and composing)**.** How many 3×3 permutation matrices are there? Pick the swap of rows 1 and 2, call it $\mathbf{P}_{12}$, and the swap of rows 2 and 3, call it $\mathbf{P}_{23}$. Compute $\mathbf{P}_{12}\mathbf{P}_{23}$ and $\mathbf{P}_{23}\mathbf{P}_{12}$. Are they equal? What does your answer say about whether the group of permutation matrices is commutative?

Exercise 10.10 (Orthogonality preserves geometry)**.** Let $\mathbf{P}$ be any 3×3 permutation matrix and $\mathbf{x} = (1, -2, 2)$. Without choosing a specific $\mathbf{P}$, explain why $\|\mathbf{Px}\| = 3$ no matter which permutation you pick. Then prove in general that if $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ and $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$ then the product $\mathbf{QR}$ is also orthogonal.

Exercise 10.11 (A proof to finish)**.** Prove that if $\mathbf{A}$ is any $m \times n$ matrix and $\mathbf{A}^\mathsf{T}\mathbf{A} = \mathbf{0}$ (the zero matrix), then $\mathbf{A} = \mathbf{0}$. *Hint:* look at the diagonal entries of $\mathbf{A}^\mathsf{T}\mathbf{A}$ and recall what they equal in terms of the columns of $\mathbf{A}$. In particular, $\mathbf{A}^\mathsf{T}\mathbf{A}$ cannot collapse to zero unless $\mathbf{A}$ already was zero, which is the seed of why $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible exactly when $\mathbf{A}$ has independent columns.

Exercise 10.12 (Elimination with row swaps: $\mathbf{PA} = \mathbf{LU}$)**.** Consider the matrix

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 3 \\ 2 & 1 & 1 \\ 4 & 5 & 2 \end{bmatrix}.$$

(a) Explain why standard Gaussian elimination cannot begin without swapping rows.

(b) Let $\mathbf{P}$ be the permutation matrix that exchanges row 1 and row 2. Write down $\mathbf{P}$ and compute the product $\mathbf{PA}$.

(c) Perform elimination on $\mathbf{PA}$ to find a unit lower-triangular matrix $\mathbf{L}$ and an upper-triangular matrix $\mathbf{U}$ such that $\mathbf{PA} = \mathbf{LU}$.

(d) Using $\mathbf{P}^{-1} = \mathbf{P}^\mathsf{T}$, write down the factorization of the original matrix as $\mathbf{A} = \mathbf{P}^\mathsf{T}\mathbf{LU}$, and verify by multiplication that it recovers $\mathbf{A}$.

Exercise 10.13 (Permutation similarity and graph isomorphism)**.** Let a 3-node network (graph) have connections given by the symmetric adjacency matrix

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix},$$

where entry $a_{ij} = 1$ indicates an undirected edge connecting node i and node j (a line network: node 1 is connected to node 2, which is connected to node 3).

(a) Let $\mathbf{P} = \left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]$ be the permutation that exchanges node 1 and node 2. Compute $\mathbf{B} = \mathbf{P}^\mathsf{T}\mathbf{AP}$.

(b) Explain why left-multiplication by $\mathbf{P}^\mathsf{T}$ permutes the rows while right-multiplication by $\mathbf{P}$ permutes the columns, so that $\mathbf{P}^\mathsf{T}\mathbf{AP}$ renames both source and target of every edge consistently.

(c) Verify that $\mathbf{B}$ is also symmetric. Which node is now the central hub connected to the other two? Conclude that $\mathbf{B}$ represents the exact same network structure, merely with the node labels renumbered.

Part III

Vector Spaces

Chapter 11

Vector Spaces and Subspaces

In the previous chapter, we finished the concrete mechanics of elimination: row exchanges reordered equations through permutation matrices $\mathbf{P}$, and transposes $\mathbf{A}^\mathsf{T}$ flipped rows into columns to link matrices with dot products, completing the factorization $\mathbf{PA} = \mathbf{LU}$. Throughout those first ten chapters, we worked concretely: we added vectors in $\mathbb{R}^2$ and $\mathbb{R}^3$, scaled them, took their dot products, packed them into rectangular matrices, and tracked their numerical entries through row operations. Every object we touched was a list of numbers. That was the right place to start, because lists of numbers are something you can visualize: an arrow on a page, a point in space, a recipe for mixing ingredients.

Now we ask a more general question. Forget for a moment *what* a vector is made of. Ask instead: what can we *do* with vectors, and which collections of them are "closed" under those operations? The answer is the idea of a *vector space*, and its most useful relative, the *subspace*.

This is the chapter where linear algebra stops being a bag of tricks for $\mathbb{R}^n$ and becomes a language. Once you have the vector-space idea, the same theorems apply to arrows, to functions, to signals, to polynomials, to images, to the solution sets of differential equations. You will prove something once and use it everywhere. We will build the idea slowly, picture first.

11.1 What "space" is supposed to capture

11.1.1 The two operations we keep using

Look back at everything we did with vectors. Strip it down and only two operations ever appeared:

- **Adding** two vectors to get a third vector: $\mathbf{u} + \mathbf{v}$.
- **Scaling** a vector by a number: $c\,\mathbf{v}$.

Dot products, lengths, and angles are wonderful, but they are *extra*. The bare skeleton of linear algebra is just "add" and "scale." A linear combination $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$ is exactly these two operations used together. Span, independence, dimension, rank, and eigenvectors will all be defined using only adding and scaling.

Intuition

A vector space is any world where "add" and "scale" both make sense and never throw you out of the world. You start inside the world, you add or scale, and you always land back inside. Nothing leaks out. That "nothing leaks out" property – called *closure* – is the foundation that keeps the entire algebraic machinery consistent.

The numbers we scale by are called *scalars*. For us they will always be real numbers (we will say a word about complex scalars later in the book). A vector space whose scalars are real numbers is called a *real vector space*.

11.1.2 Why bother abstracting?

Here is the payoff stated up front, before we have even seen the definition. Consider three collections that look nothing alike:

- all arrows in the plane $\mathbb{R}^2$;
- all polynomials of degree at most 2, like $3 - x + 5x^2$;
- all smooth functions $y(t)$ satisfying $y'' + y = 0$.

The first is geometry, the second is algebra, the third is calculus. Yet in each one you can add two members and get a member, and scale a member and get a member. They are all vector spaces. Anything we prove about "vector spaces in general" is automatically true in all three at once. That is the leverage. You do the work once.

Why this matters

Arrows in the plane and continuous functions look nothing alike, yet both obey the exact same rules of addition and scaling. Because of that shared structure, a sound wave, a grayscale image, a polynomial, and a quantum state all live in vector spaces – and every theorem we prove from the axioms applies to all of them at once.

11.2 The axioms of a vector space (made friendly)

We now write down the official rules. Do not be intimidated by the list; every line is something you already believe about ordinary arrows. The point of writing them out is so that later we can *check* whether some new, exotic collection deserves to be called a vector space.

Formal

A **(real) vector space** is a set V together with two operations – vector addition $\mathbf{u} + \mathbf{v}$ and scalar multiplication $c\,\mathbf{v}$ for $c \in \mathbb{R}$ – such that for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all scalars $c, d \in \mathbb{R}$:

Closure.

(1) $\mathbf{u} + \mathbf{v} \in V$ (sums stay inside)

(2) $c\,\mathbf{v} \in V$ (scalings stay inside)

Addition behaves.

(3) $\mathbf{u}+\mathbf{v}=\mathbf{v}+\mathbf{u}$ (commutative)

(4) $(\mathbf{u}+\mathbf{v})+\mathbf{w}=\mathbf{u}+(\mathbf{v}+\mathbf{w})$ (associative)

(5) there is a *zero vector* $\mathbf{0}\in V$ with $\mathbf{v}+\mathbf{0}=\mathbf{v}$

(6) every $\mathbf{v}$ has a *negative* $-\mathbf{v}\in V$ with $\mathbf{v}+(-\mathbf{v})=\mathbf{0}$

Scaling behaves.

(7) $1\,\mathbf{v}=\mathbf{v}$ (scaling by one does nothing)

(8) $c(d\,\mathbf{v})=(cd)\,\mathbf{v}$ (scalings compose)

(9) $c(\mathbf{u}+\mathbf{v})=c\,\mathbf{u}+c\,\mathbf{v}$ (distribute over vectors)

(10) $(c+d)\,\mathbf{v}=c\,\mathbf{v}+d\,\mathbf{v}$ (distribute over scalars)

That is ten rules. Eight of them (3 through 10) are just "the algebra you already do works the way you expect." The only two with real teeth are the first two: closure under addition and closure under scaling. Keep your eye on those.

Focus

When you meet a new candidate vector space, the rules that almost always do the work are **closure under addition** and **closure under scalar multiplication**, plus **"is 0 actually in there?"** The eight bookkeeping rules are usually inherited for free from the arithmetic of real numbers. Spend your energy on closure.

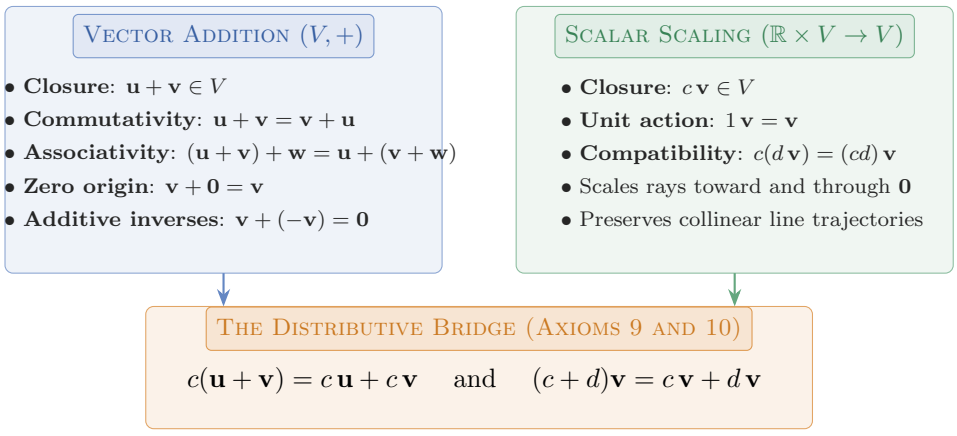


Figure 11.1: The two fundamental operations of every vector space. Addition provides commutative, reversible translation along a grid (you can reorder steps, group them, and always return to $\mathbf{0}$), while scalar multiplication defines a compatible scaling action along lines through the origin. The distributive laws bridge the two, guaranteeing that scaling a parallelogram stretches both its sides and its diagonal in unison.

History and connection

For more than two centuries following Descartes and Newton, algebra and geometry stayed tethered to coordinate axes and mechanical arrows. Vectors were either physical directed segments representing velocities and forces in $\mathbb{R}^2$ and $\mathbb{R}^3$, or concrete coordinate tuples pushed through paper-and-pencil elimination. But across the nineteenth century, as Joseph Fourier decomposed heat diffusion into infinite trigonometric series and as mathematicians investigated linear differential equations such as $y''+\omega^2 y = 0$, they noticed something fundamental: families of solutions to these equations could be added together and scaled by real numbers, obeying the same algebraic rules as geometric arrows in space. Yet their elements were continuous curves and functions, not little directed arrows.

In 1844, the German polymath Hermann Grassmann published a revolutionary work treating geometric entities—directed lines, planar areas, and volumes—as algebraic objects that could be added and multiplied. But his writing was dense, philosophical, and almost entirely neglected. In 1888, the Italian mathematician and logician Giuseppe Peano set out to give Grassmann's ideas a clean, unshakeable foundation. Armed with the tools of symbolic logic (which he pioneered), Peano published *Calcolo Geometrico*, stripping away all talk of coordinates, physical space, and geometry. He formulated for the first time in history the modern axiomatic definition of a vector space over the real numbers—the ten formal rules listed above. In the 1930s, the influential collective of French mathematicians writing under the pseudonym *Nicolas Bourbaki* elevated this axiomatic, coordinate-free approach into the universal language of modern mathematics.

Peano's abstraction initially met skepticism. Traditional geometers complained that axiomatization severed mathematics from its intuitive physical roots, turning geometric intuition into formal symbol manipulation. Applied scientists and computational mathematicians asked a skeptical question: how does verifying whether an abstract set satisfies an associative or distributive axiom help anyone solve a concrete numerical problem?

Yet the abstract viewpoint soon proved its worth, because physical arrow intuition alone stops short as soon as the objects are signals or functions. If vectors must be arrows in physical space, linear algebra cannot speak about audio signals, probability distributions, digital image matrices, or quantum mechanical states. By leaving the nature of the objects open and specifying only how they add and scale, Peano let one proof do the work of many. Prove a theorem once from the ten axioms, and that single proof immediately governs arrows in three-space, polynomials of any degree, continuous functions on an interval, and the solution sets of differential equations. What remains missing here is any notion of measurement—lengths, angles, and distances will require inner products (Chapter 17)—and in infinite-dimensional spaces, algebra alone cannot decide whether infinite sequences of vectors converge, launching modern functional analysis. Most immediately, the abstract axioms do not yet tell us how to compute within a specific matrix, which we take up next in Chapter 12 with column spaces and nullspaces.

11.2.1 Two small theorems you get for free

The axioms are spare on purpose. Notice they do *not* say "$0\,\mathbf{v} = \mathbf{0}$" or "$(-1)\mathbf{v} = -\mathbf{v}$." Those feel obvious, but in a properly austere theory we should *derive* them. Watch how the rules force them.

Proposition 11.1. *In any vector space,* $0\,\mathbf{v} = \mathbf{0}$ *for every* $\mathbf{v}$.

Proof. Start from a true fact about real numbers, $0 + 0 = 0$, and hit $\mathbf{v}$ with it. By rule (10),

$$0\,\mathbf{v} = (0 + 0)\,\mathbf{v} = 0\,\mathbf{v} + 0\,\mathbf{v}.$$

Now add the negative of $0\,\mathbf{v}$ (which exists by rule (6)) to both sides. The left becomes $\mathbf{0}$, and on the right $0\,\mathbf{v} + 0\,\mathbf{v} + (-(0\,\mathbf{v}))$ collapses to $0\,\mathbf{v}$. So $\mathbf{0} = 0\,\mathbf{v}$. Done. □

Proposition 11.2. *In any vector space,* $(-1)\,\mathbf{v} = -\mathbf{v}$.

Proof. We show $(-1)\mathbf{v}$ is *the* negative of $\mathbf{v}$, i.e. it adds to $\mathbf{0}$:

$$\mathbf{v} + (-1)\mathbf{v} = 1\,\mathbf{v} + (-1)\mathbf{v} = (1 + (-1))\,\mathbf{v} = 0\,\mathbf{v} = \mathbf{0},$$

using rule (7), then rule (10), then arithmetic, then Proposition 11.1. Because additive inverses are unique in any vector space (if $\mathbf{w}$ also satisfies $\mathbf{v} + \mathbf{w} = \mathbf{0}$, then $\mathbf{w} = \mathbf{w} + \mathbf{0} = \mathbf{w} + (\mathbf{v} + (-\mathbf{v})) = (\mathbf{w} + \mathbf{v}) + (-\mathbf{v}) = \mathbf{0} + (-\mathbf{v}) = -\mathbf{v}$), we conclude $(-1)\mathbf{v} = -\mathbf{v}$. □

Intuition

These tiny proofs are your first taste of the abstract style. We never used coordinates, never wrote a single number inside a vector. We used only the rules. That is exactly why the conclusion holds in *every* vector space at once – arrows, polynomials, functions, all of them.

11.3 The home example: $\mathbb{R}^n$

The first vector space, and the one you should always picture, is $\mathbb{R}^n$: all columns of n real numbers,

$$\mathbb{R}^n = \left\{ \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} : x_1, \ldots, x_n \in \mathbb{R} \right\},$$

with the addition and scaling you already know – componentwise. For $n = 2$ and $n = 3$ you can literally draw the vectors as arrows from the origin. For larger n you cannot draw them, but the rules are identical, so your geometric intuition keeps working.

Example 11.1. Let us sanity-check that $\mathbb{R}^2$ obeys closure. Take $\mathbf{u} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$ and the scalar $c = 5$. Then

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} 2 + (-3) \\ -1 + 4 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix} \in \mathbb{R}^2, \qquad 5\,\mathbf{u} = \begin{bmatrix} 10 \\ -5 \end{bmatrix} \in \mathbb{R}^2.$$

A sum of two columns of two reals is a column of two reals; a scalar times such a column is again such a column. Closure holds, and you can see the eight bookkeeping rules are just facts about adding and multiplying the entries.

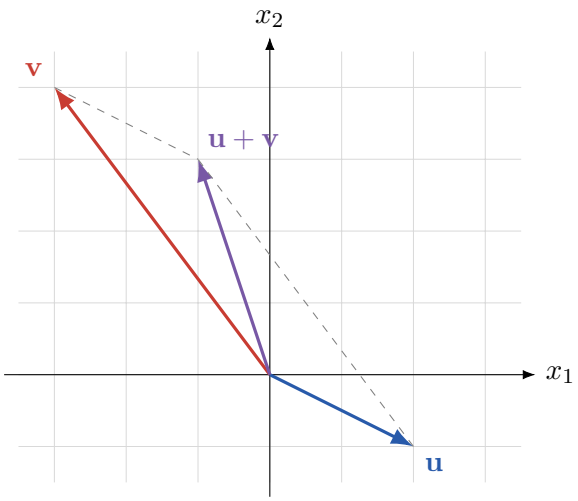


Figure 11.2: Addition in $\mathbb{R}^2$ is the parallelogram rule: slide $\mathbf{v}$ to the tip of $\mathbf{u}$ and the diagonal is $\mathbf{u}+\mathbf{v}$. The sum is another arrow from the origin – still inside $\mathbb{R}^2$. That is closure, drawn.

Before/after Strang

Strang opens his course with $\mathbb{R}^n$ and the four fundamental subspaces, and only later – sometimes in a footnote – mentions the general axioms. Reading this *before* Strang, take the axioms as the "why $\mathbb{R}^n$ works" story so that when he says "column space" or "nullspace" you already hear "a subspace" – a smaller vector space living inside $\mathbb{R}^n$. Reading this *after* Strang, the axioms are what let you carry every result you learned about $\mathbb{R}^n$ over to function spaces and operator theory without redoing a thing.

11.4 Spaces that are not columns of numbers

To feel the power of the definition, we need examples where the "vectors" are not columns at all. Function and matrix spaces arise throughout modern applications.

11.4.1 Matrices form a vector space

The set of all 2×2 real matrices, call it $M_{2\times 2}$, is a vector space. You add matrices entrywise and scale them entrywise, and both stay 2×2. The zero vector is the zero matrix $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$.

Example 11.2. With $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 3 & 0 \\ 1 & 4 \end{bmatrix}$, and $c = 2$:

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}, \qquad 2\mathbf{A} = \begin{bmatrix} 2 & 4 \\ 0 & -2 \end{bmatrix}.$$

Both are again 2×2 matrices. Here a single "vector" is a whole 2×2 grid of numbers – four numbers, so this space is isomorphic to $\mathbb{R}^4$, with entries arranged in a square array.

Pause

Pause and think: matrices with determinant zero. Does the collection of all 2×2 matrices with $\det(\mathbf{A}) = 0$ form a vector space? Test closure under addition:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Both have determinant $(1)(0) - (0)(0) = 0$, so both live in the candidate set. Yet their sum is the identity matrix $\mathbf{A} + \mathbf{B} = \mathbf{I}$, whose determinant is $1 \neq 0$. A single addition escapes the set, so singular matrices do *not* form a vector space.

11.4.2 Polynomials form a vector space

Let P_2 be all polynomials of degree at most 2:

$$p(x) = a_0 + a_1 x + a_2 x^2, \qquad a_0, a_1, a_2 \in \mathbb{R}.$$

Add two such polynomials and you get another one of degree at most 2; scale one and the degree cannot grow. The zero vector is the constant polynomial 0. Closure holds.

Example 11.3. With $p = 3 - x + 5x^2$ and $q = 1 + 2x - 5x^2$:

$$p + q = 4 + x + 0\,x^2 = 4 + x.$$

Notice the x^2 terms cancelled and the sum has degree 1 – still "degree at most 2," so still inside P_2. (This is exactly why we say "at most." The polynomials of degree *exactly* 2 are *not* a vector space, because two of them can sum to something of lower degree, and worse, the zero polynomial is missing.)

11.4.3 Functions form a vector space

This is the example that opens the most doors. Let $C[0, 1]$ be the collection of all continuous functions f on the closed interval $[0, 1]$—functions whose graphs can be drawn without lifting your pencil. Add two continuous functions *pointwise* (meaning at each input number x, you add their output heights: $(f + g)(x) = f(x) + g(x)$), and the result is guaranteed by calculus to be continuous. Scale a continuous function by multiplying every output height by c, and it stays continuous. The zero vector is the zero function $z(x) = 0$, the flat horizontal line at height zero. So $C[0, 1]$ is a vector space—an infinite-dimensional one, but a genuine vector space all the same.

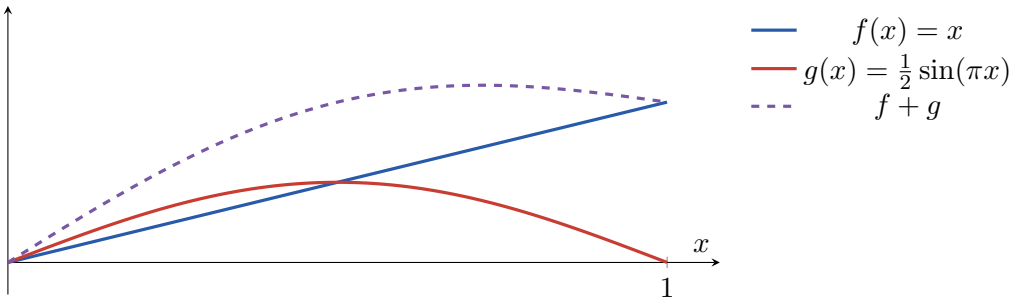


Figure 11.3: Adding functions is adding arrows "one x at a time." At each input x you add the two output heights. The dashed curve $f+g$ is the sum vector. Continuous plus continuous is continuous, so we never leave the space.

Modern application

Where function spaces run the modern world. A grayscale image is a function from pixels to brightness; adding two images (double exposure) and scaling brightness are exactly the vector-space operations, which is why a 64×64 image lives in $\mathbb{R}^{4096}$ and can be fed to a neural network as a vector. An audio clip is a function from time to amplitude; mixing two tracks is vector addition, turning the volume knob is scalar multiplication. The Fourier transform, JPEG compression, and the embeddings inside a large language model all rest on treating data as vectors in a (possibly enormous) vector space and finding a good *basis* – the very next ideas in this book. The reason a single piece of mathematics powers images, audio, and text is that all three are vector spaces.

Connection to other math

Differential equations live in function spaces. Consider all solutions of $y'' + y = 0$. If y_1 and y_2 both solve it, so does $c_1y_1 + c_2y_2$, because differentiation is linear:

$$(c_1y_1 + c_2y_2)'' + (c_1y_1 + c_2y_2) = c_1\underbrace{(y_1'' + y_1)}_{0} + c_2\underbrace{(y_2'' + y_2)}_{0} = 0.$$

So the solution set is closed under addition and scaling – it is a *subspace* of the space of all smooth functions. The two solutions $\cos t$ and $\sin t$ turn out to span it, which is exactly why "the general solution is $c_1\cos t + c_2\sin t$." That phrase from your differential-equations course is a statement about a 2-dimensional vector space. We will make "basis" and "dimension" precise soon, and you will see this example again.

11.5 Subspaces: vector spaces hiding inside vector spaces

Most of the time we do not invent a vector space from nothing. We already have a big one – usually $\mathbb{R}^n$ – and we care about some special collection of vectors sitting *inside* it: the solutions of a system, the outputs of a matrix, a plane through the origin. We want to ask: is that smaller collection itself a vector space, using the same add and scale? If so, we call it a *subspace*, and we inherit every theorem for free.

Intuition

Picture $\mathbb{R}^3$, all of 3D space. A subspace is a perfectly flat sheet (a plane) or a perfectly straight line that *passes through the origin*, extending forever in both directions, with no edges. The defining feature: if you stand on it and walk in any direction allowed by the sheet, or scale your position toward or past the origin, you stay on the sheet. The origin must be included, because scaling any vector by 0 lands you there.

11.5.1 The subspace test

We do not want to re-verify all ten axioms every time. The good news: most of them are *inherited* automatically from the big space. Commutativity, associativity, and the distributive laws are already true for the parent's vectors, so they are true for any subset of them. The only things that can go wrong are closure and the presence of $\mathbf{0}$ and negatives. That collapses to a short, practical test.

Formal

Subspace test. A nonempty subset W of a vector space V is a *subspace* if and only if:

(a) $\mathbf{0} \in W$ (it contains the origin);

(b) $\mathbf{u}, \mathbf{v} \in W \implies \mathbf{u} + \mathbf{v} \in W$ (closed under addition);

(c) $\mathbf{v} \in W,\ c \in \mathbb{R} \implies c\,\mathbf{v} \in W$ (closed under scaling).

Equivalently and most compactly: W is a subspace iff it is nonempty and for all $\mathbf{u}, \mathbf{v} \in W$ and all scalars c, d, the linear combination $c\,\mathbf{u} + d\,\mathbf{v}$ is again in W. "Closed under linear combinations" is the whole condition.

Why is condition (a) really needed if we have (b) and (c)? Because (c) with $c = 0$ forces $\mathbf{0} \in W$ *provided W has at least one vector*. So (a) is really just the reminder "W is not empty," and the fast way to disqualify a candidate is to check whether $\mathbf{0}$ is in it. If the origin is missing, stop – it is not a subspace.

Common trap

The number-one mistake: forgetting the origin. A line that does *not* pass through the origin – say all points with $x_2 = x_1 + 1$ in $\mathbb{R}^2$ – is not a subspace, even though it is perfectly straight and "looks linear." The point $\mathbf{0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ is not on it ($0 \neq 0 + 1$), and adding two of its points lands you off the line. Always, *always* check the origin first. "Through the origin" is the dividing line between a subspace and a mere shifted copy (an *affine* set – a flat line or plane displaced away from the origin by a fixed shift vector, which we meet when solving $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{b} \neq \mathbf{0}$).

11.5.2 A worked subspace and a worked non-subspace

Example 11.4. A line through the origin is a subspace. Let

$$W = \left\{ \begin{bmatrix} t \\ 2t \end{bmatrix} : t \in \mathbb{R} \right\} \subseteq \mathbb{R}^2,$$

the line $x_2 = 2x_1$. Check the three conditions.

- *Origin:* take $t = 0$ to get $\begin{bmatrix} 0 \\ 0 \end{bmatrix} \in W$. Good.
- *Addition:* $\begin{bmatrix} s \\ 2s \end{bmatrix} + \begin{bmatrix} t \\ 2t \end{bmatrix} = \begin{bmatrix} s+t \\ 2(s+t) \end{bmatrix}$, which has the form $\begin{bmatrix} r \\ 2r \end{bmatrix}$ with $r = s + t$. Still in W.
- *Scaling:* $c \begin{bmatrix} t \\ 2t \end{bmatrix} = \begin{bmatrix} ct \\ 2(ct) \end{bmatrix}$, again of the form $\begin{bmatrix} r \\ 2r \end{bmatrix}$. Still in W.

All three hold, so W is a subspace – a one-dimensional subspace, the line of slope 2.

Example 11.5. The first quadrant is not a subspace. Let $W = \left\{ \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} : x_1 \geq 0,\ x_2 \geq 0 \right\}$, the corner of the plane where both coordinates are nonnegative. The origin is in it, and the sum of two nonnegative vectors is nonnegative, so (a) and (b) hold. But scaling fails: take $\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in W$ and $c = -1$. Then $c\mathbf{v} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, which has negative entries and is *not* in W. One counterexample to one condition is enough. The first quadrant is not a subspace.

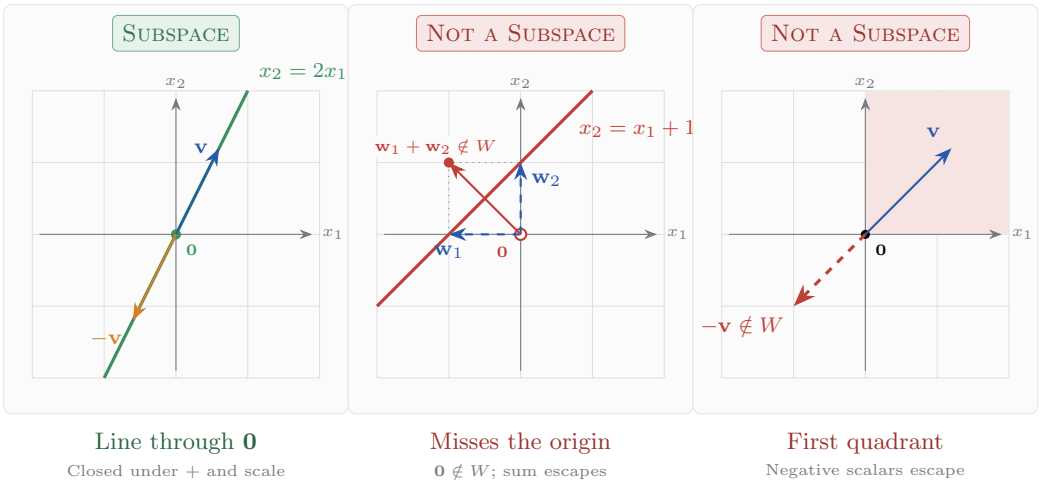


Figure 11.4: The subspace test across three test cases. *Left:* a line through the origin is closed under both addition and scalar multiplication – an authentic subspace. *Middle:* an affine line shifted off the origin fails the first checkpoint ($\mathbf{0} \notin W$), and vector addition escapes the line. *Right:* the first quadrant contains the origin and is closed under positive addition, but scaling by negative numbers escapes into the third quadrant.

Walking question

Take a walk and classify these subsets of $\mathbb{R}^2$ in your head – subspace or not, and which condition fails if not: (i) the single point $\{\mathbf{0}\}$; (ii) all of $\mathbb{R}^2$; (iii) the x-axis; (iv) the unit circle $x_1^2 + x_2^2 = 1$; (v) the two coordinate axes together (their union); (vi) all vectors with $x_1 + x_2 = 0$. Two of these are sneaky. The circle misses the origin *and* fails closure; the union

of the axes contains the origin and is closed under scaling but *not* under addition – add $\begin{bmatrix}1\\0\end{bmatrix}$ and $\begin{bmatrix}0\\1\end{bmatrix}$ and you fall off both axes.

11.6 The complete list of subspaces of $\mathbb{R}^3$

A beautiful thing happens in $\mathbb{R}^3$: there are only four *kinds* of subspace, and you can picture every one of them. This is a great reality check on the whole idea.

Theorem 11.1. *Every subspace of $\mathbb{R}^3$ is exactly one of the following:*

(i) *the **zero subspace** $\{\mathbf{0}\}$ – just the origin (dimension 0);*

(ii) *a **line through the origin** (dimension 1);*

(iii) *a **plane through the origin** (dimension 2);*

(iv) ***all of** $\mathbb{R}^3$ (dimension 3).*

There is nothing else. No curves, no half-spaces, no off-center planes.

We are not ready to prove this rigorously – it needs the idea of dimension, a few chapters ahead – but the picture is already convincing. A subspace must contain whole lines through the origin (closure under scaling) and the parallelogram-sums of any two such lines (closure under addition). The moment a subspace contains two vectors pointing in genuinely different directions, it must contain the entire flat plane they span. The moment it contains three vectors in genuinely different directions, it is forced to be all of $\mathbb{R}^3$. The "size" can only jump $0, 1, 2, 3$.

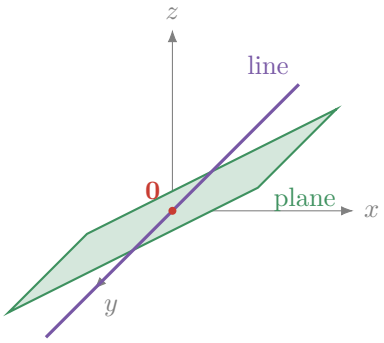


Figure 11.5: The nontrivial subspaces of $\mathbb{R}^3$: lines and planes through the origin. Every subspace contains $\mathbf{0}$ (red), and every line and plane shown extends infinitely and is perfectly flat. Adding the zero subspace $\{\mathbf{0}\}$ and all of $\mathbb{R}^3$ completes the list.

Example 11.6. A plane through the origin, checked. Let $W = \left\{ \begin{bmatrix}x\\y\\z\end{bmatrix} : x + 2y - z = 0 \right\} \subseteq \mathbb{R}^3$.

This is the plane with normal vector $\begin{bmatrix}1\\2\\-1\end{bmatrix}$ through the origin.

- *Origin:* $0 + 2(0) - 0 = 0$, so $\mathbf{0} \in W$.
- *Addition:* if $x_1 + 2y_1 - z_1 = 0$ and $x_2 + 2y_2 - z_2 = 0$, then summing, $(x_1 + x_2) + 2(y_1 + y_2) - (z_1 + z_2) = 0$, so the sum satisfies the equation.
- *Scaling:* multiplying the equation by c gives $cx + 2(cy) - (cz) = 0$, so $c\mathbf{v} \in W$.

A subspace. Contrast with the *affine* plane $x + 2y - z = 5$: the origin fails ($0 \neq 5$), and adding two solutions gives a right-hand side of 10, not 5. Not a subspace. The pattern is general: a single homogeneous equation ($= 0$, no constant term) through the origin gives a subspace; a nonzero constant on the right breaks it.

Focus

The phrase to burn in: **homogeneous means through the origin means subspace.** The solution set of $\mathbf{Ax} = \mathbf{0}$ is always a subspace (we will call it the *nullspace*). The solution set of $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{b} \neq \mathbf{0}$ is *never* a subspace – it is a shifted copy of one. This single distinction organizes the next several chapters.

11.7 Spans are subspaces

We close with the construction that makes subspaces appear everywhere. Recall the *span* of a set of vectors is all of their linear combinations:

$$\operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\} = \{c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k : c_1, \ldots, c_k \in \mathbb{R}\}.$$

We claimed earlier, when we first met span, that it produces lines and planes through the origin. Now we can say exactly why: *a span is automatically a subspace.* This is the single most common way subspaces arise.

Theorem 11.2. *For any vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ in a vector space V, the set $\operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ is a subspace of V. In fact it is the* smallest *subspace containing all of the $\mathbf{v}_i$.*

Proof. Write $S = \operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$. We run the subspace test.

Origin. Choosing all coefficients zero gives $0\mathbf{v}_1 + \cdots + 0\mathbf{v}_k = \mathbf{0} \in S$.

Closed under addition. Take two members of S,

$$\mathbf{u} = a_1\mathbf{v}_1 + \cdots + a_k\mathbf{v}_k, \qquad \mathbf{w} = b_1\mathbf{v}_1 + \cdots + b_k\mathbf{v}_k.$$

Adding and grouping by each $\mathbf{v}_i$,

$$\mathbf{u} + \mathbf{w} = (a_1 + b_1)\mathbf{v}_1 + \cdots + (a_k + b_k)\mathbf{v}_k,$$

which is again a linear combination of the $\mathbf{v}_i$, hence in S.

Closed under scaling. For any scalar c,

$$c\,\mathbf{u} = (ca_1)\mathbf{v}_1 + \cdots + (ca_k)\mathbf{v}_k \in S.$$

All three conditions hold, so S is a subspace. For "smallest": any subspace W that contains all the $\mathbf{v}_i$ must (by closure under linear combinations) contain every combination $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$, i.e. all of S. So $S \subseteq W$ for every such W; S is the smallest. □

Intuition

The proof is almost too easy, and that is the point. "A linear combination of linear combinations is again a linear combination." Adding two recipes that each only use the ingredients $\mathbf{v}_1, \dots, \mathbf{v}_k$ produces a recipe that still only uses those ingredients. You can never escape the span by combining things inside it – which is precisely closure.

Example 11.7. The span of $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$ in $\mathbb{R}^3$ is the set of all $\begin{bmatrix} a \\ b \\ a+b \end{bmatrix}$. By the theorem it is a subspace, and since the two vectors point in different directions it is a plane through the origin (Figure 11.6). Which plane? A vector $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$ lies on it exactly when $z = x+y$, that is $x+y-z = 0$ – the homogeneous plane equation with normal $\begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$. The span description and the equation description are two faces of the same subspace; converting between them is what elimination is for, coming up next.

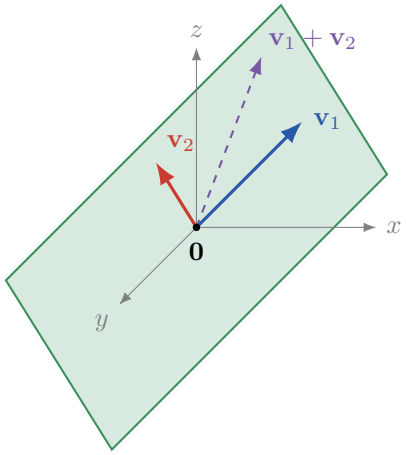


Figure 11.6: The span of two independent vectors $\mathbf{v}_1, \mathbf{v}_2$ in $\mathbb{R}^3$ is the whole plane through the origin that they lie in – here the plane $x+y-z=0$. Every linear combination (such as the dashed $\mathbf{v}_1+\mathbf{v}_2$) stays in the green sheet: that is Theorem 11.2, drawn.

Compute it yourself

By hand, with $\mathbf{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$ in $\mathbb{R}^2$:

1. Compute $2\mathbf{v}_1 + 3\mathbf{v}_2$ and confirm it is a vector in $\mathbb{R}^2$ (closure).
2. Argue that since $\mathbf{v}_1, \mathbf{v}_2$ point in different directions, their span is all of $\mathbb{R}^2$. (Hint: can you hit $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$? Solve $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.)

3. Now replace $\mathbf{v}_2$ by $\begin{bmatrix}-4\\-2\end{bmatrix}$. What is $\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$ now, and what dimension is it? (The second vector is $-2\mathbf{v}_1$, so it adds nothing.)

This little lab is the seed of *independence* and *dimension*: whether the spanning vectors genuinely add new directions.

Before/after Strang

When Strang introduces the column space $\text{col}(\mathbf{A})$ as "all combinations of the columns of $\mathbf{A}$," he is taking a span – so by Theorem 11.2 it is automatically a subspace, no further checking needed. Likewise the nullspace is a span of the special solutions. Half of Strang's "four subspaces" chapter is really this one theorem applied four times. Carry it with you.

11.8 Operations on subspaces

Two subspaces can be combined to make new ones, though one natural-looking combination fails. This sharpens your instinct for closure.

Proposition 11.3. *If U and W are subspaces of V, then their intersection $U \cap W$ (vectors in both) is a subspace.*

Proof. The origin lies in both U and W, so $\mathbf{0} \in U \cap W$. If $\mathbf{u}, \mathbf{v} \in U \cap W$, then both lie in U (so $\mathbf{u} + \mathbf{v} \in U$ and $c\mathbf{u} \in U$) and both lie in W (so $\mathbf{u} + \mathbf{v} \in W$ and $c\mathbf{u} \in W$). Hence the sum and the scaling lie in the intersection. Subspace test passed. □

Geometrically, two planes through the origin in $\mathbb{R}^3$ meet in a line through the origin (or coincide); a plane and a line meet in the origin or the whole line. Every such intersection is again a flat object through the origin – a subspace, as promised (Figure 11.7).

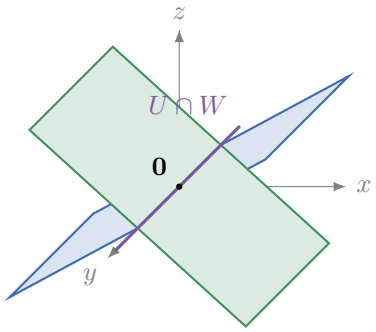


Figure 11.7: Two distinct planes through the origin in $\mathbb{R}^3$ (blue and green) intersect in a line through the origin (purple). The intersection of subspaces is always a subspace (Proposition 11.3); here it is forced to be one-dimensional.

Common trap

Unions usually fail. The *union* $U \cup W$ (vectors in either one) is almost never a subspace. Take the x-axis and the y-axis in $\mathbb{R}^2$: both are subspaces, but their union is not closed under addition, since $\begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}1\\1\end{bmatrix}$ lies on neither axis (Figure 11.8). The fix is to take not the union but the *sum* $U + W = \{\mathbf{u} + \mathbf{w} : \mathbf{u} \in U,\ \mathbf{w} \in W\}$, which *is* a subspace (and for the two axes equals all of $\mathbb{R}^2$). Intersection: yes. Sum: yes. Plain union: no.

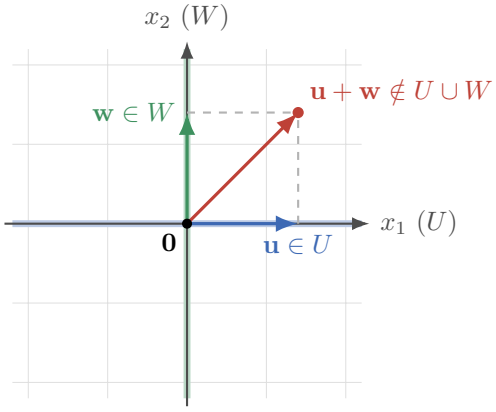


Figure 11.8: Why the union of two subspaces fails closure under addition. The x_1-axis U and the x_2-axis W in $\mathbb{R}^2$ are each genuine one-dimensional subspaces. But their union $U \cup W$ (the cross formed by the two axes) cannot hold their sum: adding $\mathbf{u} \in U$ and $\mathbf{w} \in W$ steps diagonally off both axes into the open quadrant. The union is not a subspace. The repair is the *sum* $U + W$, which fills all of $\mathbb{R}^2$.

11.9 Looking ahead

You now have the core vocabulary. A *vector space* is any world closed under adding and scaling; a *subspace* is a smaller such world living inside a bigger one, detected by the three-line test with the origin as your first checkpoint. *Spans are always subspaces*, which is how subspaces actually show up – as the column space, the nullspace, and every line and plane through the origin.

The questions that drive the next chapters are now askable in clean language. Given a subspace, how few vectors do we need to span it (a *basis*)? How many is that, exactly (its *dimension*)? When does a list of vectors waste effort by repeating directions (*dependence*)? And how do the four subspaces attached to a matrix $\mathbf{A}$ fit together? Every one of those is a question about closure, and you now know what closure means.

11.9.1 The bridge to Chapter 12: how matrices manufacture subspaces

Up to this point, our subspaces have been geometric lines and planes or algebraic collections of polynomials. But where do subspaces appear in practical computation? The answer is that **every matrix manufactures two subspaces automatically**.

Think of an $m \times n$ matrix $\mathbf{A}$ as a machine that takes an input vector $\mathbf{x} \in \mathbb{R}^n$ and produces an output vector $\mathbf{Ax} \in \mathbb{R}^m$. Ask the two most fundamental questions you can ask about any machine, and the definitions of this chapter hand you two subspaces for free:

1. **What outputs can come out of the machine?** Recall how matrix–vector multiplication acts on the columns $\mathbf{a}_1, \ldots, \mathbf{a}_n$ of $\mathbf{A}$:

$$\mathbf{Ax} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

   Every possible output is a linear combination of the columns. By Theorem 11.2, the span of any set of vectors is automatically a subspace. Therefore, all reachable outputs form a subspace of $\mathbb{R}^m$:

$$\operatorname{col}(\mathbf{A}) = \operatorname{span}\{\mathbf{a}_1, \ldots, \mathbf{a}_n\} \subseteq \mathbb{R}^m.$$

   This is the **column space** of $\mathbf{A}$. It gives a geometric answer to the question: *can we solve* $\mathbf{Ax} = \mathbf{b}$*?* The system has a solution if and only if the target vector $\mathbf{b}$ lies on the column space. If $\mathbf{b}$ sits outside that sheet, no input can ever reach it.

2. **What inputs are crushed to zero?** Look at the inputs $\mathbf{x} \in \mathbb{R}^n$ that satisfy $\mathbf{Ax} = \mathbf{0}$. This is a homogeneous system. As we saw in Example 11.6, homogeneous equations always pass the subspace test:

   - $\mathbf{A0} = \mathbf{0}$, so the origin is inside;
   - if $\mathbf{Au} = \mathbf{0}$ and $\mathbf{Av} = \mathbf{0}$, then $\mathbf{A}(\mathbf{u} + \mathbf{v}) = \mathbf{Au} + \mathbf{Av} = \mathbf{0} + \mathbf{0} = \mathbf{0}$;
   - if $\mathbf{Au} = \mathbf{0}$, then $\mathbf{A}(c\,\mathbf{u}) = c(\mathbf{Au}) = c\,\mathbf{0} = \mathbf{0}$.

   The collection of all inputs sent to zero forms a subspace of the input space $\mathbb{R}^n$:

$$\operatorname{null}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{0}\} \subseteq \mathbb{R}^n.$$

   This is the **nullspace** of $\mathbf{A}$. It gives a geometric answer to the question: *if a solution exists, is it unique?* If $\operatorname{null}(\mathbf{A})$ contains only the zero vector $\mathbf{0}$, the solution is unique. If $\operatorname{null}(\mathbf{A})$ contains an entire line or plane of directions, you can add any vector from the nullspace to a solution without changing the output: $\mathbf{A}(\mathbf{x}_{\text{particular}} + \mathbf{x}_{\text{null}}) = \mathbf{b} + \mathbf{0} = \mathbf{b}$.

Figure 11.9 brings these two spaces together into a single picture.

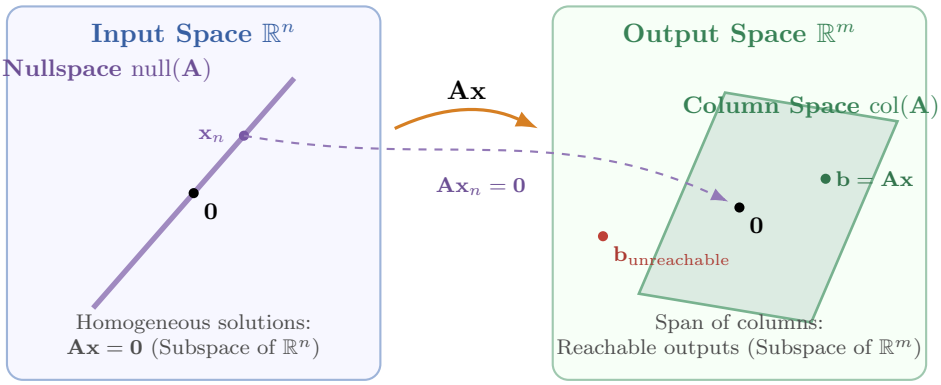


Figure 11.9: The two fundamental subspaces manufactured by any matrix $\mathbf{A}$. In the input space $\mathbb{R}^n$ (left), all solutions to $\mathbf{Ax} = \mathbf{0}$ form the *nullspace* null($\mathbf{A}$)—a subspace passing through the origin. Every vector on this line is squashed directly to $\mathbf{0} \in \mathbb{R}^m$. In the output space $\mathbb{R}^m$ (right), all linear combinations of the columns of $\mathbf{A}$ form the *column space* col($\mathbf{A}$)—a subspace containing all reachable outputs. The system $\mathbf{Ax} = \mathbf{b}$ is solvable precisely when $\mathbf{b}$ lands on this green sheet. In Chapter 12, elimination gives us the algorithm to compute both subspaces explicitly.

Here is the bridge to the next chapter: the abstract definitions of this chapter prove that col($\mathbf{A}$) and null($\mathbf{A}$) are authentic subspaces, but they do not yet give us an algorithm to find their size, their directions, or a minimal set of spanning vectors. In Chapter 12, we bring our elimination tools back to tame these two spaces, computing the exact basis of both col($\mathbf{A}$) and null($\mathbf{A}$) from the pivots and free columns of row-echelon form.

Connection to other math

The vector space framework frees us completely from coordinate axes. We no longer need separate mathematics for geometric arrows, polynomial curves, discrete audio signals, and solutions to differential equations: the three-condition subspace test ($0 \in W$, closure under addition, closure under scaling) provides an immediate test for whether any collection forms an autonomous linear universe.

Yet vector spaces in the abstract are purely qualitative arenas. They provide no tape measure and no protractor: concepts of length, distance, and perpendicularity require the inner products of Chapter 17. Nor do the abstract axioms alone give us an algorithm for solving systems or measuring the size of a subspace. In Chapter 12, we anchor these spaces back to concrete linear transformations, discovering that every matrix $\mathbf{A}$ naturally carves its world into a reachable column space col($\mathbf{A}$) and an invisible nullspace null($\mathbf{A}$).

Pause

Intuition recap – carry these six lines.

- A *vector space* is any world closed under *add* and *scale*; the eight bookkeeping axioms come free, closure is the real test.
- A *subspace* is a vector space living inside another one. Test it in three lines: contains $\mathbf{0}$, closed under $+$, closed under scaling – equivalently, closed under linear combinations.
- First reflex on any candidate: *is the origin in it?* If not, stop – not a subspace. "Homo-

geneous ($= 0$) means through the origin means subspace."

- A *span* is automatically a subspace (Theorem 11.2), in fact the smallest one containing the given vectors.
- *Intersection* of subspaces: yes. *Sum*: yes. Plain *union*: almost never (the sum escapes).
- Every matrix $\mathbf{A}$ manufactures two subspaces: the *column space* $\text{col}(\mathbf{A})$ (all reachable outputs in $\mathbb{R}^m$) and the *nullspace* $\text{null}(\mathbf{A})$ (all inputs crushed to $\mathbf{0}$ in $\mathbb{R}^n$). Chapter 12 uses elimination to compute them.

The subspace test is the cheapest check in the book—three closure questions—but the conceptual payoff is immense: prove it once for vector spaces, and it holds for arrows, matrices, polynomials, functions, and data alike.

Exercises

Exercise 11.1. For each subset of $\mathbb{R}^2$, decide whether it is a subspace. If not, name a specific condition that fails with a numerical counterexample. (a) $\{\begin{bmatrix} x \\ y \end{bmatrix} : y = 3x\}$; (b) $\{\begin{bmatrix} x \\ y \end{bmatrix} : y = 3x + 1\}$; (c) $\{\begin{bmatrix} x \\ y \end{bmatrix} : xy = 0\}$; (d) $\{\begin{bmatrix} x \\ y \end{bmatrix} : x = y = 0\}$; (e) $\{\begin{bmatrix} x \\ y \end{bmatrix} : x \geq 0\}$.

Exercise 11.2. Show that $W = \{\begin{bmatrix} x \\ y \\ z \end{bmatrix} : 2x - y + 3z = 0\}$ is a subspace of $\mathbb{R}^3$ by checking the three conditions, and find two specific vectors whose span is W. What is the normal vector to this plane?

Exercise 11.3. Explain carefully why the set of polynomials of degree *exactly* 2 is not a subspace of P_2. Give two failures: one about the zero vector, and one about closure under addition (find two degree-2 polynomials whose sum has degree less than 2).

Exercise 11.4. Inside $M_{2\times 2}$, consider the set of all matrices with trace zero, i.e. $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ with $a + d = 0$. Show it is a subspace. Then do the same for the symmetric matrices ($b = c$). For each, find a spanning set.

Exercise 11.5. In $\mathbb{R}^3$, let $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$, $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$, $\mathbf{v}_3 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$. Show that $\mathbf{v}_3 \in \text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$ and conclude that $\text{span}\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\} = \text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$. What kind of subspace of $\mathbb{R}^3$ is this (point, line, plane, or all)?

Exercise 11.6. Let U be the xy-plane and W the plane $x = z$ in $\mathbb{R}^3$. Describe $U \cap W$ explicitly (find a vector that spans it) and confirm geometrically that it is a line through the origin. Why does Proposition 11.3 guarantee in advance that the intersection is a subspace?

Exercise 11.7. Let V be the space of all functions $f : \mathbb{R} \to \mathbb{R}$. For each subset, decide whether it is a subspace and justify with the closure conditions: (a) all f with $f(0) = 0$; (b) all f with $f(0) = 1$; (c) all f with $f(1) = f(2)$; (d) all even functions ($f(-x) = f(x)$); (e) all f with $\int_0^1 f(x)\,dx = 0$.

Exercise 11.8. Prove the general fact behind the union trap: if U and W are subspaces of V and *neither* contains the other, then $U \cup W$ is not a subspace. (Hint: pick $\mathbf{u} \in U \setminus W$ and $\mathbf{w} \in W \setminus U$ and consider $\mathbf{u} + \mathbf{w}$. Show it cannot lie in U and cannot lie in W.)

Exercise 11.9. Using only the ten axioms, prove that the zero vector is unique: if $\mathbf{0}$ and $\mathbf{0}'$ both satisfy $\mathbf{v} + \mathbf{0} = \mathbf{v}$ and $\mathbf{v} + \mathbf{0}' = \mathbf{v}$ for all $\mathbf{v}$, then $\mathbf{0} = \mathbf{0}'$. (Hint: evaluate $\mathbf{0} + \mathbf{0}'$ two ways.) Then prove negatives are unique.

Exercise 11.10. Theorem 11.2 says $\operatorname{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ is the *smallest* subspace containing the $\mathbf{v}_i$. Use this to give a one-line argument that the intersection of *all* subspaces containing a set S equals $\operatorname{span}(S)$. (This is a slick alternative definition of span: "the smallest subspace containing S.")

Exercise 11.11. (Connection to calculus.) Show that the set of all differentiable functions y satisfying $y' = 3y$ is a subspace of the space of differentiable functions. Then find a nonzero function in it. (You may recall the solution is $y = Ce^{3t}$ – which says the subspace is the span of a single function. What is its dimension?)

Exercise 11.12. (Preview of Chapter 12: the nullspace.) Consider the matrix $\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \end{bmatrix}$.

(a) Check directly that $\mathbf{s}_1 = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$ and $\mathbf{s}_2 = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$ both satisfy $\mathbf{A}\mathbf{x} = \mathbf{0}$.

(b) Using the subspace test, prove that *any* linear combination $c_1\mathbf{s}_1 + c_2\mathbf{s}_2$ also satisfies $\mathbf{A}\mathbf{x} = \mathbf{0}$.

(c) Geometrically, what kind of subspace of $\mathbb{R}^3$ is $\operatorname{null}(\mathbf{A})$ (point, line, plane, or all of $\mathbb{R}^3$)?

Exercise 11.13. (Preview of Chapter 12: the column space and solvability.) For the same matrix $\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \end{bmatrix}$:

(a) Write out the column space $\operatorname{col}(\mathbf{A}) = \operatorname{span}\{\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3\} \subseteq \mathbb{R}^2$. Notice that $\mathbf{a}_2 = 2\mathbf{a}_1$ and $\mathbf{a}_3 = \mathbf{a}_1$. What kind of subspace of $\mathbb{R}^2$ is $\operatorname{col}(\mathbf{A})$?

(b) Which of the two targets $\mathbf{b}_1 = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$ and $\mathbf{b}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ can be solved as $\mathbf{A}\mathbf{x} = \mathbf{b}$? Explain why one is solvable and the other is impossible using the geometry of $\operatorname{col}(\mathbf{A})$.

Chapter 12

Column Space and Nullspace

In the last chapter we met vector spaces and their subspaces in the abstract: a subspace is a set of vectors that is closed under addition and scalar multiplication, a flat sheet through the origin that you can never fall off of. That definition is clean, but it can feel like it is floating in the air. Where do subspaces actually *come from*? Where will you meet them in practice?

The answer, and it is one of the central facts of the whole subject, is that **every matrix carries two subspaces around with it automatically**. A matrix $\mathbf{A}$ is a machine that takes input vectors $\mathbf{x}$ and produces output vectors $\mathbf{Ax}$. Ask the two most natural questions you can ask about any machine and you get two subspaces for free:

- **What can come out?** The set of all reachable outputs is the *column space* $\text{col}(\mathbf{A})$. It lives in the output space.
- **What gets crushed to zero?** The set of all inputs the machine sends to $\mathbf{0}$ is the *nullspace* $\text{null}(\mathbf{A})$. It lives in the input space.

These two spaces are the heart of this chapter, and arguably the heart of the first half of the book. The column space tells you which systems $\mathbf{Ax} = \mathbf{b}$ have a solution at all. The nullspace tells you, once you have one solution, how many others there are. Together they are the bookkeeping of solvability. And both of them, it turns out, fall directly out of elimination, the very algorithm you already know.

We build the pictures first.

12.1 The column space: what can come out

12.1.1 Reading a matrix–vector product as a combination of columns

Everything in this chapter rests on one way of looking at $\mathbf{Ax}$ that we have used before and will now lean on hard. When a matrix multiplies a vector, **the output is a linear combination of the columns of A**, and the entries of $\mathbf{x}$ are the coefficients.

Write $\mathbf{A}$ with its columns named $\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_n$:

$$\mathbf{A} = \begin{bmatrix} | & | & & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \\ | & | & & | \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Then

$$\mathbf{A}\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

You dial in the amount x_1 of the first column, x_2 of the second, and so on, and add up the arrows. Here it is with numbers.

Example 12.1. Take

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 0 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 4 \\ -1 \end{bmatrix}.$$

The mechanical row-by-row recipe gives

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1\cdot 4 + 2\cdot(-1) \\ 3\cdot 4 + 0\cdot(-1) \\ 1\cdot 4 + 1\cdot(-1) \end{bmatrix} = \begin{bmatrix} 2 \\ 12 \\ 3 \end{bmatrix}.$$

But the column picture says the same thing in a more revealing way:

$$\mathbf{A}\mathbf{x} = 4\begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix} + (-1)\begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 12 \\ 4 \end{bmatrix} + \begin{bmatrix} -2 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 12 \\ 3 \end{bmatrix}.$$

Same answer, but now you can see that the output is built out of the two columns. Vary $\mathbf{x}$ over all of $\mathbb{R}^2$ and the output ranges over every combination of those two columns.

Focus

$\mathbf{A}\mathbf{x}$ is a recipe for mixing the columns of $\mathbf{A}$. The vector $\mathbf{x}$ supplies the amounts; the columns supply the directions. *Hold onto this single sentence* and the rest of the chapter unfolds from it.

12.1.2 Definition and the picture

If $\mathbf{A}\mathbf{x}$ is always a combination of the columns, then the set of *all* outputs is exactly the set of all combinations of the columns, that is, their span.

Definition 12.1 (Column space)**.** Let $\mathbf{A}$ be an $m \times n$ matrix with columns $\mathbf{a}_1, \ldots, \mathbf{a}_n \in \mathbb{R}^m$. The **column space** of $\mathbf{A}$, written $\mathrm{col}(\mathbf{A})$ or $C(\mathbf{A})$, is the set of all linear combinations of its columns:

$$\mathrm{col}(\mathbf{A}) = \mathrm{span}\{\mathbf{a}_1, \ldots, \mathbf{a}_n\} = \{\, \mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n \,\} \subseteq \mathbb{R}^m.$$

Equivalently, $\mathrm{col}(\mathbf{A})$ is the set of all vectors $\mathbf{b}$ for which the system $\mathbf{A}\mathbf{x} = \mathbf{b}$ has at least one solution.

That last sentence is worth pausing on, because it converts a geometric object into a solvability test. Saying "$\mathbf{b}$ is in the column space" and saying "$\mathbf{A}\mathbf{x} = \mathbf{b}$ is solvable" are two phrasings of the identical fact. The column space is precisely the set of right-hand sides you can hit.

Intuition

Picture $\mathbf{A}$ as a machine with a panel of dials, one per column. Each dial direction is one column vector. The column space is *the set of all places the machine can reach by turning its dials*. If your target $\mathbf{b}$ is inside that reachable region, the system is solvable; if it sits outside, no setting of the dials will ever get you there.

Pause

Pause and think. If a matrix $\mathbf{A}$ is 5×3, in which Euclidean space does $\text{col}(\mathbf{A})$ live? In which space does $\text{null}(\mathbf{A})$ live?

Answer: Each column of $\mathbf{A}$ has 5 entries (the number of rows), so $\text{col}(\mathbf{A}) \subseteq \mathbb{R}^5$, the output space. The matrix multiplies input vectors $\mathbf{x}$ with 3 entries (the number of columns), so $\text{null}(\mathbf{A}) \subseteq \mathbb{R}^3$, the input space.

For a 3×2 matrix like the one in Example 12.1, the two columns are arrows in $\mathbb{R}^3$. As long as they do not point along the same line, all of their combinations fill out a *plane through the origin* in $\mathbb{R}^3$. The output space is three-dimensional, but the reachable part is only a two-dimensional sheet inside it. Figure 12.1 shows this.

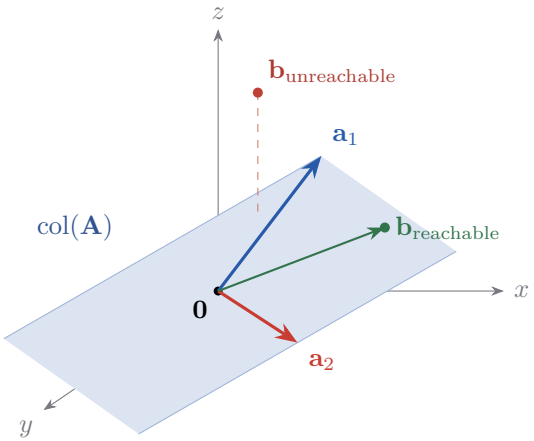


Figure 12.1: Two columns of a 3×2 matrix are arrows in $\mathbb{R}^3$. All their linear combinations fill a two-dimensional plane through the origin: the column space $\text{col}(\mathbf{A})$. Any target $\mathbf{b}_{\text{reachable}}$ on this sheet is reachable by setting the coefficients in $\mathbf{Ax} = \mathbf{b}$. Any target $\mathbf{b}_{\text{unreachable}}$ hovering off the sheet can never be reached: no linear combination of the columns can produce it.

12.1.3 Why the column space is a subspace

We claimed $\text{col}(\mathbf{A})$ is a subspace; we should not just assert it. The clean way is to use the description $\text{col}(\mathbf{A}) = \{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\}$ together with the linearity of the map $\mathbf{x} \mapsto \mathbf{Ax}$.

Proposition 12.1. *For any $m \times n$ matrix $\mathbf{A}$, the column space $\text{col}(\mathbf{A})$ is a subspace of $\mathbb{R}^m$.*

Proof. We check the three subspace conditions for $S = \{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\}$.

Contains zero. Take $\mathbf{x} = \mathbf{0}$. Then $\mathbf{A0} = \mathbf{0}$, so $\mathbf{0} \in S$.

Closed under addition. Suppose $\mathbf{u}, \mathbf{w} \in S$. Then $\mathbf{u} = \mathbf{A}\mathbf{x}$ and $\mathbf{w} = \mathbf{A}\mathbf{y}$ for some inputs $\mathbf{x}, \mathbf{y}$. By linearity,

$$\mathbf{u} + \mathbf{w} = \mathbf{A}\mathbf{x} + \mathbf{A}\mathbf{y} = \mathbf{A}(\mathbf{x} + \mathbf{y}),$$

which is again of the form $\mathbf{A}$(something), so $\mathbf{u} + \mathbf{w} \in S$.

Closed under scalar multiplication. If $\mathbf{u} = \mathbf{A}\mathbf{x} \in S$ and $c \in \mathbb{R}$, then $c\mathbf{u} = c(\mathbf{A}\mathbf{x}) = \mathbf{A}(c\mathbf{x}) \in S$.

All three hold, so $S = \text{col}(\mathbf{A})$ is a subspace. □

Notice how little work that was, and notice *why*. The two closure properties are exactly the two halves of linearity, $\mathbf{A}(\mathbf{x} + \mathbf{y}) = \mathbf{A}\mathbf{x} + \mathbf{A}\mathbf{y}$ and $\mathbf{A}(c\mathbf{x}) = c\,\mathbf{A}\mathbf{x}$, read backwards. A span is always a subspace, and the column space is a span; the proposition is really just that observation in matrix terms.

Why this matters

Notice that we did not *design* the column space to satisfy the subspace axioms. We simply asked "what outputs can come out of $\mathbf{A}$?" – and the linearity of $\mathbf{A}$ automatically forced that set of outputs to be a flat subspace through the origin.

12.1.4 The shape of the column space depends on the columns

Two columns in $\mathbb{R}^3$ *usually* span a plane, but not always. If the second column happens to be a multiple of the first, the two arrows lie on one line and their span is only that line. The column space can be smaller than you expect.

Example 12.2. Compare

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \quad \text{vs.} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 \\ 0 & 0 \\ 1 & 2 \end{bmatrix}.$$

In $\mathbf{A}$ the columns $\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}$ are not multiples of each other, so $\text{col}(\mathbf{A})$ is a plane in $\mathbb{R}^3$. In $\mathbf{B}$ the second column is exactly twice the first, $\begin{bmatrix} 2 \\ 0 \\ 2 \end{bmatrix} = 2\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$, so every combination $x_1\mathbf{b}_1 + x_2\mathbf{b}_2 = (x_1 + 2x_2)\mathbf{b}_1$ is just a scalar times $\mathbf{b}_1$. The column space of $\mathbf{B}$ is a single *line*.

The number of genuinely independent directions in the column space is its size or **dimension** (a concept we will make rigorous in Chapter 14), and it has a special name: the **rank**. For $\mathbf{A}$ in Example 12.2, the rank is 2 (a two-dimensional plane); for $\mathbf{B}$, the rank is 1 (a one-dimensional line). For now keep the intuitive slogan: *the column space is as big as the columns are independent.*

Common trap

"The column space lives in $\mathbb{R}^n$." No. Be careful which space you are in. For an $m \times n$ matrix the columns each have m entries, so $\text{col}(\mathbf{A}) \subseteq \mathbb{R}^m$, the *output* space. The nullspace, which we meet next, lives in $\mathbb{R}^n$, the input space. A 3×2 matrix has its column space in $\mathbb{R}^3$ and its

nullspace in $\mathbb{R}^2$. Keep the dimensions clear: m for the column space and n for the nullspace.

12.2 The nullspace: what gets crushed to zero

12.2.1 Definition and the picture

The column space asked about outputs. Now swing around to the input side and ask the opposite question: which inputs does the machine destroy? Which $\mathbf{x}$ satisfy $\mathbf{Ax} = \mathbf{0}$?

Definition 12.2 (Nullspace)**.** Let $\mathbf{A}$ be an $m \times n$ matrix. The **nullspace** of $\mathbf{A}$, written $\text{null}(\mathbf{A})$ or $N(\mathbf{A})$, is the set of all solutions of the homogeneous system $\mathbf{Ax} = \mathbf{0}$:

$$\text{null}(\mathbf{A}) = \{\, \mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{0} \,\} \subseteq \mathbb{R}^n.$$

The word *homogeneous* just means the right-hand side is zero. The nullspace is never empty, because $\mathbf{x} = \mathbf{0}$ is always a solution: $\mathbf{A0} = \mathbf{0}$. The real question is whether there are *other* solutions, nonzero inputs that the matrix flattens to the origin.

Intuition

Same machine, different question. The column space was "where can the dials take me?" The nullspace is "which dial settings leave me exactly where I started, at the origin?" If turning the dials by $\mathbf{x}$ and by $\mathbf{x}'$ both land you at zero, the matrix cannot tell those two settings apart at the output: it has *collapsed* a direction. The nullspace measures how much information the matrix throws away.

12.2.2 Why the nullspace is a subspace

Proposition 12.2. *For any $m \times n$ matrix $\mathbf{A}$, the nullspace $\text{null}(\mathbf{A})$ is a subspace of $\mathbb{R}^n$.*

Proof. Contains zero. $\mathbf{A0} = \mathbf{0}$, so $\mathbf{0} \in \text{null}(\mathbf{A})$.

Closed under addition. If $\mathbf{Ax} = \mathbf{0}$ and $\mathbf{Ay} = \mathbf{0}$, then

$$\mathbf{A}(\mathbf{x} + \mathbf{y}) = \mathbf{Ax} + \mathbf{Ay} = \mathbf{0} + \mathbf{0} = \mathbf{0},$$

so $\mathbf{x} + \mathbf{y} \in \text{null}(\mathbf{A})$.

Closed under scalar multiplication. If $\mathbf{Ax} = \mathbf{0}$ and $c \in \mathbb{R}$, then $\mathbf{A}(c\mathbf{x}) = c(\mathbf{Ax}) = c\mathbf{0} = \mathbf{0}$, so $c\mathbf{x} \in \text{null}(\mathbf{A})$.

Hence $\text{null}(\mathbf{A})$ is a subspace of $\mathbb{R}^n$. □

Geometrically: solutions of $\mathbf{Ax} = \mathbf{0}$ form a flat object through the origin. It might be just the origin (a single point with zero degrees of freedom), or a line (one direction of freedom), or a plane (two directions of freedom), or more. It always passes through $\mathbf{0}$, and it is always "flat" in the subspace sense, because adding two solutions or scaling a solution gives another solution.

Common trap

The *solution set of* $\mathbf{Ax} = \mathbf{b}$ *is not a subspace* unless $\mathbf{b} = \mathbf{0}$. If $\mathbf{b} \neq \mathbf{0}$ the solution set does not contain the origin (since $\mathbf{A0} = \mathbf{0} \neq \mathbf{b}$), so it fails the very first test. It is a flat sheet, yes, but a *shifted* one that misses the origin – what we will later call an affine set. Only the homogeneous equation gives a true subspace. We will see in the next chapter how the shifted solution set of $\mathbf{Ax} = \mathbf{b}$ is exactly the nullspace slid over by one particular solution.

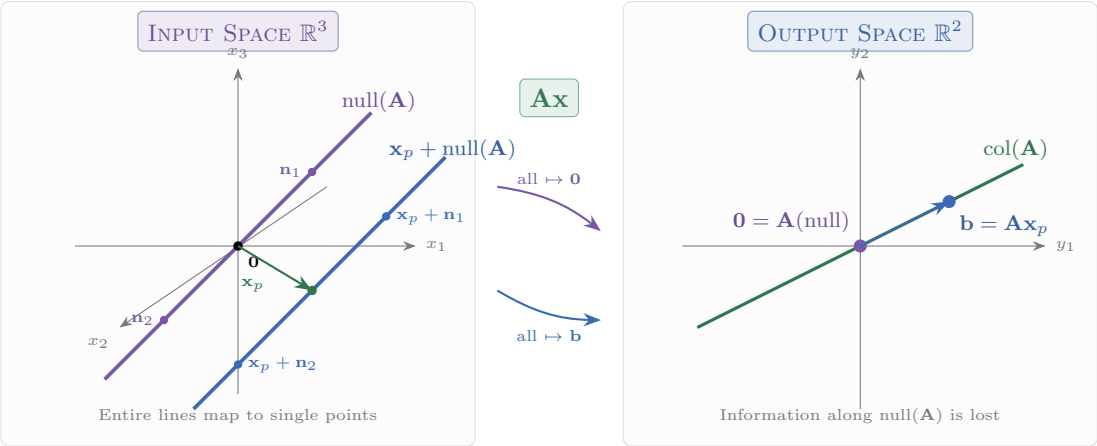


Figure 12.2: The geometry of nullspace collapse. The matrix $\mathbf{A}$ squashes the entire purple line null($\mathbf{A}$) through the origin down to the single point $\mathbf{0}$ in the output space. Consequently, every parallel shifted fiber $\mathbf{x}_p + \text{null}(\mathbf{A})$ (blue) collapses to the exact same output vector $\mathbf{b} = \mathbf{A}\mathbf{x}_p$. This explains why the complete solution to $\mathbf{Ax} = \mathbf{b}$ is an entire affine line: any two inputs that differ by a nullspace vector produce identical outputs.

12.3 Finding the nullspace by elimination

So far the nullspace is a definition. To actually *find* it, to write down all the vectors $\mathbf{x}$ with $\mathbf{Ax} = \mathbf{0}$, we use the primary computational tool from earlier chapters: row reduction. The key fact is that elimination does not disturb the nullspace at all.

12.3.1 Elimination preserves the nullspace

Lemma 12.1. *If* $\mathbf{R}$ *is obtained from* $\mathbf{A}$ *by elementary row operations (row swaps, scaling a row, adding a multiple of one row to another), then* $\mathbf{A}$ *and* $\mathbf{R}$ *have exactly the same nullspace.*

Why. Each elementary row operation can be undone by another elementary row operation, so the rows of $\mathbf{R}$ are combinations of the rows of $\mathbf{A}$ and vice versa. The equation $\mathbf{Ax} = \mathbf{0}$ says every row of $\mathbf{A}$ dotted with $\mathbf{x}$ gives 0. If that holds, then any combination of the rows also dots with $\mathbf{x}$ to give 0, so $\mathbf{Rx} = \mathbf{0}$ too; and because the operations are reversible, the converse holds as well. The solution set never changes. (This is special to row operations and the right-hand side $\mathbf{0}$: column operations would *not* preserve the nullspace.) □

This is the green light we needed. To find null($\mathbf{A}$) we may freely reduce $\mathbf{A}$ to its row echelon form, or all the way to the *reduced* row echelon form, and read off the answer there, where it is easy.

12.3.2 Pivots, free variables, and special solutions

Run elimination on $\mathbf{A}$. Some columns end up with pivots (the first nonzero entry in their row); these are the **pivot columns** and their variables are **pivot variables**. The columns *without* a pivot correspond to **free variables**. The name is exactly right: you are free to set those variables to anything you like, and the pivot variables are then forced. Each free variable gives you one "knob," and turning one knob at a time produces one **special solution**. The whole nullspace is the span of these special solutions.

Focus

The recipe.

1. Reduce $\mathbf{A}$ to reduced row echelon form $\mathbf{R}$.
2. Identify pivot columns and free columns. The number of free columns is $n -$ (number of pivots).
3. For each free variable, set it to 1 and all other free variables to 0, then solve for the pivot variables. That vector is a special solution.
4. null($\mathbf{A}$) is the span of all the special solutions.

We work through this on a concrete matrix step by step.

Example 12.3 (A full nullspace computation)**.** Find the nullspace of

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 2 & 4 \\ 1 & 2 & 3 & 6 \\ 2 & 4 & 5 & 10 \end{bmatrix}.$$

Step 1: eliminate. Subtract row 1 from row 2, and subtract 2× row 1 from row 3:

$$\begin{bmatrix} 1 & 2 & 2 & 4 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 1 & 2 \end{bmatrix}.$$

Now subtract the new row 2 from row 3:

$$\begin{bmatrix} 1 & 2 & 2 & 4 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Finally clear above the second pivot (subtract 2× row 2 from row 1) to reach reduced row echelon form:

$$\mathbf{R} = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Step 2: pivots and free variables. The pivots sit in columns 1 and 3. So x_1 and x_3 are pivot variables, while x_2 and x_4 are *free*. There are $4-2=2$ free variables, giving two independent degrees of freedom: a two-dimensional flat plane living inside $\mathbb{R}^4$.

Step 3: special solutions. The reduced rows say

$$x_1 + 2x_2 = 0 \quad \Longrightarrow \quad x_1 = -2x_2, \qquad x_3 + 2x_4 = 0 \quad \Longrightarrow \quad x_3 = -2x_4.$$

First special solution $\mathbf{s}_1$: set $x_2 = 1$, $x_4 = 0$. Then $x_1 = -2$, $x_3 = 0$:

$$\mathbf{s}_1 = \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}.$$

Second special solution $\mathbf{s}_2$: set $x_2 = 0$, $x_4 = 1$. Then $x_1 = 0$, $x_3 = -2$:

$$\mathbf{s}_2 = \begin{bmatrix} 0 \\ 0 \\ -2 \\ 1 \end{bmatrix}.$$

Step 4: the nullspace. Every solution is a combination of these:

$$\text{null}(\mathbf{A}) = \left\{ c_1 \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix} + c_2 \begin{bmatrix} 0 \\ 0 \\ -2 \\ 1 \end{bmatrix} : c_1, c_2 \in \mathbb{R} \right\}.$$

Check. Multiply $\mathbf{A}\mathbf{s}_1$: $1(-2)+2(1)+2(0)+4(0) = 0$, $1(-2)+2(1)+3(0)+6(0) = 0$, $2(-2)+4(1)+5(0)+10(0) = 0$. All zero, as required. (You can verify $\mathbf{A}\mathbf{s}_2 = \mathbf{0}$ directly.)

Compute it yourself

Find the nullspace of $\mathbf{A} = \begin{bmatrix} 1 & 3 & 1 \\ 2 & 6 & 4 \end{bmatrix}$ by hand. Reduce, spot the pivot in column 1 and another in column 3, notice x_2 is free, and produce the one special solution. You should get $\text{null}(\mathbf{A}) = \text{span}\{(-3, 1, 0)\}$, a line in $\mathbb{R}^3$. Then *verify* by computing $\mathbf{A}\,(-3, 1, 0)^{\mathsf{T}}$ and confirming it is $\mathbf{0}$.

12.3.3 Why the special solutions span the whole nullspace

We asserted that the special solutions capture *every* nullspace vector. Let us see why; the reason is clean, and it is the same reason every time.

After reduction, the free variables can be *anything*. Take any solution $\mathbf{x}$ of $\mathbf{R}\mathbf{x} = \mathbf{0}$ and read off its free-variable entries; call them $f_1, f_2, \ldots$ (in Example 12.3 these are x_2 and x_4). Now form the combination

$$f_1\mathbf{s}_1 + f_2\mathbf{s}_2 + \cdots,$$

where $\mathbf{s}_j$ is the special solution for the j-th free variable. By construction $\mathbf{s}_j$ has a 1 in free-slot j and a 0 in the other free slots, so this combination reproduces *exactly* the free entries $f_1, f_2, \ldots$ of $\mathbf{x}$. And since each $\mathbf{s}_j$ already satisfies the homogeneous equation, so does the combination, which means its pivot entries are forced to be the unique correct ones, matching $\mathbf{x}$. Two solutions with the same free entries must be the same solution. Therefore

$$\mathbf{x} = f_1\mathbf{s}_1 + f_2\mathbf{s}_2 + \cdots .$$

Every nullspace vector is a combination of the special solutions: they span the nullspace. Notice, too, that these special solutions carry no redundancy whatsoever: each special solution owns a designated free slot where it has a 1 while all the other special solutions have a 0. Because of those unique coordinates, no special solution can ever be made out of the others. A set of vectors that spans a subspace without a single wasted direction is what Chapter 14 will formally define as a **basis**. The number of vectors in that minimal set is the **dimension** of the subspace, written $\dim \operatorname{null}(\mathbf{A})$. Because every special solution comes from exactly one free variable, the dimension of the nullspace is simply the count of free variables: $n - r$.

Formal

Counting solutions of $\mathbf{Ax} = \mathbf{0}$. For an $m \times n$ matrix $\mathbf{A}$ with r pivots after elimination:

- the number of free variables (independent degrees of freedom) is $n - r$;
- $\dim \operatorname{null}(\mathbf{A}) = n - r$: the $n - r$ special solutions form a minimal, non-redundant spanning set (a *basis*, defined formally in Chapter 14);
- if $r = n$ (a pivot in every column, no free variables) then $\operatorname{null}(\mathbf{A}) = \{\mathbf{0}\}$: the only solution is the trivial one;
- if $r < n$ then there are infinitely many solutions, filling an $(n - r)$-dimensional subspace.

In particular, if a system has more unknowns than equations ($n > m \geq r$), there must be free variables, so $\mathbf{Ax} = \mathbf{0}$ always has nonzero solutions. *A wide homogeneous system can never have only the zero solution.*

Example 12.4 (Nullspace is only the origin)**.**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 3 \\ 1 & 1 \end{bmatrix} \xrightarrow{\text{eliminate}} \begin{bmatrix} 1 & 2 \\ 0 & 3 \\ 0 & 0 \end{bmatrix}.$$

Both columns have pivots ($r = 2 = n$), so there are no free variables. The only solution of $\mathbf{Ax} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$, and $\operatorname{null}(\mathbf{A}) = \{\mathbf{0}\}$. The columns are independent; the matrix crushes nothing.

12.3.4 A rank-one example tying both spaces together

Example 12.5 (One matrix, both spaces)**.**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

The second column is twice the first, so the column space is the line $\operatorname{span}\{(1, 2)\}$ in $\mathbb{R}^2$ – everything the matrix produces lies on that line. Now the nullspace. Eliminate: subtract $2\times$ row 1 from row

2 to get $\begin{bmatrix} 1 & 2 \\ 0 & 0 \end{bmatrix}$. Column 1 has the only pivot; x_2 is free. The pivot row says $x_1 + 2x_2 = 0$, i.e. $x_1 = -2x_2$. The special solution (set $x_2 = 1$) is $(-2, 1)$, so

$$\text{null}(\mathbf{A}) = \text{span}\left\{ \begin{bmatrix} -2 \\ 1 \end{bmatrix} \right\},$$

a line in $\mathbb{R}^2$. Geometrically the matrix squashes the whole plane onto a single line (the column space), and the direction $(-2, 1)$ is exactly the direction squashed to the origin (the nullspace). Figure 12.2 illustrates this collapse: one input dimension dies, and correspondingly the output is only one-dimensional.

The collapse is easiest to feel by watching what the matrix does to a whole patch of the plane. Feed the unit square into $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ and it does not come out as a tilted parallelogram, the way an invertible matrix would leave it. It comes out *flattened onto a single line*: the column space. Every point of the square is pushed onto $\text{span}\{(1, 2)\}$, and the entire nullspace direction $(-2, 1)$ is what gets ironed flat. Figure 12.3 shows the squash.

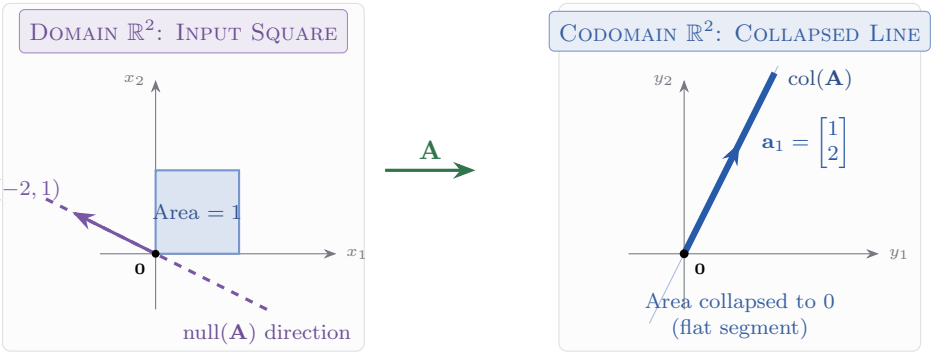


Figure 12.3: The rank-one matrix $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ flattens the two-dimensional unit square onto a one-dimensional segment of the line $\text{col}(\mathbf{A}) = \text{span}\{(1, 2)\}$. The entire nullspace direction $(-2, 1)$ is squashed to zero, destroying all 2D area.

Walking question

Take the matrix $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ out for a walk. Without writing anything down, answer: what is its column space (a line? which line?), and what direction does it crush to zero? Then ask the deeper one: the nullspace direction $(1, -1)$ and the column space direction $(1, 1)$ are perpendicular. Is that a coincidence, or is something structural going on between the rows of $\mathbf{A}$ and its nullspace? Hold that question; Chapter 15 on the four fundamental subspaces will reward it.

12.4 The two spaces as a solvability ledger

The column space and the nullspace together answer, completely, the two questions you can ask about $\mathbf{Ax} = \mathbf{b}$:

Question	**Answered by**
Does a solution *exist*?	Is $\mathbf{b} \in \text{col}(\mathbf{A})$?
If one exists, is it *unique*?	Is $\text{null}(\mathbf{A}) = \{\mathbf{0}\}$?

Existence is a question about the column space; uniqueness is a question about the nullspace. Here is the uniqueness half spelled out, because it is one of the most useful small facts in the subject.

Theorem 12.1 (Nullspace controls uniqueness)**.** *Suppose* $\mathbf{Ax} = \mathbf{b}$ *has at least one solution* $\mathbf{x}_p$*. Then the complete set of solutions is*

$$\{\, \mathbf{x}_p + \mathbf{n} : \mathbf{n} \in \text{null}(\mathbf{A}) \,\}.$$

In particular the solution is unique if and only if $\text{null}(\mathbf{A}) = \{\mathbf{0}\}$*.*

Proof. First, every such vector solves the system: $\mathbf{A}(\mathbf{x}_p + \mathbf{n}) = \mathbf{Ax}_p + \mathbf{An} = \mathbf{b} + \mathbf{0} = \mathbf{b}$.

Conversely, let $\mathbf{x}$ be *any* solution. Then

$$\mathbf{A}(\mathbf{x} - \mathbf{x}_p) = \mathbf{Ax} - \mathbf{Ax}_p = \mathbf{b} - \mathbf{b} = \mathbf{0},$$

so $\mathbf{n} := \mathbf{x} - \mathbf{x}_p$ lies in the nullspace, and $\mathbf{x} = \mathbf{x}_p + \mathbf{n}$ has the claimed form. Thus the solution set is exactly the nullspace shifted by $\mathbf{x}_p$. It collapses to the single point $\mathbf{x}_p$ precisely when the nullspace is $\{\mathbf{0}\}$. □

This is the theorem the next chapter is built on: the complete solution of $\mathbf{Ax} = \mathbf{b}$ is "one particular solution plus all of the nullspace." The nullspace is the freedom; the particular solution is one place to start. Two solutions can differ only by a nullspace vector, never by anything else. Figure 12.4 draws the picture: the solution set is the nullspace line, picked up off the origin and laid down through $\mathbf{x}_p$.

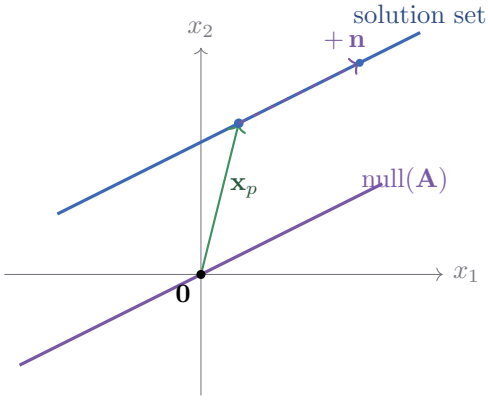


Figure 12.4: Theorem 12.1 in one picture. The homogeneous solutions null($\mathbf{A}$) form a line through the origin (purple). Adding one particular solution $\mathbf{x}_p$ slides that whole line off the origin to give the solution set of $\mathbf{Ax} = \mathbf{b}$ (blue): a parallel, *shifted* copy. Any two solutions differ by a nullspace vector $\mathbf{n}$. The set is a single point exactly when the nullspace is just $\{\mathbf{0}\}$.

Before/after Strang

If you have not yet read Strang: when you get to his lectures, notice that he introduces $C(\mathbf{A})$ and $N(\mathbf{A})$ together early on, emphasizing the column picture of $\mathbf{Ax}$ from the outset. The reason is exactly Theorem 12.1: these two subspaces are how he organizes *all* of $\mathbf{Ax} = \mathbf{b}$. If you have already finished Strang, you will recognize this chapter as the front half of his "four fundamental subspaces" framework. We have deliberately met the column space and nullspace alone first, so that when their two partners (the row space and the left nullspace) arrive, you already have half the picture and can feel the symmetry snap into place.

12.5 A modern application and a connection

Modern application

Nullspace and column space in machine learning. Stack your data as a matrix $\mathbf{X}$ with one feature per column. Fitting a linear model means finding weights $\mathbf{w}$ so that $\mathbf{Xw}$ approximates the target. Two things from this chapter bite immediately.

Redundant features make a nontrivial nullspace. If one feature is a linear combination of others – say "height in inches" alongside "height in centimeters," or a one-hot encoding that includes every category plus an all-ones intercept – then $\mathbf{X}$ has a nonzero nullspace. By Theorem 12.1 the best-fit weights are then *not unique*: you can add any nullspace vector to $\mathbf{w}$ and the predictions $\mathbf{Xw}$ do not change at all. This is the linear-algebra heart of *multicollinearity*, and it is why software either warns you, drops a column, or adds regularization to pick one answer out of the infinite flat sheet of equally good ones.

The column space is the set of achievable predictions. No matter how you choose $\mathbf{w}$, the prediction $\mathbf{Xw}$ lives in $\text{col}(\mathbf{X})$. If the true targets do not lie in that span, you simply cannot fit them exactly; the best you can do is land at the closest point of the column space. That closest point is the orthogonal projection onto $\text{col}(\mathbf{X})$, and it is exactly what least-squares regression computes – the subject of Chapter 19. So the column space is the exact menu of predictions your model is allowed to make.

Connection to other math

Kernel and image, everywhere in mathematics. The two words for our two spaces in the wider mathematical world are *kernel* (the nullspace) and *image* (the column space), and they appear for *any* linear map, not just matrices.

In calculus, differentiation $D : f \mapsto f'$ is linear. Its kernel is the set of functions with $f' = 0$ – the constants. That is why every indefinite integral comes with a "$+\,C$": the $+C$ is precisely the nullspace of D, and Theorem 12.1 is the reason the general antiderivative is "one particular antiderivative plus the kernel." The structure you learned for $\mathbf{Ax} = \mathbf{b}$ is *the same structure* as "solve $f' = g$."

The same pattern governs solutions of linear differential equations (general solution = particular solution + homogeneous solutions), and it is the seed of the rank–nullity theorem and, much deeper, the first isomorphism theorem in group and module theory. Kernel measures collapse; image measures reach. Once you see that one pair of ideas running underneath linear systems, calculus, and abstract algebra alike, the grand map of mathematics gets a good deal

smaller and friendlier.

12.6 Reading the column space off the echelon form (a caution)

Elimination gave us the nullspace cleanly. It is tempting to expect it to hand us the column space just as cleanly, but there is a subtlety worth a warning, because it trips up almost everyone once.

Common trap

The pivot columns tell you *which* original columns form a minimal spanning set (a basis) for $\mathrm{col}(\mathbf{A})$ **– but you must take them from A, not from R.** Row operations *change* the column space (they mix the entries within each column), so $\mathrm{col}(\mathbf{R}) \neq \mathrm{col}(\mathbf{A})$ in general. What *is* preserved is the linear relationships among columns: column j of $\mathbf{A}$ is a combination of earlier columns exactly when column j of $\mathbf{R}$ is. So you reduce to find *which* columns hold pivots, then go back to the *original* matrix $\mathbf{A}$ and take those columns as your minimal spanning set (what Chapter 14 will define as a *basis*).

Example 12.6. Return to Example 12.3, where

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 2 & 4 \\ 1 & 2 & 3 & 6 \\ 2 & 4 & 5 & 10 \end{bmatrix} \xrightarrow{\text{rref}} \mathbf{R} = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Pivots are in columns 1 and 3. So a minimal spanning set (basis) for $\mathrm{col}(\mathbf{A})$ is columns 1 and 3 *of* $\mathbf{A}$:

$$\mathrm{col}(\mathbf{A}) = \mathrm{span}\left\{ \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix} \right\}.$$

Columns 2 and 4 are redundant. Indeed $\mathbf{R}$ shows column 2 is $2\times$ column 1 and column 4 is $2\times$ column 3, and you can check the same multiples hold in $\mathbf{A}$: $\begin{bmatrix} 2 \\ 2 \\ 4 \end{bmatrix} = 2\begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$ and $\begin{bmatrix} 4 \\ 6 \\ 10 \end{bmatrix} = 2\begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$. The column space is a plane in $\mathbb{R}^3$, spanned by those two surviving columns. Notice the same number, $r = 2$, counts both the pivots *and* the dimension (number of independent directions) of the column space: each pivot column isolates a genuinely new direction, providing the geometric foundation of the rank theorem.

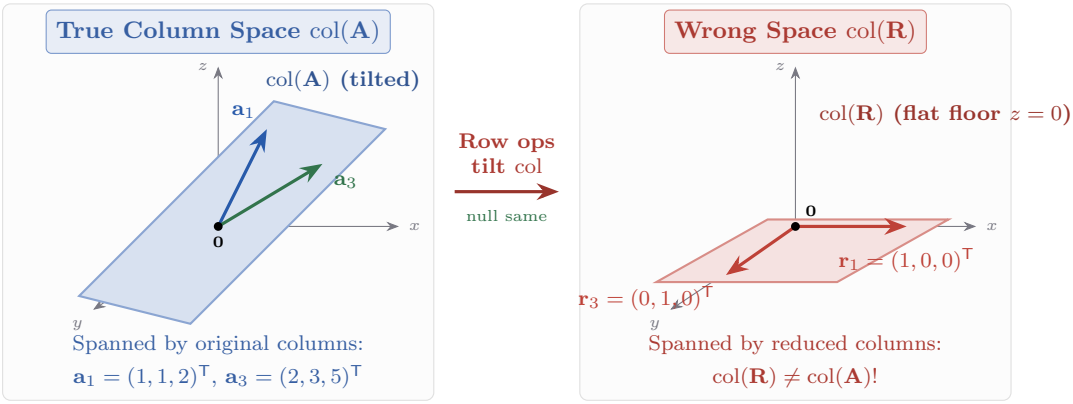


Figure 12.5: Why you must take pivot columns from the original matrix $\mathbf{A}$, not from its reduced form $\mathbf{R}$. Row operations preserve linear relationships between columns (so columns 1 and 3 remain the independent ones), but they mix row entries, tilting the actual subspace of outputs. In Example 12.6, $\text{col}(\mathbf{A})$ is a tilted plane in $\mathbb{R}^3$, whereas $\text{col}(\mathbf{R})$ has collapsed onto the horizontal coordinate floor $z = 0$.

Notice the bookkeeping: $r = 2$ pivots gave $n - r = 2$ free variables and a 2-dimensional nullspace, while the $r = 2$ pivot columns gave a 2-dimensional column space. Two pivots, doing double duty. The conservation law hiding here, $\dim \text{col}(\mathbf{A}) + \dim \text{null}(\mathbf{A}) = n$, is the celebrated *rank–nullity theorem*. We will prove it in Chapter 16 after Chapter 14 defines bases and dimension formally. For now, notice the efficiency: the same elimination, run once, completely reveals the geometry of *both* subspaces.

History and connection

Throughout the eighteenth and early nineteenth centuries, mathematicians viewed a system of linear equations purely as a mechanical list of scalar constraints, attempting to solve them by algebraic substitutions or by determinant formulas (such as Cramer's rule). But determinants were blind to structural degeneracy: whenever equations were redundant or inconsistent, the determinant collapsed to zero and gave indeterminate fractions like $\frac{0}{0}$, offering no explanation of how many solutions existed or what geometry lay behind them.

In the 1850s, the English mathematicians James Joseph Sylvester and Arthur Cayley introduced the term *matrix* and established the algebra of matrix multiplication. Yet algebraists initially resisted the idea that a matrix was anything more than a static bookkeeping grid of numbers. In the 1870s, the French mathematician Camille Jordan and the German mathematician Ferdinand Georg Frobenius took the decisive step: viewing a matrix as an active linear transformation that splits the universe into an input space and an output space. They recognized that every matrix decomposes space into two complementary structures: the *image* (what we now call the column space $\text{col}(\mathbf{A})$) and the *kernel* (what we call the nullspace $\text{null}(\mathbf{A})$). Frobenius showed that solvability is fundamentally a reachability question: $\mathbf{b}$ must lie in the subspace reachable by mixing the columns. Uniqueness, conversely, depends entirely on whether the kernel contains any nonzero directions crushed to zero.

A persistent controversy and point of confusion was the distinction between row operations and column operations. Because elementary row operations preserve all algebraic dependencies among columns, practitioners found they could read the pivot counts of both spaces directly off the echelon form $\mathbf{R}$. But early workers repeatedly fell into the trap of assuming that the

column space of $\mathbf{A}$ was spanned by the simplified columns of $\mathbf{R}$. It required careful geometric axiomatization to cement the distinction: row operations preserve the nullspace because they leave the solution set of $\mathbf{Ax} = \mathbf{0}$ completely unchanged, but they mix row entries, tilting the column space into an entirely different orientation in space.
In exact paper-and-pencil algebra, Gaussian elimination resolves the column space and nullspace completely. In modern scientific computing and machine learning, however, matrices routinely contain millions of rows and columns measured with physical noise. In floating-point arithmetic, tiny rounding errors of size 10^{-16} can cause a computer to confuse a near-zero direction with an exact nullspace, or miss a rank deficiency entirely. Developing fast, numerically stable, randomized algorithms to reliably detect and sketch the column space and nullspace of massive, streaming datasets remains an active and vital frontier of research today.

12.7 Summary

Focus

What to carry out of this chapter.

- $\mathbf{Ax}$ is a linear combination of the columns of $\mathbf{A}$, with $\mathbf{x}$ as the coefficients. Everything follows from this.
- $\text{col}(\mathbf{A}) \subseteq \mathbb{R}^m$ is the span of the columns: all reachable outputs, equivalently all $\mathbf{b}$ making $\mathbf{Ax} = \mathbf{b}$ solvable.
- $\text{null}(\mathbf{A}) \subseteq \mathbb{R}^n$ is the set of all $\mathbf{x}$ with $\mathbf{Ax} = \mathbf{0}$: the inputs crushed to zero.
- Both are subspaces, forced by the linearity of $\mathbf{A}$.
- Find the nullspace by elimination: reduce, find free variables, build one special solution per free variable, and span them. The number of independent directions of freedom is $\dim \text{null}(\mathbf{A}) = n - r$.
- Find a minimal spanning set (basis) for the column space by taking the *pivot columns of the original* $\mathbf{A}$. The number of independent directions is $\dim \text{col}(\mathbf{A}) = r$.
- Column space governs *existence*; nullspace governs *uniqueness*. The complete solution of $\mathbf{Ax} = \mathbf{b}$ is one particular solution plus all of the nullspace.

Connection to other math

The column space and nullspace give us two clean geometric answers to any linear system $\mathbf{Ax} = \mathbf{b}$. Existence is completely answered by whether $\mathbf{b} \in \text{col}(\mathbf{A})$; uniqueness is completely answered by whether $\text{null}(\mathbf{A}) = \{\mathbf{0}\}$. One pass of Gaussian elimination produces both subspaces simultaneously.
Yet knowing that $\mathbf{b}$ lies in $\text{col}(\mathbf{A})$ guarantees only that a solution exists; it does not hand us a formula for that solution. And what happens when $\mathbf{b} \notin \text{col}(\mathbf{A})$? In real-world data and physical measurements, noise almost always pushes $\mathbf{b}$ outside the column space, rendering exact solutions impossible. In Chapter 13, we assemble these two spaces into the complete solution $\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$. Later, in Chapters 18 and 19, we learn how to overcome unsolvable

systems by projecting $\mathbf{b}$ orthogonally onto $\mathrm{col}(\mathbf{A})$ to obtain the best possible least-squares fit.

12.8 Exercises

Exercise 12.1 (Warmup: combination of columns)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 1 & 3 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$, compute $\mathbf{Ax}$ two ways: by the row recipe, and by writing it as $3(\text{col } 1) + (-2)(\text{col } 2)$. Confirm the answers agree, and state which space the answer lives in, $\mathbb{R}^2$ or $\mathbb{R}^3$.

Exercise 12.2 (Is $\mathbf{b}$ reachable?)**.** Let $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$. Determine whether $\mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$ lies in $\mathrm{col}(\mathbf{A})$ by trying to solve $\mathbf{Ax} = \mathbf{b}$. If it is unreachable, give one equation that every vector in $\mathrm{col}(\mathbf{A})$ must satisfy (hint: the three entries of any column combination obey a linear relation).

Exercise 12.3 (Nullspace by elimination)**.** Find the nullspace of $\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 & 1 \\ 2 & 4 & 3 & 4 \end{bmatrix}$. Identify the pivot and free variables, write down each special solution, and state the dimension (number of independent directions) $\dim \mathrm{null}(\mathbf{A})$. Verify one special solution by direct multiplication.

Exercise 12.4 (Trivial nullspace)**.** For which value(s) of t does $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & t \end{bmatrix}$ have nullspace equal to $\{\mathbf{0}\}$? For the value of t that fails, describe the nullspace as a line and give its direction.

Exercise 12.5 (Minimal spanning set for the column space)**.** Let $\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 7 \\ 1 & 2 & 4 \end{bmatrix}$. Reduce to find the pivot columns, then give a minimal spanning set (what Chapter 14 will define as a basis) for $\mathrm{col}(\mathbf{A})$ taken from the columns of $\mathbf{A}$. How many independent directions span $\mathrm{col}(\mathbf{A})$ and $\mathrm{null}(\mathbf{A})$? Check that their counts add to 3.

Exercise 12.6 (Particular plus nullspace)**.** You are told that $\mathbf{x}_p = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$ solves $\mathbf{Ax} = \mathbf{b}$ for some matrix $\mathbf{A}$, and that $\mathrm{null}(\mathbf{A}) = \mathrm{span}\{(1, -1, 2)\}$. Write down the complete solution set of $\mathbf{Ax} = \mathbf{b}$. Is the solution unique? Give two distinct solutions explicitly.

Exercise 12.7 (More unknowns than equations)**.** Explain, in two or three sentences and citing the pivot count, why a homogeneous system of 3 equations in 5 unknowns must always have a nonzero solution. Construct one explicit 3×5 example and exhibit a nonzero nullspace vector.

Exercise 12.8 (Subspace, true or false)**.** For each set, decide whether it is a subspace and justify in one line. (a) $\{\mathbf{x} \in \mathbb{R}^3 : \mathbf{Ax} = \mathbf{0}\}$ for a fixed $\mathbf{A}$. (b) $\{\mathbf{x} \in \mathbb{R}^3 : \mathbf{Ax} = \mathbf{b}\}$ for a fixed $\mathbf{b} \neq \mathbf{0}$. (c) $\mathrm{col}(\mathbf{A})$ for a 4×2 matrix, viewed as a subset of $\mathbb{R}^4$. (d) The set of all $\mathbf{b}$ for which $\mathbf{Ax} = \mathbf{b}$ has *no* solution.

Exercise 12.9 (Geometry of the collapse)**.** Let $\mathbf{A} = \begin{bmatrix} 2 & 6 \\ 1 & 3 \end{bmatrix}$. Draw the unit square's image under $\mathbf{A}$ and explain why it is a segment, not a parallelogram. Identify the column space (the line the square collapses onto) and the nullspace (the direction collapsed to the origin), and sketch both in the plane.

Exercise 12.10 (Proof: nullspace under stacking)**.** Suppose $\mathbf{B}$ is the matrix obtained by stacking $\mathbf{A}$ on top of a copy of itself, $\mathbf{B} = \begin{bmatrix} \mathbf{A} \\ \mathbf{A} \end{bmatrix}$. Prove that $\text{null}(\mathbf{B}) = \text{null}(\mathbf{A})$. Then explain in words why this matches the intuition that duplicating equations adds no new information.

Exercise 12.11 (Challenge: nullspace of a product)**.** Let $\mathbf{A}$ be $m \times n$ and $\mathbf{B}$ be $n \times p$. Prove that $\text{null}(\mathbf{B}) \subseteq \text{null}(\mathbf{AB})$. Give a 2×2 example where the containment is *strict* (the product kills a vector that $\mathbf{B}$ does not), and explain geometrically how $\mathbf{A}$ can collapse extra directions that $\mathbf{B}$ left alive.

Chapter 13

The Complete Solution to Ax = b

So far you have learned to ask two separate questions about a matrix $\mathbf{A}$. The column space told you which right-hand sides $\mathbf{b}$ are reachable: $\mathbf{Ax} = \mathbf{b}$ has *some* solution exactly when $\mathbf{b}$ lies in $\text{col}(\mathbf{A})$. The nullspace told you about the slack: $\mathbf{Ax} = \mathbf{0}$ has nontrivial solutions exactly when the columns are dependent. In this chapter we glue those two ideas together and answer the question that systems of equations were invented to answer: *given a matrix* $\mathbf{A}$ *and a vector* $\mathbf{b}$*, describe every* $\mathbf{x}$ *with* $\mathbf{Ax} = \mathbf{b}$ *– all of them, no more and no fewer.*

That full description is called the *complete solution*, and it has a shape you can draw. It is always

$$\mathbf{x} = \underbrace{\mathbf{x}_p}_{\text{one particular solution}} + \underbrace{\mathbf{x}_n}_{\text{anything in the nullspace}}.$$

One fixed particular solution, plus the entire nullspace, slid over to sit on top of it. The rest of the chapter unpacks that one line: why it is true, how to compute each piece, and what the geometry looks like in every case (no solution, one solution, infinitely many).

13.1 One picture to hold onto

Before any algebra, here is the mental image. The map $\mathbf{x} \mapsto \mathbf{Ax}$ takes the input space and folds it down onto the column space. Solving $\mathbf{Ax} = \mathbf{b}$ means: find every input that lands exactly on the target point $\mathbf{b}$.

Intuition

Think of $\mathbf{A}$ as a machine that flattens. If two different inputs $\mathbf{x}_1$ and $\mathbf{x}_2$ both produce $\mathbf{b}$, then their difference satisfies

$$\mathbf{A}(\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{Ax}_1 - \mathbf{Ax}_2 = \mathbf{b} - \mathbf{b} = \mathbf{0}.$$

So the difference lives in the nullspace. Two solutions can differ *only* by a nullspace vector. Turn that around: take one solution $\mathbf{x}_p$ and add any nullspace vector $\mathbf{x}_n$; then $\mathbf{A}(\mathbf{x}_p + \mathbf{x}_n) = \mathbf{b} + \mathbf{0} = \mathbf{b}$, so you get another solution. **The solution set is the nullspace, picked up and translated to pass through $\mathbf{x}_p$.**

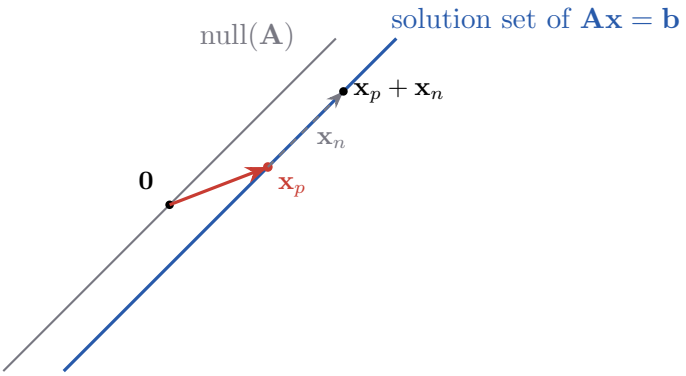


Figure 13.1: The complete solution set (blue) is the nullspace (gray, through the origin) slid over so it passes through one particular solution $\mathbf{x}_p$. Adding any nullspace vector $\mathbf{x}_n$ to $\mathbf{x}_p$ keeps you on the blue line.

A set like the blue line in Figure 13.1 – a subspace shifted off the origin – is called an *affine subspace* (in abstract algebra, you will also hear this called a *coset*: a linear subspace slid sideways by a fixed vector). It is parallel to the nullspace but generally does *not* pass through $\mathbf{0}$. The only time it passes through the origin is when $\mathbf{b} = \mathbf{0}$, because then $\mathbf{x}_p = \mathbf{0}$ works and the solution set *is* the nullspace.

Focus

Two questions, two subspaces, one formula.

- **Existence** (does a solution exist?) is a question about the *column space*: solvable $\iff$ $\mathbf{b} \in \text{col}(\mathbf{A})$.
- **Uniqueness** (is it the only one?) is a question about the *nullspace*: unique $\iff$ $\text{null}(\mathbf{A}) = \{\mathbf{0}\}$.
- When both succeed for a given $\mathbf{b}$, the answer set is $\mathbf{x}_p + \text{null}(\mathbf{A})$.

13.2 Why particular plus nullspace is everything

Now turn the picture into a theorem. The argument is short, and it is the same one that reappears for differential equations, recurrences, and every other linear problem you will meet.

Theorem 13.1 (Structure of the solution set)**.** *Suppose* $\mathbf{A}\mathbf{x}_p = \mathbf{b}$ *for some particular vector* $\mathbf{x}_p$ *(that is,* $\mathbf{b}$ *is reachable). Then the complete set of solutions to* $\mathbf{A}\mathbf{x} = \mathbf{b}$ *is*

$$\{\mathbf{x}_p + \mathbf{x}_n \ : \ \mathbf{x}_n \in \text{null}(\mathbf{A})\}.$$

If instead no particular solution exists, the solution set is empty.

Proof. We prove the two halves of set equality.

Every $\mathbf{x}_p + \mathbf{x}_n$ *is a solution.* If $\mathbf{A}\mathbf{x}_n = \mathbf{0}$, then by linearity

$$\mathbf{A}(\mathbf{x}_p + \mathbf{x}_n) = \mathbf{A}\mathbf{x}_p + \mathbf{A}\mathbf{x}_n = \mathbf{b} + \mathbf{0} = \mathbf{b}.$$

Every solution has this form. Suppose $\mathbf{Ax} = \mathbf{b}$. Set $\mathbf{x}_n := \mathbf{x} - \mathbf{x}_p$. Then

$$\mathbf{Ax}_n = \mathbf{Ax} - \mathbf{Ax}_p = \mathbf{b} - \mathbf{b} = \mathbf{0},$$

so $\mathbf{x}_n \in \text{null}(\mathbf{A})$, and $\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$ is of the claimed form. The two inclusions together give equality. □

Notice what the proof did *not* need: it never used the size of $\mathbf{A}$, never assumed a square matrix, never assumed invertibility. It used only linearity. That is why the formula is universal.

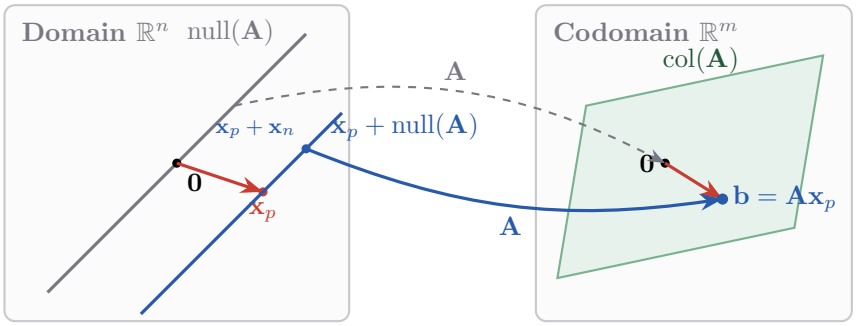


Figure 13.2: The geometric machine of $\mathbf{Ax} = \mathbf{b}$. In the domain $\mathbb{R}^n$ (left), the entire nullspace null($\mathbf{A}$) collapses to $\mathbf{0}$ in the codomain $\mathbb{R}^m$ (right). Its parallel shadow, the affine flat $\mathbf{x}_p + \text{null}(\mathbf{A})$, collapses entirely onto the single target point $\mathbf{b} \in \text{col}(\mathbf{A})$. Every point on the blue line is an equally valid solution to $\mathbf{Ax} = \mathbf{b}$.

Pause

Pause and think: what operations keep you on the solution set? If $\mathbf{x}_1$ and $\mathbf{x}_2$ are both solutions to $\mathbf{Ax} = \mathbf{b}$, is their midpoint $\frac{1}{2}(\mathbf{x}_1 + \mathbf{x}_2)$ also a solution? What about $\mathbf{x}_1 + \mathbf{x}_2$? By linearity,

$$\mathbf{A}\left(\tfrac{1}{2}\mathbf{x}_1 + \tfrac{1}{2}\mathbf{x}_2\right) = \tfrac{1}{2}\mathbf{Ax}_1 + \tfrac{1}{2}\mathbf{Ax}_2 = \tfrac{1}{2}\mathbf{b} + \tfrac{1}{2}\mathbf{b} = \mathbf{b}.$$

The midpoint is always a solution. More generally, any weighted average whose coefficients sum to 1 (an *affine combination*) stays on the solution flat. But the sum $\mathbf{x}_1 + \mathbf{x}_2$ yields $\mathbf{A}(\mathbf{x}_1 + \mathbf{x}_2) = 2\mathbf{b}$, falling completely off the target whenever $\mathbf{b} \neq \mathbf{0}$.

Common trap

The complete solution is $\mathbf{x}_p + \text{null}(\mathbf{A})$, **not** null($\mathbf{A}$) and **not** "the span of $\mathbf{x}_p$ and the nullspace." You do not scale $\mathbf{x}_p$. You add it once, as a fixed offset, and let only the nullspace part vary. Writing $c\mathbf{x}_p$ + (nullspace) is wrong: $2\mathbf{x}_p$ usually solves $\mathbf{Ax} = 2\mathbf{b}$, not $\mathbf{Ax} = \mathbf{b}$.

Before/after Strang

Strang states this as "*complete solution = particular + special.*" His "particular solution" is our $\mathbf{x}_p$, gotten by setting all the free variables to zero. His "special solutions" $\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_k$ form what we will formally define in Chapter 14 as a *basis* for the nullspace: a minimal set of independent building-block vectors whose linear combinations generate the entire nullspace without any redundancy. So $\mathbf{x}_n = c_1\mathbf{s}_1 + \cdots + c_k\mathbf{s}_k$, and the complete solution is $\mathbf{x}_p + c_1\mathbf{s}_1 + \cdots + c_k\mathbf{s}_k$. Reading this chapter *before* Strang, you now know *why* that recipe produces everything;

reading it *after*, you can see it is Theorem 13.1 with the nullspace written in a basis.

13.3 Existence: is b in the column space?

A solution exists if and only if $\mathbf{b} = \mathbf{Ax}$ for some $\mathbf{x}$ – and $\mathbf{Ax}$ is, by definition, a linear combination of the columns of $\mathbf{A}$. So the reachable right-hand sides are exactly the column space. There is nothing more to it conceptually; the work is in *detecting* membership, which elimination does for you.

When you row-reduce the augmented matrix $[\,\mathbf{A} \mid \mathbf{b}\,]$, watch the rows that become all zeros on the $\mathbf{A}$ side. If such a zero row has a nonzero entry on the $\mathbf{b}$ side, you have produced the equation $0 = (\text{nonzero})$, which is a contradiction: $\mathbf{b}$ was *not* in the column space, and there is no solution. If every all-zero row of $\mathbf{A}$ pairs with a zero in $\mathbf{b}$, the system is consistent.

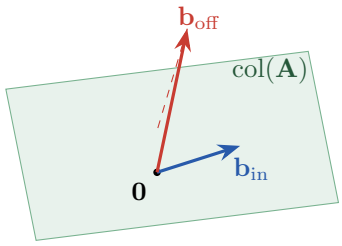


Figure 13.3: Existence is a hit-or-miss question. If $\mathbf{b}$ lies in the plane col($\mathbf{A}$) (blue), the system is solvable. If $\mathbf{b}$ sticks up out of the plane (red), no input can reach it and $\mathbf{Ax} = \mathbf{b}$ has no solution.

Example 13.1 (Spotting an inconsistent system)**.** Take

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 3 \\ 3 \end{bmatrix}.$$

Every column of $\mathbf{A}$ is a multiple of $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, so the column space is the single line through $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$. Is $\mathbf{b}$ on that line? It would need $\mathbf{b} = t \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, forcing $t = 1$ from the first entry but $t = \frac{3}{2}$ from the second. No single t works, so $\mathbf{b} \notin$ col($\mathbf{A}$) and there is *no* solution. Elimination confirms it: subtract 2×row 1 from row 2 and 3×row 1 from row 3 in $[\,\mathbf{A} \mid \mathbf{b}\,]$:

$$\left[\begin{array}{cc|c} 1 & 2 & 1 \\ 2 & 4 & 3 \\ 3 & 6 & 3 \end{array}\right] \longrightarrow \left[\begin{array}{cc|c} 1 & 2 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{array}\right].$$

Row 2 now reads $0x_1 + 0x_2 = 1$, i.e. $0 = 1$. Contradiction; inconsistent.

Example 13.2 (Same $\mathbf{A}$, a reachable $\mathbf{b}$)**.** Keep the same $\mathbf{A}$ but use $\mathbf{b} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, which *is* on

the column line. Reducing,

$$\left[\begin{array}{cc|c} 1 & 2 & 2 \\ 2 & 4 & 4 \\ 3 & 6 & 6 \end{array}\right] \longrightarrow \left[\begin{array}{cc|c} 1 & 2 & 2 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{array}\right].$$

Now the zero rows of $\mathbf{A}$ pair with zeros in $\mathbf{b}$: consistent. The one surviving equation is $x_1 + 2x_2 = 2$, which has infinitely many solutions – a whole line of them. We will write that line out in the next section.

Walking question

On your walk, fix the 3×2 matrix $\mathbf{A}$ from the examples in your head – its column space is one line in $\mathbb{R}^3$. Now imagine sweeping $\mathbf{b}$ all over $\mathbb{R}^3$. For how many $\mathbf{b}$ is the system solvable: none, a line's worth, a plane's worth, or all of $\mathbb{R}^3$? And of the solvable ones, how many have a *unique* solution? (Hint: the nullspace of this $\mathbf{A}$ is not just the origin.)

13.4 Uniqueness: is the nullspace trivial?

Existence was about $\mathbf{b}$ and the columns reaching far enough. Uniqueness is about the columns being non-redundant. By Theorem 13.1, once a solution exists, the number of solutions equals the number of nullspace vectors:

$$\text{null}(\mathbf{A}) = \{\mathbf{0}\} \implies \text{at most one solution}; \qquad \text{null}(\mathbf{A}) \neq \{\mathbf{0}\} \implies \text{either none or infinitely many}.$$

There is never a system with exactly two or exactly seventeen solutions. The solution set is an affine copy of a subspace, and a nonzero subspace already contains infinitely many vectors (you can scale any nonzero element). So the count is always 0, 1, or ∞.

Why this matters

This trichotomy – zero, one, or infinitely many – is one of the things linearity buys you. Nonlinear equations are wild: $x^2 = c$ can have 0, 1, or 2 roots; $\sin x = c$ can have infinitely many *isolated* roots. Linear systems are tame because straightness is preserved: if two solutions exist, the entire line through them solves the system too. The geometry of the answer set is always flat.

Common trap

"Square matrix" does not mean "unique solution." A 3×3 matrix can have a nontrivial nullspace (when its columns are dependent), giving zero or infinitely many solutions. Uniqueness is controlled by the nullspace, which is controlled by *independence of the columns*, not by the matrix being square. We will see all four combinations below.

13.5 Rank decides the shape

Everything – existence, uniqueness, and the geometric shape of the solution set – is governed by one central number and the matrix dimensions m and n.

Let $\mathbf{A}$ be $m \times n$ (m equations in n unknowns). Back in Chapter 12, we met the *rank* of $\mathbf{A}$, denoted r: the count of pivots that survive elimination. Think of r as counting the independent directions the matrix actually preserves. We also used the geometric idea of *dimension*—the number of independent directions, or degrees of freedom, needed to navigate a subspace. (Both concepts receive their rigorous foundation in Chapter 14, and the full Rank Theorem is proved in Chapter 16.)

Elimination splits the n input columns into two complementary camps:

- r pivot columns: these correspond to the non-redundant directions that survive into the output. They span the reachable target space, giving $\dim \operatorname{col}(\mathbf{A}) = r$.

- $n - r$ free columns: each free variable provides one independent dial of slack that gets crushed to $\mathbf{0}$. These $n - r$ independent directions generate the nullspace, giving $\dim \operatorname{null}(\mathbf{A}) = n - r$.

Together, $r + (n - r) = n$. Every column is either a pivot column or a free column: the input dimensions split cleanly between what reaches the output and what gets crushed to zero.

Combining these with Theorem 13.1 gives the master statement of the chapter.

Formal

The shape of the solution set. For $\mathbf{A} \in \mathbb{R}^{m \times n}$ of rank r and a *consistent* right-hand side $\mathbf{b}$, the solution set is an affine subspace of dimension $n - r$:

$$\mathbf{x}_p + \operatorname{null}(\mathbf{A}), \qquad \dim = n - r.$$

In particular:

- solvable for *every* $\mathbf{b} \in \mathbb{R}^m \iff r = m$ (the columns span $\mathbb{R}^m$; *full row rank*);
- solution unique whenever it exists $\iff r = n$ (the columns are independent; *full column rank*).

Read the two bullets as "existence for all $\mathbf{b}$ is about rows" and "uniqueness is about columns." The number r can be at most $\min(m, n)$, so a matrix can be full row rank, full column rank, both (then it is square and invertible), or neither.

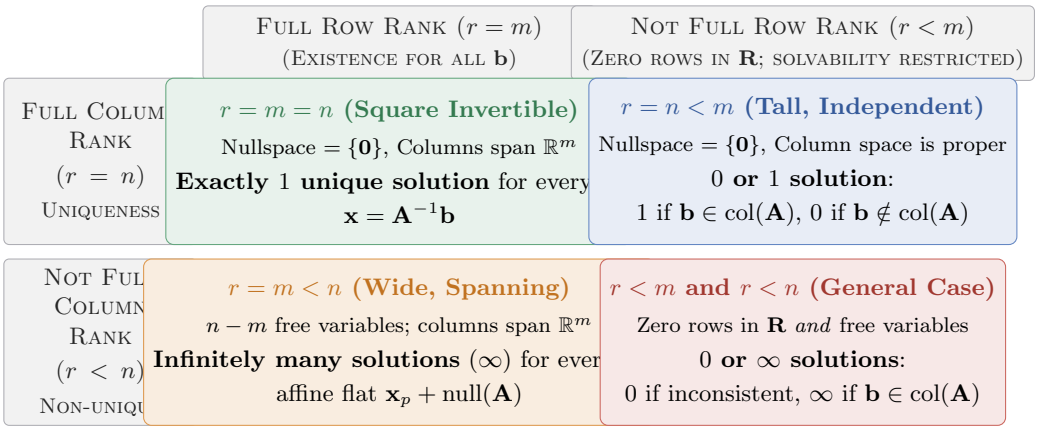


Figure 13.4: The four cases of the linear system $\mathbf{Ax} = \mathbf{b}$, categorized by row rank $r = m$ (which guarantees that columns span $\mathbb{R}^m$, ensuring existence) and column rank $r = n$ (which guarantees that null($\mathbf{A}$) = $\{\mathbf{0}\}$, ensuring uniqueness).

13.5.1 The four cases, one at a time

Case 1: $r = m = n$ (square and invertible). Full row rank gives existence for every $\mathbf{b}$; full column rank gives uniqueness. Together: *exactly one* solution for every $\mathbf{b}$, namely $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. The nullspace is $\{\mathbf{0}\}$, so $\mathbf{x}_p$ is the only solution. These are square invertible systems.

Case 2: $r = n < m$ (tall, full column rank). More equations than unknowns. The columns are independent, so the nullspace is trivial and any solution is unique – but the columns may not reach every $\mathbf{b}$, so existence can fail. Outcome: 0 *or* 1 solution. Either $\mathbf{b}$ lies in the n-dimensional column space (one solution) or it does not (none). This is the setting of *least squares*: when no exact solution exists, you will later settle for the closest reachable point.

Case 3: $r = m < n$ (wide, full row rank). More unknowns than equations. The columns span all of $\mathbb{R}^m$, so existence holds for *every* $\mathbf{b}$ – but there are $n - m > 0$ free variables, so the nullspace is nonzero and solutions are never unique. Outcome: ∞ solutions for every $\mathbf{b}$. These are *underdetermined* systems, where minimum-norm solutions are often selected.

Case 4: $r < m$ and $r < n$ (the general rectangular case). Neither full. Existence can fail *and* uniqueness fails whenever existence holds. Outcome: 0 *or* ∞. Almost every real-world data matrix is here.

Compute it yourself

Work through a small example of each case and verify the solution count.

1. Case 1: $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$. Solve; you should get the single point $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$.
2. Case 2: $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$. Try $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ (consistent) and $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}$ (inconsistent). What is special about the first $\mathbf{b}$?
3. Case 3: $\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$, $\mathbf{b} = [2]$. Write the whole plane of solutions in the form

$\mathbf{x}_p + \text{null}(\mathbf{A})$.

4. Case 4: $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. Find a $\mathbf{b}$ with no solution and a $\mathbf{b}$ with infinitely many.

13.6 A full worked example: building the complete solution

Let us run the whole machine once, slowly, on a matrix that is neither square nor full rank, so you see every moving part. Take

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 0 & 2 \\ 0 & 0 & 1 & 4 \\ 1 & 3 & 1 & 6 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 6 \\ 7 \end{bmatrix}.$$

This is 3×4: three equations, four unknowns. We expect free variables.

Step 1 – eliminate. Work on the augmented matrix. Subtract row 1 from row 3:

$$\left[\begin{array}{cccc|c} 1 & 3 & 0 & 2 & 1 \\ 0 & 0 & 1 & 4 & 6 \\ 1 & 3 & 1 & 6 & 7 \end{array}\right] \xrightarrow{R_3 - R_1} \left[\begin{array}{cccc|c} 1 & 3 & 0 & 2 & 1 \\ 0 & 0 & 1 & 4 & 6 \\ 0 & 0 & 1 & 4 & 6 \end{array}\right] \xrightarrow{R_3 - R_2} \left[\begin{array}{cccc|c} 1 & 3 & 0 & 2 & 1 \\ 0 & 0 & 1 & 4 & 6 \\ 0 & 0 & 0 & 0 & 0 \end{array}\right].$$

The last row is all zeros, including the augmented entry: **consistent**. The pivots sit in columns 1 and 3, so the rank is $r = 2$. Columns 2 and 4 have no pivot – those are the *free* columns, and x_2, x_4 are the *free variables*. The nullspace will have dimension $n - r = 4 - 2 = 2$.

Step 2 – a particular solution (free variables = 0). Set $x_2 = 0$ and $x_4 = 0$. The reduced equations read

$$x_1 + 3x_2 + 2x_4 = 1 \;\Rightarrow\; x_1 = 1, \qquad x_3 + 4x_4 = 6 \;\Rightarrow\; x_3 = 6.$$

So

$$\mathbf{x}_p = \begin{bmatrix} 1 \\ 0 \\ 6 \\ 0 \end{bmatrix}.$$

Sanity check: $\mathbf{A}\mathbf{x}_p$ should be $\mathbf{b}$. First entry $1 \cdot 1 + 3 \cdot 0 + 0 \cdot 6 + 2 \cdot 0 = 1$; second $0 + 0 + 1 \cdot 6 + 0 = 6$; third $1 + 0 + 6 + 0 = 7$. Indeed $\begin{bmatrix} 1 \\ 6 \\ 7 \end{bmatrix} = \mathbf{b}$. Good.

Step 3 – the special solutions (nullspace basis). Now solve $\mathbf{A}\mathbf{x} = \mathbf{0}$. Same pivots; the homogeneous equations are

$$x_1 = -3x_2 - 2x_4, \qquad x_3 = -4x_4.$$

Turn each free variable on by itself.

Set $x_2 = 1,\ x_4 = 0$: then $x_1 = -3,\ x_3 = 0$, giving $\mathbf{s}_1 = \begin{bmatrix} -3 \\ 1 \\ 0 \\ 0 \end{bmatrix}$.

Set $x_2 = 0,\ x_4 = 1$: then $x_1 = -2,\ x_3 = -4$, giving $\mathbf{s}_2 = \begin{bmatrix} -2 \\ 0 \\ -4 \\ 1 \end{bmatrix}$.

These two special solutions form a basis for the nullspace (a minimal generating set with no redundant directions), so $\text{null}(\mathbf{A}) = \text{span}\{\mathbf{s}_1, \mathbf{s}_2\}$ – a 2-dimensional plane through the origin in $\mathbb{R}^4$, exactly matching the free-variable count $n - r = 4 - 2 = 2$.

Step 4 – assemble the complete solution. By Theorem 13.1,

$$\mathbf{x} = \begin{bmatrix} 1 \\ 0 \\ 6 \\ 0 \end{bmatrix} + c_1 \begin{bmatrix} -3 \\ 1 \\ 0 \\ 0 \end{bmatrix} + c_2 \begin{bmatrix} -2 \\ 0 \\ -4 \\ 1 \end{bmatrix}, \qquad c_1, c_2 \in \mathbb{R}.$$

That is *every* solution: a particular point in $\mathbb{R}^4$ plus a 2-dimensional plane of freedom. Pick any c_1, c_2 and you get a valid $\mathbf{x}$; conversely every valid $\mathbf{x}$ arises from some choice. There are infinitely many solutions, organized as a flat 2-dimensional sheet.

Remark 13.1. Notice the bookkeeping that made this painless: the *same* elimination served both jobs. The pivots told us the rank and which variables are free; setting free variables to 0 produced $\mathbf{x}_p$; turning them on one at a time produced the nullspace basis. You never reduce twice.

Common trap

A frequent slip: forgetting to put the free variables *into* the special solutions. When you set $x_2 = 1$, the special solution's second entry must be 1, not 0 – the free variable is part of the vector. Each special solution has a single 1 in its own free slot and 0 in the other free slots; the pivot slots are then forced. If your special solutions do not have those isolated 1s, recheck.

13.7 Reading the four cases off your worked example

The example above had $m = 3$, $n = 4$, $r = 2$. Since $r < m$ $(2 < 3)$ it is not full row rank, so some right-hand sides have *no* solution. Since $r < n$ $(2 < 4)$ it is not full column rank, so when a solution exists it is never unique. That is Case 4. Our chosen $\mathbf{b}$ happened to be consistent, so we landed in the "∞" branch. Had we used, say, $\mathbf{b} = \begin{bmatrix} 1 \\ 6 \\ 99 \end{bmatrix}$, the last augmented row would have read $0 = 92$ and we would have been in the "0" branch.

Example 13.3 (Flipping a case by changing the shape)**.** Delete the last column of the worked $\mathbf{A}$ to get the 3×3 matrix $\mathbf{A}' = \begin{bmatrix} 1 & 3 & 0 \\ 0 & 0 & 1 \\ 1 & 3 & 1 \end{bmatrix}$. Column 2 is still a non-pivot (it equals $3\times$ column 1), so

$r = 2$ *again*, and now $r < n = 3$ and $r < m = 3$: still Case 4, still 0 or ∞. Squareness did not buy uniqueness, because the columns are still dependent. To reach Case 1 you must make the columns independent, e.g. replace column 2 by $\begin{bmatrix}0\\1\\0\end{bmatrix}$, which gives a pivot in every column.

13.8 The two "nice" rectangular extremes

The full-row-rank and full-column-rank cases are worth a moment alone, because they are the backbone of regression and of underdetermined inverse problems. Figure 13.5 contrasts their geometry: a tall full-column-rank matrix can *miss* $\mathbf{b}$ but never offers a choice, while a wide full-row-rank matrix always *reaches* $\mathbf{b}$ but always offers a whole line (or more) of choices.

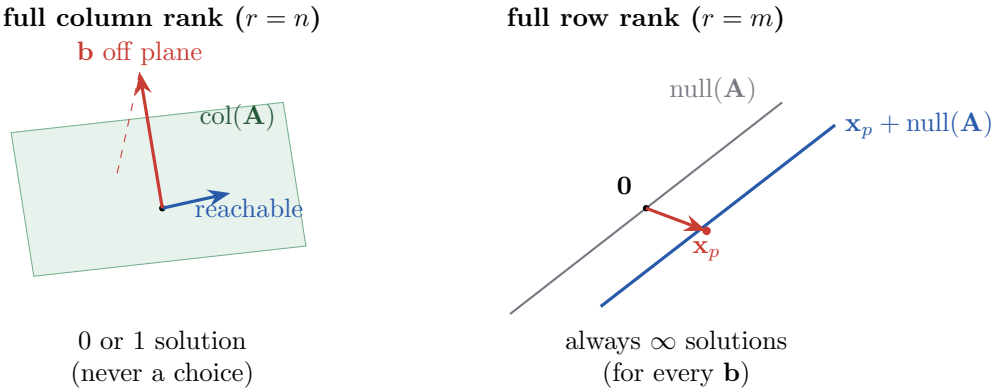


Figure 13.5: The two rectangular extremes. *Left:* full column rank — the column space may not contain $\mathbf{b}$ (no solution), but when it does the answer is a single point, since the nullspace is just $\{\mathbf{0}\}$. *Right:* full row rank — every $\mathbf{b}$ is reachable, but the nonzero nullspace slides $\mathbf{x}_p$ into a whole flat of solutions.

13.8.1 Full column rank: at most one solution

If $\text{rank}(\mathbf{A}) = n$ (every column is a pivot column), there are no free variables, so the nullspace is just $\{\mathbf{0}\}$. The complete solution collapses to a single point $\mathbf{x}_p$ whenever $\mathbf{b}$ is reachable. The tall, thin matrix

$$\mathbf{A} = \begin{bmatrix}1 & 0\\1 & 1\\1 & 2\end{bmatrix}$$

has two independent columns ($r = 2 = n$). For $\mathbf{b} = \begin{bmatrix}1\\2\\3\end{bmatrix}$ you can check $\mathbf{x} = \begin{bmatrix}1\\1\end{bmatrix}$ works and is the *only* solution. For $\mathbf{b} = \begin{bmatrix}1\\2\\4\end{bmatrix}$, elimination produces a contradiction and there is no exact solution at all – the point sticks up out of the 2-dimensional column plane in $\mathbb{R}^3$.

13.8.2 Full row rank: always at least one solution

If $\text{rank}(\mathbf{A}) = m$ (every row has a pivot), the columns span all of $\mathbb{R}^m$, so every $\mathbf{b}$ is reachable and the system is *always* consistent. The wide matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

has $r = 2 = m$. Both rows pivot, so for any $\mathbf{b} \in \mathbb{R}^2$ a solution exists; and since $n - r = 3 - 2 = 1$, the solution set is always a line. For $\mathbf{b} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$ one particular solution is found by setting the lone free variable $x_3 = 0$: back-substitution gives $x_2 = 3$, $x_1 = 0$, so $\mathbf{x}_p = \begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix}$. The nullspace special solution (set $x_3 = 1$) is $\mathbf{s} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$, so the complete solution is the line $\begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix} + c \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$.

Connection to other math

The decomposition "$\mathbf{x}_p$+ homogeneous solutions" holds for *any* linear operator L in an equation $L\mathbf{x} = \mathbf{b}$, not only for matrices. In a differential equations course you meet exactly this when solving $y'' + y = \cos t$: the general solution is one *particular* solution (a response forced by $\cos t$) plus the general solution of the *homogeneous* equation $y'' + y = 0$ (the nullspace of the operator $L = \frac{d^2}{dt^2} + 1$, namely $c_1 \cos t + c_2 \sin t$). Same theorem, same proof: L is linear, so $L(y_p + y_h) = \mathbf{b} + \mathbf{0}$. Once you see that $\frac{d^2}{dt^2} + 1$ is a linear operator with a 2-dimensional nullspace, the ODE recipe and the matrix recipe are literally the same recipe. This is the bridge to Chapter 25.

Modern application

Underdetermined systems in machine learning and imaging. Modern problems are routinely *wide*: you have far more unknowns n than measurements m, so $\mathbf{Ax} = \mathbf{b}$ has $r = m < n$ and an entire $(n - m)$-dimensional sheet of exact solutions (Case 3). Examples: reconstructing a high-resolution image from a few sensor readings (compressed sensing), or fitting an over-parameterized neural network where there are more weights than training equations. Theorem 13.1 says all these solutions are equally valid *as far as the equations are concerned* – so you must add a preference to pick one. Choosing the solution of smallest length $\|\mathbf{x}\|$ gives the *minimum-norm* solution and lands you back on the row space; choosing the sparsest $\mathbf{x}$ (fewest nonzeros) is the ℓ_1 idea behind compressed sensing. Either way, you are selecting one point $\mathbf{x}_p$ out of the affine sheet $\mathbf{x}_p + \text{null}(\mathbf{A})$. The nullspace provides the precise space of ambiguity your prior must resolve.

13.9 Recipe: the complete solution in five steps

Focus

Given $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{A} \in \mathbb{R}^{m \times n}$:

1. **Augment and reduce.** Row-reduce $[\,\mathbf{A} \mid \mathbf{b}\,]$ to a row echelon (or reduced) form. Identify the pivot columns; let r be their number.
2. **Check consistency.** If any zero row of $\mathbf{A}$ has a nonzero augmented entry, **stop: no solution**. Otherwise continue.
3. **Particular solution.** Set all free variables to 0 and solve for the pivot variables. Call the result $\mathbf{x}_p$.
4. **Special solutions.** For each free variable, set it to 1 and the other free variables to 0; solve $\mathbf{Ax} = \mathbf{0}$ for the pivot variables. This gives $\mathbf{s}_1, \ldots, \mathbf{s}_{n-r}$, which form a basis (a minimal spanning set) of the nullspace.
5. **Assemble.** The complete solution is $\mathbf{x} = \mathbf{x}_p + c_1\mathbf{s}_1 + \cdots + c_{n-r}\mathbf{s}_{n-r}$, $c_i \in \mathbb{R}$.

If $r = n$ there are no free variables and $\mathbf{x} = \mathbf{x}_p$ is unique. If $r = m$ the consistency check in step 2 can never fail.

Pause

Pause and think: must free variables always be set to zero for $\mathbf{x}_p$? In Step 3, we set all free variables to 0 to find $\mathbf{x}_p$. Could you have set them to 17 or $-\pi$ instead?
Yes: any choice of values for the free variables yields a completely valid particular solution. Setting them all to zero is simply the cleanest arithmetic: it zeroes out the free columns during back-substitution and anchors the affine flat at the point where the free coordinates vanish.

Example 13.4 (A clean square singular system, start to finish)**.**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 3 \\ 1 & 2 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 2 \\ 5 \\ 3 \end{bmatrix}.$$

Reduce $[\,\mathbf{A} \mid \mathbf{b}\,]$. Subtract $2R_1$ from R_2 and R_1 from R_3:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 2 & 4 & 3 & 5 \\ 1 & 2 & 2 & 3 \end{array}\right] \longrightarrow \left[\begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{array}\right] \xrightarrow{R_3 - R_2} \left[\begin{array}{ccc|c} 1 & 2 & 1 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{array}\right].$$

Pivots in columns 1 and 3; column 2 is free, so $r = 2$, and the last row reads $0 = 0$ – consistent.

Particular (set $x_2 = 0$): from R_2, $x_3 = 1$; from R_1, $x_1 + 2(0) + 1 = 2 \Rightarrow x_1 = 1$. So $\mathbf{x}_p = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$.

Special (set $x_2 = 1$, $\mathbf{b} \to \mathbf{0}$): $x_3 = 0$ and $x_1 + 2(1) + 0 = 0 \Rightarrow x_1 = -2$, giving $\mathbf{s} = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$. Complete solution:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + c \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}, \qquad c \in \mathbb{R}.$$

A line of solutions in $\mathbb{R}^3$ – this square matrix had $r = 2 < 3$, dependent columns, so it is squarely in Case 4 despite being 3×3.

13.10 Counting, sanity checks, and what can go wrong

A few quick mental checks save you from most errors.

- **Dimension check.** The number of special solutions must equal the number of free columns, $n - r$. If you produced three special solutions but only two columns were free, you double-counted.
- **Plug $\mathbf{x}_p$ back in.** It costs one matrix-vector product and catches arithmetic slips immediately. $\mathbf{A}\mathbf{x}_p$ must equal $\mathbf{b}$ on the nose.
- **Plug each $\mathbf{s}_i$ in.** Each must give $\mathbf{A}\mathbf{s}_i = \mathbf{0}$. If not, you mishandled a sign during back-substitution.
- **Consistency is about b only.** Changing $\mathbf{b}$ never changes the nullspace or the special solutions – those come from $\mathbf{A}$ alone. So if you solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ for several right-hand sides, you reduce $\mathbf{A}$ once and reuse the special solutions every time; only $\mathbf{x}_p$ and the consistency verdict change.

Walking question

Here is a puzzle to chew on. You are told $\mathbf{A}$ is 4×6 and that $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a solution for the specific $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$, and that the solution is *not* unique. What is the smallest the solution set's dimension could be? Could the solution ever be unique for some *other* right-hand side while staying non-unique for this one? (Think about whether "unique" is a property of $\mathbf{A}$ or of $\mathbf{b}$.)

Why this matters

Two subspaces attached to $\mathbf{A}$ – the column space living in the output $\mathbb{R}^m$ and the nullspace living in the input $\mathbb{R}^n$ – between them answer *every* question you can ask about $\mathbf{A}\mathbf{x} = \mathbf{b}$. The column space decides *whether*; the nullspace decides *how many*; and the single number r ties them together through $r = \dim \text{col}$ and $n - r = \dim \text{null}$. In Chapters 15 and 16, that same pivot count r will govern the row space and left nullspace as well.

History and connection

The decomposition of an inhomogeneous problem into one particular solution plus all solutions to the homogeneous equation is one of the grand unifying themes of mathematics. It first surfaced in the 1750s and 1760s in the work of Leonhard Euler, Joseph-Louis Lagrange, and Jean le Rond d'Alembert on linear differential equations of the form $L(y) = f(t)$ (modeling physical systems like vibrating strings and planetary orbits). They observed that if y_p is any single response satisfying $L(y_p) = f(t)$, and y_h satisfies the unforced equation $L(y_h) = 0$, then linearity guarantees that $L(y_p + y_h) = L(y_p) + L(y_h) = f(t) + 0 = f(t)$.
For the first half of the nineteenth century, linear systems of algebraic equations were tackled

without any such unifying geometry. Mathematicians like Carl Friedrich Gauss relied on manual elimination, while others leaned heavily on determinants (such as Cramer's rule). But determinants only worked for square matrices and went silent whenever equations were redundant or rectangular. Astronomers and geodesists were repeatedly stuck on systems with more equations than unknowns (overdetermined celestial observations) or fewer equations than unknowns (underdetermined constraints). Without the concepts of vector spaces, early textbooks offered clumsy, confusing lists of case-by-case rules, and practitioners frequently fell into the trap of assuming that having as many equations as unknowns guaranteed a unique solution.

Moving from coordinate-heavy formulas to this geometric picture took decades. In 1844, the German schoolmaster Hermann Grassmann published his revolutionary *Ausdehnungslehre* (Theory of Extension), introducing abstract vector spaces, linear combinations, and geometric subspaces. Because Grassmann abandoned the traditional coordinate-heavy formulas in favor of coordinate-free spaces, his contemporaries found his work nearly impenetrable, and it was largely ignored for decades. Only toward the late nineteenth century, through the work of Giuseppe Peano, Camille Jordan, and Ferdinand Georg Frobenius, did the modern geometric synthesis take hold.

That synthesis swept away the old folklore. Solvability was recognized as purely geometric: does the target vector $\mathbf{b}$ lie in the column space $\text{col}(\mathbf{A})$? When it does, the complete solution set is not a vector subspace (since scaling or adding two solutions shifts the output away from $\mathbf{b}$), but an *affine flat*—the nullspace $\text{null}(\mathbf{A})$ picked up and displaced away from the origin by an anchor $\mathbf{x}_p$. Beginners often worry that their particular solution $\mathbf{x}_p$ is "wrong" if it differs from a textbook answer. But setting free variables to zero is simply one convenient choice; every single point on the affine flat serves equally well as an anchor.

What remains open today is not how to describe this flat, but how to choose the "best" point on it. In modern machine learning, compressed sensing, and inverse problems, systems are vastly underdetermined: an imaging algorithm might have 10^5 measurements but 10^7 unknown pixels, leaving a nullspace with millions of dimensions of freedom. How to formulate principled criteria (such as minimal ℓ_1-norm for sparse recovery, or implicit regularization from gradient descent) to select the most meaningful particular solution $\mathbf{x}_p$ from an infinite sheet of exact solutions is one of the most active research frontiers in modern data science.

13.11 Summary

- Every solution of $\mathbf{Ax} = \mathbf{b}$ is one particular solution plus a nullspace vector: $\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$. The solution set is the nullspace translated to pass through $\mathbf{x}_p$ – an affine subspace of dimension $n - r$.
- **Existence** $\iff$ $\mathbf{b} \in \text{col}(\mathbf{A})$; detectable as "no zero-row contradiction" after augmented elimination. Existence for *all* $\mathbf{b}$ means full row rank $r = m$.
- **Uniqueness** $\iff$ $\text{null}(\mathbf{A}) = \{\mathbf{0}\}$, i.e. no free variables, i.e. full column rank $r = n$.
- The number of solutions is always 0, 1, or ∞ – never any other finite count.
- The four cases sort by ($r = m$?) and ($r = n$?): square-invertible (exactly one for all $\mathbf{b}$), tall full-column-rank (0 or 1), wide full-row-rank (∞ for all $\mathbf{b}$), and the general case (0 or ∞).

- The five-step recipe – reduce, check consistency, particular, special, assemble – produces the complete solution from a *single* elimination.

Connection to other math

We now have a complete constructive algorithm to describe the entire infinite solution set of any consistent rectangular system $\mathbf{Ax} = \mathbf{b}$. From a single elimination run, we identify consistency, anchor at a particular solution $\mathbf{x}_p$, and span every degree of freedom with $n - r$ special solutions along null($\mathbf{A}$).
Yet a natural question remains: while our algorithm produces $n - r$ special solutions, what mathematically guarantees that this count is minimal? Could a clever alternative elimination path span the same nullspace with fewer vectors? What does it rigorously mean for a set of vectors to have no redundancies? Chapter 14 answers this foundational question by establishing the rigorous theory of linear independence, basis, and dimension, proving (via the Steinitz exchange lemma, which shows you can swap vectors without shrinking a spanning set) that the number of degrees of freedom is an absolute geometric invariant of the subspace itself.

13.12 Exercises

Exercise 13.1 (Warmup: read the verdict)**.** For each reduced augmented matrix, state whether the system is consistent, and if so how many solutions it has.

$$\text{(a)} \left[\begin{array}{cc|c} 1 & 2 & 3 \\ 0 & 0 & 0 \end{array}\right] \quad \text{(b)} \left[\begin{array}{cc|c} 1 & 0 & 5 \\ 0 & 1 & 2 \end{array}\right] \quad \text{(c)} \left[\begin{array}{ccc|c} 1 & 0 & 4 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 7 \end{array}\right]$$

Exercise 13.2 (Particular plus special)**.** Find the complete solution of $\begin{bmatrix} 1 & 1 & 2 \\ 2 & 2 & 5 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}$. Identify $\mathbf{x}_p$, the special solution(s), and the dimension of the solution set.

Exercise 13.3 (Consistency depends on $\mathbf{b}$)**.** Let $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ -1 & -2 \end{bmatrix}$. Describe *all* vectors $\mathbf{b} \in \mathbb{R}^3$ for which $\mathbf{Ax} = \mathbf{b}$ is consistent. (Hint: what is col($\mathbf{A}$), and what linear conditions on b_1, b_2, b_3 cut it out?)

Exercise 13.4 (A square singular system)**.** For $\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix}$, decide which case (of the four) it is, then give the complete solution of $\mathbf{Ax} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$.

Exercise 13.5 (Full column rank)**.** The matrix $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$ has full column rank. For which of $\mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$ is the system consistent? When it is, find the unique solution. When it is not, explain geometrically what fails.

Exercise 13.6 (Full row rank, every **b**)**.** Show that $\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \end{bmatrix}$ has a solution for every $\mathbf{b} \in \mathbb{R}^2$, and write the complete solution of $\mathbf{Ax} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ as a particular solution plus a line.

Exercise 13.7 (Why only 0, 1, or ∞)**.** Prove that a system $\mathbf{Ax} = \mathbf{b}$ cannot have exactly two distinct solutions. (Hint: if $\mathbf{x}_1 \neq \mathbf{x}_2$ both solve it, consider $\mathbf{x}_1 + t(\mathbf{x}_2 - \mathbf{x}_1)$ for $t \in \mathbb{R}$ and use linearity.)

Exercise 13.8 (Reusing one elimination)**.** Let $\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix}$. Find the nullspace once. Then, without re-reducing $\mathbf{A}$, determine the complete solution set for $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and for $\mathbf{b} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$. Explain why the special solution is identical in both cases.

Exercise 13.9 (The shape from rank alone)**.** A matrix $\mathbf{A}$ is 5×7 with rank $r = 4$. (a) What is the dimension of the nullspace? (b) For a consistent $\mathbf{b}$, what is the dimension of the solution set? (c) Is the system solvable for *every* $\mathbf{b} \in \mathbb{R}^5$? (d) Can any right-hand side give a unique solution? Justify each answer from the four-case framework.

Exercise 13.10 (Challenge: prescribe the solution set)**.** Construct a 3×3 matrix $\mathbf{A}$ and a vector $\mathbf{b}$ so that the complete solution of $\mathbf{Ax} = \mathbf{b}$ is exactly the line $\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} + c \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$, $c \in \mathbb{R}$. (Hint: you need $\begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$ in null($\mathbf{A}$) and $\mathbf{A} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \mathbf{b}$. Build the rows so each is orthogonal to the nullspace direction, then read off $\mathbf{b}$.)

Exercise 13.11 (Challenge: a uniqueness criterion without elimination)**.** Suppose $\mathbf{A}$ is $m \times n$ and you are told that $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible. Prove that $\mathbf{Ax} = \mathbf{b}$ has *at most* one solution for every $\mathbf{b}$. (Hint: show $\mathbf{Ax} = \mathbf{0}$ forces $\mathbf{x} = \mathbf{0}$ by hitting it with $\mathbf{A}^\mathsf{T}$, or by looking at $\|\mathbf{Ax}\|^2 = \mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{Ax}$.) Which of the four cases does this rule out?

Chapter 14

Independence, Basis, and Dimension

In Chapter 13, we used Gaussian elimination to describe the column space and nullspace, finding r pivot columns for $\text{col}(\mathbf{A})$ and $n - r$ special solutions for $\text{null}(\mathbf{A})$. In each case, a small set of vectors generated an entire subspace.

This chapter answers the question behind those recipes: *how many vectors do you really need to build a subspace, and how do you know when you have just enough — no wasted vectors, no missing directions?* That story rests on three ideas:

- **Independence** — no vector in your set is redundant.
- **Spanning** — your set reaches every destination in the space.
- **Basis** — independent and spanning: minimal building blocks.

From "basis" falls a single number, the **dimension**, that pins down the true size of a subspace once and for all.

14.1 The redundant vector: a picture of dependence

Given a collection of vectors, are any of them redundant? A vector is redundant if it can be written as a linear combination of the others — adding no new direction to the span.

Intuition

Think of each vector as a direction of motion from the origin. A set of vectors is **dependent** if at least one vector points somewhere you could already reach by combining the others. That vector is redundant: removing it does not change the span. A set is **independent** if every vector provides a genuinely new direction — remove any one of them and the span strictly shrinks.

Let us make that concrete in $\mathbb{R}^2$. Take

$$\mathbf{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \qquad \mathbf{v}_3 = \begin{bmatrix} 1 \\ 5 \end{bmatrix}.$$

The first two point in different directions: neither is a scalar multiple of the other, so they span a

two-dimensional plane. But watch the third vector:

$$2\,\mathbf{v}_1 + 3\,\mathbf{v}_2 = 2\begin{bmatrix}2\\1\end{bmatrix} + 3\begin{bmatrix}-1\\1\end{bmatrix} = \begin{bmatrix}4\\2\end{bmatrix} + \begin{bmatrix}-3\\3\end{bmatrix} = \begin{bmatrix}1\\5\end{bmatrix} = \mathbf{v}_3.$$

So any linear combination of $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ is already in the span of $\mathbf{v}_1$ and $\mathbf{v}_2$. The three vectors are **dependent**: there is a redundancy in the set.

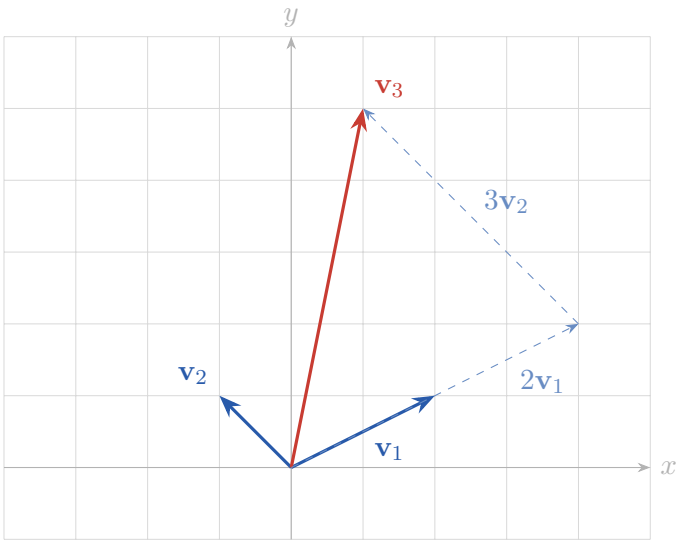


Figure 14.1: The redundant vector. $\mathbf{v}_3$ (red) is reached by walking $2\mathbf{v}_1$ then $3\mathbf{v}_2$ (dashed). It lies inside the reach of the other two, so the set $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ is dependent.

There is a symmetric way to state this redundancy without singling out one particular vector. Move everything to one side of the equation $\mathbf{v}_3 = 2\mathbf{v}_1 + 3\mathbf{v}_2$:

$$2\,\mathbf{v}_1 + 3\,\mathbf{v}_2 - \mathbf{v}_3 = \mathbf{0}.$$

We have written the zero vector as a combination of $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ *without using all-zero coefficients*. That is the defining condition for linear dependence, and it is the definition we will lean on for the rest of the book.

Focus

Dependence means the zero vector can be reached in a *nontrivial* way — with coefficients that are not all zero. Independence means the *only* linear combination that yields $\mathbf{0}$ has every coefficient equal to zero. Everything in this chapter flows from that one idea.

14.2 Linear independence, defined precisely

Definition 14.1 (Linear independence)**.** A list of vectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k$ in a vector space V is **linearly independent** if the only scalars $c_1, c_2, \ldots, c_k$ satisfying

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$$

are $c_1 = c_2 = \cdots = c_k = 0$. If *any* choice of scalars that are not all zero produces $\mathbf{0}$, the list is **linearly dependent**.

This definition is formulated around the zero vector, but its primary consequence is *uniqueness of representation*, which is what we actually care about. Suppose some vector $\mathbf{w}$ in the span could be written two different ways,

$$\mathbf{w} = a_1\mathbf{v}_1 + \cdots + a_k\mathbf{v}_k = b_1\mathbf{v}_1 + \cdots + b_k\mathbf{v}_k.$$

Subtract: $(a_1 - b_1)\mathbf{v}_1 + \cdots + (a_k - b_k)\mathbf{v}_k = \mathbf{0}$. If the $\mathbf{v}_i$ are independent, every difference $a_i - b_i$ must be zero, so the two "different" representations were the same all along. Independence is exactly the condition that *coordinates are unambiguous*. This property is what makes bases useful as coordinate systems.

Common trap

"Linearly independent" is a property of the *whole list together*, not of individual vectors. No single vector is "independent." And pairwise checks are not enough: three vectors can be independent two-at-a-time yet dependent as a trio. In $\mathbb{R}^2$,

$$\begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}0\\1\end{bmatrix}, \begin{bmatrix}1\\1\end{bmatrix}$$

has no two parallel vectors, yet $\begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\1\end{bmatrix} - \begin{bmatrix}1\\1\end{bmatrix} = \mathbf{0}$, so the three are dependent. Always test the full set at once.

Two vectors: independence is non-parallelism

For just two vectors the definition collapses to something you can see. $\{\mathbf{v}_1, \mathbf{v}_2\}$ is dependent exactly when one is a scalar multiple of the other — when they are parallel (including the degenerate cases where one is $\mathbf{0}$). If $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 = \mathbf{0}$ with, say, $c_1 \neq 0$, then $\mathbf{v}_1 = -(c_2/c_1)\mathbf{v}_2$, a scalar multiple. So:

Focus

Two nonzero vectors are independent $\iff$ they are not parallel $\iff$ they span a plane (not just a line). Independence guarantees that neither vector lies along the line spanned by the other.

Three vectors: trapped in a plane versus stepping into 3D

Two independent vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ in $\mathbb{R}^3$ span a flat two-dimensional sheet (a plane) passing through the origin. What happens when you bring in a third vector $\mathbf{v}_3$?

There are two possibilities, illustrated in Figure 14.2:

- If $\mathbf{v}_3$ lies flat inside that plane, it adds zero new territory. Any destination along $\mathbf{v}_3$ could already be reached using $\mathbf{v}_1$ and $\mathbf{v}_2$. The trio $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ is **linearly dependent**, even though no two vectors are parallel.

- If $\mathbf{v}_3$ pokes upward or downward out of the plane, it opens up the third dimension. Combinations of the three now reach every point in space. The trio $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ is **linearly independent**.

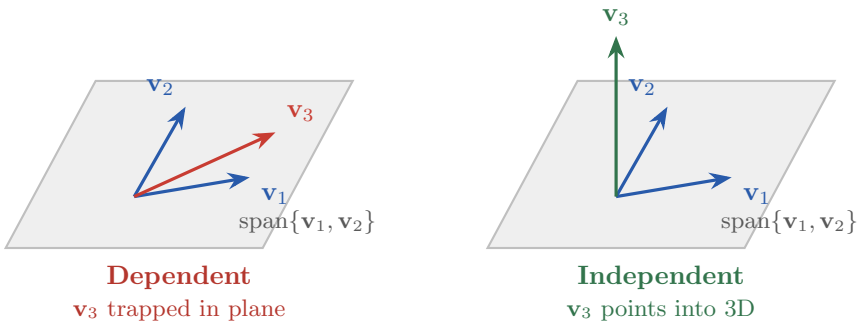


Figure 14.2: Linear dependence and independence in $\mathbb{R}^3$. Two non-parallel vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ span a flat two-dimensional plane (gray). Left: If a third vector $\mathbf{v}_3$ (red) lies flat inside that plane, it adds no new directions; the trio is **linearly dependent**. Right: If $\mathbf{v}_3$ (green) sticks out of the plane into the third dimension, it opens up new space; the trio is **linearly independent**.

Compute it yourself

Pause and think: What if you bring a *fourth* vector $\mathbf{v}_4$ into $\mathbb{R}^3$? Three independent vectors in $\mathbb{R}^3$ span all of $\mathbb{R}^3$. Any fourth vector must already lie in their span, so any four vectors in $\mathbb{R}^3$ are linearly dependent. Keep this picture in mind as we formalize the algebraic test.

Any set containing the zero vector is dependent

If $\mathbf{v}_1 = \mathbf{0}$, then $1 \cdot \mathbf{v}_1 + 0 \cdot \mathbf{v}_2 + \cdots + 0 \cdot \mathbf{v}_k = \mathbf{0}$ uses a nonzero coefficient (the leading 1), so the set is automatically dependent. Any list containing $\mathbf{0}$ is dependent. This matches the geometric intuition: the zero vector adds no direction of movement and contributes nothing new to the span.

14.3 The independence test you will actually run

To test whether a given list of vectors is linearly independent, stack the vectors as the columns of a matrix and use Gaussian elimination.

The equation $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ is precisely the matrix equation $\mathbf{Ac} = \mathbf{0}$, where the columns of $\mathbf{A}$ are the $\mathbf{v}_i$ and $\mathbf{c} = (c_1, \ldots, c_k)$. So:

Formal

Independence as a nullspace condition. Let $\mathbf{A}$ have columns $\mathbf{v}_1, \ldots, \mathbf{v}_k$. The columns are linearly independent if and only if $\text{null}(\mathbf{A}) = \{\mathbf{0}\}$, i.e. $\mathbf{Ac} = \mathbf{0}$ has *only* the trivial solution. Equivalently, after elimination, *every column is a pivot column* — there are no free variables.

A free variable allows choosing a nonzero solution of $\mathbf{Ac} = \mathbf{0}$, producing a nontrivial linear combination that sums to zero. Having no free variables means the only solution is $\mathbf{c} = \mathbf{0}$, guaranteeing

independence. Let us run it.

Example 14.1 (Testing three vectors in $\mathbb{R}^3$)**.** Are these independent?

$$\mathbf{v}_1 = \begin{bmatrix}1\\2\\3\end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix}1\\1\\1\end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix}3\\5\\7\end{bmatrix}.$$

Stack them as columns and eliminate:

$$\mathbf{A} = \begin{bmatrix}1&1&3\\2&1&5\\3&1&7\end{bmatrix} \xrightarrow[R_3-3R_1]{R_2-2R_1} \begin{bmatrix}1&1&3\\0&-1&-1\\0&-2&-2\end{bmatrix} \xrightarrow{R_3-2R_2} \begin{bmatrix}1&1&3\\0&-1&-1\\0&0&0\end{bmatrix}.$$

The third column has no pivot — column 3 is a free column. So there is a nontrivial solution and the vectors are **dependent**. Read the dependence off: back-substitute with the free variable $c_3 = 1$. Row 2 gives $-c_2 - c_3 = 0 \Rightarrow c_2 = -1$. Row 1 gives $c_1 + c_2 + 3c_3 = 0 \Rightarrow c_1 = -(-1) - 3 = -2$. Check:

$$-2\mathbf{v}_1 - \mathbf{v}_2 + \mathbf{v}_3 = -2\begin{bmatrix}1\\2\\3\end{bmatrix} - \begin{bmatrix}1\\1\\1\end{bmatrix} + \begin{bmatrix}3\\5\\7\end{bmatrix} = \begin{bmatrix}0\\0\\0\end{bmatrix}. \checkmark$$

Indeed $\mathbf{v}_3 = 2\mathbf{v}_1 + \mathbf{v}_2$.

Example 14.2 (An independent set of three vectors)**.** Change one entry:

$$\mathbf{A} = \begin{bmatrix}1&1&3\\2&1&5\\3&1&8\end{bmatrix} \xrightarrow[R_3-3R_1]{R_2-2R_1} \begin{bmatrix}1&1&3\\0&-1&-1\\0&-2&-1\end{bmatrix} \xrightarrow{R_3-2R_2} \begin{bmatrix}1&1&3\\0&-1&-1\\0&0&1\end{bmatrix}.$$

Now there are three pivots, one in every column. No free variables, so $\mathbf{Ac} = \mathbf{0}$ forces $\mathbf{c} = \mathbf{0}$: the columns are **independent**. Three independent vectors in $\mathbb{R}^3$ form a basis, and this $\mathbf{A}$ is invertible.

Compute it yourself

Test $\begin{bmatrix}1\\3\end{bmatrix}, \begin{bmatrix}2\\6\end{bmatrix}, \begin{bmatrix}0\\1\end{bmatrix}$ by hand. (i) Are the first two independent? (ii) Is the whole set? (iii) You have three vectors in $\mathbb{R}^2$ — predict the answer *before* eliminating, then confirm. What general rule about "more vectors than dimensions" are you discovering?

More vectors than components: dependence is forced

That lab points at a theorem you will use constantly. If you have more vectors than the space has components, redundancy is unavoidable.

Theorem 14.1 (Too many vectors must be dependent)**.** *Any list of k vectors in $\mathbb{R}^n$ with $k > n$ is linearly dependent.*

Proof. Form the $n \times k$ matrix $\mathbf{A}$ whose columns are the vectors. The equation $\mathbf{Ac} = \mathbf{0}$ is a homogeneous system with n equations and k unknowns. Elimination produces at most n pivots (one per row), so at most n of the k columns are pivot columns. Since $k > n$, at least one column is free, giving a free variable and hence a nonzero solution $\mathbf{c}$. That is a nontrivial dependence. □

Intuition

Three independent vectors already span all of $\mathbb{R}^3$; any additional vector is necessarily a linear combination of those three. The maximum number of linearly independent vectors is capped by the dimension of the space. This fact ensures that dimension is well-defined, as we prove below.

14.4 Spanning sets: reaching everything

Independence is about having no redundancy. Spanning is the complementary requirement: having no missing directions. We met span in Chapter 3; let us restate it with our current vocabulary.

Definition 14.2 (Spanning set)**.** Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ **span** a vector space V if every $\mathbf{w} \in V$ can be written as some linear combination $\mathbf{w} = c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$. We write $V = \text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ and call $\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ a spanning set for V.

Spanning says: *no destination in V is out of reach.* In matrix language, $\mathbf{v}_1, \ldots, \mathbf{v}_k$ span $\mathbb{R}^n$ if and only if $\mathbf{Ax} = \mathbf{b}$ has a solution for *every* $\mathbf{b} \in \mathbb{R}^n$ — equivalently, $\text{col}(\mathbf{A}) = \mathbb{R}^n$, which (Chapter 12) happens exactly when $\mathbf{A}$ has a pivot in every *row*.

Focus

Two complementary questions, two complementary pivot patterns:

- **Independence** (no redundant vectors) $\iff$ a pivot in every *column* (no free variables).
- **Spanning** $\mathbb{R}^n$ (no missing directions) $\iff$ a pivot in every *row* (no unsolvable $\mathbf{b}$).

A basis demands both at once: a pivot in every row *and* every column — which only a *square*, full-pivot matrix can deliver.

Example 14.3 (Spanning but not independent)**.** In $\mathbb{R}^2$ the three vectors $\begin{bmatrix}1\\0\end{bmatrix}$, $\begin{bmatrix}0\\1\end{bmatrix}$, $\begin{bmatrix}1\\1\end{bmatrix}$ span $\mathbb{R}^2$ — the first two alone already reach everything. But we saw they are dependent. So a spanning set may contain redundant vectors. Conversely, $\begin{bmatrix}1\\0\end{bmatrix}$ alone is independent but spans only the x-axis, not all of $\mathbb{R}^2$. A basis achieves the exact balance: it spans the entire space without containing any redundant vectors.

Figure 14.3 shows how the shape of $\mathbf{A}$ constrains where those pivots can sit.

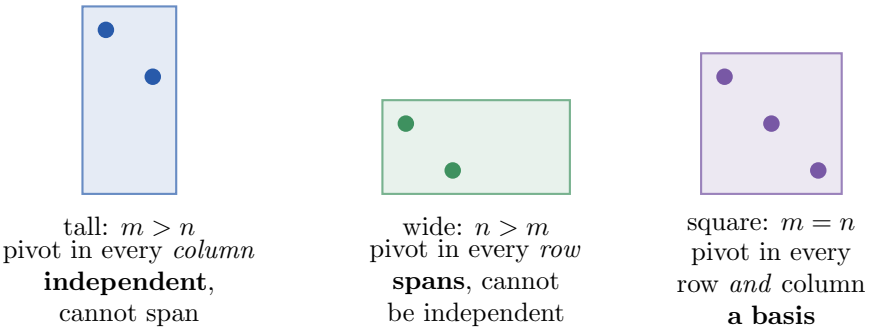


Figure 14.3: The shape of the pivots. Dots mark pivots. A *tall* matrix can have a pivot in every column (its columns are independent) but never reaches every $\mathbf{b}$. A *wide* matrix can have a pivot in every row (its columns span) but always carries a free column, so its columns are dependent. Only a *square*, full-pivot matrix achieves a pivot in every row *and* column at once — its columns are a basis.

14.5 Basis: independent and spanning

Definition 14.3 (Basis)**.** A **basis** of a vector space V is a list of vectors that is

(i) linearly independent, and

(ii) spans V.

Independent so there is nothing redundant; spanning so there is nothing missing. A basis is both the smallest set that spans the entire space and the largest linearly independent set that fits — those two extremal characterizations coincide.

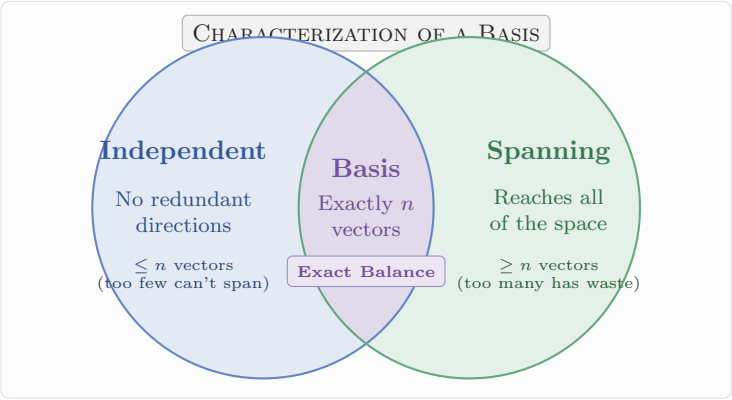


Figure 14.4: A basis lies at the intersection of two complementary conditions. Independent sets (left) have zero redundancy but may be too small to span; spanning sets (right) reach everywhere but may contain redundant vectors. A basis sits exactly at the intersection: minimally spanning and maximally independent.

Intuition

A basis is a *coordinate frame* for the space: a minimal set of independent directions such that every point has *exactly one* address. Independence guarantees the address is unique (no two combinations give the same vector); spanning guarantees every point has an address at all. A basis is the bridge between abstract vectors and the concrete tuples of numbers you compute with.

Example 14.4 (The standard basis of $\mathbb{R}^n$)**.** The vectors

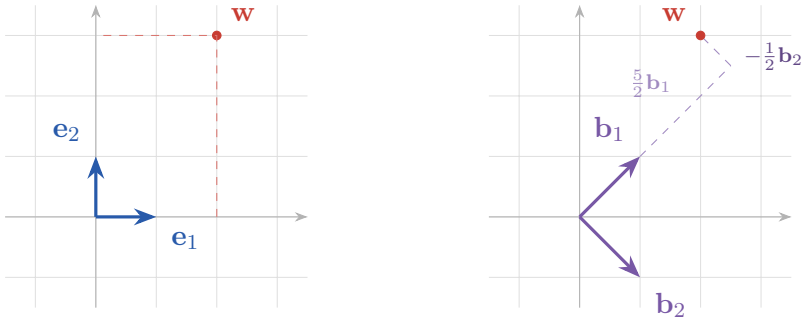
$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

form the **standard basis** of $\mathbb{R}^3$. They span (any $\mathbf{w} = w_1\mathbf{e}_1 + w_2\mathbf{e}_2 + w_3\mathbf{e}_3$) and they are independent (stacking them gives the identity matrix, all pivots). The coordinates of $\mathbf{w}$ in this basis are just its ordinary entries — which is why we usually do not even notice we are using a basis.

Example 14.5 (A non-standard basis of $\mathbb{R}^2$)**.** Consider $\mathbf{b}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Stacked,

$$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \xrightarrow{R_2 - R_1} \begin{bmatrix} 1 & 1 \\ 0 & -2 \end{bmatrix},$$

two pivots in a 2×2 matrix: independent and spanning, hence a basis. It is a perfectly good coordinate frame for the plane — just rotated and rescaled from the standard one. We will read off coordinates in this frame in Section 14.7.



standard frame: $\mathbf{w} = (2, 3)$

b frame: $\mathbf{w} = (\frac{5}{2}, -\frac{1}{2})$

Figure 14.5: The same point $\mathbf{w}$ lives in one fixed space, but its *coordinates* depend on the basis you choose. Left: in the standard frame $\mathbf{w}$ has address $(2, 3)$. Right: in the frame $\mathbf{b}_1 = (1, 1)$, $\mathbf{b}_2 = (1, -1)$, the *identical* point is reached by walking $\frac{5}{2}\mathbf{b}_1$ then $-\frac{1}{2}\mathbf{b}_2$ (dashed), so its address is $(\frac{5}{2}, -\frac{1}{2})$ — the numbers computed in Example 14.7. Changing basis is changing the ruler, not the point.

Before/after Strang

Gilbert Strang emphasizes that a basis must satisfy both conditions: the columns must be linearly independent, and they must span the space. If you are reading this before Strang: the payoff arrives in his "four subspaces" discussion, where bases for each of the four subspaces are obtained directly from the elimination of $\mathbf{A}$. If you are reading after Strang: notice that his rule "pivot columns of $\mathbf{A}$ form a basis for the column space" is Theorem 14.3 below, and the special solutions forming a basis for the nullspace is Theorem 14.4. This chapter provides the mathematical foundation for those procedures.

14.6 Dimension is well-defined

A space has many different bases (we just saw two for $\mathbb{R}^2$). But *every basis of a given space has the same number of vectors.* That common number deserves a name.

Definition 14.4 (Dimension)**.** The **dimension** of a vector space V, written $\dim V$, is the number of vectors in any basis of V. By the theorem below this number does not depend on which basis you pick.

The whole definition would be nonsense if two bases could have different sizes. The fact that they cannot is the *Steinitz exchange* idea, and it follows cleanly from Theorem 14.1 once we sharpen it slightly.

Lemma 14.1 (Independent $\leq$ spanning)**.** *If $\mathbf{u}_1, \dots, \mathbf{u}_p$ are linearly independent in V, and $\mathbf{w}_1, \dots, \mathbf{w}_q$ span V, then $p \leq q$. In words: you can never have* more *independent vectors than you have spanning vectors.*

Proof. Because the $\mathbf{w}_j$ span V, each independent vector is a combination of them: $\mathbf{u}_i = \sum_{j=1}^{q} a_{ji}\,\mathbf{w}_j$. Collect the coefficients into a $q \times p$ matrix $\mathbf{A} = (a_{ji})$. Now suppose, for contradiction, that $p > q$. Then $\mathbf{A}$ has more columns than rows, so by Theorem 14.1 (applied to its columns) the system $\mathbf{A}\mathbf{c} = \mathbf{0}$ has a nonzero solution $\mathbf{c} = (c_1, \dots, c_p)$. Form the same combination of the $\mathbf{u}_i$:

$$\sum_{i=1}^{p} c_i \mathbf{u}_i = \sum_{i=1}^{p} c_i \sum_{j=1}^{q} a_{ji}\mathbf{w}_j = \sum_{j=1}^{q} \Big(\underbrace{\textstyle\sum_{i=1}^{p} a_{ji} c_i}_{=0} \Big) \mathbf{w}_j = \mathbf{0},$$

where the inner sum is the j-th entry of $\mathbf{A}\mathbf{c} = \mathbf{0}$. So a nonzero $\mathbf{c}$ gives a nontrivial dependence among the $\mathbf{u}_i$ — contradicting their independence. Hence $p \leq q$. □

Theorem 14.2 (Dimension is well-defined)**.** *Any two bases of a vector space V have the same number of vectors.*

Proof. Let $\mathcal{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_p\}$ and $\mathcal{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_q\}$ both be bases of V. Then $\mathcal{B}$ is independent and $\mathcal{C}$ spans, so Lemma 14.1 gives $p \leq q$. By symmetry (now $\mathcal{C}$ independent, $\mathcal{B}$ spanning), $q \leq p$. Therefore $p = q$. □

Why this matters

Although a subspace contains infinitely many vectors and admits infinitely many different bases, every basis contains exactly the same number of vectors. Dimension is an intrinsic invariant of the subspace, independent of any particular choice of basis. This single integer governs the solvability of linear systems, the rank of linear transformations, and the size of the nullspace.

History and connection

The idea of dimension seems obvious in daily life: a line is one-dimensional, a sheet of paper is two-dimensional, and a room is three-dimensional. But what gives a mathematical space an *intrinsic* dimension, independent of any coordinate grid you happen to draw on it?

In 1844, the German schoolteacher Hermann Grassmann published *Die Lineare Ausdehnungslehre* (The Theory of Linear Extension). At the time, mathematicians were stuck believing that geometry required fixing an arbitrary set of coordinate axes like x, y, z. Without an external grid, they had no mathematical vocabulary to describe lines, planes, and higher-dimensional spaces. Armed only with Euclidean geometry and early algebra, Grassmann worked without fixed coordinate axes. He invented linear combinations, linear independence, and spanning sets, showing that an n-dimensional space could be understood purely through its internal directions. Because Grassmann wrote in dense, metaphysical prose, the mathematical establishment, including Carl Friedrich Gauss and August Möbius, ignored or rejected the work for over thirty years. Only in 1888 did Italian mathematician Giuseppe Peano distill Grassmann's ideas into the modern axiomatic definition of a vector space (a set where vectors can be added and scaled according to clean algebraic rules).

In 1910, Ernst Steinitz published his memoir *Algebraische Theorie der Körper* (Algebraic Theory of Fields—the study of number systems where addition, subtraction, multiplication, and division are well-behaved). Steinitz proved the exchange lemma (Lemma 14.1), establishing once and for all that no matter how two people choose a basis for the same finite-dimensional space, they must always count the exact same number of vectors. Steinitz established dimension as a structural invariant of finite-dimensional vector spaces.

Extending bases to infinite-dimensional spaces—such as $C[0, 1]$, the space of continuous functions on an interval (curves drawn without lifting your pencil)—raised a subtler question. In 1905, Georg Hamel defined a basis using strictly finite sums of basis elements (a *Hamel basis*). Proving that every vector space has a Hamel basis requires the controversial Axiom of Choice—the mathematical assumption that one can make infinitely many arbitrary selections without giving an explicit rule. The resulting Hamel bases for function spaces are unfathomably huge, and not a single concrete example can ever be written down. To escape this non-constructive trap, Polish mathematicians Juliusz Schauder and Stefan Banach in 1927 introduced *Schauder bases*, which allow infinite convergent series (building blocks that approximate any function more and more closely, like Fourier series).

Banach famously posed a challenge in his notebook at the Scottish Café in Lwów: does *every* reasonable infinite-dimensional space admit a Schauder basis? This "Basis Problem" resisted all attempts for over forty years, until Swedish mathematician Per Enflo solved it negatively in 1972 by constructing a space with no Schauder basis at all (winning the live goose promised by Stanisław Mazur in 1936). Today, the active frontier has shifted to computation: in machine learning and data compression, researchers work on *dictionary learning*, searching for algorithms that can discover the optimal, sparsest basis to represent high-dimensional images

and neural network activations.

Consequences you will reuse constantly

Corollary 14.1 (Counting shortcuts in $\mathbb{R}^n$)**.** *In an n-dimensional space:*

(a) Any independent list has at most n vectors; more than n is forced dependent.

(b) Any spanning list has at least n vectors; fewer than n cannot span.

(c) Any independent list of exactly n vectors is automatically a basis *(it must already span).*

(d) Any spanning list of exactly n vectors is automatically a basis *(it must already be independent).*

Parts (c) and (d) simplify computations: in an n-dimensional space, once a set has exactly n vectors, checking either condition is sufficient. If n vectors are independent, they automatically span; if they span, they are automatically independent.

Example 14.6 (Using the count)**.** Are $\begin{bmatrix}2\\1\\0\end{bmatrix}, \begin{bmatrix}0\\3\\1\end{bmatrix}, \begin{bmatrix}1\\0\\2\end{bmatrix}$ a basis of $\mathbb{R}^3$? There are exactly three of them in a 3-dimensional space, so by Corollary 14.1(c) it is enough to check independence. Eliminate:

$$\begin{bmatrix}2&0&1\\1&3&0\\0&1&2\end{bmatrix} \xrightarrow{R_2-\frac{1}{2}R_1} \begin{bmatrix}2&0&1\\0&3&-\frac{1}{2}\\0&1&2\end{bmatrix} \xrightarrow{R_3-\frac{1}{3}R_2} \begin{bmatrix}2&0&1\\0&3&-\frac{1}{2}\\0&0&\frac{13}{6}\end{bmatrix}.$$

Three pivots, so independent, so (no spanning check needed) a basis.

Walking question

Convince yourself geometrically of each of the four parts of Corollary 14.1 using the principle from Theorem 14.1: a space cannot contain more independent directions than its dimension, and fewer vectors cannot span the space. Which inequality in Lemma 14.1 establishes this balance?

14.7 Coordinates in a basis

Because a basis is linearly independent and spans V, every vector has a unique representation as a linear combination of basis vectors. The coefficients in this representation form its **coordinates**.

Definition 14.5 (Coordinates)**.** Let $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ be a basis of V. For $\mathbf{w} \in V$ there are unique scalars $c_1, \ldots, c_n$ with $\mathbf{w} = c_1\mathbf{b}_1 + \cdots + c_n\mathbf{b}_n$. The vector

$$[\mathbf{w}]_{\mathcal{B}} = \begin{bmatrix}c_1\\\vdots\\c_n\end{bmatrix}$$

is the **coordinate vector** of $\mathbf{w}$ with respect to $\mathcal{B}$.

Uniqueness is precisely the independence argument from Section 14.2: two addresses for the same point would subtract to a nontrivial dependence, which a basis forbids. So coordinates are well-defined, and finding them is just solving a square system.

Example 14.7 (Coordinates in the frame $\{\mathbf{b}_1, \mathbf{b}_2\}$)**.** Use $\mathbf{b}_1 = \begin{bmatrix}1\\1\end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix}1\\-1\end{bmatrix}$ from Example 14.5, and find the coordinates of $\mathbf{w} = \begin{bmatrix}2\\3\end{bmatrix}$. We need $c_1\mathbf{b}_1 + c_2\mathbf{b}_2 = \mathbf{w}$, i.e.

$$\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\begin{bmatrix}c_1\\c_2\end{bmatrix} = \begin{bmatrix}2\\3\end{bmatrix}.$$

Adding the two equations: $2c_1 = 5 \Rightarrow c_1 = \frac{5}{2}$. Subtracting: $2c_2 = -1 \Rightarrow c_2 = -\frac{1}{2}$. So

$$[\mathbf{w}]_{\mathcal{B}} = \begin{bmatrix}\frac{5}{2}\\ -\frac{1}{2}\end{bmatrix}, \qquad \text{meaning} \qquad \tfrac{5}{2}\begin{bmatrix}1\\1\end{bmatrix} - \tfrac{1}{2}\begin{bmatrix}1\\-1\end{bmatrix} = \begin{bmatrix}2\\3\end{bmatrix}. \checkmark$$

The point $\mathbf{w}$ never moved. In the standard frame its address is $(2, 3)$; in the $\mathcal{B}$ frame its address is $(\frac{5}{2}, -\frac{1}{2})$. Same point, different ruler.

Focus

To find coordinates, put the basis vectors in the columns of $\mathbf{B}$ and solve $\mathbf{B}\,[\mathbf{w}]_{\mathcal{B}} = \mathbf{w}$. Because $\mathcal{B}$ is a basis, $\mathbf{B}$ is invertible, so $[\mathbf{w}]_{\mathcal{B}} = \mathbf{B}^{-1}\mathbf{w}$ and the answer is unique. "Change of basis" is multiplication by $\mathbf{B}^{-1}$ — the full story waits in Chapter 31, but the seed is right here.

Connection to other math

This is the idea that powers *Fourier series* and *wavelets*. There, the "vectors" are functions and the basis is a family of sines and cosines (or little wavelet bumps). The Fourier coefficients of a signal are exactly its *coordinates* in that basis — exactly Definition 14.5, carried into an infinite-dimensional space. Independence becomes orthogonality, solving $\mathbf{Bc} = \mathbf{w}$ becomes an integral, and "every function has a unique coordinate list" becomes the completeness of the Fourier basis. The discrete cosine transform inside every JPEG is this chapter, applied to 8×8 blocks of pixels.

14.8 A basis for the column space

Now we put our new theory to work on the two subspaces that govern all of linear algebra. In Chapter 12, we introduced the column space $\text{col}(\mathbf{A})$ as the span of the columns of $\mathbf{A}$ — everything the matrix transformation can output. In Chapter 13, we used the pivot columns to pick out the non-redundant directions.

The columns of $\mathbf{A}$ span $\text{col}(\mathbf{A})$ by definition, but if some columns are combinations of earlier ones, the set contains redundant vectors. To reduce that spanning set to a basis, eliminate the redundant vectors while preserving the span. Elimination identifies precisely which columns to keep.

Theorem 14.3 (Pivot columns are a basis for $\text{col}(\mathbf{A})$)**.** *The pivot columns of* $\mathbf{A}$ *— the columns of the* original *matrix in the positions where its row-reduced form has pivots — form a basis for* $\text{col}(\mathbf{A})$*. Hence* $\dim \text{col}(\mathbf{A})$ *equals the number of pivots, which is the rank of* $\mathbf{A}$*.*

Why it works. Row operations do not change which combinations of columns give zero: if $\mathbf{R}$ is the reduced form of $\mathbf{A}$, then $\mathbf{Ac} = \mathbf{0}$ and $\mathbf{Rc} = \mathbf{0}$ have the same solutions $\mathbf{c}$. In $\mathbf{R}$ the pivot columns are clearly independent (each carries a pivot the others lack) and each free column is visibly a combination of earlier pivot columns. The same dependence relations $\mathbf{c}$ hold among the columns of $\mathbf{A}$ — so the pivot columns of $\mathbf{A}$ are independent, and the free columns of $\mathbf{A}$ are combinations of them. Independent and spanning the column space: a basis. (A common pitfall is discussed in the box below.) □

Example 14.8 (Reading a column-space basis)**.** Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 & 3 \\ 2 & 4 & 3 & 7 \\ 1 & 2 & 2 & 4 \end{bmatrix}.$$

Eliminate. Subtract $2R_1$ from R_2 and R_1 from R_3:

$$\begin{bmatrix} 1 & 2 & 1 & 3 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \xrightarrow{R_3 - R_2} \begin{bmatrix} 1 & 2 & 1 & 3 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Pivots sit in columns 1 and 3. So a basis for $\operatorname{col}(\mathbf{A})$ is

$$\left\{ \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} \right\} \qquad \text{(columns 1 and 3 of the } \textit{original}\ \mathbf{A}\text{)},$$

and $\dim \operatorname{col}(\mathbf{A}) = \operatorname{rank}(\mathbf{A}) = 2$. The column space is a plane in $\mathbb{R}^3$. Columns 2 and 4 were redundant: indeed column $2 = 2\times$ column 1, and you can check column $4 = 2(\text{col } 1) + (\text{col } 3)$.

Common trap

Take the pivot columns from the **original** $\mathbf{A}$, *never* the columns of the row-reduced $\mathbf{R}$. Row operations preserve the column space's *dimension* and the *dependence pattern*, but they scramble the actual column vectors — $\operatorname{col}(\mathbf{R})$ is usually a different subspace from $\operatorname{col}(\mathbf{A})$. In Example 14.8, $\mathbf{R}$'s columns all have a 0 in the last slot, so $\operatorname{col}(\mathbf{R})$ is the xy-plane; but $\operatorname{col}(\mathbf{A})$ is a *tilted* plane. Same dimension, different plane. Use $\mathbf{A}$'s columns.

Here is what that "same dimension, different plane" caution looks like. Both planes are 2-dimensional, but they sit in $\mathbb{R}^3$ at different tilts: the column space of $\mathbf{A}$ is spanned by the two genuine pivot columns $\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}$, while the row-reduced columns flatten onto the floor.

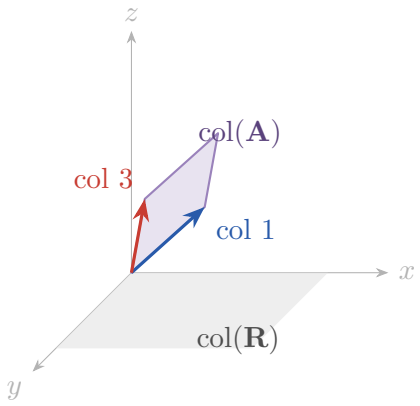


Figure 14.6: Two planes of the *same* dimension in $\mathbb{R}^3$. The row-reduced matrix $\mathbf{R}$ has all its columns lying flat in the xy-plane (gray floor), so col($\mathbf{R}$) is the floor. The original $\mathbf{A}$ spans a *tilted* plane (purple) carried by its two true pivot columns (blue and red). Equal dimension, different subspace — which is why a column-space basis must come from $\mathbf{A}$, not from $\mathbf{R}$.

14.9 A basis for the nullspace

The nullspace null($\mathbf{A}$) is the set of all vectors $\mathbf{x}$ with $\mathbf{Ax} = \mathbf{0}$ — every combination of columns that cancels out to zero. In Chapter 13, you solved this system by identifying free variables and producing the **special solutions**: set one free variable to 1, set the remaining free variables to 0, and solve for the pivot variables.

The special solutions derived from row reduction form a basis for null($\mathbf{A}$).

Theorem 14.4 (Special solutions are a basis for null($\mathbf{A}$))**.** *If* $\mathbf{A}$ *has* r *pivots and* f *free columns (so the number of columns is* $n = r + f$*), then the* f *special solutions form a basis for* null($\mathbf{A}$)*. Hence*

$$\dim \text{null}(\mathbf{A}) = f = n - r = (\#\textit{columns}) - \text{rank}(\mathbf{A}).$$

Why it works. They span. Every solution of $\mathbf{Ax} = \mathbf{0}$ is determined by its free-variable values, and the general solution is exactly the combination of the special solutions weighted by those free values — that is how we built them. *They are independent.* Look at the free-variable slots: special solution i has a 1 in slot i and 0 in the other free slots. If a combination $\sum_i c_i \mathbf{s}_i = \mathbf{0}$, then reading off free slot i gives $c_i = 0$. So all c_i vanish: independent. Independent and spanning the nullspace: a basis. □

The pattern of 1's and 0's in the free-variable slots guarantees that the special solutions are linearly independent by construction.

Example 14.9 (Reading a nullspace basis)**.** Reuse $\mathbf{A}$ from Example 14.8, whose reduced form (after one more cleanup, $R_1 - R_2$, to clear the pivot column) is

$$\mathbf{R} = \begin{bmatrix} 1 & 2 & 0 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Pivot variables: x_1, x_3. Free variables: x_2, x_4. The equations are

$$x_1 + 2x_2 + 2x_4 = 0, \qquad x_3 + x_4 = 0.$$

Special solution 1 ($x_2 = 1,\ x_4 = 0$): $x_3 = 0$, $x_1 = -2$, giving $\mathbf{s}_1 = (-2, 1, 0, 0)$. **Special solution 2** ($x_2 = 0,\ x_4 = 1$): $x_3 = -1$, $x_1 = -2$, giving $\mathbf{s}_2 = (-2, 0, -1, 1)$. So

$$\operatorname{null}(\mathbf{A}) = \operatorname{span}\left\{ \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} -2 \\ 0 \\ -1 \\ 1 \end{bmatrix} \right\}, \qquad \dim \operatorname{null}(\mathbf{A}) = 2.$$

Sanity check the bookkeeping: $n = 4$ columns, $r = 2$ pivots, so $f = n - r = 2$ free variables — exactly the two basis vectors we found. And $2 + 2 = 4$: the dimension of the column space plus the dimension of the nullspace equals the number of columns. That balance is the *Rank–Nullity Theorem*, which we formalize in the next chapter.

Modern application

Why data scientists count dimensions. A data matrix $\mathbf{X}$ stores one example per row and one feature per column. If two features are linearly dependent — say `height_cm` and `height_in`, or `revenue`, `cost`, and `profit` = `revenue` − `cost` — then those columns are redundant and $\operatorname{rank}(\mathbf{X})$ is strictly less than the number of features. In a linear regression this redundancy makes the normal equations (the system $\mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{w} = \mathbf{X}^\mathsf{T}\mathbf{y}$ used to compute best-fit weights) *singular* (non-invertible): there is an entire nullspace of weight vectors that fit the data equally well, so the "best coefficients" are not unique and the fit becomes numerically unstable.

The cure is directly in this chapter: find a *basis* for the column space of $\mathbf{X}$ (a maximal independent set of features), keep those, and drop the rest. The "effective number of independent features" is literally $\operatorname{rank}(\mathbf{X})$, and the redundant directions you delete are spanned by the special solutions in $\operatorname{null}(\mathbf{X})$. These concepts extend directly to singular value decomposition and principal component analysis in later chapters.

14.10 Bases in other vector spaces

Nothing in Definition 14.3 is restricted to $\mathbb{R}^n$. Independence, spanning, basis, and dimension apply to *any* vector space, giving these concepts their power across mathematics.

Example 14.10 (Polynomials of degree ≤ 2)**.** Let $V = \mathcal{P}_2$, the polynomials $a + bx + cx^2$. A natural basis is $\{1, x, x^2\}$: any polynomial is a unique combination of them (spanning + independence), so $\dim \mathcal{P}_2 = 3$. The coordinate vector of $3 - 5x + 2x^2$ in this basis is $(3, -5, 2)$. Different basis, different coordinates: in the basis $\{1, x - 1, (x - 1)^2\}$ (a Taylor-at-1 frame), the *same* polynomial gets a different coordinate triple, but it is still a 3-dimensional space. With a choice of basis, $\mathcal{P}_2$ is isomorphic to $\mathbb{R}^3$: coordinate vectors allow problems in abstract vector spaces to be solved using matrix computations in $\mathbb{R}^n$.

Example 14.11 (A two-dimensional space of matrices)**.** The symmetric 2×2 matrices with trace zero, $\begin{bmatrix} a & b \\ b & -a \end{bmatrix}$, form a vector space. A basis is

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

since every such matrix is a times the first plus b times the second, uniquely. Dimension 2. (Physics readers: these are two of the Pauli matrices, and this 2-dimensional space is where qubit observables live.)

Focus

The machinery is universal. Whenever you can add and scale — arrows, polynomials, matrices, functions, signals — you can ask "what is a basis?" and "what is the dimension?" The *answers* differ, but the *questions*, and the elimination-style tools for answering them, are always the ones in this chapter.

14.11 Putting it together

Pause

A quick self-check before the exercises. Make sure you can clearly explain each concept:

- **Independent** = no redundant vector = only the trivial combination gives $\mathbf{0}$ = a pivot in every column = null = $\{\mathbf{0}\}$.
- **Spans** $\mathbb{R}^n$ = reaches every $\mathbf{b}$ = a pivot in every row.
- **Basis** = independent + spanning = a minimal reaching set = a maximal fitting set = unique coordinates for every vector.
- **Dimension** = size of any basis (well-defined) = number of pivots, for a column space.
- **Column-space basis** = pivot columns of the *original* $\mathbf{A}$. **Nullspace basis** = the special solutions.

Elimination on $\mathbf{A}$ splits its columns into pivot columns and free columns. The pivot columns, taken from the original matrix, are a basis for the column space; their count is the rank. The free columns each donate one special solution, and those are a basis for the nullspace; their count is $n - r$. Two subspaces, two bases, two dimensions — all obtained from a single row reduction. In the next chapter, these subspaces and their counterparts for $\mathbf{A}^\mathsf{T}$ combine to form the Four Fundamental Subspaces.

Connection to other math

We can now determine the exact intrinsic size of any subspace ($\dim V$) and guarantee that every vector has a unique coordinate address relative to any chosen basis. The pivot columns of $\mathbf{A}$ give an explicit basis for $\text{col}(\mathbf{A})$, and the special solutions give an explicit basis for $\text{null}(\mathbf{A})$.

However, a general basis can be arbitrarily crooked and skewed. If two basis vectors point in nearly the same direction, computing coordinates requires inverting an ill-conditioned matrix (a matrix where tiny errors in the input blow up into huge inaccuracies in the output). In addition, a general basis provides no built-in concept of perpendicularity, distance, or optimal geometric projection. In Chapters 15 and 16, we discover how the dimensions of the four fundamental subspaces lock together into the Fundamental Theorem of Linear Algebra. Then, in Part IV (Chapters 17–20), we cure the challenge of crooked bases by constructing orthonormal bases (coordinate frames where every vector has unit length and is perpendicular to all the

others) via the Gram–Schmidt process and QR factorization.

Exercises

Exercise 14.1 (Warmup: parallel or not)**.** Decide whether each pair is independent. (a) $\begin{bmatrix} 2 \\ -1 \end{bmatrix}, \begin{bmatrix} -6 \\ 3 \end{bmatrix}$. (b) $\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. (c) $\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 4 \\ 7 \end{bmatrix}$. For each, state the reason in one sentence using the word "parallel" or "zero."

Exercise 14.2 (Run the test)**.** Use elimination to decide whether $\begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix}$ are independent. If they are dependent, find an explicit nontrivial combination equal to $\mathbf{0}$ and verify it.

Exercise 14.3 (Count first, then check)**.** You are handed four vectors in $\mathbb{R}^3$. Without any computation, what can you conclude about their independence, and which theorem says so? Now: are five vectors enough to *span* $\mathbb{R}^3$? Could three? Could two? Justify each with Corollary 14.1.

Exercise 14.4 (Build a basis by trimming)**.** The set $\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$ spans some subspace of $\mathbb{R}^3$. Find a basis for that subspace by selecting pivot columns, and state its dimension. Is the subspace all of $\mathbb{R}^3$, a plane, or a line?

Exercise 14.5 (Coordinates)**.** With the basis $\mathbf{b}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \mathbf{b}_2 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$ of $\mathbb{R}^2$, find $[\mathbf{w}]_{\mathcal{B}}$ for $\mathbf{w} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$. Then find the vector whose $\mathcal{B}$-coordinates are $(1, -1)$. Sketch both in the $\mathcal{B}$ frame.

Exercise 14.6 (Column space and nullspace together)**.** For

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 1 & 4 \\ 2 & 6 & 1 & 7 \\ 1 & 3 & 0 & 3 \end{bmatrix},$$

find a basis for $\operatorname{col}(\mathbf{A})$ and a basis for $\operatorname{null}(\mathbf{A})$. Verify that $\dim \operatorname{col}(\mathbf{A}) + \dim \operatorname{null}(\mathbf{A})$ equals the number of columns.

Exercise 14.7 (A basis outside $\mathbb{R}^n$)**.** Show that $\{1 + x,\ 1 - x,\ x^2\}$ is a basis of $\mathcal{P}_2$ (polynomials of degree ≤ 2). Then find the coordinates of $p(x) = 4 + 2x + 3x^2$ in this basis.

Exercise 14.8 (Extend to a basis)**.** The vectors $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$ are independent in $\mathbb{R}^3$. Find a third vector that makes them a basis of $\mathbb{R}^3$, and prove your choice works. (Hint: you only need to add one direction not already in their span; the count then guarantees a basis.)

Exercise 14.9 (Proof: a spanning set can be shrunk to a basis)**.** Suppose $\mathbf{v}_1, \ldots, \mathbf{v}_k$ span a space V but are dependent. Prove that some $\mathbf{v}_i$ is a combination of the others, and that deleting it leaves a set that still spans V. (This is the trimming process; iterating it reaches a basis.)

Exercise 14.10 (Proof: independence and uniqueness are the same thing)**.** Prove that $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are linearly independent if and only if every vector in $\operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ has *exactly one* expression as a combination $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$. (One direction is the argument in Section 14.2; supply both.)

Exercise 14.11 (Challenge: dimension by two routes)**.** Let $W = \operatorname{span}\{\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3\}$ where $\mathbf{u}_1 = \begin{bmatrix}1\\2\\0\\1\end{bmatrix}, \mathbf{u}_2 = \begin{bmatrix}2\\4\\1\\3\end{bmatrix}, \mathbf{u}_3 = \begin{bmatrix}0\\0\\1\\1\end{bmatrix}$ in $\mathbb{R}^4$. (a) Find $\dim W$ by selecting pivot columns. (b) Independently, write W as the nullspace of some matrix — i.e. find the linear equations cutting out W — and recompute $\dim W$ from that description as $4 -$ (number of independent equations). Confirm the two answers agree, and explain in a sentence why they must.

Chapter 15

The Four Fundamental Subspaces

In Chapter 14, we discovered that every subspace has an unambiguous intrinsic size: its **dimension**, counted by the number of vectors in any basis. We saw how Gaussian elimination systematically partitions the n columns of an $m \times n$ matrix into r pivot columns and $n - r$ free columns. That gave us explicit bases and exact dimension theorems for two geometric spaces: the column space $\text{col}(\mathbf{A}) \subseteq \mathbb{R}^m$ with dimension r, and the nullspace $\text{null}(\mathbf{A}) \subseteq \mathbb{R}^n$ with dimension $n - r$.

Now we look at the matrix through its dual structure: every matrix has rows as well as columns. Transposing $\mathbf{A}$ into an $n \times m$ matrix $\mathbf{A}^\mathsf{T}$ introduces two companion subspaces. Together, these form the *four fundamental subspaces*: two in the input space $\mathbb{R}^n$, and two in the output space $\mathbb{R}^m$.

In each space, the two subspaces are orthogonal complements whose dimensions add up to the dimension of the ambient space ($r + (n - r) = n$ in $\mathbb{R}^n$, and $r + (m - r) = m$ in $\mathbb{R}^m$). Gilbert Strang assembled these four spaces into a unified diagram often called "the big picture of linear algebra." Every matrix takes an r-dimensional subspace of the domain and maps it invertibly onto an r-dimensional subspace of the codomain, while sending the orthogonal complement to zero.

That is the central geometric structure. The rest of this chapter develops each space and its properties step by step.

15.1 Two spaces for the inputs, two for the outputs

Let $\mathbf{A}$ be an $m \times n$ matrix, representing a linear map from $\mathbb{R}^n$ to $\mathbb{R}^m$. There are two ambient spaces: the input space (domain) $\mathbb{R}^n$ and the output space (codomain) $\mathbb{R}^m$. Each contains exactly two of the fundamental subspaces, and in each space the two are *orthogonal complements* of each other: every vector in one meets every vector in the other at a right angle (orthogonality), and their dimensions sum to the dimension of the ambient space (complements).

Focus

The four fundamental subspaces of an $m \times n$ matrix $\mathbf{A}$ are:

- the **column space** $\text{col}(\mathbf{A}) \subseteq \mathbb{R}^m$ – all reachable outputs $\mathbf{Ax}$;
- the **nullspace** $\text{null}(\mathbf{A}) \subseteq \mathbb{R}^n$ – all inputs mapped to zero by $\mathbf{A}$;
- the **row space** $\text{row}(\mathbf{A}) = \text{col}(\mathbf{A}^\mathsf{T}) \subseteq \mathbb{R}^n$ – the span of the rows;

- the **left nullspace** $\text{null}(\mathbf{A}^\mathsf{T}) \subseteq \mathbb{R}^m$ – all output directions orthogonal to $\text{col}(\mathbf{A})$, satisfying $\mathbf{A}^\mathsf{T}\mathbf{y} = \mathbf{0}$.

Two live in $\mathbb{R}^n$ (column-vector inputs): the row space and the nullspace. Two live in $\mathbb{R}^m$ (column-vector outputs): the column space and the left nullspace.

Intuition

A matrix $\mathbf{A}$ defines a linear mapping from $\mathbb{R}^n$ to $\mathbb{R}^m$. Inside $\mathbb{R}^n$, every input vector decomposes uniquely into two orthogonal components: one in the row space and one in the nullspace. $\mathbf{A}$ maps vectors in the nullspace to $\mathbf{0}$, while mapping the row space one-to-one and onto the column space in $\mathbb{R}^m$. Any vector in $\mathbb{R}^m$ outside the column space lies in the left nullspace and cannot be reached by $\mathbf{A}$.

Before drawing the big picture, we build each space carefully and learn to read its dimension and a basis straight off the elimination of $\mathbf{A}$.

15.2 The column space: where the outputs live

Recall the definition. The column space is everything you can build out of the columns of $\mathbf{A}$:

$$\text{col}(\mathbf{A}) \;=\; \{\, \mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n \,\} \;=\; \text{span}(\text{columns of } \mathbf{A}) \;\subseteq\; \mathbb{R}^m.$$

The equation $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a solution exactly when $\mathbf{b}$ lies in this space. The column space is the set of *achievable right-hand sides*.

Example 15.1. Take

$$\mathbf{A} \;=\; \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 8 \\ 1 & 2 & 5 \end{bmatrix}.$$

Column 2 is exactly $2\times$ column 1, so it adds nothing new. Columns 1 and 3 point in different directions. The column space is therefore the plane through the origin in $\mathbb{R}^3$ spanned by $\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 3 \\ 8 \\ 5 \end{bmatrix}$. It is two-dimensional: a flat sheet inside three-space. The vector $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, say, is *not* on that sheet, so $\mathbf{A}\mathbf{x} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ has no solution.

How did we know columns 1 and 3 were the "real" ones? Elimination tells us. The pivot columns of $\mathbf{A}$ – the columns that end up holding a pivot after row reduction – are a basis for the column space.

Formal

Basis for the column space. Row-reduce $\mathbf{A}$ to find which columns contain pivots. The corresponding columns *of the original* $\mathbf{A}$ form a basis for $\text{col}(\mathbf{A})$. Therefore $\dim \text{col}(\mathbf{A}) = r$,

the number of pivots, which we call the **rank**.

Common trap

A subtle but classic mistake: after row reduction you must take the pivot columns of the *original* matrix, not the pivot columns of the reduced matrix. Row operations change the column space, because mixing the rows moves the columns around in $\mathbb{R}^m$. What row operations *do* preserve is *which* columns are dependent on which – so the *positions* of the pivots are trustworthy, but the actual basis vectors must come from $\mathbf{A}$ itself.

Here is the trap in miniature.

Compute it yourself

Reduce $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$. You get $\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$. The pivot is in column 1. The column space of the reduced matrix is the span of $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ (the x-axis). But the column space of $\mathbf{A}$ is the span of $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (the diagonal line). Those are different lines: they share the same dimension ($r = 1$) and the same pivot positions, but you must read the basis vector $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ off the original $\mathbf{A}$.

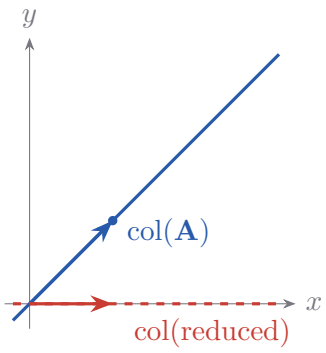


Figure 15.1: Row reduction preserves pivot *positions* but not the column space itself. For $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ the true column space is the diagonal line (blue); the reduced matrix has the x-axis (red dashed). Always read the basis off the original columns.

Pause

Pause and think: Suppose $\mathbf{A}$ is a 4×3 matrix. What is the largest possible dimension its column space can have? Could it ever fill the output room $\mathbb{R}^4$?

The column space is spanned by 3 columns living in $\mathbb{R}^4$. By Chapter 14's dimension theorem, 3 vectors cannot span a 4-dimensional space: $\dim \text{col}(\mathbf{A}) \leq 3$. So $\text{col}(\mathbf{A})$ is at most a 3D subspace inside $\mathbb{R}^4$. There is *always* at least one dimension of unreachable space left over in the output room.

15.3 The nullspace: inputs mapped to zero

The nullspace collects every input that $\mathbf{A}$ sends to zero:

$$\operatorname{null}(\mathbf{A}) \;=\; \{\,\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\,\} \;\subseteq\; \mathbb{R}^n.$$

This is a subspace because if $\mathbf{A}\mathbf{x} = \mathbf{0}$ and $\mathbf{A}\mathbf{y} = \mathbf{0}$ then $\mathbf{A}(c\mathbf{x} + d\mathbf{y}) = \mathbf{0}$ too. Geometrically, moving along a nullspace direction does not change the output: $\mathbf{A}(\mathbf{x} + \mathbf{n}) = \mathbf{A}\mathbf{x}$ for any $\mathbf{n} \in \operatorname{null}(\mathbf{A})$.

We find a basis by solving $\mathbf{A}\mathbf{x} = \mathbf{0}$ with Gaussian elimination. Setting each free variable to 1 while setting all other free variables to 0 produces one *special solution*, and the special solutions form a basis for $\operatorname{null}(\mathbf{A})$.

Example 15.2. Continue with $\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 8 \\ 1 & 2 & 5 \end{bmatrix}$ from Example 15.1. Eliminate:

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 8 \\ 1 & 2 & 5 \end{bmatrix} \xrightarrow{R_2 - 2R_1,\, R_3 - R_1} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 2 \\ 0 & 0 & 2 \end{bmatrix} \xrightarrow{R_3 - R_2} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}.$$

Pivots sit in columns 1 and 3; column 2 is free. Set the free variable $x_2 = 1$. Back-substitute: the second row gives $2x_3 = 0$, so $x_3 = 0$; the first row gives $x_1 + 2x_2 + 3x_3 = 0$, so $x_1 = -2$. The special solution is

$$\mathbf{s} = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}, \qquad \operatorname{null}(\mathbf{A}) = \operatorname{span}\Big\{ \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} \Big\}.$$

Check: $\mathbf{A}\mathbf{s} = \begin{bmatrix} 1\cdot(-2) + 2\cdot 1 + 3\cdot 0 \\ 2\cdot(-2) + 4\cdot 1 + 8\cdot 0 \\ 1\cdot(-2) + 2\cdot 1 + 5\cdot 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$. One free variable, so the nullspace is a single line in $\mathbb{R}^3$: $\dim \operatorname{null}(\mathbf{A}) = 1$.

Notice the bookkeeping that just happened. There were $n = 3$ columns. Two held pivots ($r = 2$), one was free ($n - r = 1$), and that free variable became the one dimension of the nullspace. This is no accident.

Formal

Dimension of the nullspace. For an $m \times n$ matrix of rank r, the number of free variables is $n - r$, and each gives an independent special solution. Why are they automatically independent? Because each special solution has a 1 in its own free-variable coordinate and a 0 in all other free-variable coordinates. Chapter 14 guarantees that $n - r$ linearly independent vectors form a basis of dimension $n - r$. Hence

$$\dim \operatorname{null}(\mathbf{A}) \;=\; n - r.$$

This is the *nullity* of $\mathbf{A}$, and the identity $\operatorname{rank} + \operatorname{nullity} = n$ is the celebrated **rank–nullity theorem**.

Walking question

Suppose $\mathbf{A}$ is a 3×5 matrix. On a walk, picture its 5-dimensional input space: if $\mathbf{A}$ crushes a 2-dimensional plane to zero ($\dim \text{null}(\mathbf{A}) = 2$), how many independent input directions survive into the output? Could $\text{null}(\mathbf{A})$ ever be only 1-dimensional for a 3×5 matrix, or do 3 rows force at least 2 free variables?

15.4 The row space: inputs that govern the output

The rows of $\mathbf{A}$ are vectors with n components, so they live in the domain $\mathbb{R}^n$. Their span is the row space:

$$\text{row}(\mathbf{A}) \;=\; \text{span}(\text{rows of } \mathbf{A}) \;=\; \text{col}(\mathbf{A}^{\mathsf{T}}) \;\subseteq\; \mathbb{R}^n.$$

Each component of the output is given by a dot product with a row of $\mathbf{A}$: $(\mathbf{A}\mathbf{x})_i = (\text{row } i) \cdot \mathbf{x}$. The output is built entirely from dot products of $\mathbf{x}$ against the rows. If $\mathbf{x}$ is orthogonal to every row, every dot product is zero and the output vanishes – that is exactly the nullspace condition. So the row space and the nullspace are orthogonal complements inside $\mathbb{R}^n$, a claim we prove in the next section.

How big is the row space, and what is a basis? Here the story is friendlier than for the column space.

Formal

Basis for the row space. Row operations are reversible combinations of rows, so they never change the span of the rows. The nonzero rows of the row-reduced matrix $\mathbf{R}$ span $\text{row}(\mathbf{A})$. Why are they linearly independent? Because each nonzero row has a leading pivot with only zeros below it in that column—the staircase echelon pattern prevents any row from being canceled by the rows beneath it. Thus the r nonzero rows of $\mathbf{R}$ form an explicit basis for $\text{row}(\mathbf{A}) = \text{row}(\mathbf{R})$, and

$$\dim \text{row}(\mathbf{A}) \;=\; r \;=\; \dim \text{col}(\mathbf{A}).$$

Unlike the column space, you *may* use the rows of the reduced matrix directly.

That last equation is a striking fact: the number of independent rows equals the number of independent columns. The next chapter treats it head-on, but you have already proved it – both counts equal r, the number of pivots.

Example 15.3. Same $\mathbf{A}$. Its reduced rows (from Example 15.2, after scaling the pivot to 1) are

$$\text{row}(\mathbf{A}) = \text{span}\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \; \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

A two-dimensional plane in $\mathbb{R}^3$. And look: the nullspace direction $\begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$ is orthogonal to both row vectors. Dot with $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$: $-2 + 2 + 0 = 0$. Dot with $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$: $0 + 0 + 0 = 0$. The plane (row space) and the line (nullspace) meet at a perfect right angle, and together $2 + 1 = 3$ they fill all of $\mathbb{R}^3$.

Figure 15.2 shows this split of the input room: the row space is a plane, the nullspace is the line through it at a right angle, and every input vector decomposes uniquely into a piece in each.

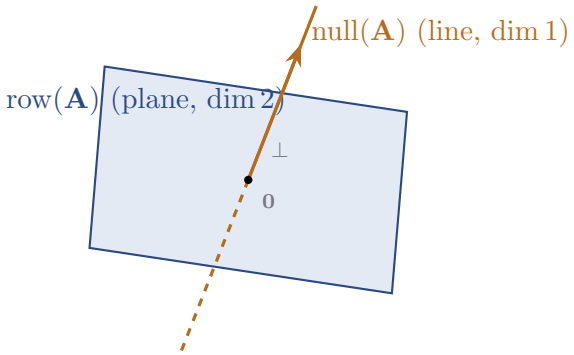


Figure 15.2: The input room $\mathbb{R}^3$ for the running matrix (Example 15.3), split into perpendicular partners: the row space is the plane spanned by $\begin{bmatrix}1\\2\\3\end{bmatrix}$ and $\begin{bmatrix}0\\0\\1\end{bmatrix}$, and the nullspace is the line through $\begin{bmatrix}-2\\1\\0\end{bmatrix}$, which stands at a right angle to that plane. Dimensions $2+1=3$ fill the room.

Before/after Strang

Strang writes the row space as $C(\mathbf{A}^\mathsf{T})$ rather than $\text{row}(\mathbf{A})$ so that all four subspaces use just two symbols, C and N: $C(\mathbf{A}^\mathsf{T})$ and $N(\mathbf{A})$ in $\mathbb{R}^n$, paired with $C(\mathbf{A})$ and $N(\mathbf{A}^\mathsf{T})$ in $\mathbb{R}^m$. That notation makes it immediately visible why transposing $\mathbf{A}$ swaps the input picture with the output picture.

15.5 The left nullspace: the unreachable output directions

The fourth space is the one students forget, because naming it requires a transpose. The *left nullspace* is the nullspace of $\mathbf{A}^\mathsf{T}$:

$$\text{null}(\mathbf{A}^\mathsf{T}) \;=\; \{\,\mathbf{y}\in\mathbb{R}^m : \mathbf{A}^\mathsf{T}\mathbf{y}=\mathbf{0}\,\} \;\subseteq\; \mathbb{R}^m.$$

Why "left"? Because $\mathbf{A}^\mathsf{T}\mathbf{y}=\mathbf{0}$ is the same as $\mathbf{y}^\mathsf{T}\mathbf{A}=\mathbf{0}^\mathsf{T}$ – a row vector $\mathbf{y}^\mathsf{T}$ multiplying $\mathbf{A}$ from the *left* and producing the zero row. So $\mathbf{y}$ records a linear combination of the *rows* of $\mathbf{A}$ that cancels to zero. These are exactly the redundancies among the rows.

Geometrically, the left nullspace lives in the output room $\mathbb{R}^m$ and consists of the directions the column space cannot reach. If $\mathbf{y}\in\text{null}(\mathbf{A}^\mathsf{T})$, then for every input $\mathbf{x}$,

$$\mathbf{y}\cdot(\mathbf{A}\mathbf{x}) = \mathbf{y}^\mathsf{T}\mathbf{A}\mathbf{x} = (\mathbf{A}^\mathsf{T}\mathbf{y})^\mathsf{T}\mathbf{x} = \mathbf{0}^\mathsf{T}\mathbf{x} = 0.$$

So $\mathbf{y}$ is perpendicular to every possible output. The left nullspace is the orthogonal complement of the column space inside $\mathbb{R}^m$.

Formal

Dimension and basis of the left nullspace. Applying the rank–nullity bookkeeping to $\mathbf{A}^\mathsf{T}$ (which is $n \times m$ with the same rank r):

$$\dim \operatorname{null}(\mathbf{A}^\mathsf{T}) \;=\; m - r.$$

A clean way to get a basis: do elimination as $\mathbf{EA} = \mathbf{R}$, tracking the operations in $\mathbf{E}$. The rows of $\mathbf{E}$ that produce the *zero rows* of $\mathbf{R}$ are exactly a basis for the left nullspace.

Example 15.4. For our running $\mathbf{A}$, the reduction $R_2 - 2R_1$, $R_3 - R_1$, then $R_3 - R_2$ combined the rows so that the final third row was all zeros. Tracking the combination, the third row of $\mathbf{R}$ came from

$$(R_3 - R_1) - (R_2 - 2R_1) = R_1 - R_2 + R_3,$$

i.e. row 1 minus row 2 plus row 3 of $\mathbf{A}$ gives the zero row. So $\mathbf{y} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$ satisfies $\mathbf{y}^\mathsf{T}\mathbf{A} = \mathbf{0}^\mathsf{T}$. Check the first component: $1 \cdot 1 - 1 \cdot 2 + 1 \cdot 1 = 0$. Second: $1 \cdot 2 - 1 \cdot 4 + 1 \cdot 2 = 0$. Third: $1 \cdot 3 - 1 \cdot 8 + 1 \cdot 5 = 0$. Indeed $\mathbf{y}^\mathsf{T}\mathbf{A} = \mathbf{0}^\mathsf{T}$. Here $m = 3$, $r = 2$, so the left nullspace is the single line $\operatorname{span}\{ \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \}$ in $\mathbb{R}^3$, dimension $m - r = 1$. And it is orthogonal to the column space plane: dot $\begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$ with column 1 $\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ to get $1 - 2 + 1 = 0$, and with column 3 $\begin{bmatrix} 3 \\ 8 \\ 5 \end{bmatrix}$ to get $3 - 8 + 5 = 0$. Right angles everywhere.

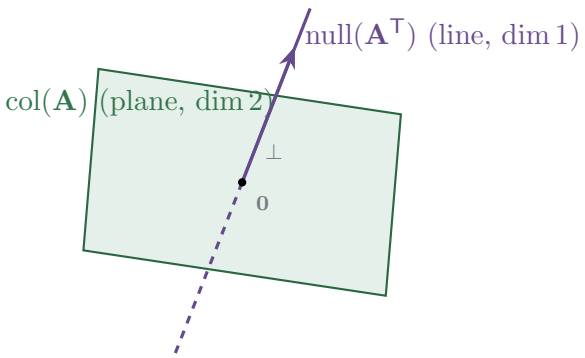


Figure 15.3: The output room $\mathbb{R}^3$ for the running matrix (Example 15.4), split into perpendicular partners: the column space is the plane spanned by $\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 3 \\ 8 \\ 5 \end{bmatrix}$, and the left nullspace is the line through $\begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$, standing at a right angle to that plane. Together $2 + 1 = 3$ dimensions fill the output room.

Pause

Pause and think: If a target output $\mathbf{b} \in \mathbb{R}^m$ has a nonzero component in the left nullspace $\text{null}(\mathbf{A}^\mathsf{T})$, can the linear system $\mathbf{Ax} = \mathbf{b}$ ever have an exact solution?

Never. Every achievable output $\mathbf{Ax}$ lives in $\text{col}(\mathbf{A})$. Because the left nullspace meets the column space at a right angle, any vector with a piece in $\text{null}(\mathbf{A}^\mathsf{T})$ sticks out of the reachable plane. No choice of input $\mathbf{x}$ can ever reach it.

Why this matters

The dimensions of the four subspaces exhibit complete symmetry. Because the matrix and its transpose share the same rank r, all four dimensions are determined by just three integers m, n, and r:

$$\dim \text{col} = r, \quad \dim \text{null}(\mathbf{A}^\mathsf{T}) = m - r \quad (\text{in } \mathbb{R}^m), \qquad \dim \text{row} = r, \quad \dim \text{null} = n - r \quad (\text{in } \mathbb{R}^n).$$

In each room the two dimensions sum to the dimension of the room: $r + (m - r) = m$ and $r + (n - r) = n$. The whole input space and the whole output space are each split cleanly into a "used" part and an "unused" part. A single rank number governs all of it.

15.6 Orthogonality: the two spaces in each room are perpendicular

We keep saying "perpendicular partners." Now we prove it, because it is the structural backbone of the big picture.

Theorem 15.1 (Orthogonality of the fundamental subspaces)**.** *For any $m \times n$ matrix $\mathbf{A}$:*

1. *In $\mathbb{R}^n$: every nullspace vector is orthogonal to every row-space vector. In symbols,* $\text{null}(\mathbf{A}) \perp \text{row}(\mathbf{A})$*, and together they fill $\mathbb{R}^n$.*
2. *In $\mathbb{R}^m$: every left-nullspace vector is orthogonal to every column-space vector. In symbols,* $\text{null}(\mathbf{A}^\mathsf{T}) \perp \text{col}(\mathbf{A})$*, and together they fill $\mathbb{R}^m$.*

Proof. Part 1. Let $\mathbf{x} \in \text{null}(\mathbf{A})$, so $\mathbf{Ax} = \mathbf{0}$. Reading $\mathbf{Ax}$ row by row, the i-th entry is $(\text{row}_i) \cdot \mathbf{x} = 0$. So $\mathbf{x}$ is orthogonal to every row, hence to every linear combination of rows, hence to the entire row space. That gives $\text{null}(\mathbf{A}) \perp \text{row}(\mathbf{A})$.

For the dimension count: $\dim \text{row} = r$ and $\dim \text{null} = n - r$. If a vector $\mathbf{v}$ belongs to both $\text{row}(\mathbf{A})$ and $\text{null}(\mathbf{A})$, it must be orthogonal to itself: $\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2 = 0$, forcing $\mathbf{v} = \mathbf{0}$. Thus the two subspaces meet only at the origin. Choose a basis of r vectors for $\text{row}(\mathbf{A})$ and a basis of $n - r$ vectors for $\text{null}(\mathbf{A})$. Together they give $r + (n - r) = n$ vectors in $\mathbb{R}^n$, and if a combination $\mathbf{u} + \mathbf{w} = \mathbf{0}$ ($\mathbf{u} \in \text{row}(\mathbf{A})$, $\mathbf{w} \in \text{null}(\mathbf{A})$), then $\mathbf{u} = -\mathbf{w}$ lies in both subspaces and hence equals $\mathbf{0}$, forcing every basis coefficient to vanish. By Corollary 14.1(c), these n independent vectors form a basis for $\mathbb{R}^n$. Thus $\text{row}(\mathbf{A})$ and $\text{null}(\mathbf{A})$ are orthogonal complements: every vector in $\mathbb{R}^n$ decomposes uniquely into a row-space piece plus a nullspace piece.

Part 2 is Part 1 applied to $\mathbf{A}^\mathsf{T}$: its row space is $\text{col}(\mathbf{A})$ and its nullspace is $\text{null}(\mathbf{A}^\mathsf{T})$, both living in $\mathbb{R}^m$. □

Intuition

Here is the cleanest way to remember which pairs are perpendicular. The nullspace is defined by "dot with every row gives zero," so the nullspace is whatever is left after you remove the row space – they are complements. Likewise the left nullspace is "dot with every column gives zero," so it is the complement of the column space. Rows pair with the nullspace; columns pair with the left nullspace. The output (col) and the unreachable directions (null($\mathbf{A}^\mathsf{T}$)) carve up $\mathbb{R}^m$; the listened-to inputs (row) and the ignored inputs (null) carve up $\mathbb{R}^n$.

Connection to other math

The phrase "orthogonal complement and the spaces fill the room" is the *Fundamental Theorem of Linear Algebra, Part 2*, and it is the finite-dimensional shadow of a deep theme in analysis. In functional analysis—the branch of mathematics that studies calculus on infinite-dimensional function spaces—the same statement is the *Fredholm alternative*: the equation $\mathbf{Ax} = \mathbf{b}$ is solvable precisely when $\mathbf{b}$ is orthogonal to every solution of the adjoint homogeneous problem $\mathbf{A}^\mathsf{T}\mathbf{y} = \mathbf{0}$ (where "adjoint" is the infinite-dimensional counterpart of the matrix transpose). That adjoint condition is exactly "$\mathbf{b} \perp$ left nullspace, i.e. $\mathbf{b} \in \text{col}(\mathbf{A})$." The four-subspace picture you are learning here scales all the way up to differential operators and underwrites the solvability theory for huge classes of differential equations.

15.7 The big picture

Now we assemble everything into the one diagram worth memorizing. Draw the input room $\mathbb{R}^n$ on the left and the output room $\mathbb{R}^m$ on the right. Inside each room, draw the two perpendicular subspaces, and show the action of $\mathbf{A}$ as arrows crossing from left to right.

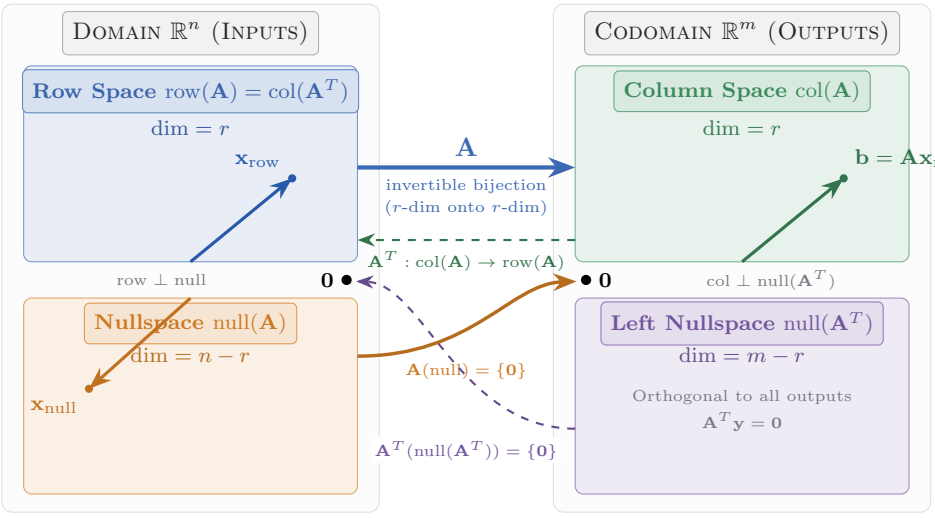


Figure 15.4: The Big Picture of Linear Algebra. The input domain $\mathbb{R}^n$ decomposes into the row space and nullspace (orthogonal complements of dimensions r and $n-r$). The codomain $\mathbb{R}^m$ decomposes into the column space and left nullspace (orthogonal complements of dimensions r and $m-r$). The matrix $\mathbf{A}$ maps the entire nullspace directly to the origin $\mathbf{0} \in \mathbb{R}^m$, while establishing an invertible, one-to-one isomorphism between the r-dimensional row space and the r-dimensional column space.

Two facts drive everything in Figure 15.4. First, **the nullspace maps to zero** by definition. Second, **the row space maps invertibly onto the column space**. Take any input $\mathbf{x} \in \mathbb{R}^n$ and split it (uniquely, by Theorem 15.1) as

$$\mathbf{x} = \mathbf{x}_{\text{row}} + \mathbf{x}_{\text{null}}, \qquad \mathbf{x}_{\text{row}} \in \text{row}(\mathbf{A}), \quad \mathbf{x}_{\text{null}} \in \text{null}(\mathbf{A}).$$

Then

$$\mathbf{A}\mathbf{x} = \mathbf{A}\mathbf{x}_{\text{row}} + \mathbf{A}\mathbf{x}_{\text{null}} = \mathbf{A}\mathbf{x}_{\text{row}} + \mathbf{0} = \mathbf{A}\mathbf{x}_{\text{row}}.$$

So the output is entirely determined by the row-space component of the input: the nullspace component does not affect the output. The map "$\mathbf{x}_{\text{row}} \mapsto \mathbf{A}\mathbf{x}_{\text{row}}$" goes from the row space (dimension r) to the column space (dimension r).

An **isomorphism** is a linear map that is both one-to-one (different inputs never collide) and onto (every target vector in the space is reached)—an invertible linear map between two vector spaces that preserves vector addition and scalar multiplication.

Theorem 15.2 (Row space to column space is an isomorphism)**.** *Restricted to the row space,* $\mathbf{A}$ *is an isomorphism from* $\text{row}(\mathbf{A})$ *onto* $\text{col}(\mathbf{A})$*: every output in* $\text{col}(\mathbf{A})$ *comes from exactly one vector in* $\text{row}(\mathbf{A})$*.*

Proof. Onto: every output $\mathbf{A}\mathbf{x}$ equals $\mathbf{A}\mathbf{x}_{\text{row}}$ by the splitting above, so every element of the column space is hit by some row-space vector.

One-to-one: suppose two row-space vectors $\mathbf{u}, \mathbf{v} \in \text{row}(\mathbf{A})$ give the same output, $\mathbf{A}\mathbf{u} = \mathbf{A}\mathbf{v}$. Then $\mathbf{A}(\mathbf{u} - \mathbf{v}) = \mathbf{0}$, so $\mathbf{u} - \mathbf{v} \in \text{null}(\mathbf{A})$. But $\mathbf{u} - \mathbf{v}$ is also in the row space (a difference of row-space vectors). A vector in both the row space and the nullspace is orthogonal to itself, so $\mathbf{u} - \mathbf{v} = \mathbf{0}$, i.e. $\mathbf{u} = \mathbf{v}$. Distinct row-space vectors give distinct outputs. □

Focus

Restricting the domain to the row space and the codomain to the column space makes every linear map invertible. By removing the nullspace from the input space and the left nullspace from the output space, $\mathbf{A}$ defines a bijective linear transformation from $\text{row}(\mathbf{A})$ to $\text{col}(\mathbf{A})$. The rank r is the intrinsic dimension of this invertible mapping.

Example 15.5. Watch the splitting in action with our $\mathbf{A}$. Take the input $\mathbf{x} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ (just turning on the free variable, plus nothing else). Decompose it. The nullspace direction is $\mathbf{n} = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$ with $\|\mathbf{n}\|^2 = 5$. Project $\mathbf{x}$ onto it (using Chapter 4's line-projection formula):

$$\mathbf{x}_{\text{null}} = \frac{\mathbf{x} \cdot \mathbf{n}}{\mathbf{n} \cdot \mathbf{n}} \mathbf{n} = \frac{(0)(-2) + (1)(1) + 0}{5} \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}.$$

Then $\mathbf{x}_{\text{row}} = \mathbf{x} - \mathbf{x}_{\text{null}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} - \frac{1}{5} \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix}$. You can verify $\mathbf{x}_{\text{row}}$ lies in the row space (it is

$\frac{2}{5}\begin{bmatrix}1\\2\\0\end{bmatrix}$, a combination of the rows) and is orthogonal to $\mathbf{n}$. Now apply $\mathbf{A}$:

$$\mathbf{Ax} = \mathbf{A}\begin{bmatrix}0\\1\\0\end{bmatrix} = \begin{bmatrix}2\\4\\2\end{bmatrix}, \qquad \mathbf{Ax}_{\text{row}} = \mathbf{A}\cdot\tfrac{1}{5}\begin{bmatrix}2\\4\\0\end{bmatrix} = \tfrac{1}{5}\begin{bmatrix}2\cdot 1+4\cdot 2\\2\cdot 2+4\cdot 4\\2\cdot 1+4\cdot 2\end{bmatrix} = \tfrac{1}{5}\begin{bmatrix}10\\20\\10\end{bmatrix} = \begin{bmatrix}2\\4\\2\end{bmatrix}.$$

Identical. The nullspace part contributed nothing; the row-space part did all the work. That is Theorem 15.2 with numbers.

Figure 15.5 illustrates this decomposition: the input $\mathbf{x}$ projects orthogonally onto the row space and onto the nullspace. $\mathbf{A}$ maps the nullspace component to zero, so the output $\mathbf{Ax}$ depends entirely on $\mathbf{x}_{\text{row}}$.

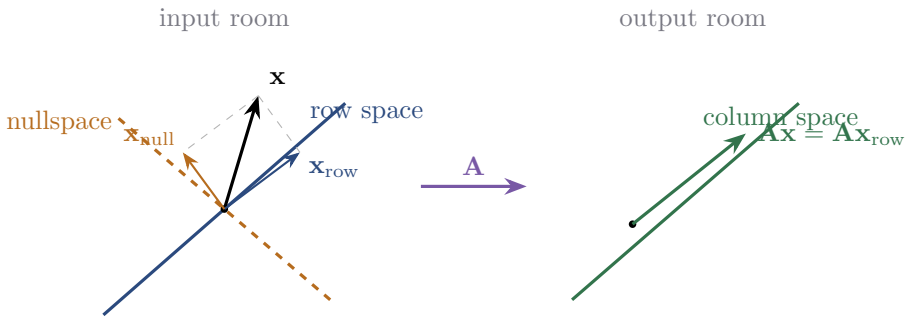


Figure 15.5: The orthogonal decomposition of Example 15.5. The input $\mathbf{x}$ resolves into a row-space component $\mathbf{x}_{\text{row}}$ and a nullspace component $\mathbf{x}_{\text{null}}$. Because $\mathbf{Ax}_{\text{null}} = \mathbf{0}$, the output depends entirely on $\mathbf{x}_{\text{row}}$ and lies in the column space.

15.8 Reading all four bases off one elimination

One row reduction hands you everything, so it is worth consolidating the recipe in one place.

Formal

The one-elimination recipe. Reduce $\mathbf{A}$ (tracking operations in $\mathbf{E}$ so that $\mathbf{EA} = \mathbf{R}$). Let the pivots sit in r columns.

- **Column space** ($\dim r$, in $\mathbb{R}^m$): the pivot columns of the *original* $\mathbf{A}$.
- **Row space** ($\dim r$, in $\mathbb{R}^n$): the nonzero rows of $\mathbf{R}$.
- **Nullspace** ($\dim n - r$, in $\mathbb{R}^n$): the special solutions, one per free column.
- **Left nullspace** ($\dim m - r$, in $\mathbb{R}^m$): the rows of $\mathbf{E}$ that produced the zero rows of $\mathbf{R}$.

Compute it yourself

Try the full recipe on

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 & 1 \\ 2 & 4 & 3 & 4 \\ 1 & 2 & 2 & 3 \end{bmatrix} \quad (m = 3,\ n = 4).$$

Eliminate to $\mathbf{R} = \begin{bmatrix} 1 & 2 & 0 & -1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ (pivots in columns 1 and 3, so $r = 2$). Now read off all four:

- **Column space**: columns 1 and 3 of $\mathbf{A}$, namely $\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}$ (dimension 2, in $\mathbb{R}^3$).
- **Row space**: $\begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 2 \end{bmatrix}$ (dimension 2, in $\mathbb{R}^4$).
- **Nullspace**: free columns are 2 and 4. Setting $x_2 = 1$ (rest free $= 0$) gives $\begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}$; setting $x_4 = 1$ gives $\begin{bmatrix} 1 \\ 0 \\ -2 \\ 1 \end{bmatrix}$ (dimension 2, in $\mathbb{R}^4$).
- **Left nullspace**: one zero row, so $\dim 1$ in $\mathbb{R}^3$. The combination $R_1 - R_2 + R_3$ of the original rows is zero, giving $\begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$.

Sanity check the dimensions: $r = 2$, $n - r = 2$ (fills $\mathbb{R}^4$), $m - r = 1$ (with $r = 2$ fills $\mathbb{R}^3$).

Common trap

Watch your rooms. The row space and nullspace are sets of 4-component vectors (they live in the *input* space $\mathbb{R}^4$); the column space and left nullspace are 3-component vectors (they live in the *output* space $\mathbb{R}^3$). A frequent error is to try to "check orthogonality" between the column space and the nullspace – but those two live in different rooms and cannot even be dotted together. Orthogonality is always *within* a room: row $\perp$ null in $\mathbb{R}^n$, column $\perp$ left-null in $\mathbb{R}^m$.

15.9 A modern application: where data lives and what is unreachable

Modern application

Recommender systems and the column space. Imagine a big matrix $\mathbf{A}$ whose rows are movies and whose columns are users, with entry A_{ij} the rating user j gave movie i. Most entries are missing; the engine's job is to fill them in. The working hypothesis behind systems like the ones that won the Netflix Prize is that tastes are *low rank*: a handful of hidden factors (how much you like comedy, action, slow burns) explain most of the ratings. "Low rank" means the rank r is small compared with m and n.

The four subspaces tell you exactly what such a model can and cannot do.

- The **column space** is the set of rating patterns the model can produce. If a real user's true ratings lie outside this r-dimensional column space, no choice of weights can reproduce them – their genuinely individual taste is, mathematically, in the **left nullspace**, perpendicular to everything the model can represent. That orthogonal leftover is precisely the part the model must give up on.
- The **nullspace** is the model's blind spot among inputs: directions in user/feature space you can change without changing a single predicted rating. Regularizers and extra data are how engineers try to shrink or pin down that nullspace so predictions become determined rather than free.

The same decomposition powers image compression, topic models for text, and the embeddings inside modern recommendation and language models: keep the r directions that carry signal (the column space), discard the rest. The rank is the budget; the four subspaces are the accounting.

Modern application

Least squares and the left nullspace, previewed. When $\mathbf{Ax} = \mathbf{b}$ has no solution – the overdetermined case at the heart of every regression – it is precisely because $\mathbf{b}$ has a piece sticking out of the column space, a piece lying in the left nullspace. The least-squares method, which you will meet soon, throws that unreachable piece away by projecting $\mathbf{b}$ onto the column space and solving for the part that *is* achievable. The left nullspace is the exact measure of "how much your data refuses to fit your model."

15.10 Worked summary example, all four at once

Example 15.6. Let

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 6 & 4 \end{bmatrix} \qquad (m = 2,\ n = 3).$$

Row 2 is twice row 1, so we expect $r = 1$. Eliminate: $R_2 - 2R_1$ gives $\mathbf{R} = \begin{bmatrix} 1 & 3 & 2 \\ 0 & 0 & 0 \end{bmatrix}$. One pivot, $r = 1$.

- **Column space** (in $\mathbb{R}^2$, $\dim 1$): pivot column 1 of $\mathbf{A}$ is $\begin{bmatrix}1\\2\end{bmatrix}$. The column space is the line through $\begin{bmatrix}1\\2\end{bmatrix}$.
- **Left nullspace** (in $\mathbb{R}^2$, $\dim m-r=1$): the operation R_2-2R_1 zeroed a row, so the combination $-2(\text{row}_1)+1(\text{row}_2)$ vanishes, i.e. $\mathbf{y}=\begin{bmatrix}-2\\1\end{bmatrix}$ gives $\mathbf{y}^\mathsf{T}\mathbf{A}=\mathbf{0}^\mathsf{T}$. Check entry by entry: $-2(1)+1(2)=0$, $-2(3)+1(6)=0$, and $-2(2)+1(4)=0$. Good. The left nullspace is the line through $\begin{bmatrix}-2\\1\end{bmatrix}$, perpendicular to the column-space direction $\begin{bmatrix}1\\2\end{bmatrix}$ (since $(-2)(1)+(1)(2)=0$). Together they fill $\mathbb{R}^2$.
- **Row space** (in $\mathbb{R}^3$, $\dim 1$): the nonzero reduced row $\begin{bmatrix}1\\3\\2\end{bmatrix}$. The line through it.
- **Nullspace** (in $\mathbb{R}^3$, $\dim n-r=2$): free columns 2, 3. Special solutions $\begin{bmatrix}-3\\1\\0\end{bmatrix}$ (set $x_2=1$) and $\begin{bmatrix}-2\\0\\1\end{bmatrix}$ (set $x_3=1$). A plane. Both are orthogonal to the row space direction $\begin{bmatrix}1\\3\\2\end{bmatrix}$: $-3+3+0=0$ and $-2+0+2=0$. The line (row space) plus the plane (nullspace) fill $\mathbb{R}^3$.

Every dimension checks: $\mathbb{R}^2=1+1$, $\mathbb{R}^3=1+2$. A rank-one matrix: all outputs are scalar multiples of a single column vector, and all row vectors are multiples of a single row vector.

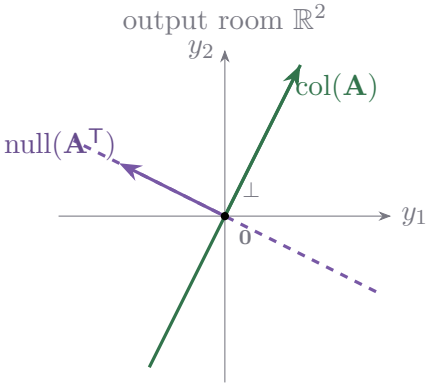


Figure 15.6: The output room $\mathbb{R}^2$ for the rank-one matrix of Example 15.6: the column space (green, the line through $\begin{bmatrix}1\\2\end{bmatrix}$) and the left nullspace (purple dashed, the perpendicular line through $\begin{bmatrix}-2\\1\end{bmatrix}$). They meet at a right angle and together fill the plane.

History and connection

For nearly a century, from Arthur Cayley and James Joseph Sylvester's work in the 1850s through the mid-twentieth century, linear algebra was taught almost exclusively through the lens of determinants (numbers computed by cross-multiplying matrix entries to test if an $n\times n$

matrix is invertible), minors (determinants of smaller square submatrices), and Cramer's rule. Rectangular $m \times n$ matrices were treated as clumsy anomalies because you cannot compute the determinant of a non-square matrix. Students spent weeks expanding tedious polynomial formulas by hand, learning almost nothing about what a transformation actually does to space.

The geometric breakthrough arrived from an unexpected direction: continuous physics. In 1903, the Swedish mathematician Ivar Fredholm was studying integral equations—equations where the unknown is an entire function rather than a list of numbers, describing physical phenomena like heat distribution and electrostatics. These act like infinite-dimensional linear systems, where determinants fail completely. Fredholm discovered what is now celebrated as the *Fredholm Alternative*: a linear system $\mathbf{Ax} = \mathbf{b}$ has an exact solution if and only if the target $\mathbf{b}$ is perpendicular to every solution of the transposed equation $\mathbf{A}^\mathsf{T}\mathbf{y} = \mathbf{0}$. In geometric terms, the outputs you can reach ($\text{col}(\mathbf{A})$) are the exact orthogonal complement of the vectors killed by the transpose:

$$\text{col}(\mathbf{A}) \;=\; (\text{null}(\mathbf{A}^\mathsf{T}))^\perp.$$

Whether a system can be solved is a test of right angles.

Despite Fredholm's discovery, textbooks clung to determinant formulas for decades. A major stumbling block was the transpose itself: students and early calculators were taught to view $\mathbf{A}^\mathsf{T}$ as a trivial typographical trick—simply "flipping rows into columns." This blind spot hid the deeper truth: the transpose represents the *adjoint* map, the machine that translates measurements in the output room back into coordinates in the input room. A second confusion arose from row reduction: row operations preserve the row space and nullspace in the input room $\mathbb{R}^n$, but they distort the column space and left nullspace in the output room $\mathbb{R}^m$.

The pedagogical revolution that placed geometry front and center was led by Gilbert Strang. In a landmark 1993 paper in the *American Mathematical Monthly* titled "The Fundamental Theorem of Linear Algebra," Strang championed the two-room diagram—two perpendicular subspaces in $\mathbb{R}^n$, two perpendicular subspaces in $\mathbb{R}^m$, joined by an invertible r-dimensional bridge—as the organizing blueprint of the entire discipline.

This four-subspace picture organizes several central tools of applied mathematics: least-squares data fitting (projecting an unreachable data vector onto the column space), the Moore–Penrose pseudoinverse, and the Singular Value Decomposition (SVD), which finds ideal perpendicular bases for all four subspaces simultaneously. Today, an active frontier lies in high dimensions and massive datasets: when data matrices have millions of noisy entries, the exact boundary between the row space and the nullspace blurs. Modern researchers rely on randomized numerical linear algebra and perturbation bounds (such as the Davis–Kahan theorem) to reliably estimate these four subspaces. At the same time, extending this clean four-subspace decomposition to higher-order *tensors* (multi-dimensional grids of data beyond 2D matrices) remains notoriously difficult: while matrix rank is computed in milliseconds via elimination, computing the rank of a 3D tensor is provably NP-hard.

15.11 What we now know

Focus

The four fundamental subspaces of an $m \times n$ matrix $\mathbf{A}$ of rank r:

Subspace	Lives in	Dimension
Column space $\text{col}(\mathbf{A})$	$\mathbb{R}^m$	r
Left nullspace $\text{null}(\mathbf{A}^\mathsf{T})$	$\mathbb{R}^m$	$m - r$
Row space $\text{row}(\mathbf{A})$	$\mathbb{R}^n$	r
Nullspace $\text{null}(\mathbf{A})$	$\mathbb{R}^n$	$n - r$

In $\mathbb{R}^n$: row space $\perp$ nullspace, dimensions add to n. In $\mathbb{R}^m$: column space $\perp$ left nullspace, dimensions add to m. And $\mathbf{A}$ maps the row space one-to-one onto the column space, collapsing the nullspace to $\mathbf{0}$.

The next chapter gives the rank itself center stage and states the Fundamental Theorem of Linear Algebra in full – you have already done most of the work.

Connection to other math

We now possess the complete structural blueprint for any linear transformation $\mathbf{A} : \mathbb{R}^n \to \mathbb{R}^m$. Both the input space $\mathbb{R}^n$ and output space $\mathbb{R}^m$ split cleanly into orthogonal complementary subspaces. Every input decomposes into a part that $\mathbf{A}$ transmits invertibly onto $\text{col}(\mathbf{A})$ and a part it maps to $\mathbf{0}$ in $\text{null}(\mathbf{A})$.

While the Big Picture reveals orthogonality and matching dimensions (r and $n - r$, r and $m - r$), it is still qualitative. It does not provide the explicit metric projection matrices to drop a vector onto $\text{col}(\mathbf{A})$ or $\text{row}(\mathbf{A})$. And when $\mathbf{b} \notin \text{col}(\mathbf{A})$, knowing that the error lives in $\text{null}(\mathbf{A}^\mathsf{T})$ does not yet tell us how to compute the best approximate solution. In Chapter 16, we prove that row rank always equals column rank ($\dim \text{row}(\mathbf{A}) = \dim \text{col}(\mathbf{A})$) for every matrix without exception, culminating in the complete Fundamental Theorem of Linear Algebra and showing how any matrix decomposes into a sum of rank-one building blocks. That sets the stage for Part IV (Chapters 17–20), where we construct orthogonal projections and least squares regression.

15.12 Exercises

Exercise 15.1 (Mental warmup: room and dimension check)**.** A matrix $\mathbf{A}$ has size 4×6 and rank $r = 3$.

(a) In your head, state the dimension of each of the four fundamental subspaces.

(b) Which two subspaces live in the input room $\mathbb{R}^6$, and what is the sum of their dimensions?

(c) Which two subspaces live in the output room $\mathbb{R}^4$, and what is the sum of their dimensions?

Exercise 15.2 (Mental warmup: which room?)**.** Let $\mathbf{A}$ be a 3×5 matrix. A student writes: "We checked that $\text{col}(\mathbf{A})$ is orthogonal to $\text{null}(\mathbf{A})$ by computing the dot product of their basis vectors." Without doing any arithmetic, explain in one sentence why this calculation is impossible.

Exercise 15.3. For $\mathbf{A} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, find a basis and the dimension of each of the four fundamental subspaces. State which room ($\mathbb{R}^2$ or $\mathbb{R}^3$) each lives in.

Exercise 15.4. Let $\mathbf{A} = \begin{bmatrix} 1 & 1 & 2 & 1 \\ 2 & 2 & 4 & 3 \\ 1 & 1 & 2 & 0 \end{bmatrix}$. Reduce it, identify the pivot and free columns, and write down all the special solutions (a basis for the nullspace). What is the nullity?

Exercise 15.5. Give a 2×2 matrix whose row-reduced form has a pivot in column 1, yet whose column space is *not* the x-axis. Use it to explain in one sentence why the column-space basis must be read from the original matrix, not the reduced one.

Exercise 15.6. For $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$, find the left nullspace directly by solving $\mathbf{A}^{\mathsf{T}}\mathbf{y} = \mathbf{0}$. Confirm its dimension equals $m - r$ and that every basis vector is orthogonal to the column space.

Exercise 15.7. A matrix $\mathbf{A}$ is 5×7 with rank 4. State the dimension of each of the four fundamental subspaces and the room each lives in. How many independent special solutions does $\mathbf{Ax} = \mathbf{0}$ have?

Exercise 15.8. For $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ and input $\mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, split $\mathbf{x}$ into its row-space and nullspace pieces and verify that $\mathbf{Ax} = \mathbf{Ax}_{\text{row}}$, the content of Theorem 15.2.

Exercise 15.9. Use the left nullspace to decide solvability. For $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 1 \end{bmatrix}$, a vector $\mathbf{b}$ makes $\mathbf{Ax} = \mathbf{b}$ solvable iff $\mathbf{b}$ is orthogonal to the left nullspace. Find that orthogonality condition explicitly and check it on $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$.

Exercise 15.10. Let $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \\ 0 & 0 \end{bmatrix}$. Identify all four subspaces, then verify by hand that $\mathbf{A}$ carries the row-space basis to the column-space basis in a one-to-one onto fashion (Theorem 15.2). What are the two stretch factors?

Exercise 15.11. *(Proof.)* Prove that for any $m \times n$ matrix $\mathbf{A}$, a vector in both the row space and the nullspace must be the zero vector. Then use this to prove that the only vector in all four subspaces simultaneously is $\mathbf{0}$ (after accounting for the rooms they live in).

Exercise 15.12. *(Symmetry challenge.)* Explain why $\operatorname{rank}(\mathbf{A}) = \operatorname{rank}(\mathbf{A}^{\mathsf{T}})$ guarantees that the column space and row space always have the *same* dimension, even when $m \neq n$ so they live in rooms of different sizes. Illustrate with a 2×5 matrix of rank 2: what are the four dimensions?

Exercise 15.13. *(Modeling.)* A data matrix $\mathbf{A}$ is 1000×50 (1000 samples, 50 features) and turns out to have rank $r = 8$. In plain language, what do the column space, nullspace, row space, and left nullspace each mean for this dataset? Which subspace measures "directions in feature space that do not affect any output," and which measures "output patterns the data can never produce"?

Chapter 16

Rank and the Fundamental Theorem of Linear Algebra

A single integer captures the true geometric dimensionality of a matrix transformation: the **rank**. A matrix may contain thousands of rows and columns yet collapse the entire input space onto a single line, or it may be small and fill the entire space it maps into. The dimensions $m \times n$ describe the size of the matrix array, but the rank measures the dimension of its image. This chapter explores what rank means, why the row rank always equals the column rank, and how rank organizes the four fundamental subspaces.

We also show that every matrix decomposes into a sum of rank-one outer products, providing the conceptual foundation for the singular value decomposition.

16.1 Rank is the true size of a matrix

Let us start with a concrete geometric example. Take the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

It is a 2×2 matrix. But watch what it does to vectors. Feed it $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and you get $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Feed it $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ and you get $\begin{bmatrix} 2 \\ 4 \end{bmatrix}$. Both outputs sit on the exact same line through the origin – the line in the direction $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$, because $\begin{bmatrix} 2 \\ 4 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. In fact *every* input comes out on that one line:

$$\mathbf{A} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + 2y \\ 2x + 4y \end{bmatrix} = (x + 2y) \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

The entire input plane $\mathbb{R}^2$ maps onto a single line. Although represented by a 2×2 array of numbers, the transformation is fundamentally one-dimensional.

Intuition

The **rank** of a matrix is the dimension of the space its outputs fill – the dimension of the column space $\text{col}(\mathbf{A})$. In plain terms, it is the number of independent directions the matrix can actually reach. A 1000×1000 matrix whose output always lands on a single line has rank 1. The dimensions $m \times n$ count the entries; the rank counts the dimension of the column space.

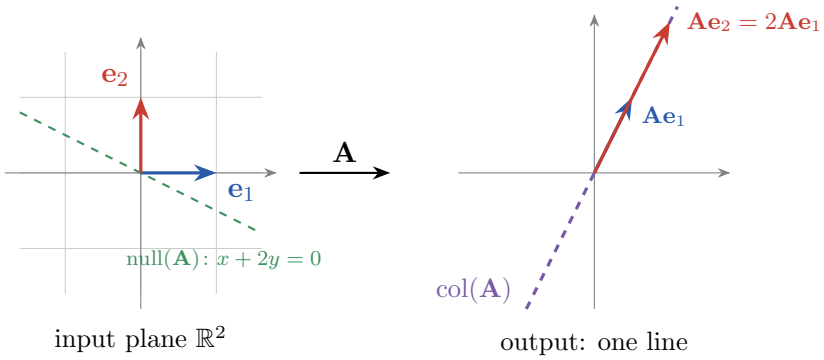


Figure 16.1: The matrix $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ takes the full two-dimensional input plane and crushes it onto a single line. Its rank is 1, while the dashed line $x + 2y = 0$ is crushed completely to the origin.

Compare this with

$$\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}.$$

One number changed – the bottom-right went from 4 to 5 – and the columns are now linearly independent. Now $\mathbf{B}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\mathbf{B}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$, and those two outputs do *not* lie on a common line: $\begin{bmatrix} 2 \\ 5 \end{bmatrix}$ is not a multiple of $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. The two columns point in different directions, so together they span all of $\mathbb{R}^2$. The rank is 2. A single entry change alters the rank from 1 to 2, transforming a singular matrix into an invertible transformation that spans all of $\mathbb{R}^2$.

Focus

Three equivalent ways to say what rank is. Hold all three; they are the same fact seen from three sides.

- **Column rank:** the number of linearly independent columns, i.e. $\dim \text{col}(\mathbf{A})$.
- **Row rank:** the number of linearly independent rows, i.e. $\dim \text{row}(\mathbf{A})$.
- **Pivot count:** the number of pivots you find when you row-reduce the matrix to echelon form.

The deep theorem of this chapter is that these three numbers are *always equal*. We call the common value $\text{rank}(\mathbf{A})$.

16.1.1 Reading rank off the echelon form

You already know elimination. It is the most reliable rank detector we have. Reduce a matrix to row echelon form and count the pivots – the leading nonzero entries, one per pivot row. That count is the rank.

Example 16.1. Find the rank of

$$\mathbf{C} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 8 \\ 3 & 6 & 7 \end{bmatrix}.$$

Eliminate below the first pivot. Subtract $2\times\text{row}_1$ from row_2 and $3\times\text{row}_1$ from row_3:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 2 \\ 0 & 0 & -2 \end{bmatrix}.$$

The second column never produced a pivot – the $(2,2)$ position is zero and so is everything below it. Move to column 3: there is a pivot in row 2. Add row_2 to row_3 to clear the last entry:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}.$$

Two pivots (in columns 1 and 3). So $\text{rank}(\mathbf{C}) = 2$. The third row was a combination of the first two all along (specifically, $\text{row}_3 = 5\,\text{row}_1 - \text{row}_2$); elimination exposed it as a row of zeros.

Common trap

Rank is *not* "the number of nonzero rows you started with," and it is not "the number of columns minus the number of zeros." A matrix can have no zero entries anywhere and still have rank 1 (every row a multiple of the first). Rank is about *independence*, not about where the zeros are. You must do the elimination – or otherwise check for dependence – to know it.

Compute it yourself

Compute the rank of each by hand, reducing to echelon form and counting pivots:

$$\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 2 \\ 1 & 4 & 3 \end{bmatrix}.$$

(Answers: 1, 2, 2. In the second, $\text{row}_3 = \text{row}_1 + \text{row}_2$; in the third, $\text{row}_3 = 2\,\text{row}_2 - \text{row}_1$. Hunt for those dependencies before you reduce – seeing them is the central skill.)

16.2 Rank-nullity: the conservation law

Here is the single most useful counting fact in linear algebra. When a matrix acts on $\mathbb{R}^n$, every one of the n input dimensions has to go *somewhere*. Some of them survive and become independent output

directions – those are counted by the rank. The rest get crushed to zero – those are counted by the dimension of the nullspace, $\dim \operatorname{null}(\mathbf{A})$, the number we call the **nullity**. (In plain terms, nullity is the dimension of the matrix's blind spot: the number of independent directions that collapse completely to the zero vector.) Nothing else can happen to a dimension. So the two counts must add up to the number of input dimensions you started with.

Theorem 16.1 (Rank–Nullity)**.** *For any $m \times n$ matrix $\mathbf{A}$ (with real entries),*

$$\operatorname{rank}(\mathbf{A}) + \dim \operatorname{null}(\mathbf{A}) = n,$$

where n is the number of columns – the dimension of the input space.

Intuition

Rank–nullity expresses an exact dimensional balance for any linear mapping $\mathbf{A} : \mathbb{R}^n \to \mathbb{R}^m$:

$$\underbrace{n}_{\text{input dimension}} = \underbrace{\operatorname{rank}(\mathbf{A})}_{\text{dimension of image}} + \underbrace{\dim \operatorname{null}(\mathbf{A})}_{\text{dimension of kernel}}.$$

The total dimension n of the domain is partitioned between the directions that contribute to the output (the row space) and the directions mapped to zero (the nullspace). Increasing the rank necessarily decreases the nullity by the same amount.

16.2.1 Why it is true, from the echelon form

The cleanest proof rides on something you already trust: every column of $\mathbf{A}$ either holds a pivot or it does not. Reduce $\mathbf{A}$ to reduced row echelon form $\mathbf{R}$. The columns split into two kinds.

- **Pivot columns:** there are $r = \operatorname{rank}(\mathbf{A})$ of them. The corresponding columns of the *original* $\mathbf{A}$ are linearly independent and form a basis for the column space.
- **Free columns:** there are $n - r$ of them, one for each free variable. Each free variable produces exactly one special solution of $\mathbf{Ax} = \mathbf{0}$, and those special solutions form a basis for the nullspace.

So the nullspace has dimension $n - r$. That is the entire theorem: $r + (n - r) = n$. Let us make the special-solution count airtight, because that is the part that does the work.

Proof. Row reduction does not change the nullspace: $\mathbf{Ax} = \mathbf{0}$ and $\mathbf{Rx} = \mathbf{0}$ have exactly the same solutions, because each elimination step is a reversible operation on the equations. So it suffices to count the nullspace dimension of $\mathbf{R}$.

Say $\mathbf{R}$ has r pivots, hence r pivot columns and $n - r$ free columns. Each free column corresponds to a free variable that we are allowed to set to anything. Form the j-th special solution by setting the j-th free variable to 1, all other free variables to 0, and solving for the pivot variables from the equations $\mathbf{Rx} = \mathbf{0}$ (each pivot variable is determined by the free ones). This gives $n - r$ vectors $\mathbf{s}_1, \ldots, \mathbf{s}_{n-r}$.

They are independent. Look only at the free-variable positions. There the vectors $\mathbf{s}_1, \ldots, \mathbf{s}_{n-r}$ read off as the standard basis vectors of $\mathbb{R}^{n-r}$ – a single 1 in distinct slots. A combination $c_1\mathbf{s}_1 + \cdots + c_{n-r}\mathbf{s}_{n-r} = \mathbf{0}$ forces every $c_j = 0$ just by reading those slots. So the special solutions are linearly independent.

They span the nullspace. Take any $\mathbf{x}$ with $\mathbf{Rx} = \mathbf{0}$. Let its free-variable values be $c_1, \ldots, c_{n-r}$. The vector $\mathbf{x} - (c_1\mathbf{s}_1 + \cdots + c_{n-r}\mathbf{s}_{n-r})$ is also in the nullspace (the nullspace is closed under combinations), and it has *all* free variables equal to zero. But once the free variables are zero, the equations $\mathbf{Rx} = \mathbf{0}$ force every pivot variable to zero too. So that difference is $\mathbf{0}$, meaning $\mathbf{x}$ *is* the combination. The special solutions span.

Independent and spanning: the $n - r$ special solutions are a basis, so $\dim \operatorname{null}(\mathbf{A}) = n - r$. Adding r to both sides gives $r + (n - r) = n$. □

Example 16.2. Take a 2×3 matrix, already in a convenient form:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix}.$$

Here $n = 3$ columns. Pivots in columns 1 and 2, so $r = \operatorname{rank}(\mathbf{A}) = 2$. Column 3 is free. Rank–nullity predicts $\dim \operatorname{null} = 3 - 2 = 1$: a single line of solutions. Let us find it. Set the free variable $x_3 = 1$; the equations $x_1 + 2x_3 = 0$ and $x_2 + 3x_3 = 0$ give $x_1 = -2$, $x_2 = -3$. The one special solution is

$$\mathbf{s} = \begin{bmatrix} -2 \\ -3 \\ 1 \end{bmatrix}, \qquad \operatorname{null}(\mathbf{A}) = \operatorname{span}\{\mathbf{s}\}, \quad \dim = 1. \checkmark$$

Two input dimensions produce nonzero output (the rank), one was mapped to zero (the nullity), and $2 + 1 = 3$. The dimension sum matches the theorem.

Walking question

A 5×5 matrix has rank 3. Without computing anything: how many dimensions does it map to zero? How many independent directions can its outputs reach? Could a different 5×5 matrix have rank 3 and map only one dimension to zero? Walk until you are certain the answer to the last question is "no, never" – and you can state clearly which dimension theorem guarantees this.

Before/after Strang

Strang emphasizes this dimension balance throughout his lectures, writing $r + (n - r) = n$ as the core identity connecting rank and nullity. In geometric terms, the dimension of the domain equals the dimension of the image plus the dimension of the kernel: $\dim(\text{domain}) = \dim(\text{image}) + \dim(\text{kernel})$. This is the rank–nullity theorem for linear maps, which serves as the linear algebra formulation of the first isomorphism theorem.

16.3 Row rank equals column rank

We have seen that "number of independent columns" and "number of independent rows" both equal the number of pivots. Because the columns live in $\mathbb{R}^m$ and the rows live in $\mathbb{R}^n$, it is not immediately obvious that the maximum number of linearly independent columns must always equal the maximum number of linearly independent rows. Yet this equality holds for every matrix.

Theorem 16.2 (Row rank = Column rank)**.** *For every matrix* $\mathbf{A}$*, the dimension of the column space equals the dimension of the row space.*

16.3.1 The decisive picture: A as a short product

Here is the argument I find most convincing, because it is constructive and you can see every piece. Let r be the column rank of $\mathbf{A}$ ($m \times n$). Pick a basis $\mathbf{c}_1, \ldots, \mathbf{c}_r$ for the column space – r vectors in $\mathbb{R}^m$. Gather them as the columns of an $m \times r$ matrix $\mathbf{C}$.

Every column $\mathbf{a}_j$ of $\mathbf{A}$ lives in the column space, so it is some combination of the basis: $\mathbf{a}_j = \mathbf{C}\mathbf{f}_j$ for a coefficient vector $\mathbf{f}_j \in \mathbb{R}^r$. Stack those coefficient vectors as columns of an $r \times n$ matrix $\mathbf{F}$. Then, column by column,

$$\mathbf{A} = \mathbf{C}\,\mathbf{F}, \qquad \underbrace{\mathbf{A}}_{m\times n} = \underbrace{\mathbf{C}}_{m\times r}\,\underbrace{\mathbf{F}}_{r\times n}.$$

This is the **rank factorization**: every matrix of column rank r is a "tall times wide" product through a waist of width r. (Figure 16.2.) Now read off the rows. In the product $\mathbf{CF}$, every row of $\mathbf{A}$ is a combination of the r rows of $\mathbf{F}$ – because row i of $\mathbf{A}$ equals (row i of $\mathbf{C}$) $\times\mathbf{F}$, a weighted sum of $\mathbf{F}$'s rows. So the row space of $\mathbf{A}$ is spanned by the r rows of $\mathbf{F}$, which means

$$\text{row rank of } \mathbf{A} \;\le\; r \;=\; \text{column rank of } \mathbf{A}.$$

That is half the theorem. For the other half, apply this same inequality to the transpose $\mathbf{A}^\mathsf{T}$. The rows of $\mathbf{A}$ are the columns of $\mathbf{A}^\mathsf{T}$ and vice versa, so the inequality for $\mathbf{A}^\mathsf{T}$ reads column rank of $\mathbf{A} \le$ row rank of $\mathbf{A}$. Two inequalities pointing opposite ways force equality. Row rank = column rank. ■

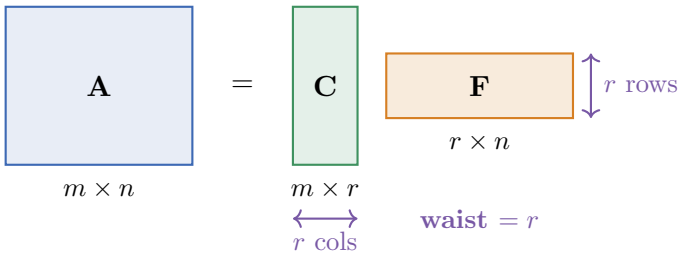


Figure 16.2: The rank factorization $\mathbf{A} = \mathbf{CF}$. Every column of $\mathbf{A}$ is built from the r columns of $\mathbf{C}$ (so column rank $\le r$); every row of $\mathbf{A}$ is built from the r rows of $\mathbf{F}$ (so row rank $\le r$). The single number r – the width of the waist – governs both.

Why this matters

Although columns live in $\mathbb{R}^m$ and rows live in $\mathbb{R}^n$, both counts reflect the same structural rank. In the rank factorization $\mathbf{A} = \mathbf{CF}$, the intermediate dimension r is the minimum inner dimension required to factor $\mathbf{A}$. This factorization demonstrates that the row rank and column rank are simply two perspectives on the same intrinsic invariant of the linear transformation.

Compute it yourself

Mental check: the bottleneck in action. If $\mathbf{A}$ is a 4×100 matrix, what is the largest possible column rank it can have? What is the largest possible row rank? (Answer: At most 4 for both. Although there are a hundred columns, they live in $\mathbb{R}^4$, so at most four can be linearly independent. And because row rank equals column rank, those four dimensions create a waist of width at most 4, preventing the hundred rows from ever spanning more than a

four-dimensional space.)

History and connection

In the nineteenth century, mathematicians studying systems of linear equations and bilinear forms (expressions $\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{y}$ where pairs of vectors interact through a coefficient grid) worked heavily in fixed coordinate systems. They knew equations could be redundant, but lacked any coordinate-free measure of a matrix's true dimension, leaving the equality of row rank and column rank an unsolved puzzle.

At the time, determinants were the supreme algebraic tool; matrices were still viewed merely as rectangular tables for calculating determinants. In 1879, Ferdinand Georg Frobenius published his landmark paper *Theorie der linearen Formen mit ganzen Coefficienten*, formally defining the **rank** of a matrix as the size of its largest non-vanishing **minor**. (A minor is simply the determinant of a square submatrix formed by crossing out unwanted rows and columns.) Because the determinant of any square matrix equals the determinant of its transpose, crossing out rows and columns yields the exact same collection of non-zero minors whether you inspect $\mathbf{A}$ or $\mathbf{A}^{\mathsf{T}}$. Frobenius's determinantal definition settled algebraically why row rank must strictly equal column rank.

Along a complementary track, the British mathematician James Joseph Sylvester developed **invariant theory** (the study of algebraic properties that remain unchanged under coordinate shifts), introducing the term "nullity" and publishing his "law of nullity" in 1884:

$$\operatorname{rank}(\mathbf{AB}) \;\geq\; \operatorname{rank}(\mathbf{A}) + \operatorname{rank}(\mathbf{B}) - n.$$

Whereas Kronecker and Frobenius favored explicit determinantal criteria, twentieth-century algebraists combined rank and nullity with linear maps to classify operators into canonical forms (building on Camille Jordan's normal form).

Yet rank faces a modern computational challenge: exact algebraic rank is brittle under floating-point roundoff. Consider the matrix $\begin{bmatrix} 1 & 1 \\ 1 & 1+10^{-16} \end{bmatrix}$. Mathematically, its rows are independent and its rank is 2. But in IEEE double-precision floating-point arithmetic, $1 + 10^{-16}$ rounds to 1, collapsing the computed rank to 1. Conversely, tiny roundoff noise can make a truly rank-1 physical matrix appear to have full rank. Traditional elimination cannot distinguish between a tiny pivot that represents genuine physical signal and one that represents roundoff garbage. This drove Gene Golub and William Kahan (1965) to develop the singular value decomposition, replacing brittle integer rank with *numerical rank*.

Example 16.3. Factor the rank-1 matrix from the opening:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}_{\mathbf{C}} \underbrace{\begin{bmatrix} 1 & 2 \end{bmatrix}}_{\mathbf{F}}.$$

The waist has width 1. The single column $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ of $\mathbf{C}$ spans the column space; the single row $\begin{bmatrix} 1 & 2 \end{bmatrix}$ of $\mathbf{F}$ spans the row space. One number, $r = 1$, and it is visibly the same on both sides.

Connection to other math

This equality parallels symmetry theorems elsewhere in mathematics, such as Clairaut's theorem on the equality of mixed partial derivatives ($f_{xy} = f_{yx}$) for smooth functions. In both settings, an underlying algebraic structure ensures that evaluating operations across rows versus columns yields identical results.

16.4 The four subspaces, paired by perpendicularity

The rank r determines the dimensions of all four fundamental subspaces. Let $\mathbf{A}$ be $m \times n$ with rank r. Two subspaces live in the input space $\mathbb{R}^n$ and two live in the output space $\mathbb{R}^m$:

$$\begin{array}{lll} \text{in } \mathbb{R}^n: & \text{row}(\mathbf{A}) & \text{dimension } r, \\ & \text{null}(\mathbf{A}) & \text{dimension } n-r, \\ \text{in } \mathbb{R}^m: & \text{col}(\mathbf{A}) & \text{dimension } r, \\ & \text{null}(\mathbf{A}^\mathsf{T}) & \text{dimension } m-r. \end{array}$$

The dimensions already obey rank–nullity twice over: in $\mathbb{R}^n$ we have $r + (n-r) = n$, and in $\mathbb{R}^m$ we have $r + (m-r) = m$. But the sharper fact is *geometric*: inside each space, the two subspaces are not just complementary in dimension – they are **orthogonal**. Every vector in one is perpendicular to every vector in the other.

Theorem 16.3 (Orthogonality of the fundamental subspaces)**.** *For any real $m \times n$ matrix $\mathbf{A}$,*

$$\text{row}(\mathbf{A}) \perp \text{null}(\mathbf{A}) \quad \text{in } \mathbb{R}^n, \qquad \text{col}(\mathbf{A}) \perp \text{null}(\mathbf{A}^\mathsf{T}) \quad \text{in } \mathbb{R}^m.$$

What is more, each pair fills its whole space: row($\mathbf{A}$) *and* null($\mathbf{A}$) *together span* $\mathbb{R}^n$*, and* col($\mathbf{A}$) *and* null($\mathbf{A}^\mathsf{T}$) *together span* $\mathbb{R}^m$*. Each subspace is the* orthogonal complement *of its partner. (An* ***orthogonal complement*** *of a subspace V, written $V^\perp$, consists of all vectors perpendicular to every vector in V.)*

Proof. The first orthogonality is almost a restatement of what $\mathbf{Ax} = \mathbf{0}$ *means.* Write $\mathbf{A}$ in terms of its rows $\mathbf{r}_1^\mathsf{T}, \ldots, \mathbf{r}_m^\mathsf{T}$. The equation $\mathbf{Ax} = \mathbf{0}$ says precisely that every row dotted with $\mathbf{x}$ is zero:

$$\mathbf{Ax} = \mathbf{0} \quad \Longleftrightarrow \quad \langle \mathbf{r}_i, \mathbf{x} \rangle = 0 \text{ for every } i.$$

So a nullspace vector $\mathbf{x}$ is perpendicular to every row, and therefore to every *combination* of rows – that is, to all of row($\mathbf{A}$). Hence null($\mathbf{A}$) $\perp$ row($\mathbf{A}$).

For the dimensions: the two subspaces have dimensions r and $n-r$, summing to n, and orthogonal subspaces meet only at $\mathbf{0}$ (a vector perpendicular to itself has length zero). Complementary dimension plus trivial intersection means they together span $\mathbb{R}^n$, and being orthogonal, each is the orthogonal complement of the other.

The second pair is the first pair applied to $\mathbf{A}^\mathsf{T}$: the column space of $\mathbf{A}$ is the row space of $\mathbf{A}^\mathsf{T}$, and the left nullspace of $\mathbf{A}$ is the nullspace of $\mathbf{A}^\mathsf{T}$. Substituting $\mathbf{A}^\mathsf{T}$ for $\mathbf{A}$ in what we just proved gives col($\mathbf{A}$) $\perp$ null($\mathbf{A}^\mathsf{T}$) filling $\mathbb{R}^m$. □

Intuition

Why must a nullspace vector be perpendicular to every row? Because $\mathbf{Ax}$ is built out of dot products of $\mathbf{x}$ with the rows. For $\mathbf{Ax}$ to be the zero vector, $\mathbf{x}$ has to give zero against *each* row – it must point in a direction the rows cannot detect at all. "Invisible to every row" is exactly "perpendicular to the row space." The nullspace is the blind spot of the rows.

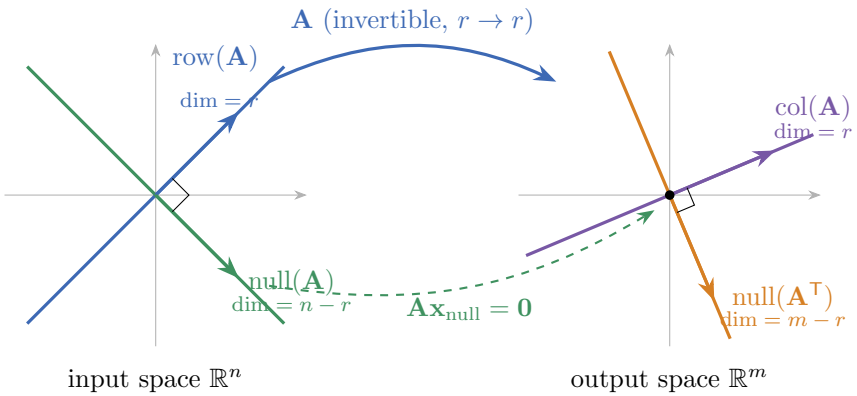


Figure 16.3: The big picture of the Fundamental Theorem of Linear Algebra. In the input space $\mathbb{R}^n$, the row space and nullspace are orthogonal complements ($r + (n - r) = n$). In the output space $\mathbb{R}^m$, the column space and left nullspace are orthogonal complements ($r + (m - r) = m$). The matrix $\mathbf{A}$ maps the row space *one-to-one onto* the column space, and annihilates the nullspace onto $\mathbf{0}$.

There is one more piece that turns these four subspaces into a single moving picture, and it is the reason this whole circle of ideas earns the grand name *the Fundamental Theorem of Linear Algebra*.

Theorem 16.4 (The action on the row space)**.** *The matrix* $\mathbf{A}$ *maps the row space* one-to-one onto *the column space. Restricted to the row space,* $\mathbf{A}$ *is invertible: distinct row-space vectors go to distinct column-space vectors, and every column-space vector is hit exactly once.*

Proof. Both spaces have the same dimension r, so we only need that $\mathbf{A}$ is one-to-one on the row space. Suppose two row-space vectors $\mathbf{u}, \mathbf{v}$ satisfy $\mathbf{Au} = \mathbf{Av}$. Then $\mathbf{A}(\mathbf{u} - \mathbf{v}) = \mathbf{0}$, so $\mathbf{u} - \mathbf{v}$ lies in the nullspace. But $\mathbf{u} - \mathbf{v}$ is also a difference of row-space vectors, hence in the row space. A vector in both the row space and the nullspace is perpendicular to itself (Theorem 16.3), so it is $\mathbf{0}$. Thus $\mathbf{u} = \mathbf{v}$: the map is one-to-one on the row space. A one-to-one linear map between spaces of equal dimension is onto as well. □

Compute it yourself

Mental check: decomposing an input vector. Because $\mathbb{R}^n = \text{row}(\mathbf{A}) \oplus \text{null}(\mathbf{A})$, every input vector $\mathbf{x} \in \mathbb{R}^n$ splits uniquely into $\mathbf{x} = \mathbf{x}_{\text{row}} + \mathbf{x}_{\text{null}}$. What is $\mathbf{Ax}$? (Answer: $\mathbf{Ax} = \mathbf{Ax}_{\text{row}} + \mathbf{Ax}_{\text{null}} = \mathbf{Ax}_{\text{row}} + \mathbf{0} = \mathbf{Ax}_{\text{row}}$. Because $\mathbf{Ax}_{\text{null}} = \mathbf{0}$, all of the output comes from $\mathbf{x}_{\text{row}}$, which $\mathbf{A}$ carries invertibly onto the column space.)

Focus

The Fundamental Theorem of Linear Algebra, summarized. Every real $m \times n$ matrix splits both its spaces into orthogonal subspaces and acts invertibly between them:

- $\mathbb{R}^n = \text{row}(\mathbf{A}) \oplus \text{null}(\mathbf{A})$, an orthogonal split into dimensions r and $n - r$.
- $\mathbb{R}^m = \text{col}(\mathbf{A}) \oplus \text{null}(\mathbf{A}^\mathsf{T})$, an orthogonal split into dimensions r and $m - r$.
- $\mathbf{A}$ sends the row space *invertibly onto* the column space, and maps the nullspace to $\mathbf{0}$.

The single number $r = \text{rank}(\mathbf{A})$ controls every dimension in the diagram. This is the structure the singular value decomposition will later make completely explicit – with orthonormal bases for all four subspaces at once.

Example 16.4. Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}.$$

Row 2 is twice row 1, so $r = 1$ ($m = 2$, $n = 3$). The dimensions must be:

$$\dim \text{row} = 1, \quad \dim \text{null} = n - r = 2, \quad \dim \text{col} = 1, \quad \dim \text{null}(\mathbf{A}^\mathsf{T}) = m - r = 1.$$

Row space ($\subset \mathbb{R}^3$): spanned by $\mathbf{r} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$. *Nullspace* ($\subset \mathbb{R}^3$): solve $x_1 + 2x_2 + 3x_3 = 0$; the two special solutions are $\begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} -3 \\ 0 \\ 1 \end{bmatrix}$. Check perpendicularity to $\mathbf{r}$:

$$\langle \mathbf{r}, \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} \rangle = -2 + 2 + 0 = 0, \qquad \langle \mathbf{r}, \begin{bmatrix} -3 \\ 0 \\ 1 \end{bmatrix} \rangle = -3 + 0 + 3 = 0. \checkmark$$

Column space ($\subset \mathbb{R}^2$): spanned by $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. *Left nullspace* ($\subset \mathbb{R}^2$): solve $\mathbf{A}^\mathsf{T}\mathbf{y} = \mathbf{0}$, i.e. $y_1 + 2y_2 = 0$ from every column; one solution direction is $\begin{bmatrix} 2 \\ -1 \end{bmatrix}$, and indeed $\langle \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ -1 \end{bmatrix} \rangle = 2 - 2 = 0$. ✓Both perpendicular splits check out, and every dimension is governed by the lone $r = 1$.

Common trap

The nullspace and the column space are *not* a perpendicular pair, even though both appear in the four-subspace list. The nullspace lives in $\mathbb{R}^n$ (input side); the column space lives in $\mathbb{R}^m$ (output side). They are in different rooms and need not even have the same length vectors. The correct partners are row $\perp$ null (both in $\mathbb{R}^n$) and col $\perp$ null($\mathbf{A}^\mathsf{T}$) (both in $\mathbb{R}^m$). Pair by *room*, not by name.

Walking question

A square 3×3 matrix has rank 2. Picture its four subspaces. The nullspace is a line in $\mathbb{R}^3$; the row space is the plane perpendicular to that line. On the output side, the column space

is a plane in $\mathbb{R}^3$ and the left nullspace is the line perpendicular to it. Now ask: is the input nullspace line the same as the output left-nullspace line? Walk until you can produce a 3×3 matrix where they differ – and one (symmetric) where they coincide.

16.5 Rank-one matrices: elementary outer products

We have seen that rank measures the width of a matrix's waist. The narrowest possible nonzero waist has width 1. A matrix of rank 1 is the simplest nontrivial matrix there is – and *every* matrix is a sum of rank-one matrices. They serve as elementary building blocks for general matrices and play a central role in the singular value decomposition.

Definition 16.1. A nonzero matrix $\mathbf{M}$ has **rank one** if it can be written as an **outer product**

$$\mathbf{M} = \mathbf{u}\,\mathbf{v}^{\mathsf{T}},$$

where $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^n$ are nonzero column vectors. Its (i, j) entry is $u_i v_j$.

Example 16.5. With $\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$,

$$\mathbf{u}\,\mathbf{v}^{\mathsf{T}} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 4 & 5 \end{bmatrix} = \begin{bmatrix} 4 & 5 \\ 8 & 10 \\ 12 & 15 \end{bmatrix}.$$

Every row is a multiple of $\begin{bmatrix} 4 & 5 \end{bmatrix}$ (the multipliers are $1, 2, 3$); every column is a multiple of $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ (the multipliers are $4, 5$). Row space is a line, column space is a line: rank exactly 1.

Intuition

A rank-one matrix $\mathbf{u}\mathbf{v}^{\mathsf{T}}$ acts on an input $\mathbf{x}$ via

$$(\mathbf{u}\mathbf{v}^{\mathsf{T}})\mathbf{x} = \mathbf{u}\,(\mathbf{v}^{\mathsf{T}}\mathbf{x}) = (\langle \mathbf{v},\, \mathbf{x} \rangle)\,\mathbf{u}.$$

The scalar product $\langle \mathbf{v},\, \mathbf{x} \rangle$ determines the magnitude, and the output always points along the line spanned by $\mathbf{u}$. The vector $\mathbf{v}$ defines the linear functional that evaluates the input, while $\mathbf{u}$ specifies the direction of the output in $\mathbb{R}^m$. Figure 16.4 illustrates this two-step geometric evaluation.

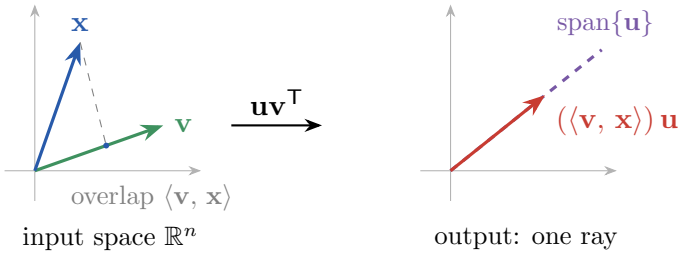


Figure 16.4: A rank-one matrix $\mathbf{u}\mathbf{v}^\mathsf{T}$ evaluated in two stages. First, the dot product $\langle \mathbf{v}, \mathbf{x} \rangle$ produces a scalar. Second, scaling $\mathbf{u}$ by this scalar yields the output. Every output lies in the one-dimensional subspace $\text{span}\{\mathbf{u}\}$.

16.5.1 Every matrix is a sum of rank-one pieces

Here is the bridge. Multiply $\mathbf{A}$ by a vector and you compute a linear combination of $\mathbf{A}$'s columns. But there is a second, equally true way to read a matrix product that reveals the fundamental rank-one building blocks.

Recall the rank factorization $\mathbf{A} = \mathbf{C}\mathbf{F}$ with waist r. A product of an $m \times r$ matrix and an $r \times n$ matrix can be expanded as a sum over the shared index: column k of $\mathbf{C}$ times row k of $\mathbf{F}$, summed. Writing $\mathbf{c}_k$ for the columns of $\mathbf{C}$ and $\mathbf{f}_k^\mathsf{T}$ for the rows of $\mathbf{F}$,

$$\mathbf{A} = \mathbf{C}\mathbf{F} = \sum_{k=1}^{r} \mathbf{c}_k \, \mathbf{f}_k^\mathsf{T}.$$

Each term $\mathbf{c}_k \mathbf{f}_k^\mathsf{T}$ is a rank-one matrix. So:

Theorem 16.5 (Rank as a sum of rank-one pieces)**.** *A matrix has rank r if and only if it can be written as a sum of r rank-one matrices, and r is the* smallest *number of rank-one matrices whose sum is $\mathbf{A}$.*

Why r is the minimum. If $\mathbf{A} = \sum_{k=1}^{p} \mathbf{u}_k \mathbf{v}_k^\mathsf{T}$, then every column of $\mathbf{A}$ is a combination of the p vectors $\mathbf{u}_1, \ldots, \mathbf{u}_p$ (each term contributes a multiple of its $\mathbf{u}_k$). So the column space is spanned by those p vectors, giving $\text{rank}(\mathbf{A}) \le p$. Hence no sum of fewer than $r = \text{rank}(\mathbf{A})$ rank-one matrices can equal $\mathbf{A}$. The factorization above achieves exactly r, so r is the minimum. □

Example 16.6. Decompose a rank-2 matrix into two rank-one atoms. Take

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 1 & 1 & 5 \end{bmatrix}.$$

Notice $\text{row}_3 = \text{row}_1 + \text{row}_2$, so $r = 2$. Columns 1 and 2, $\mathbf{c}_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$ and $\mathbf{c}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$, are a basis for the column space. Column $3 = 2\mathbf{c}_1 + 3\mathbf{c}_2$. So the coefficient matrix $\mathbf{F}$ (each column gives the recipe for the matching column of $\mathbf{A}$) is

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix}, \qquad \mathbf{C} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Now read off the two rank-one pieces – column k of $\mathbf{C}$ times row k of $\mathbf{F}$:

$$\mathbf{A} = \underbrace{\begin{bmatrix}1\\0\\1\end{bmatrix}\begin{bmatrix}1 & 0 & 2\end{bmatrix}}_{\text{piece 1}} + \underbrace{\begin{bmatrix}0\\1\\1\end{bmatrix}\begin{bmatrix}0 & 1 & 3\end{bmatrix}}_{\text{piece 2}} = \begin{bmatrix}1 & 0 & 2\\0 & 0 & 0\\1 & 0 & 2\end{bmatrix} + \begin{bmatrix}0 & 0 & 0\\0 & 1 & 3\\0 & 1 & 3\end{bmatrix}.$$

Add them and you recover $\mathbf{A}$ exactly. Two atoms, because the rank is two.

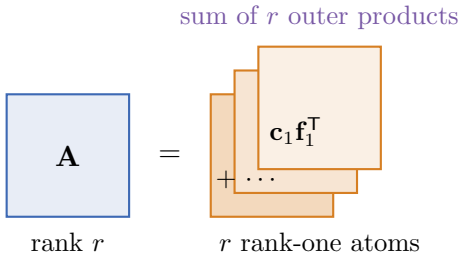


Figure 16.5: Every matrix of rank r can be written as a sum of r rank-one matrices $\mathbf{c}_k\mathbf{f}_k^\mathsf{T}$ (Theorem 16.5). When the pieces are built from orthonormal vectors and ordered by weight $\sigma_1 \geq \sigma_2 \geq \cdots > 0$ (the singular value decomposition in Chapter 29), dropping the smallest terms yields the optimal low-rank approximation – the mathematical engine of modern data compression.

Compute it yourself

By hand, write $\mathbf{A} = \begin{bmatrix}2 & 6\\1 & 3\end{bmatrix}$ as a single outer product $\mathbf{u}\mathbf{v}^\mathsf{T}$. (Hint: every column is a multiple of $\begin{bmatrix}2\\1\end{bmatrix}$, so try $\mathbf{u} = \begin{bmatrix}2\\1\end{bmatrix}$ and find the row $\mathbf{v}^\mathsf{T}$ that makes the columns $\begin{bmatrix}2\\1\end{bmatrix}$ and $\begin{bmatrix}6\\3\end{bmatrix}$ come out right. Answer: $\mathbf{v}^\mathsf{T} = \begin{bmatrix}1 & 3\end{bmatrix}$.) Then confirm $\operatorname{rank}(\mathbf{A}) = 1$ by checking the second column is $3\times$ the first.

Modern application

Low-rank approximation: how data gets compressed. A grayscale image is a matrix of pixel brightnesses – maybe 1000×1000, a million numbers. Most real images are *nearly* low rank: their information concentrates in a few dominant directions. The singular value decomposition (Chapter 29) writes any matrix as a sum of rank-one outer products ordered by importance (the strength-ordered version of Figure 16.5),

$$\mathbf{A} = \sigma_1\mathbf{u}_1\mathbf{v}_1^\mathsf{T} + \sigma_2\mathbf{u}_2\mathbf{v}_2^\mathsf{T} + \cdots + \sigma_r\mathbf{u}_r\mathbf{v}_r^\mathsf{T}, \qquad \sigma_1 \geq \sigma_2 \geq \cdots > 0,$$

and keeping only the first few terms gives the *best* approximation of that rank. Keep 50 terms of a 1000×1000 image and you store $50\,(1000 + 1000) = 100{,}000$ numbers instead of a million – a $10\times$ compression – while the picture stays recognizable. The same technique powers recommender systems (a user-by-movie ratings matrix is low rank because tastes cluster), latent-semantic analysis of text, and the parameter-efficient fine-tuning of large neural networks (LoRA adds a trained low-rank update $\mathbf{BA}$ to a frozen weight matrix). Rank-one decompositions provide the foundation for low-rank approximations across data science and numerical computing.

Pause

Recap, and what each idea costs you.

- **Rank = true reach.** The dimension the outputs fill – equally, independent columns, independent rows, or pivots. *Cost:* one elimination, $O(mn \cdot \min\{m, n\})$ arithmetic; then just count pivots.
- **Rank–nullity** (Theorem 16.1): $\text{rank} + \dim \text{null} = n$. *Cost:* free – once you know the rank and the number of columns, the nullity needs no further work.
- **Row rank = column rank** (Theorem 16.2): both count the width r of the waist in $\mathbf{A} = \mathbf{CF}$. *Cost:* the same single elimination tells you both at once.
- **Four subspaces, two perpendicular pairs** (Theorem 16.3): row $\perp$ null in $\mathbb{R}^n$, col $\perp$ null($\mathbf{A}^\mathsf{T}$) in $\mathbb{R}^m$, all dimensions set by r. *Cost:* the pivot columns and special solutions you already have hand you bases for all four.
- **Rank-one components** (Theorem 16.5): $\mathbf{A}$ is a sum of exactly r outer products $\mathbf{c}_k\mathbf{f}_k^\mathsf{T}$. *Cost:* read them straight off the rank factorization; storing them costs $r(m+n)$ numbers instead of mn – the whole point of low-rank compression.

One integer, r, paid for once, governs everything above.

Connection to other math

The rank–nullity conservation law $\text{rank}(\mathbf{A}) + \dim \text{null}(\mathbf{A}) = n$ and the outer-product expansion $\mathbf{A} = \sum_{k=1}^{r} \mathbf{c}_k\mathbf{f}_k^\mathsf{T}$ give us a complete structural inventory of what a matrix preserves and destroys. We can compress low-rank matrices into $r(m+n)$ numbers instead of mn, forming the foundation of modern data compression and machine learning.

Even so, two practical challenges remain. First, pure algebraic rank is brittle under floating-point roundoff: a matrix can be mathematically full rank while sitting within 10^{-16} of a lower-rank matrix. (In Chapter 29, the Singular Value Decomposition will address this by replacing the integer rank with a continuous spectrum of singular values.)

Second, while Theorem 16.3 establishes that the row and column spaces are perpendicular to their partner nullspaces, the basis vectors *within* each subspace may be non-orthogonal. In Chapter 17, we launch Part IV by developing orthogonal and orthonormal coordinate systems, where coordinates decouple and projections onto subspaces simplify significantly.

16.6 Exercises

Exercise 16.1. Find the rank of each matrix by reducing to echelon form and counting pivots.

$$\text{(a)} \begin{bmatrix} 3 & 6 \\ 1 & 2 \end{bmatrix}, \quad \text{(b)} \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix}, \quad \text{(c)} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 5 \end{bmatrix}.$$

For each, state the nullity using rank–nullity, then verify by finding the special solutions.

Exercise 16.2. A matrix $\mathbf{A}$ is 4×7 and has rank 3. Without seeing the entries, state: the dimension of the nullspace, the dimension of the column space, the dimension of the left nullspace, and the dimension of the row space. Which two of these subspaces live in $\mathbb{R}^4$, and which two in $\mathbb{R}^7$?

Exercise 16.3. For $\mathbf{A} = \begin{bmatrix} 1 & 2 & 2 \\ 2 & 4 & 5 \end{bmatrix}$, find a basis for the row space and a basis for the nullspace. Verify by direct dot product that every nullspace basis vector is perpendicular to every row space basis vector. (Determine the dimension of the nullspace from the rank first.)

Exercise 16.4. Write $\mathbf{A} = \begin{bmatrix} 6 & 4 & 2 \\ 9 & 6 & 3 \end{bmatrix}$ as a single outer product $\mathbf{u}\mathbf{v}^{\mathsf{T}}$. Then write its rank. Could you have known the rank just by glancing at the rows? Explain in one sentence.

Exercise 16.5. Decompose $\mathbf{A} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 0 & 1 & 1 \end{bmatrix}$ into a sum of rank-one matrices using a rank factorization $\mathbf{A} = \mathbf{C}\mathbf{F}$. First find the rank (look for a column dependency), then build $\mathbf{C}$ from a basis of the column space and $\mathbf{F}$ from the recipes for each column, and finally write $\mathbf{A} = \sum_k \mathbf{c}_k \mathbf{f}_k^{\mathsf{T}}$. Check by adding the pieces.

Exercise 16.6. Let $\mathbf{M} = \mathbf{u}\mathbf{v}^{\mathsf{T}}$ be a nonzero rank-one $n \times n$ matrix. Show that $\mathbf{M}^2 = (\langle \mathbf{v}, \mathbf{u} \rangle)\,\mathbf{M}$. Conclude that if $\langle \mathbf{v}, \mathbf{u} \rangle = 0$ then $\mathbf{M}^2 = \mathbf{0}$, even though $\mathbf{M} \neq \mathbf{0}$. Give a concrete 2×2 example with $\mathbf{M}^2 = \mathbf{0}$.

Exercise 16.7. Prove that for any matrices $\mathbf{A}$ (size $m \times k$) and $\mathbf{B}$ (size $k \times n$), $\operatorname{rank}(\mathbf{AB}) \leq \min\{\operatorname{rank}(\mathbf{A}), \operatorname{rank}(\mathbf{B})\}$. (Hint: every column of $\mathbf{AB}$ is a combination of the columns of $\mathbf{A}$; then apply the same idea to rows, or to the transpose.) Where does this say the "waist" of a product cannot be wider than either factor's waist?

Exercise 16.8. Find two rank-one matrices whose sum has rank 1, and two whose sum has rank 2. Then find two nonzero rank-one matrices whose sum is the zero matrix (rank 0). What does this show about the claim "rank of a sum equals sum of ranks"? (It is false; show the correct statement is $\operatorname{rank}(\mathbf{A} + \mathbf{B}) \leq \operatorname{rank}(\mathbf{A}) + \operatorname{rank}(\mathbf{B})$ and explain the inequality in one line using column spaces.)

Exercise 16.9. Using the rank factorization $\mathbf{A} = \mathbf{C}\mathbf{F}$ with waist $r =$ column rank, explain in your own words why the row space of $\mathbf{A}$ is spanned by the rows of $\mathbf{F}$, and hence why row rank $\leq$ column rank. Then explain why applying the same statement to $\mathbf{A}^{\mathsf{T}}$ closes the loop to equality. (Aim for a paragraph a classmate could follow without the figure.)

Exercise 16.10. Let $\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ (rank 2, $n = 3$). Take the two row-space basis vectors $\mathbf{r}_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$ and $\mathbf{r}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$ and compute $\mathbf{A}\mathbf{r}_1$ and $\mathbf{A}\mathbf{r}_2$. Verify the outputs are independent (so $\mathbf{A}$ is one-to-one on the row space, as Theorem 16.4 promises). Then take the nullspace vector $\mathbf{n} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$, confirm $\mathbf{A}\mathbf{n} = \mathbf{0}$, and note how the row space gets mapped one-to-one while the nullspace is annihilated.

Exercise 16.11. (Challenge.) Suppose $\mathbf{A}$ is $n \times n$ with $\operatorname{rank}(\mathbf{A}) = r < n$. Prove that $\mathbf{A}$ has a nonzero nullspace, and use rank–nullity to say exactly its dimension. Then prove the converse direction of the invertibility dictionary: an $n \times n$ matrix is invertible if and only if its rank is n. (Connect "rank n" to "trivial nullspace" to "columns independent" to "invertible.")

Part IV

Orthogonality

Chapter 17

Orthogonality and Orthogonal Subspaces

A central concept throughout linear algebra is **orthogonality**—the mathematical generalization of perpendicularity and right angles. When two directions are orthogonal, their components decouple completely: knowledge of one coordinate gives zero information about the other. Because of this decoupling, complex multi-dimensional problems separate into independent one-dimensional subproblems.

Here we extend perpendicularity from pairs of individual vectors to whole *subspaces* – lines, planes, and higher-dimensional subspaces that meet at right angles. The four fundamental subspaces of a matrix partition the domain and codomain into two orthogonal pairs, providing the structural foundation for projections, least squares, and the singular value decomposition. We also introduce *orthonormal* sets – coordinate systems composed of mutually perpendicular unit vectors – and the matrices built from them, the *orthogonal matrices*, which represent rotations and reflections. These transformations preserve lengths and angles exactly, making them stable tools in numerical linear algebra.

As always: picture first, then algebra, then proof.

17.1 The dot product remembers the angle

Everything here rides on one number. Given two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ written as columns, their *dot product* (or inner product) is

$$\langle \mathbf{u},\, \mathbf{v} \rangle \;=\; \mathbf{u}^{\mathsf{T}}\mathbf{v} \;=\; u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

It is a sum of products, and the answer is a single scalar. The length of a vector is recovered by dotting it with itself:

$$\|\mathbf{v}\|^2 \;=\; \langle \mathbf{v},\, \mathbf{v} \rangle \;=\; v_1^2 + \cdots + v_n^2,$$

which is just the Pythagorean theorem in n dimensions. And the dot product remembers the *angle* θ between the two vectors through the formula

$$\langle \mathbf{u},\, \mathbf{v} \rangle \;=\; \|\mathbf{u}\|\,\|\mathbf{v}\|\,\cos\theta.$$

Intuition

The dot product $\langle \mathbf{u}, \mathbf{v} \rangle = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta$ measures the directional alignment between two vectors. When two nonzero vectors point in perpendicular directions, $\cos\theta = 0$, so their dot product vanishes: $\langle \mathbf{u}, \mathbf{v} \rangle = 0$. In linear algebra, vectors with zero inner product are called **orthogonal**.

We know the angle formula from Chapter 4, but it pays to see *why* a zero dot product means a right angle from first principles, because the geometric reason is pure Pythagoras and needs no trigonometry at all.

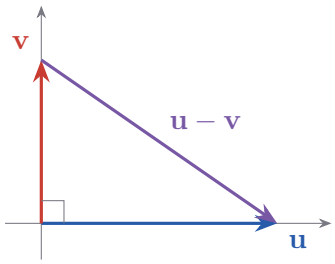


Figure 17.1: When $\mathbf{u} \perp \mathbf{v}$, the triangle with legs $\mathbf{u}$ and $\mathbf{v}$ and hypotenuse $\mathbf{u} - \mathbf{v}$ obeys Pythagoras: $\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$. That equation is *equivalent* to $\langle \mathbf{u}, \mathbf{v} \rangle = 0$.

Formal

Orthogonality via Pythagoras. For any $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$,

$$\|\mathbf{u} - \mathbf{v}\|^2 = \langle \mathbf{u} - \mathbf{v}, \, \mathbf{u} - \mathbf{v} \rangle = \|\mathbf{u}\|^2 - 2\langle \mathbf{u}, \, \mathbf{v} \rangle + \|\mathbf{v}\|^2.$$

The Pythagorean relation $\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$ holds *if and only if* the cross term vanishes, that is, if and only if $\langle \mathbf{u}, \mathbf{v} \rangle = 0$. So "right angle" and "zero dot product" are two names for the same thing.

Pause

Pause and think: The diagonals of a rectangle. If $\mathbf{u} \perp \mathbf{v}$, what is the squared length of their sum, $\|\mathbf{u} + \mathbf{v}\|^2$? Expand it:

$$\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + 2\langle \mathbf{u}, \, \mathbf{v} \rangle + \|\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2.$$

Because the cross term $2\langle \mathbf{u}, \, \mathbf{v} \rangle$ is zero, $\|\mathbf{u} + \mathbf{v}\|^2$ equals $\|\mathbf{u} - \mathbf{v}\|^2$. Geometrically, $\mathbf{u} + \mathbf{v}$ and $\mathbf{u} - \mathbf{v}$ are the two diagonals of the parallelogram spanned by $\mathbf{u}$ and $\mathbf{v}$. When the angle is $90°$, that parallelogram is a rectangle, and its two diagonals have the exact same length.

That little derivation is the model for the whole chapter: expand a squared length, watch the cross terms, and demand that they vanish. Let us make the definition official.

Definition 17.1. Two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ are **orthogonal** if $\langle \mathbf{u}, \, \mathbf{v} \rangle = \mathbf{u}^\mathsf{T}\mathbf{v} = 0$. We write $\mathbf{u} \perp \mathbf{v}$.

Remark 17.1. By this definition, the zero vector is orthogonal to every vector $\mathbf{v} \in \mathbb{R}^n$, since $\langle \mathbf{0}, \, \mathbf{v} \rangle = 0$. This convention ensures that orthogonal complements are linear subspaces containing $\mathbf{0}$.

Example 17.1. Let $\mathbf{u} = \begin{bmatrix} 3 \\ 1 \\ -2 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ 2 \\ 4 \end{bmatrix}$. Then

$$\langle \mathbf{u}, \mathbf{v} \rangle = (3)(2) + (1)(2) + (-2)(4) = 6 + 2 - 8 = 0,$$

so $\mathbf{u} \perp \mathbf{v}$. They sit at a right angle in $\mathbb{R}^3$, even though neither one is along an axis. Check Pythagoras as a sanity test: $\|\mathbf{u}\|^2 = 9 + 1 + 4 = 14$, $\|\mathbf{v}\|^2 = 4 + 4 + 16 = 24$, and $\mathbf{u} - \mathbf{v} = \begin{bmatrix} 1 \\ -1 \\ -6 \end{bmatrix}$ has $\|\mathbf{u} - \mathbf{v}\|^2 = 1 + 1 + 36 = 38 = 14 + 24$. The cross term is zero.

Common trap

Orthogonality must not be confused with collinearity. Nonzero parallel vectors have maximal inner product in absolute value; orthogonal vectors have an inner product of zero. In addition, while every set of nonzero mutually orthogonal vectors is linearly independent (as proved below), linearly independent vectors need not be orthogonal.

17.2 From vectors to subspaces

Now we lift the idea. So far "orthogonal" relates two arrows. The reach grows once we ask a whole *line* to be perpendicular to a whole *plane*, or one subspace to another.

Definition 17.2. Two subspaces V and W of $\mathbb{R}^n$ are **orthogonal**, written $V \perp W$, if *every* vector in V is orthogonal to *every* vector in W:

$$\langle \mathbf{v}, \mathbf{w} \rangle = 0 \qquad \text{for all } \mathbf{v} \in V,\ \mathbf{w} \in W.$$

This is a strong demand – it must hold for all pairs at once – but it is much easier to check than it looks. A small lemma reduces the work to testing *basis* vectors.

Lemma 17.1. *Let $V = \operatorname{span}(\mathbf{v}_1, \dots, \mathbf{v}_p)$ and $W = \operatorname{span}(\mathbf{w}_1, \dots, \mathbf{w}_q)$. If every basis vector of V is orthogonal to every basis vector of W – that is, $\langle \mathbf{v}_i, \mathbf{w}_j \rangle = 0$ for all i, j – then $V \perp W$.*

Proof. Take any $\mathbf{v} \in V$ and $\mathbf{w} \in W$. Write them in the bases: $\mathbf{v} = \sum_i a_i \mathbf{v}_i$ and $\mathbf{w} = \sum_j b_j \mathbf{w}_j$. The dot product is bilinear (it distributes over sums and pulls out scalars in each slot), so

$$\langle \mathbf{v}, \mathbf{w} \rangle = \Big\langle \textstyle\sum_i a_i \mathbf{v}_i,\ \sum_j b_j \mathbf{w}_j \Big\rangle = \sum_i \sum_j a_i b_j \langle \mathbf{v}_i, \mathbf{w}_j \rangle = \sum_i \sum_j a_i b_j \cdot 0 = 0. \qquad \square$$

Example 17.2. In $\mathbb{R}^3$, let V be the z-axis, spanned by $\mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, and let W be the xy-plane, spanned by $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$. Each basis dot product is zero: $\langle \mathbf{e}_3, \mathbf{e}_1 \rangle = 0$ and $\langle \mathbf{e}_3, \mathbf{e}_2 \rangle = 0$. By the lemma, the whole z-axis is orthogonal to the whole xy-plane. Good – that matches the picture in your head.

Common trap

Two *planes* in $\mathbb{R}^3$ can never be orthogonal subspaces in the sense of Definition 17.2, even though we casually call the floor and a wall "perpendicular." Why? Two planes through the origin in $\mathbb{R}^3$ always share a whole line (their intersection). Any nonzero vector $\mathbf{x}$ on that shared line lives in *both* planes, so we would need $\langle \mathbf{x}, \mathbf{x} \rangle = 0$, forcing $\mathbf{x} = \mathbf{0}$. The everyday "perpendicular walls" meet along an edge that belongs to both – so as *subspaces* they are not orthogonal. Orthogonal subspaces can share *only* the zero vector.

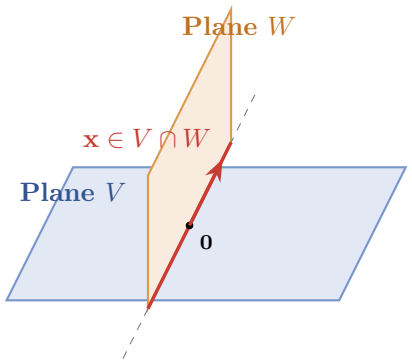


Figure 17.2: Why two planes in $\mathbb{R}^3$ can never be orthogonal subspaces. Even if they meet at 90° like the floor and a wall, they intersect along an entire line $V \cap W$. Any nonzero vector $\mathbf{x}$ on that line belongs to *both* planes. If $V \perp W$, we would have $\langle \mathbf{x}, \mathbf{x} \rangle = 0$, forcing $\mathbf{x} = \mathbf{0}$. True orthogonal subspaces can share nothing except the origin.

That last observation is so important it deserves to be a proposition.

Proposition 17.1. *If $V \perp W$ then $V \cap W = \{\mathbf{0}\}$.*

Proof. Suppose $\mathbf{x} \in V \cap W$. Then $\mathbf{x}$ is in V and also in W, so by orthogonality $\langle \mathbf{x}, \mathbf{x} \rangle = 0$. But $\langle \mathbf{x}, \mathbf{x} \rangle = \|\mathbf{x}\|^2$, and the only vector with zero length is $\mathbf{0}$. Hence $\mathbf{x} = \mathbf{0}$. □

Intuition

Orthogonal subspaces intersect only at the origin: $V \cap W = \{\mathbf{0}\}$. Geometrically, every direction in V is perpendicular to every direction in W, as seen with the z-axis and the xy-plane in $\mathbb{R}^3$.

17.3 The orthogonal complement

A line through the origin in $\mathbb{R}^3$ is orthogonal to many things, but one space perpendicular to it is the largest possible: the entire plane at right angles. That "everything perpendicular to V" is useful enough to earn its own name and symbol.

Definition 17.3. The **orthogonal complement** of a subspace $V \subseteq \mathbb{R}^n$, written $V^\perp$ (read "V perp"), is the set of all vectors orthogonal to every vector in V:

$$V^\perp = \{\, \mathbf{x} \in \mathbb{R}^n : \langle \mathbf{x}, \mathbf{v} \rangle = 0 \text{ for all } \mathbf{v} \in V \,\}.$$

A few facts are immediate and worth pinning down, since $V^\perp$ is going to do a lot of work.

Proposition 17.2. *For any subspace $V \subseteq \mathbb{R}^n$, the set $V^\perp$ is itself a subspace of $\mathbb{R}^n$.*

Proof. We check the two closure rules. First, $\mathbf{0} \in V^\perp$ since $\langle \mathbf{0}, \mathbf{v} \rangle = 0$ for all $\mathbf{v}$. Now suppose $\mathbf{x}, \mathbf{y} \in V^\perp$ and c is a scalar. For any $\mathbf{v} \in V$,

$$\langle \mathbf{x} + \mathbf{y}, \mathbf{v} \rangle = \langle \mathbf{x}, \mathbf{v} \rangle + \langle \mathbf{y}, \mathbf{v} \rangle = 0 + 0 = 0, \qquad \langle c\mathbf{x}, \mathbf{v} \rangle = c \langle \mathbf{x}, \mathbf{v} \rangle = c \cdot 0 = 0.$$

So $\mathbf{x} + \mathbf{y}$ and $c\mathbf{x}$ are again in $V^\perp$. Closed under addition and scaling: it is a subspace. □

Focus

The two facts to carry with you about $V^\perp$:

- $V^\perp$ is a subspace, and $V \perp V^\perp$ (they are orthogonal to each other).
- Together they fill the whole space cleanly: $\dim V + \dim V^\perp = n$, and $V \cap V^\perp = \{\mathbf{0}\}$. We say $\mathbb{R}^n = V \oplus V^\perp$.

Here the direct sum symbol $\oplus$ means that every vector in $\mathbb{R}^n$ decomposes into a piece in V plus a piece in $V^\perp$ in exactly one unique way, with zero overlap between the two subspaces except the origin. The word "complement" fits cleanly: $V^\perp$ supplies exactly the dimensions V is missing, with no overlap and at right angles.

The dimension count $\dim V + \dim V^\perp = n$ comes shortly, once we connect $V^\perp$ to a nullspace – the cleanest route to it. First, a hands-on computation to make the idea concrete.

Compute it yourself

Let $V = \operatorname{span}\left(\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right)$, a line in $\mathbb{R}^3$. To find $V^\perp$ we want all $\mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ with $\langle \mathbf{x}, (1,1,1) \rangle = 0$, i.e. $x + y + z = 0$. That single equation describes a *plane* through the origin. A basis for it: set the free variables and solve. Taking (y, z) free gives $\begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$. So $V^\perp$ is the 2-dimensional plane $x+y+z = 0$. Check: we have $\dim V = 1$, $\dim V^\perp = 2$, and $1+2 = 3 = n$. The dimensions add up.

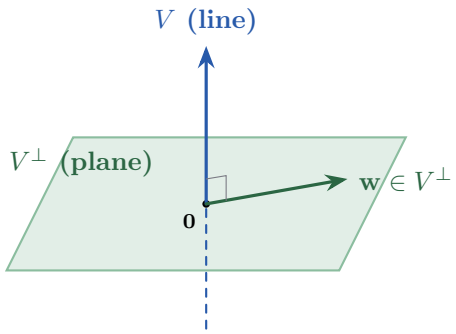


Figure 17.3: A line V and its orthogonal complement $V^\perp$, a plane in $\mathbb{R}^3$. Every vector $\mathbf{w}$ in the plane meets the line at 90°. Together their dimensions add to the full space ($1 + 2 = 3$), and they intersect only at the origin $\mathbf{0}$.

Notice the trick in the lab. Finding $V^\perp$ for a line turned into solving a single linear equation, and that equation was exactly "row vector dotted with $\mathbf{x}$ equals zero." Stack several such equations into the rows of a matrix and $V^\perp$ becomes a nullspace.

17.4 The key identity: $\text{null}(\mathbf{A}) = \text{row}(\mathbf{A})^\perp$

One observation ties orthogonal complements to matrices and sets up the whole "big picture." Look at what it means for $\mathbf{x}$ to be in the nullspace of a matrix $\mathbf{A}$. The equation $\mathbf{Ax} = \mathbf{0}$, written row by row, says

$$(\text{row 1 of } \mathbf{A}) \cdot \mathbf{x} = 0, \quad (\text{row 2 of } \mathbf{A}) \cdot \mathbf{x} = 0, \quad \ldots, \quad (\text{row } m \text{ of } \mathbf{A}) \cdot \mathbf{x} = 0.$$

Every row of $\mathbf{A}$ dots to zero with $\mathbf{x}$. So $\mathbf{x}$ is orthogonal to every row – and by Lemma 17.1, orthogonal to every *combination* of rows, i.e. to the entire row space. That is the definition of being in $\text{row}(\mathbf{A})^\perp$.

Theorem 17.1. *For any $m \times n$ matrix $\mathbf{A}$,*

$$\text{null}(\mathbf{A}) \;=\; \text{row}(\mathbf{A})^\perp,$$

as subspaces of the input space $\mathbb{R}^n$. The nullspace and the row space are orthogonal complements.

Proof. We show each set contains the other.

($\subseteq$) If $\mathbf{x} \in \text{null}(\mathbf{A})$ then $\mathbf{Ax} = \mathbf{0}$, so each row dotted with $\mathbf{x}$ is zero. Hence $\mathbf{x}$ is orthogonal to every row, and by Lemma 17.1 to all of $\text{row}(\mathbf{A})$. Thus $\mathbf{x} \in \text{row}(\mathbf{A})^\perp$.

($\supseteq$) Conversely, if $\mathbf{x} \in \text{row}(\mathbf{A})^\perp$ then $\mathbf{x}$ is orthogonal to every row in particular, so each row dotted with $\mathbf{x}$ is zero, which says $\mathbf{Ax} = \mathbf{0}$. Thus $\mathbf{x} \in \text{null}(\mathbf{A})$.

The two inclusions give equality. □

This theorem pays for the dimension law promised earlier. The Rank–Nullity theorem (from the rank chapter) says $\dim \text{row}(\mathbf{A}) + \dim \text{null}(\mathbf{A}) = n$, because the row space has dimension r (the rank) and the nullspace has dimension $n - r$. Read through Theorem 17.1, that becomes a statement purely about a subspace and its complement.

Theorem 17.2 (Dimension of the complement)**.** *For any subspace $V \subseteq \mathbb{R}^n$,*

$$\dim V + \dim V^{\perp} = n.$$

Proof. Pick a basis $\mathbf{v}_1, \ldots, \mathbf{v}_r$ of V and stack these as the *rows* of an $r \times n$ matrix $\mathbf{A}$. Then $V = \mathrm{row}(\mathbf{A})$ and $\dim V = r = \mathrm{rank}(\mathbf{A})$. By Theorem 17.1, $V^{\perp} = \mathrm{row}(\mathbf{A})^{\perp} = \mathrm{null}(\mathbf{A})$, whose dimension is $n - r$ by Rank–Nullity. So $\dim V + \dim V^{\perp} = r + (n - r) = n$. □

Corollary 17.1. $(V^{\perp})^{\perp} = V$. *Taking the complement twice returns you to where you started.*

Proof. Every vector of V is orthogonal to every vector of $V^{\perp}$, so $V \subseteq (V^{\perp})^{\perp}$. By Theorem 17.2 applied twice, $\dim(V^{\perp})^{\perp} = n - \dim V^{\perp} = n - (n - \dim V) = \dim V$. A subspace contained in another of the same dimension must equal it. □

Before/after Strang

Strang reaches this point and draws his famous "big picture": two perpendicular pairs of subspaces, the input space $\mathbb{R}^n$ split by the row space and the nullspace, the output space $\mathbb{R}^m$ split by the column space and the left nullspace. *Before* Strang, hold onto the single sentence "the nullspace is everything the rows can't see," which is exactly $\mathrm{null}(\mathbf{A}) = \mathrm{row}(\mathbf{A})^{\perp}$. *After* Strang, recognize this same orthogonality as the reason projection onto the column space (next chapter) and least squares both work: the error vector lands in the left nullspace, perpendicular to the column space.

17.5 The two orthogonal pairs among the four subspaces

We now have the first orthogonal pair for free. Apply the identity to $\mathbf{A}^{\mathsf{T}}$ instead of $\mathbf{A}$, and the second pair follows, because the row space of $\mathbf{A}^{\mathsf{T}}$ is the column space of $\mathbf{A}$, and the nullspace of $\mathbf{A}^{\mathsf{T}}$ is the left nullspace of $\mathbf{A}$.

Theorem 17.3 (The two orthogonal pairs)**.** *Let $\mathbf{A}$ be $m \times n$ with rank r. Then:*

- *In the input space $\mathbb{R}^n$:* $\mathrm{null}(\mathbf{A}) = \mathrm{row}(\mathbf{A})^{\perp}$, *and these have dimensions $n - r$ and r.*
- *In the output space $\mathbb{R}^m$:* $\mathrm{null}(\mathbf{A}^{\mathsf{T}}) = \mathrm{col}(\mathbf{A})^{\perp}$, *and these have dimensions $m - r$ and r.*

Consequently $\mathbb{R}^n = \mathrm{row}(\mathbf{A}) \oplus \mathrm{null}(\mathbf{A})$ and $\mathbb{R}^m = \mathrm{col}(\mathbf{A}) \oplus \mathrm{null}(\mathbf{A}^{\mathsf{T}})$, each a clean perpendicular split.

Proof. The first line is Theorem 17.1. For the second, replace $\mathbf{A}$ by $\mathbf{A}^{\mathsf{T}}$ in that theorem: $\mathrm{null}(\mathbf{A}^{\mathsf{T}}) = \mathrm{row}(\mathbf{A}^{\mathsf{T}})^{\perp}$. But the rows of $\mathbf{A}^{\mathsf{T}}$ are the columns of $\mathbf{A}$, so $\mathrm{row}(\mathbf{A}^{\mathsf{T}}) = \mathrm{col}(\mathbf{A})$. Substituting gives $\mathrm{null}(\mathbf{A}^{\mathsf{T}}) = \mathrm{col}(\mathbf{A})^{\perp}$. The dimension counts come from Theorem 17.2 and $\mathrm{rank}(\mathbf{A}^{\mathsf{T}}) = \mathrm{rank}(\mathbf{A}) = r$. □

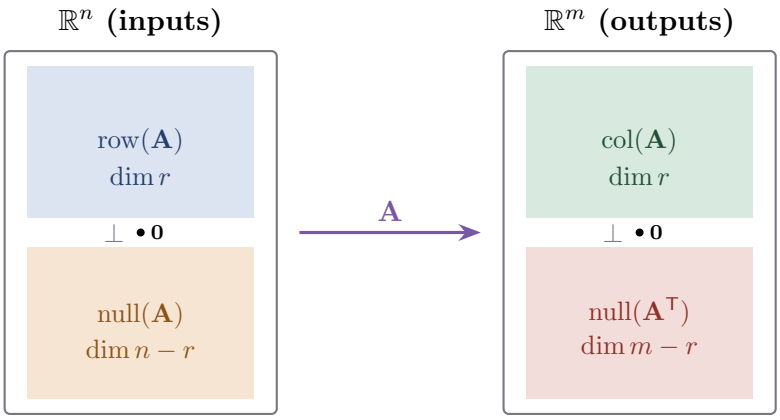


Figure 17.4: The big picture. Inside $\mathbb{R}^n$ the row space and the nullspace are orthogonal complements; inside $\mathbb{R}^m$ the column space and the left nullspace are orthogonal complements. The matrix $\mathbf{A}$ maps the row space one-to-one onto the column space, and crushes the nullspace to the origin. The little $\perp$ symbols remind you each pair meets at 90° at the origin.

Why this matters

A matrix is, on its face, just a grid of numbers. Yet hidden inside every such grid are four subspaces, locked into two perpendicular pairs, with dimensions r, $n-r$, r, $m-r$ that always add correctly. The rank r appears *twice* – once on each side – which is the deep reason "row rank equals column rank." Orthogonality is what makes the picture rigid: the complement isn't just some other subspace of the right dimension, it is the *unique* one at right angles. That rigidity is what lets us later *split* any vector uniquely into a part the matrix sees and a part it ignores – the projection behind least squares, regression, and the SVD.

Example 17.3. Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 2 \\ 2 & 4 & 5 \end{bmatrix}.$$

Row reduce: $R_2 \to R_2 - 2R_1$ gives $\begin{bmatrix} 1 & 2 & 2 \\ 0 & 0 & 1 \end{bmatrix}$, then clear above the second pivot: $R_1 \to R_1 - 2R_2$ gives $\begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. So $r = 2$.

Row space ($\subseteq \mathbb{R}^3$, dimension 2): spanned by the pivot rows $\begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$.

Nullspace ($\subseteq \mathbb{R}^3$, dimension 1): the free variable is x_2. From $x_1 + 2x_2 = 0$ and $x_3 = 0$ we get $\mathbf{n} = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$. Check the orthogonality promised by Theorem 17.3: $\langle \mathbf{n}, (1,2,0) \rangle = -2 + 2 + 0 = 0$ and $\langle \mathbf{n}, (0,0,1) \rangle = 0$. The nullspace really is perpendicular to the row space.

Column space ($\subseteq \mathbb{R}^2$, dimension 2): the pivots sit in columns 1 and 3, so a basis is $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\begin{bmatrix} 2 \\ 5 \end{bmatrix}$. These two independent vectors already fill $\mathbb{R}^2$, so $\text{col}(\mathbf{A}) = \mathbb{R}^2$.

Left nullspace ($\subseteq \mathbb{R}^2$, dimension $m - r = 0$): just $\{\mathbf{0}\}$, consistent with the column space being all of $\mathbb{R}^2$ (its complement has nothing left to be). Everything checks.

Walking question

Take a walk and answer this without paper. A matrix $\mathbf{A}$ is 4×6 with rank 3. What are the dimensions of all four subspaces, and which two live in $\mathbb{R}^6$, which two in $\mathbb{R}^4$? Then: which pair sums to 6, which pair sums to 4? If you can recite "3 and 3 in $\mathbb{R}^6$, 3 and 1 in $\mathbb{R}^4$" and say which is the row space and which the nullspace, the big picture has become yours.

17.6 Orthonormal sets and coordinate expansions

Orthogonality immediately implies linear independence.

Proposition 17.3. *A set of nonzero, mutually orthogonal vectors $\mathbf{q}_1, \dots, \mathbf{q}_k$ is linearly independent.*

Proof. Suppose $c_1\mathbf{q}_1 + \cdots + c_k\mathbf{q}_k = \mathbf{0}$. Dot both sides with $\mathbf{q}_j$. By orthogonality every term $\langle \mathbf{q}_i, \mathbf{q}_j \rangle$ vanishes except $i = j$:

$$0 = \left\langle \textstyle\sum_i c_i\mathbf{q}_i,\ \mathbf{q}_j \right\rangle = c_j \langle \mathbf{q}_j, \mathbf{q}_j \rangle = c_j \|\mathbf{q}_j\|^2.$$

Since $\mathbf{q}_j \neq \mathbf{0}$, $\|\mathbf{q}_j\|^2 > 0$, so $c_j = 0$. This holds for each j, so all coefficients are zero: the vectors are independent. □

If on top of being orthogonal each vector also has *unit length*, the structure simplifies further.

Definition 17.4. Vectors $\mathbf{q}_1, \dots, \mathbf{q}_k \in \mathbb{R}^n$ form an **orthonormal set** if they are mutually orthogonal *and* each is a unit vector:

$$\langle \mathbf{q}_i, \mathbf{q}_j \rangle \ = \ \delta_{ij} \ = \ \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

The symbol δ_{ij} is the *Kronecker delta*: 1 on the diagonal, 0 off it.

Intuition

An orthonormal set consists of mutually perpendicular unit vectors, forming a rigid coordinate frame in $\mathbb{R}^n$. The standard basis vectors $\mathbf{e}_1, \dots, \mathbf{e}_n$ form the simplest example, but any rotation or reflection of this standard frame remains orthonormal. Because the vectors are unit length and mutually orthogonal, coordinates decouple completely.

Their practical advantage is immediate: in an orthonormal basis, finding coordinates does not require solving a linear system; it requires only evaluating dot products. Suppose $\mathbf{q}_1, \dots, \mathbf{q}_n$ is an orthonormal basis of $\mathbb{R}^n$ and you want to write a vector $\mathbf{v}$ in this basis: $\mathbf{v} = c_1\mathbf{q}_1 + \cdots + c_n\mathbf{q}_n$. Dotting both sides with $\mathbf{q}_j$, every term with $i \neq j$ vanishes:

$$\langle \mathbf{v}, \mathbf{q}_j \rangle = \sum_i c_i \langle \mathbf{q}_i, \mathbf{q}_j \rangle = c_j.$$

So $c_j = \langle \mathbf{v}, \mathbf{q}_j \rangle$ – the jth coordinate is simply the dot product of $\mathbf{v}$ with the jth basis vector.

Focus

Orthonormal expansion. If $\mathbf{q}_1, \dots, \mathbf{q}_n$ is an orthonormal basis of $\mathbb{R}^n$, then every $\mathbf{v}$ has the explicit expansion

$$\mathbf{v} = \langle \mathbf{v}, \mathbf{q}_1 \rangle \mathbf{q}_1 + \langle \mathbf{v}, \mathbf{q}_2 \rangle \mathbf{q}_2 + \cdots + \langle \mathbf{v}, \mathbf{q}_n \rangle \mathbf{q}_n.$$

Each coefficient is a single dot product. This explicit formula avoids matrix inversion, making orthonormal bases computationally stable.

Pause

Pause and think: Direct coordinates. Suppose $\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3$ is an orthonormal basis of $\mathbb{R}^3$. Without doing any elimination or writing down coordinates, what is $\langle 5\mathbf{q}_1 - 4\mathbf{q}_2 + 7\mathbf{q}_3, \mathbf{q}_2 \rangle$? By orthonormality, $\langle \mathbf{q}_1, \mathbf{q}_2 \rangle = 0$, $\langle \mathbf{q}_3, \mathbf{q}_2 \rangle = 0$, and $\langle \mathbf{q}_2, \mathbf{q}_2 \rangle = 1$. The answer is -4. Each orthonormal basis vector projects out only its corresponding component.

History and connection

Systematic orthogonalization grew out of curve-fitting and integral equations around the turn of the twentieth century. In 1883, the Danish mathematician and actuary Jørgen Gram was calculating best-fit curves for forestry data. When fitting curves using ordinary powers $(1, t, t^2, \dots)$, every extra degree scrambled all previous coefficients, requiring huge, unstable systems of equations to be resolved from scratch. Gram realized that replacing polynomials with mutually perpendicular functions caused the cross-terms to vanish, allowing each coefficient to be computed independently.

Two decades later, in 1907, Erhard Schmidt and his mentor David Hilbert at Göttingen were tackling *linear integral equations*—equations where the unknown is an entire continuous curve trapped inside an integral, effectively a system with infinitely many variables. At the time, mathematicians possessed only classical calculus, determinants, and Fourier's trigonometric series; the concept of an abstract "vector space" did not yet exist. When Schmidt adapted Gram's orthogonalization procedure to continuous functions, the infinite-dimensional equations decoupled into independent one-dimensional projections, completely bypassing matrix inversion. Hilbert recognized the underlying geometry at work and formalized the framework of *Hilbert space*—an infinite-dimensional arena where functions behave like vectors and integrals play the role of dot products.

At first, applying geometric words like "perpendicular" and "projection" to spaces of infinitely many dimensions startled classical analysts accustomed to explicit coordinate formulas. Within two decades, however, Werner Heisenberg, Max Born, and Erwin Schrödinger found that Hilbert's orthogonal function spaces provided the natural mathematical setting for quantum mechanics. Today, orthogonality underpins signal processing, computer graphics, and stable algorithms like the QR factorization.

Example 17.4. The rotated pair

$$\mathbf{q}_1 = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{q}_2 = \tfrac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

is orthonormal: $\langle \mathbf{q}_1, \mathbf{q}_1 \rangle = \frac{1}{2}(1+1) = 1$, likewise for $\mathbf{q}_2$, and $\langle \mathbf{q}_1, \mathbf{q}_2 \rangle = \frac{1}{2}(-1+1) = 0$. To express

$\mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ in this basis, just dot:

$$c_1 = \langle \mathbf{v},\, \mathbf{q}_1 \rangle = \tfrac{1}{\sqrt{2}}(3+1) = \tfrac{4}{\sqrt{2}} = 2\sqrt{2}, \qquad c_2 = \langle \mathbf{v},\, \mathbf{q}_2 \rangle = \tfrac{1}{\sqrt{2}}(-3+1) = -\sqrt{2}.$$

Sanity check: $2\sqrt{2}\,\mathbf{q}_1 - \sqrt{2}\,\mathbf{q}_2 = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \mathbf{v}$. It works, and we never solved a system.

Figure 17.5 shows what this looks like: the orthonormal pair is just a copy of the standard graph paper rotated by 45°, and the coordinates c_1, c_2 are the readings of $\mathbf{v}$ along those tilted axes.

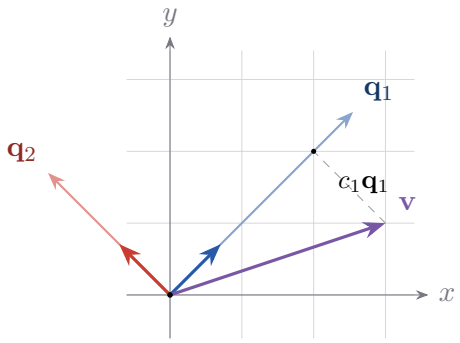


Figure 17.5: The orthonormal basis $\mathbf{q}_1, \mathbf{q}_2$ is the standard grid turned 45°. The coordinate $c_1 = \langle \mathbf{v},\, \mathbf{q}_1 \rangle$ is found by dropping $\mathbf{v}$ perpendicularly onto the $\mathbf{q}_1$ axis (dashed line), landing at $c_1\mathbf{q}_1 = (2, 2)$; $c_2 = \langle \mathbf{v},\, \mathbf{q}_2 \rangle$ is the matching reading along $\mathbf{q}_2$, landing at $c_2\mathbf{q}_2 = (1, -1)$. Coordinates by dot product, no system to solve.

Connection to other math

This expansion is the finite-dimensional ancestor of the *Fourier series*. In a function space, the sines and cosines $\{\sin(kx), \cos(kx)\}$ are mutually orthogonal under the inner product $\langle f,\, g \rangle = \int f(x)g(x)\,dx$, and the Fourier coefficients are computed by exactly the same recipe: $c_k = \langle f,\, \text{that basis function} \rangle$, an integral instead of a sum. Whole fields – signal processing, the heat equation, quantum mechanics – are built on choosing a clever orthonormal basis and reading off coordinates by integration. Chapter 32 makes this precise, but the idea is already in front of you: orthonormal means "coordinates by dot product."

17.7 Orthogonal matrices and isometries

Collect an orthonormal set into the columns of a matrix and $\mathbf{Q}^{\mathsf{T}}\mathbf{Q}$ collapses to something very simple.

Definition 17.5. A square matrix $\mathbf{Q}$ is an **orthogonal matrix** if its columns form an orthonormal set, equivalently if

$$\mathbf{Q}^{\mathsf{T}}\mathbf{Q} \;=\; \mathbf{I}.$$

Let us see why those two statements are the same. If the columns are $\mathbf{q}_1, \ldots, \mathbf{q}_n$, then the (i, j) entry of $\mathbf{Q}^{\mathsf{T}}\mathbf{Q}$ is (row i of $\mathbf{Q}^{\mathsf{T}}$) times (column j of $\mathbf{Q}$), which is $\mathbf{q}_i^{\mathsf{T}}\mathbf{q}_j = \langle \mathbf{q}_i,\, \mathbf{q}_j \rangle$. Orthonormality says that equals δ_{ij}, and the matrix of δ_{ij} entries is precisely $\mathbf{I}$. So "columns orthonormal" and "$\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$" say the same thing, entry by entry.

Common trap

Despite the standard name, an "orthogonal matrix" has *orthonormal* columns, not merely orthogonal columns. Each column must have unit length. A matrix with orthogonal columns of varying lengths is not an orthogonal matrix; the formal condition is $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$.

For a *square* $\mathbf{Q}$, the equation $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ says $\mathbf{Q}^\mathsf{T}$ is a left inverse, and since $\mathbf{Q}$ is square a left inverse is the inverse. So we get the cleanest inverse in mathematics:

$$\boxed{\mathbf{Q}^{-1} = \mathbf{Q}^\mathsf{T}}$$

To invert an orthogonal matrix you do not solve anything – you just transpose it. (As a bonus, $\mathbf{Q}^\mathsf{T}$ is then orthogonal too, so the *rows* of an orthogonal matrix are also orthonormal, and $\mathbf{Q}\mathbf{Q}^\mathsf{T} = \mathbf{I}$ as well.)

Example 17.5. The plane rotation by angle θ,

$$\mathbf{Q} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

is orthogonal. Its columns each have length $\sqrt{\cos^2\theta + \sin^2\theta} = 1$ and dot to $-\cos\theta\sin\theta + \sin\theta\cos\theta = 0$. Its inverse is the transpose, which is rotation by $-\theta$ – exactly what undoes a rotation. A reflection, e.g. $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ (flip across the x-axis), is orthogonal too. Rotations and reflections are the orthogonal matrices of the plane.

17.7.1 What orthogonal matrices preserve

Here is the property that makes orthogonal matrices central to numerical linear algebra: they move space *rigidly*. They never stretch, squash, or shear. Lengths come out unchanged, and so do angles.

Theorem 17.4 (Orthogonal matrices preserve dot products)**.** *If* $\mathbf{Q}$ *is orthogonal, then for all* $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$*,*

$$\langle \mathbf{Q}\mathbf{x},\, \mathbf{Q}\mathbf{y} \rangle = \langle \mathbf{x},\, \mathbf{y} \rangle.$$

In particular $\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|$ *(lengths preserved), and the angle between* $\mathbf{Q}\mathbf{x}$ *and* $\mathbf{Q}\mathbf{y}$ *equals the angle between* $\mathbf{x}$ *and* $\mathbf{y}$ *(angles preserved).*

Proof. Use the transpose rule $(\mathbf{Q}\mathbf{x})^\mathsf{T} = \mathbf{x}^\mathsf{T}\mathbf{Q}^\mathsf{T}$ and the defining identity $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$:

$$\langle \mathbf{Q}\mathbf{x},\, \mathbf{Q}\mathbf{y} \rangle = (\mathbf{Q}\mathbf{x})^\mathsf{T}(\mathbf{Q}\mathbf{y}) = \mathbf{x}^\mathsf{T}\mathbf{Q}^\mathsf{T}\mathbf{Q}\,\mathbf{y} = \mathbf{x}^\mathsf{T}\mathbf{I}\,\mathbf{y} = \mathbf{x}^\mathsf{T}\mathbf{y} = \langle \mathbf{x},\, \mathbf{y} \rangle.$$

Taking $\mathbf{y} = \mathbf{x}$ gives $\|\mathbf{Q}\mathbf{x}\|^2 = \|\mathbf{x}\|^2$, hence equal lengths. Since $\cos\theta = \langle \mathbf{x}, \mathbf{y} \rangle / (\|\mathbf{x}\|\|\mathbf{y}\|)$ and every quantity in that ratio is preserved, the angle is preserved too. □

Intuition

An orthogonal matrix is a *rigid motion* of space that fixes the origin: a rotation, a reflection, or a combination. Drop a constellation of points into $\mathbb{R}^n$, hit them all with $\mathbf{Q}$, and the whole constellation turns or flips as one solid object – every distance between points and every angle

is exactly what it was. Nothing is distorted. That is the geometric meaning of $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$.

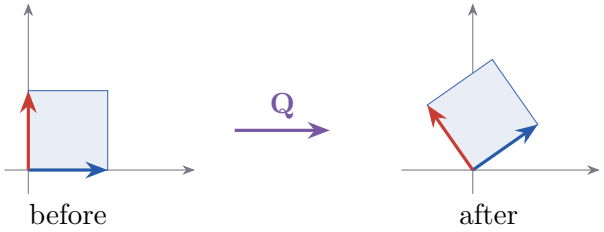


Figure 17.6: An orthogonal matrix turns (or flips) the unit square without distorting it. The sides keep length 1, the corner keeps its right angle: a rigid motion. Contrast a general matrix, which would stretch the square into a slanted parallelogram.

Compute it yourself

Verify rigidity by hand with $\theta = 90°$, so $\mathbf{Q} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$. Take $\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$. Then $\mathbf{Qx} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$ and $\mathbf{Qy} = \begin{bmatrix} -3 \\ 1 \end{bmatrix}$. Lengths: $\|\mathbf{x}\|^2 = 5 = \|\mathbf{Qx}\|^2$ and $\|\mathbf{y}\|^2 = 10 = \|\mathbf{Qy}\|^2$. Dot products: $\langle \mathbf{x}, \mathbf{y} \rangle = 2 + 3 = 5$ and $\langle \mathbf{Qx}, \mathbf{Qy} \rangle = 3 + 2 = 5$. Everything is preserved – the 90° turn moved the arrows but kept all measurements.

Remark 17.2. Taking determinants of $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$ gives $\det(\mathbf{Q})^2 = 1$, so $\det \mathbf{Q} = \pm 1$. The value $+1$ is a pure rotation (a "proper" motion that preserves handedness), and -1 is a reflection or a rotation-times-reflection (it flips orientation, turning a left hand into a right hand). Either way the volume scaling is $|\det \mathbf{Q}| = 1$: orthogonal matrices preserve volume, as a rigid motion must. We will meet determinants properly in Chapter 21; for now, note how cleanly that ± 1 falls out.

Pause

Pause and think: Real eigenvalues of a rigid motion. If an orthogonal matrix $\mathbf{Q}$ has a real eigenvalue λ with non-zero eigenvector $\mathbf{x}$, what could λ possibly be? Since $\mathbf{Q}$ preserves lengths, $\|\mathbf{Qx}\| = \|\lambda\mathbf{x}\| = |\lambda|\,\|\mathbf{x}\|$. Equating this to $\|\mathbf{x}\|$ forces $|\lambda| = 1$. So real eigenvalues of an orthogonal matrix can only be $+1$ (axes left untouched, like the rotation axis of a 3D spinning top) or -1 (directions flipped directly backwards, like in a mirror reflection). A rigid motion can never stretch or shrink an eigenvector.

Modern application

Why machine learning and graphics rely on $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$.

- *Numerical stability.* Multiplying by $\mathbf{Q}$ never amplifies error, because it never amplifies length ($\|\mathbf{Qx}\| = \|\mathbf{x}\|$). Algorithms like *QR* factorization (decomposing a matrix into an orthogonal $\mathbf{Q}$ and upper-triangular $\mathbf{R}$) and the *SVD* (singular value decomposition) are built from orthogonal matrices precisely so that rounding errors don't blow up. A general matrix can magnify a tiny error by a huge factor; an orthogonal one cannot.
- *3D graphics and robotics.* A camera turn or a robot-arm joint is an orthogonal 3×3

matrix with det = +1 (a rotation). Storing the inverse is free – just transpose – and repeated rotations never silently distort the model, because each one preserves every length and angle.

- *Random projections and PCA.* Principal component analysis (PCA, which finds the dominant directions of a data cloud) uses eigenvectors that form an orthonormal basis; projecting data onto them rotates the cloud onto its natural axes without squashing distances. "Orthonormal directions" is the data-science way of saying "uncorrelated, unit-variance coordinates."
- *Whitening and decorrelation.* Because $\mathbf{Q}$ preserves inner products, it turns one orthonormal frame into another – the exact operation behind rotating data into a basis where features no longer interfere.

The thread tying these together is Theorem 17.4: when your transformation preserves length and angle, geometry survives the computation intact.

Before/after Strang

Gilbert Strang emphasizes that orthogonal matrices are numerically ideal transformations to compute with, and that $\mathbf{Q}^{-1} = \mathbf{Q}^{\mathsf{T}}$ is the central property to remember. After Strang, you will see this same $\mathbf{Q}$ in three central factorizations: $\mathbf{A} = \mathbf{QR}$ (Gram–Schmidt, next chapters), $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^{\mathsf{T}}$ for symmetric matrices (the spectral theorem), and $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^{\mathsf{T}}$ (the SVD), where $\mathbf{U}$ and $\mathbf{V}$ are both orthogonal. In every case the orthogonal factors perform isometries (rigid rotations and reflections) while a diagonal factor handles stretching.

17.8 Splitting a vector into perpendicular parts

We close by applying the orthogonal-complement structure, which leads straight into the next chapter on projections. The focus box of the complement section established that $\mathbb{R}^n = V \oplus V^{\perp}$ for *any* subspace V: Theorem 17.2 shows the dimensions add to n, while Proposition 17.1 forces $V \cap V^{\perp} = \{\mathbf{0}\}$. The direct sum "$\oplus$" means every vector splits *uniquely* into a piece in V and a piece in $V^{\perp}$.

Theorem 17.5 (Orthogonal decomposition)**.** *Let $V \subseteq \mathbb{R}^n$ be a subspace. Every $\mathbf{x} \in \mathbb{R}^n$ can be written in exactly one way as*

$$\mathbf{x} = \mathbf{x}_V + \mathbf{x}_{V^{\perp}}, \qquad \mathbf{x}_V \in V, \quad \mathbf{x}_{V^{\perp}} \in V^{\perp}.$$

Proof. Existence. Pick a basis $\mathbf{v}_1, \dots, \mathbf{v}_r$ of V and a basis $\mathbf{w}_1, \dots, \mathbf{w}_{n-r}$ of $V^{\perp}$ (using Theorem 17.2). Combined, these n vectors are linearly independent: if $\sum_i a_i\mathbf{v}_i + \sum_j b_j\mathbf{w}_j = \mathbf{0}$, then $\sum_i a_i\mathbf{v}_i = -\sum_j b_j\mathbf{w}_j$ lies in $V \cap V^{\perp} = \{\mathbf{0}\}$ (Proposition 17.1), forcing both sums and hence all a_i, b_j to vanish. By Corollary 14.1(c), these n independent vectors form a basis of $\mathbb{R}^n$, so every $\mathbf{x} \in \mathbb{R}^n$ expands as $\mathbf{x}_V + \mathbf{x}_{V^{\perp}}$. (Once Chapter 20 shows how to build an orthonormal basis $\mathbf{q}_1, \dots, \mathbf{q}_r$ of V, we even get the explicit formula $\mathbf{x}_V = \sum_{i=1}^{r} \langle \mathbf{x}, \mathbf{q}_i \rangle \mathbf{q}_i$, since dotting $\mathbf{x} - \mathbf{x}_V$ with each $\mathbf{q}_j$ gives $\langle \mathbf{x}, \mathbf{q}_j \rangle - \langle \mathbf{x}, \mathbf{q}_j \rangle = 0$.)

Uniqueness. If $\mathbf{x} = \mathbf{a} + \mathbf{b} = \mathbf{a}' + \mathbf{b}'$ are two such splittings with $\mathbf{a}, \mathbf{a}' \in V$ and $\mathbf{b}, \mathbf{b}' \in V^{\perp}$, then $\mathbf{a} - \mathbf{a}' = \mathbf{b}' - \mathbf{b}$ lies in both V and $V^{\perp}$. By Proposition 17.1 the only such vector is $\mathbf{0}$, so $\mathbf{a} = \mathbf{a}'$ and $\mathbf{b} = \mathbf{b}'$. □

Intuition

Given any subspace $V \subseteq \mathbb{R}^n$, every vector $\mathbf{x}$ decomposes uniquely into a component $\mathbf{x}_V \in V$ and an orthogonal component $\mathbf{x}_{V^\perp} \in V^\perp$. The vector $\mathbf{x}_V$ is the orthogonal *projection* of $\mathbf{x}$ onto V, while $\mathbf{x}_{V^\perp} = \mathbf{x} - \mathbf{x}_V$ is the orthogonal error vector. In Chapter 18, this decomposition forms the basis for orthogonal projections and least-squares regression.

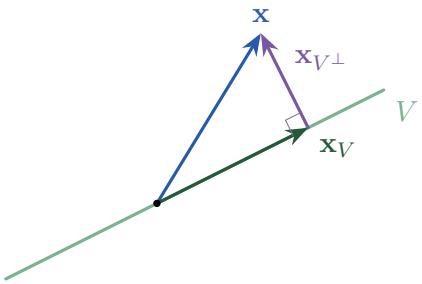


Figure 17.7: The orthogonal decomposition $\mathbf{x} = \mathbf{x}_V + \mathbf{x}_{V^\perp}$. The projection $\mathbf{x}_V$ lies in V; the residual $\mathbf{x}_{V^\perp}$ is perpendicular to V (note the right-angle mark). This decomposition is unique, providing the basis for projection and least squares.

Example 17.6. Split $\mathbf{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ along the line $V = \operatorname{span}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} \right)$ (the x-axis). The unit vector is $\mathbf{q}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, so

$$\mathbf{x}_V = \langle \mathbf{x}, \mathbf{q}_1 \rangle \, \mathbf{q}_1 = 2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad \mathbf{x}_{V^\perp} = \mathbf{x} - \mathbf{x}_V = \begin{bmatrix} 0 \\ 3 \end{bmatrix}.$$

Indeed $\mathbf{x}_V$ is on the x-axis, $\mathbf{x}_{V^\perp}$ is on the y-axis ($= V^\perp$), they are perpendicular, and they sum to $\mathbf{x}$. The projection on the axis is $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$; the orthogonal residual is $\begin{bmatrix} 0 \\ 3 \end{bmatrix}$.

Focus

Summary of core concepts. Orthogonal vectors have zero dot product. A subspace V has a unique orthogonal complement $V^\perp$ such that $\dim V + \dim V^\perp = n$. The four fundamental subspaces form two orthogonal complement pairs: $\operatorname{null}(\mathbf{A}) = \operatorname{row}(\mathbf{A})^\perp$ in $\mathbb{R}^n$ and $\operatorname{null}(\mathbf{A}^\mathsf{T}) = \operatorname{col}(\mathbf{A})^\perp$ in $\mathbb{R}^m$. Orthonormal vectors allow coordinates to be calculated directly by dot products; arranged into a square matrix, they form an orthogonal matrix $\mathbf{Q}$ with $\mathbf{Q}^{-1} = \mathbf{Q}^\mathsf{T}$ that preserves lengths and angles. Every vector decomposes uniquely into an orthogonal sum $\mathbf{x} = \mathbf{x}_V + \mathbf{x}_{V^\perp}$.

Connection to other math

Orthonormality provides direct coordinates without matrix inversion. Finding coordinates relative to an orthonormal frame requires only dot products, and orthogonal matrices $\mathbf{Q}$ act as rigid motions (isometries, transformations that preserve lengths, angles, and distances exactly) without amplifying numerical errors.
Yet real-world data and columns of matrices are almost never orthogonal when they arrive. When a system $\mathbf{Ax} = \mathbf{b}$ is inconsistent because $\mathbf{b} \notin \operatorname{col}(\mathbf{A})$, knowing that $\operatorname{col}(\mathbf{A})$ and $\operatorname{null}(\mathbf{A}^\mathsf{T})$

are orthogonal complements is not enough; we need an explicit operator to drop $\mathbf{b}$ onto $\mathrm{col}(\mathbf{A})$. In Chapter 18, we develop the algebraic machinery of *projections*, building the projection matrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ that finds the closest point on any subspace.

17.9 Exercises

Exercise 17.1. Decide which of these pairs are orthogonal, by computing the dot product: (a) $\begin{bmatrix}1\\-2\\1\end{bmatrix}$ and $\begin{bmatrix}2\\1\\0\end{bmatrix}$; (b) $\begin{bmatrix}3\\0\\-1\end{bmatrix}$ and $\begin{bmatrix}1\\5\\3\end{bmatrix}$; (c) $\begin{bmatrix}1\\1\\1\\1\end{bmatrix}$ and $\begin{bmatrix}1\\1\\-1\\-1\end{bmatrix}$. For each orthogonal pair, confirm Pythagoras: $\|\mathbf{u}-\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$.

Exercise 17.2. Let $V = \mathrm{span}\left(\begin{bmatrix}2\\-1\\2\end{bmatrix}\right) \subseteq \mathbb{R}^3$. Find a basis for $V^\perp$ by solving the single equation $2x - y + 2z = 0$. Confirm that $\dim V + \dim V^\perp = 3$, and verify each of your basis vectors really is orthogonal to $\begin{bmatrix}2\\-1\\2\end{bmatrix}$.

Exercise 17.3. Let $W = \mathrm{span}\left(\begin{bmatrix}1\\0\\1\end{bmatrix}, \begin{bmatrix}0\\1\\1\end{bmatrix}\right) \subseteq \mathbb{R}^3$. Find $W^\perp$. (Hint: stack the two vectors as rows of a 2×3 matrix $\mathbf{A}$; then $W^\perp = \mathrm{row}(\mathbf{A})^\perp = \mathrm{null}(\mathbf{A})$.) What is $\dim W^\perp$, and does it match Theorem 17.2?

Exercise 17.4. For $\mathbf{A} = \begin{bmatrix}1 & 2 & 3\\2 & 4 & 6\end{bmatrix}$, find the rank and a basis for each of the four fundamental subspaces. Then explicitly check the two orthogonality relations of Theorem 17.3: nullspace $\perp$ row space, and left nullspace $\perp$ column space.

Exercise 17.5. The vectors $\mathbf{q}_1 = \frac{1}{5}\begin{bmatrix}3\\4\end{bmatrix}$ and $\mathbf{q}_2 = \frac{1}{5}\begin{bmatrix}-4\\3\end{bmatrix}$ are orthonormal (check it). Use the orthonormal expansion formula to write $\mathbf{v} = \begin{bmatrix}5\\10\end{bmatrix}$ as $c_1\mathbf{q}_1 + c_2\mathbf{q}_2$, and verify your answer by recombining.

Exercise 17.6. Build a 3×3 orthogonal matrix whose first column is $\frac{1}{3}\begin{bmatrix}2\\2\\1\end{bmatrix}$. (Find a second unit vector orthogonal to it, then a third orthogonal to both – a cross product is a fast way to get the third.) Verify $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ for your matrix.

Exercise 17.7. Prove directly from $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ that the columns of an orthogonal matrix are orthonormal, and conversely. Then prove that if $\mathbf{Q}_1$ and $\mathbf{Q}_2$ are both orthogonal $n\times n$ matrices, so is their product $\mathbf{Q}_1\mathbf{Q}_2$. (Geometrically: composing two rigid motions gives a rigid motion.)

Exercise 17.8. Let $V = \operatorname{span}\left(\begin{bmatrix}1\\1\\0\end{bmatrix}\right)$ and $\mathbf{x} = \begin{bmatrix}3\\1\\4\end{bmatrix}$. Compute the orthogonal decomposition $\mathbf{x} = \mathbf{x}_V + \mathbf{x}_{V^\perp}$ from Theorem 17.5. (First normalize the spanning vector to a unit $\mathbf{q}_1$.) Confirm that $\langle \mathbf{x}_V, \mathbf{x}_{V^\perp} \rangle = 0$ and that the two pieces add back to $\mathbf{x}$.

Exercise 17.9. Prove that for any two subspaces, $V \subseteq W$ implies $W^\perp \subseteq V^\perp$ (taking complements reverses inclusion). Use this to give a short argument that $((V^\perp)^\perp)^\perp = V^\perp$.

Exercise 17.10. (Challenge.) Show that *every* 2×2 orthogonal matrix is either $\begin{bmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{bmatrix}$ (a rotation, $\det = +1$) or $\begin{bmatrix}\cos\theta & \sin\theta\\ \sin\theta & -\cos\theta\end{bmatrix}$ (a reflection, $\det = -1$) for some angle θ. (Hint: the first column is a unit vector, so it equals $\begin{bmatrix}\cos\theta\\ \sin\theta\end{bmatrix}$ for some θ; the second column is a unit vector orthogonal to it, leaving exactly two choices.) Interpret each case geometrically.

Exercise 17.11. (Proof.) Use the orthogonal decomposition $\mathbf{x} = \mathbf{x}_V + \mathbf{x}_{V^\perp}$ to prove the Pythagorean identity $\|\mathbf{x}\|^2 = \|\mathbf{x}_V\|^2 + \|\mathbf{x}_{V^\perp}\|^2$. Then explain in one sentence why this means the projection $\mathbf{x}_V$ is *never longer* than $\mathbf{x}$ itself – a fact you will use constantly in the projections chapter.

Exercise 17.12. (Solvability and the left nullspace.) Theorem 17.3 established that $\operatorname{null}(\mathbf{A}^\mathsf{T}) = \operatorname{col}(\mathbf{A})^\perp$, which by Corollary 17.1 means $\operatorname{col}(\mathbf{A}) = (\operatorname{null}(\mathbf{A}^\mathsf{T}))^\perp$. Explain in two plain sentences why this provides the universal test for solvability: the linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ has an exact solution if and only if $\mathbf{b}$ is orthogonal to every vector in the left nullspace $\operatorname{null}(\mathbf{A}^\mathsf{T})$. (This celebrated result is the finite-dimensional *Fredholm alternative*.)

Chapter 18

Projections

Suppose you stand in a flat field off a straight road and want the shortest walk to the road. You walk straight toward it, meeting the road at a right angle. That foot of the perpendicular is the closest point – the *orthogonal projection*.

Nothing about that picture changes when we move from a line in $\mathbb{R}^2$ to a plane in $\mathbb{R}^3$ or an r-dimensional subspace inside $\mathbb{R}^n$: to find the point in a subspace closest to $\mathbf{b}$, drop a perpendicular.

The reason projections matter so much is that linear systems in practice are often overdetermined. In $\mathbf{Ax} = \mathbf{b}$, when $\mathbf{b}$ is not in the column space $\text{col}(\mathbf{A})$, there is no exact solution. Instead of seeking an exact solution that does not exist, we find the closest reachable vector: the orthogonal projection of $\mathbf{b}$ onto $\text{col}(\mathbf{A})$. This is the foundation of least-squares approximation, linear regression, and the normal equations.

18.1 Projecting onto a line

Start with the simplest case: a single line through the origin, and a point we want to approximate. Let the line be the span of a vector $\mathbf{a}$, and let $\mathbf{b}$ be the point we are trying to reach. The line is $\{\, t\mathbf{a} : t \in \mathbb{R} \,\}$ – all scalar multiples of $\mathbf{a}$. We seek the point $\mathbf{p}$ on this line that minimizes the distance $\|\mathbf{b} - \mathbf{p}\|$.

Intuition

The projection $\mathbf{p}$ is the point on the line closest to $\mathbf{b}$. The segment connecting $\mathbf{b}$ to $\mathbf{p}$ meets the line at an exact right angle. This perpendicular vector is the *error* $\mathbf{e} = \mathbf{b} - \mathbf{p}$, and demanding that the error be orthogonal to $\mathbf{a}$ uniquely determines $\mathbf{p}$.

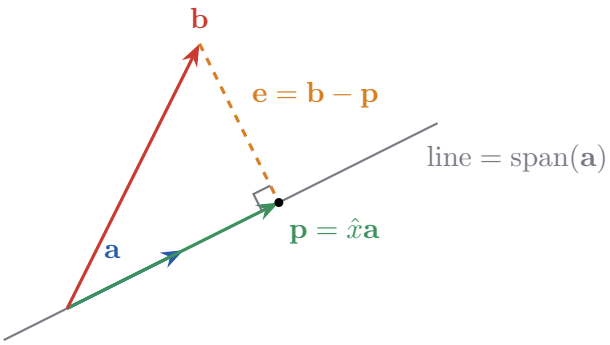


Figure 18.1: Projection onto a line. The shadow $\mathbf{p}$ is the point on span($\mathbf{a}$) closest to $\mathbf{b}$. The error $\mathbf{e} = \mathbf{b} - \mathbf{p}$ (orange, dashed) meets the line at an exact right angle.

18.1.1 Finding the projection from the right angle

Because $\mathbf{p}$ lies on the line, it is *some* multiple of $\mathbf{a}$:

$$\mathbf{p} = \hat{x}\,\mathbf{a}$$

for an unknown scalar $\hat{x}$ (read "x-hat"; the hat will always mean "our best estimate"). The whole problem is to find this one number. We have one fact to pin it down: the error $\mathbf{e} = \mathbf{b} - \mathbf{p} = \mathbf{b} - \hat{x}\mathbf{a}$ must be *perpendicular to the line*, which means perpendicular to $\mathbf{a}$. Perpendicular means the dot product is zero:

$$\mathbf{a}^{\mathsf{T}}(\mathbf{b} - \hat{x}\mathbf{a}) = 0.$$

That is one linear equation in the one unknown $\hat{x}$. Expand it:

$$\mathbf{a}^{\mathsf{T}}\mathbf{b} - \hat{x}\,\mathbf{a}^{\mathsf{T}}\mathbf{a} = 0 \qquad \Longrightarrow \qquad \hat{x} = \frac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}.$$

And then the projection itself is

$$\boxed{\mathbf{p} = \hat{x}\,\mathbf{a} = \frac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}\,\mathbf{a}.}$$

Focus

The projection of $\mathbf{b}$ onto the line span($\mathbf{a}$) is

$$\mathbf{p} = \frac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}\,\mathbf{a}, \qquad \hat{x} = \frac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}.$$

The numerator $\mathbf{a}^{\mathsf{T}}\mathbf{b}$ measures how much $\mathbf{b}$ leans along $\mathbf{a}$; the denominator $\mathbf{a}^{\mathsf{T}}\mathbf{a} = \|\mathbf{a}\|^2$ normalizes for how long $\mathbf{a}$ is. Notice $\mathbf{p}$ does not change if you rescale $\mathbf{a}$ – the line is what matters, not the particular arrow on it.

Example 18.1. Project $\mathbf{b} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ onto the line through $\mathbf{a} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ (the x-axis).

$$\mathbf{a}^{\mathsf{T}}\mathbf{b} = 1 \cdot 2 + 0 \cdot 3 = 2, \qquad \mathbf{a}^{\mathsf{T}}\mathbf{a} = 1, \qquad \hat{x} = \tfrac{2}{1} = 2,$$

so $\mathbf{p} = 2\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}2\\0\end{bmatrix}$ and the error is $\mathbf{e} = \mathbf{b} - \mathbf{p} = \begin{bmatrix}0\\3\end{bmatrix}$. Of course: projecting onto the x-axis just keeps the x-coordinate and drops the y-coordinate. And indeed $\mathbf{e} = \begin{bmatrix}0\\3\end{bmatrix}$ is perpendicular to $\mathbf{a} = \begin{bmatrix}1\\0\end{bmatrix}$, since their dot product is 0. The formula reproduces what you already knew, which is exactly what a good formula should do on an easy case.

Example 18.2. Now a case you could not eyeball. Project $\mathbf{b} = \begin{bmatrix}1\\4\end{bmatrix}$ onto the line through $\mathbf{a} = \begin{bmatrix}2\\1\end{bmatrix}$.

$$\mathbf{a}^{\mathsf{T}}\mathbf{b} = 2\cdot 1 + 1\cdot 4 = 6, \qquad \mathbf{a}^{\mathsf{T}}\mathbf{a} = 2^2 + 1^2 = 5, \qquad \hat{x} = \tfrac{6}{5}.$$

So

$$\mathbf{p} = \tfrac{6}{5}\begin{bmatrix}2\\1\end{bmatrix} = \begin{bmatrix}12/5\\6/5\end{bmatrix}, \qquad \mathbf{e} = \mathbf{b} - \mathbf{p} = \begin{bmatrix}1 - 12/5\\4 - 6/5\end{bmatrix} = \begin{bmatrix}-7/5\\14/5\end{bmatrix}.$$

Check perpendicularity: $\mathbf{a}^{\mathsf{T}}\mathbf{e} = 2(-7/5)+1(14/5) = -14/5+14/5 = 0$. The error really is orthogonal to the line. This check – "dot the error with $\mathbf{a}$ and get zero" – is the single most reliable way to catch an arithmetic slip in any projection problem.

Walking question

Take the formula $\mathbf{p} = \dfrac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}\,\mathbf{a}$ on a walk and ask: what happens if $\mathbf{b}$ is already *on* the line, say $\mathbf{b} = 5\mathbf{a}$? What should the shadow be, and does the formula give it? And what if $\mathbf{b}$ is perpendicular to $\mathbf{a}$ – where does the shadow land then? Two sanity checks that a projection must pass.

18.1.2 Packaging the projection as a matrix

Look again at $\mathbf{p} = \dfrac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}\,\mathbf{a}$. The quantity $\mathbf{a}^{\mathsf{T}}\mathbf{b}$ is a number, so we are free to slide it around. Watch:

$$\mathbf{p} = \mathbf{a}\,\frac{\mathbf{a}^{\mathsf{T}}\mathbf{b}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}} = \frac{\mathbf{a}\,\mathbf{a}^{\mathsf{T}}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}\,\mathbf{b} = \mathbf{P}\,\mathbf{b}, \qquad \text{where} \qquad \mathbf{P} = \frac{\mathbf{a}\,\mathbf{a}^{\mathsf{T}}}{\mathbf{a}^{\mathsf{T}}\mathbf{a}}.$$

Look at what happened. The object $\mathbf{a}\,\mathbf{a}^{\mathsf{T}}$ is a *column times a row* – an $n \times 1$ times a $1 \times n$ – so it is an $n \times n$ *matrix*, not a number. (Contrast $\mathbf{a}^{\mathsf{T}}\mathbf{a}$, a row times a column, which is a single number.) Dividing that matrix by the number $\mathbf{a}^{\mathsf{T}}\mathbf{a}$ gives the **projection matrix** $\mathbf{P}$. It does not depend on $\mathbf{b}$ at all. Build it once, and then projecting *any* vector onto the line is a single matrix multiply $\mathbf{p} = \mathbf{P}\mathbf{b}$.

Common trap

Keep the two products straight. $\mathbf{a}^{\mathsf{T}}\mathbf{a}$ (row times column) is a *scalar*; for $\mathbf{a} = \begin{bmatrix}2\\1\end{bmatrix}$ it is 5. But $\mathbf{a}\,\mathbf{a}^{\mathsf{T}}$ (column times row) is a *matrix*:

$$\mathbf{a}\,\mathbf{a}^{\mathsf{T}} = \begin{bmatrix}2\\1\end{bmatrix}\begin{bmatrix}2 & 1\end{bmatrix} = \begin{bmatrix}4 & 2\\2 & 1\end{bmatrix}.$$

Swapping them is the most common projection mistake. One collapses to a number; the other expands to a square matrix. Order matters with $^\mathsf{T}$.

Example 18.3. For $\mathbf{a} = \begin{bmatrix}2\\1\end{bmatrix}$ we found $\mathbf{a}\,\mathbf{a}^\mathsf{T} = \begin{bmatrix}4 & 2\\2 & 1\end{bmatrix}$ and $\mathbf{a}^\mathsf{T}\mathbf{a} = 5$, so

$$\mathbf{P} = \frac{1}{5}\begin{bmatrix}4 & 2\\2 & 1\end{bmatrix} = \begin{bmatrix}0.8 & 0.4\\0.4 & 0.2\end{bmatrix}.$$

Apply it to $\mathbf{b} = \begin{bmatrix}1\\4\end{bmatrix}$ from Example 18.2:

$$\mathbf{P}\mathbf{b} = \frac{1}{5}\begin{bmatrix}4 & 2\\2 & 1\end{bmatrix}\begin{bmatrix}1\\4\end{bmatrix} = \frac{1}{5}\begin{bmatrix}4+8\\2+4\end{bmatrix} = \frac{1}{5}\begin{bmatrix}12\\6\end{bmatrix} = \begin{bmatrix}12/5\\6/5\end{bmatrix}.$$

Same $\mathbf{p}$ as before. The matrix has packaged the whole projection into one reusable object.

This little $\mathbf{P}$ already shows the two features that turn out to define every projection matrix. It is **symmetric**: $\mathbf{P} = \mathbf{P}^\mathsf{T}$ (look at the 0.4s off the diagonal). And projecting twice does nothing new – once you are on the line, projecting again leaves you put – so $\mathbf{P}^2 = \mathbf{P}$. The second deserves a check:

$$\mathbf{P}^2 = \frac{1}{25}\begin{bmatrix}4 & 2\\2 & 1\end{bmatrix}\begin{bmatrix}4 & 2\\2 & 1\end{bmatrix} = \frac{1}{25}\begin{bmatrix}16+4 & 8+2\\8+2 & 4+1\end{bmatrix} = \frac{1}{25}\begin{bmatrix}20 & 10\\10 & 5\end{bmatrix} = \frac{1}{5}\begin{bmatrix}4 & 2\\2 & 1\end{bmatrix} = \mathbf{P}.$$

Why this matters

Geometrically, an orthogonal projection retains the component of a vector lying within a subspace and eliminates the component perpendicular to it. The algebraic property $\mathbf{P}^2 = \mathbf{P}$ reflects this stabilization: once a vector has been projected into a subspace S, applying the projection a second time leaves the vector unchanged. A matrix satisfying $\mathbf{P}^2 = \mathbf{P}$ is called *idempotent*. As we will see when studying eigenvalues, an idempotent matrix can scale eigenvectors only by 1 (on S) or by 0 (on $S^\perp$).

18.2 Why the perpendicular is the closest point

We have used the fact that the perpendicular dropped to a subspace gives the nearest point. We now give the formal proof using the Pythagorean theorem.

Theorem 18.1 (The projection is the closest point)**.** *Let* $\mathbf{p}$ *be the projection of* $\mathbf{b}$ *onto a subspace* S*, so that the error* $\mathbf{e} = \mathbf{b} - \mathbf{p}$ *is orthogonal to* S*. Then for* every *other point* $\mathbf{y} \in S$*,*

$$\|\mathbf{b} - \mathbf{y}\| \;\geq\; \|\mathbf{b} - \mathbf{p}\|,$$

with equality only when $\mathbf{y} = \mathbf{p}$*. The projection is strictly the closest point of* S *to* $\mathbf{b}$*.*

Proof. Pick any $\mathbf{y} \in S$ and split the trip from $\mathbf{b}$ to $\mathbf{y}$ into two legs through $\mathbf{p}$:

$$\mathbf{b} - \mathbf{y} = (\mathbf{b} - \mathbf{p}) + (\mathbf{p} - \mathbf{y}).$$

The first leg $\mathbf{b} - \mathbf{p} = \mathbf{e}$ is orthogonal to S. The second leg $\mathbf{p} - \mathbf{y}$ lives *inside* S (both $\mathbf{p}$ and $\mathbf{y}$ are in the subspace, and subspaces are closed under subtraction). Orthogonal-to-S and inside-S means these two legs are perpendicular to each other, so Pythagoras applies:

$$\|\mathbf{b} - \mathbf{y}\|^2 = \|\mathbf{b} - \mathbf{p}\|^2 + \|\mathbf{p} - \mathbf{y}\|^2.$$

The right side is $\|\mathbf{b} - \mathbf{p}\|^2$ plus something that is never negative, so it is at least $\|\mathbf{b} - \mathbf{p}\|^2$. It equals it only when $\|\mathbf{p} - \mathbf{y}\|^2 = 0$, i.e. when $\mathbf{y} = \mathbf{p}$. □

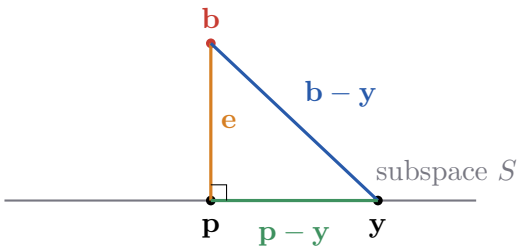


Figure 18.2: Why the perpendicular minimizes distance. Any other point $\mathbf{y}$ in S sits at the far end of a right triangle whose legs are the perpendicular error $\mathbf{e}$ and an in-subspace step. The hypotenuse $\mathbf{b} - \mathbf{y}$ is strictly longer than the perpendicular leg $\mathbf{e}$ whenever $\mathbf{y} \neq \mathbf{p}$.

Focus

The entire theory of projection rests on a single geometric principle: **the error vector is orthogonal to the subspace**. By the Pythagorean theorem, the perpendicular distance from a point to a subspace is strictly shorter than the distance to any other point in the subspace.

18.3 Projecting onto a subspace

Now we leave the line behind. Suppose S is not a single direction but a whole subspace – a plane in $\mathbb{R}^3$, or an r-dimensional flat slab inside $\mathbb{R}^n$. Give S a set of basis vectors and stack them as the columns of a matrix:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_r \end{bmatrix}, \qquad S = \text{col}(\mathbf{A}).$$

Every point of S is a combination of the columns, i.e. has the form $\mathbf{Ax}$ for some coefficient vector $\mathbf{x} \in \mathbb{R}^r$. So projecting $\mathbf{b}$ onto S means: find the coefficient vector $\hat{\mathbf{x}}$ that makes $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$ as close to $\mathbf{b}$ as possible.

Intuition

For an r-dimensional subspace, we seek the coefficient vector $\hat{\mathbf{x}} = (\hat{x}_1, \ldots, \hat{x}_r)$ determining the optimal linear combination $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} \in S$. As with a line, the condition determining $\hat{\mathbf{x}}$ is that the error vector $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ must be orthogonal to every direction in S.

18.3.1 The normal equations

The error is $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$. "Perpendicular to S" now means perpendicular to *every* column of $\mathbf{A}$, because the columns span S. (Why does checking the basis columns suffice? Any vector in S has the

form $\mathbf{Ay}$; if $\mathbf{A}^\mathsf{T}\mathbf{e} = \mathbf{0}$, then the dot product with any vector in S is $(\mathbf{Ay})^\mathsf{T}\mathbf{e} = \mathbf{y}^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{e}) = \mathbf{y}^\mathsf{T}\mathbf{0} = 0$, so $\mathbf{e}$ is automatically perpendicular to the entire subspace.) So we need $\mathbf{a}_1^\mathsf{T}\mathbf{e} = 0$, $\mathbf{a}_2^\mathsf{T}\mathbf{e} = 0$, all the way to $\mathbf{a}_r^\mathsf{T}\mathbf{e} = 0$. Stacking these r equations is exactly what $\mathbf{A}^\mathsf{T}\mathbf{e} = \mathbf{0}$ says:

$$\mathbf{A}^\mathsf{T}\mathbf{e} = \begin{bmatrix} \mathbf{a}_1^\mathsf{T} \\ \vdots \\ \mathbf{a}_r^\mathsf{T} \end{bmatrix} \mathbf{e} = \begin{bmatrix} \mathbf{a}_1^\mathsf{T}\mathbf{e} \\ \vdots \\ \mathbf{a}_r^\mathsf{T}\mathbf{e} \end{bmatrix} = \mathbf{0}.$$

Substitute $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ and expand:

$$\mathbf{A}^\mathsf{T}(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0} \qquad \Longrightarrow \qquad \boxed{\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}.}$$

These are the famous **normal equations**. They are "normal" in the old sense of the word – *normal* means *perpendicular*, and the whole system was born from the perpendicularity condition $\mathbf{A}^\mathsf{T}\mathbf{e} = \mathbf{0}$. Compare line and subspace side by side and feel how little changed:

$$\text{line:} \quad (\mathbf{a}^\mathsf{T}\mathbf{a})\,\hat{x} = \mathbf{a}^\mathsf{T}\mathbf{b}, \qquad\qquad \text{subspace:} \quad (\mathbf{A}^\mathsf{T}\mathbf{A})\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}.$$

The scalar $\mathbf{a}^\mathsf{T}\mathbf{a}$ grew into the $r \times r$ matrix $\mathbf{A}^\mathsf{T}\mathbf{A}$; the number $\mathbf{a}^\mathsf{T}\mathbf{b}$ grew into the vector $\mathbf{A}^\mathsf{T}\mathbf{b}$. Same skeleton, more dimensions.

Formal

The projection matrix. If the columns of $\mathbf{A}$ are independent, then $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible, and we may solve the normal equations:

$$\hat{\mathbf{x}} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b}, \qquad \mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\,\mathbf{b}.$$

The matrix that projects any $\mathbf{b}$ onto $\mathrm{col}(\mathbf{A})$ is therefore

$$\boxed{\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}.}$$

Notice how this collapses to the line formula when $\mathbf{A}$ is a single column $\mathbf{a}$. Then $\mathbf{A}^\mathsf{T}\mathbf{A} = \mathbf{a}^\mathsf{T}\mathbf{a}$ is the scalar we divide by, $(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}$ is just $1/(\mathbf{a}^\mathsf{T}\mathbf{a})$, and $\mathbf{P} = \mathbf{a}(\mathbf{a}^\mathsf{T}\mathbf{a})^{-1}\mathbf{a}^\mathsf{T} = \dfrac{\mathbf{a}\,\mathbf{a}^\mathsf{T}}{\mathbf{a}^\mathsf{T}\mathbf{a}}$. The grand formula and the baby formula are the same formula.

Common trap

You might be tempted to "simplify" $\mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ by distributing the inverse: $\mathbf{A}\mathbf{A}^{-1}(\mathbf{A}^\mathsf{T})^{-1}\mathbf{A}^\mathsf{T} = \mathbf{I}$. **Do not.** That algebra is illegal unless $\mathbf{A}$ is square and invertible – and if $\mathbf{A}$ were square and invertible, its column space would be *all* of $\mathbb{R}^n$, every $\mathbf{b}$ would already be in S, and projection would indeed be the identity. The interesting case is the rectangular one ($m > r$): $\mathbf{A}$ has more rows than columns, $\mathbf{A}^{-1}$ does not exist, and $\mathbf{P}$ is genuinely *not* the identity. Treat $(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}$ as one indivisible block.

Pause

Mental check: when A is square and invertible. What happens to the formula $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ if $\mathbf{A}$ is a square, invertible $n \times n$ matrix? *Pause and predict before checking*

the algebra: If the columns of $\mathbf{A}$ form a basis of all of $\mathbb{R}^n$, then $\text{col}(\mathbf{A})$ is the entire ambient space. Every vector $\mathbf{b}$ already lives in the subspace, so the closest point to $\mathbf{b}$ is $\mathbf{b}$ itself, and the projection must be the identity matrix $\mathbf{I}$. And indeed, only in this square invertible case can we legally distribute the inverse: $\mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T} = \mathbf{A}\mathbf{A}^{-1}(\mathbf{A}^\mathsf{T})^{-1}\mathbf{A}^\mathsf{T} = \mathbf{I}\cdot\mathbf{I} = \mathbf{I}$.

18.3.2 Why $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible when the columns are independent

We leaned on "$\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible" – let us prove it now. It is a small gem of an argument and it tells you exactly when projection works.

Lemma 18.1. *If the columns of $\mathbf{A}$ are linearly independent, then $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible.*

Proof. $\mathbf{A}^\mathsf{T}\mathbf{A}$ is a square $r \times r$ matrix, so it suffices to show its nullspace is trivial: the only $\mathbf{x}$ with $\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$. Suppose $\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{0}$. Dot both sides with $\mathbf{x}$:

$$\mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{x} = 0 \qquad \Longrightarrow \qquad (\mathbf{A}\mathbf{x})^\mathsf{T}(\mathbf{A}\mathbf{x}) = 0 \qquad \Longrightarrow \qquad \|\mathbf{A}\mathbf{x}\|^2 = 0.$$

A vector with zero length is the zero vector, so $\mathbf{A}\mathbf{x} = \mathbf{0}$. But the columns of $\mathbf{A}$ are independent, which means the *only* combination of them giving zero is the trivial one: $\mathbf{x} = \mathbf{0}$. Hence $\mathbf{A}^\mathsf{T}\mathbf{A}$ has trivial nullspace and is invertible. □

The middle step is a trick worth keeping: $\mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{x} = \|\mathbf{A}\mathbf{x}\|^2$ turns a matrix expression into a squared length, which can only be zero one way. You will see this move again in Chapter 27, where positive-definite matrices make it the central theme.

18.4 The two properties that define a projection

Whatever $\mathbf{A}$ is, the matrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ has two properties, and they are so characteristic that they *define* what it means to be an orthogonal projection.

Theorem 18.2 (Symmetry and idempotence)**.** *The projection matrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ satisfies*

$$\textit{(1)}\quad \mathbf{P}^\mathsf{T} = \mathbf{P} \qquad \textit{and} \qquad \textit{(2)}\quad \mathbf{P}^2 = \mathbf{P}.$$

Proof. Symmetry. Transpose, remembering that $(\mathbf{X}\mathbf{Y}\mathbf{Z})^\mathsf{T} = \mathbf{Z}^\mathsf{T}\mathbf{Y}^\mathsf{T}\mathbf{X}^\mathsf{T}$ and that $\mathbf{A}^\mathsf{T}\mathbf{A}$ is itself symmetric (so its inverse is too):

$$\mathbf{P}^\mathsf{T} = \left(\mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\right)^\mathsf{T} = (\mathbf{A}^\mathsf{T})^\mathsf{T}\left((\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\right)^\mathsf{T}\mathbf{A}^\mathsf{T} = \mathbf{A}\,(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\,\mathbf{A}^\mathsf{T} = \mathbf{P}.$$

Idempotence. Multiply $\mathbf{P}$ by itself and watch the middle collapse. Let $\mathbf{M} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}$ for brevity:

$$\mathbf{P}^2 = \mathbf{A}\mathbf{M}\mathbf{A}^\mathsf{T}\,\mathbf{A}\mathbf{M}\mathbf{A}^\mathsf{T} = \mathbf{A}\mathbf{M}\,(\mathbf{A}^\mathsf{T}\mathbf{A})\,\mathbf{M}\,\mathbf{A}^\mathsf{T}.$$

But $\mathbf{M}(\mathbf{A}^\mathsf{T}\mathbf{A}) = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}(\mathbf{A}^\mathsf{T}\mathbf{A}) = \mathbf{I}$, so the middle three factors $\mathbf{M}(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{M}$ collapse to $\mathbf{M}$, leaving $\mathbf{P}^2 = \mathbf{A}\mathbf{M}\mathbf{A}^\mathsf{T} = \mathbf{P}$. □

Intuition

Both properties are obvious *geometrically*, which is the best kind of proof to carry in your head. Idempotence $\mathbf{P}^2 = \mathbf{P}$: once $\mathbf{b}$'s shadow has landed in S, it is already in S, so projecting it again does nothing. Symmetry $\mathbf{P}^\mathsf{T} = \mathbf{P}$ is subtler but says the projection "treats both directions fairly": the part of $\mathbf{b}$ that points into $\mathbf{c}$'s shadow equals the part of $\mathbf{c}$ that points into $\mathbf{b}$'s shadow, i.e. $\mathbf{c}^\mathsf{T}\mathbf{P}\mathbf{b} = (\mathbf{P}\mathbf{c})^\mathsf{T}\mathbf{b}$. Drop symmetry and you get a *slanted* (oblique) projection that still satisfies $\mathbf{P}^2 = \mathbf{P}$ but no longer drops a true perpendicular – and no longer finds the closest point (Figure 18.3).

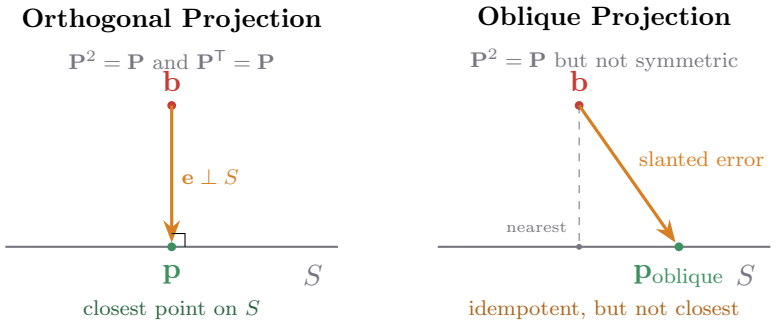


Figure 18.3: Why symmetry matters. **Left:** when $\mathbf{P}$ is both idempotent and symmetric, the error $\mathbf{e}$ drops straight down at a right angle, finding the unique nearest point. **Right:** when $\mathbf{P}^2 = \mathbf{P}$ but $\mathbf{P}^\mathsf{T} \neq \mathbf{P}$, the shadow slides along a tilted ray. It is still a projection (once on S, projecting again does nothing), but it fails to minimize the distance to $\mathbf{b}$.

Pause

Mental check: projecting the zero vector. Where does any projection matrix send $\mathbf{0}$? And if a matrix satisfies $\mathbf{P}\mathbf{v} = \mathbf{v}$ for *every* vector $\mathbf{v}$ in $\mathbb{R}^n$, what is $\mathbf{P}$? *Pause and think:* $\mathbf{P}\mathbf{0} = \mathbf{0}$, because every linear subspace passes through the origin. If $\mathbf{P}\mathbf{v} = \mathbf{v}$ everywhere, the subspace is the entire ambient space $\mathbb{R}^n$, so the error is always zero and $\mathbf{P} = \mathbf{I}$.

History and connection

The geometric idea that the shortest path from a point to a plane is the perpendicular dropped onto it was known to ancient Greek geometers. But converting that geometric intuition into an explicit algebraic operator $\mathbf{P}$ that acts on arbitrary multidimensional vectors was the achievement of nineteenth- and twentieth-century analysis.

The concept matured through physical variational principles, notably Dirichlet's principle in potential theory, where nature minimizes field energy by finding the projection of a state onto admissible boundary configurations. In the early 1900s, Maurice Fréchet and David Hilbert formalized projection theorems in general inner-product spaces. They proved that in any complete linear space, every vector decomposes uniquely into an orthogonal projection and a perpendicular error vector that achieves the absolute minimum Euclidean distance.

Remark 18.1. The complementary matrix $\mathbf{I}-\mathbf{P}$ is *also* a projection – it projects onto the orthogonal complement of S, the space where the errors live. Check: $(\mathbf{I}-\mathbf{P})^2 = \mathbf{I}-2\mathbf{P}+\mathbf{P}^2 = \mathbf{I}-2\mathbf{P}+\mathbf{P} = \mathbf{I}-\mathbf{P}$, and it is symmetric because $\mathbf{P}$ is. So $\mathbf{P}$ keeps the part of $\mathbf{b}$ inside S, and $\mathbf{I}-\mathbf{P}$ keeps the part

orthogonal to S; together they split every vector cleanly, $\mathbf{b} = \mathbf{P}\mathbf{b} + (\mathbf{I} - \mathbf{P})\mathbf{b} = \mathbf{p} + \mathbf{e}$.

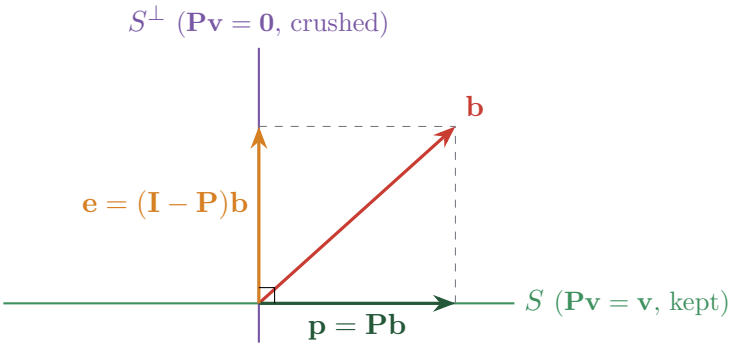


Figure 18.4: One projection splits space in two. The kept part $\mathbf{p} = \mathbf{Pb}$ lives in S (where $\mathbf{P}$ acts as the identity: $\mathbf{Pp} = \mathbf{p}$); the forgotten part $\mathbf{e} = (\mathbf{I} - \mathbf{P})\mathbf{b}$ lives in the orthogonal complement $S^\perp$ (where $\mathbf{P}$ sends everything to zero: $\mathbf{Pe} = \mathbf{0}$). Their right-angled sum rebuilds $\mathbf{b}$. (In Chapter 23, these invariant directions will be recognized as eigenspaces for scale factors 1 and 0.)

18.5 A fully worked plane in three dimensions

To see the projection formula in concrete computation, let us project a point onto a plane in $\mathbb{R}^3$ from start to finish.

Example 18.4. Project $\mathbf{b} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}$ onto the plane S spanned by $\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$. Assemble

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Step 1: form $\mathbf{A}^\mathsf{T}\mathbf{A}$.

$$\mathbf{A}^\mathsf{T}\mathbf{A} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Step 2: form $\mathbf{A}^\mathsf{T}\mathbf{b}$.

$$\mathbf{A}^\mathsf{T}\mathbf{b} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \end{bmatrix}.$$

Step 3: solve the normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$. The inverse of $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ is $\frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ (determinant $4 - 1 = 3$), so

$$\hat{\mathbf{x}} = \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 0 \end{bmatrix} = \frac{1}{3}\begin{bmatrix} 12 \\ -6 \end{bmatrix} = \begin{bmatrix} 4 \\ -2 \end{bmatrix}.$$

Step 4: form the projection $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$.

$$\mathbf{p} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ -2 \end{bmatrix} = \begin{bmatrix} 4 \\ 4-2 \\ -2 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \\ -2 \end{bmatrix}.$$

Step 5: the error and the all-important check.

$$\mathbf{e} = \mathbf{b} - \mathbf{p} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 4 \\ 2 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix}.$$

Is $\mathbf{e}$ perpendicular to the plane? Dot it with each basis vector:

$$\mathbf{a}_1^{\mathsf{T}}\mathbf{e} = 1 \cdot 2 + 1 \cdot (-2) + 0 \cdot 2 = 0, \qquad \mathbf{a}_2^{\mathsf{T}}\mathbf{e} = 0 \cdot 2 + 1 \cdot (-2) + 1 \cdot 2 = 0.$$

Both zero. The error is genuinely orthogonal to the plane, so $\mathbf{p} = \begin{bmatrix} 4 \\ 2 \\ -2 \end{bmatrix}$ is the closest point of S to $\mathbf{b}$. Notice $\mathbf{e} = \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix} = 2\begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$ points along the plane's *normal* direction $\begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$ – exactly the direction perpendicular to both $\mathbf{a}_1$ and $\mathbf{a}_2$, as it must.

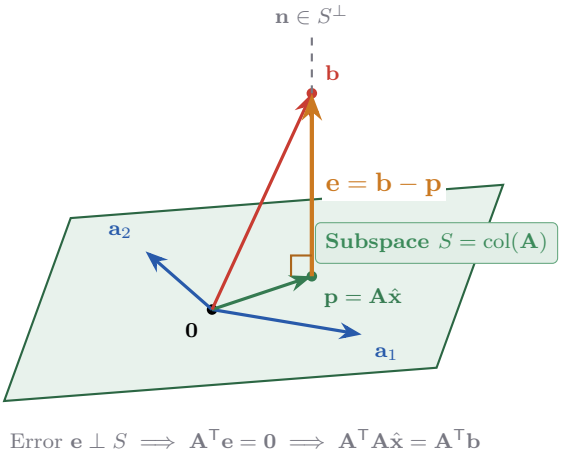


Figure 18.5: Orthogonal projection of $\mathbf{b}$ onto a 2D subspace $S = \text{col}(\mathbf{A})$. The projection $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$ is the unique closest point in the plane. The residual error vector $\mathbf{e} = \mathbf{b} - \mathbf{p}$ drops straight down, forming a right angle with the basis directions $\mathbf{a}_1, \mathbf{a}_2$ and therefore with every vector in S.

Compute it yourself

Build the full 3×3 projection matrix for that plane and feel it work. With $\mathbf{A}^\mathsf{T}\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ and inverse $\frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$, multiply out $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$. You should land on

$$\mathbf{P} = \frac{1}{3}\begin{bmatrix} 2 & 1 & -1 \\ 1 & 2 & 1 \\ -1 & 1 & 2 \end{bmatrix}.$$

Check three things by hand: (a) $\mathbf{P}$ is symmetric; (b) $\mathbf{Pb} = \begin{bmatrix} 4 \\ 2 \\ -2 \end{bmatrix}$ for $\mathbf{b} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}$, matching Step 4; (c) the *trace* (the sum of the diagonal entries, $\mathrm{tr}(\mathbf{P}) = P_{11}+P_{22}+P_{33}$) is $\frac{1}{3}(2+2+2) = 2$, which directly equals the dimension of the plane. Summing the diagonal of any projection matrix always counts the dimension of the subspace it projects onto.

18.6 Two special cases worth checking

Two extreme cases test whether you really understand $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$.

When b is already in the space. If $\mathbf{b} \in \mathrm{col}(\mathbf{A})$ then its closest point is itself, so we expect $\mathbf{Pb} = \mathbf{b}$. Indeed, $\mathbf{b} = \mathbf{Ac}$ for some $\mathbf{c}$, and

$$\mathbf{Pb} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{Ac} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{c} = \mathbf{Ac} = \mathbf{b}.$$

Projection leaves the space it projects onto fixed.

When b is orthogonal to the space. If $\mathbf{b} \perp \mathrm{col}(\mathbf{A})$ then $\mathbf{A}^\mathsf{T}\mathbf{b} = \mathbf{0}$ (every column dots to zero with $\mathbf{b}$), so $\hat{\mathbf{x}} = \mathbf{0}$ and $\mathbf{p} = \mathbf{0}$. The shadow collapses to the origin. All of $\mathbf{b}$ becomes error.

Why this matters

Compare how $\mathbf{P}$ acts on these two families of vectors. Vectors *in* the subspace S are untouched: $\mathbf{Pv} = 1 \cdot \mathbf{v}$ (they are kept). Vectors *perpendicular* to the subspace are mapped to zero: $\mathbf{Pv} = 0 \cdot \mathbf{v}$.

This is our first encounter with one of the most central themes of linear algebra: finding the special directions that a matrix merely stretches by a numerical scale factor rather than rotating. In Chapter 23, we will formalize these directions as *eigenvectors*, and their scaling factors as *eigenvalues*. A projection matrix has only two possible scaling factors: 1 (for directions inside S) and 0 (for directions perpendicular to S). The algebraic condition $\mathbf{P}^2 = \mathbf{P}$ demands this: if scaling a vector twice has the exact same effect as scaling it once ($\lambda^2 = \lambda$), the scale factor λ can only ever be 1 or 0. The number of independent directions scaled by 1 is the dimension of S—which also equals the rank of $\mathbf{P}$, and the sum of its diagonal entries (the trace). The spectral structure of an orthogonal projection is entirely determined by these two eigenvalues: 1 and 0.

18.7 Where this is heading: the inconsistent system

We opened with the promise that projection rescues the unsolvable equation $\mathbf{Ax} = \mathbf{b}$. Let us close the loop. When $\mathbf{b}$ is not in the column space, no $\mathbf{x}$ makes $\mathbf{Ax} = \mathbf{b}$. The sensible move is to solve instead the *nearest* solvable problem: replace $\mathbf{b}$ by its projection $\mathbf{p}$ onto col($\mathbf{A}$), and solve $\mathbf{A}\hat{\mathbf{x}} = \mathbf{p}$. By construction $\mathbf{p}$ *is* in the column space, so this has a solution – and it is the very $\hat{\mathbf{x}}$ from the normal equations. We have, almost without trying, derived the least-squares solution.

Focus

The bridge to least squares. To "solve" an inconsistent $\mathbf{Ax} = \mathbf{b}$, project $\mathbf{b}$ onto col($\mathbf{A}$) and accept the projection $\mathbf{p}$ as the best we can hit. The best-fit coefficients are

$$\hat{\mathbf{x}} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b},$$

and they minimize the squared error $\|\mathbf{Ax} - \mathbf{b}\|^2$ – because, by Theorem 18.1, the projection is the closest point. The next chapter puts this one idea to work on curve fitting and regression.

Modern application

Least squares is everywhere a model meets noisy data. Suppose you measure a quantity that you believe depends linearly on inputs – house price versus square footage and number of bedrooms, say. Each data point is a row of $\mathbf{A}$ (the features) and an entry of $\mathbf{b}$ (the observed price). There are far more houses than features, so $\mathbf{Ax} = \mathbf{b}$ is wildly overdetermined and has no exact solution: no single line passes through every messy point. The linear-regression coefficients that every spreadsheet, statistics package, and machine-learning library computes are *exactly* $\hat{\mathbf{x}} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b}$. The "predictions" $\hat{\mathbf{b}} = \mathbf{Pb}$ are the projection of reality onto the span of your features, and the residuals $\mathbf{e} = \mathbf{b} - \hat{\mathbf{b}}$ are the part of reality your model cannot explain – guaranteed orthogonal to every feature, which is why a well-fit linear model's residuals show no leftover linear trend. The matrix $\mathbf{P}$ even has a name in statistics: the *hat matrix*, because it puts the hat on $\mathbf{b}$ to make $\hat{\mathbf{b}}$.

Connection to other math

Projection in function spaces: Fourier series. The same perpendicular-drop works when the "vectors" are functions. Take the inner product $\langle f, g\rangle = \int_0^{2\pi} f(t)g(t)\,dt$. The functions $1, \cos t, \sin t, \cos 2t, \sin 2t, \ldots$ are mutually orthogonal under this inner product – just like an orthogonal basis of $\mathbb{R}^n$, but infinite. A Fourier series approximation of a signal f is precisely the *projection* of f onto the finite-dimensional subspace spanned by the first few of these waves. Each Fourier coefficient is a one-dimensional projection, $\hat{x}_k = \langle f, \phi_k\rangle / \langle \phi_k, \phi_k\rangle$ – the line-projection formula from Figure 18.1, now with integrals for dot products. "Find the closest combination of sines and cosines" *is* least squares in a space of functions, and the leftover (the part f minus its truncated series) is orthogonal to every wave you kept. This is the doorway from linear algebra into Fourier analysis and the whole field of signal processing.

Before/after Strang

Strang builds projections in essentially this order – line first, then subspace, then the matrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ – and he hammers the same two properties, $\mathbf{P}^\mathsf{T} = \mathbf{P}$ and $\mathbf{P}^2 = \mathbf{P}$. If you are reading this *before* Strang, the move to internalize is "error orthogonal to the space ⇒ closest point ⇒ normal equations." Everything in his projection and least-squares lectures hangs on that chain. If you are reading this *after* Strang, look ahead: $\mathbf{P}$ is your first nontrivial example of a matrix whose action is completely described by its eigenvalues (0 and 1) and eigenspaces (S and $S^\perp$). When you reach the spectral theorem and the SVD, you will recognize projection as the simplest member of the family of symmetric matrices, and you will write any symmetric matrix as a weighted sum of projections onto its eigenvectors.

18.8 What to carry forward

Focus

Summary of core concepts.

- To find the closest point of a subspace S to $\mathbf{b}$, drop a perpendicular: demand that the error $\mathbf{e} = \mathbf{b} - \mathbf{p}$ be orthogonal to S.
- **Onto a line** span($\mathbf{a}$): $\mathbf{p} = \dfrac{\mathbf{a}^\mathsf{T}\mathbf{b}}{\mathbf{a}^\mathsf{T}\mathbf{a}}\,\mathbf{a}$, $\mathbf{P} = \dfrac{\mathbf{a}\,\mathbf{a}^\mathsf{T}}{\mathbf{a}^\mathsf{T}\mathbf{a}}$.
- **Onto a subspace** col($\mathbf{A}$): solve the normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$, then $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$.
- Every orthogonal projection matrix is **symmetric** ($\mathbf{P}^\mathsf{T} = \mathbf{P}$) and **idempotent** ($\mathbf{P}^2 = \mathbf{P}$). It acts as the identity on S ($\mathbf{Pv} = \mathbf{v}$), collapses $S^\perp$ to the origin ($\mathbf{Pv} = \mathbf{0}$), and has rank($\mathbf{P}$) = dim S.
- The closest-point guarantee (Pythagoras) is what turns projection into **least squares**, the engine of regression and data fitting.

Connection to other math

Projection matrices replace impossible equations with optimal approximations. Whenever $\mathbf{Ax} = \mathbf{b}$ has no solution because $\mathbf{b} \notin \text{col}(\mathbf{A})$, the projection matrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ cleanly splits $\mathbf{b}$ into its reachable shadow $\mathbf{p} \in \text{col}(\mathbf{A})$ and its orthogonal residual $\mathbf{e} \in \text{null}(\mathbf{A}^\mathsf{T})$. Yet computing $\mathbf{P}$ directly by inverting $\mathbf{A}^\mathsf{T}\mathbf{A}$ carries a severe hidden risk: forming the normal equations squares the matrix condition number $\kappa(\mathbf{A})$ (which measures sensitivity to numerical rounding and noise), yielding $\kappa(\mathbf{A}^\mathsf{T}\mathbf{A}) = (\kappa(\mathbf{A}))^2$. If the columns of $\mathbf{A}$ are nearly parallel, $\mathbf{A}^\mathsf{T}\mathbf{A}$ becomes dangerously ill-conditioned. In Chapter 19, we apply projections to data fitting and linear regression. Then, in Chapter 20, we resolve the numerical danger of $\mathbf{A}^\mathsf{T}\mathbf{A}$ by developing the Gram–Schmidt process and QR factorization, computing orthogonal projections with pristine numerical stability.

18.9 Exercises

Exercise 18.1. (Mental checks: three instant projections.) Answer each of these without computing any matrix inverses:

(a) What is the 3×3 matrix $\mathbf{P}$ that projects every vector in $\mathbb{R}^3$ onto the xy-plane? (Think: which coordinates are kept, and which are dropped?)

(b) If $\mathbf{b}$ lies on the line spanned by $\mathbf{a}$, what is its projection $\mathbf{Pb}$? If $\mathbf{b}$ is perpendicular to $\mathbf{a}$, what is $\mathbf{Pb}$?

(c) If an orthogonal projection matrix $\mathbf{P}$ satisfies $\mathbf{P}^2 = \mathbf{P}$ and $\mathbf{P}^\mathsf{T} = \mathbf{P}$, what is $(\mathbf{I} - \mathbf{P})^2$? What is $\mathbf{P}(\mathbf{I} - \mathbf{P})$?

Exercise 18.2. Project $\mathbf{b} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ onto the line through $\mathbf{a} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Find $\hat{x}$, $\mathbf{p}$, and $\mathbf{e}$, and verify that $\mathbf{a}^\mathsf{T}\mathbf{e} = 0$.

Exercise 18.3. For $\mathbf{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$, build the line-projection matrix $\mathbf{P} = \dfrac{\mathbf{a}\,\mathbf{a}^\mathsf{T}}{\mathbf{a}^\mathsf{T}\mathbf{a}}$ explicitly as a 2×2 matrix. Confirm by direct multiplication that $\mathbf{P}^2 = \mathbf{P}$ and that $\mathbf{P}^\mathsf{T} = \mathbf{P}$.

Exercise 18.4. Let $\mathbf{P}$ be the matrix from Exercise 18.3. Compute $\mathbf{P} \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and explain geometrically why the answer is what it is. Then compute $\mathbf{P} \begin{bmatrix} 4 \\ -3 \end{bmatrix}$ and explain that answer too. (Hint: how is $\begin{bmatrix} 4 \\ -3 \end{bmatrix}$ related to $\begin{bmatrix} 3 \\ 4 \end{bmatrix}$?)

Exercise 18.5. Project $\mathbf{b} = \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}$ onto the plane spanned by $\mathbf{a}_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$. Form $\mathbf{A}^\mathsf{T}\mathbf{A}$ and $\mathbf{A}^\mathsf{T}\mathbf{b}$, solve for $\hat{\mathbf{x}}$, find $\mathbf{p}$ and $\mathbf{e}$, and check that $\mathbf{e}$ is orthogonal to both basis vectors. (You should find $\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{p} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$, and $\mathbf{e} = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$; note that $\mathbf{b}$ does *not* lie in the plane, so the projection genuinely moves it.)

Exercise 18.6. Recall that the *trace* of a square matrix is the sum of its diagonal entries, $\operatorname{tr}(\mathbf{M}) = \sum M_{ii}$, and that the trace is cyclic: $\operatorname{tr}(\mathbf{XY}) = \operatorname{tr}(\mathbf{YX})$ whenever both products are defined. Show that for the plane in Exercise 18.5 the trace of the projection matrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ equals 2, without computing all nine entries. (Hint: let $\mathbf{X} = \mathbf{A}$ and $\mathbf{Y} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$; apply the cyclic property to push $\mathbf{A}$ to the right, and watch the expression collapse to the trace of a 2×2 identity matrix.)

Exercise 18.7. Let $\mathbf{P}$ project onto a subspace S. Prove that $\mathbf{Q} = \mathbf{I} - \mathbf{P}$ is also a projection matrix (symmetric and idempotent), and argue geometrically that $\mathbf{Q}$ projects onto the orthogonal complement $S^\perp$. What is $\mathbf{PQ}$, and why does the answer make geometric sense?

Exercise 18.8. For the line through $\mathbf{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and the point $\mathbf{b} = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$ (from Example 18.2), pick any *other* point on the line, say $\mathbf{y} = 2\mathbf{a}$, and verify numerically that $\|\mathbf{b} - \mathbf{y}\| > \|\mathbf{b} - \mathbf{p}\|$. Then confirm the Pythagorean identity $\|\mathbf{b} - \mathbf{y}\|^2 = \|\mathbf{e}\|^2 + \|\mathbf{p} - \mathbf{y}\|^2$ with your numbers.

Exercise 18.9. (Preview of eigenvalues.) A scalar λ is called an *eigenvalue* of a square matrix $\mathbf{P}$ if there is a nonzero vector $\mathbf{v}$ that $\mathbf{P}$ merely scales without rotating: $\mathbf{P}\mathbf{v} = \lambda\mathbf{v}$. Using only the identity $\mathbf{P}^2 = \mathbf{P}$, prove that every such scale factor must satisfy $\lambda^2 = \lambda$, hence $\lambda \in \{0, 1\}$. (Hint: apply $\mathbf{P}$ to both sides of $\mathbf{P}\mathbf{v} = \lambda\mathbf{v}$.) Which vectors correspond to $\lambda = 1$, and which to $\lambda = 0$?

Exercise 18.10. (Challenge: why symmetry matters.) Consider $\mathbf{P} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$. Verify that $\mathbf{P}^2 = \mathbf{P}$, so $\mathbf{P}$ is idempotent, but $\mathbf{P}^{\mathsf{T}} \neq \mathbf{P}$. Show that $\mathbf{P}$ maps every vector onto the x-axis, yet does *not* send $\mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ to its *closest* point on the x-axis (which is the origin). Conclude that idempotence alone gives a projection, but only *symmetry* makes it the orthogonal, closest-point projection.

Exercise 18.11. (Challenge.) Suppose the columns of $\mathbf{A}$ are pairwise *orthonormal*: $\mathbf{A}^{\mathsf{T}}\mathbf{A} = \mathbf{I}$. Show that the projection matrix simplifies to $\mathbf{P} = \mathbf{A}\mathbf{A}^{\mathsf{T}}$, and that $\hat{\mathbf{x}} = \mathbf{A}^{\mathsf{T}}\mathbf{b}$ – each coefficient is just a single dot product $\mathbf{a}_i^{\mathsf{T}}\mathbf{b}$, with no matrix inverse anywhere. Explain why this is the computational reason Chapter 20 builds orthonormal bases via Gram–Schmidt.

Chapter 19

Least Squares and Regression

In scientific experiments and data analysis, empirical measurements rarely fit theoretical models perfectly. Even when physical laws predict a linear relationship between input and output (such as Hooke's law for an ideal spring), experimental noise, measurement errors, and uncontrolled variables ensure that measured data points scatter around the ideal curve.

In the language of linear algebra, we have an overdetermined system $\mathbf{A}\mathbf{x} = \mathbf{b}$ whose measurement vector $\mathbf{b}$ lies outside the column space $\mathrm{col}(\mathbf{A})$. Gaussian elimination reports "no exact solution," yet we do not throw the model away. Instead, we look for the parameter vector $\hat{\mathbf{x}}$ that makes the *error* $\mathbf{e} = \mathbf{b} - \mathbf{A}\mathbf{x}$ – the gap between measurements and model predictions – as small as possible. Finding $\hat{\mathbf{x}}$ requires no iterative search; it comes straight from a square linear system, the **normal equations**:

$$\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\hat{\mathbf{x}} \;=\; \mathbf{A}^{\mathsf{T}}\mathbf{b}.$$

Here, "normal" means perpendicular: the error vector stands at right angles to every column of $\mathbf{A}$. Geometrically, $\mathbf{A}\hat{\mathbf{x}}$ is the orthogonal projection of $\mathbf{b}$ onto $\mathrm{col}(\mathbf{A})$. Least squares applies the geometry of orthogonal projections to data analysis, statistics, and machine learning.

Focus

When $\mathbf{A}\mathbf{x} = \mathbf{b}$ has no solution, do not solve it – *approximate* it. Find the $\hat{\mathbf{x}}$ that makes $\mathbf{A}\hat{\mathbf{x}}$ as close to $\mathbf{b}$ as possible, measuring closeness by the squared length $\|\mathbf{b} - \mathbf{A}\mathbf{x}\|^2$. That $\hat{\mathbf{x}}$ solves the normal equations

$$\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\hat{\mathbf{x}} \;=\; \mathbf{A}^{\mathsf{T}}\mathbf{b},$$

and geometrically $\mathbf{A}\hat{\mathbf{x}}$ is the **projection of $\mathbf{b}$ onto the column space of $\mathbf{A}$**.

19.1 Why $\mathbf{A}\mathbf{x} = \mathbf{b}$ usually has no solution

Let us be concrete before we are general. Suppose you believe two quantities are related by a straight line $y = c + d\,t$, and you collect three data points:

$$(t_1, y_1) = (0, 6), \qquad (t_2, y_2) = (1, 0), \qquad (t_3, y_3) = (2, 0).$$

If a single line $y = c + dt$ passed through all three, the unknowns c, d would have to satisfy

$$\begin{aligned} c + d \cdot 0 &= 6, \\ c + d \cdot 1 &= 0, \\ c + d \cdot 2 &= 0. \end{aligned}$$

In matrix form $\mathbf{Ax} = \mathbf{b}$ with

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} c \\ d \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}.$$

Three equations, two unknowns. The first equation forces $c = 6$. The second then forces $d = -6$. But the third now reads $6 + 2(-6) = -6 \neq 0$. Contradiction. No line passes through all three points. The system is **inconsistent** – meaning there is no choice of parameters that can satisfy every equation at once.

Intuition

The columns of $\mathbf{A}$ span a two-dimensional subspace (a plane) in $\mathbb{R}^3$: the span of $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}$. Every linear combination $\mathbf{Ax} = c \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + d \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}$ lies on this plane. Because the measurement vector $\mathbf{b} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}$ does not lie in this plane, no choice of coefficients c, d can produce $\mathbf{b}$, and the system $\mathbf{Ax} = \mathbf{b}$ has no exact solution.

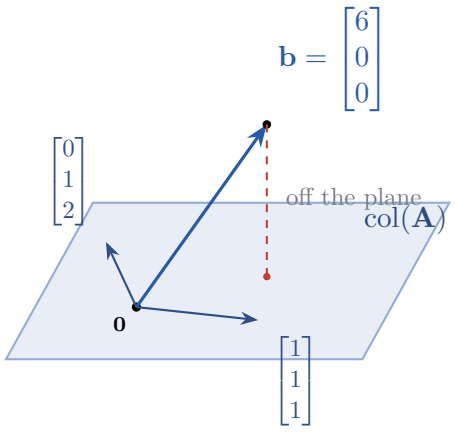


Figure 19.1: The two columns of $\mathbf{A}$, namely $(1, 1, 1)$ and $(0, 1, 2)$, span a flat plane through the origin in $\mathbb{R}^3$. Every combination $\mathbf{Ax} = c(1, 1, 1) + d(0, 1, 2)$ lands somewhere on that sheet. The data vector $\mathbf{b} = (6, 0, 0)$ points off the sheet, so no choice of c, d reaches it: the system $\mathbf{Ax} = \mathbf{b}$ is inconsistent.

Figure 19.1 draws exactly this: two columns spanning a sheet, and $\mathbf{b}$ hovering off it. This is the normal situation, not a pathology. Whenever you have *more equations than unknowns* – more data

points than parameters, which is what you always want in science – the matrix $\mathbf{A}$ is tall and skinny ($m > n$), its column space is a thin subspace of the big output space $\mathbb{R}^m$, and a measured $\mathbf{b}$ almost never lands inside it. Demanding an exact solution is demanding that noisy data fall exactly on a model. It will not.

Common trap

"No solution" from elimination does not mean "no answer." It means there is no *exact* answer, which for over-determined data is the expected state. An exact fit to many noisy points would usually mean your model has so many parameters that it is memorizing noise (a hazard known as *overfitting*). Inconsistent systems are resolved by orthogonal projection.

Pause

Pause and think. Suppose you have m data points and fit a polynomial of degree $m-1$ (for example, two points and a line $y = c_0 + c_1 t$, or three points and a parabola $y = c_0 + c_1 t + c_2 t^2$). Does $\mathbf{Ax} = \mathbf{b}$ have a solution? Yes, always: when the number of parameters n equals the number of distinct data points m, the matrix $\mathbf{A}$ is square and invertible, so an exact polynomial passes through every point with zero error. Inconsistency and the need for least squares only begin when $m > n$—when you collect more measurements than your model has parameters, so the data overconstrains the equations.

19.2 What "closest" should mean: the error vector

We give up on $\mathbf{Ax} = \mathbf{b}$ and instead ask: of all the vectors $\mathbf{Ax}$ we *can* produce (all of which lie in the column space), which one is nearest to $\mathbf{b}$? Define the **residual** or **error vector**

$$\mathbf{e} \;=\; \mathbf{b} - \mathbf{Ax}.$$

The residual is simply what remains unexplained: each component $e_i = b_i - (\mathbf{Ax})_i$ measures how far the model's prediction misses the actual measurement. We want to make this whole vector small. But "small" in what sense?

The choice that makes everything work is to minimize the **squared length**

$$\|\mathbf{e}\|^2 \;=\; \|\mathbf{b} - \mathbf{Ax}\|^2 \;=\; \sum_{i=1}^{m} \big(b_i - (\mathbf{Ax})_i\big)^2.$$

This is the sum of the squared misses, and minimizing it is what gives the method its name: **least squares**. For the line-fitting problem, $(\mathbf{Ax})_i = c + dt_i$ is the height of the line above t_i, so $\|\mathbf{e}\|^2 = \sum_i (y_i - c - dt_i)^2$ is the total squared vertical distance from the data points to the line. Notice that we measure *vertical* distances, not perpendicular distances to the line: in scientific experiments, the input settings t_i (time, temperature, load) are treated as controlled and known, whereas the output measurements y_i absorb all the random experimental noise.

Walking question

On a walk, ask yourself why we minimize the sum of *squared* misses $\sum_i e_i^2$ rather than plain absolute misses $\sum_i |e_i|$. Compare two things: what happens when you try to take a derivative of $|e_i|$ at zero, and which of the two sums equals a squared Euclidean length $\|\mathbf{e}\|^2$ where right triangles and the Pythagorean theorem apply?

The moral to carry forward: squaring the errors turns data fitting into a Euclidean *geometry* problem, and geometry is something we can solve in closed form. That is what the next section does.

19.3 The geometry: projection onto the column space

Here is the picture that organizes the entire chapter. All the achievable outputs $\mathbf{Ax}$ form the column space $\text{col}(\mathbf{A})$ – a plane (or higher-dimensional flat) sitting inside $\mathbb{R}^m$. The data $\mathbf{b}$ is a point off that plane. We want the point *in* the plane that is closest to $\mathbf{b}$.

You already know what that point is: drop a perpendicular from $\mathbf{b}$ to the plane. The foot of that perpendicular is the projection $\mathbf{p} = \text{proj}_{\text{col}(\mathbf{A})}\,\mathbf{b}$. The closest reachable vector is $\mathbf{p}$, and the $\hat{\mathbf{x}}$ we seek is the one that produces it: $\mathbf{A}\hat{\mathbf{x}} = \mathbf{p}$.

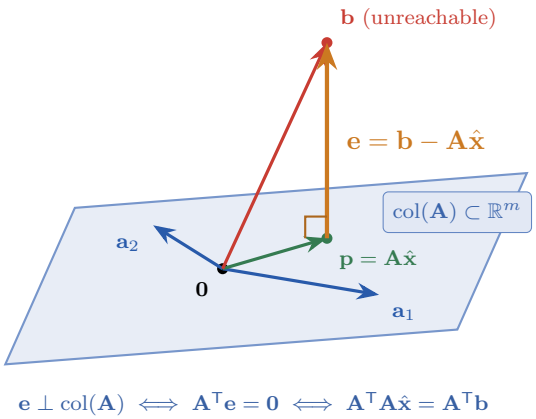


Figure 19.2: The fundamental geometry of least-squares regression. The reachable outputs $\mathbf{Ax}$ fill the column-space plane $\text{col}(\mathbf{A})$. The data vector $\mathbf{b}$ floats off the plane. The closest reachable point is the orthogonal projection $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$. The residual error vector $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ meets the plane at a right angle, orthogonal to every column of $\mathbf{A}$. That geometric perpendicularity $\mathbf{A}^\top\mathbf{e} = \mathbf{0}$ yields the normal equations.

The key fact is the right angle. The shortest segment from a point to a plane is the perpendicular one; any other segment from $\mathbf{b}$ into the plane is the hypotenuse of a right triangle and hence longer. So at the optimum, the error $\mathbf{e}$ is **orthogonal to the column space**. That single geometric statement is the complete derivation, and we now turn it into algebra.

Pause

Pause and think. What happens to this picture if $\mathbf{b}$ already lies *inside* the column space $\text{col}(\mathbf{A})$? Then $\mathbf{b}$ is already in the plane, so dropping a perpendicular moves it nowhere: the projection is $\mathbf{p} = \mathbf{b}$, and the error vector is $\mathbf{e} = \mathbf{0}$. The normal equations become $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$, which is automatically satisfied by the exact solution $\mathbf{A}\mathbf{x} = \mathbf{b}$. Least squares never distorts a problem that was already solvable; it generalizes exact linear algebra to noisy real-world data.

19.4 Deriving the normal equations

The error $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ must be orthogonal to every vector in $\text{col}(\mathbf{A})$. But every vector in the column space is a combination of the columns of $\mathbf{A}$, so it is enough that $\mathbf{e}$ be orthogonal to *each column* of $\mathbf{A}$. Orthogonality of $\mathbf{e}$ to column j means the dot product $(\text{column } j) \cdot \mathbf{e} = 0$. Stacking all n of these dot products is exactly the matrix product $\mathbf{A}^\mathsf{T}\mathbf{e}$. So the condition is

$$\mathbf{A}^\mathsf{T}\mathbf{e} \;=\; \mathbf{0} \qquad \Longleftrightarrow \qquad \mathbf{A}^\mathsf{T}\big(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}\big) \;=\; \mathbf{0}.$$

Distribute and move one term to the other side:

$$\boxed{\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} \;=\; \mathbf{A}^\mathsf{T}\mathbf{b}.}$$

These are the **normal equations**. The word "normal" here means "perpendicular" – it records the right angle in Figure 19.2. We started with an unsolvable tall system $\mathbf{A}\mathbf{x} = \mathbf{b}$ (m equations, n unknowns) and multiplied both sides by $\mathbf{A}^\mathsf{T}$ to get a *square* $n \times n$ system that is solvable.

Formal

Least-squares solution. Let $\mathbf{A}$ be $m \times n$ with linearly independent columns. Then $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible, and the unique minimizer of $\|\mathbf{b} - \mathbf{A}\mathbf{x}\|^2$ is

$$\hat{\mathbf{x}} \;=\; (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b}.$$

The projection of $\mathbf{b}$ onto $\text{col}(\mathbf{A})$ is $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b}$, and the **projection matrix** onto the column space is $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$.

History and connection

On New Year's Day in 1801, the Italian astronomer Giuseppe Piazzi looked through his telescope in Palermo and spotted a faint, moving point of light: the dwarf planet Ceres (the largest asteroid in the belt between Mars and Jupiter). For 41 days, Piazzi tracked Ceres across just nine degrees of the night sky before it disappeared into the blinding glare of the Sun. Circular orbit approximations failed completely because Ceres follows an *eccentric elliptical orbit* (an elongated oval rather than a neat circle), and forty-one days of noisy telescope angles covered barely one percent of its multi-year orbit. Most astronomers feared Ceres was lost for good.
Hearing of the crisis, a 24-year-old Carl Friedrich Gauss took up the challenge. Gauss had worked out the method of least squares privately in 1795 as an eighteen-year-old student. Using least squares to fit an overdetermined system of non-linear Keplerian equations (planetary equations derived from Johannes Kepler's laws of orbital motion) to Piazzi's noisy tracking

points, Gauss calculated where Ceres would emerge on the other side of the Sun. On December 31, 1801, exactly one year after Piazzi's discovery, the German astronomer Franz Xaver von Zach pointed his telescope precisely where Gauss had directed and rediscovered Ceres within half a degree of his prediction.

In 1805, the French mathematician Adrien-Marie Legendre published the first public description of least squares in his appendix on comet orbits (*Nouvelles méthodes pour la détermination des orbites des comètes*). When Gauss finally published his own celestial mechanics treatise in 1809 (*Theoria Motus Corporum Coelestium*) and claimed priority by stating he had used the method since 1795, a sharp priority feud erupted. Legendre protested bitterly, complaining to colleagues that Gauss was claiming credit for a discovery Legendre had published first.

Earlier, Pierre-Simon Laplace had experimented with minimizing the sum of raw absolute errors ($\sum |e_i|$, the L_1 norm, which treats small and large errors with equal proportionality). However, L_1 minimization leads to equations with sharp corners that were computationally intractable before modern linear programming (an optimization technique developed in the mid-twentieth century). The genius of minimizing squared errors (L_2, the ordinary Euclidean distance) is that differentiating a smooth quadratic produces linear equations—the normal equations $\mathbf{A}^{\mathsf{T}}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^{\mathsf{T}}\mathbf{b}$—solvable by standard elimination.

The method became standard across celestial mechanics, geodesy (mapping the curvature of the Earth), and observational science. Its main statistical weakness comes from squaring the residuals: squaring errors heavily penalizes extreme outliers, making the fit hypersensitive to corrupted measurements. Finding estimators that resist outliers while retaining computational speed remains an active subject in robust statistics.

Let us also derive the normal equations a second way, by calculus, so you trust them no matter which language you think in. Write the squared error as a function of $\mathbf{x}$:

$$f(\mathbf{x}) \;=\; \|\mathbf{b} - \mathbf{A}\mathbf{x}\|^2 \;=\; (\mathbf{b} - \mathbf{A}\mathbf{x})^{\mathsf{T}}(\mathbf{b} - \mathbf{A}\mathbf{x}) \;=\; \mathbf{b}^{\mathsf{T}}\mathbf{b} - 2\,\mathbf{x}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}\mathbf{b} + \mathbf{x}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\mathbf{x}.$$

This is a quadratic bowl in $\mathbf{x}$. Setting its gradient to zero (using $\nabla_{\mathbf{x}}(\mathbf{x}^{\mathsf{T}}\mathbf{M}\mathbf{x}) = 2\mathbf{M}\mathbf{x}$ for symmetric $\mathbf{M}$, and $\nabla_{\mathbf{x}}(\mathbf{x}^{\mathsf{T}}\mathbf{c}) = \mathbf{c}$):

$$\nabla f(\mathbf{x}) \;=\; -2\,\mathbf{A}^{\mathsf{T}}\mathbf{b} + 2\,\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\mathbf{x} \;=\; \mathbf{0} \qquad \Longrightarrow \qquad \mathbf{A}^{\mathsf{T}}\mathbf{A}\,\mathbf{x} = \mathbf{A}^{\mathsf{T}}\mathbf{b}.$$

Same equations. Because $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ is **positive semidefinite** (meaning $\mathbf{x}^{\mathsf{T}}(\mathbf{A}^{\mathsf{T}}\mathbf{A})\mathbf{x} = \|\mathbf{A}\mathbf{x}\|^2 \geq 0$ for every $\mathbf{x}$, so it never turns negative), the bowl opens *upward* in every direction like a soup bowl, so this critical point is a genuine unique minimum, not a maximum or saddle point.

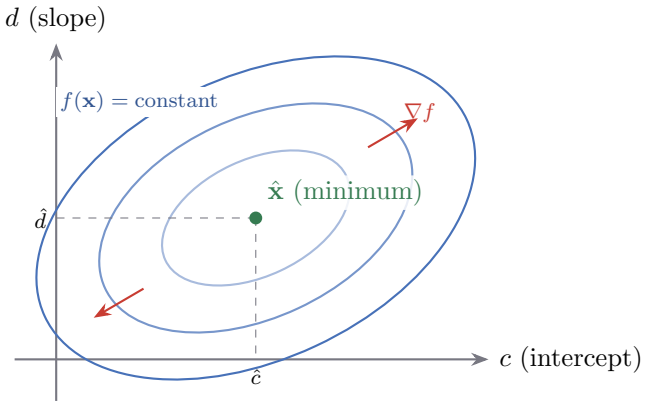


Figure 19.3: Parameter space $\mathbb{R}^n$: the calculus view. The squared error $f(\mathbf{x}) = \|\mathbf{b} - \mathbf{A}\mathbf{x}\|^2$ is a smooth quadratic bowl whose level curves are concentric ellipses centered at the optimal parameter vector $\hat{\mathbf{x}}$. The gradient $\nabla f(\mathbf{x})$ points uphill (red arrows). Setting $\nabla f(\mathbf{x}) = \mathbf{0}$ finds the unique bottom of the bowl in a single step, yielding the exact same normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$ that geometry found by dropping a perpendicular in $\mathbb{R}^m$.

Connection to other math

Two completely different roads – pure geometry (drop a perpendicular) and pure calculus (set the gradient of a quadratic to zero) – arrive at the exact same matrix equation. This reflects a theme that runs through all of optimization. Minimizing a smooth function means finding where its gradient vanishes, and for a *quadratic* function the gradient is linear, so "gradient = 0" is a linear system. Least squares is the simplest nontrivial optimization problem in the world, and it is the one case where the optimum has a clean closed form. Every fancier method – gradient descent, Newton's method, the whole edifice of convex optimization – can be understood as "what do you do when the bowl is not exactly quadratic." Here it is, so you get the answer in one step.

Why $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible

As we proved in Chapter 18, independent columns make $\mathbf{A}^\mathsf{T}\mathbf{A}$ invertible. Let us recall that short argument here because it pinpoints when least squares fails. Suppose $\mathbf{A}^\mathsf{T}\mathbf{A}\,\mathbf{x} = \mathbf{0}$. Dot both sides with $\mathbf{x}$:

$$0 \;=\; \mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{A}\,\mathbf{x} \;=\; (\mathbf{A}\mathbf{x})^\mathsf{T}(\mathbf{A}\mathbf{x}) \;=\; \|\mathbf{A}\mathbf{x}\|^2.$$

A squared length is zero only when the vector is zero, so $\mathbf{A}\mathbf{x} = \mathbf{0}$. But if the columns of $\mathbf{A}$ are independent, the only $\mathbf{x}$ with $\mathbf{A}\mathbf{x} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$. Thus the nullspace of $\mathbf{A}^\mathsf{T}\mathbf{A}$ is trivial, and a square matrix with trivial nullspace is invertible. The computation also shows $\mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{A}\,\mathbf{x} = \|\mathbf{A}\mathbf{x}\|^2 \geq 0$, which is the positive-semidefiniteness that made the bowl open upward.

Common trap

If the columns of $\mathbf{A}$ are *dependent* – say you accidentally fed in the same feature twice, or measured temperature in both Celsius and Fahrenheit – then $\mathbf{A}^\mathsf{T}\mathbf{A}$ is singular and $(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}$ does not exist. The projection $\mathbf{p}$ is still perfectly well defined (the closest point on the plane

is unique), but $\hat{\mathbf{x}}$ is not unique: many different parameter vectors produce the exact same best fit. In statistics this is called **collinearity** (or multicollinearity) – meaning two or more feature columns point along nearly the same direction, carrying redundant information that makes it impossible for the math to disentangle their individual contributions. The cure is either to drop redundant features or to add a shrinkage penalty (ridge regression, below), which restores invertibility.

19.5 Worked example: fitting a line

Let us finish the three-point problem from the start of the chapter. We had

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}, \qquad \hat{\mathbf{x}} = \begin{bmatrix} c \\ d \end{bmatrix}.$$

Build the two pieces of the normal equations. First $\mathbf{A}^{\mathsf{T}}\mathbf{A}$:

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}.$$

The top-left entry 3 is just the number of data points; the off-diagonal $3 = \sum t_i$ and the bottom-right $5 = \sum t_i^2$. Now $\mathbf{A}^{\mathsf{T}}\mathbf{b}$:

$$\mathbf{A}^{\mathsf{T}}\mathbf{b} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum t_i y_i \end{bmatrix}.$$

So the normal equations are the tidy 2×2 system

$$\begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \end{bmatrix}.$$

Solve it. From the first row $3c + 3d = 6$, i.e. $c + d = 2$. From the second $3c + 5d = 0$. Subtract three times the first from the second: $(3c + 5d) - (3c + 3d) = 2d = 0 - 6 = -6$, so $d = -3$, and then $c = 5$. The best-fit line is

$$\boxed{y = 5 - 3t.}$$

Let us check the fit. The line predicts $\hat{y} = (5, 2, -1)$ at $t = (0, 1, 2)$, while the data were $\mathbf{b} = (6, 0, 0)$. So the projection and error are

$$\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 5 \\ 2 \\ -1 \end{bmatrix}, \qquad \mathbf{e} = \mathbf{b} - \mathbf{p} = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 5 \\ 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}.$$

Two sanity checks confirm the geometry. First, the residual should be orthogonal to both columns of $\mathbf{A}$. Against the all-ones column: $1 - 2 + 1 = 0$. Against the time column $(0, 1, 2)$: $0 - 2 + 2 = 0$. Perpendicular to the plane, exactly as the picture demanded. Second, the squared error is $\|\mathbf{e}\|^2 = 1 + 4 + 1 = 6$; no other line does better. Figure 19.4 shows the line, the data, and these residuals.

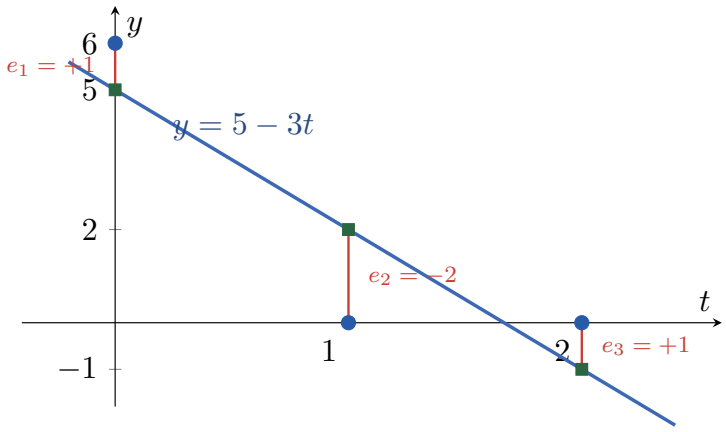


Figure 19.4: Fitting $y = c + dt$ to the three points $(0, 6), (1, 0), (2, 0)$. The blue line is the least-squares best fit $y = 5 - 3t$. Filled circles are the data $\mathbf{b}$; green squares are the predictions $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$ on the line; red segments are the residuals $e_i = b_i - p_i = (1, -2, 1)$. Least squares makes the *sum of the squared* red lengths as small as possible.

Compute it yourself

Do the whole thing yourself with a fresh tiny data set: points $(t, y) = (-1, 1), (0, 1), (1, 3)$. Build $\mathbf{A} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 3 \end{bmatrix}$, form $\mathbf{A}^{\mathsf{T}}\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$ and $\mathbf{A}^{\mathsf{T}}\mathbf{b} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$, and read off $\hat{\mathbf{x}} = \begin{bmatrix} 5/3 \\ 1 \end{bmatrix}$.

The best line is $y = \frac{5}{3} + t$. Verify the residual $\mathbf{e} = \begin{bmatrix} 1 \\ 1 \\ 3 \end{bmatrix} - \begin{bmatrix} 2/3 \\ 5/3 \\ 8/3 \end{bmatrix} = \begin{bmatrix} 1/3 \\ -2/3 \\ 1/3 \end{bmatrix}$ is orthogonal to both columns. (Notice how the centered times $t = -1, 0, 1$ made $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ *diagonal* – the next box explains why this orthogonality is such a gift.)

Why this matters

Look at what just happened to the bookkeeping. The matrix $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ for line fitting is

$$\begin{bmatrix} m & \sum t_i \\ \sum t_i & \sum t_i^2 \end{bmatrix}, \qquad \mathbf{A}^{\mathsf{T}}\mathbf{b} = \begin{bmatrix} \sum y_i \\ \sum t_i y_i \end{bmatrix}.$$

Every entry is a *sum* you compute by sweeping through the data once. The grand projection geometry of $\mathbb{R}^m$ collapses into four little sums. This is why least squares scales: fitting a line to a billion points costs the same matrix as fitting it to three – you only ever build a 2×2 system. The dimension of the data (m) never enters the size of the equations you solve, only the number of *parameters* (n) does. That decoupling is what makes regression practical on enormous data.

19.6 Fitting a parabola, and any other model linear in its parameters

Nothing about the method needed the model to be a line. Suppose the data look curved and you want the best parabola $y = c_0 + c_1 t + c_2 t^2$. The unknowns are c_0, c_1, c_2, and the model remains strictly *linear in those unknowns* even though it is quadratic in t. Each data point gives one equation $c_0 + c_1 t_i + c_2 t_i^2 = y_i$, so

$$\mathbf{A} = \begin{bmatrix} 1 & t_1 & t_1^2 \\ 1 & t_2 & t_2^2 \\ \vdots & \vdots & \vdots \\ 1 & t_m & t_m^2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

This tall matrix (where each row contains increasing powers $1, t_i, t_i^2$ of an input sample, a structure known as a **Vandermonde matrix**) translates polynomial evaluation into matrix multiplication. It goes straight into the same normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$. The recipe is identical; only the number of columns changed.

Example 19.1. Fit $y = c_0 + c_1 t + c_2 t^2$ to the four points $(t, y) = (-1, 1), (0, 1), (1, 0), (2, 4)$. Build

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 4 \end{bmatrix}.$$

Then

$$\mathbf{A}^\mathsf{T}\mathbf{A} = \begin{bmatrix} 4 & 2 & 6 \\ 2 & 6 & 8 \\ 6 & 8 & 18 \end{bmatrix}, \qquad \mathbf{A}^\mathsf{T}\mathbf{b} = \begin{bmatrix} 6 \\ 7 \\ 17 \end{bmatrix}.$$

(Each entry of $\mathbf{A}^\mathsf{T}\mathbf{A}$ is again a sum of powers of t_i: the top-left $4 = \sum t_i^0 = m$, the next diagonal $6 = \sum t_i^2$, the corner $18 = \sum t_i^4$, and so on. The entries of $\mathbf{A}^\mathsf{T}\mathbf{b}$ are $\sum y_i = 6$, $\sum t_i y_i = 7$, and $\sum t_i^2 y_i = 17$.) Solving the 3×3 system by elimination gives

$$\hat{\mathbf{x}} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 1/10 \\ -1/5 \\ 1 \end{bmatrix} \quad \Longrightarrow \quad y = 0.1 - 0.2\,t + t^2.$$

The leading coefficient came out to 1, recovering an underlying t^2 curvature, while the small constant and linear terms absorbed the scatter. The predictions are $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = \left(\frac{13}{10}, \frac{1}{10}, \frac{9}{10}, \frac{37}{10}\right)$, so the residual is $\mathbf{e} = \mathbf{b} - \mathbf{p} = \left(-\frac{3}{10}, \frac{9}{10}, -\frac{9}{10}, \frac{3}{10}\right)$, which you can confirm is orthogonal to all three columns of $\mathbf{A}$ (each dot product is 0). Figure 19.5 plots the data, the curve, and these residuals. The fit is a true parabola, not a line, yet the normal equations stayed linear throughout.

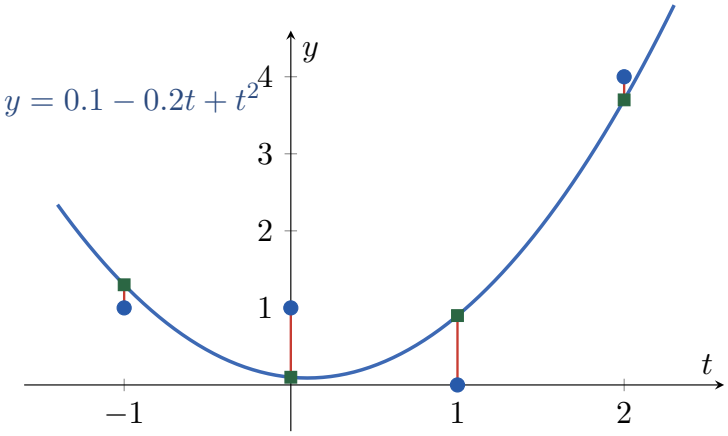


Figure 19.5: The least-squares parabola of Example 19.1. Filled circles are the data $\mathbf{b} = (1, 1, 0, 4)$; green squares are the predictions $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = (\frac{13}{10}, \frac{1}{10}, \frac{9}{10}, \frac{37}{10})$ on the curve; red segments are the residuals. Exactly the same normal-equation machine that fit a line now fits a curve – only the number of feature columns changed.

Focus

"Linear regression" does not mean "fit a straight line." It means **linear in the parameters**. You may use any features of your inputs you like – powers t, t^2, t^3, products of two variables, $\sin t$, $\log t$, a hand-crafted indicator like "is it a weekend" – and as long as the unknown coefficients enter *linearly*, the fit is a least-squares problem with the same normal equations. The columns of $\mathbf{A}$ are your chosen features evaluated at each data point. This single idea – build a feature matrix, then project – is most of practical machine learning before neural networks.

19.7 Measuring the fit: residuals and R^2

A best-fit line is only as good as the data allows. How good is it? The currency that answers this is the residual vector $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$. Its squared length $\|\mathbf{e}\|^2 = \sum e_i^2$ is the **residual sum of squares**, often written $\mathrm{SS}_{\mathrm{res}}$. Smaller is better, but it has units (squared y-units) and scale, so on its own a number like "6" means nothing. We want a unitless score that says "how much of the variation in y did the model explain?"

That score is R^2, the **coefficient of determination**, and it falls straight out of a Pythagorean theorem. Compare your model against the laziest possible model: predicting the constant mean $\bar{y} = \frac{1}{m}\sum y_i$ for every point. The lazy model's error is the **total sum of squares**

$$\mathrm{SS}_{\mathrm{tot}} = \sum_{i=1}^{m} (y_i - \bar{y})^2,$$

the raw variation in the data. Then define

$$\boxed{R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}} = 1 - \frac{\|\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}\|^2}{\|\mathbf{b} - \bar{y}\,\mathbf{1}\|^2}.}$$

If the model fits perfectly, $\mathrm{SS}_{\mathrm{res}} = 0$ and $R^2 = 1$. If the model does no better than predicting the mean, $\mathrm{SS}_{\mathrm{res}} = \mathrm{SS}_{\mathrm{tot}}$ and $R^2 = 0$. In between, R^2 is the fraction of the variance the model captured.

Intuition

R^2 compares two orthogonal distances. Total variation $\mathrm{SS}_{\mathrm{tot}}$ measures how far the data spread from the horizontal line at the sample mean $\bar{y}$; residual variation $\mathrm{SS}_{\mathrm{res}}$ measures how far the data spread from the fitted regression line. Because the model contains the constant mean as a special case (when slope $d = 0$), we always have $\mathrm{SS}_{\mathrm{res}} \le \mathrm{SS}_{\mathrm{tot}}$. An R^2 of 0.9 means that the linear model accounts for 90% of the total sample variance in the response variable, leaving 10% in the residual error.

Pause

Pause and think. Can R^2 ever be negative? Under the standard definition $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$, yes. If you fit a model without an intercept (forcing the line through the origin) and the true data sit far above the origin, the fitted line can make larger errors than the simple horizontal average $\bar{y}$. Then $\mathrm{SS}_{\mathrm{res}} > \mathrm{SS}_{\mathrm{tot}}$, and R^2 drops below zero. A negative R^2 indicates that the model fits the data worse than the simple sample mean $\bar{y}$.

Example 19.2. For our line fit $y = 5 - 3t$ we had $\mathbf{b} = (6, 0, 0)$ and residual $\mathbf{e} = (1, -2, 1)$, so $\mathrm{SS}_{\mathrm{res}} = \|\mathbf{e}\|^2 = 6$. The mean is $\bar{y} = (6 + 0 + 0)/3 = 2$, so the deviations from the mean are $(6 - 2,\ 0 - 2,\ 0 - 2) = (4, -2, -2)$ and

$$\mathrm{SS}_{\mathrm{tot}} = 4^2 + (-2)^2 + (-2)^2 = 16 + 4 + 4 = 24.$$

Therefore

$$R^2 = 1 - \frac{6}{24} = 1 - \frac{1}{4} = 0.75.$$

The line explains 75% of the variation in this (tiny, noisy) data set, leaving a quarter unexplained. With only three points and clear scatter, that is about what you would expect.

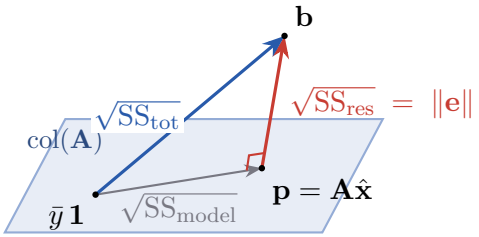


Figure 19.6: R^2 is one right triangle. Measured from the mean vector $\bar{y}\,\mathbf{1}$, the data $\mathbf{b}$ reaches up a distance $\sqrt{\mathrm{SS}_{\mathrm{tot}}}$ (the hypotenuse). The fit $\mathbf{p}$ stays in the column space, a leg of length $\sqrt{\mathrm{SS}_{\mathrm{model}}}$; the residual $\mathbf{e} = \mathbf{b} - \mathbf{p}$ is the perpendicular leg of length $\sqrt{\mathrm{SS}_{\mathrm{res}}}$. Pythagoras gives $\mathrm{SS}_{\mathrm{tot}} = \mathrm{SS}_{\mathrm{model}} + \mathrm{SS}_{\mathrm{res}}$, and $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$ is the squared fraction of the hypotenuse used up by the model leg.

Connection to other math

The Pythagorean theorem is hiding in plain sight. Because the residual $\mathbf{e}$ is orthogonal to the column space and the fit $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$ lives in it, $\mathbf{b} = \mathbf{p} + \mathbf{e}$ splits into perpendicular pieces,

and $\|\mathbf{b}\|^2 = \|\mathbf{p}\|^2 + \|\mathbf{e}\|^2$. Subtracting the mean (which the all-ones column lets the fit reproduce) turns this into $\mathrm{SS}_{\mathrm{tot}} = \mathrm{SS}_{\mathrm{model}} + \mathrm{SS}_{\mathrm{res}}$, the famous *analysis of variance* (ANOVA) decomposition drawn in Figure 19.6. The correlation coefficient r is $\pm\sqrt{R^2}$, which equals the cosine of the angle between the centered data and the centered predictor. This geometric decomposition underlies linear statistical modeling.

Common trap

A high R^2 does *not* mean your model is correct or useful. You can always raise R^2 by adding more features (more columns to $\mathbf{A}$): in the extreme, fit m points with an $(m-1)$-degree polynomial and you get $R^2 = 1$ exactly – the curve threads every point – yet the fit is a wild, useless wiggle that will predict the next data point terribly. This is **overfitting** – when a model memorizes random measurement noise rather than uncovering the underlying relationship, destroying its ability to generalize. That is why practitioners report *adjusted* R^2, hold out separate test data, or penalize parameter size. R^2 measures fit to the training sample you have, not truth, and never prediction on unseen data.

19.8 Least squares as the engine of statistics and machine learning

Everything above was phrased as "fit a curve," but the same three lines of algebra are the core of supervised learning. Let us translate.

In machine learning you have m training examples. Each example i has a row of **features** $\mathbf{a}_i^{\mathsf{T}} = (a_{i1}, \ldots, a_{in})$ (the input attributes) and a numerical **label** b_i (the target value to predict). Stack the feature rows into the **design matrix** $\mathbf{A}$ (the table of all inputs, with one row per example and one column per feature) and the target labels into $\mathbf{b}$. A **linear model** predicts $\hat{b}_i = \mathbf{a}_i^{\mathsf{T}}\mathbf{x}$ using a weight vector $\mathbf{x}$ – exactly $\mathbf{Ax}$. Training the model to minimize squared prediction error is *precisely* least squares, and the optimal weights are

$$\hat{\mathbf{x}} = (\mathbf{A}^{\mathsf{T}}\mathbf{A})^{-1}\mathbf{A}^{\mathsf{T}}\mathbf{b}.$$

The fitted weights tell you how much each feature pushes the prediction; the residuals tell you where the model is wrong; R^2 tells you how much signal you captured. This is **linear regression**, the first model in every machine-learning course and still, today, the model that runs quietly underneath an enormous fraction of deployed prediction systems because it is fast, interpretable, and hard to break.

Modern application

Why regression is everywhere. A bank scoring loan applications stacks each applicant's features (income, debt ratio, credit age) as a row of $\mathbf{A}$ and past repayment outcomes as $\mathbf{b}$; the weights $\hat{\mathbf{x}}$ become the scorecard. A climate model fits temperature against CO_2, solar cycle, and aerosol features. A sports analyst predicts a player's value from box-score features. In all of them the computational heart is the same projection onto a column space.
Three engineering notes from real practice:

- **Never invert $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ directly** on a real problem. Forming $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ squares the matrix's **condition number** $\kappa(\mathbf{A})$ (the ratio of its largest stretch factor to its smallest, measuring how severely round-off errors in the data get magnified in the answer). Instead, practical

software solves least squares via the **QR** factorization (the subject of the next chapter) or the SVD, which work directly on $\mathbf{A}$ and are numerically stable. The normal equations are how you *understand* least squares, not how you compute it in floating point.

- **Collinear features** (nearly dependent columns) make $\mathbf{A}^\mathsf{T}\mathbf{A}$ nearly singular, so tiny data changes swing the weights wildly. The standard fix is **ridge regression**: minimize $\|\mathbf{b}-\mathbf{A}\mathbf{x}\|^2+\lambda\|\mathbf{x}\|^2$, whose normal equations are $(\mathbf{A}^\mathsf{T}\mathbf{A}+\lambda\mathbf{I})\hat{\mathbf{x}}=\mathbf{A}^\mathsf{T}\mathbf{b}$. The added $\lambda\mathbf{I}$ guarantees invertibility and shrinks the weights toward zero.
- **Gradient descent** solves the very same problem one step at a time when $\mathbf{A}$ is too large to factor: it walks downhill on the bowl $f(\mathbf{x})=\|\mathbf{b}-\mathbf{A}\mathbf{x}\|^2$ (Figure 19.3), and because the bowl is convex it cannot get stuck. The least-squares bowl is the training ground on which every optimizer in deep learning was first tested.

Before/after Strang

Strang frames least squares as the meeting point of the whole first half of linear algebra: it combines the column space, orthogonality (the right angle of the error), and projections (the matrix $\mathbf{P}$) into the single normal system $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}}=\mathbf{A}^\mathsf{T}\mathbf{b}$. Before Strang, keep the geometric picture in mind: data vector off the subspace, drop a perpendicular. After Strang, notice how this same equation reappears across advanced topics – as the Moore–Penrose pseudoinverse $\mathbf{A}^+=(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$, as the minimum-norm solution via the SVD, and as the linear layer of a regression model. Gilbert Strang calls $\mathbf{A}^\mathsf{T}\mathbf{A}$ the fundamental matrix of applied mathematics, with least squares providing its primary motivation.

19.9 The projection matrix, revisited

Pulling the threads together, the fitted values are $\mathbf{p}=\mathbf{A}\hat{\mathbf{x}}=\mathbf{P}\mathbf{b}$ where

$$\mathbf{P} \;=\; \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}.$$

This $\mathbf{P}$ is the projection matrix onto $\mathrm{col}(\mathbf{A})$. As we proved in Chapter 18, it carries two signature properties that you can verify directly from its formula:

$$\mathbf{P}^\mathsf{T}=\mathbf{P} \quad \text{(symmetric)}, \qquad \mathbf{P}^2=\mathbf{P} \quad \text{(idempotent: projecting twice changes nothing)}.$$

The symmetry check: $\mathbf{P}^\mathsf{T}=\big(\mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\big)^\mathsf{T}=\mathbf{A}\big((\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\big)^\mathsf{T}\mathbf{A}^\mathsf{T}=\mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}=\mathbf{P}$, using that $\mathbf{A}^\mathsf{T}\mathbf{A}$ is symmetric so its inverse is too. The idempotence check telescopes beautifully:

$$\mathbf{P}^2=\mathbf{A}\underbrace{(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{A}}_{=\mathbf{I}}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}=\mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}=\mathbf{P}.$$

The middle $\mathbf{A}^\mathsf{T}\mathbf{A}$ cancels its own inverse, leaving $\mathbf{P}$ untouched. Geometrically: once you have landed on the plane, projecting again does nothing, because you are already there.

Example 19.3. For our line-fitting $\mathbf{A}=\begin{bmatrix}1&0\\1&1\\1&2\end{bmatrix}$ we found $\mathbf{A}^\mathsf{T}\mathbf{A}=\begin{bmatrix}3&3\\3&5\end{bmatrix}$, whose inverse is

$\frac{1}{6}\begin{bmatrix} 5 & -3 \\ -3 & 3 \end{bmatrix}$. Multiplying out $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ gives

$$\mathbf{P} = \frac{1}{6}\begin{bmatrix} 5 & 2 & -1 \\ 2 & 2 & 2 \\ -1 & 2 & 5 \end{bmatrix}.$$

Check it against the data: $\mathbf{Pb} = \frac{1}{6}\begin{bmatrix} 5 & 2 & -1 \\ 2 & 2 & 2 \\ -1 & 2 & 5 \end{bmatrix}\begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 30 \\ 12 \\ -6 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ -1 \end{bmatrix} = \mathbf{p}$, matching the projection we computed by solving the normal equations. And $\mathbf{P}$'s rows sum to 1 (it preserves the constant vector, since the constant lives in the column space), while $\operatorname{tr}\mathbf{P} = \frac{5+2+5}{6} = 2$ equals the rank – the dimension of the plane we projected onto.

Remark 19.1. The complementary matrix $\mathbf{I} - \mathbf{P}$ projects onto the orthogonal complement of the column space $(\operatorname{col}(\mathbf{A}))^\perp = \operatorname{null}(\mathbf{A}^\mathsf{T})$ (the subspace of all vectors perpendicular to every column of $\mathbf{A}$, known as the left nullspace), and it sends $\mathbf{b}$ directly to the residual: $(\mathbf{I} - \mathbf{P})\mathbf{b} = \mathbf{b} - \mathbf{p} = \mathbf{e}$. So $\mathbf{P}$ splits every data vector into "signal that the model can explain" ($\mathbf{Pb}$, in the column space) plus "noise it cannot" ($\mathbf{e}$, perpendicular to it). That decomposition $\mathbf{b} = \mathbf{Pb} + (\mathbf{I} - \mathbf{P})\mathbf{b}$ is least squares in one line.

19.10 Summary

Focus

Summary of core concepts. When $\mathbf{Ax} = \mathbf{b}$ has no solution because $\mathbf{b}$ lies outside the column space, project: the best approximate solution $\hat{\mathbf{x}}$ makes the error $\mathbf{e} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ perpendicular to $\operatorname{col}(\mathbf{A})$, which gives the normal equations

$$\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}, \qquad \hat{\mathbf{x}} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b} \text{ when columns are independent.}$$

The fit is $\mathbf{p} = \mathbf{Pb}$ with $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ symmetric and idempotent. To fit a line, parabola, or any model linear in its parameters, put your features in the columns of $\mathbf{A}$ and solve the normal equations. The coefficient of determination is $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$. Linear regression is the direct application of orthogonal projections to observational data.

Connection to other math

Every time you fit a model to noisy data, you are using least squares to drop a perpendicular from your measurements onto the subspace of achievable predictions. By replacing impossible exact equations with orthogonal projections, we extract the best available model in any dimension.

Yet solving the normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$ directly carries a hidden numerical hazard. Forming the product $\mathbf{A}^\mathsf{T}\mathbf{A}$ squares the matrix's **condition number** $\kappa(\mathbf{A})$—the ratio of its greatest stretch factor to its smallest, which measures how severely round-off errors get amplified:

$$\kappa(\mathbf{A}^\mathsf{T}\mathbf{A}) \;=\; \big(\kappa(\mathbf{A})\big)^2.$$

If two features in $\mathbf{A}$ are nearly collinear, $\kappa(\mathbf{A})$ might be 10^4. Squaring it yields 10^8, which on standard computer hardware wipes out eight decimal places of precision, turning an accurate scientific model into pure numerical noise.
Can we compute the least-squares projection *without* ever forming $\mathbf{A}^\mathsf{T}\mathbf{A}$? In Chapter 20, we will see that the answer is yes: by replacing crooked columns with an orthonormal basis through the **Gram–Schmidt process**, we factor $\mathbf{A} = \mathbf{QR}$. With orthonormal columns $\mathbf{Q}$, the matrix product $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ becomes the identity matrix, avoiding $\mathbf{A}^\mathsf{T}\mathbf{A}$ altogether and solving least squares with clean numerical stability.

Exercises

Exercise 19.1. (Mental check: exact fit vs approximation.) Suppose you fit a straight line $y = c + dt$ to just two distinct points $(t_1, y_1) = (1, 3)$ and $(t_2, y_2) = (3, 7)$. Before writing down any matrix: what is the residual vector $\mathbf{e}$? What is R^2? Explain why fitting n independent parameters to n data points never requires least squares.

Exercise 19.2. (Mental check: data already in the column space.) Suppose a vector $\mathbf{b}$ already lies in the column space of $\mathbf{A}$, so that $\mathbf{A}\mathbf{x}_0 = \mathbf{b}$ has an exact solution. What is the projection $\mathbf{p} = \mathbf{P}\mathbf{b}$? What does the residual vector $\mathbf{e}$ equal? Verify that the normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$ are satisfied by $\hat{\mathbf{x}} = \mathbf{x}_0$.

Exercise 19.3. Fit a line $y = c + dt$ by least squares to the four points $(0, 1), (1, 3), (2, 4), (3, 4)$. Form $\mathbf{A}^\mathsf{T}\mathbf{A}$ and $\mathbf{A}^\mathsf{T}\mathbf{b}$, solve the 2×2 normal equations, and report the best-fit line. Then compute the residual vector and verify it is orthogonal to both columns of $\mathbf{A}$.

Exercise 19.4. For your fit in Exercise 19.3, compute $\mathrm{SS}_{\mathrm{res}}$, $\mathrm{SS}_{\mathrm{tot}}$ (using the mean $\bar{y}$), and R^2. In one sentence, say what the resulting R^2 means about the data.

Exercise 19.5. Show that fitting the constant model $y = c$ (a horizontal line, so $\mathbf{A}$ is the single all-ones column $\mathbf{1}$) by least squares gives $\hat{c} = \bar{y}$, the mean of the data. Conclude that the mean is the least-squares "best single number" summarizing a data set, and explain why this makes $R^2 = 0$ for the constant model.

Exercise 19.6. Fit a parabola $y = c_0 + c_1 t + c_2 t^2$ to the three points $(-1, 1), (0, 0), (1, 1)$. (With three points and three parameters the fit should be *exact*; predict before computing that the residual will be zero, then confirm it by solving and checking $R^2 = 1$.)

Exercise 19.7. Suppose you shift all the times so they sum to zero (replace t_i by $t_i - \bar{t}$). Show that then $\mathbf{A}^\mathsf{T}\mathbf{A} = \begin{bmatrix} m & 0 \\ 0 & \sum t_i^2 \end{bmatrix}$ is diagonal, and hence the best slope and intercept decouple into $\hat{d} = \dfrac{\sum t_i y_i}{\sum t_i^2}$ and $\hat{c} = \bar{y}$. Explain geometrically why centering the data makes the two columns of $\mathbf{A}$ orthogonal.

Exercise 19.8. Let $\mathbf{P} = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ for an $m \times n$ matrix $\mathbf{A}$ with independent columns. Prove directly from the formula that (a) $\mathbf{P}\mathbf{p} = \mathbf{p}$ for any $\mathbf{p}$ in the column space, and (b) $\mathbf{P}\mathbf{v} = \mathbf{0}$ for any $\mathbf{v}$ orthogonal to every column of $\mathbf{A}$. Conclude that $\mathbf{P}$ acts as the identity on $\mathrm{col}(\mathbf{A})$ and as zero on its orthogonal complement.

Exercise 19.9. Using the cyclic property $\mathrm{tr}(\mathbf{XY}) = \mathrm{tr}(\mathbf{YX})$, show that $\mathrm{tr}\,\mathbf{P} = n$ for the projection matrix of an $m \times n$ matrix with independent columns. Interpret this number geometrically (what is the dimension of the space $\mathbf{P}$ projects onto?).

Exercise 19.10. (Ridge regression.) For scalar data, suppose $\mathbf{A} = \mathbf{1}$ (the all-ones column, m entries) and we minimize $\|\mathbf{b} - c\mathbf{1}\|^2 + \lambda c^2$. Derive the modified normal equation $(m + \lambda)\hat{c} = \sum b_i$, so $\hat{c} = \dfrac{m\,\bar{y}}{m + \lambda}$. Show that as $\lambda \to 0$ you recover the mean, and as $\lambda \to \infty$ the estimate shrinks to 0. Explain in words why a penalty $\lambda\|\mathbf{x}\|^2$ "shrinks" the fit.

Exercise 19.11. (Multiple regression.) You want to predict b from two features u and v with the model $b \approx x_1 u + x_2 v$ (no intercept). The data are $(u, v, b) = (1, 0, 1), (0, 1, 1), (1, 1, 3)$. Set up $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 3 \end{bmatrix}$, solve the normal equations for $\hat{\mathbf{x}}$, and interpret each weight as the effect of one feature holding the other fixed.

Exercise 19.12. (A proof about minimizing.) Let $\mathbf{p}$ be the projection of $\mathbf{b}$ onto a subspace $S = \mathrm{col}(\mathbf{A})$, so $\mathbf{b} - \mathbf{p} \perp S$. Take *any* other vector $\mathbf{q} \in S$ and prove $\|\mathbf{b} - \mathbf{q}\|^2 = \|\mathbf{b} - \mathbf{p}\|^2 + \|\mathbf{p} - \mathbf{q}\|^2$ (Pythagoras), hence $\|\mathbf{b} - \mathbf{q}\| \geq \|\mathbf{b} - \mathbf{p}\|$ with equality only when $\mathbf{q} = \mathbf{p}$. This is the rigorous statement that the projection is the unique closest point – the foundation the whole chapter rests on.

Chapter 20

Gram-Schmidt and QR

There is a recurring theme in this book: *perpendicular is easier*. Whenever two directions are at right angles, the bookkeeping simplifies. Lengths add by Pythagoras instead of by the law of cosines. Projections decouple, one coordinate at a time, instead of tangling together in a linear system. Coordinates can be read off by a single dot product instead of solved for.

Suppose you are given a set of vectors that span a subspace—the columns of a matrix, for example—but they point in oblique directions. Could you replace this basis with an **orthonormal basis**—a set of vectors that are mutually orthogonal (90°) and each of unit length—spanning the same subspace and constructed directly from the original vectors?

The procedure to do this is **Gram–Schmidt**, which means "project away what you already have, keep what is new." When you record the arithmetic Gram–Schmidt performs, that record *is* a factorization of your original matrix: $\mathbf{A} = \mathbf{QR}$, with $\mathbf{Q}$ having orthonormal columns and $\mathbf{R}$ **upper triangular** (every entry below the main diagonal is zero). This is the **QR factorization**, a standard tool in numerical linear algebra and stable least-squares solvers.

20.1 The wish: straighten a crooked basis

Consider two vectors in the plane, $\mathbf{a}_1$ and $\mathbf{a}_2$, that are linearly independent but not orthogonal (Figure 20.1). They span $\mathbb{R}^2$, so they form a valid basis. However, finding the coordinates of a vector $\mathbf{b}$ with respect to this basis requires solving a 2×2 linear system, because the directions are coupled: a vector along $\mathbf{a}_1$ has a nonzero component along $\mathbf{a}_2$.

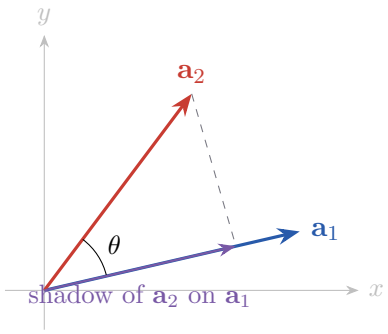


Figure 20.1: A non-orthogonal basis. The vectors $\mathbf{a}_1, \mathbf{a}_2$ span the plane but meet at an angle $\theta \neq 90°$. Projecting $\mathbf{a}_2$ onto $\mathbf{a}_1$ yields a nonzero component: the directions are coupled, so finding coordinates requires solving a system. An orthonormal basis avoids this coupling.

Pause

Pause and think. If two unit vectors $\mathbf{q}_1$ and $\mathbf{q}_2$ are perpendicular, what is the length of their sum $\mathbf{q}_1 + \mathbf{q}_2$? By Pythagoras, $\|\mathbf{q}_1 + \mathbf{q}_2\|^2 = \|\mathbf{q}_1\|^2 + \|\mathbf{q}_2\|^2 = 1 + 1 = 2$, so the length is exactly $\sqrt{2}$. With crooked vectors, cross-terms appear and spoil this simplicity. Perpendicularity removes cross-terms from every calculation.

Intuition

In an orthonormal basis, every vector has unit length and distinct vectors are mutually orthogonal. Coordinates are then computed directly by dot products: the component of $\mathbf{b}$ along $\mathbf{q}_i$ is simply $\langle \mathbf{q}_i, \mathbf{b} \rangle$, without solving a linear system. Gram–Schmidt is the algorithm that converts any basis into an orthonormal basis spanning the same space.

Recall from Chapter 18 that the projection of a vector $\mathbf{a}$ onto the line through a vector $\mathbf{q}$ is

$$\operatorname{proj}_{\mathbf{q}}(\mathbf{a}) = \frac{\langle \mathbf{q}, \mathbf{a} \rangle}{\langle \mathbf{q}, \mathbf{q} \rangle} \mathbf{q}.$$

If $\mathbf{q}$ is already a unit vector, the denominator is 1 and this simplifies to $\langle \mathbf{q}, \mathbf{a} \rangle \mathbf{q}$. That single formula is the only tool we need.

20.2 Gram-Schmidt for two vectors

Start with independent vectors $\mathbf{a}_1$ and $\mathbf{a}_2$. We want an orthonormal pair $\mathbf{q}_1, \mathbf{q}_2$ spanning the same space.

Step 1: normalize the first vector. There are no preceding directions to satisfy orthogonality against, so normalize $\mathbf{a}_1$ to unit length:

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|}.$$

Step 2: remove the part of $\mathbf{a}_2$ that lies along $\mathbf{q}_1$. The vector $\mathbf{a}_2$ has a shadow on the line of $\mathbf{q}_1$ – the component that $\mathbf{q}_1$ already covers. Subtract it off. What remains is exactly the part of $\mathbf{a}_2$

that points in a genuinely *new* direction, perpendicular to $\mathbf{q}_1$:

$$\mathbf{v}_2 \;=\; \mathbf{a}_2 - \mathrm{proj}_{\mathbf{q}_1}(\mathbf{a}_2) \;=\; \mathbf{a}_2 - \langle \mathbf{q}_1,\, \mathbf{a}_2 \rangle\, \mathbf{q}_1.$$

Then normalize:

$$\mathbf{q}_2 \;=\; \frac{\mathbf{v}_2}{\|\mathbf{v}_2\|}.$$

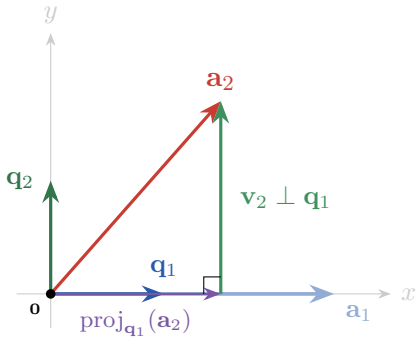


Figure 20.2: Gram–Schmidt on two vectors. The first direction $\mathbf{a}_1$ normalizes to $\mathbf{q}_1$. The second vector $\mathbf{a}_2$ casts a shadow $\mathrm{proj}_{\mathbf{q}_1}(\mathbf{a}_2) = \langle \mathbf{q}_1, \mathbf{a}_2 \rangle \mathbf{q}_1$. Subtracting that shadow leaves $\mathbf{v}_2$, which is perpendicular to $\mathbf{q}_1$. Normalizing $\mathbf{v}_2$ gives $\mathbf{q}_2$, yielding the clean orthonormal pair $(\mathbf{q}_1, \mathbf{q}_2)$.

Why is $\mathbf{v}_2$ guaranteed perpendicular to $\mathbf{q}_1$? Take the dot product and watch it cancel. Using $\langle \mathbf{q}_1, \mathbf{q}_1 \rangle = 1$,

$$\langle \mathbf{q}_1, \mathbf{v}_2 \rangle = \langle \mathbf{q}_1, \mathbf{a}_2 \rangle - \langle \mathbf{q}_1, \mathbf{a}_2 \rangle \langle \mathbf{q}_1, \mathbf{q}_1 \rangle = \langle \mathbf{q}_1, \mathbf{a}_2 \rangle - \langle \mathbf{q}_1, \mathbf{a}_2 \rangle = 0.$$

This cancellation happens because we subtracted off *exactly* the amount that overlapped $\mathbf{q}_1$, so nothing of $\mathbf{q}_1$ is left in $\mathbf{v}_2$.

Pause

Pause and think. What if $\mathbf{a}_1$ and $\mathbf{a}_2$ were already perpendicular at the start? Then $\langle \mathbf{q}_1, \mathbf{a}_2 \rangle = 0$, so the shadow is $\mathbf{0}$. The subtraction leaves $\mathbf{v}_2 = \mathbf{a}_2$ unchanged, and normalizing produces $\mathbf{q}_2 = \mathbf{a}_2 / \|\mathbf{a}_2\|$. Gram–Schmidt leaves already-perpendicular vectors completely undisturbed, adjusting only their lengths.

Compute it yourself

Let $\mathbf{a}_1 = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$.

- $\|\mathbf{a}_1\| = 3$, so $\mathbf{q}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.
- $\langle \mathbf{q}_1, \mathbf{a}_2 \rangle = 2$, so the shadow is $2\,\mathbf{q}_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$.
- $\mathbf{v}_2 = \begin{bmatrix} 2 \\ 2 \end{bmatrix} - \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$, and $\mathbf{q}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

Check: $\langle \mathbf{q}_1, \mathbf{q}_2 \rangle = 0$, both unit length. We recovered the standard axes – which makes sense,

since $\mathbf{a}_1$ was already on the x-axis. Try it again with $\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and see the frame come out tilted by 45°.

20.3 Gram-Schmidt in general

The pattern now repeats. When we reach $\mathbf{a}_k$, we already have an orthonormal set $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$. We subtract off *all* the shadows at once – the component of $\mathbf{a}_k$ along each $\mathbf{q}_i$ we have built so far – and whatever survives is perpendicular to every one of them.

Formal

The Gram–Schmidt process. Given independent vectors $\mathbf{a}_1, \dots, \mathbf{a}_n$, define for $k = 1, 2, \dots, n$:

$$\mathbf{v}_k = \mathbf{a}_k - \sum_{i=1}^{k-1} \langle \mathbf{q}_i, \mathbf{a}_k \rangle \, \mathbf{q}_i, \qquad \mathbf{q}_k = \frac{\mathbf{v}_k}{\|\mathbf{v}_k\|}.$$

Then $\mathbf{q}_1, \dots, \mathbf{q}_n$ is an orthonormal set with $\operatorname{span}\{\mathbf{q}_1, \dots, \mathbf{q}_k\} = \operatorname{span}\{\mathbf{a}_1, \dots, \mathbf{a}_k\}$ for every k.

The summation subtracts the projection of $\mathbf{a}_k$ onto each previously constructed direction $\mathbf{q}_i$. The remainder $\mathbf{v}_k$ is the component of $\mathbf{a}_k$ that is orthogonal to $\operatorname{span}\{\mathbf{q}_1, \dots, \mathbf{q}_{k-1}\}$.

Focus

Project, subtract, normalize. At step k, project $\mathbf{a}_k$ onto the subspace spanned by the earlier directions, subtract that projection to isolate the orthogonal component $\mathbf{v}_k$, and normalize $\mathbf{v}_k$ to unit length.

Let us verify both claims – orthogonality and equal spans – directly from the definition.

Proposition 20.1 (Orthogonality)**.** *For each k, the vector $\mathbf{v}_k$ defined above is orthogonal to every $\mathbf{q}_j$ with $j < k$.*

Proof. Fix $j < k$ and dot $\mathbf{q}_j$ against $\mathbf{v}_k$:

$$\langle \mathbf{q}_j, \mathbf{v}_k \rangle = \langle \mathbf{q}_j, \mathbf{a}_k \rangle - \sum_{i=1}^{k-1} \langle \mathbf{q}_i, \mathbf{a}_k \rangle \, \langle \mathbf{q}_j, \mathbf{q}_i \rangle.$$

By induction the earlier vectors $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ are already orthonormal, so $\langle \mathbf{q}_j, \mathbf{q}_i \rangle = 0$ except when $i = j$, where it equals 1. The sum therefore collapses to its single $i = j$ term:

$$\langle \mathbf{q}_j, \mathbf{v}_k \rangle = \langle \mathbf{q}_j, \mathbf{a}_k \rangle - \langle \mathbf{q}_j, \mathbf{a}_k \rangle \cdot 1 = 0. \qquad \square$$

Proposition 20.2 (Equal spans)**.** *For each k,* $\operatorname{span}\{\mathbf{q}_1, \dots, \mathbf{q}_k\} = \operatorname{span}\{\mathbf{a}_1, \dots, \mathbf{a}_k\}$.

Proof. Each $\mathbf{q}_k$ is built only from $\mathbf{a}_k$ and the earlier $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ – which in turn came from $\mathbf{a}_1, \dots, \mathbf{a}_{k-1}$ – so every $\mathbf{q}_k$ is a combination of $\mathbf{a}_1, \dots, \mathbf{a}_k$. Conversely, rearranging the definition

gives

$$\mathbf{a}_k \;=\; \|\mathbf{v}_k\|\,\mathbf{q}_k \;+\; \sum_{i=1}^{k-1} \langle \mathbf{q}_i,\, \mathbf{a}_k \rangle\, \mathbf{q}_i,$$

so every $\mathbf{a}_k$ is a combination of $\mathbf{q}_1, \ldots, \mathbf{q}_k$. Each spanning set sits inside the other; the spans are equal. Here $\|\mathbf{v}_k\| \neq 0$ because the $\mathbf{a}_k$ are independent, so no projection ever lands exactly on $\mathbf{a}_k$. □

Common trap

Normalize as you go, or keep the denominator. The clean projection formula $\langle \mathbf{q}_i,\, \mathbf{a}_k \rangle\, \mathbf{q}_i$ assumes each finished vector $\mathbf{q}_i$ has unit length. If you prefer to delay normalizing and work with the unnormalized vectors $\mathbf{v}_i$ instead, you must keep the denominator $\frac{\langle \mathbf{v}_i, \mathbf{a}_k \rangle}{\langle \mathbf{v}_i, \mathbf{v}_i \rangle}\, \mathbf{v}_i$ in every projection term. Both formulations produce identical orthogonal directions; forgetting to divide by $\|\mathbf{v}_i\|^2$ when skipping normalization is the classic student bug.

Walking question

You have built $\mathbf{q}_1, \ldots, \mathbf{q}_{k-1}$ and you are about to process $\mathbf{a}_k$. Before computing anything: what does the leftover $\mathbf{v}_k$ *mean* geometrically? (Hint: stack the $\mathbf{q}_i$ into a subspace and think about the perpendicular from $\mathbf{a}_k$ to that subspace – it is the same projection picture from the projections chapter, just onto a span instead of a single line.)

Figure 20.3 makes that perpendicular concrete. The shaded plane is the span of the directions already finished; the sum of shadows $\sum_i \langle \mathbf{q}_i,\, \mathbf{a}_k \rangle\, \mathbf{q}_i$ is the foot of the perpendicular from $\mathbf{a}_k$, and $\mathbf{v}_k$ is the perpendicular itself – the part of $\mathbf{a}_k$ that sticks out of the old subspace.

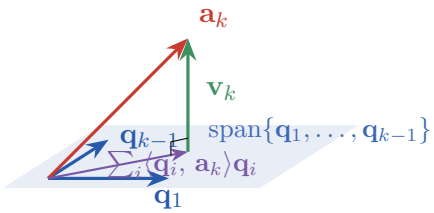


Figure 20.3: Gram–Schmidt in general is projection onto a *plane* (or higher subspace), not just a line. The foot of the perpendicular is the sum of the shadows on the finished directions; the leftover $\mathbf{v}_k$ stands straight up out of $\mathrm{span}\{\mathbf{q}_1, \ldots, \mathbf{q}_{k-1}\}$, then gets normalized to $\mathbf{q}_k$.

20.4 A fully worked three-vector example

We trace the procedure on three vectors in $\mathbb{R}^3$. Take

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \qquad \mathbf{a}_3 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

First direction. $\|\mathbf{a}_1\| = \sqrt{1^2+1^2+0^2} = \sqrt{2}$, so

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\\0\end{bmatrix}.$$

Second direction. First the overlap with $\mathbf{q}_1$:

$$\langle \mathbf{q}_1,\, \mathbf{a}_2\rangle = \frac{1}{\sqrt{2}}(1\cdot 1 + 1\cdot 0 + 0\cdot 1) = \frac{1}{\sqrt{2}}.$$

Subtract that shadow from $\mathbf{a}_2$:

$$\mathbf{v}_2 = \begin{bmatrix}1\\0\\1\end{bmatrix} - \frac{1}{\sqrt{2}}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\\0\end{bmatrix} = \begin{bmatrix}1\\0\\1\end{bmatrix} - \frac{1}{2}\begin{bmatrix}1\\1\\0\end{bmatrix} = \begin{bmatrix}\frac{1}{2}\\-\frac{1}{2}\\1\end{bmatrix}.$$

Its length is $\|\mathbf{v}_2\| = \sqrt{\frac{1}{4}+\frac{1}{4}+1} = \sqrt{\frac{3}{2}} = \dfrac{\sqrt{6}}{2}$, so

$$\mathbf{q}_2 = \frac{2}{\sqrt{6}}\begin{bmatrix}\frac{1}{2}\\-\frac{1}{2}\\1\end{bmatrix} = \frac{1}{\sqrt{6}}\begin{bmatrix}1\\-1\\2\end{bmatrix}.$$

Quick check: $\langle \mathbf{q}_1,\, \mathbf{q}_2\rangle = \frac{1}{\sqrt{2}}\frac{1}{\sqrt{6}}(1-1+0) = 0$. Perpendicular, as promised.

Third direction. Now subtract *both* shadows from $\mathbf{a}_3$. Compute the two overlaps:

$$\langle \mathbf{q}_1,\, \mathbf{a}_3\rangle = \frac{1}{\sqrt{2}}(0+1+0) = \frac{1}{\sqrt{2}}, \qquad \langle \mathbf{q}_2,\, \mathbf{a}_3\rangle = \frac{1}{\sqrt{6}}(0-1+2) = \frac{1}{\sqrt{6}}.$$

Then

$$\mathbf{v}_3 = \begin{bmatrix}0\\1\\1\end{bmatrix} - \frac{1}{\sqrt{2}}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\\0\end{bmatrix} - \frac{1}{\sqrt{6}}\cdot\frac{1}{\sqrt{6}}\begin{bmatrix}1\\-1\\2\end{bmatrix}.$$

The coefficients are $\frac{1}{2}$ and $\frac{1}{6}$, so

$$\mathbf{v}_3 = \begin{bmatrix}0\\1\\1\end{bmatrix} - \frac{1}{2}\begin{bmatrix}1\\1\\0\end{bmatrix} - \frac{1}{6}\begin{bmatrix}1\\-1\\2\end{bmatrix} = \begin{bmatrix}0-\frac{1}{2}-\frac{1}{6}\\1-\frac{1}{2}+\frac{1}{6}\\1-0-\frac{1}{3}\end{bmatrix} = \begin{bmatrix}-\frac{2}{3}\\\frac{2}{3}\\\frac{2}{3}\end{bmatrix}.$$

Its length is $\|\mathbf{v}_3\| = \sqrt{3\cdot\frac{4}{9}} = \sqrt{\frac{4}{3}} = \dfrac{2}{\sqrt{3}}$, so

$$\mathbf{q}_3 = \frac{\sqrt{3}}{2}\begin{bmatrix}-\frac{2}{3}\\\frac{2}{3}\\\frac{2}{3}\end{bmatrix} = \frac{1}{\sqrt{3}}\begin{bmatrix}-1\\1\\1\end{bmatrix}.$$

Final sanity check on the third vector against the first two:

$$\langle \mathbf{q}_1,\, \mathbf{q}_3\rangle = \tfrac{1}{\sqrt{6}}(-1+1+0) = 0, \qquad \langle \mathbf{q}_2,\, \mathbf{q}_3\rangle = \tfrac{1}{\sqrt{18}}(-1-1+2) = 0.$$

Three mutually orthogonal unit vectors, constructed from the original basis.

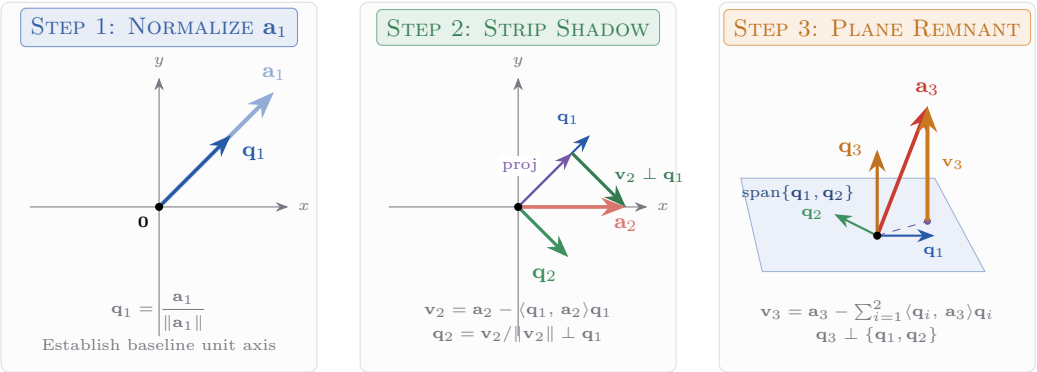


Figure 20.4: The three sequential steps of Gram–Schmidt orthogonalization. *Step 1:* normalize the first vector $\mathbf{a}_1$ to set the initial unit direction $\mathbf{q}_1$. *Step 2:* project $\mathbf{a}_2$ onto $\mathbf{q}_1$ and strip off that shadow; the leftover $\mathbf{v}_2$ is orthogonal to $\mathbf{q}_1$ and normalizes to $\mathbf{q}_2$. *Step 3:* project $\mathbf{a}_3$ onto the plane $\text{span}\{\mathbf{q}_1, \mathbf{q}_2\}$, strip off both shadows, leaving the vertical remnant $\mathbf{v}_3$ that stands perpendicular to the entire plane; normalize to $\mathbf{q}_3$.

20.5 The QR factorization

The steps of the Gram–Schmidt process directly yield a matrix factorization. Rearranging the formula from Proposition 20.2:

$$\mathbf{a}_k = \langle \mathbf{q}_1, \mathbf{a}_k \rangle \, \mathbf{q}_1 + \cdots + \langle \mathbf{q}_{k-1}, \mathbf{a}_k \rangle \, \mathbf{q}_{k-1} + \|\mathbf{v}_k\| \, \mathbf{q}_k.$$

Every original vector $\mathbf{a}_k$ is written as a combination of $\mathbf{q}_1, \ldots, \mathbf{q}_k$ – strictly *only* those, never the later $\mathbf{q}$'s, because $\mathbf{q}_{k+1}, \ldots$ did not exist yet when we processed $\mathbf{a}_k$. That "only the earlier $\mathbf{q}$'s" fact is exactly what makes a matrix *upper triangular*.

Stack the $\mathbf{a}_k$ as the columns of a matrix $\mathbf{A}$ and the $\mathbf{q}_k$ as the columns of $\mathbf{Q}$. The coefficients above fill a matrix $\mathbf{R}$, and the relation becomes a single clean equation:

$$\boxed{\mathbf{A} = \mathbf{QR}}$$

where $\mathbf{Q}$ has orthonormal columns and $\mathbf{R}$ is upper triangular with the entries

$$R_{ik} = \langle \mathbf{q}_i, \mathbf{a}_k \rangle \quad (i < k), \qquad R_{kk} = \|\mathbf{v}_k\|, \qquad R_{ik} = 0 \quad (i > k).$$

Intuition

The factorization $\mathbf{A} = \mathbf{QR}$ separates geometry from coordinates. The matrix $\mathbf{Q}$ contains the orthonormal basis vectors, while the upper-triangular matrix $\mathbf{R}$ contains the coordinates that express each original column in this basis. Reading column k of $\mathbf{A} = \mathbf{QR}$ expresses $\mathbf{a}_k$ as a linear combination of $\mathbf{q}_1, \ldots, \mathbf{q}_k$.

The triangular structure of $\mathbf{R}$ reflects the order in which Gram–Schmidt proceeds: each $\mathbf{a}_k$ depends only on $\mathbf{q}_1, \ldots, \mathbf{q}_k$. The diagonal entries $R_{kk} = \|\mathbf{v}_k\|$ are strictly positive whenever the columns $\mathbf{a}_k$ are linearly independent.

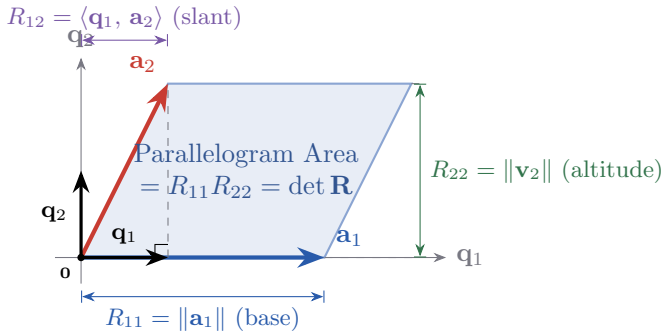


Figure 20.5: The geometric meaning of $\mathbf{A} = \mathbf{QR}$. In the orthonormal coordinates given by $(\mathbf{q}_1, \mathbf{q}_2)$, the first column $\mathbf{a}_1$ lies along $\mathbf{q}_1$ with base length $R_{11} = \|\mathbf{a}_1\|$. The second column $\mathbf{a}_2$ slants along $\mathbf{q}_1$ by $R_{12} = \langle \mathbf{q}_1, \mathbf{a}_2 \rangle$ and reaches perpendicular altitude $R_{22} = \|\mathbf{v}_2\|$ along $\mathbf{q}_2$. The area of the spanned parallelogram is base $\times$ height $= R_{11}R_{22}$, which is precisely the determinant $\det \mathbf{R}$.

Pause

Pause and think. Why must every diagonal entry $R_{kk} = \|\mathbf{v}_k\|$ be strictly positive whenever the columns of $\mathbf{A}$ are linearly independent? If $\|\mathbf{v}_k\| = 0$, then $\mathbf{v}_k = \mathbf{0}$, which would mean $\mathbf{a}_k = \sum_{i<k} \langle \mathbf{q}_i, \mathbf{a}_k \rangle \mathbf{q}_i$. In words: $\mathbf{a}_k$ would be a linear combination of the earlier vectors, contradicting independence. Linear independence guarantees $R_{kk} > 0$, ensuring $\mathbf{R}$ is invertible.

20.5.1 QR for our three-vector example

We already have everything. The dot products we computed are precisely the entries of $\mathbf{R}$. With

$$\mathbf{Q} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{bmatrix}, \qquad \mathbf{R} = \begin{bmatrix} \|\mathbf{a}_1\| & \langle \mathbf{q}_1, \mathbf{a}_2 \rangle & \langle \mathbf{q}_1, \mathbf{a}_3 \rangle \\ 0 & \|\mathbf{v}_2\| & \langle \mathbf{q}_2, \mathbf{a}_3 \rangle \\ 0 & 0 & \|\mathbf{v}_3\| \end{bmatrix} = \begin{bmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{\sqrt{6}}{2} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{2}{\sqrt{3}} \end{bmatrix}.$$

We check the columns of $\mathbf{QR}$ directly. Column 1 of $\mathbf{R}$ is $(\sqrt{2}, 0, 0)^{\mathsf{T}}$, so

$$\mathbf{Q} \begin{bmatrix} \sqrt{2} \\ 0 \\ 0 \end{bmatrix} = \sqrt{2}\,\mathbf{q}_1 = \sqrt{2} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \mathbf{a}_1. \checkmark$$

And column 2: $\mathbf{R}$'s second column is $(\frac{1}{\sqrt{2}}, \frac{\sqrt{6}}{2}, 0)^{\mathsf{T}}$, giving

$$\frac{1}{\sqrt{2}}\mathbf{q}_1 + \frac{\sqrt{6}}{2}\mathbf{q}_2 = \frac{1}{2}\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + \frac{\sqrt{6}}{2} \cdot \frac{1}{\sqrt{6}} \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + \frac{1}{2}\begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \mathbf{a}_2. \checkmark$$

Compute it yourself

Factor $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ by hand. The columns are $\mathbf{a}_1 = (1, 0, 1)^{\mathsf{T}}$ and $\mathbf{a}_2 = (2, 1, 0)^{\mathsf{T}}$.

- $\|\mathbf{a}_1\| = \sqrt{2}$, $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 0, 1)^{\mathsf{T}}$.
- $\langle \mathbf{q}_1, \mathbf{a}_2 \rangle = \frac{1}{\sqrt{2}}(2 + 0 + 0) = \sqrt{2}$.
- $\mathbf{v}_2 = (2, 1, 0)^{\mathsf{T}} - \sqrt{2} \cdot \frac{1}{\sqrt{2}}(1, 0, 1)^{\mathsf{T}} = (1, 1, -1)^{\mathsf{T}}$, $\|\mathbf{v}_2\| = \sqrt{3}$, $\mathbf{q}_2 = \frac{1}{\sqrt{3}}(1, 1, -1)^{\mathsf{T}}$.

Then $\mathbf{R} = \begin{bmatrix} \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{3} \end{bmatrix}$. Multiply $\mathbf{QR}$ back out and confirm you recover $\mathbf{A}$.

20.5.2 Properties of orthogonal matrices

A matrix whose columns are orthonormal satisfies the single most useful identity in this corner of the subject:

$$\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}.$$

Entry (i, j) of $\mathbf{Q}^{\mathsf{T}}\mathbf{Q}$ is $\langle \mathbf{q}_i, \mathbf{q}_j \rangle$, which is 1 when $i = j$ and 0 otherwise. When $\mathbf{Q}$ is square it is called **orthogonal**, and its inverse is simply its transpose: $\mathbf{Q}^{-1} = \mathbf{Q}^{\mathsf{T}}$. Orthogonal matrices preserve lengths and angles: $\|\mathbf{Qx}\| = \|\mathbf{x}\|$ and $\langle \mathbf{Qx}, \mathbf{Qy} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle$. Geometrically, multiplication by a square orthogonal matrix represents a rigid motion (a rotation or reflection). Computationally, preserving lengths prevents the amplification of numerical errors.

Before/after Strang

Strang introduces orthonormal bases and Gram–Schmidt alongside projections and least squares, expressing $\mathbf{A} = \mathbf{QR}$ in this form. The foundational geometric idea is that of Figure 20.2: project, subtract, normalize. The next sections show how $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$ simplifies the normal equations into a triangular solve, and why the modified algorithm is used in numerical software.

20.6 Why orthonormal columns make least squares easy

Recall the least-squares problem from the previous chapter: $\mathbf{Ax} = \mathbf{b}$ usually has no solution because $\mathbf{b}$ is not in the column space, so we settle for the $\hat{\mathbf{x}}$ that minimizes $\|\mathbf{Ax} - \mathbf{b}\|$. The classic route is the **normal equations**

$$\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^{\mathsf{T}}\mathbf{b}.$$

This system is mathematically exact, but explicitly forming $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ can amplify roundoff errors by squaring the **condition number** of $\mathbf{A}$. The condition number $\kappa(\mathbf{A})$ measures sensitivity to perturbations: a relative error of size ε in the data can produce a relative error of order $\kappa(\mathbf{A})\varepsilon$ in the computed solution. Forming $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ squares this quantity: $\kappa(\mathbf{A}^{\mathsf{T}}\mathbf{A}) = \kappa(\mathbf{A})^2$. For example, if $\kappa(\mathbf{A}) = 10^4$ (costing about 4 decimal digits of precision), $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ has condition number 10^8, losing 8 digits of precision before solving begins. The QR factorization avoids forming $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ directly.

Substituting $\mathbf{A} = \mathbf{QR}$ into the normal equations simplifies the system. First,

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} = (\mathbf{QR})^{\mathsf{T}}(\mathbf{QR}) = \mathbf{R}^{\mathsf{T}}\mathbf{Q}^{\mathsf{T}}\mathbf{QR} = \mathbf{R}^{\mathsf{T}}\mathbf{R},$$

using $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$. Likewise $\mathbf{A}^{\mathsf{T}}\mathbf{b} = \mathbf{R}^{\mathsf{T}}\mathbf{Q}^{\mathsf{T}}\mathbf{b}$. The normal equations become $\mathbf{R}^{\mathsf{T}}\mathbf{R}\,\hat{\mathbf{x}} = \mathbf{R}^{\mathsf{T}}\mathbf{Q}^{\mathsf{T}}\mathbf{b}$. Cancel the common $\mathbf{R}^{\mathsf{T}}$ (it is invertible, since its diagonal entries $\|\mathbf{v}_k\|$ are nonzero) to land at the strikingly simple

$$\boxed{\mathbf{R}\,\hat{\mathbf{x}} = \mathbf{Q}^{\mathsf{T}}\mathbf{b}.}$$

Focus

Least squares via QR proceeds in two steps: compute the vector $\mathbf{Q}^{\mathsf{T}}\mathbf{b}$, then solve the upper-triangular system $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^{\mathsf{T}}\mathbf{b}$ by back-substitution. This avoids forming $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and preserves the condition number $\kappa(\mathbf{A})$.

There is a clean geometric reading too. The projection of $\mathbf{b}$ onto the column space of $\mathbf{A}$ – the best approximation $\hat{\mathbf{b}}$ – is simply $\hat{\mathbf{b}} = \mathbf{QQ}^{\mathsf{T}}\mathbf{b}$, where $\mathbf{QQ}^{\mathsf{T}}$ is the projection matrix onto $\mathrm{col}(\mathbf{A})$. The orthonormal columns let you read off the projection as a sum of independent one-dimensional projections, $\hat{\mathbf{b}} = \sum_i \langle \mathbf{q}_i, \mathbf{b}\rangle\, \mathbf{q}_i$, where each coordinate is computed independently without coupling across directions (Figure 20.6).

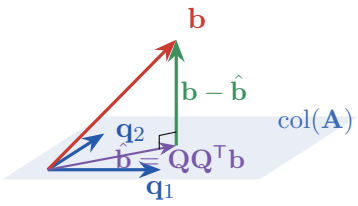


Figure 20.6: Least squares is one projection. The best approximation $\hat{\mathbf{b}}$ is the shadow of $\mathbf{b}$ on the column space, $\hat{\mathbf{b}} = \mathbf{QQ}^{\mathsf{T}}\mathbf{b}$, and the residual $\mathbf{b} - \hat{\mathbf{b}}$ stands perpendicular to that space. Orthonormal columns allow computing each coordinate of $\hat{\mathbf{b}}$ independently.

Pause

Pause and think. When $\mathbf{Q}$ is rectangular ($m \times n$ with $m > n$), $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}_n$ is the $n \times n$ identity, but $\mathbf{QQ}^{\mathsf{T}}$ is an $m \times m$ matrix that is *not* the identity. What does $\mathbf{QQ}^{\mathsf{T}}$ do? It is the orthogonal projector onto $\mathrm{col}(\mathbf{A})$: it leaves any vector already in the column space alone, and zeroes out any vector perpendicular to it.

Example 20.1 (Least squares by QR)**.** Fit a line through the three points $(0,1), (1,2), (2,2)$ – the same setup as in Chapter 19. We want $c + dx \approx y$, so

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

Gram–Schmidt on the columns gives $\mathbf{q}_1 = \frac{1}{\sqrt{3}}(1,1,1)^{\mathsf{T}}$ and, after subtracting, $\mathbf{v}_2 = (0,1,2)^{\mathsf{T}} - 1 \cdot (1,1,1)^{\mathsf{T}} = (-1,0,1)^{\mathsf{T}}$, so $\mathbf{q}_2 = \frac{1}{\sqrt{2}}(-1,0,1)^{\mathsf{T}}$. Here $\langle \mathbf{q}_1, \mathbf{a}_2\rangle = \frac{1}{\sqrt{3}}(0+1+2) = \sqrt{3}$ and $\|\mathbf{v}_2\| = \sqrt{2}$,

so

$$\mathbf{R} = \begin{bmatrix} \sqrt{3} & \sqrt{3} \\ 0 & \sqrt{2} \end{bmatrix}, \qquad \mathbf{Q}^{\mathsf{T}}\mathbf{b} = \begin{bmatrix} \frac{1}{\sqrt{3}}(1+2+2) \\ \frac{1}{\sqrt{2}}(-1+0+2) \end{bmatrix} = \begin{bmatrix} \frac{5}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}.$$

Back-substitute $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^{\mathsf{T}}\mathbf{b}$. The bottom row gives $\sqrt{2}\,d = \frac{1}{\sqrt{2}}$, so $d = \frac{1}{2}$. The top row gives $\sqrt{3}\,c + \sqrt{3}\,d = \frac{5}{\sqrt{3}}$, i.e. $c + d = \frac{5}{3}$, so $c = \frac{5}{3} - \frac{1}{2} = \frac{7}{6}$. The best line is $y = \frac{7}{6} + \frac{1}{2}x$, matching what the normal equations would give – but reached by a triangular solve.

Modern application

QR inside every regression and every deep-net training loop. When you call `numpy.linalg.lstsq`, fit an ordinary least-squares model in a statistics package, or solve the linear sub-problem inside a Gauss–Newton or Levenberg–Marquardt optimizer, the library does *not* form $\mathbf{A}^{\mathsf{T}}\mathbf{A}$. It computes a QR factorization (or the closely related SVD) and back-substitutes, exactly because squaring the condition number can destroy accuracy on real, near-collinear data. In large-scale machine learning, QR also underlies stable orthogonalization steps: QR-based whitening, and re-orthogonalizing basis vectors in Krylov solvers such as GMRES and Lanczos for huge sparse systems. It likewise powers orthogonal weight initialization, where a network's weight matrix is set to the $\mathbf{Q}$ factor of a random matrix so that signals neither explode nor vanish across layers. Whenever preserving orthogonality and vector lengths is essential, QR factorization is standard.

Connection to other math

Gram–Schmidt depends only on the inner product, not on $\mathbb{R}^n$. The process used *only* dot products, lengths, and projections. Swap the dot product for any other inner product and the identical recipe orthogonalizes those objects. The famous case: on the space of polynomials with the inner product $\langle f, g\rangle = \int_{-1}^{1} f(x)g(x)\,dx$, applying Gram–Schmidt to the plain powers $1, x, x^2, x^3, \ldots$ produces the **Legendre polynomials**, the orthogonal basis at the heart of Gaussian quadrature and spherical harmonics. Other weightings give Chebyshev, Hermite, and Laguerre families. In this way, Gram–Schmidt provides a unified procedure for constructing classical orthogonal polynomial systems in analysis.

20.7 Classical versus modified Gram-Schmidt

The version we derived is **classical** Gram–Schmidt: to build $\mathbf{v}_k$, project the *original* $\mathbf{a}_k$ onto each finished $\mathbf{q}_i$ and subtract all the shadows from $\mathbf{a}_k$ at once. In exact arithmetic, this method is exact. In floating-point arithmetic, however, computing each projection coefficient $\langle \mathbf{q}_i, \mathbf{a}_k \rangle$ from the original vector $\mathbf{a}_k$ allows roundoff errors to accumulate; when columns are nearly dependent, the computed vectors can lose orthogonality.

Modified Gram–Schmidt addresses this instability by subtracting each projection from the running remainder rather than from the original vector:

Formal

Modified Gram–Schmidt. Set $\mathbf{v} \leftarrow \mathbf{a}_k$. Then for $i = 1, 2, \ldots, k-1$ in order, update

$$\mathbf{v} \;\leftarrow\; \mathbf{v} - \langle \mathbf{q}_i,\, \mathbf{v} \rangle\, \mathbf{q}_i,$$

and finally $\mathbf{q}_k = \mathbf{v}/\|\mathbf{v}\|$.

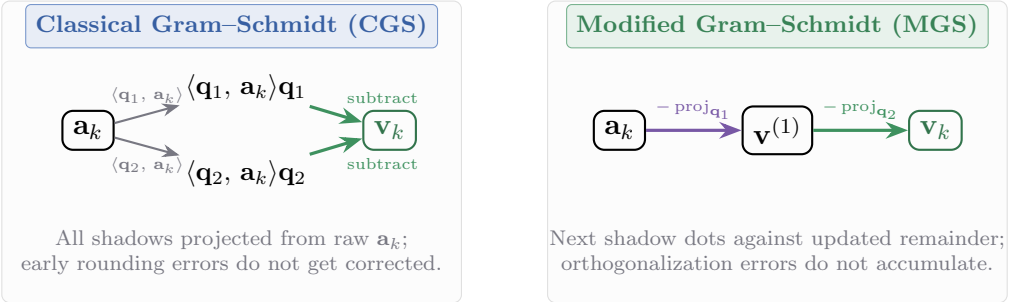


Figure 20.7: Classical versus Modified Gram–Schmidt. *Classical:* all projections are computed simultaneously from the original vector $\mathbf{a}_k$, so early errors are baked into every term. *Modified:* each shadow is subtracted sequentially, measuring overlap against the updated remainder $\mathbf{v}^{(1)}$ to prevent error accumulation in floating-point arithmetic.

In exact arithmetic this gives *exactly* the same answer as the classical version: subtracting components one at a time, each from the partially-cleaned vector, removes the same total component. The difference appears in floating-point arithmetic. After subtracting the $\mathbf{q}_1$-component, the remainder $\mathbf{v}$ is orthogonal to $\mathbf{q}_1$ to within machine precision. Computing $\langle \mathbf{q}_2,\, \mathbf{v} \rangle$ evaluates overlap against this updated remainder, preventing errors from previous projections from compounding into subsequent steps.

Common trap

The classical and modified algorithms differ by *which vector you dot against*: classical always uses the original $\mathbf{a}_k$; modified uses the running remainder $\mathbf{v}$. In exact arithmetic, both algorithms yield identical vectors. In finite-precision arithmetic, however, modified Gram–Schmidt maintains $\mathbf{Q}^{\mathsf{T}}\mathbf{Q}$ much closer to $\mathbf{I}$, making it the standard formulation in numerical code.

Remark 20.1. Even modified Gram–Schmidt is not the gold standard for the worst-conditioned problems. Production QR routines (LAPACK's `dgeqrf`) instead build $\mathbf{Q}$ from a sequence of **Householder reflections**—orthogonal transformations that reflect a vector across a hyperplane to introduce zeros below the diagonal of an entire column at once. Householder QR is backward stable and is the standard implementation in libraries such as `numpy` and MATLAB. Gram–Schmidt remains conceptually fundamental because it makes the underlying geometry explicit: repeated projection onto orthogonal subspaces. We meet Householder reflections again in Chapter 33.

History and connection

The algorithm we call Gram–Schmidt has a rich history driven by applied problems across two centuries.

The earliest formulation came from Pierre-Simon Laplace in 1816. Working on celestial mechanics – calculating the gravitational orbits of Jupiter and Saturn so that minor measurement errors would not compound over centuries – Laplace wrote down an orthogonal projection recipe to separate orbital parameters. In 1883, the Danish actuary and mathematician Jørgen Pedersen Gram independently derived the method to fit polynomial curves to actuarial tables (life expectancy and pension records) by least squares; with raw powers $1, x, x^2, \ldots$, the normal equations were nearly singular and impossible to solve reliably by hand. Twenty-four years later, in 1907, Erhard Schmidt reinvented the procedure in Göttingen to construct orthonormal function bases for Fredholm integral equations, where the unknown function appears inside an integral.

All three mathematicians worked with quill, paper, and printed logarithm tables. In exact arithmetic, Gram–Schmidt is exact. When implemented on digital computers with floating-point arithmetic, however, classical Gram–Schmidt suffers from numerical instability: rounding errors during subtractions can compound rapidly, causing computed columns to lose orthogonality.

The resolution came in two waves. First, numerical analysts introduced Modified Gram–Schmidt, which projects against the running remainder rather than the raw original, dramatically curbing error growth. Then in 1958, Alston Householder introduced elementary orthogonal reflections – reflecting entire columns onto coordinate axes at once using full orthogonal matrices, which preserves orthogonality to the full precision of machine hardware.

The naming reflects historical rediscovery: Gram's 1883 paper arose in actuarial mathematics, while Schmidt's 1907 work applied to infinite-dimensional function spaces. Laplace's 1816 formulation was overlooked for more than a century before being recognized. Together, these ideas led to the QR algorithm for eigenvalues (developed by John Francis and Vera Kublanovskaya in 1961), modern least-squares solvers, and orthogonal polynomials throughout quantum mechanics.

Why this matters

A single operation—subtracting orthogonal projections—yields an orthonormal basis (Gram–Schmidt), a matrix factorization ($\mathbf{A} = \mathbf{QR}$), a stable method for least squares ($\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\mathsf{T}\mathbf{b}$), and families of orthogonal polynomials under different inner products. The geometric principle of decomposing a vector into orthogonal components provides both theoretical clarity and computational stability throughout linear algebra.

20.8 Cost and intuition recap

The procedure is summarized by three operations: **project, subtract, normalize**. Key concepts:

- **The wish.** A crooked basis tangles every coordinate computation into a linear solve. An orthonormal basis untangles it: each coordinate is one dot product, $\langle \mathbf{q}_i, \mathbf{b} \rangle$.
- **The construction.** Gram–Schmidt builds the orthonormal frame one direction at a time. For each $\mathbf{a}_k$, subtract its shadows on the finished $\mathbf{q}_i$ and normalize what is left: $\mathbf{v}_k = \mathbf{a}_k - \sum_{i<k} \langle \mathbf{q}_i, \mathbf{a}_k \rangle \mathbf{q}_i$, $\mathbf{q}_k = \mathbf{v}_k / \|\mathbf{v}_k\|$.
- **The bookkeeping is a factorization.** Recording those dot products and lengths gives

$\mathbf{A} = \mathbf{QR}$: $\mathbf{Q}$ holds clean directions, $\mathbf{R}$ (upper triangular, positive diagonal) holds the recipe.

- **The payoff.** $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ turns the normal equations into the triangular solve $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\mathsf{T}\mathbf{b}$, dodging the condition-number squaring of $\mathbf{A}^\mathsf{T}\mathbf{A}$.

Cost. For an $m \times n$ matrix ($m \geq n$), Gram–Schmidt (classical or modified) costs about $2mn^2$ floating-point operations – the same leading order as forming and factoring $\mathbf{A}^\mathsf{T}\mathbf{A}$, but with far better accuracy because the errors never get squared. On a chalkboard the bottleneck is instead the square roots and fractions in the normalizations; choosing friendly starting vectors keeps the arithmetic clean. **Stability.** Modified Gram–Schmidt keeps $\mathbf{Q}^\mathsf{T}\mathbf{Q}$ much closer to $\mathbf{I}$ than classical; Householder QR (the production default) is better still. The geometry, though, is identical across all three: subtract the projection.

Connection to other math

Geometrically, the $\mathbf{A} = \mathbf{QR}$ factorization also measures the volume of the parallelotope spanned by the columns of $\mathbf{A}$: the orthogonal matrix $\mathbf{Q}$ acts as a rigid transformation preserving volumes and angles, while the upper-triangular matrix $\mathbf{R}$ records lengths and perpendicular altitudes. The volume of the parallelotope is given by the product of these altitudes:

$$\text{Volume} \;=\; R_{11}R_{22}\cdots R_{nn} \;=\; \det \mathbf{R}.$$

In Chapter 21 ("Determinants: Volume and Sign"), we examine how square matrices scale multi-dimensional volumes and how the sign of the determinant indicates spatial orientation. In Part VI, we study eigenvalues and eigenvectors to understand how transformations behave along invariant directions. Later, in Chapter 29, orthogonal bases and spectral theory combine in the Singular Value Decomposition (SVD), a foundational tool of applied linear algebra.

20.9 Exercises

Exercise 20.1 (Two-vector warmup)**.** Apply Gram–Schmidt to $\mathbf{a}_1 = \begin{bmatrix}2\\1\end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix}3\\4\end{bmatrix}$. Find $\mathbf{q}_1, \mathbf{q}_2$ and verify they are orthonormal. Then write down $\mathbf{R}$ and confirm $\mathbf{A} = \mathbf{QR}$.

Exercise 20.2 (A clean three-vector run)**.** Orthonormalize $\mathbf{a}_1 = \begin{bmatrix}1\\1\\1\end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix}1\\1\\0\end{bmatrix}$, $\mathbf{a}_3 = \begin{bmatrix}1\\0\\0\end{bmatrix}$. (They are independent, so the process will not break down.) Report $\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3$ and the matrix $\mathbf{R}$.

Exercise 20.3 (Read off $\mathbf{R}$ without recomputing)**.** Suppose you have already produced $\mathbf{q}_1, \mathbf{q}_2$ from $\mathbf{a}_1, \mathbf{a}_2$ and found $\|\mathbf{a}_1\| = 5$, $\langle \mathbf{q}_1, \mathbf{a}_2 \rangle = 3$, $\|\mathbf{v}_2\| = 4$. Write the full 2×2 matrix $\mathbf{R}$. What is $\det \mathbf{R}$, and why does it equal the area of the parallelogram spanned by $\mathbf{a}_1, \mathbf{a}_2$?

Exercise 20.4 (Least squares by QR)**.** Fit a line to the points $(1, 1), (2, 2), (3, 2), (4, 5)$ using the QR route: build $\mathbf{A}, \mathbf{b}$, run Gram–Schmidt to get $\mathbf{Q}, \mathbf{R}$, then solve $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\mathsf{T}\mathbf{b}$ by back-substitution. Compare your slope and intercept against solving the normal equations directly.

Exercise 20.5 (Order matters for $\mathbf{Q}$, not for the span)**.** Run Gram–Schmidt on $\mathbf{a}_1 = \begin{bmatrix}1\\0\\0\end{bmatrix}, \mathbf{a}_2 = \begin{bmatrix}1\\1\\0\end{bmatrix}$

and then again on the *reversed* list $\mathbf{a}_2, \mathbf{a}_1$. You get different orthonormal bases. Explain geometrically why the two $\mathbf{q}_1$'s point differently but the spans of $\{\mathbf{q}_1, \mathbf{q}_2\}$ are identical.

Exercise 20.6 (Why $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ but $\mathbf{Q}\mathbf{Q}^\mathsf{T} \neq \mathbf{I}$)**.** Take the $\mathbf{Q}$ from Example 20.1 (a 3×2 matrix with orthonormal columns). Compute both $\mathbf{Q}^\mathsf{T}\mathbf{Q}$ and $\mathbf{Q}\mathbf{Q}^\mathsf{T}$. One is the 2×2 identity; the other is a 3×3 projection matrix. Identify which subspace $\mathbf{Q}\mathbf{Q}^\mathsf{T}$ projects onto, and explain why it cannot be the full identity.

Exercise 20.7 (Classical vs. modified, by hand)**.** Let ε be tiny and take $\mathbf{a}_1 = \begin{bmatrix} 1 \\ \varepsilon \\ 0 \end{bmatrix}, \mathbf{a}_2 = \begin{bmatrix} 1 \\ 0 \\ \varepsilon \end{bmatrix}, \mathbf{a}_3 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ (nearly dependent). Carry out both classical and modified Gram–Schmidt symbolically, keeping terms to first order in ε. Show that in exact arithmetic both give the same $\mathbf{q}_3$, and describe in words where, with rounding, the classical version would let a $\mathbf{q}_1$-error leak into $\mathbf{q}_3$ while the modified version would not.

Exercise 20.8 (Orthogonal matrices preserve length – proof)**.** Let $\mathbf{Q}$ be square with $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$. Prove that $\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|$ for every $\mathbf{x}$, and that $\langle \mathbf{Q}\mathbf{x}, \mathbf{Q}\mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle$ for all $\mathbf{x}, \mathbf{y}$. (Start from $\|\mathbf{Q}\mathbf{x}\|^2 = (\mathbf{Q}\mathbf{x})^\mathsf{T}(\mathbf{Q}\mathbf{x})$.) Conclude that multiplication by $\mathbf{Q}$ is a rotation or reflection, and connect this to why QR-based computation does not amplify roundoff.

Exercise 20.9 (Legendre by hand)**.** Using the inner product $\langle f, g \rangle = \int_{-1}^{1} f(x)g(x)\,dx$, apply Gram–Schmidt to $1, x, x^2$. Confirm you obtain (up to scaling) 1, x, $x^2 - \frac{1}{3}$, the first three Legendre polynomials, and that they are mutually orthogonal under this integral.

Exercise 20.10 (A non-square challenge)**.** Let $\mathbf{A}$ be 4×3 with independent columns. Explain why the "thin" QR has $\mathbf{Q}$ of size 4×3 and $\mathbf{R}$ of size 3×3, and why $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}_3$ while $\mathbf{Q}\mathbf{Q}^\mathsf{T}$ is a 4×4 projector of rank 3. What does the rank-3 projector do to a vector $\mathbf{b} \in \mathbb{R}^4$, and how does that relate to least squares?

Part V

Determinants

Chapter 21

Determinants: Volume and Sign

In Chapter 20, Gram–Schmidt straightened columns into perpendicular altitudes whose product measured the volume of a box. You already know that a matrix moves all of space at once – stretching, rotating, shearing, and sometimes flattening. We now ask: *by how much does the machine inflate or deflate space?*

Feed the machine a little square of area 1. It hands you back a parallelogram. Measure that parallelogram's area. That single number – the area-scaling factor of the transformation – is almost the whole story of the determinant. We say "almost" because there is one more subtlety: the determinant also records whether the transformation reflects space, as in a mirror. So the determinant carries *two* pieces of geometric information at once: **how much volume changed**, and **whether orientation was preserved or reversed**.

Every algebraic rule in this chapter – the 2×2 formula $ad - bc$, the 3×3 expansion, the condition $\det = 0$ for collapse, and the product law $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$ – follows directly from tracking how a matrix scales volume and whether it flips orientation.

21.1 The Unit Box and Its Image

Start in the plane with the cleanest possible region: the unit square. Its two sides are the standard basis vectors,

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

and it has area exactly 1. Now act with a 2×2 matrix $\mathbf{A}$. The square has corners $\mathbf{0}$, $\mathbf{e}_1$, $\mathbf{e}_1 + \mathbf{e}_2$, $\mathbf{e}_2$, and $\mathbf{A}$ maps each corner to a new point. Because $\mathbf{A}\mathbf{e}_1$ is the first column of $\mathbf{A}$ and $\mathbf{A}\mathbf{e}_2$ is the second column, the image of the unit square is the parallelogram spanned by the two **columns** of $\mathbf{A}$.

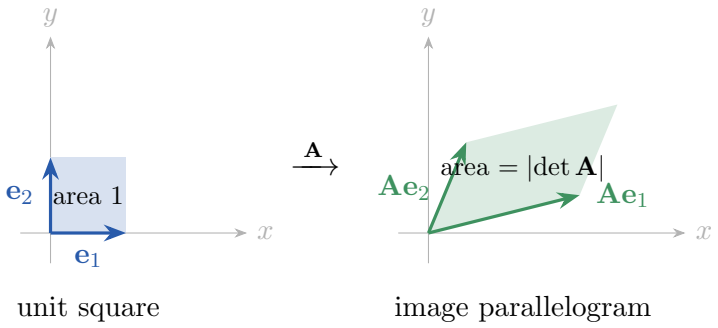


Figure 21.1: The determinant is the area-scaling factor. The machine $\mathbf{A}$ sends the unit square (area 1) to the parallelogram spanned by its columns $\mathbf{Ae}_1$ and $\mathbf{Ae}_2$. The area of that parallelogram is $|\det \mathbf{A}|$.

Intuition

The determinant of $\mathbf{A}$ is the **area of the parallelogram you get by sending the unit square through $\mathbf{A}$** – with a sign attached. The columns of $\mathbf{A}$ are the two sides of that parallelogram. A determinant of 5 means the machine multiplies every area by 5; a determinant of $\frac{1}{2}$ means it halves every area; a determinant of 0 means it flattens the square to a sliver of zero area.

Why does this scaling apply to every region, and not just the unit square? Because the machine is *linear*, and linearity is exactly the property that the grid stays a grid. If the unit square's area is multiplied by some factor k, then so is the area of the square next to it, and the one above it, and a 1000×1000 block of them tiled together. Any region you like can be approximated by tiny squares of the grid; each one gets its area scaled by k; so the whole region's area gets scaled by k. The determinant is one number that controls *all* areas at once. That is what makes it worth a name.

Focus

Hold onto three sentences. (1) $\det \mathbf{A}$ is the signed area-scaling factor of the machine $\mathbf{A}$. (2) The columns of $\mathbf{A}$ are the edges of the image of the unit box. (3) It scales *every* region by the same factor, because linear maps treat all of space uniformly.

21.2 The 2×2 Determinant: Deriving $ad - bc$

We met the formula $ad - bc$ as an algebraic condition for 2×2 invertibility back in Chapter 6 and Chapter 8. Now let us see why it had to be that exact combination of letters by computing the area of the column parallelogram directly. Take a general 2×2 matrix and read off its columns:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad \mathbf{u} = \mathbf{Ae}_1 = \begin{bmatrix} a \\ c \end{bmatrix}, \qquad \mathbf{v} = \mathbf{Ae}_2 = \begin{bmatrix} b \\ d \end{bmatrix}.$$

We want the area of the parallelogram with sides $\mathbf{u}$ and $\mathbf{v}$. The cleanest geometric derivation is to enclose the tilted parallelogram inside a bounding box of width $a+b$ and height $c+d$, then subtract the pieces that lie outside the tilt (Figure 21.2).

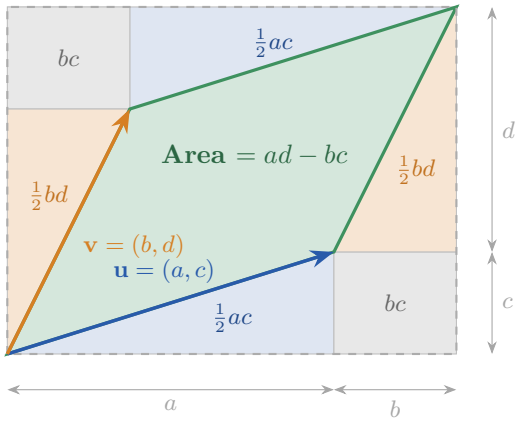


Figure 21.2: The geometric dissection derivation of $\det \mathbf{A} = ad - bc$. Enclose the parallelogram spanned by $\mathbf{u} = (a, c)$ and $\mathbf{v} = (b, d)$ in a rectangle of size $(a + b) \times (c + d)$. Subtracting the two blue triangles of area $\frac{1}{2}ac$, the two orange triangles of area $\frac{1}{2}bd$, and the two gray corner rectangles of area bc leaves exactly $(ac + ad + bc + bd) - (ac + bd + 2bc) = ad - bc$.

The total area of the bounding rectangle is:

$$\text{Total Box Area} = (a + b)(c + d) = ac + ad + bc + bd.$$

Subtract the six pieces that fall outside the parallelogram:

- Two right triangles with base a and height c: combined area $2 \times \left(\frac{1}{2}ac\right) = ac$.
- Two right triangles with base b and height d: combined area $2 \times \left(\frac{1}{2}bd\right) = bd$.
- Two corner rectangles of width b and height c: combined area $2 \times (bc) = 2bc$.

Subtracting these outer regions from the bounding box leaves the area of the parallelogram:

$$\text{Area} = (ac + ad + bc + bd) - (ac + bd + 2bc) = ad - bc.$$

This is the 2×2 formula, not as a symbol-pushing rule, but as the natural remainder when the surrounding frame is removed.

We can also reach the exact same formula through base and height using the dot product:

Intuition

The signed area of the parallelogram with edge vectors $\mathbf{u} = (u_1, u_2)$ and $\mathbf{v} = (v_1, v_2)$ is $u_1v_2 - u_2v_1$. One quick way to remember why: area is "base times height," and after you write base $= \|\mathbf{u}\|$ and height $= \|\mathbf{v}\| \sin\theta$ (the part of $\mathbf{v}$ perpendicular to $\mathbf{u}$), the product $\|\mathbf{u}\| \, \|\mathbf{v}\| \sin\theta$ expands exactly to $u_1v_2 - u_2v_1$.

Let us check that algebra directly. The area of a parallelogram is base times height. Take the base to be $\mathbf{u}$, with length $\|\mathbf{u}\|$. The height is the component of $\mathbf{v}$ perpendicular to $\mathbf{u}$, namely $\|\mathbf{v}\| \sin\theta$ where θ is the angle from $\mathbf{u}$ to $\mathbf{v}$. So

$$\text{area} = \|\mathbf{u}\| \, \|\mathbf{v}\| \, \sin\theta.$$

Now recall that the dot product gives $\|\mathbf{u}\|\,\|\mathbf{v}\|\cos\theta = u_1v_1 + u_2v_2$. There is a perpendicular twin: if you rotate $\mathbf{u}$ counterclockwise by 90° to get $\mathbf{u}^\perp = (-u_2, u_1)$, then the angle from $\mathbf{u}^\perp$ to $\mathbf{v}$ is $\theta - \frac{\pi}{2}$, so

$$\mathbf{u}^\perp \cdot \mathbf{v} = \|\mathbf{u}^\perp\|\,\|\mathbf{v}\|\,\cos\!\left(\theta - \tfrac{\pi}{2}\right) = \|\mathbf{u}\|\,\|\mathbf{v}\|\,\sin\theta = \text{signed area},$$

using $\|\mathbf{u}^\perp\| = \|\mathbf{u}\|$ and $\cos(\theta - \frac{\pi}{2}) = \sin\theta$. Computing the same dot product component by component, $\mathbf{u}^\perp \cdot \mathbf{v} = -u_2v_1 + u_1v_2 = u_1v_2 - u_2v_1$. When $\mathbf{v}$ lies counterclockwise from $\mathbf{u}$ this is positive; when it lies clockwise it is negative – the sign is the orientation we will study in Section 21.3. Plugging in our columns $\mathbf{u} = (a, c)$ and $\mathbf{v} = (b, d)$ (Figure 21.3):

$$\boxed{\det\begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.}$$

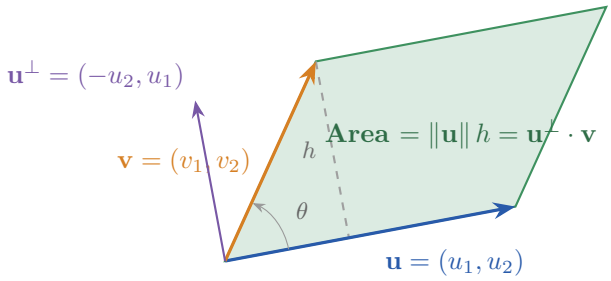


Figure 21.3: The base-and-height derivation of the 2×2 determinant. The parallelogram spanned by $\mathbf{u} = (u_1, u_2)$ and $\mathbf{v} = (v_1, v_2)$ has base length $\|\mathbf{u}\|$ and perpendicular altitude $h = \|\mathbf{v}\|\sin\theta$. Rotating $\mathbf{u}$ by 90° counterclockwise gives $\mathbf{u}^\perp = (-u_2, u_1)$, which points directly along the altitude direction. The dot product $\mathbf{u}^\perp \cdot \mathbf{v} = u_1v_2 - u_2v_1$ matches base times height, producing $ad - bc$ when $\mathbf{u} = (a, c)$ and $\mathbf{v} = (b, d)$.

Formal

For a 2×2 matrix,

$$\det\begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

Its absolute value is the area of the parallelogram spanned by the columns; its sign is + if the columns keep the same orientation as $(\mathbf{e}_1, \mathbf{e}_2)$ (counterclockwise) and − if they reverse it.

Example 21.1 (Three quick 2×2 determinants)**.** *A pure stretch.* $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ gives $\det = 2\cdot3 - 0\cdot0 = 6$. Areas multiply by 6: a 2-by-3 rectangle, exactly as you'd expect from doubling width and tripling height.

A shear. $\mathbf{S} = \begin{bmatrix} 1 & 5 \\ 0 & 1 \end{bmatrix}$ gives $\det = 1 \cdot 1 - 5 \cdot 0 = 1$. The shear slides the top of the square sideways by 5 but does *not* change its area – base 1, height 1, still area 1. This is the "deck of cards" fact made numerical.

A collapse. $\mathbf{C} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ gives $\det = 1 \cdot 4 - 2 \cdot 2 = 0$. The second column is twice the first, so both columns lie on one line; the "parallelogram" is squashed flat onto that line and has zero area. We will make a great deal of this $\det = 0$ case shortly.

Compute it yourself

Compute these three 2×2 determinants by hand, then *draw* the parallelogram of columns for each and check that the area matches:

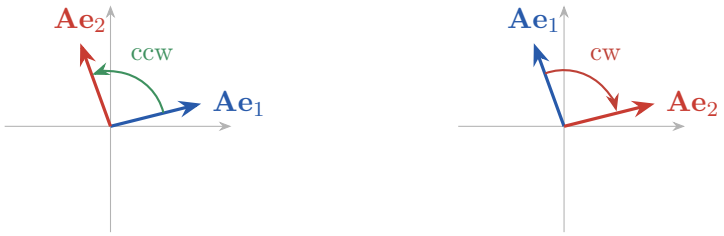
$$\begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}.$$

(Answers: 5, 1, 0. The middle one is a 90° rotation – area preserved, and we'll explain its positive sign in the next section. The last one is singular.)

21.3 Sign and Orientation: Tracking Reflections

So far we have discussed area, which is strictly non-negative. Where does the *sign* come from? It comes from orientation – indicating whether the transformation reflects space.

In the standard orientation of the plane, moving from the first basis vector $\mathbf{e}_1$ to the second $\mathbf{e}_2$ proceeds **counterclockwise**. Now act with $\mathbf{A}$ and inspect the image vectors $\mathbf{A}\mathbf{e}_1$ and $\mathbf{A}\mathbf{e}_2$. If moving from the first image column to the second is still counterclockwise, the transformation preserves orientation and $\det \mathbf{A} > 0$. If the direction is now *clockwise*, the transformation reflects the plane, and $\det \mathbf{A} < 0$.



$\det > 0$: orientation kept $\det < 0$: orientation flipped

Figure 21.4: The sign of the determinant records orientation: does the transformation keep the basis ordering counterclockwise (left, $\det > 0$) or reverse it to clockwise (right, $\det < 0$)? A negative determinant indicates that the transformation includes a reflection.

Intuition

A positive determinant corresponds to an orientation-preserving transformation: stretches, rotations, and shears. A negative determinant indicates that the transformation involves a **reflection**: the handedness of the coordinate system is reversed. The magnitude $|\det|$ tells you the factor by which area changes, while the *sign* indicates whether orientation is preserved $(+)$ or reversed $(-)$.

Example 21.2 (A reflection has determinant -1)**.** The matrix that flips the plane across the x-axis is $\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$. It leaves $\mathbf{e}_1$ alone and sends $\mathbf{e}_2$ to $-\mathbf{e}_2$. Areas are clearly unchanged (a flip is rigid), so $|\det| = 1$; but orientation is reversed, so the sign is negative:

$$\det \mathbf{F} = (1)(-1) - (0)(0) = -1. \checkmark$$

Compare the rotation by 90°, $\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$, which has $\det = (0)(0) - (-1)(1) = +1$: a rotation is rigid *and* keeps orientation, so its determinant is exactly $+1$. Reflection -1, rotation $+1$: the sign is the whole difference between them.

Common trap

A common slip is to think $\det \mathbf{A}$ is the area of the image and therefore "always positive." It is the *signed* area. The determinant $ad - bc$ can be negative, and its sign records whether the transformation reverses orientation. Only $|\det \mathbf{A}|$ is the (unsigned) area.

Pause

Pause and think: A reflection across a line in 2D or across a plane in 3D has determinant -1. What about a central inversion through the origin in 3D, sending every vector $\mathbf{x}$ to $-\mathbf{x}$? Does it reverse orientation? *Answer:* In 3D, reflecting through the origin is the matrix $\mathbf{A} = -\mathbf{I}_3 = \text{diag}(-1, -1, -1)$. Because it reverses all three coordinate axes, each axis contributes a factor of -1, giving $\det(-\mathbf{I}_3) = (-1)^3 = -1$. It flips orientation: your right hand becomes a left hand, and no rigid physical rotation in 3-space can undo that flip. (In 2D, by contrast, $\det(-\mathbf{I}_2) = (-1)^2 = +1$: flipping both planar axes is simply a 180° rotation, which preserves orientation.)

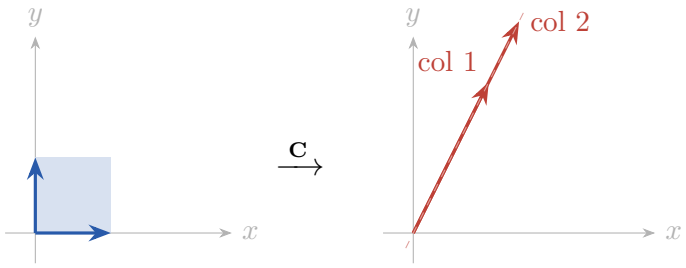
Connection to other math

Orientation connects linear algebra directly to multivariable calculus. When you change variables in a multivariable integral, $\iint f\, dx\, dy$ becomes $\iint f\, |\det J|\, du\, dv$, where J is the Jacobian matrix of the change of coordinates. That factor $|\det J|$ is the local area-scaling factor of this chapter – the change of variables formula applies the determinant-as-volume concept to infinitesimal regions. The absolute value appears because a standard scalar integral over a region depends only on the magnitude of the area element, not on the orientation of the parameterization. In oriented integration (differential forms and Stokes' theorem), the sign determines the orientation of the boundary and domain.

21.4 $\det = 0$ Means Collapse

Of all the values a determinant can take, one is special: zero. A zero determinant is the geometric signature of **collapse**, and it is the single most useful thing the determinant tells you.

If $\det \mathbf{A} = 0$, the unit square is sent to a parallelogram of zero area. But a genuine parallelogram has positive area; the only way to get area zero is for the two column vectors to lie on the *same line* (in 2D) or in the *same plane* (in 3D). The machine has squashed a full-dimensional box down into something thinner – a line segment, or a flat sheet. Space lost a dimension.



area 1 area 0: a line

Figure 21.5: $\det \mathbf{C} = 0$ for $\mathbf{C} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. Both columns land on the single line $y = 2x$, so the unit square is crushed flat to a line segment of zero area. The machine is *singular*: it has destroyed a dimension and cannot be undone.

This is the bridge between the geometry of this chapter and the algebra of everything that came before. Collapse means the columns are **linearly dependent**; equivalently, the matrix has rank less than full; equivalently, the columns fail to span all of space; equivalently, $\mathbf{Ax} = \mathbf{0}$ has a nonzero solution (some direction gets crushed to the origin); equivalently, $\mathbf{A}$ has no inverse. The determinant packages all of those conditions into a single number you can check.

Formal

For a square matrix $\mathbf{A}$, the following are equivalent:

- $\det \mathbf{A} = 0$;
- $\mathbf{A}$ is **singular** (not invertible);
- the columns of $\mathbf{A}$ are linearly dependent;
- $\operatorname{rank} \mathbf{A} < n$;
- the equation $\mathbf{Ax} = \mathbf{0}$ has a nonzero solution;
- $\mathbf{A}$ collapses $\mathbb{R}^n$ onto a subspace of dimension $< n$.

Equivalently, $\det \mathbf{A} \neq 0$ iff $\mathbf{A}$ is invertible.

Why this matters

This property is the primary reason determinants were studied centuries before matrices were formalized. A single scalar, computed directly from the entries, determines whether an entire linear system has a unique solution. The condition $\det \neq 0$ unites several fundamental properties: the columns are linearly independent, the transformation is invertible, and the null space contains only the zero vector. Compressing a solvability test for n equations into the sign-or-zero of a single value rests on a geometric criterion: *does the transformation preserve volume, or does it collapse space into fewer dimensions?*

Walking question

Picture a 3×3 matrix whose three columns lie in a common plane through the origin. The unit cube flattens into that plane, producing zero volume. Mentally verify three consequences: the determinant is 0; some nonzero vector maps to $\mathbf{0}$; and the original cube cannot be reconstructed from its image, because distinct points in $\mathbb{R}^3$ collapse onto the same point in the plane. Geometrically, this loss of dimension is why invertibility fails.

21.5 The 3×3 Determinant: Volume of the Box

In three dimensions the story is identical with "area" upgraded to "volume." The unit cube has the three basis vectors $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ as edges and volume 1. A 3×3 matrix sends it to the **parallelepiped** (a slanted box) spanned by the three columns, and $|\det \mathbf{A}|$ is that box's volume. The sign now records "right-handed" versus "left-handed" – whether the three image vectors obey the right-hand rule or its mirror.

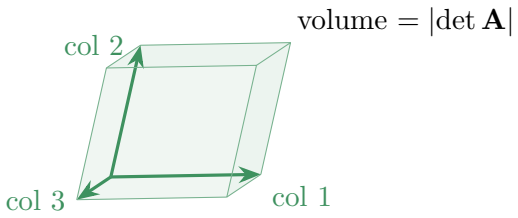


Figure 21.6: In 3D the unit cube becomes the parallelepiped spanned by the three columns of $\mathbf{A}$. Its volume is $|\det \mathbf{A}|$, and the sign tells you whether the three column vectors form a right-handed or left-handed frame.

To compute a 3×3 determinant by hand, expand along the first row using 2×2 minors. For

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix},$$

the rule (*cofactor expansion*, which we'll justify thoroughly in the next chapter) is

$$\det \mathbf{A} = a \det \begin{bmatrix} e & f \\ h & i \end{bmatrix} - b \det \begin{bmatrix} d & f \\ g & i \end{bmatrix} + c \det \begin{bmatrix} d & e \\ g & h \end{bmatrix}.$$

Notice the alternating signs $+ - +$: you cross out the row and column of each top entry, take the determinant of what's left, and attach the checkerboard sign. Written out,

$$\det \mathbf{A} = a(ei - fh) - b(di - fg) + c(dh - eg).$$

Example 21.3 (A worked 3×3)**.** Let

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 2 \\ 0 & 1 & 1 \end{bmatrix}.$$

Expanding along the top row:

$$\det \mathbf{A} = 2 \det \begin{bmatrix} 3 & 2 \\ 1 & 1 \end{bmatrix} - 0 \cdot (\cdots) + 1 \det \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}.$$

The two small determinants are $3 \cdot 1 - 2 \cdot 1 = 1$ and $1 \cdot 1 - 3 \cdot 0 = 1$, so

$$\det \mathbf{A} = 2(1) - 0 + 1(1) = 3.$$

The machine inflates volume by a factor of 3, and the positive sign says it keeps the right-handed orientation of space. You can expand along any row or column you like and get the same answer, so it pays to pick the one with the most zeros: here the first column or third row, each with a zero, would be just as fast.

Common trap

The "diagonals" trick (Sarrus' rule) – multiply the three downhill diagonals, subtract the three uphill diagonals – works **only for** 3×3. There is no 4×4 version of it; applying it in higher dimensions yields incorrect results. For larger matrices, use cofactor expansion or row reduce to triangular form (next section) and multiply the pivots. Sarrus' rule applies only to 3×3 matrices, not to general dimensions.

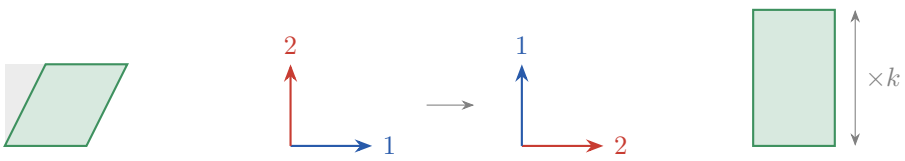
21.6 How Row Operations Change the Determinant

Computing large determinants via cofactor expansion is computationally prohibitive: an $n \times n$ expansion involves $n!$ terms, which for $n = 20$ exceeds 2.4×10^{18} arithmetic operations. The practical method uses **Gaussian elimination**. We simply track how each elementary row operation affects the determinant. There are three operations, each with a clear geometric effect.

(1) Add a multiple of one row to another: determinant unchanged. This is the shear. Adding row i to row j slides one edge of the box parallel to another; base and height are unchanged, so the volume – and hence the determinant – is unchanged. This is the primary tool: you may do as many of these as you like for free.

(2) Swap two rows: determinant flips sign. Swapping two rows swaps two edges of the box, which reverses orientation – the right hand becomes a left hand. The volume is the same but the sign flips: $\det \to -\det$.

(3) Multiply a row by a scalar k: determinant multiplies by k. Stretching one edge of the box by k stretches the volume by k. So scaling a single row by k scales the determinant by k.



(1) shear
det unchanged

(2) swap
$\det \to -\det$

(3) scale row
$\det \to k \det$

Figure 21.7: The three elementary row operations and their effect on the determinant. Shears (add a multiple of a row) leave it alone, swaps flip its sign, and scaling a row by k multiplies it by k.

These three rules turn determinant computation into elimination. Row reduce $\mathbf{A}$ to an upper-triangular matrix $\mathbf{U}$, keeping a running tally: count the swaps (each contributes a factor -1) and record any row-scalings. Then use one more beautiful fact.

Theorem 21.1 (Determinant of a triangular matrix)**.** *If* $\mathbf{T}$ *is (upper or lower) triangular, then* $\det \mathbf{T}$ *is the product of its diagonal entries.*

Why it's true, geometrically and by expansion. Geometrically: a triangular matrix's columns are already "staircased," and the box they span is a sheared rectangular box whose edges along each axis have lengths equal to the diagonal entries; shears don't change volume, so the volume is just the product of those edge lengths. Algebraically: take an upper-triangular matrix and expand along its first *column*. In an upper-triangular matrix the first column has only the top-left entry t_{11} nonzero (everything below the diagonal is zero), so only the $(1,1)$ cofactor survives, leaving t_{11} times the determinant of the smaller upper-triangular matrix obtained by deleting row 1 and column 1. Induct on that smaller matrix, and you peel off $t_{11}t_{22}\cdots t_{nn}$. (For a lower-triangular matrix, expand along the first *row* instead; the argument is the mirror image.) □

Example 21.4 (Determinant by elimination)**.** Compute $\det \mathbf{A}$ for

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 1 \\ -2 & 1 & 5 \end{bmatrix}.$$

Eliminate below the first pivot with shears (free, det unchanged): $R_2 \leftarrow R_2 - 2R_1$ and $R_3 \leftarrow R_3 + R_1$:

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & -1 \\ 0 & 2 & 6 \end{bmatrix}.$$

Now $R_3 \leftarrow R_3 - 2R_2$ (again free):

$$\mathbf{U} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 8 \end{bmatrix}.$$

No swaps, no scalings, so $\det \mathbf{A} = \det \mathbf{U} = 2 \cdot 1 \cdot 8 = 16$. The diagonal entries of $\mathbf{U}$ are exactly the pivots of elimination – which is why people say *the determinant is the product of the pivots* (times $(-1)^{\#\text{swaps}}$). If at any point a pivot had been forced to 0, the determinant would be 0 and the matrix singular.

Before/after Strang

Strang builds the determinant axiomatically: he lists three defining properties – $\det I = 1$, the sign flips under a row swap, and the determinant is linear in each row separately – and shows that those three force the formula $ad - bc$ and everything else. That is the elegant top-down route, and after Strang you should absolutely go back and admire how those three axioms generate the whole theory (rules (1), (2), (3) above are corollaries of them). We have taken a geometric approach instead: volume first, formulas second, because volume scaling provides the intuition behind Strang's axioms. "Linear in each row" corresponds to stretching one edge of the box; "swap flips the sign" reflects reversal of orientation. Both viewpoints develop the same underlying theory.

Compute it yourself

Compute $\det \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 3 \\ 1 & 0 & 8 \end{bmatrix}$ twice: once by cofactor expansion along the first row, and once by elimination to triangular form. You should get -1 both times. If your two answers disagree, the likeliest culprit is a forgotten sign in the cofactor checkerboard or an uncounted row swap.

21.7 The Product Rule: $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$

The product rule follows directly from the volume interpretation. Recall that the product $\mathbf{AB}$ corresponds to the composition "apply $\mathbf{B}$ first, then $\mathbf{A}$." Apply that composite mapping to the unit box:

- First $\mathbf{B}$ scales the box's signed volume by $\det \mathbf{B}$.
- Then $\mathbf{A}$ scales *that* signed volume by $\det \mathbf{A}$.

Scaling factors of successive operations multiply (Figure 21.8), so the total scaling is $\det \mathbf{A} \cdot \det \mathbf{B}$. And the total scaling of the combined machine $\mathbf{AB}$ is by definition $\det(\mathbf{AB})$. Therefore:

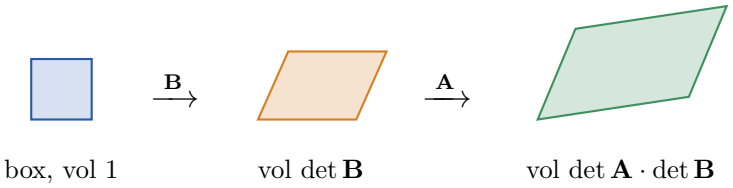


Figure 21.8: The product rule, seen as composition. Doing $\mathbf{B}$ first multiplies the box's signed volume by $\det \mathbf{B}$; then $\mathbf{A}$ multiplies that signed volume by $\det \mathbf{A}$. The combined machine $\mathbf{AB}$ therefore scales signed volume by $\det \mathbf{A} \det \mathbf{B}$, which is exactly $\det(\mathbf{AB})$.

Formal

For any two $n \times n$ matrices,

$$\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}.$$

The determinant turns "compose machines" (matrix multiplication) into "multiply numbers." Orientation behaves correctly too: two flips ($\det \mathbf{A} < 0$, $\det \mathbf{B} < 0$) compose to no flip ($\det(\mathbf{AB}) > 0$), exactly as $(-)(-) = +$.

Intuition

Volume scalings *compose by multiplication*. If $\mathbf{B}$ triples volume and $\mathbf{A}$ doubles it, then doing $\mathbf{B}$ then $\mathbf{A}$ multiplies volume by 6. That is the entire content of $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$ – a statement that would be a nightmare to prove by grinding through the entry formulas, but is a one-line truth about scaling factors.

Example 21.5 (Checking the product rule by hand)**.** Let

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 4 \\ 2 & 1 \end{bmatrix}.$$

Then $\det \mathbf{A} = 6$ and $\det \mathbf{B} = 1 - 8 = -7$, so the product rule predicts $\det(\mathbf{AB}) = 6 \cdot (-7) = -42$. Check directly:

$$\mathbf{AB} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 9 \\ 6 & 3 \end{bmatrix}, \qquad \det(\mathbf{AB}) = 12 - 54 = -42. \ \checkmark$$

The product rule has consequences that fall out instantly – each a small geometric story.

Corollary 21.1 (Determinant of an inverse)**.** *If* $\mathbf{A}$ *is invertible, then* $\det(\mathbf{A}^{-1}) = \dfrac{1}{\det \mathbf{A}}$.

Proof. Apply the product rule to $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$. Since $\det \mathbf{I} = 1$ (the identity changes no volume),

$$\det \mathbf{A} \cdot \det(\mathbf{A}^{-1}) = \det \mathbf{I} = 1,$$

so $\det(\mathbf{A}^{-1}) = 1/\det \mathbf{A}$. Geometrically: if $\mathbf{A}$ multiplies volume by 5, its inverse must divide volume by 5 to put everything back. And notice this proof *requires* $\det \mathbf{A} \neq 0$ – you cannot divide by zero, which is the algebraic version of "a collapsed box can't be un-collapsed." □

Corollary 21.2 (Powers and similar matrices)**.** $\det(\mathbf{A}^k) = (\det \mathbf{A})^k$ *for any integer* $k \geq 1$*, and similar matrices have equal determinants:* $\det(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \det \mathbf{A}$.

Proof. The first is the product rule applied k times. For the second,

$$\det(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \det(\mathbf{P}^{-1}) \det \mathbf{A} \det \mathbf{P} = \frac{1}{\det \mathbf{P}} \det \mathbf{A} \det \mathbf{P} = \det \mathbf{A}.$$

This is why the determinant is an *intrinsic* property of a linear map, independent of which basis you write it in: changing basis is exactly conjugation by $\mathbf{P}$, and the volume-scaling of a map doesn't care what coordinates you use to describe it. □

Remark 21.1 (A warning about sums). The determinant multiplies beautifully but does *not* add. In general $\det(\mathbf{A}+\mathbf{B}) \neq \det \mathbf{A} + \det \mathbf{B}$. For instance with $\mathbf{A} = \mathbf{B} = \mathbf{I}$ in $\mathbb{R}^2$, $\det(\mathbf{A}+\mathbf{B}) = \det(2\mathbf{I}) = 4$, while $\det \mathbf{A} + \det \mathbf{B} = 1 + 1 = 2$. There is no "volume of a sum" – geometrically, adding matrices does not correspond to any clean operation on the boxes, whereas multiplying them does (compose the machines).

Connection to other math

Gram–Schmidt and QR: volume as a product of altitudes. In Chapter 20, we decomposed any matrix $\mathbf{A}$ with independent columns into $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q}$ has orthonormal columns and $\mathbf{R}$ is upper triangular. Now we can see what that factorization reveals about volume. By the product rule,

$$|\det \mathbf{A}| = |\det \mathbf{Q}| \, |\det \mathbf{R}|.$$

Because $\mathbf{Q}$ is an orthogonal matrix ($\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$), it preserves lengths and angles – a rigid rotation or reflection – so $|\det \mathbf{Q}| = 1$. And because $\mathbf{R}$ is upper triangular, its determinant is

simply the product of its diagonal entries: $\det \mathbf{R} = r_{11} r_{22} \cdots r_{nn}$. Therefore,

$$|\det \mathbf{A}| = r_{11} r_{22} \cdots r_{nn}.$$

Look at what this means geometrically: $r_{11} = \|\mathbf{a}_1\|$ is the length of the first edge (the base). The entry r_{22} is the perpendicular altitude of the second column above the first. The entry r_{33} is the perpendicular altitude of the third column above the plane of the first two, and so on. The determinant is literally base $\times$ height$_1$ $\times$ height$_2$ $\times \cdots \times$ height$_{n-1}$. Gram–Schmidt's orthogonalization was building the determinant all along.

Pause

Pause and think: If $\mathbf{A}$ is a 3×3 matrix and you double every entry to form $2\mathbf{A}$, does its determinant double? *Answer:* No, it multiplies by $8 = 2^3$. Doubling the matrix scales all three edges of the parallelepiped by 2, so the volume scales by $2 \times 2 \times 2 = 8$. In general, because $k\mathbf{A} = (k\mathbf{I})\mathbf{A}$, the product rule gives $\det(k\mathbf{A}) = \det(k\mathbf{I}) \det \mathbf{A} = k^n \det \mathbf{A}$ for an $n \times n$ matrix.

Modern application

Why generative models care about determinants. A *normalizing flow* is a modern machine-learning model that learns a probability distribution by warping a simple one (a Gaussian blob) through an invertible neural network f. The change-of-variables formula says that if a point $\mathbf{z}$ has density $p_Z(\mathbf{z})$, then its image $\mathbf{x} = f(\mathbf{z})$ has density

$$p_X(\mathbf{x}) = p_Z(\mathbf{z}) \left|\det J_f(\mathbf{z})\right|^{-1},$$

where J_f is the Jacobian of f. The $|\det J_f|$ is doing *exactly* the job from this chapter: it is the local volume-scaling factor of the warp, and it has to appear because probability mass is conserved – if the map stretches a region to twice its volume, the density there must drop by half so the total still integrates to 1. Training such a model means computing (and differentiating) $\log|\det J_f|$ millions of times, so flow architectures are designed with triangular Jacobians on purpose – because, as you now know, a triangular determinant is just the product of the diagonal, cheap to compute. The entire field of normalizing flows is an engineering response to "make the determinant fast."

History and connection

Today we introduce determinants as functions of square matrices, but historically the sequence was reversed: determinants were discovered more than a century and a half *before* anyone defined a matrix. In the late seventeenth century, mathematicians faced an immediate practical roadblock: how to tell whether a large system of linear equations has a unique solution without slogging through pages of variable elimination. In Japan, Seki Takakazu (1683) worked out systematic tables of elimination coefficients up to five variables using bamboo counting rods (*sangi*, traditional sticks arranged on a counting board to represent numerical coefficients). A decade later in Europe, Gottfried Wilhelm Leibniz (1693) independently derived equivalent solvability conditions for three equations in two unknowns in a letter to Guillaume de l'Hôpital. For over a century, however, determinants remained confined to coordinate algebra, formu-

lated as polynomials of $n!$ signed terms (all $n!$ ways to pair rows with columns, which for a modest 10×10 system already exceeds 3.6 million terms). In that coordinate language, proving basic facts like the product rule $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$ required tedious algebraic substitutions. The breakthrough came when Joseph-Louis Lagrange (1773) recognized that a 3×3 determinant calculates the volume of a tetrahedron (a four-sided triangular pyramid), and Augustin-Louis Cauchy (1812) unified the subject by proving that the determinant is the signed volume scaling factor of space, establishing the modern name and notation. Carl Gustav Jacob Jacobi (1841) later extended this to calculus via the Jacobian (the matrix of local partial derivatives measuring how coordinate transformations stretch infinitesimal volume elements and flip orientation).

This geometric shift created a nineteenth-century debate. When Arthur Cayley and James Joseph Sylvester introduced the concept of a *matrix* in the 1850s, many traditional algebraists resisted, arguing that determinants were the primary objects of study and matrices were merely auxiliary notation. The dispute was settled only when linear transformations took center stage: matrices describe the linear mapping, while the determinant provides one scalar summary of that mapping.

What this theory provided was a single scalar test for invertibility ($\det \mathbf{A} \neq 0$) and a precise tracker of whether a linear transformation preserves spatial handedness or reflects it. Collapsing an $n \times n$ transformation to a single scalar, however, discards directional information. A matrix can stretch one axis by a factor of a million and compress another by a factor of a millionth while leaving the volume scaling exactly 1. In addition, calculating determinants via $n!$ permutation terms is computationally infeasible for large matrices, and floating-point round-off can underflow a small product to zero. In Chapter 22 we develop cofactors and Cramer's rule to study symbolic inverses, and in Chapter 29 we meet the singular value decomposition, which restores the individual stretch factors that the determinant aggregates together.

21.8 Putting It Together

The geometry of the transformed unit box explains the key properties of the determinant:

- The determinant is the **signed volume-scaling factor** of a linear map. Its magnitude is how much volume changes; its sign is whether orientation is preserved (+) or reversed (−).
- In 2D, $\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$, the signed area of the parallelogram of columns. In 3D, cofactor expansion gives the signed volume of the parallelepiped of columns.
- $\det \mathbf{A} = 0$ exactly when the box collapses – when columns are dependent, the map is singular, and no inverse exists. This is the determinant's most important job.
- Row operations have clean effects: shears leave det alone, swaps flip its sign, scalings multiply it. So $\det = (-1)^{\#\text{swaps}}$ times the product of the pivots – the practical way to compute.
- $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$, because volume scalings compose by multiplication. From this fall $\det(\mathbf{A}^{-1}) = 1/\det \mathbf{A}$ and the basis-independence of the determinant.

Why this matters

The determinant is the first place in linear algebra where a single scalar measures how a geometric region transforms under a linear mapping. This concept reappears throughout subsequent chapters: as the invertibility criterion, in Cramer's rule and cofactor formulas, in the characteristic polynomial $\det(\mathbf{A} - \lambda\mathbf{I})$ used to find eigenvalues, and as the product of singular values in the SVD. Each application traces back to the same geometric question: *by what factor does the transformation scale volume, and does it preserve orientation?* Viewing the determinant through volume clarifies why these algebraic formulas take the form they do.

We now understand what the determinant means: it is signed volume scaling. But two pressing practical questions remain. First, how do we peel apart a high-dimensional determinant into smaller, manageable sub-determinants without running elimination? Second, can we harness the determinant to write down an explicit, closed-form formula for the inverse matrix $\mathbf{A}^{-1}$, and for the exact solution $\mathbf{x}$ of a linear system $\mathbf{Ax} = \mathbf{b}$? In Chapter 22, we answer both questions by building cofactors, the adjugate matrix, and Cramer's rule.

21.9 Exercises

Exercise 21.1 (Mental checks: no pencil needed)**.** Answer each of these in five seconds by visualizing the geometry:

(a) If $\mathbf{A}$ is a 4×4 matrix with $\det \mathbf{A} = 3$, what is $\det(2\mathbf{A})$?

(b) A 3×3 matrix has a row of zeros. What is its determinant, and what happened to the unit cube?

(c) If you swap two columns of an $n \times n$ matrix, what happens to the determinant?

(d) If $\det \mathbf{A} = 5$ and $\det \mathbf{B} = -2$, what is $\det(\mathbf{A}^2\mathbf{B}^{-1})$?

(e) Can an orthogonal matrix have determinant equal to 3? Why or why not?

Exercise 21.2 (Warm-up areas)**.** Compute the determinant of each, and state the area (or volume) and the orientation (preserved or reversed):

$$\text{(a) } \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}, \quad \text{(b) } \begin{bmatrix} 1 & 6 \\ 0 & 1 \end{bmatrix}, \quad \text{(c) } \begin{bmatrix} 0 & 2 \\ 3 & 0 \end{bmatrix}, \quad \text{(d) } \begin{bmatrix} 3 & 6 \\ 1 & 2 \end{bmatrix}.$$

Exercise 21.3 (Draw the parallelogram)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, draw the unit square and its image under $\mathbf{A}$ on graph paper. Measure the area of the image parallelogram by the surrounding-rectangle-minus-triangles method, and confirm it equals $\det \mathbf{A}$.

Exercise 21.4 (3×3 two ways)**.** Compute $\det \begin{bmatrix} 1 & 2 & 0 \\ 3 & -1 & 2 \\ 2 & 0 & 1 \end{bmatrix}$ first by cofactor expansion along whichever row or column has the most zeros, then again by elimination to triangular form. Confirm the answers match.

Exercise 21.5 (When is it singular?)**.** Find all values of t for which $\mathbf{A} = \begin{bmatrix} t & 1 & 0 \\ 1 & t & 1 \\ 0 & 1 & t \end{bmatrix}$ is singular. (Set up $\det \mathbf{A} = 0$ and solve the resulting cubic in t.) Interpret your answer geometrically: for those t, what has happened to the box?

Exercise 21.6 (Row operations, tracked)**.** Start from $\mathbf{A} = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 0 \\ 3 & 0 & 2 \end{bmatrix}$. Reduce it to upper-triangular form, carefully recording each swap and each scaling. Use your record plus Theorem 21.1 to compute $\det \mathbf{A}$, and state how many sign flips you incurred.

Exercise 21.7 (Product rule by hand)**.** Let $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 2 & 0 \\ 1 & 5 \end{bmatrix}$. Compute $\det \mathbf{A}$, $\det \mathbf{B}$, the product $\mathbf{AB}$, and $\det(\mathbf{AB})$ directly. Verify $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$. Then compute $\det(\mathbf{A}^{-1})$ without forming $\mathbf{A}^{-1}$.

Exercise 21.8 (Scaling the whole matrix)**.** For an $n \times n$ matrix $\mathbf{A}$ and a scalar k, prove that $\det(k\mathbf{A}) = k^n \det \mathbf{A}$. (Hint: $k\mathbf{A}$ scales *every* one of the n rows by k; apply the row-scaling rule n times.) Then explain geometrically why the exponent is n and not 1: what happens to the volume of an n-dimensional box when you scale all n edges by k?

Exercise 21.9 (Orientation of a reflection)**.** Let $\mathbf{F}_\theta$ reflect the plane across the line through the origin at angle θ. Its matrix is $\mathbf{F}_\theta = \begin{bmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{bmatrix}$. Compute $\det \mathbf{F}_\theta$ and show it equals -1 for *every* θ. Explain in one sentence why this had to be the answer before you computed anything.

Exercise 21.10 (A determinant that proves dependence)**.** Three vectors $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 0 \\ 1 \\ 4 \end{bmatrix}$, $\mathbf{c} = \begin{bmatrix} 2 \\ 5 \\ 10 \end{bmatrix}$ in $\mathbb{R}^3$. By computing a single 3×3 determinant, decide whether they are linearly independent. If they are dependent, find scalars (not all zero) with $x\mathbf{a} + y\mathbf{b} + z\mathbf{c} = \mathbf{0}$, and relate your finding to the determinant being zero.

Exercise 21.11 (Challenge: the area of a triangle)**.** Show that the area of the triangle with vertices $\mathbf{p}, \mathbf{q}, \mathbf{r}$ in the plane is $\frac{1}{2}\big|\det[\,\mathbf{q} - \mathbf{p} \mid \mathbf{r} - \mathbf{p}\,]\big|$, where the 2×2 matrix has columns $\mathbf{q} - \mathbf{p}$ and $\mathbf{r} - \mathbf{p}$. Use it to find the area of the triangle with vertices $(1, 1)$, $(4, 2)$, $(2, 5)$. (Hint: the triangle is half the parallelogram spanned by the two edge vectors from $\mathbf{p}$.)

Exercise 21.12 (Challenge: a proof of the product rule for the 2×2 case)**.** Without invoking the geometric argument, prove $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$ for general 2×2 matrices by brute force: write $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} p & q \\ r & s \end{bmatrix}$, multiply them out, and expand both $\det(\mathbf{AB})$ and $(ad - bc)(ps - qr)$. Watch the cross terms cancel. (This is good algebraic exercise – and it should make you grateful for the one-line geometric proof.)

Chapter 22

Cofactors and Cramer's Rule

In the previous chapter we learned what a determinant *is*: a single number attached to a square matrix that measures signed volume – the factor by which the matrix scales areas in $\mathbb{R}^2$, volumes in $\mathbb{R}^3$, and hyper-volumes beyond, together with a sign that records whether orientation is kept or flipped. We learned the headline facts. The determinant is zero exactly when the matrix collapses space (when its columns are dependent, when it is not invertible). It multiplies: $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$. It is unchanged by the row operation that elimination uses most.

This chapter answers a different question. Not "what does the determinant mean," but "how is it *built*, and what can we build *out* of it" – a structural question rather than a geometric one. Two classical structures grow directly from the determinant: the **cofactor expansion**, which computes a big determinant by peeling it into smaller ones, and the **adjugate**, which assembles those small determinants into an explicit formula for the inverse. Cramer's rule then drops out as a one-line consequence, and it carries a clean geometric reading: each unknown in a linear system is a ratio of two volumes.

A warning before we start, because it will shape how you read everything that follows. The formulas in this chapter are some of the most elegant in linear algebra, but among the least practical for numerical calculation. They are tools for *proving theorems* and for *understanding algebraic structure*, not tools for crunching numbers on large matrices. At the end of the chapter we compare their operation counts directly against Gaussian elimination.

Focus

The whole chapter hangs on one identity, the **cofactor expansion**: a determinant equals a sum, along any single row or column, of each entry times a $\pm$ sign times the determinant of the smaller matrix you get by deleting that entry's row and column. Everything else – the inverse formula, Cramer's rule, the volume interpretation – is a consequence of reading that one identity the right way.

22.1 Peeling a Determinant Apart: Minors and Cofactors

Let us start where the geometry is easiest to hold: a 2×2 matrix, whose determinant you already know.

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad \det \mathbf{A} = ad - bc.$$

Where do the two terms in $ad - bc$ come from? The a multiplies the d – and d is exactly the lone entry left over when you cross out a's row (the top) and a's column (the left). The b multiplies c – and c is what is left when you cross out b's row and column. So the 2×2 determinant is already a sum over the top row: take each top entry, multiply it by "the determinant of what remains," and attach an alternating sign $+, -$.

That little observation is the entire idea, repeated at every size. Let us name the pieces.

Definition 22.1 (Minor)**.** Let $\mathbf{A}$ be an $n \times n$ matrix. The **minor** M_{ij} is the determinant of the $(n-1) \times (n-1)$ matrix obtained by deleting row i and column j of $\mathbf{A}$.

Definition 22.2 (Cofactor)**.** The **cofactor** C_{ij} is the signed minor

$$C_{ij} = (-1)^{i+j} M_{ij}.$$

The sign $(-1)^{i+j}$ is $+1$ when $i + j$ is even and -1 when $i + j$ is odd.

The sign pattern follows a checkerboard layout, always starting with $+$ in the top-left corner:

$$\begin{bmatrix} + & - & + & \cdots \\ - & + & - & \cdots \\ + & - & + & \cdots \\ \vdots & & & \ddots \end{bmatrix}.$$

Intuition

A minor answers "forget about entry (i, j) entirely – if I delete its whole row and column, how much volume does the leftover matrix make?" The cofactor is that same number with a sign glued on. The sign is not decoration: it is bookkeeping for orientation, the same $\pm$ that distinguishes a rotation from a reflection. When we add up entry $\times$ cofactor, the signs make the cross-terms cancel in exactly the way that recovers signed volume.

Common trap

The checkerboard sign $(-1)^{i+j}$ belongs to the *cofactor*, not the minor. A frequent error is to compute the minors correctly, then forget the sign on the off-diagonal positions and get the wrong total. Equally common: starting the checkerboard with a $-$ in the top-left. It always starts with $+$, because $(-1)^{1+1} = (-1)^2 = +1$.

Compute it yourself

Pause and think: What is the checkerboard sign for entry $(2, 3)$? For entry $(3, 1)$? Count with your fingers: row 2, column 3 gives $(-1)^{2+3} = (-1)^5 = -1$. Row 3, column 1 gives

$(-1)^{3+1} = (-1)^4 = +1$. Whenever the sum of coordinates is odd, you get a minus; whenever even, a plus.

22.1.1 The Cofactor Expansion Theorem

Now the central claim. You can compute the determinant by marching along *any* row or *any* column, multiplying each entry by its cofactor, and adding.

Theorem 22.1 (Cofactor / Laplace expansion)**.** *Let* $\mathbf{A}$ *be* $n \times n$ *with entries* a_{ij}*. For any fixed row* i*,*

$$\det \mathbf{A} \;=\; \sum_{j=1}^{n} a_{ij}\, C_{ij} \;=\; \sum_{j=1}^{n} (-1)^{i+j}\, a_{ij}\, M_{ij},$$

and for any fixed column j*,*

$$\det \mathbf{A} \;=\; \sum_{i=1}^{n} a_{ij}\, C_{ij} \;=\; \sum_{i=1}^{n} (-1)^{i+j}\, a_{ij}\, M_{ij}.$$

The key word in that statement is *any*. Expand along the top row, the bottom row, the middle column – you get the same number every time. We will not give the full combinatorial proof from the permutation definition (that is a worthwhile but heavy bookkeeping exercise), but the geometric and recursive heart of it is transparent, and we sketch it in the box below.

Formal

Why expansion works (sketch). The determinant can be *defined* as the signed sum over all permutations σ of $\{1, \dots, n\}$,

$$\det \mathbf{A} = \sum_{\sigma} \operatorname{sgn}(\sigma)\, a_{1\sigma(1)} a_{2\sigma(2)} \cdots a_{n\sigma(n)}.$$

Fix row i. Every term in that giant sum uses *exactly one* entry from row i, say $a_{i\sigma(i)}$. Group the terms by which column j that entry sits in. The terms with $\sigma(i) = j$, after factoring out a_{ij}, are precisely a signed sum over permutations of the remaining $n-1$ indices – which is the determinant of the matrix with row i and column j removed. Tracking how $\operatorname{sgn}(\sigma)$ splits gives exactly the factor $(-1)^{i+j}$. So the grouped sum is $\sum_j a_{ij}(-1)^{i+j} M_{ij}$. Because the original sum was symmetric in how we chose to single out a row, the same argument works for any row or column.

22.1.2 A worked 3×3 expansion

Let us actually do one, slowly, because the mechanics are where people stumble.

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}.$$

We expand along the **first row**. The three top entries are $2, -1, 3$, sitting in positions $(1,1)$, $(1,2)$, $(1,3)$, with signs $+, -, +$.

First term – entry 2, sign +. Delete row 1, column 1:

$$M_{11} = \det \begin{bmatrix} 4 & 5 \\ 0 & 6 \end{bmatrix} = 4 \cdot 6 - 5 \cdot 0 = 24, \qquad +2 \cdot 24 = 48.$$

Second term – entry −1, sign −. Delete row 1, column 2:

$$M_{12} = \det \begin{bmatrix} 0 & 5 \\ 1 & 6 \end{bmatrix} = 0 \cdot 6 - 5 \cdot 1 = -5, \qquad -(-1) \cdot (-5) = -5.$$

Third term – entry 3, sign +. Delete row 1, column 3:

$$M_{13} = \det \begin{bmatrix} 0 & 4 \\ 1 & 0 \end{bmatrix} = 0 \cdot 0 - 4 \cdot 1 = -4, \qquad +3 \cdot (-4) = -12.$$

Add them: $\det \mathbf{A} = 48 - 5 - 12 = 31$.

Now the payoff of "any row or column." That first column of $\mathbf{A}$ has a zero in it ($a_{21} = 0$) and a one ($a_{31} = 1$), which makes the arithmetic lighter. Expand down **column 1** instead, signs +, −, +:

$$\det \mathbf{A} = +2 \det \begin{bmatrix} 4 & 5 \\ 0 & 6 \end{bmatrix} - 0 \cdot (\cdots) + 1 \det \begin{bmatrix} -1 & 3 \\ 4 & 5 \end{bmatrix} = 2(24) + 1\big((-1)(5) - 3 \cdot 4\big) = 48 + (-17) = 31.$$

Same 31, and the middle term vanished for free. This is the one genuine computational lesson of the section, so we make it a rule.

Focus

Expand along the row or column with the most zeros. Each zero entry kills an entire cofactor – a whole smaller determinant you never have to compute. Choosing wisely can turn a painful expansion into a short one.

Compute it yourself

Compute $\det \begin{bmatrix} 1 & 2 & 0 \\ 3 & -1 & 0 \\ 4 & 5 & 2 \end{bmatrix}$ by hand. The third column has two zeros – expand along it. You should find only one nonzero cofactor survives, and it is 2 times a 2×2 determinant. (Answer: $2 \cdot (1 \cdot (-1) - 2 \cdot 3) = 2 \cdot (-7) = -14$.) Now redo it expanding along the first row and confirm you get −14 again, with more work.

Here is a picture of the peeling process: one 3×3 determinant becomes three 2×2 determinants, each living on the entries that survive a deletion.

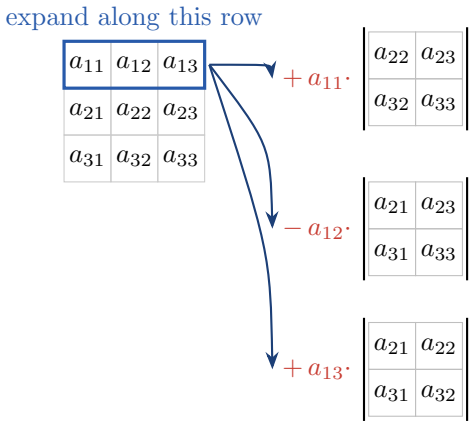


Figure 22.1: Cofactor expansion along the top row turns one 3×3 determinant into three signed 2×2 minors. Each minor lives on the entries that survive deleting the highlighted entry's row and column.

Before/after Strang

Strang introduces cofactors as the third of his "three ways to compute a determinant" (the others being elimination to triangular form, and the big permutation formula). Reading this *before* Strang, hold onto the recursive picture in Figure 22.1: a determinant of size n is a signed combination of determinants of size $n-1$. Reading it *after* Strang, notice that his "BIG formula" (the permutation sum) and the cofactor formula are the same sum, merely grouped differently – grouping by the choice in one row is exactly what produces the cofactors.

22.1.3 Cofactors and the cost of computing

It is tempting to think cofactor expansion is *the* way to find a determinant. For tiny matrices it is fine. But count the work. To expand an $n \times n$ determinant you compute n cofactors, each an $(n-1) \times (n-1)$ determinant; each of those spawns $n-1$ more, and so on. The number of multiplications grows like $n!$ – factorial. For $n = 10$ that is over three million products; for $n = 20$ it is more than the number of seconds in the age of the universe. Elimination, by contrast, reaches a triangular form (whose determinant is just the product of the pivots) in roughly $n^3/3$ operations. For $n = 20$ that is a few thousand. We will return to this contrast in Section 22.6, but we emphasize this distinction: **cofactor expansion is a reasoning tool, not a computing tool.**

The difference in efficiency is asymptotic. Figure 22.2 compares the two operation counts on a logarithmic vertical axis, illustrating how quickly the factorial curve $n!$ outpaces the cubic curve $n^3/3$.

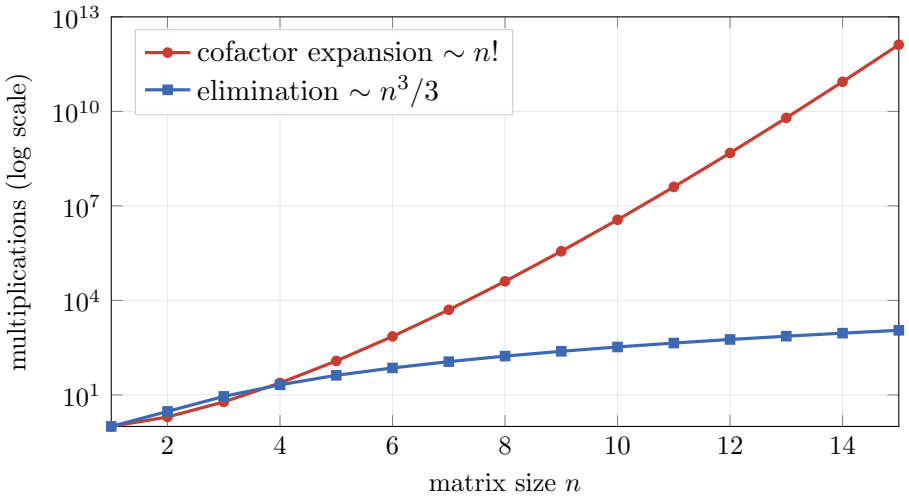


Figure 22.2: The computational cost of evaluating a determinant. On a logarithmic scale, the cofactor operation count $n!$ (red) grows rapidly, while Gaussian elimination's $n^3/3$ count (blue) remains manageable. By $n = 15$, cofactor expansion requires over a trillion multiplications, compared to approximately one thousand for elimination.

22.2 The Determinant of Special Matrices

Before deriving the inverse formula, we apply the cofactor expansion to several special families of matrices whose determinants can be evaluated immediately by inspection.

22.2.1 Triangular matrices: just multiply the diagonal

An **upper triangular** matrix has all zeros below its main diagonal, so each row only involves its own variable and later ones. A **lower triangular** matrix has all zeros above its main diagonal.

Theorem 22.2 (Determinant of a triangular matrix)**.** *If* $\mathbf{A}$ *is upper triangular or lower triangular, then*

$$\det \mathbf{A} = a_{11}\, a_{22} \cdots a_{nn},$$

the product of its diagonal entries.

Proof. Take $\mathbf{A}$ upper triangular, so every entry below the diagonal is zero. Expand along the *first column*. Only a_{11} is nonzero there, so

$$\det \mathbf{A} = a_{11}\, C_{11} = a_{11}\, M_{11},$$

and the minor M_{11} is the determinant of the $(n-1) \times (n-1)$ matrix you get by deleting the first row and column – which is *still upper triangular*, with diagonal $a_{22}, \ldots, a_{nn}$. Expanding it along *its* first column peels off a_{22}, and so on by induction until a single diagonal entry remains. Multiplying the peeled factors gives $a_{11}a_{22}\cdots a_{nn}$. The lower-triangular case is identical, expanding along the first row each time. □

Intuition

Geometrically a triangular matrix stretches space one axis at a time without ever mixing a later axis back into an earlier one. The volume scaling is then just the product of the per-axis stretches – exactly the diagonal entries. A zero on the diagonal means one axis got flattened, the volume is 0, and the matrix is singular.

Figure 22.3 shows this geometrically in the plane. The diagonal entries a_{11} and a_{22} determine the base length and the vertical height of the transformed box. The entry above the diagonal, a_{12}, acts as a pure horizontal shear: it slides the top edge sideways parallel to the base. By Cavalieri's principle (base $\times$ height), sliding the ceiling sideways changes the tilt of the walls but leaves the enclosed area untouched. The shear entry a_{12} has no say in the determinant.

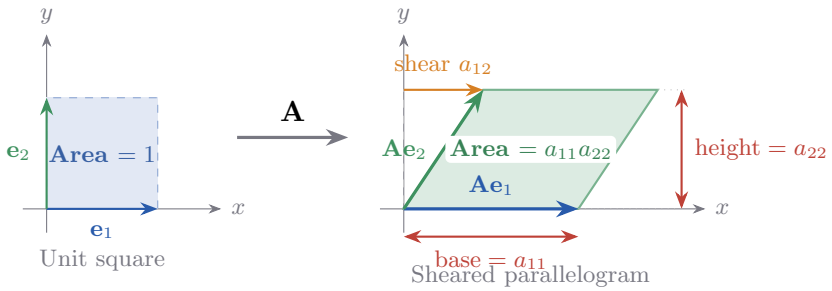


Figure 22.3: Why upper-triangular determinants ignore the entries above the diagonal. The diagonal entries a_{11} and a_{22} fix the base length and perpendicular height. The entry above the diagonal, a_{12}, acts as a horizontal shear, sliding the top edge sideways parallel to the base. By Cavalieri's principle, sliding parallel to the base preserves area: Area = base $\times$ height $= a_{11}a_{22}$, completely unaffected by a_{12}.

This is precisely *why* elimination computes determinants efficiently: row operations of the "subtract a multiple of one row from another" type do not change the determinant, and they drive any matrix to triangular form, where the answer is a single product down the diagonal. Cofactors explain *why* the triangular form is so easy; elimination delivers you *to* it cheaply.

22.2.2 A worked diagonal and triangular check

$$\det \begin{bmatrix} 3 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 5 \end{bmatrix} = 3 \cdot (-2) \cdot 5 = -30, \qquad \det \begin{bmatrix} 4 & 7 & -1 \\ 0 & 2 & 9 \\ 0 & 0 & 6 \end{bmatrix} = 4 \cdot 2 \cdot 6 = 48.$$

The upper-triangular determinant ignores the entries $7, -1, 9$ above the diagonal entirely. They contribute nothing: as established in Theorem 22.2, expanding down successive columns eliminates all above-diagonal contributions.

22.2.3 A few more determinants you can read off

- **Identity:** $\det \mathbf{I} = 1$. (Triangular with all 1's.) It scales no volume.

- **A row or column of zeros:** $\det = 0$. Expand along that row; every term has a zero factor. Geometrically the matrix flattens space.
- **Two equal rows (or columns):** $\det = 0$. The columns are dependent, so volume collapses.
- **Scaling the whole matrix:** $\det(c\mathbf{A}) = c^n \det \mathbf{A}$ for $n \times n$. Each of the n axes is stretched by c, so volume scales by c^n – not by c. This trips people up constantly.
- **Transpose:** $\det(\mathbf{A}^\mathsf{T}) = \det \mathbf{A}$. The transpose $\mathbf{A}^\mathsf{T}$ flips the matrix across its main diagonal, turning rows into columns. This equality is exactly why "any row or column" works in Theorem 22.1: the rows of $\mathbf{A}$ are the columns of $\mathbf{A}^\mathsf{T}$.

Common trap

$\det(c\mathbf{A}) = c^n \det \mathbf{A}$, not $c \det \mathbf{A}$. If you double every entry of a 3×3 matrix you multiply its determinant by $2^3 = 8$, because you doubled all three edge-lengths of the box. Likewise $\det(\mathbf{A} + \mathbf{B})$ is *not* $\det \mathbf{A} + \det \mathbf{B}$ in general – the determinant is nonlinear with respect to matrix addition, even though it is linear in each row *separately*.

Compute it yourself

Pause and think: If $\mathbf{A}$ is a 4×4 matrix with $\det \mathbf{A} = 5$, what is $\det(-\mathbf{A})$? What is $\det(2\mathbf{A})$? Before writing anything down, picture four orthogonal edges being scaled by -1 (four sign flips make $+1$, so $(-1)^4 \cdot 5 = 5$) or by 2 (each of the four dimensions doubles, scaling the 4-dimensional hypervolume by $2^4 = 16$, giving $16 \cdot 5 = 80$).

22.3 The Adjugate and the Inverse Formula

We now turn the cofactors into something structural. Collect every cofactor C_{ij} into a matrix, then *transpose* it. That transposed grid is the object this section is built around.

Definition 22.3 (Adjugate)**.** The **cofactor matrix** of $\mathbf{A}$ is the $n \times n$ matrix $\mathbf{C}$ whose (i, j) entry is the cofactor C_{ij}. The **adjugate** (or classical adjoint) is its transpose:

$$\operatorname{adj}(\mathbf{A}) = \mathbf{C}^\mathsf{T}, \qquad \text{so} \qquad \big(\operatorname{adj} \mathbf{A}\big)_{ij} = C_{ji}.$$

The name comes from the Latin *adjugare* ("to yoke together to assist"), because it is yoked to $\mathbf{A}$ to recover the identity matrix when multiplied. The transpose is easy to forget and it matters: the (i, j) entry of the adjugate is the cofactor C_{ji} with the indices *swapped*. Now the key identity.

Theorem 22.3 (Adjugate identity)**.** *For any square* $\mathbf{A}$*,*

$$\mathbf{A} \operatorname{adj}(\mathbf{A}) = \operatorname{adj}(\mathbf{A})\, \mathbf{A} = (\det \mathbf{A})\, \mathbf{I}.$$

Consequently, if $\det \mathbf{A} \neq 0$*,*

$$\boxed{\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \operatorname{adj}(\mathbf{A}).}$$

This formula provides a closed-form expression for the inverse of *any* invertible matrix without performing row reduction – expressing each entry as a ratio of sub-determinants over the determinant. The proof reveals the algebraic mechanism behind the formula.

22.3.1 Why the adjugate identity holds

Look at the (i, i) diagonal entry of the product $\mathbf{A}\,\mathrm{adj}(\mathbf{A})$. By the rule for matrix multiplication it is

$$\big(\mathbf{A}\ \mathrm{adj}\,\mathbf{A}\big)_{ii} = \sum_k a_{ik}\big(\mathrm{adj}\,\mathbf{A}\big)_{ki} = \sum_k a_{ik}\,C_{ik}.$$

But $\sum_k a_{ik}C_{ik}$ is exactly the cofactor expansion of $\det\mathbf{A}$ along row i (Theorem 22.1), so every diagonal entry equals $\det\mathbf{A}$. That handles the diagonal.

Now the off-diagonal entry (i, j) with $i \neq j$:

$$\big(\mathbf{A}\ \mathrm{adj}\,\mathbf{A}\big)_{ij} = \sum_k a_{ik}\big(\mathrm{adj}\,\mathbf{A}\big)_{kj} = \sum_k a_{ik}\,C_{jk}.$$

This sum pairs the entries of row i with the cofactors of row j. Here is the clever observation: $\sum_k a_{ik}C_{jk}$ is the cofactor expansion (along row j) of a *modified* matrix – the matrix $\mathbf{A}$ with row j *replaced by a copy of row i*. Why? Because the cofactors C_{jk} of row j depend only on the *other* rows; they never see row j itself. So feeding them the entries of row i computes the determinant of a matrix that has row i sitting in both position i and position j. That matrix has two identical rows, so its determinant is 0.

Therefore the product is $\det\mathbf{A}$ on the diagonal and 0 everywhere else – precisely $(\det\mathbf{A})\mathbf{I}$. Dividing by $\det\mathbf{A}$ (when it is nonzero) gives the inverse formula.

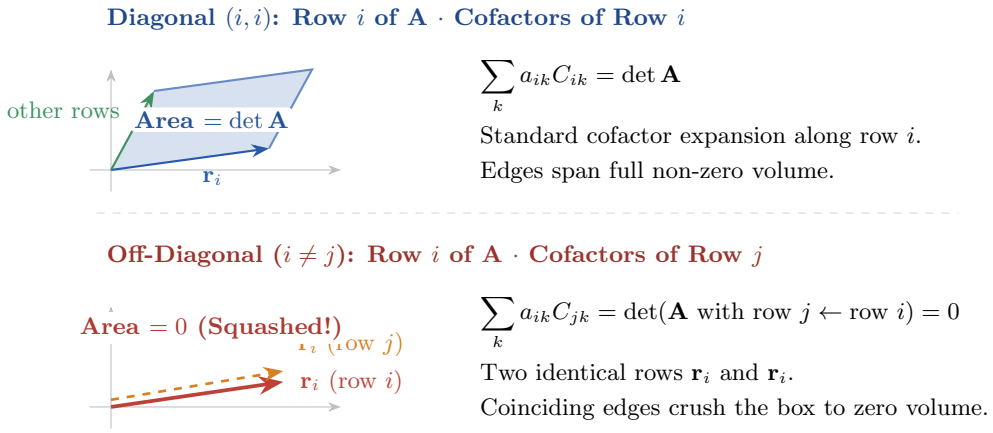


Figure 22.4: Why the adjugate identity $\mathbf{A}\,\mathrm{adj}(\mathbf{A}) = (\det\mathbf{A})\,\mathbf{I}$ holds. On the diagonal (top), dotting row i with its own cofactors computes the full volume $\det\mathbf{A}$. Off the diagonal (bottom), dotting row i with the cofactors of row j computes the volume of a box where edge j is replaced by edge i. Because two edges coincide, the box has zero width and collapses to volume 0.

Why this matters

The off-diagonal entries vanish because of a deliberate geometric cancellation: pairing one row's entries with another row's cofactors computes the determinant of a box with two identical edges, which has zero volume. That single observation – duplicate rows mean zero volume – makes the inverse formula's off-diagonal entries disappear.

22.3.2 The 2×2 inverse, finally explained

You have surely memorized the 2×2 inverse: "swap the diagonal, negate the off-diagonal, divide by the determinant." The adjugate formula is *where that recipe comes from.* For

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

the four cofactors are single entries with signs: $C_{11} = +d$, $C_{12} = -c$, $C_{21} = -b$, $C_{22} = +a$. The cofactor matrix is $\begin{bmatrix} d & -c \\ -b & a \end{bmatrix}$, and the adjugate is its transpose:

$$\operatorname{adj} \mathbf{A} = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}, \qquad \mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

There it is. The mysterious swap-and-negate is the cofactor pattern $+, -, -, +$ followed by a transpose.

Compute it yourself

Verify the adjugate identity by hand on $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$. Here $\det \mathbf{A} = 1$. Form $\operatorname{adj} \mathbf{A} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$ and multiply: $\mathbf{A} \operatorname{adj} \mathbf{A} = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix} \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = (\det \mathbf{A})\mathbf{I}$. Because $\det \mathbf{A} = 1$, the adjugate *is* the inverse here.

22.3.3 A full 3×3 inverse by cofactors

Let us do one 3×3 inverse from scratch, to see both the elegance and the labor. Take

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}.$$

First the determinant; expand along column 1 (it has a zero):

$$\det \mathbf{A} = 1 \det \begin{bmatrix} 1 & 4 \\ 6 & 0 \end{bmatrix} - 0 + 5 \det \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} = 1(0 - 24) + 5(8 - 3) = -24 + 25 = 1.$$

A determinant of 1 – convenient, because the inverse will equal the adjugate exactly. Now compute all nine cofactors. Remember the sign checkerboard.

$$C_{11} = +\det \begin{bmatrix} 1 & 4 \\ 6 & 0 \end{bmatrix} = 0 - 24 = -24, \quad C_{12} = -\det \begin{bmatrix} 0 & 4 \\ 5 & 0 \end{bmatrix} = -(0 - 20) = 20,$$

$$C_{13} = +\det \begin{bmatrix} 0 & 1 \\ 5 & 6 \end{bmatrix} = 0 - 5 = -5, \quad C_{21} = -\det \begin{bmatrix} 2 & 3 \\ 6 & 0 \end{bmatrix} = -(0 - 18) = 18,$$

$$C_{22} = +\det \begin{bmatrix} 1 & 3 \\ 5 & 0 \end{bmatrix} = 0 - 15 = -15, \quad C_{23} = -\det \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} = -(6 - 10) = 4,$$

$$C_{31} = +\det \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} = 8 - 3 = 5, \quad C_{32} = -\det \begin{bmatrix} 1 & 3 \\ 0 & 4 \end{bmatrix} = -(4 - 0) = -4,$$

$$C_{33} = +\det \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = 1 - 0 = 1.$$

The cofactor matrix and its transpose (the adjugate):

$$\mathbf{C} = \begin{bmatrix} -24 & 20 & -5 \\ 18 & -15 & 4 \\ 5 & -4 & 1 \end{bmatrix}, \qquad \operatorname{adj}\mathbf{A} = \mathbf{C}^{\mathsf{T}} = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}.$$

Since $\det\mathbf{A} = 1$,

$$\mathbf{A}^{-1} = \operatorname{adj}\mathbf{A} = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}.$$

We can verify $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ directly. Notice the computational effort: nine 2×2 determinants plus the determinant of $\mathbf{A}$, all for a single inverse. Elimination produces the same inverse with substantially fewer operations.

Before/after Strang

Strang is blunt about this formula, and rightly so: he calls the cofactor inverse a formula to *know* but rarely to *use*. The reason to learn it is that it gives the inverse *symbolically* – every entry of $\mathbf{A}^{-1}$ is a polynomial in the entries of $\mathbf{A}$ divided by $\det\mathbf{A}$. That symbolic form is what lets you *prove* general facts: that the inverse entries are continuous in $\mathbf{A}$, that the inverse of an integer matrix with $\det = \pm 1$ is again an integer matrix, and more. Numbers come from elimination; *theorems* often come from the adjugate.

Connection to other math

The adjugate gives a clean window into *when integer matrices have integer inverses*, a question at the heart of the group $\mathrm{GL}_n(\mathbb{Z})$ and of lattices and cryptography. Because every entry of $\operatorname{adj}\mathbf{A}$ is a sum of products of integer entries, it is an integer matrix whenever $\mathbf{A}$ is. So $\mathbf{A}^{-1} = \frac{1}{\det\mathbf{A}}\operatorname{adj}\mathbf{A}$ is an integer matrix exactly when $\det\mathbf{A}$ divides every adjugate entry – and in particular always when $\det\mathbf{A} = \pm 1$. Matrices with $\det = \pm 1$ are called *unimodular*; they are precisely the invertible integer matrices whose inverses are also integer matrices, and they form a group. This is why $\det = \pm 1$ shows up everywhere from lattice-based cryptography to the change-of-basis matrices that preserve a grid of points.

22.4 Cramer's Rule

Now we harvest the formula we have been building toward. Cramer's rule solves a square linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ by writing *each unknown* as a ratio of two determinants. No back substitution, no augmented matrix – just determinants.

Theorem 22.4 (Cramer's rule)**.** *Let* $\mathbf{A}$ *be* $n\times n$ *with* $\det\mathbf{A} \neq 0$*, and consider* $\mathbf{A}\mathbf{x} = \mathbf{b}$*. Let* $\mathbf{A}_i(\mathbf{b})$ *denote the matrix* $\mathbf{A}$ *with its* i*-th column replaced by* $\mathbf{b}$*. Then the unique solution has entries*

$$x_i = \frac{\det\mathbf{A}_i(\mathbf{b})}{\det\mathbf{A}}, \qquad i = 1, \ldots, n.$$

The single new idea in the statement is the matrix $\mathbf{A}_i(\mathbf{b})$: take $\mathbf{A}$, throw away its i-th column, and slot $\mathbf{b}$ into the gap. Every other column stays put. Figure 22.5 shows the surgery for a 3×3 matrix, where the second column is the one replaced.

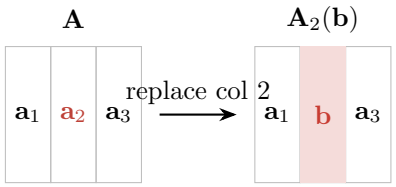


Figure 22.5: The matrix $\mathbf{A}_i(\mathbf{b})$ is $\mathbf{A}$ with its i-th column overwritten by $\mathbf{b}$ (here $i = 2$). Cramer's rule says x_i is the ratio $\det \mathbf{A}_i(\mathbf{b}) / \det \mathbf{A}$ of this matrix's determinant to the original's.

22.4.1 Where it comes from

The cleanest derivation rides on the adjugate. From $\mathbf{Ax} = \mathbf{b}$ and $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \operatorname{adj} \mathbf{A}$ we get

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{\det \mathbf{A}} \operatorname{adj}(\mathbf{A})\, \mathbf{b}.$$

The i-th entry of $\operatorname{adj}(\mathbf{A})\, \mathbf{b}$ is $\sum_k \left(\operatorname{adj} \mathbf{A}\right)_{ik} b_k = \sum_k C_{ki}\, b_k$. But $\sum_k b_k C_{ki}$ is exactly the cofactor expansion, down column i, of the matrix whose i-th column is $\mathbf{b}$ and whose other columns are those of $\mathbf{A}$ – that is, $\det \mathbf{A}_i(\mathbf{b})$. Hence $x_i = \det \mathbf{A}_i(\mathbf{b}) / \det \mathbf{A}$. Cramer's rule is the adjugate inverse, read one coordinate at a time.

Compute it yourself

Pause and think: Look at the formula $x_i = \det \mathbf{A}_i(\mathbf{b}) / \det \mathbf{A}$. What happens if $\mathbf{b}$ is already the i-th column of $\mathbf{A}$? Then $\mathbf{A}_i(\mathbf{b})$ is simply $\mathbf{A}$, so $x_i = \det \mathbf{A} / \det \mathbf{A} = 1$. And what happens to any other unknown x_j ($j \neq i$)? In $\mathbf{A}_j(\mathbf{b})$, column j has been replaced by $\mathbf{b}$ (which is column i), so that matrix has two identical columns and its determinant is zero. Thus $x_j = 0$. In one line, Cramer's rule confirms that the i-th column vector has coordinates $(0, \ldots, 1, \ldots, 0)$.

22.4.2 A worked 2×2 system

Solve

$$\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

First $\det \mathbf{A} = 2 \cdot 3 - 1 \cdot 1 = 5$. For x_1, replace column 1 by $\mathbf{b} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$:

$$\det \mathbf{A}_1(\mathbf{b}) = \det \begin{bmatrix} 5 & 1 \\ 6 & 3 \end{bmatrix} = 15 - 6 = 9, \qquad x_1 = \frac{9}{5}.$$

For x_2, replace column 2 by $\mathbf{b}$:

$$\det \mathbf{A}_2(\mathbf{b}) = \det \begin{bmatrix} 2 & 5 \\ 1 & 6 \end{bmatrix} = 12 - 5 = 7, \qquad x_2 = \frac{7}{5}.$$

Check: $2 \cdot \frac{9}{5} + \frac{7}{5} = \frac{18+7}{5} = 5$ and $\frac{9}{5} + 3 \cdot \frac{7}{5} = \frac{9+21}{5} = 6$. Both equations hold.

22.4.3 A worked 3×3 system

Solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ -1 & 3 & 1 \\ 2 & 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 5 \end{bmatrix}.$$

Determinant (expand along row 1):

$$\det \mathbf{A} = 1 \det \begin{bmatrix} 3 & 1 \\ 1 & 1 \end{bmatrix} - 0 + 2 \det \begin{bmatrix} -1 & 3 \\ 2 & 1 \end{bmatrix} = 1(3-1) + 2(-1-6) = 2 - 14 = -12.$$

Now three modified determinants. Replace column 1 by $\mathbf{b}$:

$$\det \mathbf{A}_1 = \det \begin{bmatrix} 1 & 0 & 2 \\ 2 & 3 & 1 \\ 5 & 1 & 1 \end{bmatrix} = 1(3-1) - 0 + 2(2-15) = 2 - 26 = -24, \quad x_1 = \frac{-24}{-12} = 2.$$

Replace column 2 by $\mathbf{b}$:

$$\det \mathbf{A}_2 = \det \begin{bmatrix} 1 & 1 & 2 \\ -1 & 2 & 1 \\ 2 & 5 & 1 \end{bmatrix} = 1(2-5) - 1(-1-2) + 2(-5-4) = -3 + 3 - 18 = -18, \quad x_2 = \frac{-18}{-12} = \tfrac{3}{2}.$$

Replace column 3 by $\mathbf{b}$:

$$\det \mathbf{A}_3 = \det \begin{bmatrix} 1 & 0 & 1 \\ -1 & 3 & 2 \\ 2 & 1 & 5 \end{bmatrix} = 1(15-2) - 0 + 1(-1-6) = 13 - 7 = 6, \quad x_3 = \frac{6}{-12} = -\tfrac{1}{2}.$$

So $\mathbf{x} = \left(2, \tfrac{3}{2}, -\tfrac{1}{2}\right)$. Quick check on the first equation: $1 \cdot 2 + 0 \cdot \tfrac{3}{2} + 2 \cdot (-\tfrac{1}{2}) = 2 - 1 = 1 = b_1$. Good.

Common trap

Cramer's rule needs $\det \mathbf{A} \neq 0$. If $\det \mathbf{A} = 0$ the formula divides by zero and is silent – it cannot distinguish "no solution" from "infinitely many." That is a real limitation: when a system is singular, the determinant ratios collapse and you must fall back on elimination to learn which case you are in. Cramer answers only the square, unique-solution question.

22.5 The Geometry of Cramer's Rule: Each Unknown Is a Ratio of Volumes

Formulas are easier to trust when you can see them. Cramer's rule has a direct geometric reading, and it is the reason this rule belongs in a geometry-first book at all.

Recall the picture of $\mathbf{A}\mathbf{x} = \mathbf{b}$ as a statement about columns. Writing $\mathbf{A} = [\,\mathbf{a}_1 \;\; \mathbf{a}_2 \;\; \cdots \;\; \mathbf{a}_n\,]$, the equation says

$$x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n = \mathbf{b}.$$

The unknowns x_i are the *coordinates of* $\mathbf{b}$ *in the basis of columns*. Cramer's rule recovers each coordinate as a volume ratio. Let us see exactly why in $\mathbb{R}^2$, where "volume" means area.

Take columns $\mathbf{a}_1, \mathbf{a}_2$ in the plane. The determinant $\det \mathbf{A}$ is the signed area of the parallelogram they span. Now $\mathbf{b} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2$. Consider the parallelogram built from $\mathbf{b}$ and $\mathbf{a}_2$ – its area is $\det[\,\mathbf{b}\ \ \mathbf{a}_2\,] = \det \mathbf{A}_1(\mathbf{b})$. Replace $\mathbf{b}$ by its expansion:

$$\det[\,\mathbf{b}\ \ \mathbf{a}_2\,] = \det[\,x_1\mathbf{a}_1 + x_2\mathbf{a}_2\ \ \mathbf{a}_2\,] = x_1 \det[\,\mathbf{a}_1\ \ \mathbf{a}_2\,] + x_2 \det[\,\mathbf{a}_2\ \ \mathbf{a}_2\,].$$

The second determinant is zero (two equal columns). So $\det[\,\mathbf{b}\ \ \mathbf{a}_2\,] = x_1 \det \mathbf{A}$, giving $x_1 = \det \mathbf{A}_1(\mathbf{b}) / \det \mathbf{A}$ – Cramer's rule, derived from pure area.

Intuition

Here is the one-sentence geometry. x_i **is the factor by which you must stretch the i-th column so that, together with the *other* columns, it spans the same volume as $\mathbf{b}$ does with those columns.** Replacing column i by $\mathbf{b}$ and taking the area ratio peels off exactly that stretch factor, because the other columns are shared and the determinant is linear in the column you changed.

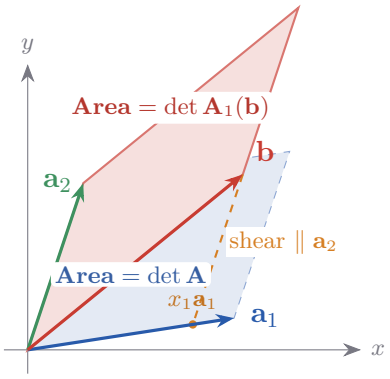


Figure 22.6: Cramer's rule in the plane. The blue parallelogram on $\mathbf{a}_1, \mathbf{a}_2$ has signed area $\det \mathbf{A}$. The red parallelogram on $\mathbf{b}, \mathbf{a}_2$ has area $\det \mathbf{A}_1(\mathbf{b})$. Because $\mathbf{b} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2$, replacing $\mathbf{a}_1$ with $\mathbf{b}$ scales the base by x_1 and then shears parallel to $\mathbf{a}_2$ (orange dashed line). By Cavalieri's principle, shearing parallel to $\mathbf{a}_2$ contributes zero extra area against $\mathbf{a}_2$, so the red area is exactly x_1 times the blue area: $x_1 = \det \mathbf{A}_1(\mathbf{b}) / \det \mathbf{A}$.

Walking question

Consider why the component of $\mathbf{b}$ along $\mathbf{a}_2$ contributes nothing to the area $\det[\,\mathbf{b}\ \mathbf{a}_2\,]$. A parallelogram whose edges are collinear collapses to zero area. Consequently, the determinant $\det[\,\mathbf{b}\ \mathbf{a}_2\,]$ isolates the component of $\mathbf{b}$ in the direction of $\mathbf{a}_1$, which is precisely $x_1\mathbf{a}_1$. This geometric cancellation explains why Cramer's rule computes each coordinate individually.

Why this matters

The fundamental reason Cramer's rule exists is that the determinant is *multilinear* – linear in each column separately – and *alternating* – zero when two columns coincide. Those two properties alone force the volume ratio to extract a single coordinate. You will meet the same pair of properties in exterior algebra and differential forms: an alternating multilinear form naturally measures oriented volume.

22.6 When These Formulas Help, and When Elimination Wins

Here is the verdict, laid out plainly.

Elimination wins for numbers. To solve $\mathbf{Ax} = \mathbf{b}$ for a specific numeric $\mathbf{A}$ and $\mathbf{b}$, Gaussian elimination costs about $n^3/3$ operations and solves all n unknowns at once. Cramer's rule asks you to compute $n + 1$ separate determinants; even computing each determinant *by elimination* costs $\sim n^3/3$, so Cramer is roughly $n + 1$ times more expensive – and if you compute those determinants by cofactor expansion the cost explodes to $\sim n \cdot n!$. For anything beyond about 3×3, Cramer and cofactors are computationally hopeless. They are also numerically fragile: dividing by a tiny determinant amplifies round-off error badly.

Cofactors and Cramer win for structure, symbols, and small sizes. Reach for them when:

- The matrix is 2×2 **or** 3×3 and you want a quick closed form – the 2×2 inverse recipe is the adjugate, and you should keep it at your fingertips.
- You need a **single unknown**, not the whole solution. Cramer gives x_3 alone without ever finding x_1, x_2 – occasionally handy in a derivation or an exam.
- You are working **symbolically** – the entries are letters or parameters – and you want to see *how* the solution depends on them. The explicit polynomial-over-determinant form is exactly right for sensitivity analysis and for proofs.
- You are **proving a theorem**. The adjugate identity, the continuity of $\mathbf{A}^{-1}$ in its entries, the integrality results in the connection box – all rest on these formulas, not on elimination.

Focus

Rule of thumb: **to get a number, eliminate; to get a formula or a proof, use cofactors.** Cramer's rule and the adjugate are the determinant's *theoretical* face. They tell you what the answer *is* as an expression, which is a different and sometimes more valuable thing than telling you what it equals as a decimal.

Modern application

Where the adjugate quietly shows up in modern work. In computer graphics and physics engines, surface *normals* must be transformed correctly when an object is deformed by a matrix $\mathbf{M}$. A normal does not transform by $\mathbf{M}$ – it transforms by the *inverse transpose* $(\mathbf{M}^{-1})^{\mathsf{T}}$, and for the common case where you only care about direction, this is proportional to the *transpose of the adjugate*, $\operatorname{adj}(\mathbf{M})^{\mathsf{T}}$. Because the adjugate avoids the $1/\det$ division, graphics code often uses it directly: it stays finite and correct even when $\det \mathbf{M}$ is small, and you only normalize the resulting direction at the end. Meanwhile in statistics, Cramer's rule is the cleanest way to *see* how a single regression coefficient depends on the data: each coefficient in the normal equations is one determinant ratio, making the effect of adding or perturbing a variable transparent. In actual software, those coefficients are computed by the far more stable **QR** or Cholesky methods rather than by evaluating Cramer's rule on a large design matrix. The formula is the explanation; elimination is the calculator.

Connection to other math

Cramer's rule and the implicit function theorem. In multivariable calculus you ask: if several quantities are tied together by equations, how does one of them change when you nudge an input? Linearizing the equations turns that question into a linear system whose matrix is the *Jacobian* – the matrix that collects all first-order partial derivatives, acting as the local best linear approximation. The implicit function theorem says you can solve for the derivative exactly when the Jacobian determinant is nonzero – and the derivative you get is, term by term, a Cramer's-rule ratio of determinants. So the condition "$\det \neq 0$" that lets Cramer run is the very same condition that lets calculus invert a map locally. The two subjects are telling the same story: a nonzero determinant means the linear approximation is invertible, so the answer is a clean ratio of volumes.

History and connection

In 1750, the Swiss mathematician Gabriel Cramer published an influential treatise on algebraic curves with a very concrete geometric puzzle in mind: finding the equation of a plane curve passing through a specified set of points. Fitting a conic section—an ellipse, parabola, or hyperbola—through five points required solving five simultaneous linear equations in five unknowns. For cubic curves, nine equations were needed. Cramer ran straight into a famous conundrum known as Cramer's paradox: two cubic curves can intersect in nine points, yet nine points were supposed to uniquely pin down a single cubic. To understand how systems of equations could become redundant or singular, Cramer sought a general algebraic formula for the unknowns.

At the time, mathematicians had no matrix notation, no vectors, no systematic elimination tables, and no subscript indices (a_{ij}). They wrote systems out with alphabetic letters ($a, b, c, d, \dots$) and untangled them through pages of exhausting substitution. In an appendix to his 1750 book, Cramer introduced an ingenious index notation that allowed him to state his celebrated rule: each unknown in an $n \times n$ system is the ratio of two determinants.

Yet this history holds a famous priority controversy. Two years earlier, in 1748, the Scottish mathematician Colin Maclaurin published the exact same determinant formulas for 2×2 and 3×3 systems in his posthumous *Treatise of Algebra* (having discovered them as early as 1729). Maclaurin published first, but Cramer's superior notation for arbitrary n captured Europe's attention, and history permanently attached Cramer's name to the rule. Two decades later, Pierre-Simon Laplace (1772) showed how to peel any determinant apart by expanding it recursively along any row or column. In 1812, Augustin-Louis Cauchy synthesized the theory, established the word "determinant" in its modern sense, and proved the adjugate identity.

A common misconception is that Cramer's rule serves as a practical numerical method for solving systems on a computer. In practice, computing Cramer's rule via cofactor expansion requires an impractical number of operations. Solving an $n \times n$ system requires $n + 1$ determinants. Peeling each into sub-determinants takes $\sim n!$ steps, demanding $(n + 1)!$ arithmetic operations in total. For a modest 100×100 system, $(101)! \approx 9.3 \times 10^{157}$ operations—vastly more than the number of seconds since the Big Bang ($\approx 4.35 \times 10^{17}$). Meanwhile, Gaussian elimination solves that same system in less than a millisecond.

While Cramer's rule is impractical for large-scale computation, it provides an exact symbolic formula for matrix inverses and solutions, showing that solutions depend smoothly and differentiably on matrix entries—a property used in the implicit function theorem in multivariable calculus and sensitivity analysis in economics.

22.7 Looking Back

We took the single number from the previous chapter – the determinant as signed volume – and learned to take it apart and put it back together.

- **Cofactor expansion** (Theorem 22.1) computes a determinant along any row or column as a signed sum of smaller determinants. Expand where the zeros are. It is recursive, exact, and – for large matrices – factorially slow.
- **Special matrices** fall out instantly: a triangular determinant is the product of its diagonal (Theorem 22.2), which is exactly why elimination computes determinants cheaply. And $\det(c\mathbf{A}) = c^n \det \mathbf{A}$, not $c \det \mathbf{A}$.
- **The adjugate** (Theorem 22.3) packages the cofactors into an explicit inverse $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \operatorname{adj} \mathbf{A}$. Its off-diagonal entries vanish because pairing one row's entries with another row's cofactors evaluates a determinant with a repeated row.
- **Cramer's rule** (Theorem 22.4) reads each unknown $x_i = \det \mathbf{A}_i(\mathbf{b}) / \det \mathbf{A}$ as a ratio of determinants, and geometrically as a ratio of volumes (Figure 22.6) – the determinant's multilinear, alternating nature extracting one coordinate at a time.
- **The verdict:** these are tools of *insight*, not of *computation*. To get a number, eliminate. To get a formula or a proof, reach for cofactors.

In Chapter 23 we apply the determinant to characteristic equations. The equation $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$ defines the *characteristic polynomial*, whose roots are the eigenvalues of $\mathbf{A}$. Cofactor expansion is how you write that polynomial down by hand for a 2×2 or 3×3 matrix, so the mechanics practiced here carry directly forward.

22.8 Exercises

Exercise 22.1 (Mental check: determinant rules)**.** A 4×4 matrix $\mathbf{A}$ has $\det \mathbf{A} = 3$. Without writing on paper, determine:

(a) $\det(2\mathbf{A})$.

(b) $\det(-\mathbf{A})$.

(c) The determinant of the matrix obtained by swapping rows 1 and 3, and then multiplying row 2 by -5.

(d) $\det(\mathbf{A}^{-1})$.

Exercise 22.2 (Mental check: cofactors and duplicated rows)**.** Suppose $\mathbf{A}$ is a 5×5 matrix whose second and fourth rows are identical.

(a) What is $\det \mathbf{A}$?

(b) Without doing any arithmetic, what is the matrix product $\mathbf{A} \operatorname{adj}(\mathbf{A})$?

(c) Look at entry $(2, 4)$ of $\mathbf{A} \operatorname{adj}(\mathbf{A})$. What combination of entries and cofactors forms this entry, and why does it vanish?

Exercise 22.3. Compute $\det \begin{bmatrix} 3 & 1 & 0 \\ 2 & -1 & 4 \\ 0 & 5 & 2 \end{bmatrix}$ twice: once by expanding along the first row, once by expanding along the first column. Confirm you get the same number, and say which expansion required less arithmetic and why.

Exercise 22.4. Find a row or column with the most zeros and use it to evaluate $\det \begin{bmatrix} 4 & 0 & 0 & 1 \\ 7 & 2 & 0 & 0 \\ 1 & 3 & -1 & 0 \\ 0 & 6 & 0 & 2 \end{bmatrix}$. (Hint: the third column has three zeros.)

Exercise 22.5. Without expanding, write down $\det \begin{bmatrix} 5 & 99 & -3 & 7 \\ 0 & 2 & 11 & 8 \\ 0 & 0 & -1 & 4 \\ 0 & 0 & 0 & 3 \end{bmatrix}$. Then explain in one sentence why all the entries above the diagonal were irrelevant.

Exercise 22.6. Let $\mathbf{A}$ be a 3×3 matrix with $\det \mathbf{A} = 4$. Compute $\det(2\mathbf{A})$, $\det(\mathbf{A}^{\mathsf{T}})$, $\det(\mathbf{A}^{-1})$, and $\det(\mathbf{A}^2)$. State the general rule you used in each case.

Exercise 22.7. For $\mathbf{A} = \begin{bmatrix} 4 & 3 \\ 6 & 5 \end{bmatrix}$, write down the cofactor matrix, the adjugate, and $\mathbf{A}^{-1}$. Verify $\mathbf{A}\,\operatorname{adj}\mathbf{A} = (\det \mathbf{A})\mathbf{I}$ by direct multiplication.

Exercise 22.8. Use Cramer's rule to solve $\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 7 \\ 9 \end{bmatrix}$. Then check your answer by substitution.

Exercise 22.9. For the system $\begin{bmatrix} 1 & 2 & -1 \\ 0 & 3 & 1 \\ 2 & 0 & 1 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 3 \\ 5 \\ 4 \end{bmatrix}$, use Cramer's rule to find *only* x_2, without computing x_1 or x_3. (This illustrates the practical advantage of Cramer's rule: isolating a single unknown without solving the entire system.)

Exercise 22.10. Compute $\mathbf{A}^{-1}$ for $\mathbf{A} = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 3 \end{bmatrix}$ by the adjugate method: find $\det \mathbf{A}$, all nine cofactors, the adjugate, and divide. Then verify one column of $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$.

Exercise 22.11. A computer does 10^9 multiplications per second. Estimate, in seconds, how long a cofactor expansion ($\sim n!$ multiplications) would take for $n = 15$ versus how long elimination ($\sim n^3/3$) would take. Use your estimates to argue, in a sentence or two, why no serious software computes determinants by cofactors.

Exercise 22.12. In $\mathbb{R}^2$ let $\mathbf{a}_1 = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, and $\mathbf{b} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$. Compute x_1, x_2 by Cramer's rule, then verify geometrically: confirm that the area of the parallelogram on $\mathbf{b}, \mathbf{a}_2$ equals x_1 times the area of the parallelogram on $\mathbf{a}_1, \mathbf{a}_2$. Sketch both parallelograms.

Exercise 22.13. (Proof.) Use the adjugate identity $\mathbf{A}\,\operatorname{adj}\mathbf{A} = (\det \mathbf{A})\mathbf{I}$ to prove that if $\det \mathbf{A} = 0$ then $\mathbf{A}\,\operatorname{adj}\mathbf{A} = \mathbf{0}$. Then argue that if in addition $\operatorname{adj}\mathbf{A} \neq \mathbf{0}$, the matrix $\mathbf{A}$ must be singular with a nontrivial null space, and that the nonzero columns of $\operatorname{adj}\mathbf{A}$ lie in that null space.

Exercise 22.14. (Challenge.) For an invertible $n \times n$ matrix $\mathbf{A}$, take determinants of both sides of $\mathbf{A}\,\operatorname{adj}\mathbf{A} = (\det \mathbf{A})\mathbf{I}$ to show that $\det\big(\operatorname{adj}\mathbf{A}\big) = (\det \mathbf{A})^{n-1}$. Check your formula on a 2×2 and a 3×3 example.

Part VI

Eigenvalues

Chapter 23

Eigenvalues and Eigenvectors

In the previous chapters, we dissected determinants—learning how cofactors and Cramer's rule unpack whether a matrix collapses volume down to zero. The determinant gave us a single number measuring how a matrix scales space. But it left an essential question unanswered: what does the matrix do to individual directions?

A matrix acts as a linear transformation: given an input vector, it produces an output vector. Typically, the output points in a new direction: the matrix may rotate, stretch, shear, or reflect the vector. In standard coordinates, x and y get mixed together, making it hard to see the geometry of the map at a glance.

For most matrices, however, there exist special directions that the transformation preserves without rotating. Along those directions, the matrix acts by pure scalar multiplication: an input vector along one of these directions maps to a scalar multiple of itself (pointing in the same or opposite direction). Those directions are the *eigenvectors*, and the scaling factor is the *eigenvalue*. Identifying them simplifies a complicated transformation into independent scalar scalings along dedicated axes. This decomposition is central to analyzing structural vibrations, Google's PageRank algorithm, the stability of differential equations, principal component analysis in data science, and rotations in computer graphics.

23.1 Directions a matrix does not turn

Start with a picture and a single concrete matrix. Take

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

This is a perfectly ordinary symmetric matrix. Watch what it does to a handful of input vectors. Apply it to $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and you get $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ – the output has swung upward, it points in a new direction. Apply it to $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ and you get $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ – swung the other way. For most inputs, $\mathbf{A}$ turns the arrow.

But now try the diagonal direction $\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$:

$$\mathbf{A}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2+1 \\ 1+2 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3\begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The output $\begin{bmatrix} 3 \\ 3 \end{bmatrix}$ points in *exactly* the same direction as the input $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. The matrix did not turn this arrow at all; it simply tripled its length. And try the anti-diagonal $\mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$:

$$\mathbf{A}\begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2-1 \\ 1-2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Same story – the direction is preserved, this time with no stretch at all (the length is unchanged because the multiplier is 1).

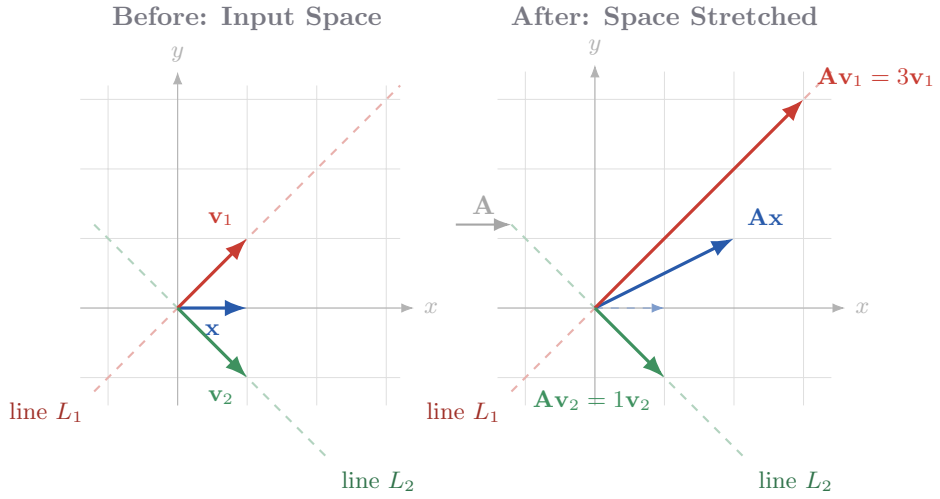


Figure 23.1: The matrix $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ turns generic vectors off their span (blue $\mathbf{x}$ turns from $(1, 0)$ to $(2, 1)$). However, vectors along the diagonal lines L_1 and L_2 stay strictly on their own lines: $\mathbf{v}_1$ is simply stretched by factor $\lambda_1 = 3$, while $\mathbf{v}_2$ is preserved with $\lambda_2 = 1$. Those invariant lines are the eigenvectors.

Intuition

An eigenvector specifies a line through the origin that is mapped into itself by the matrix. The matrix scales vectors along that line without rotating them off it. The eigenvalue is the scalar factor: $\lambda = 3$ triples length, $\lambda = 1$ leaves vectors unchanged, $\lambda = 0.5$ halves length, and $\lambda = -2$ doubles length while reversing direction.

The word *eigen* is German for "own" or "characteristic" or "proper." These are the matrix's own directions, the ones intrinsic to it. Here is the precise statement.

Definition 23.1 (Eigenvalue and eigenvector)**.** Let $\mathbf{A}$ be an $n \times n$ matrix. A nonzero vector $\mathbf{x} \in \mathbb{R}^n$ is an *eigenvector* of $\mathbf{A}$ if there is a scalar λ with

$$\boxed{\mathbf{A}\mathbf{x} = \lambda\,\mathbf{x}.}$$

The scalar λ is the *eigenvalue* belonging to $\mathbf{x}$. (We always require $\mathbf{x} \neq \mathbf{0}$. For now, our vectors live in real space $\mathbb{R}^n$. Occasionally, a real matrix forces λ to be a complex number, as when it rotates the plane; we explore what that means geometrically in Section 23.7, while the formal machinery of complex vector spaces $\mathbb{C}^n$ waits for Chapter 32 and the Chapter 39 appendix.)

Two small but essential remarks live inside that definition.

Why the vector must be nonzero. If we allowed $\mathbf{x} = \mathbf{0}$, the equation $\mathbf{A0} = \lambda\mathbf{0}$ would be true for *every* λ, which would make the eigenvalue meaningless. The zero vector has no direction, so it cannot tell us a direction the matrix preserves. We throw it out.

Why the eigenvalue *can* be zero. A perfectly good eigenvalue is $\lambda = 0$. That says $\mathbf{Ax} = \mathbf{0}$ for a nonzero $\mathbf{x}$, i.e. $\mathbf{x}$ is in the nullspace (the subspace of all vectors that $\mathbf{A}$ squashes to $\mathbf{0}$). So "0 is an eigenvalue of $\mathbf{A}$" is just a new way of saying "$\mathbf{A}$ is singular." Hold onto that; it will return.

Focus

$\mathbf{Ax} = \lambda\mathbf{x}$ is the most important equation in this half of the book. Read it as: *applying the matrix has the same effect as multiplying by a single number*, but only along the special directions $\mathbf{x}$. The number is λ; the direction is $\mathbf{x}$.

Remark 23.1 (An eigenvector is really a whole line). If $\mathbf{Ax} = \lambda\mathbf{x}$, then for any scalar $c \neq 0$ the vector $c\mathbf{x}$ is an eigenvector too, with the *same* eigenvalue: $\mathbf{A}(c\mathbf{x}) = c\,\mathbf{Ax} = c\lambda\mathbf{x} = \lambda(c\mathbf{x})$. So eigenvectors come in lines (and, as we will see, sometimes whole planes). When we "find the eigenvector" we really mean "find the line"; we usually report one convenient representative, like $\begin{bmatrix}1\\1\end{bmatrix}$ rather than $\begin{bmatrix}5\\5\end{bmatrix}$.

Pause

Pause and think: Suppose $\mathbf{v}$ is an eigenvector of a matrix $\mathbf{A}$ with eigenvalue $\lambda = 4$. What vector does $\mathbf{A}$ produce when applied to $3\mathbf{v}$? What about $-\mathbf{v}$?

By linearity, $\mathbf{A}(3\mathbf{v}) = 3(\mathbf{Av}) = 3(4\mathbf{v}) = 4(3\mathbf{v})$, and $\mathbf{A}(-\mathbf{v}) = 4(-\mathbf{v})$. In both cases, $\mathbf{A}$ simply multiplies the vector by 4. The eigenvalue is a property of the *entire line* through $\mathbf{v}$, not just the specific vector $\mathbf{v}$ itself.

Walking question

Before reading on, picture three transformations of the plane and ask which directions survive untouched. (1) A pure scaling that doubles everything: which directions are eigenvectors? (2) A reflection across the x-axis: which arrows stay on their line, and what are their eigenvalues? (3) A 90° rotation: can *any* real arrow stay on its line? Carry these on your walk; we answer all three in this chapter.

23.2 Finding the eigenvalues: the characteristic equation

Geometry told us eigenvectors exist for the matrix $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ because we got lucky and guessed $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. We need a method that does not rely on luck. The key algebraic move is this.

Start from $\mathbf{Ax} = \lambda\mathbf{x}$ and drag everything to one side. The trick is to write $\lambda\mathbf{x} = \lambda\mathbf{Ix}$ so that both sides are a matrix times $\mathbf{x}$:

$$\mathbf{Ax} - \lambda\mathbf{x} = \mathbf{0} \quad \Longrightarrow \quad \mathbf{Ax} - \lambda\mathbf{Ix} = \mathbf{0} \quad \Longrightarrow \quad (\mathbf{A} - \lambda\mathbf{I})\,\mathbf{x} = \mathbf{0}.$$

Common trap

You cannot write $\mathbf{Ax} - \lambda\mathbf{x} = (\mathbf{A} - \lambda)\mathbf{x}$ and stop there. The symbol $\mathbf{A} - \lambda$ is nonsense: $\mathbf{A}$ is a matrix and λ is a number, and you cannot subtract a number from a matrix. You must insert the identity, $\lambda\mathbf{x} = \lambda\mathbf{Ix}$, turning the scalar into the matrix $\lambda\mathbf{I} = \mathrm{diag}(\lambda, \ldots, \lambda)$. *Then* $\mathbf{A} - \lambda\mathbf{I}$ is a legitimate matrix subtraction.

Consider $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$. This is a homogeneous system, and we want a *nonzero* solution $\mathbf{x}$. When does a square system $\mathbf{Mx} = \mathbf{0}$ have a nonzero solution? Exactly when the matrix $\mathbf{M} = \mathbf{A} - \lambda\mathbf{I}$ is *singular* – when it squashes some nonzero vector down to $\mathbf{0}$, i.e. when it has a nontrivial nullspace. And we have a perfect detector for singularity from the determinant chapters: a square matrix is singular precisely when its determinant is zero.

Formal

The number λ is an eigenvalue of $\mathbf{A}$ if and only if $\mathbf{A} - \lambda\mathbf{I}$ is singular, that is, if and only if

$$\boxed{\det(\mathbf{A} - \lambda\mathbf{I}) = 0.}$$

The left-hand side, viewed as a function of λ, is a polynomial of degree n called the *characteristic polynomial* $p(\lambda)$; the equation $p(\lambda) = 0$ is the *characteristic equation*. Its roots are exactly the eigenvalues of $\mathbf{A}$.

This is the engine. To find eigenvalues you do not hunt for special vectors at all – you compute one determinant, get a polynomial in λ, and find its roots. The eigenvectors come afterward, one eigenvalue at a time.

23.2.1 The full recipe, walked through twice

Focus

The eigenvalue recipe.

1. Form $\mathbf{A} - \lambda\mathbf{I}$ (subtract λ from each diagonal entry).
2. Compute $\det(\mathbf{A} - \lambda\mathbf{I})$; this is the characteristic polynomial $p(\lambda)$.
3. Solve $p(\lambda) = 0$ for the eigenvalues $\lambda_1, \lambda_2, \ldots$
4. For each eigenvalue λ_i, solve $(\mathbf{A} - \lambda_i\mathbf{I})\mathbf{x} = \mathbf{0}$ to find its eigenvectors (its nullspace).

Example 23.1 (Our running matrix, done by the book)**.** Take $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ and rediscover its eigenstructure without guessing.

Step 1. Subtract λ from the diagonal:

$$\mathbf{A} - \lambda\mathbf{I} = \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}.$$

Step 2. Take the determinant of this 2×2 (recall $\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$):

$$p(\lambda) = (2-\lambda)(2-\lambda) - (1)(1) = (2-\lambda)^2 - 1.$$

Step 3. Set it to zero. We could expand, but it factors nicely as a difference of squares, $(2-\lambda)^2 - 1 = \big((2-\lambda)-1\big)\big((2-\lambda)+1\big) = (1-\lambda)(3-\lambda)$:

$$p(\lambda) = (1-\lambda)(3-\lambda) = 0 \quad \Longrightarrow \quad \lambda_1 = 3,\ \lambda_2 = 1.$$

There are the two stretch factors we found geometrically.

Step 4, eigenvalue $\lambda_1 = 3$. Solve $(\mathbf{A} - 3\mathbf{I})\mathbf{x} = \mathbf{0}$:

$$\mathbf{A} - 3\mathbf{I} = \begin{bmatrix} 2-3 & 1 \\ 1 & 2-3 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}.$$

The single equation both rows give is $-x_1 + x_2 = 0$, i.e. $x_1 = x_2$. So every eigenvector for $\lambda = 3$ is a multiple of $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. (Notice the matrix $\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}$ is singular – its two rows are proportional – which is exactly why $\lambda = 3$ was an eigenvalue.)

Step 4, eigenvalue $\lambda_2 = 1$. Solve $(\mathbf{A} - 1 \cdot \mathbf{I})\mathbf{x} = \mathbf{0}$:

$$\mathbf{A} - \mathbf{I} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad x_1 + x_2 = 0 \implies x_2 = -x_1.$$

So the eigenvectors for $\lambda = 1$ are multiples of $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Both lines match Figure 23.1 exactly.

Common trap

After you plug an eigenvalue back in, the matrix $\mathbf{A} - \lambda\mathbf{I}$ *must* come out singular – its rows must be dependent – or you have made an arithmetic error. If you reduce $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$ and find the only solution is $\mathbf{x} = \mathbf{0}$, something is wrong: either your λ is not really a root, or you slipped in the determinant. A correct eigenvalue always leaves you a free variable.

Example 23.2 (A non-symmetric 2×2)**.** Let $\mathbf{B} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$. Then

$$\det(\mathbf{B} - \lambda\mathbf{I}) = \det \begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix} = (4-\lambda)(3-\lambda) - (1)(2).$$

Expand: $(4-\lambda)(3-\lambda) = 12 - 7\lambda + \lambda^2$, so

$$p(\lambda) = \lambda^2 - 7\lambda + 12 - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 2)(\lambda - 5).$$

The eigenvalues are $\lambda = 5$ and $\lambda = 2$.

For $\lambda = 5$: $\mathbf{B} - 5\mathbf{I} = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}$, giving $-x_1 + x_2 = 0$, so $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

For $\lambda = 2$: $\mathbf{B} - 2\mathbf{I} = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}$, giving $2x_1 + x_2 = 0$, so $x_2 = -2x_1$ and $\mathbf{x} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$.

Sanity check the eigenvalue 5: $\mathbf{B}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4+1 \\ 2+3 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix} = 5\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Good.

Compute it yourself

Compute the eigenvalues and eigenvectors of $\mathbf{C} = \begin{bmatrix} 3 & 0 \\ 4 & 1 \end{bmatrix}$ entirely by hand, following the four steps. (Because $\mathbf{C}$ is triangular, you should be able to read its eigenvalues straight off the diagonal before you even start – see Section 23.3. Use the recipe anyway and confirm they agree.) Then verify each eigenvector by direct multiplication.

Before/after Strang

Strang introduces eigenvalues with this very same move, noting that the eigenvalues come from $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ while the eigenvectors come from the nullspace of $\mathbf{A} - \lambda\mathbf{I}$. If you are reading this *before* Strang, fix in your mind that two different ideas are stacked here: a *determinant* test finds the special numbers λ, and then a *nullspace* computation finds the special directions $\mathbf{x}$. Reading *after* Strang, notice we are about to push past the mechanics into what the eigenvalues *mean* – trace, determinant, complex rotations.

23.3 Triangular and diagonal matrices: eigenvalues for free

For certain structured matrices, the eigenvalues can be read directly from the entries. If $\mathbf{A}$ is triangular (all zeros above, or all zeros below, the diagonal), then $\mathbf{A} - \lambda\mathbf{I}$ is also triangular, and the determinant of a triangular matrix is just the product of its diagonal entries. So

$$\det(\mathbf{A} - \lambda\mathbf{I}) = (a_{11} - \lambda)(a_{22} - \lambda)\cdots(a_{nn} - \lambda),$$

and the roots are simply $\lambda = a_{11}, a_{22}, \ldots, a_{nn}$.

Proposition 23.1. *The eigenvalues of a triangular (in particular, a diagonal) matrix are exactly its diagonal entries.*

Example 23.3. For $\mathbf{T} = \begin{bmatrix} 5 & 8 & -2 \\ 0 & 1 & 7 \\ 0 & 0 & 3 \end{bmatrix}$ the eigenvalues are $5, 1, 3$ at a glance – no determinant expansion needed. (The eigenvectors still require work; only the eigenvalues are free.)

Common trap

"Eigenvalues are the diagonal entries" is true *only* for triangular matrices. For a full matrix the diagonal entries are not the eigenvalues. For Example 23.2, $\mathbf{B}$ had diagonal $4, 3$ but eigenvalues $5, 2$. What *is* always true is that the diagonal entries *sum* to the same thing the eigenvalues sum to – the trace – which is the subject of the next section ($4 + 3 = 7 = 5 + 2$).

Pause

Pause and think: A 3×3 diagonal matrix has entries $7, -2, 4$ along its main diagonal. Without writing down any characteristic polynomial, what are its eigenvalues, and what are its eigenvectors?

Because a diagonal matrix is triangular, Proposition 23.1 tells us the eigenvalues are the diagonal entries: $\lambda_1 = 7$, $\lambda_2 = -2$, and $\lambda_3 = 4$. And its eigenvectors are simply the standard unit basis vectors $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$, and $\mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, because $\mathbf{D}\mathbf{e}_1 = 7\mathbf{e}_1$, $\mathbf{D}\mathbf{e}_2 = -2\mathbf{e}_2$, and $\mathbf{D}\mathbf{e}_3 = 4\mathbf{e}_3$. A diagonal matrix is already written in its own eigen-axes.

23.4 Trace and determinant: the eigenvalues' fingerprints

Here are two facts that look like coincidences in our examples and turn out to be laws. In Example 23.2 the eigenvalues were 5 and 2. Their *sum* is 7, which is $4+3$, the sum of the diagonal entries of $\mathbf{B}$. That diagonal sum is called the *trace* of the matrix: $\operatorname{tr}\mathbf{B} = 4 + 3 = 7$. In general, for any $n \times n$ matrix $\mathbf{A}$, the trace is defined as the sum of all entries on its main diagonal:

$$\operatorname{tr}\mathbf{A} = a_{11} + a_{22} + \cdots + a_{nn}.$$

Their *product* is 10, which is exactly $\det\mathbf{B} = 4 \cdot 3 - 1 \cdot 2$. Both identities hold for every square matrix.

Theorem 23.1 (Trace and determinant from eigenvalues)**.** *Let $\mathbf{A}$ be $n \times n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$ listed with algebraic multiplicity (meaning each eigenvalue is repeated as many times as it appears as a root of $p(\lambda)$, counting complex roots). Then*

$$\lambda_1 + \lambda_2 + \cdots + \lambda_n = \operatorname{tr}\mathbf{A}, \qquad \lambda_1\lambda_2\cdots\lambda_n = \det\mathbf{A}.$$

The eigenvalues add up *to the trace and* multiply *to the determinant.*

Here is why, for the 2×2 case, where the algebra is transparent and the pattern is unmistakable. Write $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$. Its characteristic polynomial is

$$\det(\mathbf{A} - \lambda\mathbf{I}) = (a-\lambda)(d-\lambda) - bc = \lambda^2 - (a+d)\lambda + (ad - bc) = \lambda^2 - (\operatorname{tr}\mathbf{A})\,\lambda + \det\mathbf{A}.$$

So for any 2×2 matrix the characteristic polynomial is the clean little formula

$$\boxed{p(\lambda) = \lambda^2 - (\operatorname{tr}\mathbf{A})\,\lambda + \det\mathbf{A}.}$$

Now compare with the factored form. If the roots are λ_1, λ_2, then $p(\lambda) = (\lambda - \lambda_1)(\lambda - \lambda_2) = \lambda^2 - (\lambda_1 + \lambda_2)\lambda + \lambda_1\lambda_2$. Matching the coefficient of λ gives $\lambda_1 + \lambda_2 = \operatorname{tr}\mathbf{A}$, and matching the constant term

gives $\lambda_1\lambda_2 = \det \mathbf{A}$. Done. (The general $n \times n$ proof is the same idea: the characteristic polynomial is $(-1)^n(\lambda - \lambda_1)\cdots(\lambda - \lambda_n)$, and expanding it, the λ^{n-1} coefficient is $(-1)^{n-1}\sum\lambda_i = (-1)^{n-1}\operatorname{tr}\mathbf{A}$ while the constant term $p(0) = \det\mathbf{A} = \prod\lambda_i$.)

Focus

For a 2×2 you can often skip the algebra entirely. You only need two numbers: $t = \operatorname{tr}\mathbf{A}$ and $d = \det\mathbf{A}$. The eigenvalues are the two numbers that add to t and multiply to d:

$$\lambda = \frac{t \pm \sqrt{t^2 - 4d}}{2}.$$

Example 23.4 (Reading eigenvalues off trace and determinant)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: $t = 4$, $d = 4-1 = 3$. We need two numbers adding to 4 and multiplying to 3. By inspection: 3 and 1. (Or use the formula: $\lambda = \frac{4\pm\sqrt{16-12}}{2} = \frac{4\pm2}{2} = 3, 1$.) Same answer as Example 23.1, with no determinant of $\mathbf{A} - \lambda\mathbf{I}$ expanded.

Why this matters

The trace and determinant are computed directly from the entries, with no eigenvalues in sight – yet they are completely determined by the eigenvalues, which capture the matrix's deep, basis-independent character. Trace and determinant are the two simplest *fingerprints* of a matrix that survive any change of coordinates. The eigenvalues are the full fingerprint; the trace (their sum) and determinant (their product) are the first and last things you can read off without solving the whole polynomial. That is why they show up everywhere as quick sanity checks.

Compute it yourself

Use only trace and determinant – do not expand a characteristic polynomial – to find the eigenvalues of each:

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \quad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}, \quad \begin{bmatrix} 5 & 4 \\ 0 & 5 \end{bmatrix}.$$

For each, the eigenvalues add to the trace and multiply to the determinant. Notice what happens to the second one (trace 0, determinant 1): the quantity under the square root, $t^2 - 4d = -4$ (the discriminant), is negative. We deal with that in Section 23.7.

23.5 What the eigenvalues do to the picture

Eigenvalues are far more than algebraic roots of a polynomial: each carries a direct geometric meaning. Once you know the eigenvalues of a 2×2 matrix and its eigenvector directions, you can visualize the motion of the entire plane at a glance. The key is to view space through the coordinate system that the matrix itself prefers: *decompose along the eigenvectors.*

Suppose $\mathbf{A}$ has eigenvectors $\mathbf{v}_1, \mathbf{v}_2$ that span the plane, with eigenvalues λ_1, λ_2. Any vector $\mathbf{x}$ can

be written as a combination $\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2$. Then by linearity

$$\mathbf{A}\mathbf{x} = c_1\,\mathbf{A}\mathbf{v}_1 + c_2\,\mathbf{A}\mathbf{v}_2 = c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2.$$

The matrix acts *one eigendirection at a time*: it stretches the $\mathbf{v}_1$ part by λ_1 and the $\mathbf{v}_2$ part by λ_2, independently. In eigenvector coordinates, the most complicated thing this matrix can do is two separate scalings. That is the bridge to diagonalization in the next chapter. Figure 23.2 shows the two cases side by side: two positive eigenvalues turn the unit circle into a stretched ellipse, while a negative eigenvalue flips one direction end-to-end.

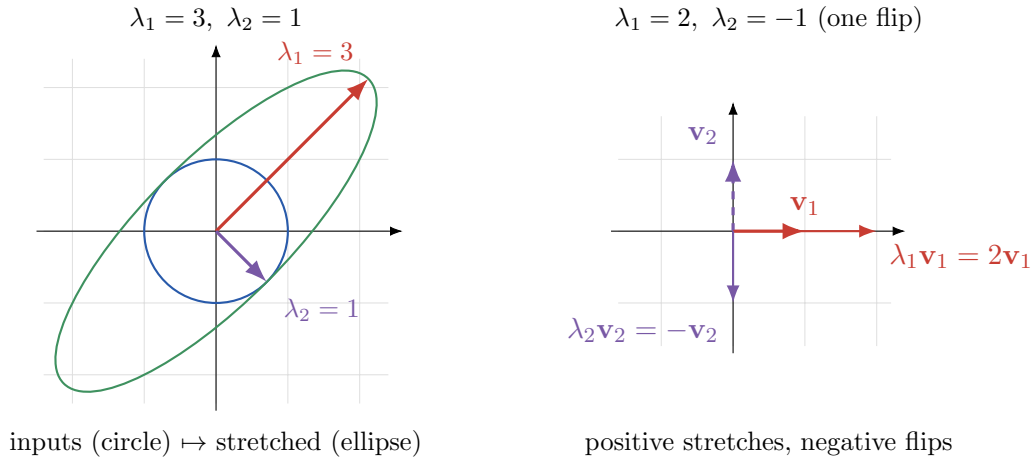


Figure 23.2: Left: when both eigenvalues are positive, the matrix turns the unit circle into an ellipse whose axes are the eigenvector directions, stretched by the eigenvalues. Right: a negative eigenvalue ($\lambda_2 = -1$) flips that eigendirection end-to-end while the positive one ($\lambda_1 = 2$) stretches.

Here is the field guide to a single real eigenvalue λ along its direction:

- $\lambda > 1$: *stretch outward* along that line (the direction is amplified). Repeatedly applying $\mathbf{A}$ sends those components flying off to infinity.
- $\lambda = 1$: *fixed direction* – vectors on that line do not move at all.
- $0 < \lambda < 1$: *shrink* toward the origin along that line. Repeated application collapses these components to nothing.
- $\lambda = 0$: *collapse* – the entire line is sent to the origin in one step. The matrix is singular; this line is the nullspace.
- $\lambda < 0$: *flip* across the origin, and stretch (if $\lambda < -1$) or shrink (if $-1 < \lambda < 0$). Each application of $\mathbf{A}$ bounces that component to the other side.

Intuition

The magnitude $|\lambda|$ governs the asymptotic behavior of each eigendirection under repeated application of the matrix. The eigenvalue with the largest magnitude $|\lambda|$ dominates: after many applications of $\mathbf{A}$, vectors generally align with the eigenvector of the largest $|\lambda|$, because that component grows fastest (or shrinks slowest) relative to the others. That fact drives the power method for computing eigenvectors, PageRank, and the stability analysis of dynamical

systems. We make it precise in Chapters 24 and 35.

Modern application

PageRank: the web's dominant eigenvector. When Google ranks web pages, it builds a giant matrix $\mathbf{M}$ describing the "random surfer": start on a page, follow a random outgoing link, repeat. The entry M_{ij} is the probability of stepping from page j to page i. The importance of every page is the eigenvector of $\mathbf{M}$ belonging to the eigenvalue $\lambda = 1$ – the one direction that the surfing process leaves *unchanged*, the steady state $\mathbf{Mx} = \mathbf{x}$. Every other eigendirection has $|\lambda| < 1$ and dies away as the surfer keeps walking, so no matter where you start, you converge to that dominant eigenvector. PageRank comes down to one instruction: "find the eigenvector for $\lambda = 1$ of a matrix with a few billion rows." The machinery of this chapter – eigenvalue 1 means a direction the matrix fixes, largest $|\lambda|$ dominates the long run – is exactly what makes it work.

Example 23.5 (Reading a matrix from its eigendata)**.** Suppose someone tells you a 2×2 matrix has eigenvalue 2 along $\begin{bmatrix}1\\0\end{bmatrix}$ and eigenvalue -1 along $\begin{bmatrix}0\\1\end{bmatrix}$. You can sketch its action immediately: it doubles everything horizontal and flips everything vertical. The matrix must be $\begin{bmatrix}2 & 0\\0 & -1\end{bmatrix}$, because that diagonal matrix has exactly those eigenvalues along the axes. A complete set of eigenvalues and eigenvectors fully characterizes the matrix.

23.6 Special matrices and their eigenvalues

A few geometric transformations you already know have eigenvalues you can read off from the geometry, with no algebra at all. Each is worth keeping as a mental anchor.

Projections. Let $\mathbf{P}$ project the plane onto a line L through the origin. If $\mathbf{x}$ already lies on L, projecting does nothing: $\mathbf{Px} = \mathbf{x}$, eigenvalue 1. If $\mathbf{x}$ is perpendicular to L, projecting kills it: $\mathbf{Px} = \mathbf{0} = 0 \cdot \mathbf{x}$, eigenvalue 0. So every projection has eigenvalues 1 and 0 – which is exactly why projections satisfy $\mathbf{P}^2 = \mathbf{P}$ (the eigenvalues satisfy $\lambda^2 = \lambda$). Figure 23.3 shows the two eigendirections.

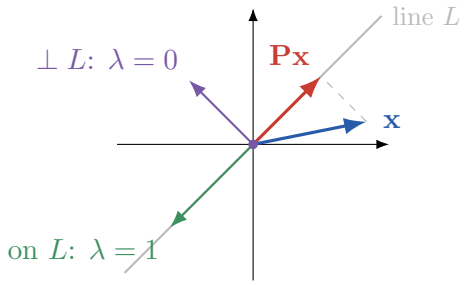


Figure 23.3: Projection onto the line L. Vectors already on L are left alone (eigenvalue 1); vectors perpendicular to L are collapsed to the origin (eigenvalue 0). A generic $\mathbf{x}$ (blue) is dropped onto its shadow $\mathbf{Px}$ on L (red) – so it is genuinely moved and is not an eigenvector.

Reflections. Let $\mathbf{R}$ reflect across a line L. Vectors on L are untouched (eigenvalue 1); vectors

perpendicular to L are sent to their negatives (eigenvalue -1). A reflection always has eigenvalues $+1$ and -1, which is why doing it twice is the identity: $\lambda^2 = 1$. Figure 23.4 shows the two eigendirections.

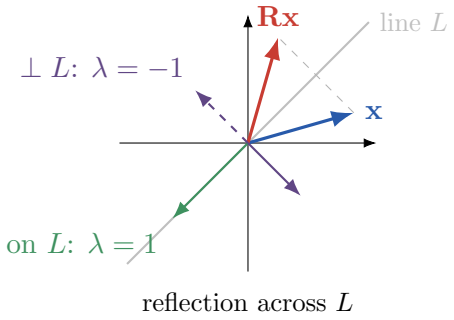


Figure 23.4: A reflection across the line L fixes the direction along L (eigenvalue $+1$) and flips the perpendicular direction (eigenvalue -1). A generic vector $\mathbf{x}$ (blue) is genuinely turned to $\mathbf{Rx}$ (red) – it is not an eigenvector.

Scalings. The matrix $c\mathbf{I}$ stretches every direction by c. So *every* nonzero vector is an eigenvector, all with the same eigenvalue c. This is the one case where eigenvectors do not pick out special lines – because the matrix has no preferred directions, every direction is equal.

Shears. The shear $\mathbf{S} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ slides the plane horizontally. Its only eigenvalue is 1 (it is triangular, both diagonal entries 1), and the only eigenvectors lie along the horizontal axis $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ – the layer that does not slide. A shear is the classic example of a matrix with a *repeated* eigenvalue but *not enough* eigenvectors to fill the plane (only one line, not two; see Figure 23.5). We will care a great deal about that shortfall when we ask which matrices can be diagonalized.

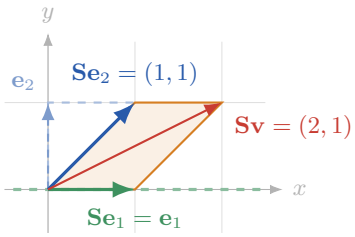


Figure 23.5: The shear matrix $\mathbf{S} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ slides horizontal lines to the right by an amount proportional to height. The horizontal axis (green) is the *only* line preserved: $\mathbf{S}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, so $\lambda = 1$ is the sole eigenvalue. Every other line tilts (for instance, vertical $\mathbf{e}_2$ swings to $(1, 1)$, and diagonal $(1, 1)$ swings to $(2, 1)$). Because no second independent line of eigenvectors exists in the plane, the shear is *defective*.

Common trap

A repeated eigenvalue does *not* guarantee a repeated supply of eigenvectors. The shear has $\lambda = 1$ twice but only a one-dimensional line of eigenvectors. Contrast $\mathbf{I}$, which also has $\lambda = 1$ twice but a whole plane of them. "How many times λ is a root" (algebraic multiplicity) and "how big the eigenvector space is" (geometric multiplicity) are different numbers; the second can be smaller. This gap is the entire reason the next chapters need "defective" matrices and the Jordan form.

23.7 Complex eigenvalues: when a matrix is really a rotation

Return to the walking question about a 90° rotation. Can a rotation have a real eigenvector? Picture it: a rotation turns *every* arrow by a quarter turn. There is no direction in the plane that stays on its own line. So a 90° rotation has *no real eigenvectors at all* – and yet the characteristic equation will still hand us two roots, because a degree-2 polynomial always has two roots over the complex numbers. Those roots are complex, and they are telling us "rotation," not "stretch."

Before computing, let us build a quick geometric bridge for what complex numbers mean in linear algebra. (We develop the full theory of complex vector spaces in Chapter 32 and the Chapter 39 appendix, but you need only one visual intuition here.)

Intuition

A geometric bridge to complex numbers. On the ordinary number line, multiplying by -1 reverses direction: it is a 180° half-turn. What operation, performed *twice in a row*, produces a 180° half-turn? A 90° quarter-turn. That is the geometric meaning of the imaginary unit $i = \sqrt{-1}$: *multiplying by i rotates the two-dimensional plane counterclockwise by* 90°. Every complex number $z = a + bi$ can be plotted as a point (a, b) in the plane. Its distance from the origin is its magnitude $r = |z| = \sqrt{a^2 + b^2}$, and its direction is an angle θ. In polar form, Euler's identity writes this as $z = re^{i\theta} = r(\cos\theta + i\sin\theta)$. Multiplying by $z = re^{i\theta}$ does two things simultaneously:

1. It *stretches* lengths by the factor $r = |z|$.
2. It *rotates* vectors by the angle θ.

When a matrix rotates space instead of just stretching it along fixed axes, its eigenvalues cannot be real numbers. Instead, they become complex numbers whose magnitude r records the stretch factor and whose angle θ records the angle of rotation.

Here is the computation. The 90° counterclockwise rotation is

$$\mathbf{Q} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Its trace is 0 and its determinant is $0 \cdot 0 - (-1)(1) = 1$, so

$$p(\lambda) = \lambda^2 - (\operatorname{tr}\mathbf{Q})\lambda + \det\mathbf{Q} = \lambda^2 + 1 = 0 \quad \Longrightarrow \quad \lambda = \pm i.$$

The eigenvalues are the imaginary numbers $+i$ and $-i$. Notice $|\lambda| = |\pm i| = 1$: the rotation does not change any length, and the magnitude of a complex eigenvalue records exactly that scaling. The

angle of the eigenvalue records the rotation: $i = e^{i\pi/2}$ sits at angle 90°, which is precisely the angle $\mathbf{Q}$ rotates by. Figure 23.6 shows both pictures at once: the turned arrows on the left, the conjugate pair $\pm i$ on the unit circle on the right.

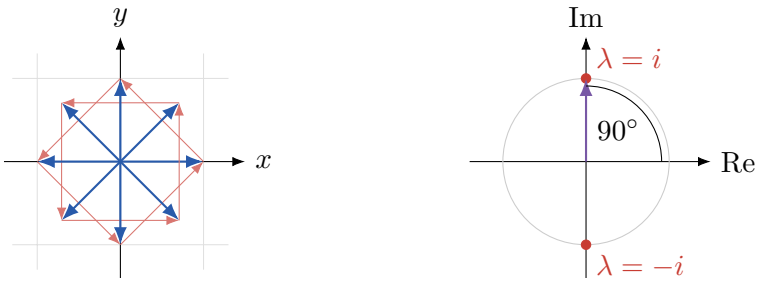


Figure 23.6: A 90° rotation has no real eigendirection (left): every arrow is turned. Its eigenvalues are complex, $\pm i$, sitting on the unit circle of the complex plane (right). The magnitude $|\lambda| = 1$ says "no stretching"; the angle of λ says "rotate by 90°."

The general rotation by angle θ,

$$\mathbf{Q}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \qquad \operatorname{tr} = 2\cos\theta, \quad \det = 1,$$

has characteristic polynomial $\lambda^2 - 2\cos\theta\,\lambda + 1 = 0$, whose roots are

$$\lambda = \cos\theta \pm \sqrt{\cos^2\theta - 1} = \cos\theta \pm i\sin\theta = e^{\pm i\theta}.$$

The eigenvalues are $e^{\pm i\theta}$: unit magnitude (no stretch, $\det = 1$), and their angle is exactly the rotation angle θ. This is the cleanest possible dictionary between a matrix and a complex number.

Focus

The complex eigenvalue dictionary. A pair of complex eigenvalues $\lambda = re^{\pm i\theta}$ means the matrix *spirals*: it rotates by the angle θ while scaling by the radius $r = |\lambda|$ per application.

- $|\lambda| = 1$: pure rotation, lengths preserved.
- $|\lambda| < 1$: inward spiral, the origin is attracting.
- $|\lambda| > 1$: outward spiral, the origin is repelling.

Real eigenvalues stretch along fixed lines; complex eigenvalues rotate-and-scale in invariant planes. Together they describe everything a real matrix can do.

Example 23.6 (A spiral matrix)**.** Consider $\mathbf{A} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$. Trace 2, determinant $1\cdot 1 - (-1)(1) = 2$, so $p(\lambda) = \lambda^2 - 2\lambda + 2 = 0$ and

$$\lambda = \frac{2 \pm \sqrt{4-8}}{2} = 1 \pm i.$$

Write $1 \pm i = \sqrt{2}\,e^{\pm i\pi/4}$: magnitude $\sqrt{2}$, angle 45°. So $\mathbf{A}$ rotates by 45° and stretches by $\sqrt{2}$ each time you apply it – an outward spiral. Geometrically $\mathbf{A} = \sqrt{2}\,\mathbf{Q}_{45°}$, which you can check directly

since $\mathbf{Q}_{45^\circ} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$. Repeatedly applying $\mathbf{A}$ to any nonzero vector traces a spiral winding out to infinity, 45° at a time (Figure 23.7).

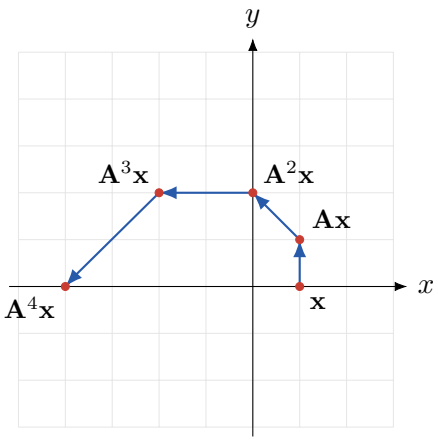


Figure 23.7: The orbit of $\mathbf{x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ under repeated application of the spiral matrix $\mathbf{A} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$. Each step turns the vector 45° counterclockwise (the angle of the complex eigenvalue) and lengthens it by $\sqrt{2}$ (the magnitude $|\lambda| > 1$), so the points wind outward. Complex eigenvalues with $|\lambda| > 1$ always produce an outward spiral like this.

Common trap

When the discriminant $t^2 - 4d$ is negative, do not declare "no eigenvalues." There are still exactly two eigenvalues; they are complex conjugates. For a *real* matrix complex eigenvalues always come in conjugate pairs $\lambda, \bar{\lambda}$, because the characteristic polynomial has real coefficients (real entries), and the complex roots of a real polynomial pair up. So a real 2×2 has either two real eigenvalues or one conjugate pair – never exactly one complex eigenvalue alone.

Connection to other math

Eigenvalues, differential equations, and stability. The verdict on $|\lambda|$ is the bridge from linear algebra to differential equations. The system $\frac{d\mathbf{x}}{dt} = \mathbf{A}\mathbf{x}$ has solutions built from $e^{\lambda t}\mathbf{v}$ for each eigenpair $(\lambda, \mathbf{v})$ – you can check that $\frac{d}{dt}\left(e^{\lambda t}\mathbf{v}\right) = \lambda e^{\lambda t}\mathbf{v} = e^{\lambda t}\mathbf{A}\mathbf{v} = \mathbf{A}\left(e^{\lambda t}\mathbf{v}\right)$. The *real part* of λ governs the long-term amplitude: $\operatorname{Re}\lambda < 0$ means $e^{\lambda t} \to 0$, a stable mode that decays; $\operatorname{Re}\lambda > 0$ means exponential growth; and the *imaginary part* sets the frequency of oscillation. A bridge oscillates, a circuit rings, a population cycles – all read off from where the eigenvalues sit in the complex plane. (We develop this fully in Chapter 25; for now, notice that the same complex eigenvalues that meant "rotation" for $\mathbf{A}\mathbf{x}$ mean "oscillation" for $e^{\mathbf{A}t}\mathbf{x}$.)

23.8 Why eigenvectors of distinct eigenvalues are independent

We keep saying "the eigenvectors span the plane" as if it were automatic. It is not automatic, as the shear matrix showed, but distinct eigenvalues always guarantee it.

Theorem 23.2 (Distinct eigenvalues give independent eigenvectors)**.** *If* $\mathbf{v}_1, \dots, \mathbf{v}_k$ *are eigenvectors of* $\mathbf{A}$ *with* distinct *eigenvalues* $\lambda_1, \dots, \lambda_k$ *(all different from one another), then* $\mathbf{v}_1, \dots, \mathbf{v}_k$ *are linearly independent.*

Proof. Take the smallest case $k = 2$ first; it carries the whole idea. Suppose $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 = \mathbf{0}$. Apply $\mathbf{A}$ to both sides; since $\mathbf{A}\mathbf{v}_i = \lambda_i\mathbf{v}_i$,

$$c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2 = \mathbf{0}.$$

Now also multiply the original relation by λ_1: $c_1\lambda_1\mathbf{v}_1 + c_2\lambda_1\mathbf{v}_2 = \mathbf{0}$. Subtract the two:

$$c_2(\lambda_2 - \lambda_1)\mathbf{v}_2 = \mathbf{0}.$$

Because $\mathbf{v}_2 \neq \mathbf{0}$ and $\lambda_2 \neq \lambda_1$, the only way out is $c_2 = 0$. Back-substituting, $c_1\mathbf{v}_1 = \mathbf{0}$ forces $c_1 = 0$ as well. So the only combination giving $\mathbf{0}$ is the trivial one: $\mathbf{v}_1, \mathbf{v}_2$ are independent. The general case is the same trick by induction: assume any $k-1$ of them are independent; a dependence among all k, hit with $\mathbf{A}$ and combined with a λ_k-multiple of itself, collapses to a dependence among $k-1$, which forces all the coefficients to vanish. □

Corollary 23.1. *If an* $n \times n$ *matrix has* n distinct *eigenvalues, its* n *eigenvectors form a basis of* $\mathbb{R}^n$ *(or* $\mathbb{C}^n$*). Every vector then decomposes into eigendirections, and the matrix acts as* n *independent scalings.*

This is the standard case: n distinct eigenvalues produce n independent eigenvectors, forming a complete eigenbasis. The shear matrix of Section 23.6 fails to have an eigenbasis precisely because its eigenvalues are repeated – and understanding when a matrix has a complete eigenbasis is the central question of diagonalization in the next chapter.

Before/after Strang

Strang states the central conclusion of this chapter directly: eigenvectors from different eigenvalues are independent, so a matrix with no repeated eigenvalues has a full set of eigenvectors. If you are heading *into* Strang, carry Theorem 23.2 as the reason diagonalization works. If you are coming *out* of Strang, this is the theorem you will lean on to write $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$ in the next chapter and to justify the spectral theorem for symmetric matrices later on.

23.9 A worked 3×3

Nothing about the recipe changes in three dimensions; the determinant is just a little bigger. Here is one worked all the way through.

Example 23.7 (Eigenstructure of a 3×3)**.** Take

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 1 \\ -1 & 0 & 1 \end{bmatrix}.$$

Characteristic polynomial. Expand $\det(\mathbf{A} - \lambda\mathbf{I})$ along the first row, which has two zeros and so is easy:

$$\det \begin{bmatrix} 2-\lambda & 0 & 0 \\ 1 & 2-\lambda & 1 \\ -1 & 0 & 1-\lambda \end{bmatrix} = (2-\lambda)\det \begin{bmatrix} 2-\lambda & 1 \\ 0 & 1-\lambda \end{bmatrix}.$$

The remaining 2×2 is triangular, so its determinant is $(2-\lambda)(1-\lambda)$. Altogether

$$p(\lambda) = (2-\lambda)\,(2-\lambda)(1-\lambda) = (2-\lambda)^2(1-\lambda).$$

The eigenvalues are $\lambda = 2$ (a double root) and $\lambda = 1$.

Check with trace and determinant. The trace is $2+2+1 = 5$, and the eigenvalues sum to $2+2+1 = 5$. Good. The determinant is $\lambda_1\lambda_2\lambda_3 = 2 \cdot 2 \cdot 1 = 4$; you can verify directly that $\det \mathbf{A} = 4$ by cofactor expansion. Both fingerprints match.

Eigenvectors for $\lambda = 1$. Solve $(\mathbf{A} - \mathbf{I})\mathbf{x} = \mathbf{0}$:

$$\mathbf{A} - \mathbf{I} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ -1 & 0 & 0 \end{bmatrix}.$$

Row one says $x_1 = 0$; row two then says $x_2 + x_3 = 0$, so $x_2 = -x_3$. Taking $x_3 = 1$ gives the eigenvector $\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$.

Eigenvectors for $\lambda = 2$. Solve $(\mathbf{A} - 2\mathbf{I})\mathbf{x} = \mathbf{0}$:

$$\mathbf{A} - 2\mathbf{I} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ -1 & 0 & -1 \end{bmatrix}.$$

The first row is all zeros; the second and third rows both reduce to $x_1 + x_3 = 0$. So there are *two* free variables (x_2 free, and x_3 free with $x_1 = -x_3$). The eigenvectors for $\lambda = 2$ fill a whole *plane*, spanned by $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ (taking $x_2 = 1, x_3 = 0$) and $\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$ (taking $x_2 = 0, x_3 = 1$). The subspace formed by all eigenvectors belonging to an eigenvalue λ, together with the zero vector $\mathbf{0}$, is called the *eigenspace* $E_\lambda = \text{null}(\mathbf{A} - \lambda\mathbf{I})$: it is the complete subspace of directions that $\mathbf{A}$ scales by λ.

Although $\lambda = 2$ is a root of multiplicity two, its eigenspace is two-dimensional. Combining this two-dimensional eigenspace with the one-dimensional eigenspace for $\lambda = 1$ yields three linearly independent eigenvectors spanning all of $\mathbb{R}^3$. This matrix has a complete eigenbasis, $\left\{ \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \right\}$, despite the repeated eigenvalue. Unlike the shear matrix, where a repeated eigenvalue produced only a one-dimensional eigenspace, here the geometric multiplicity equals the algebraic multiplicity. Figure 23.8 pictures the result: a single eigen-line for $\lambda = 1$ and an entire eigen-plane for $\lambda = 2$, together filling $\mathbb{R}^3$.

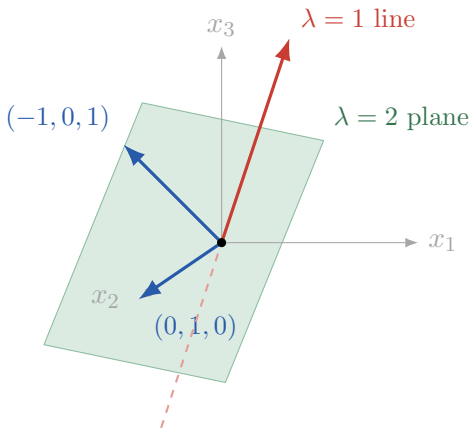


Figure 23.8: Eigenspaces of the 3×3 matrix of Example 23.7. The double eigenvalue $\lambda = 2$ has an entire plane of eigenvectors (green), spanned by $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$, while $\lambda = 1$ has a single eigen-line (red) along $\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$. The plane and the line meet only at the origin and together span $\mathbb{R}^3$, so the matrix has a complete eigenbasis.

Compute it yourself

By hand, find all eigenvalues of $\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$ (triangular – read them off), then determine the dimension of the eigenspace for $\lambda = 2$ by row-reducing $\mathbf{A} - 2\mathbf{I}$. You should find $\lambda = 2$ is a double root but its eigenspace is only one-dimensional: a 3×3 cousin of the shear, a matrix that *cannot* be diagonalized. Compare with Example 23.7, where the double root was diagonalizable.

History and connection

Eigenvalues and eigenvectors did not arise from abstract algebra; they were born from urgent physical problems in eighteenth-century astronomy and mechanics. When Leonhard Euler (1750s) studied the rotation of spinning solid bodies, he discovered that describing motion in arbitrary coordinates produced a chaotic tangle of cross-torques and wobbles. He proved that every rigid body possesses three mutually perpendicular *principal axes of inertia*—the natural directions of balance about which an object can spin cleanly without tumbling. Three decades later, Joseph-Louis Lagrange (1780s) tackled celestial stability: do the mutual gravitational attractions of the planets eventually destabilize the solar system? In standard Cartesian coordinates, the acceleration of every planet depends on every other planet. Lagrange sought a new coordinate system that would completely decouple this tangled network into independent, pure *harmonic oscillators* (simple back-and-forth periodic motions, like ideal pendulums).
In 1829, Augustin-Louis Cauchy synthesized these ideas into matrix terms, introducing the term "characteristic equation" for $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ and proving that real symmetric systems

always have purely real eigenvalues. Decades later, David Hilbert popularized the German term *Eigenwert* (meaning "inherent value" or "characteristic value"). Yet two major hurdles caught early mathematicians. The first was the defective matrix: early geometers assumed every linear transformation had a full coordinate frame of invariant axes, only to be surprised by shear matrices, which possess only a single line of eigenvectors. The second was algorithmic: because eigenvalues are polynomial roots, mathematicians hoped for explicit algebraic formulas, but by the Abel–Ruffini theorem (1824), polynomial equations of degree five or higher cannot be solved by algebraic radicals (formulas using only $+, -, \times, \div$ and root extractions like $\sqrt[n]{\cdot}$). Finding eigenvalues for general matrices is fundamentally impossible in a finite number of algebraic steps; it must be an iterative, geometric process. Around 1960, Soviet mathematician Vera Kublanovskaya and British computer scientist John Francis independently overcame this barrier by creating the QR algorithm, which iteratively drives a matrix toward triangular form to read off its eigenvalues stably.
Eigenvalues identify the invariant directions of a transformation along which coupled dynamics decouple into one-dimensional scaling, determining long-term growth, decay, or steady-state behavior. Their main limitation is that they apply only to square matrices, and for non-symmetric matrices they can fail to span the space. In Chapter 24, we assemble these invariant directions into a complete basis to diagonalize matrices, in Chapter 25 we use them to solve continuous dynamical systems, and in Chapter 29 we meet the singular value decomposition, which extends invariant stretching axes to rectangular matrices.

23.10 Pulling it together

Why this matters

The defining relation $\mathbf{Ax} = \lambda\mathbf{x}$ identifies the invariant directions along which a matrix acts by pure scalar scaling. These eigenvalues summarize essential coordinate-free properties of the transformation: their sum equals the trace, their product equals the determinant, real eigenvalues describe directional stretching, and complex eigenvalues characterize rotation. In addition, their magnitudes dictate the long-term behavior of dynamical systems under iteration. This foundation leads directly to *diagonalization* ($\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$), where the eigenvectors form a coordinate system in which the transformation is diagonal, and onward to the spectral theorem and the singular value decomposition.

Focus

What to hold onto from this chapter.

- $\mathbf{Ax} = \lambda\mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$: eigenvectors are the directions a matrix does not turn; eigenvalues are how much it scales them.
- Eigenvalues are the roots of $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$; eigenvectors are the nullspace of $\mathbf{A} - \lambda\mathbf{I}$.
- For 2×2: eigenvalues add to the trace and multiply to the determinant; $p(\lambda) = \lambda^2 - (\operatorname{tr}\mathbf{A})\lambda + \det\mathbf{A}$.
- Triangular matrices show their eigenvalues on the diagonal.

- $\lambda = 0 \Leftrightarrow$ singular; $|\lambda|$ governs long-run growth; complex eigenvalues $re^{\pm i\theta}$ mean rotate-by-θ, scale-by-r.
- Distinct eigenvalues $\Rightarrow$ independent eigenvectors $\Rightarrow$ a complete eigenbasis (the doorway to diagonalization).

Exercises

Exercise 23.1 (Quick mental checks)**.** Answer each with a brief reason, without setting up any equations:

(a) Can the zero vector $\mathbf{0}$ ever be an eigenvector? Can the number 0 ever be an eigenvalue?

(b) If a 2×2 matrix has eigenvalues $\lambda_1 = 4$ and $\lambda_2 = -3$, what are its trace and its determinant?

(c) If $\mathbf{A}\mathbf{v} = 5\mathbf{v}$, what is $\mathbf{A}^3\mathbf{v}$? What is $(\mathbf{A} + 2\mathbf{I})\mathbf{v}$?

(d) Can a 3×3 matrix with all real entries have three purely imaginary eigenvalues? (*Hint:* complex roots of real polynomials come in conjugate pairs.)

Exercise 23.2 (Warmup by hand)**.** For each matrix, find the eigenvalues and one eigenvector per eigenvalue.

$$\text{(a) } \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix} \quad \text{(b) } \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix} \quad \text{(c) } \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} \quad \text{(d) } \begin{bmatrix} 1 & 4 \\ 1 & 1 \end{bmatrix}.$$

Exercise 23.3 (Trace and determinant only)**.** Without expanding any characteristic polynomial, find the eigenvalues of $\begin{bmatrix} 6 & 2 \\ 2 & 3 \end{bmatrix}$ using only that they add to the trace and multiply to the determinant. Then verify by computing $\det(\mathbf{A} - \lambda\mathbf{I})$ directly.

Exercise 23.4 (Reflection geometry)**.** The matrix $\mathbf{R} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ reflects the plane across the line $y = x$. **(a)** Without computing, predict its eigenvalues and eigenvectors from the geometry. **(b)** Confirm with the characteristic equation. **(c)** Explain why $\mathbf{R}^2 = \mathbf{I}$ follows from the eigenvalues.

Exercise 23.5 (A scaled rotation)**.** Let $\mathbf{A} = \begin{bmatrix} 3 & -2 \\ 2 & 3 \end{bmatrix}$. **(a)** Show the eigenvalues are complex and write them in the form $re^{\pm i\theta}$. **(b)** By what angle does $\mathbf{A}$ rotate, and by what factor does it scale, each time it is applied? **(c)** What is the long-run fate of $\mathbf{A}^n\mathbf{x}$ for a generic $\mathbf{x}$ – does it spiral in or out?

Exercise 23.6 (Singular means $\lambda = 0$)**.** Show that the matrix $\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$ has 0 as an eigenvalue, find the corresponding eigenvector, and explain how this eigenvector relates to the nullspace and to the fact that $\det \mathbf{A} = 0$.

Exercise 23.7 (Eigenvalues of $\mathbf{A}^2$ and $\mathbf{A}^{-1}$)**.** Suppose $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$. **(a)** Show $\mathbf{A}^2\mathbf{x} = \lambda^2\mathbf{x}$, so λ^2 is an eigenvalue of $\mathbf{A}^2$ with the *same* eigenvector. **(b)** If $\mathbf{A}$ is invertible, show $\mathbf{A}^{-1}\mathbf{x} = \frac{1}{\lambda}\mathbf{x}$. **(c)** Show $(\mathbf{A} + c\mathbf{I})\mathbf{x} = (\lambda + c)\mathbf{x}$. Conclude that shifting a matrix by $c\mathbf{I}$ shifts every eigenvalue by c but leaves the eigenvectors alone.

Exercise 23.8 (A 3×3 from start to finish)**.** For $\mathbf{A} = \begin{bmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$, find the characteristic polynomial, all three eigenvalues, and an eigenvector for each. Check your eigenvalues against the trace and the determinant.

Exercise 23.9 (Repeated eigenvalue, two outcomes)**.** Both $\mathbf{I} = \begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$ and $\mathbf{J} = \begin{bmatrix} 5 & 1 \\ 0 & 5 \end{bmatrix}$ have the single eigenvalue $\lambda = 5$ (doubled). For each, find the dimension of the eigenspace for $\lambda = 5$. Explain in words why one of these has a full set of eigenvectors and the other does not, and connect this to the shear of Section 23.6.

Exercise 23.10 (Trace/determinant proof)**.** Prove that for any 2×2 matrix $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the characteristic polynomial equals $\lambda^2 - (\operatorname{tr}\mathbf{A})\lambda + \det\mathbf{A}$, and deduce that the two eigenvalues satisfy $\lambda_1 + \lambda_2 = \operatorname{tr}\mathbf{A}$ and $\lambda_1\lambda_2 = \det\mathbf{A}$. Then show, as a consequence, that a real 2×2 matrix has complex (non-real) eigenvalues exactly when $(\operatorname{tr}\mathbf{A})^2 < 4\det\mathbf{A}$.

Exercise 23.11 (The Markov long run – challenge)**.** Let $\mathbf{M} = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix}$ (each column sums to 1; it is a probability transition matrix). **(a)** Show $\lambda = 1$ is an eigenvalue and find its eigenvector $\mathbf{q}$; scale $\mathbf{q}$ so its entries sum to 1. **(b)** Find the other eigenvalue and confirm $|\lambda| < 1$. **(c)** Using the eigendecomposition idea of Section 23.5, argue that for *any* starting probability vector $\mathbf{p}_0$, the sequence $\mathbf{M}^n\mathbf{p}_0$ converges to $\mathbf{q}$. Why does the eigenvalue 1 "win," and what role does the second eigenvalue play in how fast you get there?

Exercise 23.12 (Complex eigenvectors – challenge)**.** For the 90° rotation $\mathbf{Q} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ with eigenvalue $\lambda = i$, solve $(\mathbf{Q} - i\mathbf{I})\mathbf{x} = \mathbf{0}$ over the complex numbers to find a complex eigenvector. Then take the eigenvalue $\lambda = -i$ and show its eigenvector is the complex conjugate of the first. (This conjugate-pairing of complex eigenvectors mirrors the conjugate-pairing of complex eigenvalues for real matrices.)

Exercise 23.13 (Build a matrix from eigendata)**.** Construct a 2×2 matrix whose eigenvalues are 3 and -1 with eigenvectors $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ respectively. *Hint:* you want $\mathbf{A}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = 3\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\mathbf{A}\begin{bmatrix} 1 \\ -1 \end{bmatrix} = -\begin{bmatrix} 1 \\ -1 \end{bmatrix}$; set up and solve for the four entries, or anticipate the next chapter and build $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$ with the eigenvectors as the columns of $\mathbf{X}$.

Chapter 24

Diagonalization

In the last chapter you learned to find a matrix's eigenvectors: the special directions a machine does not turn, only stretch. An eigenvector $\mathbf{v}$ with eigenvalue λ satisfies the one equation that this whole chapter rests on,

$$\mathbf{A}\mathbf{v} = \lambda\,\mathbf{v}.$$

That is one good fact about *one* direction. The follow-up question is the one that pays off: *what if we use the eigenvectors as our coordinate axes?* Instead of describing space with the standard grid $\mathbf{e}_1, \mathbf{e}_2, \ldots$, we lay down a new coordinate system built entirely from the directions preserved by $\mathbf{A}$.

In that coordinate system the matrix $\mathbf{A}$, which coupled the coordinates in standard axes, scales each axis by its own factor. The transformation simplifies into independent scalar operations. An **eigenbasis** is simply a coordinate grid built out of eigenvectors rather than standard perpendicular axes. Representing our matrix in this natural coordinate frame is called **diagonalization**—re-expressing the transformation so that all its action is concentrated on the main diagonal as pure, uncoupled stretches.

Diagonalization is a foundational tool of applied linear algebra. It computes $\mathbf{A}^{100}$ without a hundred multiplications, solves recurrences like Fibonacci in closed form, and explains systems of differential equations. Later, in Chapter 29, this idea broadens into the **singular value decomposition** (SVD)—a factorization that discovers optimal perpendicular stretch axes for *any* matrix, even rectangular ones or matrices that cannot be diagonalized.

We build the picture geometrically first, then write down the formula $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$ and examine its applications: powers, the geometry of working in the eigenbasis, the matrices that cannot be diagonalized, and a closed-form Fibonacci formula.

24.1 The Big Idea: Work in the Eigenbasis

Start with the picture before the formula. Suppose a 2×2 matrix $\mathbf{A}$ has two eigenvectors $\mathbf{v}_1$ and $\mathbf{v}_2$ pointing in different directions, with eigenvalues λ_1 and λ_2. Because they point in different directions, these two arrows are a *basis*: every vector $\mathbf{x}$ in the plane can be written as a combination of them,

$$\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2.$$

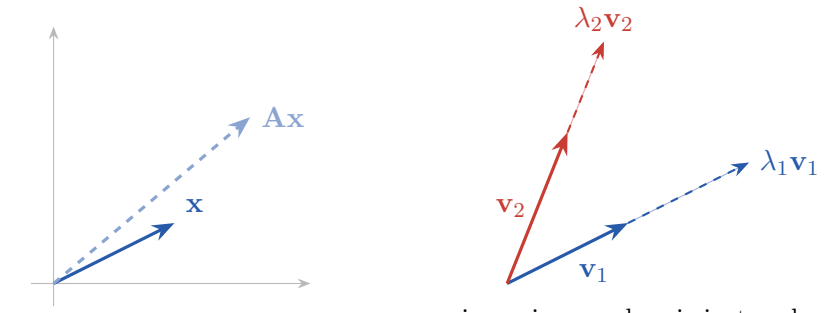
Now feed $\mathbf{x}$ to the machine. Linearity does all the work:

$$\mathbf{Ax} = \mathbf{A}(c_1\mathbf{v}_1 + c_2\mathbf{v}_2) = c_1\,\mathbf{Av}_1 + c_2\,\mathbf{Av}_2 = c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2.$$

In the eigenvector coordinates (c_1, c_2), the matrix multiplied the first coordinate by λ_1 and the second by λ_2. No mixing. The complicated machine, viewed in the right coordinates, is just two independent stretches.

Intuition

A general matrix mixes coordinates together: the new x depends on both the old x and the old y. Its eigenvectors are the directions where this coupling stops. If you represent space in terms of eigenvectors – using them as coordinate axes – then $\mathbf{A}$ acts as separate scaling factors along each axis, multiplying by the corresponding eigenvalue λ_i. **Diagonalization is the act of choosing the coordinate system in which the transformation is simplest.**



standard view: $\mathbf{A}$ mixes axes

eigen view: each axis just scales

Figure 24.1: Left: in the standard grid, $\mathbf{A}$ couples the x and y coordinates together – a generic $\mathbf{x}$ is sent to a tilted, rescaled $\mathbf{Ax}$ with coupled components. Right: along the eigenvectors $\mathbf{v}_1, \mathbf{v}_2$ the action is pure scaling – the $\mathbf{v}_1$ axis stretches by λ_1, the $\mathbf{v}_2$ axis by λ_2. The same linear map is decoupled in eigen-coordinates.

Focus

The slogan for the whole chapter: **eigenvectors are the axes in which a matrix is diagonal.** Find those axes and the matrix loses all its complexity.

24.2 The Factorization $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$

Now the algebra. The trick is to package all the eigenvectors into the columns of a single matrix and watch what $\mathbf{A}$ does to them all at once.

Suppose $\mathbf{A}$ is $n \times n$ and has n eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$. Build the **eigenvector matrix**

$$\mathbf{S} = \left[\, \mathbf{v}_1 \,\middle|\, \mathbf{v}_2 \,\middle|\, \cdots \,\middle|\, \mathbf{v}_n \,\right],$$

putting one eigenvector in each column. Now multiply $\mathbf{A}$ by $\mathbf{S}$. Matrix times matrix works column by column – the j-th column of $\mathbf{AS}$ is $\mathbf{A}$ times the j-th column of $\mathbf{S}$, which is $\mathbf{Av}_j$. But $\mathbf{v}_j$ is an

eigenvector, so $\mathbf{A}\mathbf{v}_j = \lambda_j \mathbf{v}_j$. Therefore

$$\mathbf{AS} = \left[\, \lambda_1 \mathbf{v}_1 \;\middle|\; \lambda_2 \mathbf{v}_2 \;\middle|\; \cdots \;\middle|\; \lambda_n \mathbf{v}_n \,\right].$$

Each column got scaled by its own eigenvalue. That right-hand side is exactly $\mathbf{S}$ with its columns rescaled. You rescale the columns of any matrix by multiplying *on the right* by a diagonal matrix. For example, in two dimensions,

$$\begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} \lambda_1 \mathbf{v}_1 & \lambda_2 \mathbf{v}_2 \end{bmatrix}.$$

In n dimensions, let

$$\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{bmatrix}.$$

Then $\mathbf{S\Lambda}$ takes column j of $\mathbf{S}$ and multiplies it by λ_j—precisely the right-hand side above. That proves the central identity

$$\boxed{\mathbf{AS} = \mathbf{S\Lambda}.}$$

Common trap

Order matters. $\mathbf{S\Lambda}$ (diagonal on the *right*) scales the *columns* of $\mathbf{S}$. $\mathbf{\Lambda S}$ (diagonal on the *left*) would scale the *rows* – a different matrix entirely. The eigenvalue must multiply the eigenvector it belongs to, so it has to sit on the right: $\mathbf{AS} = \mathbf{S\Lambda}$, not $\mathbf{\Lambda S}$.

So far we needed no inverse. The inverse enters only when we want to *solve* for $\mathbf{A}$. If the eigenvectors are independent, then $\mathbf{S}$ is invertible (its columns are a basis), and we may multiply $\mathbf{AS} = \mathbf{S\Lambda}$ on the right by $\mathbf{S}^{-1}$ to isolate $\mathbf{A}$, or on the left by $\mathbf{S}^{-1}$ to isolate $\mathbf{\Lambda}$:

$$\boxed{\mathbf{A} = \mathbf{S}\,\mathbf{\Lambda}\,\mathbf{S}^{-1}} \qquad \text{and} \qquad \boxed{\mathbf{\Lambda} = \mathbf{S}^{-1}\,\mathbf{A}\,\mathbf{S}.}$$

Intuition

Pause and think: what if an eigenvalue is zero? Does $\lambda = 0$ prevent diagonalization? Not at all. An eigenvalue of zero simply means that $\mathbf{A}$ is singular (non-invertible)—it collapses that eigenvector's direction to $\mathbf{0}$, so $\mathbf{\Lambda}$ has a 0 on its diagonal. But as long as $\mathbf{A}$ possesses n linearly independent eigenvectors, the matrix $\mathbf{S}$ is completely invertible and the factorization $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$ holds without hesitation. Never confuse an invertible *transformation* $\mathbf{A}$ with an invertible *coordinate change* $\mathbf{S}$.

Definition 24.1 (Diagonalizable matrix)**.** An $n \times n$ matrix $\mathbf{A}$ is **diagonalizable** if it can be written $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$ for some invertible $\mathbf{S}$ and diagonal $\mathbf{\Lambda}$. Equivalently, $\mathbf{A}$ is diagonalizable if and only if it has n linearly independent eigenvectors. Those eigenvectors are the columns of $\mathbf{S}$; the eigenvalues are the diagonal entries of $\mathbf{\Lambda}$, in the matching order.

Formal

Theorem (Diagonalization). An $n \times n$ matrix $\mathbf{A}$ is diagonalizable over $\mathbb{R}$ if and only if $\mathbb{R}^n$ has a basis consisting of eigenvectors of $\mathbf{A}$. In that case, writing those basis vectors as the columns of $\mathbf{S}$ and their eigenvalues on the diagonal of $\mathbf{\Lambda}$,

$$\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}.$$

The factorization is not unique: you may reorder the eigenpairs (permuting columns of $\mathbf{S}$ and the matching diagonal entries of $\mathbf{\Lambda}$ together) or rescale any eigenvector, and you still get $\mathbf{A}$ back.

Before/after Strang

Strang writes this factorization with the same letters: $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$, with $\mathbf{S}$ the eigenvector matrix and $\mathbf{\Lambda}$ (capital lambda) the eigenvalue matrix. He stresses, as we will, that "diagonalizing requires n independent eigenvectors." If you read this *before* Strang, the one thing to carry in: $\mathbf{S}$'s columns are eigenvectors and $\mathbf{\Lambda}$'s diagonal is eigenvalues, *in the same order*. If you read it *after*, notice we are deliberately separating two hypotheses Strang sometimes bundles: "has n eigenvalues" versus "has n independent eigenvectors." Only the second is what diagonalization needs.

24.2.1 A first worked example

Take a concrete matrix from start to finish.

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Step 1: eigenvalues. Solve $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$:

$$\det \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix} = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3) = 0,$$

so $\lambda_1 = 1$ and $\lambda_2 = 3$.

Step 2: eigenvectors. For $\lambda_1 = 1$, solve $(\mathbf{A} - \mathbf{I})\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \mathbf{v} = \mathbf{0} \implies v_1 + v_2 = 0 \implies \mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

For $\lambda_2 = 3$, solve $(\mathbf{A} - 3\mathbf{I})\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \mathbf{v} = \mathbf{0} \implies -v_1 + v_2 = 0 \implies \mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Step 3: assemble. Place eigenvectors as columns, eigenvalues on the diagonal:

$$\mathbf{S} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}, \qquad \mathbf{\Lambda} = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}.$$

The inverse of a 2×2 matrix $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ is $\frac{1}{ad-bc}\left[\begin{smallmatrix} d & -b \\ -c & a \end{smallmatrix}\right]$, and here $ad - bc = (1)(1) - (1)(-1) = 2$, so

$$\mathbf{S}^{-1} = \frac{1}{2}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}.$$

Step 4: check. Multiply $\mathbf{S\Lambda S}^{-1}$ back together and confirm we recover $\mathbf{A}$:

$$\mathbf{S\Lambda} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ -1 & 3 \end{bmatrix},$$

$$\mathbf{S\Lambda S}^{-1} = \begin{bmatrix} 1 & 3 \\ -1 & 3 \end{bmatrix} \cdot \frac{1}{2}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \mathbf{A}. \checkmark$$

Compute it yourself

Diagonalize $\mathbf{B} = \left[\begin{smallmatrix} 3 & 0 \\ 1 & 2 \end{smallmatrix}\right]$ by hand. It is lower triangular, so read the eigenvalues straight off the diagonal: $\lambda = 3$ and $\lambda = 2$. Find one eigenvector for each (you should get $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ for $\lambda = 3$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ for $\lambda = 2$), assemble $\mathbf{S}$ and $\mathbf{\Lambda}$, and multiply $\mathbf{S\Lambda S}^{-1}$ back to verify. Doing this once by hand cements the recipe forever.

24.3 Why Powers Become Easy

This is the payoff. Suppose you need $\mathbf{A}^k$ for large k – maybe you are iterating a process a thousand times. Multiplying $\mathbf{A}$ by itself a thousand times is slow and numerically dangerous. Diagonalization makes it a one-liner.

Watch the inverses cancel. Square the factorization:

$$\mathbf{A}^2 = (\mathbf{S\Lambda S}^{-1})(\mathbf{S\Lambda S}^{-1}) = \mathbf{S\Lambda}\underbrace{(\mathbf{S}^{-1}\mathbf{S})}_{=\mathbf{I}}\mathbf{\Lambda S}^{-1} = \mathbf{S\Lambda}^2\mathbf{S}^{-1}.$$

The interior $\mathbf{S}^{-1}\mathbf{S}$ collapses to the identity. Do it k times and every interior pair collapses, leaving

$$\boxed{\mathbf{A}^k = \mathbf{S}\,\mathbf{\Lambda}^k\,\mathbf{S}^{-1}.}$$

And raising a *diagonal* matrix to a power is trivial – you just raise each diagonal entry to that power, because a diagonal matrix times itself only multiplies corresponding entries:

$$\mathbf{\Lambda}^k = \begin{bmatrix} \lambda_1^k & & \\ & \ddots & \\ & & \lambda_n^k \end{bmatrix}.$$

No matrix multiplication chain at all – just n scalar powers, then two matrix products to sandwich them back. The hard part (the eigenvectors) is computed once and reused for every k.

Intuition

Think of $\mathbf{S}^{-1}$ as "translate into eigen-language," $\mathbf{\Lambda}^k$ as "do the simple thing k times – just stretch each axis," and $\mathbf{S}$ as "translate back to ordinary coordinates." Powers are hard in ordinary coordinates and trivial in eigen-coordinates, so we step into that language, do the easy thing, and step back out.

Example 24.1 (A power computed by hand)**.** Use our $\mathbf{A} = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ from above to find $\mathbf{A}^{10}$ without ten multiplications. We have $\lambda_1 = 1$, $\lambda_2 = 3$, so $\mathbf{\Lambda}^{10} = \left[\begin{smallmatrix} 1^{10} & 0 \\ 0 & 3^{10} \end{smallmatrix}\right] = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 59049 \end{smallmatrix}\right]$. Then

$$\mathbf{A}^{10} = \mathbf{S}\mathbf{\Lambda}^{10}\mathbf{S}^{-1} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 59049 \end{bmatrix} \frac{1}{2} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}.$$

Carry it out: $\mathbf{S}\mathbf{\Lambda}^{10} = \left[\begin{smallmatrix} 1 & 59049 \\ -1 & 59049 \end{smallmatrix}\right]$, and

$$\mathbf{A}^{10} = \frac{1}{2} \begin{bmatrix} 1 & 59049 \\ -1 & 59049 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 59050 & 59048 \\ 59048 & 59050 \end{bmatrix} = \begin{bmatrix} 29525 & 29524 \\ 29524 & 29525 \end{bmatrix}.$$

Notice the structure the eigenvalues predicted: the entries are huge (driven by 3^{10}) and the matrix is nearly $\frac{3^{10}}{2}$ times the rank-one "all-ones" pattern, because the $\lambda = 3$ direction dominates. That dominance is the seed of the power method and of the long-run behavior of Markov chains.

Why this matters

A power $\mathbf{A}^k$ looks like it should grow in complexity with k – more and more multiplications, entries spiraling out of control. Diagonalization shows that nothing complicated is happening at all: in its eigenbasis, the matrix acts as n independent numbers λ_i, each raised to the k. All the apparent complexity lived in the *choice of coordinates*, not in the matrix's actual behavior. Finding the eigenbasis is finding the coordinates in which the behavior is simple.

24.3.1 The long-run behavior you can read off

Because $\mathbf{A}^k$ scales the $\mathbf{v}_i$ direction by λ_i^k, the *largest* $|\lambda_i|$ wins as $k \to \infty$. If one eigenvalue is bigger in magnitude than all the others, then for large k almost every vector, when hit repeatedly by $\mathbf{A}$, lines up with that dominant eigenvector. Write any starting vector in the eigenbasis, $\mathbf{x} = \sum_i c_i \mathbf{v}_i$; then

$$\mathbf{A}^k \mathbf{x} = \sum_i c_i \, \lambda_i^k \, \mathbf{v}_i = \lambda_1^k \Big(c_1 \mathbf{v}_1 + \sum_{i \geq 2} c_i \big(\tfrac{\lambda_i}{\lambda_1}\big)^k \mathbf{v}_i \Big).$$

If $|\lambda_1| > |\lambda_i|$ for all $i \geq 2$, every ratio $(\lambda_i/\lambda_1)^k \to 0$, and the bracket collapses to $c_1 \mathbf{v}_1$. The process forgets where it started and aligns with $\mathbf{v}_1$ (Figure 24.2).

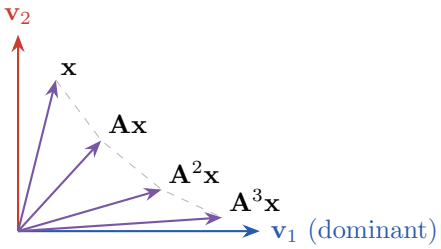


Figure 24.2: Dominant-eigenvalue behavior. Repeatedly applying $\mathbf{A}$ shrinks the $\mathbf{v}_2$ component (by the factor λ_2/λ_1 each step) while the $\mathbf{v}_1$ component grows fastest, so the images swing toward the dominant eigen-direction $\mathbf{v}_1$. This is the engine behind PageRank, the power method, and Markov-chain convergence.

Modern application

Google's PageRank and Markov chains. A web-surfing model is a giant matrix $\mathbf{A}$ whose largest eigenvalue is $\lambda_1 = 1$, with all other $|\lambda_i| < 1$. The formula above says $\mathbf{A}^k\mathbf{x}$ converges, for *any* starting distribution $\mathbf{x}$, to a multiple of the dominant eigenvector $\mathbf{v}_1$ – the **stationary distribution**. That eigenvector *is* the PageRank vector: the long-run fraction of time a random surfer spends on each page. Diagonalization does more than simplify computations here: it reveals *why* the ranking exists and converges uniquely. The "second eigenvalue" $|\lambda_2|$ controls how fast the algorithm converges – smaller $|\lambda_2|$ means PageRank settles in fewer iterations. The same mathematics governs how quickly a shuffled deck becomes random and how fast an MCMC sampler in modern Bayesian statistics mixes.

24.4 The Geometry: Three Clean Steps

The formula $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$ is best understood as a **sentence read right to left**, because matrices act on a vector from the right. To compute $\mathbf{A}\mathbf{x}$ you first apply $\mathbf{S}^{-1}$, then $\mathbf{\Lambda}$, then $\mathbf{S}$:

$$\mathbf{A}\mathbf{x} = \mathbf{S}\left(\mathbf{\Lambda}\left(\mathbf{S}^{-1}\mathbf{x}\right)\right).$$

Each factor has a clean geometric job:

- $\mathbf{S}^{-1}$: **change into the eigenbasis.** Any vector $\mathbf{x}$ can be written as a mixture of eigenvectors: $\mathbf{x} = c_1\mathbf{v}_1 + \cdots + c_n\mathbf{v}_n = \mathbf{S}\mathbf{c}$, where $\mathbf{c} = \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix}$ is the coordinate vector in the eigenbasis. Multiplying by $\mathbf{S}^{-1}$ solves for this mixture: $\mathbf{c} = \mathbf{S}^{-1}\mathbf{x}$. It translates standard coordinates into eigen-coordinates.
- $\mathbf{\Lambda}$: **stretch each eigen-axis.** In those coordinates, scale the i-th coordinate by λ_i: $\mathbf{\Lambda}\mathbf{c} = \begin{bmatrix} \lambda_1 c_1 \\ \vdots \\ \lambda_n c_n \end{bmatrix}$. Simple, decoupled, no mixing.
- $\mathbf{S}$: **change back to standard coordinates.** It re-assembles the standard vector from its (now-stretched) eigen-coordinates: $\mathbf{S}(\mathbf{\Lambda}\mathbf{c}) = \sum(c_i\lambda_i)\mathbf{v}_i = \mathbf{A}\mathbf{x}$.

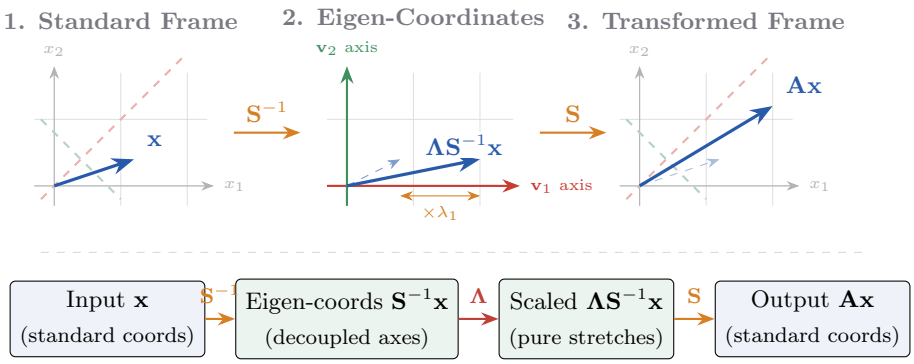


Figure 24.3: Diagonalization $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$ as a multi-stage geometric transformation. *Top lane:* $\mathbf{S}^{-1}$ re-orients coordinate axes onto the natural eigenvector directions; $\mathbf{\Lambda}$ applies simple independent scalings λ_i along those axes without cross-talk; and $\mathbf{S}$ translates the stretched vector back into standard coordinates. *Bottom lane:* the corresponding algebraic pipeline. Any complex linear transformation is simply pure axis stretching viewed in a tilted coordinate system.

This three-step sequence is an instance of a **change of basis**. In Chapter 31, we develop the full theory of changing coordinate systems for abstract vector spaces, showing how any linear map T transforms between arbitrary bases via a transition matrix $\mathbf{P}$ as $\mathbf{P}^{-1}\mathbf{A}\mathbf{P}$. Here, choosing the *eigenbasis* decouples the transformation completely, leaving the diagonal matrix $\mathbf{\Lambda} = \mathbf{S}^{-1}\mathbf{A}\mathbf{S}$. The matrix $\mathbf{A}$ in standard coordinates and its diagonal form $\mathbf{\Lambda}$ in eigen-coordinates describe the exact same transformation in two different coordinate frames.

Connection to other math

Similar matrices and change of basis. Two matrices $\mathbf{A}$ and $\mathbf{B}$ are called **similar** if $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ for some invertible $\mathbf{P}$. Similarity represents the same linear map expressed in a different basis. Diagonalization is the case where one of the similar matrices is diagonal. Similar matrices *share* their eigenvalues, their determinant ($\det \mathbf{A} = \prod \lambda_i$), their trace ($\operatorname{tr} \mathbf{A} = \sum a_{ii}$, the sum of the main-diagonal entries, which also equals $\sum \lambda_i$), and their rank—these are intrinsic properties of the underlying linear map. In Chapter 28 this analysis extends to the Jordan canonical form, which determines the simplest matrix representation when a complete basis of eigenvectors does not exist.

Walking question

Consider why the determinant of $\mathbf{A}$ equals the product of its eigenvalues, and the trace equals their sum. From $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$, the determinant satisfies $\det(\mathbf{A}) = \det(\mathbf{S})\det(\mathbf{\Lambda})\det(\mathbf{S}^{-1}) = \det(\mathbf{\Lambda}) = \prod \lambda_i$; for the trace, recall the cyclic property $\operatorname{tr}(\mathbf{X}\mathbf{Y}) = \operatorname{tr}(\mathbf{Y}\mathbf{X})$, so $\operatorname{tr}(\mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}) = \operatorname{tr}(\mathbf{\Lambda}\mathbf{S}^{-1}\mathbf{S}) = \operatorname{tr}(\mathbf{\Lambda}) = \sum \lambda_i$. Both invariants evaluate directly on the diagonal entries of $\mathbf{\Lambda}$.

24.5 Which Matrices Diagonalize? Independent Eigenvectors

Everything above needed one thing: $\mathbf{S}$ must be invertible, i.e. the eigenvectors must be *linearly independent*. When does that happen? The key fact does most of the work for us.

Theorem 24.1 (Distinct eigenvalues give independent eigenvectors)**.** *Eigenvectors corresponding to* distinct *eigenvalues are linearly independent. In particular, if an $n \times n$ matrix has n distinct eigenvalues, it has n independent eigenvectors and is therefore diagonalizable.*

Sketch for two eigenvalues. Suppose $\mathbf{v}_1, \mathbf{v}_2$ have distinct eigenvalues $\lambda_1 \neq \lambda_2$ and some combination vanishes: $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 = \mathbf{0}$. Apply $\mathbf{A}$ to get $c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2 = \mathbf{0}$. Now subtract λ_1 times the first equation: the $\mathbf{v}_1$ term cancels, leaving $c_2(\lambda_2 - \lambda_1)\mathbf{v}_2 = \mathbf{0}$. Since $\lambda_2 - \lambda_1 \neq 0$ and $\mathbf{v}_2 \neq \mathbf{0}$, we must have $c_2 = 0$, and then the first equation forces $c_1 = 0$. No nontrivial combination vanishes, so the vectors are independent.

The same idea works for any number of distinct eigenvalues: if $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$, applying $(\mathbf{A} - \lambda_k\mathbf{I})$ annihilates the last vector $\mathbf{v}_k$ entirely while scaling each earlier $\mathbf{v}_j$ by the non-zero factor $(\lambda_j - \lambda_k)$. Repeating this filtering step isolates each coefficient c_j and forces it to be zero. □

So **distinct eigenvalues guarantee diagonalizability**. This is the common, comfortable case. But the converse is false, and reading it correctly will save you from two classic traps.

Common trap

Trap 1: repeated eigenvalues do not doom a matrix. You can have repeated eigenvalues and still be perfectly diagonalizable. The identity matrix $\mathbf{I}$ has eigenvalue 1 repeated n times, yet *every* vector is an eigenvector – it is already diagonal. A repeated eigenvalue is only a *warning* that you must go check whether you can find enough independent eigenvectors for it.
Trap 2: diagonalizability is about eigenVECTORS, not eigenVALUES. The question is never "are there n eigenvalues?" (counted with repetition there always are, over $\mathbb{C}$). The question is "are there n independent eigenVECTORS?" Those are different questions, and the gap between them is the whole subject of the next section.

Intuition

Pause and think: the flat projection. Imagine a matrix $\mathbf{P}$ that projects three-dimensional space onto the horizontal xy-plane, sending (x, y, z) to $(x, y, 0)$. What are its eigenvalues? Every vector already lying in the xy-plane is completely untouched, giving eigenvalue $\lambda = 1$. Every vector along the vertical z-axis is squashed to $\mathbf{0}$, giving eigenvalue $\lambda = 0$. The characteristic equation has $\lambda = 1$ repeated twice, yet its eigenspace is the entire two-dimensional xy-plane. Together with the one-dimensional line along the z-axis, we have $2 + 1 = 3$ independent eigenvectors in $\mathbb{R}^3$. Despite the repeated eigenvalue, $\mathbf{P}$ is still diagonalizable.

24.5.1 Algebraic versus geometric multiplicity

To say precisely when a repeated eigenvalue is harmless, we need two counts.

Definition 24.2 (Two multiplicities)**.** For an eigenvalue λ of $\mathbf{A}$:

- its **algebraic multiplicity** is how many times λ appears as a root of the characteristic polynomial $\det(\mathbf{A} - \lambda\mathbf{I})$;
- its **geometric multiplicity** is the dimension of its eigenspace $\text{null}(\mathbf{A} - \lambda\mathbf{I})$ – the number of independent eigenvectors you can actually find for λ.

Always $1 \leq$ (geometric) $\leq$ (algebraic).

The matrix is diagonalizable exactly when there is no shortfall anywhere – when geometric multiplicity equals algebraic multiplicity for every eigenvalue, so the eigenvectors add up to a full basis of n.

Formal

Diagonalizability criterion. $\mathbf{A}$ is diagonalizable (over a field containing all its eigenvalues) if and only if, for every eigenvalue λ,

$$\dim \operatorname{null}(\mathbf{A} - \lambda \mathbf{I}) \;=\; (\text{algebraic multiplicity of } \lambda).$$

Equivalently, the geometric multiplicities sum to n. When some eigenvalue has geometric multiplicity strictly less than its algebraic multiplicity, the eigenvectors fall short of a basis and the matrix is **defective** – it cannot be diagonalized.

24.6 Defective Matrices: When Diagonalization Fails

Some matrices simply do not have enough eigenvectors, and no cleverness can manufacture them. The smallest and most important example is the shear.

Example 24.2 (The shear that cannot be diagonalized)**.** Consider

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

It is upper triangular, so the eigenvalues are on the diagonal: $\lambda = 1$, twice. Algebraic multiplicity of $\lambda = 1$ is 2. Now hunt for eigenvectors by solving $(\mathbf{A} - \mathbf{I})\mathbf{v} = \mathbf{0}$:

$$\mathbf{A} - \mathbf{I} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \mathbf{0} \implies v_2 = 0.$$

The only solutions are multiples of $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$. The eigenspace is a single line – geometric multiplicity 1, but algebraic multiplicity 2. There is a shortfall of one. We can never assemble two independent eigenvectors, so $\mathbf{S}$ cannot be formed, and $\mathbf{A}$ is **defective**: not diagonalizable.

Why does this matrix resist? Geometrically a shear slides the plane sideways: the horizontal axis is pinned (that is the one eigenvector), but *no other direction is merely scaled* – every other vector gets tilted toward the horizontal. There is no second axis that the machine leaves pointing the same way, so there is no second eigenvector, and hence no eigenbasis.

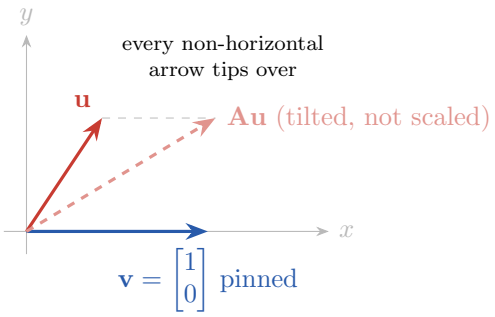


Figure 24.4: The shear $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ pins the horizontal axis (its one eigenvector) but tilts every other vector toward the horizontal – changing its *direction*, not merely its length. With only one eigendirection there is no eigenbasis, so the matrix is defective.

Intuition

A defective matrix is one that does something a diagonal matrix simply cannot: it has a direction that gets *rotated into* another, never purely scaled. Pure scaling along independent axes is the entire repertoire of a diagonal matrix, so any genuine "tipping over" that cannot be undone by a change of axes is the fingerprint of defectiveness. The repeated-eigenvalue shear is the canonical culprit.

Common trap

A repeated eigenvalue *may or may not* be defective – you must check. Compare these two, both with eigenvalue 5 doubled:

$$\begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix} \text{ (geom. mult. 2, diagonalizable)}, \qquad \begin{bmatrix} 5 & 1 \\ 0 & 5 \end{bmatrix} \text{ (geom. mult. 1, defective)}.$$

Same characteristic polynomial $(\lambda - 5)^2$, opposite fates. The off-diagonal 1 is what kills the second eigenvector. Never judge diagonalizability from the eigenvalues alone.

Remark 24.1 (There is still a "best possible" form). A defective matrix cannot be diagonalized, but it is not the end of the road. The **Jordan form** (Chapter 28) is the nearest substitute: it is *almost* diagonal, with the eigenvalues on the diagonal and a few stray 1's just above it – exactly one 1 for each missing eigenvector. The shear $\left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$ is its own Jordan form: that off-diagonal 1 records the shear left behind by the missing eigenvector.

Connection to other math

Defectiveness is fragile, and that is a story about calculus. Defective matrices are rare in the sense that an arbitrarily tiny perturbation breaks the repeated eigenvalue apart into distinct ones, instantly making the matrix diagonalizable. For instance $\left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$ is defective, but $\left[\begin{smallmatrix} 1 & 1 \\ 0 & 1+\varepsilon \end{smallmatrix}\right]$ has the two distinct eigenvalues 1 and $1 + \varepsilon$, hence two eigenvectors, for every $\varepsilon \neq 0$. As $\varepsilon \to 0$ those two eigenvectors swing toward each other and *collide*, and $\mathbf{S}^{-1}$ blows up. This is why diagonalization becomes numerically unstable near defective matrices: the eigenvectors become almost parallel and $\mathbf{S}$ becomes nearly singular. The cure that everyone

eventually reaches for is the singular value decomposition, which never has this problem, as we will see in Chapter 29.

24.7 Application: Solving the Fibonacci Recurrence

Diagonalization turns a recurrence – a rule that builds each term from the last ones – into a matrix power, which we now know how to compute exactly. The most famous recurrence in the world is Fibonacci:

$$F_0 = 0, \quad F_1 = 1, \qquad F_{k+1} = F_k + F_{k-1}.$$

So $0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots$. Is there a *closed form* for F_k – a formula that jumps straight to the thousandth term without computing the first nine hundred ninety-nine?

Step 1: make it a matrix. The trick for any recurrence is to track a little *state vector* holding enough recent terms to take the next step. Here, two terms suffice. Let $\mathbf{u}_k = \begin{bmatrix} F_{k+1} \\ F_k \end{bmatrix}$. The recurrence says

$$\begin{bmatrix} F_{k+1} \\ F_k \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_k \\ F_{k-1} \end{bmatrix}, \qquad \text{i.e.} \qquad \mathbf{u}_k = \mathbf{A}\,\mathbf{u}_{k-1}, \quad \mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

Check the top row: $F_{k+1} = 1 \cdot F_k + 1 \cdot F_{k-1}$, the recurrence itself. The bottom row just copies F_k down so it is ready to be the "previous term" next time. Iterating from $\mathbf{u}_0 = \begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$,

$$\mathbf{u}_k = \mathbf{A}^k\,\mathbf{u}_0.$$

A closed form for F_k is now a closed form for $\mathbf{A}^k$ – and we know exactly how to get that.

Step 2: diagonalize A. Characteristic polynomial:

$$\det \begin{bmatrix} 1-\lambda & 1 \\ 1 & -\lambda \end{bmatrix} = (1-\lambda)(-\lambda) - 1 = \lambda^2 - \lambda - 1 = 0.$$

The roots are the golden ratio and its conjugate:

$$\lambda_1 = \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618, \qquad \lambda_2 = \psi = \frac{1-\sqrt{5}}{2} \approx -0.618.$$

They are distinct, so $\mathbf{A}$ is diagonalizable. For eigenvectors, from $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ the bottom row gives $v_1 = \lambda v_2$, so an eigenvector for λ is $\begin{bmatrix} \lambda \\ 1 \end{bmatrix}$. Thus

$$\mathbf{S} = \begin{bmatrix} \varphi & \psi \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{\Lambda} = \begin{bmatrix} \varphi & 0 \\ 0 & \psi \end{bmatrix}.$$

Step 3: extract the formula. We want F_k, the bottom entry of $\mathbf{u}_k = \mathbf{S}\mathbf{\Lambda}^k\mathbf{S}^{-1}\mathbf{u}_0$. Rather than grind the full product, use the cleaner eigenbasis route: expand the start vector $\mathbf{u}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ in

eigenvectors, $\mathbf{u}_0 = c_1 \begin{bmatrix} \varphi \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} \psi \\ 1 \end{bmatrix}$. The two equations are $c_1\varphi + c_2\psi = 1$ and $c_1 + c_2 = 0$, giving $c_2 = -c_1$ and $c_1(\varphi - \psi) = 1$. Since $\varphi - \psi = \sqrt{5}$, we get $c_1 = 1/\sqrt{5}$ and $c_2 = -1/\sqrt{5}$. Then

$$\mathbf{u}_k = \mathbf{A}^k \mathbf{u}_0 = c_1 \varphi^k \begin{bmatrix} \varphi \\ 1 \end{bmatrix} + c_2 \psi^k \begin{bmatrix} \psi \\ 1 \end{bmatrix}.$$

Reading the bottom entry (F_k) gives the celebrated **Binet formula**:

$$\boxed{F_k = \frac{\varphi^k - \psi^k}{\sqrt{5}} = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^k - \left(\frac{1-\sqrt{5}}{2} \right)^k \right].}$$

Example 24.3 (Sanity check on small k)**.** A formula full of irrational $\sqrt{5}$'s had better produce integers. Test $k = 5$. With $\varphi^5 \approx 11.0902$ and $\psi^5 \approx -0.0902$,

$$F_5 = \frac{11.0902 - (-0.0902)}{\sqrt{5}} = \frac{11.1803}{2.2360} = 5. \checkmark$$

Directly checking the sequence $0, 1, 1, 2, 3, 5, \ldots$ confirms $F_5 = 5$. The irrational terms cancel algebraically, producing an exact integer for every k as a consequence of the pairing between conjugate eigenvalues.

Intuition

Binet's formula *is* the eigenbasis decomposition $\mathbf{A}^k\mathbf{u}_0 = \sum_i c_i \lambda_i^k \mathbf{v}_i$ written out for one specific matrix. The Fibonacci numbers are a blend of two pure exponential modes, φ^k and ψ^k. Since $|\psi| \approx 0.618 < 1$, the ψ^k mode shrinks to nothing, so for large k the Fibonacci numbers are essentially $\varphi^k/\sqrt{5}$ – they grow by a factor of φ each step (Figure 24.5). That is precisely the "dominant-eigenvalue" long-run behavior of Section 24.3.1, and it is why consecutive Fibonacci ratios F_{k+1}/F_k march toward the golden ratio.

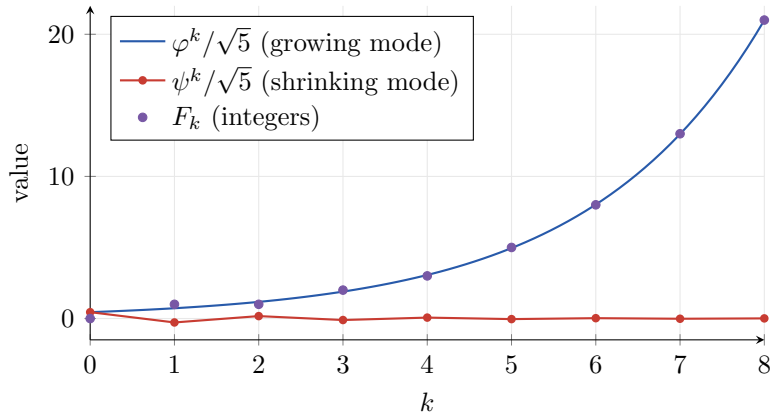


Figure 24.5: Fibonacci as two eigen-modes. $F_k = \varphi^k/\sqrt{5} - \psi^k/\sqrt{5}$ is the difference of a fast-growing mode $\varphi^k/\sqrt{5}$ (the dominant eigenvalue) and a tiny alternating mode $\psi^k/\sqrt{5}$ that decays to nothing. The integers F_k hug the growing mode almost perfectly – which is why F_k is the nearest integer to $\varphi^k/\sqrt{5}$.

Modern application

Every linear recurrence yields to this. The Fibonacci move is completely general. Any constant-coefficient linear recurrence $a_{k+1} = p_1 a_k + p_2 a_{k-1} + \cdots + p_d a_{k-d+1}$ becomes $\mathbf{u}_k = \mathbf{A}\mathbf{u}_{k-1}$ with a $d \times d$ **companion matrix** $\mathbf{A}$—a matrix built with 1's on a subdiagonal to step historical terms down and the recurrence coefficients along a row to assemble the next value. Diagonalizing $\mathbf{A}$ produces a closed form as a sum of λ_i^k modes. This is how engineers analyze the stability of discrete-time control systems and digital audio filters (stable whenever every $|\lambda_i| < 1$), how ecologists project age-structured populations with **Leslie matrices** (matrices tracking birth and survival rates across age classes), and how computer scientists determine the runtime of divide-and-conquer algorithms from their recurrence relations.

24.8 The Recipe and the Cautions

Pulling the chapter together, here is the full procedure and the places people stumble.

Focus

To diagonalize an $n \times n$ matrix $\mathbf{A}$:

1. Find all eigenvalues: solve $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$.
2. For each λ, find a basis of its eigenspace $\text{null}(\mathbf{A} - \lambda\mathbf{I})$.
3. Count: if the eigenvectors total n (every geometric multiplicity meets its algebraic multiplicity), you can diagonalize. If not, $\mathbf{A}$ is defective – stop.
4. Put the n eigenvectors as columns of $\mathbf{S}$, their eigenvalues (in the same order) on the diagonal of $\mathbf{\Lambda}$. Then $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$, and $\mathbf{A}^k = \mathbf{S}\mathbf{\Lambda}^k\mathbf{S}^{-1}$.

Common trap

Four diagonalization traps, collected.

- **Order mismatch.** The j-th column of $\mathbf{S}$ and the j-th diagonal entry of $\mathbf{\Lambda}$ must be a matched eigenpair. Shuffle one without the other and $\mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1} \neq \mathbf{A}$.
- **Confusing the two multiplicities.** "Three eigenvalues" is not the test; "three independent eigenvectors" is.
- **Forgetting complex eigenvalues.** A rotation like $\left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$ has *no real* eigenvectors (it turns every real vector), so it is not diagonalizable over $\mathbb{R}$ – but it *is* diagonalizable over $\mathbb{C}$, with eigenvalues $\pm i$. Always note which field you are working over.
- **$\mathbf{\Lambda}^k$ shortcut applies only to diagonal $\mathbf{\Lambda}$.** You may raise diagonal entries to the k individually *only because* $\mathbf{\Lambda}$ is diagonal. You cannot do this to $\mathbf{A}$ directly.

Before/after Strang

After Strang, the natural next questions are: which matrices are *guaranteed* a full set of orthogonal eigenvectors, so that $\mathbf{S}^{-1}$ is just $\mathbf{S}^{\mathsf{T}}$? That is the **spectral theorem** for sym-

metric matrices—the guarantee that every real symmetric matrix has a full set of mutually perpendicular eigenvectors with real eigenvalues—the cleanest diagonalization of all, coming in Chapter 26. And what replaces $\mathbf{S\Lambda S}^{-1}$ for rectangular or defective matrices, where eigenvectors fail us? That is the **singular value decomposition**, which extends coordinate factorizations to all matrices in Chapter 29. Diagonalization is the doorway to both.

History and connection

The drive to diagonalize matrices began with the celestial calculations of the eighteenth and nineteenth centuries. When astronomers like Pierre-Simon Laplace and Joseph-Louis Lagrange modeled the solar system, they confronted the problem of **secular perturbations**—the tiny, cumulative gravitational tugs planets exert on each other over millennia. These planetary interactions produced large systems of coupled differential equations. To determine whether the orbits of Jupiter and Saturn were eternally stable or destined to drift into collision, astronomers had to calculate states thousands of years into the future. Computing $\mathbf{A}^{100}$ by repeated hand multiplication meant ninety-nine dense matrix multiplications, compounding human arithmetic errors at each stage. Finding eigenvalues by solving for the roots of the characteristic polynomial was equally out of reach: by Niels Henrik Abel's 1824 theorem, polynomial equations of degree five and higher possess no general algebraic solution in radicals.

Equipped only with quill pens, logarithm tables, and pencil calculations, astronomers needed a direct computational path. In 1846, Carl Gustav Jacob Jacobi introduced an ingenious algorithm to diagonalize symmetric planetary matrices without calculating characteristic roots: **Jacobi rotations**, which iteratively rotate two coordinate axes at a time to systematically eliminate off-diagonal entries until the matrix relaxes into diagonal form. Arthur Cayley (1858) later organized these transformations into abstract matrix algebra, codifying diagonalization as the similarity factorization $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$.

This triumph sparked a prolonged controversy over physical systems. Physicists had long assumed that every physical matrix could be decoupled into independent vibrational modes. In 1868 and 1870, Karl Weierstrass and Camille Jordan proved this assumption false: matrices with repeated eigenvalues can be **defective**, lacking a complete basis of eigenvectors. Applied scientists initially dismissed the resulting Jordan canonical form as an artificial mathematical curiosity. In truth, defectiveness governs physical instabilities like resonance and aerodynamic flutter, where energy is transferred between modes and amplitudes grow as $t\,e^{\lambda t}$ rather than pure sinusoidal vibrations.

Diagonalization transformed applied mathematics by rendering matrix powers $\mathbf{A}^k = \mathbf{S\Lambda}^k\mathbf{S}^{-1}$ and matrix functions $f(\mathbf{A}) = \mathbf{S}f(\mathbf{\Lambda})\mathbf{S}^{-1}$ computable in a single step. It provided closed formulas for recurrences, enabled Markov-chain analysis, underpinned Google's PageRank algorithm, and became the foundation of quantum mechanics, where physical observables (energy, momentum) correspond to operators whose diagonal values are the only measurable physical states.

Yet fundamental questions remain active today. In computer arithmetic, when a matrix's eigenvectors lean almost parallel to one another, the eigenvector matrix $\mathbf{S}$ becomes **ill-conditioned**—meaning tiny rounding errors in the matrix entries cause huge errors in computed eigenvectors. For decades, numerical analysts sought a provably backward-stable algorithm that could diagonalize general non-symmetric matrices in nearly matrix-multiplication time $O(n^\omega)$. A theoretical solution appeared only in 2020 through randomized regular-

ization (Banks, Garza-Vargas, Kulkarni, and Srivastava). Meanwhile, mapping the **pseudospectrum**—the cloud of complex values that can become eigenvalues under infinitesimal perturbations—remains an essential tool and an active area of research in fluid dynamics and hydrodynamic stability.

24.8.1 Cost and intuition recap

Pause

The one-line intuition. Eigenvectors are the axes in which $\mathbf{A}$ is diagonal; $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$ says "translate into those axes, scale, translate back."

The cost. Diagonalizing once is real work: forming the characteristic polynomial and solving for eigenvalues is the expensive step (for an $n \times n$ matrix, a full eigendecomposition costs on the order of n^3 arithmetic operations, the same order as one matrix inversion). But that cost is paid *once*. After it,

- a single power $\mathbf{A}^k$ costs only n scalar powers λ_i^k plus two fixed matrix products to sandwich them – the price no longer grows with k, whereas naive repeated multiplication costs about k matrix products (order $k\,n^3$);
- the inverse is free: $\mathbf{A}^{-1} = \mathbf{S}\mathbf{\Lambda}^{-1}\mathbf{S}^{-1}$ just reciprocates the eigenvalues;
- the determinant ($\prod_i \lambda_i$) and trace ($\sum_i \lambda_i$) are read straight off $\mathbf{\Lambda}$.

The catch. You need n independent eigenvectors. Distinct eigenvalues guarantee them; repeated eigenvalues require a check (geometric multiplicity must meet algebraic multiplicity), and a defective matrix has no eigenbasis at all.

24.8.2 Looking forward: continuous systems and the matrix exponential

Throughout this chapter, our clock moved in discrete ticks: $k = 0, 1, 2, \ldots$. At each tick, the state was multiplied by $\mathbf{A}$, giving $\mathbf{u}_k = \mathbf{A}^k\mathbf{u}_0$. The eigenbasis solved that recurrence by decoupling the system into n separate, independent scalar equations $c_i\lambda_i^k$.

Continuous physical systems do not tick in discrete jumps; they flow smoothly. When the discrete recurrence $\mathbf{u}_{k+1} = \mathbf{A}\mathbf{u}_k$ is replaced by a continuous rate of change $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$, the exact same geometric strategy wins. Instead of integer powers $\mathbf{A}^k = \mathbf{S}\mathbf{\Lambda}^k\mathbf{S}^{-1}$, we form the **matrix exponential**

$$e^{\mathbf{A}t} = \mathbf{S}\,e^{\mathbf{\Lambda}t}\,\mathbf{S}^{-1} = \mathbf{S}\begin{bmatrix} e^{\lambda_1 t} & & \\ & \ddots & \\ & & e^{\lambda_n t} \end{bmatrix}\mathbf{S}^{-1}.$$

In Chapter 25, we will see how diagonalization solves coupled systems of differential equations, decoupling models from mechanics, circuits, and chemical kinetics into independent scalar exponential equations.

Exercises

Exercise 24.1 (Warm-up factorization)**.** Let $\mathbf{A} = \left[\begin{smallmatrix} 4 & 0 \\ 1 & 2 \end{smallmatrix}\right]$. Find its eigenvalues and eigenvectors, write down $\mathbf{S}$ and $\mathbf{\Lambda}$, and verify by direct multiplication that $\mathbf{S\Lambda S}^{-1} = \mathbf{A}$.

Exercise 24.2 (A power)**.** Using the diagonalization of $\mathbf{A} = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ from Section 24.2.1, compute $\mathbf{A}^5$ exactly via $\mathbf{S\Lambda}^5\mathbf{S}^{-1}$. (Check: the diagonal entries should be $\frac{1+3^5}{2} = 122$.)

Exercise 24.3 (Defective or not?)**.** For each matrix, decide whether it is diagonalizable over $\mathbb{R}$, giving the algebraic and geometric multiplicity of every eigenvalue:

$$\begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}, \qquad \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}, \qquad \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}, \qquad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Exercise 24.4 (Trace and determinant from $\mathbf{\Lambda}$)**.** Using $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$ together with $\operatorname{tr}(\mathbf{XY}) = \operatorname{tr}(\mathbf{YX})$ and $\det(\mathbf{XY}) = \det(\mathbf{X})\det(\mathbf{Y})$, prove carefully that $\operatorname{tr}\mathbf{A} = \sum_i \lambda_i$ and $\det\mathbf{A} = \prod_i \lambda_i$ for any diagonalizable $\mathbf{A}$.

Exercise 24.5 (A recurrence of your own)**.** Define $a_0 = 1$, $a_1 = 1$, and $a_{k+1} = 5a_k - 6a_{k-1}$. Write the state recurrence $\mathbf{u}_k = \mathbf{A}\mathbf{u}_{k-1}$ with $\mathbf{u}_k = \begin{bmatrix} a_{k+1} \\ a_k \end{bmatrix}$, diagonalize the 2×2 matrix $\mathbf{A}$ (its eigenvalues are nice integers), and derive a closed form $a_k = \alpha\, 2^k + \beta\, 3^k$. Determine α, β from the initial conditions.

Exercise 24.6 (Inverse from the factorization)**.** Show that if $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$ is invertible (all $\lambda_i \neq 0$), then $\mathbf{A}^{-1} = \mathbf{S\Lambda}^{-1}\mathbf{S}^{-1}$ with $\mathbf{\Lambda}^{-1} = \operatorname{diag}(1/\lambda_1, \dots, 1/\lambda_n)$. Conclude that $\mathbf{A}$ and $\mathbf{A}^{-1}$ share the same eigenvectors, with reciprocal eigenvalues.

Exercise 24.7 (Commuting matrices, shared eigenvectors)**.** Suppose $\mathbf{A}$ is diagonalizable with *distinct* eigenvalues and $\mathbf{AB} = \mathbf{BA}$. Show that every eigenvector of $\mathbf{A}$ is also an eigenvector of $\mathbf{B}$. (Hint: if $\mathbf{Av} = \lambda\mathbf{v}$, apply $\mathbf{A}$ to $\mathbf{Bv}$ and use commuting to show $\mathbf{Bv}$ lies in the same eigenspace, which is one-dimensional.) This is why commuting matrices can be "simultaneously diagonalized."

Exercise 24.8 (Building a matrix from its spectrum)**.** Construct a 2×2 matrix whose eigenvectors are $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ (eigenvalue 4) and $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (eigenvalue -1), by forming $\mathbf{S\Lambda S}^{-1}$ explicitly. Then verify your matrix really has those eigenpairs.

Exercise 24.9 (Why ψ^k is invisible)**.** From Binet's formula, prove that F_k is the *nearest integer* to $\varphi^k/\sqrt{5}$ for all $k \geq 0$. (Hint: bound $|\psi^k|/\sqrt{5} < \frac{1}{2}$.) Use this to argue that $F_{k+1}/F_k \to \varphi$, and connect this to the dominant-eigenvalue discussion of Section 24.3.1.

Exercise 24.10 (A 3×3 challenge)**.** Diagonalize

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Find all eigenvalues and a basis of each eigenspace. Is $\mathbf{A}$ diagonalizable? Identify the algebraic and geometric multiplicities of $\lambda = 2$, and explain in one sentence what the off-diagonal 1 does to your conclusion.

Chapter 25

Differential Equations and the Matrix Exponential

In the previous chapter, diagonalization uncoupled discrete matrix steps $\mathbf{u}_{k+1} = \mathbf{A}\mathbf{u}_k$ by finding coordinate axes where the matrix acts as independent dials. Now we ask the natural continuous question: what happens when the discrete ticks of a clock become a smooth, continuous flow of time?

Here is a sentence that sounds like it belongs in a calculus book, not a linear algebra book: *the rate of change of a quantity is proportional to the quantity itself.* Out of that one sentence falls the exponential function, the shape of a cooling cup of coffee, the swing of a frictionless pendulum, the growth of a population, the decay of a radioactive sample, and the fundamental structure of linear dynamical systems.

The reason linear algebra cares is that nature rarely tracks just one quantity. A predator population depends on a prey population, which depends back on the predator. A mixing tank's salt concentration depends on its neighbor's. A circuit's voltages and currents push on each other. The position and velocity of a vibrating mass feed each other. In every one of these, you have a *vector* of quantities $\mathbf{u}(t)$, and the rule for how it changes is a *matrix*:

$$\frac{d\mathbf{u}}{dt} = \mathbf{A}\,\mathbf{u}.$$

Eigenvalues and eigenvectors solve this equation directly. Everything you learned about $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ – which directions $\mathbf{A}$ leaves alone, how much it stretches along each – comes back transformed into the language of motion through time. The eigenvectors become the natural **modes** of the system – pure, independent motions where every component scales at the same rate along a fixed line without mixing with the others. The eigenvalues become growth rates and oscillation frequencies. Stability – whether the system settles, blows up, or spins forever – is read straight off the signs of the eigenvalues.

Over all of it sits one object, the **matrix exponential** $e^{\mathbf{A}t}$. It is the true matrix generalization of the scalar e^{at}, it packages the whole solution into a single propagator (a linear transformation that advances any state forward by time t), and it is where diagonalization, calculus, and geometry meet. By the end of the chapter $e^{\mathbf{A}t}$ should look less like a formula to memorize and more like the inevitable answer the scalar case was pointing at all along.

25.1 The One-Dimensional Story First

Before any matrices, nail down the scalar case, because every later formula is a dressed-up version of it.

Suppose a single number $u(t)$ changes at a rate proportional to its current size:

$$\frac{du}{dt} = a\,u, \qquad u(0) = u_0.$$

Read it out loud: the bigger u is, the faster it grows (if $a > 0$) or the faster it shrinks (if $a < 0$). What function has this self-reinforcing property? You already know it. The exponential. We can even *derive* it instead of guessing. Separate the variables:

$$\frac{du}{u} = a\,dt \implies \int \frac{du}{u} = \int a\,dt \implies \ln|u| = at + C.$$

Exponentiate both sides and absorb the constant into u_0:

$$\boxed{u(t) = u_0\,e^{at}.}$$

Intuition

The number a is a **growth rate**. If $a > 0$, u runs away to infinity. If $a < 0$, u decays to zero. If $a = 0$, u just sits at u_0 forever. The sign of a alone determines the long-term behavior, while $|a|$ sets the rate. In the matrix case, the sign of the real part of each eigenvalue determines whether its mode grows or decays.

There is a second way to write e^{at} that will matter enormously in a moment. Forget the antiderivative and ask: what power series, when you differentiate it, reproduces itself times a? (For a refresher on Taylor series and term-by-term differentiation, see Section 39.3 in the Chapter 39 appendix.) Try

$$e^{at} = 1 + at + \frac{(at)^2}{2!} + \frac{(at)^3}{3!} + \cdots = \sum_{k=0}^{\infty} \frac{(at)^k}{k!}.$$

Differentiate term by term:

$$\frac{d}{dt}\left[\sum_{k=0}^{\infty} \frac{a^k t^k}{k!}\right] = \sum_{k=1}^{\infty} \frac{a^k\,k\,t^{k-1}}{k!} = \sum_{k=1}^{\infty} \frac{a^k t^{k-1}}{(k-1)!} = a\sum_{j=0}^{\infty} \frac{a^j t^j}{j!} = a\,e^{at}.$$

So the power series *is* the solution, and it satisfies $\frac{d}{dt}e^{at} = a\,e^{at}$ with $e^{a\cdot 0} = 1$. Remember this trick. When a becomes a matrix $\mathbf{A}$, we cannot "separate variables" anymore – but we *can* still write down a power series, and it will still differentiate to itself times $\mathbf{A}$. That is the matrix exponential, already visible on the horizon.

Focus

Two faces of the same function:

$$\frac{du}{dt} = au,\ u(0) = u_0 \quad \Longleftrightarrow \quad u(t) = u_0 e^{at} = u_0 \sum_{k=0}^{\infty} \frac{(at)^k}{k!}.$$

The differential equation *defines* the behavior; the exponential *is* the behavior. Everything in this chapter promotes a to a matrix and keeps both faces.

Pause

Pause and think (mental check): If a quantity decays according to $\frac{du}{dt} = -3u$ with initial size $u(0) = 8$, what is $u(t)$? At what time does the quantity drop to 1? *Check:* $u(t) = 8e^{-3t}$. Setting $8e^{-3t} = 1$ gives $e^{3t} = 8$, so $t = \frac{\ln 8}{3} = \ln 2 \approx 0.693$.

25.2 Coupled Equations: Why a Matrix Appears

Now let two quantities interact. The smallest real example makes the point.

Example 25.1 (Two tanks trading salt)**.** Two tanks hold brine. Salt flows between them: tank 1 loses salt to tank 2 and gains some back. Suppose the amounts $u_1(t)$ and $u_2(t)$ obey

$$\frac{du_1}{dt} = -2u_1 + u_2, \qquad \frac{du_2}{dt} = \quad u_1 - 2u_2.$$

You cannot solve the first equation alone, because u_2 is in it; you cannot solve the second alone, because u_1 is in it. They are **coupled** – each variable's rate of change depends on the current value of the other, so neither can be determined in isolation. But stack them into a vector $\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$ and the coupling becomes a single matrix:

$$\frac{d}{dt}\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}\begin{bmatrix} u_1 \\ u_2 \end{bmatrix}, \qquad \text{i.e.} \qquad \frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}, \quad \mathbf{A} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}.$$

The derivative of a vector just means "differentiate each component." The matrix $\mathbf{A}$ encodes *who pushes on whom*: row i tells you how the rate of u_i depends on every u_j.

This is the general setup. A **linear system of first-order ordinary differential equations** with constant coefficients – "first-order" meaning it involves only first derivatives $\frac{d\mathbf{u}}{dt}$ (rates of change, not accelerations), and "linear" meaning rates are linear combinations of the current states – is

$$\frac{d\mathbf{u}}{dt} = \mathbf{A}\,\mathbf{u}, \qquad \mathbf{u}(0) = \mathbf{u}_0, \qquad \mathbf{u}(t) \in \mathbb{R}^n,\ \mathbf{A} \in \mathbb{R}^{n\times n}.$$

We want a formula for $\mathbf{u}(t)$: where is the state at every future (and past) time, given where it starts?

Intuition

Think of $\mathbf{Au}$ as a **velocity field**. At every point $\mathbf{u}$ of the plane (or of $\mathbb{R}^n$), the matrix hands you an arrow $\mathbf{Au}$ that says "if you are here, move *this* way, *this* fast." A solution $\mathbf{u}(t)$ is a path that, at every instant, flows along the arrow it is standing on. Solving the system means: drop a particle at $\mathbf{u}_0$ and let the arrows carry it. The whole picture is a river, and $\mathbf{A}$ is its current.

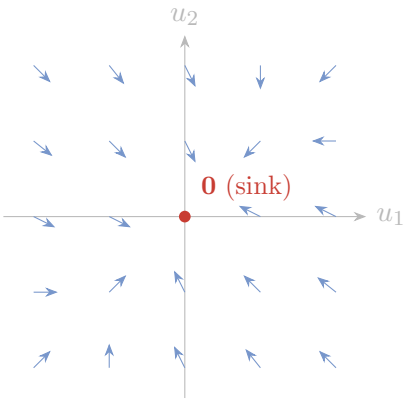


Figure 25.1: The velocity field $\mathbf{Au}$ for the two-tank matrix $\mathbf{A} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}$. Every arrow points roughly inward: every starting state drifts toward the origin. This is a *stable* system, and the chapter will explain exactly why from the eigenvalues.

25.3 One Eigenvector at a Time: Decoupling by Modes

Here is the idea that cracks the whole problem open. We do not know how to solve $\frac{d\mathbf{u}}{dt} = \mathbf{Au}$ for a general starting vector. But suppose we are lucky and the system starts *exactly along an eigenvector*. Then the matrix has nothing to do but stretch, and the vector equation collapses back into the scalar equation we already solved.

Let $\mathbf{Ax} = \lambda\mathbf{x}$, with $\mathbf{x} \neq \mathbf{0}$. Guess a solution that stays pointed along $\mathbf{x}$ for all time, only changing its length:

$$\mathbf{u}(t) = e^{\lambda t}\,\mathbf{x}.$$

Test it. The left side:

$$\frac{d\mathbf{u}}{dt} = \frac{d}{dt}\left(e^{\lambda t}\right)\mathbf{x} = \lambda\, e^{\lambda t}\,\mathbf{x}.$$

The right side:

$$\mathbf{Au} = \mathbf{A}\left(e^{\lambda t}\mathbf{x}\right) = e^{\lambda t}\,(\mathbf{Ax}) = e^{\lambda t}\,\lambda\mathbf{x} = \lambda\, e^{\lambda t}\,\mathbf{x}.$$

They match. So $\mathbf{u}(t) = e^{\lambda t}\mathbf{x}$ is a genuine solution. Along an eigendirection, the matrix system behaves *exactly* like the one-dimensional $\frac{du}{dt} = \lambda u$: the eigenvalue λ is the growth rate, the eigenvector $\mathbf{x}$ is the fixed direction.

Focus

Each eigenpair $(\lambda, \mathbf{x})$ of $\mathbf{A}$ gives one pure solution – a **mode** – of $\frac{d\mathbf{u}}{dt} = \mathbf{Au}$:

$$\mathbf{u}(t) = e^{\lambda t}\,\mathbf{x}.$$

It marches along the fixed line $\text{span}\{\mathbf{x}\}$, growing like $e^{\lambda t}$. The eigenvector sets the *direction*; the eigenvalue sets the *rate*.

Intuition

This is the deep reason eigenvectors deserve their name (*eigen* = "own/characteristic"). They are the directions in which the dynamical system does not mix anything – it just scales. A general state is a combination of these pure modes, each evolving at its own rate $e^{\lambda t}$. To solve the system, you *decompose into modes, let each evolve, and add them back up.*

25.3.1 Adding modes: the general solution by superposition

A single mode describes trajectories that start directly on an eigenvector line. To solve from an arbitrary initial state, we use the fact that the system is *linear*: if $\mathbf{u}(t)$ and $\mathbf{w}(t)$ are both solutions, then any linear combination $c_1\mathbf{u}(t) + c_2\mathbf{w}(t)$ is also a solution, through the principle of **superposition**. (This holds because differentiation and matrix multiplication both distribute over sums: $\frac{d}{dt}(c_1\mathbf{u} + c_2\mathbf{w}) = c_1\mathbf{A}\mathbf{u} + c_2\mathbf{A}\mathbf{w} = \mathbf{A}(c_1\mathbf{u} + c_2\mathbf{w})$.)

So if $\mathbf{A}$ $(n \times n)$ has n independent eigenvectors $\mathbf{x}_1, \ldots, \mathbf{x}_n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$, the general solution is the superposition of all n modes:

$$\boxed{\mathbf{u}(t) = c_1 e^{\lambda_1 t}\mathbf{x}_1 + c_2 e^{\lambda_2 t}\mathbf{x}_2 + \cdots + c_n e^{\lambda_n t}\mathbf{x}_n.}$$

The constants $c_1, \ldots, c_n$ are fixed by the initial condition. Set $t = 0$ (every exponential becomes 1):

$$\mathbf{u}(0) = c_1\mathbf{x}_1 + \cdots + c_n\mathbf{x}_n = \mathbf{u}_0.$$

That is just "write $\mathbf{u}_0$ in the eigenvector basis." In matrix form, with $\mathbf{X} = [\,\mathbf{x}_1 \;\cdots\; \mathbf{x}_n\,]$ (the eigenvector matrix written $\mathbf{S}$ in Chapter 24) and $\mathbf{c} = \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix}$,

$$\mathbf{X}\mathbf{c} = \mathbf{u}_0 \quad \Longrightarrow \quad \mathbf{c} = \mathbf{X}^{-1}\mathbf{u}_0.$$

Focus

Recipe to solve $\dfrac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$, $\mathbf{u}(0) = \mathbf{u}_0$ (diagonalizable case):

1. Find eigenvalues λ_i and eigenvectors $\mathbf{x}_i$ of $\mathbf{A}$.
2. Expand the start in that basis: solve $\mathbf{X}\mathbf{c} = \mathbf{u}_0$ for the coefficients c_i.
3. Let each coefficient ride its own exponential: $\mathbf{u}(t) = \sum_i c_i e^{\lambda_i t}\mathbf{x}_i$.

25.3.2 A fully worked example

Let us actually solve the two-tank system of Example 25.1,

$$\mathbf{A} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}, \qquad \mathbf{u}(0) = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Step 1: eigenvalues. The characteristic polynomial:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \det\begin{bmatrix} -2-\lambda & 1 \\ 1 & -2-\lambda \end{bmatrix} = (-2-\lambda)^2 - 1 = \lambda^2 + 4\lambda + 3 = (\lambda+1)(\lambda+3).$$

So $\lambda_1 = -1$ and $\lambda_2 = -3$. Both eigenvalues are negative: because each mode scales by $e^{\lambda_i t}$, negative rates mean that every component shrinks toward zero over time. The system is stable, exactly as the inward-pointing arrows in Figure 25.1 suggested.

Step 2: eigenvectors. For $\lambda_1 = -1$, solve $(\mathbf{A} + \mathbf{I})\mathbf{x} = 0$:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}\mathbf{x} = \mathbf{0} \implies x_1 = x_2 \implies \mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

For $\lambda_2 = -3$, solve $(\mathbf{A} + 3\mathbf{I})\mathbf{x} = 0$:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\mathbf{x} = \mathbf{0} \implies x_1 = -x_2 \implies \mathbf{x}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

(These eigenvectors are perpendicular because $\mathbf{A}$ is symmetric ($\mathbf{A} = \mathbf{A}^\mathsf{T}$) — a preview of the Spectral Theorem in the very next chapter.)

Step 3: match the start. We need $c_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, i.e.

$$c_1 + c_2 = 3, \qquad c_1 - c_2 = 1 \implies c_1 = 2,\ c_2 = 1.$$

Answer. Letting each coefficient ride its exponential,

$$\mathbf{u}(t) = 2\,e^{-t}\begin{bmatrix} 1 \\ 1 \end{bmatrix} + 1\,e^{-3t}\begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2e^{-t} + e^{-3t} \\ 2e^{-t} - e^{-3t} \end{bmatrix}.$$

Sanity checks: at $t = 0$ this is $\begin{bmatrix} 2+1 \\ 2-1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, correct. As $t \to \infty$, both exponentials vanish, so $\mathbf{u} \to \mathbf{0}$: all the salt equalizes and drains to the steady state, exactly as the picture promised. The e^{-3t} part (the imbalance between the tanks) dies fast; the e^{-t} part (the total amount, along $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$) lingers longest. The *slowest* mode – the eigenvalue closest to zero – dominates the late-time behavior (Figure 25.2).

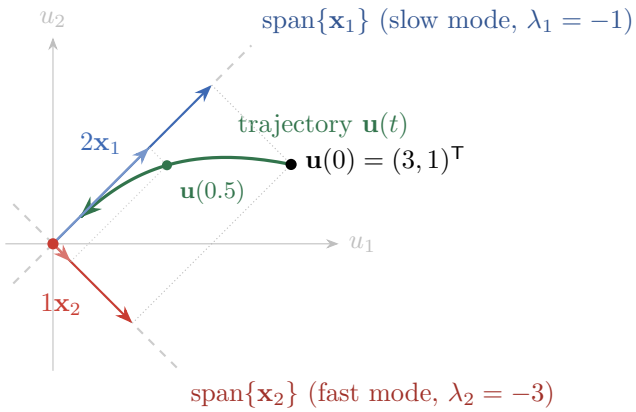


Figure 25.2: How superposition creates a curved trajectory. At $t = 0$, the initial state $\mathbf{u}(0) = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ decomposes into components along the two eigenvector lines (dashed): $2\mathbf{x}_1$ (slow mode, blue) and $1\mathbf{x}_2$ (fast mode, red). As time runs forward, the fast mode decays rapidly like e^{-3t}, collapsing the red component toward zero. The slow mode decays gently like e^{-t}, keeping the blue component alive. As a result, the sum $\mathbf{u}(t)$ quickly bends toward the slow eigendirection span$\{\mathbf{x}_1\}$ and follows it into the origin.

Compute it yourself

Take $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}$ and $\mathbf{u}(0) = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. Its eigenvalues are 1 and 2 with eigenvectors $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Find c_1, c_2 (you should get $c_1 = 1$, $c_2 = 1$, so both modes are genuinely present), write $\mathbf{u}(t) = e^t \begin{bmatrix} 1 \\ 0 \end{bmatrix} + e^{2t} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, and confirm both components blow up like e^{2t} for large t. Which mode wins the race here – and why is it the *largest* eigenvalue this time, not the smallest? (Answer: when eigenvalues are positive, the biggest one dominates growth; when negative, the one nearest zero dominates the slow decay. Either way it is the *rightmost* eigenvalue – largest real part – that sets the late-time behavior.)

Before/after Strang

Strang teaches this as "$\mathbf{u}(t) = \sum c_i e^{\lambda_i t} \mathbf{x}_i$," highlighting how eigenvalues govern dynamical behavior. *Before* Strang, the core geometric insight is that a system of linear differential equations is solved by identifying the invariant directions (eigenvectors) and their growth or decay rates (eigenvalues). *After* Strang, notice this represents diagonalization applied to time evolution, leading directly to the matrix exponential $e^{\mathbf{A}t}$.

25.4 Stability: Reading the Future from the Eigenvalues

The superposition formula makes the long-term fate of the system transparent. Each mode carries a factor $e^{\lambda_i t}$. As $t \to \infty$:

- if $\lambda_i < 0$, that mode **decays** to zero;
- if $\lambda_i > 0$, that mode **blows up**;
- if $\lambda_i = 0$, that mode **persists** at constant size.

The whole solution is a sum of such terms, so its fate is decided by the *worst* mode – the one with the largest real part (since, as we are about to see, eigenvalues may be complex and it is $\operatorname{Re}\lambda$ that sets growth or decay).

Formal

Consider $\dfrac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$ with $\mathbf{A}$ having eigenvalues $\lambda_1, \ldots, \lambda_n$.

- The system is **stable** (every solution $\to \mathbf{0}$) iff $\operatorname{Re}\lambda_i < 0$ for *all* i.
- It is **neutrally stable** (solutions stay bounded) iff $\operatorname{Re}\lambda_i \le 0$ for all i, with the zero-real-part eigenvalues not defective (an eigenvalue is **defective** if it lacks a full set of independent eigenvectors, having fewer eigenvectors than its algebraic multiplicity).
- It is **unstable** (some solution $\to \infty$) iff $\operatorname{Re}\lambda_i > 0$ for some i, or some eigenvalue with $\operatorname{Re}\lambda_i = 0$ is defective.

Why "real part"? Because eigenvalues can be complex, and that is where oscillation hides. Recall Euler's formula. If $\lambda = \sigma + i\omega$, then

$$e^{\lambda t} = e^{(\sigma+i\omega)t} = e^{\sigma t}\left(\cos\omega t + i\sin\omega t\right).$$

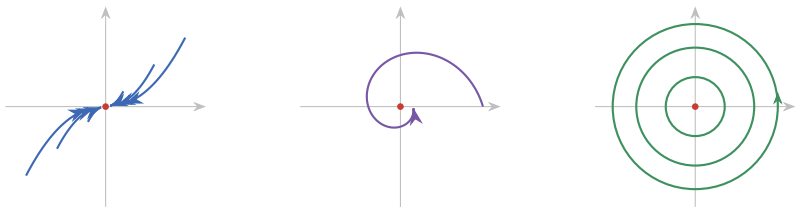
The *real part* σ controls the envelope $e^{\sigma t}$ – growth or decay – and the *imaginary part* ω controls the spinning $\cos\omega t + i\sin\omega t$ – oscillation. So:

Intuition

Split each eigenvalue $\lambda = \sigma + i\omega$ into two jobs:

$\sigma = \operatorname{Re}\lambda$ sets the **envelope**: $e^{\sigma t}$ grows ($\sigma > 0$) or shrinks ($\sigma < 0$).
$\omega = \operatorname{Im}\lambda$ sets the **frequency**: how fast the solution rotates.

A pure imaginary eigenvalue ($\sigma = 0$) means *undamped oscillation* – a frictionless pendulum circling forever. A complex eigenvalue with $\sigma < 0$ means a *damped spiral* into the origin. With $\sigma > 0$, a spiral flying out.



sink: $\sigma < 0$, real spiral sink: $\sigma < 0$, $\omega \neq 0$ center: $\sigma = 0$, $\omega \neq 0$

Figure 25.3: Phase portraits keyed to the eigenvalues. Real negative eigenvalues give a *sink* (paths slide straight in). Complex eigenvalues with negative real part give a *spiral sink*. Pure imaginary eigenvalues give a *center* of closed orbits – perpetual, undamped oscillation. Flip the sign of σ and every arrow reverses: sinks become sources, spirals fly outward.

Example 25.2 (Planar rotation: complex eigenvalues)**.** Take

$$\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad \frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}.$$

Written out, $u_1' = -u_2$ and $u_2' = u_1$. The eigenvalues satisfy $\lambda^2 + 1 = 0$, so $\lambda = \pm i$: pure imaginary, $\sigma = 0$. The theory predicts a *center* – closed circular orbits, no growth, no decay. Indeed, the solution starting at $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ is

$$\mathbf{u}(t) = \begin{bmatrix} \cos t \\ \sin t \end{bmatrix},$$

which you can verify directly: $\mathbf{u}' = \begin{bmatrix} -\sin t \\ \cos t \end{bmatrix} = \begin{bmatrix} -u_2 \\ u_1 \end{bmatrix} = \mathbf{A}\mathbf{u}$. The state runs around the unit circle forever. Notice that $\mathbf{A}$ is exactly the velocity field whose flow is rotation – the matrix that "generates" turning. Indeed, as we will see shortly, $e^{\mathbf{A}t}$ is precisely the planar rotation matrix through angle t.

Common trap

Sign confusion. The condition for *stable* continuous dynamics is $\operatorname{Re}\lambda < 0$, eigenvalues in the *left* half-plane. This is *not* the same as the rule for discrete iterations $\mathbf{u}_{k+1} = \mathbf{A}\mathbf{u}_k$, where stability means $|\lambda| < 1$ (eigenvalues inside the unit *circle*). Continuous time cares about left/right; discrete time cares about inside/outside the circle. Mixing them up is the classic exam mistake. The bridge between them is $z = e^{\lambda t}$: the left half-plane maps exactly to the inside of the unit disk (Figure 25.4).

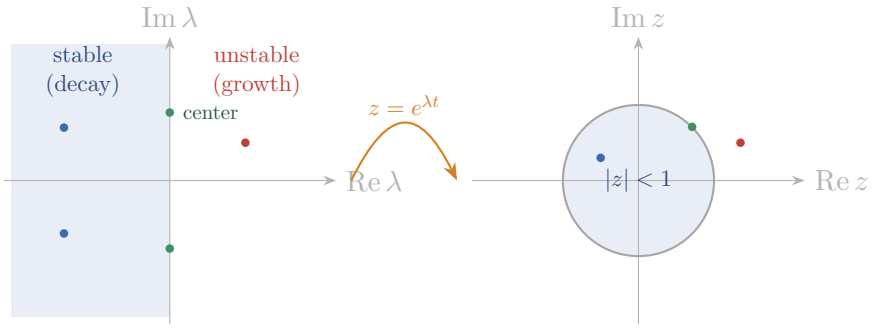


Figure 25.4: Two stability regions, and the map between them. *Left:* for continuous dynamics $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$, an eigenvalue is stabilizing when it sits in the *left half-plane* ($\operatorname{Re}\lambda < 0$); on the imaginary axis it gives a neutral center. *Right:* for discrete iterations $\mathbf{u}_{k+1} = \mathbf{A}\mathbf{u}_k$, the stable region is the *inside of the unit circle* ($|\lambda| < 1$). The exponential map $z = e^{\lambda t}$ carries the left half-plane exactly onto the open unit disk, which is why the two rules are really one rule seen through $e^{\lambda t}$.

Walking question

A system has eigenvalues -5 and -0.1. Both decay, so it is stable – but if you watched it, what would you *see*? One piece vanishes almost instantly; the other lingers fifty times longer. The long-term shape of *every* trajectory is therefore nearly the eigenvector of -0.1. Think on

your walk: in a stable system, does the smallest or the largest $|\lambda|$ shape what you actually observe after a while? (This is exactly why the "slow mode" governs the relaxation time of springs, circuits, and chemical reactions.)

25.5 The Matrix Exponential: One Object for the Whole Flow

We solved the system mode by mode. That is concrete and computational, but it hides a cleaner truth. In one dimension we never said "decompose and add" – we just wrote $u(t) = e^{at}u_0$. We want the same compact statement for vectors:

$$\mathbf{u}(t) = ? \cdot \mathbf{u}_0,$$

where the "?" is a single matrix that takes *any* start $\mathbf{u}_0$ and slides it forward to time t. That matrix is the **matrix exponential** $e^{\mathbf{A}t}$, and we define it by the very power series we rehearsed in Section 25.1, with the scalar a replaced by the matrix $\mathbf{A}$.

Definition 25.1 (Matrix exponential)**.** For a square matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$, define

$$e^{\mathbf{M}} = \sum_{k=0}^{\infty} \frac{\mathbf{M}^k}{k!} = \mathbf{I} + \mathbf{M} + \frac{\mathbf{M}^2}{2!} + \frac{\mathbf{M}^3}{3!} + \cdots.$$

Here $\mathbf{M}^0 = \mathbf{I}$. Just as an infinite series of numbers converges when its partial sums settle to a definite limit (Section 39.3), an infinite series of matrices converges when the number series in each of its n^2 entries converges. Because the entries of $\mathbf{M}^k$ grow no faster than powers of the matrix norm, the series converges entrywise and absolutely for every square matrix (Section 39.3.4 develops the calculus of these matrix series and their term-by-term derivatives). Thus $e^{\mathbf{M}}$ is always a well-defined $n \times n$ matrix. In particular, $e^{\mathbf{A}t} = \sum_{k=0}^{\infty} \frac{\mathbf{A}^k t^k}{k!}$.

Focus

$e^{\mathbf{A}t}$ is a *matrix*, not a number. It is the "time-t flow map": feed it where you are now, it returns where you will be at time t. The solution of $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$, $\mathbf{u}(0) = \mathbf{u}_0$, is

$$\boxed{\mathbf{u}(t) = e^{\mathbf{A}t}\,\mathbf{u}_0.}$$

Pause

Pause and think (mental check): If $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & -5 \end{bmatrix}$ is already diagonal, what is $e^{\mathbf{A}t}$?

Check: Because powers of a diagonal matrix are diagonal ($\mathbf{A}^k = \operatorname{diag}(2^k, (-5)^k)$), each entry computes its own scalar series: $e^{\mathbf{A}t} = \operatorname{diag}(e^{2t}, e^{-5t}) = \begin{bmatrix} e^{2t} & 0 \\ 0 & e^{-5t} \end{bmatrix}$.

Let us prove that boxed claim, and prove it the same way we did for scalars: differentiate the series term by term and watch it reproduce $\mathbf{A}$ times itself.

Theorem 25.1 (The matrix exponential solves the system)**.** *The matrix-valued function* $\Phi(t) = e^{\mathbf{A}t}$ *satisfies*

$$\frac{d}{dt}\, e^{\mathbf{A}t} = \mathbf{A}\, e^{\mathbf{A}t} = e^{\mathbf{A}t}\, \mathbf{A}, \qquad e^{\mathbf{A}\cdot 0} = \mathbf{I}.$$

Consequently $\mathbf{u}(t) = e^{\mathbf{A}t}\mathbf{u}_0$ *solves* $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$ *with* $\mathbf{u}(0) = \mathbf{u}_0$.

Proof. At $t = 0$ every term but the first vanishes, leaving $e^{\mathbf{A}\cdot 0} = \mathbf{I}$. Differentiate the series term by term (legal because it converges absolutely and uniformly on bounded t-intervals; see Section 39.3.4):

$$\frac{d}{dt}\sum_{k=0}^{\infty}\frac{\mathbf{A}^k t^k}{k!} = \sum_{k=1}^{\infty}\frac{\mathbf{A}^k\, k\, t^{k-1}}{k!} = \sum_{k=1}^{\infty}\frac{\mathbf{A}^k t^{k-1}}{(k-1)!}.$$

Factor one $\mathbf{A}$ out of the front (it commutes with every $\mathbf{A}^{k-1}$) and reindex $j = k - 1$:

$$= \mathbf{A}\sum_{j=0}^{\infty}\frac{\mathbf{A}^j t^j}{j!} = \mathbf{A}\, e^{\mathbf{A}t}.$$

Pulling the $\mathbf{A}$ out on the right instead gives $e^{\mathbf{A}t}\mathbf{A}$; the two agree because $\mathbf{A}$ commutes with its own powers. Finally, with $\mathbf{u}(t) = e^{\mathbf{A}t}\mathbf{u}_0$,

$$\frac{d\mathbf{u}}{dt} = \left(\tfrac{d}{dt}e^{\mathbf{A}t}\right)\mathbf{u}_0 = \mathbf{A}e^{\mathbf{A}t}\mathbf{u}_0 = \mathbf{A}\mathbf{u}(t),$$

and $\mathbf{u}(0) = \mathbf{I}\mathbf{u}_0 = \mathbf{u}_0$. □

Connection to other math

This is the same proof, letter for letter, as the calculus proof that $\frac{d}{dt}e^{at} = a\, e^{at}$. We replaced the scalar a by the matrix $\mathbf{A}$ and changed nothing else. A scalar identity, when evaluated on matrices, becomes an operator identity describing entire dynamic systems. Calculus supplies the power series and term-by-term differentiation (Section 39.3.4); linear algebra supplies the algebra of matrix powers $\mathbf{A}^k$. The matrix exponential connects the two subjects.

25.5.1 Properties, and the one trap

The matrix exponential keeps most of the algebra you expect, with one sharp exception you must respect.

Proposition 25.1 (Algebra of $e^{\mathbf{A}}$)**.** *For square* $\mathbf{A}, \mathbf{B}$ *of the same size:*

1. $e^{\mathbf{0}} = \mathbf{I}$.
2. $\left(e^{\mathbf{A}t}\right)^{-1} = e^{-\mathbf{A}t}$*; in particular* $e^{\mathbf{A}t}$ *is* always invertible.
3. $e^{\mathbf{A}(s+t)} = e^{\mathbf{A}s}\, e^{\mathbf{A}t}$ *(the flow composes: evolving* s *then* t *equals evolving* $s + t$*).*
4. ***If*** $\mathbf{AB} = \mathbf{BA}$**,** *then* $e^{\mathbf{A}+\mathbf{B}} = e^{\mathbf{A}}e^{\mathbf{B}}$*. If they do* not *commute, this is generally* false*.*

Property (2) is worth a pause. If $\mathbf{A}$ has eigenvalues $\lambda_1, \ldots, \lambda_n$, the eigenvalues of $e^{\mathbf{A}t}$ are $e^{\lambda_1 t}, \ldots, e^{\lambda_n t}$. Because the determinant of any matrix equals the product of its eigenvalues,

$$\det(e^{\mathbf{A}t}) = \prod_{i=1}^{n} e^{\lambda_i t} = e^{t\sum_{i=1}^{n}\lambda_i} = e^{t\,\operatorname{tr}\mathbf{A}},$$

where $\operatorname{tr}\mathbf{A}$, the **trace**, is the sum of the entries on the main diagonal (which always equals the sum of all eigenvalues). Because the scalar exponential $e^{t\,\operatorname{tr}\mathbf{A}}$ is never zero, $\det(e^{\mathbf{A}t}) \neq 0$ for all t. The flow map is always invertible: it can never flatten space into a lower-dimensional slice, two distinct initial states can never collide, and trajectories never cross. That is the matrix face of the uniqueness of solutions to differential equations.

Common trap

$e^{\mathbf{A}+\mathbf{B}} \neq e^{\mathbf{A}}e^{\mathbf{B}}$ when $\mathbf{A}$ and $\mathbf{B}$ do not commute. The familiar law $e^{x+y} = e^x e^y$ relies on $xy = yx$, which is automatic for numbers but not for matrices. The cleanest way to see the failure: expand both sides to second order. The cross term in $e^{\mathbf{A}+\mathbf{B}}$ is $\frac{1}{2}(\mathbf{A}+\mathbf{B})^2 = \frac{1}{2}(\mathbf{A}^2 + \mathbf{AB} + \mathbf{BA} + \mathbf{B}^2)$, while in $e^{\mathbf{A}}e^{\mathbf{B}}$ it is $\frac{1}{2}\mathbf{A}^2 + \mathbf{AB} + \frac{1}{2}\mathbf{B}^2$. They differ by $\frac{1}{2}(\mathbf{AB} - \mathbf{BA})$ – the **commutator**, which measures the failure of two operators to commute. Without commutativity, the exponential of a sum does not factor into a product of exponentials – an obstruction studied systematically in Lie theory.

25.6 Computing $e^{\mathbf{A}t}$ by Diagonalization

The power-series definition is foundational, but difficult to evaluate directly. Diagonalization provides a practical computational route, and reveals that the matrix exponential unifies the mode expansion from Section 25.3.1 into a single operator.

Suppose $\mathbf{A}$ is diagonalizable: $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$, where $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ holds the eigenvalues and $\mathbf{X}$ holds the eigenvectors as columns. The key observation is that powers of $\mathbf{A}$ telescope:

$$\mathbf{A}^k = (\mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1})^k = \mathbf{X}\mathbf{\Lambda}^k\mathbf{X}^{-1},$$

because the inner $\mathbf{X}^{-1}\mathbf{X}$ pairs cancel. Feed this into the series:

$$e^{\mathbf{A}t} = \sum_{k=0}^{\infty}\frac{\mathbf{A}^k t^k}{k!} = \sum_{k=0}^{\infty}\frac{\mathbf{X}\mathbf{\Lambda}^k\mathbf{X}^{-1}t^k}{k!} = \mathbf{X}\left(\sum_{k=0}^{\infty}\frac{\mathbf{\Lambda}^k t^k}{k!}\right)\mathbf{X}^{-1} = \mathbf{X}\,e^{\mathbf{\Lambda}t}\,\mathbf{X}^{-1}.$$

And a diagonal matrix exponentiates entrywise – because powers of a diagonal matrix are just the powers of its entries:

$$e^{\mathbf{\Lambda}t} = \operatorname{diag}\big(e^{\lambda_1 t}, e^{\lambda_2 t}, \ldots, e^{\lambda_n t}\big).$$

Focus

The master formula. If $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$, then

$$e^{\mathbf{A}t} = \mathbf{X} \begin{bmatrix} e^{\lambda_1 t} & & \\ & \ddots & \\ & & e^{\lambda_n t} \end{bmatrix} \mathbf{X}^{-1}.$$

Read it as a sentence: *change to eigen-coordinates ($\mathbf{X}^{-1}$), exponentiate each one-dimensional flow ($e^{\lambda_i t}$), change back ($\mathbf{X}$).* The hard matrix problem became n trivial scalar problems.

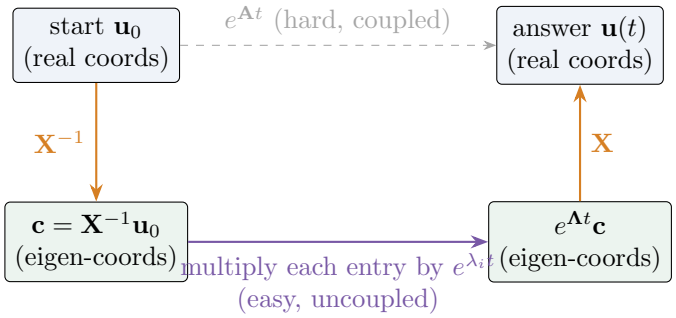


Figure 25.5: The master formula as a detour. Going straight across the top ($e^{\mathbf{A}t}$) means solving a coupled system. Instead drop down into eigen-coordinates with $\mathbf{X}^{-1}$, where the system splits into n independent scalar flows, scale each by its own $e^{\lambda_i t}$, then ride $\mathbf{X}$ back up. The two routes give the same $\mathbf{u}(t)$ – that equality *is* $e^{\mathbf{A}t} = \mathbf{X}\, e^{\mathbf{\Lambda}t}\, \mathbf{X}^{-1}$.

Intuition

This represents diagonalization applied to continuous time evolution (Figure 25.5). In the eigenbasis, the coupled system uncouples completely: each eigen-coordinate $w_i = (\mathbf{X}^{-1}\mathbf{u})_i$ obeys its own scalar differential equation $w_i' = \lambda_i w_i$, so $w_i(t) = e^{\lambda_i t} w_i(0)$. The matrices $\mathbf{X}^{-1}$ and $\mathbf{X}$ convert vectors between the standard coordinate frame and the decoupled eigenbasis.

And the connection back to modes is immediate. Multiply the master formula by $\mathbf{u}_0$ and write $\mathbf{X}^{-1}\mathbf{u}_0 = \mathbf{c}$:

$$\mathbf{u}(t) = e^{\mathbf{A}t}\mathbf{u}_0 = \mathbf{X}\, e^{\mathbf{\Lambda}t}\, \mathbf{c} = \sum_{i=1}^{n} c_i\, e^{\lambda_i t}\, \mathbf{x}_i.$$

Multiplying $e^{\mathbf{A}t}$ by $\mathbf{u}_0$ recovers the superposition formula of Section 25.3.1 directly.

Example 25.3 (Exponentiating the two-tank matrix)**.** For $\mathbf{A} = \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix}$ we found $\lambda_1 = -1, \lambda_2 = -3$ with $\mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{x}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. So

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad \mathbf{X}^{-1} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Then

$$e^{\mathbf{A}t} = \mathbf{X}\begin{bmatrix} e^{-t} & 0 \\ 0 & e^{-3t} \end{bmatrix}\mathbf{X}^{-1} = \frac{1}{2}\begin{bmatrix} e^{-t}+e^{-3t} & e^{-t}-e^{-3t} \\ e^{-t}-e^{-3t} & e^{-t}+e^{-3t} \end{bmatrix}.$$

Apply it to $\mathbf{u}_0 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$:

$$e^{\mathbf{A}t}\begin{bmatrix} 3 \\ 1 \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 3(e^{-t}+e^{-3t})+(e^{-t}-e^{-3t}) \\ 3(e^{-t}-e^{-3t})+(e^{-t}+e^{-3t}) \end{bmatrix} = \begin{bmatrix} 2e^{-t}+e^{-3t} \\ 2e^{-t}-e^{-3t} \end{bmatrix},$$

matching Section 25.3.2 exactly. The single matrix $e^{\mathbf{A}t}$ carries the whole solution for *every* starting vector at once.

Compute it yourself

Compute $e^{\mathbf{A}t}$ by hand for the rotation generator $\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ of Example 25.2, but *without* diagonalizing – use the series. Show $\mathbf{A}^2 = -\mathbf{I}$, hence $\mathbf{A}^3 = -\mathbf{A}$, $\mathbf{A}^4 = \mathbf{I}$, and the powers cycle. Group the series into even and odd terms and recognize the Taylor series of cosine and sine to get

$$e^{\mathbf{A}t} = \begin{bmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{bmatrix}.$$

The matrix exponential of the "turning" matrix is the rotation by angle t. Beautiful, and a perfect mirror of $e^{it} = \cos t + i \sin t$.

Why this matters

The scalar identity $e^{it} = \cos t + i \sin t$ – Euler's formula – is the counterpart of the matrix statement "the exponential of the rotation generator is the rotation matrix." The imaginary unit i corresponds algebraically to the matrix $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$; squaring it gives $-\mathbf{I}$, just as $i^2 = -1$. Complex numbers, rotations, oscillation, and the matrix exponential are interconnected facets of the same underlying structure.

25.7 When You Cannot Diagonalize

The master formula assumed n independent eigenvectors. What if $\mathbf{A}$ is **defective** – lacking a complete set of n independent eigenvectors, like $\mathbf{A} = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}$? Although the eigenvector decomposition is incomplete, the matrix exponential remains well-defined: its defining power series converges for *every* square matrix. You can evaluate it directly by splitting $\mathbf{A}$ into a scalar multiple of $\mathbf{I}$ plus a **nilpotent** remainder – a matrix that vanishes entirely when raised to a sufficiently high power.

Write $\mathbf{A} = \lambda\mathbf{I} + \mathbf{N}$ with $\mathbf{N} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$. These commute (every matrix commutes with scalar multiples of the identity $\lambda\mathbf{I}$), so by Proposition 25.1(4),

$$e^{\mathbf{A}t} = e^{\lambda t\mathbf{I}}\, e^{\mathbf{N}t} = e^{\lambda t}\, e^{\mathbf{N}t}.$$

Now $\mathbf{N}^2 = \mathbf{0}$ (so $\mathbf{N}$ is nilpotent of degree 2), so its series truncates cleanly after two terms:

$$e^{\mathbf{N}t} = \mathbf{I} + \mathbf{N}t = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}.$$

Therefore

$$e^{\mathbf{A}t} = e^{\lambda t}\begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} e^{\lambda t} & t\,e^{\lambda t} \\ 0 & e^{\lambda t} \end{bmatrix}.$$

Pause

Pause and think (mental check): If $\mathbf{N} = \begin{bmatrix} 0 & 3 \\ 0 & 0 \end{bmatrix}$, what is $\mathbf{N}^2$, and what is $e^{\mathbf{N}t}$? *Check:* $\mathbf{N}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} = \mathbf{0}$, so only two terms survive the power series: $e^{\mathbf{N}t} = \mathbf{I} + \mathbf{N}t = \begin{bmatrix} 1 & 3t \\ 0 & 1 \end{bmatrix}$.

Notice the brand-new ingredient: a factor of t multiplying an exponential. A defective eigenvalue produces solutions involving $t\,e^{\lambda t}$ alongside $e^{\lambda t}$. Geometrically, the deficient direction leaks polynomial growth on top of the exponential.

Remark 25.1. The general structure behind this is the **Jordan canonical form** (introduced in Chapter 28, Section 28.5) – every matrix can be expressed as $\mathbf{X}(\mathbf{\Lambda} + \mathbf{N})\mathbf{X}^{-1}$ with $\mathbf{\Lambda}$ diagonal, $\mathbf{N}$ nilpotent, and $\mathbf{\Lambda N} = \mathbf{N\Lambda}$. Because the pieces commute, $e^{\mathbf{A}t}$ always decomposes into pure exponentials multiplied by polynomials in t. For now, keep the headline: the flow map $e^{\mathbf{A}t}$ *exists and is computable for every matrix* $\mathbf{A}$, diagonalizable or not.

Connection to other math

Higher-order scalar ODEs can be rewritten directly as first-order matrix systems, and this is where the $t\,e^{\lambda t}$ terms you saw in calculus come from. The equation $y'' + 2y' + y = 0$ has the repeated root $\lambda = -1$, and its solutions are e^{-t} *and* $t\,e^{-t}$ – exactly the defective pattern above. Setting $\mathbf{u} = \begin{bmatrix} y \\ y' \end{bmatrix}$ turns the second-order equation into $\mathbf{u}' = \begin{bmatrix} 0 & 1 \\ -1 & -2 \end{bmatrix}\mathbf{u}$, a defective 2×2 system. The standard rule of multiplying by t for repeated roots in differential equations is the scalar manifestation of a nilpotent operator.

25.8 A Modern Application

Modern application

Continuous-time Markov chains and the engines behind search, epidemics, and chemistry. Many real systems hop between states at random rates: which web page a surfer visits next, whether a person is Susceptible / Infected / Recovered, which conformation a protein is in. The probability distribution $\mathbf{p}(t)$ over states evolves by a linear ODE,

$$\frac{d\mathbf{p}}{dt} = \mathbf{Q}^{\mathsf{T}}\mathbf{p},$$

where $\mathbf{Q}$ is the *generator* (rate) matrix: off-diagonal $Q_{ij} \geq 0$ is the rate of jumping $i \to j$, and each row sums to zero so probability is conserved. The solution is exactly the matrix

exponential,

$$\mathbf{p}(t) = e^{\mathbf{Q}^{\mathsf{T}} t}\,\mathbf{p}(0).$$

The eigenvalue $\lambda = 0$ (every generator has one, because rows sum to zero) carries the *steady-state distribution* – the mode that never decays, the long-run behavior of the chain. Every other eigenvalue has negative real part, and the one closest to zero sets the *mixing time*: how long until the chain forgets where it started. Search ranking, disease forecasting, queueing theory, and molecular kinetics all run on $e^{\mathbf{Q}t}$. The signs and magnitudes of $\mathbf{Q}$'s eigenvalues determine whether the chain settles to a steady state and how quickly it mixes.

Modern application

The exponential of a network. In graph theory and neuroscience, if $\mathbf{A}$ is the adjacency matrix of a network, then $(e^{\mathbf{A}})_{ij}$ – the ij entry of the matrix exponential – is a weighted count of *all* walks from node i to node j, with longer walks down-weighted by $1/k!$. This "communicability" is a standard centrality measure: it scores how easily influence (signals, rumors, current) spreads between nodes. Because $e^{\mathbf{A}} = \sum \mathbf{A}^k/k!$ and $(\mathbf{A}^k)_{ij}$ counts length-k walks, the matrix exponential of a graph is, quite literally, a generating function for its connectivity.

25.9 The Geometry of the Flow

The geometry is the point, so it is worth collecting the pictures in one place. The matrix $\mathbf{A}$ defines a velocity field; the family $\{e^{\mathbf{A}t}\}_{t\in\mathbb{R}}$ is the *flow* it generates – a one-parameter family of matrices, starting at $\mathbf{I}$ when $t = 0$, sliding every point of space along the river as t increases.

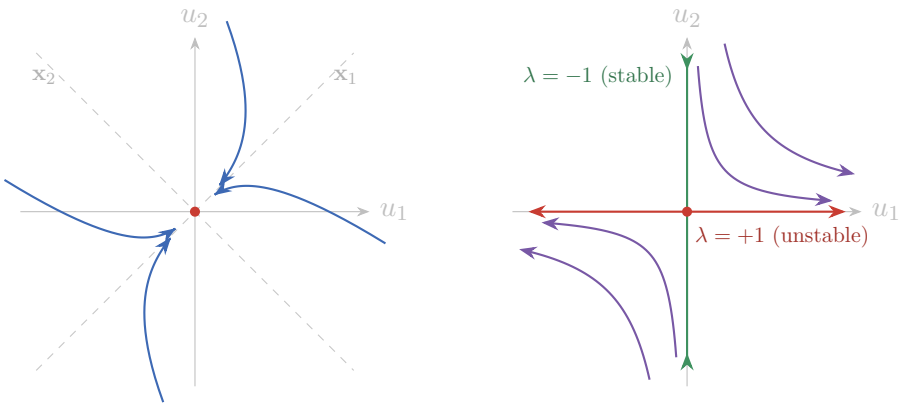


Figure 25.6: Two phase portraits. *Left:* both eigenvalues negative – a stable node. Every trajectory slides into the origin, hugging the slow eigendirection $\mathbf{x}_1$ (eigenvalue -1) on its way in, because the fast mode (-3) dies first. *Right:* one positive, one negative eigenvalue – a *saddle*. Along the green axis (stable, $\lambda = -1$) points fall in; along the red axis (unstable, $\lambda = +1$) they fly out; generic trajectories swoop in, turn, and escape. A single positive eigenvalue is enough to make a system unstable.

Intuition

The full zoo of 2×2 behaviors is organized entirely by the two eigenvalues:

- both real, same sign $\rightarrow$ **node** (in if negative, out if positive);
- real, opposite signs $\rightarrow$ **saddle** (always unstable);
- complex, nonzero real part $\rightarrow$ **spiral** (in if $\sigma < 0$);
- pure imaginary $\rightarrow$ **center** (closed orbits, neutrally stable).

Two numbers – the eigenvalues – and you know the qualitative shape of every trajectory before computing a single point. That is the power the spectrum gives you over a dynamical system.

Before/after Strang

Strang frames the climax as: *the eigenvalues of* $\mathbf{A}$ *control* $e^{\mathbf{A}t}$, and the four fundamental questions (does it grow, decay, oscillate, blow up?) are answered by the locations of the λ's. *After* Strang, the natural next step is to ask what happens when $\mathbf{A}$ is symmetric (real eigenvalues, orthogonal eigenvectors, no spirals – the spectral theorem, next chapter) and what $e^{\mathbf{A}t}$ means when $\mathbf{A}$ is skew-symmetric (pure rotation, energy conserved). The matrix exponential is the doorway from static linear algebra into the dynamics of the world.

History and connection

For two centuries after Newton, the grand goal of mathematical physics was discovering explicit algebraic formulas for physical paths and planetary orbits. But in the 1880s, the French mathematician Henri Poincaré confronted the **three-body problem** in celestial mechanics – predicting the gravitational motions of three mutually attracting bodies, such as the Sun, Earth, and Moon – and discovered that exact closed-form algebraic formulas in terms of elementary functions are impossible. Armed only with pencil-and-paper calculus, coordinate transformations, and power series, Poincaré revolutionized mathematics by shifting from algebraic formulas to qualitative geometry. Instead of hunting for an elusive formula for $\mathbf{u}(t)$, he asked what the global geometry of all trajectories looks like in space. He invented the **phase portrait** – a geometric atlas displaying all possible trajectory curves simultaneously – showing that a linear system $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$ generates a directional velocity field whose flow lines swirl, focus, saddle, or spiral around equilibrium points. Decades later, Rudolf Kalman (1960) formalized continuous-time linear **state-space models** in aerospace engineering – systems tracking physical positions and velocities via first-order vector differential equations – establishing the matrix exponential $e^{\mathbf{A}t}$ as the standard engine for spacecraft trajectory planning and robotic control.

The leap from scalar to matrix exponentials uncovered several classic traps. The most notorious algebraic pitfall is writing $e^{\mathbf{A}+\mathbf{B}} = e^{\mathbf{A}}e^{\mathbf{B}}$, which fails whenever $\mathbf{A}$ and $\mathbf{B}$ do not commute ($\mathbf{AB} \neq \mathbf{BA}$); swapping the order of non-commuting physical operations leads to genuinely different final states. A second trap is numerical: in their classic 1978 paper "Nineteen Dubious Ways to Compute the Exponential of a Matrix," Cleve Moler and Charles Van Loan proved that directly summing the Taylor series $\sum \frac{(\mathbf{A}t)^k}{k!}$ in floating-point arithmetic is disastrously unstable. Large powers of $\mathbf{A}$ with alternating signs produce giant intermediate quantities whose subtraction obliterates accuracy – a numerical breakdown called **catastrophic cancellation**. When a matrix is defective (possessing repeated eigenvalues that lack a full set of independent

eigenvectors), the flow develops polynomial prefactors like $t\,e^{\lambda t}$. In mechanical engineering, these repeated roots correspond to physical **resonance**, where an external push at a system's natural frequency pumps unbounded energy into an undamped mode.

The matrix exponential provides a continuous-time **propagator** $\mathbf{u}(t) = e^{\mathbf{A}t}\mathbf{u}_0$ that evolves any initial state forward in time, together with a direct stability test: a system decays to rest if and only if every eigenvalue of $\mathbf{A}$ has a strictly negative real part. Although real physical dynamics are nonlinear, this linear test remains valid in a tiny neighborhood of an equilibrium point where $\mathbf{A}$ serves as the **Jacobian** – the matrix of all first partial derivatives that provides the best linear approximation to a nonlinear vector field. In 1892, the Russian mathematician Aleksandr Lyapunov showed how to prove stability without solving the differential equation at all, by finding an energy-like scalar function (a **Lyapunov function**) that steadily decreases along every trajectory.

Two major challenges remain active frontiers today. First, for massive sparse systems with millions of variables (such as power transmission grids, quantum multi-body systems, or chemical reaction networks), explicitly forming or storing the dense $n \times n$ matrix $e^{\mathbf{A}t}$ is impossible. Computing the action $e^{\mathbf{A}t}\mathbf{v}$ directly on a state vector using Krylov subspace projections, without ever forming the full matrix exponential, remains an active area of numerical research. Second, **non-normal matrices** (matrices that do not commute with their transpose, $\mathbf{A}\mathbf{A}^{\mathsf{T}} \neq \mathbf{A}^{\mathsf{T}}\mathbf{A}$, meaning their eigenvectors are not perpendicular) can trigger colossal **transient growth**: trajectories can surge thousands of times larger before ultimately decaying, setting off turbulence in fluid flow even when every eigenvalue predicts stability. Analyzing such transient spikes requires the modern theory of **pseudospectra** and remains an open frontier at the boundary of linear algebra and fluid dynamics. In Chapter 26 (Section 26.1), we see how symmetric matrices rule out spirals and guarantee pure exponential decay along perpendicular axes, and in Chapter 27 (Definition 27.1), positive definite matrices provide the energy functions that certify stability.

Pause

Cost and intuition recap. What did solving $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$ actually cost, and what is the one idea to keep?

- **Intuition:** eigenvectors are the directions the flow does not mix; along each, the system is just the scalar law $w' = \lambda w$ with solution $e^{\lambda t}$. Solve, then superpose.
- **Cost (diagonalizable case):** one eigendecomposition of $\mathbf{A}$, which is $O(n^3)$ work, then *each* value $\mathbf{u}(t)$ is one change of basis, a diagonal scaling, and a change back – $O(n^2)$ per time, for as many times t as you like. You pay the $O(n^3)$ once and reuse it forever.
- **Cost (general A):** the matrix exponential $e^{\mathbf{A}t}$ still exists and is computable (scaling-and-squaring of the power series, or via Jordan/Schur form); it never needs eigenvectors to be defined.
- **The headline:** the signs of $\operatorname{Re}\lambda_i$ decide the fate of *every* trajectory before you compute a single point.

Summary

- A linear system $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$ is a velocity field; its solution is a path flowing along the arrows.
- Each eigenpair gives a pure mode $\mathbf{u}(t) = e^{\lambda t}\mathbf{x}$. By superposition the general (diagonalizable) solution is $\mathbf{u}(t) = \sum_i c_i e^{\lambda_i t}\mathbf{x}_i$, with $\mathbf{c} = \mathbf{X}^{-1}\mathbf{u}_0$.
- **Stability** is read from the eigenvalues: stable iff every $\operatorname{Re}\lambda_i < 0$. The real part is growth/decay; the imaginary part is oscillation frequency.
- The **matrix exponential** $e^{\mathbf{A}t} = \sum_k \mathbf{A}^k t^k/k!$ is the time-t flow map; $\mathbf{u}(t) = e^{\mathbf{A}t}\mathbf{u}_0$ solves the system, proved by the same term-by-term differentiation as the scalar case.
- When $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$, $e^{\mathbf{A}t} = \mathbf{X}\,e^{\mathbf{\Lambda}t}\,\mathbf{X}^{-1}$ with $e^{\mathbf{\Lambda}t} = \operatorname{diag}(e^{\lambda_i t})$ – diagonalization applied to continuous time evolution. Defective matrices add polynomial-in-t factors.
- Beware: $e^{\mathbf{A}+\mathbf{B}} = e^{\mathbf{A}}e^{\mathbf{B}}$ only when $\mathbf{AB} = \mathbf{BA}$; and continuous stability ($\operatorname{Re}\lambda < 0$) is not discrete stability ($|\lambda| < 1$).
- **Looking forward:** In this chapter, eigenvectors could lean at skewed angles and eigenvalues could wander into the complex plane, generating spirals. In Chapter 26 (Section 26.1), we see that when a matrix is symmetric ($\mathbf{A} = \mathbf{A}^{\mathsf{T}}$), no spirals can occur: every eigenvalue is real and the eigenvectors can always be chosen perpendicular, which is the content of the Spectral Theorem.

25.10 Exercises

Exercise 25.1 (Warmup: scalar fate)**.** For each, state whether $u(t) \to 0$, stays constant, or blows up, and give $u(t)$: (a) $u' = 3u,\ u(0) = 2$; (b) $u' = -\frac{1}{2}u,\ u(0) = 5$; (c) $u' = 0 \cdot u,\ u(0) = 7$.

Exercise 25.2 (Solve a 2×2 system)**.** Solve $\frac{d\mathbf{u}}{dt} = \mathbf{A}\mathbf{u}$ with $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$ and $\mathbf{u}(0) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. (Find eigenvalues/eigenvectors, expand the start, write $\mathbf{u}(t)$.) Is the system stable? What is the fate of each component as $t \to \infty$?

Exercise 25.3 (Read the portrait)**.** Classify the origin (node/saddle/spiral/center, stable or not) for each: (a) $\begin{bmatrix} -3 & 0 \\ 0 & -1 \end{bmatrix}$; (b) $\begin{bmatrix} 2 & 0 \\ 0 & -1 \end{bmatrix}$; (c) $\begin{bmatrix} -1 & -4 \\ 4 & -1 \end{bmatrix}$; (d) $\begin{bmatrix} 0 & 3 \\ -3 & 0 \end{bmatrix}$. For (c) and (d), find the eigenvalues and identify the real and imaginary parts.

Exercise 25.4 (Matrix exponential by series)**.** Let $\mathbf{N} = \begin{bmatrix} 0 & 2 \\ 0 & 0 \end{bmatrix}$. Compute $\mathbf{N}^2$, then sum the series to find $e^{\mathbf{N}t}$ exactly. Verify by checking $\frac{d}{dt}e^{\mathbf{N}t} = \mathbf{N}e^{\mathbf{N}t}$.

Exercise 25.5 (Diagonalize, then exponentiate)**.** For $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$, find $\mathbf{X}$ and $\mathbf{\Lambda}$, then assemble $e^{\mathbf{A}t} = \mathbf{X}e^{\mathbf{\Lambda}t}\mathbf{X}^{-1}$ as an explicit 2×2 matrix of exponentials. (Eigenvalues are -1 and -2.)

Exercise 25.6 (Rotation generator)**.** Let $\mathbf{A} = \begin{bmatrix} 0 & -\omega \\ \omega & 0 \end{bmatrix}$. Using $\mathbf{A}^2 = -\omega^2\mathbf{I}$, sum the even and odd parts of the series to show $e^{\mathbf{A}t} = \begin{bmatrix} \cos\omega t & -\sin\omega t \\ \sin\omega t & \cos\omega t \end{bmatrix}$. What is the period of the resulting motion?

Exercise 25.7 (Higher-order ODE as a system)**.** The damped oscillator is $y'' + 3y' + 2y = 0$. Set $\mathbf{u} = \begin{bmatrix} y \\ y' \end{bmatrix}$ and write the equivalent first-order system $\mathbf{u}' = \mathbf{A}\mathbf{u}$. Find the eigenvalues of $\mathbf{A}$ and confirm they are the roots of $\lambda^2 + 3\lambda + 2 = 0$. Conclude the general solution $y(t) = c_1 e^{-t} + c_2 e^{-2t}$, and explain why the system is stable.

Exercise 25.8 (The commutator obstruction)**.** Let $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$. Show $\mathbf{AB} \neq \mathbf{BA}$. Compute $e^{\mathbf{A}}$ and $e^{\mathbf{B}}$ (each series is short), form $e^{\mathbf{A}}e^{\mathbf{B}}$, and compare with $e^{\mathbf{A}+\mathbf{B}}$ where $\mathbf{A}+\mathbf{B} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (this one has eigenvalues ± 1). Confirm they are different, and identify the source of the discrepancy.

Exercise 25.9 (Trace and determinant of the flow)**.** Prove that $\det\!\left(e^{\mathbf{A}t}\right) = e^{t\,\operatorname{tr}\mathbf{A}}$ for diagonalizable $\mathbf{A}$. (Hint: use $e^{\mathbf{A}t} = \mathbf{X}e^{\mathbf{\Lambda}t}\mathbf{X}^{-1}$ and that det is multiplicative.) Interpret: when is the flow volume-preserving?

Exercise 25.10 (Proof: stability)**.** Suppose $\mathbf{A}$ is diagonalizable with all eigenvalues satisfying $\operatorname{Re}\lambda_i < 0$. Using $\mathbf{u}(t) = \sum_i c_i e^{\lambda_i t}\mathbf{x}_i$, prove that $\|\mathbf{u}(t)\| \to 0$ as $t \to \infty$ for *every* initial condition. Where exactly does the negativity of the real parts enter?

Exercise 25.11 (Challenge: the slow mode)**.** A stable system has eigenvalues $\lambda_1 = -0.2$ and $\lambda_2 = -5$ with eigenvectors $\mathbf{x}_1, \mathbf{x}_2$. Argue that for large t, *every* nonzero trajectory $\mathbf{u}(t)$ becomes nearly parallel to $\mathbf{x}_1$, regardless of the start (assuming $c_1 \neq 0$). Quantify "large": roughly how big must t be before the $\mathbf{x}_2$ contribution is under 1% of the $\mathbf{x}_1$ contribution, if they start with equal coefficients? (Use $e^{-5t}/e^{-0.2t} = e^{-4.8t} < 0.01$.)

Chapter 26

Symmetric Matrices and the Spectral Theorem

In Chapter 24 you learned to take a matrix apart with its eigenvectors, and in Chapter 25 you used that diagonalization to solve coupled differential equations $\mathbf{u}' = \mathbf{A}\mathbf{u}$ using the matrix exponential $e^{\mathbf{A}t} = \mathbf{P}e^{\mathbf{\Lambda}t}\mathbf{P}^{-1}$ (writing $\mathbf{P}$ for the eigenvector matrix). The central idea was decoupling: find special directions $\mathbf{x}$ where the matrix acts as pure scaling ($\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$), change to those eigenvector coordinates so each mode evolves independently, and change back. But that general story presented three complications. First, eigenvalues could be complex, turning simple growth and decay into swirling spirals and oscillations. Second, the eigenvectors could be skewed at awkward, non-perpendicular angles, distorting geometry and forcing $\mathbf{P}^{-1}$ to be an expensive matrix inverse. Third, defective matrices lacked enough eigenvectors to diagonalize at all, spawning polynomial growth terms $te^{\lambda t}$ that tangled the modes.

This chapter is about the family of matrices where these complications disappear: **symmetric matrices**. When a matrix equals its own transpose – when $\mathbf{A} = \mathbf{A}^\mathsf{T}$ – the situation changes completely. Its eigenvalues are guaranteed real: no imaginary numbers, no hidden rotations. Its eigenvectors can always be chosen *perpendicular* to one another, forming a rigid coordinate frame. And the change-of-basis matrix $\mathbf{P}$ becomes an **orthogonal matrix** $\mathbf{Q}$ (a pure rotation or reflection) whose inverse simplifies to $\mathbf{Q}^{-1} = \mathbf{Q}^\mathsf{T}$. The diagonalization collapses to

$$\boxed{\mathbf{A} = \mathbf{Q}\,\mathbf{\Lambda}\,\mathbf{Q}^\mathsf{T}}$$

The collection of eigenvalues of a matrix is called its **spectrum** – named by analogy to the spectrum of light, where a glass prism breaks a composite beam into its pure constituent colors. This factorization is the **Spectral Theorem**. It decomposes a symmetric matrix into pure, independent, perpendicular stretching actions. It underlies principal component analysis in machine learning, the geometry of quadratic forms, the vibrational modes of bridges and molecules, the covariance ellipsoids of statistics, and the singular value decomposition. We start with the geometric picture – symmetric matrices stretch space along perpendicular axes – and then prove why symmetry forces both real eigenvalues and orthogonal eigenvectors.

26.1 What Makes a Matrix Symmetric

A square matrix $\mathbf{A}$ is **symmetric** when it is unchanged by transposition: $\mathbf{A}^\mathsf{T} = \mathbf{A}$. Concretely, the entry in row i, column j equals the entry in row j, column i – the matrix is a mirror image of itself across the main diagonal.

$$\mathbf{A} = \begin{bmatrix} 2 & 5 & -1 \\ 5 & 0 & 3 \\ -1 & 3 & 4 \end{bmatrix} \quad \text{is symmetric:} \quad a_{ij} = a_{ji}.$$

Fold the matrix along the diagonal (the $2, 0, 4$) and the off-diagonal entries land on their twins: the two 5's swap, the two 3's swap, the two -1's swap. By contrast

$$\mathbf{B} = \begin{bmatrix} 1 & 2 \\ -2 & 1 \end{bmatrix}$$

is *not* symmetric, because the 2 above the diagonal faces a -2 below it.

Definition 26.1 (Symmetric matrix)**.** A real $n \times n$ matrix $\mathbf{A}$ is **symmetric** if $\mathbf{A}^\mathsf{T} = \mathbf{A}$, equivalently if $a_{ij} = a_{ji}$ for all i, j.

Focus

Symmetry is a statement about the *matrix*, but its real meaning is a statement about the *dot product*. The single identity to carry through this whole chapter is

$$\langle \mathbf{Ax},\, \mathbf{y} \rangle = \langle \mathbf{x},\, \mathbf{Ay} \rangle \qquad \text{for all } \mathbf{x}, \mathbf{y}.$$

"You may move $\mathbf{A}$ from one side of the dot product to the other for free." Every theorem in this chapter falls out of that one sentence.

Why is that the real meaning? Recall the dot product written with matrices: $\langle \mathbf{u},\, \mathbf{v} \rangle = \mathbf{u}^\mathsf{T}\mathbf{v}$. Then

$$\langle \mathbf{Ax},\, \mathbf{y} \rangle = (\mathbf{Ax})^\mathsf{T}\mathbf{y} = \mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{y} = \mathbf{x}^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{y}) = \langle \mathbf{x},\, \mathbf{A}^\mathsf{T}\mathbf{y} \rangle.$$

So $\mathbf{A}$ can always be moved across the dot product if you transpose it on the way. Symmetry, $\mathbf{A}^\mathsf{T} = \mathbf{A}$, is exactly the condition that it crosses *without changing*. Hold onto this; we will use it four times.

Intuition

A general matrix can rotate, stretch, and shear space. A symmetric matrix is much simpler: geometrically it **stretches space along a set of perpendicular axes**. It introduces no twist or non-orthogonal shear between the principal axes. This action corresponds to stretching an elastic membrane along perpendicular axes with different scaling factors.

26.1.1 Where symmetric matrices come from

You might worry that symmetric matrices are a rare, hand-picked species. The opposite is true – they are everywhere, and usually for the same structural reason: a symmetric matrix records a *relationship between pairs* that does not care about order.

- **Gram matrices.** For *any* matrix $\mathbf{M}$, the product $\mathbf{M}^\mathsf{T}\mathbf{M}$ is symmetric, because $(\mathbf{M}^\mathsf{T}\mathbf{M})^\mathsf{T} = \mathbf{M}^\mathsf{T}(\mathbf{M}^\mathsf{T})^\mathsf{T} = \mathbf{M}^\mathsf{T}\mathbf{M}$. Its (i, j) entry is the dot product of column i with column j of $\mathbf{M}$ – and the dot product does not care about order. These appear in least squares as the normal equations $\mathbf{M}^\mathsf{T}\mathbf{M}\mathbf{x} = \mathbf{M}^\mathsf{T}\mathbf{b}$.

- **Covariance matrices.** A covariance matrix measures how pairs of random variables vary together. Because the co-movement of feature i with feature j is identical to that of j with i, every covariance matrix in statistics and machine learning is symmetric. (As you will see, this is simply the Gram matrix of the mean-centered data, up to a scalar factor.)

- **Distances and similarities.** The matrix of pairwise distances or similarities between n objects is symmetric: object i is as far from j as j is from i.

- **Second derivatives.** The **Hessian matrix** of second partial derivatives, $H_{ij} = \partial^2 f/\partial x_i \partial x_j$, captures the local curvature of a multivariable surface in every direction. It is symmetric because mixed partial derivatives commute (Clairaut's theorem: $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$). Modern optimization algorithms rely on the Hessian to curve toward minima.

- **Graphs and networks.** The adjacency matrix of an undirected graph is symmetric: an edge between node i and node j is equally an edge between j and i. So is the **graph Laplacian** $\mathbf{L} = \mathbf{D} - \mathbf{A}$ (the diagonal matrix of node connection counts minus the adjacency matrix), which governs how heat, random walks, and information diffuse across a network.

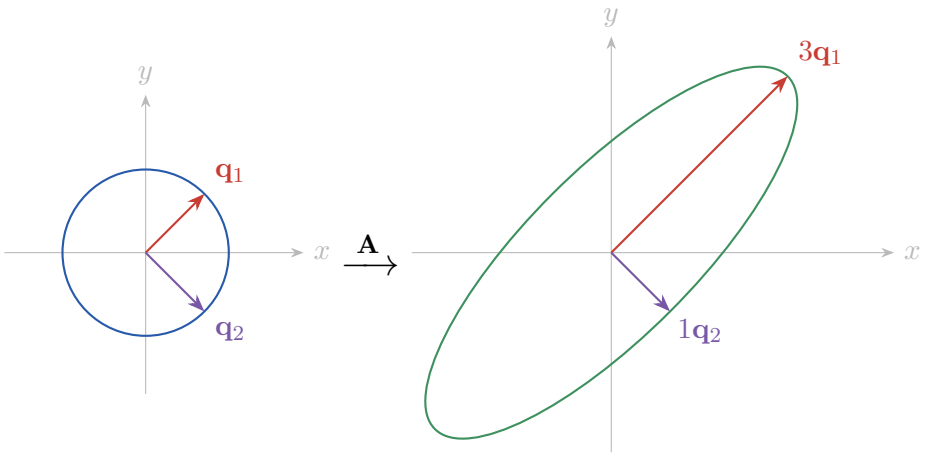
Whenever the underlying relationship is undirected – a pairing with no preferred arrow from one party to the other – the matrix that records it is symmetric. That is why this chapter pays off so widely.

26.2 The Picture: Stretching Along Perpendicular Axes

Before any theorem, let us look. Take the symmetric matrix

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

and watch what it does to the unit circle. The circle is the set of all arrows of length 1; its image under $\mathbf{A}$ tells you everything about how $\mathbf{A}$ stretches space. Figure 26.1 shows the result: the circle becomes an **ellipse**. The long and short axes of that ellipse are *perpendicular*, and they line up with the diagonal directions $(1, 1)$ and $(1, -1)$.



unit circle image: an ellipse

Figure 26.1: The symmetric matrix $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ sends the unit circle to an ellipse. The special directions $\mathbf{q}_1 \propto (1,1)$ and $\mathbf{q}_2 \propto (1,-1)$ are *perpendicular*, and $\mathbf{A}$ simply stretches them by $\lambda_1 = 3$ and $\lambda_2 = 1$. These are the **principal axes**.

Let us verify the picture by hand. Along $(1,1)$:

$$\mathbf{A}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2+1 \\ 1+2 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3\begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

So $(1,1)$ is an eigenvector with eigenvalue 3 – $\mathbf{A}$ stretches it by a factor of 3. Along $(1,-1)$:

$$\mathbf{A}\begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2-1 \\ 1-2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

So $(1,-1)$ is an eigenvector with eigenvalue 1 – $\mathbf{A}$ leaves its length alone. The two eigenvectors are perpendicular: $(1,1) \cdot (1,-1) = 1 - 1 = 0$.

That perpendicularity is not luck. It is forced by symmetry, and it is the heart of the spectral theorem. The long and short axes of the ellipse – the directions along which a symmetric matrix purely stretches – are called the **principal axes**, and they are always at right angles.

Walking question

Consider the matrix $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. We found it stretches $(1,1)$ by 3 and fixes $(1,-1)$. Now picture feeding it a non-eigenvector like $(1,0)$. It must land between the scaling along the two axes. Which way does the output arrow tilt – toward the dominant axis or the smaller one? The output always leans toward the direction of *larger* eigenvalue. Checking directly: $\mathbf{A}(1,0) = (2,1)$, which tilts toward the $(1,1)$ axis.

26.3 Real Eigenvalues

A general matrix can have complex eigenvalues – a rotation matrix has eigenvalues $\cos\theta \pm i\sin\theta$, which live off the real line and correspond to no real stretching direction at all. Symmetric matrices

never do this. Their eigenvalues are always real numbers, every time.

Theorem 26.1 (Symmetric matrices have real eigenvalues)**.** *If* $\mathbf{A}$ *is a real symmetric matrix, then every eigenvalue of* $\mathbf{A}$ *is real.*

The cleanest proof briefly steps into complex numbers, uses the conjugate, and steps back out. It is three lines once you see the trick. We use the **conjugate transpose** $\mathbf{z}^* = \overline{\mathbf{z}}^{\mathsf{T}}$: swap rows and columns while replacing every imaginary number i with $-i$. In ordinary real geometry, $\mathbf{x}^{\mathsf{T}}\mathbf{x} = \sum x_k^2$ computes the squared length of a vector; for complex vectors, the conjugate ensures that $\mathbf{z}^*\mathbf{z} = \sum_k |z_k|^2$ is a *positive real number* whenever $\mathbf{z} \neq \mathbf{0}$.

Proof. Let λ be an eigenvalue (a priori possibly complex) with eigenvector $\mathbf{z} \neq \mathbf{0}$, so $\mathbf{A}\mathbf{z} = \lambda\mathbf{z}$. Form the scalar $\mathbf{z}^*\mathbf{A}\mathbf{z}$ and compute it two ways.

First way. Substitute $\mathbf{A}\mathbf{z} = \lambda\mathbf{z}$:

$$\mathbf{z}^*\mathbf{A}\mathbf{z} = \mathbf{z}^*(\lambda\mathbf{z}) = \lambda\,(\mathbf{z}^*\mathbf{z}).$$

Second way. Take the conjugate transpose of the whole scalar. By conjugate transpose rules, $(\mathbf{z}^*\mathbf{A}\mathbf{z})^* = \mathbf{z}^*\mathbf{A}^*\mathbf{z}$. Because $\mathbf{A}$ is *real* and *symmetric*, $\mathbf{A}^* = \overline{\mathbf{A}}^{\mathsf{T}} = \mathbf{A}^{\mathsf{T}} = \mathbf{A}$, so $(\mathbf{z}^*\mathbf{A}\mathbf{z})^* = \mathbf{z}^*\mathbf{A}\mathbf{z}$. Therefore

$$\mathbf{z}^*\mathbf{A}\mathbf{z} = (\mathbf{A}\mathbf{z})^*\mathbf{z} = (\lambda\mathbf{z})^*\mathbf{z} = \overline{\lambda}\,(\mathbf{z}^*\mathbf{z}).$$

Setting the two equal: $\lambda\,(\mathbf{z}^*\mathbf{z}) = \overline{\lambda}\,(\mathbf{z}^*\mathbf{z})$. Since $\mathbf{z}^*\mathbf{z} > 0$, we may cancel it to get $\lambda = \overline{\lambda}$. A number equal to its own conjugate is real. □

Intuition

A complex eigenvalue corresponds to rotation combined with scaling. A symmetric matrix satisfies $\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{A}\mathbf{y}\rangle$, which prevents rotational coupling across orthogonal directions and restricts the transformation to pure directional scaling. Real eigenvalues are the algebraic signature of this purely scaling behavior.

26.4 Orthogonal Eigenvectors

The second fundamental property of symmetric matrices is that eigenvectors belonging to *different* eigenvalues are automatically perpendicular. The proof relies directly on the symmetry identity $\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{A}\mathbf{y}\rangle$.

Theorem 26.2 (Eigenvectors for distinct eigenvalues are orthogonal)**.** *Let* $\mathbf{A}$ *be real symmetric. If* $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ *and* $\mathbf{A}\mathbf{y} = \mu\mathbf{y}$ *with* $\lambda \neq \mu$*, then* $\langle \mathbf{x}, \mathbf{y}\rangle = 0$*.*

Proof. Compute the single number $\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle$ two ways, sliding $\mathbf{A}$ across the dot product using symmetry.

On one hand, $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$, so

$$\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle = \langle \lambda\mathbf{x}, \mathbf{y}\rangle = \lambda\langle \mathbf{x}, \mathbf{y}\rangle.$$

On the other hand, by symmetry $\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{A}\mathbf{y}\rangle$, and $\mathbf{A}\mathbf{y} = \mu\mathbf{y}$, so

$$\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{A}\mathbf{y}\rangle = \langle \mathbf{x}, \mu\mathbf{y}\rangle = \mu\langle \mathbf{x}, \mathbf{y}\rangle.$$

Subtracting, $(\lambda - \mu)\langle \mathbf{x}, \mathbf{y}\rangle = 0$. Since $\lambda \neq \mu$, the factor $\lambda - \mu \neq 0$, so $\langle \mathbf{x}, \mathbf{y}\rangle = 0$. □

That is the entire argument: push $\mathbf{A}$ onto the eigenvector on the right, pull it onto the eigenvector on the left, and the two eigenvalues must disagree unless the vectors are perpendicular. Three lines, and the perpendicular axes of every ellipse in the chapter are explained.

Example 26.1 (Checking orthogonality numerically)**.** Return to $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ with eigenvectors $\mathbf{x} = (1, 1)$ for $\lambda = 3$ and $\mathbf{y} = (1, -1)$ for $\mu = 1$. Directly checking the dot product: $\langle \mathbf{x}, \mathbf{y}\rangle = (1)(1) + (1)(-1) = 0$. Now a 3×3 example: take

$$\mathbf{B} = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 5 \end{bmatrix}.$$

The bottom-right block already exposes the eigenvector $\mathbf{e}_3 = (0, 0, 1)$ with $\lambda = 5$. The top-left 2×2 block is $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, whose eigenvectors are $(1, 1, 0)$ (eigenvalue 3) and $(1, -1, 0)$ (eigenvalue -1). All three pairwise dot products vanish:

$$(1, 1, 0) \cdot (1, -1, 0) = 0, \quad (1, 1, 0) \cdot (0, 0, 1) = 0, \quad (1, -1, 0) \cdot (0, 0, 1) = 0.$$

Three mutually perpendicular stretching axes, exactly as promised.

Pause

Pause and think: the third direction for free. Suppose a 3×3 symmetric matrix has three distinct eigenvalues, and you have already computed eigenvectors $\mathbf{x}_1 = (1, 1, 0)$ and $\mathbf{x}_2 = (1, -1, 0)$ for the first two. Do you need to set up and solve a third system of equations $(\mathbf{A} - \lambda_3 \mathbf{I})\mathbf{x} = \mathbf{0}$ to find the direction of the third eigenvector?
No: because $\mathbf{A}$ is symmetric and the eigenvalues are distinct, Theorem 26.2 guarantees that the third eigenvector $\mathbf{x}_3$ must be perpendicular to both $\mathbf{x}_1$ and $\mathbf{x}_2$. In $\mathbb{R}^3$, the only vectors perpendicular to the entire xy-plane point along the z-axis, so $\mathbf{x}_3$ must be proportional to $(0, 0, 1)$ (or their cross product $\mathbf{x}_1 \times \mathbf{x}_2$). Orthogonality hands you the final axis with zero row reduction.

Common trap

Theorem 26.2 only guarantees orthogonality across *distinct* eigenvalues. If an eigenvalue is repeated – say a double eigenvalue with a whole *plane* of eigenvectors – two eigenvectors from that same plane need not start out perpendicular. The fix is easy and important: within a repeated eigenvalue's eigenspace you are free to *choose* an orthonormal basis (Gram–Schmidt does it). So you can *always* end up with a full orthonormal set; you just may have to pick wisely inside a shared eigenspace. The full spectral theorem in the next section asserts this can always be done.

26.5 The Spectral Theorem

Combining real eigenvalues (Theorem 26.1) with an orthonormal basis of eigenvectors yields the fundamental factorization of symmetric matrices.

Formal

Theorem 26.3 (Spectral Theorem for real symmetric matrices)**.** *Let* $\mathbf{A}$ *be a real* $n \times n$ *symmetric matrix. Then there is an* ***orthogonal*** *matrix* $\mathbf{Q}$ *(its columns form an orthonormal basis of* $\mathbb{R}^n$*, so* $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ *and* $\mathbf{Q}^{-1} = \mathbf{Q}^\mathsf{T}$*) and a* ***real diagonal*** *matrix* $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ *such that*

$$\mathbf{A} = \mathbf{Q}\,\mathbf{\Lambda}\,\mathbf{Q}^\mathsf{T}.$$

The columns $\mathbf{q}_1, \ldots, \mathbf{q}_n$ *of* $\mathbf{Q}$ *are orthonormal eigenvectors of* $\mathbf{A}$*, and* $\lambda_1, \ldots, \lambda_n$ *are the corresponding (real) eigenvalues. Equivalently, every real symmetric matrix is* ***orthogonally diagonalizable****.*

The single most important structural change from ordinary diagonalization $\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}$ is that $\mathbf{P}^{-1}$ has become $\mathbf{Q}^\mathsf{T}$. The hard, expensive inverse turns into a free transpose, because the eigenvector basis is orthonormal. That one swap is what makes the spectral theorem so computationally and conceptually pleasant.

26.5.1 Reading the factorization as a sum of projections

Multiply $\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ out column-by-column and a cleaner form appears. Writing $\mathbf{Q} = [\mathbf{q}_1 \ \cdots \ \mathbf{q}_n]$, the product expands into

$$\mathbf{A} = \mathbf{Q}\,\mathbf{\Lambda}\,\mathbf{Q}^\mathsf{T} = \sum_{k=1}^{n} \lambda_k\, \mathbf{q}_k \mathbf{q}_k^\mathsf{T}.$$

Each term $\mathbf{q}_k\mathbf{q}_k^\mathsf{T}$ is an $n \times n$ matrix of rank one – it is the **projection** onto the line through $\mathbf{q}_k$ (recall the projection onto a unit vector $\mathbf{q}$ is $\mathbf{q}\mathbf{q}^\mathsf{T}$). So the spectral theorem reads:

Focus

$$\mathbf{A} = \lambda_1\, \mathbf{q}_1\mathbf{q}_1^\mathsf{T} + \lambda_2\, \mathbf{q}_2\mathbf{q}_2^\mathsf{T} + \cdots + \lambda_n\, \mathbf{q}_n\mathbf{q}_n^\mathsf{T}.$$

A symmetric matrix is a **weighted sum of perpendicular projections**. To apply $\mathbf{A}$ to a vector: project onto each principal axis, scale that piece by the axis's eigenvalue, and add the pieces back up. That is *all* a symmetric matrix does.

Figure 26.2 makes this concrete for $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ acting on $\mathbf{x} = (1, 0)$: split $\mathbf{x}$ into its components along the two perpendicular axes, scale each by its eigenvalue, and add the scaled pieces to land exactly on $\mathbf{A}\mathbf{x} = (2, 1)$.

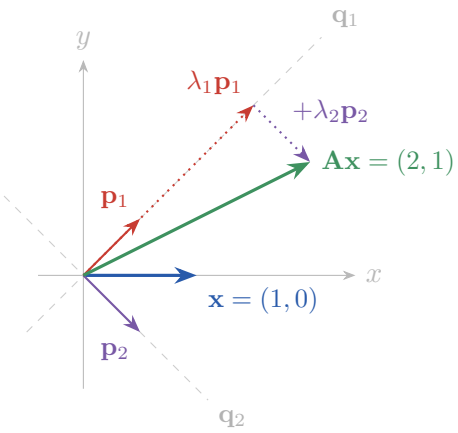


Figure 26.2: The spectral decomposition in action on $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $\mathbf{x} = (1, 0)$. First split $\mathbf{x}$ into its perpendicular components $\mathbf{p}_1 = \mathbf{q}_1\mathbf{q}_1^\mathsf{T}\mathbf{x} = (\frac{1}{2}, \frac{1}{2})$ along the strong axis and $\mathbf{p}_2 = \mathbf{q}_2\mathbf{q}_2^\mathsf{T}\mathbf{x} = (\frac{1}{2}, -\frac{1}{2})$ along the weak axis (so $\mathbf{p}_1 + \mathbf{p}_2 = \mathbf{x}$). Then scale by the eigenvalues $\lambda_1 = 3$ and $\lambda_2 = 1$ and add: $3\mathbf{p}_1 + 1\mathbf{p}_2 = (2, 1) = \mathbf{Ax}$. "Project, scale, add" is the whole machine.

Pause

Pause and think: testing the projection sum. What happens if you multiply the sum $\sum_{k=1}^{n} \lambda_k \mathbf{q}_k \mathbf{q}_k^\mathsf{T}$ directly by the first eigenvector $\mathbf{q}_1$?
Because the basis is orthonormal, $\mathbf{q}_1^\mathsf{T}\mathbf{q}_1 = 1$ while $\mathbf{q}_k^\mathsf{T}\mathbf{q}_1 = 0$ for every $k \neq 1$. Every term in the sum vanishes except the first, leaving $\lambda_1\mathbf{q}_1(1) = \lambda_1\mathbf{q}_1$. The eigenvector equation is hard-wired directly into the sum of projections.

This decomposition is called the **spectral decomposition**, and it is the direct ancestor of the singular value decomposition. When you later drop the smallest $|\lambda_k|$ terms to compress a matrix, you are doing the linear algebra behind PCA and low-rank approximation.

26.5.2 The geometry: rotate, stretch, rotate back

Read $\mathbf{Ax} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}\mathbf{x}$ right-to-left as three moves, just as you read ordinary diagonalization – but now the first and last moves are rigid.

1. $\mathbf{Q}^\mathsf{T}\mathbf{x}$: **rotate** (or reflect) space so the principal axes line up with the coordinate axes. This step measures how much of $\mathbf{x}$ points along each $\mathbf{q}_k$; its k-th entry is $\langle \mathbf{q}_k, \mathbf{x} \rangle$. Because $\mathbf{Q}$ is orthogonal, lengths and angles are perfectly preserved – nothing is distorted yet.

2. $\mathbf{\Lambda}(\cdots)$: **stretch** each coordinate axis by its eigenvalue λ_k (a negative λ_k flips that axis).

3. $\mathbf{Q}(\cdots)$: **rotate back** to the original orientation.

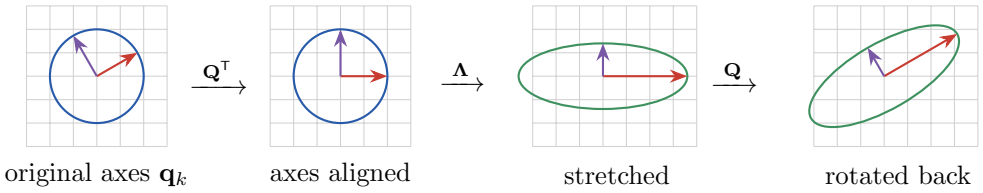


Figure 26.3: The action $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ read right to left. $\mathbf{Q}^\mathsf{T}$ rigidly rotates the principal axes onto the coordinate axes (lengths unchanged), $\mathbf{\Lambda}$ stretches each coordinate axis by its eigenvalue, and a final $\mathbf{Q}$ rotates back. A symmetric matrix is "rotate, stretch, rotate back" – with the *same* rotation undone at the end.

Before/after Strang

Strang calls this $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ and sometimes "$\mathbf{S} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$," reserving $\mathbf{S}$ for symmetric matrices. He summarizes it in three words: *real eigenvalues, orthogonal eigenvectors.* **Before** Strang, the payoff to hold onto is the picture in Figure 26.3: symmetric means perpendicular stretching axes. **After** Strang, notice that the SVD ($\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$) is the same idea generalized to *rectangular* matrices, where the input rotation $\mathbf{V}$ and the output rotation $\mathbf{U}$ are allowed to differ. When $\mathbf{A}$ is positive semidefinite, the spectral theorem is the SVD's symmetric special case, with $\mathbf{U} = \mathbf{V} = \mathbf{Q}$ and $\mathbf{\Sigma} = \mathbf{\Lambda}$ (more generally $\mathbf{\Sigma} = |\mathbf{\Lambda}|$, with $\mathbf{U}$ and $\mathbf{V}$ differing by sign flips on directions with negative eigenvalues).

26.5.3 Why it is true: a proof sketch

We have already proved the two main ingredients: eigenvalues are real (Theorem 26.1) and eigenvectors for distinct eigenvalues are orthogonal (Theorem 26.2). The only gap is *repeated* eigenvalues – we must be sure there are always enough orthonormal eigenvectors to fill an entire orthonormal basis, even when an eigenvalue repeats. The argument runs by induction on the matrix size n, using a technique known as **deflation**: peel off one known eigenvector $\mathbf{q}_1$, and show that the remaining problem drops cleanly into the perpendicular subspace of one smaller dimension, without ever leaking back.

Proof sketch. The case $n = 1$ is trivial ($\mathbf{A}$ is a single real number). Suppose the theorem holds for all symmetric matrices of size $n - 1$. Given a symmetric $n \times n$ matrix $\mathbf{A}$, a real eigenvalue λ_1 exists (the characteristic polynomial has a real root, by Theorem 26.1), with a *unit* eigenvector $\mathbf{q}_1$: $\mathbf{A}\mathbf{q}_1 = \lambda_1\mathbf{q}_1$.

Now look at the **orthogonal complement** $W = \mathbf{q}_1^\perp$, the $(n-1)$-dimensional subspace of all vectors perpendicular to $\mathbf{q}_1$. The key claim is that $\mathbf{A}$ *maps* W *into itself.* To check it, take any $\mathbf{w} \in W$, so $\langle \mathbf{q}_1, \mathbf{w} \rangle = 0$, and test whether $\mathbf{A}\mathbf{w}$ is still perpendicular to $\mathbf{q}_1$:

$$\langle \mathbf{q}_1, \mathbf{A}\mathbf{w} \rangle = \langle \mathbf{A}\mathbf{q}_1, \mathbf{w} \rangle = \langle \lambda_1\mathbf{q}_1, \mathbf{w} \rangle = \lambda_1 \langle \mathbf{q}_1, \mathbf{w} \rangle = 0.$$

(We slid $\mathbf{A}$ across the dot product using symmetry – the same move again.) So $\mathbf{A}\mathbf{w} \in W$. Restricting $\mathbf{A}$ to the $(n-1)$-dimensional space W gives a smaller symmetric operator, which by the induction hypothesis has an orthonormal eigenbasis $\mathbf{q}_2, \ldots, \mathbf{q}_n$ of W. Together with $\mathbf{q}_1$, these form an orthonormal eigenbasis of all of $\mathbb{R}^n$. Stacking them as columns of $\mathbf{Q}$ gives $\mathbf{A}\mathbf{Q} = \mathbf{Q}\mathbf{\Lambda}$, i.e. $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$. □

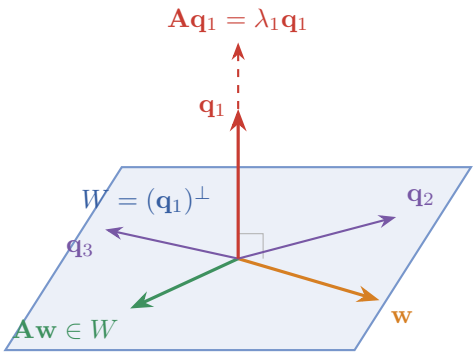


Figure 26.4: The geometric engine of the Spectral Theorem: deflation. Once the first unit eigenvector $\mathbf{q}_1$ is found, symmetry forces $\mathbf{A}$ to map the perpendicular slice $W = \mathbf{q}_1^\perp$ strictly back into itself ($\langle \mathbf{q}_1, \mathbf{A}\mathbf{w}\rangle = \lambda_1\langle \mathbf{q}_1, \mathbf{w}\rangle = 0$). The transformation never "leaks" out of W. We can therefore restrict $\mathbf{A}$ to W and repeat the argument by induction, finding an orthonormal eigenbasis $\mathbf{q}_2, \ldots, \mathbf{q}_n$ within the smaller subspace.

The engine of the whole proof is "peel off one eigenvector, and symmetry guarantees the rest of the matrix still lives cleanly in the perpendicular subspace" (Figure 26.4). Symmetry is exactly what makes the orthogonal complement *invariant*, and that is what lets the induction go through even with repeated eigenvalues.

Why this matters

The spectral theorem is among the most consequential theorems in applied mathematics. It shows that coupled pairwise relationships – covariances, quadratic energies, distances, and physical couplings – decompose into a set of independent, mutually perpendicular one-dimensional actions. Diagonalization decouples the system and reveals the natural axes along which the transformation acts by pure scaling. Vibrating structures possess normal modes; data clouds align along principal components; quantum observables admit stationary energy states. Each is an instance of the spectral decomposition applied in a different mathematical or physical setting.

26.6 Computing the Spectral Decomposition by Hand

Let us run the full procedure on a concrete matrix so the abstract $\mathbf{Q}$, $\mathbf{\Lambda}$ become numbers you can hold.

Example 26.2 (Full spectral decomposition of a 2×2 matrix)**.** Take

$$\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}.$$

Step 1: eigenvalues. Solve $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$:

$$\det \begin{bmatrix} 4-\lambda & 2 \\ 2 & 1-\lambda \end{bmatrix} = (4-\lambda)(1-\lambda) - 4 = \lambda^2 - 5\lambda + 0 = \lambda(\lambda - 5).$$

So $\lambda_1 = 5$ and $\lambda_2 = 0$. Both real, as promised. (A zero eigenvalue just means $\mathbf{A}$ flattens one direction – it is singular.)

Step 2: eigenvectors. For $\lambda_1 = 5$, solve $(\mathbf{A} - 5\mathbf{I})\mathbf{x} = \mathbf{0}$:

$$\begin{bmatrix} -1 & 2 \\ 2 & -4 \end{bmatrix} \mathbf{x} = \mathbf{0} \;\Rightarrow\; -x_1 + 2x_2 = 0 \;\Rightarrow\; \mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

For $\lambda_2 = 0$, solve $\mathbf{A}\mathbf{y} = \mathbf{0}$:

$$\begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix} \mathbf{y} = \mathbf{0} \;\Rightarrow\; 2y_1 + y_2 = 0 \;\Rightarrow\; \mathbf{y} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Check orthogonality: $(2,1) \cdot (1,-2) = 2 - 2 = 0$. Perpendicular, as Theorem 26.2 guarantees.

Step 3: normalize. Both eigenvectors have length $\sqrt{5}$, so

$$\mathbf{q}_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{q}_2 = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Step 4: assemble.

$$\mathbf{Q} = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 & 1 \\ 1 & -2 \end{bmatrix}, \qquad \mathbf{\Lambda} = \begin{bmatrix} 5 & 0 \\ 0 & 0 \end{bmatrix}.$$

Verify $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$: $\frac{1}{5}\begin{bmatrix} 2 & 1 \\ 1 & -2 \end{bmatrix}\begin{bmatrix} 2 & 1 \\ 1 & -2 \end{bmatrix} = \frac{1}{5}\begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix} = \mathbf{I}$. Good – $\mathbf{Q}$ is orthogonal.

Step 5: spectral form. As a sum of projections,

$$\mathbf{A} = 5\,\mathbf{q}_1\mathbf{q}_1^\mathsf{T} + 0\,\mathbf{q}_2\mathbf{q}_2^\mathsf{T} = 5 \cdot \frac{1}{5}\begin{bmatrix} 2 \\ 1 \end{bmatrix}\begin{bmatrix} 2 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}.$$

The whole matrix is recovered from a *single* rank-one projection scaled by 5, because the other eigenvalue is 0. That last picture – a symmetric matrix as a stack of scaled projections – is the one to remember.

Compute it yourself

Diagonalize $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ by hand, following the five steps above. You should find $\lambda = 4$ and $\lambda = 2$, eigenvectors $(1,1)$ and $(1,-1)$, and

$$\mathbf{Q} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{\Lambda} = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}.$$

Then *verify* by multiplying $\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ back out and confirming you recover $\mathbf{A}$. Finally, write $\mathbf{A}$ as $4\,\mathbf{q}_1\mathbf{q}_1^\mathsf{T} + 2\,\mathbf{q}_2\mathbf{q}_2^\mathsf{T}$ and check that too. This shows the transpose doing the work of an inverse.

26.7 The Spectral Theorem and Quadratic Forms

Symmetric matrices are the natural language of **quadratic forms** – expressions like $3x^2 + 4xy + 3y^2$ that are degree-two in the variables. Every such form can be written as $q(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ for a unique

symmetric $\mathbf{A}$. For instance

$$3x^2 + 4xy + 3y^2 = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

The off-diagonal 2's are each *half* the cross-term coefficient 4, split evenly so the matrix comes out symmetric. (That even split is exactly why quadratic forms force symmetry on us.)

The spectral theorem tells you what such a form "really" looks like. Substitute $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ and change to the rotated coordinates $\mathbf{y} = \mathbf{Q}^\mathsf{T}\mathbf{x}$:

$$q(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}\mathbf{x} = (\mathbf{Q}^\mathsf{T}\mathbf{x})^\mathsf{T}\mathbf{\Lambda}(\mathbf{Q}^\mathsf{T}\mathbf{x}) = \mathbf{y}^\mathsf{T}\mathbf{\Lambda}\mathbf{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2.$$

In the right coordinates – the principal axes – the cross terms vanish entirely and the form is a plain weighted sum of squares. The eigenvalues are the weights.

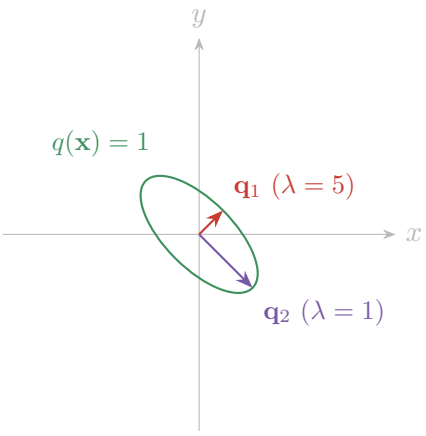


Figure 26.5: The level set $3x^2 + 4xy + 3y^2 = 1$ of a quadratic form is an ellipse whose axes are the eigenvectors of $\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$ (here $\lambda_1 = 5$ along $(1, 1)$, $\lambda_2 = 1$ along $(1, -1)$). A *larger* eigenvalue makes the form grow faster, so the ellipse is *shorter* along that axis (semi-axis length $1/\sqrt{\lambda}$).

This is the bridge to the next chapter on *positive definite* matrices: the form $q(\mathbf{x})$ is positive for every nonzero $\mathbf{x}$ exactly when all the eigenvalues λ_k are positive – because in principal coordinates it is a sum of positive multiples of squares. Symmetry plus positive eigenvalues is the algebra of bowls that curve upward in every direction, which is the geometry of a unique minimum, which is why optimization cares so much.

Connection to other math

Calculus and the second derivative test. Near a critical point, a smooth function of several variables is approximated by a quadratic form whose matrix is the symmetric *Hessian* H of second partials. The spectral theorem diagonalizes H into independent parabolas along its principal axes. The critical point is a *minimum* when all eigenvalues of H are positive (a bowl), a *maximum* when all are negative (a dome), and a *saddle* when the signs disagree – up along one principal axis, down along a perpendicular one. The familiar single-variable second-derivative test ($f'' > 0$ means minimum) is just the 1×1 case, and the multivariable test is the spectral theorem in disguise.

26.8 Why Symmetry Is Everywhere: A Modern Application

Modern application

Principal Component Analysis (PCA). Suppose you have a cloud of data – say n samples, each a point in $\mathbb{R}^d$ (pixels of a face, genes of a cell, features of a customer). Center it so the mean is at the origin and stack the samples as rows of a matrix $\mathbf{X}$. The **covariance matrix**

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}^\mathsf{T}\mathbf{X}$$

is symmetric (it is a Gram matrix, Section 26.1.1), so the spectral theorem applies: $\mathbf{C} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ with real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq 0$ and orthonormal eigenvectors $\mathbf{q}_k$.
Those eigenvectors are the **principal components**, the perpendicular directions along which the data varies. The eigenvalue λ_k is the *variance* of the data along direction $\mathbf{q}_k$. The largest eigenvalue points along the direction of greatest spread – the long axis of the data ellipsoid in Figure 26.1, now drawn by real measurements rather than a chosen matrix.
The payoff: keep only the top few $\mathbf{q}_k$ (largest variances) and you have compressed high-dimensional data onto a few perpendicular axes while losing the least information possible. Eigenfaces, topic models, the "elbow plot" of explained variance, dimensionality reduction before a neural net – all of it is this single computation: *diagonalize a symmetric covariance matrix and read its principal axes.* The reason PCA's directions come out conveniently perpendicular and its variances come out conveniently real is precisely the spectral theorem.

Remark 26.1 (Why symmetry reflects nature). Symmetry is not merely a convenient hypothesis chosen for clean theorems. Quantities such as energy, variance, distance, and mutual coupling are symmetric by construction because they measure undirected relationships between pairs, which is why the spectral theorem applies so widely.

26.9 A Gallery of Symmetric Behaviors

A few small worked cases to anchor the range of what symmetric matrices do.

Example 26.3 (Diagonal matrices: the trivial spectral case)**.** A diagonal matrix $\mathbf{D} = \operatorname{diag}(d_1, \ldots, d_n)$ is already symmetric and already diagonalized: $\mathbf{Q} = \mathbf{I}$, $\mathbf{\Lambda} = \mathbf{D}$. Its principal axes are the standard coordinate axes, and it stretches axis k by d_k. Every symmetric matrix is just a diagonal matrix viewed in a rotated coordinate system. This is the entire content of the spectral theorem in one sentence.

Example 26.4 (A reflection is symmetric)**.** The reflection across the line $y = x$ is

$$\mathbf{R} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{R}^\mathsf{T} = \mathbf{R}.$$

It is symmetric, so it has perpendicular eigenvectors with real eigenvalues. Along the mirror line $(1,1)$ nothing moves: $\mathbf{R}(1,1) = (1,1)$, eigenvalue $+1$. Perpendicular to the mirror, $(1,-1)$ flips: $\mathbf{R}(1,-1) = (-1,1) = -(1,-1)$, eigenvalue -1. So $\mathbf{\Lambda} = \operatorname{diag}(1,-1)$. The *negative* eigenvalue is the algebra of "this axis got flipped." Note a *rotation* (other than by 0 or π) is *not* symmetric – consistent with its complex eigenvalues and its lack of any real stretching axis.

Example 26.5 (Trace and determinant as quick checks)**.** Two sanity checks cost nothing. The sum of eigenvalues equals the trace, and the product equals the determinant. For $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$ from Example 26.2: $\operatorname{tr}\mathbf{A} = 4 + 1 = 5 = \lambda_1 + \lambda_2 = 5 + 0$, and $\det\mathbf{A} = 4 - 4 = 0 = \lambda_1\lambda_2 = 5 \cdot 0$. Both check out. For a 2×2 symmetric matrix this lets you read off the eigenvalues almost by inspection: they are the two numbers summing to the trace and multiplying to the determinant.

Common trap

A real matrix with real eigenvalues is *not* necessarily symmetric. The shear $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has the real (repeated) eigenvalue 1 but is not symmetric and is not even diagonalizable – it has only a one-dimensional line of eigenvectors. Symmetry is a much stronger condition than "real eigenvalues." What symmetry buys you on top of real eigenvalues is the *orthonormal basis* of eigenvectors. Do not confuse the two.

History and connection

The Spectral Theorem was forged in the study of continuum mechanics, elasticity, and celestial perturbations. When an elastic solid is deformed under pressure, the internal forces across any microscopic plane are described by the **stress tensor** (a 3×3 matrix describing the internal forces of tension, compression, and shear pushing across any cross-section of the material). Because conservation of angular momentum requires internal torques to balance, the stress tensor is strictly symmetric: $\sigma_{ij} = \sigma_{ji}$. In arbitrary coordinates, normal forces (compression and tension) are entangled with shear forces that warp right angles. In the 1750s, Leonhard Euler proved that every rotating rigid body has three perpendicular axes where rotation produces no wobble. In elasticity, the fundamental physical question was identical: does there always exist a perpendicular frame of axes where all shearing vanishes entirely, leaving only pure, independent compression or tension?

In 1829, Augustin-Louis Cauchy published the first proof of the principal axis theorem for 3×3 symmetric matrices in celestial mechanics. Yet two controversies lingered for decades. First, why must the eigenvalues of a symmetric matrix be strictly real? General matrices routinely produce pairs of complex conjugate roots corresponding to rotations. Cauchy showed that if $\mathbf{Az} = \lambda\mathbf{z}$ for a complex vector, symmetry forces $\bar{\mathbf{z}}^{\mathsf{T}}\mathbf{Az} = \lambda\|\mathbf{z}\|^2$ to equal its own conjugate transpose, ensuring that every eigenvalue is real. Second, what happens when eigenvalues are repeated? General matrices with repeated eigenvalues frequently become defective (short of eigenvectors). In 1858, Karl Weierstrass delivered the complete, general proof for $n \times n$ matrices using variational methods (finding eigenvalues by maximizing or minimizing the energy ratio $\mathbf{x}^{\mathsf{T}}\mathbf{Ax}/\|\mathbf{x}\|^2$, the Rayleigh quotient, rather than factoring polynomials). He proved that a real symmetric matrix can never be defective: its geometric multiplicity always matches its algebraic multiplicity (the dimension of each eigenspace matches how many times the eigenvalue repeats as a polynomial root), guaranteeing an orthonormal basis of eigenvectors under all circumstances.

What the Spectral Theorem delivers is the cleanest factorization in linear algebra: $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{\mathsf{T}}$, expressing any symmetric operator as a pure stretch along perpendicular directions with an inverse that costs nothing ($\mathbf{Q}^{-1} = \mathbf{Q}^{\mathsf{T}}$). Its main restriction is that it applies only to symmetric matrices; real-world data matrices and directed networks are rarely symmetric, and its eigenvalues may still be negative, creating saddle points (surfaces that curve up along

one axis and down along another) instead of energy bowls. In Chapter 27, we restrict our focus to symmetric matrices with strictly positive eigenvalues, and in Chapter 29, we liberate the Spectral Theorem from the symmetry restriction entirely, applying it to $\mathbf{A}^\mathsf{T}\mathbf{A}$ to reach the singular value decomposition for every rectangular matrix. In infinite dimensions (in quantum mechanics, where physical observables are self-adjoint differential operators), the spectrum can feature continuous unbroken bands rather than discrete eigenvalues, leading to the functional-analytic spectral theorem developed by David Hilbert and John von Neumann; the statistical behavior of eigenvalues in large random symmetric matrices remains an active frontier in mathematical physics.

26.10 Summary

A symmetric matrix $\mathbf{A} = \mathbf{A}^\mathsf{T}$ is, geometrically, a stretch along a set of perpendicular axes. The dot-product identity $\langle \mathbf{Ax}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{Ay} \rangle$ is the engine that makes everything work:

- **Real eigenvalues.** No twisting; only stretching (Theorem 26.1).
- **Orthogonal eigenvectors.** Distinct eigenvalues force perpendicular directions, and repeated ones can always be chosen perpendicular (Theorem 26.2).
- **Spectral theorem.** $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ with $\mathbf{Q}$ orthogonal and $\mathbf{\Lambda}$ real diagonal (Theorem 26.3); equivalently $\mathbf{A} = \sum_k \lambda_k \, \mathbf{q}_k \mathbf{q}_k^\mathsf{T}$, a weighted sum of perpendicular projections.
- **Geometry.** Rotate the principal axes onto the coordinate axes, stretch by the eigenvalues, rotate back – with the inverse rotation delivered for free by a transpose.
- **Cost.** The orthonormal basis provides a major computational advantage: $\mathbf{Q}^{-1} = \mathbf{Q}^\mathsf{T}$, so applying $\mathbf{A}$ or any matrix function of it ($\mathbf{A}^k$, $\sqrt{\mathbf{A}}$, $\mathbf{A}^{-1}$) reduces to "transpose, scale the eigenvalues, transpose back" – with no general matrix inversion required, and orthogonal steps that maintain numerical stability.
- **Why it matters.** Covariances, Hessians, Gram matrices, distances, and undirected networks are all symmetric, so PCA, the second-derivative test, least squares, and the SVD all stand on this theorem.

In Chapter 27, we specialize one notch further: what happens when every eigenvalue is strictly positive? The ellipse becomes a true energy ellipsoid, the quadratic form $q(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{Ax}$ curves upward in every direction like a bowl with a unique minimum at the origin, and the matrix earns the title **positive definite**. Positive definiteness gives the Cholesky factorization $\mathbf{A} = \mathbf{R}^\mathsf{T}\mathbf{R}$, inner products weighted by $\mathbf{A}$, and the local curvature condition for convex optimization.

26.11 Exercises

Exercise 26.1 (Warmup: spot the symmetric ones)**.** Which of these are symmetric? For each symmetric one, state without computing what its principal axes will be (perpendicular, real eigenvalues)

and whether you can predict any eigenvalue by inspection.

$$\begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 3 \\ -3 & 1 \end{bmatrix}, \quad \begin{bmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 5 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Exercise 26.2 (Full decomposition by hand)**.** Find the eigenvalues, orthonormal eigenvectors, $\mathbf{Q}$, and $\mathbf{\Lambda}$ for $\mathbf{A} = \begin{bmatrix} 5 & -2 \\ -2 & 5 \end{bmatrix}$. Verify $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ and that $\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ reproduces $\mathbf{A}$. Then write $\mathbf{A}$ as a sum $\lambda_1 \mathbf{q}_1\mathbf{q}_1^\mathsf{T} + \lambda_2 \mathbf{q}_2\mathbf{q}_2^\mathsf{T}$.

Exercise 26.3 (Gram matrices are always symmetric)**.** Let $\mathbf{M}$ be any (possibly rectangular) real matrix. Prove $\mathbf{M}^\mathsf{T}\mathbf{M}$ is symmetric using only transpose rules. Then compute it for $\mathbf{M} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ and confirm the result is symmetric.

Exercise 26.4 (The orthogonality proof, in your own hand)**.** Let $\mathbf{A}$ be symmetric with $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ and $\mathbf{A}\mathbf{y} = \mu\mathbf{y}$. Starting from the single quantity $\langle \mathbf{A}\mathbf{x}, \mathbf{y}\rangle$, derive $(\lambda - \mu)\langle \mathbf{x}, \mathbf{y}\rangle = 0$ and explain in one sentence why $\lambda \neq \mu$ forces orthogonality. Where exactly did you use $\mathbf{A}^\mathsf{T} = \mathbf{A}$?

Exercise 26.5 (Quadratic form to matrix and back)**.** Write $q(x, y) = 2x^2 - 6xy + 2y^2$ as $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ with $\mathbf{A}$ symmetric. Diagonalize $\mathbf{A}$, find its principal axes, and describe the level set $q(\mathbf{x}) = 1$ (ellipse? hyperbola?). What about the signs of the eigenvalues makes it one or the other?

Exercise 26.6 (Trace and determinant shortcut)**.** A symmetric 2×2 matrix has trace 6 and determinant 5. Without finding the matrix, give its eigenvalues. Are its eigenvectors guaranteed perpendicular? Why?

Exercise 26.7 (Reflections and signs)**.** The matrix $\mathbf{R} = \frac{1}{5}\begin{bmatrix} 3 & 4 \\ 4 & -3 \end{bmatrix}$ is symmetric. Show $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$ (so $\mathbf{R}$ is also orthogonal). Find its eigenvalues – you should get $+1$ and -1 – and interpret $\mathbf{R}$ geometrically as a reflection. Along which line does the $+1$ eigenvector point?

Exercise 26.8 (Functions of a symmetric matrix)**.** Using $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$, explain why $\mathbf{A}^k = \mathbf{Q}\mathbf{\Lambda}^k\mathbf{Q}^\mathsf{T}$, and why one may sensibly define $\sqrt{\mathbf{A}} = \mathbf{Q}\sqrt{\mathbf{\Lambda}}\,\mathbf{Q}^\mathsf{T}$ (taking square roots of the eigenvalues) when all $\lambda_k \geq 0$. Compute $\sqrt{\mathbf{A}}$ for $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (eigenvalues 3 and 1). What goes wrong if some eigenvalue is negative?

Exercise 26.9 (The invariance step, made concrete)**.** The proof sketch hinged on: if $\mathbf{q}_1$ is an eigenvector of symmetric $\mathbf{A}$ and $\mathbf{w} \perp \mathbf{q}_1$, then $\mathbf{A}\mathbf{w} \perp \mathbf{q}_1$ too. Verify this concretely for $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $\mathbf{q}_1 = (1, 1)$, and $\mathbf{w} = (1, -1)$: check $\mathbf{w} \perp \mathbf{q}_1$, compute $\mathbf{A}\mathbf{w}$, and confirm it is still perpendicular to $\mathbf{q}_1$.

Exercise 26.10 (Challenge: a symmetric matrix from a rotation)**.** Let $\mathbf{B} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ be a rotation. It is *not* symmetric (for generic θ). But $\mathbf{B}^\mathsf{T}\mathbf{B}$ is symmetric – compute it and identify the result. What are its eigenvalues, and what does that say about how a rotation stretches lengths? Connect your answer to the fact that rotations are rigid motions.

Exercise 26.11 (Challenge: spectral theorem fails without symmetry)**.** Give a 2×2 real matrix that has two real eigenvalues but whose eigenvectors are *not* perpendicular, demonstrating that orthogonality is a genuine consequence of symmetry and not of real eigenvalues alone. (Hint: try an upper-triangular matrix with distinct diagonal entries.) Then give a 2×2 real matrix with *no* real eigenvalues at all, and explain why it cannot be symmetric.

Chapter 27

Positive Definite Matrices

A fundamental class of symmetric matrices behaves in many respects like *positive numbers*. They have positive eigenvalues and positive pivots, their determinants are positive, and they admit unique positive definite square roots. They define quadratic forms that are strictly positive for every nonzero vector, providing the foundation for convex optimization and for covariance analysis in statistics (which records how pairs of variables fluctuate together). They play the same role as Hessians at local minima in multivariable calculus (matrices of second derivatives certifying upward curvature), stiffness matrices in structural mechanics (measuring resistance to deformation), and positive definite kernels in machine learning (measuring similarity between data points). These are the *positive definite* matrices.

If the spectral theorem of the previous chapter told you that every symmetric matrix is a set of perpendicular axes with a stretch factor on each, then this chapter asks the next natural question: *what if every stretch factor is positive?* The answer turns out to reorganize a huge amount of linear algebra around a single, very physical idea: **energy**.

27.1 The energy of a matrix

27.1.1 From a number to a matrix

Start with the humblest object: a single positive number $a > 0$. What does "positive" buy you? Multiplying by a never flips a sign: ax has the same sign as x. The function $f(x) = ax^2$ is a bowl opening upward with its bottom at $x = 0$; it is positive for every $x \neq 0$. You can take a square root $\sqrt{a}$. Every one of these facts has a matrix analogue, and the matrices that inherit them are exactly the positive definite ones.

Here is the bridge. For a number a, the quantity $ax^2 = x\,a\,x$ is the "energy." For an $n \times n$ symmetric matrix $\mathbf{A}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, the analogue is the scalar

$$q(\mathbf{x}) \;=\; \mathbf{x}^{\mathsf{T}}\mathbf{A}\,\mathbf{x}.$$

This single number – a vector, times a matrix, times the same vector again – runs through the whole chapter. It is called a **quadratic form**: a multivariable polynomial where every single term has

degree two (such as $2x_1^2 + 2x_1x_2 + 3x_2^2$), with no linear or constant pieces. Here is what it actually computes.

Example 27.1 (Unpacking $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ by hand)**.** Take

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Compute $\mathbf{A}\mathbf{x}$ first:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 2x_1 + x_2 \\ x_1 + 3x_2 \end{bmatrix}.$$

Now dot it with $\mathbf{x}$:

$$\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = x_1(2x_1 + x_2) + x_2(x_1 + 3x_2) = 2x_1^2 + 2x_1x_2 + 3x_2^2.$$

The diagonal entries 2 and 3 became the coefficients of x_1^2 and x_2^2. The off-diagonal entry 1 appears *twice* (once as A_{12}, once as A_{21}), and those two copies combine into the single cross term $2x_1x_2$. A symmetric matrix is exactly a bookkeeping device for a quadratic in several variables.

In general, expanding the product gives the clean formula

$$\mathbf{x}^\mathsf{T}\mathbf{A}\,\mathbf{x} \;=\; \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}\,x_i x_j \;=\; \sum_{i} A_{ii}\,x_i^2 \;+\; 2\sum_{i<j} A_{ij}\,x_i x_j,$$

where the last step used $A_{ij} = A_{ji}$. The diagonal carries the pure squares; the off-diagonal carries the interactions.

Intuition

Think of $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ as the *height of a curved surface* above the input point $\mathbf{x}$. For an ordinary number $a > 0$, $f(x) = ax^2$ is a parabola – a 1D valley. For a matrix, $q(\mathbf{x})$ is a multidimensional surface. The entire question of this chapter is: *is that surface a bowl that curves up in every direction?* If yes, $\mathbf{A}$ is positive definite.

Common trap

Only the *symmetric part* of a matrix affects the quadratic form. If $\mathbf{A}$ is not symmetric, then $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{x}^\mathsf{T}\mathbf{A}_{\text{sym}}\,\mathbf{x}$ where $\mathbf{A}_{\text{sym}} = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\mathsf{T})$, because the antisymmetric part contributes nothing: $\mathbf{x}^\mathsf{T}\mathbf{B}\mathbf{x} = 0$ whenever $\mathbf{B}^\mathsf{T} = -\mathbf{B}$. So when we talk about positive definiteness we *always* assume $\mathbf{A}$ is symmetric – otherwise we just replace it by its symmetric part and lose nothing.

27.1.2 The definition

Definition 27.1 (Positive definite)**.** A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is **positive definite** (PD) if

$$\mathbf{x}^\mathsf{T}\mathbf{A}\,\mathbf{x} \;>\; 0 \qquad \text{for every nonzero } \mathbf{x} \in \mathbb{R}^n.$$

It is **positive semidefinite** (PSD) if the weaker $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} \geq 0$ holds for all $\mathbf{x}$. We write $\mathbf{A} \succ 0$ for positive definite and $\mathbf{A} \succeq 0$ for positive semidefinite. Reversing the inequalities gives *negative*

(semi)definite; a matrix whose form takes both positive and negative values is **indefinite** (meaning the surface bends up along some directions and down along others, forming a saddle).

Pause

Mental check. Why does Definition 27.1 demand $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} > 0$ for *nonzero* $\mathbf{x}$, rather than all $\mathbf{x}$? Because at $\mathbf{x} = \mathbf{0}$, the form $\mathbf{0}^\mathsf{T}\mathbf{A}\mathbf{0}$ is identically zero for every matrix. The heart of positive definiteness is that as soon as you step away from the origin in *any* direction, the energy strictly climbs.

The requirement "for every nonzero $\mathbf{x}$" is essential. At $\mathbf{x} = \mathbf{0}$ the form is always 0, so we exclude it; the demand is that *moving away from the origin in any direction whatsoever* strictly increases the value. That is precisely what makes a bowl a bowl.

Focus

Hold onto this one sentence: *positive definite means the quadratic surface* $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ *is a bowl that opens strictly upward in every direction, with its unique lowest point at the origin.* Everything else in this chapter – eigenvalue tests, pivot tests, determinant tests, Cholesky, covariance – is a different way of certifying that one geometric fact.

27.1.3 Three quadratic surfaces

A 2×2 quadratic form can be one of three qualitatively different things. Plotting $z = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ over the plane shows the difference instantly.

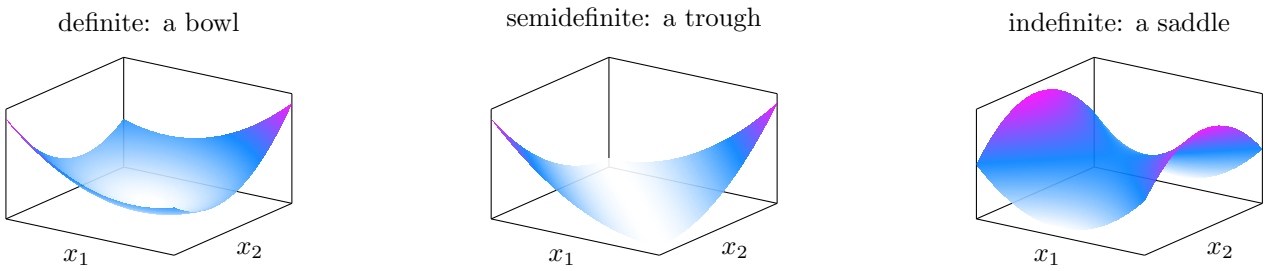


Figure 27.1: The three faces of a quadratic form $z = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$. *Left:* positive definite – every cross-section through the origin curves up, so the only minimum is at the bottom. *Middle:* positive semidefinite – a flat-bottomed trough; it never dips below zero but stays at zero along a whole line. *Right:* indefinite – the form rises in one direction and falls in another, a saddle with no minimum.

The left surface is $2x_1^2 + 2x_1x_2 + 3x_2^2$ from the positive definite $\mathbf{A}$ of Example 27.1. The middle is $(x_1 + x_2)^2$, which is never negative but *equals zero* along the entire line $x_1 = -x_2$ – that is the signature of semidefinite: a direction of zero energy. The right is $x_1^2 - x_2^2$, a saddle: walk along x_1 and the form climbs, walk along x_2 and it drops. Recognizing which picture you are in is what the tests below let you do without ever drawing the surface.

27.2 Test 1: eigenvalues

The cleanest characterization comes straight from the spectral theorem. Because $\mathbf{A}$ is symmetric, it has real eigenvalues $\lambda_1, \dots, \lambda_n$ and an *orthonormal* basis of eigenvectors $\mathbf{q}_1, \dots, \mathbf{q}_n$:

$$\mathbf{A} = \mathbf{Q}\,\mathbf{\Lambda}\,\mathbf{Q}^\mathsf{T}, \qquad \mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}, \quad \mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \dots, \lambda_n).$$

Now substitute this into the energy and change coordinates. Let $\mathbf{y} = \mathbf{Q}^\mathsf{T}\mathbf{x}$ (the coordinates of $\mathbf{x}$ in the eigenvector basis). Then

$$\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{x}^\mathsf{T}\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}\mathbf{x} = (\mathbf{Q}^\mathsf{T}\mathbf{x})^\mathsf{T}\mathbf{\Lambda}\,(\mathbf{Q}^\mathsf{T}\mathbf{x}) = \mathbf{y}^\mathsf{T}\mathbf{\Lambda}\mathbf{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2.$$

This is the most important computation in the chapter. *In the eigenvector coordinates the cross terms vanish.* The messy form $\sum A_{ij}x_i x_j$ becomes a clean weighted sum of pure squares, and the weights are the eigenvalues. Because $\mathbf{Q}$ is an orthogonal matrix, $\mathbf{x} = \mathbf{0}$ if and only if $\mathbf{y} = \mathbf{0}$, so asking "is the form positive for all $\mathbf{x} \neq \mathbf{0}$?" becomes "is $\sum \lambda_i y_i^2 > 0$ for all $\mathbf{y} \neq \mathbf{0}$?" – and that holds exactly when every $\lambda_i > 0$.

Theorem 27.1 (Eigenvalue test)**.** *A symmetric matrix $\mathbf{A}$ is positive definite if and only if all of its eigenvalues are strictly positive. It is positive semidefinite iff all eigenvalues are ≥ 0.*

Proof. ($\Leftarrow$) If every $\lambda_i > 0$ and $\mathbf{x} \neq \mathbf{0}$, then $\mathbf{y} = \mathbf{Q}^\mathsf{T}\mathbf{x} \neq \mathbf{0}$, so at least one $y_k \neq 0$, and $\sum \lambda_i y_i^2 \geq \lambda_k y_k^2 > 0$. ($\Rightarrow$) Conversely, suppose $\mathbf{A} \succ 0$ and let $\mathbf{q}_k$ be a unit eigenvector for λ_k. Plug $\mathbf{x} = \mathbf{q}_k$ into the form: $\mathbf{q}_k^\mathsf{T}\mathbf{A}\mathbf{q}_k = \mathbf{q}_k^\mathsf{T}(\lambda_k\mathbf{q}_k) = \lambda_k\|\mathbf{q}_k\|^2 = \lambda_k$. Positive definiteness forces this to be > 0, so $\lambda_k > 0$. The semidefinite statement is identical with $\geq$. □

Example 27.2 (Reading positivity off the eigenvalues)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ the characteristic equation is

$$\det(\mathbf{A} - \lambda\mathbf{I}) = (2-\lambda)(3-\lambda) - 1 = \lambda^2 - 5\lambda + 5 = 0, \qquad \lambda = \frac{5 \pm \sqrt{5}}{2}.$$

That gives $\lambda_1 \approx 3.618$ and $\lambda_2 \approx 1.382$, both positive, so $\mathbf{A} \succ 0$. Two sanity checks you can do in your head: the trace $\lambda_1 + \lambda_2 = 5 = 2 + 3$ matches the diagonal sum, and the product $\lambda_1\lambda_2 = 5 = \det\mathbf{A}$. Both being positive is *consistent* with PD, though as we will see the determinant alone is not enough.

Common trap

A positive determinant does *not* guarantee positive definiteness. Consider $\begin{bmatrix} -2 & 0 \\ 0 & -3 \end{bmatrix}$: its determinant is $+6 > 0$, but both eigenvalues are negative, so the form $-2x_1^2 - 3x_2^2$ is negative everywhere. A positive determinant in 2D only says the eigenvalues have the *same* sign – they could both be negative. You need more, which is why the determinant test below inspects *all* the leading principal minors, rather than the final determinant alone.

27.3 Test 2: pivots

Computing eigenvalues means solving a polynomial – exact but expensive. There is a much cheaper test hiding inside elimination, the very first algorithm you ever learned. Recall that Gaussian elimination without row swaps factors a matrix as $\mathbf{A} = \mathbf{L}\mathbf{U}$, where $\mathbf{L}$ is lower triangular with ones on the

diagonal and $\mathbf{U}$ is upper triangular. The diagonal entries of $\mathbf{U}$ are the **pivots** (the non-zero leading values used to clear the column entries below them during elimination).

Theorem 27.2 (Pivot test)**.** *A symmetric matrix* $\mathbf{A}$ *is positive definite if and only if elimination runs to completion* without any row exchanges *and produces* n strictly positive pivots.

Pause

Mental check. Suppose you begin elimination on a symmetric matrix $\mathbf{A}$ and find $A_{11} \leq 0$. Can $\mathbf{A}$ possibly be positive definite? No – because A_{11} is the very first pivot d_1, and it also equals the energy of the first standard basis vector: $\mathbf{e}_1^\mathsf{T}\mathbf{A}\mathbf{e}_1 = A_{11}$. If $A_{11} \leq 0$, the test vector $\mathbf{e}_1 = (1, 0, \ldots, 0)^\mathsf{T}$ already gives zero or negative energy, so the matrix fails positive definiteness before elimination takes a single step.

Here is why this is believable, and where it comes from. For a symmetric $\mathbf{A}$ the $\mathbf{LU}$ factorization can be made symmetric: $\mathbf{A} = \mathbf{L}\,\mathbf{D}\,\mathbf{L}^\mathsf{T}$, where $\mathbf{D} = \operatorname{diag}(d_1, \ldots, d_n)$ holds the pivots. (This is the symmetric cousin of $\mathbf{LU}$; $\mathbf{U} = \mathbf{D}\mathbf{L}^\mathsf{T}$.) Substitute into the energy and set $\mathbf{y} = \mathbf{L}^\mathsf{T}\mathbf{x}$:

$$\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{D}\mathbf{L}^\mathsf{T}\mathbf{x} = (\mathbf{L}^\mathsf{T}\mathbf{x})^\mathsf{T}\mathbf{D}(\mathbf{L}^\mathsf{T}\mathbf{x}) = \mathbf{y}^\mathsf{T}\mathbf{D}\mathbf{y} = d_1y_1^2 + d_2y_2^2 + \cdots + d_ny_n^2.$$

Because $\mathbf{L}^\mathsf{T}$ is invertible (ones on the diagonal), $\mathbf{x} \neq \mathbf{0}$ iff $\mathbf{y} \neq \mathbf{0}$. So exactly as with eigenvalues, the form is a weighted sum of squares – only now the weights are the *pivots*, not the eigenvalues. Positivity of the form is equivalent to positivity of every pivot. (The pivots and eigenvalues are generally different numbers; both sets happen to be all-positive together. Only the *determinant* is shared by the two sets, $\det\mathbf{A} = \prod d_i = \prod \lambda_i$; the trace equals the sum of eigenvalues but *not* the sum of pivots – for $\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ the pivots $2, \frac{5}{2}$ sum to $\frac{9}{2}$, while the trace is 5.)

Example 27.3 (Completing the square is elimination)**.** Take $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ again. One elimination step subtracts $\frac{1}{2}$ times row 1 from row 2:

$$\mathbf{A} \longrightarrow \mathbf{U} = \begin{bmatrix} 2 & 1 \\ 0 & \frac{5}{2} \end{bmatrix}, \qquad \text{pivots } d_1 = 2,\ d_2 = \tfrac{5}{2}.$$

Both positive, so $\mathbf{A} \succ 0$. And watch the pivots reappear when you *complete the square* on the quadratic form directly:

$$2x_1^2 + 2x_1x_2 + 3x_2^2 = \underbrace{2}_{d_1}\left(x_1 + \tfrac{1}{2}x_2\right)^2 + \underbrace{\tfrac{5}{2}}_{d_2}x_2^2.$$

The pivot 2 multiplies the completed square; the pivot $\frac{5}{2}$ multiplies the leftover. The multiplier $\frac{1}{2}$ inside the parenthesis is exactly the L_{21} entry from elimination. Two positive coefficients in front of two squares $\Rightarrow$ the form is a sum of nonnegative pieces that vanish only at $\mathbf{x} = \mathbf{0}$. *Completing the square and Gaussian elimination are the same computation.*

Before/after Strang

Strang loves to say "the pivots tell you the signs." This is the place that pays off. In his course the $\mathbf{LDL}^{\mathsf{T}}$ factorization and the energy test appear together, and the punch line is exactly Example 27.3: each pivot is the coefficient of a completed square. If you read this chapter *before* Strang, the thing to carry in is that elimination does far more than solve $\mathbf{Ax} = \mathbf{b}$ – it is also an immediate positivity detector. If you are here *after* Strang, notice that the pivot test is what makes positive definiteness *cheap*: $O(n^3)$ elimination versus solving for eigenvalues.

27.4 Test 3: leading minors (the determinant test)

There is a third test, beautiful because it requires no factorization at all – just determinants of the top-left corners. The kth **leading principal minor** Δ_k is the determinant of the top-left $k \times k$ block of $\mathbf{A}$ (the square submatrix formed by the first k rows and columns).

Theorem 27.3 (Sylvester's criterion)**.** *A symmetric matrix* $\mathbf{A}$ *is positive definite if and only if every leading principal minor is positive:*

$$\Delta_1 = A_{11} > 0, \quad \Delta_2 = \det \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} > 0, \quad \dots, \quad \Delta_n = \det \mathbf{A} > 0.$$

The link to the pivot test makes this almost obvious. The kth pivot equals a ratio of consecutive leading minors,

$$d_k = \frac{\Delta_k}{\Delta_{k-1}}, \qquad (\Delta_0 := 1),$$

because the product of the first k pivots is the determinant of the top-left $k \times k$ block. If all $\Delta_k > 0$ then every ratio $d_k = \Delta_k/\Delta_{k-1} > 0$, so all pivots are positive and $\mathbf{A} \succ 0$ by Theorem 27.2; conversely positive pivots make every $\Delta_k = d_1 d_2 \cdots d_k > 0$.

Example 27.4 (All three tests on one matrix)**.** Let

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix},$$

the famous "second difference" matrix. Run every test.

Leading minors: $\Delta_1 = 2 > 0$; $\Delta_2 = \det \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} = 3 > 0$; $\Delta_3 = \det \mathbf{A} = 2(4-1) - (-1)(-2-0) = 6 - 2 = 4 > 0$. All positive: PD.

Pivots: elimination gives $d_1 = \Delta_1 = 2$, $d_2 = \Delta_2/\Delta_1 = 3/2$, $d_3 = \Delta_3/\Delta_2 = 4/3$. All positive, and their product confirms $2 \cdot \frac{3}{2} \cdot \frac{4}{3} = 4 = \det \mathbf{A}$.

Eigenvalues: for this matrix they are known in closed form, $\lambda_k = 2 - 2\cos\frac{k\pi}{4}$ for $k = 1, 2, 3$, giving $2 - \sqrt{2} \approx 0.586$, 2, and $2 + \sqrt{2} \approx 3.414$ – all positive. Three tests, three different sets of positive numbers, one verdict: $\mathbf{A} \succ 0$.

Compute it yourself

Test $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ by all three methods and confirm they agree that it is *not* positive definite. *Minors:* $\Delta_1 = 1 > 0$ but $\Delta_2 = 1 - 4 = -3 < 0$. The failure of Δ_2 is the red flag. *Pivots:* $d_1 = 1$, then eliminate to get $d_2 = -3$. A negative pivot. *Eigenvalues:* $\lambda = 1 \pm 2$, i.e. 3 and -1. The negative eigenvalue means the form $x_1^2 + 4x_1x_2 + x_2^2$ is a saddle – try $\mathbf{x} = (1, -1)$ to get $1 - 4 + 1 = -2 < 0$.

Common trap

Sylvester's criterion uses the *leading* (top-left) minors and is a test for *positive definiteness only*. Do not try to detect positive *semi*definiteness by relaxing to $\Delta_k \geq 0$ – that is false. The matrix $\begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}$ has $\Delta_1 = 0$ and $\Delta_2 = 0$, both ≥ 0, yet it is negative semidefinite, not positive. For semidefiniteness you must check *all* principal minors (every diagonal sub-block, rather than only the leading ones), or simply fall back to the eigenvalue test $\lambda_i \geq 0$.

27.5 The geometry: ellipses and ellipsoids

We have three algebraic tests. The geometry behind them is what makes positive definiteness feel inevitable, and the right picture is not the 3D bowl but its *level sets* – the curves where the energy is constant.

For a positive definite 2×2 matrix, the set

$$\{\, \mathbf{x} : \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = 1 \,\}$$

is an **ellipse** centered at the origin. (If $\mathbf{A}$ were indefinite you would get a hyperbola; if singular semidefinite, a pair of parallel lines.) The ellipse is the constant-energy contour, and positive definiteness is exactly the statement that these contours are *closed loops* hugging the origin rather than open curves running off to infinity.

The spectral decomposition tells you the ellipse's shape precisely. In the eigenvector coordinates $\mathbf{y} = \mathbf{Q}^\mathsf{T}\mathbf{x}$, the equation $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = 1$ becomes

$$\lambda_1 y_1^2 + \lambda_2 y_2^2 = 1,$$

the standard equation of an ellipse with semi-axis lengths $1/\sqrt{\lambda_1}$ and $1/\sqrt{\lambda_2}$ along the eigenvector directions $\mathbf{q}_1, \mathbf{q}_2$.

Focus

The eigenvectors are the axes of the ellipse; the eigenvalues set the axis lengths. A *large* eigenvalue means the form grows fast in that direction, so the contour is *close* to the origin there (short axis $1/\sqrt{\lambda}$). A *small* eigenvalue means a long, shallow axis. Positive definite $\Leftrightarrow$ both axes are real and finite $\Leftrightarrow$ the contour is a genuine ellipse.

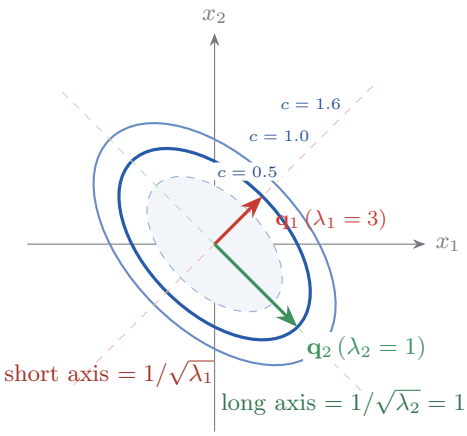


Figure 27.2: Concentric energy contours $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = c$ of the positive definite quadratic form $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. The eigenvectors $\mathbf{q}_1, \mathbf{q}_2$ point along the principal axes. The largest eigenvalue ($\lambda_1 = 3$) steepens the bowl and compresses the level curve into the *short* axis ($1/\sqrt{3}$); the smaller eigenvalue ($\lambda_2 = 1$) yields the *long* axis (1). Positive definiteness ensures every contour is a closed, nested ellipse.

In $\mathbb{R}^n$ the same statement holds: the level set $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = 1$ is an **ellipsoid** with principal axes along the eigenvectors and semi-axis lengths $1/\sqrt{\lambda_i}$. In this sense, a positive definite matrix corresponds directly to a geometric ellipsoid. This geometric interpretation is central whenever covariance matrices or Hessians appear: the level sets of the associated quadratic forms are concentric ellipsoids.

Walking question

Consider the contour $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = 1$ as one eigenvalue decreases toward 0. The semi-axis $1/\sqrt{\lambda}$ in that direction stretches toward infinity, and the ellipse elongates into an unbounded cylinder. When $\lambda = 0$, the matrix is positive semidefinite and the level set is unbounded. If λ becomes negative, the surface becomes a hyperboloid (or hyperbola in two dimensions), reflecting indefiniteness. Thus, strictly positive eigenvalues correspond precisely to closed, bounded ellipsoidal contours.

27.6 Cholesky: the square root of a matrix

A positive number $a > 0$ has a square root $\sqrt{a}$, and you can write $a = (\sqrt{a})(\sqrt{a})$. Positive definite matrices inherit this too, and the matrix version is one of the most useful factorizations in all of computation.

Theorem 27.4 (Cholesky factorization)**.** *A symmetric matrix* $\mathbf{A}$ *is positive definite if and only if it can be written*

$$\mathbf{A} = \mathbf{R}^\mathsf{T}\mathbf{R}$$

for some upper-triangular matrix $\mathbf{R}$ *with* strictly positive *diagonal entries. This* $\mathbf{R}$ *is unique, and is called the* ***Cholesky factor*** *of* $\mathbf{A}$ *– a kind of square root (an upper-triangular matrix that, when multiplied by its transpose, reconstructs* $\mathbf{A}$*).*

Why it exists. We already have $\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^\mathsf{T}$ with positive pivots d_i (the pivot test). Since each $d_i > 0$, define $\mathbf{D}^{1/2} = \operatorname{diag}(\sqrt{d_1}, \ldots, \sqrt{d_n})$, so that $\mathbf{D} = \mathbf{D}^{1/2}\mathbf{D}^{1/2}$. Then

$$\mathbf{A} = \mathbf{L}\mathbf{D}^{1/2}\mathbf{D}^{1/2}\mathbf{L}^\mathsf{T} = \left(\mathbf{D}^{1/2}\mathbf{L}^\mathsf{T}\right)^\mathsf{T}\left(\mathbf{D}^{1/2}\mathbf{L}^\mathsf{T}\right) = \mathbf{R}^\mathsf{T}\mathbf{R},$$

with $\mathbf{R} = \mathbf{D}^{1/2}\mathbf{L}^\mathsf{T}$ upper triangular and positive on the diagonal. Conversely, if $\mathbf{A} = \mathbf{R}^\mathsf{T}\mathbf{R}$ with $\mathbf{R}$ invertible then for $\mathbf{x} \neq \mathbf{0}$, $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{x}^\mathsf{T}\mathbf{R}^\mathsf{T}\mathbf{R}\mathbf{x} = \|\mathbf{R}\mathbf{x}\|^2 > 0$, because $\mathbf{R}\mathbf{x} \neq \mathbf{0}$. So $\mathbf{A} \succ 0$. □

That last line is worth a box of its own, because it is the cleanest reason positive definiteness is unavoidable in data.

Why this matters

The converse of the proof establishes that every matrix of the form $\mathbf{R}^\mathsf{T}\mathbf{R}$ is positive semidefinite, and is positive definite if and only if $\mathbf{R}$ has linearly independent columns. The associated quadratic form is a squared Euclidean norm: $\mathbf{x}^\mathsf{T}\mathbf{R}^\mathsf{T}\mathbf{R}\mathbf{x} = \|\mathbf{R}\mathbf{x}\|^2 \geq 0$. Consequently, Gram matrices, sample covariance matrices, and the normal-equation matrix $\mathbf{A}^\mathsf{T}\mathbf{A}$ in least squares are positive semidefinite by construction, representing squared lengths under linear transformations.

Example 27.5 (Computing a Cholesky factor by hand)**.** Factor $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}$ as $\mathbf{R}^\mathsf{T}\mathbf{R}$ with $\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} \\ 0 & r_{22} \end{bmatrix}$. Multiply out:

$$\mathbf{R}^\mathsf{T}\mathbf{R} = \begin{bmatrix} r_{11}^2 & r_{11}r_{12} \\ r_{11}r_{12} & r_{12}^2 + r_{22}^2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}.$$

Match entries top-left first, then sweep right and down:

$$r_{11}^2 = 4 \Rightarrow r_{11} = 2, \quad r_{11}r_{12} = 2 \Rightarrow r_{12} = 1, \quad r_{12}^2 + r_{22}^2 = 5 \Rightarrow r_{22}^2 = 4 \Rightarrow r_{22} = 2.$$

So $\mathbf{R} = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$, and you can check $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{A}$. The positive diagonal $r_{11} = r_{22} = 2$ certifies $\mathbf{A} \succ 0$. If at any step you were forced to take the square root of a *negative* number, the matrix would not be positive definite – so Cholesky is itself a positivity test, and the cheapest one in practice.

Modern application

Cholesky factorization is central to *sampling from a multivariate Gaussian*, a standard task in Bayesian modeling, Gaussian processes, generative diffusion models, and Monte Carlo simulation. To generate a random vector $\mathbf{z}$ with mean zero and prescribed positive definite covariance $\mathbf{\Sigma}$, one factors $\mathbf{\Sigma} = \mathbf{R}^\mathsf{T}\mathbf{R}$, samples a vector $\mathbf{u}$ of independent standard normal variables ($\operatorname{Cov}(\mathbf{u}) = \mathbf{I}$), and sets $\mathbf{z} = \mathbf{R}^\mathsf{T}\mathbf{u}$. Then

$$\operatorname{Cov}(\mathbf{z}) = \mathbf{R}^\mathsf{T}\operatorname{Cov}(\mathbf{u})\,\mathbf{R} = \mathbf{R}^\mathsf{T}\mathbf{I}\mathbf{R} = \mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{\Sigma}.$$

Linear transformation by $\mathbf{R}^\mathsf{T}$ imparts the required covariance structure. The Cholesky factor also provides an efficient and numerically stable method to solve linear systems $\mathbf{\Sigma}\mathbf{x} = \mathbf{b}$ via triangular substitution in $O(n^2)$ operations, and evaluates the Gaussian log-determinant directly via $\log\det\mathbf{\Sigma} = 2\sum_i \log r_{ii}$.

27.7 Positive definiteness in optimization

Positive definiteness is the multivariable version of "the second derivative is positive," and that is exactly the condition that makes a critical point a genuine *minimum*.

Recall single-variable calculus: at a critical point $f'(x^*) = 0$, you look at $f''(x^*)$. If it is positive the curve bends up and you sit at the bottom of a valley; if negative, a hilltop. The matrix generalization replaces the second derivative by the **Hessian H**, the symmetric matrix of all second partial derivatives (which measures the multivariable curvature of a function in every direction, just as $f''(x)$ measures curvature in single-variable calculus):

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}.$$

The second-order Taylor expansion of a smooth function f near a critical point $\mathbf{x}^*$ (where the gradient vanishes, $\nabla f(\mathbf{x}^*) = \mathbf{0}$) is

$$f(\mathbf{x}^* + \mathbf{h}) \;\approx\; f(\mathbf{x}^*) \;+\; \underbrace{\nabla f(\mathbf{x}^*)^{\mathsf{T}}\mathbf{h}}_{=\,0} \;+\; \tfrac{1}{2}\,\mathbf{h}^{\mathsf{T}}\mathbf{H}\,\mathbf{h}.$$

The first-order term is gone because the gradient is zero. So *the local shape of f around a critical point is governed entirely by the quadratic form* $\mathbf{h}^{\mathsf{T}}\mathbf{H}\mathbf{h}$ – and we now know exactly what its sign tells us.

Theorem 27.5 (Second-derivative test, matrix form)**.** *Let $\mathbf{x}^*$ be a critical point of a smooth $f : \mathbb{R}^n \to \mathbb{R}$ with Hessian $\mathbf{H} = \nabla^2 f(\mathbf{x}^*)$. Then*

- $\mathbf{H} \succ 0$ *(positive definite)* $\Rightarrow$ $\mathbf{x}^*$ *is a strict local* minimum*;*
- $\mathbf{H} \prec 0$ *(negative definite)* $\Rightarrow$ *a strict local* maximum*;*
- $\mathbf{H}$ *indefinite* $\Rightarrow$ *a* saddle point *(a min in some directions, a max in others).*

This is just Figure 27.1 reappearing as a calculus statement: bowl, inverted bowl, saddle. The reason gradient descent and Newton's method work so cleanly near a minimum is that the surface there is, to second order, a positive definite bowl – and a bowl has one bottom you can roll down to.

Example 27.6 (Classifying a critical point)**.** Let $f(x, y) = x^2 + xy + y^2 - 3x$. The gradient is $\nabla f = (2x + y - 3,\; x + 2y)$. Setting it to zero gives $x = 2,\; y = -1$, a single critical point. The Hessian is constant:

$$\mathbf{H} = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Leading minors: $\Delta_1 = 2 > 0$, $\Delta_2 = 4 - 1 = 3 > 0$. Positive definite, so $(2, -1)$ is a strict local minimum – and since f is a globally bowl-shaped quadratic, the global minimum. No plotting required; Sylvester's criterion did it in two determinants.

Before/after Strang

After Strang, this is the bridge to his "optimization" and "positive definite" lectures, through similarity and Jordan form in the next chapter, and onward to the SVD and PCA. The throughline is: *symmetric positive definite matrices are the well-behaved ones*, the ones whose

quadratic minimization problems have a unique answer $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$ given by solving a single linear system. Minimizing $\frac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} - \mathbf{b}^\mathsf{T}\mathbf{x}$ has gradient $\mathbf{A}\mathbf{x} - \mathbf{b}$, which vanishes at $\mathbf{A}\mathbf{x} = \mathbf{b}$; positive definiteness of $\mathbf{A}$ is exactly what guarantees that this critical point is the unique global minimum rather than a saddle. Solving a linear system *is* minimizing a quadratic bowl.

Connection to other math

Positive definiteness is the matrix realization of the general *inner product* concept that runs through analysis. Any positive definite $\mathbf{A}$ defines an inner product $\langle \mathbf{x}, \mathbf{y} \rangle_\mathbf{A} := \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{y}$, which satisfies all standard inner product axioms precisely *because* $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} > 0$ guarantees $\langle \mathbf{x}, \mathbf{x} \rangle_\mathbf{A} > 0$ for $\mathbf{x} \neq \mathbf{0}$. Geometrically, distance is measured with the ellipsoid of Figure 27.2 as the generalized unit sphere. The same construction governs the metric tensor in Riemannian geometry (a positive definite matrix at every point of a smooth manifold), energy norms in differential equations, and reproducing kernel Hilbert spaces in machine learning, where a positive definite kernel function induces an inner product space.

27.8 Covariance: positive definiteness in statistics

The other place positive definite matrices are everywhere is statistics, where the **covariance matrix** of a random vector (a symmetric table whose (i, j) entry measures how much the ith and jth variables fluctuate together) is always positive semidefinite, and positive definite as soon as the data is not degenerate.

Let $\mathbf{X} = (X_1, \ldots, X_n)$ be a random vector with mean μ. Its covariance matrix is

$$\mathbf{\Sigma} = \mathrm{Cov}(\mathbf{X}) = \mathbb{E}\big[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^\mathsf{T}\big],$$

the matrix whose (i, j) entry is $\mathrm{Cov}(X_i, X_j)$. Why must it be positive semidefinite? Take any fixed direction $\mathbf{a} \in \mathbb{R}^n$ and form the scalar random variable $\mathbf{a}^\mathsf{T}\mathbf{X}$ (a weighted combination of the coordinates). Its variance cannot be negative, and a short computation gives

$$0 \;\leq\; \mathrm{Var}(\mathbf{a}^\mathsf{T}\mathbf{X}) = \mathbf{a}^\mathsf{T}\,\mathrm{Cov}(\mathbf{X})\,\mathbf{a} = \mathbf{a}^\mathsf{T}\mathbf{\Sigma}\mathbf{a}.$$

There it is: the quadratic form $\mathbf{a}^\mathsf{T}\mathbf{\Sigma}\mathbf{a}$ is a *variance*, and variances are never negative. So $\mathbf{\Sigma} \succeq 0$ automatically, and it fails to be positive *definite* only when some combination $\mathbf{a}^\mathsf{T}\mathbf{X}$ has zero variance – i.e., the data lies exactly on a lower-dimensional flat (a perfect linear dependence among the variables).

Intuition

For centered data points $\mathbf{x}$, the covariance ellipsoid $\mathbf{x}^\mathsf{T}\mathbf{\Sigma}^{-1}\mathbf{x} = 1$ models the dispersion of the data distribution. Its principal axes (the eigenvectors of $\mathbf{\Sigma}$) align with the directions of maximum and minimum variance; because $\mathbf{\Sigma}^{-1}$ has eigenvalues $1/\lambda_i$, the semi-axis lengths $1/\sqrt{1/\lambda_i} = \sqrt{\lambda_i}$ are the standard deviations along those directions. In principal component analysis (PCA), the principal components are precisely these eigenvectors, ordered by eigenvalue magnitude. Positive definiteness guarantees that the ellipsoid is non-degenerate and full-dimensional.

Example 27.7 (A sample covariance is $\frac{1}{m}\mathbf{X}^\mathsf{T}\mathbf{X}$)**.** Stack m mean-centered data vectors as the rows of a matrix $\mathbf{X} \in \mathbb{R}^{m\times n}$. The sample covariance is $\mathbf{S} = \frac{1}{m}\mathbf{X}^\mathsf{T}\mathbf{X}$. By the Cholesky argument (Theorem 27.4) this is positive semidefinite automatically, since for any $\mathbf{a}$,

$$\mathbf{a}^\mathsf{T}\mathbf{S}\mathbf{a} = \tfrac{1}{m}\,\mathbf{a}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{a} = \tfrac{1}{m}\,\|\mathbf{X}\mathbf{a}\|^2 \;\geq\; 0,$$

again a squared length. It is strictly positive definite exactly when $\mathbf{X}$ has independent columns – when no feature is a linear combination of the others. The same matrix $\mathbf{X}^\mathsf{T}\mathbf{X}$ is the one you invert in the least-squares normal equations, and its positive definiteness is *why* ordinary least squares has a unique solution.

Common trap

In practice, a sample covariance can be singular (only positive semidefinite) – for instance, when there are fewer data points than features ($m < n$) or when columns are linearly dependent. In this setting, $\mathbf{S}$ is not invertible and Cholesky decomposition fails. The standard remedy is *regularization*: replacing $\mathbf{S}$ by $\mathbf{S} + \epsilon\mathbf{I}$ for a parameter $\epsilon > 0$. Adding $\epsilon\mathbf{I}$ shifts every eigenvalue up by ϵ, guaranteeing that the smallest eigenvalue is at least $\epsilon > 0$ and yielding a strictly positive definite matrix. This is the algebraic mechanism behind ridge regression, restoring full-dimensional geometry to the level surfaces (Figure 27.3).

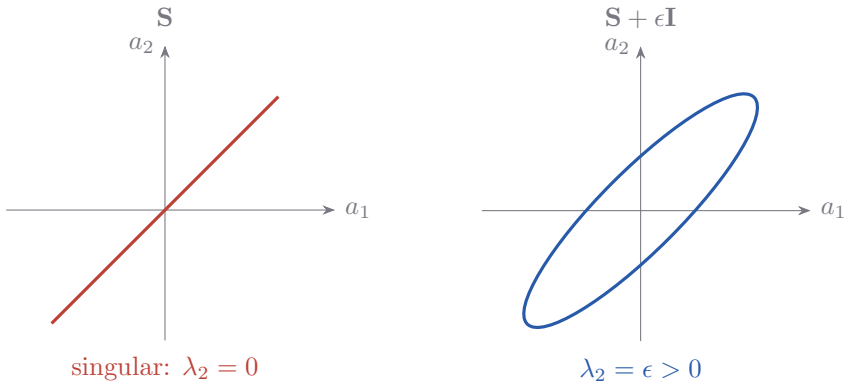


Figure 27.3: Regularization, geometrically. *Left:* a singular (positive semidefinite) covariance $\mathbf{S}$ has a zero eigenvalue, so its covariance ellipsoid collapses to a lower-dimensional segment with zero variance along that axis, preventing inversion. *Right:* adding $\epsilon\mathbf{I}$ shifts every eigenvalue by ϵ, so the smallest eigenvalue becomes $\epsilon > 0$; the collapsed contour expands into a non-degenerate, full-dimensional ellipse that can be inverted and factored by Cholesky decomposition.

27.9 A gallery of consequences

Once a matrix is known to be positive definite, a long list of pleasant facts follows at once. Here is a quick gallery; each is a short consequence of the tests above, and several make good exercises.

- **Invertible, with positive determinant.** All eigenvalues are positive, so none is zero, so $\det \mathbf{A} = \prod \lambda_i > 0$ and $\mathbf{A}^{-1}$ exists.
- **The inverse is also PD.** The eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_i$, still positive.

- **Positive diagonal.** Plugging the standard basis vector $\mathbf{e}_i$ into the form gives $A_{ii} = \mathbf{e}_i^\mathsf{T}\mathbf{A}\mathbf{e}_i > 0$. (Useful as a quick *disqualifier*: a negative or zero diagonal entry instantly rules PD out.)
- **Largest entry on the diagonal.** For a PD matrix the biggest entry in absolute value always sits on the diagonal. Since every 2×2 principal submatrix has positive determinant, $A_{ii}A_{jj} - A_{ij}^2 > 0$, so $|A_{ij}| < \sqrt{A_{ii}A_{jj}} \leq \max(A_{ii}, A_{jj})$: no off-diagonal entry can ever exceed the largest diagonal element.
- **Sums and congruence stay PD.** If $\mathbf{A}, \mathbf{B} \succ 0$ then $\mathbf{A} + \mathbf{B} \succ 0$ (add the energies), and $\mathbf{C}^\mathsf{T}\mathbf{A}\mathbf{C} \succ 0$ for any invertible $\mathbf{C}$ (called a *congruence* transformation, which corresponds to changing coordinates in a quadratic form: if $\mathbf{y} = \mathbf{C}\mathbf{x}$, then $\mathbf{x}^\mathsf{T}(\mathbf{C}^\mathsf{T}\mathbf{A}\mathbf{C})\mathbf{x} = \mathbf{y}^\mathsf{T}\mathbf{A}\mathbf{y} > 0$).
- **A unique PD square root.** There is a unique positive definite $\mathbf{A}^{1/2}$ with $\mathbf{A}^{1/2}\mathbf{A}^{1/2} = \mathbf{A}$, gotten by square-rooting the eigenvalues: $\mathbf{A}^{1/2} = \mathbf{Q}\,\mathrm{diag}(\sqrt{\lambda_i})\,\mathbf{Q}^\mathsf{T}$. (Different from the triangular Cholesky factor, but a square root all the same.)

Focus

The central takeaway: *positive definite matrices are the multidimensional counterpart to positive numbers.* They are invertible, admit unique square roots, remain closed under positive linear combinations, and ensure that quadratic optimization problems have unique global minimizers. When a theorem in optimization, statistics, or numerical analysis assumes $\mathbf{A} \succ 0$, it relies on this strictly convex, upward-curving geometry.

Connection to other math

Non-symmetric matrices: looking ahead to Chapter 28. In this chapter and the previous one, symmetric matrices provided orthogonal eigenvectors and real eigenvalues, while positive definiteness guaranteed that quadratic energy forms curve strictly upward in all directions. Under an orthogonal change of coordinates $\mathbf{Q}^\mathsf{T}\mathbf{A}\mathbf{Q}$, the quadratic form decouples into independent positive squares.

Yet most matrices in applications are not symmetric: transition matrices in Markov chains, adjacency matrices of directed graphs, and dynamical systems with friction or fluid flow. Without symmetry, eigenvectors need not be perpendicular, eigenvalues can be complex numbers, and defective matrices lack a full basis of eigenvectors.

In Chapter 28, we explore **similarity** ($\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$) to analyze what survives when diagonalization fails and how the **Jordan canonical form** captures defective transformations. Then in Chapters 29 and 30, we return to the tools of this chapter: by forming the symmetric positive semidefinite matrix $\mathbf{A}^\mathsf{T}\mathbf{A}$, we will establish that *every* matrix – square or rectangular – possesses singular values and a principal ellipsoid of its own.

History and connection

The theory of positive definite matrices originated in the physical mechanics of conservative systems (physical systems where total mechanical energy is conserved and forces arise as the gradients of a potential energy). In his 1788 treatise *Mécanique Analytique*, Joseph-Louis Lagrange formulated the universal principle of mechanical stability: an equilibrium resting position of a mechanical system is stable if and only if its potential energy achieves a strict local minimum. Expanding potential energy in a multivariable Taylor series reveals that while forces vanish at equilibrium ($\nabla V = \mathbf{0}$), whether an equilibrium is stable or unstable is

determined by the Hessian matrix of second derivatives. Lagrange needed an algebraic test on the quadratic form $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ to guarantee it curved upward in every direction without having to compute all eigenvalues by hand with paper and pencil.

Lagrange's stability criterion sparked lasting controversy. Lagrange believed that a strict local minimum of potential energy was unconditionally sufficient for stability in all mechanical systems. In 1846, Peter Gustav Lejeune Dirichlet provided a rigorous proof for analytic potentials, but whether the converse held – meaning that an equilibrium must be unstable if the potential is not a minimum – remained open for over a century, yielding subtle counterexamples for non-analytic potentials (the Painlevé problem). Meanwhile, mathematicians sought efficient algebraic tests. In 1852, English mathematician James Joseph Sylvester proved that instead of checking all $2^n - 1$ subdeterminants (determinants of smaller submatrices), testing only the n nested leading principal minors is completely sufficient, though a fully rigorous inductive demonstration connecting minors to elimination pivots took years to settle amid Victorian rivalries with Arthur Cayley over determinant invariants.

In 1910, French military geodesist (a mathematical surveyor mapping Earth's geometry) André-Louis Cholesky solved the massive normal equations (the linear systems $\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{A}^\mathsf{T}\mathbf{b}$ that solve linear least-squares problems) for topographic triangulation in Crete and North Africa. Cholesky discovered that symmetric positive definite systems admit a triangular square root $\mathbf{A} = \mathbf{R}^\mathsf{T}\mathbf{R}$ (often written $\mathbf{L}\mathbf{L}^\mathsf{T}$ with $\mathbf{L} = \mathbf{R}^\mathsf{T}$). His algorithm required half the arithmetic operations of ordinary Gaussian elimination and was unconditionally stable without row exchanges. Cholesky's method remained a classified French military secret through World War I; Cholesky died in action in 1914, and the method was only revealed posthumously in 1924 by his fellow officer Commandant Benoît in the *Bulletin Géodésique*.

Positive definiteness underlies convex optimization, Cholesky and conjugate-gradient solvers, and covariance ellipsoids whose axes define Mahalanobis distance metrics (statistical distances scaled by variance along each principal axis, so that a step along a noisy direction counts for less than a step along a tight one). Its active frontiers include high-dimensional matrix completion: when only a fraction of entries in a massive data matrix are observed, deciding whether the unobserved entries can be filled in to produce a positive definite matrix is a central challenge in semidefinite programming. Similarly, certifying whether a multivariable polynomial of degree four or higher is everywhere nonnegative (a sum of squares) is NP-hard, making semidefinite relaxations an active domain of modern optimization. Before exploring how positive semidefiniteness applies to rectangular matrices via $\mathbf{A}^\mathsf{T}\mathbf{A}$ in Chapter 29 and principal component analysis in Chapter 30, Chapter 28 first analyzes general non-symmetric matrices and similarity transformations.

27.10 Exercises

Exercise 27.1 (Warmup: expand the form)**.** For $\mathbf{A} = \begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix}$, write out $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ as a polynomial in x_1, x_2. Then complete the square to express it as $d_1(x_1 + c\,x_2)^2 + d_2x_2^2$ and read off the pivots d_1, d_2. Is $\mathbf{A}$ positive definite?

Exercise 27.2 (Three tests agree)**.** For $\mathbf{A} = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$, verify positive definiteness three ways: (a) compute both eigenvalues; (b) compute both pivots by elimination; (c) check the two leading minors.

Confirm that the products $\lambda_1\lambda_2$ and d_1d_2 both equal $\det \mathbf{A}$.

Exercise 27.3 (Find the boundary)**.** Consider $\mathbf{A}(t) = \begin{bmatrix} 2 & t \\ t & 2 \end{bmatrix}$. For which values of t is $\mathbf{A}(t)$ positive definite? Positive semidefinite? Indefinite? Use Sylvester's criterion, and describe what happens to the energy contour as t crosses each boundary value.

Exercise 27.4 (The diagonal disqualifier)**.** Explain, in one or two sentences, why a symmetric matrix with a negative entry anywhere on its diagonal cannot be positive definite. Give a 2×2 example whose diagonal is all positive but which is nonetheless *not* positive definite, and identify the test that catches it.

Exercise 27.5 (Cholesky by hand)**.** Find the Cholesky factor $\mathbf{R}$ (upper triangular, positive diagonal) of $\mathbf{A} = \begin{bmatrix} 9 & 3 \\ 3 & 5 \end{bmatrix}$ so that $\mathbf{A} = \mathbf{R}^{\mathsf{T}}\mathbf{R}$. Then use $\mathbf{R}$ to compute $\det \mathbf{A}$ from the diagonal of $\mathbf{R}$ alone (no cofactor expansion).

Exercise 27.6 (Classify the critical points)**.** Let $f(x, y) = x^3 - 3x + y^2$. Find all critical points, compute the Hessian at each, and use the second-derivative test (Theorem 27.5) to classify each as a local minimum, local maximum, or saddle. Note that the Hessian is *not* constant here.

Exercise 27.7 ($\mathbf{A}^{\mathsf{T}}\mathbf{A}$ is always PSD)**.** Let $\mathbf{B}$ be any (not necessarily square) real matrix. Prove that $\mathbf{B}^{\mathsf{T}}\mathbf{B}$ is positive semidefinite, and that it is positive definite if and only if the columns of $\mathbf{B}$ are linearly independent. (Hint: look at $\mathbf{x}^{\mathsf{T}}\mathbf{B}^{\mathsf{T}}\mathbf{B}\mathbf{x}$ and recognize it as a squared length.)

Exercise 27.8 (Eigenvalues of the regularized matrix)**.** Suppose $\mathbf{S}$ is symmetric positive semidefinite with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$, and at least one $\lambda_i = 0$ (so $\mathbf{S}$ is singular). Show that $\mathbf{S} + \epsilon\mathbf{I}$ is positive definite for every $\epsilon > 0$ by finding its eigenvalues. Why does this justify the "add $\epsilon\mathbf{I}$" regularization method?

Exercise 27.9 (Sum of PD matrices)**.** Prove that if $\mathbf{A}$ and $\mathbf{B}$ are both positive definite (same size), then so is $\mathbf{A} + \mathbf{B}$. Give a one-line energy argument, then explain why the product $\mathbf{AB}$ need *not* even be symmetric, let alone positive definite.

Exercise 27.10 (The $\mathbf{A}$-inner product)**.** Let $\mathbf{A} \succ 0$ and define $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{A}} = \mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{y}$. Verify the three inner-product axioms: symmetry $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{A}} = \langle \mathbf{y}, \mathbf{x} \rangle_{\mathbf{A}}$, linearity in the first slot, and positivity $\langle \mathbf{x}, \mathbf{x} \rangle_{\mathbf{A}} > 0$ for $\mathbf{x} \neq \mathbf{0}$. Which property uses positive definiteness, and which only uses symmetry?

Exercise 27.11 (Challenge: minors and the energy)**.** Prove directly (without invoking the eigenvalue test) that if $\mathbf{A} \succ 0$ then every leading principal submatrix $\mathbf{A}_k$ (the top-left $k \times k$ block) is also positive definite. (Hint: restrict the energy to vectors whose last $n - k$ coordinates are zero.) Explain how this gives one direction of Sylvester's criterion, $\Delta_k > 0$ for all k.

Chapter 28

Similarity and Jordan Form

In Chapters 26 and 27, we saw how cleanly symmetric and positive definite matrices behave. For symmetric matrices, the spectral theorem guarantees a full orthonormal basis of eigenvectors, real eigenvalues, and a diagonal factorization $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$; when the eigenvalues are positive, the associated quadratic forms curve upward in every direction like bowls.

Yet this chapter returns to a central question left open when we introduced diagonalization in Chapter 24: what happens when a square matrix cannot be diagonalized?

Diagonalization is the ideal outcome: in an eigenvector basis, an operator acts simply by scaling each coordinate axis independently. But not every matrix has a full basis of eigenvectors. The standard counterexample is a shear—a transformation that slides horizontal lines sideways proportionally to height:

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Its only eigenvalue is 1 (with algebraic multiplicity 2), but it possesses only *one* line of eigenvectors, not two. Because there is no basis of eigenvectors, no diagonal matrix $\mathbf{D}$ exists. The transformation cannot be decomposed into independent stretches because its action includes a shear component along the invariant line.

This chapter investigates how to represent general linear operators as simply as possible. The underlying concept is *similarity*: two matrices are similar when they represent the same linear operator under different coordinate systems. Diagonalization corresponds to the special case where the basis consists entirely of eigenvectors. When eigenvectors are missing, any square complex matrix can still be brought to a canonical, block-diagonal form called the **Jordan form**. It is diagonal except for superdiagonal 1's that couple generalized eigenvectors into chains. In this chapter, we develop the geometric meaning of similarity, the structure of generalized eigenvectors, and the classification of operators via Jordan blocks.

28.1 Similarity: The Same Operator in a New Basis

28.1.1 The change-of-basis picture

Here is the single idea the whole chapter rests on. A matrix is not really an operator; it is the *coordinates* of an operator once you have fixed a basis. Change the basis and the same operator gets a different matrix. Two matrices that describe the same underlying operator — just in different coordinate systems — are called *similar*.

Intuition

Think of an operator T as a physical motion of space: a rotation, a stretch, a shear. The motion exists before you choose any axes. When you pick a basis and write down the matrix, you are just recording that motion in a particular language. **Similar matrices are translations of the same sentence into different languages.** The change of basis is the dictionary.

Here is the dictionary made explicit. Suppose $\mathbf{A}$ is the matrix of an operator in the standard basis, and you want its matrix in a new basis whose vectors are the columns of an invertible matrix $\mathbf{P}$. To apply the operator to a vector $\mathbf{y}$ given in the new coordinates, you do three things:

1. **Translate into standard coordinates:** multiply by $\mathbf{P}$ to get $\mathbf{Py}$, the same vector spelled in the standard basis.
2. **Apply the operator:** multiply by $\mathbf{A}$ to get $\mathbf{APy}$.
3. **Translate the answer back into new coordinates:** multiply by $\mathbf{P}^{-1}$.

The whole round trip is $\mathbf{P}^{-1}\mathbf{A}\,\mathbf{P}$, and that is the matrix of the same operator in the new basis.

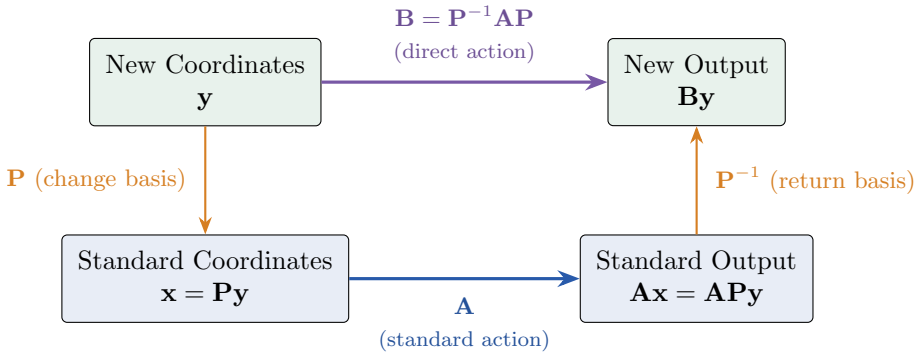


Figure 28.1: The change-of-basis commutative square. Applying $\mathbf{B}$ directly in the new coordinates equals the three-step detour: translate to standard coordinates via $\mathbf{P}$, apply the original transformation $\mathbf{A}$, then translate back via $\mathbf{P}^{-1}$. That geometric equivalence is $\mathbf{B} = \mathbf{P}^{-1}\mathbf{AP}$.

Definition 28.1 (Similar matrices)**.** Two $n \times n$ matrices $\mathbf{A}$ and $\mathbf{B}$ are **similar**, written $\mathbf{A} \sim \mathbf{B}$, if there is an invertible matrix $\mathbf{P}$ with

$$\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\,\mathbf{P}.$$

The matrix $\mathbf{P}$ is the change of basis: its columns are the new basis vectors written in the old coordinates.

Notice that diagonalization, $\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$, says exactly that $\mathbf{D} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ — that is, $\mathbf{A}$ *is similar to a diagonal matrix*. So "diagonalizable" is just a special, lucky case of similarity. The whole chapter is the study of which canonical form you can reach when you are not so lucky.

Focus

$\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ does not change *what* the operator does. It changes only the coordinate language used to describe it. Everything coordinate-free about the operator — its eigenvalues, its rank, its determinant, its trace — must therefore survive the change. That is the next thing we prove.

28.1.2 Similarity is an equivalence relation

Before we mine similar matrices for shared properties, we should check that "similar" behaves the way a notion of sameness should: every matrix is similar to itself, the relation is symmetric, and it chains.

Proposition 28.1. *Similarity is an equivalence relation:* $\mathbf{A} \sim \mathbf{A}$*; if* $\mathbf{A} \sim \mathbf{B}$ *then* $\mathbf{B} \sim \mathbf{A}$*; and if* $\mathbf{A} \sim \mathbf{B}$ *and* $\mathbf{B} \sim \mathbf{C}$ *then* $\mathbf{A} \sim \mathbf{C}$*.*

Proof. Reflexive: take $\mathbf{P} = \mathbf{I}$, so $\mathbf{A} = \mathbf{I}^{-1}\mathbf{A}\,\mathbf{I}$. *Symmetric:* if $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$, multiply to solve for $\mathbf{A}$: $\mathbf{A} = \mathbf{P}\mathbf{B}\mathbf{P}^{-1} = (\mathbf{P}^{-1})^{-1}\mathbf{B}\,(\mathbf{P}^{-1})$, so $\mathbf{B} \sim \mathbf{A}$ via $\mathbf{P}^{-1}$. *Transitive:* if $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ and $\mathbf{C} = \mathbf{Q}^{-1}\mathbf{B}\mathbf{Q}$, substitute: $\mathbf{C} = \mathbf{Q}^{-1}\mathbf{P}^{-1}\mathbf{A}\mathbf{P}\mathbf{Q} = (\mathbf{P}\mathbf{Q})^{-1}\mathbf{A}(\mathbf{P}\mathbf{Q})$, so $\mathbf{A} \sim \mathbf{C}$ via $\mathbf{P}\mathbf{Q}$. □

So the set of all $n \times n$ matrices breaks into disjoint *similarity classes*, each class being one operator seen in all possible bases. The grand goal of this chapter is to pick, from each class, one cleanest representative. For nice operators that representative is diagonal; in general it is the Jordan form.

Pause

Pause and think: Suppose $\mathbf{A}$ and $\mathbf{B}$ are similar matrices. Could $\mathbf{A}$ be invertible while $\mathbf{B}$ is singular? *Reasoning:* No. Since $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$, taking determinants immediately gives $\det\mathbf{B} = \det(\mathbf{P}^{-1})\det(\mathbf{A})\det(\mathbf{P}) = \det\mathbf{A}$. If $\det\mathbf{A} \neq 0$, then $\det\mathbf{B} \neq 0$ as well. Even more directly, we can write down the inverse of $\mathbf{B}$ explicitly: $\mathbf{B}^{-1} = (\mathbf{P}^{-1}\mathbf{A}\mathbf{P})^{-1} = \mathbf{P}^{-1}\mathbf{A}^{-1}\mathbf{P}$. If one is invertible, both are invertible, and their inverses are themselves similar.

28.2 What Similar Matrices Share

28.2.1 Eigenvalues, determinant, trace, rank

Because $\mathbf{A}$ and $\mathbf{B}$ represent the same linear operator in different bases, anything that is really an intrinsic property of the operator (and not of the chosen coordinates) must match. Here are four central invariants.

Theorem 28.1 (Similarity invariants)**.** *If* $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$*, then* $\mathbf{A}$ *and* $\mathbf{B}$ *have the same characteristic polynomial* $p(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I})$ *(the polynomial whose roots are the eigenvalues), hence the same*

eigenvalues with the same algebraic multiplicities. They also have the same determinant, the same trace, and the same rank.

Proof. Compute the characteristic polynomial of $\mathbf{B}$ directly, using $\mathbf{I} = \mathbf{P}^{-1}\mathbf{I}\mathbf{P}$:

$$\det(\mathbf{B} - \lambda\mathbf{I}) = \det(\mathbf{P}^{-1}\mathbf{A}\mathbf{P} - \lambda\mathbf{P}^{-1}\mathbf{I}\mathbf{P}) = \det\big(\mathbf{P}^{-1}(\mathbf{A} - \lambda\mathbf{I})\mathbf{P}\big).$$

Determinant is multiplicative, $\det(\mathbf{X}\mathbf{Y}) = \det\mathbf{X}\det\mathbf{Y}$, and $\det(\mathbf{P}^{-1}) = 1/\det(\mathbf{P})$, so the two $\mathbf{P}$ factors cancel:

$$\det(\mathbf{B} - \lambda\mathbf{I}) = \frac{1}{\det\mathbf{P}}\det(\mathbf{A} - \lambda\mathbf{I})\det\mathbf{P} = \det(\mathbf{A} - \lambda\mathbf{I}).$$

Same characteristic polynomial. Its roots (eigenvalues) and their multiplicities are therefore identical. Setting $\lambda = 0$ gives $\det\mathbf{B} = \det\mathbf{A}$. The trace matches because $\operatorname{tr}(\mathbf{X}\mathbf{Y}) = \operatorname{tr}(\mathbf{Y}\mathbf{X})$ for any conformable $\mathbf{X}, \mathbf{Y}$, so $\operatorname{tr}(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \operatorname{tr}(\mathbf{A}\mathbf{P}\mathbf{P}^{-1}) = \operatorname{tr}\mathbf{A}$. Finally, multiplying by the invertible matrices $\mathbf{P}$ and $\mathbf{P}^{-1}$ never changes rank, so $\operatorname{rank}\mathbf{B} = \operatorname{rank}\mathbf{A}$. □

Compute it yourself

Verify the trace and determinant are preserved with your hands. Take $\mathbf{A} = \begin{bmatrix} 2 & 3 \\ 0 & 3 \end{bmatrix}$ and the shear $\mathbf{P} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ (so $\mathbf{P}^{-1} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$). Compute $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ by hand, in two steps: $\mathbf{A}\mathbf{P} = \begin{bmatrix} 2 & 5 \\ 0 & 3 \end{bmatrix}$, then $\mathbf{P}^{-1}(\mathbf{A}\mathbf{P}) = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix} = \mathbf{B}$. Check: tr = 5 and det = 6 for both, and both have eigenvalues $2, 3$. The off-diagonal entry changed from 3 to 2; the operator did not. (Watch the order: $\mathbf{B}$ is *not* the same as $\mathbf{A}$ here, only similar to it.)

Common trap

Sharing eigenvalues is *necessary* for similarity but **not sufficient**. The two matrices

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

both have the single eigenvalue 1 with algebraic multiplicity 2, the same trace, the same determinant, the same characteristic polynomial. Yet they are *not* similar: the first is the identity (it commutes with everything, so $\mathbf{P}^{-1}\mathbf{I}\mathbf{P} = \mathbf{I}$ always), while the second is a nontrivial shear. Eigenvalues alone do not pin down a similarity class. We will need more — the Jordan form — to tell these apart.

28.2.2 Eigenvectors transform too

Eigenvalues are shared, but the eigenvectors get *relabeled* by the change of basis, which makes perfect sense: an eigenvector is a direction in space, and you wrote it in new coordinates.

Proposition 28.2. *If* $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ *and* $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$*, then* $\mathbf{w} = \mathbf{P}^{-1}\mathbf{v}$ *is an eigenvector of* $\mathbf{B}$ *with the same eigenvalue* λ*.*

Proof. $\mathbf{B}\mathbf{w} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}\,(\mathbf{P}^{-1}\mathbf{v}) = \mathbf{P}^{-1}\mathbf{A}\mathbf{v} = \mathbf{P}^{-1}(\lambda\mathbf{v}) = \lambda(\mathbf{P}^{-1}\mathbf{v}) = \lambda\mathbf{w}$. □

So eigenvalues live in the operator and survive any change of basis; eigenvectors live in the operator too, but their *coordinates* change with the basis. This is exactly why diagonalization works: the special basis is the eigenvector basis, and in those coordinates each eigenvector is a standard axis $\mathbf{e}_i$, on which the operator acts by simple scaling.

Before/after Strang

Strang introduces $\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}$ (he writes $\mathbf{S}$ for the eigenvector matrix and $\mathbf{\Lambda}$ for the diagonal of eigenvalues). Read that equation as a similarity statement: $\mathbf{\Lambda} = \mathbf{S}^{-1}\mathbf{A}\mathbf{S}$, so $\mathbf{A}$ *is similar to* $\mathbf{\Lambda}$. The present chapter is what happens when $\mathbf{S}$ cannot be formed because there are too few eigenvectors — the case Strang flags as "not diagonalizable" and largely sets aside. Here we follow it to its conclusion.

28.3 When Diagonalization Fails

28.3.1 Algebraic versus geometric multiplicity

A matrix is diagonalizable exactly when it has a full basis of eigenvectors — n independent ones for an $n \times n$ matrix. The way that can fail is subtle, and pinning it down means separating two different counts attached to each eigenvalue λ.

Definition 28.2 (Algebraic and geometric multiplicity)**.** Let λ be an eigenvalue of $\mathbf{A}$.

- Its **algebraic multiplicity** is the number of times $(t-\lambda)$ divides the characteristic polynomial $\det(\mathbf{A} - t\mathbf{I})$ — how many times λ shows up as a root.
- Its **geometric multiplicity** is $\dim \operatorname{null}(\mathbf{A} - \lambda\mathbf{I})$ — the dimension of the eigenspace, i.e. how many independent eigenvectors λ actually supplies.

It is a theorem (we will not belabor it) that for every eigenvalue,

$$1 \;\leq\; \text{geometric multiplicity} \;\leq\; \text{algebraic multiplicity}.$$

You always have at least one eigenvector, and you never have more independent eigenvectors than the root multiplicity allows. **Diagonalization succeeds exactly when geometric equals algebraic for every eigenvalue.** When some eigenvalue is "geometrically deficient" — fewer eigenvectors than its algebraic multiplicity demands — the eigenvector basis is short, and no diagonal form exists. That gap (algebraic multiplicity minus geometric multiplicity) is called the *defect* of the eigenvalue.

Intuition

The algebraic multiplicity is the multiplicity of λ as a root of the characteristic polynomial. The geometric multiplicity counts how many independent eigenvector directions the matrix provides. When geometric multiplicity is strictly less than algebraic multiplicity, the matrix is defective, and the deficiency reflects shearing behavior.

28.3.2 The canonical defective matrix

Consider the smallest defective matrix, the shear we met at the start:

$$\mathbf{J} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Its characteristic polynomial is $\det \begin{bmatrix} 1-t & 1 \\ 0 & 1-t \end{bmatrix} = (1-t)^2$, so $\lambda = 1$ has algebraic multiplicity 2. Now hunt for eigenvectors. Solve $(\mathbf{J} - \mathbf{I})\mathbf{v} = \mathbf{0}$:

$$\mathbf{J} - \mathbf{I} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} y \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies y = 0.$$

Every eigenvector is a multiple of $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$. The eigenspace is one-dimensional: geometric multiplicity 1, algebraic multiplicity 2. Defect $= 1$. There is no second independent eigenvector anywhere, so no eigenvector basis, so no diagonalization. The matrix is genuinely, irreparably non-diagonal.

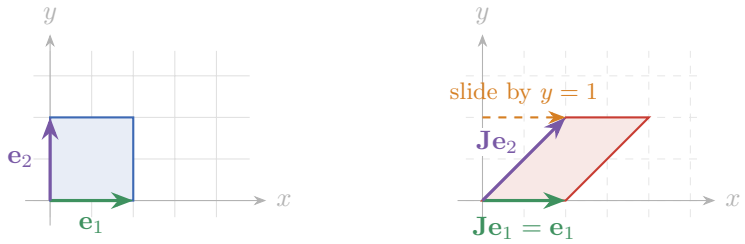


Figure 28.2: The shear $\mathbf{J} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ slides each point (x, y) to $(x + y, y)$. The horizontal axis stays fixed: $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ is the sole eigenvector direction. The vertical vector $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ is tilted to $\mathbf{J}\mathbf{e}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \mathbf{e}_1 + \mathbf{e}_2$. Because every direction off the horizontal axis is tilted toward the horizontal, no second independent eigenvector can exist.

Walking question

Take a walk and ask: why *should* a shear have only one eigenvector? An eigenvector is a direction that the operator merely scales — it does not get rotated off its own line. A shear pushes every direction except the horizontal a little toward the horizontal. So no slanted direction can survive unrotated, and only the horizontal axis is left. The geometry forces the deficiency; the algebra merely records it.

28.4 Generalized Eigenvectors: Filling the Gap

Because $\mathbf{J}$ has only one independent eigenvector, we broaden the definition to vectors annihilated by higher powers of $\mathbf{J} - \lambda\mathbf{I}$. A true eigenvector satisfies $(\mathbf{J} - \mathbf{I})\mathbf{v} = \mathbf{0}$. A *generalized* eigenvector satisfies $(\mathbf{J} - \mathbf{I})^k\mathbf{v} = \mathbf{0}$ for some integer $k \geq 1$, even if $(\mathbf{J} - \mathbf{I})\mathbf{v} \neq \mathbf{0}$.

Definition 28.3 (Generalized eigenvector)**.** A nonzero vector $\mathbf{v}$ is a **generalized eigenvector** of $\mathbf{A}$ for the eigenvalue λ if $(\mathbf{A}-\lambda\mathbf{I})^k\mathbf{v}=\mathbf{0}$ for some integer $k\geq 1$. The smallest such k is its *rank* (or *height*). An ordinary eigenvector is the case $k=1$.

To find one for $\mathbf{J}$, we need a vector that $(\mathbf{J}-\mathbf{I})$ does *not* kill but $(\mathbf{J}-\mathbf{I})^2$ does. First note

$$(\mathbf{J}-\mathbf{I})^2=\begin{bmatrix}0&1\\0&0\end{bmatrix}\begin{bmatrix}0&1\\0&0\end{bmatrix}=\begin{bmatrix}0&0\\0&0\end{bmatrix},$$

the zero matrix. So $(\mathbf{J}-\mathbf{I})^2$ kills *everything* — every vector in $\mathbb{R}^2$ is a generalized eigenvector of height at most 2. Pick one that the first power does not kill, for instance $\mathbf{w}=\begin{bmatrix}0\\1\end{bmatrix}$:

$$(\mathbf{J}-\mathbf{I})\mathbf{w}=\begin{bmatrix}0&1\\0&0\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix}=\begin{bmatrix}1\\0\end{bmatrix}=\mathbf{v}\neq\mathbf{0}.$$

So the first power of $(\mathbf{J}-\mathbf{I})$ does not annihilate $\mathbf{w}$; instead it sends $\mathbf{w}$ onto the genuine eigenvector $\mathbf{v}=\begin{bmatrix}1\\0\end{bmatrix}$. The eigenvector and the generalized eigenvector form a little chain

$$\mathbf{w}\xrightarrow{\mathbf{J}-\mathbf{I}}\mathbf{v}\xrightarrow{\mathbf{J}-\mathbf{I}}\mathbf{0}.$$

This chain is the missing structure. The pair $\{\mathbf{v},\mathbf{w}\}$ is independent (one is $\begin{bmatrix}1\\0\end{bmatrix}$, the other $\begin{bmatrix}0\\1\end{bmatrix}$), so it *is* a basis of $\mathbb{R}^2$ — not a basis of eigenvectors, but a basis of generalized eigenvectors. That is the replacement for the eigenvector basis we could not build.

Intuition

A generalized eigenvector is a vector that is "almost" an eigenvector — it is not fixed by the operator, but applying $(\mathbf{A}-\lambda\mathbf{I})$ repeatedly walks it down a staircase of vectors until it lands on a true eigenvector and then on zero. The length of that staircase is exactly the size of the Jordan block, and the staircase is called a **Jordan chain**.

28.4.1 What the chain does to the matrix

Now build the change of basis from this chain and see what falls out. Order the chain "bottom up": put the eigenvector first, the generalized eigenvector second. So the new basis is $\mathbf{v}=\begin{bmatrix}1\\0\end{bmatrix}$ then $\mathbf{w}=\begin{bmatrix}0\\1\end{bmatrix}$, giving

$$\mathbf{P}=\begin{bmatrix}\mathbf{v}&\mathbf{w}\end{bmatrix}=\begin{bmatrix}1&0\\0&1\end{bmatrix}=\mathbf{I}.$$

(For this particular $\mathbf{J}$ the chain vectors happen to be the standard basis already, which is why $\mathbf{J}$ is its own Jordan form.) The defining relations of the chain are

$$\mathbf{J}\mathbf{v}=\mathbf{v},\qquad \mathbf{J}\mathbf{w}=\mathbf{v}+\mathbf{w}.$$

Read those as columns: the operator sends the first basis vector to $1 \cdot \mathbf{v}$, and the second basis vector to $1 \cdot \mathbf{v} + 1 \cdot \mathbf{w}$. Writing those coefficients as columns gives the matrix in the chain basis:

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

The eigenvalue $\lambda = 1$ sits on the diagonal, and the single 1 *above* the diagonal is the algebraic shadow of the relation $\mathbf{J}\mathbf{w} = \lambda\mathbf{w} + \mathbf{v}$ — the "$+\mathbf{v}$" that says $\mathbf{w}$ is dragged partly onto the eigenvector. That extra 1 is the slide. It is what diagonalization could never capture, and the Jordan form keeps it on purpose.

28.5 The Jordan Form

28.5.1 Jordan blocks

The atom of the theory is a single Jordan block: an eigenvalue λ down the diagonal, with 1's on the superdiagonal.

Definition 28.4 (Jordan block)**.** A **Jordan block** $\mathbf{J}_k(\lambda)$ is the $k \times k$ matrix with λ on the diagonal, 1 on the entries just above the diagonal, and 0 everywhere else:

$$\mathbf{J}_1(\lambda) = \begin{bmatrix} \lambda \end{bmatrix}, \quad \mathbf{J}_2(\lambda) = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}, \quad \mathbf{J}_3(\lambda) = \begin{bmatrix} \lambda & 1 & 0 \\ 0 & \lambda & 1 \\ 0 & 0 & \lambda \end{bmatrix}, \ \ldots$$

A single Jordan block has exactly *one* eigenvector (geometric multiplicity 1) no matter how big it is, but algebraic multiplicity k. So a block of size k carries a defect of $k - 1$: one eigenvector and $k - 1$ generalized eigenvectors, strung in one chain of length k.

Compute it yourself

Confirm the eigenvector count for $\mathbf{J}_3(5)$. Compute $\mathbf{J}_3(5) - 5\mathbf{I} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ and solve $(\mathbf{J}_3(5) - 5\mathbf{I})\mathbf{x} = \mathbf{0}$. You should find the only constraints are $x_2 = 0$ and $x_3 = 0$, so the eigenspace is the single line $\operatorname{span}\{\mathbf{e}_1\}$ — geometric multiplicity 1, even though 5 is a triple root. Then check that $(\mathbf{J}_3(5) - 5\mathbf{I})^3 = \mathbf{0}$ but the square is not zero: the chain has length exactly 3.

28.5.2 The big theorem

Every operator, over the complex numbers, can be assembled out of these blocks — one diagonal staircase per Jordan chain — and that assembly is essentially unique.

Theorem 28.2 (Jordan canonical form)**.** *Let* $\mathbf{A}$ *be any* $n \times n$ *matrix over* $\mathbb{C}$*. Then* $\mathbf{A}$ *is similar to a block-diagonal matrix*

$$\mathbf{J} = \begin{bmatrix} \mathbf{J}_{k_1}(\lambda_1) & & \\ & \ddots & \\ & & \mathbf{J}_{k_m}(\lambda_m) \end{bmatrix},$$

a direct sum of Jordan blocks. The list of blocks — their eigenvalues and sizes — is unique up to the order in which the blocks are written. We call $\mathbf{J}$ *the* ***Jordan canonical form*** *of* $\mathbf{A}$*, and* $\mathbf{A} = \mathbf{PJP}^{-1}$ *for some invertible* $\mathbf{P}$ *whose columns are the Jordan chains.*

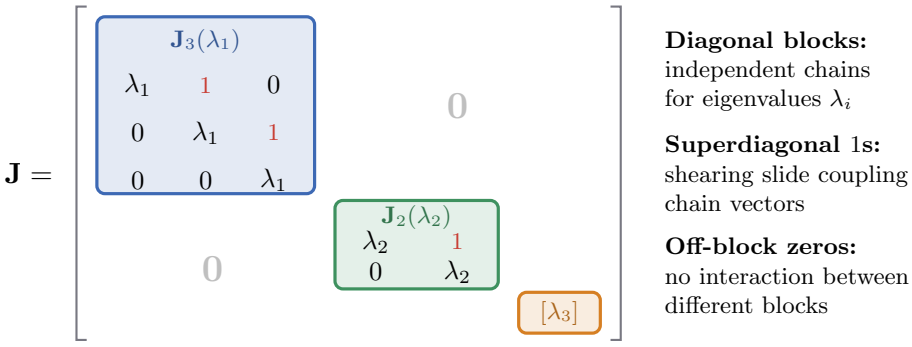


Figure 28.3: The anatomy of a matrix in Jordan canonical form. The operator decomposes into independent diagonal blocks $\mathbf{J}_k(\lambda)$. Within each block, the eigenvalue λ acts by pure scaling along the diagonal, while the superdiagonal 1's couple successive vectors along a Jordan chain. All off-block entries are strictly zero.

We will not prove existence in full — the proof is a careful induction on generalized eigenspaces, more machinery than a first reader needs. But every *idea* in the proof is already in front of you: find the eigenvectors; where you come up short, climb to generalized eigenvectors; bundle each chain into a block. What you should trust and use is the bookkeeping, which is entirely concrete.

Formal

The block structure is read off from ranks of powers of $(\mathbf{A} - \lambda\mathbf{I})$. For a fixed eigenvalue λ:

- The **number of Jordan blocks** for λ equals the geometric multiplicity $\dim \operatorname{null}(\mathbf{A}-\lambda\mathbf{I})$ — one block per genuine eigenvector.
- The **total size** of all blocks for λ equals its algebraic multiplicity.
- The **size of the largest block** for λ equals the smallest k with $\operatorname{null}(\mathbf{A} - \lambda\mathbf{I})^k = \operatorname{null}(\mathbf{A} - \lambda\mathbf{I})^{k+1}$ — the point where the null spaces stop growing.

The exact sizes come from the chain of nullity dimensions $d_j = \dim \operatorname{null}(\mathbf{A}-\lambda\mathbf{I})^j$. The number of blocks of size $\geq j$ is $d_j - d_{j-1}$, so the number of blocks of size exactly j is $(d_j - d_{j-1}) - (d_{j+1} - d_j) = 2d_j - d_{j-1} - d_{j+1}$.

Focus

Two numbers per eigenvalue carry most of the meaning. Algebraic multiplicity = *total* room the eigenvalue occupies on the diagonal. Geometric multiplicity = *number of blocks*. Diagonalizable means every block has size 1, which happens exactly when these two numbers are equal for every eigenvalue. The further apart they are, the bigger the blocks, the more shearing the operator hides.

A picture makes the bookkeeping stick. Stack the blocks for a fixed eigenvalue as rows of boxes — one row per block, the row's length equal to that block's size. The *number of rows* is the geometric multiplicity (one row per genuine eigenvector); the *total number of boxes* is the algebraic multiplicity.

The growing nullities d_j are then just the heights of the columns: d_1 is the number of rows (boxes in the first column), $d_2 - d_1$ is the boxes added in the second column, and so on. Reading the diagram by rows gives block sizes; reading it by columns gives the nullity jumps. The same staircase, two ways.

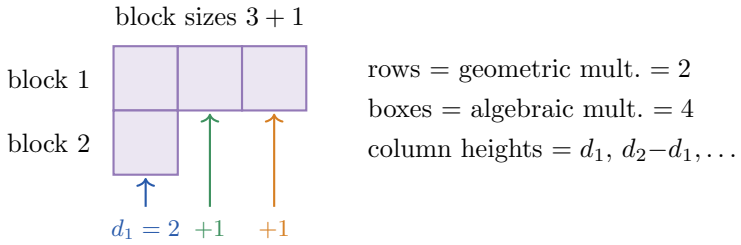


Figure 28.4: The Jordan structure for one eigenvalue drawn as a partition. Each row is a block; its length is the block size. Rows count genuine eigenvectors (geometric multiplicity); total boxes count algebraic multiplicity. The column heights are exactly the nullity increments d_1, $d_2 - d_1$, $d_3 - d_2, \ldots$, which is why the formula $2d_j - d_{j-1} - d_{j+1}$ counts blocks of a given size.

Pause

Pause and think: A 4×4 matrix has a single eigenvalue $\lambda = 3$ of algebraic multiplicity 4. You compute the rank of $\mathbf{A} - 3\mathbf{I}$ and find it is 2. What are the possible Jordan forms? *Reasoning:* By the rank-nullity theorem, the geometric multiplicity is $\dim\operatorname{null}(\mathbf{A} - 3\mathbf{I}) = 4 - 2 = 2$, which means there are exactly 2 Jordan blocks. The block sizes must sum to the algebraic multiplicity 4. The only partitions of 4 into two positive integers are $3+1$ and $2+2$. Thus, the Jordan form is either $\mathbf{J}_3(3) \oplus \mathbf{J}_1(3)$ or $\mathbf{J}_2(3) \oplus \mathbf{J}_2(3)$. One more rank calculation—evaluating the nullity of $(\mathbf{A} - 3\mathbf{I})^2$—decides between them immediately: if $d_2 = 3$, the partition is $3 + 1$; if $d_2 = 4$, it is $2 + 2$.

28.5.3 A worked 3×3 example

Run the whole machine on a concrete matrix that is *not* diagonalizable. Take

$$\mathbf{A} = \begin{bmatrix} 5 & 1 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 5 \end{bmatrix}.$$

The only eigenvalue is 5, with algebraic multiplicity 3 (it is upper triangular, so read the diagonal). Now the eigenvectors. Form

$$\mathbf{N} = \mathbf{A} - 5\mathbf{I} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Solving $\mathbf{Nx} = \mathbf{0}$ forces $x_2 = 0$; x_1 and x_3 are free. So the eigenspace is two-dimensional, $\operatorname{span}\{\mathbf{e}_1, \mathbf{e}_3\}$ — geometric multiplicity 2. By the bookkeeping rules: **2 blocks** (geometric multiplicity), **total size 3** (algebraic multiplicity). The only way to split 3 into two positive block sizes is $2 + 1$. So the Jordan form is

$$\mathbf{J} = \begin{bmatrix} 5 & 1 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 5 \end{bmatrix} = \mathbf{J}_2(5) \oplus \mathbf{J}_1(5).$$

Here $\mathbf{A}$ already *is* in Jordan form. Identifying the chains shows you where the columns of $\mathbf{P}$ come from. We need one chain of length 2 and one of length 1.

Length-1 chain (the lone eigenvector): $\mathbf{e}_3 = \begin{bmatrix}0\\0\\1\end{bmatrix}$ satisfies $\mathbf{N}\mathbf{e}_3 = \mathbf{0}$. That is a block $\mathbf{J}_1(5)$ all by itself.

Length-2 chain: we need a generalized eigenvector $\mathbf{w}$ with $\mathbf{N}\mathbf{w} \neq \mathbf{0}$ but $\mathbf{N}^2\mathbf{w} = \mathbf{0}$. Since $\mathbf{N}^2 = \mathbf{0}$ (check it), every vector dies by the second power; we just need one the first power does not kill. Try $\mathbf{w} = \mathbf{e}_2 = \begin{bmatrix}0\\1\\0\end{bmatrix}$:

$$\mathbf{N}\mathbf{e}_2 = \begin{bmatrix}1\\0\\0\end{bmatrix} = \mathbf{e}_1 \neq \mathbf{0}.$$

So the chain is $\mathbf{e}_2 \to \mathbf{e}_1 \to \mathbf{0}$, with eigenvector $\mathbf{e}_1$ at the bottom. Assembling the basis (each chain bottom-up, blocks side by side) gives $\mathbf{P} = [\,\mathbf{e}_1\ \mathbf{e}_2\ \mathbf{e}_3\,] = \mathbf{I}$, confirming $\mathbf{A} = \mathbf{J}$.

Example 28.1 (A defective matrix that is *not* already in Jordan form)**.** Now a less tidy one, where $\mathbf{P} \neq \mathbf{I}$:

$$\mathbf{A} = \begin{bmatrix}3 & 1\\ -1 & 5\end{bmatrix}.$$

Characteristic polynomial: $\det\begin{bmatrix}3-t & 1\\ -1 & 5-t\end{bmatrix} = (3-t)(5-t)+1 = t^2-8t+16 = (t-4)^2$. Eigenvalue 4, algebraic multiplicity 2. Eigenvectors:

$$\mathbf{N} = \mathbf{A} - 4\mathbf{I} = \begin{bmatrix}-1 & 1\\ -1 & 1\end{bmatrix}, \qquad \mathbf{N}\mathbf{x} = \mathbf{0} \implies -x_1 + x_2 = 0 \implies x_2 = x_1.$$

The eigenspace is the single line $\operatorname{span}\{\begin{bmatrix}1\\1\end{bmatrix}\}$ — geometric multiplicity 1, so **one block** of **size 2**: $\mathbf{J} = \mathbf{J}_2(4) = \begin{bmatrix}4 & 1\\ 0 & 4\end{bmatrix}$. Build the chain. The eigenvector is $\mathbf{v} = \begin{bmatrix}1\\1\end{bmatrix}$. For the generalized eigenvector we need $\mathbf{w}$ with $\mathbf{N}\mathbf{w} = \mathbf{v}$. Solve

$$\begin{bmatrix}-1 & 1\\ -1 & 1\end{bmatrix}\begin{bmatrix}w_1\\w_2\end{bmatrix} = \begin{bmatrix}1\\1\end{bmatrix} \implies -w_1 + w_2 = 1.$$

Any solution works; pick $\mathbf{w} = \begin{bmatrix}0\\1\end{bmatrix}$. Then

$$\mathbf{P} = \begin{bmatrix}\mathbf{v} & \mathbf{w}\end{bmatrix} = \begin{bmatrix}1 & 0\\ 1 & 1\end{bmatrix}, \qquad \mathbf{P}^{-1} = \begin{bmatrix}1 & 0\\ -1 & 1\end{bmatrix}.$$

Verify the similarity:

$$\mathbf{P}^{-1}\mathbf{A}\mathbf{P} = \begin{bmatrix}1 & 0\\ -1 & 1\end{bmatrix}\begin{bmatrix}3 & 1\\ -1 & 5\end{bmatrix}\begin{bmatrix}1 & 0\\ 1 & 1\end{bmatrix} = \begin{bmatrix}1 & 0\\ -1 & 1\end{bmatrix}\begin{bmatrix}4 & 1\\ 4 & 5\end{bmatrix} = \begin{bmatrix}4 & 1\\ 0 & 4\end{bmatrix} = \mathbf{J}_2(4). \checkmark$$

The off-diagonal 1 is exactly the relation $\mathbf{A}\mathbf{w} = 4\mathbf{w} + \mathbf{v}$ written in the chain basis.

Figure 28.5 shows this chain living in the plane. The eigenvector $\mathbf{v} = \begin{bmatrix}1\\1\end{bmatrix}$ marks the one fixed direction; the generalized eigenvector $\mathbf{w} = \begin{bmatrix}0\\1\end{bmatrix}$ is off that line, and $\mathbf{N} = \mathbf{A} - 4\mathbf{I}$ pushes it exactly back onto $\mathbf{v}$.

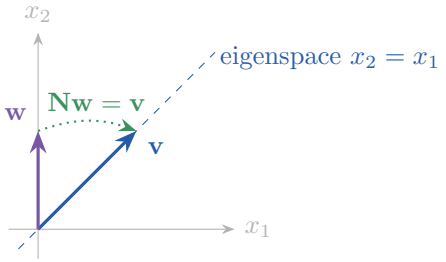


Figure 28.5: The Jordan chain of Example 28.1 drawn in $\mathbb{R}^2$. The eigenvector $\mathbf{v} = \begin{bmatrix}1\\1\end{bmatrix}$ (blue) spans the only fixed line. The generalized eigenvector $\mathbf{w} = \begin{bmatrix}0\\1\end{bmatrix}$ (purple) sits off that line, and $\mathbf{N} = \mathbf{A} - 4\mathbf{I}$ carries $\mathbf{w}$ straight onto $\mathbf{v}$ (green) — one step down the staircase before the eigenvector is annihilated.

28.6 The Geometry of a Jordan Block

A diagonal entry λ scales an axis. The superdiagonal 1 adds a *nilpotent shift*. Splitting a 2×2 block displays this separation directly:

$$\mathbf{J}_2(\lambda) = \begin{bmatrix}\lambda & 1\\ 0 & \lambda\end{bmatrix} = \underbrace{\begin{bmatrix}\lambda & 0\\ 0 & \lambda\end{bmatrix}}_{\text{scale by } \lambda} + \underbrace{\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}}_{\text{shift } \mathbf{N}}.$$

The shift part $\mathbf{N} = \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}$ satisfies $\mathbf{N}^2 = \mathbf{0}$: it is *nilpotent*. Geometrically, $\mathbf{N}$ takes the second basis vector to the first and the first to zero — it pushes everything "down the chain" one step toward the eigenvector and off the edge. The eigenvalue scales; the nilpotent slides. A Jordan block is precisely *scale-and-slide along a chain*, and that slide is the irreducible core of non-diagonalizability.

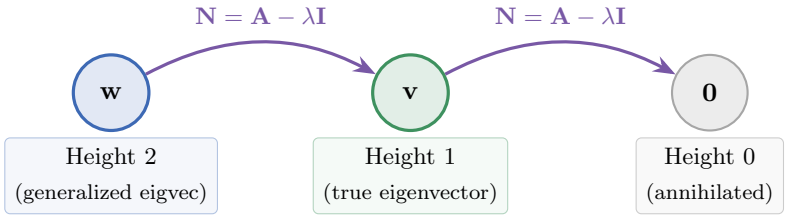


Figure 28.6: A Jordan chain of length 2. The nilpotent part $\mathbf{N} = \mathbf{A} - \lambda\mathbf{I}$ walks the generalized eigenvector $\mathbf{w}$ down to the true eigenvector $\mathbf{v}$, then down to $\mathbf{0}$. The block's size is the length of this staircase.

Why this matters

Diagonalization shows that whenever an operator possesses a full eigenbasis, it acts as a set of independent scalings along coordinate axes. The Jordan form shows that when an eigenbasis is missing, the obstruction consists entirely of coupled chains of generalized eigenvectors sliding into one another. Every square matrix over $\mathbb{C}$ decomposes into a diagonal part and a commuting nilpotent part, where the nilpotent part decomposes into canonical Jordan blocks. This reduces the classification of operators up to change of basis to a catalog of eigenvalues together with integer partitions of block sizes.

Connection to other math

The split $\mathbf{A} = \mathbf{D} + \mathbf{N}$ into a diagonalizable part and a commuting nilpotent part ($\mathbf{DN} = \mathbf{ND}$) is the *Jordan–Chevalley decomposition*. In Lie theory (the study of continuous geometric symmetries) it separates an operator into its "semisimple" component (acting by pure scaling along independent axes) and its "unipotent" component (an identity plus nilpotent part acting by pure sliding). In differential equations it is why solutions to $\mathbf{x}' = \mathbf{Ax}$ for a defective $\mathbf{A}$ pick up polynomial-times-exponential terms like $t\,e^{\lambda t}$: the matrix exponential of a Jordan block is

$$e^{t\mathbf{J}_2(\lambda)} = e^{\lambda t}\begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix},$$

because $e^{t\mathbf{N}} = \mathbf{I} + t\mathbf{N}$ (the series stops since $\mathbf{N}^2 = \mathbf{0}$). The lone superdiagonal 1 in the matrix becomes the lone t in the solution. Resonance in an undamped driven oscillator—where the response amplitude grows like t—is the physical manifestation of a repeated eigenvalue with a nontrivial Jordan block.

28.6.1 Nilpotent matrices are pure Jordan

The nilpotent case deserves its own look, because it is the whole mystery with the eigenvalue stripped away. A matrix $\mathbf{N}$ is *nilpotent* if $\mathbf{N}^k = \mathbf{0}$ for some positive integer k—repeated applications crush every vector to zero. Then its only eigenvalue is 0 (if $\mathbf{Nv} = \lambda\mathbf{v}$ then $\mathbf{0} = \mathbf{N}^k\mathbf{v} = \lambda^k\mathbf{v}$, forcing $\lambda = 0$), so its Jordan form is a direct sum of blocks $\mathbf{J}_k(0)$ — staircases of 1's with zeros on the diagonal. Every defective behavior is, up to shifting by $\lambda\mathbf{I}$, a nilpotent shift. This is why we kept saying "the slide is the core": the eigenvalue is just the diagonal backdrop, and all the interesting structure lives in the nilpotent superdiagonal.

Example 28.2 (Reading blocks from null-space growth)**.** Suppose a 4×4 nilpotent $\mathbf{N}$ has

$$\dim\operatorname{null}\mathbf{N} = 2, \quad \dim\operatorname{null}\mathbf{N}^2 = 3, \quad \dim\operatorname{null}\mathbf{N}^3 = 4, \quad \dim\operatorname{null}\mathbf{N}^4 = 4.$$

So $d_1 = 2, d_2 = 3, d_3 = 4, d_4 = 4$ (with $d_0 = 0$). Number of blocks $= d_1 = 2$. Largest block size $= 3$ (where the null spaces stop growing). The blocks of size exactly j number $2d_j - d_{j-1} - d_{j+1}$: for $j = 1$ that is $2(2) - 0 - 3 = 1$; for $j = 2$ it is $2(3) - 2 - 4 = 0$; for $j = 3$ it is $2(4) - 3 - 4 = 1$. So one block of size 1 and one of size 3: total 4, two blocks. The Jordan form is $\mathbf{J}_3(0) \oplus \mathbf{J}_1(0)$. We never computed an eigenvector explicitly — the dimensions of null spaces told us the whole structure.

28.7 A Modern Application: Stability of Iterations

Modern application

Many algorithms in machine learning and scientific computing are *iterations*: a state updates by $\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t$, so $\mathbf{x}_t = \mathbf{A}^t\mathbf{x}_0$. Think of the power method for eigenvectors, a linear recurrent network, the propagation of a gradient through the unrolled steps of a recurrent model, or a Markov chain mixing to equilibrium. Whether the state blows up, decays, or drifts is governed entirely by the powers $\mathbf{A}^t$ — and the Jordan form is the cleanest way to understand $\mathbf{A}^t$.
If $\mathbf{A}$ is diagonalizable, $\mathbf{A}^t = \mathbf{P}\mathbf{D}^t\mathbf{P}^{-1}$ and each mode just scales by λ_i^t: the spectral radius $\rho(\mathbf{A}) = \max_i |\lambda_i|$ (the largest eigenvalue magnitude, governing asymptotic exponential growth or decay) decides everything. But for a defective $\mathbf{A}$ the powers of a Jordan block carry binomial coefficients:

$$\mathbf{J}_2(\lambda)^t = \begin{bmatrix} \lambda^t & t\,\lambda^{t-1} \\ 0 & \lambda^t \end{bmatrix}, \qquad \mathbf{J}_3(\lambda)^t = \begin{bmatrix} \lambda^t & t\lambda^{t-1} & \binom{t}{2}\lambda^{t-2} \\ 0 & \lambda^t & t\lambda^{t-1} \\ 0 & 0 & \lambda^t \end{bmatrix}.$$

The off-diagonal entries grow *polynomially in* t before the λ^t takes over. This is why an iteration with $|\lambda|$ *exactly* 1 and a nontrivial Jordan block is unstable even though the spectral radius is 1: the $t\,\lambda^{t-1}$ term grows without bound. It is also why deep recurrent networks suffer "exploding/vanishing gradients" that pure eigenvalue intuition underestimates — near-defective weight matrices inject transient polynomial growth into the backpropagated signal.

Those binomial coefficients come straight from the algebra. Expand $\mathbf{J}_k(\lambda) = \lambda\mathbf{I} + \mathbf{N}$ with the binomial theorem, which is legitimate because $\lambda\mathbf{I}$ and $\mathbf{N}$ commute:

$$(\lambda\mathbf{I} + \mathbf{N})^t = \sum_{j=0}^{t} \binom{t}{j} \lambda^{t-j}\mathbf{N}^j.$$

Since $\mathbf{N}^j$ is the matrix with 1's on the j-th superdiagonal and $\mathbf{N}^k = \mathbf{0}$, only finitely many terms survive, and they land on successive superdiagonals with coefficients $\binom{t}{j}\lambda^{t-j}$. That single line explains every entry of the power formulas above.

Pause

Pause and think: If every eigenvalue of $\mathbf{A}$ satisfies $|\lambda_i| < 1$, could a defective Jordan block prevent $\mathbf{A}^t$ from decaying to $\mathbf{0}$ as $t \to \infty$? *Reasoning:* No. Although the off-diagonal entries feature polynomial prefactors like $\binom{t}{j}\lambda^{t-j}$, exponential decay always conquers polynomial growth: for any fixed j and any $|\lambda| < 1$, $\lim_{t\to\infty} t^j|\lambda|^t = 0$. The Jordan block causes a transient surge where $\|\mathbf{A}^t\|$ initially climbs, but the exponential decay eventually dominates and $\mathbf{A}^t \to \mathbf{0}$.

Figure 28.7 makes the danger visible. With eigenvalue exactly on the unit circle, $|\lambda| = 1$, a diagonalizable matrix keeps every entry bounded forever, while a single 2×2 Jordan block has an entry $t\,\lambda^{t-1}$ that grows without bound — slowly, linearly, but without bound. The spectral radius is the same in both cases; only the block structure tells them apart.

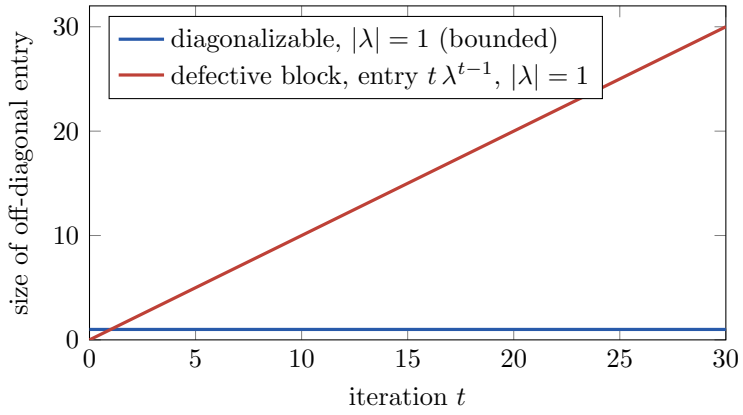


Figure 28.7: Why a defective eigenvalue on the unit circle is unstable. Both operators have spectral radius exactly 1, so eigenvalue intuition predicts "neither grows nor shrinks." But the off-diagonal entry of a 2×2 Jordan block is $t\,\lambda^{t-1}$, which marches upward linearly (red), while the diagonalizable case stays flat (blue). The transient is invisible to the eigenvalues and visible only in the Jordan structure.

28.8 Putting It All Together

Here is the recipe the whole chapter has been building, the one you can actually run on a small matrix.

Formal

Finding the Jordan form of A.

1. **Eigenvalues.** Factor the characteristic polynomial $\det(\mathbf{A} - t\mathbf{I})$. Each distinct root λ with its multiplicity is an eigenvalue with its *algebraic* multiplicity.
2. **Count blocks.** For each λ, compute $\dim \operatorname{null}(\mathbf{A} - \lambda\mathbf{I})$ — the geometric multiplicity. That is the *number of Jordan blocks* for λ.
3. **Size the blocks.** If you need the exact sizes, compute the growing nullities $d_j = \dim \operatorname{null}(\mathbf{A}-\lambda\mathbf{I})^j$ until they stop increasing, and read off block sizes from $2d_j - d_{j-1} - d_{j+1}$. (For small matrices the block sizes are usually forced once you know the algebraic and geometric multiplicities.)
4. **Build chains (if you want P).** For each block, find a top generalized eigenvector $\mathbf{w}$ and apply $(\mathbf{A} - \lambda\mathbf{I})$ repeatedly to generate the chain down to a true eigenvector. Collect all chain vectors, bottom-up within each block, as the columns of $\mathbf{P}$.
5. **Result.** $\mathbf{A} = \mathbf{P}\mathbf{J}\mathbf{P}^{-1}$ with $\mathbf{J}$ the assembled blocks.

Focus

The one sentence to carry out of this chapter: *similar matrices are one operator in different bases, and the Jordan form is the simplest basis you can always reach — a diagonal of eigenvalues plus the minimum amount of shearing the operator genuinely contains.* Diagonalizable is the case where that minimum is zero.

Remark 28.1 (A caution about the real and the numerical world). Two cautions keep us grounded. First, Jordan form lives over $\mathbb{C}$: a real matrix may require complex eigenvalues (for example, a planar rotation has eigenvalues $e^{\pm i\theta}$), so its Jordan form is complex even though the original matrix is real. Second, the Jordan form is famously *discontinuous* and numerically unstable. The matrix $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has a 2×2 block, but $\begin{bmatrix} 1 & 1 \\ 0 & 1+\varepsilon \end{bmatrix}$ for any $\varepsilon \neq 0$ is diagonalizable with two distinct eigenvalues — a tiny perturbation shatters the block. This is why numerical libraries compute the *Schur decomposition* (factoring $\mathbf{A} = \mathbf{QTQ}^*$ using an orthonormal change of basis $\mathbf{Q}$ and an upper-triangular matrix $\mathbf{T}$) or the SVD instead, never the Jordan form. The Jordan form is an essential theoretical tool for analyzing powers and exponentials of matrices, even though floating-point software relies on the Schur decomposition or SVD instead.

History and connection

The quest for canonical forms began in abstract algebra and Galois theory (the study of polynomial equations through the group of symmetries of their roots). In 1870, the French mathematician Camille Jordan published his masterwork, *Traité des substitutions et des équations algébriques*. Jordan was investigating symmetries of polynomial roots, which required classifying linear substitutions (what we now call linear transformations) over finite fields (number systems with finitely many elements where division is still possible) and over the complex numbers. When a matrix has distinct eigenvalues, ordinary diagonalization solves the classification problem completely: two matrices are similar if and only if they share the same spectrum (the complete set of eigenvalues). But for defective matrices, ordinary eigenvectors fail to span space. Jordan set out to find the simplest, most canonical form achievable for *any* linear operator under a change of basis. Working with polynomials and determinants—decades before modern axiomatic vector spaces were formalized—Jordan proved that by augmenting eigenvectors with chains of generalized eigenvectors, any square matrix uncouples into a direct sum of Jordan blocks, cleanly separating the pure stretching of the eigenvalues from the irreducible nilpotent shearing.

A heated 19th-century priority controversy soon followed. Two years earlier, in 1868, the German mathematician Karl Weierstrass had published a classification of pairs of bilinear forms using polynomial invariants he called "elementary divisors" (determinantal divisors of matrix polynomials). Weierstrass and his supporters argued that Jordan had merely translated Weierstrass's theory into the language of substitutions. Jordan countered that his block-matrix form was explicit, constructive, and provided the geometric basis that Weierstrass's invariant polynomials obscured. History ultimately credited both: Weierstrass provided the coordinate-free polynomial invariants, while Jordan gave the unforgettable visual matrix.

A century later, a second controversy erupted at the intersection of pure algebra and computational physics. Pure mathematicians celebrated the Jordan form as the definitive structural classification of linear maps. But in the 1960s, British numerical analyst James H. Wilkinson demonstrated that the Jordan canonical form is fundamentally ill-posed and uncomputable in floating-point arithmetic. The issue is severe discontinuity: consider the matrix $\begin{bmatrix} 0 & 1 \\ \varepsilon & 0 \end{bmatrix}$. When $\varepsilon = 0$, the matrix has eigenvalue 0 with only one eigenvector—a defective 2×2 Jordan block. But if $\varepsilon = 10^{-16}$, the magnitude of a single roundoff error in double-precision arithmetic, the eigenvalues split into $+10^{-8}$ and -10^{-8}, and the Jordan block shatters into a diagonalizable matrix. Because finite precision cannot distinguish whether two floating-point

numbers are truly equal or merely close, Jordan form cannot be determined reliably by an algorithm. In 1909, Issai Schur had already discovered the numerical antidote: the *Schur decomposition* ($\mathbf{A} = \mathbf{QTQ}^*$, where $\mathbf{Q}$ is unitary, preserving lengths without distortion, and $\mathbf{T}$ is upper triangular). Numerical software computes Schur forms, never Jordan forms.
The Jordan form gives a complete classification of complex matrices up to similarity and explains why defective linear systems produce polynomial-exponential terms (polynomial growth factors like t^k that multiply exponentials over time in differential equations). What remains open? The problem of *simultaneous similarity*: finding a canonical form for a pair of non-commuting matrices $(\mathbf{A}, \mathbf{B})$ under a shared basis change $\mathbf{P}^{-1}\mathbf{AP}, \mathbf{P}^{-1}\mathbf{BP}$. In modern representation theory, this is proven to be a "wild" problem—an intractable classification task equivalent to classifying representations of all finite-dimensional algebras. In applied mathematics, finding robust, stable surrogates for defective matrix dynamics (such as the pseudospectrum, which tracks how eigenvalues behave under small perturbations) remains an active and vital frontier.

Looking forward: from similarity to the SVD

Throughout this chapter, similarity forced us to play by a strict rule: we had to use the *same* change of basis $\mathbf{P}$ for both the input space and the output space ($\mathbf{B} = \mathbf{P}^{-1}\mathbf{AP}$). That rigid demand is the single reason why defective matrices cannot be diagonalized, and why Jordan form requires potentially fragile, non-orthogonal bases.

In Chapter 29, we relax that constraint. By allowing *two different* orthonormal bases—one for inputs ($\mathbf{V}$) and one for outputs ($\mathbf{U}$)—the Singular Value Decomposition ($\mathbf{A} = \mathbf{U\Sigma V}^{\mathsf{T}}$) achieves a diagonal representation for *every* matrix, whether square, rectangular, defective, or well-behaved, with complete numerical stability. While similarity is restricted to single-basis changes on square matrices, the SVD provides an orthogonal factorization for any linear map.

28.9 Exercises

Exercise 28.1 (Warmup: invariants)**.** Let $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}$ and $\mathbf{P} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$. Compute $\mathbf{B} = \mathbf{P}^{-1}\mathbf{AP}$. Verify by direct calculation that $\operatorname{tr}\mathbf{B} = \operatorname{tr}\mathbf{A}$, $\det\mathbf{B} = \det\mathbf{A}$, and that $\mathbf{A}$ and $\mathbf{B}$ have the same characteristic polynomial.

Exercise 28.2 (Geometric vs. algebraic multiplicity)**.** For each matrix, find the eigenvalues and, for each eigenvalue, its algebraic and geometric multiplicity. Which are diagonalizable?

$$\text{(a)} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \quad \text{(b)} \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix} \quad \text{(c)} \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{bmatrix}.$$

Exercise 28.3 (A chain by hand)**.** Let $\mathbf{A} = \begin{bmatrix} 6 & 1 \\ -1 & 4 \end{bmatrix}$. Show its only eigenvalue is 5 with geometric multiplicity 1. Find the eigenvector $\mathbf{v}$, then a generalized eigenvector $\mathbf{w}$ solving $(\mathbf{A} - 5\mathbf{I})\mathbf{w} = \mathbf{v}$. Form $\mathbf{P} = [\mathbf{v}\ \mathbf{w}]$ and verify $\mathbf{P}^{-1}\mathbf{AP} = \begin{bmatrix} 5 & 1 \\ 0 & 5 \end{bmatrix}$.

Exercise 28.4 (Powers of a block)**.** Using $\mathbf{J}_2(\lambda) = \lambda\mathbf{I} + \mathbf{N}$ and $\mathbf{N}^2 = \mathbf{0}$, prove by the binomial theorem that $\mathbf{J}_2(\lambda)^t = \begin{bmatrix} \lambda^t & t\lambda^{t-1} \\ 0 & \lambda^t \end{bmatrix}$. Then evaluate $\mathbf{J}_2(1)^{10}$ and $\mathbf{J}_2(\tfrac{1}{2})^{10}$ and comment on which iteration is stable.

Exercise 28.5 (Same eigenvalues, not similar)**.** Explain why $\begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$, $\begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$, and $\begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$ all share the characteristic polynomial $(t-2)^3$ yet no two of them are similar. Identify the geometric multiplicity of 2 in each, and state each matrix's Jordan form.

Exercise 28.6 (Reading blocks from nullities)**.** A 5×5 matrix has a single eigenvalue λ with $d_1 = \dim\operatorname{null}(\mathbf{A} - \lambda\mathbf{I}) = 3$, $d_2 = 4$, $d_3 = 5$, and $d_4 = 5$. How many Jordan blocks are there? What is the size of the largest block? Determine the complete list of block sizes and write the Jordan form. (Hint: use $2d_j - d_{j-1} - d_{j+1}$ with $d_0 = 0$.)

Exercise 28.7 (Nilpotent practice)**.** Let $\mathbf{N} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 0 & 3 \\ 0 & 0 & 0 \end{bmatrix}$. Confirm $\mathbf{N}^3 = \mathbf{0}$ but $\mathbf{N}^2 \neq \mathbf{0}$. What is the only eigenvalue? Using the height of the chain, argue that the Jordan form is a single block $\mathbf{J}_3(0)$.

Exercise 28.8 (Differential equations connection)**.** For $\mathbf{A} = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}$, use $e^{t\mathbf{A}} = e^{\lambda t}(\mathbf{I} + t\mathbf{N})$ to write the general solution of $\mathbf{x}' = \mathbf{A}\mathbf{x}$. Explain why a defective eigenvalue produces a $t\,e^{\lambda t}$ term, and connect this to resonance in a forced oscillator.

Exercise 28.9 (Proof: trace and similarity)**.** Prove that $\operatorname{tr}(\mathbf{XY}) = \operatorname{tr}(\mathbf{YX})$ for any conformable matrices, and use it to give a one-line proof that similar matrices have equal trace. Then explain why the converse fails: give two matrices with equal trace that are not similar.

Exercise 28.10 (Challenge: the minimal polynomial)**.** The *minimal polynomial* of $\mathbf{A}$ is the smallest-degree monic polynomial $m(t)$ with $m(\mathbf{A}) = \mathbf{0}$. Argue that for each eigenvalue λ, the exponent of $(t-\lambda)$ in $m(t)$ equals the size of the *largest* Jordan block for λ. Use this to show $\mathbf{A}$ is diagonalizable if and only if its minimal polynomial has no repeated roots. (Hint: think about what power of $\mathbf{N} = \mathbf{J}_k(\lambda) - \lambda\mathbf{I}$ first becomes zero.)

Exercise 28.11 (Challenge: classify 3×3 up to similarity)**.** Suppose a 3×3 complex matrix has characteristic polynomial $(t-1)^2(t-2)$. List all possible Jordan forms (there are exactly two), describe how to tell them apart using the geometric multiplicity of the eigenvalue 1, and state for each whether the matrix is diagonalizable.

Part VII

The SVD and Beyond (After Strang)

Chapter 29

The Singular Value Decomposition

Here is the geometric fact at the center of this chapter:

> *Every matrix – any matrix at all, square or rectangular, full rank or rank one, invertible or hopelessly singular – can be written as a rotation, then a stretch along perpendicular axes, then another rotation.*

That is the singular value decomposition, the SVD, and it is the most widely applied factorization in numerical linear algebra.

In the previous chapter, we examined similarity and the Jordan canonical form. There, we diagonalized a matrix using the *same* basis for both the input and output spaces: $\mathbf{P}^{-1}\mathbf{AP}$. That demand was restrictive. When a matrix sheared space, independent eigenvectors were missing, forcing us to settle for Jordan blocks with off-diagonal ones. Even when eigenvectors existed, they were often non-orthogonal. Eigenvalues are powerful, but they have inherent limitations: they apply only to square matrices, they can become complex even for real matrices, and defective matrices lack a full basis of eigenvectors.

The SVD avoids all of these restrictions. By choosing *two different* orthonormal bases – one for the input space and one for the output space – every matrix admits a clean diagonal representation. It applies to any $m \times n$ matrix: a 3×7 matrix of pixel intensities, a 10000×50 matrix of customer ratings, or a 2×2 shear that has only one eigenvector. The decomposition always exists, its singular values are always non-negative real numbers, and its geometric action is always the same: **rotate, stretch, rotate**.

The SVD brings together the four fundamental subspaces, orthonormal bases, projections, and the spectral theorem for symmetric matrices. When you finish this chapter you will understand principal component analysis, low-rank compression, the pseudoinverse, and a surprising amount of the linear algebra that runs underneath modern machine learning – all from one picture.

29.1 The Picture First: What a Matrix Does to a Circle

Forget formulas for a moment and let us just *watch* a matrix act. Take

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}.$$

Feed it every unit vector in the plane – the whole unit circle – and look at where the outputs land. A matrix sends straight lines to straight lines and the unit circle (which is *not* straight) to an ellipse. Always an ellipse. Figure 29.1 shows it.

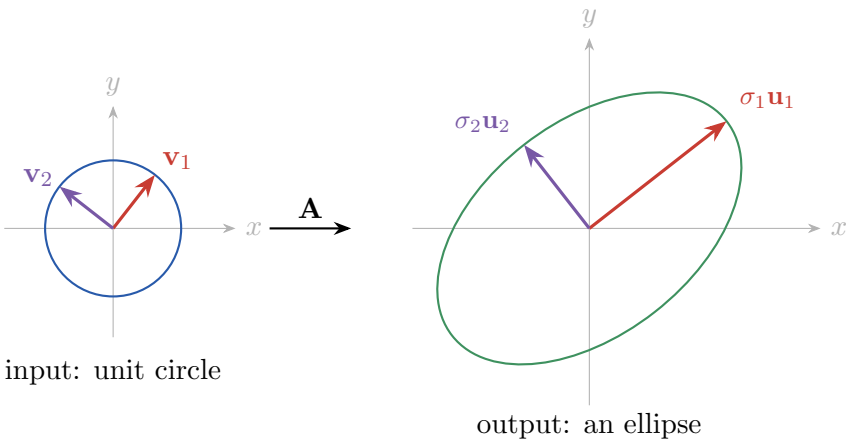


Figure 29.1: Any 2×2 matrix sends the unit circle to an ellipse. The two perpendicular input directions (red and purple, the *right singular vectors*) are the only ones that come out still perpendicular, landing on the ellipse's two axes. Their output lengths $\sigma_1 \geq \sigma_2$ are the *singular values*.

Here is the observation everything hinges on. As you sweep an input unit vector around the circle, its image sweeps around the ellipse. Most input directions get knocked off square – two perpendicular inputs usually come out *not* perpendicular, because the matrix shears as well as stretches. But there is always (at least) one special pair of perpendicular input directions that stays perpendicular on the way out. Those special inputs land exactly on the long axis and short axis of the ellipse.

Intuition

The whole SVD lives in this one picture. A matrix turns the unit circle into an ellipse. The **right singular vectors** $\mathbf{v}_1, \mathbf{v}_2$ are the perpendicular *input* directions that survive as perpendicular. The **left singular vectors** $\mathbf{u}_1, \mathbf{u}_2$ are the perpendicular *output* directions they land on – the ellipse's axes. The **singular values** $\sigma_1 \geq \sigma_2 \geq 0$ are how long those output axes are. The matrix does nothing more exotic than: line up with the $\mathbf{v}$'s, stretch by the σ's, point along the $\mathbf{u}$'s.

Why must the image always be an ellipse, and why must such a perpendicular pair always exist? That is not obvious, and we will prove it. But the geometry should already feel inevitable: an ellipse is just a stretched, rotated circle, and a matrix is just the machine that stretches and rotates. Let us turn the picture into algebra.

Pause

Pause and think: What does the output ellipse look like if $\mathbf{A}$ is a 2×2 projection matrix onto the diagonal line $y = x$?
A projection squashes everything perpendicular to the line down to zero, while vectors along the line are untouched. The unit circle collapses into a flat line segment running from $-\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$ to $+\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$. Its semi-major axis length is $\sigma_1 = 1$, and its semi-minor axis length is $\sigma_2 = 0$. A "flat ellipse" is still an ellipse.

29.2 From the Picture to the Factorization

Suppose we have found these aligned orthogonal directions. In the plane: an orthonormal pair $\mathbf{v}_1, \mathbf{v}_2$ in the input space, an orthonormal pair $\mathbf{u}_1, \mathbf{u}_2$ in the output space, and nonnegative numbers σ_1, σ_2, all linked by

$$\mathbf{A}\mathbf{v}_1 = \sigma_1 \mathbf{u}_1, \qquad \mathbf{A}\mathbf{v}_2 = \sigma_2 \mathbf{u}_2.$$

Read that as: "the input axis $\mathbf{v}_i$ gets sent to the output axis $\mathbf{u}_i$, stretched by σ_i." Now stack these two equations as columns of a single matrix equation. Put the $\mathbf{v}$'s as columns of a matrix $\mathbf{V}$, the $\mathbf{u}$'s as columns of $\mathbf{U}$:

$$\mathbf{A}\underbrace{\begin{bmatrix}\mathbf{v}_1 & \mathbf{v}_2\end{bmatrix}}_{\mathbf{V}} = \underbrace{\begin{bmatrix}\mathbf{u}_1 & \mathbf{u}_2\end{bmatrix}}_{\mathbf{U}}\underbrace{\begin{bmatrix}\sigma_1 & 0\\ 0 & \sigma_2\end{bmatrix}}_{\mathbf{\Sigma}}.$$

The right side is correct because $\mathbf{U\Sigma}$ has columns $\sigma_1\mathbf{u}_1$ and $\sigma_2\mathbf{u}_2$ – exactly what $\mathbf{A}\mathbf{v}_1$ and $\mathbf{A}\mathbf{v}_2$ are supposed to equal. So $\mathbf{AV} = \mathbf{U\Sigma}$. Because the columns of $\mathbf{V}$ are mutually perpendicular unit vectors (orthonormal), $\mathbf{V}$ is an *orthogonal matrix* – a square matrix representing a rigid rotation or reflection that preserves all lengths and angles, satisfying $\mathbf{V}^{-1} = \mathbf{V}^{\mathsf{T}}$. Multiply on the right by $\mathbf{V}^{\mathsf{T}}$ and you have it:

$$\boxed{\mathbf{A} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^{\mathsf{T}}.}$$

Focus

The SVD is just the equations $\mathbf{A}\mathbf{v}_i = \sigma_i\mathbf{u}_i$ collected into matrices. Read $\mathbf{A} = \mathbf{U\Sigma V}^{\mathsf{T}}$ **right to left** as three motions applied to an input vector $\mathbf{x}$:

1. $\mathbf{V}^{\mathsf{T}}$ **rotates** the input space so the right singular vectors $\mathbf{v}_i$ line up with the coordinate axes;
2. $\mathbf{\Sigma}$ **stretches** each coordinate axis by its singular value (and, if rectangular, changes the dimension);
3. $\mathbf{U}$ **rotates** the stretched result into the output space.

Rotate, stretch, rotate. Every matrix, no exceptions.

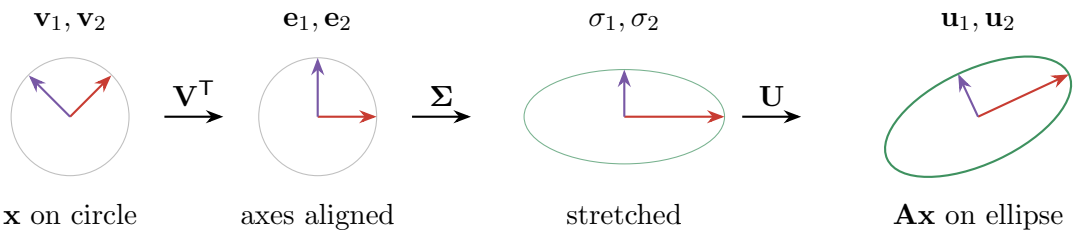


Figure 29.2: The SVD as rotate–stretch–rotate. Reading $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ from right to left: $\mathbf{V}^\mathsf{T}$ rotates the right singular vectors $\mathbf{v}_i$ onto the coordinate axes, $\mathbf{\Sigma}$ stretches along those axes by σ_1, σ_2, and $\mathbf{U}$ rotates the result into final position.

Compare this to diagonalization, $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$. There the same basis $\mathbf{X}$ appears on both sides, but it is generally *not* orthogonal, so $\mathbf{X}^{-1} \neq \mathbf{X}^\mathsf{T}$, and the picture is a skew stretch along oblique eigen-axes. The SVD pays a small price – it allows *two* different orthonormal bases, an input one $\mathbf{V}$ and an output one $\mathbf{U}$ – and in exchange it buys enormous freedom: the bases are always orthonormal, the stretches σ_i are always real and nonnegative, and the matrix need not even be square.

Before/after Strang

Strang likes to write the SVD as $\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$, calling $\mathbf{V}$ and $\mathbf{U}$ the right and left singular vector bases. If you have already taken his course, you have seen him insist that the SVD completes the four-subspaces picture. Specifically, $\mathbf{V}$ gives orthonormal bases for the row space and null space, $\mathbf{U}$ for the column space and left null space, and $\mathbf{\Sigma}$ records exactly how the row space stretches onto the column space. If you are reading this *before* Strang, hold onto the circle-to-ellipse picture; it is the one mental image that makes his blackboard derivation feel obvious.

29.3 The Formal Statement

Let us write down precisely what we are claiming, for a general $m \times n$ matrix.

Formal

Theorem 29.1 (Singular Value Decomposition)**.** *Let $\mathbf{A}$ be any real $m \times n$ matrix. Then there exist*

- *an orthogonal $m \times m$ matrix $\mathbf{U}$ (columns $\mathbf{u}_1, \ldots, \mathbf{u}_m$),*
- *an orthogonal $n \times n$ matrix $\mathbf{V}$ (columns $\mathbf{v}_1, \ldots, \mathbf{v}_n$),*
- *an $m \times n$ "diagonal" matrix $\mathbf{\Sigma}$ whose only nonzero entries are $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ in the top-left, where $r = \operatorname{rank}(\mathbf{A})$,*

such that

$$\mathbf{A} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^\mathsf{T}.$$

The numbers σ_i are the singular values*; the columns of $\mathbf{V}$ are the* right singular vectors*; the*

columns of $\mathbf{U}$ *are the* left singular vectors. *The singular values are uniquely determined by* $\mathbf{A}$.

A word on shapes, since rectangular matrices trip everyone up at first. If $\mathbf{A}$ is $m \times n$, then $\mathbf{\Sigma}$ is also $m \times n$ – the same shape as $\mathbf{A}$ – but it is as diagonal as a rectangle can be. For a 3×2 matrix,

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{bmatrix},$$

and for a 2×3 matrix,

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{bmatrix}.$$

The singular values march down the main diagonal; everything off it is zero.

Common trap

Singular values are not eigenvalues. They are nonnegative by definition, even when the matrix has negative or complex eigenvalues, even when the matrix is not square (and so has no eigenvalues at all). For the rotation by 90°, $\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$, the eigenvalues are $\pm i$ – but both singular values are 1, because a rotation stretches nothing: it sends the unit circle to itself. Eigenvalues measure "invariant directions and their scaling"; singular values measure "how much the matrix stretches space." Do not confuse them.

Pause

Pause and think: If a 2×2 diagonal matrix is $\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & -5 \end{bmatrix}$, what are its singular values? Can a singular value be negative? No: singular values measure physical stretch lengths, so by definition $\sigma_i \geq 0$, sorted as $\sigma_1 \geq \sigma_2 \geq 0$. Here $\sigma_1 = 5$ and $\sigma_2 = 3$. The minus sign is absorbed into the rotation/reflection matrices: $\mathbf{A} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = -5 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 5 \begin{bmatrix} 0 \\ -1 \end{bmatrix}$, so the input axis $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ is sent to the output axis $\mathbf{u}_2 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$.

29.3.1 The economy (thin) SVD

When m and n are very different – say a 10000×50 data matrix – the full $\mathbf{U}$ would be a gigantic 10000×10000 matrix, almost all of whose columns multiply zeros in $\mathbf{\Sigma}$. Wasteful. The **economy SVD** keeps only the columns that matter. If $r = \operatorname{rank}(\mathbf{A})$, write

$$\mathbf{A} = \mathbf{U}_r \, \mathbf{\Sigma}_r \, \mathbf{V}_r^{\mathsf{T}},$$

where $\mathbf{U}_r$ is $m \times r$ (its columns the first r left singular vectors), $\mathbf{\Sigma}_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r)$ is $r \times r$ and

genuinely square, and $\mathbf{V}_r$ is $n \times r$. Even leaner is the **outer-product form**, which we will use constantly:

$$\boxed{\mathbf{A} = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}.}$$

Each term $\sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ is a rank-one matrix – an output direction $\mathbf{u}_i$ ($m \times 1$) times an input direction $\mathbf{v}_i^{\mathsf{T}}$ ($1 \times n$), scaled by σ_i. The matrix is literally a *sum of rank-one pieces*, ordered from strongest to weakest (Figure 29.3). This representation forms the foundation of low-rank matrix approximation and data compression.

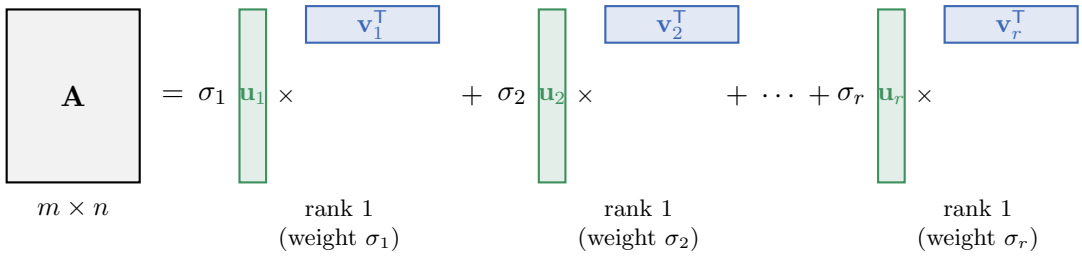


Figure 29.3: The outer-product expansion decomposes $\mathbf{A}$ into a sum of r rank-one matrices. Each slice is an outer product $\mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ (an $m \times 1$ output column times a $1 \times n$ input row) scaled by the singular value σ_i. Truncating after k slices discards only the weakest remaining patterns.

29.4 Where the Pieces Come From: $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathsf{T}}$

So far we have assumed these aligned orthonormal directions exist. Now we construct them explicitly and prove the existence of the SVD. The key step is to form symmetric matrices by multiplying $\mathbf{A}$ with its transpose: both $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathsf{T}}$ are symmetric and positive semidefinite, so the spectral theorem applies directly to them.

Suppose for the moment that $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}}$ really holds. Then watch what $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ becomes, using $\mathbf{U}^{\mathsf{T}}\mathbf{U} = \mathbf{I}$:

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}})^{\mathsf{T}}(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}}) = \mathbf{V}\mathbf{\Sigma}^{\mathsf{T}}\mathbf{U}^{\mathsf{T}}\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}} = \mathbf{V}\,(\mathbf{\Sigma}^{\mathsf{T}}\mathbf{\Sigma})\,\mathbf{V}^{\mathsf{T}}.$$

But $\mathbf{\Sigma}^{\mathsf{T}}\mathbf{\Sigma}$ is the diagonal matrix $\operatorname{diag}(\sigma_1^2, \ldots, \sigma_n^2)$. So this last line *is* an orthogonal diagonalization of $\mathbf{A}^{\mathsf{T}}\mathbf{A}$:

Focus

The right singular vectors $\mathbf{v}_i$ are the **eigenvectors of $\mathbf{A}^{\mathsf{T}}\mathbf{A}$**, and the squared singular values σ_i^2 are its **eigenvalues**:

$$\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\mathbf{v}_i = \sigma_i^2\,\mathbf{v}_i.$$

Symmetrically, the left singular vectors $\mathbf{u}_i$ are the eigenvectors of $\mathbf{A}\mathbf{A}^{\mathsf{T}}$ with the *same* eigen-

values σ_i^2:

$$\mathbf{A}\mathbf{A}^\mathsf{T}\,\mathbf{u}_i = \sigma_i^2\,\mathbf{u}_i.$$

This structural relationship provides both a constructive proof and a practical recipe. Let us see why it must work, then turn it into a procedure.

29.4.1 Why the recipe is forced (a real proof)

We build the SVD from scratch, with no prior assumption that it exists. Start with $\mathbf{A}^\mathsf{T}\mathbf{A}$. It is $n \times n$, symmetric, and *positive semidefinite*: for any $\mathbf{x}$,

$$\mathbf{x}^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{x} = (\mathbf{A}\mathbf{x})^\mathsf{T}(\mathbf{A}\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 \geq 0.$$

By the spectral theorem (Theorem 26.3 in Chapter 26), $\mathbf{A}^\mathsf{T}\mathbf{A}$ has an orthonormal basis of eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$ with real eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n$, and because it is positive semidefinite, every $\lambda_i \geq 0$. Define the singular values as the square roots,

$$\sigma_i := \sqrt{\lambda_i} \geq 0,$$

and say there are r positive ones: $\sigma_1 \geq \cdots \geq \sigma_r > 0$ and $\sigma_{r+1} = \cdots = 0$.

Now *define* the left singular vectors, for $i = 1, \ldots, r$, by normalizing the images $\mathbf{A}\mathbf{v}_i$:

$$\mathbf{u}_i := \frac{1}{\sigma_i}\,\mathbf{A}\mathbf{v}_i.$$

Two things to check: that these $\mathbf{u}_i$ are orthonormal, and that the leftover directions ($i > r$) really do contribute nothing.

Orthonormality. Compute the dot product of two of them:

$$\mathbf{u}_i^\mathsf{T}\mathbf{u}_j = \frac{1}{\sigma_i\sigma_j}(\mathbf{A}\mathbf{v}_i)^\mathsf{T}(\mathbf{A}\mathbf{v}_j) = \frac{1}{\sigma_i\sigma_j}\,\mathbf{v}_i^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{v}_j = \frac{1}{\sigma_i\sigma_j}\,\mathbf{v}_i^\mathsf{T}(\sigma_j^2\,\mathbf{v}_j) = \frac{\sigma_j}{\sigma_i}\,\mathbf{v}_i^\mathsf{T}\mathbf{v}_j.$$

Since the $\mathbf{v}$'s are orthonormal, $\mathbf{v}_i^\mathsf{T}\mathbf{v}_j$ is 1 when $i = j$ and 0 otherwise. So $\mathbf{u}_i^\mathsf{T}\mathbf{u}_j = 1$ when $i = j$ (the σ's cancel) and 0 otherwise. The $\mathbf{u}_i$ are orthonormal.

The leftovers vanish. For $i > r$ we have $\sigma_i = 0$, meaning $\lambda_i = 0$, so

$$\|\mathbf{A}\mathbf{v}_i\|^2 = \mathbf{v}_i^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{v}_i = \lambda_i\|\mathbf{v}_i\|^2 = 0,$$

hence $\mathbf{A}\mathbf{v}_i = \mathbf{0}$. These directions are crushed to nothing; they are the null space of $\mathbf{A}$.

Finally, extend $\mathbf{u}_1, \ldots, \mathbf{u}_r$ to a full orthonormal basis $\mathbf{u}_1, \ldots, \mathbf{u}_m$ of $\mathbb{R}^m$ (always possible – just complete the basis). By construction $\mathbf{A}\mathbf{v}_i = \sigma_i\mathbf{u}_i$ holds for *every* i (for $i \leq r$ by definition of $\mathbf{u}_i$; for $i > r$ both sides are zero). Collect into matrices: $\mathbf{A}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$, so $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$. □

Why this matters

Look at what just happened. We never assumed $\mathbf{A}$ was square, never assumed it was invertible, never touched a complex number. Because $\mathbf{A}^\mathsf{T}\mathbf{A}$ is always symmetric and positive semidefinite, applying the spectral theorem to $\mathbf{A}^\mathsf{T}\mathbf{A}$ yields an orthogonal decomposition for *any* matrix $\mathbf{A}$.

Common trap

Use $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ to get the $\mathbf{v}$'s, then get the $\mathbf{u}$'s *from* $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$ – do *not* find the $\mathbf{u}$'s independently as eigenvectors of $\mathbf{A}\mathbf{A}^{\mathsf{T}}$ and pair them up arbitrarily. Eigenvectors are only defined up to sign (and up to rotation within a repeated eigenspace). If you compute $\mathbf{U}$ and $\mathbf{V}$ separately, you can easily get signs that disagree, so that $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}}$ comes out as $-\mathbf{A}$ in some columns. The link $\mathbf{A}\mathbf{v}_i = \sigma_i\mathbf{u}_i$ fixes the signs consistently.

29.5 A Fully Worked 2×2 Example

Let us compute a 2×2 SVD from start to finish. Take

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}.$$

Step 1: Form $\mathbf{A}^{\mathsf{T}}\mathbf{A}$.

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix}\begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}.$$

Step 2: Eigenvalues of $\mathbf{A}^{\mathsf{T}}\mathbf{A}$. For the symmetric matrix $\begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$, the eigenvalues of $\begin{bmatrix} a & b \\ b & a \end{bmatrix}$ are $a+b$ and $a-b$ (the vectors $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ are always eigenvectors of such a matrix). So

$$\lambda_1 = 25 + 20 = 45, \qquad \lambda_2 = 25 - 20 = 5.$$

The singular values are the square roots:

$$\sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6.708, \qquad \sigma_2 = \sqrt{5} \approx 2.236.$$

Step 3: Right singular vectors (the v's). The eigenvectors are $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$; normalize them:

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Step 4: Left singular vectors (the u's) via $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$. First

$$\mathbf{A}\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix} 3 \\ 9 \end{bmatrix}, \quad \Rightarrow \quad \mathbf{u}_1 = \frac{1}{\sigma_1}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix} 3 \\ 9 \end{bmatrix} = \frac{1}{3\sqrt{5}}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix} 3 \\ 9 \end{bmatrix} = \frac{1}{\sqrt{10}}\begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Check its length: $\frac{1}{\sqrt{10}}\sqrt{1+9} = 1$. Good. Then

$$\mathbf{A}\mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}\begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix} 3 \\ -1 \end{bmatrix}, \quad \Rightarrow \quad \mathbf{u}_2 = \frac{1}{\sqrt{5}}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix} 3 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{10}}\begin{bmatrix} 3 \\ -1 \end{bmatrix}.$$

Notice $\mathbf{u}_1^{\mathsf{T}}\mathbf{u}_2 = \frac{1}{10}(1\cdot 3 + 3\cdot(-1)) = 0$: the output axes really are perpendicular, exactly as the picture promised.

Step 5: Assemble.

$$\mathbf{A} = \underbrace{\frac{1}{\sqrt{10}}\begin{bmatrix}1 & 3\\ 3 & -1\end{bmatrix}}_{\mathbf{U}}\underbrace{\begin{bmatrix}3\sqrt{5} & 0\\ 0 & \sqrt{5}\end{bmatrix}}_{\mathbf{\Sigma}}\underbrace{\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}}_{\mathbf{V}^{\mathsf{T}}}.$$

Step 6: Sanity check by multiplying $\mathbf{U\Sigma}$ first:

$$\mathbf{U\Sigma} = \frac{1}{\sqrt{10}}\begin{bmatrix}1 & 3\\ 3 & -1\end{bmatrix}\begin{bmatrix}3\sqrt{5} & 0\\ 0 & \sqrt{5}\end{bmatrix} = \frac{1}{\sqrt{10}}\begin{bmatrix}3\sqrt{5} & 3\sqrt{5}\\ 9\sqrt{5} & -\sqrt{5}\end{bmatrix}.$$

Then times $\mathbf{V}^{\mathsf{T}}$, using $\frac{\sqrt{5}}{\sqrt{10}}\cdot\frac{1}{\sqrt{2}} = \frac{\sqrt{5}}{\sqrt{20}} = \frac{\sqrt{5}}{2\sqrt{5}} = \frac{1}{2}$:

$$\mathbf{U\Sigma V}^{\mathsf{T}} = \frac{1}{2}\begin{bmatrix}3 & 3\\ 9 & -1\end{bmatrix}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}6 & 0\\ 8 & 10\end{bmatrix} = \begin{bmatrix}3 & 0\\ 4 & 5\end{bmatrix} = \mathbf{A}.\checkmark$$

It closes perfectly. Notice also that $\sigma_1\sigma_2 = 3\sqrt{5}\cdot\sqrt{5} = 15 = \det\mathbf{A}$ (since $\det\mathbf{A} = 3\cdot 5 - 0\cdot 4 = 15$). That is no accident – see the next box.

Compute it yourself

Compute the SVD of $\mathbf{A} = \begin{bmatrix}4 & 0\\ 3 & -5\end{bmatrix}$ by hand, following the six steps above. You should find $\mathbf{A}^{\mathsf{T}}\mathbf{A} = \begin{bmatrix}25 & -15\\ -15 & 25\end{bmatrix}$, eigenvalues 40 and 10, and singular values $\sigma_1 = \sqrt{40} = 2\sqrt{10}$, $\sigma_2 = \sqrt{10}$. Find the $\mathbf{v}$'s, push them through $\mathbf{A}$, and check that the two $\mathbf{u}$'s come out orthogonal. Then verify $\sigma_1\sigma_2 = |\det\mathbf{A}| = 20$.

Connection to other math

Singular values, determinant, and volume. For a square $\mathbf{A}$,

$$|\det\mathbf{A}| = \sigma_1\sigma_2\cdots\sigma_n.$$

Here is the slick reason. Determinants multiply, and orthogonal matrices have determinant ± 1 (they preserve volume), so $|\det\mathbf{A}| = |\det\mathbf{U}|\,|\det\mathbf{\Sigma}|\,|\det\mathbf{V}^{\mathsf{T}}| = 1\cdot(\sigma_1\cdots\sigma_n)\cdot 1$. Geometrically: the unit cube has volume 1; rotating ($\mathbf{U}$, $\mathbf{V}^{\mathsf{T}}$) does not change volume; only the stretch $\mathbf{\Sigma}$ does, multiplying volume by $\sigma_1\cdots\sigma_n$. The absolute value of the determinant is exactly the volume of the output box – and the SVD shows you which directions did the stretching.

29.6 The SVD and the Four Fundamental Subspaces

Recall the four fundamental subspaces of an $m\times n$ matrix $\mathbf{A}$ of rank r:

- the **column space** $\mathrm{col}(\mathbf{A})\subseteq\mathbb{R}^m$, dimension r;
- the **row space** $\mathrm{row}(\mathbf{A}) = \mathrm{col}(\mathbf{A}^{\mathsf{T}})\subseteq\mathbb{R}^n$, dimension r;
- the **null space** $\mathrm{null}(\mathbf{A})\subseteq\mathbb{R}^n$, dimension $n-r$;

- the **left null space** $\text{null}(\mathbf{A}^\mathsf{T}) \subseteq \mathbb{R}^m$, dimension $m - r$.

The SVD hands you a clean orthonormal basis for all four at once. Split the singular vectors at the rank r:

Focus

- $\mathbf{v}_1, \ldots, \mathbf{v}_r$ (positive singular values) are an orthonormal basis for the **row space**.
- $\mathbf{v}_{r+1}, \ldots, \mathbf{v}_n$ (zero singular values) are an orthonormal basis for the **null space**.
- $\mathbf{u}_1, \ldots, \mathbf{u}_r$ are an orthonormal basis for the **column space**.
- $\mathbf{u}_{r+1}, \ldots, \mathbf{u}_m$ are an orthonormal basis for the **left null space**.

And $\mathbf{A}$ maps each $\mathbf{v}_i$ ($i \leq r$) onto $\sigma_i \mathbf{u}_i$: the row space gets rotated and stretched cleanly onto the column space.

Why? We already saw that $\mathbf{v}_i$ with $\sigma_i = 0$ satisfy $\mathbf{A}\mathbf{v}_i = \mathbf{0}$, so they live in the null space; there are $n - r$ of them, which is exactly the null space's dimension, so they are a basis for it. The remaining $\mathbf{v}_1, \ldots, \mathbf{v}_r$ are orthogonal to the null space, and the orthogonal complement of the null space is the row space (fundamental theorem of linear algebra) – so they are an orthonormal basis for the row space. For the $\mathbf{u}$'s: each $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$ ($i \leq r$) is a column of $\mathbf{A}$ times a vector, hence in $\text{col}(\mathbf{A})$; there are r of them and they are orthonormal, so they are a basis for the r-dimensional column space. The leftover $\mathbf{u}_{r+1}, \ldots, \mathbf{u}_m$ are orthogonal to the column space, i.e. in the left null space.

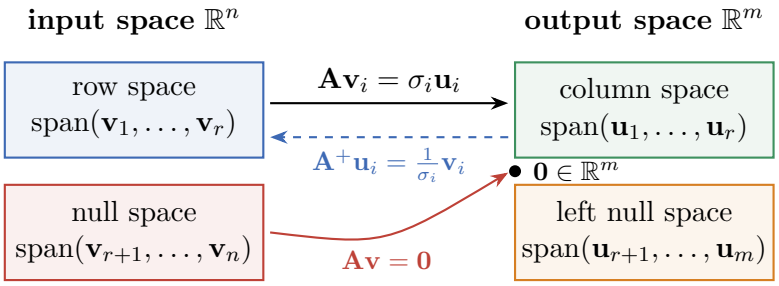


Figure 29.4: The SVD completes the four-subspaces picture. The right singular vectors split the input space $\mathbb{R}^n$ into row space (top, stretched onto the column space by σ_i) and null space (bottom, crushed to $\mathbf{0} \in \mathbb{R}^m$). The left singular vectors split the output space $\mathbb{R}^m$ into column space (the reachable image) and left null space (perpendicular to the image, never reached). The pseudoinverse $\mathbf{A}^+$ runs the top stretch in reverse, dividing each output axis by σ_i.

Pause

Pause and think: In Figure 29.4, what happens to the null space and left null space if $\mathbf{A}$ is an invertible $n \times n$ matrix?

If $\mathbf{A}$ is invertible, its rank is $r = n = m$. The null space and left null space have dimension $n - r = 0$: they both shrink to just the origin $\{\mathbf{0}\}$. The row space fills all of $\mathbb{R}^n$, the column space fills all of $\mathbb{R}^m$, and every singular value is strictly positive ($\sigma_n > 0$). The transformation is an invertible, one-to-one stretch (up to rotation) along all n axes.

Before/after Strang

This is the figure Strang calls "the big picture of linear algebra," now drawn with *orthonormal* axes. In his earlier chapters the four subspaces are there, but the bases are whatever elimination happened to give you – not orthogonal, not unit length. The SVD is the moment those four rooms get perpendicular coordinate axes and the action of $\mathbf{A}$ becomes a simple diagonal stretch from one to the other. Many readers say the four subspaces finally "clicked" only when they saw the SVD. If that is you, you are in good company.

29.7 Best Low-Rank Approximation: Eckart–Young

Now the payoff that makes the SVD indispensable in the data age. Recall the outer-product form,

$$\mathbf{A} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^{\mathsf{T}} + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^{\mathsf{T}} + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^{\mathsf{T}}.$$

The terms are sorted: $\sigma_1 \geq \sigma_2 \geq \cdots$, so the *first* term carries the most "energy," the second the next most, and so on. A natural idea: keep only the top k terms and throw away the rest. Define

$$\mathbf{A}_k := \sum_{i=1}^{k} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}.$$

This $\mathbf{A}_k$ has rank k (a sum of k rank-one pieces). It is an approximation to $\mathbf{A}$ built from the k strongest directions.

To say that $\mathbf{A}_k$ is the "best" approximation, we need a way to measure the error matrix $\mathbf{A} - \mathbf{B}$. Two standard matrix norms answer this:

- The **spectral norm** (or operator 2-norm) $\|\mathbf{M}\|_2 := \max_{\|\mathbf{x}\|=1} \|\mathbf{M}\mathbf{x}\|$ measures the *worst-case stretch* – the largest factor by which $\mathbf{M}$ can lengthen any unit input vector.
- The **Frobenius norm** $\|\mathbf{M}\|_F := \sqrt{\sum_{i,j} M_{ij}^2}$ measures the *total mass* – the square root of the sum of squared entries, treating the entire matrix like a single long Euclidean vector in $\mathbb{R}^{mn}$.

The theorem of Eckart and Young says: *this is the best rank-k approximation there is*, under both measures simultaneously. Nothing of rank k comes closer.

Formal

Theorem 29.2 (Eckart–Young, 1936)**.** *Let* $\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ *and let* $\mathbf{A}_k$ *be its truncation to the top* k *terms. Then for* every *matrix* $\mathbf{B}$ *of rank at most* k*,*

$$\|\mathbf{A} - \mathbf{A}_k\| \leq \|\mathbf{A} - \mathbf{B}\|,$$

in both the spectral norm and the Frobenius norm. The error left behind is exactly

$$\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}, \qquad \|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}.$$

This error formula explains why the SVD is so widely used in data analysis and scientific computing. The error of the best rank-k approximation is governed by the *discarded singular values*.

If $\sigma_{k+1}, \sigma_{k+2}, \ldots$ are tiny, then $\mathbf{A}_k$ is nearly equal to $\mathbf{A}$ even though it is enormously simpler. A 1000×1000 matrix whose singular values plummet after, say, σ_{20} can be captured almost perfectly by 20 rank-one pieces – that is $20 \cdot (1000 + 1000)$ numbers instead of 1,000,000. Compression, denoising, and dimensionality reduction are all this one move. Figure 29.5 shows the typical shape that makes it work: a few dominant singular values and a long, negligible tail.

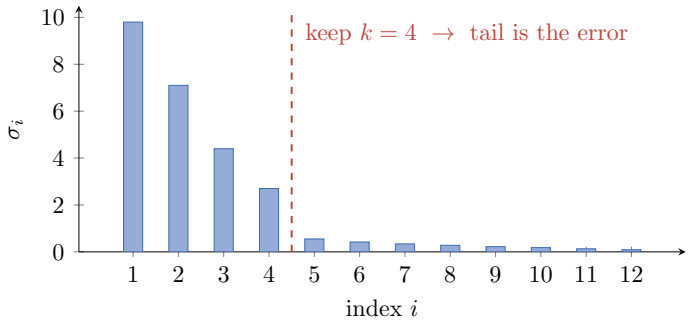


Figure 29.5: A typical singular-value spectrum: a few large values, then a long low tail. Truncating at $k = 4$ keeps the four big rank-one pieces; by Eckart–Young the leftover error is governed entirely by the discarded tail $\sigma_5, \sigma_6, \ldots$, which here is tiny. This sharp drop-off is why real data matrices compress so well.

Intuition

Think of each rank-one term $\sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$ as an elementary pattern weighted by σ_i. The SVD writes $\mathbf{A}$ as a sum of mutually orthogonal patterns ordered from largest weight to smallest, and truncating at k keeps the k strongest patterns while discarding the smallest ones. Because the directions are orthogonal, dropping the small ones costs you *only* their own energy – nothing leaks. That orthogonality is why truncating at rank k is provably optimal, minimizing total reconstruction error.

Pause

Pause and think: Suppose an $m \times n$ matrix $\mathbf{A}$ already has rank 4. What is the error of its rank-4 SVD truncation $\mathbf{A}_4$?
Since the rank is 4, all singular values past the fourth are zero: $\sigma_5 = \sigma_6 = \cdots = 0$. By Eckart–Young, the spectral error is $\sigma_5 = 0$ and the Frobenius error is $\sqrt{\sigma_5^2 + \cdots} = 0$. The approximation is completely exact: $\mathbf{A}_4 = \mathbf{A}$. Truncation only discards information when $k < r$.

29.7.1 Why the Frobenius error is what it is

We will not prove full optimality over all $\mathbf{B}$ (that takes a careful argument), but we can see *why the truncation error is the tail of the singular values*, which is the part you use in practice. The Frobenius norm squared is the sum of squares of all entries, and a beautiful fact (provable by expanding $\|\mathbf{M}\|_F^2 = \operatorname{tr}(\mathbf{M}^\mathsf{T}\mathbf{M})$ and using orthonormality) is that it equals the sum of squared singular values:

$$\|\mathbf{A}\|_F^2 = \sigma_1^2 + \sigma_2^2 + \cdots + \sigma_r^2.$$

The discarded part $\mathbf{A} - \mathbf{A}_k = \sum_{i=k+1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ is itself a (sub-)SVD with singular values $\sigma_{k+1}, \dots, \sigma_r$, so by the same fact

$$\|\mathbf{A} - \mathbf{A}_k\|_F^2 = \sigma_{k+1}^2 + \cdots + \sigma_r^2.$$

The orthogonality of the $\mathbf{u}$'s and $\mathbf{v}$'s is what makes the cross terms vanish and the energies simply add. *No information overlaps between the pieces*, so you can account for them one at a time.

Modern application

Image compression and the data that runs machine learning. Store a grayscale photo as an $m \times n$ matrix of pixel brightnesses and take its SVD. Natural images are typically low-rank: a few dozen singular values capture nearly all of what your eye sees, and the rest are essentially texture noise. Keeping the top k terms, $\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$, gives a recognizable image stored in $k(m + n + 1)$ numbers instead of mn – often a $10\times$ or $20\times$ saving with no visible loss.

The same move is everywhere in ML. **Latent semantic analysis** takes a huge term-by-document matrix and truncates its SVD to discover a few dozen "topics." **Recommender systems** treat the (mostly empty) user-by-item ratings matrix as approximately low rank and use a truncated SVD to fill in the blanks – the famous Netflix-prize approach. **Modern large language models** are compressed and fine-tuned with *low-rank adapters* (LoRA), which add a trainable rank-k update $\mathbf{BC}$ – exactly the $\sum \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ shape – to giant weight matrices, cutting the number of trainable parameters by orders of magnitude. In each setting, the Eckart–Young theorem guarantees that truncating the SVD gives the closest rank-k matrix in both the spectral and Frobenius norms.

Walking question

Take a walk and turn this over: a rank-one matrix $\sigma \mathbf{u}\mathbf{v}^{\mathsf{T}}$ is the simplest non-trivial matrix there is – one output pattern $\mathbf{u}$, one input pattern $\mathbf{v}$, one gain σ. The SVD says *every* matrix is a stack of these, ranked by importance and mutually perpendicular. Why should rank-one pieces, the humblest building blocks, be enough to reconstruct any matrix whatsoever? And why is "perpendicular and ranked" the arrangement that makes truncation optimal rather than merely convenient?

29.8 The Pseudoinverse: Solving the Unsolvable

What does it mean to "invert" a matrix that has no inverse – a rectangular one, or a singular one? The SVD gives the right answer. To invert $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}}$ you would like to undo each step: $\mathbf{U}$ is orthogonal so its inverse is $\mathbf{U}^{\mathsf{T}}$; same for $\mathbf{V}$. The only trouble is $\mathbf{\Sigma}$, which you cannot invert if some singular values are zero (you cannot divide by zero) or if it is not square. So you do the only sensible thing: invert what you can and leave the rest at zero. Define $\mathbf{\Sigma}^{+}$ by replacing each nonzero σ_i with $1/\sigma_i$ and transposing the shape. Then the **Moore–Penrose pseudoinverse** is

$$\boxed{\mathbf{A}^{+} = \mathbf{V}\,\mathbf{\Sigma}^{+}\,\mathbf{U}^{\mathsf{T}}.}$$

Focus

The pseudoinverse $\mathbf{A}^+$ reverses $\mathbf{A}$ on the directions it can (the row/column spaces, dividing by σ_i) and sends the unreachable directions to zero. When $\mathbf{A}$ is genuinely invertible, $\mathbf{A}^+ = \mathbf{A}^{-1}$ exactly. When it is not, $\mathbf{A}^+\mathbf{b}$ delivers the *least-squares, minimum-norm* solution of $\mathbf{Ax} = \mathbf{b}$ – the best you could possibly hope for.

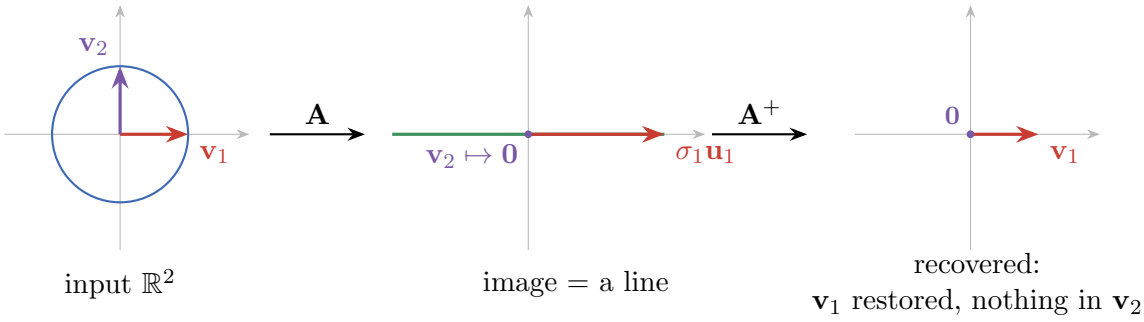


Figure 29.6: The pseudoinverse on $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$. $\mathbf{A}$ stretches $\mathbf{v}_1$ by $\sigma_1 = 2$ and crushes $\mathbf{v}_2$ to zero, so its image is just a line. $\mathbf{A}^+$ undoes the stretch on the reachable direction (multiply by $1/\sigma_1$) and, having no way to recover the crushed direction, returns nothing there. That is the least-squares, minimum-norm answer.

This closes a loop you opened in Chapters 18 and 19. The least-squares solution of an overdetermined system, the projection onto a column space, the line of best fit – all of them can be written as $\mathbf{A}^+\mathbf{b}$, and the SVD computes this solution stably even when $\mathbf{A}^\mathsf{T}\mathbf{A}$ is nearly singular (the case where the normal equations quietly blow up). This is why serious numerical software solves least squares through the SVD rather than by forming $\mathbf{A}^\mathsf{T}\mathbf{A}$ directly.

Example 29.1 (Pseudoinverse of a rank-deficient matrix)**.** Let $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$, which collapses the plane onto the x-axis (Figure 29.6). It has no inverse. Its SVD is trivial here: $\mathbf{U} = \mathbf{V} = \mathbf{I}$, $\mathbf{\Sigma} = \operatorname{diag}(2, 0)$. So

$$\mathbf{\Sigma}^+ = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{A}^+ = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{bmatrix}.$$

Check the spirit of it: $\mathbf{A}^+$ undoes the doubling on the x-axis (multiply by $\frac{1}{2}$) and does the only reasonable thing with the crushed y-direction – leaves it at zero, since no input could ever have produced a nonzero y-output.

29.9 Symmetry, Eigenvalues, and the SVD

How does the SVD relate to the eigen-decomposition you already know? They coincide in the nicest case and diverge instructively otherwise.

If $\mathbf{A}$ is **symmetric positive definite**, its spectral decomposition $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ *is* its SVD: take $\mathbf{U} = \mathbf{V} = \mathbf{Q}$ and $\sigma_i = \lambda_i$. The eigen-axes are already orthonormal and the eigenvalues already positive, so there is nothing left to do.

If $\mathbf{A}$ is **symmetric but has a negative eigenvalue**, the SVD differs by a sign. Suppose $\mathbf{Aq} = \lambda\mathbf{q}$ with $\lambda < 0$. The singular value must be positive, $\sigma = |\lambda| = -\lambda$, and to keep $\mathbf{Av} = \sigma\mathbf{u}$ intact we set $\mathbf{v} = \mathbf{q}$ but $\mathbf{u} = -\mathbf{q}$ (or vice versa) so the sign lands in $\mathbf{U}$ instead of in σ. In general: **the singular values of a symmetric matrix are the absolute values of its eigenvalues**.

Connection to other math

Polar decomposition. Group the SVD as

$$\mathbf{A} = \underbrace{(\mathbf{U}\mathbf{V}^\mathsf{T})}_{\mathbf{Q}}\underbrace{(\mathbf{V}\mathbf{\Sigma}\mathbf{V}^\mathsf{T})}_{\mathbf{P}} = \mathbf{QP}.$$

Then $\mathbf{Q} = \mathbf{U}\mathbf{V}^\mathsf{T}$ is orthogonal (a pure rotation/reflection) and $\mathbf{P} = \mathbf{V}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ is symmetric positive semidefinite (a pure stretch along orthogonal axes). This is the matrix analogue of writing a complex number as $z = e^{i\theta} \cdot r$: a rotation times a stretch. Engineers use the polar decomposition to split a deformation into "rigid rotation" plus "pure strain," and graphics programmers use it to extract the nearest true rotation from a slightly corrupted one. It falls out of the SVD in a single line.

History and connection

The singular value decomposition was discovered independently at least four times, as mathematicians in different fields stumbled upon the same geometric truth from completely different directions.

Who and when: Eugenio Beltrami (1873) in Italy and Camille Jordan (1874) in France first derived the decomposition for real square matrices. James Joseph Sylvester (1889) in England rediscovered it independently fifteen years later, calling the singular values "canonical multipliers." In 1907, Erhard Schmidt in Germany generalized the idea to integral operators in infinite dimensions. In 1936, the American psychometricians Carl Eckart and Gale Young proved the celebrated low-rank approximation theorem that made the SVD the mathematical foundation of data compression.

What problem they were stuck on: Beltrami and Jordan were studying *bilinear forms*—functions that take two vector inputs and produce a single scalar, linear in each slot (such as formulas for kinetic energy, work, or geometric surface curvature). They asked: can one choose an independent orthogonal change of coordinates for the first vector and another for the second vector that diagonalizes the formula simultaneously? Sylvester was pursuing canonical forms in invariant theory, while Schmidt was solving Fredholm integral equations—the infinite-dimensional analogue of matrix equations, where discrete sums over vector coordinates become continuous integrals over functions. Eckart and Young were trying to extract a few fundamental psychological factors (such as verbal or spatial reasoning) from large tables of student test scores, leading them to ask: what is the single best rank-k approximation to an empirical data matrix?

The tools they had: Pencil, paper, determinant expansions, and classical eigenvalue theory for quadratic forms. None of the early pioneers had access to computers. When electronic computers arrived in the 1950s, scientists immediately tried to compute the SVD using the textbook formula: form $\mathbf{A}^\mathsf{T}\mathbf{A}$, find its eigenvalues, and take square roots. This triggered a severe numerical crisis. In finite-precision floating-point arithmetic, forming $\mathbf{A}^\mathsf{T}\mathbf{A}$ squares the matrix's *condition number* (the ratio of largest to smallest stretch, $\sigma_{\max}/\sigma_{\min}$, which measures

sensitivity to perturbations). If a matrix has singular values spanning 1 down to 10^{-8}, $\mathbf{A}^\mathsf{T}\mathbf{A}$ spreads them from 1 down to 10^{-16}—right at the edge of double precision, where small singular values that reveal near-singularity are wiped out by rounding noise.

Controversy and resolution: Sylvester's 1889 paper provoked priority friction, as Jordan had published the identical canonical form fifteen years earlier. Decades later, Eckart and Young's 1936 paper was found to have been largely anticipated by Schmidt's 1907 work on integral operators, leading many historians to refer to the low-rank approximation result as the Schmidt–Eckart–Young–Mirsky theorem. The computational bottleneck was finally resolved in 1965 by Gene Golub and William Kahan, who introduced a backward-stable two-stage algorithm. Their method applies *Householder reflections* (orthogonal mirrorings across hyperplanes) directly to $\mathbf{A}$ to reduce it to *bidiagonal form* (nonzeros only on the main diagonal and the diagonal directly above it), completely avoiding the dangerous $\mathbf{A}^\mathsf{T}\mathbf{A}$.

Impact: The SVD applies to every matrix without restriction. Unlike the Jordan canonical form, which requires square matrices and breaks down when eigenvectors collide, the SVD exists unconditionally for any matrix, rectangular or defective, and provides an orthonormal coordinate frame for all four fundamental subspaces simultaneously. It gave science the pseudoinverse for ill-conditioned least squares, Principal Component Analysis for dimensionality reduction, and low-rank compression for modern deep learning.

What is still open: For two-dimensional matrices, the low-rank approximation problem is completely solved. But for *tensors*—three-dimensional or higher data arrays, such as video volumes or multi-modal embeddings—the situation is fundamentally harder. Finding the best rank-k approximation of a 3-way tensor is provably NP-hard, and the best rank-k approximation can fail to exist because tensor rank is not topologically closed (a sequence of rank-2 tensors can converge to a rank-3 tensor). At the same time, computing *sparse* singular vectors (forcing many coordinates to be exactly zero for interpretability in genetics and finance) is also NP-hard. Scalable, communication-efficient low-rank approximations for massive distributed datasets remain an active research frontier.

29.10 How It Is Computed (and Why Not the Obvious Way)

The hand recipe – diagonalize $\mathbf{A}^\mathsf{T}\mathbf{A}$, take square roots – is perfect for a 2×2 by hand and a disaster for a serious computation. The reason is *conditioning*: forming $\mathbf{A}^\mathsf{T}\mathbf{A}$ squares the singular values, and squaring spreads them out. A matrix whose singular values range from 1 down to 10^{-8} becomes, after $\mathbf{A}^\mathsf{T}\mathbf{A}$, a matrix with eigenvalues from 1 down to 10^{-16} – right at the edge of what floating-point can distinguish from zero. You can lose half your accuracy before you start.

Common trap

Never compute an SVD (or a least-squares solution) in production by forming $\mathbf{A}^\mathsf{T}\mathbf{A}$ if you can avoid it. The squaring throws away precision exactly where you need it most – in the small singular values that tell you about near-rank-deficiency. Real algorithms (Golub–Kahan bidiagonalization and its descendants) work on $\mathbf{A}$ directly, never forming $\mathbf{A}^\mathsf{T}\mathbf{A}$, and keep full accuracy. When you call `numpy.linalg.svd` or LAPACK's `dgesdd`, this is what is running.

For a feel of the cost: a full SVD of an $m \times n$ matrix takes on the order of $mn \cdot \min(m, n)$ arithmetic operations – comparable to a few matrix multiplications. For the giant, mostly-low-rank matrices of

data science, one rarely wants the full thing; *randomized* and *truncated* SVD algorithms find just the top k singular triples in roughly $O(mnk)$ time, which is what makes SVD-based methods feasible on web-scale data.

29.11 Summary: One Picture, Many Doors

Focus

Everything in this chapter hangs on one sentence and one equation.

- **Geometry:** every matrix sends the unit sphere to an ellipsoid; $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ is *rotate, stretch, rotate.*
- **Pieces:** $\mathbf{v}_i$ = eigenvectors of $\mathbf{A}^\mathsf{T}\mathbf{A}$; $\sigma_i = \sqrt{\lambda_i}$; $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$.
- **Subspaces:** $\mathbf{v}$'s split into row space / null space; $\mathbf{u}$'s split into column space / left null space; σ_i is the stretch from one to the other.
- **Approximation (Eckart–Young):** keep the top k terms of $\sum \sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$ for the best rank-k matrix; error is the tail of the singular values.
- **Inversion:** $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\mathsf{T}$ solves least squares, minimum norm, always.

In the next chapter, we apply this factorization to empirical data. When you collect multivariate observations – gene expression levels, movie ratings, or financial indicators – the data points form an elongated cloud in high dimensions. Center that cloud at its mean, stack the points as rows of a matrix, and the SVD identifies the orthogonal axes along which the data varies most: the *principal components*. Principal Component Analysis is the direct application of the SVD to sample covariance matrices. You already have the geometric foundation to see exactly how it works.

29.12 Exercises

Exercise 29.1 (Warmup: read off the SVD)**.** Write the SVD $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ of each matrix by inspection (no $\mathbf{A}^\mathsf{T}\mathbf{A}$ needed): (a) $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$; (b) $\begin{bmatrix} 0 & 3 \\ 2 & 0 \end{bmatrix}$ (hint: it is a scaling composed with a swap of axes); (c) the rotation $\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$; (d) Can any nonzero matrix have all of its singular values equal to zero? Why or why not? What are the singular values in each case?

Exercise 29.2 (A full 2×2 by hand)**.** Compute the SVD of $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ (the unit shear). Form $\mathbf{A}^\mathsf{T}\mathbf{A}$, find its eigenvalues and eigenvectors, the singular values, and the $\mathbf{u}$'s. Confirm that even though this matrix has the single repeated eigenvalue 1 and only one eigenvector, it has two distinct singular values and a complete SVD. What does this say about the difference between eigenvalues and singular values?

Exercise 29.3 (Rank-one matrices)**.** Let $\mathbf{A} = \mathbf{a}\mathbf{b}^\mathsf{T}$ for nonzero vectors $\mathbf{a} \in \mathbb{R}^m$, $\mathbf{b} \in \mathbb{R}^n$. Show that $\mathbf{A}$ has exactly one nonzero singular value, and find it in terms of $\|\mathbf{a}\|$ and $\|\mathbf{b}\|$. Identify $\mathbf{u}_1$ and $\mathbf{v}_1$.

Exercise 29.4 (Singular values vs. eigenvalues)**.** Give a 2×2 matrix whose eigenvalues are both 1 but whose larger singular value is greater than 100. (Hint: a strong shear.) Explain geometrically how a matrix can fix the length of two directions yet stretch the unit circle enormously.

Exercise 29.5 (Determinant and singular values)**.** Prove that for any square $\mathbf{A}$, $|\det \mathbf{A}| = \prod_i \sigma_i$, using $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ and the facts that det is multiplicative and $|\det \mathbf{U}| = |\det \mathbf{V}| = 1$. Deduce that $\mathbf{A}$ is invertible if and only if all its singular values are positive.

Exercise 29.6 (Frobenius norm and the singular values)**.** Using $\|\mathbf{A}\|_F^2 = \operatorname{tr}(\mathbf{A}^\mathsf{T}\mathbf{A})$, the SVD, and the fact that trace is unchanged by orthogonal similarity, prove that $\|\mathbf{A}\|_F^2 = \sigma_1^2 + \cdots + \sigma_r^2$. Then explain why the best rank-k approximation leaves Frobenius error $\sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}$.

Exercise 29.7 (Spectral norm)**.** The spectral norm is $\|\mathbf{A}\|_2 = \max_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|$. Using the SVD, prove that $\|\mathbf{A}\|_2 = \sigma_1$, the largest singular value, and that the maximizing $\mathbf{x}$ is $\mathbf{v}_1$. (Hint: write $\mathbf{x}$ in the orthonormal basis $\mathbf{v}_1, \dots, \mathbf{v}_n$ and compute $\|\mathbf{A}\mathbf{x}\|^2$.)

Exercise 29.8 (Pseudoinverse and least squares)**.** For $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}$, write the SVD of $\mathbf{A}$, form $\mathbf{A}^+$, and compute $\hat{\mathbf{x}} = \mathbf{A}^+\mathbf{b}$. Verify directly that $\hat{\mathbf{x}}$ minimizes $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|$ and explain geometrically why the residual vector $\mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 0 \\ 4 \\ 5 \end{bmatrix}$ lies entirely in the left null space of $\mathbf{A}$.

Exercise 29.9 (Polar decomposition by hand)**.** Using the SVD found in Section 29.5 for $\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$, compute the polar factors $\mathbf{Q} = \mathbf{U}\mathbf{V}^\mathsf{T}$ and $\mathbf{P} = \mathbf{V}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$. Check that $\mathbf{Q}$ is orthogonal ($\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$), that $\mathbf{P}$ is symmetric, and that $\mathbf{Q}\mathbf{P} = \mathbf{A}$.

Exercise 29.10 (Why truncation is greedy-optimal)**.** Argue, using the orthogonality of the $\mathbf{u}_i$ and $\mathbf{v}_i$, that the "energy" $\|\mathbf{A}\|_F^2$ splits additively across the rank-one terms: $\|\mathbf{A}\|_F^2 = \sum_i \sigma_i^2$ with no cross terms. Explain in one or two sentences why this additivity is exactly what makes "keep the largest σ_i" the optimal way to spend a rank budget. (This is the intuition behind Eckart–Young.)

Exercise 29.11 (Challenge: SVD of $\mathbf{A}$ vs. $\mathbf{A}^\mathsf{T}$)**.** If $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$, write down the SVD of $\mathbf{A}^\mathsf{T}$. What happens to the singular values, the left singular vectors, and the right singular vectors? Use this to explain in one sentence why $\mathbf{A}$ and $\mathbf{A}^\mathsf{T}$ always have the same singular values, the same rank, and why $\mathbf{A}^\mathsf{T}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^\mathsf{T}$ share their nonzero eigenvalues.

Chapter 30

Principal Component Analysis and Data

Imagine you have measured a thousand people and written down, for each one, their height and their arm span. You now have a cloud of a thousand dots floating in a two-dimensional plane: one axis is height, the other is arm span. If you actually plotted those dots, you would not see a shapeless blob. You would see a long, slanted, cigar-shaped smear, stretched out along the diagonal where "tall" meets "long-armed," and thin in the perpendicular direction. The cloud has a *grain* to it, like wood. There is one direction along which the data really varies, and another, perpendicular to it, along which it barely varies at all.

Principal Component Analysis – PCA – finds that grain. It hands you the directions in which your data spreads out the most, ranked from most spread to least, together with a number for "how much spread" each one carries.

In the previous chapter, we dissected matrices using the Singular Value Decomposition: every matrix, regardless of its shape, acts by rotating space, stretching along orthogonal axes, and rotating again ($\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$). What makes PCA a theorem of linear algebra rather than a purely statistical recipe is that those directions of greatest spread are the *eigenvectors of a symmetric covariance matrix* – or, equivalently and more stably, the *right singular vectors* of the centered data matrix. Symmetric matrices, the spectral theorem, and the SVD all come together directly in PCA.

The approach here is slow, geometric, and visual. We learn to *center* a cloud of data – sliding it bodily so its balance point sits at the origin – then build the *covariance matrix*, the little symmetric matrix that records how the cloud spreads and how features vary together, and watch its eigenvectors pick out the grain. From there we unpack what "variance explained" means, showing how to drop dimensions without losing meaningful signal, and connect PCA to the SVD of the centered data matrix used in numerical software. Finally comes the practical payoff: compressing images, building "eigenfaces," and making sense of the high-dimensional embedding vectors behind modern machine learning.

30.1 Data as a cloud of points

30.1.1 The data matrix

All of data analysis starts with a table. You have n **samples** (individual observations, such as people, images, transactions, or days) and for each one you record d **features** (the numerical quantities you measured, such as height, arm span, pixel brightness, or temperature). Stack each sample as a row, and you get an $n \times d$ **data matrix**

$$\mathbf{X} = \begin{bmatrix} — \ \mathbf{x}_1^\top \ — \\ — \ \mathbf{x}_2^\top \ — \\ \vdots \\ — \ \mathbf{x}_n^\top \ — \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

Row i is one sample $\mathbf{x}_i \in \mathbb{R}^d$. Column j is one feature measured across all n samples. Geometrically, each row is a single point living in d-dimensional *feature space* – the coordinate space where each axis corresponds to one measured feature – and the whole dataset is a cloud of n points floating in $\mathbb{R}^d$.

Focus

Keep the two readings of $\mathbf{X}$ straight. **Rows** are points in feature space $\mathbb{R}^d$ – this is the geometric picture PCA cares about. **Columns** are features, the things you measured. PCA asks a question about the rows (how is the cloud shaped?) and answers it with a matrix built from the columns (how do features co-vary?).

Example 30.1. Here is a tiny dataset we will carry through the chapter: five points in the plane ($n = 5$, $d = 2$).

$$\mathbf{X} = \begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 6 & 5 \\ 8 & 5 \\ 5 & 1 \end{bmatrix}.$$

Each row is a dot at (x, y). Plot them and you see five points trending up and to the right – a little cloud with a clear diagonal grain. We will find that grain by hand before the chapter is done.

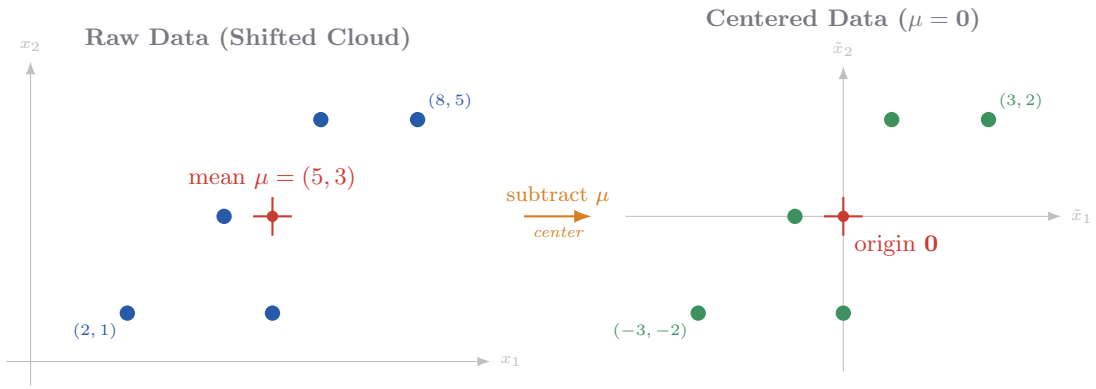


Figure 30.1: The five-point data cloud of Example 30.1. *Left:* raw data points clustered around sample mean $\mu = (5, 3)$. *Right:* centering subtracts the mean, sliding the center of gravity to the origin so that variances and covariances reflect genuine spread about zero.

30.1.2 The mean, and why we center

The first thing PCA does is move the cloud so that its center of mass sits at the origin. This step is so simple it is easy to dismiss, but it is the difference between measuring how data *spreads* and measuring how big its numbers happen to be.

The **mean** (or centroid) of the cloud is the average of all the points,

$$\bar{\mathbf{x}} \;=\; \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i \;=\; \frac{1}{n}\,\mathbf{X}^{\mathsf{T}}\mathbf{1},$$

where $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector (summing a matrix's rows is exactly right-multiplying its transpose by $\mathbf{1}$). It is itself a point in $\mathbb{R}^d$ – the balance point of the cloud, where it would sit perfectly level on a pin.

Centering the data means subtracting the mean from every point:

$$\tilde{\mathbf{x}}_i \;=\; \mathbf{x}_i - \bar{\mathbf{x}}, \qquad \mathbf{B} \;=\; \mathbf{X} - \mathbf{1}\,\bar{\mathbf{x}}^{\mathsf{T}}.$$

The centered matrix $\mathbf{B}$ holds the same cloud, slid bodily over so its centroid lands on the origin. Distances between points, angles, the whole shape – all unchanged. Only the location moved.

Intuition

PCA is about *shape*, not *position*. A cloud of stars looks the same whether you describe it from Earth or from Mars; only the coordinates of its center change. Centering throws away the arbitrary "where is the origin" information so that everything left over is about the cloud's intrinsic spread. Skipping this step is the single most common way to get nonsense out of PCA.

Example 30.2. Center the five points of Example 30.1. The column means are

$$\bar{x} = \tfrac{2+4+6+8+5}{5} = \tfrac{25}{5} = 5, \qquad \bar{y} = \tfrac{1+3+5+5+1}{5} = \tfrac{15}{5} = 3,$$

so $\bar{\mathbf{x}} = (5, 3)$. Subtracting it from every row gives

$$\mathbf{B} = \begin{bmatrix} 2-5 & 1-3 \\ 4-5 & 3-3 \\ 6-5 & 5-3 \\ 8-5 & 5-3 \\ 5-5 & 1-3 \end{bmatrix} = \begin{bmatrix} -3 & -2 \\ -1 & 0 \\ 1 & 2 \\ 3 & 2 \\ 0 & -2 \end{bmatrix}.$$

As a check, every column of $\mathbf{B}$ now sums to zero: $(-3-1+1+3+0) = 0$ and $(-2+0+2+2-2) = 0$. That is exactly what "centered at the origin" means.

Common trap

A centered column sums to zero – always check this. If after subtracting the mean your columns do not sum to (numerically) zero, you subtracted the wrong thing, subtracted a row mean instead of a column mean, or transposed your matrix. Every downstream formula in this chapter quietly assumes $\mathbf{B}$ is centered.

30.2 The covariance matrix

30.2.1 Variance along one feature

Take a single centered feature – one column of $\mathbf{B}$, say the first, with entries $b_1, \ldots, b_n$ (each already has mean zero). Its **variance** is the average squared entry,

$$s^2 \;=\; \frac{1}{n}\sum_{i=1}^{n} b_i^2 \;=\; \frac{1}{n}\,\|\text{column}\|^2.$$

Variance is just "average squared distance from the mean." A big variance means the feature is spread out; a small variance means it clusters tightly. Notice it is a squared length divided by n – a dot product of a column with itself.

(A note on the divisor: statisticians often use $\frac{1}{n-1}$ instead of $\frac{1}{n}$, which makes the variance an unbiased estimate of the true population variance. It changes every number below by the same constant factor $\frac{n}{n-1}$ and so changes *none* of the directions PCA finds. We use $\frac{1}{n}$ for clean arithmetic and will flag where the choice matters.)

30.2.2 Covariance: do two features move together?

Two features can each be spread out, but the interesting question is whether they spread *together*. When height is above its mean, is arm span usually above its mean too? The number that measures this is the **covariance** – the average product of how far two variables stray from their respective means. For two centered columns $\mathbf{a}$ and $\mathbf{b}$ of $\mathbf{B}$,

$$\text{Cov}(\mathbf{a}, \mathbf{b}) \;=\; \frac{1}{n}\sum_{i=1}^{n} a_i b_i \;=\; \frac{1}{n}\,\mathbf{a}\cdot\mathbf{b}.$$

There is the dot product again. Positive covariance: the features tend to be above or below their means *together* (the cloud tilts up-and-to-the-right). Negative covariance: when one is high the other tends to be low (the cloud tilts down-and-to-the-right). Covariance near zero: no linear tilt; the cloud is round or axis-aligned. And covariance of a feature with *itself* is exactly its variance.

Intuition

Covariance is an inner product normalized by sample size, measuring directional alignment. Two centered features have large positive covariance for the same reason two arrows have a large positive dot product – they point "the same way" across the samples. Variance is the special case where a feature is dotted with itself, the squared length. The entire covariance matrix is the table of all these pairwise dot products, scaled by $1/n$.

30.2.3 Packing every covariance into one matrix

We have d features, so there are d^2 pairwise covariances (including each feature with itself). Arrange them in a $d \times d$ grid: entry (j, k) is the covariance of feature j with feature k. This is the **covariance matrix C** – a square symmetric table holding the variance of each feature along its main diagonal and the pairwise covariances off the diagonal:

$$\boxed{\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B}} \qquad \in \mathbb{R}^{d\times d}.$$

Notice how $\mathbf{B}^\mathsf{T}\mathbf{B}$ multiplies every column of $\mathbf{B}$ against every other column; entry (j, k) is (column j) $\cdot$ (column k), which is exactly n times the covariance. Divide by n and you have the covariance matrix. The diagonal entries are the variances of the individual features; the off-diagonal entries are the covariances between feature pairs.

Formal

For a centered data matrix $\mathbf{B} \in \mathbb{R}^{n\times d}$, the **covariance matrix** is $\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B}$. It is

1. **symmetric:** $\mathbf{C}^\mathsf{T} = \frac{1}{n}(\mathbf{B}^\mathsf{T}\mathbf{B})^\mathsf{T} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B} = \mathbf{C}$, because $\mathrm{Cov}(\mathbf{a}, \mathbf{b}) = \mathrm{Cov}(\mathbf{b}, \mathbf{a})$;
2. **positive semidefinite:** for any direction $\mathbf{w} \in \mathbb{R}^d$,

$$\mathbf{w}^\mathsf{T}\mathbf{C}\,\mathbf{w} = \tfrac{1}{n}\mathbf{w}^\mathsf{T}\mathbf{B}^\mathsf{T}\mathbf{B}\mathbf{w} = \tfrac{1}{n}\|\mathbf{B}\mathbf{w}\|^2 \;\geq\; 0.$$

PCA relies directly on both properties. *Symmetric* means the spectral theorem applies: $\mathbf{C}$ has a full set of *orthonormal* eigenvectors with *real* eigenvalues. *Positive semidefinite* means those eigenvalues are all ≥ 0 – which is exactly what we need, because each one is about to be revealed as a variance, and a variance can never be negative.

Connection to other math

The quantity $\mathbf{w}^\mathsf{T}\mathbf{C}\mathbf{w}$ in the positive-semidefinite check is a **quadratic form**, the same object you met in Chapters 26 and 27 and will meet again in Chapter 36. Here it has a direct geometric interpretation: $\mathbf{w}^\mathsf{T}\mathbf{C}\mathbf{w}$ is the variance of the data *after projecting it onto the direction* $\mathbf{w}$ (when $\mathbf{w}$ is a unit vector). The covariance matrix is precisely the machine that turns a direction into "how much does the cloud spread that way." Maximizing that variance over unit vectors is an eigenvalue problem for the symmetric matrix $\mathbf{C}$.

Example 30.3. Compute the covariance matrix of our centered $\mathbf{B}$ from Example 30.2:

$$\mathbf{B} = \begin{bmatrix} -3 & -2 \\ -1 & 0 \\ 1 & 2 \\ 3 & 2 \\ 0 & -2 \end{bmatrix}.$$

We need $\mathbf{B}^\mathsf{T}\mathbf{B}$, a 2×2 matrix. Call the columns $\mathbf{a}$ (first) and $\mathbf{b}$ (second).

$$\mathbf{a} \cdot \mathbf{a} = 9 + 1 + 1 + 9 + 0 = 20, \quad \mathbf{b} \cdot \mathbf{b} = 4 + 0 + 4 + 4 + 4 = 16,$$

$$\mathbf{a} \cdot \mathbf{b} = (-3)(-2) + (-1)(0) + (1)(2) + (3)(2) + (0)(-2) = 6 + 0 + 2 + 6 + 0 = 14.$$

So

$$\mathbf{B}^\mathsf{T}\mathbf{B} = \begin{bmatrix} 20 & 14 \\ 14 & 16 \end{bmatrix}, \qquad \mathbf{C} = \frac{1}{5}\begin{bmatrix} 20 & 14 \\ 14 & 16 \end{bmatrix} = \begin{bmatrix} 4 & 2.8 \\ 2.8 & 3.2 \end{bmatrix}.$$

Read it off: feature 1 has variance 4, feature 2 has variance 3.2, and they covary positively ($2.8 > 0$) – the cloud tilts up to the right, exactly as Figure 30.1 showed.

Pause

Pause and think: Scaling the cloud. Suppose you multiply all entries in your raw dataset by 10 (for instance, converting measurements from centimeters to millimeters).

- What happens to the mean $\bar{\mathbf{x}}$ and the centered matrix $\mathbf{B}$?
- What happens to the covariance matrix $\mathbf{C}$ and its eigenvalues?
- What happens to the principal component directions $\mathbf{v}_k$?

Answer: The sample mean $\bar{\mathbf{x}}$ and the centered rows of $\mathbf{B}$ are scaled by 10. The covariance matrix $\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B}$ is scaled by $10^2 = 100$, so every eigenvalue λ_k grows by a factor of 100 (since variance is a squared distance). But the eigenvectors $\mathbf{v}_k$ do not change at all. The grain of the cloud depends on its geometric shape and orientation, not on whether you measure in centimeters or millimeters.

Compute it yourself

Build covariance intuition with three hand clouds, each centered. (a) Four points $(\pm 1, 0), (0, \pm 1)$: a round cloud – compute $\mathbf{C}$ and watch the off-diagonal vanish. (b) Four points $(\pm 1, \pm 1)$ with matching signs $(1, 1), (-1, -1)$ doubled: strong positive covariance. (c) The same with opposite signs: strong negative covariance. In each case predict the sign of the off-diagonal entry *before* you multiply $\mathbf{B}^\mathsf{T}\mathbf{B}$, then check.

30.3 Principal components: the directions of greatest spread

30.3.1 The variance of a projection

Here is the central question of the whole chapter, stated geometrically. Pick a unit direction $\mathbf{w}$ (a unit vector in $\mathbb{R}^d$). Project every centered point onto the line through $\mathbf{w}$. The projection of point

$\tilde{\mathbf{x}}_i$ is the scalar $\tilde{\mathbf{x}}_i \cdot \mathbf{w} = \tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w}$ – its signed coordinate along that line. We get n numbers, one per sample. *How spread out are they?*

Because the data is centered, those projected coordinates also have mean zero (the average of $\tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w}$ is $\bar{\tilde{\mathbf{x}}}^{\mathsf{T}}\mathbf{w} = \mathbf{0}^{\mathsf{T}}\mathbf{w} = 0$). So their variance is just the average of their squares:

$$\text{Var}(\text{projection onto } \mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(\tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w})^2 = \frac{1}{n}\|\mathbf{B}\mathbf{w}\|^2 = \mathbf{w}^{\mathsf{T}}\Big(\tfrac{1}{n}\mathbf{B}^{\mathsf{T}}\mathbf{B}\Big)\mathbf{w} = \mathbf{w}^{\mathsf{T}}\mathbf{C}\,\mathbf{w}.$$

That short calculation is the hinge of PCA, so let us say in words what it says in symbols: **the variance of the data along a unit direction $\mathbf{w}$ equals $\mathbf{w}^{\mathsf{T}}\mathbf{C}\mathbf{w}$.** The covariance matrix, sandwiched between a direction and itself, returns the spread in that direction.

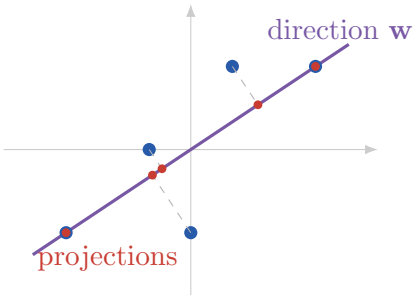


Figure 30.2: Projecting the centered cloud onto a candidate direction $\mathbf{w}$. The red dots are the projected coordinates $\tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w}$. PCA seeks the direction that makes those red dots as spread out as possible – the direction maximizing $\mathbf{w}^{\mathsf{T}}\mathbf{C}\mathbf{w}$.

30.3.2 Maximize the spread: an eigenvalue problem

We want the direction of *greatest* spread, the grain of the cloud. So we maximize $\mathbf{w}^{\mathsf{T}}\mathbf{C}\mathbf{w}$ over all unit vectors:

$$\max_{\|\mathbf{w}\|=1} \mathbf{w}^{\mathsf{T}}\mathbf{C}\,\mathbf{w}.$$

Without the constraint $\|\mathbf{w}\| = 1$ this has no answer – you could scale $\mathbf{w}$ up forever. The constraint is what makes "direction" meaningful: we ask only about the *way* $\mathbf{w}$ points, not how long it is.

Now recall the spectral theorem. Because $\mathbf{C}$ is symmetric, it has orthonormal eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_d$ with real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$ (we sort them largest first, and they are ≥ 0 by positive semidefiniteness). Write any unit vector in this eigenbasis as $\mathbf{w} = \sum_j c_j\mathbf{v}_j$ with $\sum_j c_j^2 = 1$. Then, using $\mathbf{C}\mathbf{v}_j = \lambda_j\mathbf{v}_j$ and orthonormality,

$$\mathbf{w}^{\mathsf{T}}\mathbf{C}\,\mathbf{w} = \Big(\sum_j c_j\mathbf{v}_j\Big)^{\mathsf{T}}\Big(\sum_k c_k\lambda_k\mathbf{v}_k\Big) = \sum_j \lambda_j c_j^2.$$

This is a weighted average of the eigenvalues, with weights c_j^2 that sum to 1. A weighted average is largest when all the weight sits on the biggest value: take $c_1 = 1$ and the rest zero, i.e. $\mathbf{w} = \mathbf{v}_1$. Then $\mathbf{w}^{\mathsf{T}}\mathbf{C}\mathbf{w} = \lambda_1$. So:

Formal

The principal components are the eigenvectors of the covariance matrix. Let $\mathbf{C}$ have orthonormal eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_d$ with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$. Then:

- The direction of greatest variance is $\mathbf{v}_1$, and that maximum variance is λ_1.
- Among directions *orthogonal* to $\mathbf{v}_1$, the one of greatest variance is $\mathbf{v}_2$, with variance λ_2; and so on down the list.

The eigenvectors $\mathbf{v}_k$ are the **principal components** (or principal directions); the eigenvalues λ_k are the **variances explained** by each component.

The "and so on" is worth spelling out. Why is the next direction $\mathbf{v}_2$? If we require a unit vector $\mathbf{w}$ to be orthogonal to the first principal direction $\mathbf{v}_1$, its coordinate along $\mathbf{v}_1$ must vanish: $c_1 = \mathbf{w} \cdot \mathbf{v}_1 = 0$. The weighted sum of eigenvalues then starts at the second term:

$$\mathbf{w}^{\mathsf{T}} \mathbf{C}\, \mathbf{w} \;=\; \sum_{j=2}^{d} \lambda_j c_j^2 \;\leq\; \lambda_2 \sum_{j=2}^{d} c_j^2 \;=\; \lambda_2,$$

with equality achieved when $c_2 = 1$ and all other weights are zero, which means $\mathbf{w} = \mathbf{v}_2$. Repeating this orthogonal constraint step by step picks out each eigenvector down the sorted list. The principal components thus form a complete *orthonormal* set of axes, ranked by how much of the data's spread each one carries. They are a rotated coordinate system, custom fit to your cloud.

Intuition

The covariance matrix stretches space. Feed it the unit circle of directions and it reports back a variance for each. The largest variance occurs along the top eigenvector of $\mathbf{C}$, which is the long axis of the elliptical cloud. The eigenvectors are the axes of the data ellipse; the eigenvalues are (the squares of) how far the ellipse reaches along each axis. PCA is "find the axes of the ellipse your data traces out."

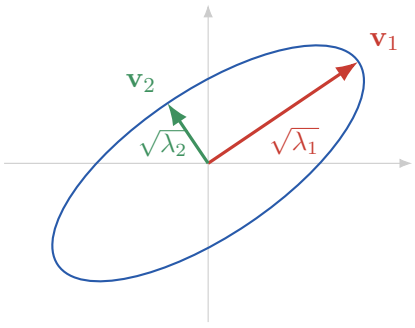


Figure 30.3: The data ellipse. Its long axis is the first principal component $\mathbf{v}_1$, its short axis the second $\mathbf{v}_2$, and the axes are orthogonal because $\mathbf{C}$ is symmetric. The reach along each axis is $\sqrt{\lambda_k}$, the standard deviation in that direction.

30.3.3 Doing it by hand on the five points

Let us finish the running example: find the principal components of

$$\mathbf{C} = \begin{bmatrix} 4 & 2.8 \\ 2.8 & 3.2 \end{bmatrix}$$

from Example 30.3. Eigenvalues come from $\det(\mathbf{C} - \lambda \mathbf{I}) = 0$:

$$(4-\lambda)(3.2-\lambda) - 2.8^2 = 0 \implies \lambda^2 - 7.2\lambda + (12.8 - 7.84) = 0 \implies \lambda^2 - 7.2\lambda + 4.96 = 0.$$

The quadratic formula gives

$$\lambda = \frac{7.2 \pm \sqrt{7.2^2 - 4(4.96)}}{2} = \frac{7.2 \pm \sqrt{51.84 - 19.84}}{2} = \frac{7.2 \pm \sqrt{32}}{2} = 3.6 \pm 2\sqrt{2}.$$

So $\lambda_1 = 3.6 + 2\sqrt{2} \approx 6.43$ and $\lambda_2 = 3.6 - 2\sqrt{2} \approx 0.77$. Sanity checks: the eigenvalues sum to $7.2 = \operatorname{tr}\mathbf{C} = 4 + 3.2$, and their product is $\lambda_1\lambda_2 = 3.6^2 - 8 = 12.96 - 8 = 4.96 = \det \mathbf{C}$. Both hold.

For the top eigenvector, solve $(\mathbf{C} - \lambda_1 \mathbf{I})\mathbf{v}_1 = \mathbf{0}$. Using the first row, $(4 - \lambda_1)v_1 + 2.8\, v_2 = 0$, so

$$v_2 / v_1 = \frac{\lambda_1 - 4}{2.8} = \frac{2\sqrt{2} - 0.4}{2.8} \approx \frac{2.428}{2.8} \approx 0.867.$$

Thus $\mathbf{v}_1 \propto (1, 0.867)$; normalized, $\mathbf{v}_1 \approx (0.756, 0.655)$. That points up and to the right at about 41° – precisely the grain we eyed in Figure 30.1. The second component is orthogonal to it, $\mathbf{v}_2 \approx (-0.655, 0.756)$, the thin direction across the cloud.

Compute it yourself

Verify the eigenvector by hand: multiply $\mathbf{C}\,\mathbf{v}_1$ with $\mathbf{v}_1 = (0.756, 0.655)$ and check you get back roughly $6.43\,\mathbf{v}_1$. ($\mathbf{C}\mathbf{v}_1 \approx (4 \cdot 0.756 + 2.8 \cdot 0.655,\ 2.8 \cdot 0.756 + 3.2 \cdot 0.655) = (4.86,\ 4.21)$, and $6.43 \cdot (0.756, 0.655) = (4.86,\ 4.21)$. It matches.)

Common trap

Eigenvectors come with a sign ambiguity: $\mathbf{v}_1$ and $-\mathbf{v}_1$ are the same principal *direction* and carry the same variance. Software may hand you either one, so a PCA plot can appear flipped from one run or library to the next. Nothing is wrong; an axis has no built-in arrowhead. Only the line matters, not which end you call positive.

30.4 Variance explained and dimensionality reduction

30.4.1 The total variance is the trace

How much spread does the cloud have *in total*? Add up the variances of all the features – the diagonal of $\mathbf{C}$ – which is $\operatorname{tr}\mathbf{C}$. But the trace is also the sum of the eigenvalues. So

$$\text{total variance} = \operatorname{tr}\mathbf{C} = \sum_{j=1}^{d} \mathbf{C}_{jj} = \sum_{k=1}^{d} \lambda_k.$$

This is the conservation law of PCA. Rotating to the principal-component axes does not create or destroy spread; it only *redistributes* it. The d messy, correlated feature-variances on the diagonal are repackaged into d clean, independent component-variances λ_k – same total, but now sorted and uncorrelated.

The **fraction of variance explained** by component k is therefore

$$\frac{\lambda_k}{\lambda_1 + \cdots + \lambda_d}.$$

In our example, component 1 explains $6.43/7.2 \approx 89.3\%$ of all the spread, and component 2 the remaining $0.77/7.2 \approx 10.7\%$. Nearly nine tenths of everything the five points do is captured by a single direction.

Why this matters

This structural property explains why PCA is so effective in practice. Real high-dimensional datasets are rarely genuinely high-dimensional. A thousand pixels of a face, a hundred survey questions, the daily prices of five hundred stocks – the underlying signal lives on a thin, gently curved, low-dimensional sheet, and most of the ambient dimensions carry almost no variance. PCA measures exactly how thin that sheet is. When the top few eigenvalues hold 95% of the total, you have learned that your 1000-dimensional data is, for all practical purposes, only a handful of dimensions deep. That fact – the *intrinsic dimension* is small – is what makes learning from data possible at all.

30.4.2 Reducing dimensions: 2D down to 1D

Now the payoff. If one direction holds almost all the variance, we can describe each point by its coordinate along that single direction and lose almost nothing. This is **dimensionality reduction**: replace each d-dimensional point by its few top principal-component coordinates.

Concretely, to project our five 2D points down to 1D, give each point its single coordinate along $\mathbf{v}_1$:

$$z_i = \tilde{\mathbf{x}}_i^{\mathsf{T}} \mathbf{v}_1.$$

Using $\mathbf{v}_1 \approx (0.756, 0.655)$ and the centered rows of $\mathbf{B}$:

$$\begin{aligned}
(-3,-2): &\quad z = -3(.756) - 2(.655) = -2.268 - 1.310 = -3.58,\\
(-1,\ 0): &\quad z = -1(.756) + 0 = -0.756,\\
(\ 1,\ 2): &\quad z = \ \ .756 + 1.310 = 2.07,\\
(\ 3,\ 2): &\quad z = 2.268 + 1.310 = 3.58,\\
(\ 0,-2): &\quad z = 0 - 1.310 = -1.31.
\end{aligned}$$

Five 2D points have become five single numbers. To *reconstruct* an approximate point, ride that coordinate back out along $\mathbf{v}_1$ and add the mean back:

$$\hat{\mathbf{x}}_i = \bar{\mathbf{x}} + z_i\, \mathbf{v}_1.$$

Every reconstructed point lands exactly on the principal line through the centroid; the part we threw away was its small wobble in the $\mathbf{v}_2$ direction. That discarded wobble has total size λ_2 – the 10.7% we chose to forget.

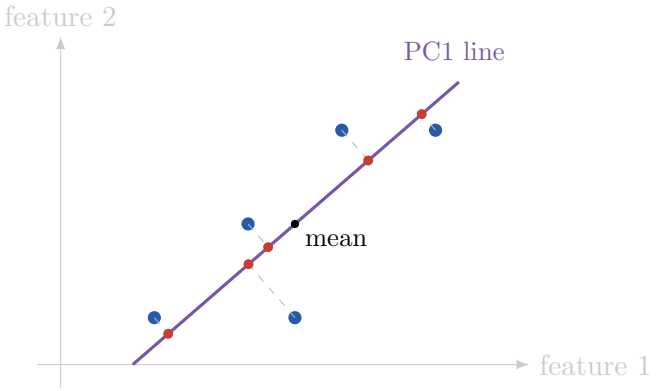


Figure 30.4: Dimensionality reduction from 2D to 1D. Each blue point collapses (dashed) onto its red reconstruction on the PC1 line. The reconstruction error is the dashed length, and PCA chooses the line that makes the *total squared* dashed length as small as possible.

30.4.3 Two faces of the same optimum

We motivated principal components as the directions of *maximum variance*. But Figure 30.4 suggests a second story: PCA is the line (or plane) that sits *closest* to the data, minimizing the total squared distance from points to their projections. These are the same optimum, and seeing why ties the chapter to the projection and least-squares chapters.

Formal

For each centered point, Pythagoras on the right triangle "point, its projection, the origin" reads

$$\underbrace{\|\tilde{\mathbf{x}}_i\|^2}_{\text{fixed}} = \underbrace{(\tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w})^2}_{\text{variance captured}} + \underbrace{\|\tilde{\mathbf{x}}_i - (\tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w})\mathbf{w}\|^2}_{\text{squared error}}.$$

Summing over all points and dividing by n: (total variance) = (variance along $\mathbf{w}$) + (mean squared error). The left side does not depend on $\mathbf{w}$. So **maximizing the variance captured is identical to minimizing the reconstruction error.**

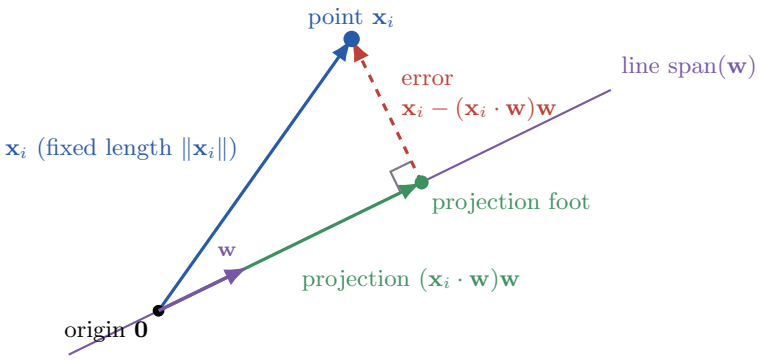


Figure 30.5: The Pythagorean decomposition for a single centered point $\mathbf{x}_i$. The distance to the origin, $\|\mathbf{x}_i\|$, is fixed. The projection along unit vector $\mathbf{w}$ forms the base of a right triangle, and the reconstruction error forms the perpendicular leg. Because base^2 + height^2 = hypotenuse^2, tilting the candidate line $\mathbf{w}$ to maximize projected spread (variance) is identical to minimizing perpendicular reconstruction error.

Before/after Strang

Strang teaches least squares as "project $\mathbf{b}$ onto the column space of $\mathbf{A}$" – find the point in a *given* subspace closest to your data. PCA is the striking inverse problem: the subspace itself is unknown, and you choose it to fit the data best. Least squares asks "how close is the data to this line?"; PCA asks "which line is the data closest to?" Same orthogonality, same Pythagoras, but now the subspace is the answer instead of the question. Reading this after Strang, you will recognize the right triangle of the formal box as the very picture from his projection chapter – only here we get to rotate the line until the error shrinks to its minimum.

30.4.4 How many components? The scree plot

To choose how many components to keep, plot the eigenvalues in decreasing order. This **scree plot** usually drops steeply and then flattens into a "rubble" tail of tiny eigenvalues – noise directions carrying negligible variance. You keep the components before the elbow. Equivalently, keep enough components to reach a target cumulative variance such as 90% or 99%.

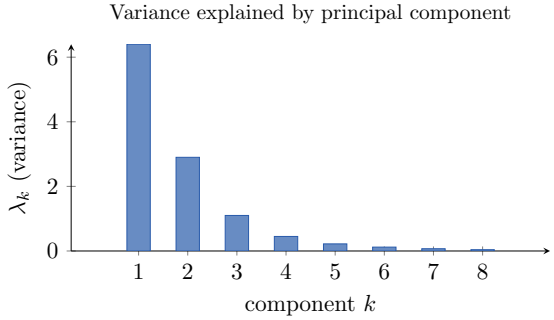


Figure 30.6: A scree plot. The first two or three components tower over a flat tail of noise. The "elbow" after component 3 says: three principal directions describe this dataset, the rest is rubble.

30.5 PCA is the SVD of the centered data

30.5.1 Why a practitioner never forms C

We built PCA on the covariance matrix $\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B}$. That is perfect for understanding, but numerically it is a trap: forming $\mathbf{B}^\mathsf{T}\mathbf{B}$ *squares* the data, and squaring squares the spread between large and small quantities, wrecking precision (a tiny eigenvalue can vanish entirely into rounding error). The professional route computes the principal components straight from $\mathbf{B}$ using the **singular value decomposition** – no covariance matrix ever formed.

Recall the SVD of the centered data: $\mathbf{B} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$, where $\mathbf{U} \in \mathbb{R}^{n\times n}$ and $\mathbf{V} \in \mathbb{R}^{d\times d}$ are orthogonal, and $\mathbf{\Sigma}$ holds the singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ on its diagonal. Now substitute into the covariance matrix:

$$\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B} = \frac{1}{n}(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T})^\mathsf{T}(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}) = \frac{1}{n}\mathbf{V}\mathbf{\Sigma}^\mathsf{T}\mathbf{U}^\mathsf{T}\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T} = \mathbf{V}\Big(\frac{\mathbf{\Sigma}^\mathsf{T}\mathbf{\Sigma}}{n}\Big)\mathbf{V}^\mathsf{T}.$$

The middle factor $\mathbf{\Sigma}^\mathsf{T}\mathbf{\Sigma}/n$ is diagonal with entries σ_k^2/n. But $\mathbf{V}(\cdots)\mathbf{V}^\mathsf{T}$ with a diagonal middle and orthogonal $\mathbf{V}$ is exactly an eigendecomposition. Therefore:

Formal

Let $\mathbf{B} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$ be the SVD of the centered data. Then the principal components and their variances are

$$\boxed{\mathbf{v}_k = k\text{th right singular vector}, \qquad \lambda_k = \frac{\sigma_k^2}{n}.}$$

The **right singular vectors of the centered data are the principal components**, and the **squared singular values (over n) are the variances explained**. The PCA scores – the coordinates of the data along the components – are the columns of $\mathbf{U}\mathbf{\Sigma}$, since $\mathbf{B}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$.

So "do PCA" and "take the SVD of the centered data, read off $\mathbf{V}$ and $\mathbf{\Sigma}$" are the same instruction. Everything from the eigenvalue story survives, just expressed in singular values – and computed without ever squaring the data.

Intuition

The SVD already told you that every matrix acts by *rotate, stretch, rotate*. For the centered data $\mathbf{B}$, the final rotation $\mathbf{V}$ aligns the coordinate axes with the cloud's own axes, and the stretches σ_k say how far the cloud reaches along each. That is the data ellipse of Figure 30.3 all over again: the SVD hands you the ellipse's axes ($\mathbf{V}$) and radii (σ_k) directly, without the detour through covariance. Figure 30.7 traces the three motions in order.

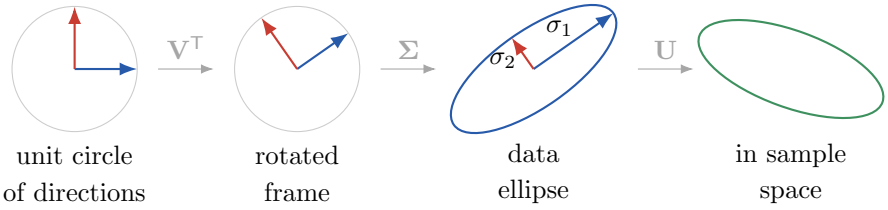


Figure 30.7: PCA is the SVD $\mathbf{B} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\mathsf{T}$ read right to left as *rotate, stretch, rotate*. The rotation $\mathbf{V}$ turns the standard axes into the cloud's own principal axes; $\boldsymbol{\Sigma}$ stretches the unit circle into the data ellipse, reaching σ_k along principal component $\mathbf{v}_k$; the final rotation $\mathbf{U}$ places that ellipse among the samples. The principal directions are the columns of $\mathbf{V}$, and the reach of the data ellipse along each is given by the singular values.

Remark 30.1. The connection runs deeper than convenience. The eigenvectors of $\mathbf{B}^\mathsf{T}\mathbf{B}$ *are* the right singular vectors of $\mathbf{B}$, with eigenvalues σ_k^2 – a fact you proved when you first built the SVD. PCA is simply that theorem applied to centered data and renamed in the language of variance. If the SVD chapter felt abstract, PCA is its concrete reason to exist.

30.5.2 Low-rank approximation is dimensionality reduction

The SVD view gives the cleanest statement of dimensionality reduction. Truncate the SVD to its top r terms,

$$\mathbf{B} \approx \mathbf{B}_r = \sum_{k=1}^{r} \sigma_k \, \mathbf{u}_k \mathbf{v}_k^\mathsf{T},$$

and the Eckart–Young theorem (Theorem 29.2) says $\mathbf{B}_r$ is the *best* rank-r approximation of $\mathbf{B}$ in the least-squares sense – no other rank-r matrix is closer. Reducing your data to its top r principal components is exactly forming this best low-rank approximation. The error you incur is governed by the singular values you dropped:

$$\frac{\|\mathbf{B} - \mathbf{B}_r\|_F^2}{\|\mathbf{B}\|_F^2} = \frac{\sigma_{r+1}^2 + \cdots + \sigma_d^2}{\sigma_1^2 + \cdots + \sigma_d^2} = \frac{\lambda_{r+1} + \cdots + \lambda_d}{\lambda_1 + \cdots + \lambda_d} = 1 - (\text{variance explained by top } r).$$

"Keep 95% of the variance" and "make the best rank-r approximation with 5% squared error" are literally the same sentence.

Pause

Pause and think: More features than samples ($d \gg n$). Suppose you sequence genetic markers for $n = 50$ patients, measuring $d = 20{,}000$ gene expression levels for each. The covariance matrix $\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B}$ has size $20{,}000 \times 20{,}000$. How many nonzero eigenvalues can $\mathbf{C}$ possibly have?

Answer: At most $n - 1 = 49$. Because the n rows of $\mathbf{B}$ are centered, they sum to zero, so $\operatorname{rank}\mathbf{B} \le n - 1 = 49$. By the rank properties from Chapter 16, $\operatorname{rank}(\mathbf{B}^\mathsf{T}\mathbf{B}) = \operatorname{rank}\mathbf{B} \le 49$. The remaining 19,951 eigenvalues are exactly zero. Computing the SVD of the $50 \times 20{,}000$ matrix $\mathbf{B}$ directly finds those 49 non-trivial directions in a fraction of a second, whereas forming and diagonalizing a $20{,}000 \times 20{,}000$ covariance matrix would waste gigabytes of memory on thousands of zeros.

30.6 PCA in the wild

30.6.1 Compressing and denoising images

A grayscale image is a matrix of pixel brightnesses. Take its SVD and keep the top r singular triplets: you reconstruct a recognizable image from a tiny fraction of the numbers. An 800×600 image has 480,000 pixels, but its rank-50 approximation needs only $50\,(800 + 600 + 1) \approx 70{,}000$ numbers – a $7\times$ compression – and often looks nearly identical, because natural images have a few dominant singular values and a long thin tail. Drop the tail and you have not only compressed the image but also *denoised* it: random noise spreads its energy evenly across all singular directions, so the small-σ tail you discard is disproportionately noise.

Compute it yourself

Make a 3×3 image $\mathbf{M}$ with rows $(1, 2, 3), (2, 4, 6), (3, 6, 9)$. Notice every row is a multiple of $(1, 2, 3)$, so $\operatorname{rank} \mathbf{M} = 1$: one singular value is nonzero and the other two are exactly 0. The rank-1 approximation is the image itself, with zero error – this picture compresses perfectly. Now nudge one entry by 0.1 and watch a small second singular value appear: that is the "detail" a rank-1 approximation would round away.

30.6.2 Eigenfaces

In the early 1990s, Turk and Pentland built one of the first working face-recognition systems using PCA. Treat each face photo as a long vector: an $N \times N$ image flattened into a point in $\mathbb{R}^{N^2}$ (a 64×64 face is a single point in 4096-dimensional space). Collect many faces as the rows of a data matrix, center them by subtracting the *average face*, and run PCA.

The top principal components are themselves images – reshape each $\mathbf{v}_k$ back into a 64×64 grid – and they are called **eigenfaces**. The first few capture the broadest ways faces differ: overall lighting, the shape of the hairline, the breadth of the jaw, the set of the eyes. Any face is then approximately the average face plus a short weighted sum of eigenfaces, so a face that once needed 4096 numbers is now described by perhaps 50 coordinates. To recognize a new face, project it into eigenface coordinates and find the nearest stored neighbor. Recognition in a 50-dimensional space, not a 4096-dimensional one – that compression is what made it run on early-1990s hardware.

Modern application

PCA inside modern machine learning. The eigenface concept applies directly across modern representation learning. Today neural networks map words, images, and user features into high-dimensional **embedding** vectors, and those embeddings are routinely analyzed and compressed with PCA. Three live uses:

- **Visualization.** Embeddings live in hundreds or thousands of dimensions. Project onto the top 2 or 3 principal components and you can *plot* them – synonyms cluster, categories separate – a sanity check practitioners run constantly. (Nonlinear cousins like t-SNE and UMAP refine the picture, but PCA is the plain linear first look.)
- **Whitening and preprocessing.** Rotating data into its principal-component axes and rescaling each by $1/\sqrt{\lambda_k}$ produces *whitened* features with identity covariance –

uncorrelated and unit-variance – which makes downstream training faster and better conditioned.

- **Compression at scale.** A retrieval system storing billions of embeddings shrinks each one with PCA before indexing, trading a little accuracy for a large cut in memory and a faster nearest-neighbor search.

The throughline of this book: the linear algebra under the newest models is the same spectral theorem you can verify on a 2×2 matrix by hand.

Common trap

Scale your features first. PCA maximizes numerical variance, and variance depends on units. If one feature is measured in millimeters and another in kilometers, the millimeter column has a much larger numerical variance simply because of the smaller unit, so the first principal component will align almost entirely with that axis. When features carry different units or wildly different scales, *standardize* each to unit variance before running PCA (this replaces the covariance matrix with the *correlation* matrix). Centering removes the location; standardizing removes the arbitrary choice of units.

Walking question

Take a walk and picture a perfectly round cloud of data – equal variance in every direction, all eigenvalues equal. What does PCA return? (Every direction is a tie for "most spread," so the principal components are not unique – any orthonormal basis works.) Now squash the cloud slightly along one axis. Suddenly there is a unique answer. Which way does $\mathbf{v}_1$ swing, and why did a tiny squash break a perfect tie? Notice how sensitive the individual eigenvectors of a near-multiple of the identity are – a first feel for why nearly-equal eigenvalues make PCA directions unstable, even as the subspace they span stays perfectly stable.

30.7 The recipe, start to finish

Focus

PCA in seven steps.

1. Arrange data as $\mathbf{X} \in \mathbb{R}^{n \times d}$: rows are samples, columns features.
2. Center: subtract the column means to get $\mathbf{B}$ (and standardize columns to unit variance if the features have different units).
3. Form the covariance $\mathbf{C} = \frac{1}{n}\mathbf{B}^{\mathsf{T}}\mathbf{B}$ *for understanding*; in practice, take the SVD $\mathbf{B} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}}$ instead.
4. Principal components are the eigenvectors of $\mathbf{C}$ = right singular vectors $\mathbf{v}_k$; variances are $\lambda_k = \sigma_k^2/n$, sorted descending.
5. Choose r from the scree plot or a cumulative-variance target.
6. Reduce: project each centered point onto the top r components, $\mathbf{z}_i = \mathbf{V}_r^{\mathsf{T}}\tilde{\mathbf{x}}_i$.

7. Reconstruct when needed: $\hat{\mathbf{x}}_i = \bar{\mathbf{x}} + \mathbf{V}_r \mathbf{z}_i$, the best rank-$r$ fit to the original.

History and connection

The idea behind principal component analysis was introduced in 1901 by the English mathematician Karl Pearson in a paper titled *On Lines and Planes of Closest Fit to Systems of Points in Space*. Pearson was analyzing biometric data—measurements of living bodies, such as human skull dimensions and limb lengths—and wanted to summarize dozens of correlated variables without throwing away the essential variation. Lacking digital computers or modern matrix algebra, Pearson turned to classical mechanics, modeling the data points as a cloud of equal-mass particles in physical space. He then calculated their principal axes of rotational inertia—the natural directions about which a spinning physical body turns smoothly without wobbling, just as Euler had analyzed for rigid solids a century earlier. Three decades later, the American statistician Harold Hotelling (1933) independently developed the algebraic framework of PCA, defining principal components as the uncorrelated linear combinations of variables that maximize projected variance.

A major controversy in Pearson's day was the distinction between ordinary least-squares regression and orthogonal regression. Standard linear regression treats one variable as special, minimizing vertical distances along an arbitrary dependent axis y. Pearson argued that in observational science and biology, nature does not label one variable as an "input" and another as an "output"; true geometric fitting requires minimizing perpendicular distances, treating every feature symmetrically. Another classic pitfall is the failure to center data: if you forget to subtract the feature means before computing components, PCA points its first axis directly from the origin toward the center of the cloud rather than along the true grain of variance. A second classic trap is scale: because variance depends on units of measurement, measuring one feature in millimeters and another in kilometers causes the big-number feature to hijack the first component entirely, requiring features to be standardized to unit variance before analysis.

PCA gives optimal linear dimensionality reduction, projecting high-dimensional data onto a low-dimensional subspace that retains maximum variance while decorrelating the coordinates. Its limitations are equally clear. It is blind to nonlinear geometry: if data lies on a curved surface—such as the classic "Swiss roll," a two-dimensional sheet curled up into a three-dimensional coil—PCA flattens the roll onto a flat plane, crushing distant points on adjacent layers together. It is also unsupervised, meaning it inspects only the cloud's geometric spread without knowing any class labels or categories; as a result, its directions of greatest variance can fail to separate distinct classes of data. These boundaries spurred the development of nonlinear manifold learning and neural autoencoders (which we explore in Chapter 34), while the underlying optimization of variance via the Rayleigh quotient $\mathbf{w}^{\mathsf{T}}\mathbf{C}\mathbf{w}/\|\mathbf{w}\|^2$ takes center stage in Chapter 36.

Why this matters

We asked a statistical question – which direction maximizes variance – and the answer is an eigenvector of the sample covariance matrix. We asked for the best low-rank summary in mean squared error, and the solution is given by the SVD. Centering, covariance, variance explained, denoising, eigenfaces, and embedding compression are all practical realizations of the spectral theorem and the singular value decomposition applied to empirical data matrices.

Looking ahead: from data axes to change of basis

Throughout this chapter, we worked with data points described in the coordinate system of our measuring tools—centimeters, dollars, pixel values. Those measurement axes are arbitrary, whereas the cloud has its own principal directions. Finding $\mathbf{v}_1, \dots, \mathbf{v}_d$ gives an orthonormal coordinate frame aligned with the spread of the data. When we expressed each point $\mathbf{x}_i$ by its projected coordinates $(z_1, \dots, z_d)$ along those principal axes, all off-diagonal covariances vanished. The variables decoupled completely, and the covariance matrix in the new basis became purely diagonal.

That is our first live encounter with a foundational theme of linear algebra: *a change of basis*. In Chapter 31, we turn from empirical data clouds to general linear transformations. A linear transformation is an intrinsic coordinate-free mapping between spaces, while a matrix representation depends on the chosen basis. Just as PCA finds a basis that diagonalizes the empirical covariance matrix, Chapter 31 develops how to construct coordinate systems that make the matrix representation of an arbitrary linear operator as simple and direct as possible.

Exercises

Exercise 30.1. The data matrix is $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 6 \\ 5 & 4 \\ 7 & 8 \end{bmatrix}$. (a) Find the mean $\bar{\mathbf{x}}$. (b) Write the centered matrix $\mathbf{B}$ and verify both columns sum to zero. (c) Compute the covariance matrix $\mathbf{C} = \frac{1}{4}\mathbf{B}^\mathsf{T}\mathbf{B}$.

Exercise 30.2. For the covariance matrix $\mathbf{C} = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix}$, find both eigenvalues and the (normalized) first principal component. What fraction of the total variance does the first component explain? Check your eigenvalues against $\operatorname{tr}\mathbf{C}$ and $\det\mathbf{C}$.

Exercise 30.3. Continuing Exercise 30.2, suppose a centered point is $\tilde{\mathbf{x}} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$. (a) Find its coordinate along the first principal component. (b) Find its rank-1 reconstruction $\hat{\mathbf{x}} = (\tilde{\mathbf{x}}^\mathsf{T}\mathbf{v}_1)\mathbf{v}_1$. (c) Verify Pythagoras: $\|\tilde{\mathbf{x}}\|^2 = (\tilde{\mathbf{x}}^\mathsf{T}\mathbf{v}_1)^2 + \|\tilde{\mathbf{x}} - \hat{\mathbf{x}}\|^2$.

Exercise 30.4. Show that for a perfectly round cloud the covariance matrix is a multiple of the identity, $\mathbf{C} = s^2\mathbf{I}$. Conclude that *every* unit vector is a principal component with the same variance, so PCA returns no preferred direction. (This is the walking-question cloud made precise.)

Exercise 30.5. Prove that the total variance of a centered dataset, $\sum_j \operatorname{Var}(\text{feature } j)$, equals $\sum_k \lambda_k$, the sum of the eigenvalues of $\mathbf{C}$. (Hint: both equal $\operatorname{tr}\mathbf{C}$; use that the trace is basis-independent, or that $\operatorname{tr}\mathbf{C} = \operatorname{tr}(\mathbf{V}\mathbf{\Lambda}\mathbf{V}^\mathsf{T}) = \operatorname{tr}\mathbf{\Lambda}$.)

Exercise 30.6. The centered data has SVD with singular values $\sigma_1 = 6,\ \sigma_2 = 3,\ \sigma_3 = 2$ and $n = 10$ samples. (a) What are the three variances λ_k? (b) What fraction of variance does a rank-2 reduction retain? (c) What is the relative squared reconstruction error $\|\mathbf{B} - \mathbf{B}_2\|_F^2 / \|\mathbf{B}\|_F^2$?

Exercise 30.7. Prove the maximum-variance characterization directly. Let $\mathbf{C}$ be symmetric positive semidefinite with top eigenvalue λ_1 and unit eigenvector $\mathbf{v}_1$. Using the spectral decomposition $\mathbf{C} = \sum_k \lambda_k \mathbf{v}_k \mathbf{v}_k^\mathsf{T}$ and writing a unit vector as $\mathbf{w} = \sum_k c_k \mathbf{v}_k$, show $\mathbf{w}^\mathsf{T}\mathbf{C}\mathbf{w} = \sum_k \lambda_k c_k^2 \le \lambda_1$, with equality iff $\mathbf{w} = \pm\mathbf{v}_1$ (assuming $\lambda_1 > \lambda_2$).

Exercise 30.8. A dataset has feature 1 in meters (variance 0.4) and feature 2 in centimeters (variance 1600), with covariance 15. (a) Find the first principal component of the raw covariance matrix and argue it points essentially along feature 2. (b) Convert feature 2 to meters and recompute; show the components change completely. Explain in one sentence why standardizing features first is essential here.

Exercise 30.9. Let $\mathbf{B} = \mathbf{a}\mathbf{w}^\mathsf{T}$ be a rank-1 centered data matrix ($\mathbf{a} \in \mathbb{R}^n$ with $\mathbf{1}^\mathsf{T}\mathbf{a} = 0$, and $\mathbf{w} \in \mathbb{R}^d$ a unit vector). (a) Show $\mathbf{C} = \frac{1}{n}\mathbf{B}^\mathsf{T}\mathbf{B}$ has rank 1. (b) Find its single nonzero eigenvalue and the corresponding principal component. (c) Interpret: what does rank-1 centered data look like as a cloud of points?

Exercise 30.10. Prove that the rank-1 PCA reconstructions $\hat{\mathbf{x}}_i = \bar{\mathbf{x}} + (\tilde{\mathbf{x}}_i^\mathsf{T}\mathbf{v}_1)\mathbf{v}_1$ of any dataset all lie on a single line through the mean, and that this line is the total-least-squares best-fit line (minimizing *perpendicular* distances, unlike ordinary regression which minimizes vertical distances). Contrast this with the least-squares regression line of Chapter 19.

Exercise 30.11. Mental checks. Answer each question in a single sentence or quick calculation:

(a) If a dataset has $n = 200$ samples and $d = 4$ features, what are the matrix dimensions of $\mathbf{X}$, the centered matrix $\mathbf{B}$, and the covariance matrix $\mathbf{C}$?

(b) Can the covariance matrix $\mathbf{C}$ ever possess a strictly negative eigenvalue? Why or why not?

(c) If the first two principal components explain 78% and 14% of the total variance respectively, what percentage of the variance is carried by all remaining components together?

(d) Suppose one software package computes the first principal component as $\mathbf{v}_1$, while a second package outputs $-\mathbf{v}_1$ on the same data. Did one of them make a calculation error?

Chapter 31

Linear Transformations and Change of Basis

In the chapters on diagonalization, the SVD, and PCA, our central move was always the same: change coordinates until the action decoupled into independent axes. Those coordinate-change formulas were practical machinery; here is the geometric principle behind them:

> *A linear transformation is a physical motion of space. The matrix is only its shadow in one particular coordinate system. Change the coordinate system and the same motion casts a different matrix.*

This is the distinction between the **operator** (the geometric act of rotating, projecting, or stretching) and its **matrix** (the table of numbers written in a chosen basis). Separating the two brings clarity. Diagonalization, Jordan form, the SVD, and PCA were never isolated tricks – they were all the same geometric sentence: *pick the right basis and the machine becomes obvious.*

This chapter builds the change-of-basis engine from first principles. We define linear maps abstractly, watch how a matrix is born when a basis is chosen, establish the translation rules between different observers, and see why similarity $\mathbf{P}^{-1}\mathbf{AP}$ is simply translating between two points of view. That coordinate-free perspective will prepare us directly for the complex vector spaces and Fourier analysis waiting ahead.

31.1 The Operator and Its Matrix Are Two Different Things

Start with a familiar geometric motion in the plane.

Take a flat sheet of paper and rotate it a quarter turn counterclockwise around its center. That is a transformation. Every point on the paper moves; the move is rigid (lengths and angles are preserved) and it fixes the center. Nothing about this act mentions numbers, axes, or a grid. The rotation is just *a thing that happens to space*. Call it T.

Now you want to *compute* with T. You lay a coordinate grid on the paper, declare two unit arrows $\mathbf{e}_1 = \begin{bmatrix}1\\0\end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix}0\\1\end{bmatrix}$, and ask: where do these two arrows go? The quarter turn sends $\mathbf{e}_1$ to $\begin{bmatrix}0\\1\end{bmatrix}$

and $\mathbf{e}_2$ to $\begin{bmatrix}-1\\0\end{bmatrix}$. Stack those answers as columns and you have the matrix

$$\mathbf{R} = \begin{bmatrix}0 & -1\\1 & 0\end{bmatrix}.$$

That matrix is the rotation *as seen in this grid*. View the experiment in the slanted basis $\mathbf{b}_1 = (1, 1)$, $\mathbf{b}_2 = (1, -1)$, and you will get a different table of numbers for the very same physical quarter turn (Figure 31.1). The rotation did not change. Your description of it did.

Intuition

A linear operator is the intrinsic geometric mapping; its matrix representation depends on the chosen coordinate frame. Change the basis, and the matrix representation changes even though the operator itself remains identical. Linear algebra often seeks a basis in which the matrix representation is as simple as possible—ideally diagonal.

Focus

Two objects, never to be confused:

- A **linear operator / transformation** $T : V \to W$ – a geometric or abstract rule, basis-free, the actual motion.
- Its **matrix** $[T]$ – a grid of numbers that only exists *after* you have chosen a basis for V and one for W.

Keep this principle in mind: the matrix of a transformation always depends on the chosen bases.

History and connection

For two centuries following René Descartes and Pierre de Fermat in the 1630s, geometry was dominated by Cartesian coordinates – assigning numbers (x, y, z) to points so geometric questions could be answered with algebraic equations. But this convenience came with an annoying flaw: equations depended entirely on where the axes happened to be drawn. Rotate your axes by thirty degrees, and a clean equation for an ellipse sprouted messy cross-terms like $3xy$.

In the early nineteenth century, this triggered a passionate dispute. Mathematicians who practiced **synthetic geometry** (the study of geometric figures using direct spatial relationships like intersections and tangents, without numbers) argued that algebra was blinding mathematicians to intrinsic geometric truths. Jean-Victor Poncelet complained that Cartesian formulas turned living geometry into mechanical bookkeeping. On the other side, champions of **analytic geometry** (solving geometric problems by computing with coordinate numbers) such as Arthur Cayley replied that pure spatial intuition could never scale to higher dimensions or systematic algorithms.

The conceptual tools to resolve the conflict arrived in two stages. First, in 1844, Hermann Grassmann published his theory of extension (*Ausdehnungslehre*), introducing vector addition and scaling abstractly without any coordinate axes – a vision Giuseppe Peano finally codified into modern vector-space axioms in 1888. Second, in 1872, twenty-three-year-old Felix Klein announced his famous Erlangen Program: geometry is not the study of coordinates, but the study of **invariants** – geometric quantities that remain unchanged under a given collection

of transformations.
Under this modern vision, coordinates are just a choice of basis. The real mathematical object is the coordinate-free transformation $T : V \to W$. When an observer switches from one coordinate frame to another using an invertible matrix $\mathbf{P}$, the matrix of the operator updates via similarity conjugation: $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$. This formula is simply an interpreter translating the same underlying motion between two languages. Today, this balance remains an active frontier: in **geometric deep learning**, researchers design **equivariant** neural networks – networks whose internal layers respect physical rotations and symmetries directly, without being fooled by arbitrary choices of coordinate frames. Meanwhile, in infinite-dimensional operator theory, the **invariant subspace problem** – asking whether every bounded linear operator on a Hilbert space preserves at least one non-trivial closed subspace – remains one of the most celebrated open questions in mathematics.

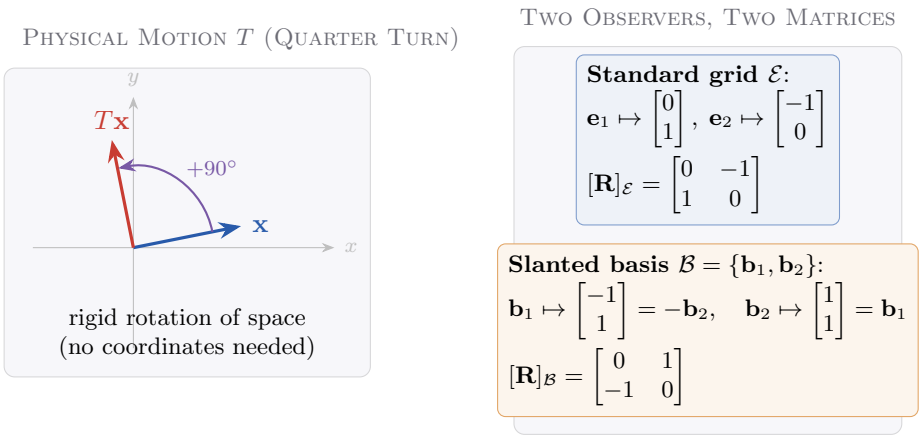


Figure 31.1: The same physical quarter turn produces different matrices in different coordinate systems. In the basis $\mathbf{b}_1 = (1, 1), \mathbf{b}_2 = (1, -1)$ the very same rotation reads $\left[\begin{smallmatrix}0 & 1\\-1 & 0\end{smallmatrix}\right]$ (verified in Exercise 31.3). The operator is fixed; the numbers reflect the coordinate vantage point.

31.2 Linear Maps, Abstractly

Before we can change bases we need a clear statement of what a linear map even is, in a way that does not lean on a grid at all. A **vector space** is simply any collection of objects – arrows in space, sequences of measurements, or polynomials – that you can add together and scale by numbers without ever leaving the collection. You have seen the two core rules of linearity many times; here they are as the whole definition.

Definition 31.1 (Linear transformation)**.** Let V and W be vector spaces over $\mathbb{R}$. A function $T : V \to W$ is a **linear transformation** (or **linear map**, or **linear operator** when $W = V$) if for all $\mathbf{u}, \mathbf{v} \in V$ and all scalars $c \in \mathbb{R}$,

$$T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v}), \qquad T(c\,\mathbf{v}) = c\,T(\mathbf{v}).$$

Equivalently, T preserves every linear combination: $T(a\mathbf{u} + b\mathbf{v}) = a\,T(\mathbf{u}) + b\,T(\mathbf{v})$.

That single "preserves combinations" line is the entire engine of the subject. Suppose $V = \mathbb{R}^n$ and you know what T does to the standard basis vectors $\mathbf{e}_1, \ldots, \mathbf{e}_n$. Then you know what T does to *everything*, because every vector is a combination of basis vectors:

$$\mathbf{x} = x_1\mathbf{e}_1 + \cdots + x_n\mathbf{e}_n \implies T(\mathbf{x}) = x_1\,T(\mathbf{e}_1) + \cdots + x_n\,T(\mathbf{e}_n).$$

A linear map is completely determined by where it sends a basis. This is not a small remark – it is the reason a finite table of numbers (the matrix) can capture a transformation of infinitely many vectors at once.

Pause

Mental check: which of these candidate maps are linear? Before continuing, test the two rules on three simple maps from $\mathbb{R}^2$ to $\mathbb{R}^2$:

(a) $T(x, y) = (2x - y,\ x + 3y)$

(b) $T(x, y) = (x + 1,\ y)$

(c) $T(x, y) = (x^2,\ y)$

Answer: Only (a) is linear. Map (b) fails because $T(0, 0) = (1, 0) \neq (0, 0)$; shifting the origin breaks scalar scaling when $c = 0$. Map (c) fails because squaring is not additive: $(u + v)^2 \neq u^2 + v^2$. A genuine linear map can only combine its input coordinates with constant multipliers.

Why this matters

Linearity leaves T no freedom beyond the basis vectors: once the images of a basis are fixed, the image of every linear combination is determined automatically. That is why knowing T on a handful of vectors pins it down everywhere – and why all of linear algebra reduces to "what happens to a basis."

Example 31.1 (Two famous linear maps that are not matrices yet)**.** Linear maps arise in settings far beyond $\mathbb{R}^n$, which is where the abstract definition shows its power.

Differentiation. Let V be the space of polynomials of degree at most 3. The derivative $D(p) = p'$ is a linear map $V \to V$: $(\,ap + bq\,)' = ap' + bq'$. No grid in sight – yet, as we will see, once we pick the basis $\{1, t, t^2, t^3\}$ it *becomes* a 4×4 matrix.

Rotation. The quarter turn T of Section 31.1 is linear: rotating a sum of arrows is the same as rotating each and adding. The matrix $\mathbf{R}$ was a consequence, not the definition.

Common trap

"Linear" does not mean "a straight-line graph" and it does not mean "$y = mx + b$." The map $x \mapsto x + 1$ is *not* linear: it moves the origin ($T(0) = 1 \neq 0$). Every genuine linear map fixes the origin, $T(\mathbf{0}) = \mathbf{0}$, because $T(\mathbf{0}) = T(0 \cdot \mathbf{0}) = 0 \cdot T(\mathbf{0}) = \mathbf{0}$. Shifts are "affine," a strictly larger and looser family.

31.3 Coordinates: A Vector's Address in a Basis

To turn an abstract map into a matrix we first need to turn an abstract vector into a column of numbers. That is what a basis does: it gives every vector an **address**.

Definition 31.2 (Coordinate vector)**.** Let $\mathcal{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ be a basis of V. Every $\mathbf{v} \in V$ is a *unique* combination $\mathbf{v} = c_1\mathbf{b}_1 + \cdots + c_n\mathbf{b}_n$. The list of weights is the **coordinate vector of v relative to** $\mathcal{B}$:

$$[\mathbf{v}]_{\mathcal{B}} = \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

Uniqueness is what makes coordinates well defined: spanning guarantees that weights exist, and linear independence guarantees that only one tuple of weights works. A coordinate vector is not the geometric vector itself, but its description *relative to the chosen basis* $\mathcal{B}$.

Example 31.2 (The same arrow, two addresses)**.** Take the geometric arrow $\mathbf{v}$ pointing to the point $(4, 3)$ in the plane. In the standard basis $\mathcal{E} = \{\mathbf{e}_1, \mathbf{e}_2\}$ its address is the boring one,

$$[\mathbf{v}]_{\mathcal{E}} = \begin{bmatrix} 4 \\ 3 \end{bmatrix},$$

which is why the standard basis feels invisible: the address equals the arrow. Now switch to the slanted basis

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{b}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

We need c_1, c_2 with $c_1\mathbf{b}_1 + c_2\mathbf{b}_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$, i.e.

$$c_1 + c_2 = 4, \qquad c_1 - c_2 = 3.$$

Adding gives $2c_1 = 7$, so $c_1 = \frac{7}{2}$ and $c_2 = \frac{1}{2}$. The same arrow now has the address

$$[\mathbf{v}]_{\mathcal{B}} = \begin{bmatrix} 7/2 \\ 1/2 \end{bmatrix}.$$

One arrow, two completely different lists of numbers. The arrow did not move; we just described it to two different people who had agreed on different axes.

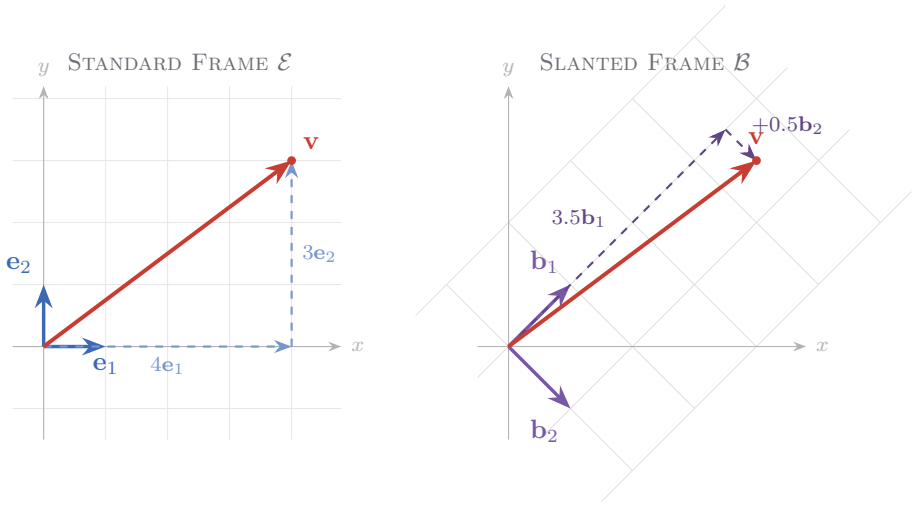


Figure 31.2: One arrow, two coordinate languages. The physical vector $\mathbf{v}$ is identical in both panels. In the standard grid $\mathcal{E}$ (left), four steps along $\mathbf{e}_1$ and three steps along $\mathbf{e}_2$ reach $(4, 3)$. In the rotated and scaled grid $\mathcal{B}$ (right), 3.5 diagonal strides along $\mathbf{b}_1$ plus 0.5 strides along $\mathbf{b}_2$ land on the exact same point: $[\mathbf{v}]_{\mathcal{B}} = (7/2,\ 1/2)$.

Focus

Coordinates are addresses, and addresses change when the basis changes. The moment you write $\begin{bmatrix} 4 \\ 3 \end{bmatrix}$ you have silently agreed on a basis. Most of the confusion in this chapter dissolves once you ask, every single time: *a column of numbers – in which basis?*

31.4 The Matrix of a Transformation in Given Bases

Now we can assemble the central object. We have an abstract linear map $T : V \to W$. We choose a basis $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ for the input space V and a basis $\mathcal{C} = \{\mathbf{c}_1, \ldots, \mathbf{c}_m\}$ for the output space W. We want a matrix that does the same job on *addresses* that T does on *vectors*.

The recipe is the one we glimpsed for rotations, now stated in full generality.

Formal

Matrix of a linear map. Let $T : V \to W$ be linear, with input basis $\mathcal{B}$ and output basis $\mathcal{C}$. For each input basis vector $\mathbf{b}_j$, compute $T(\mathbf{b}_j)$ and write its coordinates in $\mathcal{C}$. Those coordinate columns, side by side, form the matrix

$$[T]_{\mathcal{C}\leftarrow\mathcal{B}} = \Big[\ [T\mathbf{b}_1]_{\mathcal{C}} \ \Big|\ [T\mathbf{b}_2]_{\mathcal{C}} \ \Big|\ \cdots \ \Big|\ [T\mathbf{b}_n]_{\mathcal{C}}\ \Big].$$

It is the unique $m \times n$ matrix satisfying, for every $\mathbf{v} \in V$,

$$[\,T\mathbf{v}\,]_{\mathcal{C}} \ = \ [T]_{\mathcal{C}\leftarrow\mathcal{B}}\,[\mathbf{v}]_{\mathcal{B}}.$$

Read the subscript "$\mathcal{C} \leftarrow \mathcal{B}$" out loud as "out of $\mathcal{B}$, into $\mathcal{C}$." The arrow points the way the data

flows: feed in a $\mathcal{B}$-address on the right, receive a $\mathcal{C}$-address on the left. That notation will save you from a dozen sign-and-direction errors later.

Why does the recipe work? Pure linearity. Take any $\mathbf{v}$ with $\mathcal{B}$-address $\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$, so $\mathbf{v} = \sum_j x_j \mathbf{b}_j$. Apply T and use linearity:

$$T\mathbf{v} = \sum_j x_j \, T\mathbf{b}_j.$$

Now take $\mathcal{C}$-coordinates of both sides. Coordinates are themselves linear (Definition 31.2 respects sums and scalars), so

$$[T\mathbf{v}]_{\mathcal{C}} = \sum_j x_j \, [T\mathbf{b}_j]_{\mathcal{C}} = \Big[\, [T\mathbf{b}_1]_{\mathcal{C}} \,\Big|\, \cdots \,\Big|\, [T\mathbf{b}_n]_{\mathcal{C}} \,\Big] \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}.$$

The bracketed thing is exactly the matrix above, and the column on the right is $[\mathbf{v}]_{\mathcal{B}}$. Done. The columns of a transformation matrix are always "where the input basis vectors go, written in the output basis."

Pause

Mental check: predicting the matrix size. If V has dimension 4 and W has dimension 3, what are the dimensions of $[T]_{\mathcal{C}\leftarrow\mathcal{B}}$? *Answer:* 3×4. The matrix has as many columns as input basis vectors ($n = 4$), because we record the destination of each input basis element. It has as many rows as output basis vectors ($m = 3$), because each destination is written as a list of 3 coordinates in $\mathcal{C}$. Rows = output dimension, columns = input dimension.

Intuition

The standard 2×2 trick you already know – "the columns of $\mathbf{A}$ are $\mathbf{A}\mathbf{e}_1$ and $\mathbf{A}\mathbf{e}_2$" – is just this formula with $\mathcal{B} = \mathcal{C} =$ the standard basis. The general version says: it was never about $\mathbf{e}_1, \mathbf{e}_2$ specifically. It is about *whatever basis you chose*. Feed the machine the basis vectors, record where they land in your output language, stack the answers.

Example 31.3 (Differentiation becomes a matrix)**.** Let V be polynomials of degree ≤ 3 with basis $\mathcal{B} = \{1, t, t^2, t^3\}$, and let $T = D$ be the derivative, which we already know is linear (Example 31.1). Apply D to each basis vector and express the result in the same basis $\mathcal{B}$:

$$D(1) = 0, \quad D(t) = 1, \quad D(t^2) = 2t, \quad D(t^3) = 3t^2.$$

Their $\mathcal{B}$-coordinate columns are $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 3 \\ 0 \end{bmatrix}$, so

$$[D]_{\mathcal{B}\leftarrow\mathcal{B}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Sanity check on $p(t) = 5 + 2t + 7t^2 - t^3$, whose address is $[\,p\,]_{\mathcal{B}} = \begin{bmatrix} 5 \\ 2 \\ 7 \\ -1 \end{bmatrix}$:

$$[D]_{\mathcal{B}\leftarrow\mathcal{B}} \begin{bmatrix} 5 \\ 2 \\ 7 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 14 \\ -3 \\ 0 \end{bmatrix},$$

which decodes to $2 + 14t - 3t^2$. And indeed $p'(t) = 2 + 14t - 3t^2$. Calculus, done by matrix multiplication. The abstract definition was not decoration – it let us treat derivatives with the same machinery as rotations.

Connection to other math

Treating differentiation as a matrix (or, for infinite-dimensional function spaces, as a *linear operator*) is the founding move of **functional analysis** and of every numerical method for differential equations. "Discretize the derivative" means "replace the operator D by a finite matrix in some basis," which is precisely Example 31.3. Eigenvalues of D-like operators are where vibration frequencies, quantum energy levels, and the heat equation's decay rates come from.

31.4.1 A non-square example: bases on input and output

When V and W differ, the two bases really are independent choices and the "out of $\mathcal{B}$, into $\mathcal{C}$" bookkeeping becomes essential.

Example 31.4 (A map from the plane to 3-space)**.** Define $T : \mathbb{R}^2 \to \mathbb{R}^3$ by $T\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x+y \\ 2x \\ x-y \end{bmatrix}$. Take the standard input basis $\mathcal{B} = \{\mathbf{e}_1, \mathbf{e}_2\}$ but a *nonstandard* output basis

$$\mathcal{C} = \left\{ \mathbf{c}_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix},\ \mathbf{c}_2 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix},\ \mathbf{c}_3 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

First find where the input basis goes (Figure 31.3): $T\mathbf{e}_1 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$, $T\mathbf{e}_2 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$. Now write each in $\mathcal{C}$-coordinates. For $T\mathbf{e}_1 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ we need $a\mathbf{c}_1 + b\mathbf{c}_2 + c\mathbf{c}_3 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$:

$$a + b = 1, \quad c = 2, \quad a - b = 1 \ \Rightarrow\ a = 1,\ b = 0,\ c = 2,$$

so $[T\mathbf{e}_1]_{\mathcal{C}} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$. For $T\mathbf{e}_2 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \mathbf{c}_2$ directly, so $[T\mathbf{e}_2]_{\mathcal{C}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$. Hence

$$[T]_{\mathcal{C}\leftarrow\mathcal{B}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 0 \end{bmatrix}.$$

Same map T; had we used the standard output basis we would have written the columns $\begin{bmatrix}1\\2\\1\end{bmatrix}, \begin{bmatrix}1\\0\\-1\end{bmatrix}$ instead. The clever output basis revealed structure the standard one hid.

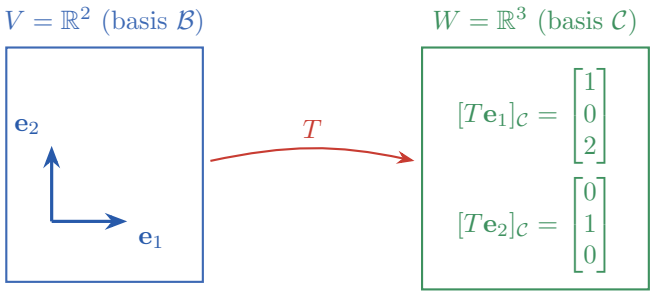


Figure 31.3: A non-square map carries the input-basis vectors $\mathbf{e}_1, \mathbf{e}_2$ of $V = \mathbb{R}^2$ into $W = \mathbb{R}^3$, where we record each landing spot in the *output* basis $\mathcal{C}$. Those two coordinate columns are the columns of $[T]_{\mathcal{C}\leftarrow\mathcal{B}}$, which is therefore 3×2: as many rows as $\dim W$, as many columns as $\dim V$.

31.5 Change of Basis: Translating Between Two Address Systems

So far we changed the matrix of a *map*. Now for the simpler-looking but foundational task: changing the address of a fixed *vector*. If two people have agreed on two different bases, how do we translate one's address into the other's? The answer is a single matrix, and almost everything else in the chapter is built from it.

Suppose $\mathcal{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ and $\mathcal{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_n\}$ are two bases of the same space V. We want a matrix $\mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}}$ that converts a $\mathcal{B}$-address into a $\mathcal{C}$-address:

$$[\mathbf{v}]_{\mathcal{C}} = \mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}}\,[\mathbf{v}]_{\mathcal{B}}.$$

This is just the matrix of the *identity map* $\mathrm{id} : V \to V$ read out of $\mathcal{B}$ and into $\mathcal{C}$, so the recipe of Section 31.4 applies verbatim: **its columns are the $\mathcal{B}$ vectors written in $\mathcal{C}$-coordinates.**

$$\mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}} = \Big[\,[\mathbf{b}_1]_{\mathcal{C}} \,\Big|\, [\mathbf{b}_2]_{\mathcal{C}} \,\Big|\, \cdots \,\Big|\, [\mathbf{b}_n]_{\mathcal{C}}\,\Big].$$

Focus

The change-of-basis matrix *into* a basis is built from the *other* basis's vectors expressed in that basis. The arrow "$\mathcal{C} \leftarrow \mathcal{B}$" records the direction: it takes $\mathcal{B}$-coordinates as input and returns $\mathcal{C}$-coordinates as output. Going the other way is the inverse: $\mathbf{P}_{\mathcal{B}\leftarrow\mathcal{C}} = \big(\mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}}\big)^{-1}$.

There is one case so common it deserves its own name. When $\mathcal{C} = \mathcal{E}$ is the standard basis of $\mathbb{R}^n$, the coordinates of $\mathbf{b}_j$ *are just* $\mathbf{b}_j$ *itself*. So the matrix that converts a $\mathcal{B}$-address into a standard address is simply the basis vectors stacked as columns:

$$\mathbf{B} = \mathbf{P}_{\mathcal{E}\leftarrow\mathcal{B}} = \big[\,\mathbf{b}_1 \,\big|\, \mathbf{b}_2 \,\big|\, \cdots \,\big|\, \mathbf{b}_n\,\big].$$

We will call this the **basis matrix $\mathbf{B}$**. It "decodes" a $\mathcal{B}$-address back into ordinary coordinates; its inverse $\mathbf{B}^{-1}$ "encodes" ordinary coordinates into the $\mathcal{B}$ language.

Common trap

The basis matrix $\mathbf{B}$ (columns = your fancy basis vectors) goes *from* fancy coordinates *to* standard ones, not the other way. People constantly write $\mathbf{B}$ when they need $\mathbf{B}^{-1}$. A reliable check: $\mathbf{B}$ applied to $\begin{bmatrix}1\\0\\\vdots\end{bmatrix}$ must return $\mathbf{b}_1$ – and it does, because that is the first column. If your translation sends the first coordinate axis to $\mathbf{b}_1$, you have the "decode" matrix $\mathbf{B}$; if it sends $\mathbf{b}_1$ to an axis, you have $\mathbf{B}^{-1}$.

Example 31.5 (Translating addresses by hand)**.** Reuse the slanted basis $\mathbf{b}_1 = \begin{bmatrix}1\\1\end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix}1\\-1\end{bmatrix}$ from Example 31.2. Stack them:

$$\mathbf{B} = \begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}, \qquad \mathbf{B}^{-1} = \frac{1}{-2}\begin{bmatrix}-1 & -1\\-1 & 1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}.$$

To get the $\mathcal{B}$-address of $\mathbf{v} = \begin{bmatrix}4\\3\end{bmatrix}$ we *encode* with $\mathbf{B}^{-1}$:

$$[\mathbf{v}]_{\mathcal{B}} = \mathbf{B}^{-1}\begin{bmatrix}4\\3\end{bmatrix} = \frac{1}{2}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}\begin{bmatrix}4\\3\end{bmatrix} = \frac{1}{2}\begin{bmatrix}7\\1\end{bmatrix} = \begin{bmatrix}7/2\\1/2\end{bmatrix},$$

matching the answer we found by solving equations earlier. And to go back, the basis matrix *decodes*:

$$\mathbf{B}\begin{bmatrix}7/2\\1/2\end{bmatrix} = \begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}\begin{bmatrix}7/2\\1/2\end{bmatrix} = \begin{bmatrix}4\\3\end{bmatrix} = \mathbf{v}. \checkmark$$

Encode with $\mathbf{B}^{-1}$, decode with $\mathbf{B}$. Two matrices, exact inverses, that carry a single arrow between two languages.

Compute it yourself

Do this one fully by hand on paper. Let $\mathbf{b}_1 = \begin{bmatrix}2\\1\end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix}1\\3\end{bmatrix}$, and $\mathbf{v} = \begin{bmatrix}5\\5\end{bmatrix}$. (a) Write the basis matrix $\mathbf{B}$. (b) Compute $\mathbf{B}^{-1}$ using the 2×2 rule $\frac{1}{ad-bc}\begin{bmatrix}d & -b\\-c & a\end{bmatrix}$. (c) Find $[\mathbf{v}]_{\mathcal{B}} = \mathbf{B}^{-1}\mathbf{v}$. (d) Verify by checking that your two weights times $\mathbf{b}_1, \mathbf{b}_2$ rebuild $\begin{bmatrix}5\\5\end{bmatrix}$. You should get weights $\begin{bmatrix}2\\1\end{bmatrix}$.

31.5.1 The two pictures of a basis matrix

There is a subtlety here that trips up nearly everyone. The matrix $\mathbf{B}$ with columns $\mathbf{b}_1, \mathbf{b}_2$ has *two* jobs that look opposite but are the same equation read two ways.

As a change of coordinates, $\mathbf{B}$ takes a list of weights and decodes it: $\mathbf{B}\begin{bmatrix}c_1\\c_2\end{bmatrix} = c_1\mathbf{b}_1 + c_2\mathbf{b}_2$. Same numbers, new meaning.

As an active transformation, the very same $\mathbf{B}$ is a machine that *moves* the standard grid to the slanted grid: it sends $\mathbf{e}_1 \mapsto \mathbf{b}_1$ and $\mathbf{e}_2 \mapsto \mathbf{b}_2$, dragging the unit square into the parallelogram spanned by the basis. Whether you call $\mathbf{B}$ "passive" (relabeling the same point) or "active" (moving the point) is a matter of story, not of arithmetic – the formula $\mathbf{Bx}$ is identical. Hold both pictures; switch between them deliberately.

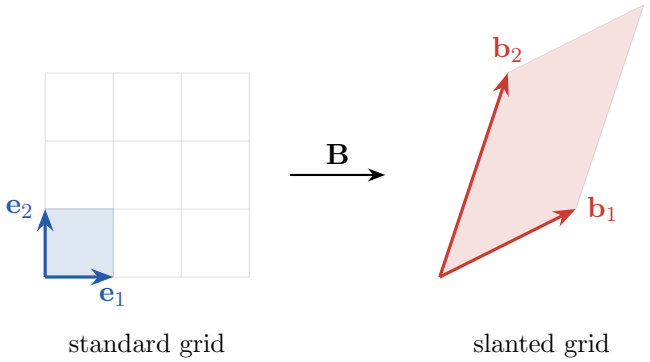


Figure 31.4: One matrix, two stories. $\mathbf{B}$ *actively* carries the unit square to the parallelogram on $\mathbf{b}_1, \mathbf{b}_2$; *passively* it decodes $\mathcal{B}$-addresses into standard ones. Same arithmetic, different narration.

31.6 The Same Operator in Different Coordinates

Now we combine the two threads – the matrix of a map, and translating addresses – into the formula this whole chapter has been walking toward. Picture an operator $T : \mathbb{R}^n \to \mathbb{R}^n$. In the standard basis it has the familiar matrix $\mathbf{A}$, so $[T\mathbf{v}]_{\mathcal{E}} = \mathbf{A}[\mathbf{v}]_{\mathcal{E}}$. A friend insists on working in a basis $\mathcal{B}$ (basis matrix $\mathbf{B}$). What does T look like to them? We want the matrix $\mathbf{A}'$ with

$$[T\mathbf{v}]_{\mathcal{B}} = \mathbf{A}'\,[\mathbf{v}]_{\mathcal{B}}.$$

Trace the action across coordinate systems to assemble the transformation matrix. Start with a $\mathcal{B}$-address. We do not know how T acts on $\mathcal{B}$-addresses directly – but we know how it acts on standard ones. So:

1. **Decode** the $\mathcal{B}$-address into standard coordinates with $\mathbf{B}$.
2. **Apply** T in standard coordinates with $\mathbf{A}$.
3. **Encode** the result back into $\mathcal{B}$-coordinates with $\mathbf{B}^{-1}$.

Composing the three steps (right to left, the way matrices act):

$$\boxed{\mathbf{A}' = \mathbf{B}^{-1}\,\mathbf{A}\,\mathbf{B}.}$$

Intuition

Read $\mathbf{B}^{-1}\mathbf{AB}$ as a three-step sequence: *translate into the standard language* ($\mathbf{B}$), *apply the transformation* ($\mathbf{A}$), and *translate back into the new language* ($\mathbf{B}^{-1}$). The two factors of $\mathbf{B}$ form a round trip into and out of standard coordinates where the action is known. The

operator T remains unchanged; the change-of-basis matrices convert coordinates on the way in and out.

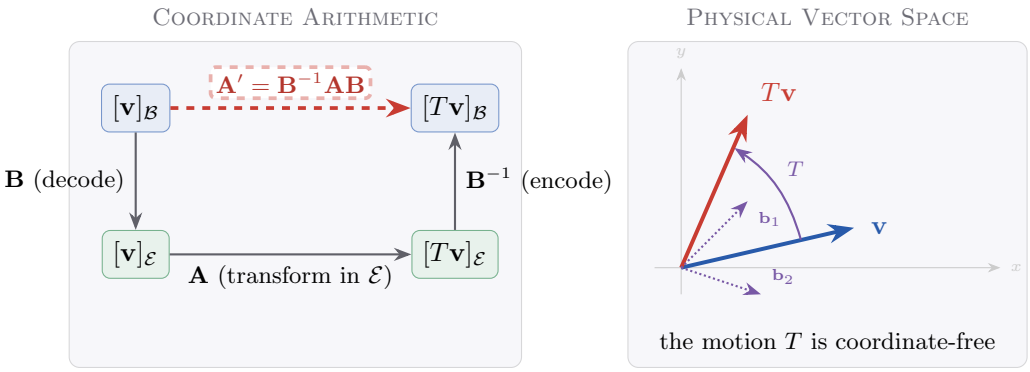


Figure 31.5: The change-of-basis commutative diagram. Left: Computing the transformed address in the new basis $[T\mathbf{v}]_{\mathcal{B}}$ can be accomplished directly by $\mathbf{A}' = \mathbf{B}^{-1}\mathbf{AB}$ (top dashed corridor), or equivalently by translating to standard coordinates with $\mathbf{B}$, executing the transformation $\mathbf{A}$, and encoding back with $\mathbf{B}^{-1}$. Right: The physical motion of space T carrying vector $\mathbf{v}$ to $T\mathbf{v}$ is completely independent of the observer's coordinate system.

Common trap

Is it $\mathbf{B}^{-1}\mathbf{AB}$ or $\mathbf{BAB}^{-1}$? Both appear in textbooks because they answer different questions, and getting them backwards is the single most common change-of-basis error. The rule that never fails: the matrix *closest to the input vector on the right* must *accept that input's coordinate type*. Our input is a $\mathcal{B}$-address, and $\mathbf{B}$ (decode) accepts $\mathcal{B}$-addresses – so $\mathbf{B}$ sits on the right. If instead you were converting a standard-coordinate matrix *from* the $\mathcal{B}$ description, the inverse would land on the right. Always sanity-check by asking what kind of address enters on the far right.

Example 31.6 (A shear in a different basis)**.** Let T have standard matrix

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}.$$

In the standard grid this is an ugly shear-plus-scale. But suppose we view it in the basis $\mathbf{b}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ (basis matrix $\mathbf{B}$ from Example 31.5). Compute $\mathbf{A}' = \mathbf{B}^{-1}\mathbf{AB}$ in stages.

First $\mathbf{AB}$:

$$\begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 2 & -2 \end{bmatrix}.$$

Then $\mathbf{B}^{-1}(\mathbf{AB})$ with $\mathbf{B}^{-1} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$:

$$\mathbf{A}' = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 3 & 1 \\ 2 & -2 \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 5 & -1 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 5/2 & -1/2 \\ 1/2 & 3/2 \end{bmatrix}.$$

This particular basis did *not* simplify $\mathbf{A}$ – the new matrix is no nicer. That is the real lesson: most bases make things worse. The art is finding the special basis. For diagonalizable matrices, the natural choice is the eigenvector basis, which we turn to next.

31.7 Similarity, Revisited

The boxed formula $\mathbf{A}' = \mathbf{B}^{-1}\mathbf{A}\mathbf{B}$ is exactly the relation called **similarity**, which we met back in Chapter 28 (Section 28.1) when studying the Jordan form. Here you see its geometric soul.

Definition 31.3 (Similar matrices)**.** Two $n \times n$ matrices $\mathbf{A}$ and $\mathbf{A}'$ are **similar** if there is an invertible matrix $\mathbf{P}$ with $\mathbf{A}' = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$.

Why this matters

Similarity is the statement that $\mathbf{A}$ and $\mathbf{A}'$ represent the same linear operator expressed in different coordinate frames. The matrix $\mathbf{P}$ converts coordinates between the two systems: $\mathbf{A}' = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$. This geometric identification explains why algebraic invariants such as trace, determinant, and eigenvalues remain identical across all similar matrices.

Because similar matrices are the same operator, they must share everything that is truly a property of the operator and not of the chosen grid. Those shared quantities are the **similarity invariants**, and they are precisely the operator's coordinate-free features.

Theorem 31.1 (Similar matrices share their invariants)**.** *If $\mathbf{A}' = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$, then $\mathbf{A}$ and $\mathbf{A}'$ have the same determinant, the same trace, the same characteristic polynomial, and hence the same eigenvalues (with multiplicities) and the same rank.*

Proof. For the determinant, use multiplicativity: $\det(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \det(\mathbf{P}^{-1})\det(\mathbf{A})\det(\mathbf{P}) = \det(\mathbf{A})$, since $\det(\mathbf{P}^{-1})\det(\mathbf{P}) = \det(\mathbf{P}^{-1}\mathbf{P}) = \det \mathbf{I} = 1$. For the characteristic polynomial, the same cancellation works with $\mathbf{A} - \lambda\mathbf{I}$:

$$\mathbf{A}' - \lambda\mathbf{I} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P} - \lambda\mathbf{P}^{-1}\mathbf{P} = \mathbf{P}^{-1}(\mathbf{A} - \lambda\mathbf{I})\mathbf{P},$$

so $\det(\mathbf{A}' - \lambda\mathbf{I}) = \det(\mathbf{A} - \lambda\mathbf{I})$ for all λ. Equal characteristic polynomials force equal eigenvalues with multiplicities. The trace is (up to sign) a coefficient of that polynomial – it equals the sum of eigenvalues – so it is shared too; equivalently use the cyclic property $\operatorname{tr}(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \operatorname{tr}(\mathbf{A}\mathbf{P}\mathbf{P}^{-1}) = \operatorname{tr}(\mathbf{A})$. Rank is preserved because multiplying by the invertible $\mathbf{P}$ and $\mathbf{P}^{-1}$ cannot change the dimension of the image. □

Pause

Mental check: can similarity change the determinant? Suppose a classmate claims that by choosing a clever basis $\mathcal{B}$, they turned a matrix $\mathbf{A}$ with $\det \mathbf{A} = 5$ into a similar matrix $\mathbf{A}'$ with $\det \mathbf{A}' = 1$. Could this claim possibly be true? *Answer:* Never. Similarity preserves the determinant: $\det(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \det \mathbf{A}$. Geometrically, the determinant measures the volume-scaling factor of the physical motion T. No change of vantage point can turn an operator that expands volume by a factor of 5 into one that preserves volume.

Before/after Strang

Strang introduces similarity right alongside diagonalization, often through the formula $\mathbf{A} = \mathbf{S}\Lambda\mathbf{S}^{-1}$, and emphasizes that "$\mathbf{A}$ and Λ have the same eigenvalues." Reading this chapter *before* Strang, hold onto the why: eigenvalues are shared because $\mathbf{A}$ and Λ are the literally-same operator in two coordinate systems, and eigenvalues belong to the operator, not the grid. Reading it *after* Strang, notice that his $\mathbf{S}$ is our basis matrix $\mathbf{B}$ – columns are eigenvectors – and his $\mathbf{S}^{-1}\mathbf{A}\mathbf{S} = \Lambda$ is our $\mathbf{B}^{-1}\mathbf{A}\mathbf{B} = \mathbf{A}'$ with the luckiest possible basis.

Common trap

Equal eigenvalues do *not* imply similar. The two matrices $\left[\begin{smallmatrix}1 & 1\\ 0 & 1\end{smallmatrix}\right]$ and $\left[\begin{smallmatrix}1 & 0\\ 0 & 1\end{smallmatrix}\right]$ both have eigenvalue 1 twice, yet they are not similar: the second is the identity, and $\mathbf{P}^{-1}\mathbf{I}\mathbf{P} = \mathbf{I}$ for every $\mathbf{P}$, so the identity is similar only to itself. Sharing invariants is necessary, not sufficient. (The full bookkeeping of what *else* is needed is the Jordan-form story.)

31.8 Choosing the Right Basis: The Natural Language of an Operator

We have built the machine: any basis $\mathcal{B}$ turns operator T into the matrix $\mathbf{B}^{-1}\mathbf{A}\mathbf{B}$. Most choices, as Example 31.6 showed, make the matrix uglier. The entire payoff of the theory is that for special operators there is a basis that makes the matrix as simple as a matrix can be – and the columns of that basis matrix $\mathbf{B}$ are the eigenvectors or singular vectors studied across this text.

31.8.1 Eigenvectors: the basis that diagonalizes

Suppose $\mathbf{A}$ has n independent eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ with eigenvalues $\lambda_1, \dots, \lambda_n$. *Use them as your basis.* Then in the eigenvector basis the operator does the simplest thing imaginable: along each axis $\mathbf{v}_j$ it just scales by λ_j and does nothing else. The matrix becomes diagonal. To see it from our recipe, build $\mathbf{B} = [\mathbf{v}_1|\cdots|\mathbf{v}_n]$ and recall $\mathbf{A}\mathbf{v}_j = \lambda_j\mathbf{v}_j$, so

$$\mathbf{A}\mathbf{B} = \big[\lambda_1\mathbf{v}_1 \,\big|\, \cdots \,\big|\, \lambda_n\mathbf{v}_n\big] = \mathbf{B}\,\mathrm{diag}(\lambda_1, \dots, \lambda_n),$$

and multiplying on the left by $\mathbf{B}^{-1}$ gives the headline result

$$\mathbf{B}^{-1}\mathbf{A}\mathbf{B} = \mathrm{diag}(\lambda_1, \dots, \lambda_n) = \Lambda.$$

This is diagonalization, and now you see it for what it is: *not* a trick, but the statement that in the eigenvector coordinate system the operator is a pure, axis-aligned stretch (Figure 31.6).

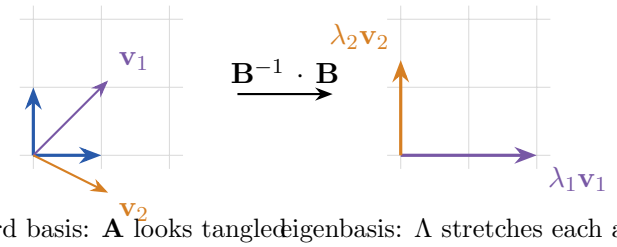


Figure 31.6: Diagonalization as a change of viewpoint. In the standard grid the eigen-directions $\mathbf{v}_1, \mathbf{v}_2$ sit at awkward angles and $\mathbf{A}$ mixes the coordinates. Rotate your axes to *be* the eigenvectors and the same operator becomes $\Lambda = \mathrm{diag}(\lambda_1, \lambda_2)$: each axis is simply scaled, nothing is mixed.

Example 31.7 (Finishing the shear example with the right basis)**.** Reconsider $\mathbf{A} = \left[\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right]$. Its only eigenvalue is 2 (repeated), with eigenvectors solving $(\mathbf{A} - 2\mathbf{I})\mathbf{v} = \mathbf{0}$, i.e. $\left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]\mathbf{v} = \mathbf{0}$, giving the single direction $\mathbf{v} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. There is only *one* independent eigenvector, not two – so no basis can diagonalize this $\mathbf{A}$. It is a **defective** matrix, and the closest we can get is the Jordan block, which is $\mathbf{A}$ itself. This is the true boundary of diagonalization, and it is exactly why the Jordan form and (for rectangular or non-diagonalizable situations) the SVD exist.

Contrast with the symmetric matrix $\mathbf{A} = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$. Its characteristic polynomial is $(2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1)$, so the eigenvalues are 3 and 1, with eigenvectors $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Two distinct eigenvalues force two independent eigenvectors, so the basis of those eigenvectors makes it the clean diagonal $\mathrm{diag}(3, 1)$. Symmetry is the friendliest case of all – it always yields a *full, orthogonal* eigenbasis, the content of the spectral theorem.

31.8.2 Singular vectors: the basis that works for every matrix

An eigenvector basis does not always exist: it requires a square matrix, and even then a defective matrix comes up short (Example 31.7). The singular value decomposition fixes this by allowing *two* bases, one for the input space and one for the output, exactly the freedom we built in Section 31.4. The SVD $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\mathsf{T}$ says: choose the orthonormal basis $\mathcal{V}$ (columns of $\mathbf{V}$) on the input side and $\mathcal{U}$ (columns of $\mathbf{U}$) on the output side, and in *those* coordinates T is the diagonal matrix Σ – it sends each input axis $\mathbf{v}_j$ straight to a stretched output axis $\sigma_j \mathbf{u}_j$. It is the change-of-basis idea pushed to its most general and most useful form: pick the best input basis and the best output basis *independently*, and *every* linear map becomes a simple nonnegative diagonal stretch (Figure 31.7).

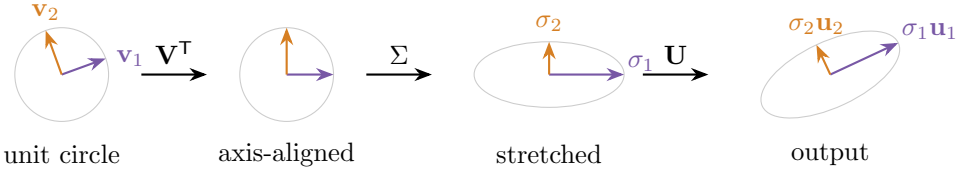


Figure 31.7: The SVD $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\mathsf{T}$ as rotate–stretch–rotate. $\mathbf{V}^\mathsf{T}$ turns the right singular vectors onto the axes, Σ stretches each axis by a singular value $\sigma_j \geq 0$ (turning the circle into an ellipse), and $\mathbf{U}$ rotates that ellipse into the output frame. In the $\mathcal{V}$,$\mathcal{U}$ bases the map is just the diagonal Σ.

Focus

The unifying slogan of the second half of the book:

Diagonalization, the spectral theorem, and the SVD are all the same move – change to the basis in which the operator is diagonal.

Diagonalization uses one eigenvector basis (square, diagonalizable case). The spectral theorem uses one *orthonormal* eigenvector basis (symmetric case). The SVD uses two orthonormal bases (every matrix, no exceptions). Each is a smarter choice of $\mathbf{B}$ in $\mathbf{B}^{-1}\mathbf{AB}$ – or of $\mathbf{U}, \mathbf{V}$ when the two sides differ.

Modern application

Why data scientists obsess over the right basis. A dataset is a cloud of points; its raw features ("height in cm," "income in dollars") form an arbitrary basis, usually correlated and redundant. In the previous chapter, we built **Principal Component Analysis** to solve this exact issue. In the language of this chapter, PCA is a change of basis: diagonalize the sample covariance matrix (which is symmetric positive semidefinite, so the spectral theorem guarantees an orthonormal eigenbasis) and rotate the data into that eigenvector basis. In those new coordinates, off-diagonal covariances vanish: the axes are completely uncoupled and ranked by variance. Keeping the top few axes gives dimensionality reduction – the mathematical foundation of data compression, recommendation engines, and feature embeddings.
In deep neural networks, each layer applies a learned linear map $\mathbf{x} \mapsto \mathbf{Wx}$ followed by a nonlinearity. Analyzing what a layer has learned almost always comes down to inspecting its singular vectors – the basis in which the weight matrix is a pure, uncoupled stretch. Choosing an adapted basis is central to data representation and machine learning.

Walking question

Take a slow walk and turn this over: you and a friend describe the exact same rotation of the plane, but you use the standard grid and she uses a skewed grid (say, with axes at 60° to each other). Your matrices of numbers are different – yet you must agree on the angle of rotation, on the determinant ($= 1$), and on the fact that there are no real eigenvectors. *Which* numbers in a matrix belong to the operator and which belong to your grid? When you can sort every number in a matrix into one of those two buckets, you have understood this chapter.

31.9 Putting It All Together: One Operator in Three Bases

We end by carrying a single operator through three coordinate systems side by side.

Example 31.8 (The same projection, three matrices)**.** Let T be the orthogonal projection of $\mathbb{R}^2$ onto the line through $\mathbf{d} = \begin{bmatrix}1\\1\end{bmatrix}$. Geometrically T is one fixed act: flatten every arrow onto that diagonal line. Watch its matrix representation shift as we change coordinate systems.

Standard basis. Using the projection formula $\operatorname{proj}_{\mathbf{d}}\mathbf{x} = \frac{\mathbf{d}\mathbf{d}^{\mathsf{T}}}{\mathbf{d}^{\mathsf{T}}\mathbf{d}}\mathbf{x}$ with $\mathbf{d}^{\mathsf{T}}\mathbf{d} = 2$,

$$\mathbf{A} = \frac{1}{2}\begin{bmatrix}1\\1\end{bmatrix}\begin{bmatrix}1 & 1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix} = \begin{bmatrix}1/2 & 1/2\\1/2 & 1/2\end{bmatrix}.$$

Off-diagonal clutter; the projection geometry is obscured by coordinate crosstalk.

Eigenvector basis. The eigenvectors are obvious from the geometry: anything *on* the line is fixed (eigenvalue 1) and anything *perpendicular* is killed (eigenvalue 0). So $\mathbf{b}_1 = \begin{bmatrix}1\\1\end{bmatrix}$ (eigenvalue 1), $\mathbf{b}_2 = \begin{bmatrix}1\\-1\end{bmatrix}$ (eigenvalue 0), $\mathbf{B} = \left[\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right]$. By the diagonalization result of Section 31.8.1,

$$\mathbf{B}^{-1}\mathbf{A}\mathbf{B} = \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}.$$

Now the matrix *tells* you the geometry: keep the first axis, delete the second. That is a projection, plainly. (Verify if you like: $\mathbf{AB} = \left[\begin{smallmatrix}1 & 0\\ 1 & 0\end{smallmatrix}\right]$, and $\mathbf{B}^{-1}$ times that is $\left[\begin{smallmatrix}1 & 0\\ 0 & 0\end{smallmatrix}\right]$.)

Some random basis, say $\mathbf{c}_1 = \begin{bmatrix}1\\0\end{bmatrix}, \mathbf{c}_2 = \begin{bmatrix}1\\2\end{bmatrix}$, $\mathbf{C} = \left[\begin{smallmatrix}1 & 1\\ 0 & 2\end{smallmatrix}\right]$, $\mathbf{C}^{-1} = \frac{1}{2}\left[\begin{smallmatrix}2 & -1\\ 0 & 1\end{smallmatrix}\right]$:

$$\mathbf{C}^{-1}\mathbf{A}\mathbf{C} = \frac{1}{2}\begin{bmatrix}2 & -1\\ 0 & 1\end{bmatrix}\cdot\frac{1}{2}\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}\cdot\begin{bmatrix}1 & 1\\ 0 & 2\end{bmatrix} = \begin{bmatrix}1/4 & 3/4\\ 1/4 & 3/4\end{bmatrix}.$$

A third matrix for the third basis – ugliest of all. Yet all three are similar, so all three share $\det = 0$, $\mathrm{tr} = 1$, and eigenvalues $\{1, 0\}$. Check: each has trace 1 ($\frac{1}{2}+\frac{1}{2}$, $1+0$, $\frac{1}{4}+\frac{3}{4}$) and determinant 0. The operator – the act of flattening onto the diagonal – never moved. We simply described it from three vantage points, and only the eigenvector vantage made the description transparent.

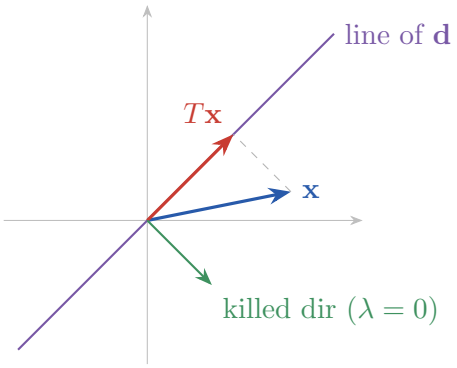


Figure 31.8: The fixed operator behind Example 31.8: project onto the line of $\mathbf{d} = (1, 1)$. On the line, vectors are fixed ($\lambda = 1$); perpendicular to it, they are erased ($\lambda = 0$). The eigenvector basis sees exactly this.

31.10 Summary

Focus

Key ideas of this chapter:

- A **linear map** is basis-free; its **matrix** exists only after you choose bases. Columns of $[T]_{\mathcal{C}\leftarrow\mathcal{B}}$ are "where the input basis vectors land, written in the output basis."

- A vector's **coordinates** are its address in a basis. The **basis matrix B** (columns = basis vectors) decodes a $\mathcal{B}$-address to standard; $\mathbf{B}^{-1}$ encodes.
- Same operator, new basis: $\mathbf{A}' = \mathbf{B}^{-1}\mathbf{A}\mathbf{B}$ – decode, apply operator, encode. This relation is **similarity**.
- Similar matrices share determinant, trace, characteristic polynomial, eigenvalues, and rank – the operator's coordinate-free invariants.
- **The central objective** is choosing the basis that makes $\mathbf{A}'$ simple: eigenvectors (diagonal), orthonormal eigenvectors (spectral theorem), singular vectors on both sides (SVD).

The boundaries of real coordinates

Yet as powerful as this machinery is, working strictly with real numbers and arbitrary coordinate frames runs into two fundamental boundaries.

The first boundary is geometric: *many real transformations cannot be diagonalized over* $\mathbb{R}$. The simplest example is the quarter turn that opened this chapter. A 90° rotation has no invariant lines in the real plane; every nonzero vector is swung into a new direction. Its characteristic polynomial is $\lambda^2+1=0$, which has no real roots. No change of real basis can ever diagonalize a rotation. Similarly, shear operators lack a full set of independent eigenvectors, leaving behind defective matrices that resist diagonal simplicity.

The second boundary is numerical: *skewed basis changes can be dangerous on a computer.* If two basis vectors in $\mathbf{P}$ point in nearly the same direction, $\mathbf{P}$ becomes ill-conditioned. Computing $\mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ in floating-point arithmetic then magnifies microscopic rounding errors into catastrophic noise.

Both limitations point toward the exact mathematical threshold we cross in the very next chapter.

To rescue rotations and guarantee that every operator has eigenvalues, we must enlarge our scalars from the real numbers $\mathbb{R}$ to the complex numbers $\mathbb{C}$. Over $\mathbb{C}$, the Fundamental Theorem of Algebra guarantees that every polynomial factors completely. Over $\mathbb{C}$, rotations acquire complex eigenvectors with eigenvalues $\pm i$.

To protect our computations from numerical instability, we will insist on bases whose vectors are mutually perpendicular and unit length. Over the complex numbers, these become **unitary matrices** ($\mathbf{U}^H\mathbf{U} = \mathbf{I}$), the complex analogues of orthogonal matrices. Unitary changes of basis never stretch lengths, never distort angles, and never amplify rounding error.

At the intersection of these two ideas stands the most celebrated unitary change of basis in modern science: the **Fourier matrix**. It maps signals from the time domain to the frequency domain, forming the basis of wave analysis, quantum mechanics, and the Fast Fourier Transform. Having mastered coordinate changes in the familiar real world, we are ready to step into the complex plane.

Exercises

Exercise 31.1 (Quick mental check: grid versus operator)**.** Which of the following attributes of an $n \times n$ matrix $\mathbf{A}$ change when you change the basis: (a) the individual matrix entries, (b) the trace, (c) the numerical coordinates of the eigenvectors, (d) the eigenvalues, (e) the determinant, (f) whether the matrix is diagonalizable? For each, state in one brief sentence whether it belongs to the grid or to the operator.

Exercise 31.2 (Reading an address)**.** Let $\mathbf{b}_1 = \begin{bmatrix}3\\1\end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix}1\\2\end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix}5\\5\end{bmatrix}$. Find $[\mathbf{v}]_{\mathcal{B}}$ two ways: (a) by solving $c_1\mathbf{b}_1 + c_2\mathbf{b}_2 = \mathbf{v}$, and (b) by computing $\mathbf{B}^{-1}\mathbf{v}$. Confirm the two answers agree.

Exercise 31.3 (Build a transformation matrix)**.** Let $T : \mathbb{R}^2 \to \mathbb{R}^2$ rotate by 90° counterclockwise. Using the basis $\mathcal{B} = \{\begin{bmatrix}1\\1\end{bmatrix}, \begin{bmatrix}1\\-1\end{bmatrix}\}$ for both input and output, compute $[T]_{\mathcal{B}\leftarrow\mathcal{B}}$ directly from the recipe (find $T\mathbf{b}_1, T\mathbf{b}_2$, then write them in $\mathcal{B}$-coordinates). Compare with $\mathbf{B}^{-1}\mathbf{R}\mathbf{B}$ where $\mathbf{R} = \left[\begin{smallmatrix}0 & -1\\ 1 & 0\end{smallmatrix}\right]$; they must match.

Exercise 31.4 (Derivative on a different basis)**.** On polynomials of degree ≤ 2, use the basis $\mathcal{B} = \{1,\ 1+t,\ 1+t+t^2\}$. Find the matrix of $D = \frac{d}{dt}$ in this basis. (Hint: compute D of each basis polynomial, then express each result as a combination of the same three basis polynomials.)

Exercise 31.5 (Active versus passive)**.** Let $\mathbf{B} = \left[\begin{smallmatrix}2 & 0\\ 0 & 1\end{smallmatrix}\right]$. (a) Describe $\mathbf{B}$ as an *active* transformation of the unit square. (b) Describe the *same* $\mathbf{B}$ as a *passive* decoding of $\mathcal{B}$-coordinates, where $\mathcal{B} = \{\begin{bmatrix}2\\0\end{bmatrix}, \begin{bmatrix}0\\1\end{bmatrix}\}$. (c) For the point with $\mathcal{B}$-address $\begin{bmatrix}1\\3\end{bmatrix}$, give its standard coordinates.

Exercise 31.6 (Diagonalize by changing basis)**.** Let $\mathbf{A} = \left[\begin{smallmatrix}4 & 1\\ 2 & 3\end{smallmatrix}\right]$. (a) Find the eigenvalues and eigenvectors. (b) Form the eigenvector basis matrix $\mathbf{B}$ and compute $\mathbf{B}^{-1}\mathbf{A}\mathbf{B}$; verify it is diagonal with the eigenvalues on the diagonal. (c) Use it to compute $\mathbf{A}^5$ via $\mathbf{A}^5 = \mathbf{B}\Lambda^5\mathbf{B}^{-1}$ without multiplying $\mathbf{A}$ by itself five times.

Exercise 31.7 (Invariants are necessary, not sufficient)**.** (a) Show $\mathbf{A} = \left[\begin{smallmatrix}3 & 0\\ 0 & 3\end{smallmatrix}\right]$ and $\mathbf{A}' = \left[\begin{smallmatrix}3 & 1\\ 0 & 3\end{smallmatrix}\right]$ have equal determinant, trace, and eigenvalues. (b) Prove they are nonetheless *not* similar. (Hint: what is $\mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ when $\mathbf{A} = 3\mathbf{I}$, for any invertible $\mathbf{P}$?)

Exercise 31.8 (Trace is a similarity invariant – prove it)**.** Prove $\operatorname{tr}(\mathbf{M}\mathbf{N}) = \operatorname{tr}(\mathbf{N}\mathbf{M})$ for square matrices of compatible size, then deduce $\operatorname{tr}(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \operatorname{tr}(\mathbf{A})$. Explain in one sentence why this means trace "belongs to the operator," not the basis.

Exercise 31.9 (Projection, three bases)**.** Let T project $\mathbb{R}^2$ onto the line through $\begin{bmatrix}2\\1\end{bmatrix}$. (a) Find the standard matrix $\mathbf{A}$ using $\frac{\mathbf{d}\mathbf{d}^{\mathsf{T}}}{\mathbf{d}^{\mathsf{T}}\mathbf{d}}$. (b) Find the eigenvector basis (the line direction and its perpendicular) and confirm $\mathbf{B}^{-1}\mathbf{A}\mathbf{B} = \left[\begin{smallmatrix}1 & 0\\ 0 & 0\end{smallmatrix}\right]$. (c) Confirm $\operatorname{tr}\mathbf{A} = 1$ and $\det\mathbf{A} = 0$ in both bases.

Exercise 31.10 (Two bases, one rectangular map)**.** Let $T : \mathbb{R}^3 \to \mathbb{R}^2$ be $T\begin{bmatrix}x\\y\\z\end{bmatrix} = \begin{bmatrix}x+z\\y-z\end{bmatrix}$. (a) Find its matrix in the standard bases. (b) Now use the input basis $\{\begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\end{bmatrix}, \begin{bmatrix}1\\0\\1\end{bmatrix}\}$ and the standard output basis; recompute the matrix from the recipe and explain which column changed and why.

Exercise 31.11 (Conceptual: which numbers belong to the grid?)**.** A student computes the matrix of a 60° rotation in some skewed basis and gets entries that are not the usual $\cos, \sin$ values – in fact the off-diagonal entries are unequal in magnitude. Without computing, explain (i) why the entries can look so different from the standard rotation matrix, and (ii) which two numbers derived from the matrix she can nonetheless predict in advance, and what they are.

Exercise 31.12 (Preview: the cyclic shift and roots of unity)**.** Consider the cyclic shift operator $S : \mathbb{R}^3 \to \mathbb{R}^3$ defined by $S(x_1, x_2, x_3) = (x_3, x_1, x_2)$, which permutes coordinate entries in a loop. (a) Write its matrix $\mathbf{P}$ in the standard basis and compute $\det \mathbf{P}$ and $\operatorname{tr} \mathbf{P}$. (b) Show that the all-ones vector $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ is an eigenvector with eigenvalue $\lambda_1 = 1$. (c) Notice that repeating the shift three times returns every vector to its original state ($\mathbf{P}^3 = \mathbf{I}$). Therefore, any eigenvalue λ must satisfy $\lambda^3 = 1$. Are there any other *real* eigenvalues or eigenvectors? How does this anticipate the complex vectors and the Fourier matrix in the next chapter?

Chapter 32

Complex Vectors, Hermitian Matrices, and the Fourier Matrix

In Chapter 31, we diagonalized linear transformations by changing coordinates to an eigenvector basis. But what happens when a matrix has no real eigenvalues at all, leaving no real directions invariant?

Every length we have measured so far has been a real length. Every angle a real angle. Our vectors lived in $\mathbb{R}^n$, our matrices had real entries, and the dot product $\mathbf{u} \cdot \mathbf{v} = \sum u_i v_i$ answered every metric question we asked. That setting is comfortable, and most of this book stays inside it.

Back in Chapters 23 and 24, however, we kept running into real matrices whose eigenvalues are not real. A rotation in the plane has no real eigenvectors at all; spin every arrow by ninety degrees and not a single direction survives. Yet the characteristic polynomial still factors, just over a bigger number system, and out come $\pm i$. The geometry was pushing us to enlarge the number field, and this chapter is where we do it.

We are going to let our vectors have *complex* entries. At first this looks like a small bookkeeping change – swap $\mathbb{R}$ for $\mathbb{C}$ – but it forces one real adjustment to the dot product, and that single adjustment ripples outward into two new families of matrices: the **Hermitian** matrices (the complex cousins of symmetric matrices) and the **unitary** matrices (the complex cousins of orthogonal matrices). These are not exotic. They are the matrices of quantum mechanics, of signal processing, of every algorithm that touches a frequency.

At the end we reach the most important unitary matrix in applied mathematics: the **Fourier matrix**. It turns a signal into its frequencies, and the algorithm computing it – the Fast Fourier Transform – is, by most accounts, the most-run numerical algorithm in the world. Everything in this chapter is built to make that matrix feel inevitable.

Slow down at the very first idea, the conjugate in the inner product. Get that one right and the rest of the chapter follows from it.

32.1 Why real numbers are not enough

Let us start with the rotation that broke things. The matrix

$$\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

spins every vector in the plane by 90° counterclockwise. Geometrically, *no* nonzero real vector $\mathbf{x}$ satisfies $\mathbf{Rx} = \lambda\mathbf{x}$: an eigenvector would have to point along the same line after rotation, and a quarter turn sends every line to a different line. So over $\mathbb{R}$, this matrix has no eigenvectors.

But the algebra does not give up. Its characteristic polynomial is

$$\det(\mathbf{R} - \lambda\mathbf{I}) = \det \begin{bmatrix} -\lambda & -1 \\ 1 & -\lambda \end{bmatrix} = \lambda^2 + 1,$$

and $\lambda^2 + 1 = 0$ has solutions $\lambda = \pm i$, where $i = \sqrt{-1}$. The eigenvalues exist; they just live in $\mathbb{C}$. And once we allow complex entries in the eigenvector too, genuine eigenvectors appear:

$$\mathbf{R} \begin{bmatrix} 1 \\ -i \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -i \end{bmatrix} = \begin{bmatrix} i \\ 1 \end{bmatrix} = i \begin{bmatrix} 1 \\ -i \end{bmatrix},$$

so $\begin{bmatrix} 1 \\ -i \end{bmatrix}$ is a genuine eigenvector with eigenvalue i. The geometry of "no fixed direction" and the algebra of "complex eigenvalues" are two faces of the same fact. Enlarging to $\mathbb{C}$ does not destroy the geometry; it completes it.

Intuition

A real matrix can "rotate" inside the plane spanned by two coordinates, and a rotation has no real eigen-direction. Complex numbers are exactly the tool for talking about rotation algebraically: multiplication by $e^{i\theta}$ *is* rotation by θ. So when a real matrix wants to rotate, its eigenvalues come out as complex numbers of the form $e^{i\theta}$, encoding the angle of the spin. Complex eigenvalues arise whenever a real linear map includes a rotation.

32.1.1 A two-minute refresher on complex numbers

We will need only a handful of facts; Chapter 39 has the full story. A complex number is $z = a + bi$ with $a, b \in \mathbb{R}$ and $i^2 = -1$. We call $a = \operatorname{Re} z$ the real part and $b = \operatorname{Im} z$ the imaginary part. Two operations matter enormously for us.

The **conjugate** flips the sign of the imaginary part:

$$\bar{z} = a - bi.$$

The **modulus** (or absolute value) is the length of z viewed as the point (a, b) in the plane:

$$|z| = \sqrt{a^2 + b^2}.$$

The reason these two travel together is a single identity:

$$z\bar{z} = (a + bi)(a - bi) = a^2 - (bi)^2 = a^2 + b^2 = |z|^2.$$

Multiplying a number by its own conjugate *kills the imaginary part* and leaves a real, nonnegative number – the squared length. That one fact drives everything in this chapter.

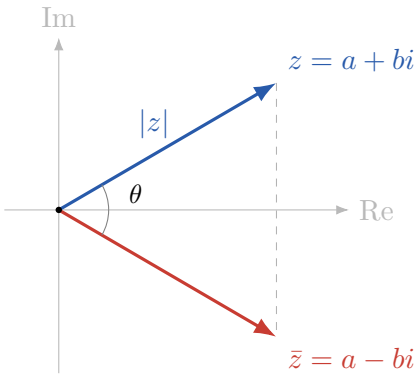


Figure 32.1: A complex number z and its conjugate $\bar{z}$ are mirror images across the real axis. Both have the same modulus $|z|$; conjugation flips the angle from $+\theta$ to $-\theta$. The product $z\bar{z} = |z|^2$ is real.

As Figure 32.1 shows, z and $\bar{z}$ are mirror images across the real axis, sharing a modulus but carrying opposite angles. The polar form makes the underlying geometry explicit. Writing $z = re^{i\theta}$ with $r = |z|$ and θ the angle, Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$ says z is the point at distance r and angle θ. Then $\bar{z} = re^{-i\theta}$ (same radius, opposite angle), and multiplication adds angles: $(r_1 e^{i\theta_1})(r_2 e^{i\theta_2}) = r_1 r_2\, e^{i(\theta_1+\theta_2)}$. Multiplying by a number of modulus 1 – a number $e^{i\theta}$ on the unit circle – is a pure rotation. Keep that picture in mind for the Fourier section.

Pause

Pause and think: If $z = 3+4i$, compute its square $z^2 = (3+4i)^2 = 9+24i-16 = -7+24i$. Notice that z^2 is not real, and its real part is negative—completely useless as a measure of size. Now compute $z\bar{z} = (3+4i)(3-4i) = 9+16 = 25$. It is strictly positive, real, and equals $|z|^2$. Squaring doubles the angle (2θ); multiplying by the conjugate cancels the angle ($\theta - \theta = 0$).

32.2 Complex vectors and the right inner product

Now stack complex numbers into a vector. A vector in $\mathbb{C}^n$ is just a list of n complex numbers,

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}, \qquad z_k \in \mathbb{C}.$$

Addition and scalar multiplication work exactly as before, now with complex scalars. The trouble – and it is a productive trouble – starts when we ask for length.

32.2.1 The naive dot product fails

In $\mathbb{R}^n$ we defined $\|\mathbf{u}\|^2 = \mathbf{u}\cdot\mathbf{u} = \sum u_i^2$, and the whole edifice of geometry stood on one fact: $\mathbf{u}\cdot\mathbf{u} \geq 0$, zero only for the zero vector. Try to copy it into $\mathbb{C}^n$ directly with the formula $\sum z_k^2$ and watch it collapse. Take the perfectly fine nonzero vector $\mathbf{z} = \begin{bmatrix} 1 \\ i \end{bmatrix}$. Then

$$\sum z_k^2 = 1^2 + i^2 = 1 + (-1) = 0.$$

A nonzero vector with "length squared" equal to zero. And it gets worse: the vector $\begin{bmatrix} 1 \\ 1+i \end{bmatrix}$ gives $1^2 + (1+i)^2 = 1 + 2i$, a non-real number, so "length squared" is not even a real number we could take the square root of. The naive copy is broken. We need a repair.

Common trap

The number-one mistake when first meeting $\mathbb{C}^n$ is to write $\|\mathbf{z}\|^2 = \sum z_k^2$. Squaring a complex number does *not* produce a length; $i^2 = -1$, not $+1$. Length must come from $|z_k|^2 = z_k\bar{z}_k$, never from z_k^2. Without the complex conjugate, nonzero vectors such as $(1, i)$ would have "length squared" equal to zero.

32.2.2 The fix: conjugate one side

We already saw the cure two sections ago. For a single number, the real, nonnegative "length squared" is $z\bar{z} = |z|^2$, not z^2. So for a vector we should sum $z_k\bar{z}_k$:

$$\|\mathbf{z}\|^2 \;=\; \sum_{k=1}^{n} z_k\bar{z}_k \;=\; \sum_{k=1}^{n} |z_k|^2 \;\geq\; 0,$$

and this is zero exactly when every $|z_k| = 0$, i.e. when $\mathbf{z} = \mathbf{0}$. Positivity is restored. Check our broken example: $\mathbf{z} = \begin{bmatrix} 1 \\ i \end{bmatrix}$ now has $\|\mathbf{z}\|^2 = |1|^2 + |i|^2 = 1 + 1 = 2$, a sensible length $\sqrt{2}$.

Generalizing from $\mathbf{z}$ with itself to two different vectors gives the **complex (or Hermitian) inner product**. We conjugate the entries of the first vector and pair them with the second:

$$\boxed{\langle \mathbf{u},\, \mathbf{v} \rangle \;=\; \sum_{k=1}^{n} \bar{u}_k\, v_k \;=\; \bar{u}_1 v_1 + \bar{u}_2 v_2 + \cdots + \bar{u}_n v_n.}$$

(Convention warning: mathematicians often conjugate the second slot instead. We pick the first; Chapter 39 notes the swap. Either way, exactly one side is conjugated.) With this definition $\langle \mathbf{z},\, \mathbf{z} \rangle = \sum \bar{z}_k z_k = \sum |z_k|^2 = \|\mathbf{z}\|^2$ is real and nonnegative – the whole point.

Definition 32.1. For $\mathbf{u}, \mathbf{v} \in \mathbb{C}^n$, the **Hermitian inner product** is $\langle \mathbf{u},\, \mathbf{v} \rangle = \sum_{k=1}^{n} \bar{u}_k v_k$, and the **norm** is $\|\mathbf{z}\| = \sqrt{\langle \mathbf{z},\, \mathbf{z} \rangle} = \sqrt{\sum_k |z_k|^2}$. Two vectors are **orthogonal** when $\langle \mathbf{u},\, \mathbf{v} \rangle = 0$.

Example 32.1. Let $\mathbf{u} = \begin{bmatrix} 1+i \\ 2 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 3 \\ 1-i \end{bmatrix}$. Conjugate the *first* vector's entries: $\bar{u}_1 = 1 - i$, $\bar{u}_2 = 2$. Then

$$\langle \mathbf{u},\, \mathbf{v} \rangle = (1-i)(3) + (2)(1-i) = (3 - 3i) + (2 - 2i) = 5 - 5i.$$

And the squared length of $\mathbf{u}$ is

$$\|\mathbf{u}\|^2 = \langle \mathbf{u}, \mathbf{u} \rangle = (1-i)(1+i) + (2)(2) = (1+1) + 4 = 6,$$

so $\|\mathbf{u}\| = \sqrt{6}$. The self inner product came out real and positive even though $\mathbf{u}$ has complex entries – the conjugate did its job.

Pause

Pause and think: What is the length of $\mathbf{w} = \begin{bmatrix} i \\ 0 \end{bmatrix}$? Using our definition, $\|\mathbf{w}\|^2 = \langle \mathbf{w}, \mathbf{w} \rangle = \bar{i} \cdot i = (-i)(i) = 1$, giving $\|\mathbf{w}\| = 1$. What would the naive sum of squares produce? It would yield $i^2 + 0 = -1$, claiming that a nonzero vector has negative squared length. The conjugate on the first coordinate is the repair that restores positive lengths.

32.2.3 Conjugate symmetry: the price of the fix

There is one property we must give up, and it is worth seeing exactly which one. In $\mathbb{R}^n$ the dot product was symmetric: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$. In $\mathbb{C}^n$ symmetry becomes **conjugate symmetry**:

$$\langle \mathbf{v}, \mathbf{u} \rangle = \overline{\langle \mathbf{u}, \mathbf{v} \rangle}.$$

Swapping the two vectors conjugates the answer. You can read it straight off the definition: $\langle \mathbf{v}, \mathbf{u} \rangle = \sum \bar{v}_k u_k$, and conjugating $\langle \mathbf{u}, \mathbf{v} \rangle = \sum \bar{u}_k v_k$ gives $\sum u_k \bar{v}_k = \sum \bar{v}_k u_k$, the same thing. In Example 32.1 we found $\langle \mathbf{u}, \mathbf{v} \rangle = 5 - 5i$; you can check directly that $\langle \mathbf{v}, \mathbf{u} \rangle = (3)(1+i) + (1+i)(2) = 5 + 5i$, indeed the conjugate.

Conjugate symmetry is required if $\langle \mathbf{z}, \mathbf{z} \rangle$ is to remain real. Conjugate symmetry *forces* $\langle \mathbf{z}, \mathbf{z} \rangle = \overline{\langle \mathbf{z}, \mathbf{z} \rangle}$, and a number equal to its own conjugate is exactly a real number.

Formal

The Hermitian inner product on $\mathbb{C}^n$ satisfies, for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{C}^n$ and $c \in \mathbb{C}$:

1. **Conjugate symmetry:** $\langle \mathbf{v}, \mathbf{u} \rangle = \overline{\langle \mathbf{u}, \mathbf{v} \rangle}$.
2. **Linearity in the second slot:** $\langle \mathbf{u}, \mathbf{v} + \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{v} \rangle + \langle \mathbf{u}, \mathbf{w} \rangle$ and $\langle \mathbf{u}, c\,\mathbf{v} \rangle = c\,\langle \mathbf{u}, \mathbf{v} \rangle$.
3. **Conjugate-linearity in the first slot:** $\langle c\,\mathbf{u}, \mathbf{v} \rangle = \bar{c}\,\langle \mathbf{u}, \mathbf{v} \rangle$.
4. **Positivity:** $\langle \mathbf{z}, \mathbf{z} \rangle = \sum |z_k|^2 \geq 0$, with equality iff $\mathbf{z} = \mathbf{0}$.

Property 3 is the one that surprises people: a scalar pulled out of the *first* slot comes out conjugated, because that slot is the conjugated one. If you scale the first vector by i, the inner product scales by $\bar{i} = -i$. Everything else behaves just like the real dot product.

Walking question

Take a walk and convince yourself of this without any formula: "length must be a real, nonnegative number, and it can only be zero for the zero vector." Now ask: what is the *simplest* change to $\sum u_k v_k$ that guarantees $\langle \mathbf{z}, \mathbf{z} \rangle \geq 0$ for every complex vector? You will rediscover the conjugate. The mathematics had no choice; the conjugate is the unique repair.

32.3 The conjugate transpose

We packaged the real dot product as a matrix product: $\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^{\mathsf{T}}\mathbf{v}$, a row times a column. The complex inner product wants the same tidy packaging, but the row has to carry that conjugate. So we bundle "transpose and conjugate" into a single operation.

Definition 32.2. The **conjugate transpose** (or **Hermitian adjoint**) of a matrix $\mathbf{A} \in \mathbb{C}^{m\times n}$ is $\mathbf{A}^* \in \mathbb{C}^{n\times m}$, obtained by transposing $\mathbf{A}$ and conjugating every entry: $(\mathbf{A}^*)_{jk} = \overline{(\mathbf{A})_{kj}}$. For a vector, $\mathbf{u}^*$ is the conjugated row.

Many books write $\mathbf{A}^H$ ("H" for Hermite) or $\mathbf{A}^\dagger$ ("dagger," the physics standard) for the same thing. We use the star $\mathbf{A}^*$. With it, the inner product is just a product:

$$\langle \mathbf{u}, \mathbf{v} \rangle \;=\; \mathbf{u}^*\mathbf{v} \;=\; \begin{bmatrix} \bar{u}_1 & \cdots & \bar{u}_n \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix}.$$

The star *is* the inner product made into matrix algebra. Notice the clean analogy: everywhere a real formula had a transpose $\mathbf{A}^{\mathsf{T}}$, the complex version has a conjugate transpose $\mathbf{A}^*$. The reversal rule survives unchanged, $(\mathbf{AB})^* = \mathbf{B}^*\mathbf{A}^*$, and $(\mathbf{A}^*)^* = \mathbf{A}$.

Example 32.2.

$$\mathbf{A} = \begin{bmatrix} 1+i & 2 \\ 0 & 3-i \end{bmatrix} \quad\Longrightarrow\quad \mathbf{A}^* = \begin{bmatrix} 1-i & 0 \\ 2 & 3+i \end{bmatrix}.$$

Transpose (swap across the diagonal), then conjugate every entry. The 2 and 0 are real, so they only move; the $1+i$ and $3-i$ both move *and* flip the sign of their imaginary part.

Focus

Translate "real → complex" by one rule: **every transpose becomes a conjugate transpose.** Dot product $\mathbf{u}^{\mathsf{T}}\mathbf{v} \to \mathbf{u}^*\mathbf{v}$. Symmetric $\mathbf{A}^{\mathsf{T}} = \mathbf{A} \to$ Hermitian $\mathbf{A}^* = \mathbf{A}$. Orthogonal $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I} \to$ unitary $\mathbf{U}^*\mathbf{U} = \mathbf{I}$. The geometry is identical; only the star changed.

32.4 Hermitian matrices: the complex symmetric

A real symmetric matrix satisfies $\mathbf{A}^{\mathsf{T}} = \mathbf{A}$, leading to the Spectral Theorem: real eigenvalues and an orthonormal basis of eigenvectors. We now identify the complex matrices that satisfy the analogous property. The answer is the matrices that equal their own conjugate transpose.

Definition 32.3. A square matrix $\mathbf{A} \in \mathbb{C}^{n\times n}$ is **Hermitian** if $\mathbf{A}^* = \mathbf{A}$, that is, $a_{jk} = \overline{a_{kj}}$ for every entry.

Two consequences are visible immediately from $a_{jk} = \overline{a_{kj}}$. On the diagonal, $j = k$ forces $a_{kk} = \overline{a_{kk}}$, so *the diagonal entries are real.* Off the diagonal, entries come in conjugate-mirror pairs across the diagonal. A real symmetric matrix is the special case where the off-diagonal pairs are equal real numbers.

Example 32.3.

$$\mathbf{A} = \begin{bmatrix} 2 & 3-i \\ 3+i & 5 \end{bmatrix}$$

is Hermitian: the diagonal $2, 5$ are real, and the off-diagonal entries $3-i$ and $3+i$ are conjugates of each other. Transpose-and-conjugate and you get $\mathbf{A}$ back. By contrast $\begin{bmatrix} 2 & 3-i \\ 3-i & 5 \end{bmatrix}$ is *not* Hermitian – the off-diagonal pair must be conjugates, not equal.

Pause

Pause and think: Could a Hermitian matrix have $4+2i$ on its main diagonal? No: the definition $\mathbf{A}^* = \mathbf{A}$ requires $(\mathbf{A})_{kk} = \overline{(\mathbf{A})_{kk}}$. The only complex numbers that equal their own conjugates are real numbers ($\mathrm{Im}(z) = 0$). Every diagonal entry of a Hermitian matrix is forced to be strictly real.

32.4.1 Real eigenvalues, for free

The headline property is the same one symmetric matrices enjoyed, and the proof is a two-line argument that shows exactly why we needed the conjugate inner product.

Theorem 32.1. *Every eigenvalue of a Hermitian matrix is real.*

Proof. Let $\mathbf{A}^* = \mathbf{A}$ and suppose $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$. Form the number $\mathbf{x}^*\mathbf{A}\mathbf{x}$ and compute it two ways. First, using $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$,

$$\mathbf{x}^*\mathbf{A}\mathbf{x} = \mathbf{x}^*(\lambda\mathbf{x}) = \lambda\,\mathbf{x}^*\mathbf{x} = \lambda\,\|\mathbf{x}\|^2.$$

Second, take the conjugate transpose of the scalar $\mathbf{x}^*\mathbf{A}\mathbf{x}$. A 1×1 matrix is its own transpose, so applying the star just conjugates the number, while on the right it distributes by the reversal rule and uses $\mathbf{A}^* = \mathbf{A}$:

$$\overline{\mathbf{x}^*\mathbf{A}\mathbf{x}} = (\mathbf{x}^*\mathbf{A}\mathbf{x})^* = \mathbf{x}^*\mathbf{A}^*\mathbf{x} = \mathbf{x}^*\mathbf{A}\mathbf{x}.$$

So $\mathbf{x}^*\mathbf{A}\mathbf{x}$ equals its own conjugate, hence is real. But it also equals $\lambda\|\mathbf{x}\|^2$, and $\|\mathbf{x}\|^2 > 0$ is a positive real. A real number divided by a positive real is real, so λ is real. □

That argument is worth rereading. It is the complex twin of the symmetric proof, and the conjugate inner product is doing all the work: $\|\mathbf{x}\|^2$ is real and positive *because* we built the inner product to make it so. Swap in the naive $\sum z_k^2$ and the proof falls apart – another sign that the conjugate was the right move.

Why this matters

This theorem explains the foundational role of Hermitian matrices in physics. In quantum mechanics, physical observables—such as energy, position, momentum, and spin—are represented by Hermitian operators. Theorem 32.1 guarantees that all measurement outcomes (the eigenvalues of the operator) are strictly real numbers, ensuring that physical predictions correspond to real-valued experimental measurements.

32.4.2 Orthogonal eigenvectors and the complex spectral theorem

Just as in the real case, eigenvectors of a Hermitian matrix for *different* eigenvalues are orthogonal. The argument is again two lines. If $\mathbf{Ax} = \lambda\mathbf{x}$ and $\mathbf{Ay} = \mu\mathbf{y}$ with $\lambda \neq \mu$ (both real, by the theorem), then

$$\lambda\langle \mathbf{x},\, \mathbf{y}\rangle = \langle \lambda\mathbf{x},\, \mathbf{y}\rangle = \langle \mathbf{Ax},\, \mathbf{y}\rangle = \mathbf{x}^*\mathbf{A}^*\mathbf{y} = \mathbf{x}^*\mathbf{Ay} = \langle \mathbf{x},\, \mathbf{Ay}\rangle = \langle \mathbf{x},\, \mu\mathbf{y}\rangle = \mu\langle \mathbf{x},\, \mathbf{y}\rangle.$$

(We used λ real so it comes out of the conjugate slot unchanged, and $\mathbf{A}^* = \mathbf{A}$ in the middle.) Thus $(\lambda-\mu)\langle \mathbf{x},\, \mathbf{y}\rangle = 0$, and since $\lambda \neq \mu$ we get $\langle \mathbf{x},\, \mathbf{y}\rangle = 0$. Distinct eigenvalues $\Rightarrow$ orthogonal eigenvectors, exactly as before.

Putting the pieces together gives the complex Spectral Theorem, the destination of all this machinery.

Theorem 32.2 (Spectral Theorem, complex form)**.** *Every Hermitian matrix* $\mathbf{A} \in \mathbb{C}^{n\times n}$ *can be written*

$$\mathbf{A} = \mathbf{U}\,\mathbf{\Lambda}\,\mathbf{U}^*,$$

where $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ *has the (real) eigenvalues on its diagonal and* $\mathbf{U}$ *is unitary, its columns an orthonormal basis of eigenvectors. Real eigenvalues, orthonormal eigenvectors – the same picture as the real symmetric case, now over* $\mathbb{C}$*.*

We will not grind out the full induction (it mirrors the real proof exactly), but the moral is clean: **Hermitian is to $\mathbb{C}$ as symmetric is to $\mathbb{R}$.** Every good thing symmetric matrices did, Hermitian matrices do, with stars in place of transposes.

Before/after Strang

Strang teaches the real Spectral Theorem $\mathbf{A} = \mathbf{Q\Lambda Q}^{\mathsf{T}}$ as the summit of his eigenvalue chapters, then notes – almost in passing – that complex entries simply replace ^{T} with the conjugate transpose. Reading this chapter *before* Strang, you will recognize his $\mathbf{A} = \mathbf{Q\Lambda Q}^{\mathsf{T}}$ as the real shadow of $\mathbf{A} = \mathbf{U\Lambda U}^*$. Reading it *after*, you now have the version that actually runs the Fourier transform and quantum mechanics – the foundation that the real symmetric factorization "$\mathbf{S} = \mathbf{Q\Lambda Q}^{\mathsf{T}}$" prepares for.

32.5 Unitary matrices: the complex orthogonal

The other star of the real theory was the orthogonal matrix $\mathbf{Q}$ with $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$ – the rotations and reflections, the rigid motions that preserve every length and angle. Their complex analog is defined by the same equation with a star.

Definition 32.4. A square matrix $\mathbf{U} \in \mathbb{C}^{n\times n}$ is **unitary** if $\mathbf{U}^*\mathbf{U} = \mathbf{I}$, equivalently $\mathbf{U}^{-1} = \mathbf{U}^*$. Its columns form an orthonormal set in $\mathbb{C}^n$.

Why "orthonormal columns"? Read off the (j,k) entry of $\mathbf{U}^*\mathbf{U}$: it is (row j of $\mathbf{U}^*$) times (column k of $\mathbf{U}$) $= \langle \mathbf{u}_j,\, \mathbf{u}_k\rangle$, the inner product of two columns of $\mathbf{U}$. Setting $\mathbf{U}^*\mathbf{U} = \mathbf{I}$ says $\langle \mathbf{u}_j,\, \mathbf{u}_k\rangle = \delta_{jk}$: each column has length 1, and different columns are orthogonal. Identical to the real story, with the Hermitian inner product.

32.5.1 Unitary means length-preserving

The defining geometric property – the reason these matrices matter – is that they preserve the complex inner product, hence all lengths and angles.

Theorem 32.3. *If* $\mathbf{U}$ *is unitary then* $\langle \mathbf{Ux}, \mathbf{Uy} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle$ *for all* $\mathbf{x}, \mathbf{y}$*, and in particular* $\|\mathbf{Ux}\| = \|\mathbf{x}\|$.

Proof. Slide the star across:

$$\langle \mathbf{Ux}, \mathbf{Uy} \rangle = (\mathbf{Ux})^*(\mathbf{Uy}) = \mathbf{x}^*\mathbf{U}^*\mathbf{Uy} = \mathbf{x}^*\mathbf{Iy} = \mathbf{x}^*\mathbf{y} = \langle \mathbf{x}, \mathbf{y} \rangle.$$

Setting $\mathbf{y} = \mathbf{x}$ gives $\|\mathbf{Ux}\|^2 = \|\mathbf{x}\|^2$. □

So a unitary matrix is a **rigid motion of** $\mathbb{C}^n$. It can rotate and reflect and mix the coordinates as wildly as it likes, but it never stretches, never shrinks, never distorts a single length. This is precisely why unitary matrices are the "safe" transformations of numerical computing and of quantum mechanics: applying one cannot amplify errors (it preserves their size) and cannot leak probability (it preserves total length). This leads directly to the next property.

Corollary 32.1. *Every eigenvalue* λ *of a unitary matrix has* $|\lambda| = 1$*: the eigenvalues live on the unit circle.*

Proof. If $\mathbf{Ux} = \lambda\mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$, then $\|\mathbf{x}\| = \|\mathbf{Ux}\| = \|\lambda\mathbf{x}\| = |\lambda|\,\|\mathbf{x}\|$, so $|\lambda| = 1$. □

That is the perfect complement to the Hermitian story. Hermitian eigenvalues are *real* (they sit on the real line); unitary eigenvalues have *modulus one* (they sit on the unit circle). Each is the complex shadow of its real cousin: symmetric eigenvalues are real, and orthogonal eigenvalues have $|\lambda| = 1$ (the real ones being ± 1). Figure 32.2 puts the two loci side by side.

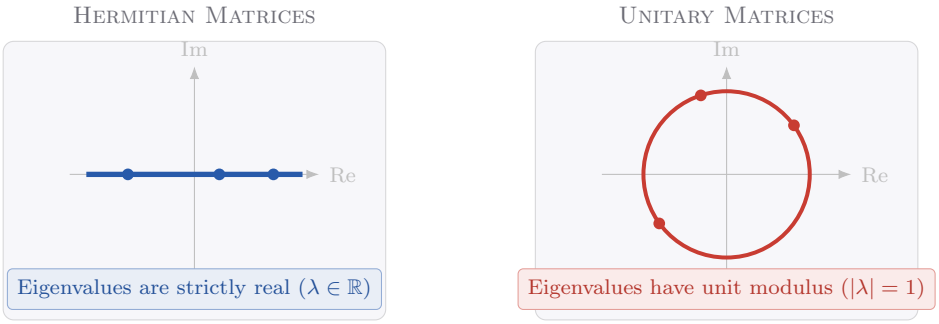


Figure 32.2: Where the eigenvalues live in the complex plane. Left: Hermitian matrices have all eigenvalues on the real line ($\lambda_j \in \mathbb{R}$), extending the symmetric spectral theorem. Right: Unitary matrices preserve lengths, constraining all eigenvalues to the unit circle ($|\lambda_j| = 1$), generalizing orthogonal rotations.

Example 32.4. The matrix

$$\mathbf{U} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ i & -i \end{bmatrix}$$

is unitary. Its columns are $\mathbf{u}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ i \end{bmatrix}$ and $\mathbf{u}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -i \end{bmatrix}$. Check: $\langle \mathbf{u}_1, \mathbf{u}_1 \rangle = \frac{1}{2}(\bar{1} \cdot 1 + \bar{i} \cdot i) = \frac{1}{2}(1 + (-i)(i)) = \frac{1}{2}(1 + 1) = 1$, length one; and $\langle \mathbf{u}_1, \mathbf{u}_2 \rangle = \frac{1}{2}(\bar{1} \cdot 1 + \bar{i} \cdot (-i)) = \frac{1}{2}(1 + (-i)(-i)) =$

$\frac{1}{2}(1+i^2) = \frac{1}{2}(1-1) = 0$, orthogonal. Two orthonormal columns, so $\mathbf{U}^*\mathbf{U} = \mathbf{I}$. (Watch the conjugate on the i in the first slot – without it you would wrongly get $\langle \mathbf{u}_1, \mathbf{u}_1 \rangle = \frac{1}{2}(1+i^2) = 0$, the same trap as before.)

Connection to other math

Unitary matrices are the gateway to *group theory*. The set of all $n \times n$ unitary matrices is closed under multiplication and inverse – a product of rigid motions is a rigid motion – so it forms a group, written $U(n)$, the *unitary group*. Its members of determinant 1 form $SU(n)$, and $SU(2)$, $SU(3)$ are the symmetry groups at the heart of the Standard Model of particle physics. The single equation $\mathbf{U}^*\mathbf{U} = \mathbf{I}$ in this chapter is the front door to the symmetry language of modern physics. The real analog $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ gives the rotation groups $O(n)$ and $SO(n)$ – the symmetries of spheres and rigid bodies.

32.6 The discrete Fourier matrix

Now we build the most useful unitary matrix in applied mathematics. The idea is an old one: a complicated signal – a sound wave, a stock price, an image row – can be rebuilt as a *sum of pure oscillations* of different frequencies. The Fourier matrix is the change of basis that swaps the "samples" description of a signal for its "frequencies" description. What makes it work is that this change of basis is *unitary* – a rigid rotation of $\mathbb{C}^n$ – so it is fully reversible and stable.

History and connection

In 1807, thirty-nine-year-old Joseph Fourier presented a radical memoir on heat diffusion to the Institut de France. To model how temperature spreads through solid metal bars, Fourier had only the tools of classical calculus and trigonometric series (infinite sums of sinusoidal waves). He proposed an audacious leap: *any* initial temperature profile $f(x)$—even one with sharp corners or abrupt step discontinuities—could be reconstructed as an infinite sum of smooth sine and cosine waves.

The examining committee—titans of analysis including Joseph-Louis Lagrange, Pierre-Simon Laplace, and Adrien-Marie Legendre—was fiercely skeptical. Lagrange stood up and objected on foundational grounds: sines and cosines are infinitely smooth and differentiable everywhere. How could adding smooth waves ever produce a sharp corner or an abrupt jump? Although the Academy awarded Fourier a mathematics prize in 1812, it refused to publish his work for fifteen years, citing a lack of mathematical rigor. It took Peter Gustav Lejeune Dirichlet in 1829 to establish the first rigorous convergence conditions, though near any sudden jump, Fourier approximations stubbornly overshoot the true value by roughly 9%—a persistent ringing known as the *Gibbs phenomenon*.

In 1858, French mathematician Charles Hermite completed the algebraic side of this story. He proved that complex matrices equal to their conjugate transpose ($\mathbf{A}^* = \mathbf{A}$, transposing entries across the diagonal and negating their imaginary parts) are guaranteed to have strictly real eigenvalues and mutually orthogonal eigenvectors. This resolved a central puzzle: how could numbers with imaginary parts yield real physical quantities? Decades later, quantum mechanics embraced Hermite's matrices as the language for measurable quantities, or *observables* (such as energy, momentum, and spin). The discrete bridge connecting both ideas is the Fourier matrix $\mathbf{F}_n$: evaluating wave frequencies at the *roots of unity* (points spaced evenly

on the complex unit circle) produces a unitary coordinate system that rotates complex space without distortion, diagonalizing circular convolutions and turning heavy filters into simple point-by-point products.

Yet the subject continues to challenge mathematicians today. In continuous harmonic analysis, proving whether Fourier series converge pointwise for general square-integrable functions stumped researchers until Lennart Carleson's proof in 1966; in two or more dimensions, open questions like the Bochner–Riesz conjecture (how spherical sums of Fourier waves converge) and the Fourier restriction conjecture remain unsolved. In algorithmic computer science, whether the discrete Fourier transform can be computed in fewer than $O(n \log n)$ operations on general hardware also remains an active open question.

32.6.1 Roots of unity: n points evenly on the circle

Everything is built from one number. Fix a size n and define

$$\omega = e^{2\pi i/n} = \cos\frac{2\pi}{n} + i \sin\frac{2\pi}{n}.$$

This ω is the point on the unit circle at angle $2\pi/n$ – one n-th of the way around. Its powers $1, \omega, \omega^2, \ldots, \omega^{n-1}$ are n points spaced evenly around the unit circle, the **n-th roots of unity**: each is a solution of $z^n = 1$. They close up perfectly, $\omega^n = e^{2\pi i} = 1$, so the powers cycle with period n ($\omega^{n+1} = \omega$, and so on). Figure 32.3 draws the eight evenly spaced roots for $n = 8$.

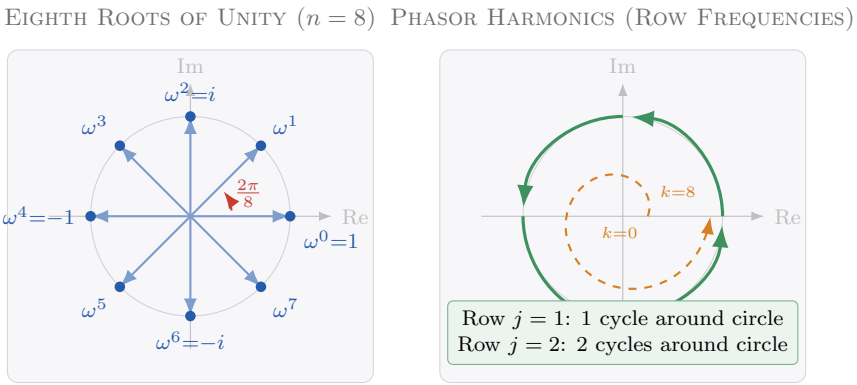


Figure 32.3: The eighth roots of unity and the discrete Fourier harmonic spiral. Left: The powers of $\omega = e^{2\pi i/8}$ march evenly around the unit circle in angular steps of $\Delta\theta = 2\pi/8$. The eight symmetric arrows cancel perfectly: $\sum_{k=0}^{7} \omega^k = 0$. Right: Multiplying by row j of the Fourier matrix evaluates the signal against a complex phasor rotating at frequency j. As discrete time k advances, ω^{jk} cycles j complete times around the origin.

The single fact we will lean on is a geometric one: **the n roots of unity sum to zero** (for $n \geq 2$). Picture n equally spaced arrows from the center of the circle – by symmetry they pull equally in all directions and cancel. (Two arrows: $1 + (-1) = 0$. Three: an equilateral triangle of arrows summing to the center.) Algebraically, $1 + \omega + \cdots + \omega^{n-1}$ is a geometric series equal to $\frac{\omega^n - 1}{\omega - 1} = \frac{1-1}{\omega-1} = 0$, since $\omega^n = 1$ and $\omega \neq 1$. Remember this cancellation; it is the gear that makes the Fourier matrix unitary.

32.6.2 Building the matrix

The $n \times n$ **Fourier matrix** $\mathbf{F} = \mathbf{F}_n$ has, in row j and column k (indexing from 0), the entry

$$(\mathbf{F})_{jk} = \omega^{jk}, \qquad j, k = 0, 1, \ldots, n-1.$$

Row j is the list of powers $1, \omega^j, \omega^{2j}, \ldots, \omega^{(n-1)j}$ – a single pure oscillation sampled at n points, oscillating j times around the circle. Multiplying a signal $\mathbf{x}$ by $\mathbf{F}$ asks, in each row j, "how much of frequency j is in $\mathbf{x}$?" For $n = 4$, with $\omega = e^{2\pi i/4} = i$ (since a quarter turn lands on i), the powers cycle $i^0, i^1, i^2, i^3 = 1, i, -1, -i$, and

$$\mathbf{F}_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}.$$

Each entry is i^{jk} reduced using $i^4 = 1$. The top row is all 1's (frequency zero – it just sums the signal, the average/DC term). The second row $1, i, -1, -i$ is one full rotation; the bottom row is the same rotation run the other way.

Compute it yourself

Compute $\mathbf{F}_4\mathbf{x}$ by hand for $\mathbf{x} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ (a single spike at time 0) and then for $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$ (a constant signal). You should find $\mathbf{F}_4 \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$ – a spike contains *every* frequency equally – and $\mathbf{F}_4 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ – a constant signal is *pure* frequency zero. A spike in time spreads across all frequencies; a flat line in time is a spike in frequency. That trade-off is the seed of the uncertainty principle.

To see why each row deserves to be called "a frequency," picture its real part $\cos(2\pi jm/n)$ as m runs over the n sample points. Row 0 is flat (no oscillation – the DC term), row 1 completes one cycle across the samples, row 2 completes two, and so on. Figure 32.4 traces these sampled cosines for $n = 8$: the Fourier matrix is just "all the pure oscillations that fit on n points," stacked as rows.

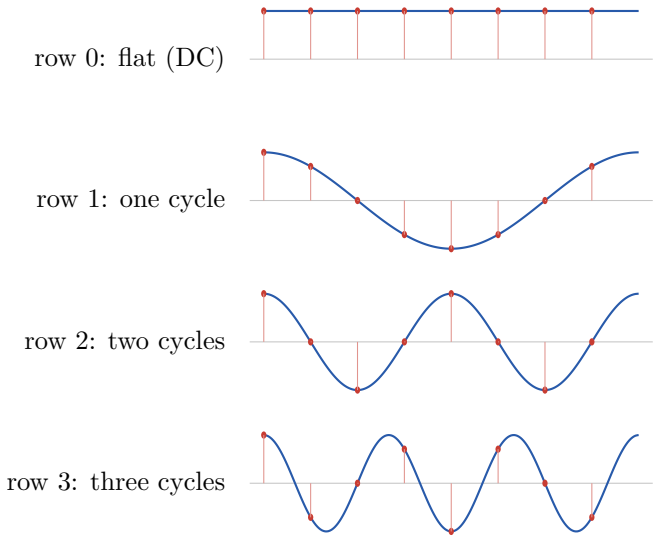


Figure 32.4: The real parts of the first four rows of $\mathbf{F}_8$, read as oscillations sampled (red dots) at the eight points $m = 0, \dots, 7$. Row j completes exactly j full cycles across the samples, so higher rows mean higher frequency. The Fourier matrix bundles every such pure oscillation that fits on n points, and asks how much of each is present in a signal.

32.6.3 Why F is (almost) unitary

Here is the payoff. The columns of $\mathbf{F}$ are *orthogonal* under the Hermitian inner product. Take column j and column k; their inner product conjugates the first (and $\overline{\omega^{mj}} = \omega^{-mj}$, since $\bar{\omega} = \omega^{-1}$ for a unit-circle number):

$$\langle \text{col } j, \text{col } k \rangle = \sum_{m=0}^{n-1} \overline{\omega^{mj}}\, \omega^{mk} = \sum_{m=0}^{n-1} \omega^{m(k-j)} = \sum_{m=0}^{n-1} \left(\omega^{\,k-j}\right)^m.$$

This is a geometric series in ω^{k-j}. If $j = k$ every term is $\omega^0 = 1$, so the sum is n. If $j \neq k$ then ω^{k-j} is itself a nontrivial root of unity, and by our cancellation fact the sum is 0. In one line:

$$\langle \text{col } j, \text{col } k \rangle = \begin{cases} n & j = k, \\ 0 & j \neq k. \end{cases}$$

Different columns are orthogonal; each column has squared length n. So $\mathbf{F}^*\mathbf{F} = n\mathbf{I}$. The columns are orthogonal but have length $\sqrt{n}$, not 1 – so $\mathbf{F}$ itself is not quite unitary. Divide by $\sqrt{n}$ to fix it.

Theorem 32.4. *The* ***normalized Fourier matrix*** $\mathbf{U} = \dfrac{1}{\sqrt{n}}\mathbf{F}$ *is unitary:* $\mathbf{U}^*\mathbf{U} = \mathbf{I}$*, equivalently* $\mathbf{F}^*\mathbf{F} = n\mathbf{I}$ *and* $\mathbf{F}^{-1} = \frac{1}{n}\mathbf{F}^*$.

This is the structural heart of the whole subject. The Fourier transform is a *rotation of* $\mathbb{C}^n$. It does not distort the signal – it merely views the same vector in a new orthonormal basis, the basis of pure frequencies. And because the inverse of a unitary matrix is just its conjugate transpose, the inverse Fourier transform costs no more than the forward one:

$$\mathbf{F}^{-1} = \frac{1}{n}\,\mathbf{F}^*, \qquad \text{so} \qquad \mathbf{x} = \frac{1}{n}\,\mathbf{F}^*(\mathbf{F}\mathbf{x}).$$

You can always get your signal back. Going to the frequency world and coming home are the same operation up to a conjugate and a factor of n.

Example 32.5. For $n = 4$ we have $\mathbf{F}_4^* = \overline{\mathbf{F}_4^{\mathsf{T}}}$. Since $\mathbf{F}_4$ is symmetric ($\omega^{jk} = \omega^{kj}$), transposing changes nothing, and conjugating sends $i \to -i$:

$$\mathbf{F}_4^* = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}, \qquad \tfrac{1}{4}\mathbf{F}_4^*\mathbf{F}_4 = \mathbf{I}.$$

You can verify a single entry: row 0 of $\mathbf{F}_4^*$ dotted with column 0 of $\mathbf{F}_4$ is $1+1+1+1 = 4$, and divided by 4 gives the 1 on the diagonal of $\mathbf{I}$. Row 0 with column 1 is $1 + i - 1 - i = 0$ – the roots-of-unity cancellation, doing its job.

Common trap

Conventions for the Fourier matrix differ across textbooks and software, and they will bite you. Some define $(\mathbf{F})_{jk} = \omega^{-jk}$ (minus sign in the exponent), some fold the $\frac{1}{\sqrt{n}}$ into the forward transform, some into the inverse, some split it as $\frac{1}{\sqrt{n}}$ on each. All are "the Fourier matrix." The invariant facts that survive every convention: the columns are orthogonal roots-of-unity vectors, and the matrix is unitary after the right normalization. Always check which convention your library (NumPy, MATLAB) uses before comparing numbers.

32.7 From the matrix to the FFT

We have a beautiful object; now make it fast. Naively, multiplying $\mathbf{F}_n$ by a vector $\mathbf{x}$ is an $n \times n$ matrix times an n-vector: about n^2 complex multiplications. For a single second of CD-quality audio, $n \approx 44{,}000$, so $n^2 \approx 2$ billion multiplications – per second of sound, for one transform. That was, for a long time, simply too slow to be practical.

The **Fast Fourier Transform** (FFT) computes the very same $\mathbf{F}_n\mathbf{x}$ in about $n \log_2 n$ operations instead of n^2. For $n = 44{,}000$ that is the difference between 2 billion and roughly 700,000 – a speedup of thousands. The idea, in one sentence, is *divide and conquer using the structure of the roots of unity.*

32.7.1 The one identity that makes it work

Split a length-n signal (with n even) into its even- and odd-indexed entries: $\mathbf{x}_{\text{even}} = (x_0, x_2, x_4, \dots)$ and $\mathbf{x}_{\text{odd}} = (x_1, x_3, x_5, \dots)$, each of length $n/2$. The key structural property is that the length-n transform of $\mathbf{x}$ can be assembled from two length-$n/2$ transforms – one of the evens, one of the odds. Writing $\omega_n = e^{2\pi i/n}$ and splitting the sum $(\mathbf{F}_n\mathbf{x})_j = \sum_k x_k \omega_n^{jk}$ into even indices $k = 2m$ and odd indices $k = 2m + 1$,

$$(\mathbf{F}_n\mathbf{x})_j = \sum_{m=0}^{n/2-1} x_{2m}\, \omega_n^{2mj} + \omega_n^{j} \sum_{m=0}^{n/2-1} x_{2m+1}\, \omega_n^{2mj}.$$

Since $\omega_n^2 = \omega_{n/2}$ (going twice as fast around the circle is the same as one step around a half-size circle), each sum is a length-$n/2$ transform. Replacing j by $j + n/2$ leaves $\omega_{n/2}^{m(j+n/2)} = \omega_{n/2}^{mj}$ unchanged, because $\omega_{n/2}^{n/2} = 1$. So

$$(\mathbf{F}_n\mathbf{x})_j \;=\; E_j \;+\; \omega_n^j\, O_j, \qquad (\mathbf{F}_n\mathbf{x})_{j+n/2} \;=\; E_j \;-\; \omega_n^j\, O_j,$$

for $j = 0, \ldots, n/2 - 1$, where $E_j = (\mathbf{F}_{n/2}\mathbf{x}_{\text{even}})_j$ and $O_j = (\mathbf{F}_{n/2}\mathbf{x}_{\text{odd}})_j$ are the half-size transforms (read periodically in j). The two outputs j and $j+n/2$ share the very same E_j, O_j and differ only by a sign on the twiddle term $\omega_n^j O_j$ – the sign flip comes from $\omega_n^{j+n/2} = -\omega_n^j$, since stepping a further half-circle negates a unit-circle number. That paired $+/-$ combine is the famous FFT *butterfly*, and it is why one combine step produces two outputs at once. Two half-size problems plus a cheap combine step. Recurse: each half splits into quarters, and so on, $\log_2 n$ levels deep, each level costing $O(n)$ work to combine. Total: $O(n \log n)$. That is the recursive heart of the algorithm – the roots of unity were *designed* to factor this way. Figure 32.5 draws the recursion tree for $n = 8$.

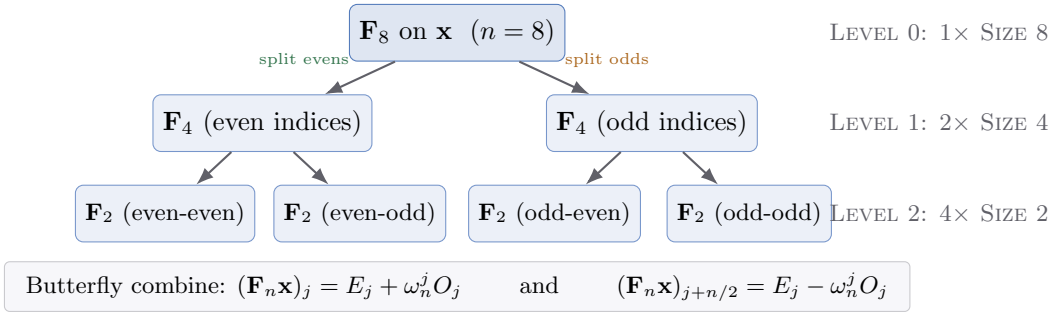


Figure 32.5: The Cooley–Tukey divide-and-conquer tree for $n = 8$. The length-8 DFT recursively splits into even and odd index subsets across $\log_2 8 = 3$ levels. At each step, results are recombined via butterfly arithmetic using twiddle factors ω_n^j, reducing computation from $O(n^2)$ down to $O(n \log n)$.

Modern application

The FFT is, by most counts, the most-executed numerical algorithm on Earth, and it is hiding everywhere:

- **Audio and music.** MP3, AAC, and Shazam all transform sound into frequencies; compression throws away the frequencies your ear cannot hear, and recognition matches a song by its frequency fingerprint.
- **Images.** JPEG splits a photo into 8×8 blocks and applies a cousin of the Fourier transform (the discrete cosine transform), then discards small high-frequency coefficients – that is where the compression comes from.
- **Wireless.** Wi-Fi, 4G/5G, and DSL use OFDM, which sends data on many frequency channels at once; the receiver runs an FFT to pull them apart.
- **Science and ML.** MRI scanners literally measure the Fourier transform of your body and FFT it back to an image; convolutions in signal processing (and fast polynomial multiplication) become cheap pointwise products in the frequency domain, because the Fourier basis diagonalizes convolution.

Every one of these rides on two facts from this chapter: the transform is unitary (so it is

invertible and stable), and it is fast (the FFT).

Connection to other math

The discrete Fourier transform is the finite shadow of the *Fourier series* from analysis, where a periodic function $f(t)$ is written as an infinite sum $\sum_k c_k e^{ikt}$ of pure waves. The functions e^{ikt} are orthonormal under the inner product $\langle f, g\rangle = \frac{1}{2\pi}\int_0^{2\pi} \overline{f(t)}g(t)\,dt$ – the *same* conjugate-inner-product idea from earlier, now with an integral in place of a sum. Sampling such a function at n equally spaced points turns the integral into the finite sum that is our matrix $\mathbf{F}_n$, and "orthonormal waves" becomes "unitary matrix." Hermitian inner products, orthogonal bases of oscillations, completeness – this is the doorway to *Fourier analysis* and *functional analysis*, where vectors become functions and matrices become operators.

32.8 A unified picture

There is an exact symmetry between the real and complex worlds. The translation is mechanical and total: replace every transpose by a conjugate transpose, and every theorem survives.

Concept	**Real ($\mathbb{R}^n$)**	**Complex ($\mathbb{C}^n$)**
Inner product	$\mathbf{u}^{\mathsf{T}}\mathbf{v} = \sum u_i v_i$	$\mathbf{u}^*\mathbf{v} = \sum \bar{u}_i v_i$
"Transpose"	$\mathbf{A}^{\mathsf{T}}$	$\mathbf{A}^*$ (conjugate transpose)
Self-adjoint	symmetric $\mathbf{A}^{\mathsf{T}} = \mathbf{A}$	Hermitian $\mathbf{A}^* = \mathbf{A}$
eigenvalues	real	real
Length-preserving	orthogonal $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$	unitary $\mathbf{U}^*\mathbf{U} = \mathbf{I}$
eigenvalues	on unit circle (± 1 if real)	on unit circle, $\lvert\lambda\rvert = 1$
Spectral theorem	$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^{\mathsf{T}}$	$\mathbf{A} = \mathbf{U}\Lambda\mathbf{U}^*$

Focus

If you remember one thing from this chapter, remember the dictionary above. Complex linear algebra parallels real linear algebra once the complex conjugate is inserted on one factor of every inner product, and with the transpose upgraded to the conjugate transpose to match. Everything else, including the Spectral Theorem and the geometry of rigid motions, carries over word for word.

Pause

Cost and intuition recap. The conjugate inner product $\langle \mathbf{u}, \mathbf{v}\rangle = \sum \bar{u}_k v_k$ costs the same n multiplications as the real dot product – one extra conjugation per entry, no asymptotic change – and it is the one repair that keeps $\|\mathbf{z}\|^2$ real and nonnegative. A Hermitian matrix gives *real* eigenvalues; a unitary matrix preserves length and gives eigenvalues of modulus one. The dense Fourier matrix-vector product costs $O(n^2)$; the FFT exploits the roots-of-unity structure to bring it down to $O(n \log n)$, which makes large-scale frequency-domain computation practical. One mantra carries the chapter: *star replaces transpose, and the conjugate goes on exactly one side.*

Why this matters

The Fourier matrix combines three central ideas of this book. It is a *change of basis*: from samples to frequencies. It is a *unitary matrix*: a rigid rotation of $\mathbb{C}^n$ that loses no information. And it *diagonalizes* every circular convolution at once, so the hardest operation in signal processing becomes pointwise multiplication. One matrix, built from n points spinning evenly around a circle, sits at the intersection of geometry, algebra, and computation – and it runs, billions of times a second, inside the device you are reading this on.

32.9 Unitary and Hermitian Matrices as the Language of Quantum Mechanics

Throughout this chapter, two matrix families have dominated the algebra: Hermitian matrices ($\mathbf{A}^* = \mathbf{A}$), whose eigenvalues are always real, and unitary matrices ($\mathbf{U}^*\mathbf{U} = \mathbf{I}$), whose columns form orthonormal frames that preserve every complex length and angle.

To a pure mathematician, these are the natural complex generalizations of symmetric and orthogonal matrices. To a physicist or a computer scientist, they are something much more fundamental: they form the complete mathematical grammar of quantum mechanics and quantum computation. Unlike classical mechanics, which describes particles by real positions and velocities, quantum mechanics represents states and measurements through complex vectors and linear operators on $\mathbb{C}^n$.

32.9.1 Quantum states as unit vectors in $\mathbb{C}^n$

In classical digital electronics, a bit is strictly 0 or 1. In quantum information, the state of the fundamental unit of storage—a **qubit**—is a nonzero vector in $\mathbb{C}^2$. Physicists write vectors using Paul Dirac's bra-ket notation. A column vector is called a "ket" and written $|\psi\rangle$. The standard basis vectors of $\mathbb{C}^2$ are labeled

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

An arbitrary qubit state is a linear combination of these basis vectors:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \qquad \alpha, \beta \in \mathbb{C}.$$

The complex scalars α and β are called **probability amplitudes**.

The conjugate transpose of the column vector $|\psi\rangle$ is the row vector $\langle\psi| = |\psi\rangle^* = \begin{bmatrix} \bar{\alpha} & \bar{\beta} \end{bmatrix}$, called a "bra." When we take the Hermitian inner product $\langle \phi, \psi \rangle = \phi^*\psi$ from Definition 32.1, Dirac's notation snaps together as $\langle \phi \mid \psi \rangle$—a "bra" followed by a "ket" forming a closed bracket.

The physical meaning of amplitudes is given by the **Born rule**: when a qubit in state $|\psi\rangle$ is measured, the system collapses to outcome 0 with probability $|\alpha|^2$ and to outcome 1 with probability $|\beta|^2$. Because the total probability of all mutually exclusive outcomes must equal 1, every physical state must be normalized to unit length:

$$\||\psi\rangle\|^2 = \langle \psi \mid \psi \rangle = |\alpha|^2 + |\beta|^2 = 1.$$

A single qubit is a unit vector on the unit sphere in $\mathbb{C}^2$. When we combine n qubits, the state vector lives in $\mathbb{C}^{2^n}$. Each of the 2^n classical bitstrings corresponds to a standard basis vector, and the register state is a unit vector whose 2^n complex components specify the amplitudes of each possible configuration.

32.9.2 Quantum gates as unitary matrices

If an isolated quantum system begins in a state of length 1, any physical transformation must maintain that length. If a transformation is represented by a linear operator $\mathbf{U}$, we require $\|\mathbf{U}|\psi\rangle\| = \||\psi\rangle\| = 1$ for every state vector $|\psi\rangle$.

By Theorem 32.3, the linear transformations that preserve complex vector norms are precisely the **unitary matrices**: $\mathbf{U}^*\mathbf{U} = \mathbf{I}$. Every valid quantum gate or time evolution in an isolated quantum computer is an isometry of complex space.

Three single-qubit unitary matrices appear constantly:

- The **Pauli-X gate** (the quantum NOT):

$$\mathbf{X} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{X}|0\rangle = |1\rangle, \quad \mathbf{X}|1\rangle = |0\rangle.$$

  It interchanges the two basis vectors, rotating or reflecting the basis.

- The **Pauli-Z gate** (the phase flip):

$$\mathbf{Z} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad \mathbf{Z}|0\rangle = |0\rangle, \quad \mathbf{Z}|1\rangle = -|1\rangle.$$

  It leaves $|0\rangle$ unaffected and negates the amplitude of $|1\rangle$.

- The **Hadamard gate H**:

$$\mathbf{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Because $\mathbf{H}$ has real symmetric entries, $\mathbf{H}^* = \mathbf{H}$, and its columns are orthonormal, $\mathbf{H}^*\mathbf{H} = \mathbf{I}$. The Hadamard gate is simultaneously unitary and Hermitian. When $\mathbf{H}$ acts on the standard basis vector $|0\rangle$, it creates an equal superposition:

$$\mathbf{H}|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle.$$

Unitary matrices preserve inner products: $\langle \mathbf{Uu}, \mathbf{Uv} \rangle = \langle \mathbf{u}, \mathbf{v} \rangle$. Two orthogonal states remain orthogonal after any unitary operation. Because every unitary matrix satisfies $\mathbf{U}^{-1} = \mathbf{U}^*$, every quantum gate is entirely reversible: applying the conjugate transpose $\mathbf{U}^*$ runs the gate backwards, restoring the original state with zero loss of information.

32.9.3 Physical observables as Hermitian matrices

When an experimentalist measures a physical quantity—energy, position, or spin—the instrument's display always shows a **real number**. A measurement in a physics laboratory never yields an outcome of $2 + 5i$ electronvolts.

This physical requirement explains why every measurable physical observable is represented by a **Hermitian matrix** $\mathbf{A}^* = \mathbf{A}$. The properties proven in Theorem 32.1 and Theorem 32.2 provide the exact physical interpretation:

1. **Real eigenvalues as observable values:** The eigenvalues $\lambda_1, \dots, \lambda_n$ of $\mathbf{A}$ are strictly real. These numbers are the only values the detector can report.

2. **Orthonormal eigenvectors as definite states:** By the Spectral Theorem, $\mathbf{A}$ has an orthonormal basis of eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ with $\mathbf{A}\mathbf{v}_k = \lambda_k \mathbf{v}_k$. A system in state $\mathbf{v}_k$ is guaranteed to yield the value λ_k with 100% certainty.

3. **Measurement collapse and projection:** Writing the eigenvectors as kets $|v_k\rangle = \mathbf{v}_k$, expand an arbitrary state $|\psi\rangle = \sum_{k=1}^{n} c_k |v_k\rangle$ with $c_k = \langle v_k \mid \psi \rangle$, measuring the observable $\mathbf{A}$ forces the state to collapse into one of the eigenvectors $\mathbf{v}_k$. The probability of this collapse is $|c_k|^2 = |\langle v_k \mid \psi \rangle|^2$.

4. **Real expectation values:** Over many identical trials, the average observed value is given by the Hermitian quadratic form

$$\langle \mathbf{A} \rangle = \langle \psi \mid \mathbf{A}\psi \rangle = |\psi\rangle^* \mathbf{A} |\psi\rangle = \sum_{k=1}^{n} \lambda_k |\langle v_k \mid \psi \rangle|^2.$$

   Because $\mathbf{A}^* = \mathbf{A}$, this expectation value is mathematically guaranteed to be real: $(|\psi\rangle^* \mathbf{A} |\psi\rangle)^* = |\psi\rangle^* \mathbf{A}^* |\psi\rangle = |\psi\rangle^* \mathbf{A} |\psi\rangle \in \mathbb{R}$.

The dictionary between linear algebra and quantum mechanics is exact: complex unit vectors represent physical states, unitary matrices represent time evolutions and computational gates, and Hermitian matrices represent observable quantities.

In Chapter 42 of the companion computer science volume (*Algorithms From the Ground Up*), we turn these unitary matrices into algorithms. We will see how quantum circuits arrange interference so that paths leading to incorrect answers cancel out, while paths leading to correct answers reinforce, enabling polynomial-time integer factorization and quadratic search.

32.10 Power and limits

Conjugating one factor of the inner product preserves the entire geometric theory over $\mathbb{C}$. We set out to give complex vectors a metric geometry. In doing so, we preserved positive lengths ($\|\mathbf{z}\|^2 > 0$ for all $\mathbf{z} \neq \mathbf{0}$), discovered that Hermitian matrices ($\mathbf{A}^* = \mathbf{A}$) behave like real symmetric matrices with strictly real eigenvalues, and proved that unitary matrices ($\mathbf{U}^*\mathbf{U} = \mathbf{I}$) preserve every length and angle in $\mathbb{C}^n$. That framework brought us the discrete Fourier transform, viewing signals as combinations of pure frequencies, and gave us the mathematical language governing quantum states, gates, and physical measurements.

Yet mathematical models always meet practical realities. The Fourier basis vectors are sinusoidal waves spanning the entire duration of a signal. If an audio recording contains a single sharp pop, or an image contains a sharp vertical edge, that localized spike spatters energy across every frequency coefficient in the transform. The standard Fourier matrix tells you which notes were sounded, but nothing about *when* they were played. Bridging that gap requires windowed Fourier transforms and wavelets, which localize waves in both time and frequency.

There is a second challenge that strikes closer to linear algebra. On paper, every formula in this chapter is exact. In a physical computer, however, numbers are represented with finite floating-point bits. Unitary transforms like the normalized FFT cope well, because a length-preserving map never amplifies rounding error; most matrices are not unitary, and solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ with them can lose many digits.

Up to this point, we have assumed that arithmetic is exact. But every practical computation runs on finite binary machines. In Chapter 33, *Numerical Linear Algebra*, we investigate condition numbers, floating-point stability, and the matrix factorizations that make scientific computing dependable.

32.11 Exercises

Exercise 32.1. Let $\mathbf{u} = \begin{bmatrix} 2i \\ 1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1+i \\ 3i \end{bmatrix}$ in $\mathbb{C}^2$. Compute $\langle \mathbf{u}, \mathbf{v} \rangle$ and $\langle \mathbf{v}, \mathbf{u} \rangle$, and verify that $\langle \mathbf{v}, \mathbf{u} \rangle = \overline{\langle \mathbf{u}, \mathbf{v} \rangle}$. Then compute $\|\mathbf{u}\|$ and $\|\mathbf{v}\|$.

Exercise 32.2. Find a nonzero vector $\mathbf{z} \in \mathbb{C}^2$ with $\sum z_k^2 = 0$ but $\|\mathbf{z}\|^2 = \sum |z_k|^2 \neq 0$. Explain in one sentence why this shows the naive "dot product" $\sum z_k^2$ cannot define a length.

Exercise 32.3. For $\mathbf{A} = \begin{bmatrix} 2 & i \\ -i & 4 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 1 & 1+i \\ 1-i & 0 \end{bmatrix}$, compute $\mathbf{A}^*$ and $\mathbf{B}^*$. Which of $\mathbf{A}, \mathbf{B}$ are Hermitian? For the Hermitian one(s), confirm by inspection that the diagonal is real and the off-diagonal entries are conjugates.

Exercise 32.4. The matrix $\mathbf{A} = \begin{bmatrix} 3 & 2i \\ -2i & 3 \end{bmatrix}$ is Hermitian. Find its eigenvalues (they should come out real, by Theorem 32.1) and a pair of orthogonal eigenvectors. Verify the orthogonality with the Hermitian inner product.

Exercise 32.5. Show that $\mathbf{Q} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ is unitary for every real θ (it is even orthogonal). Then show that $\mathbf{V} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & i \\ i & 1 \end{bmatrix}$ is unitary, and find a vector whose length it preserves to double-check Theorem 32.3.

Exercise 32.6. Using $\omega = e^{2\pi i/6}$, list the six sixth-roots of unity in the form $e^{2\pi i k/6}$ and plot them on the unit circle. Verify (by the geometric-series argument, or numerically) that they sum to zero. What is ω^3?

Exercise 32.7. Using $\mathbf{F}_4$ from the text, compute $\mathbf{F}_4\mathbf{x}$ for $\mathbf{x} = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}$. Which single frequency does the answer pick out, and why does that match the fact that $\mathbf{x}$ alternates sign every step (the *highest* frequency representable on 4 points)?

Exercise 32.8. Verify directly that $\frac{1}{4}\mathbf{F}_4^*\mathbf{F}_4 = \mathbf{I}$ by computing the inner product of column 1 with column 1 (should be 4) and column 1 with column 3 (should be 0) of $\mathbf{F}_4$. Identify the roots-of-unity cancellation in the second computation.

Exercise 32.9. (Proof.) Prove that unitary similarity preserves eigenvalues: if $\mathbf{U}$ is unitary and $\mathbf{A} = \mathbf{U}^*\mathbf{A}_0\mathbf{U}$, then $\mathbf{A}$ and $\mathbf{A}_0$ have the same characteristic polynomial, hence the same eigenvalues. Then prove the special case used in the chapter: if $\mathbf{A}_0$ is Hermitian, so is $\mathbf{A}$.

Exercise 32.10. (Challenge.) A matrix is called **normal** if $\mathbf{A}^*\mathbf{A} = \mathbf{A}\mathbf{A}^*$. Verify that every Hermitian matrix and every unitary matrix is normal. Then explain, in a sentence or two, why "normal" is exactly the class of matrices to which the complex Spectral Theorem applies (eigenvectors form an orthonormal basis) – making Hermitian and unitary two special cases of one bigger truth.

Exercise 32.11. (FFT counting.) Let $T(n)$ be the number of combine operations the FFT uses on a signal of length $n = 2^m$. The divide-and-conquer step gives roughly $T(n) = 2\,T(n/2) + n$ with $T(1) = 0$. Unroll this recurrence to show $T(n) = n \log_2 n$, and compare $T(1024)$ with the naive 1024^2.

Exercise 32.12. (Hadamard gate as an observable and a gate.) The Hadamard matrix $\mathbf{H} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ is both unitary ($\mathbf{H}^*\mathbf{H} = \mathbf{I}$) and Hermitian ($\mathbf{H}^* = \mathbf{H}$). Find its eigenvalues and verify they are real. Find a pair of orthonormal eigenvectors. Then, for the state $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, compute the expectation value $\langle \mathbf{H} \rangle = |0\rangle^* \mathbf{H} |0\rangle$.

Part VIII

Linear Algebra in the Modern World

Chapter 33

Numerical Linear Algebra

In Chapter 32, we met unitary transforms like the Fourier matrix, where exact algebra yielded fast algorithms. But every formula in this book has shared a quiet assumption: that arithmetic is exact.

We wrote ${}^{-1}\mathbf{A}$, we wrote $\sqrt{2}$, we wrote $1/3$, and we trusted those symbols to mean precisely what they say. A real computer does not get that luxury. It stores numbers in a fixed number of bits, so $1/3$ becomes $0.3333333\ldots$ chopped off somewhere, $\sqrt{2}$ becomes a nearby fraction, and every arithmetic step introduces a tiny rounding error. Most of the time those errors stay harmless. Sometimes, however, they accumulate and amplify until the computed answer has no accurate digits left.

Numerical linear algebra is the study of how to do linear algebra on a machine *and trust the result*. It splits into two questions that we will keep carefully apart:

- **Conditioning** is a property of the *problem*. Some problems are inherently delicate: tiny changes to the input force large changes in the true answer. No algorithm, however clever, can rescue an ill-conditioned problem, because the input itself is already uncertain.

- **Stability** is a property of the *algorithm*. A stable algorithm does not add much trouble of its own; an unstable one can wreck even a well-conditioned problem through its own internal rounding.

Hold those two words apart and most of the confusion in this field evaporates. The condition number measures the problem. Stability measures the method. You want both: a well-conditioned problem solved by a stable algorithm.

We will build the floating-point picture, define the condition number geometrically, see why Gaussian elimination needs pivoting and why QR provides numerical stability, then turn to iterative methods – Jacobi, Gauss–Seidel, and the conjugate gradient idea – that solve enormous sparse systems without ever forming an inverse. We close with the power method, the simplest eigenvalue algorithm, and a page of practical warnings you will be glad to have.

33.1 Floating point: numbers with a budget

A computer stores a real number the way scientific notation does, but in binary and with a fixed budget of digits. A double-precision ("float64") number looks like

$$\pm 1.\underbrace{b_1 b_2 \cdots b_{52}}_{52 \text{ bits}} \times 2^e,$$

a sign, a 52-bit fraction (the *mantissa*, which records the significant bits of the number after the normalized leading one), and an exponent e. Because the mantissa has only 52 bits, the gap between one representable number and the next is relative, not absolute: near 1 the spacing is about $2^{-52} \approx 2.2 \times 10^{-16}$, and near 1000 it is a thousand times larger. The single most important constant in the subject is this spacing at 1, called **machine epsilon**:

$$\varepsilon_{\text{mach}} \;=\; 2^{-52} \;\approx\; 2.22 \times 10^{-16}.$$

Machine epsilon is the gap between 1.0 and the very next number the computer can represent. Anything smaller than half of $\varepsilon_{\text{mach}}$ added to 1.0 rounds right back to 1.0.

Intuition

Think of the real line as a ruler whose tick marks get *farther apart* as you move away from zero, but always stay the same distance apart *relative* to where you are – about 16 significant decimal digits everywhere. When you store a number, you snap it to the nearest tick. The snap costs at most half a tick, so the relative error of storing any number x is bounded:

$$\frac{|\text{fl}(x) - x|}{|x|} \;\le\; \tfrac{1}{2}\,\varepsilon_{\text{mach}}.$$

Here $\text{fl}(x)$ means "x rounded to the nearest float." Sixteen digits sounds like plenty. The danger is never one rounding – it is what happens when millions of them interact.

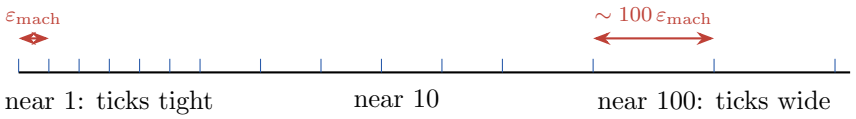


Figure 33.1: Floating-point numbers are a ruler whose tick marks spread apart as you move from zero, but stay a constant *relative* distance apart – about 16 significant digits everywhere. Storing a real number snaps it to the nearest tick, costing at most half a gap. The absolute gap near 100 is about 100 times the gap near 1.

The IEEE 754 standard guarantees that the four basic operations are *correctly rounded*: the computed $x \oplus y$ equals the true $x + y$ rounded to the nearest float. Symbolically, every operation introduces a tiny relative slip,

$$\text{fl}(x \circ y) \;=\; (x \circ y)(1 + \delta), \qquad |\delta| \le \tfrac{1}{2}\varepsilon_{\text{mach}},$$

for each of $\circ \in \{+, -, \times, \div\}$. That little $(1 + \delta)$ is the atom of all rounding analysis. Stack billions of them and you want them to stay small and *not* reinforce.

33.1.1 The cardinal sin: catastrophic cancellation

Multiplication and division are gentle: they perturb the relative error by one δ and move on. Addition is gentle too, *unless you subtract two nearly equal numbers*. That is where significant digits are lost.

Example 33.1 (Subtracting close numbers)**.** Suppose a machine keeps only 6 significant decimal digits, and you compute the difference between two quantities whose true values are

$$a = 1.234567, \qquad b = 1.234561.$$

Before subtracting, the machine must round each number to its 6-digit budget. Assuming standard round-to-nearest, a rounds up to $\mathrm{fl}(a) = 1.23457$, while b rounds down to $\mathrm{fl}(b) = 1.23456$. The machine then subtracts the two stored numbers:

$$\mathrm{fl}(a) - \mathrm{fl}(b) = 1.23457 - 1.23456 = 0.000010 = 1.00000 \times 10^{-5}.$$

The machine confidently displays six digits. But what is the true difference?

$$a - b = 1.234567 - 1.234561 = 0.000006 = 6.00000 \times 10^{-6}.$$

The computed result is off by nearly 70%. Notice where the disaster happened. The subtraction itself was performed with zero arithmetic error: $1.23457 - 1.23456$ really is 0.00001. The problem is that the five leading digits 1.2345 were identical and subtracted to zero. That cancellation wiped out the real signal and promoted the microscopic rounding noise in the sixth digit into the single leading digit of the final answer.

This is **catastrophic cancellation** – the phenomenon where subtracting two nearly equal numbers wipes out their shared leading digits and leaves only the amplified rounding noise behind. It is the mechanism behind most severe losses of precision. The subtraction itself is computed correctly; the problem is that it strips away the mask that hid the prior rounding errors in a and b. The lesson, which applies equally to elimination and least squares: *avoid subtracting quantities that are nearly equal whenever an algebraically equivalent route avoids it.*

Compute it yourself

The quadratic formula $x = \dfrac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ hides this trap. Take $a = 1$, $b = -200$, $c = 1$. Then $\sqrt{b^2 - 4ac} = \sqrt{40000 - 4} = \sqrt{39996} \approx 199.98999\ldots$. The "+" root, $\frac{200 + 199.98999}{2} \approx 199.995$, is fine. But the "−" root computes $\frac{200 - 199.98999}{2}$ – a subtraction of two nearly equal numbers – and loses most of its digits. The cure is to compute the safe root first, then get the other from $x_1 x_2 = c/a$, here $x_2 = 1/x_1$. Try both ways on a calculator that shows few digits and watch the difference.

Common trap

"Double precision has 16 digits, so I have 16 good digits." No. You *start* with 16 digits. Each catastrophic cancellation can erase a clump of them at once. A computation that subtracts two numbers agreeing in their first 12 digits keeps about 4. The bits are not lost in the subtraction – they were never there to begin with; the subtraction just exposes the lost precision.

Pause

Pause and think: Suppose you evaluate $f(x) = \sqrt{x+1} - \sqrt{x}$ when $x = 10^8$. In 16-digit floating point, $\sqrt{10^8+1} \approx 10000.00005$ and $\sqrt{10^8} = 10000$. Subtracting them directly throws away half your significant digits. How can high-school algebra save the computation? *Hint:* Multiply and divide by the conjugate $\sqrt{x+1}+\sqrt{x}$ to rewrite $f(x) = \frac{1}{\sqrt{x+1}+\sqrt{x}}$. Adding large numbers is completely benign; subtraction was the culprit.

33.2 Conditioning: how delicate is the problem?

Forget computers for a moment. Suppose someone hands you a linear system $\mathbf{Ax} = \mathbf{b}$ and warns you that $\mathbf{b}$ is only known to within a little wiggle $\Delta\mathbf{b}$ – a measurement error, say. How much can the solution $\mathbf{x}$ wiggle in response? If a small $\Delta\mathbf{b}$ can force a large $\Delta\mathbf{x}$, the problem is **ill-conditioned**, and *no* algorithm can save you: the answer genuinely is uncertain.

Geometrically, solving $\mathbf{Ax} = \mathbf{b}$ in $\mathbb{R}^2$ means intersecting two lines (the two equations). If the lines cross briskly at a wide angle, nudging one line barely moves the crossing point. But if the two lines are *nearly parallel*, the intersection point is a long, skinny wedge: a hair's nudge to either line slides the crossing point a huge distance (Figure 33.2). That near-parallel picture is exactly what ill-conditioning looks like.

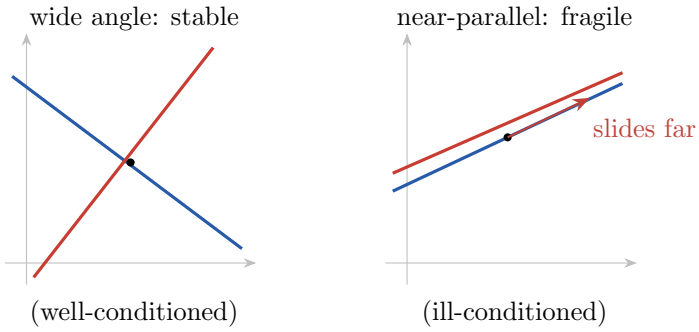


Figure 33.2: Two equations are two lines. When they cross at a wide angle (left), the solution is robust to perturbing either line. When they are nearly parallel (right), a tiny change to a line slides the intersection a long way – the hallmark of an ill-conditioned system.

33.2.1 The condition number, made precise

Let us put a number on that delicacy. Start from $\mathbf{Ax} = \mathbf{b}$ and perturb the right side to $\mathbf{b} + \Delta\mathbf{b}$, giving a new solution $\mathbf{x} + \Delta\mathbf{x}$. Subtracting the two systems, $\mathbf{A}\,\Delta\mathbf{x} = \Delta\mathbf{b}$, so $\Delta\mathbf{x} = {}^{-1}\mathbf{A}\,\Delta\mathbf{b}$. Taking norms and using the operator-norm inequality $\|\mathbf{Mv}\| \le \|\mathbf{M}\|\|\mathbf{v}\|$ (which says a matrix can never stretch any vector by more than the matrix's own maximum magnification factor):

$$\|\Delta\mathbf{x}\| \le \|{}^{-1}\mathbf{A}\|\,\|\Delta\mathbf{b}\|, \qquad \text{and from } \|\mathbf{b}\| \le \|\mathbf{A}\|\,\|\mathbf{x}\|, \quad \frac{1}{\|\mathbf{x}\|} \le \frac{\|\mathbf{A}\|}{\|\mathbf{b}\|} \quad (\mathbf{b} \neq \mathbf{0}).$$

Multiplying these two inequalities between nonnegative numbers gives the relative error bound:

$$\boxed{\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \underbrace{\|\mathbf{A}\| \, \|^{-1}\mathbf{A}\|}_{\kappa(\mathbf{A})} \cdot \frac{\|\Delta \mathbf{b}\|}{\|\mathbf{b}\|}.}$$

The amplification factor in the middle is the **condition number** $\kappa(\mathbf{A}) = \|\mathbf{A}\| \, \|^{-1}\mathbf{A}\|$. It measures how much a relative wobble in the input data can be multiplied into a relative error in the output.

Formal

For the 2-norm (the standard Euclidean length), the matrix norm $\|\mathbf{A}\|$ equals the largest singular value $\sigma_{\max}$, and $\|^{-1}\mathbf{A}\| = 1/\sigma_{\min}$. Recall that singular values represent the lengths of the principal semi-axes when $\mathbf{A}$ transforms the unit sphere into an ellipsoid. Thus:

$$\kappa_2(\mathbf{A}) \;=\; \frac{\sigma_{\max}}{\sigma_{\min}},$$

the ratio of the largest to the smallest singular value of $\mathbf{A}$. A condition number near 1 is ideal (orthogonal matrices have $\kappa_2 = 1$ exactly, preserving all lengths). A condition number of 10^k means you can expect to lose about k decimal digits of accuracy: if $\kappa_2(\mathbf{A}) \approx 10^8$ and your machine begins with 16 digits, you can trust only about $16 - 8 = 8$ digits of the computed solution.

Connection to other math

The condition number is the geometric aspect ratio revealed by the SVD. Recall from Chapter 29 that $\mathbf{A}$ maps the unit sphere into an ellipsoid whose semi-axes are the singular values $\sigma_1 \geq \cdots \geq \sigma_n$. The condition number $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ is simply the *aspect ratio* of that ellipsoid (Figure 33.3). A round sphere (aspect ratio near 1) means $\mathbf{A}$ stretches every direction about equally, and reversing it is benign. But when $\sigma_{\max} \gg \sigma_{\min}$, $\mathbf{A}$ squashes the unit sphere into an elongated needle. When you solve $\mathbf{A}\mathbf{x} = \mathbf{b}$, the inverse $^{-1}\mathbf{A}$ must stretch that needle back into a round sphere – which means any microscopic disturbance or floating-point wobble along the needle's skinny waist is magnified by the enormous factor $1/\sigma_{\min}$. Conditioning, geometry, and the SVD are one unified idea.

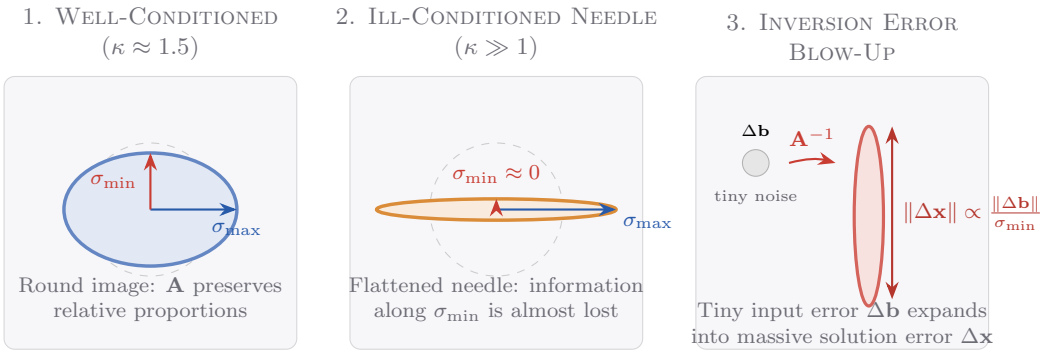


Figure 33.3: The condition number as geometric distortion and error magnification. Left: When $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ is small, $\mathbf{A}$ deforms the unit sphere into a nearly round ellipse, making inversion stable. Middle: When $\kappa_2(\mathbf{A})$ is large, $\mathbf{A}$ squashes the unit sphere into an elongated needle. Right: Inverting the system $\mathbf{Ax} = \mathbf{b}$ requires $^{-1}\mathbf{A}$ to expand the needle back out, magnifying any tiny perturbation $\Delta\mathbf{b}$ perpendicular to the needle's axis by $1/\sigma_{\min} = \kappa_2(\mathbf{A})/\sigma_{\max}$.

Example 33.2 (A mildly ill-conditioned 2×2)**.** Take

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1.001 \end{bmatrix}.$$

The two rows are *almost* the same – two nearly parallel lines. Its determinant is $1 \cdot 1.001 - 1 \cdot 1 = 0.001$, tiny, so the inverse is large:

$$^{-1}\mathbf{A} = \frac{1}{0.001} \begin{bmatrix} 1.001 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1001 & -1000 \\ -1000 & 1000 \end{bmatrix}.$$

Now watch the amplification directly. With $\mathbf{b} = \begin{bmatrix} 2 \\ 2.001 \end{bmatrix}$ the exact solution is $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Perturb the second entry of $\mathbf{b}$ by just 0.001 to $\mathbf{b}' = \begin{bmatrix} 2 \\ 2.002 \end{bmatrix}$:

$$\mathbf{x}' = ^{-1}\mathbf{Ab}' = \begin{bmatrix} 1001 \cdot 2 - 1000 \cdot 2.002 \\ -1000 \cdot 2 + 1000 \cdot 2.002 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

A 0.05% change in $\mathbf{b}$ flipped the answer from $(1, 1)$ to $(0, 2)$ – a 100% change in $\mathbf{x}$. The condition number here is roughly $\kappa_2(\mathbf{A}) \approx 4000$, and indeed the relative error was amplified by a few thousand – right up against that worst-case bound.

Pause

Pause and think: Suppose a physical system models heat diffusion, and its discretized matrix $\mathbf{A}$ has singular values $\sigma_{\max} = 100$ and $\sigma_{\min} = 10^{-4}$. What is $\kappa_2(\mathbf{A})$? If you solve $\mathbf{Ax} = \mathbf{b}$ using standard double-precision arithmetic (16 decimal digits), how many significant digits can you confidently trust in the solution vector $\mathbf{x}$? *Answer:* $\kappa_2(\mathbf{A}) = 100/10^{-4} = 10^6$. Since $\kappa_2 \approx 10^6$, input uncertainties or internal roundings can be amplified by up to a factor of one million, wiping out 6 decimal digits of precision. Out of your initial 16 digits, you should expect to retain about $16 - 6 = 10$ trustworthy digits.

Common trap

A small determinant does *not* mean ill-conditioned, and a large one does not mean well-conditioned – the determinant has the wrong units. Scale $\mathbf{A}$ by $\frac{1}{10}$ and its determinant drops by 10^n, yet $\kappa(\mathbf{A})$ does not change at all, because both $\|\mathbf{A}\|$ and $\|^{-1}\mathbf{A}\|$ rescale and the factors cancel. Conditioning is about the *ratio* of stretches $\sigma_{\max}/\sigma_{\min}$, not the overall volume det. Judge sensitivity by κ, never by det.

Walking question

The Hilbert matrix has entries $H_{ij} = 1/(i+j-1)$ – innocent-looking fractions like $1, \frac{1}{2}, \frac{1}{3}, \ldots$ on the diagonals. Its condition number explodes: about 500 for 3×3, roughly half a million for 5×5, about 1.6×10^{13} by 10×10, and past 10^{16} – beyond all double-precision hope – a few sizes later. On a walk, ask yourself why "smooth, slowly varying" matrices like this – rows that look nearly proportional – should be the dangerous ones, while a matrix of random ± 1 entries is usually tame. (Hint: think about how parallel the rows are.)

33.3 Stability: elimination versus QR

The condition number is about the problem. Now we ask about the *algorithm*. Even a perfectly well-conditioned system can be wrecked by a careless method. The classic example is Gaussian elimination without pivoting.

History and connection

In 1947, as the electronic computing era dawned with the room-sized ENIAC (a machine packed with eighteen thousand glowing vacuum tubes that could add numbers in fractions of a millisecond), John von Neumann and Herman Goldstine published a landmark paper: *Numerical Inverting of Matrices of High Order*. Before digital mainframes, solving linear systems was done by rooms of human "computers" using mechanical desk calculators; factoring even a 10×10 matrix took weeks of grueling, error-prone hand labor. Digital computers promised to solve 100×100 systems in minutes.

Yet many mathematicians doubted whether large systems could be solved accurately at all. Leading pioneers, most notably Norbert Wiener, predicted that digital computers would prove useless for large linear systems. The fear was that the tiny rounding error introduced at each elimination step would double and cascade from row to row, growing exponentially (2^n) and erasing significant digits after just twenty or thirty equations. Von Neumann and Goldstine introduced the matrix condition number $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|^{-1}\mathbf{A}\|$ to bound error growth, but their early theoretical estimates were pessimistic. Alan Turing, working on the experimental Pilot ACE computer in Britain, pushed back, suspecting that managing the size of pivots would keep the errors contained.

The breakthrough came in the 1960s from James H. Wilkinson, Turing's former collaborator at the National Physical Laboratory. Wilkinson invented **backward error analysis**—the idea of asking not how far the machine's computed solution $\hat{\mathbf{x}}$ is from the true solution $\mathbf{x}$, but rather: *what slightly perturbed problem did the machine solve exactly?* Wilkinson proved that Gaussian elimination with partial pivoting (swapping rows to place the largest available entry

on the diagonal) solves $(\mathbf{A}+\mathbf{E})\hat{\mathbf{x}} = \mathbf{b}$, where the error matrix $\mathbf{E}$ is microscopic, on the order of machine precision $\varepsilon_{\text{mach}}$. Elimination does not invent wild errors; it merely reveals the delicate sensitivity already inherent in the matrix. Wilkinson's insight transformed scientific computing from an anxious art into a rigorous science, giving birth to the foundational LINPACK and LAPACK libraries that underpin scientific computing today.

Yet a mystery remains open. In theory, Gaussian elimination with partial pivoting has a pathological worst-case error growth factor of 2^{n-1} (demonstrated by artificially constructed matrices with cascading signs). In theory, solving a 100×100 system could still destroy all accuracy. But in over seventy years of practical computation across millions of real-world scientific problems, this worst-case exponential blow-up has virtually never occurred in nature; the growth factor in practice almost never exceeds $n^{2/3}$. Explaining why partial pivoting is so reliably stable on typical matrices—a question explored today through **smoothed analysis** (measuring algorithmic behavior under small random wiggles of the inputs) and random matrix theory—remains a lively, active frontier in mathematics.

33.3.1 Why elimination needs pivoting

Recall elimination subtracts a multiple of one row from another to create zeros. The multiplier is m = (entry to kill)/(pivot). If the pivot is tiny, m is enormous, and multiplying a row by an enormous number, then subtracting, drowns the original information in roundoff.

Example 33.3 (A tiny pivot ruins a benign system)**.** Solve, on a machine carrying only 3 significant digits,

$$\begin{bmatrix} 0.0001 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

This system is perfectly well-conditioned; the true answer is about $x_1 = 1.0001\ldots$, $x_2 = 0.9999\ldots$, which rounds to $(1.00, 1.00)$ to three digits.

Without pivoting, the pivot is 0.0001 and the multiplier is $m = 1/0.0001 = 10000$. Eliminating the second row:

$$\text{row}_2 \leftarrow \text{row}_2 - 10000 \cdot \text{row}_1: \quad (1 - 10000 \cdot 0.0001)\, x_1 + (1 - 10000 \cdot 1)\, x_2 = 2 - 10000 \cdot 1.$$

The new x_2 coefficient is $1 - 10000 = -9999$, and the right side is $2 - 10000 = -9998$. On a machine with only 3 significant digits, -9999 rounds to $-1.00 \times 10^4 = -10000$, and -9998 rounds to $-1.00 \times 10^4 = -10000$. The original "1" and "2" are completely lost in the roundoff. The machine then computes:

$$x_2 = \frac{-10000}{-10000} = 1.00,$$

and back-substitution gives

$$x_1 = \frac{1 - 1 \cdot 1.00}{0.0001} = \frac{0}{0.0001} = 0.$$

The computed answer is $(0, 1.00)$ – the first coordinate is completely wrong, off by 100%, on a problem that was never delicate.

With partial pivoting, you first *swap rows* so that the largest-magnitude entry in the first column, the 1, sits in the pivot position:

$$\begin{bmatrix} 1 & 1 \\ 0.0001 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Now the multiplier is $m = 0.0001/1 = 0.0001$, small and benign. Eliminating: the x_2 coefficient becomes $1 - 0.0001 = 0.9999$, which rounds to 1.00 in 3 digits, and the right side becomes $1 - 0.0001 \cdot 2 = 0.9998$, which also rounds to 1.00. Thus $x_2 = 1.00/1.00 = 1.00$, and back-substitution yields $x_1 = 2 - 1.00 = 1.00$. The computed solution $(1.00, 1.00)$ is correct to every digit of available precision. Same problem, same 3-digit machine – simply reordering the equations saved the answer.

Focus

Partial pivoting: before eliminating in a column, swap rows so the largest-magnitude available entry becomes the pivot. This keeps every multiplier $|m| \leq 1$, so no row is blown up before being subtracted. It is cheap (a row swap), and it is why `A\b` in MATLAB and `numpy.linalg.solve` are reliable in practice. Plain LU *without* pivoting is numerically reckless; LU *with* partial pivoting, $\mathbf{PA} = \mathbf{LU}$, is the standard.

33.3.2 Backward stability, in one sentence

Pivoted Gaussian elimination is **backward stable**: the answer it returns is the *exact* solution of a slightly perturbed system $(\mathbf{A} + \mathbf{E})\hat{\mathbf{x}} = \mathbf{b}$, where $\mathbf{E}$ is tiny relative to $\mathbf{A}$. That is the gold standard for a numerical algorithm. You did not solve your problem exactly; you solved a problem exactly that sits within rounding distance of yours. Whether the *answer* is then accurate depends on the conditioning – a stable algorithm on an ill-conditioned problem still gives a large error, but it is the problem's fault, not the method's. The two ideas combine into the rule of thumb every numerical analyst carries:

$$\text{forward error} \lesssim \kappa(\mathbf{A}) \times \text{backward error}.$$

33.3.3 Why QR provides numerical stability

There is a class of matrices with condition number exactly 1: orthogonal matrices $\mathbf{Q}$, with $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$. They are rotations and reflections; they preserve length, $\|\mathbf{Qx}\| = \|\mathbf{x}\|$, so they cannot amplify any error. This is the deep reason orthogonal transformations are the safest tools in the numerical toolbox: *they never make a vector's relative error worse.*

The QR factorization $\mathbf{A} = \mathbf{QR}$ (Gram–Schmidt's stable cousins, the Householder and Givens methods, build it) exploits this. To solve $\mathbf{Ax} = \mathbf{b}$ you write $\mathbf{QRx} = \mathbf{b}$, multiply by $\mathbf{Q}^\mathsf{T}$ – free, because it is orthogonal and harmless – to get $\mathbf{Rx} = \mathbf{Q}^\mathsf{T}\mathbf{b}$, then back-substitute through the triangular $\mathbf{R}$. The orthogonal step transports the data without inflating any error (Figure 33.4).

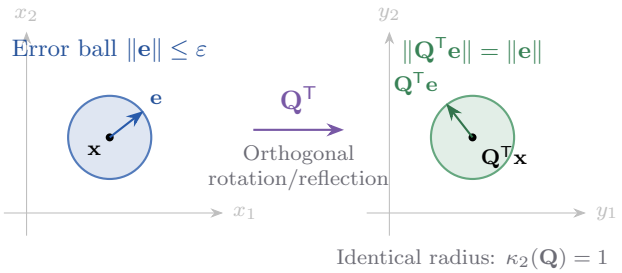


Figure 33.4: An orthogonal transformation rotates or reflects the space without stretching or squashing. A ball of uncertainty of radius ε is mapped to a congruent ball of the exact same radius ε. Because $\|\mathbf{Qx}\| = \|\mathbf{x}\|$, applying $\mathbf{Q}^\mathsf{T}$ leaves relative errors completely untouched – the reason QR-based algorithms are so robust.

Before/after Strang

Strang teaches Gaussian elimination first because it is how you compute by hand and how $\mathbf{A} = \mathbf{LU}$ is born. He then introduces $\mathbf{A} = \mathbf{QR}$ for least squares. The numerical lesson, which a first pass through Strang can leave implicit: for solving square systems, pivoted LU is fast and stable enough; but for *least squares* $\min\|\mathbf{Ax} - \mathbf{b}\|$, never form the normal equations $\mathbf{A}^\mathsf{T}\mathbf{Ax} = \mathbf{A}^\mathsf{T}\mathbf{b}$ on a computer, because $\kappa(\mathbf{A}^\mathsf{T}\mathbf{A}) = \kappa(\mathbf{A})^2$ – you square the condition number and can lose twice as many digits. Solve via QR (or the SVD) instead. That single warning is the practical heart of this chapter.

Modern application

Why your machine-learning library uses QR/SVD. A linear regression with p features fits $\min_\beta\|\mathbf{X}\beta - \mathbf{y}\|$. The textbook "closed form" $\beta = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}$ is mathematically correct and numerically treacherous: when features are correlated (which real features always are), $\mathbf{X}$ is ill-conditioned and $\mathbf{X}^\mathsf{T}\mathbf{X}$ doubles the damage. That is why `scikit-learn`'s `LinearRegression` calls LAPACK's SVD-based least-squares routine, not the normal-equation formula, and why *ridge regression* – adding $\lambda\mathbf{I}$ to $\mathbf{X}^\mathsf{T}\mathbf{X}$ – works so well: it lifts the smallest singular values away from zero, slashing the condition number and stabilizing the fit. Regularization is, in part, a conditioning fix.

33.4 Iterative methods: solving without inverting

Everything so far has been *direct*: a finite recipe (elimination, QR) that produces the answer after a fixed number of steps, roughly $\frac{2}{3}n^3$ for an $n \times n$ system. For $n = 10^6$ – a million unknowns, routine in scientific computing and graphics – that is 10^{18} operations and a dense 10^{12}-entry matrix that would not fit in memory. Direct methods die there.

But the giant matrices of practice are almost always **sparse**: mostly zeros, a handful of nonzeros per row, because each unknown couples only to its few neighbors (a pixel to adjacent pixels, a node to adjacent nodes). For these we use **iterative methods**, which never modify $\mathbf{A}$ and never form $^{-1}\mathbf{A}$. They only ever multiply $\mathbf{A}$ by a vector – cheap when $\mathbf{A}$ is sparse – generating a sequence $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots$ that converges step by step to $\mathbf{x}$.

33.4.1 Jacobi: everyone updates from last round

Split $\mathbf{A}$ into its diagonal $\mathbf{D}$ and the rest $\mathbf{R} = \mathbf{A} - \mathbf{D}$. The system $\mathbf{Ax} = \mathbf{b}$ becomes $\mathbf{Dx} = \mathbf{b} - \mathbf{Rx}$, and if we already had $\mathbf{x}$ on the right we could read off $\mathbf{x}$ on the left. We do not, so we iterate: plug in the current guess on the right to get the next guess on the left,

$$\mathbf{x}^{(k+1)} =^{-1} \mathbf{D}\big(\mathbf{b} - \mathbf{R}\,\mathbf{x}^{(k)}\big).$$

Component by component, the recipe is plain: solve the i-th equation for x_i, treating every *other* unknown as known from the previous round,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j \neq i} a_{ij}\, x_j^{(k)}\Big).$$

Example 33.4 (Two Jacobi sweeps by hand)**.** Solve

$$\begin{bmatrix} 4 & 1 \\ 2 & 5 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 1 \\ 7 \end{bmatrix}, \qquad \text{true answer } \mathbf{x} \approx \begin{bmatrix} -0.1111 \\ 1.4444 \end{bmatrix}.$$

The Jacobi updates are $x_1 \leftarrow (1 - x_2)/4$ and $x_2 \leftarrow (7 - 2x_1)/5$. Start at $\mathbf{x}^{(0)} = (0, 0)$:

$$\begin{aligned} x_1^{(1)} &= (1-0)/4 = 0.25, & x_2^{(1)} &= (7-0)/5 = 1.4, \\ x_1^{(2)} &= (1-1.4)/4 = -0.1, & x_2^{(2)} &= (7 - 2\cdot 0.25)/5 = 1.3, \\ x_1^{(3)} &= (1-1.3)/4 = -0.075, & x_2^{(3)} &= (7 - 2\cdot(-0.1))/5 = 1.44. \end{aligned}$$

Already after three sweeps we are at $(-0.075,\, 1.44)$, marching toward $(-0.11,\, 1.44)$. Each sweep used only the previous sweep's numbers – so all updates could run in parallel.

33.4.2 Gauss-Seidel: use the news as soon as you hear it

Jacobi wastes information: while computing the new x_2 it still uses the *stale* x_1, even though a fresh x_1 was just computed this very sweep. Gauss–Seidel fixes that – use each new component the instant it is available:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}\, x_j^{(k+1)} - \sum_{j>i} a_{ij}\, x_j^{(k)}\Big).$$

The only change from Jacobi is the superscript on the first sum: $j < i$ uses the already-updated values $\mathbf{x}^{(k+1)}$. Redo the example: $x_1^{(1)} = (1-0)/4 = 0.25$ as before, but now $x_2^{(1)} = (7 - 2\cdot 0.25)/5 = 1.3$ immediately uses the fresh 0.25. Next sweep $x_1^{(2)} = (1 - 1.3)/4 = -0.075$, then $x_2^{(2)} = (7 - 2\cdot(-0.075))/5 = 1.43$. After two sweeps Gauss–Seidel sits at $(-0.075, 1.43)$, where Jacobi needed three. It typically converges about twice as fast, at the cost of being inherently sequential (each component waits for the one before it).

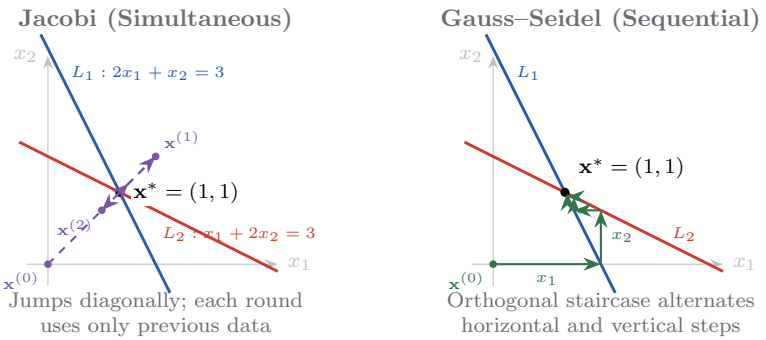


Figure 33.5: The geometry of iterative relaxation in $\mathbb{R}^2$. Each equation defines a line; the true solution $\mathbf{x}^*$ is their intersection. In Gauss–Seidel (right), updating x_1 means moving horizontally to Line 1; updating x_2 with that fresh value means moving vertically to Line 2. This creates an orthogonal staircase spiraling into $\mathbf{x}^*$. Diagonal dominance means Line 1 is steeper than Line 2, guaranteeing that each bounce contracts toward the center. Jacobi (left) computes both projections from the prior guess simultaneously, leaping diagonally.

Intuition

Both methods are "relax toward agreement." Picture each equation as a constraint, and each sweep as nudging every unknown to satisfy its own equation given everyone else's current value. Like a roomful of people each adjusting to their neighbors, the whole configuration settles toward the consensus that satisfies all equations at once. Gauss–Seidel just lets each person hear the latest gossip before adjusting, so the room settles faster.

Formal

These iterations converge for *any* starting guess precisely when the **spectral radius** of the iteration matrix is below 1. For Jacobi the iteration matrix is $\mathbf{M} = -^{-1}\mathbf{DR}$, and $\mathbf{x}^{(k+1)} - \mathbf{x} = \mathbf{M}(\mathbf{x}^{(k)} - \mathbf{x})$, so the error is multiplied by $\mathbf{M}$ each step and dies iff every eigenvalue of $\mathbf{M}$ has $|\lambda| < 1$. A clean sufficient condition you can check by eye: **strict diagonal dominance**, $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for every row – each diagonal entry bigger than the rest of its row combined – guarantees both Jacobi and Gauss–Seidel converge. Our $\left[\begin{smallmatrix} 4 & 1 \\ 2 & 5 \end{smallmatrix}\right]$ qualifies: $4 > 1$ and $5 > 2$.

Common trap

Iterative methods are not guaranteed to converge. If $\mathbf{A}$ is not diagonally dominant (or, more generally, if the iteration matrix has spectral radius ≥ 1), Jacobi and Gauss–Seidel can stall or *diverge to infinity*. Always check diagonal dominance, or a related structural property like positive-definiteness, before trusting them – and watch the residual $\|\mathbf{Ax}^{(k)} - \mathbf{b}\|$ actually shrinking.

Pause

Pause and think: Look at the symmetric matrix $\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 1 \end{bmatrix}$. Can you guarantee that Jacobi or Gauss–Seidel iteration will converge for $\mathbf{Ax} = \mathbf{b}$? *Answer:* Check strict diagonal

dominance row by row. Row 1 has $|a_{11}| = 3 > 2 = |a_{12}|$, which passes. But Row 2 has $|a_{22}| = 1 < 2 = |a_{21}|$, which fails. In fact, the Jacobi iteration matrix $\mathbf{M} = -^{-1}\mathbf{DR} = -\begin{bmatrix}1/3 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}0 & 2\\ 2 & 0\end{bmatrix} = \begin{bmatrix}0 & -2/3\\ -2 & 0\end{bmatrix}$ has eigenvalues $\lambda = \pm\sqrt{4/3} \approx \pm 1.155$. Because its spectral radius exceeds 1, errors multiply and blow up to infinity – the iteration diverges.

33.4.3 Conjugate gradient: the heavyweight

For the most important case – $\mathbf{A}$ symmetric positive definite – there is a method that leaves Jacobi and Gauss–Seidel far behind: the **conjugate gradient** (CG) method. The geometry behind it is what makes it work.

When $\mathbf{A}$ is symmetric positive definite, solving $\mathbf{Ax} = \mathbf{b}$ is *exactly* the same as minimizing the bowl-shaped quadratic

$$\phi(\mathbf{x}) = \tfrac{1}{2}\,\mathbf{x}^{\mathsf{T}}\mathbf{Ax} - \mathbf{b}^{\mathsf{T}}\mathbf{x}.$$

Its gradient is $\nabla\phi = \mathbf{Ax} - \mathbf{b}$, which is zero exactly at the solution. So solving the linear system *is* rolling to the bottom of a paraboloid bowl. Because $\mathbf{A}$ is positive definite, the bowl really is a bowl (curving up in every direction), and its level sets are nested ellipsoids – stretched into a long valley when $\mathbf{A}$ is ill-conditioned.

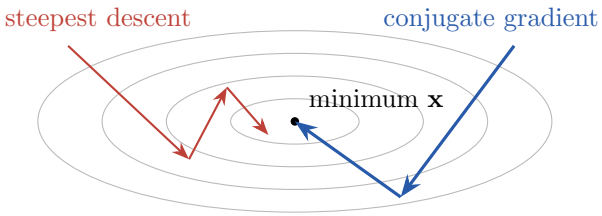


Figure 33.6: Solving $\mathbf{Ax} = \mathbf{b}$ for symmetric positive definite $\mathbf{A}$ means finding the bottom of an ellipsoidal bowl. Naive steepest descent (red) zig-zags across the stretched valley, taking tiny corrective steps forever. Conjugate gradient (blue) chooses $\mathbf{A}$-orthogonal directions and never undoes its own progress, reaching the bottom of an n-dimensional bowl in at most n steps.

The obvious way down is *steepest descent*: always step in the direction of steepest decrease (the negative gradient). On a round bowl that goes straight to the bottom, but on a long ellipsoidal valley it zig-zags maddeningly, each step ruining the progress of the last (Figure 33.6, red). Conjugate gradient's insight is to pick directions that are "$\mathbf{A}$-orthogonal" (conjugate): $\mathbf{p}_i^{\mathsf{T}}\mathbf{A}\,\mathbf{p}_j = 0$ for $i \neq j$. Geometrically, stretch coordinates by $\mathbf{y} = \mathbf{A}^{1/2}\mathbf{x}$: the elongated bowl becomes perfectly round, and $\mathbf{p}_i^{\mathsf{T}}\mathbf{A}\mathbf{p}_j = (\mathbf{A}^{1/2}\mathbf{p}_i)\cdot(\mathbf{A}^{1/2}\mathbf{p}_j) = 0$ says the search directions are ordinary perpendicular axes of that round bowl. With such directions, a step that fixes one direction never spoils another, so there is no zig-zag and no backtracking. In exact arithmetic CG reaches the exact solution of an $n \times n$ system in at most n steps – and usually far fewer, because it converges fast once it has explored the directions that matter.

Intuition

Steepest descent is like walking downhill in a narrow canyon by always facing the steepest local slope: you bounce wall to wall, creeping forward. Conjugate gradient is like realizing the canyon has a few principal directions and dispatching each one *once and for all*, so you stride to the bottom in a handful of decisive moves. The number of steps you really need is governed by $\sqrt{\kappa(\mathbf{A})}$ – which is why *preconditioning* (multiplying by an approximate inverse to shrink κ) is what makes CG practical on real problems.

Modern application

Conjugate gradient and its relatives are the quiet engine of large-scale computation. When a graphics engine simulates cloth or smoke, when an aircraft is stress-tested by finite-element analysis, or when a modern recommendation system solves a sparse linear system in millions of unknowns, a Krylov-subspace iterative solver (CG for symmetric problems, GMRES for general ones) is almost always inside the loop. They win because they touch $\mathbf{A}$ only through matrix-vector products, which respect sparsity and parallelize across thousands of cores – something a dense $^{-1}\mathbf{A}$ could never do.

33.5 The power method: finding the top eigenvalue

Iterative ideas also conquer eigenvalues. The simplest eigenvalue algorithm, the **power method**, computes the largest-magnitude eigenvalue and its eigenvector using only repeated matrix-vector multiplication.

The idea: start with almost any vector $\mathbf{v}^{(0)}$ and hit it with $\mathbf{A}$ over and over. Write $\mathbf{v}^{(0)}$ in an eigenbasis, $\mathbf{v}^{(0)} = c_1\mathbf{x}_1 + c_2\mathbf{x}_2 + \cdots + c_n\mathbf{x}_n$, where $|\lambda_1| > |\lambda_2| \geq \cdots$. Then

$$\mathbf{A}^k\mathbf{v}^{(0)} = c_1\lambda_1^k\mathbf{x}_1 + c_2\lambda_2^k\mathbf{x}_2 + \cdots = \lambda_1^k\Big(c_1\mathbf{x}_1 + c_2(\tfrac{\lambda_2}{\lambda_1})^k\mathbf{x}_2 + \cdots\Big).$$

Every ratio λ_j/λ_1 has magnitude below 1, so its k-th power decays to zero. The dominant eigendirection $\mathbf{x}_1$ survives; everything else fades. After enough multiplications the vector points (almost) exactly along $\mathbf{x}_1$. To keep the numbers from overflowing or underflowing, normalize after each step:

$$\mathbf{w} = \mathbf{A}\,\mathbf{v}^{(k)}, \qquad \mathbf{v}^{(k+1)} = \frac{\mathbf{w}}{\|\mathbf{w}\|}.$$

The eigenvalue itself falls out as the Rayleigh quotient $\lambda_1 \approx \mathbf{v}^{(k)\mathsf{T}}\mathbf{A}\,\mathbf{v}^{(k)}$, or simply as the factor by which the vector grew before normalizing.

Example 33.5 (Power method in three steps)**.** Take

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

whose true eigenvalues are 3 (eigenvector $(1,1)$) and 1 (eigenvector $(1,-1)$). Start with the deliberately lopsided $\mathbf{v}^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and just track the raw vector $\mathbf{A}^k\mathbf{v}^{(0)}$:

$$\mathbf{A}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{A}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}, \quad \mathbf{A}\begin{bmatrix} 5 \\ 4 \end{bmatrix} = \begin{bmatrix} 14 \\ 13 \end{bmatrix}, \quad \mathbf{A}\begin{bmatrix} 14 \\ 13 \end{bmatrix} = \begin{bmatrix} 41 \\ 40 \end{bmatrix}.$$

Watch the ratio of the two components head for 1 (the direction of $(1,1)$): $2/1$, then $5/4$, then $14/13$, then $41/40$. And the growth factor per step heads for 3: from $\begin{bmatrix}14\\13\end{bmatrix}$ to $\begin{bmatrix}41\\40\end{bmatrix}$ the first component went $14 \to 41$, a factor 2.93, closing on the true dominant eigenvalue $\lambda_1 = 3$. Four multiplications and we already have two good digits (Figure 33.7).

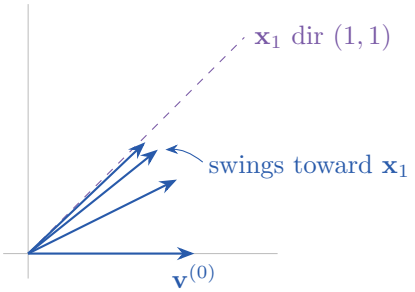


Figure 33.7: The power method in action on $\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$. Each multiplication by $\mathbf{A}$ swings the iterate closer to the dominant eigendirection $(1,1)$; the subdominant component shrinks by the factor $\lambda_2/\lambda_1 = 1/3$ each step.

Modern application

PageRank is the power method. Google's original ranking treated the web as a giant matrix $\mathbf{A}$ where A_{ij} is the probability a random surfer hops from page j to page i (Chapter 35 uses the transpose convention, with row vectors). The importance of every page is the dominant eigenvector of that billion-by-billion matrix – and you find it by exactly the iteration above: start with a uniform vector, multiply by $\mathbf{A}$, normalize, repeat. Because $\mathbf{A}$ is enormously sparse (each page links to only a handful of others), each multiplication is cheap, and the dominant eigenvalue is conveniently 1 with a clear gap to the next, so the power method converges in a few dozen sweeps over the entire web. The whole of linear algebra's spectral theory, run iteratively, ranked the internet.

Remark 33.1. The power method only finds the *top* eigenvalue, and it stalls if the top two are equal in magnitude (no gap, nothing to win the race). To get other eigenvalues you shift and invert (apply the power method to $(\mathbf{A}-\mu\mathbf{I})^{-1}$ to find the eigenvalue nearest μ), or deflate, or – for all eigenvalues at once – use the **QR algorithm**, the standard general solver: repeatedly factor $\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k$ and reassemble $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k$. This similarity iteration drives the matrix toward upper-triangular form, reading the eigenvalues off the diagonal. It is the power method's grown-up sibling and what `eig` actually calls.

33.6 Practical warnings to carry with you

Let us collect the field's hard-won wisdom into a checklist you can actually use.

Focus

Numerical commandments.

1. **Never invert a matrix to solve a system.** To solve $\mathbf{Ax} = \mathbf{b}$, factor and back-

substitute (`solve`, `A\b`) – do not compute $^{-1}\mathbf{A}$ and multiply. Forming $^{-1}\mathbf{A}$ is slower, less accurate, and destroys sparsity.

2. **Never solve least squares via the normal equations.** Use QR or SVD; $\mathbf{A}^\top\mathbf{A}$ squares the condition number.
3. **Check the condition number** (`cond(A)`) before trusting a result. If $\kappa \approx 10^k$, distrust the last k digits.
4. **Avoid subtracting nearly equal numbers.** Reformulate to dodge catastrophic cancellation when you can.
5. **Never test floating-point numbers for exact equality.** Write $|a - b| < \text{tol}$, not $a == b$; rounding makes exact equality a lie.
6. **Exploit structure.** Sparse, symmetric, positive definite, banded, and triangular matrices each admit faster, more stable specialized solvers. Tell your library what you know.
7. **Watch the residual, and remember conditioning.** A small residual $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|$ does not by itself guarantee a small error $\|\hat{\mathbf{x}} - \mathbf{x}\|$ – the gap between them is the condition number.

Why this matters

Pure linear algebra hands us perfect objects – $^{-1}\mathbf{A}$, exact eigenvalues, clean subspaces. Numerical linear algebra asks the humbler, harder question: *which of these can survive contact with a finite machine?* The answers have a pleasing shape. Orthogonal matrices survive because they preserve length. Sparse problems survive because we never form the dense inverse. Ill-conditioned problems are flagged by a large κ, while backward-stable algorithms introduce minimal error of their own. Together they show how far finite-precision arithmetic can be trusted and where its limits lie.

Pause

Before the exercises, consider the two key concepts. *Conditioning* is about the problem: $\kappa(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ measures how much a relative error in the data can be amplified into the answer; nothing can fix an ill-conditioned problem. *Stability* is about the algorithm: a backward-stable method (pivoted LU, QR) solves a nearby problem exactly and adds little of its own. The master inequality forward error $\lesssim \kappa \times$ backward error binds them. If those two sentences feel solid, you have the chapter.

33.7 The computational horizon: where numerical precision meets learning

Direct factorizations like pivoted LU, QR, and the SVD are reliable at moderate sizes, but at large scale numerical solvers face four computational limits:

- **The cubic scaling wall:** Direct factorizations like LU, QR, and the SVD scale cubically

($\mathcal{O}(n^3)$ operations) and require quadratic memory ($\mathcal{O}(n^2)$ entries). If a problem has $n = 10^7$ variables—routine in modern graph analytics, large language models, and web search—a dense direct factorization would need about 800 terabytes just to store the matrix (10^{14} entries of 8 bytes) and around 10^{21} operations to factor it.

- **Sparse fill-in:** Even when a matrix begins with 99.9% zeros, Gaussian elimination systematically turns zeros into nonzeros as rows are subtracted. This "fill-in" can rapidly exhaust computer cache and memory unless specialized graph reordering algorithms are used.
- **The wall of severe ill-conditioning:** Backward stability guarantees that the computed solution is exact for a nearby system $(\mathbf{A} + \mathbf{E})\hat{\mathbf{x}} = \mathbf{b}$. It cannot rescue a problem where $\kappa(\mathbf{A}) > 1/\varepsilon_{\text{mach}} \approx 10^{16}$. In that regime, all sixteen decimal digits of double precision are wiped out by the problem's own geometry, and the machine returns pure numerical static.
- **The spectral bottleneck:** Iterative methods depend on spectral gaps. When the subdominant eigenvalue crawls close to the dominant one ($|\lambda_2/\lambda_1| \approx 1$), the power method and basic relaxation schemes stall, requiring thousands of sweeps to gain a single digit of precision.

These constraints force a shift in practical goals. For more than half a century, the dominant goal of numerical linear algebra was to compute the exact solution of a fixed linear system to sixteen decimal digits of precision. But what happens when the data is noisy, the parameters number in the billions, and computing sixteen digits is neither possible nor useful?

In Chapter 34, we step into machine learning, where exact solves give way to low-rank approximations and gradient descent steps. The geometric tools built here—projections, singular vectors, condition numbers, and downhill steps on quadratic bowls—become the language through which models learn patterns from data.

33.8 Exercises

Exercise 33.1 (Machine epsilon)**.** Double precision has a 52-bit mantissa, so $\varepsilon_{\text{mach}} = 2^{-52}$. (a) Estimate $\varepsilon_{\text{mach}}$ in decimal and confirm it gives roughly 16 significant digits. (b) Explain in one sentence why the smallest number you can add to 1.0 and get a result different from 1.0 is about $\varepsilon_{\text{mach}}$, while near 1000 that smallest difference is about $1000\,\varepsilon_{\text{mach}}$.

Exercise 33.2 (Catastrophic cancellation)**.** Working to 4 significant decimal digits throughout, compute $f(x) = \dfrac{1 - \cos x}{x^2}$ at $x = 0.001$ directly. The true value is very close to $1/2$. (a) What do you get, and why is it badly wrong? (b) Using $1 - \cos x = 2\sin^2(x/2)$, recompute and compare. (c) State which step in the direct route caused the loss.

Exercise 33.3 (Condition number of a 2×2)**.** Let $\mathbf{A} = \left[\begin{smallmatrix} 1 & 0 \\ 0 & \delta \end{smallmatrix}\right]$ for a small $\delta > 0$. (a) Find both singular values and write down $\kappa_2(\mathbf{A})$. (b) For $\delta = 10^{-6}$, how many decimal digits of accuracy do you expect to lose when solving $\mathbf{A}\mathbf{x} = \mathbf{b}$? (c) Sketch the image of the unit circle under $\mathbf{A}$ and relate its shape to κ.

Exercise 33.4 (Determinant is not conditioning)**.** Let $\mathbf{A} = 10^{-3}\mathbf{I}_3$ (the 3×3 identity times 0.001). (a) Compute $\det \mathbf{A}$ – it is tiny. (b) Compute $\kappa_2(\mathbf{A})$. (c) Explain in one or two sentences why the small determinant is no cause for alarm here, tying it back to the trap that conditioning is about a ratio of stretches, not a volume.

Exercise 33.5 (Pivoting saves the day)**.** Consider $\left[\begin{smallmatrix} 10^{-4} & 1 \\ 1 & 1 \end{smallmatrix}\right] \mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, solved on a machine with 3 significant digits. (a) Solve by Gaussian elimination *without* pivoting and report the (wrong) answer. (b) Swap rows (partial pivoting) and solve again. (c) State the multiplier in each case and explain which one caused the trouble.

Exercise 33.6 (Diagonal dominance and convergence)**.** For $\mathbf{A} = \left[\begin{smallmatrix} 5 & 2 \\ 1 & 4 \end{smallmatrix}\right]$ and $\mathbf{b} = \begin{bmatrix} 7 \\ 6 \end{bmatrix}$: (a) Verify strict diagonal dominance. (b) Carry out three Jacobi sweeps from $\mathbf{x}^{(0)} = (0, 0)$. (c) Carry out three Gauss–Seidel sweeps from the same start, and confirm it is closer to the true solution $(\frac{8}{9}, \frac{23}{18}) \approx (0.889, 1.278)$ after the same number of sweeps.

Exercise 33.7 (Why not the normal equations)**.** Suppose $\mathbf{A}$ (tall, full column rank) has $\kappa_2(\mathbf{A}) = 10^6$. (a) What is $\kappa_2(\mathbf{A}^\mathsf{T}\mathbf{A})$? (b) Roughly how many decimal digits would you expect to lose solving the least-squares problem through the normal equations versus through QR, starting from double precision? (c) In one sentence, state the practical rule this implies.

Exercise 33.8 (Power method by hand)**.** Let $\mathbf{A} = \left[\begin{smallmatrix} 3 & 1 \\ 0 & 2 \end{smallmatrix}\right]$, whose eigenvalues are 3 and 2. Starting from $\mathbf{v}^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, perform three power-method steps (you may skip normalization and just track the vector and the ratio of components). (a) Toward which eigenvector does the iterate swing? (b) Estimate the dominant eigenvalue from the growth factor. (c) The subdominant component dies by what factor each step, and what does that predict about the speed of convergence?

Exercise 33.9 (Conjugate gradient geometry)**.** Take the symmetric positive definite $\mathbf{A} = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 100 \end{smallmatrix}\right]$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. (a) Write the quadratic $\phi(\mathbf{x})$ and describe the shape of its level sets. (b) Explain why steepest descent from a generic start zig-zags badly, referencing the condition number $\kappa_2(\mathbf{A}) = 100$. (c) Argue that conjugate gradient finishes in at most 2 steps here, and say what general statement this illustrates.

Exercise 33.10 (A proof: orthogonal matrices are perfectly conditioned)**.** Let $\mathbf{Q}$ be a real $n \times n$ orthogonal matrix, $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$. (a) Show $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$ for every $\mathbf{x}$, hence $\|\mathbf{Q}\|_2 = 1$. (b) Show ${}^{-1}\mathbf{Q} = \mathbf{Q}^\mathsf{T}$ is also orthogonal, so $\|{}^{-1}\mathbf{Q}\|_2 = 1$. (c) Conclude $\kappa_2(\mathbf{Q}) = 1$, and explain in words why this makes orthogonal transformations the safest building blocks in numerical linear algebra.

Exercise 33.11 (Residual versus error – a challenge)**.** Using $\mathbf{A} = \left[\begin{smallmatrix} 1 & 1 \\ 1 & 1.0001 \end{smallmatrix}\right]$ and the true solution $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (so $\mathbf{b} = \begin{bmatrix} 2 \\ 2.0001 \end{bmatrix}$), consider the approximate solution $\hat{\mathbf{x}} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$. (a) Compute the residual $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ and its norm. (b) Compute the error $\hat{\mathbf{x}} - \mathbf{x}$ and its norm. (c) The residual is small but the error is large. Compute $\kappa_2(\mathbf{A})$ approximately and explain how the master inequality forward error $\lesssim$ $\kappa \times$ backward error accounts for the gap.

Exercise 33.12 (The conditioning of the Fourier matrix)**.** In Chapter 32 we encountered the Discrete Fourier Transform matrix $\mathbf{F}_n$, whose entries are powers of the root of unity $\omega = e^{2\pi i/n}$. Consider the normalized Fourier matrix $\mathbf{U} = \frac{1}{\sqrt{n}}\mathbf{F}_n$. (a) Show that $\mathbf{U}$ is unitary, satisfying $\mathbf{U}^*\mathbf{U} = \mathbf{I}_n$. (b) Determine all singular values of $\mathbf{U}$. (c) Compute the 2-norm condition number $\kappa_2(\mathbf{U})$. (d) Using your result, explain why the Fast Fourier Transform (FFT) in practice does not suffer from catastrophic roundoff error accumulation, tying this back to the geometric stability of orthogonal and unitary transformations.

Chapter 34

The Linear Algebra of Machine Learning

In the previous chapter on numerical linear algebra, we saw what happens when abstract formulas meet physical machines: numbers get rounded, ill-conditioned systems amplify noise, and matrices with millions of entries cannot be inverted by exact symbolic formulas. Real computation demands stable projections, low-rank approximations, and iterative algorithms that step toward answers one matrix–vector product at a time.

This chapter brings those numerical and geometric tools together in a concrete setting. Almost every method in modern machine learning – including recommendation engines, computer vision, semantic search, and large language models – is a direct application of numerical and geometric linear algebra.

When a neural network processes an image, the image is represented as a vector of pixel intensities and multiplied by weight matrices. When a search engine measures whether "king" and "queen" are related, it computes a dot product. When a model is trained, gradient descent steps downhill on a loss surface along a vector of partial derivatives.

The aim here is to make the linear algebra under machine learning visible enough that the field stops looking mysterious and starts looking like *geometry you understand*. We go slowly, build every idea from a picture, and keep every example small enough to check by hand.

Focus

The one sentence to carry through the whole chapter: **a machine learning model is a function from vectors to vectors, built mostly out of matrix multiplications, and training is the search for the matrices that make that function fit the data.** Hold onto that and everything below is a variation on it.

34.1 Data is a matrix

Before we can do anything clever, we have to agree on how data even *enters* the machinery. Everything downstream rests on this, so let us be very concrete.

Suppose you run a small clinic and you have recorded, for each of four patients, their age, their

resting heart rate, and their systolic blood pressure. You could write this as a little table:

patient	age	heart rate	blood pressure
Ana	30	60	110
Ben	50	72	130
Cleo	40	68	120
Dev	60	80	145

What remains is a 4×3 array of numbers:

$$\mathbf{X} = \begin{bmatrix} 30 & 60 & 110 \\ 50 & 72 & 130 \\ 40 & 68 & 120 \\ 60 & 80 & 145 \end{bmatrix}.$$

This object – the *data matrix* – is where machine learning begins. The convention almost everyone uses is: **one row per example, one column per feature.** Each patient is a point; each measurement is a coordinate.

Intuition

A data matrix has two complementary readings, and you should be able to flip between them at will.

Read it by rows and each row $\mathbf{x}_i \in \mathbb{R}^3$ is a single patient: a point floating in a three-dimensional "feature space" whose axes are age, heart rate, and blood pressure. Four patients give you a cloud of four points.

Read it by columns and each column is one feature measured across the whole population: the vector $(30, 50, 40, 60)$ is "everyone's age." Now each column lives in $\mathbb{R}^4$, the "sample space," and you can ask whether two columns point in similar directions – which, you will see, is exactly asking whether two features are correlated.

This double reading drives the whole subject. The geometry of the rows (points in feature space) drives clustering, classification, and nearest-neighbor search. The geometry of the columns (features in sample space) drives correlation, covariance, and principal component analysis. *Same matrix, two geometries.*

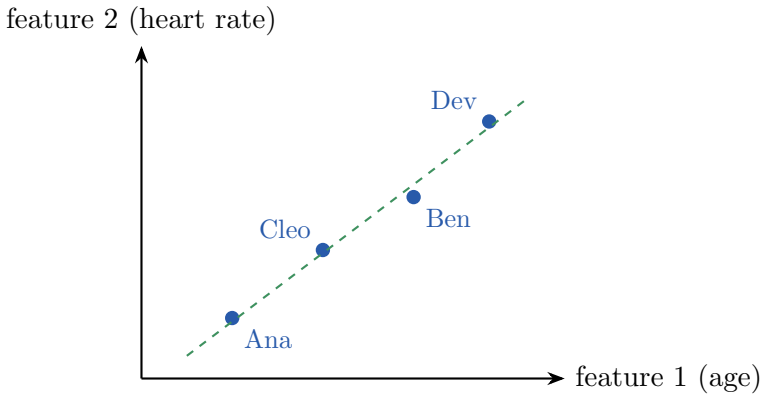


Figure 34.1: Reading the data matrix by rows: each patient is a point in feature space (only two of the three features are drawn). Already you can *see* structure – the points trend upward together, the geometric fingerprint of a correlation between age and heart rate.

Common trap

The orientation convention bites everyone at least once. In this book and in most ML libraries, examples are *rows* and features are *columns*, so $\mathbf{X}$ is $n \times d$ (n examples, d features). Classical statistics and Strang's own least-squares chapter often put the *data vector* $\mathbf{b}$ as a column and stack equations as rows – same math, transposed bookkeeping. Whenever a formula looks "backwards," the first thing to check is whether someone transposed $\mathbf{X}$. Write the shapes next to every matrix; shapes never lie.

34.1.1 Centering and the mean as a vector

Almost the first thing you do to a data matrix is *center* it: subtract, from each column, that column's average. The column means form a vector $\bar{\mathbf{x}} \in \mathbb{R}^d$, and centering replaces every row $\mathbf{x}_i$ by $\mathbf{x}_i - \bar{\mathbf{x}}$. Geometrically you are sliding the whole cloud of points so its center of mass sits at the origin.

For our clinic, the column means are $\bar{\mathbf{x}} = (45,\ 70,\ 126.25)$, and the centered matrix is

$$\mathbf{X}_c = \begin{bmatrix} -15 & -10 & -16.25 \\ 5 & 2 & 3.75 \\ -5 & -2 & -6.25 \\ 15 & 10 & 18.75 \end{bmatrix}.$$

Centering matters because nearly every "direction" question – which way does the data vary most? do two features move together? – only makes sense relative to the center. Without centering, raw vectors $\mathbf{x}_i$ point from the arbitrary coordinate origin $(0, \ldots, 0)$ rather than measuring deviations from the cloud's mean.

Compute it yourself

Take three points $(1, 2), (3, 3), (5, 7)$ as the rows of a 3×2 matrix. By hand: (a) compute the column-mean vector; (b) subtract it from each row to center the data; (c) check that the columns of your centered matrix each sum to zero. That last check is your safety net – a

centered column *always* sums to zero, so if it does not, you made an arithmetic slip.

34.1.2 The Gram matrix: all the dot products at once

Matrix algebra now provides an immediate advantage. Suppose you want every pairwise dot product between rows – a natural thing, because (as the next sections show) dot products measure similarity. You do *not* loop over pairs. You write a single matrix product:

$$\mathbf{G} = \mathbf{X}\mathbf{X}^{\mathsf{T}}, \qquad G_{ij} = \mathbf{x}_i \cdot \mathbf{x}_j.$$

The entry in row i, column j of $\mathbf{X}\mathbf{X}^{\mathsf{T}}$ is the dot product of row i of $\mathbf{X}$ with row j of $\mathbf{X}$ – that is just the definition of matrix multiplication, read carefully. So $\mathbf{G}$, the *Gram matrix*, packages all n^2 similarities into one symmetric matrix.

Flip the order and you get the other geometry. The $d \times d$ matrix $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ collects all pairwise dot products between *columns* (features); once each column is centered to mean zero, $\frac{1}{n}\mathbf{X}^{\mathsf{T}}\mathbf{X}$ is exactly the **covariance matrix** you met in Chapter 30. We will return to it; for now notice the elegant duality: $\mathbf{X}\mathbf{X}^{\mathsf{T}}$ compares examples, $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ compares features, and they share the same nonzero eigenvalues. Chapter 29 explains why; for now, notice that one little matrix product, taken two ways, generates both geometries of your data.

34.2 Linear regression: geometric least squares

The oldest learning algorithm in data analysis is linear regression, and it is pure linear algebra – in fact it is the least-squares problem you have already studied, expressed in the vocabulary of data science. Let us set it up in that notation, since the picture carries straight over to the harder models later.

34.2.1 The setup

We have inputs and we want to predict an output. Say we have the ages and heart rates of patients and we want to predict blood pressure. We propose the simplest possible model: the prediction is a *weighted sum* of the inputs, plus an offset:

$$\hat{y} = w_1 \cdot (\text{age}) + w_2 \cdot (\text{heart rate}) + b.$$

The numbers w_1, w_2 are *weights*; b is the *bias* (statisticians call it the intercept). Learning means choosing w_1, w_2, b so the predictions $\hat{y}$ match the true blood pressures as well as possible.

A standard bookkeeping trick absorbs the bias. Append a column of ones to the data matrix. Then the bias becomes "just another weight," the weight on the constant feature 1:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}, \qquad \mathbf{X} = \begin{bmatrix} 30 & 60 & 1 \\ 50 & 72 & 1 \\ 40 & 68 & 1 \\ 60 & 80 & 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix}.$$

Now the entire model is one matrix–vector product, $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. The vector of all four predictions is $\mathbf{X}$ acting on the weight vector. *The model is a linear map from weight space to prediction space.*

Focus

A linear regression model *is* the matrix–vector product $\mathbf{Xw}$. Training it means: given the targets $\mathbf{y}$, find the $\mathbf{w}$ that makes $\mathbf{Xw}$ as close to $\mathbf{y}$ as possible. That is the least-squares problem, and you already know its answer is a *projection*.

34.2.2 Why projection? The column-space picture

The geometry here is the heart of the whole section. As $\mathbf{w}$ ranges over all possible weight vectors, $\hat{\mathbf{y}} = \mathbf{Xw}$ ranges over all linear combinations of the columns of $\mathbf{X}$ – that is, over the *column space* $\mathrm{col}(\mathbf{X})$. This is a flat subspace (a plane, here a 3-dimensional one) sitting inside $\mathbb{R}^4$, the space where the target $\mathbf{y}$ lives.

The target $\mathbf{y}$ – the actual blood pressures – almost never lies exactly in that subspace. (If it did, the model would fit perfectly, and real data never cooperates.) So the best we can do is find the point *in* the column space that is closest to $\mathbf{y}$. The closest point in a subspace to an outside point is the **orthogonal projection**. The error $\mathbf{y} - \hat{\mathbf{y}}$ must be perpendicular to the subspace – because if it had any component lying *along* the subspace, you could slide $\hat{\mathbf{y}}$ in that direction and get closer.

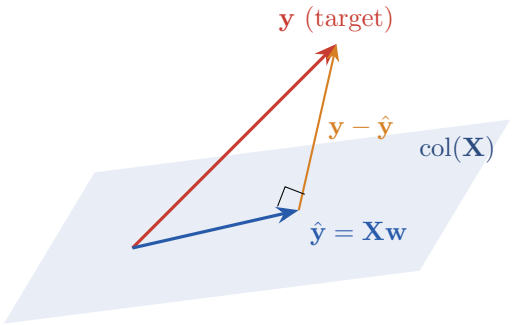


Figure 34.2: Least squares as projection. The predictions $\mathbf{Xw}$ can only reach points in the column space (the shaded plane). The best prediction is the foot of the perpendicular from the target $\mathbf{y}$. The residual $\mathbf{y} - \hat{\mathbf{y}}$ is orthogonal to the column space – that orthogonality *is* the normal equations.

"The residual is perpendicular to every column of $\mathbf{X}$" translates directly into algebra. Perpendicular to every column means $\mathbf{X}^\mathsf{T}(\mathbf{y} - \mathbf{Xw}) = \mathbf{0}$, which rearranges to the celebrated **normal equations**:

$$\boxed{\mathbf{X}^\mathsf{T}\mathbf{X}\,\mathbf{w} = \mathbf{X}^\mathsf{T}\mathbf{y}}$$

and, when $\mathbf{X}^\mathsf{T}\mathbf{X}$ is invertible, the closed-form solution

$$\mathbf{w} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}.$$

Formal

Least-squares regression. Given $\mathbf{X} \in \mathbb{R}^{n \times d}$ with linearly independent columns and $\mathbf{y} \in \mathbb{R}^n$, the weight vector minimizing the squared error $\|\mathbf{y} - \mathbf{Xw}\|^2$ is the unique solution of $\mathbf{X}^\mathsf{T}\mathbf{X}\,\mathbf{w} = \mathbf{X}^\mathsf{T}\mathbf{y}$, namely $\mathbf{w} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}$. The fitted values are $\hat{\mathbf{y}} = \mathbf{Xw} = \mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\,\mathbf{y} = \mathbf{P}\,\mathbf{y}$,

where $\mathbf{P}$ is the orthogonal projector onto $\text{col}(\mathbf{X})$.

Pause

Pause and think. Suppose the target vector $\mathbf{y}$ already lies entirely inside the column space $\text{col}(\mathbf{X})$. What is the fitted prediction $\hat{\mathbf{y}}$, what is the residual vector $\mathbf{y} - \hat{\mathbf{y}}$, and what is the minimum squared error? What if $\mathbf{y}$ is instead strictly orthogonal to $\text{col}(\mathbf{X})$?

34.2.3 A fully worked tiny fit

Let us actually solve one, small enough to do on paper. Fit a line $\hat{y} = wx + b$ to three points: $(1, 1), (2, 2), (3, 2)$. With the ones-column trick,

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \qquad \mathbf{w} = \begin{bmatrix} w \\ b \end{bmatrix}.$$

Build the two pieces of the normal equations:

$$\mathbf{X}^\top\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}, \qquad \mathbf{X}^\top\mathbf{y} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 11 \\ 5 \end{bmatrix}.$$

Solve $\begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix} \mathbf{w} = \begin{bmatrix} 11 \\ 5 \end{bmatrix}$. The determinant is $14 \cdot 3 - 6 \cdot 6 = 42 - 36 = 6$, so

$$\mathbf{w} = \frac{1}{6} \begin{bmatrix} 3 & -6 \\ -6 & 14 \end{bmatrix} \begin{bmatrix} 11 \\ 5 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 33 - 30 \\ -66 + 70 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 2/3 \end{bmatrix}.$$

So the best-fit line is $\hat{y} = \frac{1}{2}x + \frac{2}{3}$. Now do the self-check the projection picture promises – compute the residuals and confirm they are orthogonal to the column space. Predict at each x, being careful with the fractions:

$$\hat{y}(1) = \tfrac{1}{2} + \tfrac{2}{3} = \tfrac{7}{6}, \qquad \hat{y}(2) = 1 + \tfrac{2}{3} = \tfrac{5}{3}, \qquad \hat{y}(3) = \tfrac{3}{2} + \tfrac{2}{3} = \tfrac{13}{6}.$$

The residuals $\mathbf{y} - \hat{\mathbf{y}}$ are then

$$1 - \tfrac{7}{6} = -\tfrac{1}{6}, \qquad 2 - \tfrac{5}{3} = \tfrac{1}{3}, \qquad 2 - \tfrac{13}{6} = -\tfrac{1}{6},$$

so the residual vector is $\mathbf{r} = \left(-\tfrac{1}{6}, \tfrac{1}{3}, -\tfrac{1}{6}\right)$. Two orthogonality checks, one per column of $\mathbf{X}$. Dot with the ones-column: $-\tfrac{1}{6} + \tfrac{1}{3} - \tfrac{1}{6} = 0$. Dot with the x-column $(1, 2, 3)$: $-\tfrac{1}{6} + \tfrac{2}{3} - \tfrac{1}{2} = -\tfrac{1}{6} + \tfrac{4}{6} - \tfrac{3}{6} = 0$. Both zero, exactly as the projection picture promised: the residual is perpendicular to the whole column space. Get into the habit of running this check – a single nonzero dot product means an arithmetic slip is hiding somewhere, and the "residuals sum to zero" version (the ones-column check) is the fastest tripwire you have.

Common trap

The residual being orthogonal to the column space is a *theorem*, not a hope, so it is the perfect self-check. Whenever you finish a least-squares fit, dot the residual with each column of $\mathbf{X}$; every dot product must be (numerically) zero. The constant-feature version of this is the familiar fact that least-squares residuals always sum to zero when a bias term is included.

Before/after Strang

This is Strang's least-squares chapter, retold for data. Strang draws the same column space, the same perpendicular, the same normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$ – he just calls the unknown $\hat{\mathbf{x}}$ and the target $\mathbf{b}$. *Before* Strang, let this convince you that "fitting a model" is nothing more exotic than dropping a perpendicular onto a subspace. *After* Strang, notice that the projector $\mathbf{P} = \mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}$ is exactly his hat matrix, and that regularization (next box) is what you reach for when $\mathbf{X}^\mathsf{T}\mathbf{X}$ is too close to singular to invert safely.

34.2.4 When the columns nearly collide: regularization

What if two features are almost the same – say, temperature in Celsius and temperature in Fahrenheit? Then two columns of $\mathbf{X}$ are nearly parallel, $\mathbf{X}^\mathsf{T}\mathbf{X}$ is nearly singular (a tiny eigenvalue), and $(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}$ blows up: the weights become enormous and wildly sensitive to noise. The geometric reading is that the column space is nearly degenerate, so the projection is well-defined but the *coordinates* of the projection (the weights) are not pinned down.

The usual fix is **ridge regression**: add a small multiple of the identity before inverting,

$$\mathbf{w} = (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}, \qquad \lambda > 0.$$

Adding $\lambda\mathbf{I}$ lifts every eigenvalue of $\mathbf{X}^\mathsf{T}\mathbf{X}$ by λ, so the smallest eigenvalue can no longer be near zero – the matrix is comfortably invertible and the weights stay bounded. You will recognize this as the same trick that stabilizes any ill-conditioned linear system; in ML it doubles as a way to express a *prior* that weights should be small.

34.3 Vectors that mean something: embeddings and similarity

Linear regression assumed the data already arrived as numbers. But much of what we care about is words, images, users, and products – things that are not numbers at all. The central move of modern ML is to *turn those things into vectors*, called **embeddings**, in such a way that geometry encodes meaning. Once a thing is a vector, the entire toolkit of this book applies to it.

34.3.1 From words to vectors

Imagine assigning to every word a point in $\mathbb{R}^d$ (in practice d might be 300 or 4096). We want the assignment to have a property: **words used in similar contexts land near each other.** "King" and "queen" should be close; "king" and "yogurt" should be far. We will not derive the training procedure here – it is itself a learning problem – but we can completely understand how meaning, once embedded, is *read off* with linear algebra.

The tool is the dot product, normalized into the **cosine similarity**:

$$\cos\theta = \frac{\langle \mathbf{u},\, \mathbf{v}\rangle}{\|\mathbf{u}\|\,\|\mathbf{v}\|}.$$

This is the cosine of the angle between the two vectors. It is 1 when they point the same way, 0 when perpendicular, -1 when opposite. It ignores length entirely and cares only about *direction* – which is exactly right for meaning, since a word's importance (often reflected in vector length) should not change what it means.

Intuition

Why *cosine* and not plain distance? Because direction is the carrier of meaning. Two documents, one short and one long, can be "about the same thing" even though their raw vectors have very different lengths. Cosine similarity normalizes the lengths away and compares only the directions they point – the topic, not the volume. Geometrically: project both vectors onto the unit sphere, then measure the angle between them.

Pause

Pause and think. What is the cosine similarity between any nonzero vector $\mathbf{v}$ and its negative $-\mathbf{v}$? What about between $\mathbf{v}$ and $10\mathbf{v}$? Notice how cosine similarity is completely blind to scale, yet acutely sensitive to whether two vectors point in opposite directions.

34.3.2 A worked similarity

Take three tiny 2-dimensional "embeddings":

$$\mathbf{u}_{\text{king}} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{u}_{\text{queen}} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \quad \mathbf{u}_{\text{yogurt}} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}.$$

Compare king with queen:

$$\cos\theta = \frac{(2)(3)+(1)(2)}{\sqrt{2^2+1^2}\,\sqrt{3^2+2^2}} = \frac{8}{\sqrt{5}\,\sqrt{13}} = \frac{8}{\sqrt{65}} \approx \frac{8}{8.06} \approx 0.992.$$

Nearly 1: very similar, the two arrows almost overlap. Now king with yogurt:

$$\cos\theta = \frac{(2)(-1)+(1)(3)}{\sqrt{5}\,\sqrt{10}} = \frac{1}{\sqrt{50}} \approx \frac{1}{7.07} \approx 0.141.$$

Close to 0: nearly perpendicular, nearly unrelated. The geometry has correctly sorted royalty from breakfast.

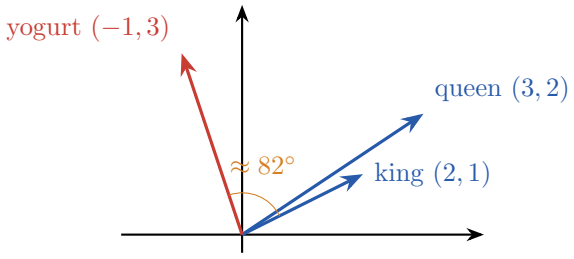


Figure 34.3: Embeddings make meaning geometric, drawn with the exact vectors above. "King" and "queen" point almost the same way (cosine ≈ 0.99, an angle of about 7°); "yogurt" points off at nearly a right angle (cosine ≈ 0.14, about 82°). Similarity is an angle.

34.3.3 Word arithmetic and the meaning of differences

The single most famous demonstration that embeddings capture structure is the analogy

$$\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}}.$$

Read it geometrically. The *difference vector* $\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}}$ is the displacement that, in this learned space, points along the "royalty" axis. Add it to "woman" and you arrive near "queen" (Figure 34.4). That a consistent direction in the embedding space corresponds to a consistent semantic relationship (gender, tense, plurality, capital-of-country) is the headline result of word embeddings – and it relies entirely on the vector addition and parallelogram rule you met in the very first chapter.

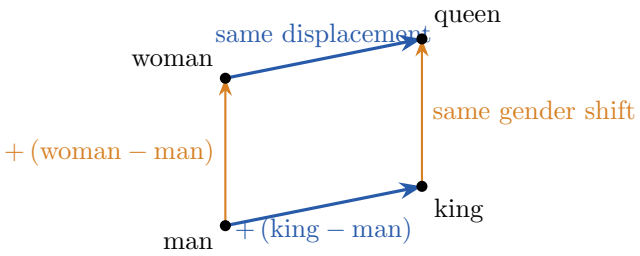


Figure 34.4: Word arithmetic is the parallelogram rule. The displacement $\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}}$ (blue) is a fixed "royalty" direction; the same blue displacement carries "woman" to "queen." Equivalently, the "gender" displacement (orange) is shared by the man→woman and king→queen edges. The four words sit at the corners of a parallelogram, so $\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}}$.

Modern application

Semantic search and retrieval. When you type a question into a modern search box, or when a retrieval-augmented language model fetches relevant documents, here is the machinery: the query is embedded into a vector $\mathbf{q}$, every document is pre-embedded into a vector $\mathbf{d}_i$, and the system returns the documents whose embeddings have the largest cosine similarity to $\mathbf{q}$. Stack all document vectors as rows of a matrix $\mathbf{D}$ and the entire search is one matrix–vector product $\mathbf{Dq}$ followed by "take the largest entries." Billions of documents, searched by meaning, reduce to a matrix hitting a vector. Specialized "vector databases" exist for exactly this and nothing more.

Connection to other math

Inner-product geometry. Cosine similarity is the concrete realization of the abstract *inner product*. The Cauchy–Schwarz inequality $|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\|\|\mathbf{v}\|$ is precisely the statement that cosine lives in $[-1, 1]$ – the angle is always real. And the "orthogonal = unrelated" intuition is the same one that makes Fourier analysis work: there, perpendicular sine waves carry independent information, exactly as perpendicular embedding directions carry independent meanings. The same inner-product geometry applies across both settings.

34.4 A neural network layer is $\mathbf{W}\mathbf{x} + \mathbf{b}$, then a bend

Now we build the object at the center of modern AI. A neural network is a stack of *layers*, and a single layer is almost entirely linear algebra. Understand one layer and you understand the whole stack, because a deep network is just layers fed into layers.

34.4.1 The two-step recipe

A layer takes an input vector $\mathbf{x} \in \mathbb{R}^d$ and produces an output vector $\mathbf{a} \in \mathbb{R}^m$ in two steps:

1. **Affine step (linear algebra):** compute $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $\mathbf{W} \in \mathbb{R}^{m \times d}$ is a weight matrix and $\mathbf{b} \in \mathbb{R}^m$ a bias vector.
2. **Nonlinear step (the bend):** apply a fixed nonlinear function σ to each entry, $\mathbf{a} = \sigma(\mathbf{z})$.

That is the entire layer: $\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$. The matrix $\mathbf{W}$ does all the mixing; the function σ does all the bending.

You understand the affine step deeply by now. $\mathbf{W}\mathbf{x}$ takes a linear combination of the input features (each output is a weighted vote over the inputs), and $+\mathbf{b}$ shifts the result. Each row of $\mathbf{W}$ is one "detector": it dots the input against a stored pattern, large when the input resembles that pattern.

34.4.2 Why the bend is non-negotiable

The most important structural fact about neural networks is a pure linear algebra argument. **Without the nonlinearity, a deep network collapses to a single layer.** Suppose you stacked two affine steps with no bend in between:

$$\mathbf{z}_2 = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}'\mathbf{x} + \mathbf{b}'.$$

The composition of two affine maps is *again one affine map*, with $\mathbf{W}' = \mathbf{W}_2\mathbf{W}_1$. Ten layers, a hundred layers – without bends they all multiply out into a single matrix. You would have spent enormous effort to build a glorified linear regression. The nonlinearity σ is precisely what breaks this collapse and lets depth buy you genuinely new, curved decision boundaries.

Focus

Matrices mix; nonlinearities bend. Stacking matrices alone gets you one bigger matrix (composition of linear maps is linear). The nonlinear σ between layers is the only reason a

deep network can do something a single matrix cannot. Linear algebra supplies the power; the bend supplies the *novelty*.

History and connection

In 1958, Cornell psychologist Frank Rosenblatt unveiled the Perceptron at the Cornell Aeronautical Laboratory. Built as a five-ton custom computer with motorized potentiometers (variable resistors turned by electric motors to adjust weights), the machine was hailed by the press as the dawn of artificial intelligence. Mathematically, it was a single flat cut through feature space: an affine hyperplane $\mathbf{w}^\mathsf{T}\mathbf{x} + b \gtrless 0$ separating positive from negative examples. Eleven years later, MIT mathematicians Marvin Minsky and Seymour Papert published a devastating geometric critique in their book *Perceptrons* (1969). They proved that a single straight boundary could not even compute the Exclusive-OR (XOR) logic gate—a four-point toy problem where opposite corners of a square share the same label. Because no single straight line can separate two diagonally opposite pairs from each other, Minsky and Papert argued that perceptron-like machines were fundamentally limited. A bitter intellectual feud broke out between connectionists, who believed in learning from data, and the symbolic AI establishment, who favored formal logic rules. The symbolic camp prevailed; funding evaporated, and neural research froze for nearly two decades in the First AI Winter.

The thaw came in 1986, when David Rumelhart, Geoffrey Hinton, and Ronald Williams demonstrated that adding hidden layers with smooth, nonlinear bends could solve XOR and discover internal representations. In 1989, Yann LeCun showed that multi-layer networks trained this way could reliably read handwritten digits on postal envelopes. The engine driving this was *backpropagation*: the multivariable chain rule written in matrix notation, where forward data multiplies by weight matrices $\mathbf{W}$ and backward error gradients multiply by their transposes $\mathbf{W}^\mathsf{T}$.

By 2017, sequential recurrent models—which processed text one slow token at a time, struggling with memory bottlenecks and fading gradients—gave way to the Transformer of Ashish Vaswani and coauthors. Their key move was replacing sequential step-by-step processing with parallel linear projections into query, key, and value subspaces ($\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$) and scaled dot-product attention. When models subsequently exploded to hundreds of billions of parameters, Edward Hu and colleagues introduced Low-Rank Adaptation (LoRA, 2021), freezing the giant weight matrix and learning only a low-rank update $\Delta\mathbf{W} = \mathbf{B}\mathbf{A}$ with $r \ll d$—compressing millions of gradient calculations onto consumer hardware through the Eckart–Young principle.

Yet basic questions remain open. Why gradient descent on non-convex surfaces with far more parameters than data points reliably discovers flat, generalizable minima rather than memorizing noise remains an unsolved problem. Equally open is the geometry of *superposition*—the phenomenon where networks compress more distinct concepts into activation vectors than there are dimensions by exploiting nearly orthogonal directions in high dimensions.

The most common bend today is the **ReLU**, "rectified linear unit," $\sigma(z) = \max(0, z)$: it passes positive values through and zeros out negatives. It is about as gentle a bend as exists – piecewise linear, just a single kink at the origin – yet stacking many of them lets the network carve input space into many linear pieces and fit highly complex functions.

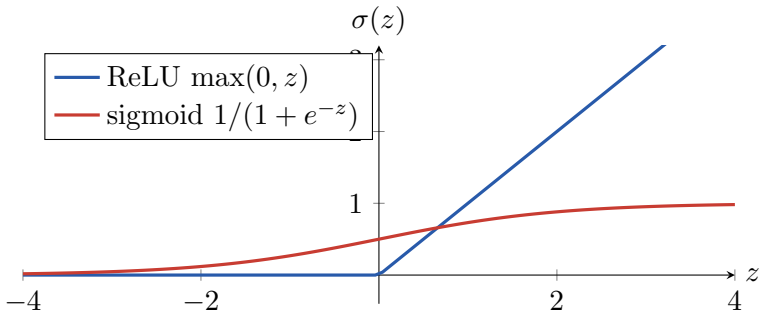


Figure 34.5: The two most common bends. ReLU (blue) is piecewise linear with a single kink at the origin: it zeros out negatives and passes positives through unchanged. The sigmoid (red) smoothly squashes the whole line into $(0, 1)$, which is why it reads a linear score as a probability. The *kink* of ReLU and the *curve* of the sigmoid are the "novelty" that pure matrix multiplication cannot supply.

34.4.3 A layer computed by hand

Let the input be $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and the layer have

$$\mathbf{W} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 0 \\ -5 \\ 1 \end{bmatrix}, \qquad \sigma = \mathrm{ReLU}.$$

First the affine step:

$$\mathbf{W}\mathbf{x} = \begin{bmatrix} 1 \cdot 1 + (-1) \cdot 2 \\ 0 \cdot 1 + 2 \cdot 2 \\ 3 \cdot 1 + 1 \cdot 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 4 \\ 5 \end{bmatrix}, \qquad \mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} = \begin{bmatrix} -1 \\ -1 \\ 6 \end{bmatrix}.$$

Then the bend, applied entrywise:

$$\mathbf{a} = \mathrm{ReLU}(\mathbf{z}) = \begin{bmatrix} \max(0, -1) \\ \max(0, -1) \\ \max(0, 6) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 6 \end{bmatrix}.$$

Two of the three neurons "switched off" (their pre-activations were negative); one fired with value 6. That pattern of on/off across neurons is how a network represents "which features are present" in this particular input.

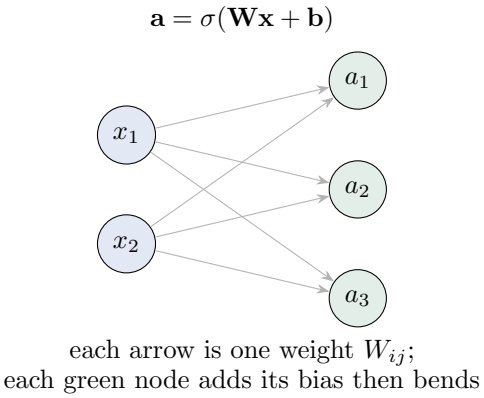


Figure 34.6: A single fully connected layer. Every input feeds every output through a weight; the weight matrix $\mathbf{W}$ *is* this complete bipartite wiring. The affine step $\mathbf{W}\mathbf{x} + \mathbf{b}$ is the wiring; the nonlinearity is applied inside each green node.

34.4.4 Classification and logistic regression

For a yes/no decision (spam or not, tumor or not), we want the model to output a *probability* in $[0, 1]$. The trick is to take a linear score and squash it through the **sigmoid** (logistic) function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \mathbf{w} \cdot \mathbf{x} + b.$$

This is **logistic regression**, and structurally it is exactly one neuron: an affine step followed by a bend (the smooth red curve of Figure 34.5). The linear part $\mathbf{w} \cdot \mathbf{x} + b$ defines a *hyperplane* in feature space – the set where the score is zero – and the sigmoid turns "how far are you from the plane, and on which side" into a probability. Points far on the positive side get probability near 1, far on the negative side near 0, and right on the plane exactly 0.5.

Intuition

A linear classifier is a *ruled line* (in 2D), *plane* (in 3D), or *hyperplane* (in general) that splits feature space into two half-spaces. The weight vector $\mathbf{w}$ is the **normal** to that hyperplane – it points in the direction of "more positive class." The bias b slides the hyperplane toward or away from the origin. Training a linear classifier is choosing the orientation $\mathbf{w}$ and the offset b of one flat cut through the data.

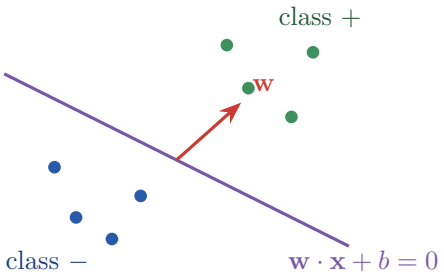


Figure 34.7: A linear classifier. The decision boundary is the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$; the weight vector $\mathbf{w}$ is its normal, pointing toward the positive class. Logistic regression wraps the signed distance to this plane in a sigmoid to read it as a probability.

34.5 Training: gradient descent and the gradient as a vector

We have models – regression, logistic, layers. We have not yet said how the optimal weights are determined, except in the one case (linear regression) where a closed-form projection yields the answer. For general models and loss functions, we rely on the primary optimization method of modern machine learning: **gradient descent**. And the gradient is, once more, a vector you can picture geometrically.

34.5.1 The loss surface

Pick a way to measure how wrong the model is – a *loss* $L(\mathbf{w})$, a single number that depends on all the weights. For regression, the natural loss is the squared error $L(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. As $\mathbf{w}$ ranges over weight space, L traces out a curved surface of hills and valleys sitting above weight space. Training means **finding the lowest point on this surface**.

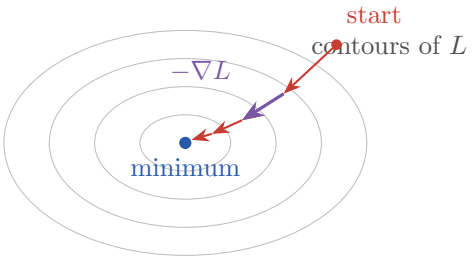


Figure 34.8: Gradient descent. The contours are level sets of the loss $L(\mathbf{w})$. At each point the gradient ∇L points uphill (steepest ascent), so we step in the direction $-\nabla L$ and roll toward the minimum. Each red arrow is one update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$.

34.5.2 The gradient points uphill

The **gradient** ∇L is the vector of partial derivatives, one per weight:

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \ldots, \frac{\partial L}{\partial w_d} \right).$$

Its defining geometric property is the one to memorize: **at any point, ∇L points in the direction of steepest increase of L, and its length is the rate of that increase.** So $-\nabla L$ points in the direction of steepest *decrease* – straight downhill. To minimize the loss, repeatedly step a little way downhill:

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})}$$

where $\eta > 0$ is the **learning rate**, the step size. That single update rule, iterated, trains essentially every model in deep learning.

Intuition

You are standing on a foggy hillside and want to reach the valley. You cannot see far, but you can feel the slope under your feet. The gradient is the direction the ground rises fastest; turn around and take a step. Repeat. The learning rate is your stride length: too small and you crawl; too large and you may overshoot the valley and bounce up the far slope. Gradient descent turns this local downhill step into an iterative algorithm.

34.5.3 Why $-\nabla L$ is steepest: a one-line derivation

This is worth deriving, because it ties the gradient to the dot product. The change in L when you step a tiny amount $\mathbf{u}$ (a unit direction) is, to first order, the *directional derivative*

$$\frac{d}{dt}\Big|_{t=0} L(\mathbf{w} + t\mathbf{u}) = \nabla L \cdot \mathbf{u} = \|\nabla L\| \, \|\mathbf{u}\| \cos\theta = \|\nabla L\| \cos\theta,$$

using $\|\mathbf{u}\| = 1$. Among all unit directions $\mathbf{u}$, this dot product is *largest* when $\cos\theta = 1$, i.e. $\mathbf{u}$ points along ∇L, and *smallest* (most negative) when $\mathbf{u}$ points along $-\nabla L$. So $-\nabla L$ is the steepest-descent direction. The whole justification for gradient descent is one application of "the dot product is maximized when the vectors align."

34.5.4 Gradient descent on our regression

Let us connect the two methods. Expand the loss as a dot product, $L(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \mathbf{y}^\mathsf{T}\mathbf{y} - 2(\mathbf{X}^\mathsf{T}\mathbf{y})^\mathsf{T}\mathbf{w} + \mathbf{w}^\mathsf{T}(\mathbf{X}^\mathsf{T}\mathbf{X})\mathbf{w}$: a constant, a linear term, and a quadratic form. Differentiating term by term (the linear term contributes $-2\mathbf{X}^\mathsf{T}\mathbf{y}$, the symmetric quadratic form $2\mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{w}$) gives

$$\nabla L = -2\,\mathbf{X}^\mathsf{T}(\mathbf{y} - \mathbf{X}\mathbf{w}) = 2\big(\mathbf{X}^\mathsf{T}\mathbf{X}\,\mathbf{w} - \mathbf{X}^\mathsf{T}\mathbf{y}\big).$$

Setting $\nabla L = \mathbf{0}$ – the condition for a flat spot, the bottom of the bowl – gives back exactly the normal equations $\mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{w} = \mathbf{X}^\mathsf{T}\mathbf{y}$. The closed-form projection answer and the gradient-descent answer are the *same point*, the minimum of the same bowl. Linear regression is the rare case where you can jump to the bottom in one step; gradient descent is what you use when no such jump exists.

Compute it yourself

Minimize $L(w) = (w-3)^2$ by hand-running gradient descent. Here $\nabla L = 2(w-3)$. Start at $w_0 = 0$ with learning rate $\eta = 0.25$. Compute $w_1 = w_0 - \eta \cdot 2(w_0 - 3)$, then w_2, then w_3. Watch the iterates march toward 3. Then try $\eta = 1.1$ and watch them *diverge* – a concrete feel for why the learning rate must not be too large.

34.6 Backpropagation: the multivariable chain rule as matrix multiplication

Gradient descent needs the gradient. For a deep network with millions of weights, how do you compute $\partial L/\partial W_{ij}$ for every single weight, efficiently? The answer is **backpropagation**, which is the multivariable chain rule from calculus organized as a sequence of matrix multiplications by *Jacobians*.

34.6.1 The Jacobian: the local linear map

A neural network is a composition of functions: input $\to$ layer 1 $\to$ layer 2 $\to \cdots \to$ loss. Each function maps vectors to vectors. The right object to describe how a vector-to-vector function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ stretches space *near a point* is its **Jacobian matrix** $\mathbf{J}$, the $m \times n$ matrix of all partial derivatives, $J_{ij} = \partial f_i/\partial x_j$.

Intuition

Zoom in far enough on any smooth function and it looks linear – that linear approximation is the Jacobian. The Jacobian is the matrix that *is* the derivative of a vector function: $\mathbf{f}(\mathbf{x} + \Delta\mathbf{x}) \approx \mathbf{f}(\mathbf{x}) + \mathbf{J}\,\Delta\mathbf{x}$. Every layer of a network has a Jacobian – the local matrix that says how a small wiggle in its input becomes a small wiggle in its output. Notice that for the affine step $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, the Jacobian with respect to $\mathbf{x}$ is exactly $\mathbf{W}$ itself, because an affine map *is* its own linear approximation.

34.6.2 Chain rule as Jacobian product

For a composition $\mathbf{f} = \mathbf{f}_3 \circ \mathbf{f}_2 \circ \mathbf{f}_1$, the multivariable chain rule says the Jacobian of the whole is the *product* of the Jacobians of the parts:

$$\mathbf{J}_{\mathbf{f}} = \mathbf{J}_3\,\mathbf{J}_2\,\mathbf{J}_1.$$

This is the same "matrices compose by multiplication" fact that has run through the entire book, now applied to derivatives. The loss is a scalar at the end of the chain, so its gradient is obtained by multiplying these Jacobians together. Backpropagation is the specific, clever *order* in which you do that multiplication.

34.6.3 Why multiply starting from the loss (backwards through the net)

The efficiency insight is purely about matrix-multiplication bookkeeping. The chain $\mathbf{J}_{\mathbf{f}} = \mathbf{J}_3\mathbf{J}_2\mathbf{J}_1$ has the loss as the last function $\mathbf{f}_3$, so its Jacobian $\mathbf{J}_3$ is the *leftmost* factor. Because the loss is one number, $\mathbf{J}_3$ is a *row vector* (a $1 \times m$ object): the gradient of a scalar. You can multiply out the product $\mathbf{J}_3\mathbf{J}_2\mathbf{J}_1$ either left-to-right (starting from $\mathbf{J}_3$) or right-to-left (starting from $\mathbf{J}_1$). Multiplying *right-to-left* – start at the input end and build up a running product – forces you to carry around full $m \times n$ matrices at each stage. Multiplying **left-to-right, starting from the scalar's row vector** $\mathbf{J}_3$, keeps a row vector at every stage:

$$\underbrace{\mathbf{J}_3}_{1\times m}\ \underbrace{\mathbf{J}_2}_{m\times k} = (\text{row})_{1\times k}, \quad \text{then} \times \mathbf{J}_1.$$

A row vector times a matrix is another row vector – cheap. **That** is backpropagation: propagate the gradient *backward* through the network, layer by layer, as a vector being hit by each layer's Jacobian (in transposed form). The "backward" in the name is just the choice to multiply the Jacobian chain in the order that keeps everything a vector instead of a matrix (Figure 34.9). Where does the "transposed form" come from? Look at one layer, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ followed by $\mathbf{a} = \sigma(\mathbf{z})$, and write gradients as columns. Given the upstream gradient $\nabla_{\mathbf{a}} L$, the elementwise bend scales each coordinate, $\delta = \nabla_{\mathbf{z}} L = \nabla_{\mathbf{a}} L \odot \sigma'(\mathbf{z})$. Since $z_i = \sum_j W_{ij} x_j + b_i$, we have $\partial z_i / \partial x_j = W_{ij}$ and $\partial z_i / \partial W_{ij} = x_j$, so one backward step yields

$$\nabla_{\mathbf{x}} L = \mathbf{W}^{\mathsf{T}} \delta, \qquad \nabla_{\mathbf{W}} L = \delta\, \mathbf{x}^{\mathsf{T}}, \qquad \nabla_{\mathbf{b}} L = \delta.$$

The row-vector product $\delta^{\mathsf{T}}\mathbf{W}$, turned back into a column, is $\mathbf{W}^{\mathsf{T}}\delta$: that is the transpose. And every weight gradient is a rank-one outer product, pairing the error δ_i arriving at output i with the activation x_j that fed it.

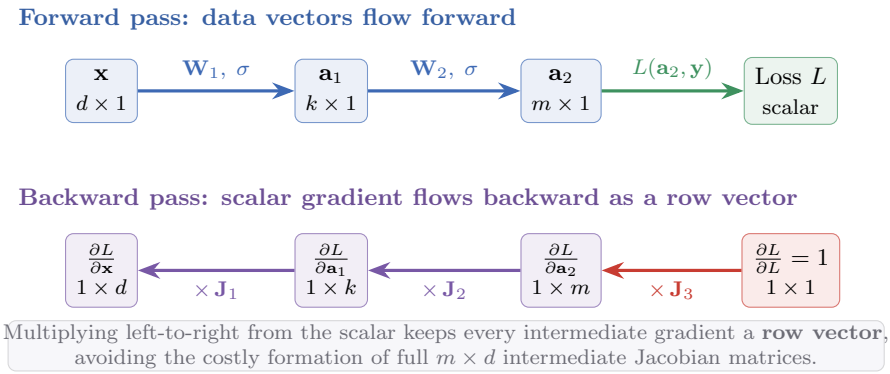


Figure 34.9: Forward evaluation versus backward propagation. Forward: input vector $\mathbf{x}$ is transformed layer-by-layer into the scalar loss L. Backward: the chain rule multiplies Jacobians from the left starting at the scalar output. Each step computes (row vector) × (Jacobian matrix) = (new row vector), never materializing a full matrix-matrix product.

Focus

Forward pass: push the input vector through the layers, computing each $\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ and remembering the intermediate values. **Backward pass:** push the gradient (a vector) back through the layers, multiplying by each layer's Jacobian, accumulating $\partial L / \partial \mathbf{W}$ along the way. Forward is matrices acting on data; backward is the same matrices (transposed) acting on gradients. Same linear algebra, run in reverse.

Connection to other math

Backpropagation as the multivariable chain rule. Everything "deep learning" about backprop is engineering – caching intermediate activations, doing it on a GPU. The mathematics is the chain rule $\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$ generalized to vectors, where the single-variable derivatives become Jacobian matrices and ordinary multiplication becomes matrix multiplication. If you have done a multivariable calculus course, you have already proven the hard part. Linear algebra supplies the only new idea: that the chain of derivatives is a chain of matrices, so the *order* of multiplication is a free choice you can exploit.

34.7 Attention: scaled dot products and matrix weights

The architecture behind modern language models – the *transformer* – is built on a mechanism called **attention**. From a mathematical perspective, attention is a structure you are now fully equipped to analyze: it is a set of dot products, scaled and normalized via softmax, used to form a weighted average. Let us assemble it from first principles.

34.7.1 Queries, keys, values

Imagine a sequence of tokens (say, words in a sentence), each already an embedding vector. Attention lets each token "look at" the others and pull in the information it needs. To do that, every token is turned into three vectors by three learned matrices:

- a **query q** – "what am I looking for?"
- a **key k** – "what do I offer?"
- a **value v** – "what will I hand over if chosen?"

Stack the queries as rows of $\mathbf{Q}$, the keys as rows of $\mathbf{K}$, the values as rows of $\mathbf{V}$. Each is just the embedding matrix times a learned weight matrix – linear algebra you have done a dozen times.

34.7.2 The core formula, read piece by piece

The famous formula is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V}.$$

It looks dense; take it apart and every piece is something from this book.

Step 1: the scores $\mathbf{Q}\mathbf{K}^{\mathsf{T}}$. This $n \times n$ matrix is, entry by entry, $(\mathbf{Q}\mathbf{K}^{\mathsf{T}})_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ – the dot product of token i's query with token j's key. It measures how much token i should care about token j. This is the Gram-matrix idea from Section 34.1 again: *all pairwise similarities at once*, as a single matrix product. Large dot product means "these two are relevant to each other."

Step 2: the scaling $1/\sqrt{d_k}$. If the coordinates of a query and a key are roughly independent with unit variance, their dot product is a sum of d_k such products, so it has variance d_k and typical size $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ brings the scores back to unit scale. Without it, large scores would push the softmax in Step 3 into saturation – nearly all weight on one entry – where its gradients vanish and training stalls.

Step 3: softmax, row by row. The softmax turns each row of scores into a probability distribution – nonnegative numbers that sum to 1:

$$\text{softmax}(\mathbf{s})_j = \frac{e^{s_j}}{\sum_k e^{s_k}}.$$

After this, row i holds the *attention weights*: how much of its new representation token i should draw from each other token. (This is the same exponential-normalization idea as the sigmoid, generalized to many outputs.)

Step 4: the weighted sum $(\cdots)\mathbf{V}$. Multiplying the attention-weight matrix by $\mathbf{V}$ takes, for each token, a *weighted average of all the value vectors*, weighted by relevance. A matrix of probabilities times a matrix of values is a matrix of convex combinations – output row i is a blend of the value vectors, heaviest on the tokens token i attended to.

Intuition

Attention is "soft lookup." A hard lookup (like a dictionary) takes a key and returns exactly one value. Attention instead compares your query to *every* key by dot product, turns those similarities into weights that sum to one, and returns a *blend* of all the values weighted by how well each key matched. The dot product asks "how relevant?"; the softmax turns relevance into "how much weight"; the multiply by $\mathbf{V}$ collects the blend. Three operations, all of them linear algebra plus one exponential.

Pause

Pause and think. Suppose token 1's query vector $\mathbf{q}_1$ happens to be orthogonal to every key vector $\mathbf{k}_j$ in the sequence. What are the dot-product scores $\mathbf{q}_1 \cdot \mathbf{k}_j$? What attention weights does the softmax produce? (Because every score is zero, every exponent e^0 is 1, yielding an equal split: $\alpha_{1j} = 1/n$ across all n tokens.)

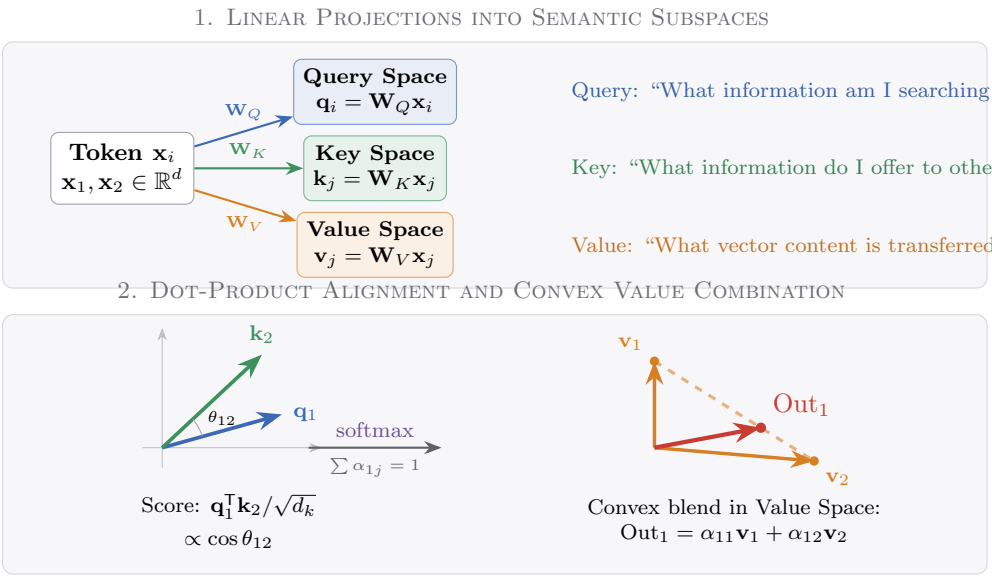


Figure 34.10: The geometric mechanism of self-attention. Top: Input token vectors are linearly projected into Query, Key, and Value semantic subspaces via learned weight matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$. Bottom: The query $\mathbf{q}_1$ acts as a directional probe, evaluating its dot-product projection against each key $\mathbf{k}_j$. Softmax normalizes these scores into non-negative weights summing to 1, which then synthesize a convex combination of value vectors in Value Space.

Modern application

Why transformers scale. The reason attention powers modern language models is that at its operational core lies dense matrix multiplication – $\mathbf{QK}^\mathsf{T}$ and then $(\cdots)\mathbf{V}$ – which is

exactly the operation GPUs are built to do astronomically fast. A model with hundreds of billions of parameters is, in the inner loop, a long sequence of big matrix multiplies. When you read that a language model was "trained on thousands of GPUs," the thing those GPUs were multiplying, over and over, were matrices of queries, keys, values, and weights. The linear algebra in this book is, quite literally, the computational engine of modern AI.

Common trap

It is tempting to think attention is "mostly nonlinear and mysterious." It is the opposite: the only nonlinearity in the whole formula is the softmax. Everything else – forming $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, scoring with $\mathbf{Q}\mathbf{K}^\mathsf{T}$, blending with $\mathbf{V}$ – is matrix multiplication. If you can multiply matrices and you understand the dot product as similarity, you understand the skeleton of a transformer.

34.8 Low rank: why a little structure goes a long way

We close with the idea that quietly makes large-scale ML feasible: **low-rank structure.** It is the SVD chapter cashed out in practice, and it explains both why models can be compressed and why they generalize at all.

34.8.1 Rank as the number of independent directions

The **rank** of a matrix is the count of independent directions in its column space – the true dimensionality of what it does. A matrix can be huge, 1000×1000, and yet have rank 5: its million entries are really controlled by only a handful of underlying patterns. The SVD makes this precise. Any matrix factors as

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\mathsf{T},$$

a sum of rank-one pieces, each a single direction $\mathbf{u}_i$ scaled by a singular value σ_i. The singular values come in decreasing order, $\sigma_1 \geq \sigma_2 \geq \cdots$, so the first few terms carry most of the action.

Intuition

A rank-one matrix $\mathbf{u}\mathbf{v}^\mathsf{T}$ is the simplest non-trivial matrix there is: every row is a multiple of $\mathbf{v}^\mathsf{T}$, every column a multiple of $\mathbf{u}$. It stores $m + n$ numbers but spreads them across $m \times n$ entries – pure structure, no independent detail. The SVD says *every* matrix is a stack of these, and that the σ_i tell you how much each layer matters. "Low rank" means a few layers already tell almost the whole story.

34.8.2 Low-rank approximation: keep the loud directions

The payoff is a theorem of Eckart and Young: the *best* rank-k approximation of any matrix – best in the least-squares sense – is obtained by keeping the top k terms of the SVD and throwing the rest

away:

$$\mathbf{A} \approx \mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}.$$

If the singular values decay quickly (and for real data they almost always do), then a small k captures almost all of $\mathbf{A}$ (Figure 34.11), and you have compressed an $m \times n$ matrix (mn numbers) down to $k(m+n)$ numbers (absorbing each σ_i into $\mathbf{u}_i$). For a 1000×1000 matrix kept at rank 10, that is a 50-fold compression.

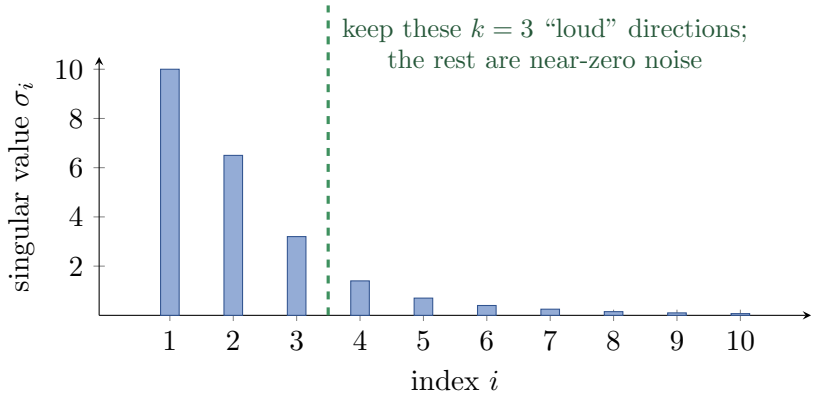


Figure 34.11: Why low-rank approximation works. For typical data the singular values σ_i fall off fast, so a few leading rank-one pieces $\sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ carry almost all of the matrix. Truncating after k terms (here $k = 3$, dashed line) is the Eckart–Young best rank-k approximation and discards only the small "tail."

Modern application

Recommender systems and model compression. The famous Netflix-style recommender is a low-rank story: the giant "users × movies" ratings matrix is modeled as a product of a thin "users × taste" matrix and a thin "taste × movies" matrix – a low-rank factorization that fills in the blanks you have not rated. The same idea now compresses enormous neural networks: a method called LoRA ("low-rank adaptation") fine-tunes a giant model by adding only a low-rank update $\mathbf{BA}$ to its weight matrices, training a few million numbers instead of billions.

34.8.3 Principal component analysis

Run the SVD on a centered data matrix and you have **PCA**, the most-used dimensionality-reduction method in all of data science. The top singular directions $\mathbf{v}_i$ are the *principal components* – the orthogonal directions along which your data varies most. Projecting the data onto the first two or three of them is how a high-dimensional dataset gets drawn as a 2D scatter plot you can actually look at. The directions of greatest variance are exactly the leading eigenvectors of the covariance matrix $\mathbf{X}_c^{\mathsf{T}} \mathbf{X}_c$ – the second geometry of your data matrix from Section 34.1, returning to close the loop.

Why this matters

Across this chapter, the dot product gave us *similarity*. Projection gave us *regression*. Matrix multiplication gave us *layers, attention, and the chain rule*. The gradient gave us *learning*. The SVD gave us *compression and the principal directions of data*. Modern machine learning relies on these core linear-algebraic structures, scaled up and run on parallel hardware.

Walking question

Take a walk and answer these without paper. (1) A neural network with ten layers but no nonlinearities – what single, simpler model is it equivalent to, and why? (2) Cosine similarity ignores vector length; name one situation where that is exactly the right thing to ignore, and one where it might throw away something you care about. (3) Gradient descent steps in the direction $-\nabla L$. If the loss surface were a perfectly round bowl, would the path to the bottom be straight or curved? What about a long, narrow valley?

Honest limits and computational traps

At scale, four mathematical bottlenecks confront every machine-learning system.

First, standard scaled dot-product attention carries a quadratic memory wall. To compare every token to every other token in a sequence of length N, the model must materialize an $N \times N$ matrix of attention scores. For a modern context of $N = 100{,}000$ tokens, storing that single matrix for every attention head consumes tens of gigabytes of GPU memory. The search for sub-quadratic, sparse, and low-rank linear attention is an active race across applied mathematics.

Second, repeated products of matrices are numerically treacherous. Pushing gradients back through fifty or a hundred layers means evaluating long product chains $\prod_{l=1}^{L} \mathbf{W}_l^{\mathsf{T}}$. As we saw in Chapter 33, if the singular values of these matrices systematically exceed 1, gradients explode exponentially into floating-point overflow; if they fall below 1, gradients vanish exponentially to machine zero. Modern architectures survive only by adding identity bypasses (residual skip connections) and normalizing activations at every layer to keep the spectrum bounded near unity.

Third, the safety of convexity is gone. Unlike linear regression, where the squared error is a convex quadratic bowl with a guaranteed global minimum reachable via normal equations or stable QR decomposition, neural loss surfaces twist into non-convex shapes. They are crowded with saddle points, flat plateaus, and ill-conditioned ravines where gradient descent can oscillate or stall without second-order curvature information.

Finally, there is the puzzle of internal representations. Although every forward and backward step is transparent linear algebra, the representations learned in intermediate layers remain difficult to interpret. High-dimensional vector spaces allow "superposition," where networks pack vastly more concepts into an activation vector than it has dimensions, aligning distinct features with nearly orthogonal directions. We can compute the numbers with complete precision, but reading off human-interpretable meaning from arbitrary coordinates remains an open frontier.

Where we go next in the book

Throughout this chapter, our data matrix $\mathbf{X}$ treated each example as an isolated row in a feature space. In Chapter 35, we connect these points into relational webs, using adjacency and Laplacian matrices to study networks, random walks, and Markov chains.

34.9 Exercises

Exercise 34.1. A data matrix has rows $(2,4),(4,6),(6,8)$. (a) Compute the column-mean vector and write the centered matrix $\mathbf{X}_c$. (b) Verify each column of $\mathbf{X}_c$ sums to zero. (c) Compute the 2×2 matrix $\mathbf{X}_c^{\mathsf{T}}\mathbf{X}_c$. Are the two features perfectly correlated? Explain geometrically what you see in the centered points.

Exercise 34.2. Fit a line $\hat{y} = wx + b$ by least squares to the points $(0,1),(1,1),(2,3)$. (a) Write $\mathbf{X}$ (with a ones-column) and $\mathbf{y}$. (b) Form and solve the normal equations $\mathbf{X}^{\mathsf{T}}\mathbf{X}\mathbf{w} = \mathbf{X}^{\mathsf{T}}\mathbf{y}$. (c) Compute the residual vector and verify it is orthogonal to both columns of $\mathbf{X}$.

Exercise 34.3. Given the embeddings $\mathbf{a} = \begin{bmatrix}1\\0\\1\end{bmatrix}$, $\mathbf{b} = \begin{bmatrix}1\\1\\0\end{bmatrix}$, $\mathbf{c} = \begin{bmatrix}0\\0\\2\end{bmatrix}$, compute all three pairwise cosine similarities. Rank the pairs from most to least similar, and say which pair is closest to orthogonal.

Exercise 34.4. A single neural-network layer has $\mathbf{W} = \begin{bmatrix}2 & 0\\ -1 & 1\\ 0 & 3\end{bmatrix}$, $\mathbf{b} = \begin{bmatrix}-3\\0\\-2\end{bmatrix}$, and ReLU activation. Compute the output for input $\mathbf{x} = \begin{bmatrix}1\\1\end{bmatrix}$. Which neurons fire?

Exercise 34.5. Show that stacking three affine layers $\mathbf{x} \mapsto \mathbf{W}_3(\mathbf{W}_2(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2)+\mathbf{b}_3$ with no nonlinearities equals a *single* affine map $\mathbf{W}'\mathbf{x}+\mathbf{b}'$. Give explicit formulas for $\mathbf{W}'$ and $\mathbf{b}'$. What does this prove about deep linear networks?

Exercise 34.6. (Gradient descent by hand.) Let $L(w) = w^2 - 4w + 5$. (a) Compute $\nabla L = L'(w)$ and find the minimizer analytically. (b) Run three steps of gradient descent from $w_0 = 0$ with learning rate $\eta = 0.3$. (c) Repeat with $\eta = 1.2$ and describe what happens. What property of η separates convergence from divergence here?

Exercise 34.7. Prove that for a unit vector $\mathbf{u}$, the directional derivative $\nabla L \cdot \mathbf{u}$ is maximized when $\mathbf{u}$ points along ∇L and minimized when it points along $-\nabla L$. (Use $\nabla L \cdot \mathbf{u} = \|\nabla L\|\cos\theta$.) Explain in one sentence why this is the entire justification for the gradient-descent update rule.

Exercise 34.8. Let $\mathbf{Q} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$ (two query rows) and $\mathbf{K} = \begin{bmatrix}1 & 0\\ 1 & 1\end{bmatrix}$ (two key rows). (a) Compute the score matrix $\mathbf{Q}\mathbf{K}^{\mathsf{T}}$. (b) Apply softmax to each row (you may leave answers in terms of e). (c) With $\mathbf{V} = \begin{bmatrix}2 & 0\\ 0 & 2\end{bmatrix}$, compute the attention output. Interpret which token each query attended to most.

Exercise 34.9. A rank-one matrix is $\mathbf{A} = \mathbf{u}\mathbf{v}^{\mathsf{T}}$ with $\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. (a) Write out $\mathbf{A}$ explicitly (it is 3×2). (b) Verify by inspection that every column is a multiple of $\mathbf{u}$ and every row a multiple of $\mathbf{v}^{\mathsf{T}}$. (c) How many numbers does it take to store $\mathbf{A}$ as a rank-one product versus as a full 3×2 table? Generalize to $m \times n$.

Exercise 34.10. (Challenge.) The covariance-style matrix $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ and the Gram matrix $\mathbf{X}\mathbf{X}^{\mathsf{T}}$ generally have different sizes. Prove that they nonetheless share the same nonzero eigenvalues. (Hint: if $\mathbf{X}^{\mathsf{T}}\mathbf{X}\,\mathbf{v} = \lambda\mathbf{v}$ with $\lambda \neq 0$, consider the vector $\mathbf{X}\mathbf{v}$ and show it is an eigenvector of $\mathbf{X}\mathbf{X}^{\mathsf{T}}$ with the same λ.) Explain why this is exactly why PCA can be computed from whichever of the two matrices is smaller.

Chapter 35

Graphs, Networks, and Markov Chains

In Chapter 34, we treated data as isolated vectors in high-dimensional feature spaces. But what happens when data points are defined not by their isolated coordinates, but by how they connect to one another?

So far in this book the vectors and matrices have mostly described *geometry*: arrows in the plane, stretches of the unit square, rotations of space. In this chapter we point the very same machinery at something that does not, at first, look geometric at all: a *network*. A web of friendships. A power grid. The links between web pages. The way a random walker hops from city to city. A spreadsheet of who emails whom.

The remarkable thing is that the structure of a network is captured *exactly* by a matrix, and the deep questions you might ask about the network (Is it connected? Where are its natural communities? Which page is the most important? Where does a random walker end up in the long run?) turn into questions about the *eigenvalues and eigenvectors* of that matrix. The graph provides the combinatorial structure, while the matrix and its spectrum provide the algebraic and geometric tools to analyze it.

By the end of this chapter you will be able to look at a graph and write down three matrices for it (incidence, adjacency, Laplacian); you will know what each row, column, and eigenvalue is telling you; and you will understand PageRank as computing the stationary eigenvector of a transition matrix.

35.1 From pictures to matrices

A **graph** is the simplest object in all of mathematics that still has interesting structure. It is a set of *nodes* (also called *vertices*) and a set of *edges*, where each edge joins two nodes. That is the whole definition. Nodes are dots; edges are the lines between them. Draw five dots, connect some of them with lines, and you have a graph.

Intuition

A graph throws away everything except *who is connected to whom*. Where you place the dots on the page does not matter – only the pattern of connections does. Two drawings that look completely different can be the "same" graph if the connection pattern matches. The job of

linear algebra is to turn that bare connection pattern into numbers we can compute with, and then to recover meaning from those numbers.

Let us fix a small running example we will use throughout. It has four nodes labeled $1, 2, 3, 4$ and five edges. Figure 35.1 shows it.

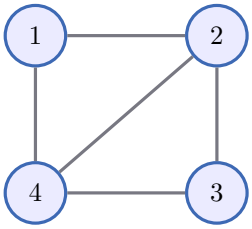


Figure 35.1: Our running graph G: four nodes, five edges. Node 2 touches three edges (it has *degree* 3); node 1 touches two. There is no edge between 1 and 3.

The *degree* of a node is the number of edges touching it. In our graph the degrees are: node 1 has degree 2, node 2 has degree 3, node 3 has degree 2, node 4 has degree 3. A useful sanity check: the degrees sum to twice the number of edges, because every edge contributes 1 to the degree of each of its two endpoints. Here $2 + 3 + 2 + 3 = 10 = 2 \times 5$. Good.

35.1.1 The adjacency matrix

The most direct way to record a graph as numbers is the **adjacency matrix A**. For a graph with n nodes it is the $n \times n$ matrix whose (i, j) entry is

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge between node } i \text{ and node } j, \\ 0 & \text{otherwise.} \end{cases}$$

For our graph, reading off Figure 35.1 row by row:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}.$$

Look at row 2: it reads $1, 0, 1, 1$, meaning node 2 connects to nodes 1, 3, and 4 but not to itself. The diagonal is all zeros (no node has an edge to itself), and the matrix is *symmetric*, $\mathbf{A} = \mathbf{A}^{\mathsf{T}}$, because "$i$ connects to j" is the same statement as "j connects to i." That symmetry is the algebraic shadow of the edges being *undirected*, and it is what will later guarantee real eigenvalues.

Focus

Three facts about an adjacency matrix of an undirected graph, worth holding onto:

- It is symmetric: $\mathbf{A} = \mathbf{A}^{\mathsf{T}}$.
- Its diagonal is zero (for a simple graph with no self-loops).

- The sum of row i equals the degree of node i. Here the row sums are $2, 3, 2, 3$ – exactly the degree sequence.

Why store a graph this way? Because once it is a matrix, *matrix operations have graph meaning.* The first example of this is already a striking one.

Intuition

Powers of A count walks. The entry $(\mathbf{A}^k)_{ij}$ is exactly the number of walks of length k from node i to node j (a walk is a sequence of edges, repeats allowed). Multiplying $\mathbf{A}$ by itself is the same as "take one more step along an edge," and matrix multiplication adds up all the ways to do that.

Let us verify the first nontrivial case. The (i, j) entry of $\mathbf{A}^2$ is $\sum_k A_{ik}A_{kj}$. The product $A_{ik}A_{kj}$ is 1 exactly when there is an edge $i-k$ *and* an edge $k-j$, i.e. when k is a common neighbor giving a length-2 walk $i \to k \to j$. Summing over k counts all such intermediate nodes. So $(\mathbf{A}^2)_{ij}$ counts length-2 walks. Let us compute it for our graph:

$$\mathbf{A}^2 = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 2 & 1 \\ 1 & 3 & 1 & 2 \\ 2 & 1 & 2 & 1 \\ 1 & 2 & 1 & 3 \end{bmatrix}.$$

Read the diagonal: $(\mathbf{A}^2)_{22} = 3$. That says there are three length-2 walks from node 2 back to node 2 – namely out to each of its 3 neighbors and back. In general $(\mathbf{A}^2)_{ii}$ is just the degree of node i. The off-diagonal entry $(\mathbf{A}^2)_{13} = 2$ says there are two length-2 paths from 1 to 3: through node 2, and through node 4. Trace your finger over Figure 35.1 to confirm it.

Pause

Pause and think: what does $\mathbf{A}^3$ say about triangles? Every closed walk of length 3 starting and ending at the same node must trace three edges forming a triangle. Since each triangle has 3 corners and can be traversed in 2 directions (clockwise or counter-clockwise), the sum of the diagonal entries $\operatorname{tr}(\mathbf{A}^3) = \sum_i (\mathbf{A}^3)_{ii}$ counts each triangle exactly $3 \times 2 = 6$ times. So the number of triangles in an undirected graph is precisely $\frac{1}{6}\operatorname{tr}(\mathbf{A}^3)$. Matrix multiplication counts geometric shapes without ever drawing them.

Compute it yourself

Compute $\mathbf{A}^3$ for the graph above and read off $(\mathbf{A}^3)_{12}$. It should equal the number of length-3 walks from node 1 to node 2. Then find them by hand on the picture (start at 1, take three edges, end at 2). Do the two answers agree?

35.1.2 Directed graphs and the adjacency matrix that is not symmetric

If edges have a *direction* – web links, one-way streets, "follows" on a social network, the flow of money – we set $A_{ij} = 1$ when there is an arrow *from i to j*. Now $\mathbf{A}$ need not be symmetric: page

i can link to page j without j linking back. We will need exactly this asymmetry for PageRank at the end of the chapter. For now, just notice that the language "$\mathbf{A} = \mathbf{A}^{\mathsf{T}}$" is precisely the dividing line between undirected and directed.

35.1.3 The incidence matrix: edges meet nodes

The adjacency matrix is node-versus-node. There is a second, equally important matrix that is *edge*-versus-node, and it is the one electrical engineers and Gil Strang reach for first. To build it, orient each edge – pick an arbitrary "from" and "to" for every edge, just to fix signs. (For an undirected graph the orientation is a bookkeeping choice; the final physics will not depend on it.)

Number the five edges of our graph and orient them: $e_1 : 1 \to 2$, $e_2 : 1 \to 4$, $e_3 : 2 \to 3$, $e_4 : 2 \to 4$, $e_5 : 3 \to 4$. The **incidence matrix B** has one row per edge and one column per node. In row e, we put -1 in the "from" column, $+1$ in the "to" column, and 0 everywhere else:

$$\mathbf{B} = \begin{array}{c} \\ e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} -1 & +1 & 0 & 0 \\ -1 & 0 & 0 & +1 \\ 0 & -1 & +1 & 0 \\ 0 & -1 & 0 & +1 \\ 0 & 0 & -1 & +1 \end{pmatrix} \end{array}.$$

Each row has exactly one -1 and one $+1$: an edge touches exactly two nodes. This matrix acts directly as a discrete difference operator.

Intuition

Think of a column vector $\mathbf{x} = (x_1, x_2, x_3, x_4)^{\mathsf{T}}$ as assigning a number to each node – a *potential* (a physical quantity such as elevation, temperature, or electrical voltage at that site). Then $\mathbf{Bx}$ assigns to each *edge* the difference of the potentials at its two ends. Row e_1 of $\mathbf{Bx}$ is $x_2 - x_1$. In this way, $\mathbf{B}$ acts as a discrete "gradient" (a difference operator measuring how steeply a quantity rises or falls along each wire): it turns node-values into edge-differences. This is the graph version of the derivative $\frac{d}{dx}$.

Let us make that concrete. Take $\mathbf{x} = \begin{bmatrix} 5 \\ 8 \\ 6 \\ 2 \end{bmatrix}$ (an elevation at each node), shown in Figure 35.2. Then

$$\mathbf{Bx} = \begin{bmatrix} -1 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & -1 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 5 \\ 8 \\ 6 \\ 2 \end{bmatrix} = \begin{bmatrix} 8-5 \\ 2-5 \\ 6-8 \\ 2-8 \\ 2-6 \end{bmatrix} = \begin{bmatrix} 3 \\ -3 \\ -2 \\ -6 \\ -4 \end{bmatrix}.$$

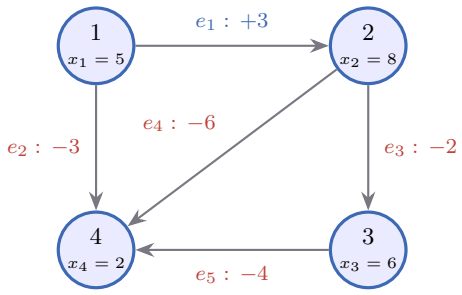


Figure 35.2: The incidence matrix $\mathbf{B}$ as a discrete gradient. Given a vector of node potentials $\mathbf{x} = (5, 8, 6, 2)^\mathsf{T}$, each row of $\mathbf{Bx}$ computes the difference (head) $-$ (tail) along that directed edge. Along edge $e_1 : 1 \to 2$, the potential increases by $x_2 - x_1 = 8 - 5 = +3$. Along edge $e_4 : 2 \to 4$, it drops by $x_4 - x_2 = 2 - 8 = -6$. If $\mathbf{x}$ were voltages, $\mathbf{Bx}$ would be the voltage drops across each component.

Each entry is the rise (or fall) along that edge. If $\mathbf{x}$ were a voltage, these would be the voltage *drops* across each wire – which, divided by resistance, give the currents. The incidence matrix is the direct algebraic encoding of Kirchhoff's laws.

Before/after Strang

If you have read Strang, you will recognize this as his beloved framework $\mathbf{A}^\mathsf{T}\mathbf{C}\mathbf{A}$ from the chapter on graphs and networks (he calls the incidence matrix $\mathbf{A}$). The four subspaces of the incidence matrix tell the whole story: its *nullspace* is the constant vectors (adding the same elevation everywhere changes no difference), and its *left nullspace* is the loops in the graph (Kirchhoff's voltage law – drops around any loop sum to zero). If you are reading this *before* Strang, just hold the picture "$\mathbf{B}$ takes differences across edges"; when you meet his $\mathbf{A}^\mathsf{T}\mathbf{A}$, you will already know it is the Laplacian we are about to build.

What lives in the nullspace of $\mathbf{B}$? We need $\mathbf{Bx} = \mathbf{0}$, i.e. every edge has zero difference, i.e. $x_i = x_j$ for every edge $i - j$. If the graph is connected, that forces *all* the x_i to be equal: the nullspace is exactly the multiples of $\mathbf{1} = (1, 1, 1, 1)^\mathsf{T}$, a one-dimensional space. This is the discrete version of "a function with zero gradient is constant." This observation returns shortly when we analyze connected components.

35.2 The graph Laplacian

We now build the single most important matrix attached to a graph. Combine the degrees and the adjacency into the **graph Laplacian**:

Definition 35.1 (Graph Laplacian)**.** Let $\mathbf{D} = \mathrm{diag}(d_1, \ldots, d_n)$ be the diagonal *degree matrix*, where d_i is the degree of node i, and let $\mathbf{A}$ be the adjacency matrix. The graph Laplacian is

$$\mathbf{L} = \mathbf{D} - \mathbf{A}.$$

Equivalently, $L_{ii} = d_i$, and for $i \neq j$, $L_{ij} = -1$ if i, j are adjacent and 0 otherwise.

For our running graph, $\mathbf{D} = \mathrm{diag}(2, 3, 2, 3)$, so

$$\mathbf{L} = \mathbf{D} - \mathbf{A} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & -1 & 0 & -1 \\ -1 & 3 & -1 & -1 \\ 0 & -1 & 2 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}.$$

Notice the structure: degrees on the diagonal, -1 wherever there is an edge, and – the key property – **every row sums to zero**. Of course it does: the diagonal is the degree, which is exactly how many -1's sit in that row. Zero row sums mean

$$\mathbf{L}\,\mathbf{1} = \mathbf{0}, \qquad \mathbf{1} = (1, 1, 1, 1)^{\mathsf{T}}.$$

So the all-ones vector is an eigenvector of $\mathbf{L}$ with eigenvalue 0. *Every* Laplacian has 0 as an eigenvalue. That single fact is the seed of everything that follows.

35.2.1 Why "Laplacian"? The connection to calculus

The name is borrowed from the Laplacian operator $\nabla^2 = \partial_{xx} + \partial_{yy}$ of multivariable calculus, and the borrowing is exact, not loose.

Connection to other math

On a grid of points spaced h apart, the second derivative is approximated by $f''(x) \approx \frac{f(x-h) - 2f(x) + f(x+h)}{h^2}$. That numerator – "twice the value here, minus the value at each neighbor" – *is* a row of the graph Laplacian of the path graph, with the sign flipped. Acting on a vector of node-values,

$$(\mathbf{L}\mathbf{x})_i = d_i\, x_i - \sum_{j \sim i} x_j = \sum_{j \sim i} (x_i - x_j),$$

which compares the value at node i to the average of its neighbors. The continuous Laplacian measures how much a function at a point exceeds its surrounding average; the graph Laplacian does the discrete same. Heat diffusion, $\partial_t u = -\mathbf{L}u$, smooths a function on a graph exactly as the heat equation smooths temperature on a metal plate. The whole field of *spectral graph theory* is multivariable calculus done on a graph instead of a continuum.

There is a second, cleaner way to see the Laplacian, and it explains why it is always "nice" (symmetric, never negative). Recall the incidence matrix $\mathbf{B}$ from §35.1.3, whose rows are edge-differences. Form $\mathbf{B}^{\mathsf{T}}\mathbf{B}$:

Theorem 35.1 (The Laplacian factorization)**.** *For any orientation of the edges,* $\mathbf{L} = \mathbf{B}^{\mathsf{T}}\mathbf{B}$.

Proof. The (i, j) entry of $\mathbf{B}^{\mathsf{T}}\mathbf{B}$ is the dot product of columns i and j of $\mathbf{B}$ – equivalently, $\sum_e B_{ei} B_{ej}$, summing over edges. Take the diagonal entry (i, i): $\sum_e B_{ei}^2$. Each edge touching node i contributes a ± 1 in that column, so $B_{ei}^2 = 1$ for each of the d_i edges at i and 0 otherwise; the sum is d_i. Now take $i \neq j$: a term $B_{ei} B_{ej}$ is nonzero only for an edge e touching *both* i and j, i.e. the edge between them (if it exists). For that edge one endpoint carries $+1$ and the other -1, so the product is -1. Hence the off-diagonal is -1 for adjacent nodes and 0 otherwise. That is exactly $\mathbf{L} = \mathbf{D} - \mathbf{A}$. □

Let us verify one entry on the running example. Column 2 of $\mathbf{B}$ is $(+1, 0, -1, -1, 0)^\mathsf{T}$ (edges e_1, e_3, e_4 touch node 2). Its own dot product is $1 + 1 + 1 = 3 = d_2$. Its dot product with column 1, which is $(-1, -1, 0, 0, 0)^\mathsf{T}$, is $(1)(-1) + 0 + 0 + 0 + 0 = -1$, matching $L_{12} = -1$. The factorization works.

Why this matters

The identity $\mathbf{L} = \mathbf{B}^\mathsf{T}\mathbf{B}$ has immediate structural consequences. Any matrix of the form $\mathbf{M}^\mathsf{T}\mathbf{M}$ is automatically *symmetric* (so it has a full set of orthogonal real eigenvectors) and *positive semidefinite* (meaning its quadratic energy $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} \geq 0$ is never negative, curving upward like a bowl, which forces every eigenvalue to be ≥ 0). Without computing a single eigenvalue we now know that the Laplacian's spectrum is real, nonnegative, and orthogonally diagonalizable. Factoring $\mathbf{L}$ as $\mathbf{B}^\mathsf{T}\mathbf{B}$ establishes both symmetry and positive semidefiniteness in a single step.

35.2.2 The energy form: what $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x}$ measures

Theorem 35.1 gives a one-line formula for the quadratic form. For any node vector $\mathbf{x}$,

$$\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} = \mathbf{x}^\mathsf{T}\mathbf{B}^\mathsf{T}\mathbf{B}\mathbf{x} = (\mathbf{B}\mathbf{x})^\mathsf{T}(\mathbf{B}\mathbf{x}) = \|\mathbf{B}\mathbf{x}\|^2 = \sum_{(i,j)\in E} (x_i - x_j)^2.$$

Read the last expression aloud: the Laplacian energy of an assignment $\mathbf{x}$ is the sum, over all edges, of the squared difference between the two endpoints. It is small when connected nodes get *similar* values and large when an edge straddles very different values.

Focus

$\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} = \sum_{(i,j)\in E}(x_i - x_j)^2 \geq 0$ is the single formula to remember about the Laplacian. It is "how wiggly is $\mathbf{x}$ along the edges of the graph." Minimizing it (subject to constraints) is the heart of spectral clustering, smoothing, and embedding. Every Laplacian eigenvalue is a value of this energy on a unit vector.

Figure 35.3 makes the formula visible on our running graph. Label nodes $\{1, 2\}$ with $+1$ and nodes $\{3, 4\}$ with -1. An edge inside a group contributes $(x_i - x_j)^2 = 0$; an edge *crossing* between the groups contributes $(+1 - (-1))^2 = 4$. Three edges cross, so the energy is $3 \times 4 = 12$ – four times the number of edges cut.

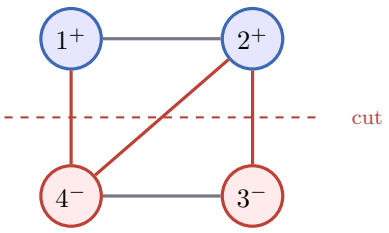


Figure 35.3: The energy $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x}$ as a cut counter. With $\mathbf{x} = (+1, +1, -1, -1)^\mathsf{T}$, the two gray edges lie inside a group and cost 0; the three red edges cross the dashed cut and each cost $(+1 - (-1))^2 = 4$. Total energy $= 12 = 4 \times$ (edges cut). Low energy means few edges straddle the boundary – the link between the Laplacian and clustering.

Now we can see *why* 0 is the smallest eigenvalue and what makes it special. The energy is zero exactly when $x_i = x_j$ across every edge – i.e. $\mathbf{x}$ is constant on each connected piece of the graph. We met this in §35.1.3: it is the nullspace of $\mathbf{B}$, hence of $\mathbf{L}$.

35.3 Eigenvalues of the Laplacian: hearing connectivity

Because $\mathbf{L}$ is symmetric positive semidefinite, we can order its eigenvalues:

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n.$$

We always have $\lambda_1 = 0$ (the constant eigenvector). The next theorem is a central result of the subject: the *number of zero eigenvalues counts the connected components.*

Theorem 35.2 (Multiplicity of zero = number of components)**.** *The multiplicity of the eigenvalue* 0 *of the graph Laplacian equals the number of connected components of the graph.*

Proof. An eigenvector for 0 is a vector $\mathbf{x}$ with $\mathbf{Lx} = \mathbf{0}$. If $\mathbf{Lx} = \mathbf{0}$ then $\mathbf{x}^\mathsf{T}\mathbf{Lx} = \|\mathbf{Bx}\|^2 = 0$; conversely, $\|\mathbf{Bx}\|^2 = 0$ forces $\mathbf{Bx} = \mathbf{0}$ and hence $\mathbf{Lx} = \mathbf{B}^\mathsf{T}(\mathbf{Bx}) = \mathbf{0}$. So $\mathbf{Lx} = \mathbf{0}$ exactly when $\sum_{(i,j)\in E}(x_i - x_j)^2 = 0$. Every squared term must vanish, so $x_i = x_j$ across each edge. Hence $\mathbf{x}$ is constant on each connected component, and free to choose a different constant per component. If there are c components, the space of such vectors has dimension exactly c – one indicator vector per component, and these are linearly independent. So the eigenspace of 0 has dimension c. □

Intuition

Here is the picture. A connected graph holds together with one "ground state" (everybody the same), so 0 appears once. Break the graph into two islands and you get *two* independent ground states – color island A red, island B blue, and there is no edge to penalize the jump. Each island contributes its own zero. The eigenvalue 0 is literally counting the islands.

This already tells us something operational: to test whether a network is connected, you do not trace paths – you compute the second-smallest eigenvalue λ_2. If $\lambda_2 > 0$, the graph is connected. If $\lambda_2 = 0$, it is disconnected.

35.3.1 The Fiedler value: connectivity by degree

The eigenvalue λ_2 is so important it has a name: the **algebraic connectivity**, or **Fiedler value**, after Miroslav Fiedler. It does not just say *whether* the graph is connected; its *size* measures *how robustly* connected the graph is. A large λ_2 means the graph is hard to cut into pieces; a small positive λ_2 means there is a near-bottleneck – a thin bridge that almost disconnects it.

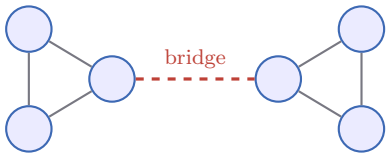


Figure 35.4: Two tight triangles joined by a single "bridge" edge. The graph is connected, so $\lambda_2 > 0$, but only barely: cutting the one dashed edge splits it. The Fiedler value λ_2 is small, and the Fiedler eigenvector takes one sign on the left cluster and the opposite sign on the right – it *finds* the cut.

Why does the eigenvector know where to cut? Because of the energy interpretation. The Fiedler eigenvector $\mathbf{v}_2$ minimizes $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} = \sum(x_i - x_j)^2$ over all unit vectors *orthogonal to the constant vector* $\mathbf{1}$ (orthogonality to the λ_1-eigenvector is forced by the spectral theorem). "Orthogonal to $\mathbf{1}$" means $\sum_i x_i = 0$: the values must be partly positive and partly negative. So $\mathbf{v}_2$ is the gentlest possible nonconstant labeling – it wants neighbors to share a value, but it is forced to use both signs. The cheapest way to do that is to make the jump happen across *few* edges, i.e. across a bottleneck. Splitting nodes by the *sign* of $\mathbf{v}_2$ is the classic **spectral bisection** heuristic.

Compute it yourself

Take the path graph on four nodes in a line, $1{-}2{-}3{-}4$. Its Laplacian is

$$\mathbf{L} = \begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}.$$

Verify $\mathbf{L}\mathbf{1} = \mathbf{0}$. Then test the simple sign pattern $\mathbf{s} = (1, 1, -1, -1)^\mathsf{T}$, which splits the path in half: compute

$$\mathbf{L}\,\mathbf{s} = (0,\ 2,\ -2,\ 0)^\mathsf{T}.$$

This is *not* a scalar multiple of $\mathbf{s}$, so $\mathbf{s}$ is not exactly an eigenvector – but notice it already carries the right idea, putting all the "action" at the two middle nodes where the cut happens. The true Fiedler vector of the 4-path is $\mathbf{v}_2 = \frac{1}{\sqrt{2}}(\cos\frac{\pi}{8}, \cos\frac{3\pi}{8}, -\cos\frac{3\pi}{8}, -\cos\frac{\pi}{8})^\mathsf{T} \approx (0.65,\ 0.27,\ -0.27,\ -0.65)^\mathsf{T}$: its values decrease smoothly from one end to the other, and its sign change happens in the middle, telling you to cut the path in half. Its eigenvalue is $\lambda_2 = 2{-}2\cos\frac{\pi}{4} = 2{-}\sqrt{2} \approx 0.586$. (The exact eigenvalues of the n-node path are $2{-}2\cos(\pi k/n)$ for $k = 0, 1, \ldots, n-1$; these correspond to the discrete cosine transform basis on a graph.)

Modern application

Spectral clustering in machine learning. Suppose you have a pile of data points – images, customers, documents – and a notion of similarity between pairs. Build a graph: nodes are data points, and you connect similar points with edges (often weighted, see below). The communities in your data are the loosely-connected clusters of this graph. Spectral clustering computes the bottom few Laplacian eigenvectors $\mathbf{v}_2, \ldots, \mathbf{v}_k$, uses them as new coordinates for each point (the i-th point becomes the row $(\,v_2(i), \ldots, v_k(i)\,)$), and runs plain k-means in that low-dimensional space. The Laplacian eigenvectors *unfold* the graph so that clusters that were tangled in the original feature space become linearly separable blobs. This is the algorithm

behind a huge amount of image segmentation, community detection in social networks, and unsupervised grouping. Its entire justification is Theorem 35.2 and the energy formula: small eigenvalues correspond to "almost components," i.e. clusters.

35.3.2 Weighted graphs

Real networks rarely have all edges equal. A friendship can be strong or weak; a road can be wide or narrow; a similarity can be near or far. Attach a weight $w_{ij} > 0$ to each edge. Then the weighted adjacency matrix has $A_{ij} = w_{ij}$, the degree becomes the *weighted* degree $d_i = \sum_j w_{ij}$, and the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ keeps every property we proved – because the energy form is now $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2$, still a sum of nonnegative terms. Heavy edges penalize differences more, so the eigenvectors try harder to keep strongly-connected nodes together. Everything in this section runs unchanged with weights; we will not belabor it, but it is what makes the machinery useful on real data.

35.4 Random walks and Markov chains

Up to this point, our view of a network has been physical and static: node potentials, edge voltage drops, Dirichlet energy, and macroscopic heat diffusion $\partial_t \mathbf{u} = -\mathbf{L}\mathbf{u}$.

What is heat diffusion when you zoom in to the particle level? It is a crowd of random walkers. If you place a drop of dye into water, each individual dye molecule simply bounces around at random. At the molecular scale, no single molecule knows calculus or gradients; it just takes jittery, random steps. Yet collectively, the cloud of particles spreads smoothly from high concentration to low concentration, governed by the Laplacian operator $-\mathbf{L}$.

Now we make that microscopic picture the main subject. Instead of tracking a continuous temperature or fluid, imagine a single walker—a web surfer clicking links, a molecule jostling in a gas, a packet routed through servers, or a coin-tossing gambler—hopping from node to node across our graph at random. Where will the walker spend its time after thousands of steps? Does where it started still matter in the long run?

This is the domain of **Markov chains**: sequences of random transitions where the probability of the next step depends solely on the current state, completely oblivious to the past. The mathematical beauty is that this random walk is the exact discrete-time sibling of our Laplacian physics. The transition matrix that advances the walker is $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{L}$. The one-step change in the probability (row) vector is $\pi(\mathbf{P} - \mathbf{I}) = -(\pi\mathbf{D}^{-1})\mathbf{L}$ – the Laplacian acting on probability *per unit degree*, π_i/d_i, which plays the role of a voltage. The stationary equilibrium where the walker settles down is the state where those "voltages" are level, $\pi_i \propto d_i$, so probability currents into each node balance the currents out: Kirchhoff's current law, rewritten for probability.

Intuition

A Markov chain is a memoryless random walk. The walker sits at some node, looks at the outgoing edges, and picks one at random according to fixed probabilities – then forgets how it got there. "Memoryless" (the *Markov property*) means the next step depends only on *where you are*, never on the history of how you arrived. The entire future is encoded in one matrix

of one-step probabilities.

History and connection

The story of matrices on networks unfolds through electrical circuits, a fierce clash between theology and mathematics in Tsarist Russia, and a web search revolution.

In 1847, twenty-three-year-old German physicist Gustav Kirchhoff needed a systematic way to calculate currents and voltages across complex wire circuits. Working with pencil, paper, and physical conservation laws, he realized that the topology of the circuit could be encoded into an incidence matrix and what we now call the graph Laplacian. His matrix methods proved that currents around closed loops sum to zero and total currents entering any node balance to zero.

Sixty years later, in the early 1900s in Imperial Russia, an intense dispute erupted. Pavel Nekrasov, a mathematician and Tsarist educational official, published papers claiming that the classical *Law of Large Numbers*—the mathematical guarantee that the running average of many random trials settles down to a predictable expectation—strictly required every trial to be statistically independent. Nekrasov argued that this mathematical independence reflected divine free will, asserting that independent moral choices were necessary for predictable social order.

Andrey Markov was appalled by what he saw as theological doctrine masquerading as mathematics. In 1906, Markov set out to prove that statistical independence was completely unnecessary: even sequences of events with strong memory can converge to predictable long-term averages. Lacking computing machines, Markov spent months laboriously tallying consonant and vowel transitions across the first 20,000 letters of Alexander Pushkin's verse novel *Eugene Onegin*. He showed that a consonant was followed by a vowel with probability 0.663 and another consonant with probability 0.337, yet the overall fraction of vowels in the text stabilized at 43.2%—establishing the foundations of **Markov chains**.

Around the same time, Oskar Perron (1907) and Georg Frobenius (1912) proved the *Perron–Frobenius theorem*: any matrix whose entries are all strictly positive numbers has a unique, positive leading eigenvalue with a strictly positive eigenvector. For row-stochastic transition matrices (where each row represents outgoing probabilities summing to 100%), this guaranteed a unique stationary equilibrium distribution. Decades later in 1973, Czech mathematician Miroslav Fiedler proved that the second eigenvalue of Kirchhoff's Laplacian measured how tightly knit a graph was, providing the mathematical foundation for spectral clustering algorithms that partition networks.

In 1998, Stanford graduate students Larry Page and Sergey Brin combined both traditions to organize web search. Early search engines ranked pages by keyword counts, leaving them vulnerable to manipulation. Page and Brin modeled the web as a giant directed Markov chain where links represented votes of confidence. But raw web graphs contained "spider traps" (closed loops of pages where a random walker gets trapped forever) and dead ends. Brin and Page solved both by introducing a teleportation probability ($1-\alpha \approx 0.15$), allowing the walker to jump uniformly to any page on the web. This made the Google matrix strictly positive, allowing the *power method* (repeatedly multiplying a probability vector by the matrix) to converge reliably to a unique ranking: PageRank.

Today, major frontiers remain open: defending spectral ranking algorithms against adversarial manipulation (such as coordinated "link farms" engineered to deceive the dominant eigenvector), and efficiently updating stationary distributions on dynamic graphs whose billions of

edges change millions of times per second. An equally active challenge is constructing optimal "expander graphs"—sparse networks with the largest possible spectral gap, achieving rapid mixing times for computer networks and cryptography.

Make this precise with a small *directed, weighted* example – a tiny model of weather, say, or of a user browsing three pages.

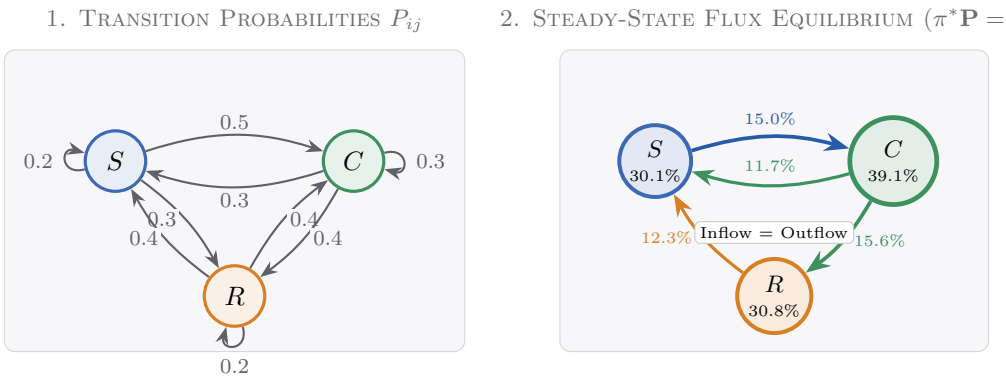


Figure 35.5: A three-state weather Markov chain and its steady-state probability equilibrium. Left: Transition probabilities P_{ij} for Sunny (S), Cloudy (C), Rainy (R), where every node's outgoing arrows sum to 1. Right: The stationary distribution $\pi^* \approx (0.301, 0.391, 0.308)$ acts as an invariant probability fluid. At equilibrium, the probability mass entering any state via incoming flows exactly balances the probability mass exiting via outgoing flows, making π^* the left eigenvector for $\lambda = 1$.

Collect the transition probabilities into a matrix. We will use the **row-stochastic** convention: P_{ij} is the probability of going *from* state i *to* state j, so each *row* sums to 1. From Figure 35.5, ordering states S, C, R:

$$\mathbf{P} = \begin{bmatrix} P_{SS} & P_{SC} & P_{SR} \\ P_{CS} & P_{CC} & P_{CR} \\ P_{RS} & P_{RC} & P_{RR} \end{bmatrix} = \begin{bmatrix} 0.2 & 0.5 & 0.3 \\ 0.3 & 0.3 & 0.4 \\ 0.4 & 0.4 & 0.2 \end{bmatrix}.$$

Check the rows: $0.2+0.5+0.3 = 1$, $0.3+0.3+0.4 = 1$, $0.4+0.4+0.2 = 1$. A matrix with nonnegative entries and rows summing to 1 is called a **stochastic matrix** (or *transition matrix*). Its rows are probability distributions.

Focus

A stochastic matrix has nonnegative entries and *rows summing to* 1. The all-ones vector is a *right* eigenvector with eigenvalue 1: $\mathbf{P1} = \mathbf{1}$, because each row sums to 1. Hold this thought – the interesting eigenvector, the stationary distribution, will be the *left* eigenvector for the same eigenvalue 1.

35.4.1 Distributions evolve by matrix multiplication

Represent the walker's location as a *row* probability vector $\pi = (\pi_S, \pi_C, \pi_R)$ with $\pi_i \geq 0$ and $\sum_i \pi_i = 1$. Say today we are certain it is sunny: $\pi^{(0)} = (1, 0, 0)$. What is the distribution tomorrow? The probability of being cloudy tomorrow is the probability we are in each state today times the probability that state goes to cloudy – a dot product with column C of $\mathbf{P}$. Stacked together, *the update is a vector–matrix product*:

$$\pi^{(1)} = \pi^{(0)}\mathbf{P}.$$

With $\pi^{(0)} = (1, 0, 0)$ this just reads off the first row: $\pi^{(1)} = (0.2, 0.5, 0.3)$. After two days, $\pi^{(2)} = \pi^{(1)}\mathbf{P}$:

$$\pi^{(2)} = (0.2,\ 0.5,\ 0.3)\begin{bmatrix} 0.2 & 0.5 & 0.3 \\ 0.3 & 0.3 & 0.4 \\ 0.4 & 0.4 & 0.2 \end{bmatrix} = (0.31,\ 0.37,\ 0.32).$$

Let us check the first entry: $0.2(0.2) + 0.5(0.3) + 0.3(0.4) = 0.04 + 0.15 + 0.12 = 0.31$. Good – and notice the new vector still sums to 1 ($0.31 + 0.37 + 0.32 = 1$), as it must: a stochastic matrix maps probability distributions to probability distributions. In general $\pi^{(k)} = \pi^{(0)}\mathbf{P}^k$. Evolving the chain k steps is multiplying by $\mathbf{P}$ a total of k times.

Pause

Pause and think: what if every row of P were identical? Suppose that regardless of whether today is sunny, cloudy, or rainy, tomorrow's weather is always drawn from a fixed probability distribution $\mathbf{p} = (0.3,\ 0.4,\ 0.3)$—meaning every row of $\mathbf{P}$ is identical to $\mathbf{p}$. Then for *any* starting distribution $\pi^{(0)}$, the very first step gives:

$$\pi^{(1)} = \pi^{(0)}\mathbf{P} = \left(\sum_i \pi_i^{(0)}\right)\mathbf{p} = \mathbf{p}.$$

The chain completely forgets its starting point in a single step. In that extreme case, $\mathbf{p}$ is the stationary distribution, and every eigenvalue other than $\lambda_1 = 1$ is 0. In general chains, rows are not identical, so it takes several steps to "mix," but the principle is the same: matrix powers repeatedly damp out the non-stationary components until only the dominant eigenvector survives.

Common trap

Mind the convention. With the *row-stochastic* matrix above, distributions are *row* vectors and you multiply on the *left*: $\pi \mapsto \pi\mathbf{P}$. Many texts (and most physics writing) instead use *column*-stochastic matrices, where columns sum to 1, distributions are column vectors, and you multiply on the right: $\pi \mapsto \mathbf{P}\pi$. The two are transposes of each other. Pick one and stay with it for a whole calculation; mixing them silently transposes your transition probabilities and produces nonsense that still sums to 1, so it is a maddening bug to catch. In this chapter: rows sum to 1, distributions on the left.

35.4.2 The stationary distribution is a left eigenvector

Run the chain a few more steps and watch:

$$\begin{aligned}\pi^{(2)} &= (0.310,\ 0.370,\ 0.320),\\ \pi^{(3)} &= (0.301,\ 0.394,\ 0.305),\\ \pi^{(4)} &= (0.300,\ 0.391,\ 0.309),\\ &\vdots\end{aligned}$$

The numbers are settling down. Figure 35.6 plots each coordinate against the step number: starting from the spike "certainly sunny," all three probabilities swing once and then lock onto fixed levels.

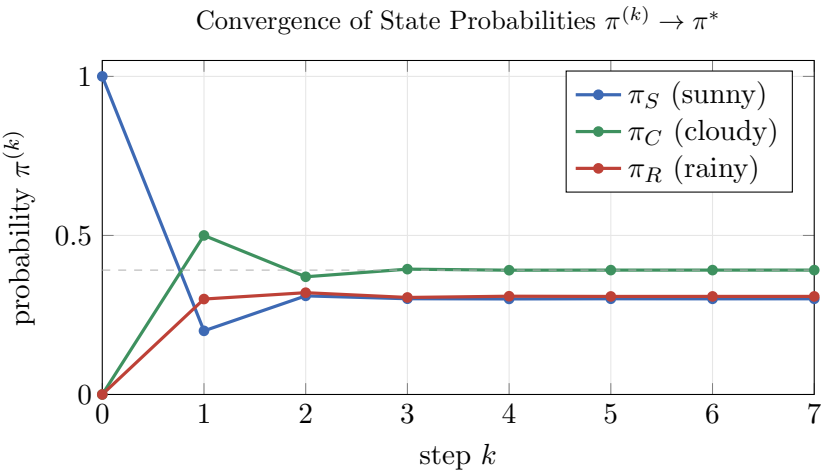


Figure 35.6: The weather chain forgets its start. Each coordinate of $\pi^{(k)}$ converges to the stationary value $\pi^* \approx (0.301, 0.391, 0.308)$ within a handful of steps, no matter that we began "certainly sunny." The geometric rate of approach is set by the second eigenvalue $|\lambda_2|$ of $\mathbf{P}$.

They are converging to a vector π^* that does not change when you push it through $\mathbf{P}$ one more time:

$$\pi^*\mathbf{P} = \pi^*.$$

This is the **stationary distribution**. Algebraically it says π^* is a *left* eigenvector of $\mathbf{P}$ with eigenvalue 1 – equivalently a right eigenvector of $\mathbf{P}^{\mathsf{T}}$. This gives the central result of this section:

Focus

The long-run behavior of a Markov chain is an eigenvector problem. The fraction of time the walker spends in each state, after wandering forever, is the left eigenvector of the transition matrix $\mathbf{P}$ for eigenvalue 1, normalized to sum to 1. Find that eigenvector and you have predicted the future of the whole system.

Let us actually solve $\pi^*\mathbf{P} = \pi^*$ for our weather chain. Writing $\pi^* = (a, b, c)$, the equation $\pi^*(\mathbf{P} - \mathbf{I}) = \mathbf{0}$ together with $a + b + c = 1$ gives the system (one equation per column of $\mathbf{P}$, dropping one as

redundant):

$$0.2a + 0.3b + 0.4c = a,$$
$$0.5a + 0.3b + 0.4c = b,$$
$$a + b + c = 1.$$

From the first equation, $0.3b + 0.4c = 0.8a$. From the second, $0.5a + 0.4c = 0.7b$. Solving this 3×3 system (a good by-hand exercise) yields the exact answer

$$\pi^* = \left(\tfrac{40}{133},\ \tfrac{52}{133},\ \tfrac{41}{133}\right) \approx (0.3008,\ 0.3910,\ 0.3083),$$

exactly the vector the iteration was crawling toward. In the long run the weather is sunny about 30% of days, cloudy about 39%, rainy about 31% – regardless of today's weather. The influence of the initial distribution decays completely, a hallmark of an ergodic Markov chain.

35.4.3 When is there a unique stationary distribution it converges to?

Not every chain settles down to a single answer. Two conditions can prevent convergence, and the Perron–Frobenius theorem guarantees convergence when both are excluded.

Formal

Perron–Frobenius for stochastic matrices. If $\mathbf{P}$ is the transition matrix of a finite Markov chain that is *irreducible* (you can get from any state to any other, following arrows) and *aperiodic* (it is not locked into a fixed cycle of return times), then:

1. $\mathbf{P}$ has a unique stationary distribution π^*, a positive left eigenvector for the eigenvalue 1;
2. 1 is the largest eigenvalue in magnitude, and it is simple (multiplicity one);
3. for *every* starting distribution, $\pi^{(0)}\mathbf{P}^k \to \pi^*$ as $k \to \infty$.

The rate of convergence is governed by the *second-largest* eigenvalue λ_2 in magnitude: the distance to π^* shrinks like $|\lambda_2|^k$. This gap $1 - |\lambda_2|$ is the *spectral gap*, the mixing speed of the chain.

Intuition

Reducible means the graph splits into pieces a walker can get stuck inside – then the long-run answer depends on where you started, so there is no single stationary distribution. *Periodic* means the chain marches in lockstep (think bipartite: odd steps here, even steps there); the distribution oscillates forever instead of settling. Rule out both – by making sure every state can reach every other and that there is at least one self-loop or odd cycle to break the rhythm – and the chain is guaranteed to converge to one universal distribution. The weather chain has all positive transitions, so it is strictly irreducible and aperiodic, explaining why it converged so cleanly.

Connection to other math

The eigenvalue λ_2 controlling Markov mixing and the eigenvalue λ_2 controlling graph connectivity (the Fiedler value) reflect the same spectral phenomenon. For a random walk on an undirected graph, the transition matrix is $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{L}$. The matrix $\mathbf{D}^{-1}\mathbf{L}$

is similar to the symmetric *normalized Laplacian* $\mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$ (conjugate by $\mathbf{D}^{1/2}$), so its eigenvalues λ_k are real and nonnegative, and $\mathbf{P}$'s eigenvalues are $\mu_k = 1 - \lambda_k$. A large Laplacian gap means a large Markov gap means fast mixing means no bottleneck. A graph that is hard to cut is a graph a random walker mixes through quickly. Connectivity, clustering, and mixing time provide three complementary perspectives on the spectrum of the Laplacian.

35.5 PageRank: importance as an eigenvector

We close with the application that made all of this famous: Google's PageRank. The problem is to rank the pages of the web by "importance" using only the link structure – no human judgment, no reading the text, just who links to whom.

Intuition

The defining idea is self-referential: *a page is important if important pages link to it.* A link is a vote, but votes are weighted by the importance of the voter, and the voter's importance is decided the same way. What looks circular in words is simply an eigenvector equation. "$\mathbf{x}$ is proportional to the incoming weighted votes, which are proportional to $\mathbf{x}$" is precisely $\mathbf{M}\mathbf{x} = \mathbf{x}$.

Model the web as a directed graph: nodes are pages, an edge $i \to j$ means page i links to page j. Imagine a **random surfer** who, at each step, picks uniformly at random one of the outgoing links of the current page and clicks it. If page i has d_i outgoing links, the transition probability from i to each linked page j is $1/d_i$. This is just the Markov chain $\mathbf{P}$ from the previous section, built from the link graph. PageRank is *the stationary distribution of the random surfer*: the fraction of time an endlessly-clicking surfer spends on each page. Important pages are the ones the surfer keeps landing on.

Take a tiny four-page web. Page 1 links to 2 and 3; page 2 links to 3; page 3 links to 1; page 4 links to 3.

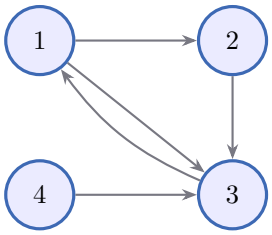


Figure 35.7: A four-page web. Page 3 is linked from 1, 2, and 4 – it should rank highest. Page 1 gets its importance from page 3, which is itself highly ranked, so 1 does well too despite only one inbound link. PageRank captures exactly this "importance flows backward along links" intuition.

Page 1 has two out-links, so it sends probability $1/2$ to each of 2 and 3. Pages 2, 3, 4 each have a single out-link and send all their probability along it. The row-stochastic transition matrix is

$$\mathbf{P} = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

We want the left eigenvector for eigenvalue 1. On a real web graph, however, two obstacles prevent us from using $\mathbf{P}$ directly.

35.5.1 Two snags and the "teleport" that fixes both

Dangling nodes. A page with *no* outgoing links (a PDF, a dead end) gives a zero row – the surfer falls off the edge of the web and probability leaks away; $\mathbf{P}$ is no longer stochastic. **Reducibility and periodicity.** Even with no dead ends, the real web has spider-traps and disconnected islands, so Perron–Frobenius does not apply and there may be no unique stationary distribution.

Brin and Page fixed both at once with a single idea: **teleportation**. With probability α (the *damping factor*, classically $\alpha = 0.85$) the surfer follows a random link as before; with probability $1 - \alpha$ the surfer gets bored and jumps to a page chosen *uniformly at random* from all n pages. The new transition matrix is

$$\mathbf{G} = \alpha \mathbf{P} + (1 - \alpha)\, \tfrac{1}{n}\, \mathbf{J},$$

where $\mathbf{J}$ is the all-ones $n \times n$ matrix (and dangling rows of $\mathbf{P}$ are first replaced by uniform rows $\frac{1}{n}\mathbf{1}^{\mathsf{T}}$, so the surfer "teleports out" of dead ends). This $\mathbf{G}$ is the **Google matrix**.

Why this matters

Teleportation resolves both issues through a single convex combination. Because $\mathbf{J}/n$ has *every* entry positive, $\mathbf{G}$ has every entry strictly positive: from any page you can reach any other in one step (you might teleport there). Strict positivity makes $\mathbf{G}$ irreducible *and* aperiodic in a single stroke, so Perron–Frobenius applies with no exceptions: a unique, positive, stationary PageRank vector *always* exists and the iteration *always* converges.

The PageRank vector $\mathbf{r}$ is the stationary distribution of $\mathbf{G}$:

$$\mathbf{r}\,\mathbf{G} = \mathbf{r}, \qquad \sum_i r_i = 1.$$

Equivalently, written as the update each entry must satisfy (this is the form you will see in code),

$$r_j = \frac{1 - \alpha}{n} + \alpha \sum_{i \to j} \frac{r_i}{d_i},$$

"my rank is a baseline teleport share plus a weighted sum of the ranks flowing in from pages that link to me, each split evenly among its out-links." Read it twice; it is the whole algorithm.

Pause

Pause and think: what do the extremes $\alpha = 1$ and $\alpha = 0$ mean? If you set $\alpha = 1$, the Google matrix becomes the raw web matrix $\mathbf{P}$: you trust hyperlinks completely, but dead ends leak probability and spider traps capture all the rank. If you set $\alpha = 0$, $\mathbf{G}$ collapses to $\frac{1}{n}\mathbf{J}$: links are ignored entirely, and every page is awarded an identical score of $1/n$. The classic choice $\alpha = 0.85$ means the surfer follows true hyperlinks 85% of the time, while the 15% teleportation fraction guarantees that Perron–Frobenius holds, that spider traps drain, and that no page is left with rank zero.

35.5.2 The power method: how it is actually computed

The web has billions of pages; you cannot diagonalize a billion-by-billion matrix. But you do not need to. You only need the *dominant* eigenvector, and there is a one-line method for that, which you have already been using without naming it: just iterate.

Intuition

The power method. Start from any distribution $\pi^{(0)}$ (say uniform) and repeatedly multiply: $\pi^{(k+1)} = \pi^{(k)}\mathbf{G}$. Write the start in the eigenvector basis; each multiplication multiplies the component along eigenvector $\mathbf{v}_k$ by λ_k. Since $\lambda_1 = 1$ dominates and every other $|\lambda_k| < 1$ shrinks toward zero, all the other components die off and what survives is the λ_1-eigenvector – the PageRank vector.

This is exactly the "run the chain forward and watch it settle" computation from §35.4, now recognized as the power method. Each step is one sparse matrix–vector product – cheap, because each page has only a handful of links. Convergence is geometric at rate $\alpha = 0.85$ (the spectral gap of $\mathbf{G}$ is controlled by the damping factor; in fact $|\lambda_2| \le \alpha$ for the Google matrix), so a few dozen iterations suffice to rank the entire web. That is why the choice $\alpha = 0.85$ is a tuning knob: lower α means faster convergence but more weight on random teleporting and less on the actual link structure.

Compute it yourself

Run three steps of the power method by hand on the four-page web of Figure 35.7, using the *undamped* $\mathbf{P}$ for simplicity and starting from uniform $\pi^{(0)} = (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$. Compute $\pi^{(1)} = \pi^{(0)}\mathbf{P}$, then $\pi^{(2)}$, then $\pi^{(3)}$. Watch page 3's entry climb and page 4's entry collapse toward 0 (nothing links to page 4, so the surfer never returns to it). You are watching importance flow through the graph in real time. Then reflect: with damping, page 4 would keep a small floor of $\frac{1-\alpha}{n}$ – teleportation guarantees no page is ranked exactly zero.

Modern application

PageRank is everywhere now, not just search. The same "stationary distribution of a well-chosen random walk" template reappears across modern computing. *Personalized PageRank* (teleporting back to one user's pages, not uniformly) powers friend and product recommendations, *TextRank* runs PageRank on a graph of sentences for automatic summarization, and *citation analysis* ranks scientific papers and journals. In the same spirit, biologists

rank genes and proteins in interaction networks ("GeneRank"), while random-walk node embeddings (DeepWalk, node2vec) – the graph analog of word embeddings – feed graph neural networks. Every one of them is the chapter you just read: build a transition matrix from a network, find the eigenvector for eigenvalue 1.

Walking question

Take this on a walk. You have seen the eigenvalue 1 appear twice – as the stationary distribution of a Markov chain, and in PageRank – and the eigenvalue 0 appear twice as well – counting connected components and as the constant ground state of the Laplacian. Why does the leading structural eigenvalue (0 for $\mathbf{L}$, 1 for $\mathbf{P}$) carry the most important information? What is the relationship between $\mathbf{L} = \mathbf{D} - \mathbf{A}$ and $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$, and how does the eigenvalue 0 of one become the eigenvalue 1 of the other? (Hint: $\mathbf{P} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{L}$.) Hold that identity in your head and the two halves of this chapter fuse into one.

35.6 Looking back

We started with simple dots and lines on a page and arrived at the algorithm that organizes the billions of documents on the World Wide Web. Every step came from translating a network into a matrix and reading off its eigenvalues and eigenvectors.

The central lesson is that a network's combinatorial shape, its physical behavior, and its probabilistic dynamics are not separate subjects. They are three facets of one unified algebraic structure:

- The **adjacency matrix** $\mathbf{A}$ records connections; its powers $\mathbf{A}^k$ count paths and walks, turning graph traversals into matrix multiplication.
- The **incidence matrix** $\mathbf{B}$ acts as a discrete spatial derivative, mapping node potentials to edge differences and factoring the Laplacian as $\mathbf{L} = \mathbf{B}^\mathsf{T}\mathbf{B}$.
- The **graph Laplacian** $\mathbf{L} = \mathbf{D} - \mathbf{A}$ is symmetric and positive semidefinite; its quadratic energy $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} = \sum_{(i,j)\in E}(x_i - x_j)^2$ measures how much a labeling fluctuates across edges. The multiplicity of its zero eigenvalue counts disconnected components, and the Fiedler value λ_2 identifies sparse cuts, enabling spectral clustering without testing exponentially many subsets.
- A **Markov chain** advances probability distributions via $\pi \mapsto \pi\mathbf{P}$. Its long-term equilibrium is the left eigenvector of $\mathbf{P}$ for eigenvalue 1, guaranteed unique and reachable by Perron–Frobenius when the chain is irreducible and aperiodic.
- **PageRank** is the stationary distribution of a random surfer, regularized by teleportation so that Perron–Frobenius always holds, and extracted efficiently by the power method.

The identity $\mathbf{P} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{L}$ connects the two views: a random walk is the discrete-time heat equation. The stationary distribution where probability ceases to shift is the state where probability currents into each node balance the currents out—Kirchhoff's current law reborn as conservation of probability.

In practical computation, this spectral framework transforms problems that seem combinatorially hopeless into routine linear algebra. Partitioning a graph into two clusters of minimal cut size is an NP-hard problem; you cannot check all 2^n subsets of nodes. The Fiedler vector provides an

immediate, continuous relaxation that separates clusters along their natural boundary. Similarly, the Google matrix $\mathbf{G} = \alpha\mathbf{P} + \frac{1-\alpha}{n}\mathbf{J}$ (with $\mathbf{J} = \mathbf{1}\mathbf{1}^\mathsf{T}$ the all-ones matrix) would appear to be a memory disaster: adding teleportation makes $\mathbf{G}$ completely dense, with n^2 nonzero entries. On a web of billions of pages, storing $\mathbf{G}$ explicitly would require exabytes of RAM. But because the dense part is a rank-one matrix $\mathbf{1}\mathbf{1}^\mathsf{T}$, we never form $\mathbf{G}$ directly. Since $\pi\mathbf{1}\mathbf{1}^\mathsf{T} = (\sum_i \pi_i)\mathbf{1}^\mathsf{T} = \mathbf{1}^\mathsf{T}$ for a probability vector, each step multiplies by the sparse link matrix ($\alpha\pi\mathbf{P}$) and adds the same constant $\frac{1-\alpha}{n}$ to every entry, keeping every power-method iteration fast and linear in the number of edges.

Yet one must remain honest about the boundaries of the theory. While the Fiedler vector offers an effective cut, Cheeger's inequality guarantees only an approximation, not the exact minimum. At the same time, the speed at which a Markov chain converges is dictated by the spectral gap $1 - |\lambda_2|$. On networks with severe bottlenecks—like two massive clusters connected by a narrow bridge—$|\lambda_2|$ hugs 1, and a random walker can take millions of steps before crossing over, slowing mixing to a crawl. Finally, standard Markov chains assume time-invariant transition probabilities, whereas real social networks and web traffic are bursty, dynamic, and memory-dependent.

Looking ahead: from graph energy to optimization

When we derived the Fiedler vector, we minimized the graph Dirichlet energy $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x} = \sum_{(i,j)\in E}(x_i - x_j)^2$, our first encounter with a quadratic form. In Chapter 36, we study general quadratic forms $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$, using eigenvalues to characterize multivariable curvature and solve constrained optimization problems.

Exercises

Exercise 35.1 (Build the three matrices)**.** Consider the path graph on five nodes in a line: 1–2–3–4–5.

1. Write down its adjacency matrix $\mathbf{A}$ and its degree matrix $\mathbf{D}$.
2. Write down the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ and verify that every row sums to zero.
3. Orient each edge left-to-right and write the 4×5 incidence matrix $\mathbf{B}$. Confirm by direct multiplication that $\mathbf{B}^\mathsf{T}\mathbf{B} = \mathbf{L}$.

Exercise 35.2 (Walks by hand)**.** For the running graph of Figure 35.1, we computed $\mathbf{A}^2$. Compute $\mathbf{A}^3$ and verify that $(\mathbf{A}^3)_{11}$ equals the number of length-3 closed walks from node 1 back to node 1. List those walks explicitly on the picture. (Hint: a length-3 closed walk must use a triangle.)

Exercise 35.3 (Counting components by eigenvalue)**.** A graph on six nodes has two triangles, $\{1, 2, 3\}$ and $\{4, 5, 6\}$, with no edges between them. Without computing all the eigenvalues, state the multiplicity of the eigenvalue 0 of its Laplacian and write down a basis for the corresponding eigenspace. Then add a single edge $3-4$ joining the triangles; what is the new multiplicity of 0, and why?

Exercise 35.4 (The energy form)**.** Let $\mathbf{L}$ be the Laplacian of the running graph (Figure 35.1) and let $\mathbf{x} = (1, 1, -1, -1)^\mathsf{T}$ (label nodes $1, 2$ "+" and nodes $3, 4$ "−").

1. Compute $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x}$ two ways: directly as a matrix product, and as $\sum_{(i,j)\in E}(x_i - x_j)^2$. They must agree.

2. Which edges contribute to the sum? Interpret the value as "the number of edges cut" by the partition $\{1, 2\}$ versus $\{3, 4\}$.

Exercise 35.5 (Stationary distribution)**.** A frog hops between two lily pads, A and B. From A it stays with probability 0.7 and jumps to B with probability 0.3. From B it stays with probability 0.4 and jumps to A with probability 0.6.

1. Write the 2×2 row-stochastic transition matrix $\mathbf{P}$.
2. Find the stationary distribution by solving $\pi\mathbf{P} = \pi$ with $\pi_A + \pi_B = 1$.
3. Start at $\pi^{(0)} = (1, 0)$ and compute $\pi^{(1)}, \pi^{(2)}, \pi^{(3)}$ to watch the convergence. What are the two eigenvalues of $\mathbf{P}$, and how does the second one predict the speed you observe?

Exercise 35.6 (Why rows summing to one matters)**.** Prove that if $\mathbf{P}$ is row-stochastic (nonnegative entries, each row summing to 1), then $\mathbf{P1} = \mathbf{1}$, so 1 is an eigenvalue with right eigenvector $\mathbf{1}$. Then explain why this does *not* immediately give you the stationary distribution – which eigenvector (left or right) do you actually need, and why are they different for a non-symmetric $\mathbf{P}$?

Exercise 35.7 (A reducible chain)**.** Consider the chain on states $\{1, 2, 3\}$ with

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}.$$

1. Draw the transition graph. Which states are "absorbing" (once entered, never left)?
2. Show that both $(1, 0, 0)$ and $(0, 0, 1)$ are stationary distributions. Why does this not contradict Perron–Frobenius?
3. Starting from $(0, 1, 0)$, where does the chain end up, and how does the answer depend on the coin flips? (This is why reducibility breaks uniqueness.)

Exercise 35.8 (PageRank by hand)**.** Use the four-page web of Figure 35.7 with damping factor $\alpha = 0.85$.

1. Take the (dangling-free) transition matrix $\mathbf{P}$ given in the text and form the Google matrix $\mathbf{G} = \alpha\mathbf{P} + \frac{1-\alpha}{4}\mathbf{J}$ explicitly (a 4×4 matrix).
2. Starting from uniform $\pi^{(0)} = (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$, run two power-method steps $\pi^{(0)}\mathbf{G}$ and $(\pi^{(0)}\mathbf{G})\mathbf{G}$.
3. Rank the four pages by your $\pi^{(2)}$. Does page 3 come out on top? Does page 4 retain a small nonzero rank, and why is teleportation responsible?

Exercise 35.9 (The bridge between the two halves)**.** For an undirected, connected graph let $\mathbf{L} = \mathbf{D} - \mathbf{A}$ be the Laplacian and $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ be the random-walk transition matrix.

1. Verify the identity $\mathbf{P} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{L}$.
2. Deduce that if $\mathbf{x}$ is a (right) eigenvector of $\mathbf{D}^{-1}\mathbf{L}$ with eigenvalue μ, it is an eigenvector of $\mathbf{P}$ with eigenvalue $1 - \mu$. In particular the Laplacian's eigenvalue 0 (constant eigenvector $\mathbf{1}$) maps to $\mathbf{P1} = \mathbf{1}$.
3. The stationary distribution is a *left* eigenvector, $\pi\mathbf{P} = \pi$. Show this holds exactly when $(\pi\mathbf{D}^{-1})\mathbf{L} = \mathbf{0}$, and use Theorem 35.2 to conclude that on a connected graph $\pi_i = d_i / \sum_k d_k$. Explain in words how this links "connected $\Rightarrow$ unique constant ground state" with "connected $\Rightarrow$ unique stationary distribution."

Exercise 35.10 (Fiedler in miniature, a proof)**.** Let $\mathbf{L}$ be the Laplacian of a graph with eigenvalues $0 = \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$. Using the energy form $\mathbf{x}^{\mathsf{T}}\mathbf{L}\mathbf{x} = \sum_{(i,j)\in E}(x_i - x_j)^2$ and the fact (from the spectral theorem for symmetric matrices) that

$$\lambda_2 = \min_{\substack{\mathbf{x}\perp\mathbf{1}\\ \mathbf{x}\neq\mathbf{0}}} \frac{\mathbf{x}^{\mathsf{T}}\mathbf{L}\mathbf{x}}{\mathbf{x}^{\mathsf{T}}\mathbf{x}},$$

prove that $\lambda_2 = 0$ *if and only if* the graph is disconnected. (One direction: if disconnected, exhibit a nonconstant $\mathbf{x} \perp \mathbf{1}$ with zero energy. Other direction: if $\lambda_2 = 0$, build a second independent constant-on-components vector and conclude there are ≥ 2 components.) This is Theorem 35.2 in the smallest nontrivial case.

Chapter 36

Optimization and Quadratic Forms

In Chapter 35, we saw electrical networks find currents that minimize dissipated power, and random walks settle into equilibrium. Both illustrate a universal theme: many mathematical problems reduce to a search for an optimal configuration. Examples include finding the best-fit line through a data set, the most efficient distribution of current through a circuit, the configuration of neural network weights that minimizes loss, or the shape a soap film adopts to minimize surface area. In each case, an objective function assigns a real number to every choice – a cost, an error, or an energy – and the goal is to minimize or maximize this value. That is the core problem of *optimization*.

This chapter rests on one observation. Most of the smooth cost surfaces that actually matter, at least near their lowest points, look like the same simple shape: a *bowl*. In Chapter 27 we met the algebraic engine of that bowl: the symmetric matrix $\mathbf{A}$ acting in the quadratic form $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$. There we saw that positive eigenvalues make $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ strictly positive. Here we turn that insight into an algorithm. The geometry of a quadratic form – bowl, dome, saddle, or trough – is governed entirely by the eigenvalues and eigenvectors of $\mathbf{A}$. So the linear algebra you already know turns out to be the exact language of multivariable curvature, equilibrium, and constrained optimization.

We will build the picture first, learning to read off from a matrix whether a surface curves up into a bowl (a minimum), down into a dome (a maximum), or twists into a saddle (neither). Along the way we will see that "positive definite", "convex", and "has a unique global minimum" are three phrasings of one geometric idea, and that least squares simply sets the gradient of an error bowl to zero. We will also see how constrained optimization – whether subject to smooth equality constraints or boundary inequalities – connects directly to eigenvalues via Lagrange multipliers and the Karush–Kuhn–Tucker conditions.

36.1 Curved surfaces and their bottoms

Fix the mental picture once and for all. Imagine a function f that takes a point $\mathbf{x}$ in the plane and returns a single height $f(\mathbf{x})$. Plot that height as a surface sitting above the plane. Optimization is the question: *over all points* $\mathbf{x}$*, where is the surface lowest?*

Intuition

Picture a marble rolling on the surface. It rolls downhill and settles at a low point. The places where it can come to rest are the places where the ground is *flat* – no slope in any direction to push the marble along. Those flat spots are the *critical points*. Most of optimization is two separate questions: (1) where is the ground flat? and (2) of those flat spots, which are genuine bottoms (minima), which are tops (maxima), and which are passes that go up one way and down another (saddles)?

The first question is about the *gradient* – the slope. The second is about *curvature* – how the surface bends – and curvature is where the quadratic forms live. Here are the three archetypal shapes, because every smooth surface, zoomed in close enough to a flat spot, looks like one of them.

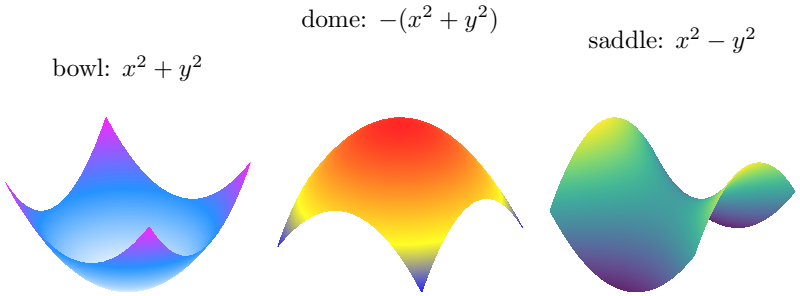


Figure 36.1: The three basic shapes of a quadratic surface. The *bowl* curves up in every direction (a minimum at the center). The *dome* curves down in every direction (a maximum). The *saddle* curves up along one axis and down along another (neither). Reading which one you have from the eigenvalues of its matrix is the first key step in multivariable optimization.

The three formulas under those pictures – x^2+y^2, $-(x^2+y^2)$, x^2-y^2 – are our first quadratic forms. Notice they are built only from *squares and products* of the coordinates: x^2, y^2, xy. No lonely x or y term, no constant. That is the defining feature, and it is exactly the kind of expression a symmetric matrix produces.

36.2 Quadratic forms: $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$

In Chapter 27, we introduced the quadratic form as an energy test for positive definite matrices. Let us briefly revisit how the matrix builds the polynomial, keeping our eyes focused now on the geometry of slopes, level curves, and algorithms. Take a symmetric 2×2 matrix and sandwich it between a column vector on the right and its transpose (a row vector) on the left:

$$\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

Multiply it out carefully. First the matrix times the vector:

$$\begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ bx + cy \end{bmatrix}.$$

Then the row vector times that:

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} ax + by \\ bx + cy \end{bmatrix} = x(ax + by) + y(bx + cy) = ax^2 + 2b\,xy + cy^2.$$

There it is. The sandwich $\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x}$ collapses to a tidy polynomial in which every term has total degree exactly two. The diagonal entries a, c become the pure-square coefficients; the off-diagonal entry b becomes the coefficient of the cross term xy, doubled because it appears in both off-diagonal slots.

Definition 36.1 (Quadratic form)**.** Let $\mathbf{A}$ be an $n \times n$ symmetric matrix. The function

$$Q(\mathbf{x}) \;=\; \mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} \;=\; \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}\, x_i x_j$$

is the *quadratic form* associated with $\mathbf{A}$. It maps a vector $\mathbf{x} \in \mathbb{R}^n$ to a single real number, and every term is degree two in the coordinates.

Focus

A quadratic form is the matrix world's version of "$y = ax^2$." One number a tells you whether the parabola opens up or down and how sharply. A whole symmetric matrix $\mathbf{A}$ tells you the same thing in many dimensions at once: which directions open up, which open down, and how sharply in each.

36.2.1 Why symmetric, and why the factor of two

You might worry: what if the matrix is not symmetric? Suppose $\mathbf{M}$ has off-diagonal entries p (top right) and q (bottom left) that differ. Working out $\mathbf{x}^{\mathsf{T}}\mathbf{M}\mathbf{x}$ gives $ax^2 + (p+q)xy + cy^2$. Only the *sum* $p+q$ survives – the form cannot tell $p = 1, q = 3$ from $p = q = 2$. So for any matrix we could replace it by its "symmetric part" $\frac{1}{2}(\mathbf{M}+\mathbf{M}^{\mathsf{T}})$ without changing the form at all. Since the quadratic form only ever sees the symmetric part, we lose nothing by *always* taking $\mathbf{A}$ symmetric, and we gain a great deal: the spectral theory of symmetric matrices (real eigenvalues, orthogonal eigenvectors) becomes available to read the geometry.

Common trap

The factor of 2 trips everyone up once. If you want the cross-term $6xy$ in your form, the matrix entry b is *half* of that, $b = 3$, because b appears twice (top-right and bottom-left) and the two copies add. So

$$3x^2 + 6xy + 5y^2 \;\longleftrightarrow\; \mathbf{A} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad \text{not } \begin{bmatrix} 3 & 6 \\ 6 & 5 \end{bmatrix}.$$

Always halve the cross-term coefficient when you build the matrix; always double the off-diagonal when you read the form back out.

Compute it yourself

Build the symmetric matrix for each form, then read it back to check. (a) $2x^2+4xy+7y^2$. (b) $x^2-6xy+y^2$. (c) $5x^2+5y^2$ (no cross term). (d) In three variables, $x^2+y^2+z^2+2xy+2yz$ – careful, there is no xz term, so the $(1,3)$ entry is 0. Answers: (a) $\begin{bmatrix} 2 & 2 \\ 2 & 7 \end{bmatrix}$; (b) $\begin{bmatrix} 1 & -3 \\ -3 & 1 \end{bmatrix}$; (c)

$\begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$; (d) $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$.

36.2.2 A first worked form and its level curves

Take $\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$, a diagonal matrix, so its form is $Q(x, y) = 3x^2 + y^2$ with no cross term. Where is Q equal to a constant, say $Q = 3$? That is the equation $3x^2 + y^2 = 3$, i.e. $x^2 + y^2/3 = 1$ – an ellipse, short in the x-direction (radius 1) and tall in the y-direction (radius $\sqrt{3}$). Set $Q = 12$ and you get a bigger ellipse of the same shape. The level curves are a nested family of ellipses, all centered at the origin, and the surface $z = Q(x, y)$ is a bowl whose cross-sections are exactly these ellipses.

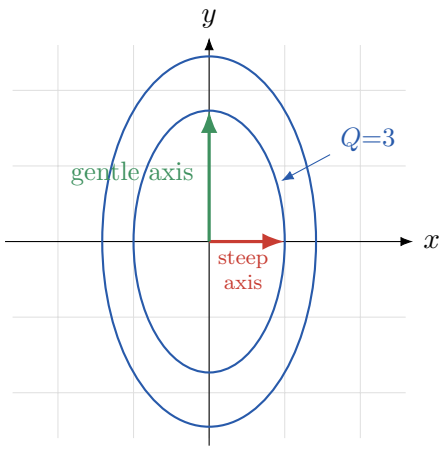


Figure 36.2: Level curves of $Q(x, y) = 3x^2 + y^2$. Each blue curve is a set $Q = \text{const}$, an ellipse. The form grows *fastest* along the short x-axis (eigenvalue 3) and *slowest* along the long y-axis (eigenvalue 1): big eigenvalue means steep wall, so the ellipse is squeezed in that direction.

Notice the eigenvalues of this diagonal matrix are 3 and 1, sitting right on the diagonal, with eigenvectors along the axes. The big eigenvalue 3 goes with the *steep* direction (the wall of the bowl rises fast, so the ellipse is short there); the small eigenvalue 1 goes with the *gentle* direction. This is the whole secret of quadratic-form geometry, and it survives even when the matrix is not diagonal – we just have to rotate to the eigenvector axes first.

Pause

Mental check: reading the shape. If $Q(x, y) = 5x^2 + 2y^2$, what are the semi-axes of the ellipse $Q(x, y) = 10$? In the x-direction, $5x^2 = 10 \Rightarrow x = \pm\sqrt{2}$. In the y-direction, $2y^2 = 10 \Rightarrow y = \pm\sqrt{5}$. The ellipse is wider in y ($\sqrt{5} \approx 2.24$) and narrower in x ($\sqrt{2} \approx 1.41$). Why is it narrower along x? Because the coefficient 5 is larger: the bowl rises more steeply in x, so the contour height 10 is reached much closer to the origin.

36.3 Diagonalizing the form: the principal axes

What if there is a cross term? Take

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \qquad Q(x, y) = 2x^2 + 2xy + 2y^2.$$

The level curves $Q = \text{const}$ are still ellipses, but now they are *tilted* – their axes do not line up with the coordinate axes, precisely because of the xy term. The cross term is the algebraic fingerprint of a tilt. To visualize the bowl's principal geometry we want to rotate our coordinates until the tilt vanishes. And we already know how: diagonalize the symmetric matrix.

Recall from the spectral theorem that a symmetric matrix factors as $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{\mathsf{T}}$, where $\mathbf{Q}$ is orthogonal (its columns are orthonormal eigenvectors) and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ holds the eigenvalues. For our $\mathbf{A}$ the eigenvalues are 3 and 1 with eigenvectors $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ (you verified these back in the eigenvalue chapter). Now substitute the factorization into the form and introduce *new coordinates* $\mathbf{y} = \mathbf{Q}^{\mathsf{T}}\mathbf{x}$ – the coordinates of $\mathbf{x}$ in the eigenvector frame:

$$Q(\mathbf{x}) = \mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} = \mathbf{x}^{\mathsf{T}}\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{\mathsf{T}}\mathbf{x} = (\mathbf{Q}^{\mathsf{T}}\mathbf{x})^{\mathsf{T}}\mathbf{\Lambda}(\mathbf{Q}^{\mathsf{T}}\mathbf{x}) = \mathbf{y}^{\mathsf{T}}\mathbf{\Lambda}\mathbf{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2.$$

The cross term is gone. In the rotated coordinates the form is purely a sum of weighted squares, and the weights are the eigenvalues. This is the *principal axes theorem*, and most of the rest of the chapter leans on it.

Formal

Principal axes theorem. Let $\mathbf{A}$ be symmetric with eigenvalues $\lambda_1, \ldots, \lambda_n$ and an orthonormal eigenbasis forming the columns of an orthogonal matrix $\mathbf{Q}$. In the rotated coordinates $\mathbf{y} = \mathbf{Q}^{\mathsf{T}}\mathbf{x}$,

$$\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} \;=\; \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2.$$

Every quadratic form is, in the right rotated frame, a weighted sum of pure squares. The eigenvectors point along the principal axes of the level-set ellipsoids; the eigenvalues control how far those axes stretch.

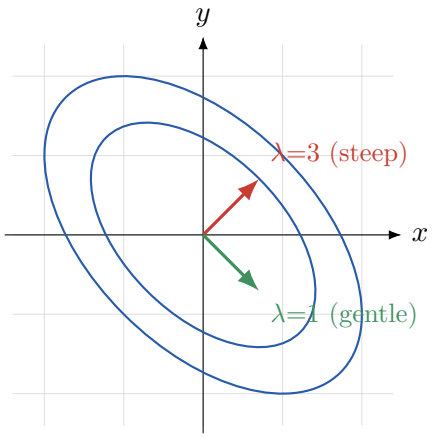


Figure 36.3: The form $2x^2 + 2xy + 2y^2$ has *tilted* elliptical level curves. The cross term rotated the picture by 45°. The eigenvectors $\begin{bmatrix}1\\1\end{bmatrix}$ and $\begin{bmatrix}1\\-1\end{bmatrix}$ are the principal axes; the bowl is steepest along the eigenvalue-3 direction and gentlest along the eigenvalue-1 direction.

Why this matters

Pause on how much this collapses. A general symmetric $n \times n$ form has up to $n(n+1)/2$ independent coefficients – a messy soup of squares and cross terms. The principal axes theorem says: *in the right orthonormal frame, all of that is just n numbers, the eigenvalues, stretching n perpendicular axes.* Curvature in many dimensions, which sounds impossibly complicated, is simply a handful of independent one-dimensional parabolas glued together at right angles. This expresses the core message of diagonalization in geometric terms: rotating to the eigenbasis decouples the coordinate directions.

36.4 Reading the shape: definiteness

Now we can answer the question the whole chapter is built around. Look at the diagonal form $\lambda_1 y_1^2 + \cdots + \lambda_n y_n^2$. Its sign at any nonzero $\mathbf{y}$ is controlled entirely by the signs of the eigenvalues.

- If *every* $\lambda_i > 0$, then every term is positive (for $\mathbf{y} \neq \mathbf{0}$), so $Q > 0$ always. The surface is a genuine **bowl**: a minimum at the origin. We call $\mathbf{A}$ *positive definite*.
- If every $\lambda_i < 0$, then $Q < 0$ always: a **dome**, a maximum. *Negative definite.*
- If the eigenvalues have *mixed* signs – some positive, some negative – then Q goes up along some axes and down along others: a **saddle**. *Indefinite.*
- If some eigenvalue is 0 (and the rest share a sign), the form flattens out along that axis – a trough or ridge rather than a strict bowl. *Positive (or negative) semidefinite.*

Formal

Definiteness via eigenvalues. For a symmetric matrix $\mathbf{A}$ with eigenvalues $\lambda_1, \dots, \lambda_n$:

$$\begin{aligned}
&\text{positive definite} && \iff \text{all } \lambda_i > 0 \iff \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} > 0 \text{ for all } \mathbf{x} \neq \mathbf{0},\\
&\text{positive semidefinite} && \iff \text{all } \lambda_i \geq 0,\\
&\text{negative definite} && \iff \text{all } \lambda_i < 0,\\
&\text{indefinite} && \iff \text{some } \lambda_i > 0 \text{ and some } \lambda_j < 0.
\end{aligned}$$

Before/after Strang

Strang devotes a whole chapter to positive definite matrices, and he loves to give the energy test $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} > 0$ as the *definition*, with the eigenvalue test and the pivot test as equivalent "tests." Reading this chapter before Strang, hold onto the geometry: positive definite *means* the surface is a bowl, so optimization works. Reading it after Strang, you now see why he cared so much – the bowl is exactly the surface on which Newton's method, conjugate gradients, and every clean convex algorithm converge.

36.4.1 Checking definiteness without computing eigenvalues

Computing eigenvalues is work. For 2×2 matrices there is a lightning-fast test using the trace and determinant. The eigenvalues satisfy $\lambda_1\lambda_2 = \det \mathbf{A}$ and $\lambda_1 + \lambda_2 = \operatorname{tr} \mathbf{A}$. So:

- $\det \mathbf{A} > 0$ means the eigenvalues share a sign (their product is positive). Then the trace tells you which: $\operatorname{tr} \mathbf{A} > 0 \Rightarrow$ both positive $\Rightarrow$ *positive definite*; $\operatorname{tr} \mathbf{A} < 0 \Rightarrow$ both negative $\Rightarrow$ negative definite.
- $\det \mathbf{A} < 0$ means the eigenvalues have opposite signs $\Rightarrow$ *indefinite*, a saddle.

Example 36.1 (The trace–determinant test in action)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: $\det = 4 - 1 = 3 > 0$ and $\operatorname{tr} = 4 > 0$, so positive definite – a bowl. Indeed its eigenvalues were $3, 1$. For $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$: $\det = 1 - 4 = -3 < 0$, so indefinite – a saddle (eigenvalues 3 and -1). Two innocent-looking matrices, just an entry swapped, with completely different geometry: one a bowl, the other a saddle.

For larger matrices the clean generalization is *Sylvester's criterion*: a symmetric matrix is positive definite exactly when all its *leading principal minors* – the determinants of the top-left 1×1, 2×2, $\dots$, $n \times n$ blocks – are positive. This criterion directly reflects the pivot test: each positive minor guarantees one positive pivot in elimination, and positive pivots correspond to positive eigenvalues.

Example 36.2 (A 3×3 check by minors)**.** Test $\mathbf{A} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$ (it will reappear as a famous matrix below). Leading minors: $|2| = 2 > 0$; $\det \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} = 3 > 0$; and the full determinant is $2(4 - 1) - (-1)(-2 - 0) = 6 - 2 = 4 > 0$. All positive, so $\mathbf{A}$ is positive definite. Its form $2x^2 + 2y^2 + 2z^2 - 2xy - 2yz$ is a 3-D bowl.

Common trap

Positive *diagonal entries do not* make a matrix positive definite. The diagonal entries are just $\mathbf{e}_i^{\mathsf{T}}\mathbf{A}\mathbf{e}_i$, the values of the form along the coordinate axes – they say nothing about diagonal directions. The matrix $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ has positive diagonal yet is indefinite. You must check the off-diagonal pull, which is what the determinant minors do.

36.5 Gradients, Hessians, and where the marble stops

Time to connect the static picture of forms to the dynamic act of *finding* a minimum of a general (not necessarily quadratic) function $f(\mathbf{x})$. Two objects do all the work.

The **gradient** ∇f collects the partial derivatives into a vector:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix}.$$

Geometrically it points in the direction of *steepest ascent*, and its length is the steepness. At a flat spot – a candidate minimum or maximum – the ground has no slope, so $\nabla f = \mathbf{0}$. That is the first-order condition.

Focus

First-order condition (stationarity). At any minimum or maximum in the interior, $\nabla f(\mathbf{x}^\star) = \mathbf{0}$. Setting the gradient to zero is how we *find* candidate optima. But a zero gradient alone cannot tell bowl from dome from saddle – for that we need curvature.

The **Hessian H** is the matrix of second partial derivatives:

$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \cdots & \dfrac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \dfrac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

Because mixed partials are equal (for nice functions), $\mathbf{H}$ is automatically *symmetric*. The Hessian is the symmetric matrix whose quadratic form *is* the local curvature of f. Here is the bridge, written precisely.

Formal

Second-order Taylor expansion. Near a stationary point $\mathbf{x}^\star$ (where $\nabla f = \mathbf{0}$), with $\mathbf{h} = \mathbf{x} - \mathbf{x}^\star$,

$$f(\mathbf{x}^\star + \mathbf{h}) \;\approx\; f(\mathbf{x}^\star) \;+\; \underbrace{\nabla f(\mathbf{x}^\star)^{\mathsf{T}}\mathbf{h}}_{=0} \;+\; \tfrac{1}{2}\,\mathbf{h}^{\mathsf{T}}\mathbf{H}(\mathbf{x}^\star)\,\mathbf{h}.$$

So up to a constant, the surface near a flat spot *is* the quadratic form of its Hessian. The

bowl/dome/saddle classification of the form transfers directly to the function.

Intuition

The gradient finds the flat spots; the Hessian reads the local shape there. This is exactly the single-variable second-derivative test you already know – "$f' = 0$ and $f'' > 0$ means a minimum" – promoted to many dimensions, with the scalar $f'' > 0$ replaced by the matrix statement "$\mathbf{H}$ is positive definite."

History and connection

In 1847, Augustin-Louis Cauchy introduced gradient descent while calculating the astronomical orbits of celestial bodies and comets. Cauchy realized that stepping in the direction of the negative gradient $-\nabla f$ guarantees an immediate decrease in function height. But he was quickly halted by a severe geometric trap: in long, narrow ravines where one direction is steep and the perpendicular direction is shallow, steepest descent bounces back and forth between the ravine walls instead of marching down the valley floor.

For a century, Newton's method offered the textbook fix by using the second-derivative Hessian matrix $\mathbf{H}$ to divide out curvature and leap straight to the bottom: $\Delta\mathbf{x} = -\mathbf{H}^{-1}\nabla f$. But storing and inverting an $n \times n$ Hessian costs $\mathcal{O}(n^3)$ operations and consumes n^2 memory words, an impossible barrier for early computers when n reached thousands of variables. Worse, near a saddle point where the Hessian has negative eigenvalues, Newton's step can launch the search uphill toward a maximum.

In 1952, Magnus Hestenes and Eduard Stiefel resolved this computational bottleneck by inventing the **Conjugate Gradient (CG)** method for symmetric positive-definite systems. Armed with the spectral geometry of quadratic forms, they constructed search directions that are mutually orthogonal with respect to the energy matrix: $\mathbf{d}_i^\mathsf{T}\mathbf{A}\mathbf{d}_j = 0$. Each step completely eliminates error along an entire principal axis of the elliptical bowl without disturbing progress made in earlier steps. CG reaches high precision in only $\mathcal{O}(\sqrt{\kappa}\log(1/\epsilon))$ matrix-vector multiplications without ever storing an inverse matrix, enabling the simulation of large-scale physical structures and fluid flows.

Meanwhile, a second optimization revolution was unfolding around inequality constraints. In 1939, a young graduate student at the University of Chicago named William Karush derived the foundational conditions for inequality-constrained optimization in his master's thesis. But World War II dispersed mathematicians into war research, and Karush's thesis sat unpublished and forgotten in the university library. Twelve years later, in 1951, Harold Kuhn and Albert W. Tucker independently rediscovered and published the identical conditions at Princeton, showing how Lagrange multipliers apply to solid boundary walls. When Kuhn later discovered Karush's 1939 thesis, he and Tucker ensured historical justice by championing Karush's priority, permanently renaming the result the **Karush–Kuhn–Tucker (KKT)** conditions. The KKT conditions provided the foundation for modern mathematical programming, economic game theory, and machine learning algorithms such as support vector machines.

Today, an active frontier is high-dimensional non-convex optimization. In machine learning models with billions of parameters, cost surfaces contain saddle points where gradients vanish but Hessians have mixed eigenvalues. Explaining when gradient methods avoid saddles, and finding efficient algorithms for non-convex quadratic programming (which is NP-hard), remain open questions.

Formal

Second-order conditions. Suppose $\nabla f(\mathbf{x}^\star) = \mathbf{0}$ and let $\mathbf{H} = \mathbf{H}(\mathbf{x}^\star)$.

$\mathbf{H}$ positive definite	$\Rightarrow$	strict local minimum (bowl),
$\mathbf{H}$ negative definite	$\Rightarrow$	strict local maximum (dome),
$\mathbf{H}$ indefinite	$\Rightarrow$	saddle point (no extremum),
$\mathbf{H}$ semidefinite (a zero eigenvalue)	$\Rightarrow$	inconclusive; look higher.

Example 36.3 (Finding and classifying critical points)**.** Let $f(x, y) = x^2 + xy + y^2 - 3y$. The gradient is

$$\nabla f = \begin{bmatrix} 2x + y \\ x + 2y - 3 \end{bmatrix}.$$

Set it to zero: $2x + y = 0$ gives $y = -2x$; substitute into $x + 2y - 3 = 0$ to get $x - 4x - 3 = 0$, so $x = -1$, $y = 2$. The lone critical point is $(-1, 2)$. The Hessian is the constant matrix

$$\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

whose $\det = 3 > 0$ and $\mathrm{tr} = 4 > 0$: positive definite. So $(-1, 2)$ is a strict minimum. We did not even need the eigenvalues – the trace–determinant test settled it. (And because $\mathbf{H}$ is constant, this minimum is the *global* one, as the next section explains.)

Example 36.4 (A saddle)**.** Let $g(x, y) = x^2 - y^2 + 2x$. Then $\nabla g = \begin{bmatrix} 2x + 2 \\ -2y \end{bmatrix} = \mathbf{0}$ at $(-1, 0)$, and $\mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$, indefinite ($\det = -4 < 0$). The point $(-1, 0)$ is a saddle: walk along x and you climb, walk along y and you descend. A marble placed exactly there is balanced but unstable.

Walking question

Take a function you can picture, like the height of a mountain pass. Stand at the saddle of the pass. In your mind, walk a tiny step in four directions: toward each peak (uphill) and down each valley (downhill). That up-one-way, down-the-other feeling *is* indefiniteness – the Hessian has one positive and one negative eigenvalue, pointing along the ridge and along the trail. Where exactly do the positive and negative eigenvector directions point on a real mountain pass?

36.6 Positive definite means convex means one bottom

For a *quadratic* function the Hessian is the same matrix everywhere – the curvature never changes. That is what makes quadratic bowls so well behaved: if the Hessian is positive definite, the function is a single perfect bowl with exactly one minimum, no other critical points, no surprises. Let us make "single perfect bowl" precise with the word *convex*.

Definition 36.2 (Convex function)**.** A function f is *convex* if for any two points $\mathbf{x}, \mathbf{z}$ and any $t \in [0, 1]$,

$$f\big(t\mathbf{x} + (1 - t)\mathbf{z}\big) \;\le\; tf(\mathbf{x}) + (1 - t)f(\mathbf{z}).$$

Geometrically: the chord joining any two points on the graph lies *on or above* the graph. The surface never bulges above its own secant lines – it always cups upward.

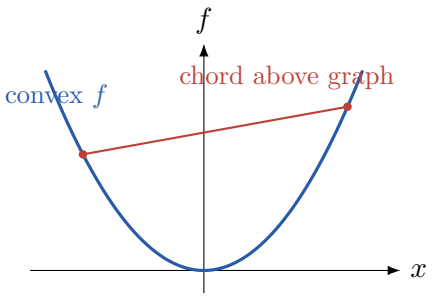


Figure 36.4: Convexity: every chord (red) lies on or above the graph (blue). A positive-definite quadratic is convex in every direction at once, which is why it has exactly one lowest point and downhill always leads there.

The connection back to our matrices is exact.

Theorem 36.1 (PD Hessian $\Leftrightarrow$ convex quadratic)**.** *A quadratic function* $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} - \mathbf{b}^\mathsf{T}\mathbf{x} + c$ *with symmetric* $\mathbf{A}$ *is convex if and only if* $\mathbf{A}$ *is positive semidefinite, and* strictly *convex (a unique global minimum) if and only if* $\mathbf{A}$ *is positive definite. More generally, a twice-differentiable function is convex if and only if its Hessian is positive semidefinite* everywhere.

Why, in one line of algebra. Convexity of a twice-differentiable function is equivalent to its second derivative being nonnegative along every line. Along the line $\mathbf{x}(t) = \mathbf{p} + t\mathbf{d}$, the second derivative of f is $\frac{d^2}{dt^2} f = \mathbf{d}^\mathsf{T}\mathbf{H}\mathbf{d}$. Demanding this be ≥ 0 for every direction $\mathbf{d}$ is *exactly* the definition of $\mathbf{H}$ positive semidefinite. For the quadratic, $\mathbf{H} = \mathbf{A}$ is constant, so one check settles convexity globally; positive definiteness (> 0 strictly) forces a single bottom. □

Why this matters

This is why linear algebra sits at the center of optimization. "Convex" is the dividing line between problems we can solve reliably and problems that hide local traps. And convex, for the smooth quadratic-like surfaces that dominate science and machine learning, *means positive definite Hessian* – an eigenvalue condition. Three vocabularies – the analyst's "convex," the optimizer's "unique global minimum," the algebraist's "positive definite" – name a single geometric fact: the surface is a bowl.

36.6.1 Solving the bowl directly

For a convex quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} - \mathbf{b}^\mathsf{T}\mathbf{x}$ with $\mathbf{A}$ positive definite, we do not need to roll downhill at all – we can jump straight to the bottom. Set the gradient to zero. Differentiating,

$$\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0} \quad \Longrightarrow \quad \boxed{\mathbf{A}\mathbf{x} = \mathbf{b}.}$$

The minimizer of the bowl is the solution of a linear system. Read that backwards and it is startling: *solving* $\mathbf{A}\mathbf{x} = \mathbf{b}$ *is the same as finding the bottom of an energy bowl.* This is the variational view

of linear systems – the one that underlies the finite-element method, where physical equilibria are computed by minimizing energy, and the conjugate-gradient algorithm, which solves $\mathbf{Ax} = \mathbf{b}$ by literally rolling toward the bottom of this bowl.

Connection to other math

The bridge to calculus and physics goes further than it first appears. In mechanics, a system at rest sits at a minimum of potential energy; the condition "∇(energy) $= 0$" is the equilibrium equation, and stability is the Hessian being positive definite. In differential equations, the matrix $\mathbf{A} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$ from Example 36.2 is the discrete second-derivative operator; its positive definiteness is the discrete statement that $-u''$ stores energy. The quadratic form $\frac{1}{2}\mathbf{x}^{\mathsf{T}}\mathbf{Ax}$ is a discretized $\int \frac{1}{2}(u')^2$. Optimization, linear systems, and the calculus of variations are one subject seen from three sides.

36.7 Least squares is the gradient of a bowl

You met least squares as projection: to "solve" an overdetermined system $\mathbf{Ax} = \mathbf{b}$ with no exact solution, find the $\mathbf{x}$ that makes $\mathbf{Ax}$ as close to $\mathbf{b}$ as possible. Optimization gives the same answer from a fresh angle, and it puts the machinery of this chapter to work on a concrete problem.

We want to minimize the squared error

$$E(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|^2 = (\mathbf{Ax} - \mathbf{b})^{\mathsf{T}}(\mathbf{Ax} - \mathbf{b}).$$

Expand it – and watch a quadratic form appear:

$$E(\mathbf{x}) = \mathbf{x}^{\mathsf{T}}(\mathbf{A}^{\mathsf{T}}\mathbf{A})\mathbf{x} \;-\; 2\,\mathbf{b}^{\mathsf{T}}\mathbf{Ax} \;+\; \mathbf{b}^{\mathsf{T}}\mathbf{b}.$$

The matrix of the quadratic part is $\mathbf{A}^{\mathsf{T}}\mathbf{A}$, which is always symmetric and positive semidefinite – and positive *definite* as soon as $\mathbf{A}$ has independent columns. So E is a convex bowl, and its unique bottom is found by setting the gradient to zero:

$$\nabla E(\mathbf{x}) = 2\mathbf{A}^{\mathsf{T}}\mathbf{Ax} - 2\mathbf{A}^{\mathsf{T}}\mathbf{b} = \mathbf{0} \quad \Longrightarrow \quad \boxed{\mathbf{A}^{\mathsf{T}}\mathbf{A}\,\mathbf{x} = \mathbf{A}^{\mathsf{T}}\mathbf{b}.}$$

These are the *normal equations* – the very ones the projection picture produced. The two derivations agree because they are the same fact: the closest point in a subspace, and the bottom of the error bowl, are one and the same.

Example 36.5 (A tiny least-squares fit done as optimization)**.** Fit a line $y = c + dx$ through the three points $(0,1), (1,2), (2,2)$. The model says

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} c \\ d \end{bmatrix}.$$

Then $\mathbf{A}^{\mathsf{T}}\mathbf{A} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$ (note $\det = 6 > 0$, so positive definite – a bowl) and $\mathbf{A}^{\mathsf{T}}\mathbf{b} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$. Solving $\begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}\begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$ gives, after elimination, $d = \frac{1}{2}$ and $c = \frac{7}{6}$. The best-fit line is $y = \frac{7}{6} + \frac{1}{2}x$. We found it by locating the bottom of a two-dimensional bowl over the (c,d)-plane.

Modern application

Training a linear model in machine learning is exactly this bowl, scaled up. A linear regressor with weight vector $\mathbf{w}$ and bias predicts $\hat{y} = \mathbf{w}^\mathsf{T}\mathbf{x} + b$, and training minimizes mean squared error – a convex quadratic in $(\mathbf{w}, b)$ whose Hessian is essentially $\mathbf{X}^\mathsf{T}\mathbf{X}/n$, the (scaled) data covariance matrix. Because that Hessian is positive semidefinite, the loss is convex and gradient descent cannot get stuck: there is one valley. The *eigenvalues* of $\mathbf{X}^\mathsf{T}\mathbf{X}$ even set the speed limit – the ratio of largest to smallest eigenvalue (the *condition number*) tells you how stretched the bowl is, and a long thin bowl is what makes gradient descent zig-zag slowly (Figure 36.5). This is why practitioners *normalize* their features: doing so reshapes the bowl toward a round one (eigenvalues closer together), and the marble rolls straight to the bottom. The whole intuition – convex loss, eigenvalue-controlled curvature, preconditioning to round the bowl – is the content of this chapter.

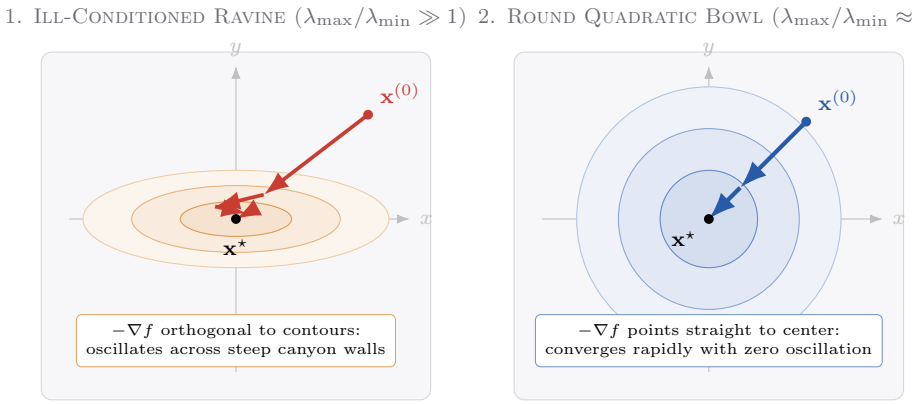


Figure 36.5: Gradient descent trajectories on quadratic form level curves. Left: On an ill-conditioned quadratic form with wide eigenvalue spread ($\lambda_{\max} \gg \lambda_{\min}$), level curves are tightly compressed ellipses. The gradient $-\nabla f$ is orthogonal to the local tangent, causing steps to bounce back and forth between steep walls while progressing slowly down the gentle valley floor. Right: On a well-conditioned or preconditioned bowl ($\lambda_{\max} \approx \lambda_{\min}$), level curves are circular and the negative gradient points straight toward the global minimum $\mathbf{x}^\star$.

36.8 Constrained optimization: a first look at Lagrange

So far the marble could roll anywhere. But real problems come fenced in: *minimize this, subject to a constraint.* Find the closest point on a circle to the origin's bowl; find the direction of maximum stretch on the unit sphere; allocate a fixed budget. The constraint confines the marble to a curve or surface, and the geometry of the answer changes in a clean, memorable way.

The setup: minimize $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$. Think of f as the bowl and $g = 0$ as a fence – a curve drawn on the floor that the marble must stay on.

Intuition

Walk along the fence $g = 0$, watching the height f. You are at a constrained minimum when, locally, you cannot lower f by moving along the fence either way – the fence is momentarily *level* with respect to f. That happens precisely when the fence runs *tangent* to a level curve of f. And "tangent to a level curve" means the two gradients line up: ∇f (perpendicular to f's level curves) is parallel to ∇g (perpendicular to the fence).

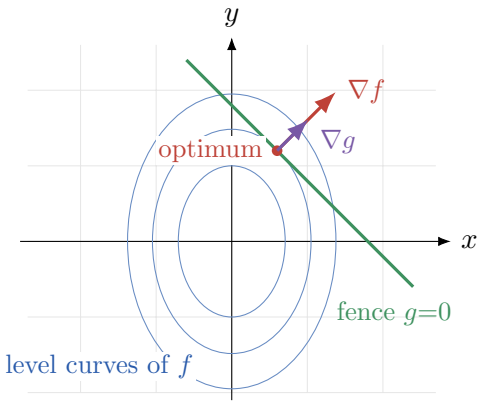


Figure 36.6: Constrained optimum. The marble must stay on the green fence $g = 0$. The optimal point is where the fence lies *tangent* to a level curve of f. There the gradients ∇f and ∇g point along the same line: $\nabla f = \lambda \nabla g$.

That alignment is the *method of Lagrange multipliers*.

Formal

Lagrange multipliers. To minimize (or maximize) $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$, look for points where

$$\nabla f(\mathbf{x}) = \lambda \nabla g(\mathbf{x}), \qquad g(\mathbf{x}) = 0,$$

for some scalar λ (the *multiplier*). Equivalently, form the *Lagrangian* $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda\, g(\mathbf{x})$ and set all its partial derivatives to zero. The multiplier λ measures how hard the constraint is pushing: it is the rate at which the optimal value would improve if you loosened the constraint.

Example 36.6 (Closest point on a line – the projection, rediscovered)**.** Minimize $f(x, y) = x^2 + y^2$ (squared distance from the origin) subject to $g(x, y) = x + y - 2 = 0$. Then $\nabla f = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$ and $\nabla g = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. The condition $\nabla f = \lambda \nabla g$ gives $2x = \lambda$ and $2y = \lambda$, so $x = y$. Plug into the fence $x + y = 2$: $x = y = 1$. The closest point on the line $x + y = 2$ to the origin is $(1, 1)$, at distance $\sqrt{2}$ – exactly what an orthogonal projection would give, now obtained by tangency.

36.8.1 Optimization recovers eigenvalues

Applying Lagrange multipliers on the unit sphere leads straight back to eigenvalues. Maximize the quadratic form $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ over the *unit sphere* $\|\mathbf{x}\| = 1$. In words: *in which direction does the form grow*

the most, among all unit vectors? Set up Lagrange with $f = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ and constraint $g = \mathbf{x}^\mathsf{T}\mathbf{x} - 1 = 0$. Using $\nabla(\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}) = 2\mathbf{A}\mathbf{x}$ and $\nabla(\mathbf{x}^\mathsf{T}\mathbf{x}) = 2\mathbf{x}$, the Lagrange condition $\nabla f = \lambda \nabla g$ becomes

$$2\mathbf{A}\mathbf{x} = \lambda\, 2\mathbf{x} \qquad \Longrightarrow \qquad \boxed{\mathbf{A}\mathbf{x} = \lambda \mathbf{x}.}$$

The eigenvalue equation, falling out of a constrained optimization. The critical points of a quadratic form on the unit sphere are exactly the *eigenvectors*, and the value of the form there is the *eigenvalue*: $\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \mathbf{x}^\mathsf{T}(\lambda\mathbf{x}) = \lambda$. So the *maximum* of the form on the sphere is the *largest* eigenvalue, attained at its eigenvector; the minimum is the smallest eigenvalue.

Formal

Rayleigh quotient. For symmetric $\mathbf{A}$, the *Rayleigh quotient* $R(\mathbf{x}) = \dfrac{\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}}{\mathbf{x}^\mathsf{T}\mathbf{x}}$ satisfies

$$\lambda_{\min} \;\le\; R(\mathbf{x}) \;\le\; \lambda_{\max} \qquad \text{for all } \mathbf{x} \neq \mathbf{0},$$

with equality exactly at the corresponding eigenvectors. The biggest and smallest eigenvalues are the maximum and minimum of the form on the unit sphere.

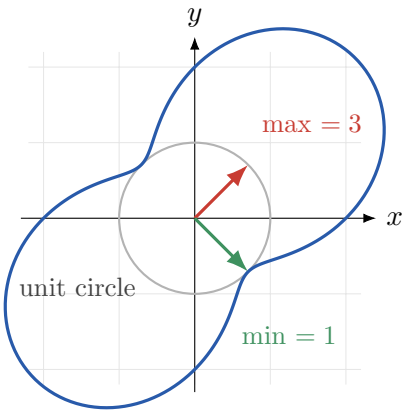


Figure 36.7: The Rayleigh quotient of $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ made visible. Along each unit direction (gray circle) we plot the value $R(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ as the radius of the blue curve. It bulges farthest out in the top eigenvector direction $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (value $\lambda_{\max} = 3$) and pulls in closest along $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ (value $\lambda_{\min} = 1$). Maximizing a form on the sphere *is* finding the top eigenvector.

Example 36.7 (Rayleigh on a 2×2)**.** For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (eigenvalues 3 and 1), the form $2x^2 + 2xy + 2y^2$ on the unit circle ranges between 1 and 3. Its maximum 3 is hit at $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (the form there is $\frac{1}{2}(2 + 2 + 2) = 3$), its minimum 1 at $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ ($\frac{1}{2}(2 - 2 + 2) = 1$). The ellipse of Figure 36.3 is squashed precisely between these extremes.

Why this matters

Notice how the theory comes full circle. We began the book with eigenvectors as "directions a matrix does not turn." Now they return as "directions in which a quadratic form is stationary on the sphere" – the steepest and flattest directions of the bowl, the principal axes of the ellipse, the answer to a constrained optimization. Same vectors, three completely different stories: invariance, curvature, and optimality. It is also why this exact computation powers *principal component analysis*: the first principal component of a data set is the unit direction maximizing the variance $\mathbf{w}^{\mathsf{T}}\mathbf{C}\mathbf{w}$ on the sphere – the top eigenvector of the covariance matrix, found by the Rayleigh quotient.

36.9 Constrained convex optimization, duality, and the KKT conditions

In Section 36.8, an equality constraint $g(\mathbf{x}) = 0$ forced motion along a specified curve or surface. Stepping to either side was strictly forbidden. But in engineering, economics, and machine learning, real-world constraints rarely confine us to curves. Instead, they provide feasible regions bounded by one-sided barriers:

- A financial portfolio must keep asset weights non-negative: $w_i \geq 0$.
- A factory cannot exceed its raw material budget: $\sum c_i x_i \leq B$.
- A machine learning classifier must keep data points outside a decision margin: $y_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) \geq 1$.

These are *inequality constraints*. They define feasible regions with an interior and a boundary. To handle them, we need to understand the geometry of the feasible region, the forces that keep an optimizer from crossing its boundary, and the Karush–Kuhn–Tucker (KKT) conditions that govern equilibrium.

36.9.1 Why convexity matters geometrically

Before adding constraints, when can we trust that rolling downhill leads to the global minimum?

On a non-convex surface, optimization is difficult. The surface may have many local minima, plateaus, and saddle points. An algorithm stepping downhill can settle into a poor local minimum while a much better point exists elsewhere.

Convexity is the geometric property that dismantles this trap. It requires two ingredients: a convex function, and a convex domain.

Definition 36.3 (Convex set)**.** A set $C \subseteq \mathbb{R}^n$ is *convex* if for any two points $\mathbf{x}, \mathbf{z} \in C$ and any scalar $t \in [0, 1]$, the line segment connecting them lies entirely within C:

$$t\mathbf{x} + (1 - t)\mathbf{z} \in C.$$

Picture walking in a straight line between any two spots in your yard. If the yard has no indentations, holes, or pinched waists, your path never steps outside the property line. A solid sphere, an ellipse, a

cube, and a half-space $\{\mathbf{x} : \mathbf{a}^\mathsf{T}\mathbf{x} \le b\}$ are all convex sets. A crescent moon, an hourglass, or a donut are not.

Now place a convex function f on top of this convex set C. As we saw in Definition 36.2, a function is convex when the straight line chord between any two points on its graph sits on or above the graph. When f is twice differentiable, Theorem 36.1 guarantees that this is equivalent to the Hessian matrix being positive semidefinite everywhere:

$$\nabla^2 f(\mathbf{x}) \succeq 0 \qquad \text{for all } \mathbf{x} \in C.$$

The eigenvalues of the Hessian are non-negative everywhere in the domain. The surface never flexes downward into domes or saddles; it cups upward in every direction.

Formal

Local minima are global in convex optimization. Let $f : C \to \mathbb{R}$ be a convex function defined on a convex set C. Then:

1. Every local minimum of f on C is a global minimum of f on C.
2. The set of all global minimizers is convex.
3. If f is strictly convex (for instance, if $\nabla^2 f(\mathbf{x}) \succ 0$ is strictly positive definite), the global minimum is unique.

Why local implies global. Suppose $\mathbf{x}^\star \in C$ is a local minimum, but there exists some other point $\mathbf{z} \in C$ with $f(\mathbf{z}) < f(\mathbf{x}^\star)$. Because C is convex, the entire line segment $\mathbf{x}(t) = (1-t)\mathbf{x}^\star + t\mathbf{z}$ belongs to C for all $t \in (0, 1]$. By the definition of a convex function,

$$f(\mathbf{x}(t)) \;\le\; (1-t)f(\mathbf{x}^\star) + tf(\mathbf{z}) \;<\; (1-t)f(\mathbf{x}^\star) + tf(\mathbf{x}^\star) \;=\; f(\mathbf{x}^\star).$$

As $t \to 0$, the point $\mathbf{x}(t)$ approaches $\mathbf{x}^\star$ arbitrarily closely, yet $f(\mathbf{x}(t)) < f(\mathbf{x}^\star)$. This contradicts the premise that $\mathbf{x}^\star$ was a local minimum. Hence no such downhill point $\mathbf{z}$ can exist: $\mathbf{x}^\star$ must be a global minimum. □

This proof reveals why both ingredients are required. If the function is not convex, the graph can dip between two points. If the set is not convex, the connecting segment could pass through forbidden territory outside C, preventing the straight-line descent. When both f and C are convex, every local minimum is guaranteed to be a global minimum.

36.9.2 Inequality constraints as physical walls

Now let us add constraints. We wish to solve the standard inequality-constrained problem:

$$\text{minimize } f(\mathbf{x}) \qquad \text{subject to } g(\mathbf{x}) \le 0.$$

The function $g(\mathbf{x})$ defines our territory: the region where $g(\mathbf{x}) < 0$ is the interior of the yard, the set $g(\mathbf{x}) = 0$ is the boundary fence, and points with $g(\mathbf{x}) > 0$ lie in forbidden space.

Picture rolling a marble on the surface $f(\mathbf{x})$ inside this fenced yard. Gravity pulls the marble downhill along the direction of steepest descent, which is the negative gradient vector $-\nabla f(\mathbf{x})$. When the marble comes to rest at an optimal point $\mathbf{x}^\star$, where does it sit?

There are two cases, illustrated in Figure 36.8:

1. **Case 1: Inactive constraint ($g(\mathbf{x}^\star) < 0$, $\mu = 0$).** The bottom of the bowl lies safely inside the yard. The marble rolls down and settles at the natural unconstrained floor, where the ground is completely flat:

$$\nabla f(\mathbf{x}^\star) = \mathbf{0}.$$

   Because the marble never touches the boundary fence, the fence exerts zero physical force on it. If we think of the fence as a physical barrier exerting an inward normal reaction force proportional to a Lagrange multiplier μ, that force must be zero: $\mu = 0$. Removing the fence entirely would not change the position of the marble at all.

2. **Case 2: Active constraint ($g(\mathbf{x}^\star) = 0$, $\mu > 0$).** The true bottom of the bowl lies outside the fence, in forbidden territory. The marble rolls downhill until it collides with the fence boundary $g(\mathbf{x}) = 0$. It wants to continue rolling downhill into the forbidden zone, so the downhill pull of gravity $-\nabla f(\mathbf{x}^\star)$ points directly into the exterior, where $g(\mathbf{x}) > 0$.

   Because the gradient $\nabla g(\mathbf{x}^\star)$ always points toward the direction of increasing g (outward across the fence), the downhill gravity vector $-\nabla f(\mathbf{x}^\star)$ aligns with the outward normal $\nabla g(\mathbf{x}^\star)$. To prevent the marble from rolling through, the solid fence pushes back inward along the inward normal $-\nabla g(\mathbf{x}^\star)$. We write this inward normal reaction force as

$$\mathbf{F}_{\text{wall}} = -\mu \nabla g(\mathbf{x}^\star).$$

   Because physical walls can only push to keep an object out – they cannot act as sticky glue or reach out to pull the marble from afar – the multiplier must be non-negative:

$$\mu \geq 0.$$

   At equilibrium, the downhill pull of gravity and the inward push of the wall must cancel each other out:

$$-\nabla f(\mathbf{x}^\star) + \mathbf{F}_{\text{wall}} \;=\; -\nabla f(\mathbf{x}^\star) - \mu \nabla g(\mathbf{x}^\star) \;=\; \mathbf{0} \quad \Longrightarrow \quad \nabla f(\mathbf{x}^\star) + \mu \nabla g(\mathbf{x}^\star) \;=\; \mathbf{0}.$$

These two physical cases condense into a single algebraic equation:

$$\mu \cdot g(\mathbf{x}^\star) = 0.$$

Consider what this equation requires. If the constraint is inactive ($g < 0$), the multiplier μ must be 0. If the multiplier is strictly positive ($\mu > 0$), the marble must be touching the wall ($g = 0$). At least one of the two factors must be zero. This is the condition of **complementary slackness**: *you cannot feel a force from a wall you are not touching.*

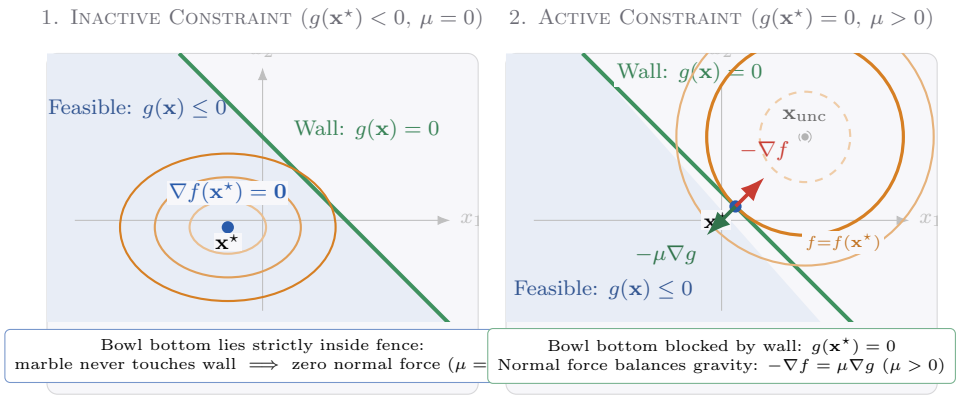


Figure 36.8: The two physical regimes of Karush–Kuhn–Tucker optimization. Left (Panel A): The unconstrained minimizer $\mathbf{x}^\star$ sits strictly inside the feasible region ($g(\mathbf{x}^\star) < 0$). The marble rests at the bottom of the bowl with zero slope ($\nabla f = \mathbf{0}$) and never touches the wall ($\mu = 0$). Right (Panel B): The unconstrained bottom $\mathbf{x}_{\text{unc}}$ lies outside in forbidden territory. The marble is stopped by the boundary wall $g(\mathbf{x}^\star) = 0$, where the inward normal force $-\mu\nabla g(\mathbf{x}^\star)$ ($\mu > 0$) balances the downhill pull of gravity $-\nabla f(\mathbf{x}^\star)$.

36.9.3 The Karush–Kuhn–Tucker (KKT) conditions

When a problem involves multiple inequality constraints $g_i(\mathbf{x}) \le 0$ ($i = 1, \dots, m$) and equality constraints $h_j(\mathbf{x}) = 0$ ($j = 1, \dots, p$), each constraint contributes its own normal force to the marble.

The general optimization problem is:

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \le 0, \quad i = 1, \dots, m, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p. \end{aligned}$$

We collect the objective and all constraints into a single function, the *Lagrangian*:

$$\mathcal{L}(\mathbf{x}, \mu, \lambda) \;=\; f(\mathbf{x}) \;+\; \sum_{i=1}^{m} \mu_i g_i(\mathbf{x}) \;+\; \sum_{j=1}^{p} \lambda_j h_j(\mathbf{x}),$$

where each μ_i is the multiplier for inequality wall $g_i \le 0$, and each λ_j is the multiplier for equality groove $h_j = 0$.

Formal

The Karush–Kuhn–Tucker (KKT) conditions. Let $\mathbf{x}^\star$ be an optimal point and let $(\mu^\star, \lambda^\star)$ be the associated Lagrange multipliers. Then they satisfy four conditions:

1. **Primal Feasibility:**

$$g_i(\mathbf{x}^\star) \;\le\; 0 \quad (i = 1, \dots, m), \qquad h_j(\mathbf{x}^\star) \;=\; 0 \quad (j = 1, \dots, p).$$

2. **Dual Feasibility:**

$$\mu_i^\star \;\ge\; 0 \quad (i = 1, \dots, m).$$

3. **Complementary Slackness:**

$$\mu_i^\star \cdot g_i(\mathbf{x}^\star) \;=\; 0 \quad (i = 1, \ldots, m).$$

4. **Stationarity:**

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^\star, \mu^\star, \lambda^\star) \;=\; \nabla f(\mathbf{x}^\star) \;+\; \sum_{i=1}^{m} \mu_i^\star \nabla g_i(\mathbf{x}^\star) \;+\; \sum_{j=1}^{p} \lambda_j^\star \nabla h_j(\mathbf{x}^\star) \;=\; \mathbf{0}.$$

Notice the physical meaning embedded in each line:

- *Primal feasibility* ensures the marble is inside the allowed territory.
- *Dual feasibility* ensures solid walls push inward rather than pulling like glue. Equality multipliers λ_j are free to be positive or negative because an equality constraint is a two-sided track that can push from either direction.
- *Complementary slackness* confirms that a wall only pushes if the marble is actively pressed against it ($g_i = 0$); if the marble is in the interior ($g_i < 0$), the multiplier μ_i must vanish.
- *Stationarity* asserts that the sum of all forces – gravity $-\nabla f$, wall pushes $-\mu_i \nabla g_i$, and track reactions $-\lambda_j \nabla h_j$ – equals zero.

36.9.4 Lagrange duality and Slater's condition

There is a second, complementary angle on these multipliers: the perspective of *duality*.

Consider the minimax game hidden inside the Lagrangian. If we choose a vector $\mathbf{x}$ that violates a constraint – say, $g_1(\mathbf{x}) > 0$ – then a challenger who wants to maximize $\mathcal{L}(\mathbf{x}, \mu, \lambda)$ with respect to $\mu \geq \mathbf{0}$ can choose $\mu_1 \to +\infty$, driving $\mathcal{L}$ to positive infinity. If $\mathbf{x}$ is feasible, however, each $g_i(\mathbf{x}) \leq 0$, so the best the challenger can do is set $\mu_i = 0$ whenever $g_i < 0$, giving an optimal value of exactly $f(\mathbf{x})$.

Therefore, the original primal optimization problem can be phrased as:

$$p^\star \;=\; \min_{\mathbf{x}} \; \max_{\mu \geq \mathbf{0},\, \lambda} \; \mathcal{L}(\mathbf{x}, \mu, \lambda).$$

Now reverse the order of operations. What happens if we maximize first, and minimize second? Define the **Lagrange dual function**:

$$d(\mu, \lambda) \;=\; \inf_{\mathbf{x}} \; \mathcal{L}(\mathbf{x}, \mu, \lambda).$$

Because the dual function is the pointwise infimum of a collection of affine functions in (μ, λ), it is *always concave*, regardless of whether the original objective f was convex.

For any valid choice of multipliers $\mu \geq \mathbf{0}$ and any feasible point $\tilde{\mathbf{x}}$,

$$d(\mu, \lambda) \;\leq\; \mathcal{L}(\tilde{\mathbf{x}}, \mu, \lambda) \;=\; f(\tilde{\mathbf{x}}) + \sum_{i=1}^{m} \underbrace{\mu_i g_i(\tilde{\mathbf{x}})}_{\leq 0} + \sum_{j=1}^{p} \lambda_j \underbrace{h_j(\tilde{\mathbf{x}})}_{=0} \;\leq\; f(\tilde{\mathbf{x}}).$$

Because this holds for every feasible $\tilde{\mathbf{x}}$, taking the infimum on the right yields the inequality of **weak duality**:

$$d(\mu, \lambda) \;\leq\; p^\star \qquad \text{for all } \mu \geq \mathbf{0},\ \lambda.$$

Every evaluation of the dual function establishes a lower bound on the optimal value $p^\star$ of our original problem.

The **dual problem** seeks the tightest possible lower bound:

$$d^\star \;=\; \max_{\mu \geq \mathbf{0},\, \lambda} d(\mu, \lambda).$$

Weak duality ensures that $d^\star \leq p^\star$. The gap $p^\star - d^\star \geq 0$ is the *duality gap*.

When does the gap close completely, so that $d^\star = p^\star$? Under **strong duality**, solving the dual problem gives the exact minimum of the primal problem.

Theorem 36.2 (Slater's condition for strong duality)**.** *Suppose the objective f and the inequality functions $g_1, \ldots, g_m$ are convex, and the equality functions $h_1, \ldots, h_p$ are affine ($h_j(\mathbf{x}) = \mathbf{a}_j^\mathsf{T}\mathbf{x} - b_j$). If there exists at least one point $\mathbf{x}_0$ that is* strictly feasible*, meaning*

$$g_i(\mathbf{x}_0) < 0 \quad (i = 1, \ldots, m) \qquad \textit{and} \qquad h_j(\mathbf{x}_0) = 0 \quad (j = 1, \ldots, p),$$

*then strong duality holds ($d^\star = p^\star$), and the dual maximum is attained. Under Slater's condition, the KKT conditions are both **necessary and sufficient** for global optimality.*

Slater's condition requires only that the feasible region possesses a genuine interior: the fence must enclose some volume, rather than collapsing into an empty set or a flat pancake. When it holds, solving the KKT equations guarantees you have found the unique global optimum.

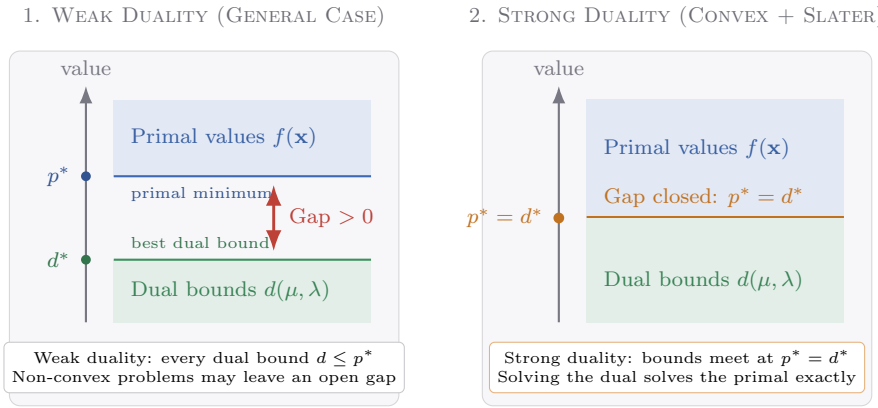


Figure 36.9: The geometry of weak and strong duality. Left: For a general optimization problem, every evaluation of the Lagrange dual function $d(\mu, \lambda)$ yields a certified lower bound on the primal optimal value $p^\star$ (weak duality), but an unbridgeable gap $p^\star - d^\star > 0$ may remain. Right: When the objective and constraints are convex and Slater's condition holds, the gap closes completely ($p^\star = d^\star$), allowing us to solve the primal problem by solving its dual.

36.9.5 A worked two-dimensional quadratic program

Let us work through a concrete problem from beginning to end, watching how complementary slackness reveals whether the boundary wall is active.

Example 36.8 (A constrained quadratic bowl)**.** Minimize the distance from $(3, 2)$ subject to a diagonal wall:

$$\text{minimize } f(x_1, x_2) = (x_1 - 3)^2 + (x_2 - 2)^2 \qquad \text{subject to } x_1 + x_2 \leq 3.$$

Step 1: Inspect the unconstrained minimum. The objective f is a circular paraboloid centered at $(3, 2)$, with minimum value 0. Let us check whether this point satisfies the constraint:

$$g(3, 2) = 3 + 2 - 3 = 2 > 0.$$

The natural bottom of the bowl lies in forbidden territory. Hence the marble cannot settle in the interior; it must be stopped by the boundary.

Step 2: Apply complementary slackness. Because $g(\mathbf{x}^\star) < 0$ is impossible (the unconstrained point violates the fence), complementary slackness $\mu \cdot g(\mathbf{x}^\star) = 0$ dictates that the constraint must be **active**:

$$x_1^\star + x_2^\star = 3 \qquad \text{and} \qquad \mu > 0.$$

Step 3: Stationarity. The gradient of f and the gradient of the constraint $g(x_1, x_2) = x_1 + x_2 - 3$ are:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} 2(x_1 - 3) \\ 2(x_2 - 2) \end{bmatrix}, \qquad \nabla g(\mathbf{x}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Setting the gradient of the Lagrangian to zero gives:

$$\nabla f(\mathbf{x}) + \mu \nabla g(\mathbf{x}) = \begin{bmatrix} 2(x_1 - 3) + \mu \\ 2(x_2 - 2) + \mu \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Solving for the coordinates in terms of the multiplier μ:

$$x_1 = 3 - \frac{\mu}{2}, \qquad x_2 = 2 - \frac{\mu}{2}.$$

Step 4: Solve for the multiplier. Substitute these coordinates into the active boundary equation $x_1 + x_2 = 3$:

$$\left(3 - \frac{\mu}{2}\right) + \left(2 - \frac{\mu}{2}\right) \;=\; 3 \quad \Longrightarrow \quad 5 - \mu = 3 \quad \Longrightarrow \quad \mu = 2.$$

Step 5: Verify dual feasibility and find the optimum. We found $\mu = 2 > 0$, satisfying the dual feasibility condition $\mu \geq 0$. The optimal coordinates are:

$$x_1^\star = 3 - \frac{2}{2} = 2, \qquad x_2^\star = 2 - \frac{2}{2} = 1.$$

The constrained minimum is $\mathbf{x}^\star = (2, 1)$, with optimal objective value:

$$f(2, 1) = (2 - 3)^2 + (1 - 2)^2 = (-1)^2 + (-1)^2 = 2.$$

Step 6: Check the physical force balance. Evaluate the gradients at $\mathbf{x}^\star = (2, 1)$:

$$\nabla f(2, 1) = \begin{bmatrix} 2(2 - 3) \\ 2(1 - 2) \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}, \qquad \nabla g(2, 1) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The downhill pull of gravity is:

$$-\nabla f(\mathbf{x}^\star) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

The inward reaction force from the wall is:

$$\mathbf{F}_{\text{wall}} = -\mu \nabla g(\mathbf{x}^\star) = -2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

The two forces are equal in magnitude and point in precisely opposite directions:

$$-\nabla f(\mathbf{x}^\star) + \mathbf{F}_{\text{wall}} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} -2 \\ -2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The marble rests in perfect mechanical equilibrium against the wall at $(2, 1)$.

Modern application

Why Support Vector Machines only care about boundary points. In classification problems, a linear Support Vector Machine (SVM) separates two clouds of data points (labeled $y_i \in \{-1, +1\}$) by finding a decision hyperplane $\mathbf{w}^\mathsf{T}\mathbf{x} + b = 0$ with the widest possible empty corridor (the *margin*) between them. Maximizing this margin is equivalent to minimizing the energy $\frac{1}{2}\|\mathbf{w}\|^2$ subject to keeping every data point on the correct side of the margin boundary:

$$y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \;\geq\; 1 \quad\Longleftrightarrow\quad g_i(\mathbf{w}, b) \;=\; 1 - y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \;\leq\; 0 \qquad (i = 1, \ldots, N).$$

Every single training example represents an inequality wall.
Now write down KKT complementary slackness for this system:

$$\mu_i \cdot \big(1 - y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)\big) \;=\; 0 \qquad (i = 1, \ldots, N).$$

And examine the stationarity condition with respect to the weight vector $\mathbf{w}$:

$$\nabla_{\mathbf{w}}\mathcal{L} \;=\; \mathbf{w} - \sum_{i=1}^{N} \mu_i y_i \mathbf{x}_i \;=\; \mathbf{0} \quad\Longrightarrow\quad \mathbf{w}^\star \;=\; \sum_{i=1}^{N} \mu_i^\star y_i \mathbf{x}_i.$$

Look at the consequence of these two lines:

- If a training example $\mathbf{x}_i$ lies safely inside its class territory, away from the margin boundary ($y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) > 1$), then $g_i < 0$. Complementary slackness forces its multiplier to be identically zero: $\mu_i^\star = 0$.
- Because $\mu_i^\star = 0$, that data point vanishes completely from the sum for $\mathbf{w}^\star$.
- Only the rare points that lie directly on the boundary of the margin corridor ($g_i = 0$) can possess a positive multiplier $\mu_i^\star > 0$.

These critical boundary examples are the **support vectors**. If your dataset contains ten million images, but only twenty of them press against the margin wall, the optimal classifier $\mathbf{w}^\star$ depends solely on those twenty images. You could delete the other 9,999,980 examples from your hard drive, retrain the model, and the decision boundary would not move by a fraction of a millimeter. The sparsity of support vector machines follows directly from KKT complementary slackness.

36.10 Putting it together: a worked classification

Let us run the full pipeline once on a three-variable function, using everything in the chapter.

Example 36.9 (End to end)**.** Minimize $f(x, y, z) = x^2 + 2y^2 + 3z^2 - 2xy - 2yz - 4x$.

Step 1 – gradient. Differentiate:

$$\nabla f = \begin{bmatrix} 2x - 2y - 4 \\ 4y - 2x - 2z \\ 6z - 2y \end{bmatrix}.$$

Step 2 – find the flat spot. Set $\nabla f = \mathbf{0}$. From the third equation $z = \frac{y}{3}$. Substitute into the second: $4y - 2x - \frac{2y}{3} = 0$, i.e. $x = \frac{5y}{3}$. The first equation $2x - 2y - 4 = 0$ gives $x - y = 2$; with $x = \frac{5y}{3}$ that is $\frac{5y}{3} - y = 2$, so $\frac{2y}{3} = 2$, $y = 3$, then $x = 5$, $z = 1$. The lone critical point is $(5, 3, 1)$.

Step 3 – Hessian. Second derivatives give the constant symmetric matrix

$$\mathbf{H} = \begin{bmatrix} 2 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 6 \end{bmatrix}.$$

Step 4 – definiteness by leading minors. $|2| = 2 > 0$; $\det \begin{bmatrix} 2 & -2 \\ -2 & 4 \end{bmatrix} = 8 - 4 = 4 > 0$; and the full determinant is $2(24 - 4) - (-2)(-12 - 0) + 0 = 40 - 24 = 16 > 0$. All three leading minors are positive, so $\mathbf{H}$ is positive definite.

Step 5 – conclude. The Hessian is positive definite everywhere (it is constant), so f is strictly convex and $(5, 3, 1)$ is the unique *global* minimum. The bowl has exactly one bottom, and we have found it.

Pause

If that example felt like a lot, notice it was only four moves, each one a tool you already owned: take the gradient (calculus), solve a linear system for the flat spot (elimination), build the Hessian (more calculus), and test definiteness (minors / eigenvalues). Geometry-first the whole way: find the flat spot, then check it cups upward. Reread it once and the pattern, not the arithmetic, is what should stick.

36.11 What to carry forward

Focus

The six ideas of this chapter.

1. A *quadratic form* $\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x}$ (with $\mathbf{A}$ symmetric) is the matrix version of ax^2: bowl, dome, or saddle.
2. *Principal axes:* rotate to the eigenvectors and the form becomes $\lambda_1 y_1^2 + \cdots + \lambda_n y_n^2$ – a

sum of squares weighted by eigenvalues.

3. *Definiteness* is just the signs of the eigenvalues: all positive ⇒ bowl (minimum), mixed ⇒ saddle. Check it fast with trace/determinant or leading minors.
4. To *find* optima of a general f: gradient zero locates flat spots; the *Hessian* (a quadratic form) classifies them. Positive definite Hessian ⇔ convex ⇔ unique minimum.
5. *Equality constraints* align gradients ($\nabla f = \lambda \nabla g$). On the unit sphere this regenerates the eigenvalue problem, and the Rayleigh quotient bounds the form between $\lambda_{\min}$ and $\lambda_{\max}$.
6. *Inequality walls and KKT:* Solid boundary walls ($g_i(\mathbf{x}) \leq 0$) can only push inward ($\mu_i \geq 0$). Complementary slackness ($\mu_i g_i(\mathbf{x}^\star) = 0$) means an untouched wall exerts zero force. For convex problems with Slater's condition, the four KKT conditions give an exact characterization of the global optimum.

Optimization turned out not to be a new subject at all. It is the geometry of symmetric matrices – curvature, bowls, and principal axes – with a thin layer of calculus (the gradient) on top to locate the flat spots. Positive definiteness, which looked like a technical condition back in the matrix chapters, is revealed as the single most useful property a problem can have: it guarantees that a unique global minimum exists and that descent methods converge to it.

In large-scale computation, however, three practical obstacles remain. If the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ is large, circular level curves stretch into narrow, steep-walled canyons; standard gradient descent bounces endlessly between the walls, requiring thousands of iterations where preconditioning could have rounded the bowl. On the non-convex surfaces of deep learning, stationary points $\nabla f = \mathbf{0}$ are overwhelmingly high-dimensional saddles rather than true minima, where uncorrected second-order steps can launch an optimizer uphill. In addition, storing an explicit $n \times n$ Hessian requires $\mathcal{O}(n^2)$ memory: for a modern neural network with a billion parameters, writing down the Hessian would demand exabytes of storage, forcing practitioners to optimize with matrix-free products or limited-memory approximations like L-BFGS.

Notice, too, the boundary of this chapter: optimization took place in finite-dimensional space $\mathbb{R}^n$. In Chapter 37, *Connections: Groups, Tensors, and Functional Analysis*, we extend linear algebra in three major directions: into infinite-dimensional Hilbert spaces, matrix Lie groups, and multilinear tensors.

Exercises

Exercise 36.1 (Mental check: shapes from signs)**.** Without writing down any scratch paper calculations, determine the geometry:

1. A symmetric 3×3 matrix has eigenvalues $7.2, 1.4, -0.05$. Is the critical point at the origin a minimum, a maximum, or a saddle?
2. A 2×2 symmetric matrix has $\det \mathbf{A} = -4$. Can the surface $z = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ ever form an upward bowl? Why or why not?
3. If $\mathbf{A}$ is positive definite and you scale all of its entries by a positive scalar $c > 0$, does the

minimizer of $\frac{1}{2}\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} - \mathbf{b}^{\mathsf{T}}\mathbf{x}$ move? (Hint: what does setting the gradient to zero yield?)

Exercise 36.2 (Forms and matrices)**.** Write the symmetric matrix for each quadratic form, then state its trace and determinant. (a) $4x^2 + 2xy + y^2$. (b) $x^2 - 4xy + 4y^2$. (c) $3x^2 - 2xy + 3y^2$. Which (if any) are positive definite by the trace–determinant test?

Exercise 36.3 (Bowl, dome, or saddle)**.** Classify the origin for each form using eigenvalue signs. (a) $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$. (b) $\begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix}$. (c) $\begin{bmatrix} -2 & 0 \\ 0 & -5 \end{bmatrix}$. (d) $\begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$ – watch the determinant on this last one and say what shape you get.

Exercise 36.4 (Critical points by hand)**.** For $f(x, y) = x^2 + 4y^2 + 2xy - 6x$, find the gradient, solve for the critical point, build the Hessian, and classify the point. Is it a local or a global extremum, and why?

Exercise 36.5 (A genuine saddle)**.** Show that $f(x, y) = xy$ has a single critical point at the origin and that it is a saddle. Find the two eigenvector directions of the Hessian and describe what f does as you walk along each.

Exercise 36.6 (Least squares as a bowl)**.** Fit a line $y = c + dx$ to the points $(1, 1), (2, 2), (3, 2)$ by minimizing squared error. Write $E(c, d)$ as a quadratic form plus linear terms, set its gradient to zero, and solve the resulting normal equations. Confirm the quadratic part's matrix is positive definite (so the bowl has a unique bottom).

Exercise 36.7 (Sylvester practice)**.** Use leading principal minors to decide whether each is positive definite. (a) $\begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}$. (b) $\begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 5 \end{bmatrix}$. For any that fail, say what shape (an indefinite saddle) you get instead by estimating eigenvalue signs.

Exercise 36.8 (Lagrange tangency)**.** Find the maximum and minimum of $f(x, y) = 2x + 3y$ on the circle $x^2 + y^2 = 1$ using $\nabla f = \lambda \nabla g$. Interpret the answer geometrically (which direction on the circle is "most aligned" with the vector $\begin{bmatrix} 2 \\ 3 \end{bmatrix}$?).

Exercise 36.9 (Rayleigh quotient)**.** For $\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$, compute the eigenvalues, then state the maximum and minimum of $\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x}$ over unit vectors and the unit vectors where each is attained. Verify by evaluating the form at $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Exercise 36.10 (Condition number and the shape of the bowl)**.** The "stretch" of a positive-definite bowl is $\lambda_{\max}/\lambda_{\min}$. For $\mathbf{A} = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$, compute this ratio and sketch the level-curve ellipse of each. Which bowl would gradient descent navigate faster, and why? (This is the feature-scaling story from the machine-learning box.)

Exercise 36.11 (Prove the convexity criterion)**.** Prove that the quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x}$ with symmetric positive-definite $\mathbf{A}$ is strictly convex, directly from the chord definition. (Hint: for $t \in (0, 1)$, expand $f(t\mathbf{x}+(1-t)\mathbf{z})$ and show $tf(\mathbf{x})+(1-t)f(\mathbf{z})-f(t\mathbf{x}+(1-t)\mathbf{z}) = \frac{1}{2}t(1-t)(\mathbf{x}-\mathbf{z})^{\mathsf{T}}\mathbf{A}(\mathbf{x}-\mathbf{z}) > 0$ whenever $\mathbf{x} \neq \mathbf{z}$.)

Exercise 36.12 (Inequality constraints and complementary slackness)**.** Minimize the quadratic form $f(x_1, x_2) = (x_1 - 4)^2 + (x_2 - 4)^2$ subject to the linear inequality constraint $x_1 + 2x_2 \leq 6$. (a) Find the unconstrained minimizer $\mathbf{x}_{\text{unc}}$ and verify that it violates the constraint. (b) Formulate the KKT conditions and determine the Lagrange multiplier μ. (c) Compute the constrained minimizer $\mathbf{x}^\star$ and confirm that the downhill gradient $-\nabla f(\mathbf{x}^\star)$ is balanced by the inward normal reaction force of the wall.

Chapter 37

Connections: Groups, Tensors, and Functional Analysis

In Chapter 36, we used quadratic forms to find the lowest points of energy bowls, completing our tour of finite-dimensional optimization. But linear algebra does not end where the dimension stops being finite, or where matrices act on more than two vectors.

You have come a long way. You started with arrows in the plane and the simple act of adding them tip to tail. You learned to span, to project, to factor, to find the directions a matrix refuses to turn. By now linear algebra probably feels less like a subject and more like a *language* – a way of talking about straightness, about combining things proportionally, about the geometry hiding inside arrays of numbers.

This last chapter is a set of open doors. We are not going to master any of the three subjects in the title; each is a whole field with its own thick books and its own community. Our goal is to examine how each area builds on the foundational concepts developed throughout this book. When you later encounter Lie groups, quantum mechanics, the finite element method, or modern neural networks, you will recognize these advanced frameworks as natural extensions of linear algebra.

Three doors. Behind the first, **groups**: the matrices that preserve length and angle – rotations and reflections – form a structure of their own, and the study of how such structures *act* on vector spaces is called representation theory, the bridge from linear algebra to symmetry. Behind the second, **tensors**: what happens when "linear in one vector" becomes "linear in several vectors at once," the natural home of multilinear maps, of the determinant, and of every modern machine-learning framework. Behind the third, **functional analysis**: what survives when the vectors become *functions* and the dimension becomes infinite, where Fourier series turn out to be nothing more exotic than an orthonormal basis you cannot see all of at once.

Focus

The single idea threading this whole chapter: the patterns of finite-dimensional linear algebra – basis, orthogonality, projection, eigenvalues, the action of a linear map – do not stop at the edge of $\mathbb{R}^n$. They keep going. They organize symmetry (groups), they stack up (tensors), and they survive the leap to infinity (Hilbert spaces). These directions show how the core ideas of linear algebra extend into broader mathematical settings.

37.1 Matrix groups: the geometry that linear algebra preserves

Let us begin where your geometric instincts are strongest: rigid motions of the plane. Pick up the plane $\mathbb{R}^2$ and move it so that no distances change and no angles change – so that the unit square stays a unit square, just possibly turned or flipped. What are you allowed to do? You may *rotate* about the origin, and you may *reflect* across a line through the origin. That is all (if we insist the origin stays put). Every such motion is a linear map, and we have met its matrices before.

A rotation by angle θ has matrix

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

and a reflection across the line at angle $\phi/2$ has matrix

$$\mathbf{F}(\phi) = \begin{bmatrix} \cos\phi & \sin\phi \\ \sin\phi & -\cos\phi \end{bmatrix}.$$

What these matrices have in common is the defining property of *not distorting*: their columns are orthonormal. Recall from the orthogonality chapter that a matrix with orthonormal columns satisfies $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$, equivalently $\mathbf{Q}^{\mathsf{T}} = \mathbf{Q}^{-1}$. Such matrices are called *orthogonal*.

Definition 37.1 (Orthogonal group)**.** The set of all $n \times n$ real matrices with $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$ is called the *orthogonal group* $O(n)$. The subset with $\det \mathbf{Q} = +1$ is the *special orthogonal group* $SO(n)$ – the pure rotations.

Pause

Pause and think. Why does having orthonormal columns automatically force $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$? Write $\mathbf{Q} = [\mathbf{q}_1 \cdots \mathbf{q}_n]$. Row i of $\mathbf{Q}^{\mathsf{T}}$ meets column j of $\mathbf{Q}$ in the dot product $\mathbf{q}_i^{\mathsf{T}}\mathbf{q}_j$. If $i = j$, the column is a unit vector, giving length-squared 1. If $i \neq j$, the distinct columns are perpendicular, giving 0. The product matrix has 1s on the diagonal and 0s everywhere else – exactly the identity matrix $\mathbf{I}$.

Why the word *group*? Because this collection of matrices is closed under exactly the operations that matter for "doing one motion after another."

Intuition

Forget the formal axioms for a moment and think physically. If you rotate the plane and then rotate again, you have rotated – a single motion of the same kind. If you do a rigid motion, you can always *undo* it with another rigid motion. And "do nothing" is itself a (boring) rigid motion. So the rigid motions form a self-contained world: combine two and you stay inside, every one has an inverse inside, and there is a do-nothing element. That self-containedness *is* what "group" means.

Formal

A *group* is a set G equipped with an associative operation for combining two elements (here, matrix multiplication) that is *closed*, meaning the combination of any two elements of G stays in G. In addition, G contains an *identity* element e with $eg = ge = g$ for all g (here $\mathbf{I}$), and

every $g \in G$ has an *inverse* $g^{-1} \in G$ with $gg^{-1} = g^{-1}g = e$ (here $\mathbf{Q}^{-1} = \mathbf{Q}^{\mathsf{T}}$). A group need not be commutative, and $O(n)$ for $n \geq 2$ is not.

History and connection

In the 1870s, Norwegian mathematician Sophus Lie set out to do for differential equations what Évariste Galois had done for polynomial equations: classify their solutions by their underlying continuous symmetries. Rather than relying on isolated integration tricks, Lie studied smooth families of geometric transformations—now called **Lie groups**—and discovered that their curved geometric structure is governed by their tangent directions at the do-nothing identity element. This tangent space is the **Lie algebra** of infinitesimal generators, which records instantaneous velocities of transformation (such as angular velocity during a rotation). Exponentiating a skew-symmetric matrix generator $e^{t\mathbf{X}}$ turns these linear infinitesimal steps into finite, smooth rotations in $SO(n)$, creating an exact bridge between linear algebra and differential equations.

In the late 1920s, Hermann Weyl and Eugene Wigner carried group theory into quantum physics, proving that physical symmetries dictate the existence of quantum spin, angular momentum, and the classification of elementary particles. Yet this abstraction met fierce controversy: prominent physicists initially revolted against what they called the "*Gruppenpest*" (the "group pestilence"), complaining that abstract group representations were needlessly mathematical compared to direct wave calculations. The pestilence won: today the Standard Model of particle physics is formulated entirely in the language of Lie gauge groups, $U(1) \times SU(2) \times SU(3)$.

Meanwhile, David Hilbert in the early 1900s and Stefan Banach in the 1920s pushed linear algebra beyond finite dimensions into complete spaces of functions: **Banach spaces** (vector spaces with lengths, or norms, where every Cauchy sequence converges) and **Hilbert spaces** (which add an inner product to measure angles and perpendicularity). Early mathematicians resisted treating functions as vectors, wondering how an infinite continuum of values could ever behave like a Euclidean column of numbers. Hilbert proved that in the square-integrable function space L^2, the inner product $\int f(x)g(x)\,dx$ gives functions lengths, angles, and orthogonal projections identical to Euclidean space.

When Albert Einstein, Boris Podolsky, and Nathan Rosen challenged quantum mechanics in 1935 with the EPR paradox, dismissing quantum entanglement as "spooky action at a distance," linear algebra provided the exact resolution through **tensor products**. In a composite state space $\mathcal{H}_A \otimes \mathcal{H}_B$, an entangled state vector $\mathbf{w}$ cannot be factored as a simple product $\mathbf{u} \otimes \mathbf{v}$. Non-local correlations are an intrinsic geometric property of tensor product spaces. Today, major questions remain active: proving whether quantum Yang–Mills gauge theory has a positive mass gap is a Clay Millennium Problem, while in multilinear algebra, simply computing the rank of a 3-way tensor is NP-hard.

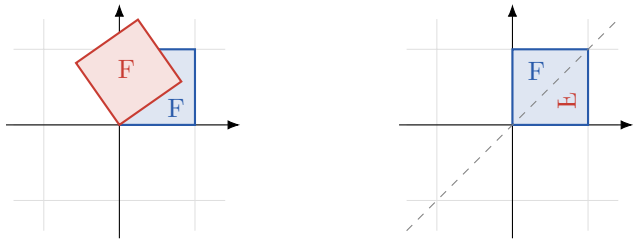
Let us check that $O(n)$ really is a group, because the proof is three lines of linear algebra you can already do, and it shows how the abstract axioms cash out into facts about matrices.

Proposition 37.1 ($O(n)$ is a group)**.** *If* $\mathbf{P}^{\mathsf{T}}\mathbf{P} = \mathbf{I}$ *and* $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$*, then* $\mathbf{PQ}$ *is orthogonal, and* $\mathbf{P}^{-1}$ *is orthogonal.*

Proof. For the product, $(\mathbf{PQ})^{\mathsf{T}}(\mathbf{PQ}) = \mathbf{Q}^{\mathsf{T}}\mathbf{P}^{\mathsf{T}}\mathbf{PQ} = \mathbf{Q}^{\mathsf{T}}\mathbf{IQ} = \mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$. For the inverse, since $\mathbf{P}^{-1} = \mathbf{P}^{\mathsf{T}}$ we have $(\mathbf{P}^{-1})^{\mathsf{T}}\mathbf{P}^{-1} = (\mathbf{P}^{\mathsf{T}})^{\mathsf{T}}\mathbf{P}^{\mathsf{T}} = \mathbf{PP}^{\mathsf{T}} = \mathbf{I}$, where the last equality holds because

a square matrix with $\mathbf{P}^{\mathsf{T}}\mathbf{P} = \mathbf{I}$ also satisfies $\mathbf{P}\mathbf{P}^{\mathsf{T}} = \mathbf{I}$. The identity $\mathbf{I}$ is orthogonal, and matrix multiplication is always associative. All four axioms hold. □

The determinant condition cuts the orthogonal group into two clean halves. An orthogonal matrix has $\det = \pm 1$ (because $1 = \det \mathbf{I} = \det(\mathbf{Q}^{\mathsf{T}}\mathbf{Q}) = (\det \mathbf{Q})^2$). The $+1$ half is the rotations, $SO(n)$. The -1 half is the reflections (and rotation–reflection combinations). The two halves cannot be deformed into one another without passing through a matrix of determinant 0, which is not invertible – so a reflection is geometrically a different *kind* of motion from a rotation. You cannot continuously turn a left hand into a right hand.



$SO(2)$: rotate, $\det = +1$ reflection, $\det = -1$

Figure 37.1: Left: a rotation in $SO(2)$ turns the unit square (and the letter F) without flipping it. Right: a reflection across the dashed line $y = x$ mirrors the F without distortion. Both maps preserve all lengths and angles, making them orthogonal; they live in the two disconnected halves $\det = +1$ and $\det = -1$ of $O(2)$.

37.1.1 $SO(2)$ is a circle, and that circle is the key to everything

Here is the first genuinely new idea. The rotations of the plane are parametrized by a single angle θ, and θ lives on a *circle* (because θ and $\theta + 2\pi$ give the same rotation). So $SO(2)$ is not just a set of matrices – it has a *shape*, and that shape is a circle. A group that is also a smooth geometric space (a *manifold* – a space that locally looks like flat Euclidean space, allowing calculus to be done on it) is called a *Lie group*, and from here the subject opens out into a vast territory of mathematics and physics. Figure 37.2 draws the identification: each rotation matrix $\mathbf{R}(\theta)$ is one point on the unit circle, and composing rotations slides you around it by adding angles.

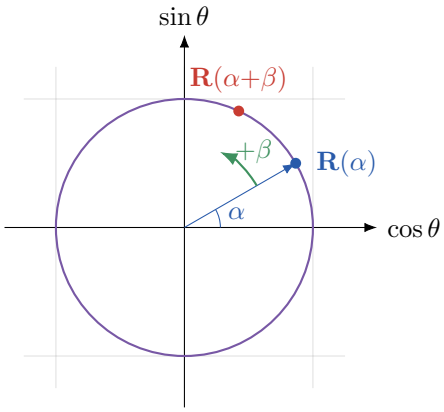


Figure 37.2: $SO(2)$ *is* a circle. Each rotation $\mathbf{R}(\theta)$ corresponds to the point $(\cos\theta, \sin\theta)$, and the group law $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha+\beta)$ is just sliding along the circle by the added angle β. "Do nothing" sits at angle 0; the inverse of a point is its mirror across the horizontal axis.

Watch how the group law becomes the addition of angles. Multiply two rotations:

$$\mathbf{R}(\alpha)\mathbf{R}(\beta) = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & -\sin\beta \\ \sin\beta & \cos\beta \end{bmatrix} = \begin{bmatrix} \cos(\alpha+\beta) & -\sin(\alpha+\beta) \\ \sin(\alpha+\beta) & \cos(\alpha+\beta) \end{bmatrix} = \mathbf{R}(\alpha+\beta).$$

Look closely: working out that product *is* a derivation of the angle-addition formulas for sine and cosine. The top-left entry is $\cos\alpha\cos\beta - \sin\alpha\sin\beta$, which equals $\cos(\alpha+\beta)$. Trigonometry's hardest identities fall out of the statement "rotating by α then by β is the same as rotating by $\alpha+\beta$." The group law and the angle-addition identity express the same algebraic relationship.

Connection to other math

This is why unit complex numbers and planar rotations have the exact same algebra. Identify the matrix $\mathbf{R}(\theta)$ with the complex number $e^{i\theta} = \cos\theta + i\sin\theta$, sitting on the unit circle in $\mathbb{C}$. Then $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha+\beta)$ becomes $e^{i\alpha}e^{i\beta} = e^{i(\alpha+\beta)}$, the ordinary rule for multiplying exponentials. The circle group $SO(2)$, the unit complex numbers $\{e^{i\theta}\}$, and the angles modulo 2π are three faces of one object. When you later meet "$U(1)$ gauge symmetry" in physics, it is this circle, and nothing more frightening.

In three dimensions the story gets richer and more physical. $SO(3)$, the rotations of space, is a three-dimensional Lie group (you need three numbers to specify a rotation: an axis, which costs two numbers, and an angle of turn, which costs one). Euler's theorem – itself pure linear algebra – says every element of $SO(3)$ is a rotation about *some* axis. We can prove this directly from the eigenvalues of $\mathbf{R}$.

Proposition 37.2 (Every rotation of space has an axis)**.** *Every $\mathbf{R} \in SO(3)$ has $+1$ as an eigenvalue. The eigenvector is the axis of rotation – the line $\mathbf{R}$ leaves fixed.*

Proof. The eigenvalues of $\mathbf{R}$ are the roots of its characteristic polynomial $\det(\mathbf{R} - \lambda\mathbf{I})$, a real cubic, so at least one root is real (complex roots of a real polynomial come in conjugate pairs). Being orthogonal forces every eigenvalue to have modulus 1 (an orthogonal map preserves length, so $\|\mathbf{Rx}\| = \|\mathbf{x}\|$ forces $|\lambda| = 1$ on any eigenvector), so the real eigenvalues are ± 1. Their product is $\det\mathbf{R} = +1$. The three eigenvalues are therefore either $\{1, 1, 1\}$, or $\{1, -1, -1\}$, or $\{1, e^{i\theta}, e^{-i\theta}\}$ – in every case a $+1$ appears. The vector $\mathbf{x}$ with $\mathbf{Rx} = \mathbf{x}$ is a fixed direction, i.e. the axis. □

This is why a spinning top has an axis, why the Earth has a North Pole, and why any sequence of 3D rotations you compose is still just *one* rotation about *one* axis. Figure 37.3 shows the picture: the $+1$-eigenvector is the fixed line, and the plane perpendicular to it spins by some angle θ.

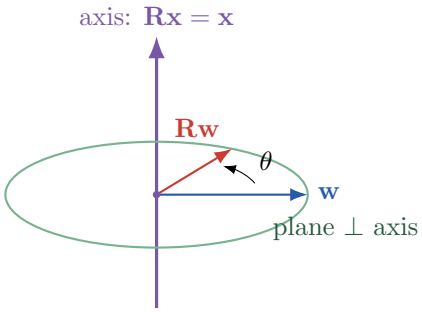


Figure 37.3: Euler's theorem geometrically. Every $\mathbf{R} \in SO(3)$ fixes a line (its $+1$-eigenvector, the purple axis) and rotates the perpendicular plane by an angle θ; the complex eigenvalues $e^{\pm i\theta}$ record that in-plane spin. A composition of rotations is again one such axis-plus-angle.

Modern application

Rotations in graphics, robotics, and aerospace. Every time a game engine turns your character's head, or a drone holds level, or a satellite points its camera, a member of $SO(3)$ is being computed. Because composing rotations as 3×3 matrices accumulates numerical rounding error (the matrix slowly stops being strictly orthogonal), practitioners often store rotations as *unit quaternions* – four-component numbers $q = w + xi + yj + zk$ with $\|q\| = 1$. Unit quaternions avoid "gimbal lock" (a physical trap where two rotation axes align and lose a degree of freedom) and form a *double cover* of $SO(3)$, meaning that the two antipodal quaternions q and $-q$ correspond to the exact same spatial rotation. They renormalize with a single vector scaling and interpolate smoothly along the shortest arc on the four-dimensional sphere (an algorithm known as *slerp*, for spherical linear interpolation). That quaternions are the ideal representation is a geometric theorem about the topology of $SO(3)$, discovered by studying its symmetry group.

37.1.2 The complex cousin: unitary matrices

Everything above has a complex twin. Replace "preserves length in $\mathbb{R}^n$" with "preserves length in $\mathbb{C}^n$," where length uses the Hermitian inner product $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_k \overline{u_k} v_k$. The condition $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$ becomes $\mathbf{U}^*\mathbf{U} = \mathbf{I}$, where $\mathbf{U}^*$ is the conjugate transpose. Such matrices are *unitary*, and they form the *unitary group* $U(n)$.

Focus

Orthogonal is to real space as unitary is to complex space. $O(n)$ preserves $\|\mathbf{x}\|^2 = \mathbf{x}^{\mathsf{T}}\mathbf{x}$; $U(n)$ preserves $\|\mathbf{x}\|^2 = \mathbf{x}^*\mathbf{x}$. Both are the "rigid motions" of their space – the matrices that never stretch, shrink, or shear.

The unitary group is the mathematical home of quantum mechanics: the state of a quantum system is a unit vector in $\mathbb{C}^n$, and the laws of physics evolve it by unitary matrices, precisely because unitary

matrices are the ones that keep the total probability (the squared length) equal to 1. When you read that "time evolution is unitary," it means "the dynamics is a rigid rotation of the complex state space." The Fourier matrix from the complex chapter is unitary (after scaling by $1/\sqrt{n}$); so is every quantum logic gate.

37.1.3 Representation theory: letting a group act

We have been treating groups as collections of matrices. But many groups arise abstractly – the symmetries of a molecule, the permutations of a deck of cards, the rotations that leave a crystal looking the same – with no matrices in sight. *Representation theory* is the art of giving such a group a set of matrices to *be*, so that the full toolbox of linear algebra can be brought to bear on it.

Definition 37.2 (Representation)**.** A *representation* of a group G on a vector space V is a rule ρ that assigns to each group element g an invertible matrix $\rho(g)$ acting on V, in a way that respects the group law: $\rho(gh) = \rho(g)\,\rho(h)$.

The slogan is: *a representation turns multiplication in the group into matrix multiplication.* In plain terms, if you have an abstract symmetry operation, a representation gives it a concrete matrix so it can actually transform vectors. Once you have that, you can ask all your favorite linear-algebra questions about the symmetry. Which directions does every symmetry leave alone (the *invariant subspaces*)? Can we change basis so that every $\rho(g)$ becomes block-diagonal at once, splitting the space into atomic, indivisible building blocks (the *irreducible representations* – representations that contain no smaller invariant subspaces)? This last question – simultaneously block-diagonalizing a whole family of matrices – is the deep generalization of diagonalizing a single matrix.

Example 37.1 (The smallest interesting representation)**.** Take the group with two elements: "do nothing" (e) and "flip" (f), where $ff = e$. Represent it on $\mathbb{R}^2$ by

$$\rho(e) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \rho(f) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Check the group law: $\rho(f)\rho(f) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \rho(e)$. Good. Now ask the representation-theory question: change basis to make *both* matrices diagonal at once. The flip $\rho(f)$ has eigenvectors $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (eigenvalue $+1$) and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ (eigenvalue -1); the identity is already diagonal in any basis. In the basis $\{\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix}\}$ *both* group elements are diagonal:

$$\rho(e) \rightsquigarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \rho(f) \rightsquigarrow \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

The two-dimensional representation has split into two one-dimensional pieces: the *symmetric* line $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$, on which the flip does nothing, and the *antisymmetric* line $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$, on which the flip negates. Those are the two irreducible representations of the flip group. Every even/odd decomposition you have ever seen – even and odd functions, symmetric and antisymmetric tensors – is this same split. Figure 37.4 shows the two invariant lines and the coordinate-swap geometry behind them.

Pause

Pause and think. Look at how any vector decomposes along these two invariant lines. Take $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. We can split it into symmetric and antisymmetric halves:

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \underbrace{\frac{x_1 + x_2}{2}\begin{bmatrix} 1 \\ 1 \end{bmatrix}}_{\text{symmetric } (+1 \text{ eigenvector})} + \underbrace{\frac{x_1 - x_2}{2}\begin{bmatrix} 1 \\ -1 \end{bmatrix}}_{\text{antisymmetric } (-1 \text{ eigenvector})}.$$

Applying the flip swaps coordinates, which leaves the symmetric piece untouched and negates the antisymmetric piece. The group symmetry has decoupled $\mathbb{R}^2$ into two independent, uncoupled one-dimensional subspaces.

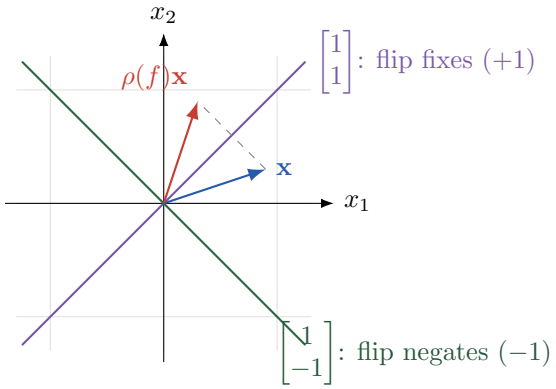


Figure 37.4: The flip $\rho(f) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ swaps coordinates, which is reflection across the symmetric line $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. That line and the antisymmetric line $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ are the invariant subspaces: in this rotated basis *both* group matrices are diagonal at once. Simultaneously diagonalizing the whole family is the representation-theory analogue of diagonalizing a single matrix.

Why this matters

Representation theory is one of the most beautiful unifications in mathematics because it explains *why* eigenvectors and "special bases" keep appearing in physics. A physical system with a symmetry can always be organized into the irreducible pieces of that symmetry, and those pieces are forced to be the natural coordinates. The energy levels of the hydrogen atom, the multiplets of particle physics, the selection rules of spectroscopy, the normal modes of a symmetric molecule – all are the irreducible representations of the system's symmetry group, found by simultaneously diagonalizing a family of matrices. Diagonalization, which you learned to do for one matrix, becomes the engine that classifies the structure of the physical world.

Before/after Strang

Strang's course gives you orthogonal matrices, $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$, and the spectral theorem for symmetric matrices, but it (rightly, for a first course) stops at single matrices. The leap here is conceptual: collect the orthogonal matrices into the *group* $O(n)$, and ask not "diagonalize this one matrix" but "find the common structure of this whole family of matrices." That reframing – from one operator to a symmetry group of operators – is the doorway from a linear-algebra course into modern geometry and physics.

37.2 Tensors: linearity in several slots at once

So far a linear map takes *one* vector as input and acts linearly on it: $\mathbf{A}(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha\mathbf{A}\mathbf{x} + \beta\mathbf{A}\mathbf{y}$. What if a map takes *two* vectors at once and is linear in *each* slot separately? That is the idea of a tensor, and you have been using one since the dot-product chapter without naming it.

37.2.1 The dot product is a bilinear form

Consider the map that takes two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and returns the number $\mathbf{x}^\mathsf{T}\mathbf{y}$. Hold $\mathbf{y}$ fixed and it is linear in $\mathbf{x}$; hold $\mathbf{x}$ fixed and it is linear in $\mathbf{y}$. Linear in each slot – that is *bilinear*. More generally a bilinear form is

$$B(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\mathsf{T}\mathbf{M}\mathbf{y} = \sum_{i,j} M_{ij}\, x_i\, y_j,$$

and the matrix $\mathbf{M}$ *is* the tensor: a grid of numbers M_{ij}, one for each pair (row-slot, column-slot). The quadratic forms from the optimization chapter are the special case $\mathbf{y} = \mathbf{x}$.

Intuition

A vector is a list of numbers indexed by *one* index: v_i. A matrix is a grid indexed by *two*: M_{ij}. A tensor just keeps going – a box of numbers indexed by three, T_{ijk}, or more. The number of indices is the *order* (or "rank," though that word is overloaded). Vectors are order-1 tensors, matrices are order-2 tensors, and the determinant of a 3×3 matrix is computed by an order-3 tensor: the Levi-Civita symbol ε_{ijk}, a bookkeeping device that equals $+1$ for even permutations of $(1, 2, 3)$, -1 for odd permutations, and 0 whenever any index repeats. The natural generalization is to allow any number of indices.

The key property that makes tensors more than "arrays" is how they transform when you change basis. A genuine tensor carries one transformation rule per index, so that the multilinear map it represents stays the same object even as its coordinates change. This is the precise sense behind the slogan that "a tensor is a thing that transforms like a tensor" – a phrasing that sounds circular only until you have seen the change-of-basis chapter. Once you have, it is exact: a tensor is a multilinear map, and its array of components depends on the chosen basis in the same way a matrix's entries do.

37.2.2 The tensor product: building bigger spaces by multiplication

Here is the constructive heart of the subject. Given two vectors $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^n$, form their *outer product*, the $m \times n$ matrix

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\,\mathbf{v}^{\mathsf{T}}, \qquad (\mathbf{u} \otimes \mathbf{v})_{ij} = u_i v_j.$$

You met this in the SVD chapter: a rank-one matrix is exactly an outer product. The collection of all such products, and all their sums, is the *tensor product space* $\mathbb{R}^m \otimes \mathbb{R}^n$, which can be identified with the space of $m \times n$ matrices. Its dimension is $m \cdot n$ – dimensions *multiply* under $\otimes$, where they would *add* under the direct sum $\oplus$.

Compute it yourself

Compute one outer product by hand and watch the rank-one structure appear. With $\mathbf{u} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 3 \\ 1 \\ -1 \end{bmatrix}$,

$$\mathbf{u} \otimes \mathbf{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 1 & -1 \\ 6 & 2 & -2 \end{bmatrix}.$$

Notice every row is a multiple of $\begin{bmatrix} 3 & 1 & -1 \end{bmatrix}$ and every column is a multiple of $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. That is what rank one *means*, and it is the simplest possible nonzero tensor. The SVD decomposes any matrix into a linear combination of rank-one tensor products: $\mathbf{A} = \sum_k \sigma_k\, \mathbf{u}_k \otimes \mathbf{v}_k$.

Pause

Pause and think. Why is any nonzero outer product $\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^{\mathsf{T}}$ automatically rank 1? Look at column j: it is simply $v_j \mathbf{u}$. Every single column is a scalar multiple of $\mathbf{u}$, so all columns lie on the one-dimensional line $\mathrm{span}\{\mathbf{u}\}$. The column space has dimension 1.

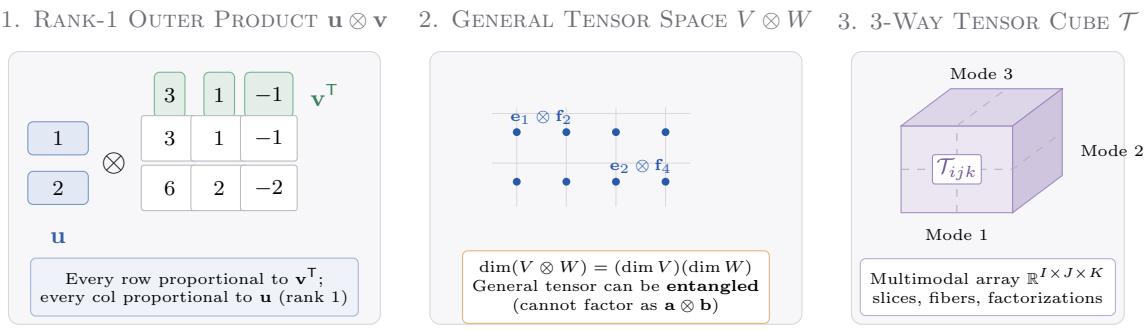


Figure 37.5: The geometry of the tensor product. Left: A rank-one tensor is the outer product $\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^{\mathsf{T}}$, where entries are pairwise products and every column is proportional to $\mathbf{u}$. Middle: The full tensor space $V \otimes W$ is the linear span of all such outer products, with dimension multiplying: $\dim(V \otimes W) = (\dim V)(\dim W)$. High-rank tensors are *entangled* (cannot be factored as a single outer product). Right: Higher-order tensors form multi-way arrays $\mathcal{T} \in \mathbb{R}^{I \times J \times K}$ indexed along multiple modes, analyzed via tensor factorizations (such as CP and Tucker decompositions, which generalize the SVD to multi-index data).

37.2.3 The determinant is an alternating multilinear form

You have already met one of the most important tensors in mathematics and not realized its pedigree: the determinant. Think of det as a map that takes the n *columns* of a matrix, $\mathbf{a}_1, \ldots, \mathbf{a}_n$, and returns a number. From the determinant chapter you know two of its properties:

- it is linear in each column separately (scale a column, the determinant scales; add columns, the determinant adds) – so det is *multilinear*;
- swapping two columns flips the sign – so det is *alternating*.

A multilinear, alternating function of n vectors in $\mathbb{R}^n$ is unique up to scale, and fixing $\det \mathbf{I} = 1$ pins it down completely. That single sentence is the modern definition of the determinant, and it explains the signed-volume picture geometrically: the only consistent way to assign a signed volume to a parallelepiped that is additive in each edge and flips sign when you swap two edges is the determinant. The whole theory of Λ^n, the "exterior algebra" (the algebra of oriented lengths, areas, and volumes) that underlies differential forms in multivariable calculus, grows from this one alternating tensor.

Connection to other math

Tensors in calculus and relativity. The gradient is a tensor (it tells you how a scalar changes per direction); the Jacobian is a tensor (linear-in-each-slot change of variables); the metric tensor g_{ij} of a curved space is a bilinear form $\langle \mathbf{u}, \mathbf{v} \rangle_g = \sum g_{ij} u^i v^j$ that measures length when the usual dot product no longer applies. Einstein's general relativity is written entirely in tensors because the laws of physics must be the same in every coordinate system, and a tensor is precisely an object whose meaning survives a change of basis. Differential forms, the objects you integrate in multivariable calculus, are the alternating tensors of this section.

Modern application

Why deep learning runs on tensors. The libraries powering modern machine learning – PyTorch, TensorFlow (the name is a giveaway), JAX – have one central data type: the tensor, a multi-index array. A batch of B color images of size $H \times W$ is a four-index tensor $X_{b,c,h,w}$ (batch, channel, height, width). A neural network is a long composition of multilinear operations (matrix multiplies, convolutions, which are tensor contractions) interleaved with simple nonlinearities. *Tensor contraction* – summing over a shared index, exactly like matrix multiplication's $\sum_k A_{ik} B_{kj}$ – is the central computational operation, and the entire efficiency of GPUs comes from doing billions of these contractions in parallel. When you hear "GPT has a trillion parameters," those parameters live in tensors, and training adjusts them by gradients that are themselves computed by the chain rule applied to tensor contractions. The order-2 special case you already know, $\mathbf{AB}$, is the seed of all of it.

Common trap

"Tensor" in a machine-learning library means only *multidimensional array* – a box of numbers with a shape. The mathematician's tensor carries the extra promise of a change-of-basis transformation law. A PyTorch tensor of shape $(3, 3)$ is not automatically a geometric tensor in the relativity sense; it is just storage. Both uses are legitimate and they share an ancestor,

but do not assume a coding "tensor" obeys the transformation rules until someone has told you what its indices *mean*. The data type is more permissive than the mathematical object.

37.3 Functional analysis: linear algebra in infinitely many dimensions

Every space in this book has been $\mathbb{R}^n$ or $\mathbb{C}^n$ for some finite n. But the abstract definition of a vector space – a set where you can add and scale, obeying the usual rules – never mentioned a dimension. What happens if the vectors are *functions*, and the space is infinite-dimensional? Almost everything you know survives, with one subtle and important repair.

37.3.1 Functions are vectors

Consider all continuous functions on the interval $[0, 1]$. You can add two of them pointwise, $(f + g)(x) = f(x) + g(x)$, and scale one, $(cf)(x) = cf(x)$, and every vector space axiom holds. So this set of functions *is* a vector space. But what is its dimension? There is no finite list of functions whose combinations give every continuous function. The space is infinite-dimensional.

Intuition

Think of a function on $[0, 1]$ as a vector with infinitely many components – one component $f(x)$ for each point x. A vector in $\mathbb{R}^3$ has three slots; a function has a continuum of slots. The dot product of two vectors, $\sum_i u_i v_i$, becomes a *sum over all the slots*, which is an *integral*:

$$\langle f,\, g\rangle = \int_0^1 f(x)\, g(x)\, dx.$$

Read the integral sign as "add up the products component by component, where the components are indexed by x instead of by i." The inner product keeps the same algebraic rules, with an integral in place of a finite sum.

With an inner product comes *everything geometric*. The length of a function is $\|f\| = \sqrt{\langle f,\, f\rangle} = \sqrt{\int_0^1 f^2}$. Two functions are *orthogonal* when $\int_0^1 fg = 0$. The angle between functions, the projection of one function onto another, the Cauchy–Schwarz inequality – all of it carries over verbatim, because every one of those facts was proved from the inner product axioms alone, never from the dimension being finite.

Example 37.2 (Two orthogonal functions)**.** On $[0, 2\pi]$ with $\langle f,\, g\rangle = \int_0^{2\pi} fg\, dx$, are $\sin x$ and $\cos x$ orthogonal? Compute:

$$\langle \sin,\, \cos\rangle = \int_0^{2\pi} \sin x \cos x\, dx = \tfrac{1}{2}\int_0^{2\pi} \sin 2x\, dx = \tfrac{1}{2}\Big[-\tfrac{1}{2}\cos 2x\Big]_0^{2\pi} = 0.$$

Yes – sin and cos are perpendicular vectors in function space. More is true: the functions $\sin x$, $\cos x$, $\sin 2x$, $\cos 2x$, $\sin 3x$, and so on are *all* mutually orthogonal. You can check, for integers $m \neq n$, that $\int_0^{2\pi} \sin(mx)\sin(nx)\, dx = 0$. We are staring at an infinite orthogonal set – an orthogonal *basis* for a space we cannot draw.

Pause

Pause and think. What is the visual picture of $\int_0^{2\pi} \sin x \cos x \, dx = 0$? The product function $(\sin x)(\cos x) = \frac{1}{2}\sin 2x$ oscillates above and below the horizontal axis. Over one full cycle $[0, 2\pi]$, the positive area on $[0, \pi/2]$ and $[\pi, 3\pi/2]$ is perfectly balanced by the negative area on $[\pi/2, \pi]$ and $[3\pi/2, 2\pi]$. Just as two vectors in $\mathbb{R}^2$ are orthogonal when their positive and negative coordinate products cancel ($u_1v_1 + u_2v_2 = 0$), two functions are orthogonal when their continuous product cancels across the domain.

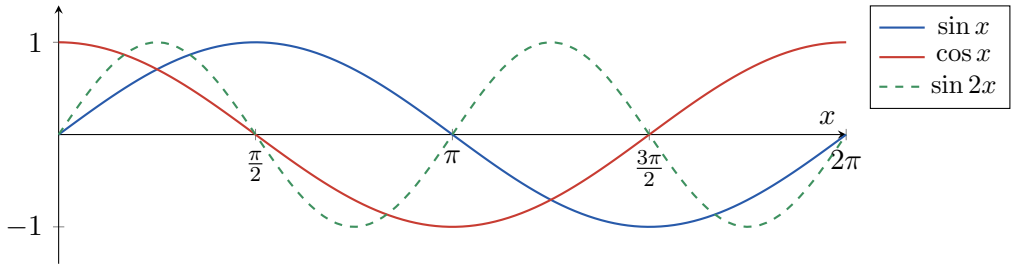


Figure 37.6: Three members of an infinite orthogonal family. Under the inner product $\int_0^{2\pi} fg \, dx$, any two distinct curves here are perpendicular – their products integrate to zero. This is the basis behind Fourier series.

37.3.2 Fourier series: expanding in the orthonormal basis you cannot see

In $\mathbb{R}^n$, once you have an orthonormal basis $\mathbf{q}_1, \ldots, \mathbf{q}_n$, you expand any vector by projecting onto each basis vector and adding up:

$$\mathbf{v} = \sum_k \langle \mathbf{q}_k, \mathbf{v} \rangle \, \mathbf{q}_k.$$

Each coefficient $\langle \mathbf{q}_k, \mathbf{v} \rangle$ is "how much of $\mathbf{v}$ points along $\mathbf{q}_k$." That expansion relies only on orthogonality, whether the sum has three terms or infinitely many. Apply it to function space with the trig basis, and out drops the *Fourier series*:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} \Big(a_k \cos kx + b_k \sin kx \Big), \qquad a_k = \frac{1}{\pi} \int_0^{2\pi} f(x) \cos kx \, dx, \quad b_k = \frac{1}{\pi} \int_0^{2\pi} f(x) \sin kx \, dx.$$

Focus

Those integral formulas for a_k and b_k are standard orthogonal projections. They follow directly from the projection formula $\frac{\langle \mathbf{q}_k, \mathbf{v} \rangle}{\|\mathbf{q}_k\|^2} \mathbf{q}_k$ from the orthogonality chapter. Why is there a $1/\pi$ in front of the integrals, and why is the constant term written as $a_0/2$? Because our trigonometric basis functions are orthogonal, but not normalized to unit length:

$$\|\cos kx\|^2 = \int_0^{2\pi} \cos^2(kx) \, dx = \pi, \qquad \|\sin kx\|^2 = \int_0^{2\pi} \sin^2(kx) \, dx = \pi$$

for every $k \geq 1$. Dividing by $\|\mathbf{q}_k\|^2 = \pi$ produces the factor $1/\pi$. For the constant basis function $q_0(x) = 1$, its squared length is $\|1\|^2 = \int_0^{2\pi} 1^2 \, dx = 2\pi$, so its projection is $\frac{1}{2\pi} \int_0^{2\pi} f(x) \, dx = \frac{a_0}{2}$. A Fourier coefficient is literally the projection of a function onto an

orthogonal basis direction. Fourier analysis *is* the "expand in an orthogonal basis" theorem, run in infinite dimensions.

Let us see one coefficient computed, so the abstraction touches the ground.

Example 37.3 (A Fourier coefficient as a projection)**.** Take the simple function $f(x) = x$ on $[0, 2\pi]$ and project it onto $\sin x$. The coefficient is

$$b_1 = \frac{1}{\pi}\int_0^{2\pi} x \sin x \, dx.$$

Integrate by parts with $u = x$, $dv = \sin x \, dx$, so $du = dx$, $v = -\cos x$:

$$\int_0^{2\pi} x \sin x \, dx = \big[-x\cos x\big]_0^{2\pi} + \int_0^{2\pi} \cos x \, dx = -2\pi\cos 2\pi + \big[\sin x\big]_0^{2\pi} = -2\pi + 0 = -2\pi.$$

Therefore $b_1 = \frac{1}{\pi}(-2\pi) = -2$. The function $f(x) = x$ has component -2 along $\sin x$, directly analogous to how $\begin{bmatrix}3\\4\end{bmatrix}$ has component 3 along $\mathbf{e}_1$. Same operation, infinite-dimensional setting.

Before/after Strang

Strang devotes a beautiful section to the Fourier matrix and the Discrete Fourier Transform – the *finite*, n-point version, where the orthonormal basis is the columns of a unitary matrix and the "integral" is a plain finite sum. The continuous Fourier series of this section is what the DFT becomes as the number of sample points goes to infinity. Read both together and the message is unmistakable: the DFT is a change of basis to an orthonormal basis (Strang's emphasis), and the Fourier series is the same change of basis when the vectors are functions.

37.3.3 The repair: completeness and Hilbert space

Infinite dimensions introduce one important subtlety that never arises in $\mathbb{R}^n$. In $\mathbb{R}^n$, every *Cauchy sequence* (a sequence whose terms bunch closer and closer together, so $\|\mathbf{v}_m - \mathbf{v}_n\| \to 0$ as $m, n \to \infty$) automatically converges to a limit that lives safely inside $\mathbb{R}^n$. In finite dimensions, you can never escape the space simply by taking limits.

In infinite dimensions, this safety net snaps. You can easily build a sequence of smooth, continuous functions that get closer and closer to one another under the L^2 inner-product distance, yet whose limit develops a vertical cliff – a jump discontinuity. Figure 37.7 shows how this happens: the ramp steepens, the difference area between successive approximations vanishes, yet the limit function is a discontinuous step function $\operatorname{sgn}(x)$, which has been expelled from the space of continuous functions $C[-1, 1]$. A space where this defect never happens – where every Cauchy sequence has its limit safely inside – is called *complete*. An infinite-dimensional inner-product space that is complete is a *Hilbert space*.

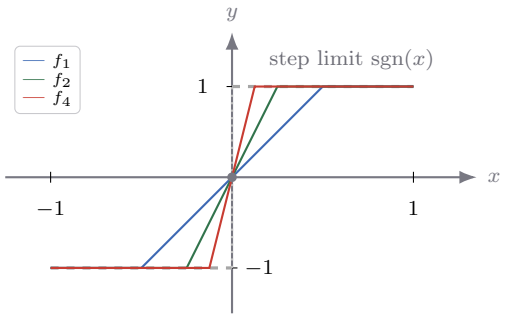


Figure 37.7: Why the space of continuous functions $C[-1,1]$ fails to be complete. The continuous functions $f_n(x)$ ramp linearly from -1 to $+1$ over the narrowing window $[-1/n, 1/n]$. As $n \to \infty$, the difference between f_m and f_n has shrinking area, so (f_n) is a Cauchy sequence in the L^2 norm. Yet its limit is the discontinuous step function $\operatorname{sgn}(x)$, which is not continuous. A Hilbert space repairs this defect by including all such limits.

Formal

A *Hilbert space* is a vector space H with an inner product $\langle \cdot, \cdot \rangle$ that is *complete*: every sequence $(\mathbf{v}_n)$ with $\|\mathbf{v}_m - \mathbf{v}_n\| \to 0$ as $m, n \to \infty$ converges to some limit *in* H. Finite-dimensional inner-product spaces ($\mathbb{R}^n$, $\mathbb{C}^n$) are automatically Hilbert spaces. The fundamental infinite-dimensional example is $L^2[a,b]$, the square-integrable functions with $\langle f, g \rangle = \int_a^b \overline{f} g \, dx$.

Why does completeness matter? Because it is exactly what makes the infinite Fourier sum *mean something*. Each partial sum of a Fourier series is a function in our space; completeness guarantees the limit is too, so "f equals its Fourier series" is a legitimate equation (in the L^2 sense of length). Completeness is the essential property required for infinite series to have well-defined limits in the space, which is why Hilbert spaces form the standard setting for functional analysis. Inside a Hilbert space, the entire geometric apparatus of this book – bases, projections onto subspaces, the Pythagorean theorem, even a version of the spectral theorem for the right operators – holds exactly as you would hope.

Why this matters

The geometric framework built from vectors in $\mathbb{R}^n$ – length, angle, perpendicularity, projection, and orthonormal expansion – extends naturally to infinite-dimensional spaces. Quantum mechanics is formulated in a Hilbert space, where observables are self-adjoint operators, states are unit vectors, and measurable quantities are eigenvalues. Similarly, the heat and wave equations are solved by expanding in an orthonormal basis of eigenfunctions, decoupling the system across modes. The projection formula applies equally to finite-dimensional vectors and continuous functions, serving as a unifying foundation for signal processing, quantum physics, and differential equations.

Connection to other math

Operators as infinite matrices. In a Hilbert space, the role of a matrix is played by a *linear operator* on functions – differentiation $\frac{d}{dx}$, integration, multiplication by x. Many of these have eigenfunctions and eigenvalues. The operator $-\frac{d^2}{dx^2}$ on $[0, \pi]$ (with the function vanishing at

the ends) has eigenfunctions $\sin(kx)$ with eigenvalues k^2 – a "diagonalization" with infinitely many eigenpairs. Solving a differential equation by Fourier series *is* eigenvector decomposition for this operator. The spectral theorem for symmetric matrices, $\mathbf{A} = \sum \lambda_k \mathbf{q}_k \mathbf{q}_k^\mathsf{T}$, has a Hilbert-space sibling that powers the entire theory of partial differential equations.

37.4 Looking back, looking out

Stand back and look at the three doors together, because they are really one doorway seen from three angles.

- **Groups** took the matrices that preserve geometry and asked about *their* structure – how they combine, what symmetries they encode, and how a symmetry can *act* on a space. A central theme was simultaneous diagonalization: the irreducible representations correspond to the simultaneous eigen-decomposition of a symmetry group.
- **Tensors** took linearity-in-one-slot and let it happen in several slots at once. Outer products, the determinant as an alternating form, and the multi-index arrays of machine learning are all the same idea: stack the bilinear structure you already had, index by index.
- **Functional analysis** took the dimension to infinity and discovered that geometry – orthogonality, projection, bases – not only survives but becomes the natural language of Fourier analysis, differential equations, and quantum mechanics, once you pay the price of completeness.

Walking question

Take a walk and try to find, in your own life or studies, one example behind each door. A symmetry of an object you can pick up (a group). A quantity that depends linearly on two separate things (a tensor). A signal – music, a heartbeat, a spoken word – that is naturally a sum of pure tones (a Fourier expansion in a function space). When you can name one of each from memory, the abstract chapter above has become yours.

None of this replaces the careful study these subjects deserve; each door opens onto a long hallway.

Focus

The conceptual arc of the book: vectors and span define a *space*; the dot product defines *geometry*; matrices represent *linear maps*; eigenvalues and the SVD identify *optimal coordinates*. This chapter showed these principles extending to groups, tensors, and infinite dimensions.

Boundaries in multilinear and infinite-dimensional spaces

Venturing into these wider territories does come with honest boundaries. In tensor spaces, dimensions multiply exponentially ($\dim V^{\otimes k} = d^k$), so tracking just 50 interacting quantum particles requires 2^{50} amplitudes—exceeding the memory of supercomputers and forcing physicists to invent tensor network approximations. In infinite-dimensional Hilbert spaces, closed unit spheres are no longer compact, operators can have continuous spectra without a single discrete eigenvector (like position

and momentum in quantum physics), and differential operators like $\frac{d}{dx}$ are unbounded, requiring careful topological care with dense domains and Sobolev spaces.

For reference, two companion appendices provide background: **Chapter 38** reviews sets, mappings, and field axioms, and **Chapter 39** reviews summation notation, complex arithmetic, and matrix calculus.

Exercises

Exercise 37.1 (Mental check: Determinants of orthogonal matrices)**.** Without writing down any coordinates, use the identity $\mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I}$ and the determinant product rule $\det(\mathbf{AB}) = (\det \mathbf{A})(\det \mathbf{B})$ to show in one line that every orthogonal matrix must have $\det \mathbf{Q} = \pm 1$. Can an orthogonal matrix have $\det \mathbf{Q} = 2$? Why geometrically can a rigid motion never scale volumes?

Exercise 37.2 (Closure of $SO(2)$ by computation)**.** Multiply $\mathbf{R}(30^\circ)$ and $\mathbf{R}(45^\circ)$ as 2×2 matrices and verify directly that the product equals $\mathbf{R}(75^\circ)$. Use your computation to state the angle-addition formula for $\cos(\alpha + \beta)$ that appears in the top-left entry.

Exercise 37.3 (Reflections are orthogonal but not rotations)**.** Let $\mathbf{F} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. Verify $\mathbf{F}^{\mathsf{T}}\mathbf{F} = \mathbf{I}$ (so $\mathbf{F} \in O(2)$) and compute $\det \mathbf{F}$. To which half of $O(2)$ does $\mathbf{F}$ belong? Find its eigenvalues and eigenvectors and describe the reflection geometrically (across which line?).

Exercise 37.4 (Euler's axis in action)**.** Consider the rotation of $\mathbb{R}^3$ that cycles the axes, $\mathbf{R}\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} z \\ x \\ y \end{bmatrix}$, with matrix $\mathbf{R} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$. Verify $\mathbf{R} \in SO(3)$ (orthogonal, determinant $+1$). Then find a nonzero vector with $\mathbf{Rx} = \mathbf{x}$ – the axis guaranteed by Euler's theorem.

Exercise 37.5 (A two-element representation)**.** The cyclic group of order 2 can be represented on $\mathbb{R}^2$ by sending the nontrivial element to $\mathbf{A} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$. Check that $\mathbf{A}^2 = \mathbf{I}$ (so the group law is respected). Is this representation already split into one-dimensional invariant pieces? Identify the two invariant lines and say what the group element does on each.

Exercise 37.6 (Outer products and rank)**.** Let $\mathbf{u} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$. Compute the outer product $\mathbf{u}\,\mathbf{v}^{\mathsf{T}}$ as a 3×2 matrix. Without further computation, state its rank and explain why every nonzero 3×2 outer product has that same rank.

Exercise 37.7 (The determinant is alternating)**.** Using only multilinearity and the alternating property (sign flips when two columns are swapped), prove that a 2×2 matrix with two equal columns has determinant 0. Then explain in one sentence why this matches the geometric meaning of determinant as signed area.

Exercise 37.8 (Orthogonality of functions)**.** On $[0, 2\pi]$ with $\langle f, g \rangle = \int_0^{2\pi} f g \, dx$, verify that $\sin x$ and $\sin 2x$ are orthogonal. (Hint: use the product-to-sum identity $\sin A \sin B = \frac{1}{2}[\cos(A - B) - \cos(A + B)]$.) What is $\|\sin x\|^2 = \int_0^{2\pi} \sin^2 x \, dx$?

Exercise 37.9 (A Fourier coefficient)**.** For $f(x) = x$ on $[0, 2\pi]$, compute the coefficient $a_1 = \frac{1}{\pi}\int_0^{2\pi} x\cos x\,dx$ by integration by parts. Combined with the worked example $b_1 = -2$, write the first sine-and-cosine terms of the Fourier series of f.

Exercise 37.10 (Projection in function space)**.** In $\mathbb{R}^n$ the projection of $\mathbf{v}$ onto a unit vector $\mathbf{q}$ is $\langle \mathbf{q}, \mathbf{v}\rangle \mathbf{q}$. Translate this into function space: write the formula for the projection of a function f on $[0, 2\pi]$ onto the (not-yet-unit) direction $\sin x$, remembering to divide by $\|\sin x\|^2$. This single term is the "best multiple of $\sin x$" approximating f in the least-squares sense.

Exercise 37.11 (Why completeness matters – a discussion problem)**.** Consider continuous functions on $[-1, 1]$ and a sequence f_n that is -1 for $x < -\frac{1}{n}$, $+1$ for $x > \frac{1}{n}$, and linear in between. Sketch f_1, f_2, f_3. Argue informally that the f_n get closer and closer to one another in the L^2 length $\|f_m - f_n\|$, yet their "limit" is a discontinuous step function. Explain in a sentence why this shows the continuous functions are *not* complete, motivating the larger Hilbert space L^2.

Exercise 37.12 (Challenge: the trace is a tensor contraction)**.** The trace of a matrix is $\operatorname{tr}\mathbf{A} = \sum_i A_{ii}$. Viewing $\mathbf{A}$ as an order-2 tensor A_{ij}, the trace "contracts" the two indices by setting them equal and summing. Prove the cyclic property $\operatorname{tr}(\mathbf{AB}) = \operatorname{tr}(\mathbf{BA})$ by writing both sides as the double sum $\sum_{i,j} A_{ij}B_{ji}$, and explain why this makes the trace independent of the basis (i.e. $\operatorname{tr}(\mathbf{P}^{-1}\mathbf{AP}) = \operatorname{tr}\mathbf{A}$).

Exercise 37.13 (Challenge: $U(1)$ and the circle)**.** Show that the map $\theta \mapsto e^{i\theta}$ from angles to unit complex numbers sends addition of angles to multiplication of complex numbers, and that $\theta \mapsto \mathbf{R}(\theta)$ does the same into $SO(2)$. Conclude that the unit circle, the group $SO(2)$, and the unit complex numbers $U(1)$ are "the same group" (isomorphic). What is the inverse of $e^{i\theta}$, and what rotation does it correspond to?

Part IX

Mathematical Background

Chapter 38

Appendix: Sets, Functions, and Fields

Every chapter of linear algebra, from the first vector in $\mathbb{R}^2$ to the singular value decomposition, rests on three foundations: *sets* (the collections objects live in), *functions* (the maps that transform one set into another), and *fields* (the number systems where scaling and division are guaranteed to work).

This appendix is the toolbox under the workbench. Nothing here is "linear algebra" proper, but everything here is the vocabulary linear algebra is spoken in. When the main chapters say "the set of all vectors $\mathbf{v}$ such that $\mathbf{A}\mathbf{v} = \mathbf{0}$," or "the function T is one-to-one," or "let the scalars come from a field $\mathbb{F}$," they are quietly assuming you are comfortable with these three ideas. Most readers have met them before in scattered courses. The goal here is to gather them into one calm, geometry-first reference you can return to whenever a definition in the text feels like it skipped a step.

You do not need to read this front to back. Skim it, notice what is familiar, and circle back to whatever a later chapter leans on. But if any of the following ever make you hesitate – what exactly a function "onto" means, why $\mathbb{R}$ and $\mathbb{C}$ get to be the scalars while the integers $\mathbb{Z}$ do not, or how to read a proof without panicking – spend twenty unhurried minutes here. It pays for itself many times over.

38.1 Sets: collections of mathematical objects

38.1.1 What a set is, and the three things you do with it

A *set* is a collection of distinct objects, called its *elements* or *members*. That is the entire definition, and its plainness is the point. A set does not care about order, does not care about repetition, and does not (by itself) come with any operations. It is simply a collection of elements.

We write $x \in S$ to say "x is an element of S," and $x \notin S$ for "x is not." Two sets are *equal* exactly when they have the same elements – nothing more is required:

$$\{1, 2, 3\} = \{3, 1, 2\} = \{1, 1, 2, 3\}.$$

Order and duplicates are invisible to a set.

Intuition

Picture a set as a loop drawn on paper with dots inside it. The loop's shape, the position of the dots, the order you drew them – all irrelevant. The only question a set ever answers is: "is this thing in, or out?" Everything else in this section is bookkeeping built on that one yes/no question.

There are two standard ways to name a set. You can *list* its elements,

$$S = \{2, 4, 6, 8\},$$

or you can describe them with a *rule*, read "the set of all x such that…":

$$S = \{x \in \mathbb{N} : x \text{ is even and } 2 \le x \le 8\}.$$

The colon (some authors use a vertical bar |) means "such that." The part before it says where x lives; the part after it is the membership test. This builder notation is everywhere in linear algebra. The null space of a matrix, for instance, is

$$\text{null}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\},$$

"the set of all vectors $\mathbf{x}$ in $\mathbb{R}^n$ that $\mathbf{A}$ sends to zero." If you can read that line aloud and know what it is collecting, you can read most of this book.

38.1.2 Numbers you already trust, named

A handful of number sets recur so often they get their own blackboard letters. You have seen them; here they are in one place, each nested inside the next.

$\mathbb{N} = \{0, 1, 2, 3, \dots\}$	the *natural numbers* (counting numbers)
$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$	the *integers*
$\mathbb{Q} = \{p/q : p, q \in \mathbb{Z},\ q \neq 0\}$	the *rationals* (fractions)
$\mathbb{R}$	the *reals* (the whole number line)
$\mathbb{C} = \{a + bi : a, b \in \mathbb{R}\}$	the *complex* numbers

Each set sits inside the next: $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$ (Figure 38.1). The symbol $\subset$ means *subset* – every element of the left set is also an element of the right one. (Some authors write $\subseteq$ to stress that equality is allowed; we use $\subset$ to mean "is contained in," allowing equality, throughout.) The jump from $\mathbb{Q}$ to $\mathbb{R}$ fills in the "gaps" like $\sqrt{2}$ and π that no fraction reaches; the jump from $\mathbb{R}$ to $\mathbb{C}$ adds a square root of -1. We will lean on exactly two of these, $\mathbb{R}$ and $\mathbb{C}$, as our scalars, and Section 38.3 explains the precise reason those two are the comfortable choices.

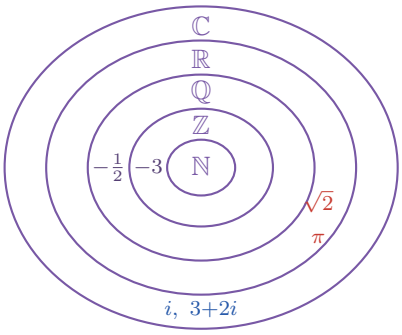


Figure 38.1: The standard number systems nest like Russian dolls: $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$. Each layer adds something the previous one was missing – negatives, then fractions, then irrationals like $\sqrt{2}$ and π, then a square root of -1.

A couple of special sets round out the vocabulary. The *empty set* $\varnothing = \{\,\}$ has no elements at all; it is the set-theoretic zero, and it turns up whenever a membership test is never satisfied (a system with no solution has an empty solution set). And for any set S we may ask its *cardinality* $|S|$, the number of elements in it: $|\{2, 4, 6, 8\}| = 4$, while $|\mathbb{R}|$ is infinite.

38.1.3 Combining sets

Given two sets A and B living inside some common universe, three operations let you build new sets. Each has a one-line definition and a one-picture meaning (Figure 38.2).

- **Union** $A \cup B = \{x : x \in A \text{ or } x \in B\}$ – everything in either bag. ("Or" in mathematics is always inclusive: in one, the other, or both.)
- **Intersection** $A \cap B = \{x : x \in A \text{ and } x \in B\}$ – everything in both bags at once.
- **Difference** $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$ – what is left of A after deleting anything that is also in B.

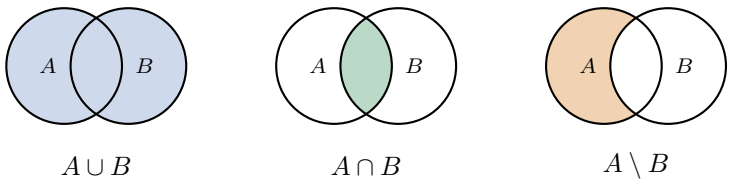


Figure 38.2: The three ways to combine two sets, shaded. Union keeps everything in either circle, intersection keeps the overlap, difference keeps the part of A outside B.

These basic set operations appear throughout the book. Two lines in the plane that cross meet in the *intersection* of their solution sets – a single point. Two lines that are parallel and distinct have empty intersection – the system has no solution. The span of a collection of vectors is built so that it is closed under combination, and subspaces will turn out to be closed under intersection but *not* under union, a subtlety the chapter on vector spaces makes much of.

38.1.4 Ordered pairs and the product $\mathbb{R}^n$

A set forgets order, but geometry cannot afford to: the point $(1, 2)$ is not the point $(2, 1)$. So we need a companion idea, the *ordered pair* (a, b), where position carries meaning and $(a, b) = (c, d)$ only when $a = c$ *and* $b = d$.

The *Cartesian product* of two sets collects all ordered pairs:

$$A \times B = \{(a, b) : a \in A,\ b \in B\}.$$

The most important instance in all of this book is

$$\mathbb{R}^2 = \mathbb{R} \times \mathbb{R} = \{(x, y) : x, y \in \mathbb{R}\},$$

the plane, and more generally $\mathbb{R}^n$, the set of ordered lists of n real numbers. That little superscript n in $\mathbb{R}^n$ is a Cartesian product done n times. Every vector you draw, every data point, every state of a system lives in some $\mathbb{R}^n$. The whole subject is, in one sense, the study of structure-preserving maps between these product sets.

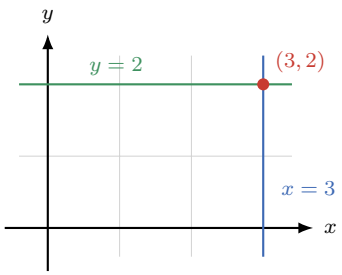


Figure 38.3: The product $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$ as a grid. Fixing the first coordinate at $x = 3$ (blue) traces out a vertical copy of $\mathbb{R}$; fixing the second at $y = 2$ (green) traces a horizontal copy. Their unique crossing is the ordered pair $(3, 2)$ – position carries meaning, so $(3, 2) \neq (2, 3)$.

Pause

Pause and think. Why is the Cartesian product $A \times B$ different from $B \times A$? If $A = \{1, 2\}$ and $B = \{3, 4\}$, write out both sets of pairs: $A \times B = \{(1, 3), (1, 4), (2, 3), (2, 4)\}$, while $B \times A = \{(3, 1), (3, 2), (4, 1), (4, 2)\}$. In linear algebra, why does position matter so decisively when we write a column vector $\begin{bmatrix} x \\ y \end{bmatrix}$? Because position encodes geometric direction: the vector $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ steps along the horizontal axis, while $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ steps along the vertical axis. A set forgets order; a Cartesian product preserves it.

Focus

The four set ideas you actually need downstream: (1) builder notation $\{x : \text{rule}\}$ to *define* subspaces and solution sets; (2) subset $\subset$ to say one space sits inside another; (3) intersection $\cap$ for "solves both conditions"; (4) the product $\mathbb{R}^n$ as the home of vectors (Figure 38.3). If those four feel automatic, the rest of this section is just polish.

38.2 Functions: mappings between sets

38.2.1 Domain, codomain, rule

A *function* f from a set A to a set B, written

$$f : A \to B,$$

is a rule that assigns to *each* element $a \in A$ exactly *one* element $f(a) \in B$. The set A is the *domain* (the legal inputs), and B is the *codomain* (the declared pool of possible outputs). The two words "each" and "exactly one" carry all the weight: every input must get an answer, and no input may get two.

Intuition

A function is a machine. You feed in one item from the input bag A; out comes one item from the output bag B. The machine is deterministic (same input, same output every time) and total (it never jams on a legal input). That is the whole contract. A matrix $\mathbf{A}$ acting by $\mathbf{x} \mapsto \mathbf{Ax}$ is exactly such a machine, with domain $\mathbb{R}^n$ and codomain $\mathbb{R}^m$.

Notice the asymmetry. Every input is used exactly once, but outputs are under no such rule: an output in B may be hit by several inputs, or by none at all. Those two loopholes – "hit more than once" and "hit not at all" – are precisely what the next two definitions police.

The *image* (or *range*) of f is the set of outputs that actually occur:

$$\operatorname{im}(f) = \{f(a) : a \in A\} \subset B.$$

The image can be smaller than the codomain. The squaring function $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, has codomain $\mathbb{R}$ but image only $[0, \infty)$ – negative numbers are declared possible outputs yet never produced. For a matrix, the image is exactly the *column space*: the set of all $\mathbf{b}$ you can actually reach as $\mathbf{Ax}$.

38.2.2 Injective, surjective, bijective

These three words sound forbidding and mean something you can see in a picture (Figure 38.4). They describe how the arrows of a function land on the codomain.

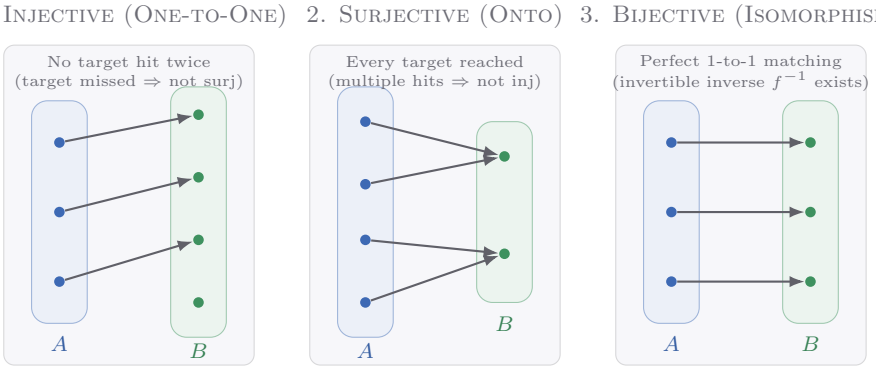


Figure 38.4: The three structural archetypes of a mapping. Left: Injective (no two domain elements map to the same target; $\text{null}\,\mathbf{A} = \{\mathbf{0}\}$). Middle: Surjective (the entire codomain is covered; $\text{col}\,\mathbf{A} = B$). Right: Bijective (both injective and surjective – a perfect matching, representing an invertible transformation or isomorphism).

Formal

Let $f : A \to B$.

- f is **injective** (one-to-one) if distinct inputs give distinct outputs: $a_1 \neq a_2 \implies f(a_1) \neq f(a_2)$. Equivalently (and this is the form you use in proofs), $f(a_1) = f(a_2) \implies a_1 = a_2$. No output is hit twice.
- f is **surjective** (onto) if every element of the codomain is hit: for every $b \in B$ there is some $a \in A$ with $f(a) = b$. The image fills all of B. No output is missed.
- f is **bijective** if it is both injective and surjective: a perfect one-to-one pairing between A and B, with nothing doubled and nothing left out.

Common trap

"Injective" and "surjective" are statements about a function *together with its declared codomain*, not about the formula alone. The map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = e^x$, is not surjective (-1 is never an output). But the very same formula seen as $f : \mathbb{R} \to (0, \infty)$ *is* surjective. Change the codomain and you can change the verdict. Always ask "into which B?" before you answer.

Example 38.1 (Testing all three on small maps)**.** Take inputs and outputs in $\mathbb{R}$ unless noted.

- $f(x) = 2x$ on $\mathbb{R} \to \mathbb{R}$. Injective: if $2x_1 = 2x_2$ then $x_1 = x_2$. Surjective: any y is hit by $x = y/2$. So bijective.
- $f(x) = x^2$ on $\mathbb{R} \to \mathbb{R}$. Not injective: $f(3) = f(-3) = 9$. Not surjective: -4 is never an output. Fails both.
- $f(x) = x^3$ on $\mathbb{R} \to \mathbb{R}$. Injective (cubing is strictly increasing) and surjective (every real has a real cube root). Bijective.
- $f(x) = e^x$ on $\mathbb{R} \to \mathbb{R}$. Injective but not surjective, as in the trap above.

38.2.3 Why this is the spine of linear algebra

Every statement about a matrix $\mathbf{A}$ being invertible is directly an expression of injectivity, surjectivity, or bijectivity. Look at the map $T(\mathbf{x}) = \mathbf{A}\mathbf{x}$ for an $n \times n$ matrix.

- T is **injective** exactly when the only solution of $\mathbf{A}\mathbf{x} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$ – that is, when the null space is trivial, when the columns are independent. Two different inputs giving the same output would mean $\mathbf{A}(\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{0}$ with a nonzero difference.

- T is **surjective** exactly when the column space is all of $\mathbb{R}^n$ – every target $\mathbf{b}$ is reachable, so $\mathbf{A}\mathbf{x} = \mathbf{b}$ always has a solution.

- T is **bijective** exactly when $\mathbf{A}$ is invertible. Then, and only then, can you undo the machine.

So the entire "invertibility" story – independent columns, trivial null space, full column space, nonzero determinant – is the demand that one linear machine be a bijection.

Pause

Pause and think. Suppose a linear map $T : \mathbb{R}^3 \to \mathbb{R}^2$ is represented by a 2×3 matrix $\mathbf{A}$. Can T ever be injective? Picture sending three independent directions of space into a two-dimensional plane: three column vectors in $\mathbb{R}^2$ cannot be linearly independent. At least one nonzero vector must get crushed to the origin ($\text{null}(\mathbf{A}) \neq \{\mathbf{0}\}$). Collisions are mathematically unavoidable, so T can never be injective. Can T be surjective? Yes: as long as the columns span the whole plane $\mathbb{R}^2$ ($\text{rank}\,\mathbf{A} = 2$), every output point is reached.

38.2.4 Inverses and composition

When $f : A \to B$ is bijective, you can run it backwards. The *inverse* $f^{-1} : B \to A$ sends each output back to the unique input that produced it, and it satisfies

$$f^{-1}(f(a)) = a \quad \text{and} \quad f(f^{-1}(b)) = b.$$

Only bijections have inverses: you need surjectivity so that every b has *some* preimage to return to, and injectivity so that the preimage is *unique* (otherwise f^{-1} would not be a function). This is the abstract reason a non-square or rank-deficient matrix has no inverse – the underlying map fails one of the two halves.

Composition chains two machines: if $f : A \to B$ and $g : B \to C$, then $g \circ f : A \to C$ means "do f, then g," so $(g \circ f)(a) = g(f(a))$. Read it right to left, the way function notation forces (Figure 38.5). This is exactly how matrix multiplication is born: applying $\mathbf{A}$ then $\mathbf{B}$ to a vector is the single matrix $\mathbf{B}\mathbf{A}$, and the order matters because composition does not commute. The notation $\mathbf{B}\mathbf{A}$ with $\mathbf{A}$ on the right is precisely $g \circ f$ with f on the right.

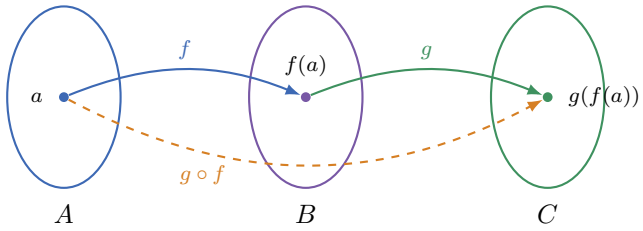


Figure 38.5: Composition runs right to left: $g \circ f$ first applies f (blue), landing in B, then applies g (green), landing in C. The dashed arrow is the single combined machine $g \circ f$. Matrix multiplication $\mathbf{BA}$ is this picture with $f = \mathbf{A}$ nearest the input.

Connection to other math

The set of all bijections of a set to itself, under composition, is the prototype of a *group* – composition is associative, identity is "do nothing," and every bijection has an inverse. When the bijections are *linear*, this group is the general linear group $GL_n(\mathbb{R})$ of invertible matrices, the central object of the chapter on groups and symmetry. So "bijection plus composition" forms the foundation of group theory, with invertible matrices serving as its primary linear realization.

Modern application

In machine learning a *neural network* is a long composition $f_L \circ \cdots \circ f_2 \circ f_1$ of simple maps, each a linear function followed by an elementwise nonlinearity. Training adjusts the linear pieces; the chain rule of calculus – itself a statement about how derivatives compose – propagates error backwards through the composition (this is "backpropagation"). *Normalizing flows* go further and insist every layer be a *bijection*, precisely so the network can be run backwards to generate data and so that probabilities can be tracked exactly through the chain. In architectures like normalizing flows, injectivity, surjectivity, and composition act directly as design constraints.

38.3 Fields: the numbers you are allowed to scale by

38.3.1 The question hiding behind "scalar"

Every time you write $c\,\mathbf{v}$ you are scaling a vector by a number c. Which numbers are legal? The main text answers "real numbers, usually," and then, in the chapter on complex matrices, "complex numbers." But *why those two*, and not the integers, or just the positive numbers, or some other pool? Linear algebra needs its scalars to support a specific list of guarantees – and any number system meeting that list is called a *field*. $\mathbb{R}$ and $\mathbb{C}$ are simply the two fields you already know and trust.

Intuition

A field is a number system in which you can add, subtract, multiply, and *divide* by any nonzero element, with all the familiar rules of arithmetic holding. The make-or-break ability is division: solving $c\,\mathbf{v} = \mathbf{w}$ for $\mathbf{v}$ means multiplying by $1/c$, and the whole machinery of pivots, inverses,

and bases depends on being able to do that for any nonzero scalar. Fields are exactly the number systems where "divide by the pivot" never fails.

38.3.2 The definition, friendly first

A field has two operations, $+$ and $\times$, and asks that each be as well-behaved as possible. The cleanest way to say it: addition and multiplication both "undo" themselves.

Formal

A **field** is a set $\mathbb{F}$ with two operations $+$ and $\cdot$ such that for all $a, b, c \in \mathbb{F}$:

- **Addition** is commutative and associative, has an identity 0, and every a has a negative $-a$ with $a + (-a) = 0$.
- **Multiplication** is commutative and associative, has an identity $1 \neq 0$, and every *nonzero* a has an inverse a^{-1} with $a \cdot a^{-1} = 1$.
- **Distributivity** links them: $a \cdot (b + c) = a \cdot b + a \cdot c$.

The single line with real teeth is "every nonzero a has an inverse a^{-1}." That is division. Everything else on the list you have believed since grade school. Compare with the vector-space axioms in the main text: they have the same shape, and that is no accident – a field is, among other things, a tiny one-dimensional vector space over itself, and the field axioms are exactly the guarantees the scalars must provide so the vector-space axioms can do their work.

History and connection

For centuries, algebraists hunted for a general formula to solve polynomial equations using *radicals*—formulas built only from addition, subtraction, multiplication, division, and roots like square roots ($\sqrt{\cdot}$) or cube roots ($\sqrt[3]{\cdot}$). The quadratic formula was known in antiquity; Tartaglia, Cardano, and Ferrari conquered cubic and quartic equations in Renaissance Italy. But the general degree-5 equation, the *quintic*, stubbornly defeated every attack.

In May 1832, on the eve of his fatal duel at age twenty, the French mathematician Évariste Galois stayed awake through the night, feverishly summarizing his mathematical discoveries in a letter to his friend Auguste Chevalier. Galois solved the riddle by changing the question. Instead of hunting for formulas, he looked at the symmetries of the solutions. He realized that the roots of a polynomial generate an *algebraic number field*—a self-contained number system formed by adjoining the roots to the rational numbers $\mathbb{Q}$, where you can add, subtract, multiply, and divide without ever leaving the set. The shufflings of the roots that leave all algebraic relationships intact form a symmetry structure called a *group*. Galois proved that an equation can be solved by radicals if and only if its symmetry group can be broken down in a specific step-by-step way; the general quintic fails because the symmetry group of five roots lacks that nested structure.

Galois's ideas were too revolutionary for his contemporaries. He had submitted earlier manuscripts to the French Academy of Sciences, but Augustin-Louis Cauchy lost one, Joseph Fourier died with another unread on his desk, and Siméon Denis Poisson rejected a third as "incomprehensible." Galois was killed the morning after writing his letter. Decades later, Richard Dedekind formalized these self-contained systems, coining the word *Körper* (German

for "body" or "field") in 1871. His abstract approach faced bitter resistance from Leopold Kronecker, who insisted that mathematics must remain rooted in concrete integer arithmetic ("God made the integers, all else is the work of man").
In 1910, Ernst Steinitz published *Algebraische Theorie der Körper*, establishing the modern axiomatic foundation for fields. Steinitz proved that every field is governed by its *characteristic*—the number of times you must add 1 to itself to wrap back to zero. If you never reach zero (as in $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$), the characteristic is zero; if adding 1 to itself p times reaches zero, the characteristic is a prime number p, producing finite "clock arithmetic" fields $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$.
This has immediate consequences for linear algebra. Almost every algebraic theorem in this book—Gaussian elimination, vector space dimensions, rank-nullity, matrix inversion, and determinants—relies strictly on the field axioms. It does not care whether the scalars are smooth measurements on the real line or discrete bits modulo 2. While continuous geometry and physics need $\mathbb{R}$ and $\mathbb{C}$, modern computer science, error-correcting codes (like Reed–Solomon codes used in CDs and deep-space communication), and cryptography (such as the AES encryption standard and elliptic curves) operate entirely on vector spaces over finite fields like $\mathbb{F}_2$ and $\mathbb{F}_{2^8}$.
Yet mysteries remain. While finite fields are completely classified, the celebrated *Inverse Galois Problem*—which asks whether every finite group can be realized as the symmetry group of some algebraic number field over $\mathbb{Q}$—remains unsolved after nearly two hundred years.

38.3.3 Who passes, who fails

Run the systems we tabulated earlier past the field test, watching the division line.

System	Field?	Reason
$\mathbb{R}$ (reals)	yes	add, subtract, multiply, divide by nonzero – all fine
$\mathbb{C}$ (complex)	yes	same, and even better: every polynomial has a root
$\mathbb{Q}$ (rationals)	yes	fractions of fractions are fractions
$\mathbb{Z}$ (integers)	**no**	$1/2 \notin \mathbb{Z}$: cannot divide
$\mathbb{N}$ (naturals)	**no**	no negatives, no division

This is the precise reason linear algebra "over the integers" is harder and stranger: $\mathbb{Z}$ is not a field, so you cannot freely divide by a pivot, and Gaussian elimination stalls. The instant you allow yourself fractions – step up to $\mathbb{Q}$ or $\mathbb{R}$ – elimination flows, every nonzero pivot is invertible, and the clean theory you know returns. When the main text says "let the scalars be real," it is really saying "let the scalars be a field, and $\mathbb{R}$ is the one we picture."

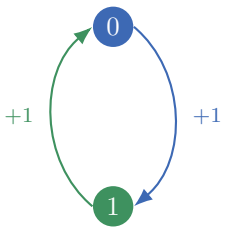
Pause

Pause and think. Why is solving linear systems over the integers $\mathbb{Z}$ fundamentally different from solving them over $\mathbb{Q}$ or $\mathbb{R}$? Consider the simple equation $3x = 5$. Over the rationals $\mathbb{Q}$, we multiply both sides by the inverse $3^{-1} = 1/3$ to find the unique solution $x = 5/3$. Over $\mathbb{Z}$, the number 3 has no multiplicative inverse, so the equation has no integer solution at all. In Gaussian elimination, row reduction relies constantly on dividing a row by its pivot entry to create a leading 1 and clear entries below it. Without division, elimination grinds to a halt.

Example 38.2 (A finite field: arithmetic on a clock)**.** Fields need not be infinite. Take $\mathbb{F}_2 = \{0, 1\}$ with addition and multiplication done "modulo 2" – keep only the remainder after dividing by 2 (Figure 38.6):

$$0+0=0, \quad 0+1=1, \quad 1+1=0, \qquad 0\cdot 0=0, \quad 0\cdot 1=0, \quad 1\cdot 1=1.$$

Check the teeth: the only nonzero element is 1, and $1\cdot 1 = 1$, so 1 is its own inverse ($1^{-1} = 1$). Division works, so $\mathbb{F}_2$ is a genuine field with just two elements – it is the arithmetic of parity, of a single bit, of XOR. Linear algebra over $\mathbb{F}_2$ is the mathematics of error-correcting codes and cryptography, and every theorem about bases and rank you prove "over a field" holds there verbatim.



Addition in $\mathbb{F}_2$ (XOR)

$1 + 1 = 0$ (toggle wheel)

Addition (+)

+	0	1
0	0	1
1	1	0

Multiplication (·)

·	0	1
0	0	0
1	0	1

Every nonzero element (1) has a reciprocal:
$1 \cdot 1 = 1 \implies 1^{-1} = 1$. Division is guaranteed.

Figure 38.6: Arithmetic in the two-element finite field $\mathbb{F}_2 = \{0, 1\}$. Left: Addition is a two-state wheel where adding 1 toggles between 0 and 1, so $1 + 1 = 0$ (the XOR operation in computer hardware). Right: Cayley tables. The only nonzero element, 1, satisfies $1 \cdot 1 = 1$, making it its own inverse ($1^{-1} = 1$). All field axioms hold on just two symbols.

Why this matters

Here is what the field abstraction buys you. You prove the theory of bases, dimension, rank, and linear maps *once*, assuming only "the scalars form a field." That single proof then pays out over $\mathbb{R}$ (geometry and data), over $\mathbb{C}$ (signals, quantum mechanics, the spectral theorem), over $\mathbb{Q}$ (exact symbolic computation), and over $\mathbb{F}_2$ and its finite cousins (coding theory, cryptography). Proving a theorem once from the field axioms makes it available across all four settings without repeating the algebra.

38.3.4 Why $\mathbb{R}$ and why $\mathbb{C}$

Among all fields, two earn their place as the default scalars for very human reasons.

$\mathbb{R}$ is the field of *measurement*. Lengths, angles, masses, probabilities, coordinates of a point – the physical world is reported in real numbers, and the real line is *complete* (no gaps), which is exactly what makes limits, continuity, and hence calculus possible. When linear algebra meets the world, it meets it over $\mathbb{R}$.

$\mathbb{C}$ is the field of *completion*. The reals have a flaw: the innocent equation $x^2 + 1 = 0$ has no real solution. Adjoining a single new number i with $i^2 = -1$ repairs this flaw *totally* – over $\mathbb{C}$ every polynomial of degree n has exactly n roots (the Fundamental Theorem of Algebra). For linear algebra this is not a curiosity; it is the reason every square matrix has a full set of eigenvalues over

$\mathbb{C}$, why rotations have "imaginary" eigenvalues, and why the spectral story is cleanest in the complex setting. In short, $\mathbb{R}$ models real measurements while $\mathbb{C}$ guarantees a full set of roots and eigenvalues.

Connection to other math

The two reasons above point at two different branches of mathematics. The completeness of $\mathbb{R}$ is an *analysis* fact – it is what lets you take limits, and it underwrites every convergence argument in numerical linear algebra and in the calculus that powers optimization. The algebraic closure of $\mathbb{C}$ is an *algebra* fact – it is what lets every characteristic polynomial factor completely, and it is the bridge to the theory of eigenvalues, Jordan form, and beyond. Choosing $\mathbb{R}$ or $\mathbb{C}$ as your scalars quietly decides which of these two great traditions you are leaning on.

38.4 Just enough proof literacy

The main chapters prove things, gently but for real. You do not need to be a proof athlete to follow them, but a few patterns recur so often that recognizing them turns proofs from intimidating to almost predictable. This section is a field guide to those patterns.

38.4.1 Reading the logical connectives

Three little phrases do most of the work in any proof.

- **Implication** $P \implies Q$: "if P, then Q." To prove it, you assume P is true and argue your way to Q. You never have to worry about the case where P is false – the claim says nothing there.
- **Equivalence** $P \iff Q$: "P if and only if Q," often abbreviated "iff." This is two implications bundled: $P \implies Q$ *and* $Q \implies P$. To prove an iff you must prove *both* directions, and forgetting one is the most common gap in a beginner's proof.
- **Quantifiers** "for all" ($\forall$) and "there exists" ($\exists$). "For all" is a promise about every case; to *disprove* it you need just one counterexample. "There exists" is a claim that at least one case works; to prove it you exhibit one witness, and to disprove it you must rule out all cases.

Common trap

$P \implies Q$ does *not* give you $Q \implies P$. "If $\mathbf{x}$ is the zero vector, then $\mathbf{Ax} = \mathbf{0}$" is always true; its reverse "if $\mathbf{Ax} = \mathbf{0}$ then $\mathbf{x} = \mathbf{0}$" is true only for special matrices (the injective ones). Confusing an implication with its converse is the single most frequent logical slip in linear algebra (Figure 38.7). When a theorem is an "iff," respect that it is making two separate promises.

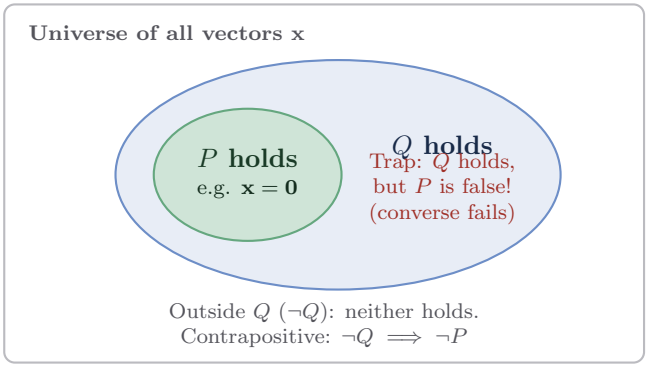


Figure 38.7: The geometry of logical implication $P \implies Q$ viewed as set containment ($P \subset Q$). If a case lands inside the inner disc P, it automatically lands inside Q (direct implication). If it lands outside Q, it cannot possibly be inside P (the contrapositive $\neg Q \implies \neg P$). The converse $Q \implies P$ fails because of the outer ring $Q \setminus P$: for example, $\mathbf{Ax} = \mathbf{0}$ holds for all vectors in null($\mathbf{A}$), not just $\mathbf{x} = \mathbf{0}$.

38.4.2 The handful of proof shapes

Almost every proof in this book is one of four moves.

Direct proof. Assume the hypothesis, walk forward through definitions and algebra, arrive at the conclusion. Most "closure" arguments are direct: to show a set is a subspace you take two members, add them, and check the sum is still a member – one short computation.

Proof by contradiction. Suppose the claim is *false*, then derive something impossible (like $0 = 1$, or a vector that is both zero and nonzero). The impossibility means your supposition was wrong, so the claim is true. The classic example is showing a set of vectors is independent: assume a nontrivial combination equals zero, then squeeze out a contradiction.

Proof by contrapositive. To prove $P \implies Q$, prove instead the logically identical $\neg Q \implies \neg P$ ("not Q implies not P"). These two statements are *always* equivalent, so you may prove whichever is easier. Often the contrapositive is the friendlier road.

Proof by induction. To prove a statement for all natural numbers n: show it for the base case $n = 0$ (or 1), then show that its truth at n forces its truth at $n + 1$. Like dominoes – knock over the first, guarantee each knocks the next, and they all fall. Dimension arguments and "every matrix can be reduced in finitely many steps" arguments often run on induction.

Example 38.3 (A direct proof you can already follow)**.** *Claim.* If $f : A \to B$ and $g : B \to C$ are both injective, then $g \circ f : A \to C$ is injective.

Proof. We use the proof-friendly form of injectivity: "equal outputs force equal inputs." Suppose $(g \circ f)(a_1) = (g \circ f)(a_2)$, i.e. $g(f(a_1)) = g(f(a_2))$. Since g is injective, equal outputs of g force equal inputs, so $f(a_1) = f(a_2)$. Since f is injective, this in turn forces $a_1 = a_2$. We assumed equal outputs of $g \circ f$ and deduced equal inputs, so $g \circ f$ is injective. ∎

Read that proof twice. Nothing in it is clever; it is just the definitions, applied in order, with each "since" pointing back to exactly one hypothesis. The vast majority of proofs in this book have that same unhurried texture. When one feels hard, slow down and ask of each sentence: which definition or hypothesis is this using? Almost always the answer is sitting one line up.

Walking question

On your next walk, take the claim "if $g \circ f$ is injective, then f is injective" (note: only f, not g). Is it true? Try to either prove it directly or build a tiny counterexample with three- or four-element sets and arrows on paper. Then ask the mirror question about surjectivity: if $g \circ f$ is surjective, which of f, g must be surjective? Getting these straight in your head, away from a screen, cements the whole section.

Before/after Strang

Strang's lectures move fast through this background – he assumes you can read set-builder notation and knows you will pick up "onto" and "one-to-one" from context. *Before* Strang, this appendix is the gentle on-ramp: get comfortable here and his definitions will feel like old friends rather than new hurdles. *After* Strang, return for the field perspective – once you have seen eigenvalues fail to be real and then reappear over $\mathbb{C}$, the "why $\mathbb{R}$ and why $\mathbb{C}$" section explains the pattern you noticed but he did not pause to name.

Focus

Summary of core definitions: A **set** answers "in or out." A **function** is a deterministic machine; **injective** means no output is reused, **surjective** means no output is missed, and **bijective** (both) means it can be run backwards – which for a matrix is exactly invertibility. A **field** is a number system where you can divide by anything nonzero; $\mathbb{R}$ and $\mathbb{C}$ are the two we use because one measures the world and the other completes the algebra. And a **proof** is almost always just the definitions applied in order. Carry these and the main text will rarely surprise you.

38.5 Power and limits

Linear algebra does not require vectors to be literal arrows on graph paper. By framing the subject through sets and fields, any collection satisfying the vector-space axioms over a field $\mathbb{F}$ inherits the main theorems: Gaussian elimination, dimension, rank-nullity, and invertibility all hold unchanged. Matrices act as functions between these spaces: a trivial null space means injectivity (no two inputs collide), a full column space means surjectivity (no output is missed), and invertibility is bijectivity (the map can be reversed). Over finite fields like $\mathbb{F}_2$, this abstraction underlies error-correcting codes and digital cryptography.

Yet the tools have clear limits. Over finite fields, Euclidean geometric intuitions no longer apply: there is no order ($x > 0$ is meaningless), and lengths and angles vanish. In $\mathbb{F}_2^2$, the nonzero vector $\begin{bmatrix}1\\1\end{bmatrix}$ has dot product $1 \cdot 1 + 1 \cdot 1 = 1 + 1 = 0$, meaning it is orthogonal to itself. Naive set comprehension also runs into paradoxes like Russell's paradox, though linear algebra avoids them by working inside explicit ambient spaces like $\mathbb{R}^n$ or $\mathbb{F}^n$.

With sets, functions, and fields established, we turn next to the computational machinery of linear algebra: summation notation ($\sum$), complex numbers and polar forms ($e^{i\theta}$), and multivariable matrix calculus in Appendix B (Chapter 39).

38.6 Exercises

Exercise 38.1 (Reading builder notation)**.** Translate each set into plain English and list its elements (or describe them) explicitly.

$$\text{(a) } \{n \in \mathbb{Z} : -3 \le n \le 2\}, \qquad \text{(b) } \{x \in \mathbb{R} : x^2 = 4\}, \qquad \text{(c) } \{(x, y) \in \mathbb{R}^2 : x + y = 1\}.$$

For (c), what geometric object is it?

Exercise 38.2 (Set combinations)**.** Let $A = \{1, 2, 3, 4, 5\}$ and $B = \{2, 4, 6\}$. Compute $A \cup B$, $A \cap B$, $A \setminus B$, and $B \setminus A$. Then verify on this example that $A \setminus B$ and $B \setminus A$ are generally different sets.

Exercise 38.3 (Classifying functions)**.** For each function decide whether it is injective, surjective, both, or neither, and justify in one sentence. Take domain and codomain $\mathbb{R} \to \mathbb{R}$ unless stated.

$$\text{(a) } f(x) = 3x - 1, \quad \text{(b) } f(x) = x^2 + 1, \quad \text{(c) } f(x) = x^3 - x, \quad \text{(d) } f : \mathbb{R} \to [0, \infty),\ f(x) = x^2.$$

Exercise 38.4 (Codomain matters)**.** The formula $f(x) = \tan(x)$ on the open interval $(-\pi/2, \pi/2)$ can be viewed as a map into $\mathbb{R}$. Argue that it is bijective onto $\mathbb{R}$, and explain in words why shrinking the domain (say to $(0, \pi/2)$) would ruin surjectivity. What does this say in general about the two loopholes a function has?

Exercise 38.5 (Inverses need bijections)**.** Explain, using the definitions of injective and surjective, exactly which property fails when you try to invert $f(x) = x^2$ on $\mathbb{R} \to \mathbb{R}$. Then show that restricting both domain and codomain to $[0, \infty)$ makes it bijective, and find the inverse formula.

Exercise 38.6 (Is it a field?)**.** Decide whether each is a field, and if not, name the axiom(s) that fail.

$$\text{(a) } \mathbb{Q}, \qquad \text{(b) the even integers } 2\mathbb{Z}, \qquad \text{(c) } \{a + b\sqrt{2} : a, b \in \mathbb{Q}\}.$$

For (c) you may take on faith that sums and products stay in the set; focus on whether nonzero elements have inverses. (Hint for (c): rationalize $\dfrac{1}{a + b\sqrt{2}}$.)

Exercise 38.7 (The two-element field in action)**.** In $\mathbb{F}_2 = \{0, 1\}$ with arithmetic modulo 2, solve the linear system

$$x + y = 1, \qquad y + z = 0, \qquad x + z = 1$$

for $x, y, z \in \mathbb{F}_2$. (Remember $1 + 1 = 0$.) How many solutions are there? Contrast this with how the same-shaped system would behave over $\mathbb{R}$.

Exercise 38.8 (Implication versus converse)**.** For a matrix $\mathbf{A}$, consider P: "the columns of $\mathbf{A}$ are linearly independent" and Q: "$\mathbf{Ax} = \mathbf{b}$ has a solution for every $\mathbf{b}$." For a general (non-square) matrix, can P hold while Q fails? Can Q hold while P fails? For a *square* matrix, explain why they become equivalent. (You may quote results from the main text; the point is to practice separating an implication from its converse.)

Exercise 38.9 (A composition proof)**.** Prove, in the unhurried style of Example 38.3: if $f : A \to B$ and $g : B \to C$ are both surjective, then $g \circ f$ is surjective. (Start from an arbitrary $c \in C$ and walk backwards through the two surjectivity assumptions.)

Exercise 38.10 (Counterexample hunting)**.** Decide whether each statement is true; if false, give the smallest counterexample you can (small finite sets and arrows on paper are ideal).

(a) If $g \circ f$ is injective, then g is injective.

(b) If $g \circ f$ is injective, then f is injective.

(c) If $g \circ f$ is surjective, then g is surjective.

Exercise 38.11 (Connecting to the matrix story)**.** Let $T : \mathbb{R}^3 \to \mathbb{R}^2$ be $T(\mathbf{x}) = \mathbf{A}\mathbf{x}$ for some 2×3 matrix $\mathbf{A}$. Using the function definitions only, explain why T *cannot* be injective no matter what $\mathbf{A}$ is, but *can* be surjective. Relate your reasoning to the sizes 3 (inputs) and 2 (outputs), and to the idea of a null space.

Exercise 38.12 (Contrapositive and eigenvalues)**.** Prove the following statement by contrapositive: "If an $n \times n$ matrix $\mathbf{A}$ is invertible, then $\lambda = 0$ is not an eigenvalue of $\mathbf{A}$." Recall that λ is an eigenvalue of $\mathbf{A}$ if there exists a *nonzero* vector $\mathbf{v}$ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$.

(a) Formulate the precise contrapositive statement ($\neg Q \implies \neg P$): what does it mean for $\lambda = 0$ to be an eigenvalue of $\mathbf{A}$?

(b) If $\mathbf{A}\mathbf{v} = 0\mathbf{v} = \mathbf{0}$ for some $\mathbf{v} \neq \mathbf{0}$, what does this imply about the null space $\text{null}(\mathbf{A})$ and the injectivity of the map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$?

(c) Conclude that $\mathbf{A}$ cannot be invertible, completing the proof.

Chapter 39

Appendix: Sums, Complex Numbers, and Calculus

Following Chapter 38's foundation of sets and fields, this appendix gathers three computational tools that appear throughout linear algebra: the *summation sigma* (for compact matrix and dot products), *complex numbers* (essential for eigenvalues and rotations), and a thin slice of *calculus* (derivatives, gradients, and integration for optimization and differential equations). None of it is difficult. But if any of it feels shaky, the linear algebra built on top of it will feel shaky too, so we make each piece concrete.

You do not need to read this appendix front to back. Treat it as a workshop to visit whenever a main chapter writes something like $\sum_k a_{ik}b_{kj}$, or $e^{i\theta}$, or ∇f, and you want the exact definition of that symbol.

Before/after Strang

Strang's *Introduction to Linear Algebra* assumes you already carry this toolbox: it writes matrix products as sums over a shared index, jumps to complex eigenvalues for rotations without pausing, and uses ∇ and the matrix exponential when applications call for them. If those steps ever seemed unexplained, read the matching section here *before* the corresponding Strang chapter, then return. *After* Strang, this appendix is the place to re-anchor the three habits that the whole subject rests on: a sum is a linear loop, complex multiplication is rotate-and-scale, and setting a derivative to zero is how optimization becomes a linear system.

39.1 Sigma Notation: A Loop on Paper

Suppose you want to add up the first five squares: $1 + 4 + 9 + 16 + 25$. Writing the whole sum is fine for five terms. For five hundred it is hopeless, and for "the first n" it is impossible, because you do not know n yet. Mathematicians solved this with one symbol, the capital Greek letter sigma, $\sum$, which means *add these up*.

Intuition

Read $\sum_{k=1}^{n} a_k$ out loud as a little program: "start a running total at zero; let k count from 1 up to n; each step, add a_k to the total; when k passes n, stop and report the total." The $\sum$ is a `for` loop, and a_k is the body of the loop. That is the whole idea.

The anatomy of the symbol:

$$\underbrace{\sum_{\underbrace{k}_{\text{index}}=\underbrace{1}_{\text{start}}}^{\overbrace{n}^{\text{stop}}}}\ \underbrace{a_k}_{\text{term}} \;=\; a_1 + a_2 + a_3 + \cdots + a_n.$$

The letter k is the *index of summation*; it is a dummy variable, meaning it is a temporary placeholder whose actual name has no meaning outside the loop. Writing $\sum_{k=1}^{n} a_k$ or $\sum_{j=1}^{n} a_j$ or $\sum_{i=1}^{n} a_i$ all mean exactly the same number. The index exists only inside the sum and never appears as a free variable in the final result.

Example 39.1 (Unrolling a few sums)**.**

$$\sum_{k=1}^{5} k^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55.$$

$$\sum_{k=0}^{3} 2^k = 2^0 + 2^1 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15.$$

$$\sum_{k=1}^{4} 3 = 3 + 3 + 3 + 3 = 12 \qquad \text{(the term does not have to mention } k\text{)}.$$

The last one is worth a pause: a sum of a constant c taken n times is just nc. In symbols $\sum_{k=1}^{n} c = nc$. This is the first of several formulas we will collect.

Pause

Pause and think: What is $\sum_{k=1}^{5} 10$? In your head: there is no k in the summand, so you are simply adding five copies of 10, giving 50. What about $\sum_{k=1}^{4}(k-(k-1))$? Before calculating anything, write the first two terms: $(1-0)+(2-1)+\cdots$. Every inner term cancels out, leaving only the very last upper value minus the starting lower value: $4-0=4$ (a telescoping sum).

39.1.1 The three rules that make sigma algebra work

Because a sum is just repeated addition, it inherits the friendly properties of addition. Three of them get used over and over, so we name them.

Formal

For any numbers a_k, b_k and any constant c:

$$\text{(Constant out)} \quad \sum_{k=1}^{n} c\,a_k = c\sum_{k=1}^{n} a_k, \tag{39.1}$$

$$\text{(Split a sum)} \quad \sum_{k=1}^{n}(a_k + b_k) = \sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k, \tag{39.2}$$

$$\text{(Split the range)} \quad \sum_{k=1}^{n} a_k = \sum_{k=1}^{m} a_k + \sum_{k=m+1}^{n} a_k \quad (1 \le m < n). \tag{39.3}$$

Rule (39.1) is just the distributive law spread across many terms: $ca_1 + ca_2 + \cdots = c(a_1 + a_2 + \cdots)$. Rule (39.2) is the commutativity of addition: you may regroup $(a_1 + b_1) + (a_2 + b_2) + \cdots$ as $(a_1 + a_2 + \cdots) + (b_1 + b_2 + \cdots)$. Rule (39.3) just says you may add the first chunk and the second chunk separately. Together these three are *linearity* of the sum, and linearity is the reason $\sum$ plays so nicely with linear algebra.

Common trap

A constant pulls out, but a *product of two indexed things does not split.* In general

$$\sum_k a_k b_k \;\neq\; \Big(\sum_k a_k\Big)\Big(\sum_k b_k\Big).$$

Check it on $a = b = (1, 1)$: the left side is $1{\cdot}1{+}1{\cdot}1 = 2$, the right side is $(1{+}1)(1{+}1) = 4$. The left side $\sum_k a_k b_k$ is exactly the dot product $\mathbf{a}{\cdot}\mathbf{b}$, and the whole point of the dot product is that it is *not* the product of the sums. Keeping this straight is what keeps matrix multiplication straight.

39.1.2 Four closed-form sums worth memorizing

A few sums collapse to a clean formula. You will meet them when counting operations in elimination, in variance formulas, and in the geometric series behind Markov chains and the Neumann series.

Formal

$$\sum_{k=1}^{n} 1 = n, \tag{39.4}$$

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}, \tag{39.5}$$

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}, \tag{39.6}$$

$$\sum_{k=0}^{n-1} r^k = \frac{1-r^n}{1-r} \quad (r \neq 1), \qquad \sum_{k=0}^{\infty} r^k = \frac{1}{1-r} \quad (|r| < 1). \tag{39.7}$$

The triangular-number formula (39.5) has the famous one-line proof attributed to a young Gauss: write the sum forwards and backwards and add columns,

$$\begin{array}{rcccccc} S & = & 1 & + & 2 & + & \cdots + n, \\ S & = & n & + & (n-1) & + & \cdots + 1, \\ \hline 2S & = & (n+1) & + & (n+1) & + & \cdots + (n+1), \end{array}$$

which is n copies of $(n+1)$, so $2S = n(n+1)$ and $S = n(n+1)/2$. Figure 39.1 turns this into a picture: a staircase of dots, plus a second copy flipped on top, fills a clean $n \times (n+1)$ rectangle.

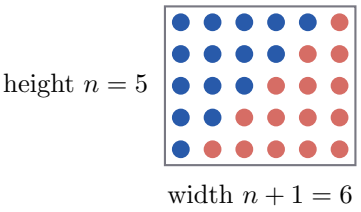


Figure 39.1: Gauss's sum as a picture, for $n = 5$. The blue staircase is $1+2+3+4+5$. A red copy, flipped, completes a 5×6 rectangle of 30 dots. So the blue half is $30/2 = 15 = \frac{5 \cdot 6}{2}$.

Why does the sum of squares $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$ scale as $n^3/3$? Think of stacking discrete cubes to build a pyramid, or taking an integral: $\int_0^n x^2\,dx = n^3/3$. Summing constants produces a linear result ($\sim n$), summing linear terms k produces a quadratic ($\sim n^2/2$), and summing squares must produce a cubic ($\sim n^3/3$). Expanding $\frac{n(n+1)(2n+1)}{6} = \frac{n^3}{3} + \frac{n^2}{2} + \frac{n}{6}$ reveals that exact volume scaling, with the smaller lower-degree terms accounting for the jagged edges of the discrete steps.

The geometric series (39.7) is the most important of the four. Its derivation is a telescoping trick. Let $S = 1 + r + r^2 + \cdots + r^{n-1}$. Multiply by r and subtract:

$$S - rS = (1 + r + \cdots + r^{n-1}) - (r + r^2 + \cdots + r^n) = 1 - r^n,$$

because every middle term cancels. So $S(1-r) = 1 - r^n$, giving $S = (1-r^n)/(1-r)$. If $|r| < 1$ then $r^n \to 0$ as $n \to \infty$, and the infinite sum is $1/(1-r)$.

Connection to other math

The matrix version of $\dfrac{1}{1-r} = 1 + r + r^2 + \cdots$ is the *Neumann series*: if a square matrix $\mathbf{A}$ is small enough (its eigenvalues all have $|\lambda| < 1$), then

$$(\mathbf{I} - \mathbf{A})^{-1} = \mathbf{I} + \mathbf{A} + \mathbf{A}^2 + \mathbf{A}^3 + \cdots .$$

The same telescoping proof works: $(\mathbf{I} - \mathbf{A})(\mathbf{I} + \mathbf{A} + \cdots + \mathbf{A}^{n-1}) = \mathbf{I} - \mathbf{A}^n$, and $\mathbf{A}^n \to 0$. This is exactly the algebra behind ranking the web (PageRank) and behind the steady state of a Markov chain. Scalar intuition transfers almost word for word once you are willing to multiply matrices.

39.1.3 Double sums and why matrix multiplication looks the way it does

When you nest one loop inside another you get a *double sum*:

$$\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij} \;=\; \sum_{i=1}^{m} \Big(\sum_{j=1}^{n} a_{ij} \Big).$$

The inner sum (over j) runs to completion for each fixed value of the outer index i; then the outer sum adds those partial totals. If you imagine the numbers a_{ij} laid out in a grid with i choosing the row and j the column, a double sum just adds up every cell of the grid. And "add up every cell" does not care whether you sweep row-by-row or column-by-column, so you may swap the order whenever the index ranges are independent:

$$\sum_{i} \sum_{j} a_{ij} \;=\; \sum_{j} \sum_{i} a_{ij}.$$

Figure 39.2 makes this physical: row totals added together or column totals added together both reach the exact same grand sum.

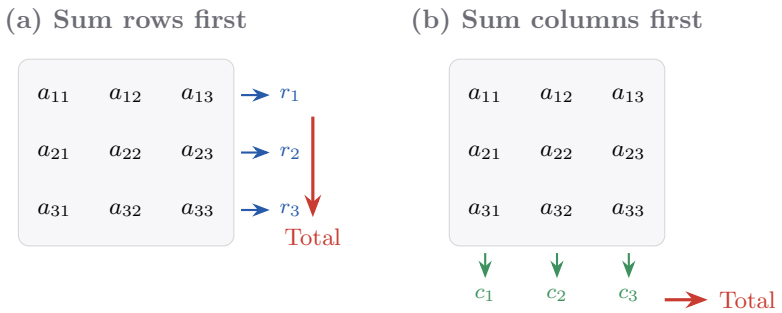


Figure 39.2: Swapping summation order in a double sum. In (a), the inner sum sweeps across each row to produce row totals $r_i = \sum_j a_{ij}$, which the outer sum adds up. In (b), the inner sum sweeps down each column to produce column totals $c_j = \sum_i a_{ij}$, which the outer sum combines. Because every entry in the grid is counted exactly once with no omissions or duplicates, both paths yield the identical grand total: $\sum_i \sum_j a_{ij} = \sum_j \sum_i a_{ij}$.

This single fact is the engine room of matrix algebra. The (i, j) entry of a product $\mathbf{AB}$ is a sum over

a shared index:

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Read it as the dot product of row i of $\mathbf{A}$ with column j of $\mathbf{B}$: k marches along the row and down the column simultaneously. The trace is a single sum down the diagonal, $\operatorname{tr}(\mathbf{A}) = \sum_i a_{ii}$. For a product $\mathbf{AB}$, the diagonal entries are $(\mathbf{AB})_{ii} = \sum_k a_{ik} b_{ki}$, so

$$\operatorname{tr}(\mathbf{AB}) = \sum_{i=1}^{m} \sum_{k=1}^{n} a_{ik} b_{ki}.$$

Inside the double sum, a_{ik} and b_{ki} are ordinary scalar numbers, so their product commutes: $a_{ik} b_{ki} = b_{ki} a_{ik}$. Swapping the order of summation now gives

$$\operatorname{tr}(\mathbf{AB}) = \sum_{i=1}^{m} \sum_{k=1}^{n} a_{ik} b_{ki} = \sum_{k=1}^{n} \sum_{i=1}^{m} b_{ki} a_{ik} = \sum_{k=1}^{n} (\mathbf{BA})_{kk} = \operatorname{tr}(\mathbf{BA}).$$

The cyclic invariance of the trace is an immediate consequence of swapping two nested loops.

Focus

Hold onto three readings of $\sum$: (1) it is a `for` loop; (2) it is *linear*, so constants come out and sums split; (3) when two of them nest, you may swap the order. Almost every "index gymnastics" computation in this book is one of these three moves applied carefully.

Walking question

On your walk, write the 2×2 product $\mathbf{AB}$ entry by entry using only the formula $(\mathbf{AB})_{ij} = \sum_k a_{ik} b_{kj}$, with no matrix picture in front of you. Then check it against the rows-times-columns picture you already know. Convince yourself the symbol and the picture are the same object seen from two sides.

39.2 Complex Numbers: Rotation Made into Arithmetic

The eigenvalue chapter hits a wall the first time it meets a rotation: a 90° turn in the plane has *no* real eigenvalue, because no real vector keeps its direction under a quarter turn. The fix is to enlarge the number system. Complex numbers are what you get when you decide that the equation $x^2 = -1$ should have a solution after all. What you gain is large: complex multiplication *is* rotation-and-scaling, which is precisely the geometry eigenvalues are trying to describe.

39.2.1 The number i and the complex plane

Define a new number i by the single rule

$$i^2 = -1.$$

A *complex number* is then anything of the form $z = a + bi$ with a, b real. We call $a = \operatorname{Re}(z)$ the real part and $b = \operatorname{Im}(z)$ the imaginary part (note: b is a real number; "imaginary" is just a historical label). The set of all of them is $\mathbb{C}$.

The picture carries the idea. Plot $z = a+bi$ as the point (a, b) in a plane, with the real axis horizontal and the imaginary axis vertical. This is the *complex plane*: geometrically it is $\mathbb{R}^2$ equipped with complex multiplication.

History and connection

In 1545, the Italian physician and mathematician Gerolamo Cardano published *Ars Magna* (The Great Art), presenting algebraic formulas for solving cubic ($x^3 + px = q$) and quartic polynomial equations. These formulas had been discovered through secret mathematical duels by Scipione del Ferro, Niccolò Tartaglia, and Cardano's student Lodovico Ferrari. But the formula ran headlong into an alarming mystery known as the *casus irreducibilis* (the "irreducible case"): whenever a cubic equation had three distinct real roots, the algebraic formula stubbornly demanded taking square roots of negative numbers. For instance, when applied to $x^3 = 15x + 4$, Cardano's formula produced

$$x = \sqrt[3]{2 + \sqrt{-121}} + \sqrt[3]{2 - \sqrt{-121}}.$$

At the time, mathematicians possessed only geometric algebra inherited from Euclid and arithmetic on positive quantities; negative numbers were barely trusted, and the square root of a negative quantity was regarded as pure nonsense. Yet $x = 4$ was an obvious real solution ($4^3 = 64 = 15(4) + 4$). Cardano was baffled, calling these expressions "as subtle as they are useless" and dismissing them as "mental torture."

In 1572, Rafael Bombelli of Bologna resolved the controversy in his treatise *L'Algebra*. Bombelli had the insight to treat $\sqrt{-1}$ as an authentic quantity with consistent multiplication rules: $(+i)(+i) = -1$, $(-i)(-i) = -1$, and $(+i)(-i) = 1$. He discovered that $(2 + i)^3 = 2 + 11i = 2 + \sqrt{-121}$ and $(2 - i)^3 = 2 - 11i = 2 - \sqrt{-121}$. When added, the imaginary terms cancel out completely, leaving $(2 + i) + (2 - i) = 4$.

Suspicion lingered for two more centuries. René Descartes coined the disparaging label "imaginary" in 1637 to imply these numbers were fictitious non-entities, and Gottfried Wilhelm Leibniz described $\sqrt{-1}$ as "an amphibian between being and non-being" in his writings. The fog lifted when Leonhard Euler introduced $e^{i\theta} = \cos\theta + i\sin\theta$ in 1748, and Caspar Wessel, Jean-Robert Argand, and Carl Friedrich Gauss gave complex numbers a geometric home: points (a, b) in a plane where multiplication means rigid rotation and scaling.

This geometric view supported rigorous proofs of the Fundamental Theorem of Algebra (every n-th degree polynomial has exactly n complex roots), electrical circuit analysis, and quantum mechanics, where physical states live in complex vector spaces. Yet deep mysteries remain open: Bernhard Riemann's 1859 hypothesis asserts that all non-trivial zeros of the Riemann zeta function $\zeta(s) = \sum_{n=1}^{\infty} n^{-s}$ (a function built by summing reciprocal powers across the complex plane) lie precisely on the "critical line" $\mathrm{Re}(s) = \frac{1}{2}$. Intimately connected to the distribution of prime numbers and the eigenvalues of chaotic quantum systems, proving or disproving the Riemann Hypothesis remains the most celebrated open problem in all of mathematics.

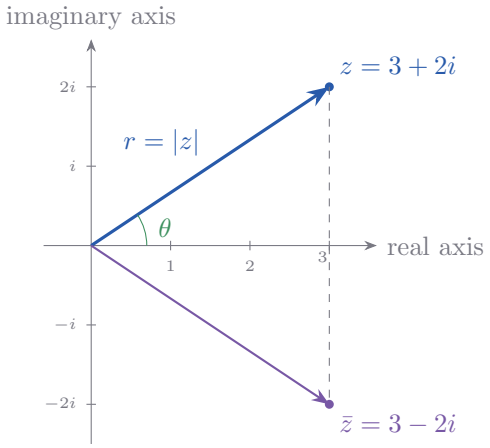


Figure 39.3: A complex number $z = a + bi$ lives at the point (a, b) in the complex plane. Its length $r = |z|$ and angle θ from the positive real axis are its polar coordinates. The conjugate $\bar{z} = a - bi$ is its mirror image across the real axis; the vertical dashed line emphasizes that both share the same real component $a = 3$.

39.2.2 Arithmetic: add like vectors, multiply by rotating

Addition is componentwise, exactly like adding vectors in $\mathbb{R}^2$:

$$(a + bi) + (c + di) = (a + c) + (b + d)i.$$

Multiplication uses ordinary distribution, then collapses i^2 to -1:

$$(a + bi)(c + di) = ac + adi + bci + bd\,i^2 = (ac - bd) + (ad + bc)i.$$

Example 39.2 (A product worked term by term)**.**

$$(2 + 3i)(1 - 4i) = 2 - 8i + 3i - 12i^2 = 2 - 8i + 3i + 12 = 14 - 5i.$$

The $-12i^2$ became $+12$ because $i^2 = -1$. This sign change is the defining feature of complex multiplication.

Two more pieces of vocabulary. The *conjugate* of $z = a + bi$ is $\bar{z} = a - bi$, the reflection of z across the real axis. The *modulus* (or absolute value, or length) is

$$|z| = \sqrt{a^2 + b^2},$$

the ordinary distance from the origin. Conjugate and modulus are tied together by a clean identity:

$$z\bar{z} = (a + bi)(a - bi) = a^2 - (bi)^2 = a^2 + b^2 = |z|^2.$$

So $z\bar{z}$ is always a nonnegative *real* number, namely $|z|^2$. This is exactly how you *divide*: to compute $1/z$, multiply top and bottom by $\bar{z}$ to make the denominator real.

Example 39.3 (Division by realizing the denominator)**.**

$$\frac{1}{2 + 3i} = \frac{1}{2 + 3i} \cdot \frac{2 - 3i}{2 - 3i} = \frac{2 - 3i}{2^2 + 3^2} = \frac{2 - 3i}{13} = \frac{2}{13} - \frac{3}{13}i.$$

Pause

Pause and think: What is i^4? In your head: $i^2 = -1$, so $i^4 = (i^2)^2 = (-1)^2 = 1$. Geometrically, multiplying by i is a 90° quarter-turn; four quarter-turns make a complete 360° circle, returning right back to $+1$. What is the modulus of $3 + 4i$? By Pythagoras, $|3 + 4i| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$.

Common trap

The modulus is *not* additive: $|z + w| \neq |z| + |w|$ in general (the triangle inequality only gives $\leq$). But the modulus *is* multiplicative: $|zw| = |z|\,|w|$ always. Multiplication respects lengths; addition does not. The next subsection explains why, in one picture.

39.2.3 Polar form and Euler's formula

Switch a complex number to polar coordinates. If z has length $r = |z|$ and makes angle θ with the positive real axis, then $a = r\cos\theta$ and $b = r\sin\theta$, so

$$z = r(\cos\theta + i\sin\theta).$$

Now multiply two numbers written this way, $z_1 = r_1(\cos\alpha + i\sin\alpha)$ and $z_2 = r_2(\cos\beta + i\sin\beta)$. Expand and use the angle-addition formulas for sine and cosine:

$$z_1 z_2 = r_1 r_2\big[(\cos\alpha\cos\beta - \sin\alpha\sin\beta) + i(\sin\alpha\cos\beta + \cos\alpha\sin\beta)\big] = r_1 r_2\big(\cos(\alpha+\beta) + i\sin(\alpha+\beta)\big).$$

Intuition

To multiply complex numbers, multiply the lengths and add the angles. That is the entire geometric content. Multiplication by a complex number is a *rotate-and-scale* of the plane. Multiplying by i (length 1, angle 90°) is a pure quarter-turn; check it: $i \cdot 1 = i$ (the point 1 swings up to i), and $i \cdot i = -1$ (another quarter-turn lands on -1). The mysterious rule $i^2 = -1$ is just "two quarter-turns make a half-turn."

Figure 39.4 shows the rule in one picture: stack the two angles tip to tail and multiply the two lengths.

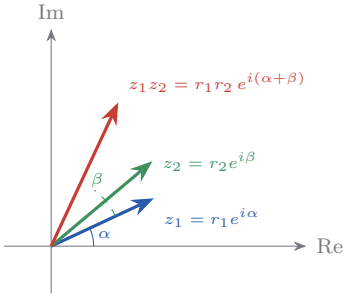


Figure 39.4: Multiplication adds angles and multiplies lengths. The product $z_1 z_2$ (red) sits at angle $\alpha + \beta$ with length $r_1 r_2$. Here the green dashed arc shows the angle β stacked directly onto α, and the red arrow is longer than either factor because $r_1 r_2 > r_1, r_2$ when both lengths exceed 1.

The "add the angles" rule is the exact rule for exponents, $e^{\alpha}e^{\beta} = e^{\alpha+\beta}$, and Euler made that connection precise.

Formal

Euler's formula. For any real θ,

$$e^{i\theta} = \cos\theta + i\sin\theta.$$

Consequently every complex number has the compact polar form $z = r\,e^{i\theta}$, with $r = |z|$ and θ its angle. In this notation, multiplication is just $(r_1e^{i\alpha})(r_2e^{i\beta}) = r_1r_2\,e^{i(\alpha+\beta)}$, and the special value at $\theta = \pi$ is the celebrated identity $e^{i\pi} + 1 = 0$.

Why is Euler's formula true? The quickest rigorous argument comes from power series. The exponential, sine, and cosine each have a Taylor series (we meet Taylor series in Section 39.3):

$$e^{x} = \sum_{k=0}^{\infty} \frac{x^k}{k!}, \qquad \cos\theta = \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k}}{(2k)!}, \qquad \sin\theta = \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!}.$$

Substitute $x = i\theta$ into the exponential series and use the cycle $i^0 = 1,\ i^1 = i,\ i^2 = -1,\ i^3 = -i,\ i^4 = 1, \ldots$:

$$e^{i\theta} = 1 + i\theta + \frac{(i\theta)^2}{2!} + \frac{(i\theta)^3}{3!} + \cdots = \underbrace{\left(1 - \tfrac{\theta^2}{2!} + \tfrac{\theta^4}{4!} - \cdots\right)}_{\cos\theta} + i\,\underbrace{\left(\theta - \tfrac{\theta^3}{3!} + \tfrac{\theta^5}{5!} - \cdots\right)}_{\sin\theta}.$$

The real terms reassemble the cosine series and the imaginary terms reassemble the sine series. Done.

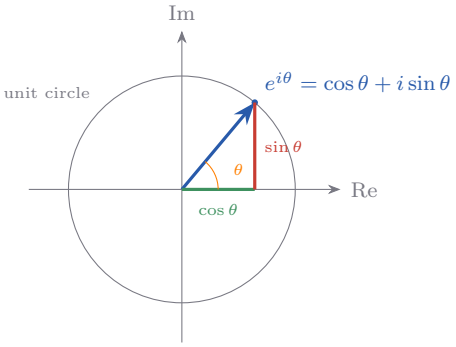


Figure 39.5: Euler's formula geometrically: $e^{i\theta}$ rides around the unit circle. Its horizontal shadow is $\cos\theta$, its vertical shadow is $\sin\theta$. Increasing θ just walks the point counterclockwise; multiplying by $e^{i\theta}$ rotates the whole plane by θ.

Example 39.4 (De Moivre powers)**.** Polar form makes powers trivial. Since $(re^{i\theta})^n = r^n e^{in\theta}$, raising to a power scales the length and multiplies the angle. Take $z = 1 + i$, which has $r = \sqrt{2}$ and $\theta = 45^\circ = \pi/4$. Then

$$(1+i)^8 = (\sqrt{2})^8\, e^{i\cdot 8\pi/4} = 16\, e^{i2\pi} = 16.$$

Try multiplying $(1 + i)$ by itself eight times by hand to appreciate how much labor polar form just saved.

Modern application

The discrete Fourier transform, which underlies JPEG, MP3, and every digital signal pipeline, is built entirely from the numbers $\omega^k = e^{-2\pi ik/n}$ — equally spaced points around the unit circle, called *roots of unity* because their n-th power equals 1. The transform writes a signal as a combination of these pure rotations at different speeds. Because each ω^k has modulus 1, the transform never inflates energy; that length-preservation is exactly the statement that the Fourier matrix is *unitary*, the complex cousin of an orthogonal matrix (its columns are perpendicular unit vectors in $\mathbb{C}^n$, preserving lengths and inner products during transformation). The Fourier chapter of this book takes $e^{i\theta}$ as its alphabet, so being fluent here pays off directly there.

Connection to other math

Complex numbers are why the eigenvalues of a real matrix can come in conjugate pairs $\lambda = a \pm bi$. Geometrically such a pair means the matrix is doing a rotation-and-scale inside some two-dimensional invariant plane: the modulus $|\lambda| = \sqrt{a^2 + b^2}$ is the per-step *stretch*, and the argument $\theta = \arg \lambda$ (the angle the number makes with the positive real axis) is the per-step *turn*. A discrete dynamical system $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k$ spirals outward when $|\lambda| > 1$, spirals inward when $|\lambda| < 1$, and circles forever when $|\lambda| = 1$. The whole qualitative story lives in one complex number.

Compute it yourself

Verify the rotation claim with a 2×2 matrix. The rotation by $90°$ is $\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$. Compute its eigenvalues by solving $\det(\mathbf{R} - \lambda \mathbf{I}) = \lambda^2 + 1 = 0$, which gives $\lambda = \pm i$. There are no real eigenvectors (no real arrow survives a quarter-turn pointing the same way), and the eigenvalues are pure imaginary with $|\lambda| = 1$ — a turn with no stretch, exactly as the picture demands. Now do the same for $\begin{bmatrix} 0 & -2 \\ 2 & 0 \end{bmatrix}$ and watch $|\lambda|$ become 2.

39.3 Just Enough Calculus

Two chapters of this book reach outside linear algebra into calculus. The differential-equations chapter needs *derivatives* (to say what "rate of change" means) and a little *integration* (to define the matrix exponential and to take inner products of functions). The optimization chapter needs the *gradient* (the multivariable derivative) to find where a function is flat, which is where minima live. This section builds exactly those tools and stops.

39.3.1 The derivative as a slope, and as a best linear fit

Start with one variable. The derivative of f at a point x measures how fast f is changing there. Geometrically it is the slope of the tangent line: zoom in on the graph near x until the curve looks straight, and the derivative is that straight line's slope.

Formal

The *derivative* of f at x is the limit of slopes of secant lines (straight lines connecting two nearby points on the curve),

$$f'(x) = \frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h},$$

when the limit exists. Equivalently, near x the function is well approximated by a line: $f(x+h) \approx f(x) + f'(x)\,h$ for small h.

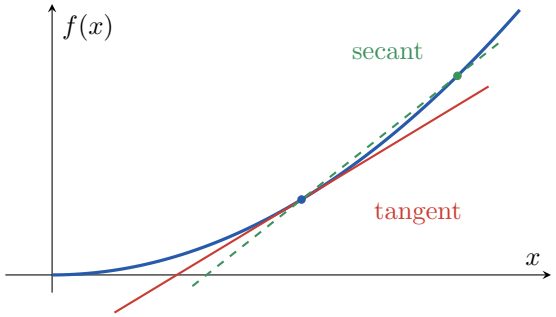


Figure 39.6: The secant slope $\frac{f(x+h)-f(x)}{h}$ (green) is the average rate of change over a step h. As the step shrinks, the secant pivots toward the tangent (red), whose slope is the derivative $f'(x)$.

Example 39.5 (A derivative from the definition)**.** Let $f(x) = x^2$. Then

$$\frac{f(x+h) - f(x)}{h} = \frac{(x+h)^2 - x^2}{h} = \frac{2xh + h^2}{h} = 2x + h \xrightarrow[h \to 0]{} 2x.$$

So $f'(x) = 2x$. The h in the numerator is what lets the fraction survive division by h; sending $h \to 0$ at the end discards the leftover h and leaves the clean slope $2x$.

You will not re-derive every derivative from the limit. A handful of rules and a short table cover everything the main chapters use.

Formal

The rules you actually need.

Power:	$(x^n)' = nx^{n-1}$,	Exponential:	$(e^{ax})' = a\,e^{ax}$,
Sine:	$(\sin x)' = \cos x$,	Cosine:	$(\cos x)' = -\sin x$,
Constant/linear:	$(c)' = 0,\ (x)' = 1$,	Sum:	$(f+g)' = f' + g'$,
Constant multiple:	$(cf)' = cf'$,	Product:	$(fg)' = f'g + fg'$,
Chain:	$\big(f(g(x))\big)' = f'(g(x))\,g'(x)$.		

The sum and constant-multiple rules say the derivative is a *linear* operation: $(af + bg)' = af' + bg'$. That linearity is why differentiation itself can be treated as a matrix in the change-of-basis and functional-analysis chapters — a linear map on a space of functions. The chain rule is the one to be most careful with; it forms the computational engine behind backpropagation.

Example 39.6 (The chain rule in action)**.** Differentiate $f(x) = e^{-x^2}$ (the bell-curve shape). Outer function e^u with $u = -x^2$:

$$f'(x) = e^{-x^2} \cdot \frac{d}{dx}(-x^2) = e^{-x^2} \cdot (-2x) = -2x\, e^{-x^2}.$$

This is zero only at $x = 0$, the peak of the bell, which matches the picture: the curve is flat exactly at its top.

Pause

Pause and think: For $f(x) = x^3$, what is the slope of the tangent line at $x = 0$? By the power rule, $f'(x) = 3x^2$, so $f'(0) = 0$: the cubic curve momentarily flattens out horizontal at the origin before continuing its climb. What about $f(x) = 5x + 7$? Its derivative is 5 everywhere, confirming that the tangent slope of a straight line is always its constant rate of change.

39.3.2 From slope to optimization: where the derivative is zero

The reason calculus shows up in the optimization chapter is one sentence: *at a smooth minimum or maximum, the slope is zero.* If $f'(x_\star) \neq 0$ then the tangent line tilts, and stepping slightly downhill makes f smaller, so $x_\star$ cannot be a minimum. Setting $f'(x) = 0$ and solving locates the candidates.

Example 39.7 (Least squares in one variable)**.** Suppose data points have heights $y_1, \ldots, y_n$ and you want the single constant c that best fits them in the least-squares sense, i.e. the c minimizing

$$E(c) = \sum_{k=1}^{n} (y_k - c)^2.$$

Differentiate with respect to c, using the chain rule on each term and the linearity of $\sum$:

$$E'(c) = \sum_{k=1}^{n} 2(y_k - c)(-1) = -2 \sum_{k=1}^{n} (y_k - c).$$

Set $E'(c) = 0$: then $\sum_k (y_k - c) = 0$, i.e. $\sum_k y_k = nc$, so $c = \frac{1}{n} \sum_k y_k$. The best constant fit is the *average*. This little calculation is the seed of the whole least-squares and regression story, where "set the derivative to zero" becomes the normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b}$.

39.3.3 Gradients: the derivative for many variables

When f depends on several variables, $f(x_1, \ldots, x_n)$, there is a slope in each coordinate direction. The *partial derivative* $\partial f / \partial x_j$ is just the ordinary derivative in the x_j direction, holding the other variables fixed. Stack all of them into a vector and you get the *gradient*.

Formal

The *gradient* of $f\colon \mathbb{R}^n \to \mathbb{R}$ is the vector of its partial derivatives,

$$\nabla f = \begin{bmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix}.$$

Near a point $\mathbf{x}$, the function is approximated by the linear formula $f(\mathbf{x}+\mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x}) \cdot \mathbf{h}$, the multivariable analog of $f(x+h) \approx f(x) + f'(x)h$.

The geometry of the gradient is worth holding onto.

Intuition

At any point, ∇f points in the direction of *steepest increase* of f, and its length is how steep that climb is. Walk along ∇f and f rises as fast as possible; walk along $-\nabla f$ and f falls as fast as possible (this is *gradient descent*, an iterative algorithm that repeatedly steps downhill to find a minimum). Walk *perpendicular* to ∇f and f does not change at all: you are following a *level curve*, a contour of constant height, like an elevation line on a topographic map. A contour map of a mountain makes this visible: the gradient is the arrow pointing straight uphill, always strictly at right angles to the contour lines.

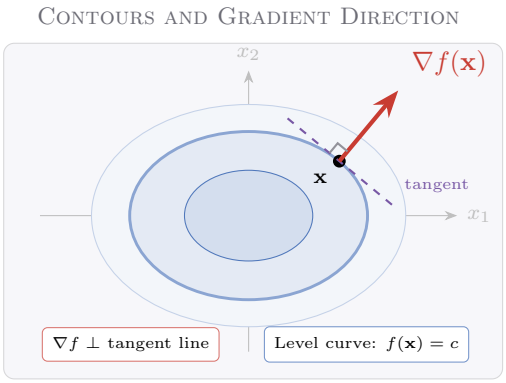


Figure 39.7: Level curves of the quadratic bowl $f(x_1, x_2) = x_1^2 + 2x_2^2$. The gradient vector $\nabla f = (2x_1, 4x_2)$ points in the direction of steepest uphill ascent and is strictly perpendicular to the level contour through $\mathbf{x}$, as shown by the right-angle indicator against the purple tangent line. At the minimum $(0, 0)$, the gradient vanishes ($\nabla f = \mathbf{0}$).

Example 39.8 (Computing a gradient)**.** Let $f(x_1, x_2) = x_1^2 + 2x_2^2$. Holding x_2 fixed, $\partial f/\partial x_1 = 2x_1$; holding x_1 fixed, $\partial f/\partial x_2 = 4x_2$. So

$$\nabla f = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}.$$

The only point where $\nabla f = \mathbf{0}$ is the origin, and the bowl shape tells us it is a minimum.

Example 39.9 (The gradient of a quadratic form)**.** The optimization chapter lives on quadratic forms (functions where every term is degree two or one), $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} - \mathbf{b}^\mathsf{T}\mathbf{x}$ with $\mathbf{A}$ symmetric.

Its gradient is the tidy

$$\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b},$$

so setting $\nabla f = \mathbf{0}$ recovers the linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$. When $\mathbf{A}$ is positive definite (all eigenvalues $\lambda_i > 0$), the surface curves upward in every direction like a bowl, guaranteeing that this critical point is a unique global minimum. Let us verify the 2×2 case with $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$ and $\mathbf{b} = \mathbf{0}$: then $\frac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = x_1^2 + 2x_2^2$, whose gradient we just found to be $(2x_1, 4x_2) = \mathbf{A}\mathbf{x}$. The abstract formula and the hand computation agree.

Formal

Matrix calculus derivatives, step by step. Where does $\nabla_\mathbf{x}(\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}) = 2\mathbf{A}\mathbf{x}$ come from? Expand the quadratic form as an index sum:

$$f(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x} = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}x_i x_j.$$

To find component k of the gradient, differentiate with respect to x_k. The variable x_k appears whenever $i = k$ or $j = k$:

- When $i = k$ and $j \neq k$: the terms are $\sum_{j\neq k} A_{kj}x_k x_j$, with derivative $\sum_{j\neq k} A_{kj}x_j$.
- When $j = k$ and $i \neq k$: the terms are $\sum_{i\neq k} A_{ik}x_i x_k$, with derivative $\sum_{i\neq k} A_{ik}x_i$.
- When $i = j = k$: the single diagonal term is $A_{kk}x_k^2$, with derivative $2A_{kk}x_k$.

Combining all three pieces:

$$\frac{\partial}{\partial x_k}(\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}) = \sum_{j=1}^{n} A_{kj}x_j + \sum_{i=1}^{n} A_{ik}x_i = (\mathbf{A}\mathbf{x})_k + (\mathbf{A}^\mathsf{T}\mathbf{x})_k.$$

Stacking all k rows yields the general matrix calculus identity:

$$\boxed{\nabla_\mathbf{x}(\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\mathsf{T})\mathbf{x}.}$$

When $\mathbf{A}$ is symmetric ($\mathbf{A} = \mathbf{A}^\mathsf{T}$), this becomes $2\mathbf{A}\mathbf{x}$. Dividing by 2 gives $\nabla(\frac{1}{2}\mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}) = \mathbf{A}\mathbf{x}$. Similarly, for a linear form $\mathbf{b}^\mathsf{T}\mathbf{x} = \sum b_i x_i$, $\frac{\partial}{\partial x_k}(\mathbf{b}^\mathsf{T}\mathbf{x}) = b_k$, so $\nabla_\mathbf{x}(\mathbf{b}^\mathsf{T}\mathbf{x}) = \mathbf{b}$.

Modern application

Training a neural network is gradient descent on a function of millions of variables. The loss L is a scalar measuring how wrong the network is; its gradient ∇L with respect to all the weights is computed by *backpropagation*, which is the chain rule from Section 39.3.1 applied layer by layer. Each training step nudges the weights along $-\nabla L$. Every matrix multiply inside a network is the linear algebra; every gradient step is this calculus.

39.3.4 A little integration

The differential-equations chapter needs integration in two small ways, so we collect just those.

First, the integral as *accumulated area / undoing a derivative*. The definite integral $\int_a^b f(x)\,dx$ is the signed area under the graph of f between a and b. The Fundamental Theorem of Calculus (the foundational principle establishing that differentiation and integration are inverse operations) ties it to derivatives: if $F' = f$ then

$$\int_a^b f(x)\,dx = F(b) - F(a).$$

Integration is the reverse of differentiation, just as $\sum$ is a discrete cousin of $\int$ (both accumulate). For instance, since $(e^{ax})' = ae^{ax}$, we get $\int e^{ax}\,dx = \frac{1}{a}e^{ax} + C$. That single antiderivative is what lets the matrix exponential $e^{\mathbf{A}t} = \sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!}$ solve $\frac{d}{dt}\mathbf{x} = \mathbf{A}\mathbf{x}$: differentiate the series term by term and the same series reappears, multiplied by $\mathbf{A}$.

Example 39.10 (Differentiating the matrix-exponential series)**.** Treat the series $e^{\mathbf{A}t} = \mathbf{I} + \mathbf{A}t + \frac{\mathbf{A}^2t^2}{2!} + \frac{\mathbf{A}^3t^3}{3!} + \cdots$ and differentiate term by term in t using the power rule:

$$\frac{d}{dt}e^{\mathbf{A}t} = \mathbf{A} + \mathbf{A}^2t + \frac{\mathbf{A}^3t^2}{2!} + \cdots = \mathbf{A}\left(\mathbf{I} + \mathbf{A}t + \frac{\mathbf{A}^2t^2}{2!} + \cdots\right) = \mathbf{A}\,e^{\mathbf{A}t}.$$

So $\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}(0)$ satisfies $\mathbf{x}'(t) = \mathbf{A}\mathbf{x}(t)$. The scalar fact $(e^{at})' = ae^{at}$ became its matrix twin with no new ideas, only the power rule and the linearity of differentiation.

Second, the integral as an *inner product of functions*. In the Fourier and functional-analysis chapters, the dot product $\mathbf{u}\cdot\mathbf{v} = \sum_k u_k v_k$ gets upgraded for functions to

$$\langle f,\, g\rangle = \int_a^b f(x)\,g(x)\,dx.$$

The sum over coordinates becomes an integral over the continuum of inputs; everything else — length $\|f\| = \sqrt{\langle f,\, f\rangle}$, orthogonality $\langle f,\, g\rangle = 0$, projection, even Gram–Schmidt — transfers verbatim. This is why a chapter on integrals belongs in a linear algebra book at all: the integral is the continuous analogue of the dot product. Figure 39.8 makes this bridge visible: discrete Riemann rectangles melt smoothly into continuous area.

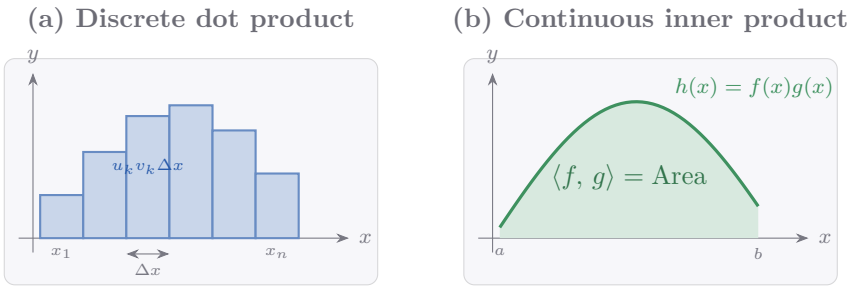


Figure 39.8: The bridge from vector dot products to function inner products. In (a), sampling two functions at n discrete points turns them into vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$; their weighted dot product $\sum_{k=1}^{n} f(x_k)g(x_k)\Delta x$ is the total area of approximating rectangles. In (b), letting the step size $\Delta x \to 0$ melts the rectangles into the smooth definite integral $\langle f,\, g\rangle = \int_a^b f(x)g(x)\,dx$. The continuous inner product is simply a Euclidean dot product evaluated across a continuum of components.

Connection to other math

The parallel runs deep. A finite vector is a function from $\{1, \dots, n\}$ to $\mathbb{R}$; a function on $[a, b]$ is a vector with a continuum of components. Under that dictionary, $\sum_k u_k v_k$ and $\int uv\, dx$ are the same construction, summation just becomes integration. Orthonormal bases become Fourier series; the spectral theorem for symmetric matrices becomes the spectral theory of differential operators. Carrying the geometry of $\mathbb{R}^n$ into spaces of functions is the subject called *functional analysis*, and the single bridge you cross to get there is replacing $\sum$ by $\int$.

Focus

The calculus this book needs reduces to four reflexes. (1) A derivative is a slope and a best linear approximation. (2) The gradient collects the partial derivatives into a vector that points uphill and vanishes at optima. (3) Setting a derivative or gradient to zero is how optimization meets linear systems. (4) An integral accumulates, undoes a derivative, and serves as the inner product of functions. Everything else you can look up.

Limitations and Conventions

Each of these tools demands respect for its boundaries. Complex numbers guarantee that every $n \times n$ matrix has a full set of n eigenvalues, but they lose size comparisons: the complex field $\mathbb{C}$ cannot be ordered, so inequalities like $z_1 < z_2$ have no meaning. In matrix calculus, literature remains split between numerator layout (gradient as column vector) and denominator layout (gradient as row vector), causing frequent transpose issues. Finally, when sums or integrals extend to infinity, linear algebra must lean on analysis: infinite series $\sum_{k=0}^{\infty} \mathbf{A}^k$ require eigenvalue bounds ($|\lambda| < 1$) to converge.

Where We Go Next Across the Curriculum

Linear algebra connects directly to the rest of the mathematical curriculum:

- *Calculus From the Ground Up:* Extends the single-variable slopes and gradients met here to multivariable Taylor series, Jacobians, and Hessians.
- *Probability From the Ground Up:* Interprets quadratic forms $\mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{x}$ as Gaussian densities, projections as conditional expectations, and SVD as principal component analysis.
- *Signal Processing From the Ground Up:* Expands Euler's formula $e^{i\theta}$ and roots of unity into the continuous Fourier transform and filter design.
- *Algorithms From the Ground Up:* Develops gradient steps and normal equations into numerical methods for large-scale computation.

39.4 Exercises

Exercise 39.1 (Unrolling sums)**.** Evaluate each by writing out the terms. (a) $\sum_{k=1}^{4}(2k-1)$. (b) $\sum_{k=0}^{4}\frac{1}{2^k}$. (c) $\sum_{i=1}^{3}\sum_{j=1}^{2}(i+j)$. For (b), compare your answer with the geometric-series formula (39.7) and with the infinite sum $\sum_{k=0}^{\infty}2^{-k}$.

Exercise 39.2 (Sigma algebra)**.** Using only the rules (39.1)–(39.3) and the closed forms (39.4)–(39.6), show that

$$\sum_{k=1}^{n}(3k^2-2k+5)=\frac{n(n+1)(2n+1)}{2}-n(n+1)+5n,$$

and simplify. Then check your formula at $n=2$ by adding the two terms directly.

Exercise 39.3 (Order swapping is the trace identity)**.** For $\mathbf{A}=\begin{bmatrix}1&2\\0&3\end{bmatrix}$ and $\mathbf{B}=\begin{bmatrix}4&1\\1&0\end{bmatrix}$, compute $\operatorname{tr}(\mathbf{AB})$ and $\operatorname{tr}(\mathbf{BA})$ from the definition $\operatorname{tr}(\mathbf{M})=\sum_i m_{ii}$, and confirm they are equal. Then explain in one sentence which summation-order swap makes this true in general.

Exercise 39.4 (Complex arithmetic)**.** Let $z=3-i$ and $w=1+2i$. Compute $z+w$, zw, $\bar{z}$, $|z|$, and z/w (realize the denominator). Plot z, w, and zw in the complex plane and verify by eye that the angle of zw is the sum of the angles of z and w.

Exercise 39.5 (Multiplying is rotating)**.** Write $z=2e^{i\pi/3}$ in the form $a+bi$. Then compute z^3 two ways: once by cubing $a+bi$ directly, and once with polar form $(re^{i\theta})^3=r^3e^{3i\theta}$. Confirm both give the same number, and describe in words the rotation and scaling that cubing performed.

Exercise 39.6 (Euler from the series)**.** Using the power-series argument of Section 39.2.3, evaluate $e^{i\pi/2}$ by separating real and imaginary parts, and confirm it equals i. Then use $e^{i\theta}$ to give a one-line proof of the double-angle identities $\cos 2\theta=\cos^2\theta-\sin^2\theta$ and $\sin 2\theta=2\sin\theta\cos\theta$ (hint: expand $(e^{i\theta})^2=e^{2i\theta}$ two ways).

Exercise 39.7 (Derivatives from scratch and from rules)**.** (a) Use the limit definition to show that the derivative of $f(x)=1/x$ is $-1/x^2$. (b) Use the product and chain rules to differentiate $g(x)=x^2e^{-x}$ and find where $g'(x)=0$. Interpret those points on the graph of g.

Exercise 39.8 (Least squares is calculus)**.** You have points $(1,1),(2,2),(3,2)$ and want the best-fitting line through the origin, $y=cx$, in the least-squares sense: minimize $E(c)=\sum_k(y_k-cx_k)^2$. Differentiate E with respect to c, set $E'(c)=0$, and solve for c. Then identify which dot products your answer is built from, and connect it to the formula $c=(\mathbf{x}\cdot\mathbf{y})/(\mathbf{x}\cdot\mathbf{x})$ for projecting $\mathbf{y}$ onto $\mathbf{x}$.

Exercise 39.9 (A gradient and its zero)**.** Let $f(x_1,x_2)=x_1^2+x_1x_2+x_2^2-3x_1$. Compute ∇f, set it to the zero vector, and solve the resulting 2×2 linear system for the critical point. Write that linear system in the form $\mathbf{Ax}=\mathbf{b}$ and identify the symmetric matrix $\mathbf{A}$; relate it to the formula $\nabla f=\mathbf{Ax}-\mathbf{b}$ from Example 39.9.

Exercise 39.10 (Integration as a dot product)**.** On the interval $[-\pi,\pi]$ with inner product $\langle f,g\rangle=\int_{-\pi}^{\pi}f(x)g(x)\,dx$, show that $\sin x$ and $\cos x$ are orthogonal, that is $\langle\sin,\cos\rangle=0$. (Hint: $\sin x\cos x=\frac{1}{2}\sin 2x$.) Then compute $\|\sin\|^2=\langle\sin,\sin\rangle$. What you have just verified is the first step in building the Fourier orthonormal basis.

Exercise 39.11 (Geometric series, scalar to matrix)**.** (a) Derive $\sum_{k=0}^{n-1} r^k = (1 - r^n)/(1 - r)$ by the telescoping trick (multiply by r and subtract). (b) For the matrix $\mathbf{A} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$, compute $\mathbf{I} + \mathbf{A} + \mathbf{A}^2 + \cdots$ entry by entry using the scalar formula on the diagonal, and check that your answer equals $(\mathbf{I} - \mathbf{A})^{-1}$ by inverting $\mathbf{I} - \mathbf{A}$ directly.

Exercise 39.12 (Spiral or decay, from one complex number)**.** The matrix $\mathbf{A} = \begin{bmatrix} 0.8 & -0.6 \\ 0.6 & 0.8 \end{bmatrix}$ has eigenvalues $\lambda = 0.8 \pm 0.6i$. Compute $|\lambda|$ and $\arg \lambda$ (in degrees). Based on the Section 39.2 connection box, predict whether the trajectory $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k$ spirals outward, spirals inward, or circles forever, and by what angle per step. Then scale $\mathbf{A}$ by 1.25 and redo the prediction.